		Section	Page
1.	Introduction	1	3
2.	The character set	17	13
3.	Input and output	25	15
4.	String handling	38	21
5.	On-line and off-line printing	54	27
6.	Reporting errors	76	36
7.	Arithmetic with scaled dimensions	103	44
7b.	Random numbers	114	48
8.	Packed data	132	55
9.	Dynamic memory allocation	137	57
10.	Data structures for boxes and their friends	155	63
11.	Memory layout	187	74
12.	Displaying boxes	199	79
13.	Destroying boxes	225	88
14.	Copying boxes	229	90
15.	The command codes	233	92
16.	The semantic nest	237	96
17.	The table of equivalents	246	101
18.	The hash table	282	125
19.	Saving and restoring equivalents	298	135
20.	Token lists	319	142
21.	Introduction to the syntactic routines	327	146
22.	Input stacks and states	330	149
23.	Maintaining the input stacks	351	159
24.	Getting the next token	362	162
25.	Expanding the next token	396	174
26.	Basic scanning subroutines	436	186
27.	Building token lists	499	212
28.	Conditional processing	522	225
29.	File names	546	233
30.	Font metric data	574	242
31.	Device-independent file format	619	261
32.	Shipping pages out	628	268
32b.	pdfTeX output low-level subroutines (equivalents)	682	292
33.	Packaging	683	293
34.	Data structures for math mode	722	305
35.	Subroutines for math mode	741	315
36.	Typesetting math formulas	762	332
37.	Alignment	816	360
38.	Breaking paragraphs into lines	861	377
39.	Breaking paragraphs into lines, continued	910	397
40.	Pre-hyphenation	939	410
41.	Post-hyphenation	953	417
42.	Hyphenation	973	428
43.	Initializing the hyphenation tables	996	434
44.	Breaking vertical lists into pages	1021	444
45.	The page builder	1034	450
46.	The chief executive	1083	467
47.	Building boxes and lists	1109	483
48.	Building math lists	1190	507
49.	Mode-independent processing	1262	529
50.	Dumping and undumping the tables	1353	554
51.	The main program	1384	564
52.	Debugging	1392	569
53.	Extensions	1394	571
53a.	The extended features of ε-TeX	1451	593
54.	System-dependent changes	1678	666

March 21, 2022 at 18:46

1. Introduction. This is XeTeX, a program derived from and extending the capabilities of TeX, a document compiler intended to produce typesetting of high quality. The Pascal program that follows is the definition of TeX82, a standard version of TeX that is designed to be highly portable so that identical output will be obtainable on a great variety of computers.

The main purpose of the following program is to explain the algorithms of TeX as clearly as possible. As a result, the program will not necessarily be very efficient when a particular Pascal compiler has translated it into a particular machine language. However, the program has been written so that it can be tuned to run efficiently in a wide variety of operating environments by making comparatively few changes. Such flexibility is possible because the documentation that follows is written in the `WEB` language, which is at a higher level than Pascal; the preprocessing step that converts `WEB` to Pascal is able to introduce most of the necessary refinements. Semi-automatic translation to other languages is also feasible, because the program below does not make extensive use of features that are peculiar to Pascal.

A large piece of software like TeX has inherent complexity that cannot be reduced below a certain level of difficulty, although each individual part is fairly simple by itself. The `WEB` language is intended to make the algorithms as readable as possible, by reflecting the way the individual program pieces fit together and by providing the cross-references that connect different parts. Detailed comments about what is going on, and about why things were done in certain ways, have been liberally sprinkled throughout the program. These comments explain features of the implementation, but they rarely attempt to explain the TeX language itself, since the reader is supposed to be familiar with *The TeXbook*.

2. The present implementation has a long ancestry, beginning in the summer of 1977, when Michael F. Plass and Frank M. Liang designed and coded a prototype based on some specifications that the author had made in May of that year. This original protoTEX included macro definitions and elementary manipulations on boxes and glue, but it did not have line-breaking, page-breaking, mathematical formulas, alignment routines, error recovery, or the present semantic nest; furthermore, it used character lists instead of token lists, so that a control sequence like `\halign` was represented by a list of seven characters. A complete version of TEX was designed and coded by the author in late 1977 and early 1978; that program, like its prototype, was written in the SAIL language, for which an excellent debugging system was available. Preliminary plans to convert the SAIL code into a form somewhat like the present "web" were developed by Luis Trabb Pardo and the author at the beginning of 1979, and a complete implementation was created by Ignacio A. Zabala in 1979 and 1980. The TEX82 program, which was written by the author during the latter part of 1981 and the early part of 1982, also incorporates ideas from the 1979 implementation of TEX in MESA that was written by Leonidas Guibas, Robert Sedgewick, and Douglas Wyatt at the Xerox Palo Alto Research Center. Several hundred refinements were introduced into TEX82 based on the experiences gained with the original implementations, so that essentially every part of the system has been substantially improved. After the appearance of "Version 0" in September 1982, this program benefited greatly from the comments of many other people, notably David R. Fuchs and Howard W. Trickey. A final revision in September 1989 extended the input character set to eight-bit codes and introduced the ability to hyphenate words from different languages, based on some ideas of Michael J. Ferguson.

No doubt there still is plenty of room for improvement, but the author is firmly committed to keeping TEX82 "frozen" from now on; stability and reliability are to be its main virtues.

On the other hand, the `WEB` description can be extended without changing the core of TEX82 itself, and the program has been designed so that such extensions are not extremely difficult to make. The *banner* string defined here should be changed whenever TEX undergoes any modifications, so that it will be clear which version of TEX might be the guilty party when a problem arises.

This program contains code for various features extending TEX, therefore this program is called 'XeTeX' and not 'TEX'; the official name 'TEX' by itself is reserved for software systems that are fully compatible with each other. A special test suite called the "`TRIP` test" is available for helping to determine whether a particular implementation deserves to be known as 'TEX' [cf. Stanford Computer Science report CS1027, November 1984].

A similar test suite called the "`e-TRIP` test" is available for helping to determine whether a particular implementation deserves to be known as 'ε-TEX'.

define *eTeX_version* = 2 { `\eTeXversion` }
define *eTeX_revision* ≡ `".6"` { `\eTeXrevision` }
define *eTeX_version_string* ≡ `´-2.6´` { current ε-TEX version }

define *XeTeX_version* = 0 { `\XeTeXversion` }
define *XeTeX_revision* ≡ `".999994"` { `\XeTeXrevision` }
define *XeTeX_version_string* ≡ `´-0.999994´` { current XeTeX version }

define *XeTeX_banner* ≡ `´This␣is␣XeTeX,␣Version␣3.141592653´`, *eTeX_version_string*,
XeTeX_version_string { printed when XeTeX starts }

define *banner* ≡ `´This␣is␣TeX,␣Version␣3.141592653´` { printed when TEX starts }

define *TEX* ≡ *XETEX* { change program name into *XETEX* }

define *TeXXeT_code* = 0 { the TEX--XET feature is optional }

define *XeTeX_dash_break_code* = 1 { non-zero to enable breaks after en- and em-dashes }

define *XeTeX_upwards_code* = 2 { non-zero if the main vertical list is being built upwards }
define *XeTeX_use_glyph_metrics_code* = 3 { non-zero to use exact glyph height/depth }
define *XeTeX_inter_char_tokens_code* = 4 { non-zero to enable `\XeTeXinterchartokens` insertion }

define *XeTeX_input_normalization_code* = 5 { normalization mode:, 1 for NFC, 2 for NFD, else none }

define *XeTeX_default_input_mode_code* = 6 { input mode for newly opened files }
define *XeTeX_input_mode_auto* = 0

define *XeTeX_input_mode_utf8* = 1
define *XeTeX_input_mode_utf16be* = 2
define *XeTeX_input_mode_utf16le* = 3
define *XeTeX_input_mode_raw* = 4
define *XeTeX_input_mode_icu_mapping* = 5
define *XeTeX_default_input_encoding_code* = 7 { *str_number* of encoding name if *mode* = ICU }
define *XeTeX_tracing_fonts_code* = 8 { non-zero to log native fonts used }
define *XeTeX_interword_space_shaping_code* = 9 { controls shaping of space chars in context when using native fonts; set to 1 for contextual adjustment of space width only, and 2 for full cross-space shaping (e.g. multi-word ligatures) }
define *XeTeX_generate_actual_text_code* = 10 { controls output of /ActualText for native-word nodes }
define *XeTeX_hyphenatable_length_code* = 11 { sets maximum hyphenatable word length }
define *eTeX_states* = 12 { number of ε-TeX state variables in *eqtb* }

3. Different Pascals have slightly different conventions, and the present program expresses TeX in terms of the Pascal that was available to the author in 1982. Constructions that apply to this particular compiler, which we shall call Pascal-H, should help the reader see how to make an appropriate interface for other systems if necessary. (Pascal-H is Charles Hedrick's modification of a compiler for the DECsystem-10 that was originally developed at the University of Hamburg; cf. *Software—Practice and Experience* **6** (1976), 29–42. The TeX program below is intended to be adaptable, without extensive changes, to most other versions of Pascal, so it does not fully use the admirable features of Pascal-H. Indeed, a conscious effort has been made here to avoid using several idiosyncratic features of standard Pascal itself, so that most of the code can be translated mechanically into other high-level languages. For example, the '**with**' and '*new*' features are not used, nor are pointer types, set types, or enumerated scalar types; there are no '**var**' parameters, except in the case of files — ε-TeX, however, does use '**var**' parameters for the *reverse* function; there are no tag fields on variant records; there are no assignments *real* ← *integer*; no procedures are declared local to other procedures.)

The portions of this program that involve system-dependent code, where changes might be necessary because of differences between Pascal compilers and/or differences between operating systems, can be identified by looking at the sections whose numbers are listed under 'system dependencies' in the index. Furthermore, the index entries for 'dirty Pascal' list all places where the restrictions of Pascal have not been followed perfectly, for one reason or another.

Incidentally, Pascal's standard *round* function can be problematical, because it disagrees with the IEEE floating-point standard. Many implementors have therefore chosen to substitute their own home-grown rounding procedure.

4. The program begins with a normal Pascal program heading, whose components will be filled in later, using the conventions of WEB. For example, the portion of the program called '⟨ Global variables 13 ⟩' below will be replaced by a sequence of variable declarations that starts in §13 of this documentation. In this way, we are able to define each individual global variable when we are prepared to understand what it means; we do not have to define all of the globals at once. Cross references in §13, where it says "See also sections 20, 26, ...," also make it possible to look at the set of all global variables, if desired. Similar remarks apply to the other portions of the program heading.

Actually the heading shown here is not quite normal: The **program** line does not mention any *output* file, because Pascal-H would ask the TeX user to specify a file name if *output* were specified here.

```
define mtype ≡ t@&y@&p@&e  { this is a WEB coding trick: }
format mtype ≡ type  { 'mtype' will be equivalent to 'type' }
format type ≡ true  { but 'type' will not be treated as a reserved word }
⟨ Compiler directives 9 ⟩
program TEX;  { all file names are defined dynamically }
  label ⟨ Labels in the outer block 6 ⟩
  const ⟨ Constants in the outer block 11 ⟩
  mtype ⟨ Types in the outer block 18 ⟩
  var ⟨ Global variables 13 ⟩
  procedure initialize;  { this procedure gets things started properly }
    var ⟨ Local variables for initialization 19 ⟩
    begin ⟨ Initialize whatever TeX might access 8 ⟩
    end;
  ⟨ Basic printing procedures 57 ⟩
  ⟨ Error handling procedures 82 ⟩
```

5. The overall TeX program begins with the heading just shown, after which comes a bunch of procedure declarations and function declarations. Finally we will get to the main program, which begins with the comment '*start_here*'. If you want to skip down to the main program now, you can look up '*start_here*' in the index. But the author suggests that the best way to understand this program is to follow pretty much the order of TeX's components as they appear in the WEB description you are now reading, since the present ordering is intended to combine the advantages of the "bottom up" and "top down" approaches to the problem of understanding a somewhat complicated system.

6. Three labels must be declared in the main program, so we give them symbolic names.

```
define start_of_TEX = 1  { go here when TeX's variables are initialized }
define end_of_TEX = 9998  { go here to close files and terminate gracefully }
define final_end = 9999  { this label marks the ending of the program }
⟨ Labels in the outer block 6 ⟩ ≡
  start_of_TEX, end_of_TEX, final_end;  { key control points }
```

This code is used in section 4.

7. Some of the code below is intended to be used only when diagnosing the strange behavior that sometimes occurs when TeX is being installed or when system wizards are fooling around with TeX without quite knowing what they are doing. Such code will not normally be compiled; it is delimited by the codewords '**debug** ... **gubed**', with apologies to people who wish to preserve the purity of English.

Similarly, there is some conditional code delimited by '**stat** ... **tats**' that is intended for use when statistics are to be kept about TeX's memory usage. The **stat** ... **tats** code also implements diagnostic information for `\tracingparagraphs`, `\tracingpages`, and `\tracingrestores`.

define *debug* ≡ `@{` { change this to '*debug* ≡ ' when debugging }
define *gubed* ≡ `@}` { change this to '*gubed* ≡ ' when debugging }
format *debug* ≡ *begin*
format *gubed* ≡ *end*

define *stat* ≡ `@{` { change this to '*stat* ≡ ' when gathering usage statistics }
define *tats* ≡ `@}` { change this to '*tats* ≡ ' when gathering usage statistics }
format *stat* ≡ *begin*
format *tats* ≡ *end*

8. This program has two important variations: (1) There is a long and slow version called `INITEX`, which does the extra calculations needed to initialize TeX's internal tables; and (2) there is a shorter and faster production version, which cuts the initialization to a bare minimum. Parts of the program that are needed in (1) but not in (2) are delimited by the codewords '**init** ... **tini**'.

define *init* ≡ { change this to '*init* ≡ `@{`' in the production version }
define *tini* ≡ { change this to '*tini* ≡ `@}`' in the production version }
format *init* ≡ *begin*
format *tini* ≡ *end*

⟨ Initialize whatever TeX might access 8 ⟩ ≡
⟨ Set initial values of key variables 23 ⟩
init ⟨ Initialize table entries (done by `INITEX` only) 189 ⟩ **tini**

This code is used in section 4.

9. If the first character of a Pascal comment is a dollar sign, Pascal-H treats the comment as a list of "compiler directives" that will affect the translation of this program into machine language. The directives shown below specify full checking and inclusion of the Pascal debugger when TeX is being debugged, but they cause range checking and other redundant code to be eliminated when the production system is being generated. Arithmetic overflow will be detected in all cases.

⟨ Compiler directives 9 ⟩ ≡
`@{@&$`$C-, A+, D-$`@}` { no range check, catch arithmetic overflow, no debug overhead }
debug `@{@&$`$C+, D+$`@}` **gubed** { but turn everything on when debugging }

This code is used in section 4.

10. This TeX implementation conforms to the rules of the *Pascal User Manual* published by Jensen and Wirth in 1975, except where system-dependent code is necessary to make a useful system program, and except in another respect where such conformity would unnecessarily obscure the meaning and clutter up the code: We assume that **case** statements may include a default case that applies if no matching label is found. Thus, we shall use constructions like

case x **of**
1: ⟨ code for $x = 1$ ⟩;
3: ⟨ code for $x = 3$ ⟩;
othercases ⟨ code for $x \neq 1$ and $x \neq 3$ ⟩
endcases

since most Pascal compilers have plugged this hole in the language by incorporating some sort of default mechanism. For example, the Pascal-H compiler allows '*others*:' as a default label, and other Pascals allow syntaxes like '**else**' or '**otherwise**' or '*otherwise*:', etc. The definitions of **othercases** and **endcases** should be changed to agree with local conventions. Note that no semicolon appears before **endcases** in this program, so the definition of **endcases** should include a semicolon if the compiler wants one. (Of course, if no default mechanism is available, the **case** statements of TeX will have to be laboriously extended by listing all remaining cases. People who are stuck with such Pascals have, in fact, done this, successfully but not happily!)

define *othercases* ≡ *others*: { default for cases not listed explicitly }
define *endcases* ≡ **end** { follows the default case in an extended **case** statement }
format *othercases* ≡ *else*
format *endcases* ≡ *end*

11. The following parameters can be changed at compile time to extend or reduce TeX's capacity. They may have different values in `INITEX` and in production versions of TeX.

⟨ Constants in the outer block 11 ⟩ ≡

mem_max = 30000;
{ greatest index in TeX's internal *mem* array; must be strictly less than *max_halfword*; must be equal to *mem_top* in `INITEX`, otherwise ≥ *mem_top* }
mem_min = 0; { smallest index in TeX's internal *mem* array; must be *min_halfword* or more; must be equal to *mem_bot* in `INITEX`, otherwise ≤ *mem_bot* }
buf_size = 500; { maximum number of characters simultaneously present in current lines of open files and in control sequences between `\csname` and `\endcsname`; must not exceed *max_halfword* }
error_line = 72; { width of context lines on terminal error messages }
half_error_line = 42; { width of first lines of contexts in terminal error messages; should be between 30 and *error_line* − 15 }
max_print_line = 79; { width of longest text lines output; should be at least 60 }
stack_size = 200; { maximum number of simultaneous input sources }
max_in_open = 6;
{ maximum number of input files and error insertions that can be going on simultaneously }
font_max = 75; { maximum internal font number; must not exceed *max_quarterword* and must be at most *font_base* + 256 }
font_mem_size = 20000; { number of words of *font_info* for all fonts }
param_size = 60; { maximum number of simultaneous macro parameters }
nest_size = 40; { maximum number of semantic levels simultaneously active }
max_strings = 3000; { maximum number of strings; must not exceed *max_halfword* }
string_vacancies = 8000; { the minimum number of characters that should be available for the user's control sequences and font names, after TeX's own error messages are stored }
pool_size = 32000; { maximum number of characters in strings, including all error messages and help texts, and the names of all fonts and control sequences; must exceed *string_vacancies* by the total length of TeX's own strings, which is currently about 23000 }
save_size = 600; { space for saving values outside of current group; must be at most *max_halfword* }
trie_size = 8000; { space for hyphenation patterns; should be larger for `INITEX` than it is in production versions of TeX }
trie_op_size = 500; { space for "opcodes" in the hyphenation patterns }
dvi_buf_size = 800; { size of the output buffer; must be a multiple of 8 }
file_name_size = 40; { file names shouldn't be longer than this }
pool_name = `'TeXformats:TEX.POOL␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣'`;
{ string of length *file_name_size*; tells where the string pool appears }

This code is used in section 4.

12. Like the preceding parameters, the following quantities can be changed at compile time to extend or reduce TeX's capacity. But if they are changed, it is necessary to rerun the initialization program `INITEX` to generate new tables for the production TeX program. One can't simply make helter-skelter changes to the following constants, since certain rather complex initialization numbers are computed from them. They are defined here using `WEB` macros, instead of being put into Pascal's **const** list, in order to emphasize this distinction.

define *mem_bot* = 0
{ smallest index in the *mem* array dumped by `INITEX`; must not be less than *mem_min* }
define *mem_top* ≡ 30000 { largest index in the *mem* array dumped by `INITEX`; must be substantially larger than *mem_bot* and not greater than *mem_max* }
define *font_base* = 0 { smallest internal font number; must not be less than *min_quarterword* }
define *hash_size* = 2100 { maximum number of control sequences; it should be at most about (*mem_max* − *mem_min*)/10 }
define *hash_prime* = 1777 { a prime number equal to about 85% of *hash_size* }
define *hyph_size* = 307 { another prime; the number of `\hyphenation` exceptions }
define *biggest_char* = 65535 { the largest allowed character number; must be ≤ *max_quarterword*, this refers to UTF16 codepoints that we store in strings, etc; actual character codes can exceed this range, up to *biggest_usv* }
define *too_big_char* = 65536 { *biggest_char* + 1 }
define *biggest_usv* = "10FFFF { the largest Unicode Scalar Value }
define *too_big_usv* = "110000 { *biggest_usv* + 1 }
define *number_usvs* = "110000 { *biggest_usv* + 1 }
define *special_char* = "110001 { *biggest_usv* + 2 }
define *biggest_reg* = 255 { the largest allowed register number; must be ≤ *max_quarterword* }
define *number_regs* = 256 { *biggest_reg* + 1 }
define *font_biggest* = 255 { the real biggest font }
define *number_fonts* = *font_biggest* − *font_base* + 2
define *number_math_families* = 256
define *number_math_fonts* = *number_math_families* + *number_math_families* + *number_math_families*
define *math_font_biggest* = *number_math_fonts* − 1
define *text_size* = 0 { size code for the largest size in a family }
define *script_size* = *number_math_families* { size code for the medium size in a family }
define *script_script_size* = *number_math_families* + *number_math_families*
{ size code for the smallest size in a family }
define *biggest_lang* = 255 { the largest hyphenation language }
define *too_big_lang* = 256 { *biggest_lang* + 1 }
define *hyphenatable_length_limit* = 4095
{ hard limit for hyphenatable length; runtime value is *max_hyphenatable_length* }

13. In case somebody has inadvertently made bad settings of the "constants," TeX checks them using a global variable called *bad*.

This is the first of many sections of TeX where global variables are defined.

⟨ Global variables 13 ⟩ ≡
bad: *integer*; { is some "constant" wrong? }

See also sections 20, 26, 30, 32, 39, 50, 54, 61, 77, 80, 83, 100, 108, 114, 121, 137, 138, 139, 140, 146, 181, 190, 199, 207, 239, 272, 279, 282, 283, 301, 316, 327, 331, 334, 335, 338, 339, 340, 363, 391, 397, 416, 421, 422, 444, 472, 481, 515, 524, 528, 547, 548, 555, 562, 567, 574, 584, 585, 590, 628, 631, 641, 652, 682, 685, 686, 695, 703, 726, 762, 767, 812, 818, 862, 869, 871, 873, 876, 881, 887, 895, 920, 940, 953, 959, 961, 975, 980, 997, 1001, 1004, 1025, 1034, 1036, 1043, 1086, 1128, 1320, 1335, 1353, 1359, 1385, 1396, 1400, 1429, 1449, 1462, 1470, 1515, 1561, 1584, 1625, 1627, 1646, 1653, 1669, and 1670.

This code is used in section 4.

14. Later on we will say '**if** *mem_max* ≥ *max_halfword* **then** *bad* ← 14', or something similar. (We can't do that until *max_halfword* has been defined.)

```
⟨ Check the "constant" values for consistency 14 ⟩ ≡
  bad ← 0;
  if (half_error_line < 30) ∨ (half_error_line > error_line − 15) then bad ← 1;
  if max_print_line < 60 then bad ← 2;
  if dvi_buf_size mod 8 ≠ 0 then bad ← 3;
  if mem_bot + 1100 > mem_top then bad ← 4;
  if hash_prime > hash_size then bad ← 5;
  if max_in_open ≥ 128 then bad ← 6;
  if mem_top < 256 + 11 then bad ← 7;  { we will want null_list > 255 }
```

See also sections 133, 320, 557, and 1303.

This code is used in section 1386.

15. Labels are given symbolic names by the following definitions, so that occasional **goto** statements will be meaningful. We insert the label '*exit*' just before the '**end**' of a procedure in which we have used the '**return**' statement defined below; the label '*restart*' is occasionally used at the very beginning of a procedure; and the label '*reswitch*' is occasionally used just prior to a **case** statement in which some cases change the conditions and we wish to branch to the newly applicable case. Loops that are set up with the **loop** construction defined below are commonly exited by going to '*done*' or to '*found*' or to '*not_found*', and they are sometimes repeated by going to '*continue*'. If two or more parts of a subroutine start differently but end up the same, the shared code may be gathered together at '*common_ending*'.

Incidentally, this program never declares a label that isn't actually used, because some fussy Pascal compilers will complain about redundant labels.

```
define exit = 10  { go here to leave a procedure }
define restart = 20  { go here to start a procedure again }
define reswitch = 21  { go here to start a case statement again }
define continue = 22  { go here to resume a loop }
define done = 30  { go here to exit a loop }
define done1 = 31  { like done, when there is more than one loop }
define done2 = 32  { for exiting the second loop in a long block }
define done3 = 33  { for exiting the third loop in a very long block }
define done4 = 34  { for exiting the fourth loop in an extremely long block }
define done5 = 35  { for exiting the fifth loop in an immense block }
define done6 = 36  { for exiting the sixth loop in a block }
define found = 40  { go here when you've found it }
define found1 = 41  { like found, when there's more than one per routine }
define found2 = 42  { like found, when there's more than two per routine }
define not_found = 45  { go here when you've found nothing }
define not_found1 = 46  { like not_found, when there's more than one }
define not_found2 = 47  { like not_found, when there's more than two }
define not_found3 = 48  { like not_found, when there's more than three }
define not_found4 = 49  { like not_found, when there's more than four }
define common_ending = 50  { go here when you want to merge with another branch }
```

16. Here are some macros for common programming idioms.

```
define incr(#) ≡ # ← # + 1   { increase a variable by unity }
define decr(#) ≡ # ← # − 1   { decrease a variable by unity }
define negate(#) ≡ # ← −#   { change the sign of a variable }
define loop ≡ while true do   { repeat over and over until a goto happens }
format loop ≡ xclause   { WEB's xclause acts like 'while true do' }
define do_nothing ≡   { empty statement }
define return ≡ goto exit   { terminate a procedure call }
format return ≡ nil
define empty = 0   { symbolic name for a null constant }
```

17. The character set. In order to make TeX readily portable to a wide variety of computers, all of its input text is converted to an internal eight-bit code that includes standard ASCII, the "American Standard Code for Information Interchange." This conversion is done immediately when each character is read in. Conversely, characters are converted from ASCII to the user's external representation just before they are output to a text file.

Such an internal code is relevant to users of TeX primarily because it governs the positions of characters in the fonts. For example, the character 'A' has ASCII code $65 = \acute{}101$, and when TeX typesets this letter it specifies character number 65 in the current font. If that font actually has 'A' in a different position, TeX doesn't know what the real position is; the program that does the actual printing from TeX's device-independent files is responsible for converting from ASCII to a particular font encoding.

TeX's internal code also defines the value of constants that begin with a reverse apostrophe; and it provides an index to the `\catcode`, `\mathcode`, `\uccode`, `\lccode`, and `\delcode` tables.

18. Characters of text that have been converted to TeX's internal form are said to be of type *ASCII_code*, which is a subrange of the integers. For xetex, we rename *ASCII_code* as *UTF16_code*. But we also have a new type *UTF8_code*, used when we construct filenames to pass to the system libraries.

define *ASCII_code* ≡ *UTF16_code*
define *packed_ASCII_code* ≡ *packed_UTF16_code*

⟨ Types in the outer block 18 ⟩ ≡
ASCII_code = 0 .. *biggest_char*; { 16-bit numbers }
UTF8_code = 0 .. 255; { 8-bit numbers }
UnicodeScalar = 0 .. *biggest_usv*; { Unicode scalars }

See also sections 25, 38, 105, 113, 135, 174, 238, 299, 330, 583, 630, 974, 979, and 1488.

This code is used in section 4.

19. The original Pascal compiler was designed in the late 60s, when six-bit character sets were common, so it did not make provision for lowercase letters. Nowadays, of course, we need to deal with both capital and small letters in a convenient way, especially in a program for typesetting; so the present specification of TeX has been written under the assumption that the Pascal compiler and run-time system permit the use of text files with more than 64 distinguishable characters. More precisely, we assume that the character set contains at least the letters and symbols associated with ASCII codes $\acute{}40$ through $\acute{}176$; all of these characters are now available on most computer terminals.

Since we are dealing with more characters than were present in the first Pascal compilers, we have to decide what to call the associated data type. Some Pascals use the original name *char* for the characters in text files, even though there now are more than 64 such characters, while other Pascals consider *char* to be a 64-element subrange of a larger data type that has some other name.

In order to accommodate this difference, we shall use the name *text_char* to stand for the data type of the characters that are converted to and from *ASCII_code* when they are input and output. We shall also assume that *text_char* consists of the elements *chr*(*first_text_char*) through *chr*(*last_text_char*), inclusive. The following definitions should be adjusted if necessary.

define *text_char* ≡ *char* { the data type of characters in text files }
define *first_text_char* = 0 { ordinal number of the smallest element of *text_char* }
define *last_text_char* = *biggest_char* { ordinal number of the largest element of *text_char* }

⟨ Local variables for initialization 19 ⟩ ≡
i: *integer*;

See also sections 188 and 981.

This code is used in section 4.

20. The TeX processor converts between ASCII code and the user's external character set by means of arrays *xord* and *xchr* that are analogous to Pascal's *ord* and *chr* functions.

⟨ Global variables 13 ⟩ +≡
xchr: **array** [*ASCII_code*] **of** *text_char*; { specifies conversion of output characters }

21. Since we are assuming that our Pascal system is able to read and write the visible characters of standard ASCII (although not necessarily using the ASCII codes to represent them), the following assignment statements initialize the standard part of the *xchr* array properly, without needing any system-dependent changes. On the other hand, it is possible to implement TeX with less complete character sets, and in such cases it will be necessary to change something here.

22. Some of the ASCII codes without visible characters have been given symbolic names in this program because they are used with a special meaning.

define *null_code* = ´0 { ASCII code that might disappear }
define *carriage_return* = ´15 { ASCII code used at end of line }
define *invalid_code* = ´177 { ASCII code that many systems prohibit in text files }

23. The ASCII code is "standard" only to a certain extent, since many computer installations have found it advantageous to have ready access to more than 94 printing characters. Appendix C of *The TeXbook* gives a complete specification of the intended correspondence between characters and TeX's internal representation.

If TeX is being used on a garden-variety Pascal for which only standard ASCII codes will appear in the input and output files, it doesn't really matter what codes are specified in *xchr*[0 .. ´37], but the safest policy is to blank everything out by using the code shown below.

However, other settings of *xchr* will make TeX more friendly on computers that have an extended character set, so that users can type things like '≠' instead of '`\ne`'. People with extended character sets can assign codes arbitrarily, giving an *xchr* equivalent to whatever characters the users of TeX are allowed to have in their input files. It is best to make the codes correspond to the intended interpretations as shown in Appendix C whenever possible; but this is not necessary. For example, in countries with an alphabet of more than 26 letters, it is usually best to map the additional letters into codes less than ´40. To get the most "permissive" character set, change ´␣´ on the right of these assignment statements to *chr*(*i*).

⟨ Set initial values of key variables 23 ⟩ ≡
for *i* ← 0 **to** ´37 **do** *xchr*[*i*] ← ´␣´;
for *i* ← ´177 **to** ´377 **do** *xchr*[*i*] ← ´␣´;

See also sections 24, 62, 78, 81, 84, 101, 122, 191, 241, 280, 284, 302, 317, 398, 417, 473, 516, 525, 556, 586, 591, 629, 632, 642, 687, 696, 704, 727, 819, 941, 982, 1044, 1087, 1321, 1336, 1354, 1397, 1412, 1516, 1562, 1628, 1647, and 1671.

This code is used in section 8.

24. The following system-independent code makes the *xord* array contain a suitable inverse to the information in *xchr*. Note that if *xchr*[*i*] = *xchr*[*j*] where *i* < *j* < ´177, the value of *xord*[*xchr*[*i*]] will turn out to be *j* or more; hence, standard ASCII code numbers will be used instead of codes below ´40 in case there is a coincidence.

⟨ Set initial values of key variables 23 ⟩ +≡
for *i* ← 0 **to** ´176 **do** *xord*[*xchr*[*i*]] ← *i*;

25. Input and output. The bane of portability is the fact that different operating systems treat input and output quite differently, perhaps because computer scientists have not given sufficient attention to this problem. People have felt somehow that input and output are not part of "real" programming. Well, it is true that some kinds of programming are more fun than others. With existing input/output conventions being so diverse and so messy, the only sources of joy in such parts of the code are the rare occasions when one can find a way to make the program a little less bad than it might have been. We have two choices, either to attack I/O now and get it over with, or to postpone I/O until near the end. Neither prospect is very attractive, so let's get it over with.

The basic operations we need to do are (1) inputting and outputting of text, to or from a file or the user's terminal; (2) inputting and outputting of eight-bit bytes, to or from a file; (3) instructing the operating system to initiate ("open") or to terminate ("close") input or output from a specified file; (4) testing whether the end of an input file has been reached.

TeX needs to deal with two kinds of files. We shall use the term *alpha_file* for a file that contains textual data, and the term *byte_file* for a file that contains eight-bit binary information. These two types turn out to be the same on many computers, but sometimes there is a significant distinction, so we shall be careful to distinguish between them. Standard protocols for transferring such files from computer to computer, via high-speed networks, are now becoming available to more and more communities of users.

The program actually makes use also of a third kind of file, called a *word_file*, when dumping and reloading base information for its own initialization. We shall define a word file later; but it will be possible for us to specify simple operations on word files before they are defined.

```
⟨ Types in the outer block 18 ⟩ +≡
  eight_bits = 0 .. 255;  { unsigned one-byte quantity }
  alpha_file = packed file of text_char;  { files that contain textual data }
  byte_file = packed file of eight_bits;  { files that contain binary data }
```

26. Most of what we need to do with respect to input and output can be handled by the I/O facilities that are standard in Pascal, i.e., the routines called *get*, *put*, *eof*, and so on. But standard Pascal does not allow file variables to be associated with file names that are determined at run time, so it cannot be used to implement TeX; some sort of extension to Pascal's ordinary *reset* and *rewrite* is crucial for our purposes. We shall assume that *name_of_file* is a variable of an appropriate type such that the Pascal run-time system being used to implement TeX can open a file whose external name is specified by *name_of_file*.

```
⟨ Global variables 13 ⟩ +≡
name_of_file: packed array [1 .. file_name_size] of char;
    { on some systems this may be a record variable }
name_of_file16: array [1 .. file_name_size] of UTF16_code;
    { but sometimes we need a UTF16 version of the name }
name_length: 0 .. file_name_size;
    { this many characters are actually relevant in name_of_file (the rest are blank) }
name_length16: 0 .. file_name_size;
```

27. The Pascal-H compiler with which the present version of TEX was prepared has extended the rules of Pascal in a very convenient way. To open file f, we can write

reset$(f, name, \texttt{'/O'})$ for input;
rewrite$(f, name, \texttt{'/O'})$ for output.

The '*name*' parameter, which is of type '**packed array** [⟨*any*⟩] **of** *char*', stands for the name of the external file that is being opened for input or output. Blank spaces that might appear in *name* are ignored.

The '/O' parameter tells the operating system not to issue its own error messages if something goes wrong. If a file of the specified name cannot be found, or if such a file cannot be opened for some other reason (e.g., someone may already be trying to write the same file), we will have *erstat*$(f) \neq 0$ after an unsuccessful *reset* or *rewrite*. This allows TEX to undertake appropriate corrective action.

TEX's file-opening procedures return *false* if no file identified by *name_of_file* could be opened.

```
define reset_OK(#) ≡ erstat(#) = 0
define rewrite_OK(#) ≡ erstat(#) = 0
function a_open_in(var f : alpha_file): boolean; { open a text file for input }
  begin reset(f, name_of_file, '/O'); a_open_in ← reset_OK(f);
  end;
function a_open_out(var f : alpha_file): boolean; { open a text file for output }
  begin rewrite(f, name_of_file, '/O'); a_open_out ← rewrite_OK(f);
  end;
function b_open_in(var f : byte_file): boolean; { open a binary file for input }
  begin reset(f, name_of_file, '/O'); b_open_in ← reset_OK(f);
  end;
function b_open_out(var f : byte_file): boolean; { open a binary file for output }
  begin rewrite(f, name_of_file, '/O'); b_open_out ← rewrite_OK(f);
  end;
function w_open_in(var f : word_file): boolean; { open a word file for input }
  begin reset(f, name_of_file, '/O'); w_open_in ← reset_OK(f);
  end;
function w_open_out(var f : word_file): boolean; { open a word file for output }
  begin rewrite(f, name_of_file, '/O'); w_open_out ← rewrite_OK(f);
  end;
```

28. Files can be closed with the Pascal-H routine '*close*(f)', which should be used when all input or output with respect to f has been completed. This makes f available to be opened again, if desired; and if f was used for output, the *close* operation makes the corresponding external file appear on the user's area, ready to be read.

These procedures should not generate error messages if a file is being closed before it has been successfully opened.

```
procedure a_close(var f : alpha_file); { close a text file }
  begin close(f);
  end;
procedure b_close(var f : byte_file); { close a binary file }
  begin close(f);
  end;
procedure w_close(var f : word_file); { close a word file }
  begin close(f);
  end;
```

29. Binary input and output are done with Pascal's ordinary *get* and *put* procedures, so we don't have to make any other special arrangements for binary I/O. Text output is also easy to do with standard Pascal routines. The treatment of text input is more difficult, however, because of the necessary translation to *ASCII_code* values. TeX's conventions should be efficient, and they should blend nicely with the user's operating environment.

30. Input from text files is read one line at a time, using a routine called *input_ln*. This function is defined in terms of global variables called *buffer*, *first*, and *last* that will be described in detail later; for now, it suffices for us to know that *buffer* is an array of *ASCII_code* values, and that *first* and *last* are indices into this array representing the beginning and ending of a line of text.

```
⟨ Global variables 13 ⟩ +≡
buffer: array [0 .. buf_size] of ASCII_code;  { lines of characters being read }
first: 0 .. buf_size;  { the first unused position in buffer }
last: 0 .. buf_size;  { end of the line just input to buffer }
max_buf_stack: 0 .. buf_size;  { largest index used in buffer }
```

31. The *input_ln* function brings the next line of input from the specified file into available positions of the buffer array and returns the value *true*, unless the file has already been entirely read, in which case it returns *false* and sets $last \leftarrow first$. In general, the *ASCII_code* numbers that represent the next line of the file are input into $buffer[first]$, $buffer[first+1]$, ..., $buffer[last-1]$; and the global variable *last* is set equal to *first* plus the length of the line. Trailing blanks are removed from the line; thus, either $last = first$ (in which case the line was entirely blank) or $buffer[last-1] \neq$ "␣".

An overflow error is given, however, if the normal actions of *input_ln* would make $last \geq buf_size$; this is done so that other parts of TeX can safely look at the contents of $buffer[last+1]$ without overstepping the bounds of the *buffer* array. Upon entry to *input_ln*, the condition $first < buf_size$ will always hold, so that there is always room for an "empty" line.

The variable *max_buf_stack*, which is used to keep track of how large the *buf_size* parameter must be to accommodate the present job, is also kept up to date by *input_ln*.

If the *bypass_eoln* parameter is *true*, *input_ln* will do a *get* before looking at the first character of the line; this skips over an *eoln* that was in $f\uparrow$. The procedure does not do a *get* when it reaches the end of the line; therefore it can be used to acquire input from the user's terminal as well as from ordinary text files.

Standard Pascal says that a file should have *eoln* immediately before *eof*, but TeX needs only a weaker restriction: If *eof* occurs in the middle of a line, the system function *eoln* should return a *true* result (even though $f\uparrow$ will be undefined).

Since the inner loop of *input_ln* is part of TeX's "inner loop"—each character of input comes in at this place—it is wise to reduce system overhead by making use of special routines that read in an entire array of characters at once, if such routines are available. The following code uses standard Pascal to illustrate what needs to be done, but finer tuning is often possible at well-developed Pascal sites.

```
function input_ln(var f : alpha_file; bypass_eoln : boolean): boolean;
        { inputs the next line or returns false }
  var last_nonblank: 0 .. buf_size;  { last with trailing blanks removed }
  begin if bypass_eoln then
    if ¬eof(f) then get(f);  { input the first character of the line into f↑ }
  last ← first;  { cf. Matthew 19:30 }
  if eof(f) then input_ln ← false
  else begin last_nonblank ← first;
    while ¬eoln(f) do
      begin if last ≥ max_buf_stack then
        begin max_buf_stack ← last + 1;
        if max_buf_stack = buf_size then ⟨Report overflow of the input buffer, and abort 35⟩;
        end;
      buffer[last] ← xord[f↑]; get(f); incr(last);
      if buffer[last − 1] ≠ "␣" then last_nonblank ← last;
      end;
    last ← last_nonblank; input_ln ← true;
    end;
  end;
```

32. The user's terminal acts essentially like other files of text, except that it is used both for input and for output. When the terminal is considered an input file, the file variable is called *term_in*, and when it is considered an output file the file variable is *term_out*.

```
⟨Global variables 13⟩ +≡
term_in: alpha_file;  { the terminal as an input file }
term_out: alpha_file;  { the terminal as an output file }
```

33. Here is how to open the terminal files in Pascal-H. The '`/I`' switch suppresses the first *get*.

define *t_open_in* ≡ *reset*(*term_in*, '`TTY:`', '`/O/I`') { open the terminal for text input }
define *t_open_out* ≡ *rewrite*(*term_out*, '`TTY:`', '`/O`') { open the terminal for text output }

34. Sometimes it is necessary to synchronize the input/output mixture that happens on the user's terminal, and three system-dependent procedures are used for this purpose. The first of these, *update_terminal*, is called when we want to make sure that everything we have output to the terminal so far has actually left the computer's internal buffers and been sent. The second, *clear_terminal*, is called when we wish to cancel any input that the user may have typed ahead (since we are about to issue an unexpected error message). The third, *wake_up_terminal*, is supposed to revive the terminal if the user has disabled it by some instruction to the operating system. The following macros show how these operations can be specified in Pascal-H:

define *update_terminal* ≡ *break*(*term_out*) { empty the terminal output buffer }
define *clear_terminal* ≡ *break_in*(*term_in*, *true*) { clear the terminal input buffer }
define *wake_up_terminal* ≡ *do_nothing* { cancel the user's cancellation of output }

35. We need a special routine to read the first line of TeX input from the user's terminal. This line is different because it is read before we have opened the transcript file; there is sort of a "chicken and egg" problem here. If the user types '`\input paper`' on the first line, or if some macro invoked by that line does such an `\input`, the transcript file will be named '`paper.log`'; but if no `\input` commands are performed during the first line of terminal input, the transcript file will acquire its default name '`texput.log`'. (The transcript file will not contain error messages generated by the first line before the first `\input` command.)

The first line is even more special if we are lucky enough to have an operating system that treats TeX differently from a run-of-the-mill Pascal object program. It's nice to let the user start running a TeX job by typing a command line like '`tex paper`'; in such a case, TeX will operate as if the first line of input were '`paper`', i.e., the first line will consist of the remainder of the command line, after the part that invoked TeX.

The first line is special also because it may be read before TeX has input a format file. In such cases, normal error messages cannot yet be given. The following code uses concepts that will be explained later. (If the Pascal compiler does not support non-local **goto**, the statement '**goto** *final_end*' should be replaced by something that quietly terminates the program.)

⟨ Report overflow of the input buffer, and abort 35 ⟩ ≡

```
if format_ident = 0 then
  begin write_ln(term_out, 'Buffer␣size␣exceeded!'); goto final_end;
  end
else begin cur_input.loc_field ← first; cur_input.limit_field ← last − 1;
  overflow("buffer␣size", buf_size);
  end
```

This code is used in sections 31 and 1567.

36. Different systems have different ways to get started. But regardless of what conventions are adopted, the routine that initializes the terminal should satisfy the following specifications:

1) It should open file *term_in* for input from the terminal. (The file *term_out* will already be open for output to the terminal.)
2) If the user has given a command line, this line should be considered the first line of terminal input. Otherwise the user should be prompted with '`**`', and the first line of input should be whatever is typed in response.
3) The first line of input, which might or might not be a command line, should appear in locations *first* to *last* − 1 of the *buffer* array.
4) The global variable *loc* should be set so that the character to be read next by TeX is in *buffer*[*loc*]. This character should not be blank, and we should have *loc* < *last*.

(It may be necessary to prompt the user several times before a non-blank line comes in. The prompt is '`**`' instead of the later '`*`' because the meaning is slightly different: '`\input`' need not be typed immediately after '`**`'.)

define *loc* ≡ *cur_input.loc_field* { location of first unread character in *buffer* }

37. The following program does the required initialization without retrieving a possible command line. It should be clear how to modify this routine to deal with command lines, if the system permits them.

```
function init_terminal: boolean; { gets the terminal input started }
  label exit;
  begin t_open_in;
  loop begin wake_up_terminal; write(term_out, '**'); update_terminal;
    if ¬input_ln(term_in, true) then { this shouldn't happen }
      begin write_ln(term_out); write(term_out, '! End of file on the terminal... why?');
      init_terminal ← false; return;
      end;
    loc ← first;
    while (loc < last) ∧ (buffer[loc] = " ") do incr(loc);
    if loc < last then
      begin init_terminal ← true; return; { return unless the line was all blank }
      end;
    write_ln(term_out, 'Please type the name of your input file.');
    end;
exit: end;
```

38. String handling. Control sequence names and diagnostic messages are variable-length strings of eight-bit characters. Since Pascal does not have a well-developed string mechanism, TeX does all of its string processing by homegrown methods.

Elaborate facilities for dynamic strings are not needed, so all of the necessary operations can be handled with a simple data structure. The array *str_pool* contains all of the (eight-bit) ASCII codes in all of the strings, and the array *str_start* contains indices of the starting points of each string. Strings are referred to by integer numbers, so that string number s comprises the characters *str_pool*$[j]$ for *str_start_macro*$[s] \le j <$ *str_start_macro*$[s+1]$. Additional integer variables *pool_ptr* and *str_ptr* indicate the number of entries used so far in *str_pool* and *str_start*, respectively; locations *str_pool*$[$*pool_ptr*$]$ and *str_start_macro*$[$*str_ptr*$]$ are ready for the next string to be allocated.

String numbers 0 to 255 are reserved for strings that correspond to single ASCII characters. This is in accordance with the conventions of WEB, which converts single-character strings into the ASCII code number of the single character involved, while it converts other strings into integers and builds a string pool file. Thus, when the string constant `"."` appears in the program below, WEB converts it into the integer 46, which is the ASCII code for a period, while WEB will convert a string like `"hello"` into some integer greater than 255. String number 46 will presumably be the single character '.'; but some ASCII codes have no standard visible representation, and TeX sometimes needs to be able to print an arbitrary ASCII character, so the first 256 strings are used to specify exactly what should be printed for each of the 256 possibilities.

Elements of the *str_pool* array must be ASCII codes that can actually be printed; i.e., they must have an *xchr* equivalent in the local character set. (This restriction applies only to preloaded strings, not to those generated dynamically by the user.)

Some Pascal compilers won't pack integers into a single byte unless the integers lie in the range $-128 \,..\, 127$. To accommodate such systems we access the string pool only via macros that can easily be redefined.

```
define si(#) ≡ #   { convert from ASCII_code to packed_ASCII_code }
define so(#) ≡ #   { convert from packed_ASCII_code to ASCII_code }
define str_start_macro(#) ≡ str_start[(#) − too_big_char]
```

```
⟨ Types in the outer block 18 ⟩ +≡
  pool_pointer = 0 .. pool_size;   { for variables that point into str_pool }
  str_number = 0 .. max_strings;   { for variables that point into str_start }
  packed_ASCII_code = 0 .. biggest_char;   { elements of str_pool array }
```

39.
```
⟨ Global variables 13 ⟩ +≡
str_pool: packed array [pool_pointer] of packed_ASCII_code;   { the characters }
str_start: array [str_number] of pool_pointer;   { the starting pointers }
pool_ptr: pool_pointer;   { first unused position in str_pool }
str_ptr: str_number;   { number of the current string being created }
init_pool_ptr: pool_pointer;   { the starting value of pool_ptr }
init_str_ptr: str_number;   { the starting value of str_ptr }
```

40. Several of the elementary string operations are performed using WEB macros instead of Pascal procedures, because many of the operations are done quite frequently and we want to avoid the overhead of procedure calls. For example, here is a simple macro that computes the length of a string.

```
function length(s : str_number): integer;   { the number of characters in string number s }
  begin if (s ≥ "10000) then length ← str_start_macro(s + 1) − str_start_macro(s)
  else if (s ≥ "20) ∧ (s < "7F) then length ← 1
    else if (s ≤ "7F) then length ← 3
      else if (s < "100) then length ← 4
        else length ← 8
  end;
```

41. The length of the current string is called *cur_length*:

define *cur_length* ≡ (*pool_ptr* − *str_start_macro*(*str_ptr*))

42. Strings are created by appending character codes to *str_pool*. The *append_char* macro, defined here, does not check to see if the value of *pool_ptr* has gotten too high; this test is supposed to be made before *append_char* is used. There is also a *flush_char* macro, which erases the last character appended.

To test if there is room to append l more characters to *str_pool*, we shall write *str_room*(l), which aborts TeX and gives an apologetic error message if there isn't enough room.

```
define append_char(#) ≡   { put ASCII_code # at the end of str_pool }
        begin str_pool[pool_ptr] ← si(#); incr(pool_ptr);
        end
define flush_char ≡ decr(pool_ptr)   { forget the last character in the pool }
define str_room(#) ≡   { make sure that the pool hasn't overflowed }
        begin if pool_ptr + # > pool_size then overflow("pool␣size", pool_size − init_pool_ptr);
        end
```

43. Once a sequence of characters has been appended to *str_pool*, it officially becomes a string when the function *make_string* is called. This function returns the identification number of the new string as its value.

```
function make_string: str_number;   { current string enters the pool }
  begin if str_ptr = max_strings then overflow("number␣of␣strings", max_strings − init_str_ptr);
  incr(str_ptr); str_start_macro(str_ptr) ← pool_ptr; make_string ← str_ptr − 1;
  end;
```

44. To destroy the most recently made string, we say *flush_string*.

```
define flush_string ≡
          begin decr(str_ptr); pool_ptr ← str_start_macro(str_ptr);
          end
procedure append_str(s : str_number);   { append an existing string to the current string }
  var i: integer; j: pool_pointer;
  begin i ← length(s); str_room(i); j ← str_start_macro(s);
  while (i > 0) do
    begin append_char(str_pool[j]); incr(j); decr(i);
    end;
  end;
```

45. The following subroutine compares string s with another string of the same length that appears in *buffer* starting at position k; the result is *true* if and only if the strings are equal. Empirical tests indicate that *str_eq_buf* is used in such a way that it tends to return *true* about 80 percent of the time.

```
function str_eq_buf(s : str_number; k : integer): boolean;  { test equality of strings }
  label not_found;  { loop exit }
  var j: pool_pointer;  { running index }
    result: boolean;  { result of comparison }
  begin j ← str_start_macro(s);
  while j < str_start_macro(s + 1) do
    begin if buffer[k] ≥ "10000 then
      if so(str_pool[j]) ≠ "D800 + (buffer[k] − "10000) div "400 then
        begin result ← false; goto not_found;
        end
      else if so(str_pool[j + 1]) ≠ "DC00 + (buffer[k] − "10000) mod "400 then
          begin result ← false; goto not_found;
          end
        else incr(j)
    else if so(str_pool[j]) ≠ buffer[k] then
        begin result ← false; goto not_found;
        end;
    incr(j); incr(k);
    end;
  result ← true;
not_found: str_eq_buf ← result;
  end;
```

46. Here is a similar routine, but it compares two strings in the string pool, and it does not assume that they have the same length.

```
function str_eq_str(s,t : str_number): boolean;  { test equality of strings }
  label not_found;  { loop exit }
  var j,k: pool_pointer;  { running indices }
    result: boolean;  { result of comparison }
  begin result ← false;
  if length(s) ≠ length(t) then goto not_found;
  if (length(s) = 1) then
    begin if s < 65536 then
      begin if t < 65536 then
        begin if s ≠ t then goto not_found;
        end
      else begin if s ≠ str_pool[str_start_macro(t)] then goto not_found;
        end;
      end
    else begin if t < 65536 then
        begin if str_pool[str_start_macro(s)] ≠ t then goto not_found;
        end
      else begin if str_pool[str_start_macro(s)] ≠ str_pool[str_start_macro(t)] then goto not_found;
        end;
      end;
    end
  else begin j ← str_start_macro(s); k ← str_start_macro(t);
    while j < str_start_macro(s + 1) do
      begin if str_pool[j] ≠ str_pool[k] then goto not_found;
      incr(j); incr(k);
      end;
    end;
  result ← true;
not_found: str_eq_str ← result;
  end;
```

47. The initial values of *str_pool*, *str_start*, *pool_ptr*, and *str_ptr* are computed by the INITEX program, based in part on the information that WEB has output while processing TeX.

```
  init function get_strings_started: boolean;
        { initializes the string pool, but returns false if something goes wrong }
  label done, exit;
  var m,n: text_char;  { characters input from pool_file }
    g: str_number;  { garbage }
    a: integer;  { accumulator for check sum }
    c: boolean;  { check sum has been checked }
  begin pool_ptr ← 0; str_ptr ← 0; str_start[0] ← 0; ⟨ Make the first 256 strings 48 ⟩;
  ⟨ Read the other strings from the TEX.POOL file and return true, or give an error message and return
      false 51 ⟩;
exit: end;
  tini
```

48. The first 65536 strings will consist of a single character only. But we don't actually make them; they're simulated on the fly.

⟨ Make the first 256 strings 48 ⟩ ≡
```
begin str_ptr ← too_big_char; str_start_macro(str_ptr) ← pool_ptr;
end
```
This code is used in section 47.

49. The first 128 strings will contain 95 standard ASCII characters, and the other 33 characters will be printed in three-symbol form like '^^A' unless a system-dependent change is made here. Installations that have an extended character set, where for example *xchr*[*'32*] = *'≠'*, would like string *'32* to be the single character *'32* instead of the three characters *'136*, *'136*, *'132* (^^Z). On the other hand, even people with an extended character set will want to represent string *'15* by ^^M, since *'15* is *carriage_return*; the idea is to produce visible strings instead of tabs or line-feeds or carriage-returns or bell-rings or characters that are treated anomalously in text files.

Unprintable characters of codes 128–255 are, similarly, rendered ^^80–^^ff.

The boolean expression defined here should be *true* unless TeX internal code number k corresponds to a non-troublesome visible symbol in the local character set. An appropriate formula for the extended character set recommended in *The TeXbook* would, for example, be '$k \in [0, '10 \mathinner{..} '12, '14, '15, '33, '177 \mathinner{..} '377]$'. If character k cannot be printed, and $k < '200$, then character $k + '100$ or $k - '100$ must be printable; moreover, ASCII codes $['41 \mathinner{..} '46, '60 \mathinner{..} '71, '136, '141 \mathinner{..} '146, '160 \mathinner{..} '171]$ must be printable. Thus, at least 80 printable characters are needed.

50. When the WEB system program called TANGLE processes the TEX.WEB description that you are now reading, it outputs the Pascal program TEX.PAS and also a string pool file called TEX.POOL. The INITEX program reads the latter file, where each string appears as a two-digit decimal length followed by the string itself, and the information is recorded in TeX's string memory.

⟨ Global variables 13 ⟩ +≡
```
init pool_file: alpha_file; { the string-pool file output by TANGLE }
tini
```

51.
```
define bad_pool(#) ≡
    begin wake_up_terminal; write_ln(term_out, #); a_close(pool_file); get_strings_started ← false;
    return;
    end
```
⟨ Read the other strings from the TEX.POOL file and return *true*, or give an error message and return *false* 51 ⟩ ≡
```
name_of_file ← pool_name; { we needn't set name_length }
if a_open_in(pool_file) then
  begin c ← false;
  repeat ⟨ Read one string, but return false if the string memory space is getting too tight for
      comfort 52 ⟩;
  until c;
  a_close(pool_file); get_strings_started ← true;
  end
else bad_pool('!␣I␣can't␣read␣TEX.POOL.')
```
This code is used in section 47.

52. ⟨ Read one string, but return *false* if the string memory space is getting too tight for comfort 52 ⟩ ≡

```
begin if eof(pool_file) then bad_pool('!␣TEX.POOL␣has␣no␣check␣sum.');
read(pool_file, m, n); { read two digits of string length }
if m = '*' then ⟨ Check the pool check sum 53 ⟩
else begin if (xord[m] < "0") ∨ (xord[m] > "9") ∨ (xord[n] < "0") ∨ (xord[n] > "9") then
    bad_pool('!␣TEX.POOL␣line␣doesn''t␣begin␣with␣two␣digits.');
  l ← xord[m] * 10 + xord[n] − "0" * 11; { compute the length }
  if pool_ptr + l + string_vacancies > pool_size then bad_pool('!␣You␣have␣to␣increase␣POOLSIZE.');
  for k ← 1 to l do
    begin if eoln(pool_file) then m ← '␣' else read(pool_file, m);
    append_char(xord[m]);
    end;
  read_ln(pool_file); g ← make_string;
  end;
end
```

This code is used in section 51.

53. The WEB operation @$ denotes the value that should be at the end of this TEX.POOL file; any other value means that the wrong pool file has been loaded.

⟨ Check the pool check sum 53 ⟩ ≡

```
begin a ← 0; k ← 1;
loop begin if (xord[n] < "0") ∨ (xord[n] > "9") then
    bad_pool('!␣TEX.POOL␣check␣sum␣doesn''t␣have␣nine␣digits.');
  a ← 10 * a + xord[n] − "0";
  if k = 9 then goto done;
  incr(k); read(pool_file, n);
  end;
done: if a ≠ @$ then bad_pool('!␣TEX.POOL␣doesn''t␣match;␣TANGLE␣me␣again.');
  c ← true;
end
```

This code is used in section 52.

54. On-line and off-line printing. Messages that are sent to a user's terminal and to the transcript-log file are produced by several '*print*' procedures. These procedures will direct their output to a variety of places, based on the setting of the global variable *selector*, which has the following possible values:

term_and_log, the normal setting, prints on the terminal and on the transcript file.
log_only, prints only on the transcript file.
term_only, prints only on the terminal.
no_print, doesn't print at all. This is used only in rare cases before the transcript file is open.
pseudo, puts output into a cyclic buffer that is used by the *show_context* routine; when we get to that routine we shall discuss the reasoning behind this curious mode.
new_string, appends the output to the current string in the string pool.
0 to 15, prints on one of the sixteen files for `\write` output.

The symbolic names '*term_and_log*', etc., have been assigned numeric codes that satisfy the convenient relations $no_print + 1 = term_only$, $no_print + 2 = log_only$, $term_only + 2 = log_only + 1 = term_and_log$.

Three additional global variables, *tally* and *term_offset* and *file_offset*, record the number of characters that have been printed since they were most recently cleared to zero. We use *tally* to record the length of (possibly very long) stretches of printing; *term_offset* and *file_offset*, on the other hand, keep track of how many characters have appeared so far on the current line that has been output to the terminal or to the transcript file, respectively.

define *no_print* = 16 { *selector* setting that makes data disappear }
define *term_only* = 17 { printing is destined for the terminal only }
define *log_only* = 18 { printing is destined for the transcript file only }
define *term_and_log* = 19 { normal *selector* setting }
define *pseudo* = 20 { special *selector* setting for *show_context* }
define *new_string* = 21 { printing is deflected to the string pool }
define *max_selector* = 21 { highest selector setting }

⟨ Global variables 13 ⟩ +≡
log_file: *alpha_file*; { transcript of TeX session }
selector: 0 .. *max_selector*; { where to print a message }
dig: **array** [0 .. 22] **of** 0 .. 15; { digits in a number being output }
tally: *integer*; { the number of characters recently printed }
term_offset: 0 .. *max_print_line*; { the number of characters on the current terminal line }
file_offset: 0 .. *max_print_line*; { the number of characters on the current file line }
trick_buf: **array** [0 .. *error_line*] **of** *ASCII_code*; { circular buffer for pseudoprinting }
trick_count: *integer*; { threshold for pseudoprinting, explained later }
first_count: *integer*; { another variable for pseudoprinting }

55. ⟨ Initialize the output routines 55 ⟩ ≡
selector ← *term_only*; *tally* ← 0; *term_offset* ← 0; *file_offset* ← 0;
See also sections 65, 563, and 568.
This code is used in section 1386.

56. Macro abbreviations for output to the terminal and to the log file are defined here for convenience. Some systems need special conventions for terminal output, and it is possible to adhere to those conventions by changing *wterm*, *wterm_ln*, and *wterm_cr* in this section.

define *wterm*(#) ≡ *write*(*term_out*, #)
define *wterm_ln*(#) ≡ *write_ln*(*term_out*, #)
define *wterm_cr* ≡ *write_ln*(*term_out*)
define *wlog*(#) ≡ *write*(*log_file*, #)
define *wlog_ln*(#) ≡ *write_ln*(*log_file*, #)
define *wlog_cr* ≡ *write_ln*(*log_file*)

57. To end a line of text output, we call *print_ln*.

```
⟨ Basic printing procedures 57 ⟩ ≡
procedure print_ln; { prints an end-of-line }
  begin case selector of
  term_and_log: begin wterm_cr; wlog_cr; term_offset ← 0; file_offset ← 0;
    end;
  log_only: begin wlog_cr; file_offset ← 0;
    end;
  term_only: begin wterm_cr; term_offset ← 0;
    end;
  no_print, pseudo, new_string: do_nothing;
  othercases write_ln(write_file[selector])
  endcases;
  end; { tally is not affected }
```

See also sections 58, 59, 63, 66, 67, 68, 69, 292, 293, 553, 741, 1415, and 1633.

This code is used in section 4.

58. The *print_raw_char* procedure sends one character to the desired destination, using the *xchr* array to map it into an external character compatible with *input_ln*. All printing comes through *print_ln*, *print_char* or *print_visible_char*. When printing a multi-byte character, the boolean parameter *incr_offset* is set *false* except for the very last byte, to avoid calling *print_ln* in the middle of such character.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_raw_char(s : ASCII_code; incr_offset : boolean);  { prints a single character }
  label exit;  { label is not used but nonetheless kept (for other changes?) }
  begin case selector of
  term_and_log: begin wterm(xchr[s]); wlog(xchr[s]);
    if incr_offset then
      begin incr(term_offset); incr(file_offset);
      end;
    if term_offset = max_print_line then
      begin wterm_cr; term_offset ← 0;
      end;
    if file_offset = max_print_line then
      begin wlog_cr; file_offset ← 0;
      end;
    end;
  log_only: begin wlog(xchr[s]);
    if incr_offset then incr(file_offset);
    if file_offset = max_print_line then print_ln;
    end;
  term_only: begin wterm(xchr[s]);
    if incr_offset then incr(term_offset);
    if term_offset = max_print_line then print_ln;
    end;
  no_print: do_nothing;
  pseudo: if tally < trick_count then trick_buf[tally mod error_line] ← s;
  new_string: begin if pool_ptr < pool_size then append_char(s);
    end;  { we drop characters if the string space is full }
  othercases write(write_file[selector], xchr[s])
  endcases;
  incr(tally);
exit: end;
```

59. The *print_char* procedure sends one character to the desired destination. Control sequence names, file names and string constructed with `\string` might contain *ASCII_code* values that can't be printed using *print_raw_char*. These characters will be printed in three- or four-symbol form like '`^^A`' or '`^^e4`', unless the -8bit option is enabled. Output that goes to the terminal and/or log file is treated differently when it comes to determining whether a character is printable.

```
define print_visible_char(#) ≡ print_raw_char(#, true)
define print_lc_hex(#) ≡ l ← #;
    if l < 10 then print_visible_char(l + "0") else print_visible_char(l − 10 + "a")

⟨ Basic printing procedures 57 ⟩ +≡
procedure print_char(s : integer); { prints a single character }
  label exit;
  var l: small_number;
  begin if (selector > pseudo) ∧ (¬doing_special) then
      { "printing" to a new string, encode as UTF-16 rather than UTF-8 }
    begin if s ≥ ″10000 then
      begin print_visible_char(″D800 + (s − ″10000) div ″400);
      print_visible_char(″DC00 + (s − ″10000) mod ″400);
      end
    else print_visible_char(s);
    return;
    end;
  if ⟨ Character s is the current new-line character 270 ⟩ then
    if selector < pseudo then
      begin print_ln; return;
      end;
  if (s < 32) ∧ (eight_bit_p = 0) ∧ (¬doing_special) then { control char: ^^X }
    begin print_visible_char("^"); print_visible_char("^"); print_visible_char(s + 64);
    end
  else if s < 127 then { printable ASCII }
      print_visible_char(s)
    else if (s = 127) then { DEL }
        begin if (eight_bit_p = 0) ∧ (¬doing_special) then
          begin print_visible_char("^"); print_visible_char("^"); print_visible_char("?")
          end
        else print_visible_char(s)
        end
      else if (s < ″A0) ∧ (eight_bit_p = 0) ∧ (¬doing_special) then { C1 controls: ^^xx }
          begin print_visible_char("^"); print_visible_char("^"); print_lc_hex((s mod ″100) div ″10);
          print_lc_hex(s mod ″10);
          end
        else if selector = pseudo then print_visible_char(s)
            { Don't UTF8-encode text in trick_buf, we'll handle that when printing error context. }
          else begin { char ≥ 128: encode as UTF8 }
            if s < ″800 then
              begin print_raw_char(″C0 + s div ″40, false); print_raw_char(″80 + s mod ″40, true);
              end
            else if s < ″10000 then
                begin print_raw_char(″E0 + (s div ″1000), false);
                print_raw_char(″80 + (s mod ″1000) div ″40, false);
                print_raw_char(″80 + (s mod ″40), true);
                end
              else begin print_raw_char(″F0 + (s div ″40000), false);
```

```
                print_raw_char("80 + (s mod "40000) div "1000, false);
                print_raw_char("80 + (s mod "1000) div "40, false);
                print_raw_char("80 + (s mod "40), true);
                end
            end;
exit: end;
```

60. **define** *native_room*(#) ≡
```
        while native_text_size ≤ native_len + # do
          begin native_text_size ← native_text_size + 128;
          native_text ← xrealloc(native_text, native_text_size * sizeof(UTF16_code));
          end
  define append_native(#) ≡
        begin native_text[native_len] ← #; incr(native_len);
        end
```

61. ⟨ Global variables 13 ⟩ +≡
```
doing_special: boolean;
native_text: ↑UTF16_code;  { buffer for collecting native-font strings }
native_text_size: integer;  { size of buffer }
native_len: integer;
save_native_len: integer;
```

62. ⟨ Set initial values of key variables 23 ⟩ +≡
```
  doing_special ← false; native_text_size ← 128;
  native_text ← xmalloc(native_text_size * sizeof(UTF16_code));
```

63. An entire string is output by calling *print*. Note that if we are outputting the single standard ASCII character `c`, we could call *print*(`"c"`), since `"c"` = 99 is the number of a single-character string, as explained above. But *print_char*(`"c"`) is quicker, so TEX goes directly to the *print_char* routine when it knows that this is safe. (The present implementation assumes that it is always safe to print a visible ASCII character.)

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print(s : integer); { prints string s }
  label exit;
  var j: pool_pointer; { current character code position }
    nl: integer; { new-line character to restore }
  begin if s ≥ str_ptr then s ← "???" { this can't happen }
  else if s < biggest_char then
      if s < 0 then s ← "???" { can't happen }
      else begin if selector > pseudo then
          begin print_char(s); return; { internal strings are not expanded }
          end;
        if (⟨ Character s is the current new-line character 270 ⟩) then
          if selector < pseudo then
            begin print_ln; return;
            end;
        nl ← new_line_char; new_line_char ← −1; print_char(s); new_line_char ← nl; return;
        end;
  j ← str_start_macro(s);
  while j < str_start_macro(s + 1) do
    begin if (so(str_pool[j]) ≥ "D800) ∧ (so(str_pool[j]) ≤ "DBFF) ∧ (j + 1 <
          str_start_macro(s + 1)) ∧ (so(str_pool[j + 1]) ≥ "DC00) ∧ (so(str_pool[j + 1]) ≤ "DFFF) then
      begin print_char("10000 + (so(str_pool[j]) − "D800) ∗ "400 + so(str_pool[j + 1]) − "DC00); j ← j + 2;
      end
    else begin print_char(so(str_pool[j])); incr(j);
      end;
    end;
exit: end;
```

64. Old versions of TEX needed a procedure called *slow_print* whose function is now subsumed by *print* and the new functionality of *print_char* and *print_visible_char*. We retain the old name *slow_print* here as a possible aid to future software archæologists.

```
define slow_print ≡ print
```

65. Here is the very first thing that TEX prints: a headline that identifies the version number and format package. The *term_offset* variable is temporarily incorrect, but the discrepancy is not serious since we assume that this part of the program is system dependent.

```
⟨ Initialize the output routines 55 ⟩ +≡
  wterm(banner);
  if format_ident = 0 then wterm_ln(´␣(no␣format␣preloaded)´)
  else begin slow_print(format_ident); print_ln;
    end;
  update_terminal;
```

66. The procedure *print_nl* is like *print*, but it makes sure that the string appears at the beginning of a new line.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_nl(s : str_number); { prints string s at beginning of line }
  begin if ((term_offset > 0) ∧ (odd(selector))) ∨ ((file_offset > 0) ∧ (selector ≥ log_only)) then print_ln;
  print(s);
  end;
```

67. The procedure *print_esc* prints a string that is preceded by the user's escape character (which is usually a backslash).

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_esc(s : str_number); { prints escape character, then s }
  var c: integer; { the escape character code }
  begin ⟨ Set variable c to the current escape character 269 ⟩;
  if c ≥ 0 then
    if c ≤ biggest_usv then print_char(c);
  slow_print(s);
  end;
```

68. An array of digits in the range 0 .. 15 is printed by *print_the_digs*.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_the_digs(k : eight_bits); { prints dig[k − 1] . . . dig[0] }
  begin while k > 0 do
    begin decr(k);
    if dig[k] < 10 then print_char("0" + dig[k])
    else print_char("A" − 10 + dig[k]);
    end;
  end;
```

69. The following procedure, which prints out the decimal representation of a given integer n, has been written carefully so that it works properly if $n = 0$ or if $(-n)$ would cause overflow. It does not apply **mod** or **div** to negative arguments, since such operations are not implemented consistently by all Pascal compilers.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_int(n : integer); { prints an integer in decimal form }
  var k: 0 .. 23; { index to current digit; we assume that |n| < 10^23 }
    m: integer; { used to negate n in possibly dangerous cases }
  begin k ← 0;
  if n < 0 then
    begin print_char("-");
    if n > −100000000 then negate(n)
    else begin m ← −1 − n; n ← m div 10; m ← (m mod 10) + 1; k ← 1;
      if m < 10 then dig[0] ← m
      else begin dig[0] ← 0; incr(n);
        end;
      end;
    end;
  repeat dig[k] ← n mod 10; n ← n div 10; incr(k);
  until n = 0;
  print_the_digs(k);
  end;
```

70. Here is a trivial procedure to print two digits; it is usually called with a parameter in the range $0 \le n \le 99$.

```
procedure print_two(n : integer);  { prints two least significant digits }
  begin n ← abs(n) mod 100; print_char("0" + (n div 10)); print_char("0" + (n mod 10));
  end;
```

71. Hexadecimal printing of nonnegative integers is accomplished by *print_hex*.

```
procedure print_hex(n : integer);  { prints a positive integer in hexadecimal form }
  var k: 0 .. 22;  { index to current digit; we assume that 0 ≤ n < 16^22 }
  begin k ← 0; print_char("""");
  repeat dig[k] ← n mod 16; n ← n div 16; incr(k);
  until n = 0;
  print_the_digs(k);
  end;
```

72. Old versions of TeX needed a procedure called *print_ASCII* whose function is now subsumed by *print*. We retain the old name here as a possible aid to future software archæologists.

```
  define print_ASCII ≡ print
```

73. Roman numerals are produced by the *print_roman_int* routine. Readers who like puzzles might enjoy trying to figure out how this tricky code works; therefore no explanation will be given. Notice that 1990 yields `mcmxc`, not `mxm`.

```
procedure print_roman_int(n : integer);
  label exit;
  var j, k: pool_pointer;  { mysterious indices into str_pool }
    u, v: nonnegative_integer;  { mysterious numbers }
  begin j ← str_start_macro("m2d5c2l5x2v5i"); v ← 1000;
  loop begin while n ≥ v do
      begin print_char(so(str_pool[j])); n ← n − v;
      end;
    if n ≤ 0 then return;  { nonpositive input produces no output }
    k ← j + 2; u ← v div (so(str_pool[k − 1]) − "0");
    if str_pool[k − 1] = si("2") then
      begin k ← k + 2; u ← u div (so(str_pool[k − 1]) − "0");
      end;
    if n + u ≥ v then
      begin print_char(so(str_pool[k])); n ← n + u;
      end
    else begin j ← j + 2; v ← v div (so(str_pool[j − 1]) − "0");
      end;
    end;
exit: end;
```

74. The *print* subroutine will not print a string that is still being created. The following procedure will.

```
procedure print_current_string;  { prints a yet-unmade string }
  var j: pool_pointer;  { points to current character code }
  begin j ← str_start_macro(str_ptr);
  while j < pool_ptr do
    begin print_char(so(str_pool[j])); incr(j);
    end;
  end;
```

75. Here is a procedure that asks the user to type a line of input, assuming that the *selector* setting is either *term_only* or *term_and_log*. The input is placed into locations *first* through *last* − 1 of the *buffer* array, and echoed on the transcript file if appropriate.

This procedure is never called when *interaction* < *scroll_mode*.

```
define prompt_input(#) ≡
        begin wake_up_terminal; print(#); term_input;
        end  { prints a string and gets a line of input }
procedure term_input;  { gets a line from the terminal }
  var k: 0 .. buf_size;  { index into buffer }
  begin update_terminal;  { now the user sees the prompt for sure }
  if ¬input_ln(term_in, true) then fatal_error("End␣of␣file␣on␣the␣terminal!");
  term_offset ← 0;  { the user's line ended with ⟨return⟩ }
  decr(selector);  { prepare to echo the input }
  if last ≠ first then
    for k ← first to last − 1 do print(buffer[k]);
  print_ln; incr(selector);  { restore previous status }
  end;
```

76. Reporting errors. When something anomalous is detected, TeX typically does something like this:

```
print_err("Something␣anomalous␣has␣been␣detected");
help3("This␣is␣the␣first␣line␣of␣my␣offer␣to␣help.")
("This␣is␣the␣second␣line.␣I´m␣trying␣to")
("explain␣the␣best␣way␣for␣you␣to␣proceed.");
error;
```

A two-line help message would be given using *help2*, etc.; these informal helps should use simple vocabulary that complements the words used in the official error message that was printed. (Outside the U.S.A., the help messages should preferably be translated into the local vernacular. Each line of help is at most 60 characters long, in the present implementation, so that *max_print_line* will not be exceeded.)

The *print_err* procedure supplies a '`!`' before the official message, and makes sure that the terminal is awake if a stop is going to occur. The *error* procedure supplies a '`.`' after the official message, then it shows the location of the error; and if *interaction* = *error_stop_mode*, it also enters into a dialog with the user, during which time the help message may be printed.

77. The global variable *interaction* has four settings, representing increasing amounts of user interaction:

```
define batch_mode = 0   { omits all stops and omits terminal output }
define nonstop_mode = 1   { omits all stops }
define scroll_mode = 2   { omits error stops }
define error_stop_mode = 3   { stops at every opportunity to interact }
define print_err(#) ≡
        begin if interaction = error_stop_mode then wake_up_terminal;
        print_nl("!␣"); print(#);
        end
```

⟨ Global variables 13 ⟩ +≡
interaction: *batch_mode* .. *error_stop_mode*; { current level of interaction }

78. ⟨ Set initial values of key variables 23 ⟩ +≡
interaction ← *error_stop_mode*;

79. TeX is careful not to call *error* when the print *selector* setting might be unusual. The only possible values of *selector* at the time of error messages are

no_print (when *interaction* = *batch_mode* and *log_file* not yet open);
term_only (when *interaction* > *batch_mode* and *log_file* not yet open);
log_only (when *interaction* = *batch_mode* and *log_file* is open);
term_and_log (when *interaction* > *batch_mode* and *log_file* is open).

⟨ Initialize the print *selector* based on *interaction* 79 ⟩ ≡
if *interaction* = *batch_mode* **then** *selector* ← *no_print* **else** *selector* ← *term_only*

This code is used in sections 1319 and 1391.

80. A global variable *deletions_allowed* is set *false* if the *get_next* routine is active when *error* is called; this ensures that *get_next* and related routines like *get_token* will never be called recursively. A similar interlock is provided by *set_box_allowed*.

The global variable *history* records the worst level of error that has been detected. It has four possible values: *spotless*, *warning_issued*, *error_message_issued*, and *fatal_error_stop*.

Another global variable, *error_count*, is increased by one when an *error* occurs without an interactive dialog, and it is reset to zero at the end of every paragraph. If *error_count* reaches 100, TeX decides that there is no point in continuing further.

define *spotless* = 0 { *history* value when nothing has been amiss yet }
define *warning_issued* = 1 { *history* value when *begin_diagnostic* has been called }
define *error_message_issued* = 2 { *history* value when *error* has been called }
define *fatal_error_stop* = 3 { *history* value when termination was premature }

⟨ Global variables 13 ⟩ +≡
deletions_allowed: *boolean*; { is it safe for *error* to call *get_token*? }
set_box_allowed: *boolean*; { is it safe to do a `\setbox` assignment? }
history: *spotless* .. *fatal_error_stop*; { has the source input been clean so far? }
error_count: −1 .. 100; { the number of scrolled errors since the last paragraph ended }

81. The value of *history* is initially *fatal_error_stop*, but it will be changed to *spotless* if TeX survives the initialization process.

⟨ Set initial values of key variables 23 ⟩ +≡
deletions_allowed ← *true*; *set_box_allowed* ← *true*; *error_count* ← 0; { *history* is initialized elsewhere }

82. Since errors can be detected almost anywhere in TeX, we want to declare the error procedures near the beginning of the program. But the error procedures in turn use some other procedures, which need to be declared *forward* before we get to *error* itself.

It is possible for *error* to be called recursively if some error arises when *get_token* is being used to delete a token, and/or if some fatal error occurs while TeX is trying to fix a non-fatal one. But such recursion is never more than two levels deep.

⟨ Error handling procedures 82 ⟩ ≡
procedure *normalize_selector*; *forward*;
procedure *get_token*; *forward*;
procedure *term_input*; *forward*;
procedure *show_context*; *forward*;
procedure *begin_file_reading*; *forward*;
procedure *open_log_file*; *forward*;
procedure *close_files_and_terminate*; *forward*;
procedure *clear_for_error_prompt*; *forward*;
procedure *give_err_help*; *forward*;
debug procedure *debug_help*; *forward*; **gubed**

See also sections 85, 86, 97, 98, 99, and 1455.

This code is used in section 4.

83. Individual lines of help are recorded in the array *help_line*, which contains entries in positions 0 .. (*help_ptr* − 1). They should be printed in reverse order, i.e., with *help_line*[0] appearing last.

```
define hlp1(#) ≡ help_line[0] ← #; end
define hlp2(#) ≡ help_line[1] ← #; hlp1
define hlp3(#) ≡ help_line[2] ← #; hlp2
define hlp4(#) ≡ help_line[3] ← #; hlp3
define hlp5(#) ≡ help_line[4] ← #; hlp4
define hlp6(#) ≡ help_line[5] ← #; hlp5
define help0 ≡ help_ptr ← 0  { sometimes there might be no help }
define help1 ≡ begin help_ptr ← 1; hlp1  { use this with one help line }
define help2 ≡ begin help_ptr ← 2; hlp2  { use this with two help lines }
define help3 ≡ begin help_ptr ← 3; hlp3  { use this with three help lines }
define help4 ≡ begin help_ptr ← 4; hlp4  { use this with four help lines }
define help5 ≡ begin help_ptr ← 5; hlp5  { use this with five help lines }
define help6 ≡ begin help_ptr ← 6; hlp6  { use this with six help lines }
```

```
⟨ Global variables 13 ⟩ +≡
help_line: array [0 .. 5] of str_number;  { helps for the next error }
help_ptr: 0 .. 6;  { the number of help lines present }
use_err_help: boolean;  { should the err_help list be shown? }
```

84.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  help_ptr ← 0; use_err_help ← false;
```

85. The *jump_out* procedure just cuts across all active procedure levels and goes to *end_of_TEX*. This is the only nontrivial **goto** statement in the whole program. It is used when there is no recovery from a particular error.

Some Pascal compilers do not implement non-local **goto** statements. In such cases the body of *jump_out* should simply be '*close_files_and_terminate*;' followed by a call on some system procedure that quietly terminates the program.

```
⟨ Error handling procedures 82 ⟩ +≡
procedure jump_out;
  begin goto end_of_TEX;
  end;
```

86. Here now is the general *error* routine.

```
⟨ Error handling procedures 82 ⟩ +≡
procedure error;  { completes the job of error reporting }
  label continue, exit;
  var c: ASCII_code;  { what the user types }
    s1, s2, s3, s4: integer;  { used to save global variables when deleting tokens }
  begin if history < error_message_issued then history ← error_message_issued;
  print_char("."); show_context;
  if interaction = error_stop_mode then ⟨ Get user's advice and return 87 ⟩;
  incr(error_count);
  if error_count = 100 then
    begin print_nl("(That␣makes␣100␣errors;␣please␣try␣again.)"); history ← fatal_error_stop;
    jump_out;
    end;
  ⟨ Put help message on the transcript file 94 ⟩;
exit: end;
```

87. ⟨ Get user's advice and **return** 87 ⟩ ≡

```
loop begin continue: if interaction ≠ error_stop_mode then return;
  clear_for_error_prompt; prompt_input("? ");
  if last = first then return;
  c ← buffer[first];
  if c ≥ "a" then c ← c + "A" − "a";  { convert to uppercase }
  ⟨ Interpret code c and return if done 88 ⟩;
  end
```

This code is used in section 86.

88. It is desirable to provide an 'E' option here that gives the user an easy way to return from TeX to the system editor, with the offending line ready to be edited. But such an extension requires some system wizardry, so the present implementation simply types out the name of the file that should be edited and the relevant line number.

There is a secret 'D' option available when the debugging routines haven't been commented out.

⟨ Interpret code c and **return** if done 88 ⟩ ≡

```
case c of
"0","1","2","3","4","5","6","7","8","9": if deletions_allowed then
    ⟨ Delete c − "0" tokens and goto continue 92 ⟩;
debug "D": begin debug_help; goto continue; end; gubed
"E": if base_ptr > 0 then
    if input_stack[base_ptr].name_field ≥ 256 then
      begin print_nl("You want to edit file ");  slow_print(input_stack[base_ptr].name_field);
      print(" at line ");  print_int(line);  interaction ← scroll_mode;  jump_out;
      end;
"H": ⟨ Print the help information and goto continue 93 ⟩;
"I": ⟨ Introduce new material from the terminal and return 91 ⟩;
"Q","R","S": ⟨ Change the interaction level and return 90 ⟩;
"X": begin interaction ← scroll_mode;  jump_out;
  end;
othercases do_nothing
endcases;
⟨ Print the menu of available options 89 ⟩
```

This code is used in section 87.

89. ⟨ Print the menu of available options 89 ⟩ ≡

```
begin print("Type <return> to proceed, S to scroll future error messages,");
print_nl("R to run without stopping, Q to run quietly,");
print_nl("I to insert something, ");
if base_ptr > 0 then
  if input_stack[base_ptr].name_field ≥ 256 then print("E to edit your file,");
if deletions_allowed then
  print_nl("1 or ... or 9 to ignore the next 1 to 9 tokens of input,");
print_nl("H for help, X to quit.");
end
```

This code is used in section 88.

90. Here the author of TeX apologizes for making use of the numerical relation between `"Q"`, `"R"`, `"S"`, and the desired interaction settings *batch_mode*, *nonstop_mode*, *scroll_mode*.

```
⟨ Change the interaction level and return 90 ⟩ ≡
  begin error_count ← 0; interaction ← batch_mode + c − "Q"; print("OK, entering ");
  case c of
  "Q": begin print_esc("batchmode"); decr(selector);
    end;
  "R": print_esc("nonstopmode");
  "S": print_esc("scrollmode");
  end; { there are no other cases }
  print("..."); print_ln; update_terminal; return;
  end
```

This code is used in section 88.

91. When the following code is executed, *buffer*[(*first* + 1) .. (*last* − 1)] may contain the material inserted by the user; otherwise another prompt will be given. In order to understand this part of the program fully, you need to be familiar with TeX's input stacks.

```
⟨ Introduce new material from the terminal and return 91 ⟩ ≡
  begin begin_file_reading; { enter a new syntactic level for terminal input }
    { now state = mid_line, so an initial blank space will count as a blank }
  if last > first + 1 then
    begin loc ← first + 1; buffer[first] ← " ";
    end
  else begin prompt_input("insert>"); loc ← first;
    end;
  first ← last; cur_input.limit_field ← last − 1; { no end_line_char ends this line }
  return;
  end
```

This code is used in section 88.

92. We allow deletion of up to 99 tokens at a time.

```
⟨ Delete c − "0" tokens and goto continue 92 ⟩ ≡
  begin s1 ← cur_tok; s2 ← cur_cmd; s3 ← cur_chr; s4 ← align_state; align_state ← 1000000;
  OK_to_interrupt ← false;
  if (last > first + 1) ∧ (buffer[first + 1] ≥ "0") ∧ (buffer[first + 1] ≤ "9") then
    c ← c * 10 + buffer[first + 1] − "0" * 11
  else c ← c − "0";
  while c > 0 do
    begin get_token; { one-level recursive call of error is possible }
    decr(c);
    end;
  cur_tok ← s1; cur_cmd ← s2; cur_chr ← s3; align_state ← s4; OK_to_interrupt ← true;
  help2("I have just deleted some text, as you asked.")
  ("You can now delete more, or insert, or whatever."); show_context; goto continue;
  end
```

This code is used in section 88.

93. ⟨ Print the help information and **goto** *continue* 93 ⟩ ≡

```
begin if use_err_help then
  begin give_err_help; use_err_help ← false;
  end
else begin if help_ptr = 0 then help2("Sorry, I don't know how to help in this situation.")
    ("Maybe you should try asking a human?");
  repeat decr(help_ptr); print(help_line[help_ptr]); print_ln;
  until help_ptr = 0;
  end;
help4("Sorry, I already gave what help I could...")
("Maybe you should try asking a human?")
("An error might have occurred before I noticed any problems.")
("``If all else fails, read the instructions.''");
goto continue;
end
```

This code is used in section 88.

94. ⟨ Put help message on the transcript file 94 ⟩ ≡

```
if interaction > batch_mode then decr(selector); { avoid terminal output }
if use_err_help then
  begin print_ln; give_err_help;
  end
else while help_ptr > 0 do
    begin decr(help_ptr); print_nl(help_line[help_ptr]);
    end;
print_ln;
if interaction > batch_mode then incr(selector); { re-enable terminal output }
print_ln
```

This code is used in section 86.

95. A dozen or so error messages end with a parenthesized integer, so we save a teeny bit of program space by declaring the following procedure:

```
procedure int_error(n : integer);
  begin print("("); print_int(n); print_char(")"); error;
  end;
```

96. In anomalous cases, the print selector might be in an unknown state; the following subroutine is called to fix things just enough to keep running a bit longer.

```
procedure normalize_selector;
  begin if log_opened then selector ← term_and_log
  else selector ← term_only;
  if job_name = 0 then open_log_file;
  if interaction = batch_mode then decr(selector);
  end;
```

97. The following procedure prints TeX's last words before dying.

```
define succumb ≡
        begin if interaction = error_stop_mode then interaction ← scroll_mode;
              { no more interaction }
        if log_opened then error;
        debug if interaction > batch_mode then debug_help;
        gubed
        history ← fatal_error_stop; jump_out;  { irrecoverable error }
        end
```

```
⟨ Error handling procedures 82 ⟩ +≡
procedure fatal_error(s : str_number);  { prints s, and that's it }
  begin normalize_selector;
  print_err("Emergency␣stop"); help1(s); succumb;
  end;
```

98. Here is the most dreaded error message.

```
⟨ Error handling procedures 82 ⟩ +≡
procedure overflow(s : str_number; n : integer);  { stop due to finiteness }
  begin normalize_selector; print_err("TeX␣capacity␣exceeded,␣sorry␣[");
  print(s); print_char("="); print_int(n); print_char("]");
  help2("If␣you␣really␣absolutely␣need␣more␣capacity,")
  ("you␣can␣ask␣a␣wizard␣to␣enlarge␣me."); succumb;
  end;
```

99. The program might sometime run completely amok, at which point there is no choice but to stop. If no previous error has been detected, that's bad news; a message is printed that is really intended for the TeX maintenance person instead of the user (unless the user has been particularly diabolical). The index entries for 'this can't happen' may help to pinpoint the problem.

```
⟨ Error handling procedures 82 ⟩ +≡
procedure confusion(s : str_number);  { consistency check violated; s tells where }
  begin normalize_selector;
  if history < error_message_issued then
    begin print_err("This␣can´t␣happen␣("); print(s); print_char(")");
    help1("I´m␣broken.␣Please␣show␣this␣to␣someone␣who␣can␣fix␣can␣fix");
    end
  else begin print_err("I␣can´t␣go␣on␣meeting␣you␣like␣this");
    help2("One␣of␣your␣faux␣pas␣seems␣to␣have␣wounded␣me␣deeply...")
    ("in␣fact,␣I´m␣barely␣conscious.␣Please␣fix␣it␣and␣try␣again.");
    end;
  succumb;
  end;
```

100. Users occasionally want to interrupt TeX while it's running. If the Pascal runtime system allows this, one can implement a routine that sets the global variable *interrupt* to some nonzero value when such an interrupt is signalled. Otherwise there is probably at least a way to make *interrupt* nonzero using the Pascal debugger.

```
define check_interrupt ≡
        begin if interrupt ≠ 0 then pause_for_instructions;
        end
```

```
⟨ Global variables 13 ⟩ +≡
interrupt: integer;  { should TeX pause for instructions? }
OK_to_interrupt: boolean;  { should interrupts be observed? }
```

101. ⟨ Set initial values of key variables 23 ⟩ +≡
interrupt ← 0; *OK_to_interrupt* ← *true*;

102. When an interrupt has been detected, the program goes into its highest interaction level and lets the user have nearly the full flexibility of the *error* routine. TeX checks for interrupts only at times when it is safe to do this.

```
procedure pause_for_instructions;
  begin if OK_to_interrupt then
    begin interaction ← error_stop_mode;
    if (selector = log_only) ∨ (selector = no_print) then incr(selector);
    print_err("Interruption"); help3("You␣rang?")
    ("Try␣to␣insert␣an␣instruction␣for␣me␣(e.g.,␣`I\showlists´),")
    ("unless␣you␣just␣want␣to␣quit␣by␣typing␣`X´."); deletions_allowed ← false; error;
    deletions_allowed ← true; interrupt ← 0;
    end;
  end;
```

103. Arithmetic with scaled dimensions. The principal computations performed by TEX are done entirely in terms of integers less than 2^{31} in magnitude; and divisions are done only when both dividend and divisor are nonnegative. Thus, the arithmetic specified in this program can be carried out in exactly the same way on a wide variety of computers, including some small ones. Why? Because the arithmetic calculations need to be spelled out precisely in order to guarantee that TEX will produce identical output on different machines. If some quantities were rounded differently in different implementations, we would find that line breaks and even page breaks might occur in different places. Hence the arithmetic of TEX has been designed with care, and systems that claim to be implementations of TEX82 should follow precisely the calculations as they appear in the present program.

(Actually there are three places where TEX uses **div** with a possibly negative numerator. These are harmless; see **div** in the index. Also if the user sets the `\time` or the `\year` to a negative value, some diagnostic information will involve negative-numerator division. The same remarks apply for **mod** as well as for **div**.)

104. Here is a routine that calculates half of an integer, using an unambiguous convention with respect to signed odd numbers.

```
function half(x : integer): integer;
  begin if odd(x) then half ← (x + 1) div 2
  else half ← x div 2;
  end;
```

105. Fixed-point arithmetic is done on *scaled integers* that are multiples of 2^{-16}. In other words, a binary point is assumed to be sixteen bit positions from the right end of a binary computer word.

```
  define unity ≡ '200000   { 2^16, represents 1.00000 }
  define two ≡ '400000   { 2^17, represents 2.00000 }
⟨ Types in the outer block 18 ⟩ +≡
  scaled = integer;   { this type is used for scaled integers }
  nonnegative_integer = 0 .. '17777777777;   { 0 ≤ x < 2^31 }
  small_number = 0 .. hyphenatable_length_limit;   { this type is self-explanatory }
```

106. The following function is used to create a scaled integer from a given decimal fraction $(.d_0 d_1 \ldots d_{k-1})$, where $0 \le k \le 17$. The digit d_i is given in $dig[i]$, and the calculation produces a correctly rounded result.

```
function round_decimals(k : small_number): scaled;   { converts a decimal fraction }
  var a: integer;   { the accumulator }
  begin a ← 0;
  while k > 0 do
    begin decr(k); a ← (a + dig[k] * two) div 10;
    end;
  round_decimals ← (a + 1) div 2;
  end;
```

107. Conversely, here is a procedure analogous to *print_int*. If the output of this procedure is subsequently read by TeX and converted by the *round_decimals* routine above, it turns out that the original value will be reproduced exactly; the "simplest" such decimal number is output, but there is always at least one digit following the decimal point.

The invariant relation in the **repeat** loop is that a sequence of decimal digits yet to be printed will yield the original number if and only if they form a fraction f in the range $s - \delta \le 10 \cdot 2^{16} f < s$. We can stop if and only if $f = 0$ satisfies this condition; the loop will terminate before s can possibly become zero.

```
procedure print_scaled(s : scaled); { prints scaled real, rounded to five digits }
  var delta: scaled; { amount of allowable inaccuracy }
  begin if s < 0 then
    begin print_char("-"); negate(s); { print the sign, if negative }
    end;
  print_int(s div unity); { print the integer part }
  print_char("."); s ← 10 * (s mod unity) + 5; delta ← 10;
  repeat if delta > unity then s ← s + '100000 - 50000; { round the last digit }
    print_char("0" + (s div unity)); s ← 10 * (s mod unity); delta ← delta * 10;
  until s ≤ delta;
  end;
```

108. Physical sizes that a TeX user specifies for portions of documents are represented internally as scaled points. Thus, if we define an 'sp' (scaled point) as a unit equal to 2^{-16} printer's points, every dimension inside of TeX is an integer number of sp. There are exactly 4,736,286.72 sp per inch. Users are not allowed to specify dimensions larger than $2^{30} - 1$ sp, which is a distance of about 18.892 feet (5.7583 meters); two such quantities can be added without overflow on a 32-bit computer.

The present implementation of TeX does not check for overflow when dimensions are added or subtracted. This could be done by inserting a few dozen tests of the form '**if** $x \ge$ *'10000000000* **then** *report_overflow*', but the chance of overflow is so remote that such tests do not seem worthwhile.

TeX needs to do only a few arithmetic operations on scaled quantities, other than addition and subtraction, and the following subroutines do most of the work. A single computation might use several subroutine calls, and it is desirable to avoid producing multiple error messages in case of arithmetic overflow; so the routines set the global variable *arith_error* to *true* instead of reporting errors directly to the user. Another global variable, *remainder*, holds the remainder after a division.

```
⟨ Global variables 13 ⟩ +≡
arith_error: boolean; { has arithmetic overflow occurred recently? }
remainder: scaled; { amount subtracted to get an exact division }
```

109. The first arithmetical subroutine we need computes $nx + y$, where x and y are *scaled* and n is an integer. We will also use it to multiply integers.

```
define nx_plus_y(#) ≡ mult_and_add(#, '7777777777)
define mult_integers(#) ≡ mult_and_add(#, 0, '17777777777)
function mult_and_add(n : integer; x, y, max_answer : scaled): scaled;
  begin if n < 0 then
    begin negate(x); negate(n);
    end;
  if n = 0 then mult_and_add ← y
  else if ((x ≤ (max_answer - y) div n) ∧ (-x ≤ (max_answer + y) div n)) then mult_and_add ← n * x + y
    else begin arith_error ← true; mult_and_add ← 0;
      end;
  end;
```

110. We also need to divide scaled dimensions by integers.

```
function x_over_n(x : scaled; n : integer): scaled;
  var negative: boolean;  { should remainder be negated? }
  begin negative ← false;
  if n = 0 then
    begin arith_error ← true; x_over_n ← 0; remainder ← x;
    end
  else begin if n < 0 then
      begin negate(x); negate(n); negative ← true;
      end;
    if x ≥ 0 then
      begin x_over_n ← x div n; remainder ← x mod n;
      end
    else begin x_over_n ← −((−x) div n); remainder ← −((−x) mod n);
      end;
    end;
  if negative then negate(remainder);
  end;
```

111. Then comes the multiplication of a scaled number by a fraction n/d, where n and d are nonnegative integers $\le 2^{16}$ and d is positive. It would be too dangerous to multiply by n and then divide by d, in separate operations, since overflow might well occur; and it would be too inaccurate to divide by d and then multiply by n. Hence this subroutine simulates 1.5-precision arithmetic.

```
function xn_over_d(x : scaled; n, d : integer): scaled;
  var positive: boolean;  { was x ≥ 0? }
    t, u, v: nonnegative_integer;  { intermediate quantities }
  begin if x ≥ 0 then positive ← true
  else begin negate(x); positive ← false;
    end;
  t ← (x mod ´100000) * n; u ← (x div ´100000) * n + (t div ´100000);
  v ← (u mod d) * ´100000 + (t mod ´100000);
  if u div d ≥ ´100000 then arith_error ← true
  else u ← ´100000 * (u div d) + (v div d);
  if positive then
    begin xn_over_d ← u; remainder ← v mod d;
    end
  else begin xn_over_d ← −u; remainder ← −(v mod d);
    end;
  end;
```

112. The next subroutine is used to compute the "badness" of glue, when a total t is supposed to be made from amounts that sum to s. According to *The TEXbook*, the badness of this situation is $100(t/s)^3$; however, badness is simply a heuristic, so we need not squeeze out the last drop of accuracy when computing it. All we really want is an approximation that has similar properties.

The actual method used to compute the badness is easier to read from the program than to describe in words. It produces an integer value that is a reasonably close approximation to $100(t/s)^3$, and all implementations of TEX should use precisely this method. Any badness of 2^{13} or more is treated as infinitely bad, and represented by 10000.

It is not difficult to prove that

$$badness(t+1,s) \geq badness(t,s) \geq badness(t,s+1).$$

The badness function defined here is capable of computing at most 1095 distinct values, but that is plenty.

```
define inf_bad = 10000 { infinitely bad value }
function badness(t,s : scaled): halfword; { compute badness, given t ≥ 0 }
  var r: integer; { approximation to αt/s, where α^3 ≈ 100 · 2^18 }
  begin if t = 0 then badness ← 0
  else if s ≤ 0 then badness ← inf_bad
    else begin if t ≤ 7230584 then r ← (t * 297) div s { 297^3 = 99.94 × 2^18 }
      else if s ≥ 1663497 then r ← t div (s div 297)
        else r ← t;
      if r > 1290 then badness ← inf_bad { 1290^3 < 2^31 < 1291^3 }
      else badness ← (r * r * r + ´400000) div ´1000000;
      end; { that was r^3/2^18, rounded to the nearest integer }
  end;
```

113. When TEX "packages" a list into a box, it needs to calculate the proportionality ratio by which the glue inside the box should stretch or shrink. This calculation does not affect TEX's decision making, so the precise details of rounding, etc., in the glue calculation are not of critical importance for the consistency of results on different computers.

We shall use the type *glue_ratio* for such proportionality ratios. A glue ratio should take the same amount of memory as an *integer* (usually 32 bits) if it is to blend smoothly with TEX's other data structures. Thus *glue_ratio* should be equivalent to *short_real* in some implementations of Pascal. Alternatively, it is possible to deal with glue ratios using nothing but fixed-point arithmetic; see *TUGboat* **3**,1 (March 1982), 10–27. (But the routines cited there must be modified to allow negative glue ratios.)

```
define set_glue_ratio_zero(#) ≡ # ← 0.0 { store the representation of zero ratio }
define set_glue_ratio_one(#) ≡ # ← 1.0 { store the representation of unit ratio }
define float(#) ≡ # { convert from glue_ratio to type real }
define unfloat(#) ≡ # { convert from real to type glue_ratio }
define float_constant(#) ≡ #.0 { convert integer constant to real }
⟨ Types in the outer block 18 ⟩ +≡
  glue_ratio = real; { one-word representation of a glue expansion factor }
```

114. Random numbers.

This section is (almost) straight from MetaPost. I had to change the types (use *integer* instead of *fraction*), but that should not have any influence on the actual calculations (the original comments refer to quantities like *fraction_four* (2^{30}), and that is the same as the numeric representation of *maxdimen*).

I've copied the low-level variables and routines that are needed, but only those (e.g. *m_log*), not the accompanying ones like *m_exp*. Most of the following low-level numeric routines are only needed within the calculation of *norm_rand*. I've been forced to rename *make_fraction* to *make_frac* because TeX already has a routine by that name with a wholly different function (it creates a *fraction_noad* for math typesetting) – Taco

And now let's complete our collection of numeric utility routines by considering random number generation. METAPOST generates pseudo-random numbers with the additive scheme recommended in Section 3.6 of *The Art of Computer Programming*; however, the results are random fractions between 0 and *fraction_one* − 1, inclusive.

There's an auxiliary array *randoms* that contains 55 pseudo-random fractions. Using the recurrence $x_n = (x_{n-55} - x_{n-31}) \bmod 2^{28}$, we generate batches of 55 new x_n's at a time by calling *new_randoms*. The global variable *j_random* tells which element has most recently been consumed.

⟨ Global variables 13 ⟩ +≡
randoms: **array** [0 .. 54] **of** *integer*; { the last 55 random values generated }
j_random: 0 .. 54; { the number of unused *randoms* }
random_seed: *scaled*; { the default random seed }

115. A small bit of metafont is needed.

define *fraction_half* ≡ *´1000000000* { 2^{27}, represents 0.50000000 }
define *fraction_one* ≡ *´2000000000* { 2^{28}, represents 1.00000000 }
define *fraction_four* ≡ *´10000000000* { 2^{30}, represents 4.00000000 }
define *el_gordo* ≡ *´17777777777* { $2^{31} - 1$, the largest value that METAPOST likes }
define *halfp*(#) ≡ (#) **div** 2
define *double*(#) ≡ # ← # + # { multiply a variable by two }

116. The *make_frac* routine produces the *fraction* equivalent of p/q, given integers p and q; it computes the integer $f = \lfloor 2^{28} p/q + \frac{1}{2} \rfloor$, when p and q are positive. If p and q are both of the same scaled type t, the "type relation" $make_frac(t,t) = fraction$ is valid; and it's also possible to use the subroutine "backwards," using the relation $make_frac(t, fraction) = t$ between scaled types.

If the result would have magnitude 2^{31} or more, *make_frac* sets *arith_error* ← *true*. Most of METAPOST's internal computations have been designed to avoid this sort of error.

If this subroutine were programmed in assembly language on a typical machine, we could simply compute $(2^{28} * p)$ **div** q, since a double-precision product can often be input to a fixed-point division instruction. But when we are restricted to Pascal arithmetic it is necessary either to resort to multiple-precision maneuvering or to use a simple but slow iteration. The multiple-precision technique would be about three times faster than the code adopted here, but it would be comparatively long and tricky, involving about sixteen additional multiplications and divisions.

This operation is part of METAPOST's "inner loop"; indeed, it will consume nearly 10% of the running time (exclusive of input and output) if the code below is left unchanged. A machine-dependent recoding will therefore make METAPOST run faster. The present implementation is highly portable, but slow; it avoids multiplication and division except in the initial stage. System wizards should be careful to replace it with a routine that is guaranteed to produce identical results in all cases.

As noted below, a few more routines should also be replaced by machine-dependent code, for efficiency. But when a procedure is not part of the "inner loop," such changes aren't advisable; simplicity and robustness are preferable to trickery, unless the cost is too high.

```
function make_frac(p,q : integer): integer;
  var f: integer;   { the fraction bits, with a leading 1 bit }
    n: integer;   { the integer part of |p/q| }
    negative: boolean;   { should the result be negated? }
    be_careful: integer;   { disables certain compiler optimizations }
  begin if p ≥ 0 then negative ← false
  else begin negate(p); negative ← true;
    end;
  if q ≤ 0 then
    begin debug if q = 0 then confusion("/"); gubed
    negate(q); negative ← ¬negative;
    end;
  n ← p div q; p ← p mod q;
  if n ≥ 8 then
    begin arith_error ← true;
    if negative then make_frac ← −el_gordo else make_frac ← el_gordo;
    end
  else begin n ← (n − 1) * fraction_one; ⟨ Compute f = ⌊2^28(1 + p/q) + 1/2⌋ 117 ⟩;
    if negative then make_frac ← −(f + n) else make_frac ← f + n;
    end;
  end;
```

117. The **repeat** loop here preserves the following invariant relations between f, p, and q: (i) $0 \le p < q$; (ii) $fq + p = 2^k(q + p_0)$, where k is an integer and p_0 is the original value of p.

Notice that the computation specifies $(p-q)+p$ instead of $(p+p)-q$, because the latter could overflow. Let us hope that optimizing compilers do not miss this point; a special variable *be_careful* is used to emphasize the necessary order of computation. Optimizing compilers should keep *be_careful* in a register, not store it in memory.

```
⟨ Compute f = ⌊2^28(1 + p/q) + 1/2⌋ 117 ⟩ ≡
  f ← 1;
  repeat be_careful ← p − q; p ← be_careful + p;
    if p ≥ 0 then f ← f + f + 1
    else begin double(f); p ← p + q;
      end;
  until f ≥ fraction_one;
  be_careful ← p − q;
  if be_careful + p ≥ 0 then incr(f)
```

This code is used in section 116.

118.

```
function take_frac(q : integer; f : integer): integer;
  var p: integer; { the fraction so far }
    negative: boolean; { should the result be negated? }
    n: integer; { additional multiple of q }
    be_careful: integer; { disables certain compiler optimizations }
  begin ⟨ Reduce to the case that f ≥ 0 and q > 0 119 ⟩;
  if f < fraction_one then n ← 0
  else begin n ← f div fraction_one; f ← f mod fraction_one;
    if q ≤ el_gordo div n then n ← n * q
    else begin arith_error ← true; n ← el_gordo;
      end;
    end;
  f ← f + fraction_one; ⟨ Compute p = ⌊qf/2^28 + 1/2⌋ − q 120 ⟩;
  be_careful ← n − el_gordo;
  if be_careful + p > 0 then
    begin arith_error ← true; n ← el_gordo − p;
    end;
  if negative then take_frac ← −(n + p)
  else take_frac ← n + p;
  end;
```

119. ⟨ Reduce to the case that $f \ge 0$ and $q > 0$ 119 ⟩ ≡

```
  if f ≥ 0 then negative ← false
  else begin negate(f); negative ← true;
    end;
  if q < 0 then
    begin negate(q); negative ← ¬negative;
    end;
```

This code is used in section 118.

120. The invariant relations in this case are (i) $\lfloor (qf+p)/2^k \rfloor = \lfloor qf_0/2^{28} + \frac{1}{2} \rfloor$, where k is an integer and f_0 is the original value of f; (ii) $2^k \le f < 2^{k+1}$.

```
⟨ Compute p = ⌊qf/2^28 + 1/2⌋ − q 120 ⟩ ≡
  p ← fraction_half; { that's 2^27; the invariants hold now with k = 28 }
  if q < fraction_four then
    repeat if odd(f) then p ← halfp(p + q) else p ← halfp(p);
      f ← halfp(f);
    until f = 1
  else repeat if odd(f) then p ← p + halfp(q − p) else p ← halfp(p);
      f ← halfp(f);
    until f = 1
```

This code is used in section 118.

121. The subroutines for logarithm and exponential involve two tables. The first is simple: $two_to_the[k]$ equals 2^k. The second involves a bit more calculation, which the author claims to have done correctly: $spec_log[k]$ is 2^{27} times $\ln\bigl(1/(1-2^{-k})\bigr) = 2^{-k} + \frac{1}{2}2^{-2k} + \frac{1}{3}2^{-3k} + \cdots$, rounded to the nearest integer.

```
⟨ Global variables 13 ⟩ +≡
two_to_the: array [0 .. 30] of integer; { powers of two }
spec_log: array [1 .. 28] of integer; { special logarithms }
```

122.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  two_to_the[0] ← 1;
  for k ← 1 to 30 do two_to_the[k] ← 2 * two_to_the[k − 1];
  spec_log[1] ← 93032640; spec_log[2] ← 38612034; spec_log[3] ← 17922280; spec_log[4] ← 8662214;
  spec_log[5] ← 4261238; spec_log[6] ← 2113709; spec_log[7] ← 1052693; spec_log[8] ← 525315;
  spec_log[9] ← 262400; spec_log[10] ← 131136; spec_log[11] ← 65552; spec_log[12] ← 32772;
  spec_log[13] ← 16385;
  for k ← 14 to 27 do spec_log[k] ← two_to_the[27 − k];
  spec_log[28] ← 1;
```

123.

```
function m_log(x : integer): integer;
  var y, z: integer; { auxiliary registers }
    k: integer; { iteration counter }
  begin if x ≤ 0 then ⟨ Handle non-positive logarithm 125 ⟩
  else begin y ← 1302456956 + 4 − 100; { 14 × 2^27 ln 2 ≈ 1302456956.421063 }
    z ← 27595 + 6553600; { and 2^16 × .421063 ≈ 27595 }
    while x < fraction_four do
      begin double(x); y ← y − 93032639; z ← z − 48782;
      end; { 2^27 ln 2 ≈ 93032639.74436163 and 2^16 × .74436163 ≈ 48782 }
    y ← y + (z div unity); k ← 2;
    while x > fraction_four + 4 do
      ⟨ Increase k until x can be multiplied by a factor of 2^−k, and adjust y accordingly 124 ⟩;
    m_log ← y div 8;
    end;
  end;
```

124. ⟨ Increase k until x can be multiplied by a factor of 2^{-k}, and adjust y accordingly 124 ⟩ ≡

```
begin z ← ((x − 1) div two_to_the[k]) + 1;  { z = ⌈x/2^k⌉ }
while x < fraction_four + z do
  begin z ← halfp(z + 1); k ← k + 1;
  end;
y ← y + spec_log[k]; x ← x − z;
end
```

This code is used in section 123.

125. ⟨ Handle non-positive logarithm 125 ⟩ ≡

```
begin print_err("Logarithm␣of␣"); print_scaled(x); print("␣has␣been␣replaced␣by␣0");
help2("Since␣I␣don't␣take␣logs␣of␣non-positive␣numbers,")
("I'm␣zeroing␣this␣one.␣Proceed,␣with␣fingers␣crossed."); error; m_log ← 0;
end
```

This code is used in section 123.

126. The following somewhat different subroutine tests rigorously if ab is greater than, equal to, or less than cd, given integers (a, b, c, d). In most cases a quick decision is reached. The result is $+1$, 0, or -1 in the three respective cases.

```
define return_sign(#) ≡
        begin ab_vs_cd ← #; return;
        end
function ab_vs_cd(a, b, c, d : integer): integer;
  label exit;
  var q, r: integer;  { temporary registers }
  begin ⟨ Reduce to the case that a, c ≥ 0, b, d > 0 127 ⟩;
  loop begin q ← a div d; r ← c div b;
    if q ≠ r then
      if q > r then return_sign(1) else return_sign(−1);
    q ← a mod d; r ← c mod b;
    if r = 0 then
      if q = 0 then return_sign(0) else return_sign(1);
    if q = 0 then return_sign(−1);
    a ← b; b ← q; c ← d; d ← r;
    end;  { now a > d > 0 and c > b > 0 }
exit: end;
```

127. ⟨ Reduce to the case that $a, c \geq 0$, $b, d > 0$ 127 ⟩ ≡

```
if a < 0 then
  begin negate(a); negate(b);
  end;
if c < 0 then
  begin negate(c); negate(d);
  end;
if d ≤ 0 then
  begin if b ≥ 0 then
    if ((a = 0) ∨ (b = 0)) ∧ ((c = 0) ∨ (d = 0)) then return_sign(0)
    else return_sign(1);
  if d = 0 then
    if a = 0 then return_sign(0) else return_sign(−1);
  q ← a; a ← c; c ← q; q ← −b; b ← −d; d ← q;
  end
else if b ≤ 0 then
    begin if b < 0 then
      if a > 0 then return_sign(−1);
    if c = 0 then return_sign(0)
    else return_sign(−1);
    end
```

This code is used in section 126.

128. To consume a random integer, the program below will say '*next_random*' and then it will fetch *randoms*[*j_random*].

```
define next_random ≡
        if j_random = 0 then new_randoms
        else decr(j_random)
procedure new_randoms;
  var k: 0 .. 54;  { index into randoms }
    x: integer;  { accumulator }
  begin for k ← 0 to 23 do
    begin x ← randoms[k] − randoms[k + 31];
    if x < 0 then x ← x + fraction_one;
    randoms[k] ← x;
    end;
  for k ← 24 to 54 do
    begin x ← randoms[k] − randoms[k − 24];
    if x < 0 then x ← x + fraction_one;
    randoms[k] ← x;
    end;
  j_random ← 54;
  end;
```

129. To initialize the *randoms* table, we call the following routine.

```
procedure init_randoms(seed : integer);
  var j, jj, k: integer;  { more or less random integers }
    i: 0 .. 54;  { index into randoms }
  begin j ← abs(seed);
  while j ≥ fraction_one do j ← halfp(j);
  k ← 1;
  for i ← 0 to 54 do
    begin jj ← k; k ← j − k; j ← jj;
    if k < 0 then k ← k + fraction_one;
    randoms[(i ∗ 21) mod 55] ← j;
    end;
  new_randoms; new_randoms; new_randoms;  { "warm up" the array }
  end;
```

130. To produce a uniform random number in the range $0 \le u < x$ or $0 \ge u > x$ or $0 = u = x$, given a *scaled* value x, we proceed as shown here.

Note that the call of *take_frac* will produce the values 0 and x with about half the probability that it will produce any other particular values between 0 and x, because it rounds its answers.

```
function unif_rand(x : integer): integer;
  var y: integer;  { trial value }
  begin next_random; y ← take_frac(abs(x), randoms[j_random]);
  if y = abs(x) then unif_rand ← 0
  else if x > 0 then unif_rand ← y
    else unif_rand ← −y;
  end;
```

131. Finally, a normal deviate with mean zero and unit standard deviation can readily be obtained with the ratio method (Algorithm 3.4.1R in *The Art of Computer Programming*).

```
function norm_rand: integer;
  var x, u, l: integer;  { what the book would call 2^16 X, 2^28 U, and −2^24 ln U }
  begin repeat repeat next_random; x ← take_frac(112429, randoms[j_random] − fraction_half);
          { 2^16 √(8/e) ≈ 112428.82793 }
      next_random; u ← randoms[j_random];
    until abs(x) < u;
    x ← make_frac(x, u); l ← 139548960 − m_log(u);  { 2^24 · 12 ln 2 ≈ 139548959.6165 }
  until ab_vs_cd(1024, l, x, x) ≥ 0;
  norm_rand ← x;
  end;
```

132. Packed data. In order to make efficient use of storage space, TeX bases its major data structures on a *memory_word*, which contains either a (signed) integer, possibly scaled, or a (signed) *glue_ratio*, or a small number of fields that are one half or one quarter of the size used for storing integers.

If x is a variable of type *memory_word*, it contains up to four fields that can be referred to as follows:

$x.int$	(an *integer*)
$x.sc$	(a *scaled* integer)
$x.gr$	(a *glue_ratio*)
$x.hh.lh$, $x.hh.rh$	(two halfword fields)
$x.hh.b0$, $x.hh.b1$, $x.hh.rh$	(two quarterword fields, one halfword field)
$x.qqqq.b0$, $x.qqqq.b1$, $x.qqqq.b2$, $x.qqqq.b3$	(four quarterword fields)

This is somewhat cumbersome to write, and not very readable either, but macros will be used to make the notation shorter and more transparent. The Pascal code below gives a formal definition of *memory_word* and its subsidiary types, using packed variant records. TeX makes no assumptions about the relative positions of the fields within a word.

Since we are assuming 32-bit integers, a halfword must contain at least 16 bits, and a quarterword must contain at least 8 bits. But it doesn't hurt to have more bits; for example, with enough 36-bit words you might be able to have *mem_max* as large as 262142, which is eight times as much memory as anybody had during the first four years of TeX's existence.

N.B.: Valuable memory space will be dreadfully wasted unless TeX is compiled by a Pascal that packs all of the *memory_word* variants into the space of a single integer. This means, for example, that *glue_ratio* words should be *short_real* instead of *real* on some computers. Some Pascal compilers will pack an integer whose subrange is '0 .. 255' into an eight-bit field, but others insist on allocating space for an additional sign bit; on such systems you can get 256 values into a quarterword only if the subrange is '−128 .. 127'.

The present implementation tries to accommodate as many variations as possible, so it makes few assumptions. If integers having the subrange '*min_quarterword* .. *max_quarterword*' can be packed into a quarterword, and if integers having the subrange '*min_halfword* .. *max_halfword*' can be packed into a halfword, everything should work satisfactorily.

It is usually most efficient to have *min_quarterword* = *min_halfword* = 0, so one should try to achieve this unless it causes a severe problem. The values defined here are recommended for most 32-bit computers.

```
define min_quarterword = 0  { smallest allowable value in a quarterword }
define max_quarterword = "FFFF  { largest allowable value in a quarterword }
define min_halfword ≡ -"FFFFFFF  { smallest allowable value in a halfword }
define max_halfword ≡ "3FFFFFFF  { largest allowable value in a halfword }
```

133. Here are the inequalities that the quarterword and halfword values must satisfy (or rather, the inequalities that they mustn't satisfy):

```
⟨ Check the "constant" values for consistency 14 ⟩ +≡
  init if (mem_min ≠ mem_bot) ∨ (mem_max ≠ mem_top) then bad ← 10;
  tini
  if (mem_min > mem_bot) ∨ (mem_max < mem_top) then bad ← 10;
  if (min_quarterword > 0) ∨ (max_quarterword < "7FFF) then bad ← 11;
  if (min_halfword > 0) ∨ (max_halfword < "3FFFFFFF) then bad ← 12;
  if (min_quarterword < min_halfword) ∨ (max_quarterword > max_halfword) then bad ← 13;
  if (mem_min < min_halfword) ∨ (mem_max ≥ max_halfword) ∨
      (mem_bot − mem_min > max_halfword + 1) then bad ← 14;
  if (font_base < min_quarterword) ∨ (font_max > max_quarterword) then bad ← 15;
  if font_max > font_base + 256 then bad ← 16;
  if (save_size > max_halfword) ∨ (max_strings > max_halfword) then bad ← 17;
  if buf_size > max_halfword then bad ← 18;
  if max_quarterword − min_quarterword < "FFFF then bad ← 19;
```

134. The operation of adding or subtracting *min_quarterword* occurs quite frequently in TeX, so it is convenient to abbreviate this operation by using the macros *qi* and *qo* for input and output to and from quarterword format.

The inner loop of TeX will run faster with respect to compilers that don't optimize expressions like '$x+0$' and '$x-0$', if these macros are simplified in the obvious way when *min_quarterword* = 0.

```
define qi(#) ≡ # + min_quarterword  { to put an eight_bits item into a quarterword }
define qo(#) ≡ # − min_quarterword  { to take an eight_bits item out of a quarterword }
define hi(#) ≡ # + min_halfword  { to put a sixteen-bit item into a halfword }
define ho(#) ≡ # − min_halfword  { to take a sixteen-bit item from a halfword }
```

135. The reader should study the following definitions closely:

```
define sc ≡ int  { scaled data is equivalent to integer }
⟨ Types in the outer block 18 ⟩ +≡
  quarterword = min_quarterword .. max_quarterword;  { 1/4 of a word }
  halfword = min_halfword .. max_halfword;  { 1/2 of a word }
  two_choices = 1 .. 2;  { used when there are two variants in a record }
  four_choices = 1 .. 4;  { used when there are four variants in a record }
  two_halves = packed record rh: halfword;
    case two_choices of
    1: (lh : halfword);
    2: (b0 : quarterword; b1 : quarterword);
    end;
  four_quarters = packed record b0: quarterword;
    b1: quarterword;
    b2: quarterword;
    b3: quarterword;
    end;
  memory_word = record
    case four_choices of
    1: (int : integer);
    2: (gr : glue_ratio);
    3: (hh : two_halves);
    4: (qqqq : four_quarters);
    end;
  word_file = gzFile;
```

136. When debugging, we may want to print a *memory_word* without knowing what type it is; so we print it in all modes.

```
debug procedure print_word(w : memory_word);  { prints w in all ways }
begin print_int(w.int); print_char("␣");
print_scaled(w.sc); print_char("␣");
print_scaled(round(unity * float(w.gr))); print_ln;
print_int(w.hh.lh); print_char("="); print_int(w.hh.b0); print_char(":"); print_int(w.hh.b1);
print_char(";"); print_int(w.hh.rh); print_char("␣");
print_int(w.qqqq.b0); print_char(":"); print_int(w.qqqq.b1); print_char(":"); print_int(w.qqqq.b2);
print_char(":"); print_int(w.qqqq.b3);
end;
gubed
```

137. Dynamic memory allocation. The TeX system does nearly all of its own memory allocation, so that it can readily be transported into environments that do not have automatic facilities for strings, garbage collection, etc., and so that it can be in control of what error messages the user receives. The dynamic storage requirements of TeX are handled by providing a large array *mem* in which consecutive blocks of words are used as nodes by the TeX routines.

Pointer variables are indices into this array, or into another array called *eqtb* that will be explained later. A pointer variable might also be a special flag that lies outside the bounds of *mem*, so we allow pointers to assume any *halfword* value. The minimum halfword value represents a null pointer. TeX does not assume that *mem*[*null*] exists.

define *pointer* ≡ *halfword* { a flag or a location in *mem* or *eqtb* }
define *null* ≡ *min_halfword* { the null pointer }

⟨ Global variables 13 ⟩ +≡
temp_ptr: *pointer*; { a pointer variable for occasional emergency use }

138. The *mem* array is divided into two regions that are allocated separately, but the dividing line between these two regions is not fixed; they grow together until finding their "natural" size in a particular job. Locations less than or equal to *lo_mem_max* are used for storing variable-length records consisting of two or more words each. This region is maintained using an algorithm similar to the one described in exercise 2.5–19 of *The Art of Computer Programming*. However, no size field appears in the allocated nodes; the program is responsible for knowing the relevant size when a node is freed. Locations greater than or equal to *hi_mem_min* are used for storing one-word records; a conventional `AVAIL` stack is used for allocation in this region.

Locations of *mem* between *mem_bot* and *mem_top* may be dumped as part of preloaded format files, by the `INITEX` preprocessor. Production versions of TeX may extend the memory at both ends in order to provide more space; locations between *mem_min* and *mem_bot* are always used for variable-size nodes, and locations between *mem_top* and *mem_max* are always used for single-word nodes.

The key pointers that govern *mem* allocation have a prescribed order:

$$null \le mem_min \le mem_bot < lo_mem_max < hi_mem_min < mem_top \le mem_end \le mem_max .$$

Empirical tests show that the present implementation of TeX tends to spend about 9% of its running time allocating nodes, and about 6% deallocating them after their use.

⟨ Global variables 13 ⟩ +≡
mem: **array** [*mem_min* .. *mem_max*] **of** *memory_word*; { the big dynamic storage area }
lo_mem_max: *pointer*; { the largest location of variable-size memory in use }
hi_mem_min: *pointer*; { the smallest location of one-word memory in use }

139. In order to study the memory requirements of particular applications, it is possible to prepare a version of TeX that keeps track of current and maximum memory usage. When code between the delimiters **stat** ... **tats** is not "commented out," TeX will run a bit slower but it will report these statistics when *tracing_stats* is sufficiently large.

⟨ Global variables 13 ⟩ +≡
var_used, *dyn_used*: *integer*; { how much memory is in use }

140. Let's consider the one-word memory region first, since it's the simplest. The pointer variable *mem_end* holds the highest-numbered location of *mem* that has ever been used. The free locations of *mem* that occur between *hi_mem_min* and *mem_end*, inclusive, are of type *two_halves*, and we write *info*(*p*) and *link*(*p*) for the *lh* and *rh* fields of *mem*[*p*] when it is of this type. The single-word free locations form a linked list

$$avail,\ link(avail),\ link(link(avail)),\ \ldots$$

terminated by *null*.

```
define link(#) ≡ mem[#].hh.rh   { the link field of a memory word }
define info(#) ≡ mem[#].hh.lh   { the info field of a memory word }
⟨ Global variables 13 ⟩ +≡
avail: pointer;   { head of the list of available one-word nodes }
mem_end: pointer;   { the last one-word node used in mem }
```

141. If memory is exhausted, it might mean that the user has forgotten a right brace. We will define some procedures later that try to help pinpoint the trouble.

```
⟨ Declare the procedure called show_token_list 322 ⟩
⟨ Declare the procedure called runaway 336 ⟩
```

142. The function *get_avail* returns a pointer to a new one-word node whose *link* field is null. However, TeX will halt if there is no more room left.

If the available-space list is empty, i.e., if *avail* = *null*, we try first to increase *mem_end*. If that cannot be done, i.e., if *mem_end* = *mem_max*, we try to decrease *hi_mem_min*. If that cannot be done, i.e., if *hi_mem_min* = *lo_mem_max* + 1, we have to quit.

```
function get_avail: pointer;   { single-word node allocation }
  var p: pointer;   { the new node being got }
  begin p ← avail;   { get top location in the avail stack }
  if p ≠ null then avail ← link(avail)   { and pop it off }
  else if mem_end < mem_max then   { or go into virgin territory }
      begin incr(mem_end); p ← mem_end;
      end
    else begin decr(hi_mem_min); p ← hi_mem_min;
      if hi_mem_min ≤ lo_mem_max then
        begin runaway;   { if memory is exhausted, display possible runaway text }
        overflow("main␣memory␣size", mem_max + 1 − mem_min);   { quit; all one-word nodes are busy }
        end;
      end;
  link(p) ← null;   { provide an oft-desired initialization of the new node }
  stat incr(dyn_used); tats   { maintain statistics }
  get_avail ← p;
  end;
```

143. Conversely, a one-word node is recycled by calling *free_avail*. This routine is part of TeX's "inner loop," so we want it to be fast.

```
define free_avail(#) ≡   { single-word node liberation }
        begin link(#) ← avail; avail ← #;
        stat decr(dyn_used); tats
        end
```

144. There's also a *fast_get_avail* routine, which saves the procedure-call overhead at the expense of extra programming. This routine is used in the places that would otherwise account for the most calls of *get_avail*.

```
define fast_get_avail(#) ≡
    begin # ← avail; { avoid get_avail if possible, to save time }
    if # = null then # ← get_avail
    else begin avail ← link(#); link(#) ← null;
      stat incr(dyn_used); tats
      end;
    end
```

145. The procedure *flush_list*(*p*) frees an entire linked list of one-word nodes that starts at position *p*.

```
procedure flush_list(p : pointer); { makes list of single-word nodes available }
  var q, r: pointer; { list traversers }
  begin if p ≠ null then
    begin r ← p;
    repeat q ← r; r ← link(r);
      stat decr(dyn_used); tats
    until r = null; { now q is the last node on the list }
    link(q) ← avail; avail ← p;
    end;
  end;
```

146. The available-space list that keeps track of the variable-size portion of *mem* is a nonempty, doubly-linked circular list of empty nodes, pointed to by the roving pointer *rover*.

Each empty node has size 2 or more; the first word contains the special value *max_halfword* in its *link* field and the size in its *info* field; the second word contains the two pointers for double linking.

Each nonempty node also has size 2 or more. Its first word is of type *two_halves*, and its *link* field is never equal to *max_halfword*. Otherwise there is complete flexibility with respect to the contents of its other fields and its other words.

(We require *mem_max* < *max_halfword* because terrible things can happen when *max_halfword* appears in the *link* field of a nonempty node.)

```
define empty_flag ≡ max_halfword { the link of an empty variable-size node }
define is_empty(#) ≡ (link(#) = empty_flag) { tests for empty node }
define node_size ≡ info { the size field in empty variable-size nodes }
define llink(#) ≡ info(# + 1) { left link in doubly-linked list of empty nodes }
define rlink(#) ≡ link(# + 1) { right link in doubly-linked list of empty nodes }
⟨ Global variables 13 ⟩ +≡
rover: pointer; { points to some node in the list of empties }
```

147. A call to *get_node* with argument s returns a pointer to a new node of size s, which must be 2 or more. The *link* field of the first word of this new node is set to null. An overflow stop occurs if no suitable space exists.

If *get_node* is called with $s = 2^{30}$, it simply merges adjacent free areas and returns the value *max_halfword*.

```
function get_node(s : integer): pointer; { variable-size node allocation }
  label found, exit, restart;
  var p: pointer; { the node currently under inspection }
    q: pointer; { the node physically after node p }
    r: integer; { the newly allocated node, or a candidate for this honor }
    t: integer; { temporary register }
  begin restart: p ← rover; { start at some free node in the ring }
  repeat ⟨ Try to allocate within node p and its physical successors, and goto found if allocation was
      possible 149 ⟩;
    p ← rlink(p); { move to the next node in the ring }
  until p = rover; { repeat until the whole list has been traversed }
  if s = ´10000000000 then
    begin get_node ← max_halfword; return;
    end;
  if lo_mem_max + 2 < hi_mem_min then
    if lo_mem_max + 2 ≤ mem_bot + max_halfword then
      ⟨ Grow more variable-size memory and goto restart 148 ⟩;
  overflow("main␣memory␣size", mem_max + 1 − mem_min); { sorry, nothing satisfactory is left }
found: link(r) ← null; { this node is now nonempty }
  stat var_used ← var_used + s; { maintain usage statistics }
  tats
  get_node ← r;
exit: end;
```

148. The lower part of *mem* grows by 1000 words at a time, unless we are very close to going under. When it grows, we simply link a new node into the available-space list. This method of controlled growth helps to keep the *mem* usage consecutive when TeX is implemented on "virtual memory" systems.

```
⟨ Grow more variable-size memory and goto restart 148 ⟩ ≡
  begin if hi_mem_min − lo_mem_max ≥ 1998 then t ← lo_mem_max + 1000
  else t ← lo_mem_max + 1 + (hi_mem_min − lo_mem_max) div 2; { lo_mem_max + 2 ≤ t < hi_mem_min }
  p ← llink(rover); q ← lo_mem_max; rlink(p) ← q; llink(rover) ← q;
  if t > mem_bot + max_halfword then t ← mem_bot + max_halfword;
  rlink(q) ← rover; llink(q) ← p; link(q) ← empty_flag; node_size(q) ← t − lo_mem_max;
  lo_mem_max ← t; link(lo_mem_max) ← null; info(lo_mem_max) ← null; rover ← q; goto restart;
  end
```

This code is used in section 147.

149. Empirical tests show that the routine in this section performs a node-merging operation about 0.75 times per allocation, on the average, after which it finds that $r > p + 1$ about 95% of the time.

```
⟨ Try to allocate within node p and its physical successors, and goto found if allocation was possible 149 ⟩ ≡
  q ← p + node_size(p);  { find the physical successor }
  while is_empty(q) do  { merge node p with node q }
    begin t ← rlink(q);
    if q = rover then rover ← t;
    llink(t) ← llink(q); rlink(llink(q)) ← t;
    q ← q + node_size(q);
    end;
  r ← q − s;
  if r > p + 1 then ⟨ Allocate from the top of node p and goto found 150 ⟩;
  if r = p then
    if rlink(p) ≠ p then ⟨ Allocate entire node p and goto found 151 ⟩;
  node_size(p) ← q − p  { reset the size in case it grew }
```

This code is used in section 147.

150.
```
⟨ Allocate from the top of node p and goto found 150 ⟩ ≡
  begin node_size(p) ← r − p;  { store the remaining size }
  rover ← p;  { start searching here next time }
  goto found;
  end
```

This code is used in section 149.

151. Here we delete node p from the ring, and let *rover* rove around.

```
⟨ Allocate entire node p and goto found 151 ⟩ ≡
  begin rover ← rlink(p); t ← llink(p); llink(rover) ← t; rlink(t) ← rover; goto found;
  end
```

This code is used in section 149.

152. Conversely, when some variable-size node p of size s is no longer needed, the operation *free_node*(p, s) will make its words available, by inserting p as a new empty node just before where *rover* now points.

```
procedure free_node(p : pointer; s : halfword);  { variable-size node liberation }
  var q: pointer;  { llink(rover) }
  begin node_size(p) ← s; link(p) ← empty_flag; q ← llink(rover); llink(p) ← q; rlink(p) ← rover;
      { set both links }
  llink(rover) ← p; rlink(q) ← p;  { insert p into the ring }
  stat var_used ← var_used − s; tats  { maintain statistics }
  end;
```

153. Just before INITEX writes out the memory, it sorts the doubly linked available space list. The list is probably very short at such times, so a simple insertion sort is used. The smallest available location will be pointed to by *rover*, the next-smallest by *rlink*(*rover*), etc.

```
init procedure sort_avail; { sorts the available variable-size nodes by location }
var p, q, r: pointer; { indices into mem }
  old_rover: pointer; { initial rover setting }
begin p ← get_node('10000000000); { merge adjacent free areas }
p ← rlink(rover); rlink(rover) ← max_halfword; old_rover ← rover;
while p ≠ old_rover do ⟨ Sort p into the list starting at rover and advance p to rlink(p) 154 ⟩;
p ← rover;
while rlink(p) ≠ max_halfword do
  begin llink(rlink(p)) ← p; p ← rlink(p);
  end;
rlink(p) ← rover; llink(rover) ← p;
end;
tini
```

154. The following **while** loop is guaranteed to terminate, since the list that starts at *rover* ends with *max_halfword* during the sorting procedure.

```
⟨ Sort p into the list starting at rover and advance p to rlink(p) 154 ⟩ ≡
  if p < rover then
    begin q ← p; p ← rlink(q); rlink(q) ← rover; rover ← q;
    end
  else begin q ← rover;
    while rlink(q) < p do q ← rlink(q);
    r ← rlink(p); rlink(p) ← rlink(q); rlink(q) ← p; p ← r;
    end
```

This code is used in section 153.

155. Data structures for boxes and their friends. From the computer's standpoint, TEX's chief mission is to create horizontal and vertical lists. We shall now investigate how the elements of these lists are represented internally as nodes in the dynamic memory.

A horizontal or vertical list is linked together by *link* fields in the first word of each node. Individual nodes represent boxes, glue, penalties, or special things like discretionary hyphens; because of this variety, some nodes are longer than others, and we must distinguish different kinds of nodes. We do this by putting a '*type*' field in the first word, together with the link and an optional '*subtype*'.

define *type*(#) ≡ *mem*[#].*hh*.*b0* { identifies what kind of node this is }
define *subtype*(#) ≡ *mem*[#].*hh*.*b1* { secondary identification in some cases }

156. A *char_node*, which represents a single character, is the most important kind of node because it accounts for the vast majority of all boxes. Special precautions are therefore taken to ensure that a *char_node* does not take up much memory space. Every such node is one word long, and in fact it is identifiable by this property, since other kinds of nodes have at least two words, and they appear in *mem* locations less than *hi_mem_min*. This makes it possible to omit the *type* field in a *char_node*, leaving us room for two bytes that identify a *font* and a *character* within that font.

Note that the format of a *char_node* allows for up to 256 different fonts and up to 256 characters per font; but most implementations will probably limit the total number of fonts to fewer than 75 per job, and most fonts will stick to characters whose codes are less than 128 (since higher codes are more difficult to access on most keyboards).

Extensions of TEX intended for oriental languages will need even more than 256×256 possible characters, when we consider different sizes and styles of type. It is suggested that Chinese and Japanese fonts be handled by representing such characters in two consecutive *char_node* entries: The first of these has *font* = *font_base*, and its *link* points to the second; the second identifies the font and the character dimensions. The saving feature about oriental characters is that most of them have the same box dimensions. The *character* field of the first *char_node* is a "*charext*" that distinguishes between graphic symbols whose dimensions are identical for typesetting purposes. (See the METAFONT manual.) Such an extension of TEX would not be difficult; further details are left to the reader.

In order to make sure that the *character* code fits in a quarterword, TEX adds the quantity *min_quarterword* to the actual code.

Character nodes appear only in horizontal lists, never in vertical lists.

define *is_char_node*(#) ≡ (# ≥ *hi_mem_min*) { does the argument point to a *char_node*? }
define *font* ≡ *type* { the font code in a *char_node* }
define *character* ≡ *subtype* { the character code in a *char_node* }

157. An *hlist_node* stands for a box that was made from a horizontal list. Each *hlist_node* is seven words long, and contains the following fields (in addition to the mandatory *type* and *link*, which we shall not mention explicitly when discussing the other node types): The *height* and *width* and *depth* are scaled integers denoting the dimensions of the box. There is also a *shift_amount* field, a scaled integer indicating how much this box should be lowered (if it appears in a horizontal list), or how much it should be moved to the right (if it appears in a vertical list). There is a *list_ptr* field, which points to the beginning of the list from which this box was fabricated; if *list_ptr* is *null*, the box is empty. Finally, there are three fields that represent the setting of the glue: *glue_set*(*p*) is a word of type *glue_ratio* that represents the proportionality constant for glue setting; *glue_sign*(*p*) is *stretching* or *shrinking* or *normal* depending on whether or not the glue should stretch or shrink or remain rigid; and *glue_order*(*p*) specifies the order of infinity to which glue setting applies (*normal*, *fil*, *fill*, or *filll*). The *subtype* field is not used in TeX. In ε-TeX the *subtype* field records the box direction mode *box_lr*.

define *hlist_node* = 0 { *type* of hlist nodes }
define *box_node_size* = 7 { number of words to allocate for a box node }
define *width_offset* = 1 { position of *width* field in a box node }
define *depth_offset* = 2 { position of *depth* field in a box node }
define *height_offset* = 3 { position of *height* field in a box node }
define *width*(#) ≡ *mem*[# + *width_offset*].*sc* { width of the box, in sp }
define *depth*(#) ≡ *mem*[# + *depth_offset*].*sc* { depth of the box, in sp }
define *height*(#) ≡ *mem*[# + *height_offset*].*sc* { height of the box, in sp }
define *shift_amount*(#) ≡ *mem*[# + 4].*sc* { repositioning distance, in sp }
define *list_offset* = 5 { position of *list_ptr* field in a box node }
define *list_ptr*(#) ≡ *link*(# + *list_offset*) { beginning of the list inside the box }
define *glue_order*(#) ≡ *subtype*(# + *list_offset*) { applicable order of infinity }
define *glue_sign*(#) ≡ *type*(# + *list_offset*) { stretching or shrinking }
define *normal* = 0 { the most common case when several cases are named }
define *stretching* = 1 { glue setting applies to the stretch components }
define *shrinking* = 2 { glue setting applies to the shrink components }
define *glue_offset* = 6 { position of *glue_set* in a box node }
define *glue_set*(#) ≡ *mem*[# + *glue_offset*].*gr* { a word of type *glue_ratio* for glue setting }

158. The *new_null_box* function returns a pointer to an *hlist_node* in which all subfields have the values corresponding to '`\hbox{}`'. (The *subtype* field is set to *min_quarterword*, for historic reasons that are no longer relevant.)

function *new_null_box*: *pointer*; { creates a new box node }
var *p*: *pointer*; { the new node }
begin *p* ← *get_node*(*box_node_size*); *type*(*p*) ← *hlist_node*; *subtype*(*p*) ← *min_quarterword*;
width(*p*) ← 0; *depth*(*p*) ← 0; *height*(*p*) ← 0; *shift_amount*(*p*) ← 0; *list_ptr*(*p*) ← *null*;
glue_sign(*p*) ← *normal*; *glue_order*(*p*) ← *normal*; *set_glue_ratio_zero*(*glue_set*(*p*)); *new_null_box* ← *p*;
end;

159. A *vlist_node* is like an *hlist_node* in all respects except that it contains a vertical list.

define *vlist_node* = 1 { *type* of vlist nodes }

160. A *rule_node* stands for a solid black rectangle; it has *width*, *depth*, and *height* fields just as in an *hlist_node*. However, if any of these dimensions is -2^{30}, the actual value will be determined by running the rule up to the boundary of the innermost enclosing box. This is called a "running dimension." The *width* is never running in an hlist; the *height* and *depth* are never running in a vlist.

define *rule_node* = 2 { *type* of rule nodes }
define *rule_node_size* = 4 { number of words to allocate for a rule node }
define *null_flag* ≡ −´10000000000 { -2^{30}, signifies a missing item }
define *is_running*(#) ≡ (# = *null_flag*) { tests for a running dimension }

161. A new rule node is delivered by the *new_rule* function. It makes all the dimensions "running," so you have to change the ones that are not allowed to run.

function *new_rule*: *pointer*;
var *p*: *pointer*; { the new node }
begin *p* ← *get_node*(*rule_node_size*); *type*(*p*) ← *rule_node*; *subtype*(*p*) ← 0; { the *subtype* is not used }
width(*p*) ← *null_flag*; *depth*(*p*) ← *null_flag*; *height*(*p*) ← *null_flag*; *new_rule* ← *p*;
end;

162. Insertions are represented by *ins_node* records, where the *subtype* indicates the corresponding box number. For example, '`\insert 250`' leads to an *ins_node* whose *subtype* is 250 + *min_quarterword*. The *height* field of an *ins_node* is slightly misnamed; it actually holds the natural height plus depth of the vertical list being inserted. The *depth* field holds the *split_max_depth* to be used in case this insertion is split, and the *split_top_ptr* points to the corresponding *split_top_skip*. The *float_cost* field holds the *floating_penalty* that will be used if this insertion floats to a subsequent page after a split insertion of the same class. There is one more field, the *ins_ptr*, which points to the beginning of the vlist for the insertion.

define *ins_node* = 3 { *type* of insertion nodes }
define *ins_node_size* = 5 { number of words to allocate for an insertion }
define *float_cost*(#) ≡ *mem*[# + 1].*int* { the *floating_penalty* to be used }
define *ins_ptr*(#) ≡ *info*(# + 4) { the vertical list to be inserted }
define *split_top_ptr*(#) ≡ *link*(# + 4) { the *split_top_skip* to be used }

163. A *mark_node* has a *mark_ptr* field that points to the reference count of a token list that contains the user's `\mark` text. In addition there is a *mark_class* field that contains the mark class.

define *mark_node* = 4 { *type* of a mark node }
define *small_node_size* = 2 { number of words to allocate for most node types }
define *mark_ptr*(#) ≡ *link*(# + 1) { head of the token list for a mark }
define *mark_class*(#) ≡ *info*(# + 1) { the mark class }

164. An *adjust_node*, which occurs only in horizontal lists, specifies material that will be moved out into the surrounding vertical list; i.e., it is used to implement TeX's '`\vadjust`' operation. The *adjust_ptr* field points to the vlist containing this material.

define *adjust_node* = 5 { *type* of an adjust node }
define *adjust_pre* ≡ *subtype* { **if** *subtype* ≠ 0 it is pre-adjustment }
{ *append_list* is used to append a list to *tail* }
define *append_list*(#) ≡
begin *link*(*tail*) ← *link*(#); *append_list_end*
define *append_list_end*(#) ≡ *tail* ← #;
end
define *adjust_ptr*(#) ≡ *mem*[# + 1].*int* { vertical list to be moved out of horizontal list }

165. A *ligature_node*, which occurs only in horizontal lists, specifies a character that was fabricated from the interaction of two or more actual characters. The second word of the node, which is called the *lig_char* word, contains *font* and *character* fields just as in a *char_node*. The characters that generated the ligature have not been forgotten, since they are needed for diagnostic messages and for hyphenation; the *lig_ptr* field points to a linked list of character nodes for all original characters that have been deleted. (This list might be empty if the characters that generated the ligature were retained in other nodes.)

The *subtype* field is 0, plus 2 and/or 1 if the original source of the ligature included implicit left and/or right boundaries.

define *ligature_node* = 6 { *type* of a ligature node }
define *lig_char*(#) ≡ # + 1 { the word where the ligature is to be found }
define *lig_ptr*(#) ≡ *link*(*lig_char*(#)) { the list of characters }

166. The *new_ligature* function creates a ligature node having given contents of the *font*, *character*, and *lig_ptr* fields. We also have a *new_lig_item* function, which returns a two-word node having a given *character* field. Such nodes are used for temporary processing as ligatures are being created.

```
function new_ligature(f, c : quarterword; q : pointer): pointer;
  var p: pointer; { the new node }
  begin p ← get_node(small_node_size); type(p) ← ligature_node; font(lig_char(p)) ← f;
  character(lig_char(p)) ← c; lig_ptr(p) ← q; subtype(p) ← 0; new_ligature ← p;
  end;
function new_lig_item(c : quarterword): pointer;
  var p: pointer; { the new node }
  begin p ← get_node(small_node_size); character(p) ← c; lig_ptr(p) ← null; new_lig_item ← p;
  end;
```

167. A *disc_node*, which occurs only in horizontal lists, specifies a "discretionary" line break. If such a break occurs at node p, the text that starts at *pre_break*(p) will precede the break, the text that starts at *post_break*(p) will follow the break, and text that appears in the next *replace_count*(p) nodes will be ignored. For example, an ordinary discretionary hyphen, indicated by '\-', yields a *disc_node* with *pre_break* pointing to a *char_node* containing a hyphen, *post_break* = *null*, and *replace_count* = 0. All three of the discretionary texts must be lists that consist entirely of character, kern, box, rule, and ligature nodes.

If *pre_break*(p) = *null*, the *ex_hyphen_penalty* will be charged for this break. Otherwise the *hyphen_penalty* will be charged. The texts will actually be substituted into the list by the line-breaking algorithm if it decides to make the break, and the discretionary node will disappear at that time; thus, the output routine sees only discretionaries that were not chosen.

define *disc_node* = 7 { *type* of a discretionary node }
define *replace_count* ≡ *subtype* { how many subsequent nodes to replace }
define *pre_break* ≡ *llink* { text that precedes a discretionary break }
define *post_break* ≡ *rlink* { text that follows a discretionary break }

```
function new_disc: pointer; { creates an empty disc_node }
  var p: pointer; { the new node }
  begin p ← get_node(small_node_size); type(p) ← disc_node; replace_count(p) ← 0; pre_break(p) ← null;
  post_break(p) ← null; new_disc ← p;
  end;
```

168. A *whatsit_node* is a wild card reserved for extensions to TeX. The *subtype* field in its first word says what '*whatsit*' it is, and implicitly determines the node size (which must be 2 or more) and the format of the remaining words. When a *whatsit_node* is encountered in a list, special actions are invoked; knowledgeable people who are careful not to mess up the rest of TeX are able to make TeX do new things by adding code at the end of the program. For example, there might be a 'TeXnicolor' extension to specify different colors of ink, and the whatsit node might contain the desired parameters.

The present implementation of TeX treats the features associated with '`\write`' and '`\special`' as if they were extensions, in order to illustrate how such routines might be coded. We shall defer further discussion of extensions until the end of this program.

define *whatsit_node* = 8 { *type* of special extension nodes }

169. To support "native" fonts, we build *native_word_node*s, which are variable size whatsits. These have the same *width*, *depth*, and *height* fields as a *box_node*, at offsets 1-3, and then a word containing a size field for the node, a font number, a length, and a glyph count. Then there is a field containing a C pointer to a glyph info array; this and the glyph count are set by *set_native_metrics*. Copying and freeing of these nodes needs to take account of this! This is followed by $2 * length$ bytes, for the actual characters of the string (in UTF-16).

So *native_node_size*, which does not include any space for the actual text, is 6.

0-3 whatsits subtypes are used for open, write, close, special; 4 is language; pdfTeX uses up through 30-something, so we use subtypes starting from 40.

There are also *glyph_node*s; these are like *native_word_node*s in having *width*, *depth*, and *height* fields, but then they contain a glyph ID rather than size and length fields, and there's no subsidiary C pointer.

```
define native_word_node = 40  { subtype of whatsits that hold native_font words }
define native_word_node_AT = 41  { a native_word_node that should output ActualText }
define is_native_word_subtype(#) ≡ ((subtype(#) ≥ native_word_node) ∧ (subtype(#) ≤
        native_word_node_AT))
define glyph_node = 42  { subtype in whatsits that hold glyph numbers }
define native_node_size = 6  { size of a native_word node (plus the actual chars) – see also xetex.h }
define glyph_node_size = 5
define native_size(#) ≡ mem[# + 4].qqqq.b0
define native_font(#) ≡ mem[# + 4].qqqq.b1
define native_length(#) ≡ mem[# + 4].qqqq.b2
define native_glyph_count(#) ≡ mem[# + 4].qqqq.b3
define native_glyph_info_ptr(#) ≡ mem[# + 5].ptr
define native_glyph_info_size = 10
        { number of bytes of info per glyph: 16-bit glyph ID, 32-bit x and y coords }
define native_glyph ≡ native_length  { in glyph_nodes, we store the glyph number here }
define free_native_glyph_info(#) ≡
      begin if native_glyph_info_ptr(#) ≠ null_ptr then
        begin libc_free(native_glyph_info_ptr(#)); native_glyph_info_ptr(#) ← null_ptr;
        native_glyph_count(#) ← 0;
        end
      end
procedure copy_native_glyph_info(src : pointer; dest : pointer);
  var glyph_count: integer;
  begin if native_glyph_info_ptr(src) ≠ null_ptr then
    begin glyph_count ← native_glyph_count(src);
    native_glyph_info_ptr(dest) ← xmalloc_array(char, glyph_count * native_glyph_info_size);
    memcpy(native_glyph_info_ptr(dest), native_glyph_info_ptr(src), glyph_count * native_glyph_info_size);
    native_glyph_count(dest) ← glyph_count;
    end
  end;
```

170. Picture files are handled with nodes that include fields for the transform associated with the picture, and a pathname for the picture file itself. They also have the *width*, *depth*, and *height* fields of a *box_node* at offsets 1-3. (*depth* will always be zero, as it happens.)

So *pic_node_size*, which does not include any space for the picture file pathname, is 7.

A *pdf_node* is just like *pic_node*, but generate a different `XDV` file code.

define *pic_node* = 43 { *subtype* in whatsits that hold picture file references }
define *pdf_node* = 44 { *subtype* in whatsits that hold PDF page references }

define *pic_node_size* = 9 { must sync with `xetex.h` }
define *pic_path_length*(#) ≡ *mem*[# + 4].*hh*.*b0*
define *pic_page*(#) ≡ *mem*[# + 4].*hh*.*b1*
define *pic_transform1*(#) ≡ *mem*[# + 5].*hh*.*lh*
define *pic_transform2*(#) ≡ *mem*[# + 5].*hh*.*rh*
define *pic_transform3*(#) ≡ *mem*[# + 6].*hh*.*lh*
define *pic_transform4*(#) ≡ *mem*[# + 6].*hh*.*rh*
define *pic_transform5*(#) ≡ *mem*[# + 7].*hh*.*lh*
define *pic_transform6*(#) ≡ *mem*[# + 7].*hh*.*rh*
define *pic_pdf_box*(#) ≡ *mem*[# + 8].*hh*.*b0*

171. A *math_node*, which occurs only in horizontal lists, appears before and after mathematical formulas. The *subtype* field is *before* before the formula and *after* after it. There is a *width* field, which represents the amount of surrounding space inserted by `\mathsurround`.

In addition a *math_node* with *subtype* > *after* and *width* = 0 will be (ab)used to record a regular *math_node* reinserted after being discarded at a line break or one of the text direction primitives (`\beginL`, `\endL`, `\beginR`, and `\endR`).

define *math_node* = 9 { *type* of a math node }
define *before* = 0 { *subtype* for math node that introduces a formula }
define *after* = 1 { *subtype* for math node that winds up a formula }

define *M_code* = 2
define *begin_M_code* = *M_code* + *before* { *subtype* for `\beginM` node }
define *end_M_code* = *M_code* + *after* { *subtype* for `\endM` node }
define *L_code* = 4
define *begin_L_code* = *L_code* + *begin_M_code* { *subtype* for `\beginL` node }
define *end_L_code* = *L_code* + *end_M_code* { *subtype* for `\endL` node }
define *R_code* = *L_code* + *L_code*
define *begin_R_code* = *R_code* + *begin_M_code* { *subtype* for `\beginR` node }
define *end_R_code* = *R_code* + *end_M_code* { *subtype* for `\endR` node }

define *end_LR*(#) ≡ *odd*(*subtype*(#))
define *end_LR_type*(#) ≡ (*L_code* ∗ (*subtype*(#) **div** *L_code*) + *end_M_code*)
define *begin_LR_type*(#) ≡ (# − *after* + *before*)

function *new_math*(*w* : *scaled*; *s* : *small_number*): *pointer*;
var *p*: *pointer*; { the new node }
begin *p* ← *get_node*(*small_node_size*); *type*(*p*) ← *math_node*; *subtype*(*p*) ← *s*; *width*(*p*) ← *w*;
new_math ← *p*;
end;

172. TeX makes use of the fact that *hlist_node*, *vlist_node*, *rule_node*, *ins_node*, *mark_node*, *adjust_node*, *ligature_node*, *disc_node*, *whatsit_node*, and *math_node* are at the low end of the type codes, by permitting a break at glue in a list if and only if the *type* of the previous node is less than *math_node*. Furthermore, a node is discarded after a break if its type is *math_node* or more.

define *precedes_break*(#) ≡ (*type*(#) < *math_node*)
define *non_discardable*(#) ≡ (*type*(#) < *math_node*)

173. A *glue_node* represents glue in a list. However, it is really only a pointer to a separate glue specification, since TeX makes use of the fact that many essentially identical nodes of glue are usually present. If *p* points to a *glue_node*, *glue_ptr*(*p*) points to another packet of words that specify the stretch and shrink components, etc.

Glue nodes also serve to represent leaders; the *subtype* is used to distinguish between ordinary glue (which is called *normal*) and the three kinds of leaders (which are called *a_leaders*, *c_leaders*, and *x_leaders*). The *leader_ptr* field points to a rule node or to a box node containing the leaders; it is set to *null* in ordinary glue nodes.

Many kinds of glue are computed from TeX's "skip" parameters, and it is helpful to know which parameter has led to a particular glue node. Therefore the *subtype* is set to indicate the source of glue, whenever it originated as a parameter. We will be defining symbolic names for the parameter numbers later (e.g., *line_skip_code* = 0, *baseline_skip_code* = 1, etc.); it suffices for now to say that the *subtype* of parametric glue will be the same as the parameter number, plus one.

In math formulas there are two more possibilities for the *subtype* in a glue node: *mu_glue* denotes an `\mskip` (where the units are scaled `mu` instead of scaled `pt`); and *cond_math_glue* denotes the '`\nonscript`' feature that cancels the glue node immediately following if it appears in a subscript.

define *glue_node* = 10 { *type* of node that points to a glue specification }
define *cond_math_glue* = 98 { special *subtype* to suppress glue in the next node }
define *mu_glue* = 99 { *subtype* for math glue }
define *a_leaders* = 100 { *subtype* for aligned leaders }
define *c_leaders* = 101 { *subtype* for centered leaders }
define *x_leaders* = 102 { *subtype* for expanded leaders }
define *glue_ptr* ≡ *llink* { pointer to a glue specification }
define *leader_ptr* ≡ *rlink* { pointer to box or rule node for leaders }

174. A glue specification has a halfword reference count in its first word, representing *null* plus the number of glue nodes that point to it (less one). Note that the reference count appears in the same position as the *link* field in list nodes; this is the field that is initialized to *null* when a node is allocated, and it is also the field that is flagged by *empty_flag* in empty nodes.

Glue specifications also contain three *scaled* fields, for the *width*, *stretch*, and *shrink* dimensions. Finally, there are two one-byte fields called *stretch_order* and *shrink_order*; these contain the orders of infinity (*normal*, *fil*, *fill*, or *filll*) corresponding to the stretch and shrink values.

define *glue_spec_size* = 4 { number of words to allocate for a glue specification }
define *glue_ref_count*(#) ≡ *link*(#) { reference count of a glue specification }
define *stretch*(#) ≡ *mem*[# + 2].*sc* { the stretchability of this glob of glue }
define *shrink*(#) ≡ *mem*[# + 3].*sc* { the shrinkability of this glob of glue }
define *stretch_order* ≡ *type* { order of infinity for stretching }
define *shrink_order* ≡ *subtype* { order of infinity for shrinking }
define *fil* = 1 { first-order infinity }
define *fill* = 2 { second-order infinity }
define *filll* = 3 { third-order infinity }

⟨ Types in the outer block 18 ⟩ +≡
glue_ord = *normal* .. *filll*; { infinity to the 0, 1, 2, or 3 power }

175. Here is a function that returns a pointer to a copy of a glue spec. The reference count in the copy is *null*, because there is assumed to be exactly one reference to the new specification.

```
function new_spec(p : pointer): pointer; { duplicates a glue specification }
  var q: pointer; { the new spec }
  begin q ← get_node(glue_spec_size);
  mem[q] ← mem[p]; glue_ref_count(q) ← null;
  width(q) ← width(p); stretch(q) ← stretch(p); shrink(q) ← shrink(p); new_spec ← q;
  end;
```

176. And here's a function that creates a glue node for a given parameter identified by its code number; for example, *new_param_glue*(*line_skip_code*) returns a pointer to a glue node for the current `\lineskip`.

```
function new_param_glue(n : small_number): pointer;
  var p: pointer; { the new node }
    q: pointer; { the glue specification }
  begin p ← get_node(small_node_size); type(p) ← glue_node; subtype(p) ← n + 1; leader_ptr(p) ← null;
  q ← ⟨ Current mem equivalent of glue parameter number n 250 ⟩; glue_ptr(p) ← q;
  incr(glue_ref_count(q)); new_param_glue ← p;
  end;
```

177. Glue nodes that are more or less anonymous are created by *new_glue*, whose argument points to a glue specification.

```
function new_glue(q : pointer): pointer;
  var p: pointer; { the new node }
  begin p ← get_node(small_node_size); type(p) ← glue_node; subtype(p) ← normal;
  leader_ptr(p) ← null; glue_ptr(p) ← q; incr(glue_ref_count(q)); new_glue ← p;
  end;
```

178. Still another subroutine is needed: This one is sort of a combination of *new_param_glue* and *new_glue*. It creates a glue node for one of the current glue parameters, but it makes a fresh copy of the glue specification, since that specification will probably be subject to change, while the parameter will stay put. The global variable *temp_ptr* is set to the address of the new spec.

```
function new_skip_param(n : small_number): pointer;
  var p: pointer; { the new node }
  begin temp_ptr ← new_spec(⟨ Current mem equivalent of glue parameter number n 250 ⟩);
  p ← new_glue(temp_ptr); glue_ref_count(temp_ptr) ← null; subtype(p) ← n + 1; new_skip_param ← p;
  end;
```

179. A *kern_node* has a *width* field to specify a (normally negative) amount of spacing. This spacing correction appears in horizontal lists between letters like A and V when the font designer said that it looks better to move them closer together or further apart. A kern node can also appear in a vertical list, when its '*width*' denotes additional spacing in the vertical direction. The *subtype* is either *normal* (for kerns inserted from font information or math mode calculations) or *explicit* (for kerns inserted from `\kern` and `\/` commands) or *acc_kern* (for kerns inserted from non-math accents) or *mu_glue* (for kerns inserted from `\mkern` specifications in math formulas).

define *kern_node* = 11 { *type* of a kern node }
define *explicit* = 1 { *subtype* of kern nodes from `\kern` and `\/` }
define *acc_kern* = 2 { *subtype* of kern nodes from accents }
define *space_adjustment* = 3
{ *subtype* of kern nodes from `\XeTeXinterwordspaceshaping` adjustment }
{ memory structure for marginal kerns }
define *margin_kern_node* = 40
define *margin_kern_node_size* = 3
define *margin_char*(#) ≡ *info*(# + 2) { unused for now; relevant for font expansion }
{ *subtype* of marginal kerns }
define *left_side* ≡ 0
define *right_side* ≡ 1
{ base for lp/rp codes starts from 2: 0 for *hyphen_char*, 1 for *skew_char* }
define *lp_code_base* ≡ 2
define *rp_code_base* ≡ 3
define *max_hlist_stack* = 512 { maximum fill level for *hlist_stack* }
{ maybe good if larger than 2 ∗ *max_quarterword*, so that box nesting level would overflow first }

180. The *new_kern* function creates a kern node having a given width.

```
function new_kern(w : scaled): pointer;
  var p: pointer; { the new node }
  begin p ← get_node(small_node_size); type(p) ← kern_node; subtype(p) ← normal; width(p) ← w;
  new_kern ← p;
  end;
```

181. ⟨ Global variables 13 ⟩ +≡

```
last_leftmost_char: pointer;
last_rightmost_char: pointer;
hlist_stack: array [0 .. max_hlist_stack] of pointer;
    { stack for find_protchar_left( ) and find_protchar_right( ) }
hlist_stack_level: 0 .. max_hlist_stack; { fill level for hlist_stack }
first_p: pointer; { to access the first node of the paragraph }
global_prev_p: pointer;
    { to access prev_p in line_break; should be kept in sync with prev_p by update_prev_p }
```

182. A *penalty_node* specifies the penalty associated with line or page breaking, in its *penalty* field. This field is a fullword integer, but the full range of integer values is not used: Any penalty ≥ 10000 is treated as infinity, and no break will be allowed for such high values. Similarly, any penalty ≤ −10000 is treated as negative infinity, and a break will be forced.

define *penalty_node* = 12 { *type* of a penalty node }
define *inf_penalty* = *inf_bad* { "infinite" penalty value }
define *eject_penalty* = −*inf_penalty* { "negatively infinite" penalty value }
define *penalty*(#) ≡ *mem*[# + 1].*int* { the added cost of breaking a list here }

183. Anyone who has been reading the last few sections of the program will be able to guess what comes next.

```
function new_penalty(m : integer): pointer;
  var p: pointer;   { the new node }
  begin p ← get_node(small_node_size); type(p) ← penalty_node; subtype(p) ← 0;
      { the subtype is not used }
  penalty(p) ← m; new_penalty ← p;
  end;
```

184. You might think that we have introduced enough node types by now. Well, almost, but there is one more: An *unset_node* has nearly the same format as an *hlist_node* or *vlist_node*; it is used for entries in `\halign` or `\valign` that are not yet in their final form, since the box dimensions are their "natural" sizes before any glue adjustment has been made. The *glue_set* word is not present; instead, we have a *glue_stretch* field, which contains the total stretch of order *glue_order* that is present in the hlist or vlist being boxed. Similarly, the *shift_amount* field is replaced by a *glue_shrink* field, containing the total shrink of order *glue_sign* that is present. The *subtype* field is called *span_count*; an unset box typically contains the data for *qo*(*span_count*) + 1 columns. Unset nodes will be changed to box nodes when alignment is completed.

```
define unset_node = 13   { type for an unset node }
define glue_stretch(#) ≡ mem[# + glue_offset].sc   { total stretch in an unset node }
define glue_shrink ≡ shift_amount   { total shrink in an unset node }
define span_count ≡ subtype   { indicates the number of spanned columns }
```

185. In fact, there are still more types coming. When we get to math formula processing we will see that a *style_node* has *type* = 14; and a number of larger type codes will also be defined, for use in math mode only.

186. Warning: If any changes are made to these data structure layouts, such as changing any of the node sizes or even reordering the words of nodes, the *copy_node_list* procedure and the memory initialization code below may have to be changed. Such potentially dangerous parts of the program are listed in the index under 'data structure assumptions'. However, other references to the nodes are made symbolically in terms of the `WEB` macro definitions above, so that format changes will leave TeX's other algorithms intact.

187. Memory layout. Some areas of *mem* are dedicated to fixed usage, since static allocation is more efficient than dynamic allocation when we can get away with it. For example, locations *mem_bot* to *mem_bot* + 3 are always used to store the specification for glue that is '`0pt plus 0pt minus 0pt`'. The following macro definitions accomplish the static allocation by giving symbolic names to the fixed positions. Static variable-size nodes appear in locations *mem_bot* through *lo_mem_stat_max*, and static single-word nodes appear in locations *hi_mem_stat_min* through *mem_top*, inclusive. It is harmless to let *lig_trick* and *garbage* share the same location of *mem*.

define *zero_glue* ≡ *mem_bot* { specification for `0pt plus 0pt minus 0pt` }
define *fil_glue* ≡ *zero_glue* + *glue_spec_size* { `0pt plus 1fil minus 0pt` }
define *fill_glue* ≡ *fil_glue* + *glue_spec_size* { `0pt plus 1fill minus 0pt` }
define *ss_glue* ≡ *fill_glue* + *glue_spec_size* { `0pt plus 1fil minus 1fil` }
define *fil_neg_glue* ≡ *ss_glue* + *glue_spec_size* { `0pt plus -1fil minus 0pt` }
define *lo_mem_stat_max* ≡ *fil_neg_glue* + *glue_spec_size* − 1
{ largest statically allocated word in the variable-size *mem* }
define *page_ins_head* ≡ *mem_top* { list of insertion data for current page }
define *contrib_head* ≡ *mem_top* − 1 { vlist of items not yet on current page }
define *page_head* ≡ *mem_top* − 2 { vlist for current page }
define *temp_head* ≡ *mem_top* − 3 { head of a temporary list of some kind }
define *hold_head* ≡ *mem_top* − 4 { head of a temporary list of another kind }
define *adjust_head* ≡ *mem_top* − 5 { head of adjustment list returned by *hpack* }
define *active* ≡ *mem_top* − 7 { head of active list in *line_break*, needs two words }
define *align_head* ≡ *mem_top* − 8 { head of preamble list for alignments }
define *end_span* ≡ *mem_top* − 9 { tail of spanned-width lists }
define *omit_template* ≡ *mem_top* − 10 { a constant token list }
define *null_list* ≡ *mem_top* − 11 { permanently empty list }
define *lig_trick* ≡ *mem_top* − 12 { a ligature masquerading as a *char_node* }
define *garbage* ≡ *mem_top* − 12 { used for scrap information }
define *backup_head* ≡ *mem_top* − 13 { head of token list built by *scan_keyword* }
define *pre_adjust_head* ≡ *mem_top* − 14 { head of pre-adjustment list returned by *hpack* }
define *hi_mem_stat_min* ≡ *mem_top* − 14 { smallest statically allocated word in the one-word *mem* }
define *hi_mem_stat_usage* = 15 { the number of one-word nodes always present }

188. The following code gets *mem* off to a good start, when TeX is initializing itself the slow way.

⟨ Local variables for initialization 19 ⟩ +≡
k: *integer*; { index into *mem*, *eqtb*, etc. }

189. ⟨ Initialize table entries (done by INITEX only) 189 ⟩ ≡

```
for k ← mem_bot + 1 to lo_mem_stat_max do mem[k].sc ← 0; { all glue dimensions are zeroed }
k ← mem_bot; while k ≤ lo_mem_stat_max do { set first words of glue specifications }
  begin glue_ref_count(k) ← null + 1; stretch_order(k) ← normal; shrink_order(k) ← normal;
  k ← k + glue_spec_size;
  end;
stretch(fil_glue) ← unity; stretch_order(fil_glue) ← fil;
stretch(fill_glue) ← unity; stretch_order(fill_glue) ← fill;
stretch(ss_glue) ← unity; stretch_order(ss_glue) ← fil;
shrink(ss_glue) ← unity; shrink_order(ss_glue) ← fil;
stretch(fil_neg_glue) ← −unity; stretch_order(fil_neg_glue) ← fil;
rover ← lo_mem_stat_max + 1; link(rover) ← empty_flag; { now initialize the dynamic memory }
node_size(rover) ← 1000; { which is a 1000-word available node }
llink(rover) ← rover; rlink(rover) ← rover;
lo_mem_max ← rover + 1000; link(lo_mem_max) ← null; info(lo_mem_max) ← null;
for k ← hi_mem_stat_min to mem_top do mem[k] ← mem[lo_mem_max]; { clear list heads }
⟨ Initialize the special list heads and constant nodes 838 ⟩;
avail ← null; mem_end ← mem_top; hi_mem_min ← hi_mem_stat_min;
    { initialize the one-word memory }
var_used ← lo_mem_stat_max + 1 − mem_bot; dyn_used ← hi_mem_stat_usage; { initialize statistics }
```

See also sections 248, 254, 258, 266, 276, 285, 587, 1000, 1005, 1270, 1355, 1432, 1463, 1629, and 1665.

This code is used in section 8.

190. If TeX is extended improperly, the *mem* array might get screwed up. For example, some pointers might be wrong, or some "dead" nodes might not have been freed when the last reference to them disappeared. Procedures *check_mem* and *search_mem* are available to help diagnose such problems. These procedures make use of two arrays called *free* and *was_free* that are present only if TeX's debugging routines have been included. (You may want to decrease the size of *mem* while you are debugging.)

⟨ Global variables 13 ⟩ +≡

```
debug free: packed array [mem_min .. mem_max] of boolean; { free cells }
was_free: packed array [mem_min .. mem_max] of boolean; { previously free cells }
was_mem_end, was_lo_max, was_hi_min: pointer; { previous mem_end, lo_mem_max, and hi_mem_min }
panicking: boolean; { do we want to check memory constantly? }
gubed
```

191. ⟨ Set initial values of key variables 23 ⟩ +≡

```
debug was_mem_end ← mem_min; { indicate that everything was previously free }
was_lo_max ← mem_min; was_hi_min ← mem_max; panicking ← false;
gubed
```

192. Procedure *check_mem* makes sure that the available space lists of *mem* are well formed, and it optionally prints out all locations that are reserved now but were free the last time this procedure was called.

```
debug procedure check_mem(print_locs : boolean);
label done1, done2;  { loop exits }
var p, q: pointer;  { current locations of interest in mem }
  clobbered: boolean;  { is something amiss? }
begin for p ← mem_min to lo_mem_max do free[p] ← false;  { you can probably do this faster }
for p ← hi_mem_min to mem_end do free[p] ← false;  { ditto }
⟨ Check single-word avail list 193 ⟩;
⟨ Check variable-size avail list 194 ⟩;
⟨ Check flags of unavailable nodes 195 ⟩;
if print_locs then ⟨ Print newly busy locations 196 ⟩;
for p ← mem_min to lo_mem_max do was_free[p] ← free[p];
for p ← hi_mem_min to mem_end do was_free[p] ← free[p];  { was_free ← free might be faster }
was_mem_end ← mem_end; was_lo_max ← lo_mem_max; was_hi_min ← hi_mem_min;
end;
gubed
```

193. ⟨ Check single-word *avail* list 193 ⟩ ≡

```
p ← avail; q ← null; clobbered ← false;
while p ≠ null do
  begin if (p > mem_end) ∨ (p < hi_mem_min) then clobbered ← true
  else if free[p] then clobbered ← true;
  if clobbered then
    begin print_nl("AVAIL␣list␣clobbered␣at␣"); print_int(q); goto done1;
    end;
  free[p] ← true; q ← p; p ← link(q);
  end;
done1:
```

This code is used in section 192.

194. ⟨ Check variable-size *avail* list 194 ⟩ ≡

```
p ← rover; q ← null; clobbered ← false;
repeat if (p ≥ lo_mem_max) ∨ (p < mem_min) then clobbered ← true
  else if (rlink(p) ≥ lo_mem_max) ∨ (rlink(p) < mem_min) then clobbered ← true
    else if ¬(is_empty(p)) ∨ (node_size(p) < 2) ∨ (p + node_size(p) > lo_mem_max) ∨
            (llink(rlink(p)) ≠ p) then clobbered ← true;
  if clobbered then
    begin print_nl("Double-AVAIL␣list␣clobbered␣at␣"); print_int(q); goto done2;
    end;
  for q ← p to p + node_size(p) − 1 do  { mark all locations free }
    begin if free[q] then
      begin print_nl("Doubly␣free␣location␣at␣"); print_int(q); goto done2;
      end;
    free[q] ← true;
    end;
  q ← p; p ← rlink(p);
until p = rover;
done2:
```

This code is used in section 192.

195. ⟨ Check flags of unavailable nodes 195 ⟩ ≡

```
p ← mem_min;
while p ≤ lo_mem_max do { node p should not be empty }
  begin if is_empty(p) then
    begin print_nl("Bad␣flag␣at␣"); print_int(p);
    end;
  while (p ≤ lo_mem_max) ∧ ¬free[p] do incr(p);
  while (p ≤ lo_mem_max) ∧ free[p] do incr(p);
  end
```

This code is used in section 192.

196. ⟨ Print newly busy locations 196 ⟩ ≡

```
begin print_nl("New␣busy␣locs:");
for p ← mem_min to lo_mem_max do
  if ¬free[p] ∧ ((p > was_lo_max) ∨ was_free[p]) then
    begin print_char("␣"); print_int(p);
    end;
for p ← hi_mem_min to mem_end do
  if ¬free[p] ∧ ((p < was_hi_min) ∨ (p > was_mem_end) ∨ was_free[p]) then
    begin print_char("␣"); print_int(p);
    end;
end
```

This code is used in section 192.

197. The *search_mem* procedure attempts to answer the question "Who points to node p?" In doing so, it fetches *link* and *info* fields of *mem* that might not be of type *two_halves*. Strictly speaking, this is undefined in Pascal, and it can lead to "false drops" (words that seem to point to p purely by coincidence). But for debugging purposes, we want to rule out the places that do *not* point to p, so a few false drops are tolerable.

```
debug procedure search_mem(p : pointer); { look for pointers to p }
var q: integer; { current position being searched }
begin for q ← mem_min to lo_mem_max do
  begin if link(q) = p then
    begin print_nl("LINK("); print_int(q); print_char(")");
    end;
  if info(q) = p then
    begin print_nl("INFO("); print_int(q); print_char(")");
    end;
  end;
for q ← hi_mem_min to mem_end do
  begin if link(q) = p then
    begin print_nl("LINK("); print_int(q); print_char(")");
    end;
  if info(q) = p then
    begin print_nl("INFO("); print_int(q); print_char(")");
    end;
  end;
⟨ Search eqtb for equivalents equal to p 281 ⟩;
⟨ Search save_stack for equivalents that point to p 315 ⟩;
⟨ Search hyph_list for pointers to p 987 ⟩;
end;
gubed
```

198. Some stuff for character protrusion.

```
procedure pdf_error(t, p : str_number);
  begin normalize_selector; print_err("Error");
  if t ≠ 0 then
    begin print("␣("); print(t); print(")");
    end;
  print(":␣"); print(p); succumb;
  end;
function prev_rightmost(s, e : pointer): pointer;
        { finds the node preceding the rightmost node e; s is some node before e }
  var p: pointer;
  begin prev_rightmost ← null; p ← s;
  if p = null then return;
  while link(p) ≠ e do
    begin p ← link(p);
    if p = null then return;
    end;
  prev_rightmost ← p;
  end;
function round_xn_over_d(x : scaled; n, d : integer): scaled;
  var positive: boolean;   { was x ≥ 0? }
    t, u, v: nonnegative_integer;   { intermediate quantities }
  begin if x ≥ 0 then positive ← true
  else begin negate(x); positive ← false;
    end;
  t ← (x mod ´100000) * n; u ← (x div ´100000) * n + (t div ´100000);
  v ← (u mod d) * ´100000 + (t mod ´100000);
  if u div d ≥ ´100000 then arith_error ← true
  else u ← ´100000 * (u div d) + (v div d);
  v ← v mod d;
  if 2 * v ≥ d then incr(u);
  if positive then round_xn_over_d ← u
  else round_xn_over_d ← −u;
  end; ⟨ Declare procedures that need to be declared forward for pdfTeX 1411 ⟩
```

199. Displaying boxes. We can reinforce our knowledge of the data structures just introduced by considering two procedures that display a list in symbolic form. The first of these, called *short_display*, is used in "overfull box" messages to give the top-level description of a list. The other one, called *show_node_list*, prints a detailed description of exactly what is in the data structure.

The philosophy of *short_display* is to ignore the fine points about exactly what is inside boxes, except that ligatures and discretionary breaks are expanded. As a result, *short_display* is a recursive procedure, but the recursion is never more than one level deep.

A global variable *font_in_short_display* keeps track of the font code that is assumed to be present when *short_display* begins; deviations from this font will be printed.

```
⟨ Global variables 13 ⟩ +≡
font_in_short_display: integer;   { an internal font number }
```

200. Boxes, rules, inserts, whatsits, marks, and things in general that are sort of "complicated" are indicated only by printing '`[]`'.

```
procedure short_display(p : integer);   { prints highlights of list p }
  var n: integer;   { for replacement counts }
  begin while p > mem_min do
    begin if is_char_node(p) then
      begin if p ≤ mem_end then
        begin if font(p) ≠ font_in_short_display then
          begin if (font(p) < font_base) ∨ (font(p) > font_max) then print_char("*")
          else ⟨ Print the font identifier for font(p) 297 ⟩;
          print_char("␣"); font_in_short_display ← font(p);
          end;
        print_ASCII(qo(character(p)));
        end;
      end
    else ⟨ Print a short indication of the contents of node p 201 ⟩;
    p ← link(p);
    end;
  end;
```

201. ⟨ Print a short indication of the contents of node p 201 ⟩ ≡

```
case type(p) of
hlist_node, vlist_node, ins_node, mark_node, adjust_node, unset_node: print("[]");
whatsit_node: case subtype(p) of
  native_word_node, native_word_node_AT: begin if native_font(p) ≠ font_in_short_display then
      begin print_esc(font_id_text(native_font(p))); print_char("␣");
      font_in_short_display ← native_font(p);
      end;
    print_native_word(p);
    end;
  othercases print("[]")
  endcases;
rule_node: print_char("|");
glue_node: if glue_ptr(p) ≠ zero_glue then print_char("␣");
math_node: if subtype(p) ≥ L_code then print("[]")
  else print_char("$");
ligature_node: short_display(lig_ptr(p));
disc_node: begin short_display(pre_break(p)); short_display(post_break(p));
  n ← replace_count(p);
  while n > 0 do
    begin if link(p) ≠ null then p ← link(p);
    decr(n);
    end;
  end;
othercases do_nothing
endcases
```

This code is used in section 200.

202. The *show_node_list* routine requires some auxiliary subroutines: one to print a font-and-character combination, one to print a token list without its reference count, and one to print a rule dimension.

```
procedure print_font_and_char(p : integer); { prints char_node data }
  begin if p > mem_end then print_esc("CLOBBERED.")
  else begin if (font(p) < font_base) ∨ (font(p) > font_max) then print_char("*")
    else ⟨ Print the font identifier for font(p) 297 ⟩;
    print_char("␣"); print_ASCII(qo(character(p)));
    end;
  end;
procedure print_mark(p : integer); { prints token list data in braces }
  begin print_char("{");
  if (p < hi_mem_min) ∨ (p > mem_end) then print_esc("CLOBBERED.")
  else show_token_list(link(p), null, max_print_line − 10);
  print_char("}");
  end;
procedure print_rule_dimen(d : scaled); { prints dimension in rule node }
  begin if is_running(d) then print_char("*")
  else print_scaled(d);
  end;
```

203. Then there is a subroutine that prints glue stretch and shrink, possibly followed by the name of finite units:

```
procedure print_glue(d : scaled; order : integer; s : str_number);  { prints a glue component }
  begin print_scaled(d);
  if (order < normal) ∨ (order > filll) then print("foul")
  else if order > normal then
      begin print("fil");
      while order > fil do
        begin print_char("l"); decr(order);
        end;
      end
    else if s ≠ 0 then print(s);
  end;
```

204. The next subroutine prints a whole glue specification.

```
procedure print_spec(p : integer; s : str_number);  { prints a glue specification }
  begin if (p < mem_min) ∨ (p ≥ lo_mem_max) then print_char("*")
  else begin print_scaled(width(p));
    if s ≠ 0 then print(s);
    if stretch(p) ≠ 0 then
      begin print(" plus "); print_glue(stretch(p), stretch_order(p), s);
      end;
    if shrink(p) ≠ 0 then
      begin print(" minus "); print_glue(shrink(p), shrink_order(p), s);
      end;
    end;
  end;
```

205. We also need to declare some procedures that appear later in this documentation.

⟨ Declare procedures needed for displaying the elements of mlists 733 ⟩
⟨ Declare the procedure called *print_skip_param* 251 ⟩

206. Since boxes can be inside of boxes, *show_node_list* is inherently recursive, up to a given maximum number of levels. The history of nesting is indicated by the current string, which will be printed at the beginning of each line; the length of this string, namely *cur_length*, is the depth of nesting.

Recursive calls on *show_node_list* therefore use the following pattern:

```
define node_list_display(#) ≡
    begin append_char("."); show_node_list(#); flush_char;
    end  { str_room need not be checked; see show_box below }
```

207. A global variable called *depth_threshold* is used to record the maximum depth of nesting for which *show_node_list* will show information. If we have *depth_threshold* = 0, for example, only the top level information will be given and no sublists will be traversed. Another global variable, called *breadth_max*, tells the maximum number of items to show at each level; *breadth_max* had better be positive, or you won't see anything.

```
⟨ Global variables 13 ⟩ +≡
depth_threshold: integer;  { maximum nesting depth in box displays }
breadth_max: integer;  { maximum number of items shown at the same list level }
```

208. Now we are ready for *show_node_list* itself. This procedure has been written to be "extra robust" in the sense that it should not crash or get into a loop even if the data structures have been messed up by bugs in the rest of the program. You can safely call its parent routine *show_box*(*p*) for arbitrary values of *p* when you are debugging TeX. However, in the presence of bad data, the procedure may fetch a *memory_word* whose variant is different from the way it was stored; for example, it might try to read *mem*[*p*].*hh* when *mem*[*p*] contains a scaled integer, if *p* is a pointer that has been clobbered or chosen at random.

```
procedure show_node_list(p : integer); { prints a node list symbolically }
  label exit;
  var n: integer; { the number of items already printed at this level }
    i: integer; { temp index for printing chars of picfile paths }
    g: real; { a glue ratio, as a floating point number }
  begin if cur_length > depth_threshold then
    begin if p > null then print("␣[]"); { indicate that there's been some truncation }
    return;
    end;
  n ← 0;
  while p > mem_min do
    begin print_ln; print_current_string; { display the nesting history }
    if p > mem_end then { pointer out of range }
      begin print("Bad␣link,␣display␣aborted."); return;
      end;
    incr(n);
    if n > breadth_max then { time to stop }
      begin print("etc."); return;
      end;
    ⟨ Display node p 209 ⟩;
    p ← link(p);
    end;
exit: end;
```

209. ⟨ Display node p 209 ⟩ ≡
```
if is_char_node(p) then print_font_and_char(p)
else case type(p) of
  hlist_node, vlist_node, unset_node: ⟨ Display box p 210 ⟩;
  rule_node: ⟨ Display rule p 213 ⟩;
  ins_node: ⟨ Display insertion p 214 ⟩;
  whatsit_node: ⟨ Display the whatsit node p 1416 ⟩;
  glue_node: ⟨ Display glue p 215 ⟩;
  kern_node: ⟨ Display kern p 217 ⟩;
  margin_kern_node: begin print_esc("kern"); print_scaled(width(p));
    if subtype(p) = left_side then print("␣(left␣margin)")
    else print("␣(right␣margin)");
    end;
  math_node: ⟨ Display math node p 218 ⟩;
  ligature_node: ⟨ Display ligature p 219 ⟩;
  penalty_node: ⟨ Display penalty p 220 ⟩;
  disc_node: ⟨ Display discretionary p 221 ⟩;
  mark_node: ⟨ Display mark p 222 ⟩;
  adjust_node: ⟨ Display adjustment p 223 ⟩;
  ⟨ Cases of show_node_list that arise in mlists only 732 ⟩
  othercases print("Unknown␣node␣type!")
  endcases
```
This code is used in section 208.

210. ⟨ Display box p 210 ⟩ ≡
```
begin if type(p) = hlist_node then print_esc("h")
else if type(p) = vlist_node then print_esc("v")
  else print_esc("unset");
print("box("); print_scaled(height(p)); print_char("+"); print_scaled(depth(p)); print(")x");
print_scaled(width(p));
if type(p) = unset_node then ⟨ Display special fields of the unset node p 211 ⟩
else begin ⟨ Display the value of glue_set(p) 212 ⟩;
  if shift_amount(p) ≠ 0 then
    begin print(",␣shifted␣"); print_scaled(shift_amount(p));
    end;
  if eTeX_ex then ⟨ Display if this box is never to be reversed 1514 ⟩;
  end;
node_list_display(list_ptr(p)); { recursive call }
end
```
This code is used in section 209.

211. ⟨ Display special fields of the unset node p 211 ⟩ ≡

```
begin if span_count(p) ≠ min_quarterword then
  begin print("␣("); print_int(qo(span_count(p)) + 1); print("␣columns)");
  end;
if glue_stretch(p) ≠ 0 then
  begin print(",␣stretch␣"); print_glue(glue_stretch(p), glue_order(p), 0);
  end;
if glue_shrink(p) ≠ 0 then
  begin print(",␣shrink␣"); print_glue(glue_shrink(p), glue_sign(p), 0);
  end;
end
```

This code is used in section 210.

212. The code will have to change in this place if *glue_ratio* is a structured type instead of an ordinary *real*. Note that this routine should avoid arithmetic errors even if the *glue_set* field holds an arbitrary random value. The following code assumes that a properly formed nonzero *real* number has absolute value 2^{20} or more when it is regarded as an integer; this precaution was adequate to prevent floating point underflow on the author's computer.

⟨ Display the value of *glue_set*(*p*) 212 ⟩ ≡

```
g ← float(glue_set(p));
if (g ≠ float_constant(0)) ∧ (glue_sign(p) ≠ normal) then
  begin print(",␣glue␣set␣");
  if glue_sign(p) = shrinking then print("-␣");
  if abs(mem[p + glue_offset].int) < ´4000000 then print("?.?")
  else if abs(g) > float_constant(20000) then
      begin if g > float_constant(0) then print_char(">")
      else print("<␣-");
      print_glue(20000 * unity, glue_order(p), 0);
      end
    else print_glue(round(unity * g), glue_order(p), 0);
  end
```

This code is used in section 210.

213. ⟨ Display rule p 213 ⟩ ≡

```
begin print_esc("rule("); print_rule_dimen(height(p)); print_char("+"); print_rule_dimen(depth(p));
print(")x"); print_rule_dimen(width(p));
end
```

This code is used in section 209.

214. ⟨ Display insertion p 214 ⟩ ≡

```
begin print_esc("insert"); print_int(qo(subtype(p))); print(",␣natural␣size␣");
print_scaled(height(p)); print(";␣split("); print_spec(split_top_ptr(p), 0); print_char(",");
print_scaled(depth(p)); print(");␣float␣cost␣"); print_int(float_cost(p)); node_list_display(ins_ptr(p));
    { recursive call }
end
```

This code is used in section 209.

215. ⟨ Display glue p 215 ⟩ ≡
```
if subtype(p) ≥ a_leaders then ⟨ Display leaders p 216 ⟩
else begin print_esc("glue");
  if subtype(p) ≠ normal then
    begin print_char("(");
    if subtype(p) < cond_math_glue then print_skip_param(subtype(p) − 1)
    else if subtype(p) = cond_math_glue then print_esc("nonscript")
      else print_esc("mskip");
    print_char(")");
    end;
  if subtype(p) ≠ cond_math_glue then
    begin print_char("␣");
    if subtype(p) < cond_math_glue then print_spec(glue_ptr(p), 0)
    else print_spec(glue_ptr(p), "mu");
    end;
  end
```
This code is used in section 209.

216. ⟨ Display leaders p 216 ⟩ ≡
```
begin print_esc("");
if subtype(p) = c_leaders then print_char("c")
else if subtype(p) = x_leaders then print_char("x");
print("leaders␣"); print_spec(glue_ptr(p), 0); node_list_display(leader_ptr(p));   { recursive call }
end
```
This code is used in section 215.

217. An "explicit" kern value is indicated implicitly by an explicit space.

⟨ Display kern p 217 ⟩ ≡
```
if subtype(p) ≠ mu_glue then
  begin print_esc("kern");
  if subtype(p) ≠ normal then print_char("␣");
  print_scaled(width(p));
  if subtype(p) = acc_kern then print("␣(for␣accent)")
  else if subtype(p) = space_adjustment then print("␣(space␣adjustment)");
  end
else begin print_esc("mkern"); print_scaled(width(p)); print("mu");
  end
```
This code is used in section 209.

218. ⟨ Display math node p 218 ⟩ ≡

```
if subtype(p) > after then
  begin if end_LR(p) then print_esc("end")
  else print_esc("begin");
  if subtype(p) > R_code then print_char("R")
  else if subtype(p) > L_code then print_char("L")
    else print_char("M");
  end
else begin print_esc("math");
  if subtype(p) = before then print("on")
  else print("off");
  if width(p) ≠ 0 then
    begin print(",␣surrounded␣"); print_scaled(width(p));
    end;
  end
```

This code is used in section 209.

219. ⟨ Display ligature p 219 ⟩ ≡

```
begin print_font_and_char(lig_char(p)); print("␣(ligature␣");
if subtype(p) > 1 then print_char("|");
font_in_short_display ← font(lig_char(p)); short_display(lig_ptr(p));
if odd(subtype(p)) then print_char("|");
print_char(")");
end
```

This code is used in section 209.

220. ⟨ Display penalty p 220 ⟩ ≡

```
begin print_esc("penalty␣"); print_int(penalty(p));
end
```

This code is used in section 209.

221. The *post_break* list of a discretionary node is indicated by a prefixed '`|`' instead of the '`.`' before the *pre_break* list.

⟨ Display discretionary p 221 ⟩ ≡

```
begin print_esc("discretionary");
if replace_count(p) > 0 then
  begin print("␣replacing␣"); print_int(replace_count(p));
  end;
node_list_display(pre_break(p)); { recursive call }
append_char("|"); show_node_list(post_break(p)); flush_char; { recursive call }
end
```

This code is used in section 209.

222. ⟨ Display mark p 222 ⟩ ≡

```
begin print_esc("mark");
if mark_class(p) ≠ 0 then
  begin print_char("s"); print_int(mark_class(p));
  end;
print_mark(mark_ptr(p));
end
```

This code is used in section 209.

223. ⟨ Display adjustment p 223 ⟩ ≡

```
begin print_esc("vadjust");
if adjust_pre(p) ≠ 0 then print("␣pre␣");
node_list_display(adjust_ptr(p)); { recursive call }
end
```

This code is used in section 209.

224. The recursive machinery is started by calling *show_box*.

```
procedure show_box(p : pointer);
  begin ⟨ Assign the values depth_threshold ← show_box_depth and breadth_max ← show_box_breadth 262 ⟩;
  if breadth_max ≤ 0 then breadth_max ← 5;
  if pool_ptr + depth_threshold ≥ pool_size then depth_threshold ← pool_size − pool_ptr − 1;
      { now there's enough room for prefix string }
  show_node_list(p); { the show starts at p }
  print_ln;
  end;
procedure short_display_n(p, m : integer); { prints highlights of list p }
  begin breadth_max ← m; depth_threshold ← pool_size − pool_ptr − 1; show_node_list(p);
    { the show starts at p }
  end;
```

225. Destroying boxes. When we are done with a node list, we are obliged to return it to free storage, including all of its sublists. The recursive procedure *flush_node_list* does this for us.

226. First, however, we shall consider two non-recursive procedures that do simpler tasks. The first of these, *delete_token_ref*, is called when a pointer to a token list's reference count is being removed. This means that the token list should disappear if the reference count was *null*, otherwise the count should be decreased by one.

```
define token_ref_count(#) ≡ info(#)  { reference count preceding a token list }
procedure delete_token_ref(p : pointer);
        { p points to the reference count of a token list that is losing one reference }
  begin if token_ref_count(p) = null then flush_list(p)
  else decr(token_ref_count(p));
  end;
```

227. Similarly, *delete_glue_ref* is called when a pointer to a glue specification is being withdrawn.

```
define fast_delete_glue_ref(#) ≡
        begin if glue_ref_count(#) = null then free_node(#, glue_spec_size)
        else decr(glue_ref_count(#));
        end
procedure delete_glue_ref(p : pointer);  { p points to a glue specification }
    fast_delete_glue_ref(p);
```

228. Now we are ready to delete any node list, recursively. In practice, the nodes deleted are usually charnodes (about 2/3 of the time), and they are glue nodes in about half of the remaining cases.

```
procedure flush_node_list(p : pointer); { erase list of nodes starting at p }
  label done; { go here when node p has been freed }
  var q: pointer; { successor to node p }
  begin while p ≠ null do
    begin q ← link(p);
    if is_char_node(p) then free_avail(p)
    else begin case type(p) of
      hlist_node, vlist_node, unset_node: begin flush_node_list(list_ptr(p)); free_node(p, box_node_size);
        goto done;
        end;
      rule_node: begin free_node(p, rule_node_size); goto done;
        end;
      ins_node: begin flush_node_list(ins_ptr(p)); delete_glue_ref(split_top_ptr(p));
        free_node(p, ins_node_size); goto done;
        end;
      whatsit_node: ⟨ Wipe out the whatsit node p and goto done 1418 ⟩;
      glue_node: begin fast_delete_glue_ref(glue_ptr(p));
        if leader_ptr(p) ≠ null then flush_node_list(leader_ptr(p));
        end;
      kern_node, math_node, penalty_node: do_nothing;
      margin_kern_node: begin free_node(p, margin_kern_node_size); goto done;
        end;
      ligature_node: flush_node_list(lig_ptr(p));
      mark_node: delete_token_ref(mark_ptr(p));
      disc_node: begin flush_node_list(pre_break(p)); flush_node_list(post_break(p));
        end;
      adjust_node: flush_node_list(adjust_ptr(p));
      ⟨ Cases of flush_node_list that arise in mlists only 740 ⟩
      othercases confusion("flushing")
      endcases;
      free_node(p, small_node_size);
    done: end;
    p ← q;
    end;
  end;
```

229. Copying boxes. Another recursive operation that acts on boxes is sometimes needed: The procedure *copy_node_list* returns a pointer to another node list that has the same structure and meaning as the original. Note that since glue specifications and token lists have reference counts, we need not make copies of them. Reference counts can never get too large to fit in a halfword, since each pointer to a node is in a different memory address, and the total number of memory addresses fits in a halfword.

(Well, there actually are also references from outside *mem*; if the *save_stack* is made arbitrarily large, it would theoretically be possible to break TeX by overflowing a reference count. But who would want to do that?)

```
define add_token_ref(#) ≡ incr(token_ref_count(#))  { new reference to a token list }
define add_glue_ref(#) ≡ incr(glue_ref_count(#))  { new reference to a glue spec }
```

230. The copying procedure copies words en masse without bothering to look at their individual fields. If the node format changes—for example, if the size is altered, or if some link field is moved to another relative position—then this code may need to be changed too.

```
function copy_node_list(p : pointer): pointer;
        { makes a duplicate of the node list that starts at p and returns a pointer to the new list }
  var h: pointer;  { temporary head of copied list }
    q: pointer;  { previous position in new list }
    r: pointer;  { current node being fabricated for new list }
    words: 0 .. 5;  { number of words remaining to be copied }
  begin h ← get_avail; q ← h;
  while p ≠ null do
    begin ⟨ Make a copy of node p in node r 231 ⟩;
    link(q) ← r; q ← r; p ← link(p);
    end;
  link(q) ← null; q ← link(h); free_avail(h); copy_node_list ← q;
  end;
```

231. ⟨ Make a copy of node p in node r 231 ⟩ ≡

```
  words ← 1;  { this setting occurs in more branches than any other }
  if is_char_node(p) then r ← get_avail
  else ⟨ Case statement to copy different types and set words to the number of initial words not yet
        copied 232 ⟩;
  while words > 0 do
    begin decr(words); mem[r + words] ← mem[p + words];
    end
```

This code is used in section 230.

232. ⟨ Case statement to copy different types and set *words* to the number of initial words not yet copied 232 ⟩ ≡

```
case type(p) of
hlist_node, vlist_node, unset_node: begin r ← get_node(box_node_size); mem[r + 6] ← mem[p + 6];
  mem[r + 5] ← mem[p + 5]; { copy the last two words }
  list_ptr(r) ← copy_node_list(list_ptr(p)); { this affects mem[r + 5] }
  words ← 5;
  end;
rule_node: begin r ← get_node(rule_node_size); words ← rule_node_size;
  end;
ins_node: begin r ← get_node(ins_node_size); mem[r + 4] ← mem[p + 4]; add_glue_ref(split_top_ptr(p));
  ins_ptr(r) ← copy_node_list(ins_ptr(p)); { this affects mem[r + 4] }
  words ← ins_node_size − 1;
  end;
whatsit_node: ⟨ Make a partial copy of the whatsit node p and make r point to it; set words to the
    number of initial words not yet copied 1417 ⟩;
glue_node: begin r ← get_node(small_node_size); add_glue_ref(glue_ptr(p)); glue_ptr(r) ← glue_ptr(p);
  leader_ptr(r) ← copy_node_list(leader_ptr(p));
  end;
kern_node, math_node, penalty_node: begin r ← get_node(small_node_size); words ← small_node_size;
  end;
margin_kern_node: begin r ← get_node(margin_kern_node_size); words ← margin_kern_node_size;
  end;
ligature_node: begin r ← get_node(small_node_size); mem[lig_char(r)] ← mem[lig_char(p)];
    { copy font and character }
  lig_ptr(r) ← copy_node_list(lig_ptr(p));
  end;
disc_node: begin r ← get_node(small_node_size); pre_break(r) ← copy_node_list(pre_break(p));
  post_break(r) ← copy_node_list(post_break(p));
  end;
mark_node: begin r ← get_node(small_node_size); add_token_ref(mark_ptr(p));
  words ← small_node_size;
  end;
adjust_node: begin r ← get_node(small_node_size); adjust_ptr(r) ← copy_node_list(adjust_ptr(p));
  end; { words = 1 = small_node_size − 1 }
othercases confusion("copying")
endcases
```

This code is used in section 231.

233. The command codes. Before we can go any further, we need to define symbolic names for the internal code numbers that represent the various commands obeyed by TeX. These codes are somewhat arbitrary, but not completely so. For example, the command codes for character types are fixed by the language, since a user says, e.g., '`\catcode `\$ = 3`' to make `$` a math delimiter, and the command code *math_shift* is equal to 3. Some other codes have been made adjacent so that **case** statements in the program need not consider cases that are widely spaced, or so that **case** statements can be replaced by **if** statements.

At any rate, here is the list, for future reference. First come the "catcode" commands, several of which share their numeric codes with ordinary commands when the catcode cannot emerge from TeX's scanning routine.

define *escape* = 0 { escape delimiter (called \ in *The TeXbook*) }
define *relax* = 0 { do nothing (`\relax`) }
define *left_brace* = 1 { beginning of a group (`{`) }
define *right_brace* = 2 { ending of a group (`}`) }
define *math_shift* = 3 { mathematics shift character (`$`) }
define *tab_mark* = 4 { alignment delimiter (`&`, `\span`) }
define *car_ret* = 5 { end of line (*carriage_return*, `\cr`, `\crcr`) }
define *out_param* = 5 { output a macro parameter }
define *mac_param* = 6 { macro parameter symbol (`#`) }
define *sup_mark* = 7 { superscript (`^`) }
define *sub_mark* = 8 { subscript (`_`) }
define *ignore* = 9 { characters to ignore (`^^@`) }
define *endv* = 9 { end of $\langle v_j \rangle$ list in alignment template }
define *spacer* = 10 { characters equivalent to blank space (␣) }
define *letter* = 11 { characters regarded as letters (`A..Z`, `a..z`) }
define *other_char* = 12 { none of the special character types }
define *active_char* = 13 { characters that invoke macros (`~`) }
define *par_end* = 13 { end of paragraph (`\par`) }
define *match* = 13 { match a macro parameter }
define *comment* = 14 { characters that introduce comments (`%`) }
define *end_match* = 14 { end of parameters to macro }
define *stop* = 14 { end of job (`\end`, `\dump`) }
define *invalid_char* = 15 { characters that shouldn't appear (`^^?`) }
define *delim_num* = 15 { specify delimiter numerically (`\delimiter`) }
define *max_char_code* = 15 { largest catcode for individual characters }

234. Next are the ordinary run-of-the-mill command codes. Codes that are *min_internal* or more represent internal quantities that might be expanded by '`\the`'.

define *char_num* = 16 { character specified numerically (`\char`) }
define *math_char_num* = 17 { explicit math code (`\mathchar`) }
define *mark* = 18 { mark definition (`\mark`) }
define *xray* = 19 { peek inside of TeX (`\show`, `\showbox`, etc.) }
define *make_box* = 20 { make a box (`\box`, `\copy`, `\hbox`, etc.) }
define *hmove* = 21 { horizontal motion (`\moveleft`, `\moveright`) }
define *vmove* = 22 { vertical motion (`\raise`, `\lower`) }
define *un_hbox* = 23 { unglue a box (`\unhbox`, `\unhcopy`) }
define *un_vbox* = 24 { unglue a box (`\unvbox`, `\unvcopy`) }
{ (or `\pagediscards`, `\splitdiscards`) }
define *remove_item* = 25 { nullify last item (`\unpenalty`, `\unkern`, `\unskip`) }
define *hskip* = 26 { horizontal glue (`\hskip`, `\hfil`, etc.) }
define *vskip* = 27 { vertical glue (`\vskip`, `\vfil`, etc.) }
define *mskip* = 28 { math glue (`\mskip`) }
define *kern* = 29 { fixed space (`\kern`) }
define *mkern* = 30 { math kern (`\mkern`) }
define *leader_ship* = 31 { use a box (`\shipout`, `\leaders`, etc.) }
define *halign* = 32 { horizontal table alignment (`\halign`) }
define *valign* = 33 { vertical table alignment (`\valign`) }
{ or text direction directives (`\beginL`, etc.) }
define *no_align* = 34 { temporary escape from alignment (`\noalign`) }
define *vrule* = 35 { vertical rule (`\vrule`) }
define *hrule* = 36 { horizontal rule (`\hrule`) }
define *insert* = 37 { vlist inserted in box (`\insert`) }
define *vadjust* = 38 { vlist inserted in enclosing paragraph (`\vadjust`) }
define *ignore_spaces* = 39 { gobble *spacer* tokens (`\ignorespaces`) }
define *after_assignment* = 40 { save till assignment is done (`\afterassignment`) }
define *after_group* = 41 { save till group is done (`\aftergroup`) }
define *break_penalty* = 42 { additional badness (`\penalty`) }
define *start_par* = 43 { begin paragraph (`\indent`, `\noindent`) }
define *ital_corr* = 44 { italic correction (`\/`) }
define *accent* = 45 { attach accent in text (`\accent`) }
define *math_accent* = 46 { attach accent in math (`\mathaccent`) }
define *discretionary* = 47 { discretionary texts (`\-`, `\discretionary`) }
define *eq_no* = 48 { equation number (`\eqno`, `\leqno`) }
define *left_right* = 49 { variable delimiter (`\left`, `\right`) }
{ (or `\middle`) }
define *math_comp* = 50 { component of formula (`\mathbin`, etc.) }
define *limit_switch* = 51 { diddle limit conventions (`\displaylimits`, etc.) }
define *above* = 52 { generalized fraction (`\above`, `\atop`, etc.) }
define *math_style* = 53 { style specification (`\displaystyle`, etc.) }
define *math_choice* = 54 { choice specification (`\mathchoice`) }
define *non_script* = 55 { conditional math glue (`\nonscript`) }
define *vcenter* = 56 { vertically center a vbox (`\vcenter`) }
define *case_shift* = 57 { force specific case (`\lowercase`, `\uppercase`) }
define *message* = 58 { send to user (`\message`, `\errmessage`) }
define *extension* = 59 { extensions to TeX (`\write`, `\special`, etc.) }
define *in_stream* = 60 { files for reading (`\openin`, `\closein`) }
define *begin_group* = 61 { begin local grouping (`\begingroup`) }
define *end_group* = 62 { end local grouping (`\endgroup`) }

define *omit* = 63 { omit alignment template (`\omit`) }
define *ex_space* = 64 { explicit space (`\␣`) }
define *no_boundary* = 65 { suppress boundary ligatures (`\noboundary`) }
define *radical* = 66 { square root and similar signs (`\radical`) }
define *end_cs_name* = 67 { end control sequence (`\endcsname`) }
define *min_internal* = 68 { the smallest code that can follow `\the` }
define *char_given* = 68 { character code defined by `\chardef` }
define *math_given* = 69 { math code defined by `\mathchardef` }
define *XeTeX_math_given* = 70 { extended math code defined by `\Umathchardef` }
define *last_item* = 71 { most recent item (`\lastpenalty`, `\lastkern`, `\lastskip`) }
define *max_non_prefixed_command* = 71 { largest command code that can't be `\global` }

235. The next codes are special; they all relate to mode-independent assignment of values to TeX's internal registers or tables. Codes that are *max_internal* or less represent internal quantities that might be expanded by '`\the`'.

define *toks_register* = 72 { token list register (`\toks`) }
define *assign_toks* = 73 { special token list (`\output`, `\everypar`, etc.) }
define *assign_int* = 74 { user-defined integer (`\tolerance`, `\day`, etc.) }
define *assign_dimen* = 75 { user-defined length (`\hsize`, etc.) }
define *assign_glue* = 76 { user-defined glue (`\baselineskip`, etc.) }
define *assign_mu_glue* = 77 { user-defined muglue (`\thinmuskip`, etc.) }
define *assign_font_dimen* = 78 { user-defined font dimension (`\fontdimen`) }
define *assign_font_int* = 79 { user-defined font integer (`\hyphenchar`, `\skewchar`) }
define *set_aux* = 80 { specify state info (`\spacefactor`, `\prevdepth`) }
define *set_prev_graf* = 81 { specify state info (`\prevgraf`) }
define *set_page_dimen* = 82 { specify state info (`\pagegoal`, etc.) }
define *set_page_int* = 83 { specify state info (`\deadcycles`, `\insertpenalties`) }
{ (or `\interactionmode`) }
define *set_box_dimen* = 84 { change dimension of box (`\wd`, `\ht`, `\dp`) }
define *set_shape* = 85 { specify fancy paragraph shape (`\parshape`) }
{ (or `\interlinepenalties`, etc.) }
define *def_code* = 86 { define a character code (`\catcode`, etc.) }
define *XeTeX_def_code* = 87 { `\Umathcode`, `\Udelcode` }
define *def_family* = 88 { declare math fonts (`\textfont`, etc.) }
define *set_font* = 89 { set current font (font identifiers) }
define *def_font* = 90 { define a font file (`\font`) }
define *register* = 91 { internal register (`\count`, `\dimen`, etc.) }
define *max_internal* = 91 { the largest code that can follow `\the` }
define *advance* = 92 { advance a register or parameter (`\advance`) }
define *multiply* = 93 { multiply a register or parameter (`\multiply`) }
define *divide* = 94 { divide a register or parameter (`\divide`) }
define *prefix* = 95 { qualify a definition (`\global`, `\long`, `\outer`) }
{ (or `\protected`) }
define *let* = 96 { assign a command code (`\let`, `\futurelet`) }
define *shorthand_def* = 97 { code definition (`\chardef`, `\countdef`, etc.) }
define *read_to_cs* = 98 { read into a control sequence (`\read`) }
{ (or `\readline`) }
define *def* = 99 { macro definition (`\def`, `\gdef`, `\xdef`, `\edef`) }
define *set_box* = 100 { set a box (`\setbox`) }
define *hyph_data* = 101 { hyphenation data (`\hyphenation`, `\patterns`) }
define *set_interaction* = 102 { define level of interaction (`\batchmode`, etc.) }
define *max_command* = 102 { the largest command code seen at *big_switch* }

236. The remaining command codes are extra special, since they cannot get through TeX's scanner to the main control routine. They have been given values higher than *max_command* so that their special nature is easily discernible. The "expandable" commands come first.

define *undefined_cs* = *max_command* + 1 { initial state of most *eq_type* fields }
define *expand_after* = *max_command* + 2 { special expansion (`\expandafter`) }
define *no_expand* = *max_command* + 3 { special nonexpansion (`\noexpand`) }
define *input* = *max_command* + 4 { input a source file (`\input`, `\endinput`) }
{ (or `\scantokens`) }
define *if_test* = *max_command* + 5 { conditional text (`\if`, `\ifcase`, etc.) }
define *fi_or_else* = *max_command* + 6 { delimiters for conditionals (`\else`, etc.) }
define *cs_name* = *max_command* + 7 { make a control sequence from tokens (`\csname`) }
define *convert* = *max_command* + 8 { convert to text (`\number`, `\string`, etc.) }
define *the* = *max_command* + 9 { expand an internal quantity (`\the`) }
{ (or `\unexpanded`, `\detokenize`) }
define *top_bot_mark* = *max_command* + 10 { inserted mark (`\topmark`, etc.) }
define *call* = *max_command* + 11 { non-long, non-outer control sequence }
define *long_call* = *max_command* + 12 { long, non-outer control sequence }
define *outer_call* = *max_command* + 13 { non-long, outer control sequence }
define *long_outer_call* = *max_command* + 14 { long, outer control sequence }
define *end_template* = *max_command* + 15 { end of an alignment template }
define *dont_expand* = *max_command* + 16 { the following token was marked by `\noexpand` }
define *glue_ref* = *max_command* + 17 { the equivalent points to a glue specification }
define *shape_ref* = *max_command* + 18 { the equivalent points to a parshape specification }
define *box_ref* = *max_command* + 19 { the equivalent points to a box node, or is *null* }
define *data* = *max_command* + 20 { the equivalent is simply a halfword number }

237. The semantic nest. TEX is typically in the midst of building many lists at once. For example, when a math formula is being processed, TEX is in math mode and working on an mlist; this formula has temporarily interrupted TEX from being in horizontal mode and building the hlist of a paragraph; and this paragraph has temporarily interrupted TEX from being in vertical mode and building the vlist for the next page of a document. Similarly, when a `\vbox` occurs inside of an `\hbox`, TEX is temporarily interrupted from working in restricted horizontal mode, and it enters internal vertical mode. The "semantic nest" is a stack that keeps track of what lists and modes are currently suspended.

At each level of processing we are in one of six modes:

vmode stands for vertical mode (the page builder);
hmode stands for horizontal mode (the paragraph builder);
mmode stands for displayed formula mode;
$-$*vmode* stands for internal vertical mode (e.g., in a `\vbox`);
$-$*hmode* stands for restricted horizontal mode (e.g., in an `\hbox`);
$-$*mmode* stands for math formula mode (not displayed).

The mode is temporarily set to zero while processing `\write` texts.

Numeric values are assigned to *vmode*, *hmode*, and *mmode* so that TEX's "big semantic switch" can select the appropriate thing to do by computing the value $abs(mode) + cur_cmd$, where *mode* is the current mode and *cur_cmd* is the current command code.

```
define vmode = 1  { vertical mode }
define hmode = vmode + max_command + 1  { horizontal mode }
define mmode = hmode + max_command + 1  { math mode }
procedure print_mode(m : integer);  { prints the mode represented by m }
  begin if m > 0 then
    case m div (max_command + 1) of
    0: print("vertical");
    1: print("horizontal");
    2: print("display␣math");
    end
  else if m = 0 then print("no")
    else case (−m) div (max_command + 1) of
      0: print("internal␣vertical");
      1: print("restricted␣horizontal");
      2: print("math");
      end;
  print("␣mode");
  end;
```

238. The state of affairs at any semantic level can be represented by five values:

mode is the number representing the semantic mode, as just explained.

head is a *pointer* to a list head for the list being built; *link*(*head*) therefore points to the first element of the list, or to *null* if the list is empty.

tail is a *pointer* to the final node of the list being built; thus, *tail* = *head* if and only if the list is empty.

prev_graf is the number of lines of the current paragraph that have already been put into the present vertical list.

aux is an auxiliary *memory_word* that gives further information that is needed to characterize the situation.

In vertical mode, *aux* is also known as *prev_depth*; it is the scaled value representing the depth of the previous box, for use in baseline calculations, or it is ≤ -1000pt if the next box on the vertical list is to be exempt from baseline calculations. In horizontal mode, *aux* is also known as *space_factor* and *clang*; it holds the current space factor used in spacing calculations, and the current language used for hyphenation. (The value of *clang* is undefined in restricted horizontal mode.) In math mode, *aux* is also known as *incompleat_noad*; if not *null*, it points to a record that represents the numerator of a generalized fraction for which the denominator is currently being formed in the current list.

There is also a sixth quantity, *mode_line*, which correlates the semantic nest with the user's input; *mode_line* contains the source line number at which the current level of nesting was entered. The negative of this line number is the *mode_line* at the level of the user's output routine.

A seventh quantity, *eTeX_aux*, is used by the extended features ε-TeX. In vertical modes it is known as *LR_save* and holds the LR stack when a paragraph is interrupted by a displayed formula. In display math mode it is known as *LR_box* and holds a pointer to a prototype box for the display. In math mode it is known as *delim_ptr* and points to the most recent *left_noad* or *middle_noad* of a *math_left_group*.

In horizontal mode, the *prev_graf* field is used for initial language data.

The semantic nest is an array called *nest* that holds the *mode*, *head*, *tail*, *prev_graf*, *aux*, and *mode_line* values for all semantic levels below the currently active one. Information about the currently active level is kept in the global quantities *mode*, *head*, *tail*, *prev_graf*, *aux*, and *mode_line*, which live in a Pascal record that is ready to be pushed onto *nest* if necessary.

define *ignore_depth* ≡ −65536000 { *prev_depth* value that is ignored }

⟨ Types in the outer block 18 ⟩ +≡

```
list_state_record = record mode_field: −mmode .. mmode; head_field, tail_field: pointer;
  eTeX_aux_field: pointer;
  pg_field, ml_field: integer; aux_field: memory_word;
  end;
```

239. **define** *mode* ≡ *cur_list.mode_field* { current mode }
define *head* ≡ *cur_list.head_field* { header node of current list }
define *tail* ≡ *cur_list.tail_field* { final node on current list }
define *eTeX_aux* ≡ *cur_list.eTeX_aux_field* { auxiliary data for ε-TeX }
define *LR_save* ≡ *eTeX_aux* { LR stack when a paragraph is interrupted }
define *LR_box* ≡ *eTeX_aux* { prototype box for display }
define *delim_ptr* ≡ *eTeX_aux* { most recent left or right noad of a math left group }
define *prev_graf* ≡ *cur_list.pg_field* { number of paragraph lines accumulated }
define *aux* ≡ *cur_list.aux_field* { auxiliary data about the current list }
define *prev_depth* ≡ *aux.sc* { the name of *aux* in vertical mode }
define *space_factor* ≡ *aux.hh.lh* { part of *aux* in horizontal mode }
define *clang* ≡ *aux.hh.rh* { the other part of *aux* in horizontal mode }
define *incompleat_noad* ≡ *aux.int* { the name of *aux* in math mode }
define *mode_line* ≡ *cur_list.ml_field* { source file line number at beginning of list }

⟨ Global variables 13 ⟩ +≡
nest: **array** [0 .. *nest_size*] **of** *list_state_record*;
nest_ptr: 0 .. *nest_size*; { first unused location of *nest* }
max_nest_stack: 0 .. *nest_size*; { maximum of *nest_ptr* when pushing }
cur_list: *list_state_record*; { the "top" semantic state }
shown_mode: −*mmode* .. *mmode*; { most recent mode shown by `\tracingcommands` }

240. Here is a common way to make the current list grow:

define *tail_append*(#) ≡
begin *link*(*tail*) ← #; *tail* ← *link*(*tail*);
end

241. We will see later that the vertical list at the bottom semantic level is split into two parts; the "current page" runs from *page_head* to *page_tail*, and the "contribution list" runs from *contrib_head* to *tail* of semantic level zero. The idea is that contributions are first formed in vertical mode, then "contributed" to the current page (during which time the page-breaking decisions are made). For now, we don't need to know any more details about the page-building process.

⟨ Set initial values of key variables 23 ⟩ +≡
nest_ptr ← 0; *max_nest_stack* ← 0; *mode* ← *vmode*; *head* ← *contrib_head*; *tail* ← *contrib_head*;
eTeX_aux ← *null*; *prev_depth* ← *ignore_depth*; *mode_line* ← 0; *prev_graf* ← 0; *shown_mode* ← 0;
⟨ Start a new current page 1045 ⟩;

242. When TeX's work on one level is interrupted, the state is saved by calling *push_nest*. This routine changes *head* and *tail* so that a new (empty) list is begun; it does not change *mode* or *aux*.

procedure *push_nest*; { enter a new semantic level, save the old }
begin if *nest_ptr* > *max_nest_stack* **then**
begin *max_nest_stack* ← *nest_ptr*;
if *nest_ptr* = *nest_size* **then** *overflow*("`semantic␣nest␣size`", *nest_size*);
end;
nest[*nest_ptr*] ← *cur_list*; { stack the record }
incr(*nest_ptr*); *head* ← *get_avail*; *tail* ← *head*; *prev_graf* ← 0; *mode_line* ← *line*; *eTeX_aux* ← *null*;
end;

243. Conversely, when TEX is finished on the current level, the former state is restored by calling *pop_nest*. This routine will never be called at the lowest semantic level, nor will it be called unless *head* is a node that should be returned to free memory.

```
procedure pop_nest; { leave a semantic level, re-enter the old }
  begin free_avail(head); decr(nest_ptr); cur_list ← nest[nest_ptr];
  end;
```

244. Here is a procedure that displays what TEX is working on, at all levels.

```
procedure print_totals; forward;
procedure show_activities;
  var p: 0 .. nest_size; { index into nest }
    m: −mmode .. mmode; { mode }
    a: memory_word; { auxiliary }
    q, r: pointer; { for showing the current page }
    t: integer; { ditto }
  begin nest[nest_ptr] ← cur_list; { put the top level into the array }
  print_nl(""); print_ln;
  for p ← nest_ptr downto 0 do
    begin m ← nest[p].mode_field; a ← nest[p].aux_field; print_nl("###␣"); print_mode(m);
    print("␣entered␣at␣line␣"); print_int(abs(nest[p].ml_field));
    if m = hmode then
      if nest[p].pg_field ≠ ´40600000 then
        begin print("␣(language"); print_int(nest[p].pg_field mod ´200000); print(":hyphenmin");
        print_int(nest[p].pg_field div ´20000000); print_char(",");
        print_int((nest[p].pg_field div ´200000) mod ´100); print_char(")");
        end;
    if nest[p].ml_field < 0 then print("␣(\output␣routine)");
    if p = 0 then
      begin ⟨ Show the status of the current page 1040 ⟩;
      if link(contrib_head) ≠ null then print_nl("###␣recent␣contributions:");
      end;
    show_box(link(nest[p].head_field)); ⟨ Show the auxiliary field, a 245 ⟩;
    end;
  end;
```

245. ⟨ Show the auxiliary field, a 245 ⟩ ≡

```
case abs(m) div (max_command + 1) of
0: begin print_nl("prevdepth␣");
  if a.sc ≤ ignore_depth then print("ignored")
  else print_scaled(a.sc);
  if nest[p].pg_field ≠ 0 then
    begin print(",␣prevgraf␣"); print_int(nest[p].pg_field); print("␣line");
    if nest[p].pg_field ≠ 1 then print_char("s");
    end;
  end;
1: begin print_nl("spacefactor␣"); print_int(a.hh.lh);
  if m > 0 then if a.hh.rh > 0 then
      begin print(",␣current␣language␣"); print_int(a.hh.rh); end;
  end;
2: if a.int ≠ null then
    begin print("this␣will␣begin␣denominator␣of:"); show_box(a.int); end;
end { there are no other cases }
```

This code is used in section 244.

246. The table of equivalents. Now that we have studied the data structures for TeX's semantic routines, we ought to consider the data structures used by its syntactic routines. In other words, our next concern will be the tables that TeX looks at when it is scanning what the user has written.

The biggest and most important such table is called *eqtb*. It holds the current "equivalents" of things; i.e., it explains what things mean or what their current values are, for all quantities that are subject to the nesting structure provided by TeX's grouping mechanism. There are six parts to *eqtb*:

1) *eqtb*[*active_base* .. (*hash_base* − 1)] holds the current equivalents of single-character control sequences.
2) *eqtb*[*hash_base* .. (*glue_base* − 1)] holds the current equivalents of multiletter control sequences.
3) *eqtb*[*glue_base* .. (*local_base* − 1)] holds the current equivalents of glue parameters like the current baselineskip.
4) *eqtb*[*local_base* .. (*int_base* − 1)] holds the current equivalents of local halfword quantities like the current box registers, the current "catcodes," the current font, and a pointer to the current paragraph shape.
5) *eqtb*[*int_base* .. (*dimen_base* − 1)] holds the current equivalents of fullword integer parameters like the current hyphenation penalty.
6) *eqtb*[*dimen_base* .. *eqtb_size*] holds the current equivalents of fullword dimension parameters like the current hsize or amount of hanging indentation.

Note that, for example, the current amount of baselineskip glue is determined by the setting of a particular location in region 3 of *eqtb*, while the current meaning of the control sequence '`\baselineskip`' (which might have been changed by `\def` or `\let`) appears in region 2.

247. Each entry in *eqtb* is a *memory_word*. Most of these words are of type *two_halves*, and subdivided into three fields:

1) The *eq_level* (a quarterword) is the level of grouping at which this equivalent was defined. If the level is *level_zero*, the equivalent has never been defined; *level_one* refers to the outer level (outside of all groups), and this level is also used for global definitions that never go away. Higher levels are for equivalents that will disappear at the end of their group.
2) The *eq_type* (another quarterword) specifies what kind of entry this is. There are many types, since each TeX primitive like `\hbox`, `\def`, etc., has its own special code. The list of command codes above includes all possible settings of the *eq_type* field.
3) The *equiv* (a halfword) is the current equivalent value. This may be a font number, a pointer into *mem*, or a variety of other things.

define *eq_level_field*(#) ≡ #.*hh.b1*
define *eq_type_field*(#) ≡ #.*hh.b0*
define *equiv_field*(#) ≡ #.*hh.rh*
define *eq_level*(#) ≡ *eq_level_field*(*eqtb*[#]) { level of definition }
define *eq_type*(#) ≡ *eq_type_field*(*eqtb*[#]) { command code for equivalent }
define *equiv*(#) ≡ *equiv_field*(*eqtb*[#]) { equivalent value }
define *level_zero* = *min_quarterword* { level for undefined quantities }
define *level_one* = *level_zero* + 1 { outermost level for defined quantities }

248. Many locations in *eqtb* have symbolic names. The purpose of the next paragraphs is to define these names, and to set up the initial values of the equivalents.

In the first region we have *number_usvs* equivalents for "active characters" that act as control sequences, followed by *number_usvs* equivalents for single-character control sequences.

Then comes region 2, which corresponds to the hash table that we will define later. The maximum address in this region is used for a dummy control sequence that is perpetually undefined. There also are several locations for control sequences that are perpetually defined (since they are used in error recovery).

define *active_base* = 1 { beginning of region 1, for active character equivalents }
define *single_base* = *active_base* + *number_usvs* { equivalents of one-character control sequences }
define *null_cs* = *single_base* + *number_usvs* { equivalent of `\csname\endcsname` }
define *hash_base* = *null_cs* + 1 { beginning of region 2, for the hash table }
define *frozen_control_sequence* = *hash_base* + *hash_size* { for error recovery }
define *frozen_protection* = *frozen_control_sequence* { inaccessible but definable }
define *frozen_cr* = *frozen_control_sequence* + 1 { permanent '`\cr`' }
define *frozen_end_group* = *frozen_control_sequence* + 2 { permanent '`\endgroup`' }
define *frozen_right* = *frozen_control_sequence* + 3 { permanent '`\right`' }
define *frozen_fi* = *frozen_control_sequence* + 4 { permanent '`\fi`' }
define *frozen_end_template* = *frozen_control_sequence* + 5 { permanent '`\endtemplate`' }
define *frozen_endv* = *frozen_control_sequence* + 6 { second permanent '`\endtemplate`' }
define *frozen_relax* = *frozen_control_sequence* + 7 { permanent '`\relax`' }
define *end_write* = *frozen_control_sequence* + 8 { permanent '`\endwrite`' }
define *frozen_dont_expand* = *frozen_control_sequence* + 9 { permanent '`\notexpanded:`' }
define *prim_size* = 2100 { maximum number of primitives }
define *frozen_null_font* = *frozen_control_sequence* + 10 { permanent '`\nullfont`' }
define *frozen_primitive* = *frozen_control_sequence* + 11 { permanent '`\pdfprimitive`' }
define *prim_eqtb_base* = *frozen_primitive* + 1
define *font_id_base* = *frozen_null_font* − *font_base* { begins table of 257 permanent font identifiers }
define *undefined_control_sequence* = *frozen_null_font* + 257 { dummy location }
define *glue_base* = *undefined_control_sequence* + 1 { beginning of region 3 }

⟨ Initialize table entries (done by `INITEX` only) 189 ⟩ +≡
eq_type(*undefined_control_sequence*) ← *undefined_cs*; *equiv*(*undefined_control_sequence*) ← *null*;
eq_level(*undefined_control_sequence*) ← *level_zero*;
for *k* ← *active_base* **to** *undefined_control_sequence* − 1 **do** *eqtb*[*k*] ← *eqtb*[*undefined_control_sequence*];

249. Here is a routine that displays the current meaning of an *eqtb* entry in region 1 or 2. (Similar routines for the other regions will appear below.)

⟨ Show equivalent *n*, in region 1 or 2 249 ⟩ ≡
begin *sprint_cs*(*n*); *print_char*("="); *print_cmd_chr*(*eq_type*(*n*), *equiv*(*n*));
if *eq_type*(*n*) ≥ *call* **then**
begin *print_char*(":"); *show_token_list*(*link*(*equiv*(*n*)), *null*, 32);
end;
end

This code is used in section 278.

250. Region 3 of *eqtb* contains the *number_regs* `\skip` registers, as well as the glue parameters defined here. It is important that the "muskip" parameters have larger numbers than the others.

define *line_skip_code* = 0 { interline glue if *baseline_skip* is infeasible }
define *baseline_skip_code* = 1 { desired glue between baselines }
define *par_skip_code* = 2 { extra glue just above a paragraph }
define *above_display_skip_code* = 3 { extra glue just above displayed math }
define *below_display_skip_code* = 4 { extra glue just below displayed math }
define *above_display_short_skip_code* = 5 { glue above displayed math following short lines }
define *below_display_short_skip_code* = 6 { glue below displayed math following short lines }
define *left_skip_code* = 7 { glue at left of justified lines }
define *right_skip_code* = 8 { glue at right of justified lines }
define *top_skip_code* = 9 { glue at top of main pages }
define *split_top_skip_code* = 10 { glue at top of split pages }
define *tab_skip_code* = 11 { glue between aligned entries }
define *space_skip_code* = 12 { glue between words (if not *zero_glue*) }
define *xspace_skip_code* = 13 { glue after sentences (if not *zero_glue*) }
define *par_fill_skip_code* = 14 { glue on last line of paragraph }
define *XeTeX_linebreak_skip_code* = 15 { glue introduced at potential linebreak location }
define *thin_mu_skip_code* = 16 { thin space in math formula }
define *med_mu_skip_code* = 17 { medium space in math formula }
define *thick_mu_skip_code* = 18 { thick space in math formula }
define *glue_pars* = 19 { total number of glue parameters }
define *skip_base* = *glue_base* + *glue_pars* { table of *number_regs* "skip" registers }
define *mu_skip_base* = *skip_base* + *number_regs* { table of *number_regs* "muskip" registers }
define *local_base* = *mu_skip_base* + *number_regs* { beginning of region 4 }

define *skip*(#) ≡ *equiv*(*skip_base* + #) { *mem* location of glue specification }
define *mu_skip*(#) ≡ *equiv*(*mu_skip_base* + #) { *mem* location of math glue spec }
define *glue_par*(#) ≡ *equiv*(*glue_base* + #) { *mem* location of glue specification }
define *line_skip* ≡ *glue_par*(*line_skip_code*)
define *baseline_skip* ≡ *glue_par*(*baseline_skip_code*)
define *par_skip* ≡ *glue_par*(*par_skip_code*)
define *above_display_skip* ≡ *glue_par*(*above_display_skip_code*)
define *below_display_skip* ≡ *glue_par*(*below_display_skip_code*)
define *above_display_short_skip* ≡ *glue_par*(*above_display_short_skip_code*)
define *below_display_short_skip* ≡ *glue_par*(*below_display_short_skip_code*)
define *left_skip* ≡ *glue_par*(*left_skip_code*)
define *right_skip* ≡ *glue_par*(*right_skip_code*)
define *top_skip* ≡ *glue_par*(*top_skip_code*)
define *split_top_skip* ≡ *glue_par*(*split_top_skip_code*)
define *tab_skip* ≡ *glue_par*(*tab_skip_code*)
define *space_skip* ≡ *glue_par*(*space_skip_code*)
define *xspace_skip* ≡ *glue_par*(*xspace_skip_code*)
define *par_fill_skip* ≡ *glue_par*(*par_fill_skip_code*)
define *XeTeX_linebreak_skip* ≡ *glue_par*(*XeTeX_linebreak_skip_code*)
define *thin_mu_skip* ≡ *glue_par*(*thin_mu_skip_code*)
define *med_mu_skip* ≡ *glue_par*(*med_mu_skip_code*)
define *thick_mu_skip* ≡ *glue_par*(*thick_mu_skip_code*)

⟨ Current *mem* equivalent of glue parameter number *n* 250 ⟩ ≡
glue_par(*n*)

This code is used in sections 176 and 178.

251. Sometimes we need to convert TeX's internal code numbers into symbolic form. The *print_skip_param* routine gives the symbolic name of a glue parameter.

```
⟨ Declare the procedure called print_skip_param 251 ⟩ ≡
procedure print_skip_param(n : integer);
  begin case n of
  line_skip_code: print_esc("lineskip");
  baseline_skip_code: print_esc("baselineskip");
  par_skip_code: print_esc("parskip");
  above_display_skip_code: print_esc("abovedisplayskip");
  below_display_skip_code: print_esc("belowdisplayskip");
  above_display_short_skip_code: print_esc("abovedisplayshortskip");
  below_display_short_skip_code: print_esc("belowdisplayshortskip");
  left_skip_code: print_esc("leftskip");
  right_skip_code: print_esc("rightskip");
  top_skip_code: print_esc("topskip");
  split_top_skip_code: print_esc("splittopskip");
  tab_skip_code: print_esc("tabskip");
  space_skip_code: print_esc("spaceskip");
  xspace_skip_code: print_esc("xspaceskip");
  par_fill_skip_code: print_esc("parfillskip");
  XeTeX_linebreak_skip_code: print_esc("XeTeXlinebreakskip");
  thin_mu_skip_code: print_esc("thinmuskip");
  med_mu_skip_code: print_esc("medmuskip");
  thick_mu_skip_code: print_esc("thickmuskip");
  othercases print("[unknown␣glue␣parameter!]")
  endcases;
  end;
```

This code is used in section 205.

252. The symbolic names for glue parameters are put into TeX's hash table by using the routine called *primitive*, defined below. Let us enter them now, so that we don't have to list all those parameter names anywhere else.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ ≡
```
primitive("lineskip", assign_glue, glue_base + line_skip_code);
primitive("baselineskip", assign_glue, glue_base + baseline_skip_code);
primitive("parskip", assign_glue, glue_base + par_skip_code);
primitive("abovedisplayskip", assign_glue, glue_base + above_display_skip_code);
primitive("belowdisplayskip", assign_glue, glue_base + below_display_skip_code);
primitive("abovedisplayshortskip", assign_glue, glue_base + above_display_short_skip_code);
primitive("belowdisplayshortskip", assign_glue, glue_base + below_display_short_skip_code);
primitive("leftskip", assign_glue, glue_base + left_skip_code);
primitive("rightskip", assign_glue, glue_base + right_skip_code);
primitive("topskip", assign_glue, glue_base + top_skip_code);
primitive("splittopskip", assign_glue, glue_base + split_top_skip_code);
primitive("tabskip", assign_glue, glue_base + tab_skip_code);
primitive("spaceskip", assign_glue, glue_base + space_skip_code);
primitive("xspaceskip", assign_glue, glue_base + xspace_skip_code);
primitive("parfillskip", assign_glue, glue_base + par_fill_skip_code);
primitive("XeTeXlinebreakskip", assign_glue, glue_base + XeTeX_linebreak_skip_code);
primitive("thinmuskip", assign_mu_glue, glue_base + thin_mu_skip_code);
primitive("medmuskip", assign_mu_glue, glue_base + med_mu_skip_code);
primitive("thickmuskip", assign_mu_glue, glue_base + thick_mu_skip_code);
```
See also sections 256, 264, 274, 295, 364, 410, 418, 445, 450, 503, 522, 526, 588, 828, 1037, 1106, 1112, 1125, 1142, 1161, 1168, 1195, 1210, 1223, 1232, 1242, 1262, 1273, 1276, 1284, 1304, 1308, 1316, 1326, 1331, 1340, 1345, and 1398.

This code is used in section 1390.

253. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ ≡
```
assign_glue, assign_mu_glue: if chr_code < skip_base then print_skip_param(chr_code − glue_base)
  else if chr_code < mu_skip_base then
      begin print_esc("skip"); print_int(chr_code − skip_base);
      end
    else begin print_esc("muskip"); print_int(chr_code − mu_skip_base);
      end;
```
See also sections 257, 265, 275, 296, 365, 411, 419, 446, 451, 504, 523, 527, 829, 1038, 1107, 1113, 1126, 1143, 1162, 1169, 1197, 1211, 1224, 1233, 1243, 1263, 1274, 1277, 1285, 1305, 1309, 1315, 1317, 1327, 1332, 1341, 1346, 1349, and 1401.

This code is used in section 328.

254. All glue parameters and registers are initially '`0pt plus0pt minus0pt`'.

⟨ Initialize table entries (done by `INITEX` only) 189 ⟩ +≡
```
equiv(glue_base) ← zero_glue; eq_level(glue_base) ← level_one; eq_type(glue_base) ← glue_ref;
for k ← glue_base + 1 to local_base − 1 do eqtb[k] ← eqtb[glue_base];
glue_ref_count(zero_glue) ← glue_ref_count(zero_glue) + local_base − glue_base;
```

255. ⟨ Show equivalent n, in region 3 255 ⟩ ≡

```
if n < skip_base then
  begin print_skip_param(n − glue_base); print_char("=");
  if n < glue_base + thin_mu_skip_code then print_spec(equiv(n), "pt")
  else print_spec(equiv(n), "mu");
  end
else if n < mu_skip_base then
    begin print_esc("skip"); print_int(n − skip_base); print_char("="); print_spec(equiv(n), "pt");
    end
  else begin print_esc("muskip"); print_int(n − mu_skip_base); print_char("=");
    print_spec(equiv(n), "mu");
    end
```

This code is used in section 278.

256. Region 4 of *eqtb* contains the local quantities defined here. The bulk of this region is taken up by five tables that are indexed by eight-bit characters; these tables are important to both the syntactic and semantic portions of TeX. There are also a bunch of special things like font and token parameters, as well as the tables of `\toks` and `\box` registers.

define *par_shape_loc* = *local_base* { specifies paragraph shape }
define *output_routine_loc* = *local_base* + 1 { points to token list for `\output` }
define *every_par_loc* = *local_base* + 2 { points to token list for `\everypar` }
define *every_math_loc* = *local_base* + 3 { points to token list for `\everymath` }
define *every_display_loc* = *local_base* + 4 { points to token list for `\everydisplay` }
define *every_hbox_loc* = *local_base* + 5 { points to token list for `\everyhbox` }
define *every_vbox_loc* = *local_base* + 6 { points to token list for `\everyvbox` }
define *every_job_loc* = *local_base* + 7 { points to token list for `\everyjob` }
define *every_cr_loc* = *local_base* + 8 { points to token list for `\everycr` }
define *err_help_loc* = *local_base* + 9 { points to token list for `\errhelp` }
define *tex_toks* = *local_base* + 10 { end of TeX's token list parameters }

define *etex_toks_base* = *tex_toks* { base for ε-TeX's token list parameters }
define *every_eof_loc* = *etex_toks_base* { points to token list for `\everyeof` }
define *XeTeX_inter_char_loc* = *every_eof_loc* + 1 { not really used, but serves as a flag }
define *etex_toks* = *XeTeX_inter_char_loc* + 1 { end of ε-TeX's token list parameters }

define *toks_base* = *etex_toks* { table of *number_regs* token list registers }

define *etex_pen_base* = *toks_base* + *number_regs* { start of table of ε-TeX's penalties }
define *inter_line_penalties_loc* = *etex_pen_base* { additional penalties between lines }
define *club_penalties_loc* = *etex_pen_base* + 1 { penalties for creating club lines }
define *widow_penalties_loc* = *etex_pen_base* + 2 { penalties for creating widow lines }
define *display_widow_penalties_loc* = *etex_pen_base* + 3 { ditto, just before a display }
define *etex_pens* = *etex_pen_base* + 4 { end of table of ε-TeX's penalties }

define *box_base* = *etex_pens* { table of *number_regs* box registers }
define *cur_font_loc* = *box_base* + *number_regs* { internal font number outside math mode }
define *math_font_base* = *cur_font_loc* + 1 { table of *number_math_fonts* math font numbers }
define *cat_code_base* = *math_font_base* + *number_math_fonts*
{ table of *number_usvs* command codes (the "catcodes") }
define *lc_code_base* = *cat_code_base* + *number_usvs* { table of *number_usvs* lowercase mappings }
define *uc_code_base* = *lc_code_base* + *number_usvs* { table of *number_usvs* uppercase mappings }
define *sf_code_base* = *uc_code_base* + *number_usvs* { table of *number_usvs* spacefactor mappings }
define *math_code_base* = *sf_code_base* + *number_usvs* { table of *number_usvs* math mode mappings }
define *int_base* = *math_code_base* + *number_usvs* { beginning of region 5 }

define *par_shape_ptr* ≡ *equiv*(*par_shape_loc*)
define *output_routine* ≡ *equiv*(*output_routine_loc*)
define *every_par* ≡ *equiv*(*every_par_loc*)
define *every_math* ≡ *equiv*(*every_math_loc*)
define *every_display* ≡ *equiv*(*every_display_loc*)
define *every_hbox* ≡ *equiv*(*every_hbox_loc*)
define *every_vbox* ≡ *equiv*(*every_vbox_loc*)
define *every_job* ≡ *equiv*(*every_job_loc*)
define *every_cr* ≡ *equiv*(*every_cr_loc*)
define *err_help* ≡ *equiv*(*err_help_loc*)
define *toks*(#) ≡ *equiv*(*toks_base* + #)
define *box*(#) ≡ *equiv*(*box_base* + #)
define *cur_font* ≡ *equiv*(*cur_font_loc*)
define *fam_fnt*(#) ≡ *equiv*(*math_font_base* + #)
define *cat_code*(#) ≡ *equiv*(*cat_code_base* + #)

define *lc_code*(#) ≡ *equiv*(*lc_code_base* + #)
define *uc_code*(#) ≡ *equiv*(*uc_code_base* + #)
define *sf_code*(#) ≡ *equiv*(*sf_code_base* + #)
define *math_code*(#) ≡ *equiv*(*math_code_base* + #)
{ Note: *math_code*(*c*) is the true math code plus *min_halfword* }

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("output", *assign_toks*, *output_routine_loc*); *primitive*("everypar", *assign_toks*, *every_par_loc*);
primitive("everymath", *assign_toks*, *every_math_loc*);
primitive("everydisplay", *assign_toks*, *every_display_loc*);
primitive("everyhbox", *assign_toks*, *every_hbox_loc*); *primitive*("everyvbox", *assign_toks*, *every_vbox_loc*);
primitive("everyjob", *assign_toks*, *every_job_loc*); *primitive*("everycr", *assign_toks*, *every_cr_loc*);
primitive("errhelp", *assign_toks*, *err_help_loc*);

257. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
assign_toks: if chr_code ≥ toks_base then
    begin print_esc("toks"); print_int(chr_code − toks_base);
    end
  else case chr_code of
    output_routine_loc: print_esc("output");
    every_par_loc: print_esc("everypar");
    every_math_loc: print_esc("everymath");
    every_display_loc: print_esc("everydisplay");
    every_hbox_loc: print_esc("everyhbox");
    every_vbox_loc: print_esc("everyvbox");
    every_job_loc: print_esc("everyjob");
    every_cr_loc: print_esc("everycr");
      ⟨ Cases of assign_toks for print_cmd_chr 1468 ⟩
    othercases print_esc("errhelp")
    endcases;
```

258. We initialize most things to null or undefined values. An undefined font is represented by the internal code *font_base*.

However, the character code tables are given initial values based on the conventional interpretation of ASCII code. These initial values should not be changed when TeX is adapted for use with non-English languages; all changes to the initialization conventions should be made in format packages, not in TeX itself, so that global interchange of formats is possible.

define *null_font* ≡ *font_base*
define *var_fam_class* = 7
define *active_math_char* = "1FFFFF
define *is_active_math_char*(#) ≡ *math_char_field*(#) = *active_math_char*
define *is_var_family*(#) ≡ *math_class_field*(#) = 7

⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
par_shape_ptr ← *null*; *eq_type*(*par_shape_loc*) ← *shape_ref*; *eq_level*(*par_shape_loc*) ← *level_one*;
for *k* ← *etex_pen_base* **to** *etex_pens* − 1 **do** *eqtb*[*k*] ← *eqtb*[*par_shape_loc*];
for *k* ← *output_routine_loc* **to** *toks_base* + *number_regs* − 1 **do** *eqtb*[*k*] ← *eqtb*[*undefined_control_sequence*];
box(0) ← *null*; *eq_type*(*box_base*) ← *box_ref*; *eq_level*(*box_base*) ← *level_one*;
for *k* ← *box_base* + 1 **to** *box_base* + *number_regs* − 1 **do** *eqtb*[*k*] ← *eqtb*[*box_base*];
cur_font ← *null_font*; *eq_type*(*cur_font_loc*) ← *data*; *eq_level*(*cur_font_loc*) ← *level_one*;
for *k* ← *math_font_base* **to** *math_font_base* + *number_math_fonts* − 1 **do** *eqtb*[*k*] ← *eqtb*[*cur_font_loc*];
equiv(*cat_code_base*) ← 0; *eq_type*(*cat_code_base*) ← *data*; *eq_level*(*cat_code_base*) ← *level_one*;
for *k* ← *cat_code_base* + 1 **to** *int_base* − 1 **do** *eqtb*[*k*] ← *eqtb*[*cat_code_base*];
for *k* ← 0 **to** *number_usvs* − 1 **do**
  begin *cat_code*(*k*) ← *other_char*; *math_code*(*k*) ← *hi*(*k*); *sf_code*(*k*) ← 1000;
  end;
cat_code(*carriage_return*) ← *car_ret*; *cat_code*("␣") ← *spacer*; *cat_code*("\") ← *escape*;
cat_code("%") ← *comment*; *cat_code*(*invalid_code*) ← *invalid_char*; *cat_code*(*null_code*) ← *ignore*;
for *k* ← "0" **to** "9" **do** *math_code*(*k*) ← *hi*(*k* + *set_class_field*(*var_fam_class*));
for *k* ← "A" **to** "Z" **do**
  begin *cat_code*(*k*) ← *letter*; *cat_code*(*k* + "a" − "A") ← *letter*;
  math_code(*k*) ← *hi*(*k* + *set_family_field*(1) + *set_class_field*(*var_fam_class*));
  math_code(*k* + "a" − "A") ← *hi*(*k* + "a" − "A" + *set_family_field*(1) + *set_class_field*(*var_fam_class*));
  lc_code(*k*) ← *k* + "a" − "A"; *lc_code*(*k* + "a" − "A") ← *k* + "a" − "A";
  uc_code(*k*) ← *k*; *uc_code*(*k* + "a" − "A") ← *k*;
  sf_code(*k*) ← 999;
  end;

259. ⟨ Show equivalent n, in region 4 259 ⟩ ≡

```
if (n = par_shape_loc) ∨ ((n ≥ etex_pen_base) ∧ (n < etex_pens)) then
  begin print_cmd_chr(set_shape, n); print_char("=");
  if equiv(n) = null then print_char("0")
  else if n > par_shape_loc then
      begin print_int(penalty(equiv(n))); print_char("␣"); print_int(penalty(equiv(n) + 1));
      if penalty(equiv(n)) > 1 then print_esc("ETC.");
      end
    else print_int(info(par_shape_ptr));
  end
else if n < toks_base then
    begin print_cmd_chr(assign_toks, n); print_char("=");
    if equiv(n) ≠ null then show_token_list(link(equiv(n)), null, 32);
    end
  else if n < box_base then
      begin print_esc("toks"); print_int(n − toks_base); print_char("=");
      if equiv(n) ≠ null then show_token_list(link(equiv(n)), null, 32);
      end
    else if n < cur_font_loc then
        begin print_esc("box"); print_int(n − box_base); print_char("=");
        if equiv(n) = null then print("void")
        else begin depth_threshold ← 0; breadth_max ← 1; show_node_list(equiv(n));
          end;
        end
      else if n < cat_code_base then ⟨ Show the font identifier in eqtb[n] 260 ⟩
        else ⟨ Show the halfword code in eqtb[n] 261 ⟩
```

This code is used in section 278.

260. ⟨ Show the font identifier in $eqtb[n]$ 260 ⟩ ≡

```
begin if n = cur_font_loc then print("current␣font")
else if n < math_font_base + script_size then
    begin print_esc("textfont"); print_int(n − math_font_base);
    end
  else if n < math_font_base + script_script_size then
      begin print_esc("scriptfont"); print_int(n − math_font_base − script_size);
      end
    else begin print_esc("scriptscriptfont"); print_int(n − math_font_base − script_script_size);
      end;
print_char("=");
print_esc(hash[font_id_base + equiv(n)].rh);   { that's font_id_text(equiv(n)) }
end
```

This code is used in section 259.

261. ⟨ Show the halfword code in *eqtb*[*n*] 261 ⟩ ≡

```
if n < math_code_base then
  begin if n < lc_code_base then
    begin print_esc("catcode"); print_int(n − cat_code_base);
    end
  else if n < uc_code_base then
      begin print_esc("lccode"); print_int(n − lc_code_base);
      end
    else if n < sf_code_base then
        begin print_esc("uccode"); print_int(n − uc_code_base);
        end
      else begin print_esc("sfcode"); print_int(n − sf_code_base);
        end;
  print_char("="); print_int(equiv(n));
  end
else begin print_esc("mathcode"); print_int(n − math_code_base); print_char("=");
  print_int(ho(equiv(n)));
  end
```

This code is used in section 259.

262. Region 5 of *eqtb* contains the integer parameters and registers defined here, as well as the *del_code* table. The latter table differs from the *cat_code* .. *math_code* tables that precede it, since delimiter codes are fullword integers while the other kinds of codes occupy at most a halfword. This is what makes region 5 different from region 4. We will store the *eq_level* information in an auxiliary array of quarterwords that will be defined later.

define *pretolerance_code* = 0 { badness tolerance before hyphenation }
define *tolerance_code* = 1 { badness tolerance after hyphenation }
define *line_penalty_code* = 2 { added to the badness of every line }
define *hyphen_penalty_code* = 3 { penalty for break after discretionary hyphen }
define *ex_hyphen_penalty_code* = 4 { penalty for break after explicit hyphen }
define *club_penalty_code* = 5 { penalty for creating a club line }
define *widow_penalty_code* = 6 { penalty for creating a widow line }
define *display_widow_penalty_code* = 7 { ditto, just before a display }
define *broken_penalty_code* = 8 { penalty for breaking a page at a broken line }
define *bin_op_penalty_code* = 9 { penalty for breaking after a binary operation }
define *rel_penalty_code* = 10 { penalty for breaking after a relation }
define *pre_display_penalty_code* = 11 { penalty for breaking just before a displayed formula }
define *post_display_penalty_code* = 12 { penalty for breaking just after a displayed formula }
define *inter_line_penalty_code* = 13 { additional penalty between lines }
define *double_hyphen_demerits_code* = 14 { demerits for double hyphen break }
define *final_hyphen_demerits_code* = 15 { demerits for final hyphen break }
define *adj_demerits_code* = 16 { demerits for adjacent incompatible lines }
define *mag_code* = 17 { magnification ratio }
define *delimiter_factor_code* = 18 { ratio for variable-size delimiters }
define *looseness_code* = 19 { change in number of lines for a paragraph }
define *time_code* = 20 { current time of day }
define *day_code* = 21 { current day of the month }
define *month_code* = 22 { current month of the year }
define *year_code* = 23 { current year of our Lord }
define *show_box_breadth_code* = 24 { nodes per level in *show_box* }
define *show_box_depth_code* = 25 { maximum level in *show_box* }
define *hbadness_code* = 26 { hboxes exceeding this badness will be shown by *hpack* }
define *vbadness_code* = 27 { vboxes exceeding this badness will be shown by *vpack* }
define *pausing_code* = 28 { pause after each line is read from a file }
define *tracing_online_code* = 29 { show diagnostic output on terminal }
define *tracing_macros_code* = 30 { show macros as they are being expanded }
define *tracing_stats_code* = 31 { show memory usage if TeX knows it }
define *tracing_paragraphs_code* = 32 { show line-break calculations }
define *tracing_pages_code* = 33 { show page-break calculations }
define *tracing_output_code* = 34 { show boxes when they are shipped out }
define *tracing_lost_chars_code* = 35 { show characters that aren't in the font }
define *tracing_commands_code* = 36 { show command codes at *big_switch* }
define *tracing_restores_code* = 37 { show equivalents when they are restored }
define *uc_hyph_code* = 38 { hyphenate words beginning with a capital letter }
define *output_penalty_code* = 39 { penalty found at current page break }
define *max_dead_cycles_code* = 40 { bound on consecutive dead cycles of output }
define *hang_after_code* = 41 { hanging indentation changes after this many lines }
define *floating_penalty_code* = 42 { penalty for insertions held over after a split }
define *global_defs_code* = 43 { override `\global` specifications }
define *cur_fam_code* = 44 { current family }
define *escape_char_code* = 45 { escape character for token output }
define *default_hyphen_char_code* = 46 { value of `\hyphenchar` when a font is loaded }

define *default_skew_char_code* = 47 { value of `\skewchar` when a font is loaded }
define *end_line_char_code* = 48 { character placed at the right end of the buffer }
define *new_line_char_code* = 49 { character that prints as *print_ln* }
define *language_code* = 50 { current hyphenation table }
define *left_hyphen_min_code* = 51 { minimum left hyphenation fragment size }
define *right_hyphen_min_code* = 52 { minimum right hyphenation fragment size }
define *holding_inserts_code* = 53 { do not remove insertion nodes from `\box255` }
define *error_context_lines_code* = 54 { maximum intermediate line pairs shown }
define *tex_int_pars* = 55 { total number of TeX's integer parameters }
define *etex_int_base* = *tex_int_pars* { base for ε-TeX's integer parameters }
define *tracing_assigns_code* = *etex_int_base* { show assignments }
define *tracing_groups_code* = *etex_int_base* + 1 { show save/restore groups }
define *tracing_ifs_code* = *etex_int_base* + 2 { show conditionals }
define *tracing_scan_tokens_code* = *etex_int_base* + 3 { show pseudo file open and close }
define *tracing_nesting_code* = *etex_int_base* + 4 { show incomplete groups and ifs within files }
define *pre_display_direction_code* = *etex_int_base* + 5 { text direction preceding a display }
define *last_line_fit_code* = *etex_int_base* + 6 { adjustment for last line of paragraph }
define *saving_vdiscards_code* = *etex_int_base* + 7 { save items discarded from vlists }
define *saving_hyph_codes_code* = *etex_int_base* + 8 { save hyphenation codes for languages }
define *suppress_fontnotfound_error_code* = *etex_int_base* + 9 { suppress errors for missing fonts }
define *XeTeX_linebreak_locale_code* = *etex_int_base* + 10
{ string number of locale to use for linebreak locations }
define *XeTeX_linebreak_penalty_code* = *etex_int_base* + 11
{ penalty to use at locale-dependent linebreak locations }
define *XeTeX_protrude_chars_code* = *etex_int_base* + 12
{ protrude chars at left/right edge of paragraphs }
define *eTeX_state_code* = *etex_int_base* + 13 { ε-TeX state variables }
define *etex_int_pars* = *eTeX_state_code* + *eTeX_states* { total number of ε-TeX's integer parameters }
define *int_pars* = *etex_int_pars* { total number of integer parameters }
define *count_base* = *int_base* + *int_pars* { *number_regs* user `\count` registers }
define *del_code_base* = *count_base* + *number_regs* { *number_usvs* delimiter code mappings }
define *dimen_base* = *del_code_base* + *number_usvs* { beginning of region 6 }
define *del_code*(#) ≡ *eqtb*[*del_code_base* + #].*int*
define *count*(#) ≡ *eqtb*[*count_base* + #].*int*
define *int_par*(#) ≡ *eqtb*[*int_base* + #].*int* { an integer parameter }
define *pretolerance* ≡ *int_par*(*pretolerance_code*)
define *tolerance* ≡ *int_par*(*tolerance_code*)
define *line_penalty* ≡ *int_par*(*line_penalty_code*)
define *hyphen_penalty* ≡ *int_par*(*hyphen_penalty_code*)
define *ex_hyphen_penalty* ≡ *int_par*(*ex_hyphen_penalty_code*)
define *club_penalty* ≡ *int_par*(*club_penalty_code*)
define *widow_penalty* ≡ *int_par*(*widow_penalty_code*)
define *display_widow_penalty* ≡ *int_par*(*display_widow_penalty_code*)
define *broken_penalty* ≡ *int_par*(*broken_penalty_code*)
define *bin_op_penalty* ≡ *int_par*(*bin_op_penalty_code*)
define *rel_penalty* ≡ *int_par*(*rel_penalty_code*)
define *pre_display_penalty* ≡ *int_par*(*pre_display_penalty_code*)
define *post_display_penalty* ≡ *int_par*(*post_display_penalty_code*)
define *inter_line_penalty* ≡ *int_par*(*inter_line_penalty_code*)
define *double_hyphen_demerits* ≡ *int_par*(*double_hyphen_demerits_code*)
define *final_hyphen_demerits* ≡ *int_par*(*final_hyphen_demerits_code*)
define *adj_demerits* ≡ *int_par*(*adj_demerits_code*)

define *mag* ≡ *int_par*(*mag_code*)
define *delimiter_factor* ≡ *int_par*(*delimiter_factor_code*)
define *looseness* ≡ *int_par*(*looseness_code*)
define *time* ≡ *int_par*(*time_code*)
define *day* ≡ *int_par*(*day_code*)
define *month* ≡ *int_par*(*month_code*)
define *year* ≡ *int_par*(*year_code*)
define *show_box_breadth* ≡ *int_par*(*show_box_breadth_code*)
define *show_box_depth* ≡ *int_par*(*show_box_depth_code*)
define *hbadness* ≡ *int_par*(*hbadness_code*)
define *vbadness* ≡ *int_par*(*vbadness_code*)
define *pausing* ≡ *int_par*(*pausing_code*)
define *tracing_online* ≡ *int_par*(*tracing_online_code*)
define *tracing_macros* ≡ *int_par*(*tracing_macros_code*)
define *tracing_stats* ≡ *int_par*(*tracing_stats_code*)
define *tracing_paragraphs* ≡ *int_par*(*tracing_paragraphs_code*)
define *tracing_pages* ≡ *int_par*(*tracing_pages_code*)
define *tracing_output* ≡ *int_par*(*tracing_output_code*)
define *tracing_lost_chars* ≡ *int_par*(*tracing_lost_chars_code*)
define *tracing_commands* ≡ *int_par*(*tracing_commands_code*)
define *tracing_restores* ≡ *int_par*(*tracing_restores_code*)
define *uc_hyph* ≡ *int_par*(*uc_hyph_code*)
define *output_penalty* ≡ *int_par*(*output_penalty_code*)
define *max_dead_cycles* ≡ *int_par*(*max_dead_cycles_code*)
define *hang_after* ≡ *int_par*(*hang_after_code*)
define *floating_penalty* ≡ *int_par*(*floating_penalty_code*)
define *global_defs* ≡ *int_par*(*global_defs_code*)
define *cur_fam* ≡ *int_par*(*cur_fam_code*)
define *escape_char* ≡ *int_par*(*escape_char_code*)
define *default_hyphen_char* ≡ *int_par*(*default_hyphen_char_code*)
define *default_skew_char* ≡ *int_par*(*default_skew_char_code*)
define *end_line_char* ≡ *int_par*(*end_line_char_code*)
define *new_line_char* ≡ *int_par*(*new_line_char_code*)
define *language* ≡ *int_par*(*language_code*)
define *left_hyphen_min* ≡ *int_par*(*left_hyphen_min_code*)
define *right_hyphen_min* ≡ *int_par*(*right_hyphen_min_code*)
define *holding_inserts* ≡ *int_par*(*holding_inserts_code*)
define *error_context_lines* ≡ *int_par*(*error_context_lines_code*)
define *tracing_assigns* ≡ *int_par*(*tracing_assigns_code*)
define *tracing_groups* ≡ *int_par*(*tracing_groups_code*)
define *tracing_ifs* ≡ *int_par*(*tracing_ifs_code*)
define *tracing_scan_tokens* ≡ *int_par*(*tracing_scan_tokens_code*)
define *tracing_nesting* ≡ *int_par*(*tracing_nesting_code*)
define *pre_display_direction* ≡ *int_par*(*pre_display_direction_code*)
define *last_line_fit* ≡ *int_par*(*last_line_fit_code*)
define *saving_vdiscards* ≡ *int_par*(*saving_vdiscards_code*)
define *saving_hyph_codes* ≡ *int_par*(*saving_hyph_codes_code*)
define *suppress_fontnotfound_error* ≡ *int_par*(*suppress_fontnotfound_error_code*)
define *XeTeX_linebreak_locale* ≡ *int_par*(*XeTeX_linebreak_locale_code*)
define *XeTeX_linebreak_penalty* ≡ *int_par*(*XeTeX_linebreak_penalty_code*)
define *XeTeX_protrude_chars* ≡ *int_par*(*XeTeX_protrude_chars_code*)

⟨ Assign the values *depth_threshold* ← *show_box_depth* and *breadth_max* ← *show_box_breadth* 262 ⟩ ≡

depth_threshold ← *show_box_depth*; *breadth_max* ← *show_box_breadth*

This code is used in section 224.

263. We can print the symbolic name of an integer parameter as follows.

```
procedure print_param(n : integer);
  begin case n of
  pretolerance_code: print_esc("pretolerance");
  tolerance_code: print_esc("tolerance");
  line_penalty_code: print_esc("linepenalty");
  hyphen_penalty_code: print_esc("hyphenpenalty");
  ex_hyphen_penalty_code: print_esc("exhyphenpenalty");
  club_penalty_code: print_esc("clubpenalty");
  widow_penalty_code: print_esc("widowpenalty");
  display_widow_penalty_code: print_esc("displaywidowpenalty");
  broken_penalty_code: print_esc("brokenpenalty");
  bin_op_penalty_code: print_esc("binoppenalty");
  rel_penalty_code: print_esc("relpenalty");
  pre_display_penalty_code: print_esc("predisplaypenalty");
  post_display_penalty_code: print_esc("postdisplaypenalty");
  inter_line_penalty_code: print_esc("interlinepenalty");
  double_hyphen_demerits_code: print_esc("doublehyphendemerits");
  final_hyphen_demerits_code: print_esc("finalhyphendemerits");
  adj_demerits_code: print_esc("adjdemerits");
  mag_code: print_esc("mag");
  delimiter_factor_code: print_esc("delimiterfactor");
  looseness_code: print_esc("looseness");
  time_code: print_esc("time");
  day_code: print_esc("day");
  month_code: print_esc("month");
  year_code: print_esc("year");
  show_box_breadth_code: print_esc("showboxbreadth");
  show_box_depth_code: print_esc("showboxdepth");
  hbadness_code: print_esc("hbadness");
  vbadness_code: print_esc("vbadness");
  pausing_code: print_esc("pausing");
  tracing_online_code: print_esc("tracingonline");
  tracing_macros_code: print_esc("tracingmacros");
  tracing_stats_code: print_esc("tracingstats");
  tracing_paragraphs_code: print_esc("tracingparagraphs");
  tracing_pages_code: print_esc("tracingpages");
  tracing_output_code: print_esc("tracingoutput");
  tracing_lost_chars_code: print_esc("tracinglostchars");
  tracing_commands_code: print_esc("tracingcommands");
  tracing_restores_code: print_esc("tracingrestores");
  uc_hyph_code: print_esc("uchyph");
  output_penalty_code: print_esc("outputpenalty");
  max_dead_cycles_code: print_esc("maxdeadcycles");
  hang_after_code: print_esc("hangafter");
  floating_penalty_code: print_esc("floatingpenalty");
  global_defs_code: print_esc("globaldefs");
  cur_fam_code: print_esc("fam");
  escape_char_code: print_esc("escapechar");
  default_hyphen_char_code: print_esc("defaulthyphenchar");
  default_skew_char_code: print_esc("defaultskewchar");
  end_line_char_code: print_esc("endlinechar");
```

new_line_char_code: *print_esc*("newlinechar");
language_code: *print_esc*("language");
left_hyphen_min_code: *print_esc*("lefthyphenmin");
right_hyphen_min_code: *print_esc*("righthyphenmin");
holding_inserts_code: *print_esc*("holdinginserts");
error_context_lines_code: *print_esc*("errorcontextlines");
XeTeX_linebreak_penalty_code: *print_esc*("XeTeXlinebreakpenalty");
XeTeX_protrude_chars_code: *print_esc*("XeTeXprotrudechars");
⟨ Cases for *print_param* 1469 ⟩
othercases *print*("[unknown␣integer␣parameter!]")
endcases;
end;

264. The integer parameter names must be entered into the hash table.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("pretolerance", *assign_int*, *int_base* + *pretolerance_code*);
primitive("tolerance", *assign_int*, *int_base* + *tolerance_code*);
primitive("linepenalty", *assign_int*, *int_base* + *line_penalty_code*);
primitive("hyphenpenalty", *assign_int*, *int_base* + *hyphen_penalty_code*);
primitive("exhyphenpenalty", *assign_int*, *int_base* + *ex_hyphen_penalty_code*);
primitive("clubpenalty", *assign_int*, *int_base* + *club_penalty_code*);
primitive("widowpenalty", *assign_int*, *int_base* + *widow_penalty_code*);
primitive("displaywidowpenalty", *assign_int*, *int_base* + *display_widow_penalty_code*);
primitive("brokenpenalty", *assign_int*, *int_base* + *broken_penalty_code*);
primitive("binoppenalty", *assign_int*, *int_base* + *bin_op_penalty_code*);
primitive("relpenalty", *assign_int*, *int_base* + *rel_penalty_code*);
primitive("predisplaypenalty", *assign_int*, *int_base* + *pre_display_penalty_code*);
primitive("postdisplaypenalty", *assign_int*, *int_base* + *post_display_penalty_code*);
primitive("interlinepenalty", *assign_int*, *int_base* + *inter_line_penalty_code*);
primitive("doublehyphendemerits", *assign_int*, *int_base* + *double_hyphen_demerits_code*);
primitive("finalhyphendemerits", *assign_int*, *int_base* + *final_hyphen_demerits_code*);
primitive("adjdemerits", *assign_int*, *int_base* + *adj_demerits_code*);
primitive("mag", *assign_int*, *int_base* + *mag_code*);
primitive("delimiterfactor", *assign_int*, *int_base* + *delimiter_factor_code*);
primitive("looseness", *assign_int*, *int_base* + *looseness_code*);
primitive("time", *assign_int*, *int_base* + *time_code*);
primitive("day", *assign_int*, *int_base* + *day_code*);
primitive("month", *assign_int*, *int_base* + *month_code*);
primitive("year", *assign_int*, *int_base* + *year_code*);
primitive("showboxbreadth", *assign_int*, *int_base* + *show_box_breadth_code*);
primitive("showboxdepth", *assign_int*, *int_base* + *show_box_depth_code*);
primitive("hbadness", *assign_int*, *int_base* + *hbadness_code*);
primitive("vbadness", *assign_int*, *int_base* + *vbadness_code*);
primitive("pausing", *assign_int*, *int_base* + *pausing_code*);
primitive("tracingonline", *assign_int*, *int_base* + *tracing_online_code*);
primitive("tracingmacros", *assign_int*, *int_base* + *tracing_macros_code*);
primitive("tracingstats", *assign_int*, *int_base* + *tracing_stats_code*);
primitive("tracingparagraphs", *assign_int*, *int_base* + *tracing_paragraphs_code*);
primitive("tracingpages", *assign_int*, *int_base* + *tracing_pages_code*);
primitive("tracingoutput", *assign_int*, *int_base* + *tracing_output_code*);
primitive("tracinglostchars", *assign_int*, *int_base* + *tracing_lost_chars_code*);
primitive("tracingcommands", *assign_int*, *int_base* + *tracing_commands_code*);
primitive("tracingrestores", *assign_int*, *int_base* + *tracing_restores_code*);
primitive("uchyph", *assign_int*, *int_base* + *uc_hyph_code*);
primitive("outputpenalty", *assign_int*, *int_base* + *output_penalty_code*);
primitive("maxdeadcycles", *assign_int*, *int_base* + *max_dead_cycles_code*);
primitive("hangafter", *assign_int*, *int_base* + *hang_after_code*);
primitive("floatingpenalty", *assign_int*, *int_base* + *floating_penalty_code*);
primitive("globaldefs", *assign_int*, *int_base* + *global_defs_code*);
primitive("fam", *assign_int*, *int_base* + *cur_fam_code*);
primitive("escapechar", *assign_int*, *int_base* + *escape_char_code*);
primitive("defaulthyphenchar", *assign_int*, *int_base* + *default_hyphen_char_code*);
primitive("defaultskewchar", *assign_int*, *int_base* + *default_skew_char_code*);
primitive("endlinechar", *assign_int*, *int_base* + *end_line_char_code*);
primitive("newlinechar", *assign_int*, *int_base* + *new_line_char_code*);

primitive("language", *assign_int*, *int_base* + *language_code*);
primitive("lefthyphenmin", *assign_int*, *int_base* + *left_hyphen_min_code*);
primitive("righthyphenmin", *assign_int*, *int_base* + *right_hyphen_min_code*);
primitive("holdinginserts", *assign_int*, *int_base* + *holding_inserts_code*);
primitive("errorcontextlines", *assign_int*, *int_base* + *error_context_lines_code*);
primitive("XeTeXlinebreakpenalty", *assign_int*, *int_base* + *XeTeX_linebreak_penalty_code*);
primitive("XeTeXprotrudechars", *assign_int*, *int_base* + *XeTeX_protrude_chars_code*);

265. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
assign_int: **if** *chr_code* < *count_base* **then** *print_param*(*chr_code* − *int_base*)
else begin *print_esc*("count"); *print_int*(*chr_code* − *count_base*);
end;

266. The integer parameters should really be initialized by a macro package; the following initialization does the minimum to keep TeX from complete failure.

⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
for *k* ← *int_base* **to** *del_code_base* − 1 **do** *eqtb*[*k*].*int* ← 0;
mag ← 1000; *tolerance* ← 10000; *hang_after* ← 1; *max_dead_cycles* ← 25; *escape_char* ← "\";
end_line_char ← *carriage_return*;
for *k* ← 0 **to** *number_usvs* − 1 **do** *del_code*(*k*) ← −1;
del_code(".") ← 0; { this null delimiter is used in error recovery }

267. The following procedure, which is called just before TeX initializes its input and output, establishes the initial values of the date and time. Since standard Pascal cannot provide such information, something special is needed. The program here simply assumes that suitable values appear in the global variables *sys_time*, *sys_day*, *sys_month*, and *sys_year* (which are initialized to noon on 4 July 1776, in case the implementor is careless).

procedure *fix_date_and_time*;
begin *sys_time* ← 12 ∗ 60; *sys_day* ← 4; *sys_month* ← 7; *sys_year* ← 1776; { self-evident truths }
time ← *sys_time*; { minutes since midnight }
day ← *sys_day*; { day of the month }
month ← *sys_month*; { month of the year }
year ← *sys_year*; { Anno Domini }
end;

268. ⟨ Show equivalent *n*, in region 5 268 ⟩ ≡
begin if *n* < *count_base* **then** *print_param*(*n* − *int_base*)
else if *n* < *del_code_base* **then**
begin *print_esc*("count"); *print_int*(*n* − *count_base*);
end
else begin *print_esc*("delcode"); *print_int*(*n* − *del_code_base*);
end;
print_char("="); *print_int*(*eqtb*[*n*].*int*);
end

This code is used in section 278.

269. ⟨ Set variable *c* to the current escape character 269 ⟩ ≡
c ← *escape_char*

This code is used in section 67.

270. ⟨ Character s is the current new-line character 270 ⟩ ≡

```
s = new_line_char
```

This code is used in sections 59 and 63.

271. TEX is occasionally supposed to print diagnostic information that goes only into the transcript file, unless *tracing_online* is positive. Here are two routines that adjust the destination of print commands:

```
procedure begin_diagnostic; { prepare to do some tracing }
  begin old_setting ← selector;
  if (tracing_online ≤ 0) ∧ (selector = term_and_log) then
    begin decr(selector);
    if history = spotless then history ← warning_issued;
    end;
  end;
procedure end_diagnostic(blank_line : boolean); { restore proper conditions after tracing }
  begin print_nl("");
  if blank_line then print_ln;
  selector ← old_setting;
  end;
```

272. Of course we had better declare a few more global variables, if the previous routines are going to work.

```
⟨ Global variables 13 ⟩ +≡
old_setting: 0 .. max_selector;
sys_time, sys_day, sys_month, sys_year: integer; { date and time supplied by external system }
```

273. The final region of *eqtb* contains the dimension parameters defined here, and the *number_regs* `\dimen` registers.

define *par_indent_code* = 0 { indentation of paragraphs }
define *math_surround_code* = 1 { space around math in text }
define *line_skip_limit_code* = 2 { threshold for *line_skip* instead of *baseline_skip* }
define *hsize_code* = 3 { line width in horizontal mode }
define *vsize_code* = 4 { page height in vertical mode }
define *max_depth_code* = 5 { maximum depth of boxes on main pages }
define *split_max_depth_code* = 6 { maximum depth of boxes on split pages }
define *box_max_depth_code* = 7 { maximum depth of explicit vboxes }
define *hfuzz_code* = 8 { tolerance for overfull hbox messages }
define *vfuzz_code* = 9 { tolerance for overfull vbox messages }
define *delimiter_shortfall_code* = 10 { maximum amount uncovered by variable delimiters }
define *null_delimiter_space_code* = 11 { blank space in null delimiters }
define *script_space_code* = 12 { extra space after subscript or superscript }
define *pre_display_size_code* = 13 { length of text preceding a display }
define *display_width_code* = 14 { length of line for displayed equation }
define *display_indent_code* = 15 { indentation of line for displayed equation }
define *overfull_rule_code* = 16 { width of rule that identifies overfull hboxes }
define *hang_indent_code* = 17 { amount of hanging indentation }
define *h_offset_code* = 18 { amount of horizontal offset when shipping pages out }
define *v_offset_code* = 19 { amount of vertical offset when shipping pages out }
define *emergency_stretch_code* = 20 { reduces badnesses on final pass of line-breaking }
define *pdf_page_width_code* = 21 { page width of the PDF output }
define *pdf_page_height_code* = 22 { page height of the PDF output }
define *dimen_pars* = 23 { total number of dimension parameters }
define *scaled_base* = *dimen_base* + *dimen_pars* { table of *number_regs* user-defined `\dimen` registers }
define *eqtb_size* = *scaled_base* + *biggest_reg* { largest subscript of *eqtb* }

define *dimen*(#) ≡ *eqtb*[*scaled_base* + #].*sc*
define *dimen_par*(#) ≡ *eqtb*[*dimen_base* + #].*sc* { a scaled quantity }
define *par_indent* ≡ *dimen_par*(*par_indent_code*)
define *math_surround* ≡ *dimen_par*(*math_surround_code*)
define *line_skip_limit* ≡ *dimen_par*(*line_skip_limit_code*)
define *hsize* ≡ *dimen_par*(*hsize_code*)
define *vsize* ≡ *dimen_par*(*vsize_code*)
define *max_depth* ≡ *dimen_par*(*max_depth_code*)
define *split_max_depth* ≡ *dimen_par*(*split_max_depth_code*)
define *box_max_depth* ≡ *dimen_par*(*box_max_depth_code*)
define *hfuzz* ≡ *dimen_par*(*hfuzz_code*)
define *vfuzz* ≡ *dimen_par*(*vfuzz_code*)
define *delimiter_shortfall* ≡ *dimen_par*(*delimiter_shortfall_code*)
define *null_delimiter_space* ≡ *dimen_par*(*null_delimiter_space_code*)
define *script_space* ≡ *dimen_par*(*script_space_code*)
define *pre_display_size* ≡ *dimen_par*(*pre_display_size_code*)
define *display_width* ≡ *dimen_par*(*display_width_code*)
define *display_indent* ≡ *dimen_par*(*display_indent_code*)
define *overfull_rule* ≡ *dimen_par*(*overfull_rule_code*)
define *hang_indent* ≡ *dimen_par*(*hang_indent_code*)
define *h_offset* ≡ *dimen_par*(*h_offset_code*)
define *v_offset* ≡ *dimen_par*(*v_offset_code*)
define *emergency_stretch* ≡ *dimen_par*(*emergency_stretch_code*)
define *pdf_page_width* ≡ *dimen_par*(*pdf_page_width_code*)

```
  define pdf_page_height ≡ dimen_par(pdf_page_height_code)
procedure print_length_param(n : integer);
  begin case n of
  par_indent_code: print_esc("parindent");
  math_surround_code: print_esc("mathsurround");
  line_skip_limit_code: print_esc("lineskiplimit");
  hsize_code: print_esc("hsize");
  vsize_code: print_esc("vsize");
  max_depth_code: print_esc("maxdepth");
  split_max_depth_code: print_esc("splitmaxdepth");
  box_max_depth_code: print_esc("boxmaxdepth");
  hfuzz_code: print_esc("hfuzz");
  vfuzz_code: print_esc("vfuzz");
  delimiter_shortfall_code: print_esc("delimitershortfall");
  null_delimiter_space_code: print_esc("nulldelimiterspace");
  script_space_code: print_esc("scriptspace");
  pre_display_size_code: print_esc("predisplaysize");
  display_width_code: print_esc("displaywidth");
  display_indent_code: print_esc("displayindent");
  overfull_rule_code: print_esc("overfullrule");
  hang_indent_code: print_esc("hangindent");
  h_offset_code: print_esc("hoffset");
  v_offset_code: print_esc("voffset");
  emergency_stretch_code: print_esc("emergencystretch");
  pdf_page_width_code: print_esc("pdfpagewidth");
  pdf_page_height_code: print_esc("pdfpageheight");
  othercases print("[unknown␣dimen␣parameter!]")
  endcases;
  end;
```

274. ⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("parindent", *assign_dimen*, *dimen_base* + *par_indent_code*);
primitive("mathsurround", *assign_dimen*, *dimen_base* + *math_surround_code*);
primitive("lineskiplimit", *assign_dimen*, *dimen_base* + *line_skip_limit_code*);
primitive("hsize", *assign_dimen*, *dimen_base* + *hsize_code*);
primitive("vsize", *assign_dimen*, *dimen_base* + *vsize_code*);
primitive("maxdepth", *assign_dimen*, *dimen_base* + *max_depth_code*);
primitive("splitmaxdepth", *assign_dimen*, *dimen_base* + *split_max_depth_code*);
primitive("boxmaxdepth", *assign_dimen*, *dimen_base* + *box_max_depth_code*);
primitive("hfuzz", *assign_dimen*, *dimen_base* + *hfuzz_code*);
primitive("vfuzz", *assign_dimen*, *dimen_base* + *vfuzz_code*);
primitive("delimitershortfall", *assign_dimen*, *dimen_base* + *delimiter_shortfall_code*);
primitive("nulldelimiterspace", *assign_dimen*, *dimen_base* + *null_delimiter_space_code*);
primitive("scriptspace", *assign_dimen*, *dimen_base* + *script_space_code*);
primitive("predisplaysize", *assign_dimen*, *dimen_base* + *pre_display_size_code*);
primitive("displaywidth", *assign_dimen*, *dimen_base* + *display_width_code*);
primitive("displayindent", *assign_dimen*, *dimen_base* + *display_indent_code*);
primitive("overfullrule", *assign_dimen*, *dimen_base* + *overfull_rule_code*);
primitive("hangindent", *assign_dimen*, *dimen_base* + *hang_indent_code*);
primitive("hoffset", *assign_dimen*, *dimen_base* + *h_offset_code*);
primitive("voffset", *assign_dimen*, *dimen_base* + *v_offset_code*);
primitive("emergencystretch", *assign_dimen*, *dimen_base* + *emergency_stretch_code*);
primitive("pdfpagewidth", *assign_dimen*, *dimen_base* + *pdf_page_width_code*);
primitive("pdfpageheight", *assign_dimen*, *dimen_base* + *pdf_page_height_code*);

275. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
assign_dimen: **if** *chr_code* < *scaled_base* **then** *print_length_param*(*chr_code* − *dimen_base*)
else begin *print_esc*("dimen"); *print_int*(*chr_code* − *scaled_base*);
end;

276. ⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
for *k* ← *dimen_base* **to** *eqtb_size* **do** *eqtb*[*k*].*sc* ← 0;

277. ⟨ Show equivalent *n*, in region 6 277 ⟩ ≡
begin if *n* < *scaled_base* **then** *print_length_param*(*n* − *dimen_base*)
else begin *print_esc*("dimen"); *print_int*(*n* − *scaled_base*);
end;
print_char("="); *print_scaled*(*eqtb*[*n*].*sc*); *print*("pt");
end

This code is used in section 278.

278. Here is a procedure that displays the contents of *eqtb*[*n*] symbolically.

```
⟨ Declare the procedure called print_cmd_chr 328 ⟩
  stat procedure show_eqtb(n : pointer);
  begin if n < active_base then print_char("?")  { this can't happen }
  else if n < glue_base then ⟨ Show equivalent n, in region 1 or 2 249 ⟩
    else if n < local_base then ⟨ Show equivalent n, in region 3 255 ⟩
      else if n < int_base then ⟨ Show equivalent n, in region 4 259 ⟩
        else if n < dimen_base then ⟨ Show equivalent n, in region 5 268 ⟩
          else if n ≤ eqtb_size then ⟨ Show equivalent n, in region 6 277 ⟩
            else print_char("?");  { this can't happen either }
  end;
  tats
```

279. The last two regions of *eqtb* have fullword values instead of the three fields *eq_level*, *eq_type*, and *equiv*. An *eq_type* is unnecessary, but TeX needs to store the *eq_level* information in another array called *xeq_level*.

```
⟨ Global variables 13 ⟩ +≡
eqtb: array [active_base .. eqtb_size] of memory_word;
xeq_level: array [int_base .. eqtb_size] of quarterword;
```

280.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  for k ← int_base to eqtb_size do xeq_level[k] ← level_one;
```

281. When the debugging routine *search_mem* is looking for pointers having a given value, it is interested only in regions 1 to 3 of *eqtb*, and in the first part of region 4.

```
⟨ Search eqtb for equivalents equal to p 281 ⟩ ≡
  for q ← active_base to box_base + biggest_reg do
    begin if equiv(q) = p then
      begin print_nl("EQUIV("); print_int(q); print_char(")");
      end;
    end
```

This code is used in section 197.

282. The hash table. Control sequences are stored and retrieved by means of a fairly standard hash table algorithm called the method of "coalescing lists" (cf. Algorithm 6.4C in *The Art of Computer Programming*). Once a control sequence enters the table, it is never removed, because there are complicated situations involving `\gdef` where the removal of a control sequence at the end of a group would be a mistake preventable only by the introduction of a complicated reference-count mechanism.

The actual sequence of letters forming a control sequence identifier is stored in the *str_pool* array together with all the other strings. An auxiliary array *hash* consists of items with two halfword fields per word. The first of these, called $next(p)$, points to the next identifier belonging to the same coalesced list as the identifier corresponding to p; and the other, called $text(p)$, points to the *str_start* entry for p's identifier. If position p of the hash table is empty, we have $text(p) = 0$; if position p is either empty or the end of a coalesced hash list, we have $next(p) = 0$. An auxiliary pointer variable called *hash_used* is maintained in such a way that all locations $p \geq hash_used$ are nonempty. The global variable *cs_count* tells how many multiletter control sequences have been defined, if statistics are being kept.

A global boolean variable called *no_new_control_sequence* is set to *true* during the time that new hash table entries are forbidden.

```
define next(#) ≡ hash[#].lh  { link for coalesced lists }
define text(#) ≡ hash[#].rh  { string number for control sequence name }
define hash_is_full ≡ (hash_used = hash_base)  { test if all positions are occupied }
define font_id_text(#) ≡ text(font_id_base + #)  { a frozen font identifier's name }
```

⟨ Global variables 13 ⟩ +≡

```
hash: array [hash_base .. undefined_control_sequence − 1] of two_halves;  { the hash table }
hash_used: pointer;  { allocation pointer for hash }
no_new_control_sequence: boolean;  { are new identifiers legal? }
cs_count: integer;  { total number of known identifiers }
```

283. Primitive support needs a few extra variables and definitions

```
define prim_prime = 1777  { about 85% of primitive_size }
define prim_base = 1
define prim_next(#) ≡ prim[#].lh  { link for coalesced lists }
define prim_text(#) ≡ prim[#].rh  { string number for control sequence name, plus one }
define prim_is_full ≡ (prim_used = prim_base)  { test if all positions are occupied }
define prim_eq_level_field(#) ≡ #.hh.b1
define prim_eq_type_field(#) ≡ #.hh.b0
define prim_equiv_field(#) ≡ #.hh.rh
define prim_eq_level(#) ≡ prim_eq_level_field(eqtb[prim_eqtb_base + #])  { level of definition }
define prim_eq_type(#) ≡ prim_eq_type_field(eqtb[prim_eqtb_base + #])  { command code for equivalent }
define prim_equiv(#) ≡ prim_equiv_field(eqtb[prim_eqtb_base + #])  { equivalent value }
define undefined_primitive = 0
```

⟨ Global variables 13 ⟩ +≡

```
prim: array [0 .. prim_size] of two_halves;  { the primitives table }
prim_used: pointer;  { allocation pointer for prim }
```

284. ⟨ Set initial values of key variables 23 ⟩ +≡

```
no_new_control_sequence ← true;  { new identifiers are usually forbidden }
prim_next(0) ← 0; prim_text(0) ← 0;
for k ← 1 to prim_size do prim[k] ← prim[0];
next(hash_base) ← 0; text(hash_base) ← 0;
for k ← hash_base + 1 to undefined_control_sequence − 1 do hash[k] ← hash[hash_base];
```

285. ⟨ Initialize table entries (done by `INITEX` only) 189 ⟩ +≡
```
prim_used ← prim_size; { nothing is used }
hash_used ← frozen_control_sequence; { nothing is used }
cs_count ← 0; eq_type(frozen_dont_expand) ← dont_expand;
text(frozen_dont_expand) ← "notexpanded:"; eq_type(frozen_primitive) ← ignore_spaces;
equiv(frozen_primitive) ← 1; eq_level(frozen_primitive) ← level_one;
text(frozen_primitive) ← "primitive";
```

286. Here is the subroutine that searches the hash table for an identifier that matches a given string of length $l > 0$ appearing in $buffer[j \mathinner{..} (j + l - 1)]$. If the identifier is found, the corresponding hash table address is returned. Otherwise, if the global variable *no_new_control_sequence* is *true*, the dummy address *undefined_control_sequence* is returned. Otherwise the identifier is inserted into the hash table and its location is returned.

```
function id_lookup(j, l : integer): pointer; { search the hash table }
  label found; { go here if you found it }
  var h: integer; { hash code }
    d: integer; { number of characters in incomplete current string }
    p: pointer; { index in hash array }
    k: pointer; { index in buffer array }
    ll: integer; { length in UTF16 code units }
  begin ⟨ Compute the hash code h 288 ⟩;
  p ← h + hash_base; { we start searching here; note that 0 ≤ h < hash_prime }
  ll ← l;
  for d ← 0 to l − 1 do
    if buffer[j + d] ≥ "10000 then incr(ll);
  loop begin if text(p) > 0 then
      if length(text(p)) = ll then
        if str_eq_buf(text(p), j) then goto found;
    if next(p) = 0 then
      begin if no_new_control_sequence then p ← undefined_control_sequence
      else ⟨ Insert a new control sequence after p, then make p point to it 287 ⟩;
      goto found;
      end;
    p ← next(p);
    end;
found: id_lookup ← p;
  end;
```

287. ⟨ Insert a new control sequence after p, then make p point to it 287 ⟩ ≡

```
begin if text(p) > 0 then
  begin repeat if hash_is_full then overflow("hash␣size", hash_size);
    decr(hash_used);
  until text(hash_used) = 0;  { search for an empty location in hash }
  next(p) ← hash_used; p ← hash_used;
  end;
str_room(ll); d ← cur_length;
while pool_ptr > str_start_macro(str_ptr) do
  begin decr(pool_ptr); str_pool[pool_ptr + l] ← str_pool[pool_ptr];
  end;  { move current string up to make room for another }
for k ← j to j + l − 1 do
  begin if buffer[k] < "10000 then append_char(buffer[k])
  else begin append_char("D800 + (buffer[k] − "10000) div "400);
    append_char("DC00 + (buffer[k] − "10000) mod "400);
    end
  end;
text(p) ← make_string; pool_ptr ← pool_ptr + d;
stat incr(cs_count); tats
end
```

This code is used in section 286.

288. The value of *hash_prime* should be roughly 85% of *hash_size*, and it should be a prime number. The theory of hashing tells us to expect fewer than two table probes, on the average, when the search is successful. [See J. S. Vitter, *Journal of the ACM* **30** (1983), 231–258.]

⟨ Compute the hash code h 288 ⟩ ≡

```
h ← 0;
for k ← j to j + l − 1 do
  begin h ← h + h + buffer[k];
  while h ≥ hash_prime do h ← h − hash_prime;
  end
```

This code is used in section 286.

289. Here is the subroutine that searches the primitive table for an identifier

```
function prim_lookup(s : str_number): pointer; { search the primitives table }
  label found; { go here if you found it }
  var h: integer; { hash code }
    p: pointer; { index in hash array }
    k: pointer; { index in string pool }
    j, l: integer;
  begin if s ≤ biggest_char then
    begin if s < 0 then
      begin p ← undefined_primitive; goto found;
      end
    else p ← (s mod prim_prime) + prim_base; { we start searching here }
    end
  else begin j ← str_start_macro(s);
    if s = str_ptr then l ← cur_length
    else l ← length(s);
    ⟨ Compute the primitive code h 291 ⟩;
    p ← h + prim_base; { we start searching here; note that 0 ≤ h < prim_prime }
    end;
  loop begin if prim_text(p) > 1 + biggest_char then { p points a multi-letter primitive }
      begin if length(prim_text(p) − 1) = l then
        if str_eq_str(prim_text(p) − 1, s) then goto found;
      end
    else if prim_text(p) = 1 + s then goto found; { p points a single-letter primitive }
    if prim_next(p) = 0 then
      begin if no_new_control_sequence then p ← undefined_primitive
      else ⟨ Insert a new primitive after p, then make p point to it 290 ⟩;
      goto found;
      end;
    p ← prim_next(p);
    end;
found: prim_lookup ← p;
  end;
```

290. ⟨ Insert a new primitive after p, then make p point to it 290 ⟩ ≡

```
  begin if prim_text(p) > 0 then
    begin repeat if prim_is_full then overflow("primitive␣size", prim_size);
      decr(prim_used);
    until prim_text(prim_used) = 0; { search for an empty location in prim }
    prim_next(p) ← prim_used; p ← prim_used;
    end;
  prim_text(p) ← s + 1;
  end
```

This code is used in section 289.

291. The value of *prim_prime* should be roughly 85% of *prim_size*, and it should be a prime number.

```
⟨ Compute the primitive code h 291 ⟩ ≡
  h ← str_pool[j];
  for k ← j + 1 to j + l − 1 do
    begin h ← h + h + str_pool[k];
    while h ≥ prim_prime do h ← h − prim_prime;
    end
```

This code is used in section 289.

292. Single-character control sequences do not need to be looked up in a hash table, since we can use the character code itself as a direct address. The procedure *print_cs* prints the name of a control sequence, given a pointer to its address in *eqtb*. A space is printed after the name unless it is a single nonletter or an active character. This procedure might be invoked with invalid data, so it is "extra robust." The individual characters must be printed one at a time using *print*, since they may be unprintable.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_cs(p : integer); { prints a purported control sequence }
  begin if p < hash_base then { single character }
    if p ≥ single_base then
      if p = null_cs then
        begin print_esc("csname"); print_esc("endcsname"); print_char("␣");
        end
      else begin print_esc(p − single_base);
        if cat_code(p − single_base) = letter then print_char("␣");
        end
    else if p < active_base then print_esc("IMPOSSIBLE.")
      else print_char(p − active_base)
  else if p ≥ undefined_control_sequence then print_esc("IMPOSSIBLE.")
    else if (text(p) < 0) ∨ (text(p) ≥ str_ptr) then print_esc("NONEXISTENT.")
      else begin if (p ≥ prim_eqtb_base) ∧ (p < frozen_null_font) then
          print_esc(prim_text(p − prim_eqtb_base) − 1)
        else print_esc(text(p));
        print_char("␣");
        end;
  end;
```

293. Here is a similar procedure; it avoids the error checks, and it never prints a space after the control sequence.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure sprint_cs(p : pointer); { prints a control sequence }
  begin if p < hash_base then
    if p < single_base then print_char(p − active_base)
    else if p < null_cs then print_esc(p − single_base)
      else begin print_esc("csname"); print_esc("endcsname");
        end
  else if (p ≥ prim_eqtb_base) ∧ (p < frozen_null_font) then print_esc(prim_text(p − prim_eqtb_base) − 1)
    else print_esc(text(p));
  end;
```

294. We need to put TeX's "primitive" control sequences into the hash table, together with their command code (which will be the *eq_type*) and an operand (which will be the *equiv*). The *primitive* procedure does this, in a way that no TeX user can. The global value *cur_val* contains the new *eqtb* pointer after *primitive* has acted.

```
init procedure primitive(s : str_number; c : quarterword; o : halfword);
var k: pool_pointer;  { index into str_pool }
  j: 0 .. buf_size;  { index into buffer }
  l: small_number;  { length of the string }
  prim_val: integer;  { needed to fill prim_eqtb }
begin if s < 256 then
  begin cur_val ← s + single_base; prim_val ← prim_lookup(s);
  end
else begin k ← str_start_macro(s); l ← str_start_macro(s + 1) − k;
      { we will move s into the (possibly non-empty) buffer }
  if first + l > buf_size + 1 then overflow("buffer␣size", buf_size);
  for j ← 0 to l − 1 do buffer[first + j] ← so(str_pool[k + j]);
  cur_val ← id_lookup(first, l);  { no_new_control_sequence is false }
  flush_string; text(cur_val) ← s;  { we don't want to have the string twice }
  prim_val ← prim_lookup(s);
  end;
eq_level(cur_val) ← level_one; eq_type(cur_val) ← c; equiv(cur_val) ← o;
prim_eq_level(prim_val) ← level_one; prim_eq_type(prim_val) ← c; prim_equiv(prim_val) ← o;
end;
tini
```

295. Many of TeX's primitives need no *equiv*, since they are identifiable by their *eq_type* alone. These primitives are loaded into the hash table as follows:

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("␣", *ex_space*, 0);
primitive("/", *ital_corr*, 0);
primitive("accent", *accent*, 0);
primitive("advance", *advance*, 0);
primitive("afterassignment", *after_assignment*, 0);
primitive("aftergroup", *after_group*, 0);
primitive("begingroup", *begin_group*, 0);
primitive("char", *char_num*, 0);
primitive("csname", *cs_name*, 0);
primitive("delimiter", *delim_num*, 0);
primitive("XeTeXdelimiter", *delim_num*, 1);
primitive("Udelimiter", *delim_num*, 1);
primitive("divide", *divide*, 0);
primitive("endcsname", *end_cs_name*, 0);
primitive("endgroup", *end_group*, 0); *text*(*frozen_end_group*) ← "endgroup";
eqtb[*frozen_end_group*] ← *eqtb*[*cur_val*];
primitive("expandafter", *expand_after*, 0);
primitive("font", *def_font*, 0);
primitive("fontdimen", *assign_font_dimen*, 0);
primitive("halign", *halign*, 0);
primitive("hrule", *hrule*, 0);
primitive("ignorespaces", *ignore_spaces*, 0);
primitive("insert", *insert*, 0);
primitive("mark", *mark*, 0);
primitive("mathaccent", *math_accent*, 0);
primitive("XeTeXmathaccent", *math_accent*, 1);
primitive("Umathaccent", *math_accent*, 1);
primitive("mathchar", *math_char_num*, 0);
primitive("XeTeXmathcharnum", *math_char_num*, 1);
primitive("Umathcharnum", *math_char_num*, 1);
primitive("XeTeXmathchar", *math_char_num*, 2);
primitive("Umathchar", *math_char_num*, 2);
primitive("mathchoice", *math_choice*, 0);
primitive("multiply", *multiply*, 0);
primitive("noalign", *no_align*, 0);
primitive("noboundary", *no_boundary*, 0);
primitive("noexpand", *no_expand*, 0);
primitive("primitive", *no_expand*, 1);
primitive("nonscript", *non_script*, 0);
primitive("omit", *omit*, 0);
primitive("parshape", *set_shape*, *par_shape_loc*);
primitive("penalty", *break_penalty*, 0);
primitive("prevgraf", *set_prev_graf*, 0);
primitive("radical", *radical*, 0);
primitive("XeTeXradical", *radical*, 1);
primitive("Uradical", *radical*, 1);
primitive("read", *read_to_cs*, 0);
primitive("relax", *relax*, *too_big_usv*); { cf. *scan_file_name* }
text(*frozen_relax*) ← "relax"; *eqtb*[*frozen_relax*] ← *eqtb*[*cur_val*];

primitive("setbox", *set_box*, 0);
primitive("the", *the*, 0);
primitive("toks", *toks_register*, *mem_bot*);
primitive("vadjust", *vadjust*, 0);
primitive("valign", *valign*, 0);
primitive("vcenter", *vcenter*, 0);
primitive("vrule", *vrule*, 0);

296. Each primitive has a corresponding inverse, so that it is possible to display the cryptic numeric contents of *eqtb* in symbolic form. Every call of *primitive* in this program is therefore accompanied by some straightforward code that forms part of the *print_cmd_chr* routine below.

```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
accent: print_esc("accent");
advance: print_esc("advance");
after_assignment: print_esc("afterassignment");
after_group: print_esc("aftergroup");
assign_font_dimen: print_esc("fontdimen");
begin_group: print_esc("begingroup");
break_penalty: print_esc("penalty");
char_num: print_esc("char");
cs_name: print_esc("csname");
def_font: print_esc("font");
delim_num: if chr_code = 1 then print_esc("Udelimiter")
  else print_esc("delimiter");
divide: print_esc("divide");
end_cs_name: print_esc("endcsname");
end_group: print_esc("endgroup");
ex_space: print_esc("␣");
expand_after: if chr_code = 0 then print_esc("expandafter")
      ⟨ Cases of expandafter for print_cmd_chr 1574 ⟩;
halign: print_esc("halign");
hrule: print_esc("hrule");
ignore_spaces: if chr_code = 0 then print_esc("ignorespaces")
  else print_esc("primitive");
insert: print_esc("insert");
ital_corr: print_esc("/");
mark: begin print_esc("mark");
  if chr_code > 0 then print_char("s");
  end;
math_accent: if chr_code = 1 then print_esc("Umathaccent")
  else print_esc("mathaccent");
math_char_num: if chr_code = 2 then print_esc("Umathchar")
  else if chr_code = 1 then print_esc("Umathcharnum")
    else print_esc("mathchar");
math_choice: print_esc("mathchoice");
multiply: print_esc("multiply");
no_align: print_esc("noalign");
no_boundary: print_esc("noboundary");
no_expand: if chr_code = 0 then print_esc("noexpand")
  else print_esc("primitive");
non_script: print_esc("nonscript");
omit: print_esc("omit");
radical: if chr_code = 1 then print_esc("Uradical")
  else print_esc("radical");
read_to_cs: if chr_code = 0 then print_esc("read") ⟨ Cases of read for print_cmd_chr 1571 ⟩;
relax: print_esc("relax");
set_box: print_esc("setbox");
set_prev_graf: print_esc("prevgraf");
set_shape: case chr_code of
  par_shape_loc: print_esc("parshape");
```

```
    ⟨ Cases of set_shape for print_cmd_chr 1676 ⟩
  end; { there are no other cases }
the: if chr_code = 0 then print_esc("the") ⟨ Cases of the for print_cmd_chr 1497 ⟩;
toks_register: ⟨ Cases of toks_register for print_cmd_chr 1644 ⟩;
vadjust: print_esc("vadjust");
valign: if chr_code = 0 then print_esc("valign")
  ⟨ Cases of valign for print_cmd_chr 1512 ⟩;
vcenter: print_esc("vcenter");
vrule: print_esc("vrule");
```

297. We will deal with the other primitives later, at some point in the program where their *eq_type* and *equiv* values are more meaningful. For example, the primitives for math mode will be loaded when we consider the routines that deal with formulas. It is easy to find where each particular primitive was treated by looking in the index at the end; for example, the section where `"radical"` entered *eqtb* is listed under '`\radical` primitive'. (Primitives consisting of a single nonalphabetic character, like '`\/`', are listed under 'Single-character primitives'.)

Meanwhile, this is a convenient place to catch up on something we were unable to do before the hash table was defined:

```
⟨ Print the font identifier for font(p) 297 ⟩ ≡
  print_esc(font_id_text(font(p)))
```

This code is used in sections 200 and 202.

298. Saving and restoring equivalents. The nested structure provided by '`{...}`' groups in TeX means that *eqtb* entries valid in outer groups should be saved and restored later if they are overridden inside the braces. When a new *eqtb* value is being assigned, the program therefore checks to see if the previous entry belongs to an outer level. In such a case, the old value is placed on the *save_stack* just before the new value enters *eqtb*. At the end of a grouping level, i.e., when the right brace is sensed, the *save_stack* is used to restore the outer values, and the inner ones are destroyed.

Entries on the *save_stack* are of type *memory_word*. The top item on this stack is *save_stack*[*p*], where $p = save_ptr - 1$; it contains three fields called *save_type*, *save_level*, and *save_index*, and it is interpreted in one of five ways:

1) If *save_type*(*p*) = *restore_old_value*, then *save_index*(*p*) is a location in *eqtb* whose current value should be destroyed at the end of the current group and replaced by *save_stack*[$p-1$]. Furthermore if *save_index*(*p*) $\geq$ *int_base*, then *save_level*(*p*) should replace the corresponding entry in *xeq_level*.

2) If *save_type*(*p*) = *restore_zero*, then *save_index*(*p*) is a location in *eqtb* whose current value should be destroyed at the end of the current group, when it should be replaced by the value of *eqtb*[*undefined_control_sequence*].

3) If *save_type*(*p*) = *insert_token*, then *save_index*(*p*) is a token that should be inserted into TeX's input when the current group ends.

4) If *save_type*(*p*) = *level_boundary*, then *save_level*(*p*) is a code explaining what kind of group we were previously in, and *save_index*(*p*) points to the level boundary word at the bottom of the entries for that group. Furthermore, in extended ε-TeX mode, *save_stack*[$p-1$] contains the source line number at which the current level of grouping was entered.

5) If *save_type*(*p*) = *restore_sa*, then *sa_chain* points to a chain of sparse array entries to be restored at the end of the current group. Furthermore *save_index*(*p*) and *save_level*(*p*) should replace the values of *sa_chain* and *sa_level* respectively.

define *save_type*(#) ≡ *save_stack*[#].*hh.b0* { classifies a *save_stack* entry }
define *save_level*(#) ≡ *save_stack*[#].*hh.b1* { saved level for regions 5 and 6, or group code }
define *save_index*(#) ≡ *save_stack*[#].*hh.rh* { *eqtb* location or token or *save_stack* location }
define *restore_old_value* = 0 { *save_type* when a value should be restored later }
define *restore_zero* = 1 { *save_type* when an undefined entry should be restored }
define *insert_token* = 2 { *save_type* when a token is being saved for later use }
define *level_boundary* = 3 { *save_type* corresponding to beginning of group }
define *restore_sa* = 4 { *save_type* when sparse array entries should be restored }

⟨ Declare ε-TeX procedures for tracing and input 314 ⟩

299. Here are the group codes that are used to discriminate between different kinds of groups. They allow TeX to decide what special actions, if any, should be performed when a group ends.

Some groups are not supposed to be ended by right braces. For example, the '$' that begins a math formula causes a *math_shift_group* to be started, and this should be terminated by a matching '$'. Similarly, a group that starts with `\left` should end with `\right`, and one that starts with `\begingroup` should end with `\endgroup`.

define *bottom_level* = 0 { group code for the outside world }
define *simple_group* = 1 { group code for local structure only }
define *hbox_group* = 2 { code for '`\hbox{...}`' }
define *adjusted_hbox_group* = 3 { code for '`\hbox{...}`' in vertical mode }
define *vbox_group* = 4 { code for '`\vbox{...}`' }
define *vtop_group* = 5 { code for '`\vtop{...}`' }
define *align_group* = 6 { code for '`\halign{...}`', '`\valign{...}`' }
define *no_align_group* = 7 { code for '`\noalign{...}`' }
define *output_group* = 8 { code for output routine }
define *math_group* = 9 { code for, e.g., '`^{...}`' }
define *disc_group* = 10 { code for '`\discretionary{...}{...}{...}`' }
define *insert_group* = 11 { code for '`\insert{...}`', '`\vadjust{...}`' }
define *vcenter_group* = 12 { code for '`\vcenter{...}`' }
define *math_choice_group* = 13 { code for '`\mathchoice{...}{...}{...}{...}`' }
define *semi_simple_group* = 14 { code for '`\begingroup...\endgroup`' }
define *math_shift_group* = 15 { code for '`$...$`' }
define *math_left_group* = 16 { code for '`\left...\right`' }
define *max_group_code* = 16

⟨ Types in the outer block 18 ⟩ +≡
group_code = 0 .. *max_group_code*; { *save_level* for a level boundary }

300. The global variable *cur_group* keeps track of what sort of group we are currently in. Another global variable, *cur_boundary*, points to the topmost *level_boundary* word. And *cur_level* is the current depth of nesting. The routines are designed to preserve the condition that no entry in the *save_stack* or in *eqtb* ever has a level greater than *cur_level*.

301. ⟨ Global variables 13 ⟩ +≡
save_stack: **array** [0 .. *save_size*] **of** *memory_word*;
save_ptr: 0 .. *save_size*; { first unused entry on *save_stack* }
max_save_stack: 0 .. *save_size*; { maximum usage of save stack }
cur_level: *quarterword*; { current nesting level for groups }
cur_group: *group_code*; { current group type }
cur_boundary: 0 .. *save_size*; { where the current level begins }

302. At this time it might be a good idea for the reader to review the introduction to *eqtb* that was given above just before the long lists of parameter names. Recall that the "outer level" of the program is *level_one*, since undefined control sequences are assumed to be "defined" at *level_zero*.

⟨ Set initial values of key variables 23 ⟩ +≡
save_ptr ← 0; *cur_level* ← *level_one*; *cur_group* ← *bottom_level*; *cur_boundary* ← 0; *max_save_stack* ← 0;

303. The following macro is used to test if there is room for up to seven more entries on *save_stack*. By making a conservative test like this, we can get by with testing for overflow in only a few places.

```
define check_full_save_stack ≡
    if save_ptr > max_save_stack then
      begin max_save_stack ← save_ptr;
      if max_save_stack > save_size − 7 then overflow("save␣size", save_size);
      end
```

304. Procedure *new_save_level* is called when a group begins. The argument is a group identification code like '*hbox_group*'. After calling this routine, it is safe to put five more entries on *save_stack*.

In some cases integer-valued items are placed onto the *save_stack* just below a *level_boundary* word, because this is a convenient place to keep information that is supposed to "pop up" just when the group has finished. For example, when '`\hbox to 100pt{...}`' is being treated, the 100pt dimension is stored on *save_stack* just before *new_save_level* is called.

We use the notation *saved*(*k*) to stand for an integer item that appears in location *save_ptr* + *k* of the save stack.

```
define saved(#) ≡ save_stack[save_ptr + #].int
procedure new_save_level(c : group_code); { begin a new level of grouping }
  begin check_full_save_stack;
  if eTeX_ex then
    begin saved(0) ← line; incr(save_ptr);
    end;
  save_type(save_ptr) ← level_boundary; save_level(save_ptr) ← cur_group;
  save_index(save_ptr) ← cur_boundary;
  if cur_level = max_quarterword then
    overflow("grouping␣levels", max_quarterword − min_quarterword);
        { quit if (cur_level + 1) is too big to be stored in eqtb }
  cur_boundary ← save_ptr; cur_group ← c;
  stat if tracing_groups > 0 then group_trace(false);
  tats
  incr(cur_level); incr(save_ptr);
  end;
```

305. Just before an entry of *eqtb* is changed, the following procedure should be called to update the other data structures properly. It is important to keep in mind that reference counts in *mem* include references from within *save_stack*, so these counts must be handled carefully.

```
procedure eq_destroy(w : memory_word); { gets ready to forget w }
  var q: pointer; { equiv field of w }
  begin case eq_type_field(w) of
  call, long_call, outer_call, long_outer_call: delete_token_ref(equiv_field(w));
  glue_ref: delete_glue_ref(equiv_field(w));
  shape_ref: begin q ← equiv_field(w); { we need to free a \parshape block }
    if q ≠ null then free_node(q, info(q) + info(q) + 1);
    end; { such a block is 2n + 1 words long, where n = info(q) }
  box_ref: flush_node_list(equiv_field(w));
    ⟨ Cases for eq_destroy 1645 ⟩
  othercases do_nothing
  endcases;
  end;
```

306. To save a value of *eqtb*[*p*] that was established at level *l*, we can use the following subroutine.

```
procedure eq_save(p : pointer; l : quarterword);  { saves eqtb[p] }
  begin check_full_save_stack;
  if l = level_zero then save_type(save_ptr) ← restore_zero
  else begin save_stack[save_ptr] ← eqtb[p]; incr(save_ptr); save_type(save_ptr) ← restore_old_value;
    end;
  save_level(save_ptr) ← l; save_index(save_ptr) ← p; incr(save_ptr);
  end;
```

307. The procedure *eq_define* defines an *eqtb* entry having specified *eq_type* and *equiv* fields, and saves the former value if appropriate. This procedure is used only for entries in the first four regions of *eqtb*, i.e., only for entries that have *eq_type* and *equiv* fields. After calling this routine, it is safe to put four more entries on *save_stack*, provided that there was room for four more entries before the call, since *eq_save* makes the necessary test.

```
  define assign_trace(#) ≡
          stat if tracing_assigns > 0 then restore_trace(#);
          tats
procedure eq_define(p : pointer; t : quarterword; e : halfword);  { new data for eqtb }
  label exit;
  begin if eTeX_ex ∧ (eq_type(p) = t) ∧ (equiv(p) = e) then
    begin assign_trace(p, "reassigning")
    eq_destroy(eqtb[p]); return;
    end;
  assign_trace(p, "changing")
  if eq_level(p) = cur_level then eq_destroy(eqtb[p])
  else if cur_level > level_one then eq_save(p, eq_level(p));
  eq_level(p) ← cur_level; eq_type(p) ← t; equiv(p) ← e; assign_trace(p, "into")
exit: end;
```

308. The counterpart of *eq_define* for the remaining (fullword) positions in *eqtb* is called *eq_word_define*. Since *xeq_level*[*p*] ≥ *level_one* for all *p*, a '*restore_zero*' will never be used in this case.

```
procedure eq_word_define(p : pointer; w : integer);
  label exit;
  begin if eTeX_ex ∧ (eqtb[p].int = w) then
    begin assign_trace(p, "reassigning")
    return;
    end;
  assign_trace(p, "changing")
  if xeq_level[p] ≠ cur_level then
    begin eq_save(p, xeq_level[p]); xeq_level[p] ← cur_level;
    end;
  eqtb[p].int ← w; assign_trace(p, "into")
exit: end;
```

309. The *eq_define* and *eq_word_define* routines take care of local definitions. Global definitions are done in almost the same way, but there is no need to save old values, and the new value is associated with *level_one*.

```
procedure geq_define(p : pointer; t : quarterword; e : halfword);  { global eq_define }
  begin assign_trace(p, "globally␣changing")
  begin eq_destroy(eqtb[p]); eq_level(p) ← level_one; eq_type(p) ← t; equiv(p) ← e;
  end; assign_trace(p, "into")
  end;
procedure geq_word_define(p : pointer; w : integer);  { global eq_word_define }
  begin assign_trace(p, "globally␣changing")
  begin eqtb[p].int ← w; xeq_level[p] ← level_one;
  end; assign_trace(p, "into")
  end;
```

310. Subroutine *save_for_after* puts a token on the stack for save-keeping.

```
procedure save_for_after(t : halfword);
  begin if cur_level > level_one then
    begin check_full_save_stack; save_type(save_ptr) ← insert_token; save_level(save_ptr) ← level_zero;
    save_index(save_ptr) ← t; incr(save_ptr);
    end;
  end;
```

311. The *unsave* routine goes the other way, taking items off of *save_stack*. This routine takes care of restoration when a level ends; everything belonging to the topmost group is cleared off of the save stack.

```
procedure back_input; forward;
procedure unsave;  { pops the top level off the save stack }
  label done;
  var p: pointer;  { position to be restored }
    l: quarterword;  { saved level, if in fullword regions of eqtb }
    t: halfword;  { saved value of cur_tok }
    a: boolean;  { have we already processed an \aftergroup ? }
  begin a ← false;
  if cur_level > level_one then
    begin decr(cur_level); ⟨ Clear off top level from save_stack 312 ⟩;
    end
  else confusion("curlevel");  { unsave is not used when cur_group = bottom_level }
  end;
```

312. ⟨ Clear off top level from *save_stack* 312 ⟩ ≡

```
loop begin decr(save_ptr);
  if save_type(save_ptr) = level_boundary then goto done;
  p ← save_index(save_ptr);
  if save_type(save_ptr) = insert_token then ⟨ Insert token p into TEX's input 356 ⟩
  else if save_type(save_ptr) = restore_sa then
      begin sa_restore; sa_chain ← p; sa_level ← save_level(save_ptr);
      end
    else begin if save_type(save_ptr) = restore_old_value then
        begin l ← save_level(save_ptr); decr(save_ptr);
        end
      else save_stack[save_ptr] ← eqtb[undefined_control_sequence];
      ⟨ Store save_stack[save_ptr] in eqtb[p], unless eqtb[p] holds a global value 313 ⟩;
      end;
  end;
done: stat if tracing_groups > 0 then group_trace(true);
  tats
  if grp_stack[in_open] = cur_boundary then group_warning;
      { groups possibly not properly nested with files }
  cur_group ← save_level(save_ptr); cur_boundary ← save_index(save_ptr);
  if eTeX_ex then decr(save_ptr)
```

This code is used in section 311.

313. A global definition, which sets the level to *level_one*, will not be undone by *unsave*. If at least one global definition of *eqtb*[*p*] has been carried out within the group that just ended, the last such definition will therefore survive.

⟨ Store *save_stack*[*save_ptr*] in *eqtb*[*p*], unless *eqtb*[*p*] holds a global value 313 ⟩ ≡

```
if p < int_base then
  if eq_level(p) = level_one then
    begin eq_destroy(save_stack[save_ptr]);  { destroy the saved value }
    stat if tracing_restores > 0 then restore_trace(p, "retaining");
    tats
    end
  else begin eq_destroy(eqtb[p]);  { destroy the current value }
    eqtb[p] ← save_stack[save_ptr];  { restore the saved value }
    stat if tracing_restores > 0 then restore_trace(p, "restoring");
    tats
    end
else if xeq_level[p] ≠ level_one then
    begin eqtb[p] ← save_stack[save_ptr]; xeq_level[p] ← l;
    stat if tracing_restores > 0 then restore_trace(p, "restoring");
    tats
    end
  else begin stat if tracing_restores > 0 then restore_trace(p, "retaining");
    tats
    end
```

This code is used in section 312.

314. ⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ ≡
```
stat procedure restore_trace(p : pointer; s : str_number);  { eqtb[p] has just been restored or retained }
begin begin_diagnostic; print_char("{"); print(s); print_char("␣"); show_eqtb(p); print_char("}");
end_diagnostic(false);
end;
tats
```
See also sections 1471, 1472, 1567, 1568, 1585, 1587, 1588, 1632, 1634, 1648, 1649, 1650, 1651, and 1652.

This code is used in section 298.

315. When looking for possible pointers to a memory location, it is helpful to look for references from *eqtb* that might be waiting on the save stack. Of course, we might find spurious pointers too; but this routine is merely an aid when debugging, and at such times we are grateful for any scraps of information, even if they prove to be irrelevant.

⟨ Search *save_stack* for equivalents that point to *p* 315 ⟩ ≡
```
if save_ptr > 0 then
  for q ← 0 to save_ptr − 1 do
    begin if equiv_field(save_stack[q]) = p then
      begin print_nl("SAVE("); print_int(q); print_char(")");
      end;
    end
```
This code is used in section 197.

316. Most of the parameters kept in *eqtb* can be changed freely, but there's an exception: The magnification should not be used with two different values during any TEX job, since a single magnification is applied to an entire run. The global variable *mag_set* is set to the current magnification whenever it becomes necessary to "freeze" it at a particular value.

⟨ Global variables 13 ⟩ +≡
```
mag_set: integer;  { if nonzero, this magnification should be used henceforth }
```

317. ⟨ Set initial values of key variables 23 ⟩ +≡
```
mag_set ← 0;
```

318. The *prepare_mag* subroutine is called whenever TEX wants to use *mag* for magnification.
```
procedure prepare_mag;
begin if (mag_set > 0) ∧ (mag ≠ mag_set) then
  begin print_err("Incompatible␣magnification␣("); print_int(mag); print(");");
  print_nl("␣the␣previous␣value␣will␣be␣retained");
  help2("I␣can␣handle␣only␣one␣magnification␣ratio␣per␣job.␣So␣I´ve")
  ("reverted␣to␣the␣magnification␣you␣used␣earlier␣on␣this␣run.");
  int_error(mag_set); geq_word_define(int_base + mag_code, mag_set);  { mag ← mag_set }
  end;
if (mag ≤ 0) ∨ (mag > 32768) then
  begin print_err("Illegal␣magnification␣has␣been␣changed␣to␣1000");
  help1("The␣magnification␣ratio␣must␣be␣between␣1␣and␣32768."); int_error(mag);
  geq_word_define(int_base + mag_code, 1000);
  end;
mag_set ← mag;
end;
```

319. Token lists. A TeX token is either a character or a control sequence, and it is represented internally in one of two ways: (1) A character whose ASCII code number is c and whose command code is m is represented as the number $2^{21}m + c$; the command code is in the range $1 \le m \le 14$. (2) A control sequence whose *eqtb* address is p is represented as the number $cs_token_flag + p$. Here $cs_token_flag = 2^{25} - 1$ is larger than $2^{21}m + c$, yet it is small enough that $cs_token_flag + p < max_halfword$; thus, a token fits comfortably in a halfword.

A token t represents a *left_brace* command if and only if $t < left_brace_limit$; it represents a *right_brace* command if and only if we have $left_brace_limit \le t < right_brace_limit$; and it represents a *match* or *end_match* command if and only if $match_token \le t \le end_match_token$. The following definitions take care of these token-oriented constants and a few others.

define *cs_token_flag* = ″1FFFFFF { amount added to the *eqtb* location in a token that stands for a control sequence; is a multiple of ″10000, less 1 }
define *max_char_val* = ″200000 { to separate char and command code }
define *left_brace_token* = ″200000 { $2^{21} \cdot$ *left_brace* }
define *left_brace_limit* = ″400000 { $2^{21} \cdot (left_brace + 1)$ }
define *right_brace_token* = ″400000 { $2^{21} \cdot$ *right_brace* }
define *right_brace_limit* = ″600000 { $2^{21} \cdot (right_brace + 1)$ }
define *math_shift_token* = ″600000 { $2^{21} \cdot$ *math_shift* }
define *tab_token* = ″800000 { $2^{21} \cdot$ *tab_mark* }
define *out_param_token* = ″A00000 { $2^{21} \cdot$ *out_param* }
define *space_token* = ″1400020 { $2^{21} \cdot$ *spacer* + "␣" }
define *letter_token* = ″1600000 { $2^{21} \cdot$ *letter* }
define *other_token* = ″1800000 { $2^{21} \cdot$ *other_char* }
define *match_token* = ″1A00000 { $2^{21} \cdot$ *match* }
define *end_match_token* = ″1C00000 { $2^{21} \cdot$ *end_match* }
define *protected_token* = *end_match_token* + 1 { $2^{21} \cdot$ *end_match* + 1 }

320. ⟨ Check the "constant" values for consistency 14 ⟩ +≡
if *cs_token_flag* + *undefined_control_sequence* > *max_halfword* **then** *bad* ← 21;

321. A token list is a singly linked list of one-word nodes in *mem*, where each word contains a token and a link. Macro definitions, output-routine definitions, marks, `\write` texts, and a few other things are remembered by TeX in the form of token lists, usually preceded by a node with a reference count in its *token_ref_count* field. The token stored in location p is called *info*(p).

Three special commands appear in the token lists of macro definitions. When $m = match$, it means that TeX should scan a parameter for the current macro; when $m = end_match$, it means that parameter matching should end and TeX should start reading the macro text; and when $m = out_param$, it means that TeX should insert parameter number c into the text at this point.

The enclosing `{` and `}` characters of a macro definition are omitted, but an output routine will be enclosed in braces.

Here is an example macro definition that illustrates these conventions. After TeX processes the text

```
\def\mac a#1#2 \b {#1\-a ##1#2 #2}
```

the definition of `\mac` is represented as a token list containing

(reference count), *letter* `a`, *match* `#`, *match* `#`, *spacer* ␣, `\b`, *end_match*, *out_param* 1, `\-`, *letter* `a`, *spacer* ␣, *mac_param* `#`, *other_char* `1`, *out_param* 2, *spacer* ␣, *out_param* 2.

The procedure *scan_toks* builds such token lists, and *macro_call* does the parameter matching.

Examples such as

```
\def\m{\def\m{a}␣b}
```

explain why reference counts would be needed even if TeX had no `\let` operation: When the token list for `\m` is being read, the redefinition of `\m` changes the *eqtb* entry before the token list has been fully consumed, so we dare not simply destroy a token list when its control sequence is being redefined.

If the parameter-matching part of a definition ends with '`#{`', the corresponding token list will have '`{`' just before the '*end_match*' and also at the very end. The first '`{`' is used to delimit the parameter; the second one keeps the first from disappearing.

322. The procedure *show_token_list*, which prints a symbolic form of the token list that starts at a given node p, illustrates these conventions. The token list being displayed should not begin with a reference count. However, the procedure is intended to be robust, so that if the memory links are awry or if p is not really a pointer to a token list, nothing catastrophic will happen.

An additional parameter q is also given; this parameter is either null or it points to a node in the token list where a certain magic computation takes place that will be explained later. (Basically, q is non-null when we are printing the two-line context information at the time of an error message; q marks the place corresponding to where the second line should begin.)

For example, if p points to the node containing the first `a` in the token list above, then *show_token_list* will print the string

‘`a#1#2␣\b␣->#1\-a␣##1#2␣#2`’;

and if q points to the node containing the second `a`, the magic computation will be performed just before the second `a` is printed.

The generation will stop, and ‘`\ETC.`’ will be printed, if the length of printing exceeds a given limit l. Anomalous entries are printed in the form of control sequences that are not followed by a blank space, e.g., ‘`\BAD.`’; this cannot be confused with actual control sequences because a real control sequence named `BAD` would come out ‘`\BAD␣`’.

```
⟨ Declare the procedure called show_token_list 322 ⟩ ≡
procedure show_token_list(p, q : integer; l : integer);
  label exit;
  var m, c: integer;  { pieces of a token }
    match_chr: integer;  { character used in a 'match' }
    n: ASCII_code;  { the highest parameter number, as an ASCII digit }
  begin match_chr ← "#"; n ← "0"; tally ← 0;
  while (p ≠ null) ∧ (tally < l) do
    begin if p = q then ⟨ Do magic computation 350 ⟩;
    ⟨ Display token p, and return if there are problems 323 ⟩;
    p ← link(p);
    end;
  if p ≠ null then print_esc("ETC.");
exit: end;
```

This code is used in section 141.

```
323. ⟨ Display token p, and return if there are problems 323 ⟩ ≡
  if (p < hi_mem_min) ∨ (p > mem_end) then
    begin print_esc("CLOBBERED."); return;
    end;
  if info(p) ≥ cs_token_flag then print_cs(info(p) − cs_token_flag)
  else begin m ← info(p) div max_char_val; c ← info(p) mod max_char_val;
    if info(p) < 0 then print_esc("BAD.")
    else ⟨ Display the token (m, c) 324 ⟩;
    end
```

This code is used in section 322.

324. The procedure usually "learns" the character code used for macro parameters by seeing one in a *match* command before it runs into any *out_param* commands.

```
⟨ Display the token (m, c) 324 ⟩ ≡
  case m of
  left_brace, right_brace, math_shift, tab_mark, sup_mark, sub_mark, spacer, letter, other_char: print_char(c);
  mac_param: begin print_char(c); print_char(c);
    end;
  out_param: begin print_char(match_chr);
    if c ≤ 9 then print_char(c + "0")
    else begin print_char("!"); return;
      end;
    end;
  match: begin match_chr ← c; print_char(c); incr(n); print_char(n);
    if n > "9" then return;
    end;
  end_match: if c = 0 then print("->");
    othercases print_esc("BAD.")
  endcases
```

This code is used in section 323.

325. Here's the way we sometimes want to display a token list, given a pointer to its reference count; the pointer may be null.

```
procedure token_show(p : pointer);
  begin if p ≠ null then show_token_list(link(p), null, 10000000);
  end;
```

326. The *print_meaning* subroutine displays *cur_cmd* and *cur_chr* in symbolic form, including the expansion of a macro or mark.

```
procedure print_meaning;
  begin print_cmd_chr(cur_cmd, cur_chr);
  if cur_cmd ≥ call then
    begin print_char(":"); print_ln; token_show(cur_chr);
    end
  else if (cur_cmd = top_bot_mark) ∧ (cur_chr < marks_code) then
      begin print_char(":"); print_ln; token_show(cur_mark[cur_chr]);
      end;
  end;
```

327. Introduction to the syntactic routines. Let's pause a moment now and try to look at the Big Picture. The TeX program consists of three main parts: syntactic routines, semantic routines, and output routines. The chief purpose of the syntactic routines is to deliver the user's input to the semantic routines, one token at a time. The semantic routines act as an interpreter responding to these tokens, which may be regarded as commands. And the output routines are periodically called on to convert box-and-glue lists into a compact set of instructions that will be sent to a typesetter. We have discussed the basic data structures and utility routines of TeX, so we are good and ready to plunge into the real activity by considering the syntactic routines.

Our current goal is to come to grips with the *get_next* procedure, which is the keystone of TeX's input mechanism. Each call of *get_next* sets the value of three variables *cur_cmd*, *cur_chr*, and *cur_cs*, representing the next input token.

> *cur_cmd* denotes a command code from the long list of codes given above;
> *cur_chr* denotes a character code or other modifier of the command code;
> *cur_cs* is the *eqtb* location of the current control sequence,
> if the current token was a control sequence, otherwise it's zero.

Underlying this external behavior of *get_next* is all the machinery necessary to convert from character files to tokens. At a given time we may be only partially finished with the reading of several files (for which `\input` was specified), and partially finished with the expansion of some user-defined macros and/or some macro parameters, and partially finished with the generation of some text in a template for `\halign`, and so on. When reading a character file, special characters must be classified as math delimiters, etc.; comments and extra blank spaces must be removed, paragraphs must be recognized, and control sequences must be found in the hash table. Furthermore there are occasions in which the scanning routines have looked ahead for a word like '`plus`' but only part of that word was found, hence a few characters must be put back into the input and scanned again.

To handle these situations, which might all be present simultaneously, TeX uses various stacks that hold information about the incomplete activities, and there is a finite state control for each level of the input mechanism. These stacks record the current state of an implicitly recursive process, but the *get_next* procedure is not recursive. Therefore it will not be difficult to translate these algorithms into low-level languages that do not support recursion.

```
⟨ Global variables 13 ⟩ +≡
cur_cmd: eight_bits; { current command set by get_next }
cur_chr: halfword; { operand of current command }
cur_cs: pointer; { control sequence found here, zero if none found }
cur_tok: halfword; { packed representative of cur_cmd and cur_chr }
```

328. The *print_cmd_chr* routine prints a symbolic interpretation of a command code and its modifier. This is used in certain '`You can't`' error messages, and in the implementation of diagnostic routines like `\show`.

The body of *print_cmd_chr* is a rather tedious listing of print commands, and most of it is essentially an inverse to the *primitive* routine that enters a TeX primitive into *eqtb*. Therefore much of this procedure appears elsewhere in the program, together with the corresponding *primitive* calls.

```
define chr_cmd(#) ≡
    begin print(#);
    if chr_code < "10000 then print_ASCII(chr_code)
    else print_char(chr_code); { non-Plane 0 Unicodes can't be sent through print_ASCII }
    end
```

⟨ Declare the procedure called *print_cmd_chr* 328 ⟩ ≡

```
procedure print_cmd_chr(cmd : quarterword; chr_code : halfword);
  var n: integer; { temp variable }
    font_name_str: str_number; { local vars for \fontname quoting extension }
    quote_char: UTF16_code;
  begin case cmd of
  left_brace: chr_cmd("begin-group␣character␣");
  right_brace: chr_cmd("end-group␣character␣");
  math_shift: chr_cmd("math␣shift␣character␣");
  mac_param: chr_cmd("macro␣parameter␣character␣");
  sup_mark: chr_cmd("superscript␣character␣");
  sub_mark: chr_cmd("subscript␣character␣");
  endv: print("end␣of␣alignment␣template");
  spacer: chr_cmd("blank␣space␣");
  letter: chr_cmd("the␣letter␣");
  other_char: chr_cmd("the␣character␣");
  ⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩
  othercases print("[unknown␣command␣code!]")
  endcases;
  end;
```

See also section 1457.

This code is used in section 278.

329. Here is a procedure that displays the current command.

```
procedure show_cur_cmd_chr;
  var n: integer;  { level of \if...\fi nesting }
    l: integer;  { line where \if started }
    p: pointer;
  begin begin_diagnostic; print_nl("{");
  if mode ≠ shown_mode then
    begin print_mode(mode); print(": "); shown_mode ← mode;
    end;
  print_cmd_chr(cur_cmd, cur_chr);
  if tracing_ifs > 0 then
    if cur_cmd ≥ if_test then
      if cur_cmd ≤ fi_or_else then
        begin print(": ");
        if cur_cmd = fi_or_else then
          begin print_cmd_chr(if_test, cur_if); print_char(" "); n ← 0; l ← if_line;
          end
        else begin n ← 1; l ← line;
          end;
        p ← cond_ptr;
        while p ≠ null do
          begin incr(n); p ← link(p);
          end;
        print("(level "); print_int(n); print_char(")"); print_if_line(l);
        end;
  print_char("}"); end_diagnostic(false);
  end;
```

330. Input stacks and states. This implementation of TEX uses two different conventions for representing sequential stacks.

1) If there is frequent access to the top entry, and if the stack is essentially never empty, then the top entry is kept in a global variable (even better would be a machine register), and the other entries appear in the array *stack*[0 .. (*ptr* − 1)]. For example, the semantic stack described above is handled this way, and so is the input stack that we are about to study.

2) If there is infrequent top access, the entire stack contents are in the array *stack*[0 .. (*ptr* − 1)]. For example, the *save_stack* is treated this way, as we have seen.

The state of TEX's input mechanism appears in the input stack, whose entries are records with six fields, called *state*, *index*, *start*, *loc*, *limit*, and *name*. This stack is maintained with convention (1), so it is declared in the following way:

```
⟨ Types in the outer block 18 ⟩ +≡
  in_state_record = record state_field, index_field: quarterword;
    start_field, loc_field, limit_field, name_field: halfword;
    end;
```

331.
```
⟨ Global variables 13 ⟩ +≡
input_stack: array [0 .. stack_size] of in_state_record;
input_ptr: 0 .. stack_size; { first unused location of input_stack }
max_in_stack: 0 .. stack_size; { largest value of input_ptr when pushing }
cur_input: in_state_record; { the "top" input state, according to convention (1) }
```

332. We've already defined the special variable *loc* ≡ *cur_input.loc_field* in our discussion of basic input-output routines. The other components of *cur_input* are defined in the same way:

```
define state ≡ cur_input.state_field { current scanner state }
define index ≡ cur_input.index_field { reference for buffer information }
define start ≡ cur_input.start_field { starting position in buffer }
define limit ≡ cur_input.limit_field { end of current line in buffer }
define name ≡ cur_input.name_field { name of the current file }
```

333. Let's look more closely now at the control variables (*state*, *index*, *start*, *loc*, *limit*, *name*), assuming that TeX is reading a line of characters that have been input from some file or from the user's terminal. There is an array called *buffer* that acts as a stack of all lines of characters that are currently being read from files, including all lines on subsidiary levels of the input stack that are not yet completed. TeX will return to the other lines when it is finished with the present input file.

(Incidentally, on a machine with byte-oriented addressing, it might be appropriate to combine *buffer* with the *str_pool* array, letting the buffer entries grow downward from the top of the string pool and checking that these two tables don't bump into each other.)

The line we are currently working on begins in position *start* of the buffer; the next character we are about to read is *buffer*[*loc*]; and *limit* is the location of the last character present. If $loc > limit$, the line has been completely read. Usually *buffer*[*limit*] is the *end_line_char*, denoting the end of a line, but this is not true if the current line is an insertion that was entered on the user's terminal in response to an error message.

The *name* variable is a string number that designates the name of the current file, if we are reading a text file. It is zero if we are reading from the terminal; it is $n + 1$ if we are reading from input stream n, where $0 \le n \le 16$. (Input stream 16 stands for an invalid stream number; in such cases the input is actually from the terminal, under control of the procedure *read_toks*.) Finally $18 \le name \le 19$ indicates that we are reading a pseudo file created by the `\scantokens` command.

The *state* variable has one of three values, when we are scanning such files:

1) *state* = *mid_line* is the normal state.
2) *state* = *skip_blanks* is like *mid_line*, but blanks are ignored.
3) *state* = *new_line* is the state at the beginning of a line.

These state values are assigned numeric codes so that if we add the state code to the next character's command code, we get distinct values. For example, '*mid_line* + *spacer*' stands for the case that a blank space character occurs in the middle of a line when it is not being ignored; after this case is processed, the next value of *state* will be *skip_blanks*.

define *mid_line* = 1 { *state* code when scanning a line of characters }
define *skip_blanks* = 2 + *max_char_code* { *state* code when ignoring blanks }
define *new_line* = 3 + *max_char_code* + *max_char_code* { *state* code at start of line }

334. Additional information about the current line is available via the *index* variable, which counts how many lines of characters are present in the buffer below the current level. We have $index = 0$ when reading from the terminal and prompting the user for each line; then if the user types, e.g., '`\input paper`', we will have $index = 1$ while reading the file `paper.tex`. However, it does not follow that *index* is the same as the input stack pointer, since many of the levels on the input stack may come from token lists. For example, the instruction '`\input paper`' might occur in a token list.

The global variable *in_open* is equal to the *index* value of the highest non-token-list level. Thus, the number of partially read lines in the buffer is $in_open + 1$, and we have $in_open = index$ when we are not reading a token list.

If we are not currently reading from the terminal, or from an input stream, we are reading from the file variable *input_file*[*index*]. We use the notation *terminal_input* as a convenient abbreviation for $name = 0$, and *cur_file* as an abbreviation for *input_file*[*index*].

The global variable *line* contains the line number in the topmost open file, for use in error messages. If we are not reading from the terminal, *line_stack*[*index*] holds the line number for the enclosing level, so that *line* can be restored when the current file has been read. Line numbers should never be negative, since the negative of the current line number is used to identify the user's output routine in the *mode_line* field of the semantic nest entries.

If more information about the input state is needed, it can be included in small arrays like those shown here. For example, the current page or segment number in the input file might be put into a variable *page*, maintained for enclosing levels in '*page_stack*: **array** [1 .. *max_in_open*] **of** *integer*' by analogy with *line_stack*.

```
define terminal_input ≡ (name = 0)   { are we reading from the terminal? }
define cur_file ≡ input_file[index]   { the current alpha_file variable }
```

⟨ Global variables 13 ⟩ +≡

```
in_open: 0 .. max_in_open;   { the number of lines in the buffer, less one }
open_parens: 0 .. max_in_open;   { the number of open text files }
input_file: array [1 .. max_in_open] of alpha_file;
line: integer;   { current line number in the current source file }
line_stack: array [1 .. max_in_open] of integer;
```

335. Users of TEX sometimes forget to balance left and right braces properly, and one of the ways TEX tries to spot such errors is by considering an input file as broken into subfiles by control sequences that are declared to be `\outer`.

A variable called *scanner_status* tells TEX whether or not to complain when a subfile ends. This variable has six possible values:

normal, means that a subfile can safely end here without incident.

skipping, means that a subfile can safely end here, but not a file, because we're reading past some conditional text that was not selected.

defining, means that a subfile shouldn't end now because a macro is being defined.

matching, means that a subfile shouldn't end now because a macro is being used and we are searching for the end of its arguments.

aligning, means that a subfile shouldn't end now because we are not finished with the preamble of an `\halign` or `\valign`.

absorbing, means that a subfile shouldn't end now because we are reading a balanced token list for `\message`, `\write`, etc.

If the *scanner_status* is not *normal*, the variable *warning_index* points to the *eqtb* location for the relevant control sequence name to print in an error message.

```
define skipping = 1  { scanner_status when passing conditional text }
define defining = 2  { scanner_status when reading a macro definition }
define matching = 3  { scanner_status when reading macro arguments }
define aligning = 4  { scanner_status when reading an alignment preamble }
define absorbing = 5  { scanner_status when reading a balanced text }
⟨ Global variables 13 ⟩ +≡
scanner_status: normal .. absorbing;  { can a subfile end now? }
warning_index: pointer;  { identifier relevant to non-normal scanner status }
def_ref: pointer;  { reference count of token list being defined }
```

336. Here is a procedure that uses *scanner_status* to print a warning message when a subfile has ended, and at certain other crucial times:

```
⟨ Declare the procedure called runaway 336 ⟩ ≡
procedure runaway;
  var p: pointer;  { head of runaway list }
  begin if scanner_status > skipping then
    begin print_nl("Runaway␣");
    case scanner_status of
    defining: begin print("definition"); p ← def_ref;
      end;
    matching: begin print("argument"); p ← temp_head;
      end;
    aligning: begin print("preamble"); p ← hold_head;
      end;
    absorbing: begin print("text"); p ← def_ref;
      end;
    end;  { there are no other cases }
    print_char("?"); print_ln; show_token_list(link(p), null, error_line − 10);
    end;
  end;
```

This code is used in section 141.

337. However, all this discussion about input state really applies only to the case that we are inputting from a file. There is another important case, namely when we are currently getting input from a token list. In this case *state* = *token_list*, and the conventions about the other state variables are different:

loc is a pointer to the current node in the token list, i.e., the node that will be read next. If *loc* = *null*, the token list has been fully read.

start points to the first node of the token list; this node may or may not contain a reference count, depending on the type of token list involved.

token_type, which takes the place of *index* in the discussion above, is a code number that explains what kind of token list is being scanned.

name points to the *eqtb* address of the control sequence being expanded, if the current token list is a macro.

param_start, which takes the place of *limit*, tells where the parameters of the current macro begin in the *param_stack*, if the current token list is a macro.

The *token_type* can take several values, depending on where the current token list came from:

parameter, if a parameter is being scanned;
u_template, if the $\langle u_j \rangle$ part of an alignment template is being scanned;
v_template, if the $\langle v_j \rangle$ part of an alignment template is being scanned;
backed_up, if the token list being scanned has been inserted as 'to be read again';
inserted, if the token list being scanned has been inserted as the text expansion of a `\count` or similar variable;
macro, if a user-defined control sequence is being scanned;
output_text, if an `\output` routine is being scanned;
every_par_text, if the text of `\everypar` is being scanned;
every_math_text, if the text of `\everymath` is being scanned;
every_display_text, if the text of `\everydisplay` is being scanned;
every_hbox_text, if the text of `\everyhbox` is being scanned;
every_vbox_text, if the text of `\everyvbox` is being scanned;
every_job_text, if the text of `\everyjob` is being scanned;
every_cr_text, if the text of `\everycr` is being scanned;
mark_text, if the text of a `\mark` is being scanned;
write_text, if the text of a `\write` is being scanned.

The codes for *output_text*, *every_par_text*, etc., are equal to a constant plus the corresponding codes for token list parameters *output_routine_loc*, *every_par_loc*, etc. The token list begins with a reference count if and only if *token_type* ≥ *macro*.

Since ε-TeX's additional token list parameters precede *toks_base*, the corresponding token types must precede *write_text*.

define *token_list* = 0 { *state* code when scanning a token list }
define *token_type* ≡ *index* { type of current token list }
define *param_start* ≡ *limit* { base of macro parameters in *param_stack* }
define *parameter* = 0 { *token_type* code for parameter }
define *u_template* = 1 { *token_type* code for $\langle u_j \rangle$ template }
define *v_template* = 2 { *token_type* code for $\langle v_j \rangle$ template }
define *backed_up* = 3 { *token_type* code for text to be reread }
define *backed_up_char* = 4 { special code for backed-up char from XeTeXinterchartoks hook }
define *inserted* = 5 { *token_type* code for inserted texts }
define *macro* = 6 { *token_type* code for defined control sequences }
define *output_text* = 7 { *token_type* code for output routines }
define *every_par_text* = 8 { *token_type* code for `\everypar` }
define *every_math_text* = 9 { *token_type* code for `\everymath` }
define *every_display_text* = 10 { *token_type* code for `\everydisplay` }
define *every_hbox_text* = 11 { *token_type* code for `\everyhbox` }

define *every_vbox_text* = 12 { *token_type* code for `\everyvbox` }
define *every_job_text* = 13 { *token_type* code for `\everyjob` }
define *every_cr_text* = 14 { *token_type* code for `\everycr` }
define *mark_text* = 15 { *token_type* code for `\topmark`, etc. }
define *eTeX_text_offset* = *output_routine_loc* − *output_text*
define *every_eof_text* = *every_eof_loc* − *eTeX_text_offset* { *token_type* code for `\everyeof` }
define *inter_char_text* = *XeTeX_inter_char_loc* − *eTeX_text_offset*
{ *token_type* code for `\XeTeXinterchartoks` }
define *write_text* = *toks_base* − *eTeX_text_offset* { *token_type* code for `\write` }

338. The *param_stack* is an auxiliary array used to hold pointers to the token lists for parameters at the current level and subsidiary levels of input. This stack is maintained with convention (2), and it grows at a different rate from the others.

⟨ Global variables 13 ⟩ +≡
param_stack: **array** [0 .. *param_size*] **of** *pointer*; { token list pointers for parameters }
param_ptr: 0 .. *param_size*; { first unused entry in *param_stack* }
max_param_stack: *integer*; { largest value of *param_ptr*, will be ≤ *param_size* + 9 }

339. The input routines must also interact with the processing of `\halign` and `\valign`, since the appearance of tab marks and `\cr` in certain places is supposed to trigger the beginning of special ⟨v_j⟩ template text in the scanner. This magic is accomplished by an *align_state* variable that is increased by 1 when a '`{`' is scanned and decreased by 1 when a '`}`' is scanned. The *align_state* is nonzero during the ⟨u_j⟩ template, after which it is set to zero; the ⟨v_j⟩ template begins when a tab mark or `\cr` occurs at a time that *align_state* = 0.

⟨ Global variables 13 ⟩ +≡
align_state: *integer*; { group level with respect to current alignment }

340. Thus, the "current input state" can be very complicated indeed; there can be many levels and each level can arise in a variety of ways. The *show_context* procedure, which is used by TeX's error-reporting routine to print out the current input state on all levels down to the most recent line of characters from an input file, illustrates most of these conventions. The global variable *base_ptr* contains the lowest level that was displayed by this procedure.

⟨ Global variables 13 ⟩ +≡
base_ptr: 0 .. *stack_size*; { shallowest level shown by *show_context* }

341. The status at each level is indicated by printing two lines, where the first line indicates what was read so far and the second line shows what remains to be read. The context is cropped, if necessary, so that the first line contains at most *half_error_line* characters, and the second contains at most *error_line*. Non-current input levels whose *token_type* is '*backed_up*' are shown only if they have not been fully read.

```
procedure show_context; { prints where the scanner is }
  label done;
  var old_setting: 0 .. max_selector; { saved selector setting }
    nn: integer; { number of contexts shown so far, less one }
    bottom_line: boolean; { have we reached the final context to be shown? }
    ⟨ Local variables for formatting calculations 345 ⟩
  begin base_ptr ← input_ptr; input_stack[base_ptr] ← cur_input; { store current state }
  nn ← −1; bottom_line ← false;
  loop begin cur_input ← input_stack[base_ptr]; { enter into the context }
    if (state ≠ token_list) then
      if (name > 19) ∨ (base_ptr = 0) then bottom_line ← true;
    if (base_ptr = input_ptr) ∨ bottom_line ∨ (nn < error_context_lines) then
      ⟨ Display the current context 342 ⟩
    else if nn = error_context_lines then
        begin print_nl("..."); incr(nn); { omitted if error_context_lines < 0 }
        end;
    if bottom_line then goto done;
    decr(base_ptr);
    end;
done: cur_input ← input_stack[input_ptr]; { restore original state }
  end;
```

342. ⟨ Display the current context 342 ⟩ ≡

```
  begin if (base_ptr = input_ptr) ∨ (state ≠ token_list) ∨ (token_type ≠ backed_up) ∨ (loc ≠ null) then
        { we omit backed-up token lists that have already been read }
    begin tally ← 0; { get ready to count characters }
    old_setting ← selector;
    if state ≠ token_list then
      begin ⟨ Print location of current line 343 ⟩;
      ⟨ Pseudoprint the line 348 ⟩;
      end
    else begin ⟨ Print type of token list 344 ⟩;
      ⟨ Pseudoprint the token list 349 ⟩;
      end;
    selector ← old_setting; { stop pseudoprinting }
    ⟨ Print two lines using the tricky pseudoprinted information 347 ⟩;
    incr(nn);
    end;
  end
```

This code is used in section 341.

343. This routine should be changed, if necessary, to give the best possible indication of where the current line resides in the input file. For example, on some systems it is best to print both a page and line number.

```
⟨ Print location of current line 343 ⟩ ≡
  if name ≤ 17 then
    if terminal_input then
      if base_ptr = 0 then print_nl("<*>")
      else print_nl("<insert> ")
    else begin print_nl("<read ");
      if name = 17 then print_char("*") else print_int(name − 1);
      print_char(">");
      end
  else begin print_nl("l.");
    if index = in_open then print_int(line)
    else print_int(line_stack[index + 1]); { input from a pseudo file }
    end;
  print_char(" ")
```

This code is used in section 342.

344.

```
⟨ Print type of token list 344 ⟩ ≡
  case token_type of
  parameter: print_nl("<argument> ");
  u_template, v_template: print_nl("<template> ");
  backed_up, backed_up_char: if loc = null then print_nl("<recently read> ")
    else print_nl("<to be read again> ");
  inserted: print_nl("<inserted text> ");
  macro: begin print_ln; print_cs(name);
    end;
  output_text: print_nl("<output> ");
  every_par_text: print_nl("<everypar> ");
  every_math_text: print_nl("<everymath> ");
  every_display_text: print_nl("<everydisplay> ");
  every_hbox_text: print_nl("<everyhbox> ");
  every_vbox_text: print_nl("<everyvbox> ");
  every_job_text: print_nl("<everyjob> ");
  every_cr_text: print_nl("<everycr> ");
  mark_text: print_nl("<mark> ");
  every_eof_text: print_nl("<everyeof> ");
  inter_char_text: print_nl("<XeTeXinterchartoks> ");
  write_text: print_nl("<write> ");
  othercases print_nl("?") { this should never happen }
  endcases
```

This code is used in section 342.

345. Here it is necessary to explain a little trick. We don't want to store a long string that corresponds to a token list, because that string might take up lots of memory; and we are printing during a time when an error message is being given, so we dare not do anything that might overflow one of TeX's tables. So 'pseudoprinting' is the answer: We enter a mode of printing that stores characters into a buffer of length *error_line*, where character $k+1$ is placed into *trick_buf*$[k \bmod$ *error_line*$]$ if $k <$ *trick_count*, otherwise character k is dropped. Initially we set *tally* $\leftarrow 0$ and *trick_count* $\leftarrow 1000000$; then when we reach the point where transition from line 1 to line 2 should occur, we set *first_count* $\leftarrow$ *tally* and *trick_count* $\leftarrow \max($*error_line*, *tally* $+1+$ *error_line* $-$ *half_error_line*$)$. At the end of the pseudoprinting, the values of *first_count*, *tally*, and *trick_count* give us all the information we need to print the two lines, and all of the necessary text is in *trick_buf*.

Namely, let l be the length of the descriptive information that appears on the first line. The length of the context information gathered for that line is $k =$ *first_count*, and the length of the context information gathered for line 2 is $m = \min($*tally*, *trick_count*$) - k$. If $l + k \le h$, where $h =$ *half_error_line*, we print *trick_buf*$[0 \,..\, k-1]$ after the descriptive information on line 1, and set $n \leftarrow l + k$; here n is the length of line 1. If $l + k > h$, some cropping is necessary, so we set $n \leftarrow h$ and print '...' followed by

$$\textit{trick_buf}[(l + k - h + 3) \,..\, k - 1],$$

where subscripts of *trick_buf* are circular modulo *error_line*. The second line consists of n spaces followed by *trick_buf*$[k \,..\, (k + m - 1)]$, unless $n + m >$ *error_line*; in the latter case, further cropping is done. This is easier to program than to explain.

⟨ Local variables for formatting calculations 345 ⟩ ≡
i: 0 .. *buf_size*; { index into *buffer* }
j: 0 .. *buf_size*; { end of current line in *buffer* }
l: 0 .. *half_error_line*; { length of descriptive information on line 1 }
m: *integer*; { context information gathered for line 2 }
n: 0 .. *error_line*; { length of line 1 }
p: *integer*; { starting or ending place in *trick_buf* }
q: *integer*; { temporary index }
This code is used in section 341.

346. The following code sets up the print routines so that they will gather the desired information.

```
define begin_pseudoprint ≡
        begin l ← tally; tally ← 0; selector ← pseudo; trick_count ← 1000000;
        end
define set_trick_count ≡
        begin first_count ← tally; trick_count ← tally + 1 + error_line − half_error_line;
        if trick_count < error_line then trick_count ← error_line;
        end
```

347. And the following code uses the information after it has been gathered.

```
⟨ Print two lines using the tricky pseudoprinted information 347 ⟩ ≡
  if trick_count = 1000000 then set_trick_count; { set_trick_count must be performed }
  if tally < trick_count then m ← tally − first_count
  else m ← trick_count − first_count; { context on line 2 }
  if l + first_count ≤ half_error_line then
    begin p ← 0; n ← l + first_count;
    end
  else begin print("..."); p ← l + first_count − half_error_line + 3; n ← half_error_line;
    end;
  for q ← p to first_count − 1 do print_char(trick_buf[q mod error_line]);
  print_ln;
  for q ← 1 to n do print_visible_char("␣"); { print n spaces to begin line 2 }
  if m + n ≤ error_line then p ← first_count + m
  else p ← first_count + (error_line − n − 3);
  for q ← first_count to p − 1 do print_char(trick_buf[q mod error_line]);
  if m + n > error_line then print("...")
```

This code is used in section 342.

348. But the trick is distracting us from our current goal, which is to understand the input state. So let's concentrate on the data structures that are being pseudoprinted as we finish up the *show_context* procedure.

```
⟨ Pseudoprint the line 348 ⟩ ≡
  begin_pseudoprint;
  if buffer[limit] = end_line_char then j ← limit
  else j ← limit + 1; { determine the effective end of the line }
  if j > 0 then
    for i ← start to j − 1 do
      begin if i = loc then set_trick_count;
      print_char(buffer[i]);
      end
```

This code is used in section 342.

349.

```
⟨ Pseudoprint the token list 349 ⟩ ≡
  begin_pseudoprint;
  if token_type < macro then show_token_list(start, loc, 100000)
  else show_token_list(link(start), loc, 100000) { avoid reference count }
```

This code is used in section 342.

350. Here is the missing piece of *show_token_list* that is activated when the token beginning line 2 is about to be shown:

```
⟨ Do magic computation 350 ⟩ ≡
  set_trick_count
```

This code is used in section 322.

351. Maintaining the input stacks. The following subroutines change the input status in commonly needed ways.

First comes *push_input*, which stores the current state and creates a new level (having, initially, the same properties as the old).

```
define push_input ≡    { enter a new input level, save the old }
    begin if input_ptr > max_in_stack then
      begin max_in_stack ← input_ptr;
      if input_ptr = stack_size then overflow("input␣stack␣size", stack_size);
      end;
    input_stack[input_ptr] ← cur_input;    { stack the record }
    incr(input_ptr);
    end
```

352. And of course what goes up must come down.

```
define pop_input ≡    { leave an input level, re-enter the old }
    begin decr(input_ptr); cur_input ← input_stack[input_ptr];
    end
```

353. Here is a procedure that starts a new level of token-list input, given a token list p and its type t. If $t = macro$, the calling routine should set *name* and *loc*.

```
define back_list(#) ≡ begin_token_list(#, backed_up)    { backs up a simple token list }
define ins_list(#) ≡ begin_token_list(#, inserted)    { inserts a simple token list }
procedure begin_token_list(p : pointer; t : quarterword);
  begin push_input; state ← token_list; start ← p; token_type ← t;
  if t ≥ macro then    { the token list starts with a reference count }
    begin add_token_ref(p);
    if t = macro then param_start ← param_ptr
    else begin loc ← link(p);
      if tracing_macros > 1 then
        begin begin_diagnostic; print_nl("");
        case t of
        mark_text: print_esc("mark");
        write_text: print_esc("write");
        othercases print_cmd_chr(assign_toks, t − output_text + output_routine_loc)
        endcases;
        print("->"); token_show(p); end_diagnostic(false);
        end;
      end;
    end
  else loc ← p;
  end;
```

354. When a token list has been fully scanned, the following computations should be done as we leave that level of input. The *token_type* tends to be equal to either *backed_up* or *inserted* about 2/3 of the time.

```
procedure end_token_list; { leave a token-list input level }
  begin if token_type ≥ backed_up then { token list to be deleted }
    begin if token_type ≤ inserted then flush_list(start)
    else begin delete_token_ref(start); { update reference count }
      if token_type = macro then { parameters must be flushed }
        while param_ptr > param_start do
          begin decr(param_ptr); flush_list(param_stack[param_ptr]);
          end;
      end;
    end
  else if token_type = u_template then
      if align_state > 500000 then align_state ← 0
      else fatal_error("(interwoven␣alignment␣preambles␣are␣not␣allowed)");
  pop_input; check_interrupt;
  end;
```

355. Sometimes TEX has read too far and wants to "unscan" what it has seen. The *back_input* procedure takes care of this by putting the token just scanned back into the input stream, ready to be read again. This procedure can be used only if *cur_tok* represents the token to be replaced. Some applications of TEX use this procedure a lot, so it has been slightly optimized for speed.

```
procedure back_input; { undoes one token of input }
  var p: pointer; { a token list of length one }
  begin while (state = token_list) ∧ (loc = null) ∧ (token_type ≠ v_template) do end_token_list;
        { conserve stack space }
  p ← get_avail; info(p) ← cur_tok;
  if cur_tok < right_brace_limit then
    if cur_tok < left_brace_limit then decr(align_state)
    else incr(align_state);
  push_input; state ← token_list; start ← p; token_type ← backed_up; loc ← p;
      { that was back_list(p), without procedure overhead }
  end;
```

356. ⟨ Insert token p into TEX's input 356 ⟩ ≡

```
  begin t ← cur_tok; cur_tok ← p;
  if a then
    begin p ← get_avail; info(p) ← cur_tok; link(p) ← loc; loc ← p; start ← p;
    if cur_tok < right_brace_limit then
      if cur_tok < left_brace_limit then decr(align_state)
      else incr(align_state);
    end
  else begin back_input; a ← eTeX_ex;
    end;
  cur_tok ← t;
  end
```

This code is used in section 312.

357. The *back_error* routine is used when we want to replace an offending token just before issuing an error message. This routine, like *back_input*, requires that *cur_tok* has been set. We disable interrupts during the call of *back_input* so that the help message won't be lost.

```
procedure back_error; { back up one token and call error }
  begin OK_to_interrupt ← false; back_input; OK_to_interrupt ← true; error;
  end;
procedure ins_error; { back up one inserted token and call error }
  begin OK_to_interrupt ← false; back_input; token_type ← inserted; OK_to_interrupt ← true; error;
  end;
```

358. The *begin_file_reading* procedure starts a new level of input for lines of characters to be read from a file, or as an insertion from the terminal. It does not take care of opening the file, nor does it set *loc* or *limit* or *line*.

```
procedure begin_file_reading;
  begin if in_open = max_in_open then overflow("text␣input␣levels", max_in_open);
  if first = buf_size then overflow("buffer␣size", buf_size);
  incr(in_open); push_input; index ← in_open; eof_seen[index] ← false;
  grp_stack[index] ← cur_boundary; if_stack[index] ← cond_ptr; line_stack[index] ← line; start ← first;
  state ← mid_line; name ← 0; { terminal_input is now true }
  end;
```

359. Conversely, the variables must be downdated when such a level of input is finished:

```
procedure end_file_reading;
  begin first ← start; line ← line_stack[index];
  if (name = 18) ∨ (name = 19) then pseudo_close
  else if name > 17 then u_close(cur_file); { forget it }
  pop_input; decr(in_open);
  end;
```

360. In order to keep the stack from overflowing during a long sequence of inserted '`\show`' commands, the following routine removes completed error-inserted lines from memory.

```
procedure clear_for_error_prompt;
  begin while (state ≠ token_list) ∧ terminal_input ∧ (input_ptr > 0) ∧ (loc > limit) do end_file_reading;
  print_ln; clear_terminal;
  end;
```

361. To get TeX's whole input mechanism going, we perform the following actions.

```
⟨ Initialize the input routines 361 ⟩ ≡
  begin input_ptr ← 0; max_in_stack ← 0; in_open ← 0; open_parens ← 0; max_buf_stack ← 0;
  grp_stack[0] ← 0; if_stack[0] ← null; param_ptr ← 0; max_param_stack ← 0; first ← buf_size;
  repeat buffer[first] ← 0; decr(first);
  until first = 0;
  scanner_status ← normal; warning_index ← null; first ← 1; state ← new_line; start ← 1; index ← 0;
  line ← 0; name ← 0; force_eof ← false; align_state ← 1000000;
  if ¬init_terminal then goto final_end;
  limit ← last; first ← last + 1; { init_terminal has set loc and last }
  end
```

This code is used in section 1391.

362. Getting the next token. The heart of TeX's input mechanism is the *get_next* procedure, which we shall develop in the next few sections of the program. Perhaps we shouldn't actually call it the "heart," however, because it really acts as TeX's eyes and mouth, reading the source files and gobbling them up. And it also helps TeX to regurgitate stored token lists that are to be processed again.

The main duty of *get_next* is to input one token and to set *cur_cmd* and *cur_chr* to that token's command code and modifier. Furthermore, if the input token is a control sequence, the *eqtb* location of that control sequence is stored in *cur_cs*; otherwise *cur_cs* is set to zero.

Underlying this simple description is a certain amount of complexity because of all the cases that need to be handled. However, the inner loop of *get_next* is reasonably short and fast.

When *get_next* is asked to get the next token of a `\read` line, it sets $cur_cmd = cur_chr = cur_cs = 0$ in the case that no more tokens appear on that line. (There might not be any tokens at all, if the *end_line_char* has *ignore* as its catcode.)

363. The value of *par_loc* is the *eqtb* address of '`\par`'. This quantity is needed because a blank line of input is supposed to be exactly equivalent to the appearance of `\par`; we must set $cur_cs \leftarrow par_loc$ when detecting a blank line.

```
⟨ Global variables 13 ⟩ +≡
par_loc: pointer; { location of '\par' in eqtb }
par_token: halfword; { token representing '\par' }
```

364.
```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("par", par_end, too_big_usv); { cf. scan_file_name }
  par_loc ← cur_val; par_token ← cs_token_flag + par_loc;
```

365.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
par_end: print_esc("par");
```

366. Before getting into *get_next*, let's consider the subroutine that is called when an '`\outer`' control sequence has been scanned or when the end of a file has been reached. These two cases are distinguished by *cur_cs*, which is zero at the end of a file.

```
procedure check_outer_validity;
  var p: pointer; { points to inserted token list }
    q: pointer; { auxiliary pointer }
  begin if scanner_status ≠ normal then
    begin deletions_allowed ← false; ⟨ Back up an outer control sequence so that it can be reread 367 ⟩;
    if scanner_status > skipping then ⟨ Tell the user what has run away and try to recover 368 ⟩
    else begin print_err("Incomplete␣"); print_cmd_chr(if_test, cur_if);
      print("␣;␣all␣text␣was␣ignored␣after␣line␣"); print_int(skip_line);
      help3("A␣forbidden␣control␣sequence␣occurred␣in␣skipped␣text.")
      ("This␣kind␣of␣error␣happens␣when␣you␣say␣`\if...´␣and␣forget")
      ("the␣matching␣`\fi´␣I´ve␣inserted␣a␣`\fi´;␣this␣might␣work.");
      if cur_cs ≠ 0 then cur_cs ← 0
      else help_line[2] ← "The␣file␣ended␣while␣I␣was␣skipping␣conditional␣text.";
      cur_tok ← cs_token_flag + frozen_fi; ins_error;
      end;
    deletions_allowed ← true;
    end;
  end;
```

367. An outer control sequence that occurs in a `\read` will not be reread, since the error recovery for `\read` is not very powerful.

```
⟨ Back up an outer control sequence so that it can be reread 367 ⟩ ≡
  if cur_cs ≠ 0 then
    begin if (state = token_list) ∨ (name < 1) ∨ (name > 17) then
      begin p ← get_avail; info(p) ← cs_token_flag + cur_cs; back_list(p);
          { prepare to read the control sequence again }
      end;
    cur_cmd ← spacer; cur_chr ← " "; { replace it by a space }
    end
```

This code is used in section 366.

368.

```
⟨ Tell the user what has run away and try to recover 368 ⟩ ≡
  begin runaway; { print a definition, argument, or preamble }
  if cur_cs = 0 then print_err("File ended")
  else begin cur_cs ← 0; print_err("Forbidden control sequence found");
    end;
  print(" while scanning "); ⟨ Print either 'definition' or 'use' or 'preamble' or 'text', and insert
      tokens that should lead to recovery 369 ⟩;
  print(" of "); sprint_cs(warning_index);
  help4("I suspect you have forgotten a `}', causing me")
  ("to read past where you wanted me to stop.")
  ("I'll try to recover; but if the error is serious,")
  ("you'd better type `E' or `X' now and fix your file.");
  error;
  end
```

This code is used in section 366.

369. The recovery procedure can't be fully understood without knowing more about the TeX routines that should be aborted, but we can sketch the ideas here: For a runaway definition or a runaway balanced text we will insert a right brace; for a runaway preamble, we will insert a special `\cr` token and a right brace; and for a runaway argument, we will set *long_state* to *outer_call* and insert `\par`.

```
⟨ Print either 'definition' or 'use' or 'preamble' or 'text', and insert tokens that should lead to
      recovery 369 ⟩ ≡
  p ← get_avail;
  case scanner_status of
  defining: begin print("definition"); info(p) ← right_brace_token + "}";
    end;
  matching: begin print("use"); info(p) ← par_token; long_state ← outer_call;
    end;
  aligning: begin print("preamble"); info(p) ← right_brace_token + "}"; q ← p; p ← get_avail;
    link(p) ← q; info(p) ← cs_token_flag + frozen_cr; align_state ← −1000000;
    end;
  absorbing: begin print("text"); info(p) ← right_brace_token + "}";
    end;
  end; { there are no other cases }
  ins_list(p)
```

This code is used in section 368.

370. We need to mention a procedure here that may be called by *get_next*.

```
procedure firm_up_the_line; forward;
```

371. Now we're ready to take the plunge into *get_next* itself. Parts of this routine are executed more often than any other instructions of TeX.

```
define switch = 25  { a label in get_next }
define start_cs = 26  { another }
define not_exp = 27
procedure get_next;  { sets cur_cmd, cur_chr, cur_cs to next token }
  label restart,  { go here to get the next input token }
    switch,  { go here to eat the next character from a file }
    reswitch,  { go here to digest it again }
    start_cs,  { go here to start looking for a control sequence }
    found,  { go here when a control sequence has been found }
    not_exp,  { go here when 'turned out not to start an expanded code }
    exit;  { go here when the next input token has been got }
  var k: 0 .. buf_size;  { an index into buffer }
    t: halfword;  { a token }
    cat: 0 .. max_char_code;  { cat_code(cur_chr), usually }
    c: UnicodeScalar;  { constituent of a possible expanded code }
    lower: UTF16_code;  { lower surrogate of a possible UTF-16 compound }
    d: small_number;  { number of excess characters in an expanded code }
    sup_count: small_number;  { number of identical sup_mark characters }
  begin restart: cur_cs ← 0;
  if state ≠ token_list then ⟨ Input from external file, goto restart if no input found 373 ⟩
  else ⟨ Input from token list, goto restart if end of list or if a parameter needs to be expanded 387 ⟩;
  ⟨ If an alignment entry has just ended, take appropriate action 372 ⟩;
exit: end;
```

372. An alignment entry ends when a tab or `\cr` occurs, provided that the current level of braces is the same as the level that was present at the beginning of that alignment entry; i.e., provided that *align_state* has returned to the value it had after the $\langle u_j \rangle$ template for that entry.

```
⟨ If an alignment entry has just ended, take appropriate action 372 ⟩ ≡
  if cur_cmd ≤ car_ret then
    if cur_cmd ≥ tab_mark then
      if align_state = 0 then ⟨ Insert the ⟨v_j⟩ template and goto restart 837 ⟩
```

This code is used in section 371.

373. ⟨ Input from external file, **goto** *restart* if no input found 373 ⟩ ≡
```
begin switch: if loc ≤ limit then { current line not yet finished }
  begin cur_chr ← buffer[loc]; incr(loc);
  if (cur_chr ≥ "D800) ∧ (cur_chr < "DC00) ∧ (loc ≤ limit) ∧ (buffer[loc] ≥ "DC00) ∧ (buffer[loc] < "E000)
      then
    begin lower ← buffer[loc] − "DC00; incr(loc); cur_chr ← "10000 + (cur_chr − "D800) * 1024 + lower;
    end;
  reswitch: cur_cmd ← cat_code(cur_chr); ⟨ Change state if necessary, and goto switch if the current
      character should be ignored, or goto reswitch if the current character changes to another 374 ⟩;
  end
else begin state ← new_line;
  ⟨ Move to next line of file, or goto restart if there is no next line, or return if a \read line has
      finished 390 ⟩;
  check_interrupt; goto switch;
  end;
end
```
This code is used in section 371.

374. The following 48-way switch accomplishes the scanning quickly, assuming that a decent Pascal compiler has translated the code. Note that the numeric values for *mid_line*, *skip_blanks*, and *new_line* are spaced apart from each other by *max_char_code* + 1, so we can add a character's command code to the state to get a single number that characterizes both.

```
define any_state_plus(#) ≡ mid_line + #, skip_blanks + #, new_line + #
```

⟨ Change state if necessary, and **goto** *switch* if the current character should be ignored, or **goto** *reswitch* if the current character changes to another 374 ⟩ ≡
```
case state + cur_cmd of
⟨ Cases where character is ignored 375 ⟩: goto switch;
any_state_plus(escape): ⟨ Scan a control sequence and set state ← skip_blanks or mid_line 384 ⟩;
any_state_plus(active_char): ⟨ Process an active-character control sequence and set state ← mid_line 383 ⟩;
any_state_plus(sup_mark): ⟨ If this sup_mark starts an expanded character like ^^A or ^^df, then goto
    reswitch, otherwise set state ← mid_line 382 ⟩;
any_state_plus(invalid_char): ⟨ Decry the invalid character and goto restart 376 ⟩;
⟨ Handle situations involving spaces, braces, changes of state 377 ⟩
othercases do_nothing
endcases
```
This code is used in section 373.

375. ⟨ Cases where character is ignored 375 ⟩ ≡
```
any_state_plus(ignore), skip_blanks + spacer, new_line + spacer
```
This code is used in section 374.

376. We go to *restart* instead of to *switch*, because *state* might equal *token_list* after the error has been dealt with (cf. *clear_for_error_prompt*).

⟨ Decry the invalid character and **goto** *restart* 376 ⟩ ≡
```
begin print_err("Text␣line␣contains␣an␣invalid␣character");
help2("A␣funny␣symbol␣that␣I␣can´t␣read␣has␣just␣been␣input.")
("Continue,␣and␣I´ll␣forget␣that␣it␣ever␣happened.");
deletions_allowed ← false; error; deletions_allowed ← true; goto restart;
end
```
This code is used in section 374.

377. **define** *add_delims_to*(#) ≡ # + *math_shift*, # + *tab_mark*, # + *mac_param*, # + *sub_mark*, # + *letter*, # + *other_char*

⟨ Handle situations involving spaces, braces, changes of state 377 ⟩ ≡
mid_line + *spacer*: ⟨ Enter *skip_blanks* state, emit a space 379 ⟩;
mid_line + *car_ret*: ⟨ Finish line, emit a space 378 ⟩;
skip_blanks + *car_ret*, *any_state_plus*(*comment*): ⟨ Finish line, **goto** *switch* 380 ⟩;
new_line + *car_ret*: ⟨ Finish line, emit a `\par` 381 ⟩;
mid_line + *left_brace*: *incr*(*align_state*);
skip_blanks + *left_brace*, *new_line* + *left_brace*: **begin** *state* ← *mid_line*; *incr*(*align_state*);
end;
mid_line + *right_brace*: *decr*(*align_state*);
skip_blanks + *right_brace*, *new_line* + *right_brace*: **begin** *state* ← *mid_line*; *decr*(*align_state*);
end;
add_delims_to(*skip_blanks*), *add_delims_to*(*new_line*): *state* ← *mid_line*;
This code is used in section 374.

378. When a character of type *spacer* gets through, its character code is changed to "␣" = ′40. This means that the ASCII codes for tab and space, and for the space inserted at the end of a line, will be treated alike when macro parameters are being matched. We do this since such characters are indistinguishable on most computer terminal displays.

⟨ Finish line, emit a space 378 ⟩ ≡
begin *loc* ← *limit* + 1; *cur_cmd* ← *spacer*; *cur_chr* ← "␣";
end
This code is used in section 377.

379. The following code is performed only when *cur_cmd* = *spacer*.

⟨ Enter *skip_blanks* state, emit a space 379 ⟩ ≡
begin *state* ← *skip_blanks*; *cur_chr* ← "␣";
end
This code is used in section 377.

380. ⟨ Finish line, **goto** *switch* 380 ⟩ ≡
begin *loc* ← *limit* + 1; **goto** *switch*;
end
This code is used in section 377.

381. ⟨ Finish line, emit a `\par` 381 ⟩ ≡
begin *loc* ← *limit* + 1; *cur_cs* ← *par_loc*; *cur_cmd* ← *eq_type*(*cur_cs*); *cur_chr* ← *equiv*(*cur_cs*);
if *cur_cmd* ≥ *outer_call* **then** *check_outer_validity*;
end
This code is used in section 377.

382. Notice that a code like `^^8` becomes `x` if not followed by a hex digit.

```
define is_hex(#) ≡ (((# ≥ "0") ∧ (# ≤ "9")) ∨ ((# ≥ "a") ∧ (# ≤ "f")))
define hex_to_cur_chr ≡
        if c ≤ "9" then cur_chr ← c − "0" else cur_chr ← c − "a" + 10;
      if cc ≤ "9" then cur_chr ← 16 ∗ cur_chr + cc − "0"
      else cur_chr ← 16 ∗ cur_chr + cc − "a" + 10
define long_hex_to_cur_chr ≡
        if c ≤ "9" then cur_chr ← c − "0" else cur_chr ← c − "a" + 10;
      if cc ≤ "9" then cur_chr ← 16 ∗ cur_chr + cc − "0"
      else cur_chr ← 16 ∗ cur_chr + cc − "a" + 10;
      if ccc ≤ "9" then cur_chr ← 16 ∗ cur_chr + ccc − "0"
      else cur_chr ← 16 ∗ cur_chr + ccc − "a" + 10;
      if cccc ≤ "9" then cur_chr ← 16 ∗ cur_chr + cccc − "0"
      else cur_chr ← 16 ∗ cur_chr + cccc − "a" + 10
⟨ If this sup_mark starts an expanded character like ^^A or ^^df, then goto reswitch, otherwise set
      state ← mid_line 382 ⟩ ≡
  begin if cur_chr = buffer[loc] then
    if loc < limit then
      begin sup_count ← 2;
          { we have ↑↑ and another char; check how many ↑s we have altogether, up to a max of 6 }
      while (sup_count < 6) ∧ (loc + 2 ∗ sup_count − 2 ≤ limit) ∧ (cur_chr = buffer[loc + sup_count − 1])
          do incr(sup_count);  { check whether we have enough hex chars for the number of ↑s }
      for d ← 1 to sup_count do
        if ¬is_hex(buffer[loc + sup_count − 2 + d]) then  { found a non-hex char, so do single ↑↑X style }
          begin c ← buffer[loc + 1];
          if c < ´200 then
            begin loc ← loc + 2;
            if c < ´100 then cur_chr ← c + ´100 else cur_chr ← c − ´100;
            goto reswitch;
            end;
          goto not_exp;
          end;  { there were the right number of hex chars, so convert them }
      cur_chr ← 0;
      for d ← 1 to sup_count do
        begin c ← buffer[loc + sup_count − 2 + d];
        if c ≤ "9" then cur_chr ← 16 ∗ cur_chr + c − "0"
        else cur_chr ← 16 ∗ cur_chr + c − "a" + 10;
        end;  { check the resulting value is within the valid range }
      if cur_chr > biggest_usv then
        begin cur_chr ← buffer[loc]; goto not_exp;
        end;
      loc ← loc + 2 ∗ sup_count − 1; goto reswitch;
      end;
not_exp: state ← mid_line;
  end
```

This code is used in section 374.

383. ⟨ Process an active-character control sequence and set *state* ← *mid_line* 383 ⟩ ≡

```
begin cur_cs ← cur_chr + active_base; cur_cmd ← eq_type(cur_cs); cur_chr ← equiv(cur_cs);
state ← mid_line;
if cur_cmd ≥ outer_call then check_outer_validity;
end
```

This code is used in section 374.

384. Control sequence names are scanned only when they appear in some line of a file; once they have been scanned the first time, their *eqtb* location serves as a unique identification, so TeX doesn't need to refer to the original name any more except when it prints the equivalent in symbolic form.

The program that scans a control sequence has been written carefully in order to avoid the blowups that might otherwise occur if a malicious user tried something like '`\catcode´15=0`'. The algorithm might look at *buffer*[*limit* + 1], but it never looks at *buffer*[*limit* + 2].

If expanded characters like '`^^A`' or '`^^df`' appear in or just following a control sequence name, they are converted to single characters in the buffer and the process is repeated, slowly but surely.

⟨ Scan a control sequence and set *state* ← *skip_blanks* or *mid_line* 384 ⟩ ≡

```
begin if loc > limit then cur_cs ← null_cs { state is irrelevant in this case }
else begin start_cs: k ← loc; cur_chr ← buffer[k]; cat ← cat_code(cur_chr); incr(k);
  if cat = letter then state ← skip_blanks
  else if cat = spacer then state ← skip_blanks
    else state ← mid_line;
  if (cat = letter) ∧ (k ≤ limit) then ⟨ Scan ahead in the buffer until finding a nonletter; if an expanded
      code is encountered, reduce it and goto start_cs; otherwise if a multiletter control sequence is
      found, adjust cur_cs and loc, and goto found 386 ⟩
  else ⟨ If an expanded code is present, reduce it and goto start_cs 385 ⟩; { At this point, we have a
      single-character cs name in the buffer. But if the character code is > "FFFF, we treat it like a
      multiletter name for string purposes, because we use UTF-16 in the string pool. }
  if buffer[loc] > "FFFF then
    begin cur_cs ← id_lookup(loc, 1); incr(loc); goto found;
    end;
  cur_cs ← single_base + buffer[loc]; incr(loc);
  end;
found: cur_cmd ← eq_type(cur_cs); cur_chr ← equiv(cur_cs);
  if cur_cmd ≥ outer_call then check_outer_validity;
  end
```

This code is used in section 374.

385. Whenever we reach the following piece of code, we will have $cur_chr = buffer[k-1]$ and $k \le limit+1$ and $cat = cat_code(cur_chr)$. If an expanded code like `^^A` or `^^df` appears in $buffer[(k-1)\,..\,(k+1)]$ or $buffer[(k-1)\,..\,(k+2)]$, we will store the corresponding code in $buffer[k-1]$ and shift the rest of the buffer left two or three places.

```
⟨ If an expanded code is present, reduce it and goto start_cs 385 ⟩ ≡
  begin if (cat = sup_mark) ∧ (buffer[k] = cur_chr) ∧ (k < limit) then
    begin sup_count ← 2;
        { we have ↑↑ and another char; check how many ↑s we have altogether, up to a max of 6 }
    while (sup_count < 6) ∧ (k + 2 * sup_count − 2 ≤ limit) ∧ (buffer[k + sup_count − 1] = cur_chr) do
      incr(sup_count);  { check whether we have enough hex chars for the number of ↑s }
    for d ← 1 to sup_count do
      if ¬is_hex(buffer[k + sup_count − 2 + d]) then  { found a non-hex char, so do single ↑↑X style }
        begin c ← buffer[k + 1];
        if c < ´200 then
          begin if c < ´100 then buffer[k − 1] ← c + ´100 else buffer[k − 1] ← c − ´100;
          d ← 2; limit ← limit − d;
          while k ≤ limit do
            begin buffer[k] ← buffer[k + d]; incr(k);
            end;
          goto start_cs;
          end
        else sup_count ← 0;
        end;
    if sup_count > 0 then  { there were the right number of hex chars, so convert them }
      begin cur_chr ← 0;
      for d ← 1 to sup_count do
        begin c ← buffer[k + sup_count − 2 + d];
        if c ≤ "9" then cur_chr ← 16 * cur_chr + c − "0"
        else cur_chr ← 16 * cur_chr + c − "a" + 10;
        end;  { check the resulting value is within the valid range }
      if cur_chr > biggest_usv then cur_chr ← buffer[k]
      else begin buffer[k − 1] ← cur_chr; d ← 2 * sup_count − 1;
          { shift the rest of the buffer left by d chars }
        limit ← limit − d;
        while k ≤ limit do
          begin buffer[k] ← buffer[k + d]; incr(k);
          end;
        goto start_cs;
        end
      end
    end
  end
```

This code is used in sections 384 and 386.

386. ⟨ Scan ahead in the buffer until finding a nonletter; if an expanded code is encountered, reduce it and **goto** *start_cs*; otherwise if a multiletter control sequence is found, adjust *cur_cs* and *loc*, and **goto** *found* 386 ⟩ ≡

```
begin repeat cur_chr ← buffer[k]; cat ← cat_code(cur_chr); incr(k);
until (cat ≠ letter) ∨ (k > limit);
⟨ If an expanded code is present, reduce it and goto start_cs 385 ⟩;
if cat ≠ letter then decr(k); { now k points to first nonletter }
if k > loc + 1 then { multiletter control sequence has been scanned }
  begin cur_cs ← id_lookup(loc, k − loc); loc ← k; goto found;
  end;
end
```

This code is used in section 384.

387. Let's consider now what happens when *get_next* is looking at a token list.

⟨ Input from token list, **goto** *restart* if end of list or if a parameter needs to be expanded 387 ⟩ ≡

```
if loc ≠ null then { list not exhausted }
  begin t ← info(loc); loc ← link(loc); { move to next }
  if t ≥ cs_token_flag then { a control sequence token }
    begin cur_cs ← t − cs_token_flag; cur_cmd ← eq_type(cur_cs); cur_chr ← equiv(cur_cs);
    if cur_cmd ≥ outer_call then
      if cur_cmd = dont_expand then ⟨ Get the next token, suppressing expansion 388 ⟩
      else check_outer_validity;
    end
  else begin cur_cmd ← t div max_char_val; cur_chr ← t mod max_char_val;
    case cur_cmd of
    left_brace: incr(align_state);
    right_brace: decr(align_state);
    out_param: ⟨ Insert macro parameter and goto restart 389 ⟩;
    othercases do_nothing
    endcases;
    end;
  end
else begin { we are done with this token list }
  end_token_list; goto restart; { resume previous level }
  end
```

This code is used in section 371.

388. The present point in the program is reached only when the *expand* routine has inserted a special marker into the input. In this special case, *info*(*loc*) is known to be a control sequence token, and *link*(*loc*) = *null*.

```
define no_expand_flag = special_char { this characterizes a special variant of relax }
```

⟨ Get the next token, suppressing expansion 388 ⟩ ≡

```
begin cur_cs ← info(loc) − cs_token_flag; loc ← null;
cur_cmd ← eq_type(cur_cs); cur_chr ← equiv(cur_cs);
if cur_cmd > max_command then
  begin cur_cmd ← relax; cur_chr ← no_expand_flag;
  end;
end
```

This code is used in section 387.

389. ⟨ Insert macro parameter and **goto** *restart* 389 ⟩ ≡
begin *begin_token_list*(*param_stack*[*param_start* + *cur_chr* − 1], *parameter*); **goto** *restart*;
end

This code is used in section 387.

390. All of the easy branches of *get_next* have now been taken care of. There is one more branch.

define *end_line_char_inactive* ≡ (*end_line_char* < 0) ∨ (*end_line_char* > 255)

⟨ Move to next line of file, or **goto** *restart* if there is no next line, or **return** if a `\read` line has finished 390 ⟩ ≡
if *name* > 17 **then** ⟨ Read next line of file into *buffer*, or **goto** *restart* if the file has ended 392 ⟩
else begin if ¬*terminal_input* **then** { `\read` line has ended }
begin *cur_cmd* ← 0; *cur_chr* ← 0; **return**;
end;
if *input_ptr* > 0 **then** { text was inserted during error recovery }
begin *end_file_reading*; **goto** *restart*; { resume previous level }
end;
if *selector* < *log_only* **then** *open_log_file*;
if *interaction* > *nonstop_mode* **then**
begin if *end_line_char_inactive* **then** *incr*(*limit*);
if *limit* = *start* **then** { previous line was empty }
print_nl("(Please␣type␣a␣command␣or␣say␣`\end´)");
print_ln; *first* ← *start*; *prompt_input*("*"); { input on-line into *buffer* }
limit ← *last*;
if *end_line_char_inactive* **then** *decr*(*limit*)
else *buffer*[*limit*] ← *end_line_char*;
first ← *limit* + 1; *loc* ← *start*;
end
else *fatal_error*("***␣(job␣aborted,␣no␣legal␣\end␣found)");
{ nonstop mode, which is intended for overnight batch processing, never waits for on-line input }
end

This code is used in section 373.

391. The global variable *force_eof* is normally *false*; it is set *true* by an `\endinput` command.

⟨ Global variables 13 ⟩ +≡
force_eof: *boolean*; { should the next `\input` be aborted early? }

392. ⟨ Read next line of file into *buffer*, or **goto** *restart* if the file has ended 392 ⟩ ≡

```
begin incr(line); first ← start;
if ¬force_eof then
  if name ≤ 19 then
    begin if pseudo_input then { not end of file }
      firm_up_the_line { this sets limit }
    else if (every_eof ≠ null) ∧ ¬eof_seen[index] then
        begin limit ← first − 1; eof_seen[index] ← true; { fake one empty line }
        begin_token_list(every_eof, every_eof_text); goto restart;
        end
      else force_eof ← true;
    end
  else begin if input_ln(cur_file, true) then { not end of file }
      firm_up_the_line { this sets limit }
    else if (every_eof ≠ null) ∧ ¬eof_seen[index] then
        begin limit ← first − 1; eof_seen[index] ← true; { fake one empty line }
        begin_token_list(every_eof, every_eof_text); goto restart;
        end
      else force_eof ← true;
    end;
if force_eof then
  begin if tracing_nesting > 0 then
    if (grp_stack[in_open] ≠ cur_boundary) ∨ (if_stack[in_open] ≠ cond_ptr) then file_warning;
        { give warning for some unfinished groups and/or conditionals }
  if name ≥ 19 then
    begin print_char(")"); decr(open_parens); update_terminal; { show user that file has been read }
    end;
  force_eof ← false; end_file_reading; { resume previous level }
  check_outer_validity; goto restart;
  end;
if end_line_char_inactive then decr(limit)
else buffer[limit] ← end_line_char;
first ← limit + 1; loc ← start; { ready to read }
end
```

This code is used in section 390.

393. If the user has set the *pausing* parameter to some positive value, and if nonstop mode has not been selected, each line of input is displayed on the terminal and the transcript file, followed by '=>'. TeX waits for a response. If the response is simply *carriage_return*, the line is accepted as it stands, otherwise the line typed is used instead of the line in the file.

```
procedure firm_up_the_line;
  var k: 0 .. buf_size;  { an index into buffer }
  begin limit ← last;
  if pausing > 0 then
    if interaction > nonstop_mode then
      begin wake_up_terminal; print_ln;
      if start < limit then
        for k ← start to limit − 1 do print(buffer[k]);
      first ← limit; prompt_input("=>");  { wait for user response }
      if last > first then
        begin for k ← first to last − 1 do  { move line down in buffer }
          buffer[k + start − first] ← buffer[k];
        limit ← start + last − first;
        end;
      end;
  end;
```

394. Since *get_next* is used so frequently in TeX, it is convenient to define three related procedures that do a little more:

get_token not only sets *cur_cmd* and *cur_chr*, it also sets *cur_tok*, a packed halfword version of the current token.

get_x_token, meaning "get an expanded token," is like *get_token*, but if the current token turns out to be a user-defined control sequence (i.e., a macro call), or a conditional, or something like `\topmark` or `\expandafter` or `\csname`, it is eliminated from the input by beginning the expansion of the macro or the evaluation of the conditional.

x_token is like *get_x_token* except that it assumes that *get_next* has already been called.

In fact, these three procedures account for almost every use of *get_next*.

395. No new control sequences will be defined except during a call of *get_token*, or when `\csname` compresses a token list, because *no_new_control_sequence* is always *true* at other times.

```
procedure get_token;  { sets cur_cmd, cur_chr, cur_tok }
  begin no_new_control_sequence ← false; get_next; no_new_control_sequence ← true;
  if cur_cs = 0 then cur_tok ← (cur_cmd * max_char_val) + cur_chr
  else cur_tok ← cs_token_flag + cur_cs;
  end;
```

396. Expanding the next token. Only a dozen or so command codes > *max_command* can possibly be returned by *get_next*; in increasing order, they are *undefined_cs*, *expand_after*, *no_expand*, *input*, *if_test*, *fi_or_else*, *cs_name*, *convert*, *the*, *top_bot_mark*, *call*, *long_call*, *outer_call*, *long_outer_call*, and *end_template*.

The *expand* subroutine is used when *cur_cmd* > *max_command*. It removes a "call" or a conditional or one of the other special operations just listed. It follows that *expand* might invoke itself recursively. In all cases, *expand* destroys the current token, but it sets things up so that the next *get_next* will deliver the appropriate next token. The value of *cur_tok* need not be known when *expand* is called.

Since several of the basic scanning routines communicate via global variables, their values are saved as local variables of *expand* so that recursive calls don't invalidate them.

```
⟨ Declare the procedure called macro_call 423 ⟩
⟨ Declare the procedure called insert_relax 413 ⟩
⟨ Declare ε-TeX procedures for expanding 1563 ⟩
procedure pass_text; forward;
procedure start_input; forward;
procedure conditional; forward;
procedure get_x_token; forward;
procedure conv_toks; forward;
procedure ins_the_toks; forward;
procedure expand;
  label reswitch;
  var t: halfword; { token that is being "expanded after" }
    b: boolean; { keep track of nested csnames }
    p, q, r: pointer; { for list manipulation }
    j: 0 .. buf_size; { index into buffer }
    cv_backup: integer; { to save the global quantity cur_val }
    cvl_backup, radix_backup, co_backup: small_number; { to save cur_val_level, etc. }
    backup_backup: pointer; { to save link(backup_head) }
    save_scanner_status: small_number; { temporary storage of scanner_status }
  begin cv_backup ← cur_val; cvl_backup ← cur_val_level; radix_backup ← radix; co_backup ← cur_order;
  backup_backup ← link(backup_head);
reswitch: if cur_cmd < call then ⟨ Expand a nonmacro 399 ⟩
  else if cur_cmd < end_template then macro_call
    else ⟨ Insert a token containing frozen_endv 409 ⟩;
  cur_val ← cv_backup; cur_val_level ← cvl_backup; radix ← radix_backup; cur_order ← co_backup;
  link(backup_head) ← backup_backup;
  end;
```

397. ⟨ Global variables 13 ⟩ +≡

```
is_in_csname: boolean;
```

398. ⟨ Set initial values of key variables 23 ⟩ +≡

```
  is_in_csname ← false;
```

399. ⟨ Expand a nonmacro 399 ⟩ ≡

```
begin if tracing_commands > 1 then show_cur_cmd_chr;
case cur_cmd of
top_bot_mark: ⟨ Insert the appropriate mark text into the scanner 420 ⟩;
expand_after: if cur_chr = 0 then ⟨ Expand the token after the next token 400 ⟩
  else ⟨ Negate a boolean conditional and goto reswitch 1576 ⟩;
no_expand: if cur_chr = 0 then ⟨ Suppress expansion of the next token 401 ⟩
  else ⟨ Implement \primitive 402 ⟩;
cs_name: ⟨ Manufacture a control sequence name 406 ⟩;
convert: conv_toks;  { this procedure is discussed in Part 27 below }
the: ins_the_toks;  { this procedure is discussed in Part 27 below }
if_test: conditional;  { this procedure is discussed in Part 28 below }
fi_or_else: ⟨ Terminate the current conditional and skip to \fi 545 ⟩;
input: ⟨ Initiate or terminate input from a file 412 ⟩;
othercases ⟨ Complain about an undefined macro 404 ⟩
endcases;
end
```

This code is used in section 396.

400. It takes only a little shuffling to do what TeX calls `\expandafter`.

⟨ Expand the token after the next token 400 ⟩ ≡

```
begin get_token; t ← cur_tok; get_token;
if cur_cmd > max_command then expand else back_input;
cur_tok ← t; back_input;
end
```

This code is used in section 399.

401. The implementation of `\noexpand` is a bit trickier, because it is necessary to insert a special '*dont_expand*' marker into TeX's reading mechanism. This special marker is processed by *get_next*, but it does not slow down the inner loop.

Since `\outer` macros might arise here, we must also clear the *scanner_status* temporarily.

⟨ Suppress expansion of the next token 401 ⟩ ≡

```
begin save_scanner_status ← scanner_status; scanner_status ← normal; get_token;
scanner_status ← save_scanner_status; t ← cur_tok; back_input;
    { now start and loc point to the backed-up token t }
if t ≥ cs_token_flag then
  begin p ← get_avail; info(p) ← cs_token_flag + frozen_dont_expand; link(p) ← loc; start ← p;
  loc ← p;
  end;
end
```

This code is used in section 399.

402. The `\primitive` handling. If the primitive meaning of the next token is an expandable command, it suffices to replace the current token with the primitive one and restart *expand*/

Otherwise, the token we just read has to be pushed back, as well as a token matching the internal form of `\primitive`, that is sneaked in as an alternate form of *ignore_spaces*.

Simply pushing back a token that matches the correct internal command does not work, because approach would not survive roundtripping to a temporary file.

```
⟨ Implement \primitive 402 ⟩ ≡
  begin save_scanner_status ← scanner_status; scanner_status ← normal; get_token;
  scanner_status ← save_scanner_status;
  if cur_cs < hash_base then cur_cs ← prim_lookup(cur_cs − single_base)
  else cur_cs ← prim_lookup(text(cur_cs));
  if cur_cs ≠ undefined_primitive then
    begin t ← prim_eq_type(cur_cs);
    if t > max_command then
      begin cur_cmd ← t; cur_chr ← prim_equiv(cur_cs);
      cur_tok ← (cur_cmd * max_char_val) + cur_chr; cur_cs ← 0; goto reswitch;
      end
    else begin back_input; { now loc and start point to a one-item list }
      p ← get_avail; info(p) ← cs_token_flag + frozen_primitive; link(p) ← loc; loc ← p; start ← p;
      end;
    end;
  end
```

This code is used in section 399.

403. This block deals with unexpandable `\primitive` appearing at a spot where an integer or an internal values should have been found. It fetches the next token then resets *cur_cmd*, *cur_cs*, and *cur_tok*, based on the primitive value of that token. No expansion takes place, because the next token may be all sorts of things. This could trigger further expansion creating new errors.

```
⟨ Reset cur_tok for unexpandable primitives, goto restart 403 ⟩ ≡
  begin get_token;
  if cur_cs < hash_base then cur_cs ← prim_lookup(cur_cs − single_base)
  else cur_cs ← prim_lookup(text(cur_cs));
  if cur_cs ≠ undefined_primitive then
    begin cur_cmd ← prim_eq_type(cur_cs); cur_chr ← prim_equiv(cur_cs);
    cur_cs ← prim_eqtb_base + cur_cs; cur_tok ← cs_token_flag + cur_cs;
    end
  else begin cur_cmd ← relax; cur_chr ← 0; cur_tok ← cs_token_flag + frozen_relax;
    cur_cs ← frozen_relax;
    end;
  goto restart;
  end
```

This code is used in sections 447 and 474.

404. ⟨ Complain about an undefined macro 404 ⟩ ≡

```
begin print_err("Undefined␣control␣sequence");
help5("The␣control␣sequence␣at␣the␣end␣of␣the␣top␣line")
("of␣your␣error␣message␣was␣never␣\def´ed.␣If␣you␣have")
("misspelled␣it␣(e.g.,␣`\hobx´),␣type␣`I´␣and␣the␣correct")
("spelling␣(e.g.,␣`I\hbox´).␣Otherwise␣just␣continue,")
("and␣I´ll␣forget␣about␣whatever␣was␣undefined."); error;
end
```

This code is used in section 399.

405. The *expand* procedure and some other routines that construct token lists find it convenient to use the following macros, which are valid only if the variables p and q are reserved for token-list building.

```
define store_new_token(#) ≡
        begin q ← get_avail; link(p) ← q; info(q) ← #; p ← q;  { link(p) is null }
        end
define fast_store_new_token(#) ≡
        begin fast_get_avail(q); link(p) ← q; info(q) ← #; p ← q;  { link(p) is null }
        end
```

406. ⟨ Manufacture a control sequence name 406 ⟩ ≡

```
begin r ← get_avail; p ← r;  { head of the list of characters }
b ← is_in_csname; is_in_csname ← true;
repeat get_x_token;
  if cur_cs = 0 then store_new_token(cur_tok);
until cur_cs ≠ 0;
if cur_cmd ≠ end_cs_name then ⟨ Complain about missing \endcsname 407 ⟩;
is_in_csname ← b; ⟨ Look up the characters of list r in the hash table, and set cur_cs 408 ⟩;
flush_list(r);
if eq_type(cur_cs) = undefined_cs then
  begin eq_define(cur_cs, relax, too_big_usv);  { N.B.: The save_stack might change }
  end;  { the control sequence will now match '\relax' }
cur_tok ← cur_cs + cs_token_flag; back_input;
end
```

This code is used in section 399.

407. ⟨ Complain about missing \endcsname 407 ⟩ ≡

```
begin print_err("Missing␣"); print_esc("endcsname"); print("␣inserted");
help2("The␣control␣sequence␣marked␣<to␣be␣read␣again>␣should")
("not␣appear␣between␣\csname␣and␣\endcsname."); back_error;
end
```

This code is used in sections 406 and 1578.

408. ⟨ Look up the characters of list r in the hash table, and set *cur_cs* 408 ⟩ ≡

```
j ← first; p ← link(r);
while p ≠ null do
  begin if j ≥ max_buf_stack then
    begin max_buf_stack ← j + 1;
    if max_buf_stack = buf_size then overflow("buffer␣size", buf_size);
    end;
  buffer[j] ← info(p) mod max_char_val; incr(j); p ← link(p);
  end;
if (j > first + 1) ∨ (buffer[first] > "FFFF) then
  begin no_new_control_sequence ← false; cur_cs ← id_lookup(first, j − first);
  no_new_control_sequence ← true;
  end
else if j = first then cur_cs ← null_cs  { the list is empty }
  else cur_cs ← single_base + buffer[first]  { the list has length one }
```

This code is used in section 406.

409. An *end_template* command is effectively changed to an *endv* command by the following code. (The reason for this is discussed below; the *frozen_end_template* at the end of the template has passed the *check_outer_validity* test, so its mission of error detection has been accomplished.)

⟨ Insert a token containing *frozen_endv* 409 ⟩ ≡

```
begin cur_tok ← cs_token_flag + frozen_endv; back_input;
end
```

This code is used in section 396.

410. The processing of `\input` involves the *start_input* subroutine, which will be declared later; the processing of `\endinput` is trivial.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡

```
primitive("input", input, 0);
primitive("endinput", input, 1);
```

411. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
input: if chr_code = 0 then print_esc("input")
  ⟨ Cases of input for print_cmd_chr 1559 ⟩
else print_esc("endinput");
```

412. ⟨ Initiate or terminate input from a file 412 ⟩ ≡

```
if cur_chr = 1 then force_eof ← true
⟨ Cases for input 1560 ⟩
else if name_in_progress then insert_relax
  else start_input
```

This code is used in section 399.

413. Sometimes the expansion looks too far ahead, so we want to insert a harmless `\relax` into the user's input.

⟨ Declare the procedure called *insert_relax* 413 ⟩ ≡

```
procedure insert_relax;
  begin cur_tok ← cs_token_flag + cur_cs; back_input; cur_tok ← cs_token_flag + frozen_relax; back_input;
  token_type ← inserted;
  end;
```

This code is used in section 396.

414. Here is a recursive procedure that is TeX's usual way to get the next token of input. It has been slightly optimized to take account of common cases.

```
procedure get_x_token; { sets cur_cmd, cur_chr, cur_tok, and expands macros }
  label restart, done;
  begin restart: get_next;
  if cur_cmd ≤ max_command then goto done;
  if cur_cmd ≥ call then
    if cur_cmd < end_template then macro_call
    else begin cur_cs ← frozen_endv; cur_cmd ← endv; goto done; { cur_chr = null_list }
      end
  else expand;
  goto restart;
done: if cur_cs = 0 then cur_tok ← (cur_cmd * max_char_val) + cur_chr
  else cur_tok ← cs_token_flag + cur_cs;
  end;
```

415. The *get_x_token* procedure is essentially equivalent to two consecutive procedure calls: *get_next*; *x_token*.

```
procedure x_token; { get_x_token without the initial get_next }
  begin while cur_cmd > max_command do
    begin expand; get_next;
    end;
  if cur_cs = 0 then cur_tok ← (cur_cmd * max_char_val) + cur_chr
  else cur_tok ← cs_token_flag + cur_cs;
  end;
```

416. A control sequence that has been `\def`'ed by the user is expanded by TeX's *macro_call* procedure.

Before we get into the details of *macro_call*, however, let's consider the treatment of primitives like `\topmark`, since they are essentially macros without parameters. The token lists for such marks are kept in a global array of five pointers; we refer to the individual entries of this array by symbolic names *top_mark*, etc. The value of *top_mark* is either *null* or a pointer to the reference count of a token list.

```
define marks_code ≡ 5 { add this for \topmarks etc. }
define top_mark_code = 0 { the mark in effect at the previous page break }
define first_mark_code = 1 { the first mark between top_mark and bot_mark }
define bot_mark_code = 2 { the mark in effect at the current page break }
define split_first_mark_code = 3 { the first mark found by \vsplit }
define split_bot_mark_code = 4 { the last mark found by \vsplit }
define top_mark ≡ cur_mark[top_mark_code]
define first_mark ≡ cur_mark[first_mark_code]
define bot_mark ≡ cur_mark[bot_mark_code]
define split_first_mark ≡ cur_mark[split_first_mark_code]
define split_bot_mark ≡ cur_mark[split_bot_mark_code]
```

⟨ Global variables 13 ⟩ +≡

```
cur_mark: array [top_mark_code .. split_bot_mark_code] of pointer; { token lists for marks }
```

417. ⟨ Set initial values of key variables 23 ⟩ +≡

```
top_mark ← null; first_mark ← null; bot_mark ← null; split_first_mark ← null; split_bot_mark ← null;
```

418. ⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
```
primitive("topmark", top_bot_mark, top_mark_code);
primitive("firstmark", top_bot_mark, first_mark_code);
primitive("botmark", top_bot_mark, bot_mark_code);
primitive("splitfirstmark", top_bot_mark, split_first_mark_code);
primitive("splitbotmark", top_bot_mark, split_bot_mark_code);
```

419. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
```
top_bot_mark: begin case (chr_code mod marks_code) of
  first_mark_code: print_esc("firstmark");
  bot_mark_code: print_esc("botmark");
  split_first_mark_code: print_esc("splitfirstmark");
  split_bot_mark_code: print_esc("splitbotmark");
  othercases print_esc("topmark")
  endcases;
  if chr_code ≥ marks_code then print_char("s");
  end;
```

420. The following code is activated when *cur_cmd* = *top_bot_mark* and when *cur_chr* is a code like *top_mark_code*.

⟨ Insert the appropriate mark text into the scanner 420 ⟩ ≡
```
begin t ← cur_chr mod marks_code;
if cur_chr ≥ marks_code then scan_register_num else cur_val ← 0;
if cur_val = 0 then cur_ptr ← cur_mark[t]
else ⟨ Compute the mark pointer for mark type t and class cur_val 1635 ⟩;
if cur_ptr ≠ null then begin_token_list(cur_ptr, mark_text);
end
```
This code is used in section 399.

421. Now let's consider *macro_call* itself, which is invoked when TeX is scanning a control sequence whose *cur_cmd* is either *call*, *long_call*, *outer_call*, or *long_outer_call*. The control sequence definition appears in the token list whose reference count is in location *cur_chr* of *mem*.

The global variable *long_state* will be set to *call* or to *long_call*, depending on whether or not the control sequence disallows `\par` in its parameters. The *get_next* routine will set *long_state* to *outer_call* and emit `\par`, if a file ends or if an `\outer` control sequence occurs in the midst of an argument.

⟨ Global variables 13 ⟩ +≡
```
long_state: call .. long_outer_call; { governs the acceptance of \par }
```

422. The parameters, if any, must be scanned before the macro is expanded. Parameters are token lists without reference counts. They are placed on an auxiliary stack called *pstack* while they are being scanned, since the *param_stack* may be losing entries during the matching process. (Note that *param_stack* can't be gaining entries, since *macro_call* is the only routine that puts anything onto *param_stack*, and it is not recursive.)

⟨ Global variables 13 ⟩ +≡
```
pstack: array [0 .. 8] of pointer; { arguments supplied to a macro }
```

423. After parameter scanning is complete, the parameters are moved to the *param_stack*. Then the macro body is fed to the scanner; in other words, *macro_call* places the defined text of the control sequence at the top of TeX's input stack, so that *get_next* will proceed to read it next.

The global variable *cur_cs* contains the *eqtb* address of the control sequence being expanded, when *macro_call* begins. If this control sequence has not been declared `\long`, i.e., if its command code in the *eq_type* field is not *long_call* or *long_outer_call*, its parameters are not allowed to contain the control sequence `\par`. If an illegal `\par` appears, the macro call is aborted, and the `\par` will be rescanned.

```
⟨ Declare the procedure called macro_call 423 ⟩ ≡
procedure macro_call; { invokes a user-defined control sequence }
  label exit, continue, done, done1, found;
  var r: pointer; { current node in the macro's token list }
    p: pointer; { current node in parameter token list being built }
    q: pointer; { new node being put into the token list }
    s: pointer; { backup pointer for parameter matching }
    t: pointer; { cycle pointer for backup recovery }
    u, v: pointer; { auxiliary pointers for backup recovery }
    rbrace_ptr: pointer; { one step before the last right_brace token }
    n: small_number; { the number of parameters scanned }
    unbalance: halfword; { unmatched left braces in current parameter }
    m: halfword; { the number of tokens or groups (usually) }
    ref_count: pointer; { start of the token list }
    save_scanner_status: small_number; { scanner_status upon entry }
    save_warning_index: pointer; { warning_index upon entry }
    match_chr: ASCII_code; { character used in parameter }
  begin save_scanner_status ← scanner_status; save_warning_index ← warning_index;
  warning_index ← cur_cs; ref_count ← cur_chr; r ← link(ref_count); n ← 0;
  if tracing_macros > 0 then ⟨ Show the text of the macro being expanded 435 ⟩;
  if info(r) = protected_token then r ← link(r);
  if info(r) ≠ end_match_token then ⟨ Scan the parameters and make link(r) point to the macro body;
        but return if an illegal \par is detected 425 ⟩;
  ⟨ Feed the macro body and its parameters to the scanner 424 ⟩;
exit: scanner_status ← save_scanner_status; warning_index ← save_warning_index;
  end;
```

This code is used in section 396.

424. Before we put a new token list on the input stack, it is wise to clean off all token lists that have recently been depleted. Then a user macro that ends with a call to itself will not require unbounded stack space.

```
⟨ Feed the macro body and its parameters to the scanner 424 ⟩ ≡
  while (state = token_list) ∧ (loc = null) ∧ (token_type ≠ v_template) do end_token_list;
        { conserve stack space }
  begin_token_list(ref_count, macro); name ← warning_index; loc ← link(r);
  if n > 0 then
    begin if param_ptr + n > max_param_stack then
      begin max_param_stack ← param_ptr + n;
      if max_param_stack > param_size then overflow("parameter␣stack␣size", param_size);
      end;
    for m ← 0 to n − 1 do param_stack[param_ptr + m] ← pstack[m];
    param_ptr ← param_ptr + n;
    end
```

This code is used in section 423.

425. At this point, the reader will find it advisable to review the explanation of token list format that was presented earlier, since many aspects of that format are of importance chiefly in the *macro_call* routine.

The token list might begin with a string of compulsory tokens before the first *match* or *end_match*. In that case the macro name is supposed to be followed by those tokens; the following program will set $s = null$ to represent this restriction. Otherwise s will be set to the first token of a string that will delimit the next parameter.

```
⟨ Scan the parameters and make link(r) point to the macro body; but return if an illegal \par is
      detected 425 ⟩ ≡
  begin scanner_status ← matching; unbalance ← 0; long_state ← eq_type(cur_cs);
  if long_state ≥ outer_call then long_state ← long_state − 2;
  repeat link(temp_head) ← null;
    if (info(r) ≥ end_match_token) ∨ (info(r) < match_token) then s ← null
    else begin match_chr ← info(r) − match_token; s ← link(r); r ← s; p ← temp_head; m ← 0;
      end;
    ⟨ Scan a parameter until its delimiter string has been found; or, if s = null, simply scan the delimiter
        string 426 ⟩;
      { now info(r) is a token whose command code is either match or end_match }
  until info(r) = end_match_token;
  end
```

This code is used in section 423.

426. If $info(r)$ is a *match* or *end_match* command, it cannot be equal to any token found by *get_token*. Therefore an undelimited parameter—i.e., a *match* that is immediately followed by *match* or *end_match*—will always fail the test '$cur_tok = info(r)$' in the following algorithm.

```
⟨ Scan a parameter until its delimiter string has been found; or, if s = null, simply scan the delimiter
      string 426 ⟩ ≡
continue: get_token;   { set cur_tok to the next token of input }
  if cur_tok = info(r) then ⟨ Advance r; goto found if the parameter delimiter has been fully matched,
        otherwise goto continue 428 ⟩;
  ⟨ Contribute the recently matched tokens to the current parameter, and goto continue if a partial match
      is still in effect; but abort if s = null 431 ⟩;
  if cur_tok = par_token then
    if long_state ≠ long_call then ⟨ Report a runaway argument and abort 430 ⟩;
  if cur_tok < right_brace_limit then
    if cur_tok < left_brace_limit then ⟨ Contribute an entire group to the current parameter 433 ⟩
    else ⟨ Report an extra right brace and goto continue 429 ⟩
  else ⟨ Store the current token, but goto continue if it is a blank space that would become an undelimited
        parameter 427 ⟩;
  incr(m);
  if info(r) > end_match_token then goto continue;
  if info(r) < match_token then goto continue;
found: if s ≠ null then ⟨ Tidy up the parameter just scanned, and tuck it away 434 ⟩
```

This code is used in section 425.

427. ⟨ Store the current token, but **goto** *continue* if it is a blank space that would become an undelimited parameter 427 ⟩ ≡

```
begin if cur_tok = space_token then
  if info(r) ≤ end_match_token then
    if info(r) ≥ match_token then goto continue;
store_new_token(cur_tok);
end
```

This code is used in section 426.

428. A slightly subtle point arises here: When the parameter delimiter ends with '`#{`', the token list will have a left brace both before and after the *end_match*. Only one of these should affect the *align_state*, but both will be scanned, so we must make a correction.

⟨ Advance r; **goto** *found* if the parameter delimiter has been fully matched, otherwise **goto** *continue* 428 ⟩ ≡

```
begin r ← link(r);
if (info(r) ≥ match_token) ∧ (info(r) ≤ end_match_token) then
  begin if cur_tok < left_brace_limit then decr(align_state);
  goto found;
  end
else goto continue;
end
```

This code is used in section 426.

429. ⟨ Report an extra right brace and **goto** *continue* 429 ⟩ ≡

```
begin back_input; print_err("Argument␣of␣"); sprint_cs(warning_index); print("␣has␣an␣extra␣}");
help6("I´ve␣run␣across␣a␣`}´␣that␣doesn´t␣seem␣to␣match␣anything.")
("For␣example,␣`\def\a#1{...}´␣and␣`\a}´␣would␣produce")
("this␣error.␣If␣you␣simply␣proceed␣now,␣the␣`\par´␣that")
("I´ve␣just␣inserted␣will␣cause␣me␣to␣report␣a␣runaway")
("argument␣that␣might␣be␣the␣root␣of␣the␣problem.␣But␣if")
("your␣`}´␣was␣spurious,␣just␣type␣`2´␣and␣it␣will␣go␣away."); incr(align_state);
long_state ← call; cur_tok ← par_token; ins_error; goto continue;
end { a white lie; the \par won't always trigger a runaway }
```

This code is used in section 426.

430. If *long_state* = *outer_call*, a runaway argument has already been reported.

⟨ Report a runaway argument and abort 430 ⟩ ≡

```
begin if long_state = call then
  begin runaway; print_err("Paragraph␣ended␣before␣"); sprint_cs(warning_index);
  print("␣was␣complete");
  help3("I␣suspect␣you´ve␣forgotten␣a␣`}´,␣causing␣me␣to␣apply␣this")
  ("control␣sequence␣to␣too␣much␣text.␣How␣can␣we␣recover?")
  ("My␣plan␣is␣to␣forget␣the␣whole␣thing␣and␣hope␣for␣the␣best."); back_error;
  end;
pstack[n] ← link(temp_head); align_state ← align_state − unbalance;
for m ← 0 to n do flush_list(pstack[m]);
return;
end
```

This code is used in sections 426 and 433.

431. When the following code becomes active, we have matched tokens from s to the predecessor of r, and we have found that *cur_tok* $\neq$ *info*(r). An interesting situation now presents itself: If the parameter is to be delimited by a string such as '`ab`', and if we have scanned '`aa`', we want to contribute one '`a`' to the current parameter and resume looking for a '`b`'. The program must account for such partial matches and for others that can be quite complex. But most of the time we have $s = r$ and nothing needs to be done.

Incidentally, it is possible for `\par` tokens to sneak in to certain parameters of non-`\long` macros. For example, consider a case like '`\def\a#1\par!{...}`' where the first `\par` is not followed by an exclamation point. In such situations it does not seem appropriate to prohibit the `\par`, so TeX keeps quiet about this bending of the rules.

```
⟨ Contribute the recently matched tokens to the current parameter, and goto continue if a partial match is
      still in effect; but abort if s = null 431 ⟩ ≡
  if s ≠ r then
    if s = null then ⟨ Report an improper use of the macro and abort 432 ⟩
    else begin t ← s;
      repeat store_new_token(info(t)); incr(m); u ← link(t); v ← s;
        loop begin if u = r then
            if cur_tok ≠ info(v) then goto done
            else begin r ← link(v); goto continue;
              end;
          if info(u) ≠ info(v) then goto done;
          u ← link(u); v ← link(v);
          end;
      done: t ← link(t);
      until t = r;
      r ← s;  { at this point, no tokens are recently matched }
      end
```

This code is used in section 426.

432.

```
⟨ Report an improper use of the macro and abort 432 ⟩ ≡
  begin print_err("Use␣of␣"); sprint_cs(warning_index); print("␣doesn't␣match␣its␣definition");
  help4("If␣you␣say,␣e.g.,␣`\def\a1{...}',␣then␣you␣must␣always")
  ("put␣`1'␣after␣`\a',␣since␣control␣sequence␣names␣are")
  ("made␣up␣of␣letters␣only.␣The␣macro␣here␣has␣not␣been")
  ("followed␣by␣the␣required␣stuff,␣so␣I'm␣ignoring␣it."); error; return;
  end
```

This code is used in section 431.

433.

```
⟨ Contribute an entire group to the current parameter 433 ⟩ ≡
  begin unbalance ← 1;
  loop begin fast_store_new_token(cur_tok); get_token;
    if cur_tok = par_token then
      if long_state ≠ long_call then ⟨ Report a runaway argument and abort 430 ⟩;
    if cur_tok < right_brace_limit then
      if cur_tok < left_brace_limit then incr(unbalance)
      else begin decr(unbalance);
        if unbalance = 0 then goto done1;
        end;
    end;
done1: rbrace_ptr ← p; store_new_token(cur_tok);
  end
```

This code is used in section 426.

434. If the parameter consists of a single group enclosed in braces, we must strip off the enclosing braces. That's why *rbrace_ptr* was introduced.

⟨ Tidy up the parameter just scanned, and tuck it away 434 ⟩ ≡

```
begin if (m = 1) ∧ (info(p) < right_brace_limit) then
  begin link(rbrace_ptr) ← null; free_avail(p); p ← link(temp_head); pstack[n] ← link(p); free_avail(p);
  end
else pstack[n] ← link(temp_head);
incr(n);
if tracing_macros > 0 then
  begin begin_diagnostic; print_nl(match_chr); print_int(n); print("<-");
  show_token_list(pstack[n − 1], null, 1000); end_diagnostic(false);
  end;
end
```

This code is used in section 426.

435. ⟨ Show the text of the macro being expanded 435 ⟩ ≡

```
begin begin_diagnostic; print_ln; print_cs(warning_index); token_show(ref_count);
end_diagnostic(false);
end
```

This code is used in section 423.

436. Basic scanning subroutines. Let's turn now to some procedures that TeX calls upon frequently to digest certain kinds of patterns in the input. Most of these are quite simple; some are quite elaborate. Almost all of the routines call *get_x_token*, which can cause them to be invoked recursively.

437. The *scan_left_brace* routine is called when a left brace is supposed to be the next non-blank token. (The term "left brace" means, more precisely, a character whose catcode is *left_brace*.) TeX allows `\relax` to appear before the *left_brace*.

```
procedure scan_left_brace; { reads a mandatory left_brace }
  begin ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
  if cur_cmd ≠ left_brace then
    begin print_err("Missing␣{␣inserted");
    help4("A␣left␣brace␣was␣mandatory␣here,␣so␣I´ve␣put␣one␣in.")
    ("You␣might␣want␣to␣delete␣and/or␣insert␣some␣corrections")
    ("so␣that␣I␣will␣find␣a␣matching␣right␣brace␣soon.")
    ("(If␣you´re␣confused␣by␣all␣this,␣try␣typing␣`I}´␣now.)"); back_error;
    cur_tok ← left_brace_token + "{"; cur_cmd ← left_brace; cur_chr ← "{"; incr(align_state);
    end;
  end;
```

438. ⟨ Get the next non-blank non-relax non-call token 438 ⟩ ≡

```
repeat get_x_token;
until (cur_cmd ≠ spacer) ∧ (cur_cmd ≠ relax)
```

This code is used in sections 437, 1132, 1138, 1205, 1214, 1265, 1280, and 1324.

439. The *scan_optional_equals* routine looks for an optional '`=`' sign preceded by optional spaces; '`\relax`' is not ignored here.

```
procedure scan_optional_equals;
  begin ⟨ Get the next non-blank non-call token 440 ⟩;
  if cur_tok ≠ other_token + "=" then back_input;
  end;
```

440. ⟨ Get the next non-blank non-call token 440 ⟩ ≡

```
repeat get_x_token;
until cur_cmd ≠ spacer
```

This code is used in sections 439, 475, 490, 538, 561, 612, 1099, 1595, and 1596.

441. In case you are getting bored, here is a slightly less trivial routine: Given a string of lowercase letters, like '`pt`' or '`plus`' or '`width`', the *scan_keyword* routine checks to see whether the next tokens of input match this string. The match must be exact, except that uppercase letters will match their lowercase counterparts; uppercase equivalents are determined by subtracting `"a" − "A"`, rather than using the *uc_code* table, since TeX uses this routine only for its own limited set of keywords.

If a match is found, the characters are effectively removed from the input and *true* is returned. Otherwise *false* is returned, and the input is left essentially unchanged (except for the fact that some macros may have been expanded, etc.).

```
function scan_keyword(s : str_number): boolean; { look for a given string }
  label exit;
  var p: pointer; { tail of the backup list }
    q: pointer; { new node being added to the token list via store_new_token }
    k: pool_pointer; { index into str_pool }
    save_cur_cs: pointer; { to save cur_cs }
  begin p ← backup_head; link(p) ← null;
  if s < too_big_char then
    begin while true do
      begin get_x_token; { recursion is possible here }
      if (cur_cs = 0) ∧ ((cur_chr = s) ∨ (cur_chr = s − "a" + "A")) then
        begin store_new_token(cur_tok); flush_list(link(backup_head)); scan_keyword ← true; return;
        end
      else if (cur_cmd ≠ spacer) ∨ (p ≠ backup_head) then
          begin back_input;
          if p ≠ backup_head then back_list(link(backup_head));
          scan_keyword ← false; return;
          end;
      end;
    end;
  k ← str_start_macro(s); save_cur_cs ← cur_cs;
  while k < str_start_macro(s + 1) do
    begin get_x_token; { recursion is possible here }
    if (cur_cs = 0) ∧ ((cur_chr = so(str_pool[k])) ∨ (cur_chr = so(str_pool[k]) − "a" + "A")) then
      begin store_new_token(cur_tok); incr(k);
      end
    else if (cur_cmd ≠ spacer) ∨ (p ≠ backup_head) then
        begin back_input;
        if p ≠ backup_head then back_list(link(backup_head));
        cur_cs ← save_cur_cs; scan_keyword ← false; return;
        end;
    end;
  flush_list(link(backup_head)); scan_keyword ← true;
exit: end;
```

442. Here is a procedure that sounds an alarm when mu and non-mu units are being switched.

```
procedure mu_error;
  begin print_err("Incompatible␣glue␣units");
  help1("I´m␣going␣to␣assume␣that␣1mu=1pt␣when␣they´re␣mixed."); error;
  end;
```

443. The next routine '*scan_something_internal*' is used to fetch internal numeric quantities like '`\hsize`', and also to handle the '`\the`' when expanding constructions like '`\the\toks0`' and '`\the\baselineskip`'. Soon we will be considering the *scan_int* procedure, which calls *scan_something_internal*; on the other hand, *scan_something_internal* also calls *scan_int*, for constructions like '`\catcode`\$`' or '`\fontdimen 3 \ff`'. So we have to declare *scan_int* as a *forward* procedure. A few other procedures are also declared at this point.

procedure *scan_int*; *forward*; { scans an integer value }
⟨ Declare procedures that scan restricted classes of integers 467 ⟩
⟨ Declare ε-TeX procedures for scanning 1492 ⟩
⟨ Declare procedures that scan font-related stuff 612 ⟩

444. TeX doesn't know exactly what to expect when *scan_something_internal* begins. For example, an integer or dimension or glue value could occur immediately after '`\hskip`'; and one can even say `\the` with respect to token lists in constructions like '`\xdef\o{\the\output}`'. On the other hand, only integers are allowed after a construction like '`\count`'. To handle the various possibilities, *scan_something_internal* has a *level* parameter, which tells the "highest" kind of quantity that *scan_something_internal* is allowed to produce. Six levels are distinguished, namely *int_val*, *dimen_val*, *glue_val*, *mu_val*, *ident_val*, and *tok_val*.

The output of *scan_something_internal* (and of the other routines *scan_int*, *scan_dimen*, and *scan_glue* below) is put into the global variable *cur_val*, and its level is put into *cur_val_level*. The highest values of *cur_val_level* are special: *mu_val* is used only when *cur_val* points to something in a "muskip" register, or to one of the three parameters `\thinmuskip`, `\medmuskip`, `\thickmuskip`; *ident_val* is used only when *cur_val* points to a font identifier; *tok_val* is used only when *cur_val* points to *null* or to the reference count of a token list. The last two cases are allowed only when *scan_something_internal* is called with *level* = *tok_val*.

If the output is glue, *cur_val* will point to a glue specification, and the reference count of that glue will have been updated to reflect this reference; if the output is a nonempty token list, *cur_val* will point to its reference count, but in this case the count will not have been updated. Otherwise *cur_val* will contain the integer or scaled value in question.

define *int_val* = 0 { integer values }
define *dimen_val* = 1 { dimension values }
define *glue_val* = 2 { glue specifications }
define *mu_val* = 3 { math glue specifications }
define *ident_val* = 4 { font identifier }
define *tok_val* = 5 { token lists }
define *inter_char_val* = 6 { inter-character (class) token lists }

⟨ Global variables 13 ⟩ +≡
cur_val: *integer*; { value returned by numeric scanners }
cur_val1: *integer*; { value returned by numeric scanners }
cur_val_level: *int_val* .. *tok_val*; { the "level" of this value }

445. The hash table is initialized with '`\count`', '`\dimen`', '`\skip`', and '`\muskip`' all having *register* as their command code; they are distinguished by the *chr_code*, which is either *int_val*, *dimen_val*, *glue_val*, or *mu_val* more than *mem_bot* (dynamic variable-size nodes cannot have these values)

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("count", *register*, *mem_bot* + *int_val*); *primitive*("dimen", *register*, *mem_bot* + *dimen_val*);
primitive("skip", *register*, *mem_bot* + *glue_val*); *primitive*("muskip", *register*, *mem_bot* + *mu_val*);

446. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
register: ⟨ Cases of *register* for *print_cmd_chr* 1643 ⟩;

447. OK, we're ready for *scan_something_internal* itself. A second parameter, *negative*, is set *true* if the value that is found should be negated. It is assumed that *cur_cmd* and *cur_chr* represent the first token of the internal quantity to be scanned; an error will be signalled if *cur_cmd* $<$ *min_internal* or *cur_cmd* $>$ *max_internal*.

```
define scanned_result_end(#) ≡ cur_val_level ← #; end
define scanned_result(#) ≡ begin cur_val ← #; scanned_result_end
define char_class_limit = "1000
define char_class_ignored ≡ char_class_limit
define char_class_boundary ≡ (char_class_ignored − 1)
procedure scan_something_internal(level : small_number; negative : boolean);
        { fetch an internal parameter }
  label exit, restart;
  var m: halfword;  { chr_code part of the operand token }
    n, k, kk: integer;  { accumulators }
    q, r: pointer;  { general purpose indices }
    tx: pointer;  { effective tail node }
    i: four_quarters;  { character info }
    p: 0 .. nest_size;  { index into nest }
  begin restart: m ← cur_chr;
  case cur_cmd of
  def_code: ⟨ Fetch a character code from some table 448 ⟩;
  XeTeX_def_code: begin scan_usv_num;
    if m = sf_code_base then
      begin scanned_result(ho(sf_code(cur_val) div "10000))(int_val)
      end
    else if m = math_code_base then
        begin scanned_result(ho(math_code(cur_val)))(int_val)
        end
      else if m = math_code_base + 1 then
          begin print_err("Can't use \Umathcode as a number (try \Umathcodenum)");
          help2("\Umathcode is for setting a mathcode from separate values;")
          ("use \Umathcodenum to access them as single values."); error;
          scanned_result(0)(int_val)
          end
        else if m = del_code_base then
            begin scanned_result(ho(del_code(cur_val)))(int_val)
            end
          else begin print_err("Can't use \Udelcode as a number (try \Udelcodenum)");
            help2("\Udelcode is for setting a delcode from separate values;")
            ("use \Udelcodenum to access them as single values."); error;
            scanned_result(0)(int_val);
            end;
    end;
  toks_register, assign_toks, def_family, set_font, def_font: ⟨ Fetch a token list or font identifier, provided
        that level = tok_val 449 ⟩;
  assign_int: scanned_result(eqtb[m].int)(int_val);
  assign_dimen: scanned_result(eqtb[m].sc)(dimen_val);
  assign_glue: scanned_result(equiv(m))(glue_val);
  assign_mu_glue: scanned_result(equiv(m))(mu_val);
  set_aux: ⟨ Fetch the space_factor or the prev_depth 452 ⟩;
  set_prev_graf: ⟨ Fetch the prev_graf 456 ⟩;
  set_page_int: ⟨ Fetch the dead_cycles or the insert_penalties 453 ⟩;
```

set_page_dimen: ⟨ Fetch something on the *page_so_far* 455 ⟩;
set_shape: ⟨ Fetch the *par_shape* size 457 ⟩;
set_box_dimen: ⟨ Fetch a box dimension 454 ⟩;
char_given, *math_given*, *XeTeX_math_given*: *scanned_result*(*cur_chr*)(*int_val*);
assign_font_dimen: ⟨ Fetch a font dimension 459 ⟩;
assign_font_int: ⟨ Fetch a font integer 460 ⟩;
register: ⟨ Fetch a register 461 ⟩;
last_item: ⟨ Fetch an item in the current node, if appropriate 458 ⟩;
ignore_spaces: { trap unexpandable primitives }
if *cur_chr* = 1 **then** ⟨ Reset *cur_tok* for unexpandable primitives, goto restart 403 ⟩;
othercases ⟨ Complain that `\the` can't do this; give zero result 462 ⟩
endcases;
while *cur_val_level* > *level* **do** ⟨ Convert *cur_val* to a lower level 463 ⟩;
⟨ Fix the reference count, if any, and negate *cur_val* if *negative* 464 ⟩;
exit: **end**;

448. ⟨ Fetch a character code from some table 448 ⟩ ≡

```
begin scan_usv_num;
if m = math_code_base then
  begin cur_val1 ← ho(math_code(cur_val));
  if is_active_math_char(cur_val1) then cur_val1 ← "8000
  else if (math_class_field(cur_val1) > 7)∨(math_fam_field(cur_val1) > 15)∨(math_char_field(cur_val1) >
      255) then
    begin print_err("Extended␣mathchar␣used␣as␣mathchar");
    help2("A␣mathchar␣number␣must␣be␣between␣0␣and␣""7FFF.")
    ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val1); cur_val1 ← 0;
    end;
  cur_val1 ← (math_class_field(cur_val1) * "1000) + (math_fam_field(cur_val1) * "100) +
    math_char_field(cur_val1); scanned_result(cur_val1)(int_val)
  end
else if m = del_code_base then
    begin cur_val1 ← del_code(cur_val);
    if cur_val1 ≥ "40000000 then
      begin print_err("Extended␣delcode␣used␣as␣delcode");
      help2("A␣delimiter␣code␣must␣be␣between␣0␣and␣""7FFFFFF.")
      ("I␣changed␣this␣one␣to␣zero."); error; scanned_result(0)(int_val);
      end
    else begin scanned_result(cur_val1)(int_val);
      end
    end
  else if m < sf_code_base then scanned_result(equiv(m + cur_val))(int_val)
    else if m < math_code_base then scanned_result(equiv(m + cur_val) mod "10000)(int_val)
      else scanned_result(eqtb[m + cur_val].int)(int_val);
end
```

This code is used in section 447.

449. ⟨ Fetch a token list or font identifier, provided that *level* = *tok_val* 449 ⟩ ≡

```
if level ≠ tok_val then
  begin print_err("Missing␣number,␣treated␣as␣zero");
  help3("A␣number␣should␣have␣been␣here;␣I␣inserted␣`0´.")
  ("(If␣you␣can´t␣figure␣out␣why␣I␣needed␣to␣see␣a␣number,")
  ("look␣up␣`weird␣error´␣in␣the␣index␣to␣The␣TeXbook.)"); back_error;
  scanned_result(0)(dimen_val);
  end
else if cur_cmd ≤ assign_toks then
    begin if cur_cmd < assign_toks then { cur_cmd = toks_register }
      if m = mem_bot then
        begin scan_register_num;
        if cur_val < 256 then cur_val ← equiv(toks_base + cur_val)
        else begin find_sa_element(tok_val, cur_val, false);
          if cur_ptr = null then cur_val ← null
          else cur_val ← sa_ptr(cur_ptr);
          end;
        end
      else cur_val ← sa_ptr(m)
    else if cur_chr = XeTeX_inter_char_loc then
        begin scan_char_class_not_ignored; cur_ptr ← cur_val; scan_char_class_not_ignored;
        find_sa_element(inter_char_val, cur_ptr * char_class_limit + cur_val, false);
        if cur_ptr = null then cur_val ← null
        else cur_val ← sa_ptr(cur_ptr);
        end
      else cur_val ← equiv(m);
    cur_val_level ← tok_val;
    end
  else begin back_input; scan_font_ident; scanned_result(font_id_base + cur_val)(ident_val);
    end
```

This code is used in section 447.

450. Users refer to '`\the\spacefactor`' only in horizontal mode, and to '`\the\prevdepth`' only in vertical mode; so we put the associated mode in the modifier part of the *set_aux* command. The *set_page_int* command has modifier 0 or 1, for '`\deadcycles`' and '`\insertpenalties`', respectively. The *set_box_dimen* command is modified by either *width_offset*, *height_offset*, or *depth_offset*. And the *last_item* command is modified by either *int_val*, *dimen_val*, *glue_val*, *input_line_no_code*, or *badness_code*. ε-TeX inserts *last_node_type_code* after *glue_val* and adds the codes for its extensions: *eTeX_version_code*, ...

define *last_node_type_code* = *glue_val* + 1 { code for `\lastnodetype` }
define *input_line_no_code* = *glue_val* + 2 { code for `\inputlineno` }
define *badness_code* = *input_line_no_code* + 1 { code for `\badness` }

define *pdftex_first_rint_code* = *badness_code* + 1 { base for pdfTeX's command codes }
define *pdf_last_x_pos_code* = *pdftex_first_rint_code* + 6 { code for `\pdflastxpos` }
define *pdf_last_y_pos_code* = *pdftex_first_rint_code* + 7 { code for `\pdflastypos` }
define *elapsed_time_code* = *pdftex_first_rint_code* + 10 { code for `\elapsedtime` }
define *pdf_shell_escape_code* = *pdftex_first_rint_code* + 11 { code for `\shellescape` }
define *random_seed_code* = *pdftex_first_rint_code* + 12 { code for `\randomseed` }
define *pdftex_last_item_codes* = *pdftex_first_rint_code* + 12 { end of pdfTeX's command codes }

define *eTeX_int* = *pdftex_last_item_codes* + 1 { first of ε-TeX codes for integers }

define *XeTeX_int* = *eTeX_int* + 8 { base for XeTeX's command codes }
define *XeTeX_version_code* = *XeTeX_int* + 0 { code for `\XeTeXversion` }
define *XeTeX_count_glyphs_code* = *XeTeX_int* + 1 { code for `\XeTeXcountglyphs` }
define *XeTeX_count_variations_code* = *XeTeX_int* + 2 { Deprecated }
define *XeTeX_variation_code* = *XeTeX_int* + 3 { Deprecated }
define *XeTeX_find_variation_by_name_code* = *XeTeX_int* + 4 { Deprecated }
define *XeTeX_variation_min_code* = *XeTeX_int* + 5 { Deprecated }
define *XeTeX_variation_max_code* = *XeTeX_int* + 6 { Deprecated }
define *XeTeX_variation_default_code* = *XeTeX_int* + 7 { Deprecated }
define *XeTeX_count_features_code* = *XeTeX_int* + 8 { code for `\XeTeXcountfeatures` }
define *XeTeX_feature_code_code* = *XeTeX_int* + 9 { code for `\XeTeXfeaturecode` }
define *XeTeX_find_feature_by_name_code* = *XeTeX_int* + 10 { code for `\XeTeXfindfeaturebyname` }
define *XeTeX_is_exclusive_feature_code* = *XeTeX_int* + 11 { code for `\XeTeXisexclusivefeature` }
define *XeTeX_count_selectors_code* = *XeTeX_int* + 12 { code for `\XeTeXcountselectors` }
define *XeTeX_selector_code_code* = *XeTeX_int* + 13 { code for `\XeTeXselectorcode` }
define *XeTeX_find_selector_by_name_code* = *XeTeX_int* + 14 { code for `\XeTeXfindselectorbyname` }
define *XeTeX_is_default_selector_code* = *XeTeX_int* + 15 { code for `\XeTeXisdefaultselector` }
define *XeTeX_OT_count_scripts_code* = *XeTeX_int* + 16 { code for `\XeTeXOTcountscripts` }
define *XeTeX_OT_count_languages_code* = *XeTeX_int* + 17 { code for `\XeTeXOTcountlanguages` }
define *XeTeX_OT_count_features_code* = *XeTeX_int* + 18 { code for `\XeTeXOTcountfeatures` }
define *XeTeX_OT_script_code* = *XeTeX_int* + 19 { code for `\XeTeXOTscripttag` }
define *XeTeX_OT_language_code* = *XeTeX_int* + 20 { code for `\XeTeXOTlanguagetag` }
define *XeTeX_OT_feature_code* = *XeTeX_int* + 21 { code for `\XeTeXOTfeaturetag` }
define *XeTeX_map_char_to_glyph_code* = *XeTeX_int* + 22 { code for `\XeTeXcharglyph` }
define *XeTeX_glyph_index_code* = *XeTeX_int* + 23 { code for `\XeTeXglyphindex` }
define *XeTeX_font_type_code* = *XeTeX_int* + 24 { code for `\XeTeXfonttype` }
define *XeTeX_first_char_code* = *XeTeX_int* + 25 { code for `\XeTeXfirstfontchar` }
define *XeTeX_last_char_code* = *XeTeX_int* + 26 { code for `\XeTeXlastfontchar` }
define *XeTeX_pdf_page_count_code* = *XeTeX_int* + 27 { code for `\XeTeXpdfpagecount` }
define *XeTeX_last_item_codes* = *XeTeX_int* + 27 { end of XeTeX's command codes }

define *XeTeX_dim* = *XeTeX_last_item_codes* + 1 { first of XeTeX codes for dimensions }
define *XeTeX_glyph_bounds_code* = *XeTeX_dim* + 0 { code for `\XeTeXglyphbounds` }
define *XeTeX_last_dim_codes* = *XeTeX_dim* + 0 { end of XeTeX's command codes }

define *eTeX_dim* = *XeTeX_last_dim_codes* + 1 { first of ε-TeX codes for dimensions }

define *eTeX_glue* = *eTeX_dim* + 9 { first of ε-TeX codes for glue }
define *eTeX_mu* = *eTeX_glue* + 1 { first of ε-TeX codes for muglue }
define *eTeX_expr* = *eTeX_mu* + 1 { first of ε-TeX codes for expressions }

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("spacefactor", *set_aux*, *hmode*); *primitive*("prevdepth", *set_aux*, *vmode*);
primitive("deadcycles", *set_page_int*, 0); *primitive*("insertpenalties", *set_page_int*, 1);
primitive("wd", *set_box_dimen*, *width_offset*); *primitive*("ht", *set_box_dimen*, *height_offset*);
primitive("dp", *set_box_dimen*, *depth_offset*); *primitive*("lastpenalty", *last_item*, *int_val*);
primitive("lastkern", *last_item*, *dimen_val*); *primitive*("lastskip", *last_item*, *glue_val*);
primitive("inputlineno", *last_item*, *input_line_no_code*); *primitive*("badness", *last_item*, *badness_code*);
primitive("pdflastxpos", *last_item*, *pdf_last_x_pos_code*);
primitive("pdflastypos", *last_item*, *pdf_last_y_pos_code*);
primitive("elapsedtime", *last_item*, *elapsed_time_code*);
primitive("shellescape", *last_item*, *pdf_shell_escape_code*);
primitive("randomseed", *last_item*, *random_seed_code*);

451. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
set_aux: **if** *chr_code* = *vmode* **then** *print_esc*("prevdepth") **else** *print_esc*("spacefactor");
set_page_int: **if** *chr_code* = 0 **then** *print_esc*("deadcycles")
⟨ Cases of *set_page_int* for *print_cmd_chr* 1503 ⟩ **else** *print_esc*("insertpenalties");
set_box_dimen: **if** *chr_code* = *width_offset* **then** *print_esc*("wd")
else if *chr_code* = *height_offset* **then** *print_esc*("ht")
else *print_esc*("dp");
last_item: **case** *chr_code* **of**
int_val: *print_esc*("lastpenalty");
dimen_val: *print_esc*("lastkern");
glue_val: *print_esc*("lastskip");
input_line_no_code: *print_esc*("inputlineno");
⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩
pdf_last_x_pos_code: *print_esc*("pdflastxpos");
pdf_last_y_pos_code: *print_esc*("pdflastypos");
elapsed_time_code: *print_esc*("elapsedtime");
pdf_shell_escape_code: *print_esc*("shellescape");
random_seed_code: *print_esc*("randomseed");
othercases *print_esc*("badness")
endcases;

452. ⟨ Fetch the *space_factor* or the *prev_depth* 452 ⟩ ≡
if *abs*(*mode*) ≠ *m* **then**
begin *print_err*("Improper␣"); *print_cmd_chr*(*set_aux*, *m*);
help4("You␣can␣refer␣to␣\spacefactor␣only␣in␣horizontal␣mode;")
("you␣can␣refer␣to␣\prevdepth␣only␣in␣vertical␣mode;␣and")
("neither␣of␣these␣is␣meaningful␣inside␣\write.␣So")
("I´m␣forgetting␣what␣you␣said␣and␣using␣zero␣instead."); *error*;
if *level* ≠ *tok_val* **then** *scanned_result*(0)(*dimen_val*)
else *scanned_result*(0)(*int_val*);
end
else if *m* = *vmode* **then** *scanned_result*(*prev_depth*)(*dimen_val*)
else *scanned_result*(*space_factor*)(*int_val*)

This code is used in section 447.

453. ⟨ Fetch the *dead_cycles* or the *insert_penalties* 453 ⟩ ≡
begin if $m = 0$ **then** *cur_val* ← *dead_cycles*
⟨ Cases for 'Fetch the *dead_cycles* or the *insert_penalties*' 1504 ⟩
else *cur_val* ← *insert_penalties*; *cur_val_level* ← *int_val*;
end
This code is used in section 447.

454. ⟨ Fetch a box dimension 454 ⟩ ≡
begin *scan_register_num*; *fetch_box*(*q*);
if $q = null$ **then** *cur_val* ← 0 **else** *cur_val* ← *mem*[$q + m$].*sc*;
cur_val_level ← *dimen_val*;
end
This code is used in section 447.

455. Inside an `\output` routine, a user may wish to look at the page totals that were present at the moment when output was triggered.

define *max_dimen* ≡ ′7777777777 { $2^{30} - 1$ }

⟨ Fetch something on the *page_so_far* 455 ⟩ ≡
begin if (*page_contents* = *empty*) ∧ (¬*output_active*) **then**
if $m = 0$ **then** *cur_val* ← *max_dimen* **else** *cur_val* ← 0
else *cur_val* ← *page_so_far*[m];
cur_val_level ← *dimen_val*;
end
This code is used in section 447.

456. ⟨ Fetch the *prev_graf* 456 ⟩ ≡
if *mode* = 0 **then** *scanned_result*(0)(*int_val*) { *prev_graf* = 0 within `\write` }
else begin *nest*[*nest_ptr*] ← *cur_list*; *p* ← *nest_ptr*;
while *abs*(*nest*[*p*].*mode_field*) ≠ *vmode* **do** *decr*(*p*);
scanned_result(*nest*[*p*].*pg_field*)(*int_val*);
end
This code is used in section 447.

457. ⟨ Fetch the *par_shape* size 457 ⟩ ≡
begin if $m >$ *par_shape_loc* **then** ⟨ Fetch a penalties array element 1677 ⟩
else if *par_shape_ptr* = *null* **then** *cur_val* ← 0
else *cur_val* ← *info*(*par_shape_ptr*);
cur_val_level ← *int_val*;
end
This code is used in section 447.

458. Here is where `\lastpenalty`, `\lastkern`, `\lastskip`, and `\lastnodetype` are implemented. The reference count for `\lastskip` will be updated later.

We also handle `\inputlineno` and `\badness` here, because they are legal in similar contexts.

The macro *find_effective_tail_eTeX* sets *tx* to the last non-`\endM` node of the current list.

```
define find_effective_tail_eTeX ≡ tx ← tail;
    if ¬is_char_node(tx) then
      if (type(tx) = math_node) ∧ (subtype(tx) = end_M_code) then
        begin r ← head;
        repeat q ← r; r ← link(q);
        until r = tx;
        tx ← q;
        end
define find_effective_tail ≡ find_effective_tail_eTeX
```

⟨ Fetch an item in the current node, if appropriate 458 ⟩ ≡

```
if m ≥ input_line_no_code then
  if m ≥ eTeX_glue then ⟨ Process an expression and return 1591 ⟩
  else if m ≥ XeTeX_dim then
      begin case m of
        ⟨ Cases for fetching a dimension value 1458 ⟩
      end; { there are no other cases }
      cur_val_level ← dimen_val;
      end
    else begin case m of
      input_line_no_code: cur_val ← line;
      badness_code: cur_val ← last_badness;
      elapsed_time_code: cur_val ← get_microinterval;
      random_seed_code: cur_val ← random_seed;
      pdf_shell_escape_code: begin if shellenabledp then
          begin if restrictedshell then cur_val ← 2
          else cur_val ← 1;
          end
        else cur_val ← 0;
        end;
        ⟨ Cases for fetching an integer value 1454 ⟩
      end; { there are no other cases }
      cur_val_level ← int_val;
      end
else begin if cur_chr = glue_val then cur_val ← zero_glue else cur_val ← 0;
  find_effective_tail;
  if cur_chr = last_node_type_code then
    begin cur_val_level ← int_val;
    if (tx = head) ∨ (mode = 0) then cur_val ← −1;
    end
  else cur_val_level ← cur_chr;
  if ¬is_char_node(tx) ∧ (mode ≠ 0) then
    case cur_chr of
    int_val: if type(tx) = penalty_node then cur_val ← penalty(tx);
    dimen_val: if type(tx) = kern_node then cur_val ← width(tx);
    glue_val: if type(tx) = glue_node then
        begin cur_val ← glue_ptr(tx);
        if subtype(tx) = mu_glue then cur_val_level ← mu_val;
        end;
```

```
    last_node_type_code: if type(tx) ≤ unset_node then cur_val ← type(tx) + 1
      else cur_val ← unset_node + 2;
    end { there are no other cases }
  else if (mode = vmode) ∧ (tx = head) then
    case cur_chr of
    int_val: cur_val ← last_penalty;
    dimen_val: cur_val ← last_kern;
    glue_val: if last_glue ≠ max_halfword then cur_val ← last_glue;
    last_node_type_code: cur_val ← last_node_type;
    end; { there are no other cases }
  end
```

This code is used in section 447.

459. ⟨ Fetch a font dimension 459 ⟩ ≡

```
begin find_font_dimen(false); font_info[fmem_ptr].sc ← 0;
scanned_result(font_info[cur_val].sc)(dimen_val);
end
```

This code is used in section 447.

460. ⟨ Fetch a font integer 460 ⟩ ≡

```
begin scan_font_ident;
if m = 0 then scanned_result(hyphen_char[cur_val])(int_val)
else if m = 1 then scanned_result(skew_char[cur_val])(int_val)
  else begin n ← cur_val;
    if is_native_font(n) then scan_glyph_number(n)
    else scan_char_num;
    k ← cur_val;
    case m of
    lp_code_base: scanned_result(get_cp_code(n, k, left_side))(int_val);
    rp_code_base: scanned_result(get_cp_code(n, k, right_side))(int_val);
    end;
    end;
end
```

This code is used in section 447.

461. ⟨ Fetch a register 461 ⟩ ≡

```
begin if (m < mem_bot) ∨ (m > lo_mem_stat_max) then
  begin cur_val_level ← sa_type(m);
  if cur_val_level < glue_val then cur_val ← sa_int(m)
  else cur_val ← sa_ptr(m);
  end
else begin scan_register_num; cur_val_level ← m − mem_bot;
  if cur_val > 255 then
    begin find_sa_element(cur_val_level, cur_val, false);
    if cur_ptr = null then
      if cur_val_level < glue_val then cur_val ← 0
      else cur_val ← zero_glue
    else if cur_val_level < glue_val then cur_val ← sa_int(cur_ptr)
      else cur_val ← sa_ptr(cur_ptr);
    end
  else case cur_val_level of
    int_val: cur_val ← count(cur_val);
    dimen_val: cur_val ← dimen(cur_val);
    glue_val: cur_val ← skip(cur_val);
    mu_val: cur_val ← mu_skip(cur_val);
    end; { there are no other cases }
  end;
end
```

This code is used in section 447.

462. ⟨ Complain that `\the` can't do this; give zero result 462 ⟩ ≡

```
begin print_err("You␣can´t␣use␣`"); print_cmd_chr(cur_cmd, cur_chr); print("´␣after␣");
print_esc("the"); help1("I´m␣forgetting␣what␣you␣said␣and␣using␣zero␣instead."); error;
if level ≠ tok_val then scanned_result(0)(dimen_val)
else scanned_result(0)(int_val);
end
```

This code is used in section 447.

463. When a *glue_val* changes to a *dimen_val*, we use the width component of the glue; there is no need to decrease the reference count, since it has not yet been increased. When a *dimen_val* changes to an *int_val*, we use scaled points so that the value doesn't actually change. And when a *mu_val* changes to a *glue_val*, the value doesn't change either.

⟨ Convert *cur_val* to a lower level 463 ⟩ ≡

```
begin if cur_val_level = glue_val then cur_val ← width(cur_val)
else if cur_val_level = mu_val then mu_error;
decr(cur_val_level);
end
```

This code is used in section 447.

464. If *cur_val* points to a glue specification at this point, the reference count for the glue does not yet include the reference by *cur_val*. If *negative* is *true*, *cur_val_level* is known to be $\le$ *mu_val*.

```
⟨ Fix the reference count, if any, and negate cur_val if negative 464 ⟩ ≡
  if negative then
    if cur_val_level ≥ glue_val then
      begin cur_val ← new_spec(cur_val); ⟨ Negate all three glue components of cur_val 465 ⟩;
      end
    else negate(cur_val)
  else if (cur_val_level ≥ glue_val) ∧ (cur_val_level ≤ mu_val) then add_glue_ref(cur_val)
```

This code is used in section 447.

465.

```
⟨ Negate all three glue components of cur_val 465 ⟩ ≡
  begin negate(width(cur_val)); negate(stretch(cur_val)); negate(shrink(cur_val));
  end
```

This code is used in sections 464 and 1591.

466. Our next goal is to write the *scan_int* procedure, which scans anything that TeX treats as an integer. But first we might as well look at some simple applications of *scan_int* that have already been made inside of *scan_something_internal*.

467. ⟨ Declare procedures that scan restricted classes of integers 467 ⟩ ≡

```
procedure scan_glyph_number(f : internal_font_number);
        { scan a glyph ID for native font f, identified by Unicode value or name or glyph number }
  begin if scan_keyword("/") then   { set cp value by glyph name }
    begin scan_and_pack_name;   { result is in nameoffile }
    scanned_result(map_glyph_to_index(f))(int_val);
    end
  else if scan_keyword("u") then   { set cp value by unicode }
      begin scan_char_num; scanned_result(map_char_to_glyph(f, cur_val))(int_val);
      end
    else scan_int;
  end;
procedure scan_char_class;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > char_class_limit) then
    begin print_err("Bad␣character␣class");
    help2("A␣character␣class␣must␣be␣between␣0␣and␣4096.")
    ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
procedure scan_char_class_not_ignored;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > char_class_limit) then
    begin print_err("Bad␣character␣class");
    help2("A␣class␣for␣inter-character␣transitions␣must␣be␣between␣0␣and␣4095.")
    ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
procedure scan_eight_bit_int;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > 255) then
    begin print_err("Bad␣register␣code");
    help2("A␣register␣code␣or␣char␣class␣must␣be␣between␣0␣and␣255.")
    ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
```

See also sections 468, 469, 470, 471, and 1622.

This code is used in section 443.

468. ⟨ Declare procedures that scan restricted classes of integers 467 ⟩ +≡

```
procedure scan_usv_num;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > biggest_usv) then
    begin print_err("Bad␣character␣code");
    help2("A␣Unicode␣scalar␣value␣must␣be␣between␣0␣and␣""10FFFF.")
    ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
procedure scan_char_num;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > biggest_char) then
    begin print_err("Bad␣character␣code");
    help2("A␣character␣number␣must␣be␣between␣0␣and␣65535.")
    ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
```

469. While we're at it, we might as well deal with similar routines that will be needed later.

```
⟨ Declare procedures that scan restricted classes of integers 467 ⟩ +≡
procedure scan_xetex_math_char_int;
  begin scan_int;
  if is_active_math_char(cur_val) then
    begin if cur_val ≠ active_math_char then
      begin print_err("Bad active XeTeX math code");
      help2("Since I ignore class and family for active math chars,")
      ("I changed this one to ""1FFFFF."); int_error(cur_val); cur_val ← active_math_char;
      end
    end
  else if math_char_field(cur_val) > biggest_usv then
      begin print_err("Bad XeTeX math character code");
      help2("Since I expected a character number between 0 and ""10FFFF,")
      ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
      end;
  end;
procedure scan_math_class_int;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > 7) then
    begin print_err("Bad math class");
    help2("Since I expected to read a number between 0 and 7,")
    ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
procedure scan_math_fam_int;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > number_math_families − 1) then
    begin print_err("Bad math family");
    help2("Since I expected to read a number between 0 and 255,")
    ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
procedure scan_four_bit_int;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > 15) then
    begin print_err("Bad number");
    help2("Since I expected to read a number between 0 and 15,")
    ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
```

470.

```
⟨ Declare procedures that scan restricted classes of integers 467 ⟩ +≡
procedure scan_fifteen_bit_int;
  begin scan_int;
  if (cur_val < 0) ∨ (cur_val > ´77777) then
    begin print_err("Bad mathchar"); help2("A mathchar number must be between 0 and 32767.")
    ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
    end;
  end;
```

471. ⟨ Declare procedures that scan restricted classes of integers 467 ⟩ +≡
procedure *scan_delimiter_int*;
begin *scan_int*;
if (*cur_val* < 0) ∨ (*cur_val* > ´777777777) **then**
begin *print_err*("Bad␣delimiter␣code");
help2("A␣numeric␣delimiter␣code␣must␣be␣between␣0␣and␣2^{27}-1.")
("I␣changed␣this␣one␣to␣zero."); *int_error*(*cur_val*); *cur_val* ← 0;
end;
end;

472. An integer number can be preceded by any number of spaces and '+' or '-' signs. Then comes either a decimal constant (i.e., radix 10), an octal constant (i.e., radix 8, preceded by ´), a hexadecimal constant (radix 16, preceded by "), an alphabetic constant (preceded by `), or an internal variable. After scanning is complete, *cur_val* will contain the answer, which must be at most $2^{31} - 1 = 2147483647$ in absolute value. The value of *radix* is set to 10, 8, or 16 in the cases of decimal, octal, or hexadecimal constants, otherwise *radix* is set to zero. An optional space follows a constant.

define *octal_token* = *other_token* + "´" { apostrophe, indicates an octal constant }
define *hex_token* = *other_token* + """ { double quote, indicates a hex constant }
define *alpha_token* = *other_token* + "`" { reverse apostrophe, precedes alpha constants }
define *point_token* = *other_token* + "." { decimal point }
define *continental_point_token* = *other_token* + "," { decimal point, Eurostyle }

⟨ Global variables 13 ⟩ +≡
radix: *small_number*; { *scan_int* sets this to 8, 10, 16, or zero }

473. We initialize the following global variables just in case *expand* comes into action before any of the basic scanning routines has assigned them a value.

⟨ Set initial values of key variables 23 ⟩ +≡
cur_val ← 0; *cur_val_level* ← *int_val*; *radix* ← 0; *cur_order* ← *normal*;

474. The *scan_int* routine is used also to scan the integer part of a fraction; for example, the '3' in '3.14159' will be found by *scan_int*. The *scan_dimen* routine assumes that *cur_tok* = *point_token* after the integer part of such a fraction has been scanned by *scan_int*, and that the decimal point has been backed up to be scanned again.

procedure *scan_int*; { sets *cur_val* to an integer }
label *done*, *restart*;
var *negative*: *boolean*; { should the answer be negated? }
m: *integer*; { 2^{31} **div** *radix*, the threshold of danger }
d: *small_number*; { the digit just scanned }
vacuous: *boolean*; { have no digits appeared? }
OK_so_far: *boolean*; { has an error message been issued? }
begin *radix* ← 0; *OK_so_far* ← *true*;
⟨ Get the next non-blank non-sign token; set *negative* appropriately 475 ⟩;
restart: **if** *cur_tok* = *alpha_token* **then** ⟨ Scan an alphabetic character code into *cur_val* 476 ⟩
else if *cur_tok* = *cs_token_flag* + *frozen_primitive* **then**
⟨ Reset *cur_tok* for unexpandable primitives, goto restart 403 ⟩
else if (*cur_cmd* ≥ *min_internal*) ∧ (*cur_cmd* ≤ *max_internal*) **then**
scan_something_internal(*int_val*, *false*)
else ⟨ Scan a numeric constant 478 ⟩;
if *negative* **then** *negate*(*cur_val*);
end;

475. ⟨ Get the next non-blank non-sign token; set *negative* appropriately 475 ⟩ ≡

```
negative ← false;
repeat ⟨ Get the next non-blank non-call token 440 ⟩;
  if cur_tok = other_token + "-" then
    begin negative ← ¬negative; cur_tok ← other_token + "+";
    end;
until cur_tok ≠ other_token + "+"
```

This code is used in sections 474, 482, and 496.

476. A space is ignored after an alphabetic character constant, so that such constants behave like numeric ones.

⟨ Scan an alphabetic character code into *cur_val* 476 ⟩ ≡

```
begin get_token; { suppress macro expansion }
if cur_tok < cs_token_flag then
  begin cur_val ← cur_chr;
  if cur_cmd ≤ right_brace then
    if cur_cmd = right_brace then incr(align_state)
    else decr(align_state);
  end
else if cur_tok < cs_token_flag + single_base then cur_val ← cur_tok − cs_token_flag − active_base
  else cur_val ← cur_tok − cs_token_flag − single_base;
if cur_val > biggest_usv then
  begin print_err("Improper␣alphabetic␣constant");
  help2("A␣one-character␣control␣sequence␣belongs␣after␣a␣`␣mark.")
  ("So␣I´m␣essentially␣inserting␣\0␣here."); cur_val ← "0"; back_error;
  end
else ⟨ Scan an optional space 477 ⟩;
end
```

This code is used in section 474.

477. ⟨ Scan an optional space 477 ⟩ ≡

```
begin get_x_token;
if cur_cmd ≠ spacer then back_input;
end
```

This code is used in sections 476, 482, 490, and 1254.

478. ⟨ Scan a numeric constant 478 ⟩ ≡

```
begin radix ← 10; m ← 214748364;
if cur_tok = octal_token then
  begin radix ← 8; m ← ´2000000000; get_x_token;
  end
else if cur_tok = hex_token then
    begin radix ← 16; m ← ´1000000000; get_x_token;
    end;
vacuous ← true; cur_val ← 0;
⟨ Accumulate the constant until cur_tok is not a suitable digit 479 ⟩;
if vacuous then ⟨ Express astonishment that no number was here 480 ⟩
else if cur_cmd ≠ spacer then back_input;
end
```

This code is used in section 474.

479. **define** *infinity* ≡ ´17777777777 { the largest positive value that TEX knows }
define *zero_token* = *other_token* + "0" { zero, the smallest digit }
define *A_token* = *letter_token* + "A" { the smallest special hex digit }
define *other_A_token* = *other_token* + "A" { special hex digit of type *other_char* }

⟨ Accumulate the constant until *cur_tok* is not a suitable digit 479 ⟩ ≡

```
loop begin if (cur_tok < zero_token + radix) ∧ (cur_tok ≥ zero_token) ∧ (cur_tok ≤ zero_token + 9)
        then d ← cur_tok − zero_token
    else if radix = 16 then
        if (cur_tok ≤ A_token + 5) ∧ (cur_tok ≥ A_token) then d ← cur_tok − A_token + 10
        else if (cur_tok ≤ other_A_token + 5) ∧ (cur_tok ≥ other_A_token) then
            d ← cur_tok − other_A_token + 10
        else goto done
      else goto done;
    vacuous ← false;
    if (cur_val ≥ m) ∧ ((cur_val > m) ∨ (d > 7) ∨ (radix ≠ 10)) then
      begin if OK_so_far then
        begin print_err("Number␣too␣big");
        help2("I␣can␣only␣go␣up␣to␣2147483647=´17777777777=""7FFFFFFF,")
        ("so␣I´m␣using␣that␣number␣instead␣of␣yours."); error; cur_val ← infinity;
        OK_so_far ← false;
        end;
      end
    else cur_val ← cur_val * radix + d;
    get_x_token;
    end;
done:
```

This code is used in section 478.

480. ⟨ Express astonishment that no number was here 480 ⟩ ≡

```
begin print_err("Missing␣number,␣treated␣as␣zero");
help3("A␣number␣should␣have␣been␣here;␣I␣inserted␣`0´.")
("(If␣you␣can´t␣figure␣out␣why␣I␣needed␣to␣see␣a␣number,")
("look␣up␣`weird␣error´␣in␣the␣index␣to␣The␣TeXbook.)"); back_error;
end
```

This code is used in section 478.

481. The *scan_dimen* routine is similar to *scan_int*, but it sets *cur_val* to a *scaled* value, i.e., an integral number of sp. One of its main tasks is therefore to interpret the abbreviations for various kinds of units and to convert measurements to scaled points.

There are three parameters: *mu* is *true* if the finite units must be 'mu', while *mu* is *false* if 'mu' units are disallowed; *inf* is *true* if the infinite units 'fil', 'fill', 'filll' are permitted; and *shortcut* is *true* if *cur_val* already contains an integer and only the units need to be considered.

The order of infinity that was found in the case of infinite glue is returned in the global variable *cur_order*.

⟨ Global variables 13 ⟩ +≡

```
cur_order: glue_ord; { order of infinity found by scan_dimen }
```

482. Constructions like '`-´77 pt`' are legal dimensions, so *scan_dimen* may begin with *scan_int*. This explains why it is convenient to use *scan_int* also for the integer part of a decimal fraction.

Several branches of *scan_dimen* work with *cur_val* as an integer and with an auxiliary fraction f, so that the actual quantity of interest is $cur_val + f/2^{16}$. At the end of the routine, this "unpacked" representation is put into the single word *cur_val*, which suddenly switches significance from *integer* to *scaled*.

```
define attach_fraction = 88  { go here to pack cur_val and f into cur_val }
define attach_sign = 89  { go here when cur_val is correct except perhaps for sign }
define scan_normal_dimen ≡ scan_dimen(false, false, false)
procedure xetex_scan_dimen(mu, inf, shortcut, requires_units : boolean);  { sets cur_val to a dimension }
  label done, done1, done2, found, not_found, attach_fraction, attach_sign;
  var negative: boolean;  { should the answer be negated? }
    f: integer;  { numerator of a fraction whose denominator is 2^16 }
    ⟨ Local variables for dimension calculations 485 ⟩
  begin f ← 0; arith_error ← false; cur_order ← normal; negative ← false;
  if ¬shortcut then
    begin ⟨ Get the next non-blank non-sign token; set negative appropriately 475 ⟩;
    if (cur_cmd ≥ min_internal) ∧ (cur_cmd ≤ max_internal) then
      ⟨ Fetch an internal dimension and goto attach_sign, or fetch an internal integer 484 ⟩
    else begin back_input;
      if cur_tok = continental_point_token then cur_tok ← point_token;
      if cur_tok ≠ point_token then scan_int
      else begin radix ← 10; cur_val ← 0;
        end;
      if cur_tok = continental_point_token then cur_tok ← point_token;
      if (radix = 10) ∧ (cur_tok = point_token) then ⟨ Scan decimal fraction 487 ⟩;
      end;
    end;
  if cur_val < 0 then  { in this case f = 0 }
    begin negative ← ¬negative; negate(cur_val);
    end;
  if requires_units then
    begin ⟨ Scan units and set cur_val to x · (cur_val + f/2^16), where there are x sp per unit; goto
        attach_sign if the units are internal 488 ⟩;
    ⟨ Scan an optional space 477 ⟩;
    end
  else begin if cur_val ≥ ´40000 then arith_error ← true
    else cur_val ← cur_val * unity + f;
    end;
attach_sign: if arith_error ∨ (abs(cur_val) ≥ ´10000000000) then
    ⟨ Report that this dimension is out of range 495 ⟩;
  if negative then negate(cur_val);
  end;
procedure scan_dimen(mu, inf, shortcut : boolean);
  begin xetex_scan_dimen(mu, inf, shortcut, true);
  end;
```

483. For XeTeX, we have an additional version *scan_decimal*, like *scan_dimen* but without any scanning of units.

```
procedure scan_decimal;  { sets cur_val to a quantity expressed as a decimal fraction }
  begin xetex_scan_dimen(false, false, false, false);
  end;
```

484. ⟨ Fetch an internal dimension and **goto** *attach_sign*, or fetch an internal integer 484 ⟩ ≡

```
if mu then
  begin scan_something_internal(mu_val, false); ⟨ Coerce glue to a dimension 486 ⟩;
  if cur_val_level = mu_val then goto attach_sign;
  if cur_val_level ≠ int_val then mu_error;
  end
else begin scan_something_internal(dimen_val, false);
  if cur_val_level = dimen_val then goto attach_sign;
  end
```

This code is used in section 482.

485. ⟨ Local variables for dimension calculations 485 ⟩ ≡

```
num, denom: 1 .. 65536;  { conversion ratio for the scanned units }
k, kk: small_number;  { number of digits in a decimal fraction }
p, q: pointer;  { top of decimal digit stack }
v: scaled;  { an internal dimension }
save_cur_val: integer;  { temporary storage of cur_val }
```

This code is used in section 482.

486. The following code is executed when *scan_something_internal* was called asking for *mu_val*, when we really wanted a "mudimen" instead of "muglue."

⟨ Coerce glue to a dimension 486 ⟩ ≡

```
if cur_val_level ≥ glue_val then
  begin v ← width(cur_val); delete_glue_ref(cur_val); cur_val ← v;
  end
```

This code is used in sections 484 and 490.

487. When the following code is executed, we have *cur_tok* = *point_token*, but this token has been backed up using *back_input*; we must first discard it.

It turns out that a decimal point all by itself is equivalent to '`0.0`'. Let's hope people don't use that fact.

⟨ Scan decimal fraction 487 ⟩ ≡

```
begin k ← 0; p ← null; get_token;  { point_token is being re-scanned }
loop begin get_x_token;
  if (cur_tok > zero_token + 9) ∨ (cur_tok < zero_token) then goto done1;
  if k < 17 then  { digits for k ≥ 17 cannot affect the result }
    begin q ← get_avail; link(q) ← p; info(q) ← cur_tok − zero_token; p ← q; incr(k);
    end;
  end;
done1: for kk ← k downto 1 do
  begin dig[kk − 1] ← info(p); q ← p; p ← link(p); free_avail(q);
  end;
f ← round_decimals(k);
if cur_cmd ≠ spacer then back_input;
end
```

This code is used in section 482.

488. Now comes the harder part: At this point in the program, *cur_val* is a nonnegative integer and $f/2^{16}$ is a nonnegative fraction less than 1; we want to multiply the sum of these two quantities by the appropriate factor, based on the specified units, in order to produce a *scaled* result, and we want to do the calculation with fixed point arithmetic that does not overflow.

```
⟨ Scan units and set cur_val to x · (cur_val + f/2^16), where there are x sp per unit; goto attach_sign if the
      units are internal 488 ⟩ ≡
  if inf then ⟨ Scan for fil units; goto attach_fraction if found 489 ⟩;
  ⟨ Scan for units that are internal dimensions; goto attach_sign with cur_val set if found 490 ⟩;
  if mu then ⟨ Scan for mu units and goto attach_fraction 491 ⟩;
  if scan_keyword("true") then ⟨ Adjust for the magnification ratio 492 ⟩;
  if scan_keyword("pt") then goto attach_fraction;  { the easy case }
  ⟨ Scan for all other units and adjust cur_val and f accordingly; goto done in the case of scaled
      points 493 ⟩;
attach_fraction: if cur_val ≥ ´40000 then arith_error ← true
  else cur_val ← cur_val * unity + f;
done:
```

This code is used in section 482.

489. A specification like '`fillll`' or '`fill L L L`' will lead to two error messages (one for each additional keyword `"l"`).

```
⟨ Scan for fil units; goto attach_fraction if found 489 ⟩ ≡
  if scan_keyword("fil") then
    begin cur_order ← fil;
    while scan_keyword("l") do
      begin if cur_order = filll then
        begin print_err("Illegal␣unit␣of␣measure␣("); print("replaced␣by␣filll)");
        help1("I␣dddon´t␣go␣any␣higher␣than␣filll.");  error;
        end
      else incr(cur_order);
      end;
    goto attach_fraction;
    end
```

This code is used in section 488.

490. ⟨ Scan for units that are internal dimensions; **goto** *attach_sign* with *cur_val* set if found 490 ⟩ ≡

```
save_cur_val ← cur_val; ⟨ Get the next non-blank non-call token 440 ⟩;
if (cur_cmd < min_internal) ∨ (cur_cmd > max_internal) then back_input
else begin if mu then
    begin scan_something_internal(mu_val, false); ⟨ Coerce glue to a dimension 486 ⟩;
    if cur_val_level ≠ mu_val then mu_error;
    end
  else scan_something_internal(dimen_val, false);
  v ← cur_val; goto found;
  end;
if mu then goto not_found;
if scan_keyword("em") then v ← (⟨ The em width for cur_font 593 ⟩)
else if scan_keyword("ex") then v ← (⟨ The x-height for cur_font 594 ⟩)
  else goto not_found;
⟨ Scan an optional space 477 ⟩;
found: cur_val ← nx_plus_y(save_cur_val, v, xn_over_d(v, f, ´200000)); goto attach_sign;
not_found:
```

This code is used in section 488.

491. ⟨ Scan for `mu` units and **goto** *attach_fraction* 491 ⟩ ≡

```
if scan_keyword("mu") then goto attach_fraction
else begin print_err("Illegal␣unit␣of␣measure␣("); print("mu␣inserted)");
  help4("The␣unit␣of␣measurement␣in␣math␣glue␣must␣be␣mu.")
  ("To␣recover␣gracefully␣from␣this␣error,␣it´s␣best␣to")
  ("delete␣the␣erroneous␣units;␣e.g.,␣type␣`2´␣to␣delete")
  ("two␣letters.␣(See␣Chapter␣27␣of␣The␣TeXbook.)"); error; goto attach_fraction;
  end
```

This code is used in section 488.

492. ⟨ Adjust for the magnification ratio 492 ⟩ ≡

```
begin prepare_mag;
if mag ≠ 1000 then
  begin cur_val ← xn_over_d(cur_val, 1000, mag); f ← (1000 * f + ´200000 * remainder) div mag;
  cur_val ← cur_val + (f div ´200000); f ← f mod ´200000;
  end;
end
```

This code is used in section 488.

493. The necessary conversion factors can all be specified exactly as fractions whose numerator and denominator sum to 32768 or less. According to the definitions here, 2660 dd ≈ 1000.33297 mm; this agrees well with the value 1000.333 mm cited by Bosshard in *Technische Grundlagen zur Satzherstellung* (Bern, 1980).

```
define set_conversion_end(#) ≡ denom ← #;
    end
define set_conversion(#) ≡ begin num ← #; set_conversion_end
```

⟨ Scan for all other units and adjust *cur_val* and *f* accordingly; **goto** *done* in the case of scaled points 493 ⟩ ≡

```
if scan_keyword("in") then set_conversion(7227)(100)
else if scan_keyword("pc") then set_conversion(12)(1)
  else if scan_keyword("cm") then set_conversion(7227)(254)
    else if scan_keyword("mm") then set_conversion(7227)(2540)
      else if scan_keyword("bp") then set_conversion(7227)(7200)
        else if scan_keyword("dd") then set_conversion(1238)(1157)
          else if scan_keyword("cc") then set_conversion(14856)(1157)
            else if scan_keyword("sp") then goto done
              else ⟨ Complain about unknown unit and goto done2 494 ⟩;
cur_val ← xn_over_d(cur_val, num, denom); f ← (num * f + ´200000 * remainder) div denom;
cur_val ← cur_val + (f div ´200000); f ← f mod ´200000;
done2:
```

This code is used in section 488.

494. ⟨ Complain about unknown unit and **goto** *done2* 494 ⟩ ≡

```
begin print_err("Illegal␣unit␣of␣measure␣("); print("pt␣inserted)");
help6("Dimensions␣can␣be␣in␣units␣of␣em,␣ex,␣in,␣pt,␣pc,")
("cm,␣mm,␣dd,␣cc,␣bp,␣or␣sp;␣but␣yours␣is␣a␣new␣one!")
("I´ll␣assume␣that␣you␣meant␣to␣say␣pt,␣for␣printer´s␣points.")
("To␣recover␣gracefully␣from␣this␣error,␣it´s␣best␣to")
("delete␣the␣erroneous␣units;␣e.g.,␣type␣`2´␣to␣delete")
("two␣letters.␣(See␣Chapter␣27␣of␣The␣TeXbook.)"); error; goto done2;
end
```

This code is used in section 493.

495. ⟨ Report that this dimension is out of range 495 ⟩ ≡

```
begin print_err("Dimension␣too␣large");
help2("I␣can´t␣work␣with␣sizes␣bigger␣than␣about␣19␣feet.")
("Continue␣and␣I´ll␣use␣the␣largest␣value␣I␣can.");
error; cur_val ← max_dimen; arith_error ← false;
end
```

This code is used in section 482.

496. The final member of TeX's value-scanning trio is *scan_glue*, which makes *cur_val* point to a glue specification. The reference count of that glue spec will take account of the fact that *cur_val* is pointing to it.

The *level* parameter should be either *glue_val* or *mu_val*.

Since *scan_dimen* was so much more complex than *scan_int*, we might expect *scan_glue* to be even worse. But fortunately, it is very simple, since most of the work has already been done.

```
procedure scan_glue(level : small_number); { sets cur_val to a glue spec pointer }
  label exit;
  var negative: boolean; { should the answer be negated? }
    q: pointer; { new glue specification }
    mu: boolean; { does level = mu_val? }
  begin mu ← (level = mu_val); ⟨ Get the next non-blank non-sign token; set negative appropriately 475 ⟩;
  if (cur_cmd ≥ min_internal) ∧ (cur_cmd ≤ max_internal) then
    begin scan_something_internal(level, negative);
    if cur_val_level ≥ glue_val then
      begin if cur_val_level ≠ level then mu_error;
      return;
      end;
    if cur_val_level = int_val then scan_dimen(mu, false, true)
    else if level = mu_val then mu_error;
    end
  else begin back_input; scan_dimen(mu, false, false);
    if negative then negate(cur_val);
    end;
  ⟨ Create a new glue specification whose width is cur_val; scan for its stretch and shrink components 497 ⟩;
exit: end;

  ⟨ Declare procedures needed for expressions 1593 ⟩
```

497. ⟨ Create a new glue specification whose width is *cur_val*; scan for its stretch and shrink components 497 ⟩ ≡

```
  q ← new_spec(zero_glue); width(q) ← cur_val;
  if scan_keyword("plus") then
    begin scan_dimen(mu, true, false); stretch(q) ← cur_val; stretch_order(q) ← cur_order;
    end;
  if scan_keyword("minus") then
    begin scan_dimen(mu, true, false); shrink(q) ← cur_val; shrink_order(q) ← cur_order;
    end;
  cur_val ← q
```

This code is used in section 496.

498. Here's a similar procedure that returns a pointer to a rule node. This routine is called just after TeX has seen `\hrule` or `\vrule`; therefore *cur_cmd* will be either *hrule* or *vrule*. The idea is to store the default rule dimensions in the node, then to override them if '`height`' or '`width`' or '`depth`' specifications are found (in any order).

```
  define default_rule = 26214  { 0.4 pt }
function scan_rule_spec: pointer;
  label reswitch;
  var q: pointer;  { the rule node being created }
  begin q ← new_rule;  { width, depth, and height all equal null_flag now }
  if cur_cmd = vrule then width(q) ← default_rule
  else begin height(q) ← default_rule; depth(q) ← 0;
    end;
reswitch: if scan_keyword("width") then
    begin scan_normal_dimen; width(q) ← cur_val; goto reswitch;
    end;
  if scan_keyword("height") then
    begin scan_normal_dimen; height(q) ← cur_val; goto reswitch;
    end;
  if scan_keyword("depth") then
    begin scan_normal_dimen; depth(q) ← cur_val; goto reswitch;
    end;
  scan_rule_spec ← q;
  end;
```

499. Building token lists. The token lists for macros and for other things like `\mark` and `\output` and `\write` are produced by a procedure called *scan_toks*.

Before we get into the details of *scan_toks*, let's consider a much simpler task, that of converting the current string into a token list. The *str_toks* function does this; it classifies spaces as type *spacer* and everything else as type *other_char*.

The token list created by *str_toks* begins at *link*(*temp_head*) and ends at the value p that is returned. (If $p = temp_head$, the list is empty.)

The *str_toks_cat* function is the same, except that the catcode *cat* is stamped on all the characters, unless zero is passed in which case it chooses *spacer* or *other_char* automatically.

```
⟨ Declare ε-TeX procedures for token lists 1493 ⟩
function str_toks_cat(b : pool_pointer; cat : small_number): pointer;
        { changes the string str_pool[b .. pool_ptr] to a token list }
  var p: pointer; { tail of the token list }
    q: pointer; { new node being added to the token list via store_new_token }
    t: halfword; { token being appended }
    k: pool_pointer; { index into str_pool }
  begin str_room(1); p ← temp_head; link(p) ← null; k ← b;
  while k < pool_ptr do
    begin t ← so(str_pool[k]);
    if (t = "␣") ∧ (cat = 0) then t ← space_token
    else begin if (t ≥ "D800) ∧ (t ≤ "DBFF) ∧ (k + 1 < pool_ptr) ∧ (so(str_pool[k + 1]) ≥
          "DC00) ∧ (so(str_pool[k + 1]) ≤ "DFFF) then
        begin incr(k); t ← "10000 + (t − "D800) ∗ "400 + (so(str_pool[k]) − "DC00);
        end;
      if cat = 0 then t ← other_token + t
      else if cat = active_char then t ← cs_token_flag + active_base + t
        else t ← max_char_val ∗ cat + t;
      end;
    fast_store_new_token(t); incr(k);
    end;
  pool_ptr ← b; str_toks_cat ← p;
  end;
function str_toks(b : pool_pointer): pointer;
  begin str_toks ← str_toks_cat(b, 0);
  end;
```

500. The main reason for wanting *str_toks* is the next function, *the_toks*, which has similar input/output characteristics.

This procedure is supposed to scan something like '`\skip\count12`', i.e., whatever can follow '`\the`', and it constructs a token list containing something like '`-3.0pt minus 0.5fill`'.

```
function the_toks: pointer;
  label exit;
  var old_setting: 0 .. max_selector;  { holds selector setting }
    p, q, r: pointer;  { used for copying a token list }
    b: pool_pointer;  { base of temporary string }
    c: small_number;  { value of cur_chr }
  begin ⟨ Handle \unexpanded or \detokenize and return 1498 ⟩;
  get_x_token; scan_something_internal(tok_val, false);
  if cur_val_level ≥ ident_val then ⟨ Copy the token list 501 ⟩
  else begin old_setting ← selector; selector ← new_string; b ← pool_ptr;
    case cur_val_level of
    int_val: print_int(cur_val);
    dimen_val: begin print_scaled(cur_val); print("pt");
      end;
    glue_val: begin print_spec(cur_val, "pt"); delete_glue_ref(cur_val);
      end;
    mu_val: begin print_spec(cur_val, "mu"); delete_glue_ref(cur_val);
      end;
    end;  { there are no other cases }
    selector ← old_setting; the_toks ← str_toks(b);
    end;
exit: end;
```

501. ⟨ Copy the token list 501 ⟩ ≡

```
  begin p ← temp_head; link(p) ← null;
  if cur_val_level = ident_val then store_new_token(cs_token_flag + cur_val)
  else if cur_val ≠ null then
      begin r ← link(cur_val);  { do not copy the reference count }
      while r ≠ null do
        begin fast_store_new_token(info(r)); r ← link(r);
        end;
      end;
  the_toks ← p;
  end
```

This code is used in section 500.

502. Here's part of the *expand* subroutine that we are now ready to complete:

```
procedure ins_the_toks;
  begin link(garbage) ← the_toks; ins_list(link(temp_head));
  end;
```

503. The primitives `\number`, `\romannumeral`, `\string`, `\meaning`, `\fontname`, and `\jobname` are defined as follows.

ε-TeX adds `\eTeXrevision` such that *job_name_code* remains last.

```
define number_code = 0 { command code for \number }
define roman_numeral_code = 1 { command code for \romannumeral }
define string_code = 2 { command code for \string }
define meaning_code = 3 { command code for \meaning }
define font_name_code = 4 { command code for \fontname }
define etex_convert_base = 5 { base for ε-TeX's command codes }
define eTeX_revision_code = etex_convert_base { command code for \eTeXrevision }
define etex_convert_codes = etex_convert_base + 1 { end of ε-TeX's command codes }
define expanded_code = etex_convert_codes { command code for \expanded }
define pdftex_first_expand_code = expanded_code + 1 { base for pdfTeX's command codes }
define left_margin_kern_code = pdftex_first_expand_code + 9 { command code for \leftmarginkern }
define right_margin_kern_code = pdftex_first_expand_code + 10 { command code for \rightmarginkern }
define pdf_strcmp_code = pdftex_first_expand_code + 11 { command code for \strcmp }
define pdf_creation_date_code = pdftex_first_expand_code + 15 { command code for \creationdate }
define pdf_file_mod_date_code = pdftex_first_expand_code + 16 { command code for \filemoddate }
define pdf_file_size_code = pdftex_first_expand_code + 17 { command code for \filesize }
define pdf_mdfive_sum_code = pdftex_first_expand_code + 18 { command code for \mdfivesum }
define pdf_file_dump_code = pdftex_first_expand_code + 19 { command code for \filedump }
define uniform_deviate_code = pdftex_first_expand_code + 22 { command code for \uniformdeviate }
define normal_deviate_code = pdftex_first_expand_code + 23 { command code for \normaldeviate }
define pdftex_convert_codes = pdftex_first_expand_code + 26 { end of pdfTeX's command codes }
define XeTeX_first_expand_code = pdftex_convert_codes { base for XeTeX's command codes }
define XeTeX_revision_code = XeTeX_first_expand_code + 0 { command code for \XeTeXrevision }
define XeTeX_variation_name_code = XeTeX_first_expand_code + 1
    { command code for \XeTeXvariationname }
define XeTeX_feature_name_code = XeTeX_first_expand_code + 2
    { command code for \XeTeXfeaturename }
define XeTeX_selector_name_code = XeTeX_first_expand_code + 3
    { command code for \XeTeXselectornamename }
define XeTeX_glyph_name_code = XeTeX_first_expand_code + 4 { command code for \XeTeXglyphname }
define XeTeX_Uchar_code = XeTeX_first_expand_code + 5 { command code for \Uchar }
define XeTeX_Ucharcat_code = XeTeX_first_expand_code + 6 { command code for \Ucharcat }
define XeTeX_convert_codes = XeTeX_first_expand_code + 7 { end of XeTeX's command codes }
define job_name_code = XeTeX_convert_codes { command code for \jobname }
```

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡

```
primitive("number", convert, number_code);
primitive("romannumeral", convert, roman_numeral_code);
primitive("string", convert, string_code);
primitive("meaning", convert, meaning_code);
primitive("fontname", convert, font_name_code);

primitive("expanded", convert, expanded_code);

primitive("leftmarginkern", convert, left_margin_kern_code);
primitive("rightmarginkern", convert, right_margin_kern_code);
primitive("creationdate", convert, pdf_creation_date_code);
primitive("filemoddate", convert, pdf_file_mod_date_code);
primitive("filesize", convert, pdf_file_size_code);
primitive("mdfivesum", convert, pdf_mdfive_sum_code);
primitive("filedump", convert, pdf_file_dump_code);
```

primitive("strcmp", *convert*, *pdf_strcmp_code*);
primitive("uniformdeviate", *convert*, *uniform_deviate_code*);
primitive("normaldeviate", *convert*, *normal_deviate_code*);

primitive("jobname", *convert*, *job_name_code*);
primitive("Uchar", *convert*, *XeTeX_Uchar_code*);
primitive("Ucharcat", *convert*, *XeTeX_Ucharcat_code*);

504. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
convert: case chr_code of
  number_code: print_esc("number");
  roman_numeral_code: print_esc("romannumeral");
  string_code: print_esc("string");
  meaning_code: print_esc("meaning");
  font_name_code: print_esc("fontname");
  eTeX_revision_code: print_esc("eTeXrevision");
  expanded_code: print_esc("expanded");
  left_margin_kern_code: print_esc("leftmarginkern");
  right_margin_kern_code: print_esc("rightmarginkern");
  pdf_creation_date_code: print_esc("creationdate");
  pdf_file_mod_date_code: print_esc("filemoddate");
  pdf_file_size_code: print_esc("filesize");
  pdf_mdfive_sum_code: print_esc("mdfivesum");
  pdf_file_dump_code: print_esc("filedump");
  pdf_strcmp_code: print_esc("strcmp");
  uniform_deviate_code: print_esc("uniformdeviate");
  normal_deviate_code: print_esc("normaldeviate");
    ⟨ Cases of convert for print_cmd_chr 1459 ⟩
  othercases print_esc("jobname")
  endcases;
```

505. The procedure *conv_toks* uses *str_toks* to insert the token list for *convert* functions into the scanner; '`\outer`' control sequences are allowed to follow '`\string`' and '`\meaning`'.

The extra temp string u is needed because *pdf_scan_ext_toks* incorporates any pending string in its output. In order to save such a pending string, we have to create a temporary string that is destroyed immediately after.

```
define save_cur_string ≡
        if str_start_macro(str_ptr) < pool_ptr then u ← make_string
        else u ← 0
define restore_cur_string ≡
        if u ≠ 0 then decr(str_ptr)
procedure conv_toks;
  var old_setting: 0 .. max_selector; { holds selector setting }
    save_warning_index, save_def_ref: pointer; boolvar: boolean; { temp boolean }
    s: str_number; u: str_number; j: integer; c: small_number; { desired type of conversion }
    save_scanner_status: small_number; { scanner_status upon entry }
    b: pool_pointer; { base of temporary string }
    fnt, arg1, arg2: integer; { args for XeTeX extensions }
    font_name_str: str_number; { local vars for \fontname quoting extension }
    i: small_number; quote_char: UTF16_code; cat: small_number;
        { desired catcode, or 0 for automatic spacer/other_char selection }
    saved_chr: UnicodeScalar; p, q: pointer;
  begin cat ← 0; c ← cur_chr; ⟨ Scan the argument for command c 506 ⟩;
  old_setting ← selector; selector ← new_string; b ← pool_ptr; ⟨ Print the result of command c 507 ⟩;
  selector ← old_setting; link(garbage) ← str_toks_cat(b, cat); ins_list(link(temp_head));
  end;
```

506. Not all catcode values are allowed by `\Ucharcat`:

define *illegal_Ucharcat_catcode*(#) ≡ (# < *left_brace*) ∨ (# > *active_char*) ∨ (# = *out_param*) ∨ (# = *ignore*)

⟨ Scan the argument for command *c* 506 ⟩ ≡

case *c* **of**
number_code, *roman_numeral_code*: *scan_int*;
string_code, *meaning_code*: **begin** *save_scanner_status* ← *scanner_status*; *scanner_status* ← *normal*;
get_token; *scanner_status* ← *save_scanner_status*;
end;
font_name_code: *scan_font_ident*;
eTeX_revision_code: *do_nothing*;
expanded_code: **begin** *save_scanner_status* ← *scanner_status*; *save_warning_index* ← *warning_index*;
save_def_ref ← *def_ref*; *save_cur_string*; *scan_pdf_ext_toks*; *warning_index* ← *save_warning_index*;
scanner_status ← *save_scanner_status*; *ins_list*(*link*(*def_ref*)); *free_avail*(*def_ref*);
def_ref ← *save_def_ref*; *restore_cur_string*; **return**;
end;
left_margin_kern_code, *right_margin_kern_code*: **begin** *scan_register_num*; *fetch_box*(*p*);
if (*p* = *null*) ∨ (*type*(*p*) ≠ *hlist_node*) **then** *pdf_error*("marginkern", "a␣non-empty␣hbox␣expected")
end;
pdf_creation_date_code: **begin** *b* ← *pool_ptr*; *getcreationdate*; *link*(*garbage*) ← *str_toks*(*b*);
ins_list(*link*(*temp_head*)); **return**;
end;
pdf_file_mod_date_code: **begin** *save_scanner_status* ← *scanner_status*;
save_warning_index ← *warning_index*; *save_def_ref* ← *def_ref*; *save_cur_string*; *scan_pdf_ext_toks*;
if *selector* = *new_string* **then**
pdf_error("tokens", "tokens_to_string()␣called␣while␣selector␣=␣new_string");
old_setting ← *selector*; *selector* ← *new_string*;
show_token_list(*link*(*def_ref*), *null*, *pool_size* − *pool_ptr*); *selector* ← *old_setting*; *s* ← *make_string*;
delete_token_ref(*def_ref*); *def_ref* ← *save_def_ref*; *warning_index* ← *save_warning_index*;
scanner_status ← *save_scanner_status*; *b* ← *pool_ptr*; *getfilemoddate*(*s*); *link*(*garbage*) ← *str_toks*(*b*);
if *flushable*(*s*) **then** *flush_string*;
ins_list(*link*(*temp_head*)); *restore_cur_string*; **return**;
end;
pdf_file_size_code: **begin** *save_scanner_status* ← *scanner_status*; *save_warning_index* ← *warning_index*;
save_def_ref ← *def_ref*; *save_cur_string*; *scan_pdf_ext_toks*;
if *selector* = *new_string* **then**
pdf_error("tokens", "tokens_to_string()␣called␣while␣selector␣=␣new_string");
old_setting ← *selector*; *selector* ← *new_string*;
show_token_list(*link*(*def_ref*), *null*, *pool_size* − *pool_ptr*); *selector* ← *old_setting*; *s* ← *make_string*;
delete_token_ref(*def_ref*); *def_ref* ← *save_def_ref*; *warning_index* ← *save_warning_index*;
scanner_status ← *save_scanner_status*; *b* ← *pool_ptr*; *getfilesize*(*s*); *link*(*garbage*) ← *str_toks*(*b*);
if *flushable*(*s*) **then** *flush_string*;
ins_list(*link*(*temp_head*)); *restore_cur_string*; **return**;
end;
pdf_mdfive_sum_code: **begin** *save_scanner_status* ← *scanner_status*;
save_warning_index ← *warning_index*; *save_def_ref* ← *def_ref*; *save_cur_string*;
boolvar ← *scan_keyword*("file"); *scan_pdf_ext_toks*;
if *selector* = *new_string* **then**
pdf_error("tokens", "tokens_to_string()␣called␣while␣selector␣=␣new_string");
old_setting ← *selector*; *selector* ← *new_string*; *show_token_list*(*link*(*def_ref*), *null*, *pool_size* − *pool_ptr*);
selector ← *old_setting*; *s* ← *make_string*; *delete_token_ref*(*def_ref*); *def_ref* ← *save_def_ref*;
warning_index ← *save_warning_index*; *scanner_status* ← *save_scanner_status*; *b* ← *pool_ptr*;
getmd5sum(*s*, *boolvar*); *link*(*garbage*) ← *str_toks*(*b*);

```
  if flushable(s) then flush_string;
  ins_list(link(temp_head)); restore_cur_string; return;
  end;
pdf_file_dump_code: begin save_scanner_status ← scanner_status; save_warning_index ← warning_index;
  save_def_ref ← def_ref; save_cur_string; { scan offset }
  cur_val ← 0;
  if (scan_keyword("offset")) then
    begin scan_int;
    if (cur_val < 0) then
      begin print_err("Bad file offset");
      help2("A file offset must be between 0 and 2^{31}-1,")
      ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
      end;
    end;
  i ← cur_val; { scan length }
  cur_val ← 0;
  if (scan_keyword("length")) then
    begin scan_int;
    if (cur_val < 0) then
      begin print_err("Bad dump length");
      help2("A dump length must be between 0 and 2^{31}-1,")
      ("I changed this one to zero."); int_error(cur_val); cur_val ← 0;
      end;
    end;
  j ← cur_val; { scan file name }
  scan_pdf_ext_toks;
  if selector = new_string then
    pdf_error("tokens", "tokens_to_string() called while selector = new_string");
  old_setting ← selector; selector ← new_string;
  show_token_list(link(def_ref), null, pool_size − pool_ptr); selector ← old_setting; s ← make_string;
  delete_token_ref(def_ref); def_ref ← save_def_ref; warning_index ← save_warning_index;
  scanner_status ← save_scanner_status; b ← pool_ptr; getfiledump(s, i, j); link(garbage) ← str_toks(b);
  if flushable(s) then flush_string;
  ins_list(link(temp_head)); restore_cur_string; return;
  end;
pdf_strcmp_code: begin save_scanner_status ← scanner_status; save_warning_index ← warning_index;
  save_def_ref ← def_ref; save_cur_string; compare_strings; def_ref ← save_def_ref;
  warning_index ← save_warning_index; scanner_status ← save_scanner_status; restore_cur_string;
  end;
XeTeX_Uchar_code: scan_usv_num;
XeTeX_Ucharcat_code: begin scan_usv_num; saved_chr ← cur_val; scan_int;
  if illegal_Ucharcat_catcode(cur_val) then
    begin print_err("Invalid code("); print_int(cur_val);
    print("), should be in the ranges 1..4, 6..8, 10..13");
    help1("I´m going to use 12 instead of that illegal code value.");
    error; cat ← 12;
    end
  else cat ← cur_val;
  cur_val ← saved_chr;
  end;
  ⟨ Cases of 'Scan the argument for command c' 1460 ⟩
job_name_code: if job_name = 0 then open_log_file;
```

uniform_deviate_code: *scan_int*;
normal_deviate_code: *do_nothing*;
end { there are no other cases }

This code is used in section 505.

507. ⟨ Print the result of command *c* 507 ⟩ ≡

```
case c of
number_code: print_int(cur_val);
roman_numeral_code: print_roman_int(cur_val);
string_code: if cur_cs ≠ 0 then sprint_cs(cur_cs)
  else print_char(cur_chr);
meaning_code: print_meaning;
font_name_code: begin font_name_str ← font_name[cur_val];
  if is_native_font(cur_val) then
    begin quote_char ← """";
    for i ← 0 to length(font_name_str) − 1 do
      if str_pool[str_start_macro(font_name_str) + i] = """" then quote_char ← "´";
    print_char(quote_char); print(font_name_str); print_char(quote_char);
    end
  else print(font_name_str);
  if font_size[cur_val] ≠ font_dsize[cur_val] then
    begin print("␣at␣"); print_scaled(font_size[cur_val]); print("pt");
    end;
  end;
eTeX_revision_code: print(eTeX_revision);
left_margin_kern_code: begin p ← list_ptr(p);
  while (p ≠ null) ∧ (cp_skipable(p) ∨ ((¬is_char_node(p)) ∧ (type(p) = glue_node) ∧ (subtype(p) =
      left_skip_code + 1))) do p ← link(p);
  if (p ≠ null) ∧ (¬is_char_node(p)) ∧ (type(p) = margin_kern_node) ∧ (subtype(p) = left_side) then
    print_scaled(width(p))
  else print("0");
  print("pt");
  end;
right_margin_kern_code: begin q ← list_ptr(p); p ← prev_rightmost(q, null);
  while (p ≠ null) ∧ (cp_skipable(p) ∨ ((¬is_char_node(p)) ∧ (type(p) = glue_node) ∧ (subtype(p) =
      right_skip_code + 1))) do p ← prev_rightmost(q, p);
  if (p ≠ null) ∧ (¬is_char_node(p)) ∧ (type(p) = margin_kern_node) ∧ (subtype(p) = right_side) then
    print_scaled(width(p))
  else print("0");
  print("pt");
  end;
pdf_strcmp_code: print_int(cur_val);
uniform_deviate_code: print_int(unif_rand(cur_val));
normal_deviate_code: print_int(norm_rand);
XeTeX_Uchar_code, XeTeX_Ucharcat_code: print_char(cur_val);
  ⟨ Cases of 'Print the result of command c' 1461 ⟩
job_name_code: print_file_name(job_name, 0, 0);
end { there are no other cases }
```

This code is used in section 505.

508. Now we can't postpone the difficulties any longer; we must bravely tackle *scan_toks*. This function returns a pointer to the tail of a new token list, and it also makes *def_ref* point to the reference count at the head of that list.

There are two boolean parameters, *macro_def* and *xpand*. If *macro_def* is true, the goal is to create the token list for a macro definition; otherwise the goal is to create the token list for some other TeX primitive: `\mark`, `\output`, `\everypar`, `\lowercase`, `\uppercase`, `\message`, `\errmessage`, `\write`, or `\special`. In the latter cases a left brace must be scanned next; this left brace will not be part of the token list, nor will the matching right brace that comes at the end. If *xpand* is false, the token list will simply be copied from the input using *get_token*. Otherwise all expandable tokens will be expanded until unexpandable tokens are left, except that the results of expanding '`\the`' are not expanded further. If both *macro_def* and *xpand* are true, the expansion applies only to the macro body (i.e., to the material following the first *left_brace* character).

The value of *cur_cs* when *scan_toks* begins should be the *eqtb* address of the control sequence to display in "runaway" error messages.

```
function scan_toks(macro_def, xpand : boolean): pointer;
  label found, continue, done, done1, done2;
  var t: halfword; { token representing the highest parameter number }
    s: halfword; { saved token }
    p: pointer; { tail of the token list being built }
    q: pointer; { new node being added to the token list via store_new_token }
    unbalance: halfword; { number of unmatched left braces }
    hash_brace: halfword; { possible '#{' token }
  begin if macro_def then scanner_status ← defining else scanner_status ← absorbing;
  warning_index ← cur_cs; def_ref ← get_avail; token_ref_count(def_ref) ← null; p ← def_ref;
  hash_brace ← 0; t ← zero_token;
  if macro_def then ⟨Scan and build the parameter part of the macro definition 509⟩
  else scan_left_brace; { remove the compulsory left brace }
  ⟨Scan and build the body of the token list; goto found when finished 512⟩;
found: scanner_status ← normal;
  if hash_brace ≠ 0 then store_new_token(hash_brace);
  scan_toks ← p;
  end;
```

509. ⟨Scan and build the parameter part of the macro definition 509⟩ ≡

```
  begin loop
    begin continue: get_token; { set cur_cmd, cur_chr, cur_tok }
    if cur_tok < right_brace_limit then goto done1;
    if cur_cmd = mac_param then ⟨If the next character is a parameter number, make cur_tok a match
        token; but if it is a left brace, store 'left_brace, end_match', set hash_brace, and goto done 511⟩;
    store_new_token(cur_tok);
    end;
done1: store_new_token(end_match_token);
  if cur_cmd = right_brace then ⟨Express shock at the missing left brace; goto found 510⟩;
done: end
```

This code is used in section 508.

510. ⟨Express shock at the missing left brace; goto *found* 510⟩ ≡

```
  begin print_err("Missing␣{␣inserted"); incr(align_state);
  help2("Where␣was␣the␣left␣brace?␣You␣said␣something␣like␣`\def\a}´,")
  ("which␣I´m␣going␣to␣interpret␣as␣`\def\a{}´."); error; goto found;
  end
```

This code is used in section 509.

511. ⟨ If the next character is a parameter number, make *cur_tok* a *match* token; but if it is a left brace, store '*left_brace*, *end_match*', set *hash_brace*, and **goto** *done* 511 ⟩ ≡

```
begin s ← match_token + cur_chr; get_token;
if cur_tok < left_brace_limit then
  begin hash_brace ← cur_tok; store_new_token(cur_tok); store_new_token(end_match_token);
  goto done;
  end;
if t = zero_token + 9 then
  begin print_err("You already have nine parameters");
  help2("I'm going to ignore the # sign you just used,")
  ("as well as the token that followed it."); error; goto continue;
  end
else begin incr(t);
  if cur_tok ≠ t then
    begin print_err("Parameters must be numbered consecutively");
    help2("I've inserted the digit you should have used after the #.")
    ("Type `1' to delete what you did use."); back_error;
    end;
  cur_tok ← s;
  end;
end
```

This code is used in section 509.

512. ⟨ Scan and build the body of the token list; **goto** *found* when finished 512 ⟩ ≡

```
unbalance ← 1;
loop begin if xpand then ⟨ Expand the next part of the input 513 ⟩
  else get_token;
  if cur_tok < right_brace_limit then
    if cur_cmd < right_brace then incr(unbalance)
    else begin decr(unbalance);
      if unbalance = 0 then goto found;
      end
  else if cur_cmd = mac_param then
      if macro_def then ⟨ Look for parameter number or ## 514 ⟩;
  store_new_token(cur_tok);
  end
```

This code is used in section 508.

513. Here we insert an entire token list created by *the_toks* without expanding it further.

```
⟨ Expand the next part of the input 513 ⟩ ≡
  begin loop
    begin get_next;
    if cur_cmd ≥ call then
      if info(link(cur_chr)) = protected_token then
        begin cur_cmd ← relax; cur_chr ← no_expand_flag;
        end;
    if cur_cmd ≤ max_command then goto done2;
    if cur_cmd ≠ the then expand
    else begin q ← the_toks;
      if link(temp_head) ≠ null then
        begin link(p) ← link(temp_head); p ← q;
        end;
      end;
    end;
done2: x_token
  end
```

This code is used in section 512.

514.

```
⟨ Look for parameter number or ## 514 ⟩ ≡
  begin s ← cur_tok;
  if xpand then get_x_token
  else get_token;
  if cur_cmd ≠ mac_param then
    if (cur_tok ≤ zero_token) ∨ (cur_tok > t) then
      begin print_err("Illegal␣parameter␣number␣in␣definition␣of␣");
      sprint_cs(warning_index);
      help3("You␣meant␣to␣type␣##␣instead␣of␣#,␣right?")
      ("Or␣maybe␣a␣}␣was␣forgotten␣somewhere␣earlier,␣and␣things")
      ("are␣all␣screwed␣up?␣I´m␣going␣to␣assume␣that␣you␣meant␣##.");
      back_error; cur_tok ← s;
      end
    else cur_tok ← out_param_token − "0" + cur_chr;
  end
```

This code is used in section 512.

515. Another way to create a token list is via the `\read` command. The sixteen files potentially usable for reading appear in the following global variables. The value of *read_open*[*n*] will be *closed* if stream number *n* has not been opened or if it has been fully read; *just_open* if an `\openin` but not a `\read` has been done; and *normal* if it is open and ready to read the next line.

```
  define closed = 2   { not open, or at end of file }
  define just_open = 1   { newly opened, first line not yet read }
⟨ Global variables 13 ⟩ +≡
read_file: array [0 .. 15] of unicode_file;   { used for \read }
read_open: array [0 .. 16] of normal .. closed;   { state of read_file[n] }
```

516.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  for k ← 0 to 16 do read_open[k] ← closed;
```

517. The *read_toks* procedure constructs a token list like that for any macro definition, and makes *cur_val* point to it. Parameter r points to the control sequence that will receive this token list.

```
procedure read_toks(n : integer; r : pointer; j : halfword);
  label done;
  var p: pointer;  { tail of the token list }
    q: pointer;  { new node being added to the token list via store_new_token }
    s: integer;  { saved value of align_state }
    m: small_number;  { stream number }
  begin scanner_status ← defining; warning_index ← r; def_ref ← get_avail;
  token_ref_count(def_ref) ← null; p ← def_ref;  { the reference count }
  store_new_token(end_match_token);
  if (n < 0) ∨ (n > 15) then m ← 16 else m ← n;
  s ← align_state; align_state ← 1000000;  { disable tab marks, etc. }
  repeat ⟨ Input and store tokens from the next line of the file 518 ⟩;
  until align_state = 1000000;
  cur_val ← def_ref; scanner_status ← normal; align_state ← s;
  end;
```

518.

```
⟨ Input and store tokens from the next line of the file 518 ⟩ ≡
  begin_file_reading; name ← m + 1;
  if read_open[m] = closed then ⟨ Input for \read from the terminal 519 ⟩
  else if read_open[m] = just_open then ⟨ Input the first line of read_file[m] 520 ⟩
    else ⟨ Input the next line of read_file[m] 521 ⟩;
  limit ← last;
  if end_line_char_inactive then decr(limit)
  else buffer[limit] ← end_line_char;
  first ← limit + 1; loc ← start; state ← new_line;
  ⟨ Handle \readline and goto done 1572 ⟩;
  loop begin get_token;
    if cur_tok = 0 then goto done;  { cur_cmd = cur_chr = 0 will occur at the end of the line }
    if align_state < 1000000 then  { unmatched '}' aborts the line }
      begin repeat get_token;
      until cur_tok = 0;
      align_state ← 1000000; goto done;
      end;
    store_new_token(cur_tok);
    end;
done: end_file_reading
```

This code is used in section 517.

519. Here we input on-line into the *buffer* array, prompting the user explicitly if $n \geq 0$. The value of n is set negative so that additional prompts will not be given in the case of multi-line input.

```
⟨ Input for \read from the terminal 519 ⟩ ≡
  if interaction > nonstop_mode then
    if n < 0 then prompt_input("")
    else begin wake_up_terminal; print_ln; sprint_cs(r); prompt_input("="); n ← −1;
      end
  else fatal_error("***␣(cannot␣\read␣from␣terminal␣in␣nonstop␣modes)")
```

This code is used in section 518.

520. The first line of a file must be treated specially, since *input_ln* must be told not to start with *get*.

```
⟨ Input the first line of read_file[m] 520 ⟩ ≡
  if input_ln(read_file[m], false) then read_open[m] ← normal
  else begin u_close(read_file[m]); read_open[m] ← closed;
    end
```

This code is used in section 518.

521. An empty line is appended at the end of a *read_file*.

```
⟨ Input the next line of read_file[m] 521 ⟩ ≡
  begin if ¬input_ln(read_file[m], true) then
    begin u_close(read_file[m]); read_open[m] ← closed;
    if align_state ≠ 1000000 then
      begin runaway; print_err("File␣ended␣within␣"); print_esc("read");
      help1("This␣\read␣has␣unbalanced␣braces."); align_state ← 1000000; limit ← 0; error;
      end;
    end;
  end
```

This code is used in section 518.

522. Conditional processing. We consider now the way TeX handles various kinds of `\if` commands.

define *unless_code* = 32 { amount added for '`\unless`' prefix }

define *if_char_code* = 0 { '`\if`' }
define *if_cat_code* = 1 { '`\ifcat`' }
define *if_int_code* = 2 { '`\ifnum`' }
define *if_dim_code* = 3 { '`\ifdim`' }
define *if_odd_code* = 4 { '`\ifodd`' }
define *if_vmode_code* = 5 { '`\ifvmode`' }
define *if_hmode_code* = 6 { '`\ifhmode`' }
define *if_mmode_code* = 7 { '`\ifmmode`' }
define *if_inner_code* = 8 { '`\ifinner`' }
define *if_void_code* = 9 { '`\ifvoid`' }
define *if_hbox_code* = 10 { '`\ifhbox`' }
define *if_vbox_code* = 11 { '`\ifvbox`' }
define *ifx_code* = 12 { '`\ifx`' }
define *if_eof_code* = 13 { '`\ifeof`' }
define *if_true_code* = 14 { '`\iftrue`' }
define *if_false_code* = 15 { '`\iffalse`' }
define *if_case_code* = 16 { '`\ifcase`' }
define *if_primitive_code* = 21 { '`\ifprimitive`' }

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive(`"if"`, *if_test*, *if_char_code*); *primitive*(`"ifcat"`, *if_test*, *if_cat_code*);
primitive(`"ifnum"`, *if_test*, *if_int_code*); *primitive*(`"ifdim"`, *if_test*, *if_dim_code*);
primitive(`"ifodd"`, *if_test*, *if_odd_code*); *primitive*(`"ifvmode"`, *if_test*, *if_vmode_code*);
primitive(`"ifhmode"`, *if_test*, *if_hmode_code*); *primitive*(`"ifmmode"`, *if_test*, *if_mmode_code*);
primitive(`"ifinner"`, *if_test*, *if_inner_code*); *primitive*(`"ifvoid"`, *if_test*, *if_void_code*);
primitive(`"ifhbox"`, *if_test*, *if_hbox_code*); *primitive*(`"ifvbox"`, *if_test*, *if_vbox_code*);
primitive(`"ifx"`, *if_test*, *ifx_code*); *primitive*(`"ifeof"`, *if_test*, *if_eof_code*);
primitive(`"iftrue"`, *if_test*, *if_true_code*); *primitive*(`"iffalse"`, *if_test*, *if_false_code*);
primitive(`"ifcase"`, *if_test*, *if_case_code*); *primitive*(`"ifprimitive"`, *if_test*, *if_primitive_code*);

523. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
if_test: **begin if** *chr_code* ≥ *unless_code* **then** *print_esc*("unless");
case *chr_code* **mod** *unless_code* **of**
if_cat_code: *print_esc*("ifcat");
if_int_code: *print_esc*("ifnum");
if_dim_code: *print_esc*("ifdim");
if_odd_code: *print_esc*("ifodd");
if_vmode_code: *print_esc*("ifvmode");
if_hmode_code: *print_esc*("ifhmode");
if_mmode_code: *print_esc*("ifmmode");
if_inner_code: *print_esc*("ifinner");
if_void_code: *print_esc*("ifvoid");
if_hbox_code: *print_esc*("ifhbox");
if_vbox_code: *print_esc*("ifvbox");
ifx_code: *print_esc*("ifx");
if_eof_code: *print_esc*("ifeof");
if_true_code: *print_esc*("iftrue");
if_false_code: *print_esc*("iffalse");
if_case_code: *print_esc*("ifcase");
if_primitive_code: *print_esc*("ifprimitive");
⟨ Cases of *if_test* for *print_cmd_chr* 1575 ⟩
othercases *print_esc*("if")
endcases;
end;

524. Conditions can be inside conditions, and this nesting has a stack that is independent of the *save_stack*.

Four global variables represent the top of the condition stack: *cond_ptr* points to pushed-down entries, if any; *if_limit* specifies the largest code of a *fi_or_else* command that is syntactically legal; *cur_if* is the name of the current type of conditional; and *if_line* is the line number at which it began.

If no conditions are currently in progress, the condition stack has the special state *cond_ptr* = *null*, *if_limit* = *normal*, *cur_if* = 0, *if_line* = 0. Otherwise *cond_ptr* points to a two-word node; the *type*, *subtype*, and *link* fields of the first word contain *if_limit*, *cur_if*, and *cond_ptr* at the next level, and the second word contains the corresponding *if_line*.

define *if_node_size* = 2 { number of words in stack entry for conditionals }
define *if_line_field*(#) ≡ *mem*[# + 1].*int*
define *if_code* = 1 { code for \if... being evaluated }
define *fi_code* = 2 { code for \fi }
define *else_code* = 3 { code for \else }
define *or_code* = 4 { code for \or }

⟨ Global variables 13 ⟩ +≡
cond_ptr: *pointer*; { top of the condition stack }
if_limit: *normal* .. *or_code*; { upper bound on *fi_or_else* codes }
cur_if: *small_number*; { type of conditional being worked on }
if_line: *integer*; { line where that conditional began }

525. ⟨ Set initial values of key variables 23 ⟩ +≡
cond_ptr ← *null*; *if_limit* ← *normal*; *cur_if* ← 0; *if_line* ← 0;

526. ⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("fi", *fi_or_else*, *fi_code*); *text*(*frozen_fi*) ← "fi"; *eqtb*[*frozen_fi*] ← *eqtb*[*cur_val*];
primitive("or", *fi_or_else*, *or_code*); *primitive*("else", *fi_or_else*, *else_code*);

527. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
fi_or_else: **if** *chr_code* = *fi_code* **then** *print_esc*("fi")
else if *chr_code* = *or_code* **then** *print_esc*("or")
else *print_esc*("else");

528. When we skip conditional text, we keep track of the line number where skipping began, for use in error messages.

⟨ Global variables 13 ⟩ +≡
skip_line: *integer*; { skipping began here }

529. Here is a procedure that ignores text until coming to an `\or`, `\else`, or `\fi` at the current level of `\if`...`\fi` nesting. After it has acted, *cur_chr* will indicate the token that was found, but *cur_tok* will not be set (because this makes the procedure run faster).

procedure *pass_text*;
label *done*;
var *l*: *integer*; { level of `\if`...`\fi` nesting }
save_scanner_status: *small_number*; { *scanner_status* upon entry }
begin *save_scanner_status* ← *scanner_status*; *scanner_status* ← *skipping*; *l* ← 0; *skip_line* ← *line*;
loop begin *get_next*;
if *cur_cmd* = *fi_or_else* **then**
begin if *l* = 0 **then goto** *done*;
if *cur_chr* = *fi_code* **then** *decr*(*l*);
end
else if *cur_cmd* = *if_test* **then** *incr*(*l*);
end;
done: *scanner_status* ← *save_scanner_status*;
if *tracing_ifs* > 0 **then** *show_cur_cmd_chr*;
end;

530. When we begin to process a new `\if`, we set *if_limit* ← *if_code*; then if `\or` or `\else` or `\fi` occurs before the current `\if` condition has been evaluated, `\relax` will be inserted. For example, a sequence of commands like '`\ifvoid1\else...\fi`' would otherwise require something after the '`1`'.

⟨ Push the condition stack 530 ⟩ ≡
begin *p* ← *get_node*(*if_node_size*); *link*(*p*) ← *cond_ptr*; *type*(*p*) ← *if_limit*; *subtype*(*p*) ← *cur_if*;
if_line_field(*p*) ← *if_line*; *cond_ptr* ← *p*; *cur_if* ← *cur_chr*; *if_limit* ← *if_code*; *if_line* ← *line*;
end

This code is used in section 533.

531. ⟨ Pop the condition stack 531 ⟩ ≡
begin if *if_stack*[*in_open*] = *cond_ptr* **then** *if_warning*;
{ conditionals possibly not properly nested with files }
p ← *cond_ptr*; *if_line* ← *if_line_field*(*p*); *cur_if* ← *subtype*(*p*); *if_limit* ← *type*(*p*); *cond_ptr* ← *link*(*p*);
free_node(*p*, *if_node_size*);
end

This code is used in sections 533, 535, 544, and 545.

532. Here's a procedure that changes the *if_limit* code corresponding to a given value of *cond_ptr*.

```
procedure change_if_limit(l : small_number; p : pointer);
  label exit;
  var q: pointer;
  begin if p = cond_ptr then if_limit ← l  { that's the easy case }
  else begin q ← cond_ptr;
    loop begin if q = null then confusion("if");
      if link(q) = p then
        begin type(q) ← l; return;
        end;
      q ← link(q);
      end;
    end;
exit: end;
```

533. A condition is started when the *expand* procedure encounters an *if_test* command; in that case *expand* reduces to *conditional*, which is a recursive procedure.

```
procedure conditional;
  label exit, common_ending;
  var b: boolean;  { is the condition true? }
    e: boolean;  { keep track of nested csnames }
    r: "<" .. ">";  { relation to be evaluated }
    m, n: integer;  { to be tested against the second operand }
    p, q: pointer;  { for traversing token lists in \ifx tests }
    save_scanner_status: small_number;  { scanner_status upon entry }
    save_cond_ptr: pointer;  { cond_ptr corresponding to this conditional }
    this_if: small_number;  { type of this conditional }
    is_unless: boolean;  { was this if preceded by '\unless' ? }
  begin if tracing_ifs > 0 then
    if tracing_commands ≤ 1 then show_cur_cmd_chr;
  ⟨ Push the condition stack 530 ⟩; save_cond_ptr ← cond_ptr; is_unless ← (cur_chr ≥ unless_code);
  this_if ← cur_chr mod unless_code;
  ⟨ Either process \ifcase or set b to the value of a boolean condition 536 ⟩;
  if is_unless then b ← ¬b;
  if tracing_commands > 1 then ⟨ Display the value of b 537 ⟩;
  if b then
    begin change_if_limit(else_code, save_cond_ptr); return;  { wait for \else or \fi }
    end;
  ⟨ Skip to \else or \fi, then goto common_ending 535 ⟩;
common_ending: if cur_chr = fi_code then ⟨ Pop the condition stack 531 ⟩
  else if_limit ← fi_code;  { wait for \fi }
exit: end;
```

534. In a construction like '`\if\iftrue abc\else d\fi`', the first `\else` that we come to after learning that the `\if` is false is not the `\else` we're looking for. Hence the following curious logic is needed.

535. ⟨ Skip to `\else` or `\fi`, then **goto** *common_ending* 535 ⟩ ≡

```
loop begin pass_text;
  if cond_ptr = save_cond_ptr then
    begin if cur_chr ≠ or_code then goto common_ending;
    print_err("Extra␣"); print_esc("or");
    help1("I´m␣ignoring␣this;␣it␣doesn´t␣match␣any␣\if.");
    error;
    end
  else if cur_chr = fi_code then ⟨ Pop the condition stack 531 ⟩;
  end
```

This code is used in section 533.

536. ⟨ Either process `\ifcase` or set b to the value of a boolean condition 536 ⟩ ≡

```
case this_if of
if_char_code, if_cat_code: ⟨ Test if two characters match 541 ⟩;
if_int_code, if_dim_code: ⟨ Test relation between integers or dimensions 538 ⟩;
if_odd_code: ⟨ Test if an integer is odd 539 ⟩;
if_vmode_code: b ← (abs(mode) = vmode);
if_hmode_code: b ← (abs(mode) = hmode);
if_mmode_code: b ← (abs(mode) = mmode);
if_inner_code: b ← (mode < 0);
if_void_code, if_hbox_code, if_vbox_code: ⟨ Test box register status 540 ⟩;
ifx_code: ⟨ Test if two tokens match 542 ⟩;
if_eof_code: begin scan_four_bit_int; b ← (read_open[cur_val] = closed);
  end;
if_true_code: b ← true;
if_false_code: b ← false;
  ⟨ Cases for conditional 1577 ⟩
if_case_code: ⟨ Select the appropriate case and return or goto common_ending 544 ⟩;
if_primitive_code: begin save_scanner_status ← scanner_status; scanner_status ← normal; get_next;
  scanner_status ← save_scanner_status;
  if cur_cs < hash_base then m ← prim_lookup(cur_cs − single_base)
  else m ← prim_lookup(text(cur_cs));
  b ← ((cur_cmd ≠ undefined_cs) ∧ (m ≠ undefined_primitive) ∧ (cur_cmd = prim_eq_type(m)) ∧ (cur_chr =
      prim_equiv(m)));
  end;
end { there are no other cases }
```

This code is used in section 533.

537. ⟨ Display the value of b 537 ⟩ ≡

```
begin begin_diagnostic;
if b then print("{true}") else print("{false}");
end_diagnostic(false);
end
```

This code is used in section 533.

538. Here we use the fact that `"<"`, `"="`, and `">"` are consecutive ASCII codes.

```
⟨ Test relation between integers or dimensions 538 ⟩ ≡
  begin if this_if = if_int_code then scan_int else scan_normal_dimen;
  n ← cur_val; ⟨ Get the next non-blank non-call token 440 ⟩;
  if (cur_tok ≥ other_token + "<") ∧ (cur_tok ≤ other_token + ">") then r ← cur_tok − other_token
  else begin print_err("Missing␣=␣inserted␣for␣"); print_cmd_chr(if_test, this_if);
    help1("I␣was␣expecting␣to␣see␣`<´,␣`=´,␣or␣`>´.␣Didn´t.");
    back_error; r ← "=";
    end;
  if this_if = if_int_code then scan_int else scan_normal_dimen;
  case r of
  "<": b ← (n < cur_val);
  "=": b ← (n = cur_val);
  ">": b ← (n > cur_val);
  end;
  end
```

This code is used in section 536.

539.

```
⟨ Test if an integer is odd 539 ⟩ ≡
  begin scan_int; b ← odd(cur_val);
  end
```

This code is used in section 536.

540.

```
⟨ Test box register status 540 ⟩ ≡
  begin scan_register_num; fetch_box(p);
  if this_if = if_void_code then b ← (p = null)
  else if p = null then b ← false
    else if this_if = if_hbox_code then b ← (type(p) = hlist_node)
      else b ← (type(p) = vlist_node);
  end
```

This code is used in section 536.

541. An active character will be treated as category 13 following `\if\noexpand` or following `\ifcat\noexpand`. We use the fact that active characters have the smallest tokens, among all control sequences.

```
define get_x_token_or_active_char ≡
    begin get_x_token;
    if cur_cmd = relax then
      if cur_chr = no_expand_flag then
        begin cur_cmd ← active_char; cur_chr ← cur_tok − cs_token_flag − active_base;
        end;
    end
```

```
⟨ Test if two characters match 541 ⟩ ≡
  begin get_x_token_or_active_char;
  if (cur_cmd > active_char) ∨ (cur_chr > biggest_usv) then   { not a character }
    begin m ← relax; n ← too_big_usv;
    end
  else begin m ← cur_cmd; n ← cur_chr;
    end;
  get_x_token_or_active_char;
  if (cur_cmd > active_char) ∨ (cur_chr > biggest_usv) then
    begin cur_cmd ← relax; cur_chr ← too_big_usv;
    end;
  if this_if = if_char_code then b ← (n = cur_chr) else b ← (m = cur_cmd);
  end
```

This code is used in section 536.

542. Note that '`\ifx`' will declare two macros different if one is *long* or *outer* and the other isn't, even though the texts of the macros are the same.

We need to reset *scanner_status*, since `\outer` control sequences are allowed, but we might be scanning a macro definition or preamble.

```
⟨ Test if two tokens match 542 ⟩ ≡
  begin save_scanner_status ← scanner_status; scanner_status ← normal; get_next; n ← cur_cs;
  p ← cur_cmd; q ← cur_chr; get_next;
  if cur_cmd ≠ p then b ← false
  else if cur_cmd < call then b ← (cur_chr = q)
    else ⟨ Test if two macro texts match 543 ⟩;
  scanner_status ← save_scanner_status;
  end
```

This code is used in section 536.

543. Note also that '`\ifx`' decides that macros `\a` and `\b` are different in examples like this:

```
\def\a{\c}    \def\c{}
\def\b{\d}    \def\d{}
```

⟨ Test if two macro texts match 543 ⟩ ≡
```
begin p ← link(cur_chr); q ← link(equiv(n)); { omit reference counts }
if p = q then b ← true
else begin while (p ≠ null) ∧ (q ≠ null) do
    if info(p) ≠ info(q) then p ← null
    else begin p ← link(p); q ← link(q);
      end;
  b ← ((p = null) ∧ (q = null));
  end;
end
```
This code is used in section 542.

544. ⟨ Select the appropriate case and **return** or **goto** *common_ending* 544 ⟩ ≡
```
begin scan_int; n ← cur_val; { n is the number of cases to pass }
if tracing_commands > 1 then
  begin begin_diagnostic; print("{case␣"); print_int(n); print_char("}"); end_diagnostic(false);
  end;
while n ≠ 0 do
  begin pass_text;
  if cond_ptr = save_cond_ptr then
    if cur_chr = or_code then decr(n)
    else goto common_ending
  else if cur_chr = fi_code then ⟨ Pop the condition stack 531 ⟩;
  end;
change_if_limit(or_code, save_cond_ptr); return; { wait for \or, \else, or \fi }
end
```
This code is used in section 536.

545. The processing of conditionals is complete except for the following code, which is actually part of *expand*. It comes into play when `\or`, `\else`, or `\fi` is scanned.

⟨ Terminate the current conditional and skip to `\fi` 545 ⟩ ≡
```
begin if tracing_ifs > 0 then
  if tracing_commands ≤ 1 then show_cur_cmd_chr;
if cur_chr > if_limit then
  if if_limit = if_code then insert_relax { condition not yet evaluated }
  else begin print_err("Extra␣"); print_cmd_chr(fi_or_else, cur_chr);
    help1("I´m␣ignoring␣this;␣it␣doesn´t␣match␣any␣\if."); error;
    end
else begin while cur_chr ≠ fi_code do pass_text; { skip to \fi }
  ⟨ Pop the condition stack 531 ⟩;
  end;
end
```
This code is used in section 399.

546. File names. It's time now to fret about file names. Besides the fact that different operating systems treat files in different ways, we must cope with the fact that completely different naming conventions are used by different groups of people. The following programs show what is required for one particular operating system; similar routines for other systems are not difficult to devise.

TEX assumes that a file name has three parts: the name proper; its "extension"; and a "file area" where it is found in an external file system. The extension of an input file or a write file is assumed to be '`.tex`' unless otherwise specified; it is '`.log`' on the transcript file that records each run of TEX; it is '`.tfm`' on the font metric files that describe characters in the fonts TEX uses; it is '`.dvi`' on the output files that specify typesetting information; and it is '`.fmt`' on the format files written by `INITEX` to initialize TEX. The file area can be arbitrary on input files, but files are usually output to the user's current area. If an input file cannot be found on the specified area, TEX will look for it on a special system area; this special area is intended for commonly used input files like `webmac.tex`.

Simple uses of TEX refer only to file names that have no explicit extension or area. For example, a person usually says '`\input paper`' or '`\font\tenrm = helvetica`' instead of '`\input paper.new`' or '`\font\tenrm = <csd.knuth>test`'. Simple file names are best, because they make the TEX source files portable; whenever a file name consists entirely of letters and digits, it should be treated in the same way by all implementations of TEX. However, users need the ability to refer to other files in their environment, especially when responding to error messages concerning unopenable files; therefore we want to let them use the syntax that appears in their favorite operating system.

547. In order to isolate the system-dependent aspects of file names, the system-independent parts of TEX are expressed in terms of three system-dependent procedures called *begin_name*, *more_name*, and *end_name*. In essence, if the user-specified characters of the file name are $c_1 \ldots c_n$, the system-independent driver program does the operations

$$\textit{begin_name};\ \textit{more_name}(c_1);\ \ldots\ ;\ \textit{more_name}(c_n);\ \textit{end_name}.$$

These three procedures communicate with each other via global variables. Afterwards the file name will appear in the string pool as three strings called *cur_name*, *cur_area*, and *cur_ext*; the latter two are null (i.e., `""`), unless they were explicitly specified by the user.

Actually the situation is slightly more complicated, because TEX needs to know when the file name ends. The *more_name* routine is a function (with side effects) that returns *true* on the calls $\textit{more_name}(c_1)$, ..., $\textit{more_name}(c_{n-1})$. The final call $\textit{more_name}(c_n)$ returns *false*; or, it returns *true* and the token following c_n is something like '`\hbox`' (i.e., not a character). In other words, *more_name* is supposed to return *true* unless it is sure that the file name has been completely scanned; and *end_name* is supposed to be able to finish the assembly of *cur_name*, *cur_area*, and *cur_ext* regardless of whether $\textit{more_name}(c_n)$ returned *true* or *false*.

⟨ Global variables 13 ⟩ +≡
cur_name: *str_number*; { name of file just scanned }
cur_area: *str_number*; { file area just scanned, or `""` }
cur_ext: *str_number*; { file extension just scanned, or `""` }

548. The file names we shall deal with for illustrative purposes have the following structure: If the name contains '`>`' or '`:`', the file area consists of all characters up to and including the final such character; otherwise the file area is null. If the remaining file name contains '`.`', the file extension consists of all such characters from the first remaining '`.`' to the end, otherwise the file extension is null.

We can scan such file names easily by using two global variables that keep track of the occurrences of area and extension delimiters:

⟨ Global variables 13 ⟩ +≡
area_delimiter: *pool_pointer*; { the most recent '`>`' or '`:`', if any }
ext_delimiter: *pool_pointer*; { the relevant '`.`', if any }
file_name_quote_char: *UTF16_code*;

549. Input files that can't be found in the user's area may appear in a standard system area called *TEX_area*. Font metric files whose areas are not given explicitly are assumed to appear in a standard system area called *TEX_font_area*. These system area names will, of course, vary from place to place.

```
define TEX_area ≡ "TeXinputs:"
define TEX_font_area ≡ "TeXfonts:"
```

550. Here now is the first of the system-dependent routines for file name scanning.

```
procedure begin_name;
  begin area_delimiter ← 0; ext_delimiter ← 0; file_name_quote_char ← 0;
  end;
```

551. And here's the second. The string pool might change as the file name is being scanned, since a new `\csname` might be entered; therefore we keep *area_delimiter* and *ext_delimiter* relative to the beginning of the current string, instead of assigning an absolute address like *pool_ptr* to them.

```
function more_name(c : ASCII_code): boolean;
  begin if c = " " then more_name ← false
  else begin str_room(1); append_char(c); { contribute c to the current string }
    if (c = ">") ∨ (c = ":") then
      begin area_delimiter ← cur_length; ext_delimiter ← 0;
      end
    else if (c = ".") ∧ (ext_delimiter = 0) then ext_delimiter ← cur_length;
    more_name ← true;
    end;
  end;
```

552. The third.

```
procedure end_name;
  begin if str_ptr + 3 > max_strings then overflow("number of strings", max_strings − init_str_ptr);
  if area_delimiter = 0 then cur_area ← ""
  else begin cur_area ← str_ptr; str_start_macro(str_ptr + 1) ← str_start_macro(str_ptr) + area_delimiter;
    incr(str_ptr);
    end;
  if ext_delimiter = 0 then
    begin cur_ext ← ""; cur_name ← make_string;
    end
  else begin cur_name ← str_ptr;
    str_start_macro(str_ptr + 1) ← str_start_macro(str_ptr) + ext_delimiter − area_delimiter − 1;
    incr(str_ptr); cur_ext ← make_string;
    end;
  end;
```

553. Conversely, here is a routine that takes three strings and prints a file name that might have produced them. (The routine is system dependent, because some operating systems put the file area last instead of first.)

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_file_name(n, a, e : integer);
  begin slow_print(a); slow_print(n); slow_print(e);
  end;
```

554. Another system-dependent routine is needed to convert three internal TEX strings into the *name_of_file* value that is used to open files. The present code allows both lowercase and uppercase letters in the file name.

```
define append_to_name(#) ≡
        begin c ← #; incr(k);
        if k ≤ file_name_size then name_of_file[k] ← xchr[c];
        end
procedure pack_file_name(n, a, e : str_number);
  var k: integer; { number of positions filled in name_of_file }
    c: ASCII_code; { character being packed }
    j: pool_pointer; { index into str_pool }
  begin k ← 0;
  for j ← str_start_macro(a) to str_start_macro(a + 1) − 1 do append_to_name(so(str_pool[j]));
  for j ← str_start_macro(n) to str_start_macro(n + 1) − 1 do append_to_name(so(str_pool[j]));
  for j ← str_start_macro(e) to str_start_macro(e + 1) − 1 do append_to_name(so(str_pool[j]));
  if k ≤ file_name_size then name_length ← k else name_length ← file_name_size;
  for k ← name_length + 1 to file_name_size do name_of_file[k] ← ´␣´;
  end;
```

555. A messier routine is also needed, since format file names must be scanned before TEX's string mechanism has been initialized. We shall use the global variable *TEX_format_default* to supply the text for default system areas and extensions related to format files.

```
define format_default_length = 20 { length of the TEX_format_default string }
define format_area_length = 11 { length of its area part }
define format_ext_length = 4 { length of its '.fmt' part }
define format_extension = ".fmt" { the extension, as a WEB constant }
⟨ Global variables 13 ⟩ +≡
TEX_format_default: packed array [1 .. format_default_length] of char;
```

556.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  TEX_format_default ← ´TeXformats:plain.fmt´;
```

557.
```
⟨ Check the "constant" values for consistency 14 ⟩ +≡
  if format_default_length > file_name_size then bad ← 31;
```

558. Here is the messy routine that was just mentioned. It sets *name_of_file* from the first n characters of *TEX_format_default*, followed by *buffer*[a .. b], followed by the last *format_ext_length* characters of *TEX_format_default*.

We dare not give error messages here, since TeX calls this routine before the *error* routine is ready to roll. Instead, we simply drop excess characters, since the error will be detected in another way when a strange file name isn't found.

```
procedure pack_buffered_name(n : small_number; a, b : integer);
  var k: integer;  { number of positions filled in name_of_file }
    c: ASCII_code;  { character being packed }
    j: integer;  { index into buffer or TEX_format_default }
  begin if n + b − a + 1 + format_ext_length > file_name_size then
    b ← a + file_name_size − n − 1 − format_ext_length;
  k ← 0;
  for j ← 1 to n do append_to_name(xord[TEX_format_default[j]]);
  for j ← a to b do append_to_name(buffer[j]);
  for j ← format_default_length − format_ext_length + 1 to format_default_length do
    append_to_name(xord[TEX_format_default[j]]);
  if k ≤ file_name_size then name_length ← k else name_length ← file_name_size;
  for k ← name_length + 1 to file_name_size do name_of_file[k] ← ´␣´;
  end;
```

559. Here is the only place we use *pack_buffered_name*. This part of the program becomes active when a "virgin" TeX is trying to get going, just after the preliminary initialization, or when the user is substituting another format file by typing '`&`' after the initial '`**`' prompt. The buffer contains the first line of input in *buffer*[*loc* .. (*last* − 1)], where *loc* < *last* and *buffer*[*loc*] ≠ "␣".

```
⟨ Declare the function called open_fmt_file 559 ⟩ ≡
function open_fmt_file: boolean;
  label found, exit;
  var j: 0 .. buf_size;  { the first space after the format file name }
  begin j ← loc;
  if buffer[loc] = "&" then
    begin incr(loc); j ← loc; buffer[last] ← "␣";
    while buffer[j] ≠ "␣" do incr(j);
    pack_buffered_name(0, loc, j − 1);  { try first without the system file area }
    if w_open_in(fmt_file) then goto found;
    pack_buffered_name(format_area_length, loc, j − 1);  { now try the system format file area }
    if w_open_in(fmt_file) then goto found;
    wake_up_terminal; wterm_ln(´Sorry,␣I␣can´´t␣find␣that␣format;´, ´␣will␣try␣PLAIN.´);
    update_terminal;
    end;  { now pull out all the stops: try for the system plain file }
  pack_buffered_name(format_default_length − format_ext_length, 1, 0);
  if ¬w_open_in(fmt_file) then
    begin wake_up_terminal; wterm_ln(´I␣can´´t␣find␣the␣PLAIN␣format␣file!´);
    open_fmt_file ← false; return;
    end;
found: loc ← j; open_fmt_file ← true;
exit: end;
```

This code is used in section 1357.

560. Operating systems often make it possible to determine the exact name (and possible version number) of a file that has been opened. The following routine, which simply makes a TeX string from the value of *name_of_file*, should ideally be changed to deduce the full name of file f, which is the file most recently opened, if it is possible to do this in a Pascal program.

This routine might be called after string memory has overflowed, hence we dare not use '*str_room*'.

```
function make_name_string: str_number;
  var k: 0 .. file_name_size;  { index into name_of_file }
  begin if (pool_ptr + name_length > pool_size) ∨ (str_ptr = max_strings) ∨ (cur_length > 0) then
    make_name_string ← "?"
  else begin make_utf16_name;
    for k ← 0 to name_length16 − 1 do append_char(name_of_file16[k]);
    make_name_string ← make_string;
    end;
  end;
function u_make_name_string(var f : unicode_file): str_number;
  begin u_make_name_string ← make_name_string;
  end;
function a_make_name_string(var f : alpha_file): str_number;
  begin a_make_name_string ← make_name_string;
  end;
function b_make_name_string(var f : byte_file): str_number;
  begin b_make_name_string ← make_name_string;
  end;
function w_make_name_string(var f : word_file): str_number;
  begin w_make_name_string ← make_name_string;
  end;
```

561. Now let's consider the "driver" routines by which TeX deals with file names in a system-independent manner. First comes a procedure that looks for a file name in the input by calling *get_x_token* for the information.

```
procedure scan_file_name;
  label done;
  begin name_in_progress ← true; begin_name; ⟨ Get the next non-blank non-call token 440 ⟩;
  loop begin if (cur_cmd > other_char) ∨ (cur_chr > biggest_char) then  { not a character }
      begin back_input; goto done;
      end;
    if ¬more_name(cur_chr) then goto done;
    get_x_token;
    end;
done: end_name; name_in_progress ← false;
  end;
```

562. The global variable *name_in_progress* is used to prevent recursive use of *scan_file_name*, since the *begin_name* and other procedures communicate via global variables. Recursion would arise only by devious tricks like '`\input\input f`'; such attempts at sabotage must be thwarted. Furthermore, *name_in_progress* prevents `\input` from being initiated when a font size specification is being scanned.

Another global variable, *job_name*, contains the file name that was first `\input` by the user. This name is extended by '`.log`' and '`.dvi`' and '`.fmt`' in the names of TeX's output files.

⟨ Global variables 13 ⟩ +≡
name_in_progress: *boolean*; { is a file name being scanned? }
job_name: *str_number*; { principal file name }
log_opened: *boolean*; { has the transcript file been opened? }

563. Initially *job_name* = 0; it becomes nonzero as soon as the true name is known. We have *job_name* = 0 if and only if the '`log`' file has not been opened, except of course for a short time just after *job_name* has become nonzero.

⟨ Initialize the output routines 55 ⟩ +≡
job_name ← 0; *name_in_progress* ← *false*; *log_opened* ← *false*;

564. Here is a routine that manufactures the output file names, assuming that *job_name* ≠ 0. It ignores and changes the current settings of *cur_area* and *cur_ext*.

define *pack_cur_name* ≡ *pack_file_name*(*cur_name*, *cur_area*, *cur_ext*)

procedure *pack_job_name*(*s* : *str_number*); { *s* = "`.log`", *output_file_extension*, or *format_extension* }
begin *cur_area* ← ""; *cur_ext* ← *s*; *cur_name* ← *job_name*; *pack_cur_name*;
end;

565. If some trouble arises when TeX tries to open a file, the following routine calls upon the user to supply another file name. Parameter *s* is used in the error message to identify the type of file; parameter *e* is the default extension if none is given. Upon exit from the routine, variables *cur_name*, *cur_area*, *cur_ext*, and *name_of_file* are ready for another attempt at file opening.

procedure *prompt_file_name*(*s*, *e* : *str_number*);
label *done*;
var *k*: 0 .. *buf_size*; { index into *buffer* }
begin if *interaction* = *scroll_mode* **then** *wake_up_terminal*;
if *s* = "`input␣file␣name`" **then** *print_err*("`I␣can´t␣find␣file␣`")
else *print_err*("`I␣can´t␣write␣on␣file␣`");
print_file_name(*cur_name*, *cur_area*, *cur_ext*); *print*("`´.`");
if *e* = "`.tex`" **then** *show_context*;
print_nl("`Please␣type␣another␣`"); *print*(*s*);
if *interaction* < *scroll_mode* **then** *fatal_error*("`***␣(job␣aborted,␣file␣error␣in␣nonstop␣mode)`");
clear_terminal; *prompt_input*("`:␣`"); ⟨ Scan file name in the buffer 566 ⟩;
if *cur_ext* = "" **then** *cur_ext* ← *e*;
pack_cur_name;
end;

566. ⟨ Scan file name in the buffer 566 ⟩ ≡

```
begin begin_name; k ← first;
while (buffer[k] = "␣") ∧ (k < last) do incr(k);
loop begin if k = last then goto done;
  if ¬more_name(buffer[k]) then goto done;
  incr(k);
  end;
done: end_name;
end
```

This code is used in section 565.

567. Here's an example of how these conventions are used. Whenever it is time to ship out a box of stuff, we shall use the macro *ensure_dvi_open*.

```
define ensure_dvi_open ≡
        if output_file_name = 0 then
          begin if job_name = 0 then open_log_file;
          pack_job_name(output_file_extension);
          while ¬dvi_open_out(dvi_file) do
            prompt_file_name("file␣name␣for␣output", output_file_extension);
          output_file_name ← b_make_name_string(dvi_file);
          end
```

⟨ Global variables 13 ⟩ +≡

```
output_file_extension: str_number;
no_pdf_output: boolean;
dvi_file: byte_file;  { the device-independent output goes here }
output_file_name: str_number;  { full name of the output file }
log_name: str_number;  { full name of the log file }
```

568. ⟨ Initialize the output routines 55 ⟩ +≡

```
output_file_name ← 0;
if no_pdf_output then output_file_extension ← ".xdv"
else output_file_extension ← ".pdf";
```

569. The *open_log_file* routine is used to open the transcript file and to help it catch up to what has previously been printed on the terminal.

```
procedure open_log_file;
  var old_setting: 0 .. max_selector;  { previous selector setting }
    k: 0 .. buf_size;  { index into months and buffer }
    l: 0 .. buf_size;  { end of first input line }
    months: packed array [1 .. 36] of char;  { abbreviations of month names }
  begin old_setting ← selector;
  if job_name = 0 then job_name ← "texput";
  pack_job_name(".log");
  while ¬a_open_out(log_file) do ⟨ Try to get a different log file name 570 ⟩;
  log_name ← a_make_name_string(log_file); selector ← log_only; log_opened ← true;
  ⟨ Print the banner line, including the date and time 571 ⟩;
  input_stack[input_ptr] ← cur_input;  { make sure bottom level is in memory }
  print_nl("**"); l ← input_stack[0].limit_field;  { last position of first line }
  if buffer[l] = end_line_char then decr(l);
  for k ← 1 to l do print(buffer[k]);
  print_ln;  { now the transcript file contains the first line of input }
  selector ← old_setting + 2;  { log_only or term_and_log }
  end;
```

570. Sometimes *open_log_file* is called at awkward moments when TeX is unable to print error messages or even to *show_context*. The *prompt_file_name* routine can result in a *fatal_error*, but the *error* routine will not be invoked because *log_opened* will be false.

The normal idea of *batch_mode* is that nothing at all should be written on the terminal. However, in the unusual case that no log file could be opened, we make an exception and allow an explanatory message to be seen.

Incidentally, the program always refers to the log file as a '`transcript file`', because some systems cannot use the extension '`.log`' for this file.

```
⟨ Try to get a different log file name 570 ⟩ ≡
  begin selector ← term_only; prompt_file_name("transcript␣file␣name", ".log");
  end
```

This code is used in section 569.

571.

```
⟨ Print the banner line, including the date and time 571 ⟩ ≡
  begin wlog(banner); slow_print(format_ident); print("␣␣"); print_int(sys_day); print_char("␣");
  months ← ´JANFEBMARAPRMAYJUNJULAUGSEPOCTNOVDEC´;
  for k ← 3 * sys_month − 2 to 3 * sys_month do wlog(months[k]);
  print_char("␣"); print_int(sys_year); print_char("␣"); print_two(sys_time div 60); print_char(":");
  print_two(sys_time mod 60);
  if eTeX_ex then
    begin ; wlog_cr; wlog(´entering␣extended␣mode´);
    end;
  end
```

This code is used in section 569.

572. Let's turn now to the procedure that is used to initiate file reading when an '`\input`' command is being processed. Beware: For historic reasons, this code foolishly conserves a tiny bit of string pool space; but that can confuse the interactive '`E`' option.

```
procedure start_input; { TeX will \input something }
  label done;
  begin scan_file_name; { set cur_name to desired file name }
  if cur_ext = "" then cur_ext ← ".tex";
  pack_cur_name;
  loop begin begin_file_reading; { set up cur_file and new level of input }
    if a_open_in(cur_file) then goto done;
    if cur_area = "" then
      begin pack_file_name(cur_name, TEX_area, cur_ext);
      if a_open_in(cur_file) then goto done;
      end;
    end_file_reading; { remove the level that didn't work }
    prompt_file_name("input␣file␣name", ".tex");
    end;
done: name ← a_make_name_string(cur_file);
  if job_name = 0 then
    begin job_name ← cur_name; open_log_file;
    end; { open_log_file doesn't show_context, so limit and loc needn't be set to meaningful values yet }
  if term_offset + length(name) > max_print_line − 2 then print_ln
  else if (term_offset > 0) ∨ (file_offset > 0) then print_char("␣");
  print_char("("); incr(open_parens); slow_print(name); update_terminal; state ← new_line;
  if name = str_ptr − 1 then { conserve string pool space (but see note above) }
    begin flush_string; name ← cur_name;
    end;
  ⟨ Read the first line of the new file 573 ⟩;
  end;
```

573. Here we have to remember to tell the *input_ln* routine not to start with a *get*. If the file is empty, it is considered to contain a single blank line.

```
⟨ Read the first line of the new file 573 ⟩ ≡
  begin line ← 1;
  if input_ln(cur_file, false) then do_nothing;
  firm_up_the_line;
  if end_line_char_inactive then decr(limit)
  else buffer[limit] ← end_line_char;
  first ← limit + 1; loc ← start;
  end
```

This code is used in section 572.

574. Font metric data. TEX gets its knowledge about fonts from font metric files, also called TFM files; the 'T' in 'TFM' stands for TEX, but other programs know about them too.

The information in a TFM file appears in a sequence of 8-bit bytes. Since the number of bytes is always a multiple of 4, we could also regard the file as a sequence of 32-bit words, but TEX uses the byte interpretation. The format of TFM files was designed by Lyle Ramshaw in 1980. The intent is to convey a lot of different kinds of information in a compact but useful form.

⟨ Global variables 13 ⟩ +≡
tfm_file: *byte_file*;

575. The first 24 bytes (6 words) of a TFM file contain twelve 16-bit integers that give the lengths of the various subsequent portions of the file. These twelve integers are, in order:

lf = length of the entire file, in words;
lh = length of the header data, in words;
bc = smallest character code in the font;
ec = largest character code in the font;
nw = number of words in the width table;
nh = number of words in the height table;
nd = number of words in the depth table;
ni = number of words in the italic correction table;
nl = number of words in the lig/kern table;
nk = number of words in the kern table;
ne = number of words in the extensible character table;
np = number of font parameter words.

They are all nonnegative and less than 2^{15}. We must have $bc - 1 \le ec \le 255$, and

$$lf = 6 + lh + (ec - bc + 1) + nw + nh + nd + ni + nl + nk + ne + np.$$

Note that a font may contain as many as 256 characters (if $bc = 0$ and $ec = 255$), and as few as 0 characters (if $bc = ec + 1$).

Incidentally, when two or more 8-bit bytes are combined to form an integer of 16 or more bits, the most significant bytes appear first in the file. This is called BigEndian order.

576. The rest of the TFM file may be regarded as a sequence of ten data arrays having the informal specification

header: **array** [0 .. $lh - 1$] **of** *stuff*
char_info: **array** [bc .. ec] **of** *char_info_word*
width: **array** [0 .. $nw - 1$] **of** *fix_word*
height: **array** [0 .. $nh - 1$] **of** *fix_word*
depth: **array** [0 .. $nd - 1$] **of** *fix_word*
italic: **array** [0 .. $ni - 1$] **of** *fix_word*
lig_kern: **array** [0 .. $nl - 1$] **of** *lig_kern_command*
kern: **array** [0 .. $nk - 1$] **of** *fix_word*
exten: **array** [0 .. $ne - 1$] **of** *extensible_recipe*
param: **array** [1 .. np] **of** *fix_word*

The most important data type used here is a *fix_word*, which is a 32-bit representation of a binary fraction. A *fix_word* is a signed quantity, with the two's complement of the entire word used to represent negation. Of the 32 bits in a *fix_word*, exactly 12 are to the left of the binary point; thus, the largest *fix_word* value is $2048 - 2^{-20}$, and the smallest is -2048. We will see below, however, that all but two of the *fix_word* values must lie between -16 and $+16$.

577. The first data array is a block of header information, which contains general facts about the font. The header must contain at least two words, *header*[0] and *header*[1], whose meaning is explained below. Additional header information of use to other software routines might also be included, but TeX82 does not need to know about such details. For example, 16 more words of header information are in use at the Xerox Palo Alto Research Center; the first ten specify the character coding scheme used (e.g., '`XEROX text`' or '`TeX math symbols`'), the next five give the font identifier (e.g., '`HELVETICA`' or '`CMSY`'), and the last gives the "face byte." The program that converts `DVI` files to Xerox printing format gets this information by looking at the `TFM` file, which it needs to read anyway because of other information that is not explicitly repeated in `DVI` format.

header[0] is a 32-bit check sum that TeX will copy into the `DVI` output file. Later on when the `DVI` file is printed, possibly on another computer, the actual font that gets used is supposed to have a check sum that agrees with the one in the `TFM` file used by TeX. In this way, users will be warned about potential incompatibilities. (However, if the check sum is zero in either the font file or the `TFM` file, no check is made.) The actual relation between this check sum and the rest of the `TFM` file is not important; the check sum is simply an identification number with the property that incompatible fonts almost always have distinct check sums.

header[1] is a *fix_word* containing the design size of the font, in units of TeX points. This number must be at least 1.0; it is fairly arbitrary, but usually the design size is 10.0 for a "10 point" font, i.e., a font that was designed to look best at a 10-point size, whatever that really means. When a TeX user asks for a font '`at` δ `pt`', the effect is to override the design size and replace it by δ, and to multiply the x and y coordinates of the points in the font image by a factor of δ divided by the design size. *All other dimensions in the* `TFM` *file are fix_word numbers in design-size units*, with the exception of *param*[1] (which denotes the slant ratio). Thus, for example, the value of *param*[6], which defines the `em` unit, is often the *fix_word* value $2^{20} = 1.0$, since many fonts have a design size equal to one em. The other dimensions must be less than 16 design-size units in absolute value; thus, *header*[1] and *param*[1] are the only *fix_word* entries in the whole `TFM` file whose first byte might be something besides 0 or 255.

578. Next comes the *char_info* array, which contains one *char_info_word* per character. Each word in this part of the file contains six fields packed into four bytes as follows.

first byte: *width_index* (8 bits)
second byte: *height_index* (4 bits) times 16, plus *depth_index* (4 bits)
third byte: *italic_index* (6 bits) times 4, plus *tag* (2 bits)
fourth byte: *remainder* (8 bits)

The actual width of a character is *width*[*width_index*], in design-size units; this is a device for compressing information, since many characters have the same width. Since it is quite common for many characters to have the same height, depth, or italic correction, the `TFM` format imposes a limit of 16 different heights, 16 different depths, and 64 different italic corrections.

The italic correction of a character has two different uses. (a) In ordinary text, the italic correction is added to the width only if the TeX user specifies '`\/`' after the character. (b) In math formulas, the italic correction is always added to the width, except with respect to the positioning of subscripts.

Incidentally, the relation *width*[0] = *height*[0] = *depth*[0] = *italic*[0] = 0 should always hold, so that an index of zero implies a value of zero. The *width_index* should never be zero unless the character does not exist in the font, since a character is valid if and only if it lies between *bc* and *ec* and has a nonzero *width_index*.

579. The *tag* field in a *char_info_word* has four values that explain how to interpret the *remainder* field.

tag = 0 (*no_tag*) means that *remainder* is unused.

tag = 1 (*lig_tag*) means that this character has a ligature/kerning program starting at position *remainder* in the *lig_kern* array.

tag = 2 (*list_tag*) means that this character is part of a chain of characters of ascending sizes, and not the largest in the chain. The *remainder* field gives the character code of the next larger character.

tag = 3 (*ext_tag*) means that this character code represents an extensible character, i.e., a character that is built up of smaller pieces so that it can be made arbitrarily large. The pieces are specified in *exten*[*remainder*].

Characters with *tag* = 2 and *tag* = 3 are treated as characters with *tag* = 0 unless they are used in special circumstances in math formulas. For example, the `\sum` operation looks for a *list_tag*, and the `\left` operation looks for both *list_tag* and *ext_tag*.

define *no_tag* = 0 { vanilla character }
define *lig_tag* = 1 { character has a ligature/kerning program }
define *list_tag* = 2 { character has a successor in a charlist }
define *ext_tag* = 3 { character is extensible }

580. The *lig_kern* array contains instructions in a simple programming language that explains what to do for special letter pairs. Each word in this array is a *lig_kern_command* of four bytes.

first byte: *skip_byte*, indicates that this is the final program step if the byte is 128 or more, otherwise the next step is obtained by skipping this number of intervening steps.

second byte: *next_char*, "if *next_char* follows the current character, then perform the operation and stop, otherwise continue."

third byte: *op_byte*, indicates a ligature step if less than 128, a kern step otherwise.

fourth byte: *remainder*.

In a kern step, an additional space equal to $kern[256 * (op_byte - 128) + remainder]$ is inserted between the current character and *next_char*. This amount is often negative, so that the characters are brought closer together by kerning; but it might be positive.

There are eight kinds of ligature steps, having *op_byte* codes $4a+2b+c$ where $0 \le a \le b+c$ and $0 \le b, c \le 1$. The character whose code is *remainder* is inserted between the current character and *next_char*; then the current character is deleted if $b = 0$, and *next_char* is deleted if $c = 0$; then we pass over a characters to reach the next current character (which may have a ligature/kerning program of its own).

If the very first instruction of the *lig_kern* array has *skip_byte* = 255, the *next_char* byte is the so-called boundary character of this font; the value of *next_char* need not lie between *bc* and *ec*. If the very last instruction of the *lig_kern* array has *skip_byte* = 255, there is a special ligature/kerning program for a boundary character at the left, beginning at location $256 * op_byte + remainder$. The interpretation is that TeX puts implicit boundary characters before and after each consecutive string of characters from the same font. These implicit characters do not appear in the output, but they can affect ligatures and kerning.

If the very first instruction of a character's *lig_kern* program has *skip_byte* > 128, the program actually begins in location $256 * op_byte + remainder$. This feature allows access to large *lig_kern* arrays, because the first instruction must otherwise appear in a location ≤ 255.

Any instruction with *skip_byte* > 128 in the *lig_kern* array must satisfy the condition

$$256 * op_byte + remainder < nl.$$

If such an instruction is encountered during normal program execution, it denotes an unconditional halt; no ligature or kerning command is performed.

define *stop_flag* ≡ *qi*(128) { value indicating 'STOP' in a lig/kern program }
define *kern_flag* ≡ *qi*(128) { op code for a kern step }
define *skip_byte*(#) ≡ #.*b0*
define *next_char*(#) ≡ #.*b1*
define *op_byte*(#) ≡ #.*b2*
define *rem_byte*(#) ≡ #.*b3*

581. Extensible characters are specified by an *extensible_recipe*, which consists of four bytes called *top*, *mid*, *bot*, and *rep* (in this order). These bytes are the character codes of individual pieces used to build up a large symbol. If *top*, *mid*, or *bot* are zero, they are not present in the built-up result. For example, an extensible vertical line is like an extensible bracket, except that the top and bottom pieces are missing.

Let T, M, B, and R denote the respective pieces, or an empty box if the piece isn't present. Then the extensible characters have the form TR^kMR^kB from top to bottom, for some $k \ge 0$, unless M is absent; in the latter case we can have TR^kB for both even and odd values of k. The width of the extensible character is the width of R; and the height-plus-depth is the sum of the individual height-plus-depths of the components used, since the pieces are butted together in a vertical list.

define *ext_top*(#) ≡ #.*b0* { *top* piece in a recipe }
define *ext_mid*(#) ≡ #.*b1* { *mid* piece in a recipe }
define *ext_bot*(#) ≡ #.*b2* { *bot* piece in a recipe }
define *ext_rep*(#) ≡ #.*b3* { *rep* piece in a recipe }

582. The final portion of a TFM file is the *param* array, which is another sequence of *fix_word* values.

param[1] = *slant* is the amount of italic slant, which is used to help position accents. For example, *slant* = .25 means that when you go up one unit, you also go .25 units to the right. The *slant* is a pure number; it's the only *fix_word* other than the design size itself that is not scaled by the design size.

param[2] = *space* is the normal spacing between words in text. Note that character "␣" in the font need not have anything to do with blank spaces.

param[3] = *space_stretch* is the amount of glue stretching between words.

param[4] = *space_shrink* is the amount of glue shrinking between words.

param[5] = *x_height* is the size of one ex in the font; it is also the height of letters for which accents don't have to be raised or lowered.

param[6] = *quad* is the size of one em in the font.

param[7] = *extra_space* is the amount added to *param*[2] at the ends of sentences.

If fewer than seven parameters are present, TeX sets the missing parameters to zero. Fonts used for math symbols are required to have additional parameter information, which is explained later.

define *slant_code* = 1
define *space_code* = 2
define *space_stretch_code* = 3
define *space_shrink_code* = 4
define *x_height_code* = 5
define *quad_code* = 6
define *extra_space_code* = 7

583. So that is what TFM files hold. Since TeX has to absorb such information about lots of fonts, it stores most of the data in a large array called *font_info*. Each item of *font_info* is a *memory_word*; the *fix_word* data gets converted into *scaled* entries, while everything else goes into words of type *four_quarters*.

When the user defines \font\f, say, TeX assigns an internal number to the user's font \f. Adding this number to *font_id_base* gives the *eqtb* location of a "frozen" control sequence that will always select the font.

⟨ Types in the outer block 18 ⟩ +≡
internal_font_number = *font_base* .. *font_max*; { *font* in a *char_node* }
font_index = 0 .. *font_mem_size*; { index into *font_info* }

584. Here now is the (rather formidable) array of font arrays.

```
define otgr_font_flag = "FFFE
define aat_font_flag = "FFFF
define is_aat_font(#) ≡ (font_area[#] = aat_font_flag)
define is_ot_font(#) ≡ ((font_area[#] = otgr_font_flag) ∧ (usingOpenType(font_layout_engine[#])))
define is_gr_font(#) ≡ ((font_area[#] = otgr_font_flag) ∧ (usingGraphite(font_layout_engine[#])))
define is_otgr_font(#) ≡ (font_area[#] = otgr_font_flag)
define is_native_font(#) ≡ (is_aat_font(#) ∨ is_otgr_font(#))  { native fonts have font_area = 65534 or
        65535, which would be a string containing an invalid Unicode character }
define
        is_new_mathfont(#) ≡ ((font_area[#] = otgr_font_flag) ∧
        (isOpenTypeMathFont(font_layout_engine[#])))
define non_char ≡ qi(too_big_char)  { a halfword code that can't match a real character }
define non_address = 0  { a spurious bchar_label }
```

⟨ Global variables 13 ⟩ +≡

```
font_info: array [font_index] of memory_word;  { the big collection of font data }
fmem_ptr: font_index;  { first unused word of font_info }
font_ptr: internal_font_number;  { largest internal font number in use }
font_check: array [internal_font_number] of four_quarters;  { check sum }
font_size: array [internal_font_number] of scaled;  { "at" size }
font_dsize: array [internal_font_number] of scaled;  { "design" size }
font_params: array [internal_font_number] of font_index;  { how many font parameters are present }
font_name: array [internal_font_number] of str_number;  { name of the font }
font_area: array [internal_font_number] of str_number;  { area of the font }
font_bc: array [internal_font_number] of eight_bits;  { beginning (smallest) character code }
font_ec: array [internal_font_number] of eight_bits;  { ending (largest) character code }
font_glue: array [internal_font_number] of pointer;
        { glue specification for interword space, null if not allocated }
font_used: array [internal_font_number] of boolean;
        { has a character from this font actually appeared in the output? }
hyphen_char: array [internal_font_number] of integer;  { current \hyphenchar values }
skew_char: array [internal_font_number] of integer;  { current \skewchar values }
bchar_label: array [internal_font_number] of font_index;
        { start of lig_kern program for left boundary character, non_address if there is none }
font_bchar: array [internal_font_number] of min_quarterword .. non_char;
        { boundary character, non_char if there is none }
font_false_bchar: array [internal_font_number] of min_quarterword .. non_char;
        { font_bchar if it doesn't exist in the font, otherwise non_char }
```

585. Besides the arrays just enumerated, we have directory arrays that make it easy to get at the individual entries in *font_info*. For example, the *char_info* data for character c in font f will be in *font_info*[*char_base*[f] + c].*qqqq*; and if w is the *width_index* part of this word (the *b0* field), the width of the character is *font_info*[*width_base*[f] + w].*sc*. (These formulas assume that *min_quarterword* has already been added to c and to w, since TeX stores its quarterwords that way.)

```
⟨ Global variables 13 ⟩ +≡
char_base: array [internal_font_number] of integer; { base addresses for char_info }
width_base: array [internal_font_number] of integer; { base addresses for widths }
height_base: array [internal_font_number] of integer; { base addresses for heights }
depth_base: array [internal_font_number] of integer; { base addresses for depths }
italic_base: array [internal_font_number] of integer; { base addresses for italic corrections }
lig_kern_base: array [internal_font_number] of integer; { base addresses for ligature/kerning programs }
kern_base: array [internal_font_number] of integer; { base addresses for kerns }
exten_base: array [internal_font_number] of integer; { base addresses for extensible recipes }
param_base: array [internal_font_number] of integer; { base addresses for font parameters }
```

586.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  for k ← font_base to font_max do font_used[k] ← false;
```

587. TeX always knows at least one font, namely the null font. It has no characters, and its seven parameters are all equal to zero.

```
⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
  font_ptr ← null_font; fmem_ptr ← 7; font_name[null_font] ← "nullfont"; font_area[null_font] ← "";
  hyphen_char[null_font] ← "-"; skew_char[null_font] ← −1; bchar_label[null_font] ← non_address;
  font_bchar[null_font] ← non_char; font_false_bchar[null_font] ← non_char; font_bc[null_font] ← 1;
  font_ec[null_font] ← 0; font_size[null_font] ← 0; font_dsize[null_font] ← 0; char_base[null_font] ← 0;
  width_base[null_font] ← 0; height_base[null_font] ← 0; depth_base[null_font] ← 0;
  italic_base[null_font] ← 0; lig_kern_base[null_font] ← 0; kern_base[null_font] ← 0;
  exten_base[null_font] ← 0; font_glue[null_font] ← null; font_params[null_font] ← 7;
  param_base[null_font] ← −1;
  for k ← 0 to 6 do font_info[k].sc ← 0;
```

588.
```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("nullfont", set_font, null_font); text(frozen_null_font) ← "nullfont";
  eqtb[frozen_null_font] ← eqtb[cur_val];
```

589. Of course we want to define macros that suppress the detail of how font information is actually packed, so that we don't have to write things like

$$font_info[width_base[f] + font_info[char_base[f] + c].qqqq.b0].sc$$

too often. The WEB definitions here make $char_info(f)(c)$ the *four_quarters* word of font information corresponding to character c of font f. If q is such a word, $char_width(f)(q)$ will be the character's width; hence the long formula above is at least abbreviated to

$$char_width(f)(char_info(f)(c)).$$

Usually, of course, we will fetch q first and look at several of its fields at the same time.

The italic correction of a character will be denoted by $char_italic(f)(q)$, so it is analogous to $char_width$. But we will get at the height and depth in a slightly different way, since we usually want to compute both height and depth if we want either one. The value of $height_depth(q)$ will be the 8-bit quantity

$$b = height_index \times 16 + depth_index,$$

and if b is such a byte we will write $char_height(f)(b)$ and $char_depth(f)(b)$ for the height and depth of the character c for which $q = char_info(f)(c)$. Got that?

The tag field will be called $char_tag(q)$; the remainder byte will be called $rem_byte(q)$, using a macro that we have already defined above.

Access to a character's *width*, *height*, *depth*, and *tag* fields is part of TeX's inner loop, so we want these macros to produce code that is as fast as possible under the circumstances.

```
define char_info_end(#) ≡ # ] .qqqq
define char_info(#) ≡ font_info [ char_base[#] + char_info_end
define char_width_end(#) ≡ #.b0 ] .sc
define char_width(#) ≡ font_info [ width_base[#] + char_width_end
define char_exists(#) ≡ (#.b0 > min_quarterword)
define char_italic_end(#) ≡ (qo(#.b2)) div 4 ] .sc
define char_italic(#) ≡ font_info [ italic_base[#] + char_italic_end
define height_depth(#) ≡ qo(#.b1)
define char_height_end(#) ≡ (#) div 16 ] .sc
define char_height(#) ≡ font_info [ height_base[#] + char_height_end
define char_depth_end(#) ≡ (#) mod 16 ] .sc
define char_depth(#) ≡ font_info [ depth_base[#] + char_depth_end
define char_tag(#) ≡ ((qo(#.b2)) mod 4)
```

590. The global variable *null_character* is set up to be a word of *char_info* for a character that doesn't exist. Such a word provides a convenient way to deal with erroneous situations.

```
⟨ Global variables 13 ⟩ +≡
null_character: four_quarters; { nonexistent character information }
```

591.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  null_character.b0 ← min_quarterword; null_character.b1 ← min_quarterword;
  null_character.b2 ← min_quarterword; null_character.b3 ← min_quarterword;
```

592. Here are some macros that help process ligatures and kerns. We write *char_kern*(*f*)(*j*) to find the amount of kerning specified by kerning command *j* in font *f*. If *j* is the *char_info* for a character with a ligature/kern program, the first instruction of that program is either *i* = *font_info*[*lig_kern_start*(*f*)(*j*)] or *font_info*[*lig_kern_restart*(*f*)(*i*)], depending on whether or not *skip_byte*(*i*) ≤ *stop_flag*.

The constant *kern_base_offset* should be simplified, for Pascal compilers that do not do local optimization.

define *char_kern_end*(#) ≡ 256 ∗ *op_byte*(#) + *rem_byte*(#)] *.sc*
define *char_kern*(#) ≡ *font_info* [*kern_base*[#] + *char_kern_end*
define *kern_base_offset* ≡ 256 ∗ (128 + *min_quarterword*)
define *lig_kern_start*(#) ≡ *lig_kern_base*[#] + *rem_byte* { beginning of lig/kern program }
define *lig_kern_restart_end*(#) ≡ 256 ∗ *op_byte*(#) + *rem_byte*(#) + 32768 − *kern_base_offset*
define *lig_kern_restart*(#) ≡ *lig_kern_base*[#] + *lig_kern_restart_end*

593. Font parameters are referred to as *slant*(*f*), *space*(*f*), etc.

define *param_end*(#) ≡ *param_base*[#]] *.sc*
define *param*(#) ≡ *font_info* [# + *param_end*
define *slant* ≡ *param*(*slant_code*) { slant to the right, per unit distance upward }
define *space* ≡ *param*(*space_code*) { normal space between words }
define *space_stretch* ≡ *param*(*space_stretch_code*) { stretch between words }
define *space_shrink* ≡ *param*(*space_shrink_code*) { shrink between words }
define *x_height* ≡ *param*(*x_height_code*) { one ex }
define *quad* ≡ *param*(*quad_code*) { one em }
define *extra_space* ≡ *param*(*extra_space_code*) { additional space at end of sentence }

⟨ The em width for *cur_font* 593 ⟩ ≡
quad(*cur_font*)

This code is used in section 490.

594. ⟨ The x-height for *cur_font* 594 ⟩ ≡
x_height(*cur_font*)

This code is used in section 490.

595. TeX checks the information of a TFM file for validity as the file is being read in, so that no further checks will be needed when typesetting is going on. The somewhat tedious subroutine that does this is called *read_font_info*. It has four parameters: the user font identifier u, the file name and area strings *nom* and *aire*, and the "at" size s. If s is negative, it's the negative of a scale factor to be applied to the design size; $s = -1000$ is the normal case. Otherwise s will be substituted for the design size; in this case, s must be positive and less than 2048 pt (i.e., it must be less than 2^{27} when considered as an integer).

The subroutine opens and closes a global file variable called *tfm_file*. It returns the value of the internal font number that was just loaded. If an error is detected, an error message is issued and no font information is stored; *null_font* is returned in this case.

```
define bad_tfm = 11 { label for read_font_info }
define abort ≡ goto bad_tfm { do this when the TFM data is wrong }
function read_font_info(u : pointer; nom, aire : str_number; s : scaled): internal_font_number;
        { input a TFM file }
  label done, bad_tfm, not_found;
  var k: font_index; { index into font_info }
    file_opened: boolean; { was tfm_file successfully opened? }
    lf, lh, bc, ec, nw, nh, nd, ni, nl, nk, ne, np: halfword; { sizes of subfiles }
    f: internal_font_number; { the new font's number }
    g: internal_font_number; { the number to return }
    a, b, c, d: eight_bits; { byte variables }
    qw: four_quarters; sw: scaled; { accumulators }
    bch_label: integer; { left boundary start location, or infinity }
    bchar: 0 .. 256; { boundary character, or 256 }
    z: scaled; { the design size or the "at" size }
    alpha: integer; beta: 1 .. 16; { auxiliary quantities used in fixed-point multiplication }
  begin g ← null_font;
  file_opened ← false; pack_file_name(nom, aire, cur_ext);
  if XeTeX_tracing_fonts_state > 0 then
    begin begin_diagnostic; print_nl("Requested␣font␣"""); print_c_string(stringcast(name_of_file + 1));
    print('"');
    if s < 0 then
      begin print("␣scaled␣"); print_int(−s);
      end
    else begin print("␣at␣"); print_scaled(s); print("pt");
      end;
    end_diagnostic(false);
    end;
  if quoted_filename then
    begin { quoted name, so try for a native font }
    g ← load_native_font(u, nom, aire, s);
    if g ≠ null_font then goto done;
    end; { it was an unquoted name, or not found as an installed font, so try for a TFM file }
  ⟨Read and check the font data if file exists; abort if the TFM file is malformed; if there's no room for this
      font, say so and goto done; otherwise incr(font_ptr) and goto done 597⟩;
  if g ≠ null_font then goto done;
  if ¬quoted_filename then
    begin { we failed to find a TFM file, so try for a native font }
    g ← load_native_font(u, nom, aire, s);
    if g ≠ null_font then goto done
    end;
bad_tfm: if suppress_fontnotfound_error = 0 then
    begin ⟨Report that the font won't be loaded 596⟩;
```

```
    end;
done: if file_opened then b_close(tfm_file);
  if XeTeX_tracing_fonts_state > 0 then
    begin if g = null_font then
      begin begin_diagnostic; print_nl("␣->␣font␣not␣found,␣using␣""nullfont""");
      end_diagnostic(false);
      end
    else if file_opened then
        begin begin_diagnostic; print_nl("␣->␣"); print_c_string(stringcast(name_of_file + 1));
        end_diagnostic(false);
        end;
    end;
  read_font_info ← g;
  end;
```

596. There are programs called `TFtoPL` and `PLtoTF` that convert between the `TFM` format and a symbolic property-list format that can be easily edited. These programs contain extensive diagnostic information, so TEX does not have to bother giving precise details about why it rejects a particular `TFM` file.

```
  define start_font_error_message ≡ print_err("Font␣"); sprint_cs(u); print_char("=");
        if file_name_quote_char ≠ 0 then print_char(file_name_quote_char);
        print_file_name(nom, aire, cur_ext);
        if file_name_quote_char ≠ 0 then print_char(file_name_quote_char);
        if s ≥ 0 then
          begin print("␣at␣"); print_scaled(s); print("pt");
          end
        else if s ≠ −1000 then
            begin print("␣scaled␣"); print_int(−s);
            end
```

⟨ Report that the font won't be loaded 596 ⟩ ≡

```
  start_font_error_message;
  if file_opened then print("␣not␣loadable:␣Bad␣metric␣(TFM)␣file")
  else print("␣not␣loadable:␣Metric␣(TFM)␣file␣not␣found");
  help5("I␣wasn´t␣able␣to␣read␣the␣size␣data␣for␣this␣font,")
  ("so␣I␣will␣ignore␣the␣font␣specification.")
  ("[Wizards␣can␣fix␣TFM␣files␣using␣TFtoPL/PLtoTF.]")
  ("You␣might␣try␣inserting␣a␣different␣font␣spec;")
  ("e.g.,␣type␣`I\font<same␣font␣id>=<substitute␣font␣name>´."); error
```

This code is used in section 595.

597. ⟨ Read and check the font data if file exists; *abort* if the TFM file is malformed; if there's no room for this font, say so and **goto** *done*; otherwise *incr*(*font_ptr*) and **goto** *done* 597 ⟩ ≡

```
⟨ Open tfm_file for input and begin 598 ⟩;
⟨ Read the TFM size fields 600 ⟩;
⟨ Use size fields to allocate font information 601 ⟩;
⟨ Read the TFM header 603 ⟩;
⟨ Read character data 604 ⟩;
⟨ Read box dimensions 606 ⟩;
⟨ Read ligature/kern program 608 ⟩;
⟨ Read extensible character recipes 609 ⟩;
⟨ Read font parameters 610 ⟩;
⟨ Make final adjustments and goto done 611 ⟩;
end
```

This code is used in section 595.

598. ⟨ Open *tfm_file* for input and **begin** 598 ⟩ ≡

```
if aire = "" then pack_file_name(nom, TEX_font_area, ".tfm")
else pack_file_name(nom, aire, ".tfm");
check_for_tfm_font_mapping;
if b_open_in(tfm_file) then
  begin file_opened ← true
```

This code is used in section 597.

599. Note: A malformed TFM file might be shorter than it claims to be; thus *eof*(*tfm_file*) might be true when *read_font_info* refers to *tfm_file*↑ or when it says *get*(*tfm_file*). If such circumstances cause system error messages, you will have to defeat them somehow, for example by defining *fget* to be '**begin** *get*(*tfm_file*); **if** *eof*(*tfm_file*) **then** *abort*; **end**'.

```
define fget ≡ get(tfm_file)
define fbyte ≡ tfm_file↑
define read_sixteen(#) ≡
        begin # ← fbyte;
        if # > 127 then abort;
        fget; # ← # * ´400 + fbyte;
        end
define store_four_quarters(#) ≡
        begin fget; a ← fbyte; qw.b0 ← qi(a); fget; b ← fbyte; qw.b1 ← qi(b); fget; c ← fbyte;
        qw.b2 ← qi(c); fget; d ← fbyte; qw.b3 ← qi(d); # ← qw;
        end
```

600. ⟨ Read the TFM size fields 600 ⟩ ≡

```
begin read_sixteen(lf); fget; read_sixteen(lh); fget; read_sixteen(bc); fget; read_sixteen(ec);
if (bc > ec + 1) ∨ (ec > 255) then abort;
if bc > 255 then { bc = 256 and ec = 255 }
  begin bc ← 1; ec ← 0;
  end;
fget; read_sixteen(nw); fget; read_sixteen(nh); fget; read_sixteen(nd); fget; read_sixteen(ni); fget;
read_sixteen(nl); fget; read_sixteen(nk); fget; read_sixteen(ne); fget; read_sixteen(np);
if lf ≠ 6 + lh + (ec − bc + 1) + nw + nh + nd + ni + nl + nk + ne + np then abort;
if (nw = 0) ∨ (nh = 0) ∨ (nd = 0) ∨ (ni = 0) then abort;
end
```

This code is used in section 597.

601. The preliminary settings of the index-offset variables *char_base*, *width_base*, *lig_kern_base*, *kern_base*, and *exten_base* will be corrected later by subtracting *min_quarterword* from them; and we will subtract 1 from *param_base* too. It's best to forget about such anomalies until later.

```
⟨ Use size fields to allocate font information 601 ⟩ ≡
  lf ← lf − 6 − lh; { lf words should be loaded into font_info }
  if np < 7 then lf ← lf + 7 − np; { at least seven parameters will appear }
  if (font_ptr = font_max) ∨ (fmem_ptr + lf > font_mem_size) then
    ⟨ Apologize for not loading the font, goto done 602 ⟩;
  f ← font_ptr + 1; char_base[f] ← fmem_ptr − bc; width_base[f] ← char_base[f] + ec + 1;
  height_base[f] ← width_base[f] + nw; depth_base[f] ← height_base[f] + nh;
  italic_base[f] ← depth_base[f] + nd; lig_kern_base[f] ← italic_base[f] + ni;
  kern_base[f] ← lig_kern_base[f] + nl − kern_base_offset;
  exten_base[f] ← kern_base[f] + kern_base_offset + nk; param_base[f] ← exten_base[f] + ne
```

This code is used in section 597.

602.

```
⟨ Apologize for not loading the font, goto done 602 ⟩ ≡
  begin start_font_error_message; print("␣not␣loaded:␣Not␣enough␣room␣left");
  help4("I´m␣afraid␣I␣won´t␣be␣able␣to␣make␣use␣of␣this␣font,")
  ("because␣my␣memory␣for␣character-size␣data␣is␣too␣small.")
  ("If␣you´re␣really␣stuck,␣ask␣a␣wizard␣to␣enlarge␣me.")
  ("Or␣maybe␣try␣`I\font<same␣font␣id>=<name␣of␣loaded␣font>´."); error; goto done;
  end
```

This code is used in sections 601 and 744.

603. Only the first two words of the header are needed by TEX82.

```
⟨ Read the TFM header 603 ⟩ ≡
  begin if lh < 2 then abort;
  store_four_quarters(font_check[f]); fget; read_sixteen(z); { this rejects a negative design size }
  fget; z ← z ∗ ´400 + fbyte; fget; z ← (z ∗ ´20) + (fbyte div ´20);
  if z < unity then abort;
  while lh > 2 do
    begin fget; fget; fget; fget; decr(lh); { ignore the rest of the header }
    end;
  font_dsize[f] ← z;
  if s ≠ −1000 then
    if s ≥ 0 then z ← s
    else z ← xn_over_d(z, −s, 1000);
  font_size[f] ← z;
  end
```

This code is used in section 597.

604. ⟨ Read character data 604 ⟩ ≡

```
for k ← fmem_ptr to width_base[f] − 1 do
  begin store_four_quarters(font_info[k].qqqq);
  if (a ≥ nw) ∨ (b div ´20 ≥ nh) ∨ (b mod ´20 ≥ nd) ∨ (c div 4 ≥ ni) then abort;
  case c mod 4 of
  lig_tag: if d ≥ nl then abort;
  ext_tag: if d ≥ ne then abort;
  list_tag: ⟨ Check for charlist cycle 605 ⟩;
  othercases do_nothing { no_tag }
  endcases;
  end
```

This code is used in section 597.

605. We want to make sure that there is no cycle of characters linked together by *list_tag* entries, since such a cycle would get TeX into an endless loop. If such a cycle exists, the routine here detects it when processing the largest character code in the cycle.

```
define check_byte_range(#) ≡
          begin if (# < bc) ∨ (# > ec) then abort
          end
define current_character_being_worked_on ≡ k + bc − fmem_ptr
```

⟨ Check for charlist cycle 605 ⟩ ≡

```
begin check_byte_range(d);
while d < current_character_being_worked_on do
  begin qw ← char_info(f)(d); { N.B.: not qi(d), since char_base[f] hasn't been adjusted yet }
  if char_tag(qw) ≠ list_tag then goto not_found;
  d ← qo(rem_byte(qw)); { next character on the list }
  end;
if d = current_character_being_worked_on then abort; { yes, there's a cycle }
not_found: end
```

This code is used in section 604.

606. A *fix_word* whose four bytes are (a, b, c, d) from left to right represents the number

$$x = \begin{cases} b \cdot 2^{-4} + c \cdot 2^{-12} + d \cdot 2^{-20}, & \text{if } a = 0; \\ -16 + b \cdot 2^{-4} + c \cdot 2^{-12} + d \cdot 2^{-20}, & \text{if } a = 255. \end{cases}$$

(No other choices of a are allowed, since the magnitude of a number in design-size units must be less than 16.) We want to multiply this quantity by the integer z, which is known to be less than 2^{27}. If $z < 2^{23}$, the individual multiplications $b \cdot z$, $c \cdot z$, $d \cdot z$ cannot overflow; otherwise we will divide z by 2, 4, 8, or 16, to obtain a multiplier less than 2^{23}, and we can compensate for this later. If z has thereby been replaced by $z' = z/2^e$, let $\beta = 2^{4-e}$; we shall compute

$$\lfloor (b + c \cdot 2^{-8} + d \cdot 2^{-16})\, z'/\beta \rfloor$$

if $a = 0$, or the same quantity minus $\alpha = 2^{4+e} z'$ if $a = 255$. This calculation must be done exactly, in order to guarantee portability of TeX between computers.

```
define store_scaled(#) ≡
        begin fget; a ← fbyte; fget; b ← fbyte; fget; c ← fbyte; fget; d ← fbyte;
        sw ← (((((d * z) div ´400) + (c * z)) div ´400) + (b * z)) div beta;
        if a = 0 then # ← sw else if a = 255 then # ← sw − alpha else abort;
        end
```

```
⟨ Read box dimensions 606 ⟩ ≡
  begin ⟨ Replace z by z′ and compute α, β 607 ⟩;
  for k ← width_base[f] to lig_kern_base[f] − 1 do store_scaled(font_info[k].sc);
  if font_info[width_base[f]].sc ≠ 0 then abort;   { width[0] must be zero }
  if font_info[height_base[f]].sc ≠ 0 then abort;   { height[0] must be zero }
  if font_info[depth_base[f]].sc ≠ 0 then abort;   { depth[0] must be zero }
  if font_info[italic_base[f]].sc ≠ 0 then abort;   { italic[0] must be zero }
  end
```

This code is used in section 597.

607.

```
⟨ Replace z by z′ and compute α, β 607 ⟩ ≡
  begin alpha ← 16;
  while z ≥ ´40000000 do
    begin z ← z div 2; alpha ← alpha + alpha;
    end;
  beta ← 256 div alpha; alpha ← alpha * z;
  end
```

This code is used in section 606.

608. **define** *check_existence*(#) ≡
begin *check_byte_range*(#); *qw* ← *char_info*(*f*)(#); { N.B.: not *qi*(#) }
if ¬*char_exists*(*qw*) **then** *abort*;
end

⟨ Read ligature/kern program 608 ⟩ ≡
bch_label ← ′77777; *bchar* ← 256;
if *nl* > 0 **then**
begin for *k* ← *lig_kern_base*[*f*] **to** *kern_base*[*f*] + *kern_base_offset* − 1 **do**
begin *store_four_quarters*(*font_info*[*k*].*qqqq*);
if *a* > 128 **then**
begin if 256 ∗ *c* + *d* ≥ *nl* **then** *abort*;
if *a* = 255 **then**
if *k* = *lig_kern_base*[*f*] **then** *bchar* ← *b*;
end
else begin if *b* ≠ *bchar* **then** *check_existence*(*b*);
if *c* < 128 **then** *check_existence*(*d*) { check ligature }
else if 256 ∗ (*c* − 128) + *d* ≥ *nk* **then** *abort*; { check kern }
if *a* < 128 **then**
if *k* − *lig_kern_base*[*f*] + *a* + 1 ≥ *nl* **then** *abort*;
end;
end;
if *a* = 255 **then** *bch_label* ← 256 ∗ *c* + *d*;
end;
for *k* ← *kern_base*[*f*] + *kern_base_offset* **to** *exten_base*[*f*] − 1 **do** *store_scaled*(*font_info*[*k*].*sc*);
This code is used in section 597.

609. ⟨ Read extensible character recipes 609 ⟩ ≡
for *k* ← *exten_base*[*f*] **to** *param_base*[*f*] − 1 **do**
begin *store_four_quarters*(*font_info*[*k*].*qqqq*);
if *a* ≠ 0 **then** *check_existence*(*a*);
if *b* ≠ 0 **then** *check_existence*(*b*);
if *c* ≠ 0 **then** *check_existence*(*c*);
check_existence(*d*);
end
This code is used in section 597.

610. We check to see that the TFM file doesn't end prematurely; but no error message is given for files having more than *lf* words.

⟨ Read font parameters 610 ⟩ ≡
begin for *k* ← 1 **to** *np* **do**
if *k* = 1 **then** { the *slant* parameter is a pure number }
begin *fget*; *sw* ← *fbyte*;
if *sw* > 127 **then** *sw* ← *sw* − 256;
fget; *sw* ← *sw* ∗ ′400 + *fbyte*; *fget*; *sw* ← *sw* ∗ ′400 + *fbyte*; *fget*;
font_info[*param_base*[*f*]].*sc* ← (*sw* ∗ ′20) + (*fbyte* **div** ′20);
end
else *store_scaled*(*font_info*[*param_base*[*f*] + *k* − 1].*sc*);
if *eof*(*tfm_file*) **then** *abort*;
for *k* ← *np* + 1 **to** 7 **do** *font_info*[*param_base*[*f*] + *k* − 1].*sc* ← 0;
end
This code is used in section 597.

611. Now to wrap it up, we have checked all the necessary things about the TFM file, and all we need to do is put the finishing touches on the data for the new font.

```
define adjust(#) ≡ #[f] ← qo(#[f])  { correct for the excess min_quarterword that was added }
⟨ Make final adjustments and goto done 611 ⟩ ≡
  if np ≥ 7 then font_params[f] ← np else font_params[f] ← 7;
  hyphen_char[f] ← default_hyphen_char; skew_char[f] ← default_skew_char;
  if bch_label < nl then bchar_label[f] ← bch_label + lig_kern_base[f]
  else bchar_label[f] ← non_address;
  font_bchar[f] ← qi(bchar); font_false_bchar[f] ← qi(bchar);
  if bchar ≤ ec then
    if bchar ≥ bc then
      begin qw ← char_info(f)(bchar);  { N.B.: not qi(bchar) }
      if char_exists(qw) then font_false_bchar[f] ← non_char;
      end;
  font_name[f] ← nom; font_area[f] ← aire; font_bc[f] ← bc; font_ec[f] ← ec; font_glue[f] ← null;
  adjust(char_base); adjust(width_base); adjust(lig_kern_base); adjust(kern_base); adjust(exten_base);
  decr(param_base[f]); fmem_ptr ← fmem_ptr + lf; font_ptr ← f; g ← f;
  font_mapping[f] ← load_tfm_font_mapping; goto done
```

This code is used in section 597.

612. Before we forget about the format of these tables, let's deal with two of TeX's basic scanning routines related to font information.

```
⟨ Declare procedures that scan font-related stuff 612 ⟩ ≡
procedure scan_font_ident;
  var f: internal_font_number; m: halfword;
  begin ⟨ Get the next non-blank non-call token 440 ⟩;
  if cur_cmd = def_font then f ← cur_font
  else if cur_cmd = set_font then f ← cur_chr
    else if cur_cmd = def_family then
        begin m ← cur_chr; scan_math_fam_int; f ← equiv(m + cur_val);
        end
      else begin print_err("Missing␣font␣identifier");
        help2("I␣was␣looking␣for␣a␣control␣sequence␣whose")
        ("current␣meaning␣has␣been␣defined␣by␣\font."); back_error; f ← null_font;
        end;
  cur_val ← f;
  end;
```

See also section 613.

This code is used in section 443.

613. The following routine is used to implement '`\fontdimen` *n f*'. The boolean parameter *writing* is set *true* if the calling program intends to change the parameter value.

```
⟨ Declare procedures that scan font-related stuff 612 ⟩ +≡
procedure find_font_dimen(writing : boolean); { sets cur_val to font_info location }
  var f: internal_font_number; n: integer; { the parameter number }
  begin scan_int; n ← cur_val; scan_font_ident; f ← cur_val;
  if n ≤ 0 then cur_val ← fmem_ptr
  else begin if writing ∧ (n ≤ space_shrink_code) ∧ (n ≥ space_code) ∧ (font_glue[f] ≠ null) then
      begin delete_glue_ref(font_glue[f]); font_glue[f] ← null;
      end;
    if n > font_params[f] then
      if f < font_ptr then cur_val ← fmem_ptr
      else ⟨ Increase the number of parameters in the last font 615 ⟩
    else cur_val ← n + param_base[f];
    end;
  ⟨ Issue an error message if cur_val = fmem_ptr 614 ⟩;
  end;
```

614.
```
⟨ Issue an error message if cur_val = fmem_ptr 614 ⟩ ≡
  if cur_val = fmem_ptr then
    begin print_err("Font␣"); print_esc(font_id_text(f)); print("␣has␣only␣");
    print_int(font_params[f]); print("␣fontdimen␣parameters");
    help2("To␣increase␣the␣number␣of␣font␣parameters,␣you␣must")
    ("use␣\fontdimen␣immediately␣after␣the␣\font␣is␣loaded."); error;
    end
```

This code is used in section 613.

615.
```
⟨ Increase the number of parameters in the last font 615 ⟩ ≡
  begin repeat if fmem_ptr = font_mem_size then overflow("font␣memory", font_mem_size);
    font_info[fmem_ptr].sc ← 0; incr(fmem_ptr); incr(font_params[f]);
  until n = font_params[f];
  cur_val ← fmem_ptr − 1; { this equals param_base[f] + font_params[f] }
  end
```

This code is used in section 613.

616. When TEX wants to typeset a character that doesn't exist, the character node is not created; thus the output routine can assume that characters exist when it sees them. The following procedure prints a warning message unless the user has suppressed it.

```
⟨ Declare subroutines for new_character 616 ⟩ ≡
procedure char_warning(f : internal_font_number; c : integer);
  var old_setting: integer;  { saved value of tracing_online }
  begin if tracing_lost_chars > 0 then
    begin old_setting ← tracing_online;
    if eTeX_ex ∧ (tracing_lost_chars > 1) then tracing_online ← 1;
    begin begin_diagnostic; print_nl("Missing␣character:␣There␣is␣no␣");
    if c < "10000 then print_ASCII(c)
    else print_char(c);  { non-Plane 0 Unicodes can't be sent through print_ASCII }
    print("␣in␣font␣"); slow_print(font_name[f]); print_char("!"); end_diagnostic(false);
    end; tracing_online ← old_setting;
    end;
  end;
```

See also section 744.

This code is used in section 617.

617. We need a few subroutines for *new_character*.

```
⟨ Declare subroutines for new_character 616 ⟩
```

618. Here is a function that returns a pointer to a character node for a given character in a given font. If that character doesn't exist, *null* is returned instead.

```
function new_character(f : internal_font_number; c : eight_bits): pointer;
  label exit;
  var p: pointer;  { newly allocated node }
  begin if font_bc[f] ≤ c then
    if font_ec[f] ≥ c then
      if char_exists(char_info(f)(qi(c))) then
        begin p ← get_avail; font(p) ← f; character(p) ← qi(c); new_character ← p; return;
        end;
  char_warning(f, c); new_character ← null;
exit: end;
```

619. Device-independent file format. The most important output produced by a run of TeX is the "device independent" (`DVI`) file that specifies where characters and rules are to appear on printed pages. The form of these files was designed by David R. Fuchs in 1979. Almost any reasonable typesetting device can be driven by a program that takes `DVI` files as input, and dozens of such `DVI`-to-whatever programs have been written. Thus, it is possible to print the output of TeX on many different kinds of equipment, using TeX as a device-independent "front end."

A `DVI` file is a stream of 8-bit bytes, which may be regarded as a series of commands in a machine-like language. The first byte of each command is the operation code, and this code is followed by zero or more bytes that provide parameters to the command. The parameters themselves may consist of several consecutive bytes; for example, the '*set_rule*' command has two parameters, each of which is four bytes long. Parameters are usually regarded as nonnegative integers; but four-byte-long parameters, and shorter parameters that denote distances, can be either positive or negative. Such parameters are given in two's complement notation. For example, a two-byte-long distance parameter has a value between -2^{15} and $2^{15}-1$. As in `TFM` files, numbers that occupy more than one byte position appear in BigEndian order.

XeTeX extends the format of `DVI` with its own commands, and thus produced "extended device independent" (`XDV`) files.

A `DVI` file consists of a "preamble," followed by a sequence of one or more "pages," followed by a "postamble." The preamble is simply a *pre* command, with its parameters that define the dimensions used in the file; this must come first. Each "page" consists of a *bop* command, followed by any number of other commands that tell where characters are to be placed on a physical page, followed by an *eop* command. The pages appear in the order that TeX generated them. If we ignore *nop* commands and *fnt_def* commands (which are allowed between any two commands in the file), each *eop* command is immediately followed by a *bop* command, or by a *post* command; in the latter case, there are no more pages in the file, and the remaining bytes form the postamble. Further details about the postamble will be explained later.

Some parameters in `DVI` commands are "pointers." These are four-byte quantities that give the location number of some other byte in the file; the first byte is number 0, then comes number 1, and so on. For example, one of the parameters of a *bop* command points to the previous *bop*; this makes it feasible to read the pages in backwards order, in case the results are being directed to a device that stacks its output face up. Suppose the preamble of a `DVI` file occupies bytes 0 to 99. Now if the first page occupies bytes 100 to 999, say, and if the second page occupies bytes 1000 to 1999, then the *bop* that starts in byte 1000 points to 100 and the *bop* that starts in byte 2000 points to 1000. (The very first *bop*, i.e., the one starting in byte 100, has a pointer of -1.)

620. The `DVI` format is intended to be both compact and easily interpreted by a machine. Compactness is achieved by making most of the information implicit instead of explicit. When a `DVI`-reading program reads the commands for a page, it keeps track of several quantities: (a) The current font f is an integer; this value is changed only by *fnt* and *fnt_num* commands. (b) The current position on the page is given by two numbers called the horizontal and vertical coordinates, h and v. Both coordinates are zero at the upper left corner of the page; moving to the right corresponds to increasing the horizontal coordinate, and moving down corresponds to increasing the vertical coordinate. Thus, the coordinates are essentially Cartesian, except that vertical directions are flipped; the Cartesian version of (h, v) would be $(h, -v)$. (c) The current spacing amounts are given by four numbers w, x, y, and z, where w and x are used for horizontal spacing and where y and z are used for vertical spacing. (d) There is a stack containing (h, v, w, x, y, z) values; the `DVI` commands *push* and *pop* are used to change the current level of operation. Note that the current font f is not pushed and popped; the stack contains only information about positioning.

The values of h, v, w, x, y, and z are signed integers having up to 32 bits, including the sign. Since they represent physical distances, there is a small unit of measurement such that increasing h by 1 means moving a certain tiny distance to the right. The actual unit of measurement is variable, as explained below; TeX sets things up so that its `DVI` output is in sp units, i.e., scaled points, in agreement with all the *scaled* dimensions in TeX's data structures.

621. Here is a list of all the commands that may appear in a `XDV` file. Each command is specified by its symbolic name (e.g., *bop*), its opcode byte (e.g., 139), and its parameters (if any). The parameters are followed by a bracketed number telling how many bytes they occupy; for example, 'p[4]' means that parameter p is four bytes long.

set_char_0 0. Typeset character number 0 from font f such that the reference point of the character is at (h, v). Then increase h by the width of that character. Note that a character may have zero or negative width, so one cannot be sure that h will advance after this command; but h usually does increase.

set_char_1 through *set_char_127* (opcodes 1 to 127). Do the operations of *set_char_0*; but use the character whose number matches the opcode, instead of character 0.

set1 128 c[1]. Same as *set_char_0*, except that character number c is typeset. TeX82 uses this command for characters in the range $128 \le c < 256$.

set2 129 c[2]. Same as *set1*, except that c is two bytes long, so it is in the range $0 \le c < 65536$. TeX82 never uses this command, but it should come in handy for extensions of TeX that deal with oriental languages.

set3 130 c[3]. Same as *set1*, except that c is three bytes long, so it can be as large as $2^{24} - 1$. Not even the Chinese language has this many characters, but this command might prove useful in some yet unforeseen extension.

set4 131 c[4]. Same as *set1*, except that c is four bytes long. Imagine that.

set_rule 132 a[4] b[4]. Typeset a solid black rectangle of height a and width b, with its bottom left corner at (h, v). Then set $h \leftarrow h + b$. If either $a \le 0$ or $b \le 0$, nothing should be typeset. Note that if $b < 0$, the value of h will decrease even though nothing else happens. See below for details about how to typeset rules so that consistency with METAFONT is guaranteed.

put1 133 c[1]. Typeset character number c from font f such that the reference point of the character is at (h, v). (The 'put' commands are exactly like the 'set' commands, except that they simply put out a character or a rule without moving the reference point afterwards.)

put2 134 c[2]. Same as *set2*, except that h is not changed.

put3 135 c[3]. Same as *set3*, except that h is not changed.

put4 136 c[4]. Same as *set4*, except that h is not changed.

put_rule 137 a[4] b[4]. Same as *set_rule*, except that h is not changed.

nop 138. No operation, do nothing. Any number of *nop*'s may occur between `DVI` commands, but a *nop* cannot be inserted between a command and its parameters or between two parameters.

bop 139 c_0[4] c_1[4] ... c_9[4] p[4]. Beginning of a page: Set $(h, v, w, x, y, z) \leftarrow (0, 0, 0, 0, 0, 0)$ and set the stack empty. Set the current font f to an undefined value. The ten c_i parameters hold the values of `\count0` ... `\count9` in TeX at the time `\shipout` was invoked for this page; they can be used to identify pages, if a user wants to print only part of a `DVI` file. The parameter p points to the previous *bop* in the file; the first *bop* has $p = -1$.

eop 140. End of page: Print what you have read since the previous *bop*. At this point the stack should be empty. (The `DVI`-reading programs that drive most output devices will have kept a buffer of the material that appears on the page that has just ended. This material is largely, but not entirely, in order by v coordinate and (for fixed v) by h coordinate; so it usually needs to be sorted into some order that is appropriate for the device in question.)

push 141. Push the current values of (h, v, w, x, y, z) onto the top of the stack; do not change any of these values. Note that f is not pushed.

pop 142. Pop the top six values off of the stack and assign them respectively to (h, v, w, x, y, z). The number of pops should never exceed the number of pushes, since it would be highly embarrassing if the stack were empty at the time of a *pop* command.

right1 143 b[1]. Set $h \leftarrow h+b$, i.e., move right b units. The parameter is a signed number in two's complement notation, $-128 \le b < 128$; if $b < 0$, the reference point moves left.

right2 144 b[2]. Same as *right1*, except that b is a two-byte quantity in the range $-32768 \le b < 32768$.

right3 145 b[3]. Same as *right1*, except that b is a three-byte quantity in the range $-2^{23} \le b < 2^{23}$.

right4 146 b[4]. Same as *right1*, except that b is a four-byte quantity in the range $-2^{31} \le b < 2^{31}$.

w0 147. Set $h \leftarrow h + w$; i.e., move right w units. With luck, this parameterless command will usually suffice, because the same kind of motion will occur several times in succession; the following commands explain how w gets particular values.

w1 148 b[1]. Set $w \leftarrow b$ and $h \leftarrow h + b$. The value of b is a signed quantity in two's complement notation, $-128 \le b < 128$. This command changes the current w spacing and moves right by b.

w2 149 b[2]. Same as *w1*, but b is two bytes long, $-32768 \le b < 32768$.

w3 150 b[3]. Same as *w1*, but b is three bytes long, $-2^{23} \le b < 2^{23}$.

w4 151 b[4]. Same as *w1*, but b is four bytes long, $-2^{31} \le b < 2^{31}$.

x0 152. Set $h \leftarrow h + x$; i.e., move right x units. The 'x' commands are like the 'w' commands except that they involve x instead of w.

x1 153 b[1]. Set $x \leftarrow b$ and $h \leftarrow h + b$. The value of b is a signed quantity in two's complement notation, $-128 \le b < 128$. This command changes the current x spacing and moves right by b.

x2 154 b[2]. Same as *x1*, but b is two bytes long, $-32768 \le b < 32768$.

x3 155 b[3]. Same as *x1*, but b is three bytes long, $-2^{23} \le b < 2^{23}$.

x4 156 b[4]. Same as *x1*, but b is four bytes long, $-2^{31} \le b < 2^{31}$.

down1 157 a[1]. Set $v \leftarrow v + a$, i.e., move down a units. The parameter is a signed number in two's complement notation, $-128 \le a < 128$; if $a < 0$, the reference point moves up.

down2 158 a[2]. Same as *down1*, except that a is a two-byte quantity in the range $-32768 \le a < 32768$.

down3 159 a[3]. Same as *down1*, except that a is a three-byte quantity in the range $-2^{23} \le a < 2^{23}$.

down4 160 a[4]. Same as *down1*, except that a is a four-byte quantity in the range $-2^{31} \le a < 2^{31}$.

y0 161. Set $v \leftarrow v + y$; i.e., move down y units. With luck, this parameterless command will usually suffice, because the same kind of motion will occur several times in succession; the following commands explain how y gets particular values.

y1 162 a[1]. Set $y \leftarrow a$ and $v \leftarrow v + a$. The value of a is a signed quantity in two's complement notation, $-128 \le a < 128$. This command changes the current y spacing and moves down by a.

y2 163 a[2]. Same as *y1*, but a is two bytes long, $-32768 \le a < 32768$.

y3 164 a[3]. Same as *y1*, but a is three bytes long, $-2^{23} \le a < 2^{23}$.

y4 165 a[4]. Same as *y1*, but a is four bytes long, $-2^{31} \le a < 2^{31}$.

z0 166. Set $v \leftarrow v + z$; i.e., move down z units. The 'z' commands are like the 'y' commands except that they involve z instead of y.

z1 167 a[1]. Set $z \leftarrow a$ and $v \leftarrow v + a$. The value of a is a signed quantity in two's complement notation, $-128 \le a < 128$. This command changes the current z spacing and moves down by a.

z2 168 a[2]. Same as *z1*, but a is two bytes long, $-32768 \le a < 32768$.

z3 169 a[3]. Same as *z1*, but a is three bytes long, $-2^{23} \le a < 2^{23}$.

z4 170 a[4]. Same as *z1*, but a is four bytes long, $-2^{31} \le a < 2^{31}$.

fnt_num_0 171. Set $f \leftarrow 0$. Font 0 must previously have been defined by a *fnt_def* instruction, as explained below.

fnt_num_1 through *fnt_num_63* (opcodes 172 to 234). Set $f \leftarrow 1$, ..., $f \leftarrow 63$, respectively.

fnt1 235 k[1]. Set $f \leftarrow k$. TeX82 uses this command for font numbers in the range $64 \le k < 256$.

fnt2 236 k[2]. Same as *fnt1*, except that k is two bytes long, so it is in the range $0 \le k < 65536$. TeX82 never generates this command, but large font numbers may prove useful for specifications of color or texture, or they may be used for special fonts that have fixed numbers in some external coding scheme.

fnt3 237 $k[3]$. Same as *fnt1*, except that k is three bytes long, so it can be as large as $2^{24}-1$.

fnt4 238 $k[4]$. Same as *fnt1*, except that k is four bytes long; this is for the really big font numbers (and for the negative ones).

xxx1 239 $k[1]$ $x[k]$. This command is undefined in general; it functions as a $(k+2)$-byte *nop* unless special `DVI`-reading programs are being used. TeX82 generates *xxx1* when a short enough `\special` appears, setting k to the number of bytes being sent. It is recommended that x be a string having the form of a keyword followed by possible parameters relevant to that keyword.

xxx2 240 $k[2]$ $x[k]$. Like *xxx1*, but $0 \le k < 65536$.

xxx3 241 $k[3]$ $x[k]$. Like *xxx1*, but $0 \le k < 2^{24}$.

xxx4 242 $k[4]$ $x[k]$. Like *xxx1*, but k can be ridiculously large. TeX82 uses *xxx4* when sending a string of length 256 or more.

fnt_def1 243 $k[1]$ $c[4]$ $s[4]$ $d[4]$ $a[1]$ $l[1]$ $n[a+l]$. Define font k, where $0 \le k < 256$; font definitions will be explained shortly.

fnt_def2 244 $k[2]$ $c[4]$ $s[4]$ $d[4]$ $a[1]$ $l[1]$ $n[a+l]$. Define font k, where $0 \le k < 65536$.

fnt_def3 245 $k[3]$ $c[4]$ $s[4]$ $d[4]$ $a[1]$ $l[1]$ $n[a+l]$. Define font k, where $0 \le k < 2^{24}$.

fnt_def4 246 $k[4]$ $c[4]$ $s[4]$ $d[4]$ $a[1]$ $l[1]$ $n[a+l]$. Define font k, where $-2^{31} \le k < 2^{31}$.

pre 247 $i[1]$ *num*$[4]$ *den*$[4]$ *mag*$[4]$ $k[1]$ $x[k]$. Beginning of the preamble; this must come at the very beginning of the file. Parameters i, *num*, *den*, *mag*, k, and x are explained below.

post 248. Beginning of the postamble, see below.

post_post 249. Ending of the postamble, see below.

Commands 250–255 are undefined in normal `DVI` files, but the following commands are used in `XDV` files.

define_native_font 252 $k[4]$ $s[4]$ *flags*$[2]$ $l[1]$ $n[l]$ $i[4]$
if (*flags* $\wedge$ *COLORED*) **then** *rgba*$[4]$
if (*flags* $\wedge$ *EXTEND*) **then** *extend*$[4]$
if (*flags* $\wedge$ *SLANT*) **then** *slant*$[4]$
if (*flags* $\wedge$ *EMBOLDEN*) **then** *embolden*$[4]$

set_glyphs 253 $w[4]$ $k[2]$ $xy[8k]$ $g[2k]$.

set_text_and_glyphs 254 $l[2]$ $t[2l]$ $w[4]$ $k[2]$ $xy[8k]$ $g[2k]$.

Commands 250 and 255 are undefined in normal `XDV` files.

622. **define** *set_char_0* = 0 { typeset character 0 and move right }
define *set1* = 128 { typeset a character and move right }
define *set_rule* = 132 { typeset a rule and move right }
define *put_rule* = 137 { typeset a rule }
define *nop* = 138 { no operation }
define *bop* = 139 { beginning of page }
define *eop* = 140 { ending of page }
define *push* = 141 { save the current positions }
define *pop* = 142 { restore previous positions }
define *right1* = 143 { move right }
define *w0* = 147 { move right by w }
define *w1* = 148 { move right and set w }
define *x0* = 152 { move right by x }
define *x1* = 153 { move right and set x }
define *down1* = 157 { move down }
define *y0* = 161 { move down by y }
define *y1* = 162 { move down and set y }
define *z0* = 166 { move down by z }
define *z1* = 167 { move down and set z }
define *fnt_num_0* = 171 { set current font to 0 }
define *fnt1* = 235 { set current font }
define *xxx1* = 239 { extension to DVI primitives }
define *xxx4* = 242 { potentially long extension to DVI primitives }
define *fnt_def1* = 243 { define the meaning of a font number }
define *pre* = 247 { preamble }
define *post* = 248 { postamble beginning }
define *post_post* = 249 { postamble ending }
define *define_native_font* = 252 { define native font }
define *set_glyphs* = 253 { sequence of glyphs with individual x-y coordinates }
define *set_text_and_glyphs* = 254 { run of Unicode (UTF16) text followed by positioned glyphs }

623. The preamble contains basic information about the file as a whole. As stated above, there are six parameters:

$$i[1]\ num[4]\ den[4]\ mag[4]\ k[1]\ x[k].$$

The i byte identifies DVI format; in XeTeX this byte is set to 7, as we have new DVI opcodes, while in TeX82 it is always set to 2. (The value $i = 3$ is used for an extended format that allows a mixture of right-to-left and left-to-right typesetting. Older versions of XeTeX used $i = 4$, $i = 5$ and $i = 6$.)

The next two parameters, *num* and *den*, are positive integers that define the units of measurement; they are the numerator and denominator of a fraction by which all dimensions in the DVI file could be multiplied in order to get lengths in units of 10^{-7} meters. Since 7227pt = 254cm, and since TeX works with scaled points where there are 2^{16} sp in a point, TeX sets $num/den = (254 \cdot 10^5)/(7227 \cdot 2^{16}) = 25400000/473628672$.

The *mag* parameter is what TeX calls \mag, i.e., 1000 times the desired magnification. The actual fraction by which dimensions are multiplied is therefore $mag \cdot num/1000den$. Note that if a TeX source document does not call for any '**true**' dimensions, and if you change it only by specifying a different \mag setting, the DVI file that TeX creates will be completely unchanged except for the value of *mag* in the preamble and postamble. (Fancy DVI-reading programs allow users to override the *mag* setting when a DVI file is being printed.)

Finally, k and x allow the DVI writer to include a comment, which is not interpreted further. The length of comment x is k, where $0 \le k < 256$.

define *id_byte* = 7 { identifies the kind of DVI files described here }

624. Font definitions for a given font number k contain further parameters

$$c[4]\ s[4]\ d[4]\ a[1]\ l[1]\ n[a+l].$$

The four-byte value c is the check sum that TeX found in the TFM file for this font; c should match the check sum of the font found by programs that read this DVI file.

Parameter s contains a fixed-point scale factor that is applied to the character widths in font k; font dimensions in TFM files and other font files are relative to this quantity, which is called the "at size" elsewhere in this documentation. The value of s is always positive and less than 2^{27}. It is given in the same units as the other DVI dimensions, i.e., in sp when TeX82 has made the file. Parameter d is similar to s; it is the "design size," and (like s) it is given in DVI units. Thus, font k is to be used at $mag \cdot s/1000d$ times its normal size.

The remaining part of a font definition gives the external name of the font, which is an ASCII string of length $a + l$. The number a is the length of the "area" or directory, and l is the length of the font name itself; the standard local system font area is supposed to be used when $a = 0$. The n field contains the area in its first a bytes.

Font definitions must appear before the first use of a particular font number. Once font k is defined, it must not be defined again; however, we shall see below that font definitions appear in the postamble as well as in the pages, so in this sense each font number is defined exactly twice, if at all. Like *nop* commands, font definitions can appear before the first *bop*, or between an *eop* and a *bop*.

625. Sometimes it is desirable to make horizontal or vertical rules line up precisely with certain features in characters of a font. It is possible to guarantee the correct matching between DVI output and the characters generated by METAFONT by adhering to the following principles: (1) The METAFONT characters should be positioned so that a bottom edge or left edge that is supposed to line up with the bottom or left edge of a rule appears at the reference point, i.e., in row 0 and column 0 of the METAFONT raster. This ensures that the position of the rule will not be rounded differently when the pixel size is not a perfect multiple of the units of measurement in the DVI file. (2) A typeset rule of height $a > 0$ and width $b > 0$ should be equivalent to a METAFONT-generated character having black pixels in precisely those raster positions whose METAFONT coordinates satisfy $0 \le x < \alpha b$ and $0 \le y < \alpha a$, where α is the number of pixels per DVI unit.

626. The last page in a DVI file is followed by '*post*'; this command introduces the postamble, which summarizes important facts that TeX has accumulated about the file, making it possible to print subsets of the data with reasonable efficiency. The postamble has the form

post $p[4]$ *num*$[4]$ *den*$[4]$ *mag*$[4]$ $l[4]$ $u[4]$ $s[2]$ $t[2]$
⟨ font definitions ⟩
post_post $q[4]$ $i[1]$ 223's$[\ge 4]$

Here p is a pointer to the final *bop* in the file. The next three parameters, *num*, *den*, and *mag*, are duplicates of the quantities that appeared in the preamble.

Parameters l and u give respectively the height-plus-depth of the tallest page and the width of the widest page, in the same units as other dimensions of the file. These numbers might be used by a DVI-reading program to position individual "pages" on large sheets of film or paper; however, the standard convention for output on normal size paper is to position each page so that the upper left-hand corner is exactly one inch from the left and the top. Experience has shown that it is unwise to design DVI-to-printer software that attempts cleverly to center the output; a fixed position of the upper left corner is easiest for users to understand and to work with. Therefore l and u are often ignored.

Parameter s is the maximum stack depth (i.e., the largest excess of *push* commands over *pop* commands) needed to process this file. Then comes t, the total number of pages (*bop* commands) present.

The postamble continues with font definitions, which are any number of *fnt_def* commands as described above, possibly interspersed with *nop* commands. Each font number that is used in the DVI file must be defined exactly twice: Once before it is first selected by a *fnt* command, and once in the postamble.

627. The last part of the postamble, following the *post_post* byte that signifies the end of the font definitions, contains q, a pointer to the *post* command that started the postamble. An identification byte, i, comes next; this currently equals 2, as in the preamble.

The i byte is followed by four or more bytes that are all equal to the decimal number 223 (i.e., *'337* in octal). TeX puts out four to seven of these trailing bytes, until the total length of the file is a multiple of four bytes, since this works out best on machines that pack four bytes per word; but any number of 223's is allowed, as long as there are at least four of them. In effect, 223 is a sort of signature that is added at the very end.

This curious way to finish off a `DVI` file makes it feasible for `DVI`-reading programs to find the postamble first, on most computers, even though TeX wants to write the postamble last. Most operating systems permit random access to individual words or bytes of a file, so the `DVI` reader can start at the end and skip backwards over the 223's until finding the identification byte. Then it can back up four bytes, read q, and move to byte q of the file. This byte should, of course, contain the value 248 (*post*); now the postamble can be read, so the `DVI` reader can discover all the information needed for typesetting the pages. Note that it is also possible to skip through the `DVI` file at reasonably high speed to locate a particular page, if that proves desirable. This saves a lot of time, since `DVI` files used in production jobs tend to be large.

Unfortunately, however, standard Pascal does not include the ability to access a random position in a file, or even to determine the length of a file. Almost all systems nowadays provide the necessary capabilities, so `DVI` format has been designed to work most efficiently with modern operating systems. But if `DVI` files have to be processed under the restrictions of standard Pascal, one can simply read them from front to back, since the necessary header information is present in the preamble and in the font definitions. (The l and u and s and t parameters, which appear only in the postamble, are "frills" that are handy but not absolutely necessary.)

628. Shipping pages out. After considering TeX's eyes and stomach, we come now to the bowels.

The *ship_out* procedure is given a pointer to a box; its mission is to describe that box in DVI form, outputting a "page" to *dvi_file*. The DVI coordinates $(h, v) = (0, 0)$ should correspond to the upper left corner of the box being shipped.

Since boxes can be inside of boxes inside of boxes, the main work of *ship_out* is done by two mutually recursive routines, *hlist_out* and *vlist_out*, which traverse the hlists and vlists inside of horizontal and vertical boxes.

As individual pages are being processed, we need to accumulate information about the entire set of pages, since such statistics must be reported in the postamble. The global variables *total_pages*, *max_v*, *max_h*, *max_push*, and *last_bop* are used to record this information.

The variable *doing_leaders* is *true* while leaders are being output. The variable *dead_cycles* contains the number of times an output routine has been initiated since the last *ship_out*.

A few additional global variables are also defined here for use in *vlist_out* and *hlist_out*. They could have been local variables, but that would waste stack space when boxes are deeply nested, since the values of these variables are not needed during recursive calls.

⟨ Global variables 13 ⟩ +≡
total_pages: *integer*; { the number of pages that have been shipped out }
max_v: *scaled*; { maximum height-plus-depth of pages shipped so far }
max_h: *scaled*; { maximum width of pages shipped so far }
max_push: *integer*; { deepest nesting of *push* commands encountered so far }
last_bop: *integer*; { location of previous *bop* in the DVI output }
dead_cycles: *integer*; { recent outputs that didn't ship anything out }
doing_leaders: *boolean*; { are we inside a leader box? }
c, *f*: *quarterword*; { character and font in current *char_node* }
rule_ht, *rule_dp*, *rule_wd*: *scaled*; { size of current rule being output }
g: *pointer*; { current glue specification }
lq, *lr*: *integer*; { quantities used in calculations for leaders }

629. ⟨ Set initial values of key variables 23 ⟩ +≡
total_pages ← 0; *max_v* ← 0; *max_h* ← 0; *max_push* ← 0; *last_bop* ← −1; *doing_leaders* ← *false*;
dead_cycles ← 0; *cur_s* ← −1;

630. The DVI bytes are output to a buffer instead of being written directly to the output file. This makes it possible to reduce the overhead of subroutine calls, thereby measurably speeding up the computation, since output of DVI bytes is part of TeX's inner loop. And it has another advantage as well, since we can change instructions in the buffer in order to make the output more compact. For example, a '*down2*' command can be changed to a '*y2*', thereby making a subsequent '*y0*' command possible, saving two bytes.

The output buffer is divided into two parts of equal size; the bytes found in *dvi_buf*[0 .. *half_buf* − 1] constitute the first half, and those in *dvi_buf*[*half_buf* .. *dvi_buf_size* − 1] constitute the second. The global variable *dvi_ptr* points to the position that will receive the next output byte. When *dvi_ptr* reaches *dvi_limit*, which is always equal to one of the two values *half_buf* or *dvi_buf_size*, the half buffer that is about to be invaded next is sent to the output and *dvi_limit* is changed to its other value. Thus, there is always at least a half buffer's worth of information present, except at the very beginning of the job.

Bytes of the DVI file are numbered sequentially starting with 0; the next byte to be generated will be number *dvi_offset* + *dvi_ptr*. A byte is present in the buffer only if its number is ≥ *dvi_gone*.

⟨ Types in the outer block 18 ⟩ +≡
dvi_index = 0 .. *dvi_buf_size*; { an index into the output buffer }

631. Some systems may find it more efficient to make *dvi_buf* a **packed** array, since output of four bytes at once may be facilitated.

```
⟨ Global variables 13 ⟩ +≡
dvi_buf: array [dvi_index] of eight_bits;  { buffer for DVI output }
half_buf: dvi_index;  { half of dvi_buf_size }
dvi_limit: dvi_index;  { end of the current half buffer }
dvi_ptr: dvi_index;  { the next available buffer address }
dvi_offset: integer;  { dvi_buf_size times the number of times the output buffer has been fully emptied }
dvi_gone: integer;  { the number of bytes already output to dvi_file }
```

632. Initially the buffer is all in one piece; we will output half of it only after it first fills up.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  half_buf ← dvi_buf_size div 2; dvi_limit ← dvi_buf_size; dvi_ptr ← 0; dvi_offset ← 0; dvi_gone ← 0;
```

633. The actual output of *dvi_buf*[*a* . . *b*] to *dvi_file* is performed by calling *write_dvi*(*a*, *b*). For best results, this procedure should be optimized to run as fast as possible on each particular system, since it is part of TeX's inner loop. It is safe to assume that *a* and *b* + 1 will both be multiples of 4 when *write_dvi*(*a*, *b*) is called; therefore it is possible on many machines to use efficient methods to pack four bytes per word and to output an array of words with one system call.

```
procedure write_dvi(a, b : dvi_index);
  var k: dvi_index;
  begin for k ← a to b do write(dvi_file, dvi_buf[k]);
  end;
```

634. To put a byte in the buffer without paying the cost of invoking a procedure each time, we use the macro *dvi_out*.

```
  define dvi_out(#) ≡ begin dvi_buf[dvi_ptr] ← #; incr(dvi_ptr);
        if dvi_ptr = dvi_limit then dvi_swap;
        end
procedure dvi_swap;  { outputs half of the buffer }
  begin if dvi_limit = dvi_buf_size then
    begin write_dvi(0, half_buf − 1); dvi_limit ← half_buf; dvi_offset ← dvi_offset + dvi_buf_size;
    dvi_ptr ← 0;
    end
  else begin write_dvi(half_buf, dvi_buf_size − 1); dvi_limit ← dvi_buf_size;
    end;
  dvi_gone ← dvi_gone + half_buf;
  end;
```

635. Here is how we clean out the buffer when TeX is all through; *dvi_ptr* will be a multiple of 4.

```
⟨ Empty the last bytes out of dvi_buf 635 ⟩ ≡
  if dvi_limit = half_buf then write_dvi(half_buf, dvi_buf_size − 1);
  if dvi_ptr > 0 then write_dvi(0, dvi_ptr − 1)
```

This code is used in section 680.

636. The *dvi_four* procedure outputs four bytes in two's complement notation, without risking arithmetic overflow.

```
procedure dvi_four(x : integer);
  begin if x ≥ 0 then dvi_out(x div ´100000000)
  else begin x ← x + ´10000000000; x ← x + ´10000000000; dvi_out((x div ´100000000) + 128);
    end;
  x ← x mod ´100000000; dvi_out(x div ´200000); x ← x mod ´200000; dvi_out(x div ´400);
  dvi_out(x mod ´400);
  end;
procedure dvi_two(s : UTF16_code);
  begin dvi_out(s div ´400); dvi_out(s mod ´400);
  end;
```

637. A mild optimization of the output is performed by the *dvi_pop* routine, which issues a *pop* unless it is possible to cancel a '*push pop*' pair. The parameter to *dvi_pop* is the byte address following the old *push* that matches the new *pop*.

```
procedure dvi_pop(l : integer);
  begin if (l = dvi_offset + dvi_ptr) ∧ (dvi_ptr > 0) then decr(dvi_ptr)
  else dvi_out(pop);
  end;
```

638. Here's a procedure that outputs a font definition. Since TeX82 uses at most 256 different fonts per job, *fnt_def1* is always used as the command code.

```
procedure dvi_native_font_def(f : internal_font_number);
  var font_def_length, i: integer;
  begin dvi_out(define_native_font); dvi_four(f − font_base − 1); font_def_length ← make_font_def(f);
  for i ← 0 to font_def_length − 1 do dvi_out(xdv_buffer[i]);
  end;
procedure dvi_font_def(f : internal_font_number);
  var k: pool_pointer; { index into str_pool }
    l: integer; { length of name without mapping option }
    begin if is_native_font(f) then dvi_native_font_def(f)
    else begin dvi_out(fnt_def1); dvi_out(f − font_base − 1);
      dvi_out(qo(font_check[f].b0)); dvi_out(qo(font_check[f].b1)); dvi_out(qo(font_check[f].b2));
      dvi_out(qo(font_check[f].b3));
      dvi_four(font_size[f]); dvi_four(font_dsize[f]);
      dvi_out(length(font_area[f])); ⟨ Output the font name whose internal number is f 639 ⟩;
      end;
```

639. ⟨ Output the font name whose internal number is f 639 ⟩ ≡

```
l ← 0; k ← str_start_macro(font_name[f]); { search for colon; we will truncate the name there }
while (l = 0) ∧ (k < str_start_macro(font_name[f] + 1)) do
  begin if so(str_pool[k]) = ":" then l ← k − str_start_macro(font_name[f]);
  incr(k);
  end;
if l = 0 then l ← length(font_name[f]); { no colon found }
dvi_out(l);
for k ← str_start_macro(font_area[f]) to str_start_macro(font_area[f] + 1) − 1 do
  dvi_out(so(str_pool[k]));
for k ← str_start_macro(font_name[f]) to str_start_macro(font_name[f]) + l − 1 do
  dvi_out(so(str_pool[k]));
end ;
```

This code is used in section 638.

640. Versions of TeX intended for small computers might well choose to omit the ideas in the next few parts of this program, since it is not really necessary to optimize the DVI code by making use of the *w0*, *x0*, *y0*, and *z0* commands. Furthermore, the algorithm that we are about to describe does not pretend to give an optimum reduction in the length of the DVI code; after all, speed is more important than compactness. But the method is surprisingly effective, and it takes comparatively little time.

We can best understand the basic idea by first considering a simpler problem that has the same essential characteristics. Given a sequence of digits, say 3 1 4 1 5 9 2 6 5 3 5 8 9, we want to assign subscripts d, y, or z to each digit so as to maximize the number of "y-hits" and "z-hits"; a y-hit is an instance of two appearances of the same digit with the subscript y, where no y's intervene between the two appearances, and a z-hit is defined similarly. For example, the sequence above could be decorated with subscripts as follows:

$$3_z\,1_y\,4_d\,1_y\,5_y\,9_d\,2_d\,6_d\,5_y\,3_z\,5_y\,8_d\,9_d.$$

There are three y-hits ($1_y \ldots 1_y$ and $5_y \ldots 5_y \ldots 5_y$) and one z-hit ($3_z \ldots 3_z$); there are no d-hits, since the two appearances of 9_d have d's between them, but we don't count d-hits so it doesn't matter how many there are. These subscripts are analogous to the DVI commands called *down*, y, and z, and the digits are analogous to different amounts of vertical motion; a y-hit or z-hit corresponds to the opportunity to use the one-byte commands *y0* or *z0* in a DVI file.

TeX's method of assigning subscripts works like this: Append a new digit, say δ, to the right of the sequence. Now look back through the sequence until one of the following things happens: (a) You see δ_y or δ_z, and this was the first time you encountered a y or z subscript, respectively. Then assign y or z to the new δ; you have scored a hit. (b) You see δ_d, and no y subscripts have been encountered so far during this search. Then change the previous δ_d to δ_y (this corresponds to changing a command in the output buffer), and assign y to the new δ; it's another hit. (c) You see δ_d, and a y subscript has been seen but not a z. Change the previous δ_d to δ_z and assign z to the new δ. (d) You encounter both y and z subscripts before encountering a suitable δ, or you scan all the way to the front of the sequence. Assign d to the new δ; this assignment may be changed later.

The subscripts $3_z\,1_y\,4_d \ldots$ in the example above were, in fact, produced by this procedure, as the reader can verify. (Go ahead and try it.)

641. In order to implement such an idea, TeX maintains a stack of pointers to the *down*, *y*, and *z* commands that have been generated for the current page. And there is a similar stack for *right*, *w*, and *x* commands. These stacks are called the down stack and right stack, and their top elements are maintained in the variables *down_ptr* and *right_ptr*.

Each entry in these stacks contains four fields: The *width* field is the amount of motion down or to the right; the *location* field is the byte number of the DVI command in question (including the appropriate *dvi_offset*); the *link* field points to the next item below this one on the stack; and the *info* field encodes the options for possible change in the DVI command.

```
define movement_node_size = 3  { number of words per entry in the down and right stacks }
define location(#) ≡ mem[# + 2].int  { DVI byte number for a movement command }
⟨ Global variables 13 ⟩ +≡
down_ptr, right_ptr: pointer;  { heads of the down and right stacks }
```

642.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  down_ptr ← null; right_ptr ← null;
```

643. Here is a subroutine that produces a DVI command for some specified downward or rightward motion. It has two parameters: *w* is the amount of motion, and *o* is either *down1* or *right1*. We use the fact that the command codes have convenient arithmetic properties: *y1* − *down1* = *w1* − *right1* and *z1* − *down1* = *x1* − *right1*.

```
procedure movement(w : scaled; o : eight_bits);
  label exit, found, not_found, 2, 1;
  var mstate: small_number;  { have we seen a y or z? }
    p, q: pointer;  { current and top nodes on the stack }
    k: integer;  { index into dvi_buf, modulo dvi_buf_size }
  begin q ← get_node(movement_node_size);  { new node for the top of the stack }
  width(q) ← w; location(q) ← dvi_offset + dvi_ptr;
  if o = down1 then
    begin link(q) ← down_ptr; down_ptr ← q;
    end
  else begin link(q) ← right_ptr; right_ptr ← q;
    end;
  ⟨ Look at the other stack entries until deciding what sort of DVI command to generate; goto found if
      node p is a "hit" 647 ⟩;
  ⟨ Generate a down or right command for w and return 646 ⟩;
found: ⟨ Generate a y0 or z0 command in order to reuse a previous appearance of w 645 ⟩;
exit: end;
```

644. The *info* fields in the entries of the down stack or the right stack have six possible settings: *y_here* or *z_here* mean that the DVI command refers to y or z, respectively (or to w or x, in the case of horizontal motion); *yz_OK* means that the DVI command is *down* (or *right*) but can be changed to either y or z (or to either w or x); *y_OK* means that it is *down* and can be changed to y but not z; *z_OK* is similar; and *d_fixed* means it must stay *down*.

The four settings *yz_OK*, *y_OK*, *z_OK*, *d_fixed* would not need to be distinguished from each other if we were simply solving the digit-subscripting problem mentioned above. But in TeX's case there is a complication because of the nested structure of *push* and *pop* commands. Suppose we add parentheses to the digit-subscripting problem, redefining hits so that $\delta_y \ldots \delta_y$ is a hit if all y's between the δ's are enclosed in properly nested parentheses, and if the parenthesis level of the right-hand δ_y is deeper than or equal to that of the left-hand one. Thus, '(' and ')' correspond to '*push*' and '*pop*'. Now if we want to assign a subscript to the final 1 in the sequence

$$2_y\, 7_d\, 1_d\, (\, 8_z\, 2_y\, 8_z\,)\, 1$$

we cannot change the previous 1_d to 1_y, since that would invalidate the $2_y \ldots 2_y$ hit. But we can change it to 1_z, scoring a hit since the intervening 8_z's are enclosed in parentheses.

The program below removes movement nodes that are introduced after a *push*, before it outputs the corresponding *pop*.

```
define y_here = 1   { info when the movement entry points to a y command }
define z_here = 2   { info when the movement entry points to a z command }
define yz_OK = 3    { info corresponding to an unconstrained down command }
define y_OK = 4     { info corresponding to a down that can't become a z }
define z_OK = 5     { info corresponding to a down that can't become a y }
define d_fixed = 6  { info corresponding to a down that can't change }
```

645. When the *movement* procedure gets to the label *found*, the value of *info*(*p*) will be either *y_here* or *z_here*. If it is, say, *y_here*, the procedure generates a *y0* command (or a *w0* command), and marks all *info* fields between q and p so that y is not OK in that range.

```
⟨ Generate a y0 or z0 command in order to reuse a previous appearance of w 645 ⟩ ≡
  info(q) ← info(p);
  if info(q) = y_here then
    begin dvi_out(o + y0 − down1); { y0 or w0 }
    while link(q) ≠ p do
      begin q ← link(q);
      case info(q) of
      yz_OK: info(q) ← z_OK;
      y_OK: info(q) ← d_fixed;
      othercases do_nothing
      endcases;
      end;
    end
  else begin dvi_out(o + z0 − down1); { z0 or x0 }
    while link(q) ≠ p do
      begin q ← link(q);
      case info(q) of
      yz_OK: info(q) ← y_OK;
      z_OK: info(q) ← d_fixed;
      othercases do_nothing
      endcases;
      end;
    end
```

This code is used in section 643.

646. ⟨ Generate a *down* or *right* command for w and **return** 646 ⟩ ≡

```
info(q) ← yz_OK;
if abs(w) ≥ ´40000000 then
  begin dvi_out(o + 3);  { down4 or right4 }
  dvi_four(w); return;
  end;
if abs(w) ≥ ´100000 then
  begin dvi_out(o + 2);  { down3 or right3 }
  if w < 0 then w ← w + ´100000000;
  dvi_out(w div ´200000); w ← w mod ´200000; goto 2;
  end;
if abs(w) ≥ ´200 then
  begin dvi_out(o + 1);  { down2 or right2 }
  if w < 0 then w ← w + ´200000;
  goto 2;
  end;
dvi_out(o);  { down1 or right1 }
if w < 0 then w ← w + ´400;
goto 1;
2: dvi_out(w div ´400);
1: dvi_out(w mod ´400); return
```

This code is used in section 643.

647. As we search through the stack, we are in one of three states, *y_seen*, *z_seen*, or *none_seen*, depending on whether we have encountered *y_here* or *z_here* nodes. These states are encoded as multiples of 6, so that they can be added to the *info* fields for quick decision-making.

```
define none_seen = 0  { no y_here or z_here nodes have been encountered yet }
define y_seen = 6  { we have seen y_here but not z_here }
define z_seen = 12  { we have seen z_here but not y_here }
```

⟨ Look at the other stack entries until deciding what sort of DVI command to generate; **goto** *found* if node p is a "hit" 647 ⟩ ≡

```
p ← link(q); mstate ← none_seen;
while p ≠ null do
  begin if width(p) = w then ⟨ Consider a node with matching width; goto found if it's a hit 648 ⟩
  else case mstate + info(p) of
    none_seen + y_here: mstate ← y_seen;
    none_seen + z_here: mstate ← z_seen;
    y_seen + z_here, z_seen + y_here: goto not_found;
    othercases do_nothing
    endcases;
  p ← link(p);
  end;
not_found:
```

This code is used in section 643.

648. We might find a valid hit in a y or z byte that is already gone from the buffer. But we can't change bytes that are gone forever; "the moving finger writes, ..."

```
⟨ Consider a node with matching width; goto found if it's a hit 648 ⟩ ≡
  case mstate + info(p) of
  none_seen + yz_OK, none_seen + y_OK, z_seen + yz_OK, z_seen + y_OK:
    if location(p) < dvi_gone then goto not_found
    else ⟨ Change buffered instruction to y or w and goto found 649 ⟩;
  none_seen + z_OK, y_seen + yz_OK, y_seen + z_OK:
    if location(p) < dvi_gone then goto not_found
    else ⟨ Change buffered instruction to z or x and goto found 650 ⟩;
  none_seen + y_here, none_seen + z_here, y_seen + z_here, z_seen + y_here: goto found;
  othercases do_nothing
  endcases
```

This code is used in section 647.

649.
```
⟨ Change buffered instruction to y or w and goto found 649 ⟩ ≡
  begin k ← location(p) − dvi_offset;
  if k < 0 then k ← k + dvi_buf_size;
  dvi_buf[k] ← dvi_buf[k] + y1 − down1; info(p) ← y_here; goto found;
  end
```

This code is used in section 648.

650.
```
⟨ Change buffered instruction to z or x and goto found 650 ⟩ ≡
  begin k ← location(p) − dvi_offset;
  if k < 0 then k ← k + dvi_buf_size;
  dvi_buf[k] ← dvi_buf[k] + z1 − down1; info(p) ← z_here; goto found;
  end
```

This code is used in section 648.

651. In case you are wondering when all the movement nodes are removed from TeX's memory, the answer is that they are recycled just before *hlist_out* and *vlist_out* finish outputting a box. This restores the down and right stacks to the state they were in before the box was output, except that some *info*'s may have become more restrictive.

```
procedure prune_movements(l : integer); { delete movement nodes with location ≥ l }
  label done, exit;
  var p: pointer; { node being deleted }
  begin while down_ptr ≠ null do
    begin if location(down_ptr) < l then goto done;
    p ← down_ptr; down_ptr ← link(p); free_node(p, movement_node_size);
    end;
done: while right_ptr ≠ null do
    begin if location(right_ptr) < l then return;
    p ← right_ptr; right_ptr ← link(p); free_node(p, movement_node_size);
    end;
exit: end;
```

652. The actual distances by which we want to move might be computed as the sum of several separate movements. For example, there might be several glue nodes in succession, or we might want to move right by the width of some box plus some amount of glue. More importantly, the baselineskip distances are computed in terms of glue together with the depth and height of adjacent boxes, and we want the DVI file to lump these three quantities together into a single motion.

Therefore, TeX maintains two pairs of global variables: *dvi_h* and *dvi_v* are the h and v coordinates corresponding to the commands actually output to the DVI file, while *cur_h* and *cur_v* are the coordinates corresponding to the current state of the output routines. Coordinate changes will accumulate in *cur_h* and *cur_v* without being reflected in the output, until such a change becomes necessary or desirable; we can call the *movement* procedure whenever we want to make *dvi_h* = *cur_h* or *dvi_v* = *cur_v*.

The current font reflected in the DVI output is called *dvi_f*; there is no need for a '*cur_f*' variable.

The depth of nesting of *hlist_out* and *vlist_out* is called *cur_s*; this is essentially the depth of *push* commands in the DVI output.

For mixed direction text (TeX--XeT) the current text direction is called *cur_dir*. As the box being shipped out will never be used again and soon be recycled, we can simply reverse any R-text (i.e., right-to-left) segments of hlist nodes as well as complete hlist nodes embedded in such segments. Moreover this can be done iteratively rather than recursively. There are, however, two complications related to leaders that require some additional bookkeeping: (1) One and the same hlist node might be used more than once (but never inside both L- and R-text); and (2) leader boxes inside hlists must be aligned with respect to the left edge of the original hlist.

A math node is changed into a kern node whenever the text direction remains the same, it is replaced by an *edge_node* if the text direction changes; the subtype of an an *hlist_node* inside R-text is changed to *reversed* once its hlist has been reversed.

```
define reversed = 1  { subtype for an hlist_node whose hlist has been reversed }
define dlist = 2  { subtype for an hlist_node from display math mode }
define box_lr(#) ≡ (qo(subtype(#)))  { direction mode of a box }
define set_box_lr(#) ≡ subtype(#) ← set_box_lr_end
define set_box_lr_end(#) ≡ qi(#)

define left_to_right = 0
define right_to_left = 1
define reflected ≡ 1 − cur_dir  { the opposite of cur_dir }

define synch_h ≡
        if cur_h ≠ dvi_h then
          begin movement(cur_h − dvi_h, right1); dvi_h ← cur_h;
          end
define synch_v ≡
        if cur_v ≠ dvi_v then
          begin movement(cur_v − dvi_v, down1); dvi_v ← cur_v;
          end
```

⟨ Global variables 13 ⟩ +≡

```
dvi_h, dvi_v: scaled;  { a DVI reader program thinks we are here }
cur_h, cur_v: scaled;  { TeX thinks we are here }
dvi_f: internal_font_number;  { the current font }
cur_s: integer;  { current depth of output box nesting, initially −1 }
```

653. ⟨ Initialize variables as *ship_out* begins 653 ⟩ ≡
```
dvi_h ← 0; dvi_v ← 0; cur_h ← h_offset; dvi_f ← null_font;
⟨ Calculate page dimensions and margins 1428 ⟩;
ensure_dvi_open;
if total_pages = 0 then
  begin dvi_out(pre); dvi_out(id_byte); { output the preamble }
  dvi_four(25400000); dvi_four(473628672); { conversion ratio for sp }
  prepare_mag; dvi_four(mag); { magnification factor is frozen }
  old_setting ← selector; selector ← new_string; print("␣XeTeX␣output␣"); print_int(year);
  print_char("."); print_two(month); print_char("."); print_two(day); print_char(":");
  print_two(time div 60); print_two(time mod 60); selector ← old_setting; dvi_out(cur_length);
  for s ← str_start_macro(str_ptr) to pool_ptr − 1 do dvi_out(so(str_pool[s]));
  pool_ptr ← str_start_macro(str_ptr); { flush the current string }
  end
```
This code is used in section 678.

654. When *hlist_out* is called, its duty is to output the box represented by the *hlist_node* pointed to by *temp_ptr*. The reference point of that box has coordinates (*cur_h*, *cur_v*).

Similarly, when *vlist_out* is called, its duty is to output the box represented by the *vlist_node* pointed to by *temp_ptr*. The reference point of that box has coordinates (*cur_h*, *cur_v*).

```
procedure vlist_out; forward; { hlist_out and vlist_out are mutually recursive }
```

655. The recursive procedures *hlist_out* and *vlist_out* each have local variables *save_h* and *save_v* to hold the values of *dvi_h* and *dvi_v* just before entering a new level of recursion. In effect, the values of *save_h* and *save_v* on TeX's run-time stack correspond to the values of h and v that a DVI-reading program will push onto its coordinate stack.

```
define move_past = 13 { go to this label when advancing past glue or a rule }
define fin_rule = 14 { go to this label to finish processing a rule }
define next_p = 15 { go to this label when finished with node p }
define check_next = 1236
define end_node_run = 1237
⟨ Declare procedures needed in hlist_out, vlist_out 1431 ⟩
procedure hlist_out; { output an hlist_node box }
  label reswitch, move_past, fin_rule, next_p;
  var base_line: scaled; { the baseline coordinate for this box }
    left_edge: scaled; { the left coordinate for this box }
    save_h, save_v: scaled; { what dvi_h and dvi_v should pop to }
    this_box: pointer; { pointer to containing box }
    g_order: glue_ord; { applicable order of infinity for glue }
    g_sign: normal .. shrinking; { selects type of glue }
    p: pointer; { current position in the hlist }
    save_loc: integer; { DVI byte location upon entry }
    leader_box: pointer; { the leader box being replicated }
    leader_wd: scaled; { width of leader box being replicated }
    lx: scaled; { extra space between leader boxes }
    outer_doing_leaders: boolean; { were we doing leaders? }
    edge: scaled; { right edge of sub-box or leader space }
    prev_p: pointer; { one step behind p }
    len: integer; { length of scratch string for native word output }
    q, r: pointer; k, j: integer; glue_temp: real; { glue value before rounding }
    cur_glue: real; { glue seen so far }
    cur_g: scaled; { rounded equivalent of cur_glue times the glue ratio }
  begin cur_g ← 0; cur_glue ← float_constant(0); this_box ← temp_ptr; g_order ← glue_order(this_box);
  g_sign ← glue_sign(this_box);
  if XeTeX_interword_space_shaping_state > 1 then
    begin ⟨ Merge sequences of words using native fonts and inter-word spaces into single nodes 656 ⟩;
    end;
  p ← list_ptr(this_box); incr(cur_s);
  if cur_s > 0 then dvi_out(push);
  if cur_s > max_push then max_push ← cur_s;
  save_loc ← dvi_offset + dvi_ptr; base_line ← cur_v; prev_p ← this_box + list_offset;
  ⟨ Initialize hlist_out for mixed direction typesetting 1524 ⟩;
  left_edge ← cur_h;
  while p ≠ null do ⟨ Output node p for hlist_out and move to the next node, maintaining the condition
      cur_v = base_line 658 ⟩;
  ⟨ Finish hlist_out for mixed direction typesetting 1525 ⟩;
  prune_movements(save_loc);
  if cur_s > 0 then dvi_pop(save_loc);
  decr(cur_s);
  end;
```

656. Extra stuff for justifiable AAT text; need to merge runs of words and normal spaces.

```
define is_native_word_node(#) ≡ (((#) ≠ null) ∧ (¬is_char_node(#)) ∧ (type(#) =
        whatsit_node) ∧ (is_native_word_subtype(#)))
define is_glyph_node(#) ≡ (((#) ≠ null ∧ (¬is_char_node(#)) ∧ (type(#) = whatsit_node) ∧ (subtype(#) =
        glyph_node)))
define
        node_is_invisible_to_interword_space(#) ≡ ¬is_char_node(#) ∧ ((type(#) =
        penalty_node) ∨ (type(#) = ins_node) ∨ (type(#) = mark_node) ∨ (type(#) =
        adjust_node) ∨ ((type(#) = whatsit_node) ∧ (subtype(#) ≤ 4)))
        { This checks for subtypes in the range open/write/close/special/language, but the definitions
        haven't appeared yet in the .web file so we cheat. }
⟨ Merge sequences of words using native fonts and inter-word spaces into single nodes 656 ⟩ ≡
  p ← list_ptr(this_box); prev_p ← this_box + list_offset;
  while p ≠ null do
    begin if link(p) ≠ null then
      begin { not worth looking ahead at the end }
      if is_native_word_node(p) ∧ (font_letter_space[native_font(p)] = 0) then
        begin { got a word in an AAT font, might be the start of a run }
        r ← p; { r is start of possible run }
        k ← native_length(r); q ← link(p);
      check_next: ⟨ Advance q past ignorable nodes 657 ⟩;
        if (q ≠ null) ∧ ¬is_char_node(q) then
          begin if (type(q) = glue_node) ∧ (subtype(q) = normal) then
            begin if (glue_ptr(q) = font_glue[native_font(r)]) then
              begin { found a normal space; if the next node is another word in the same font, we'll
                  merge }
              q ← link(q); ⟨ Advance q past ignorable nodes 657 ⟩;
              if is_native_word_node(q) ∧ (native_font(q) = native_font(r)) then
                begin p ← q; { record new tail of run in p }
                k ← k + 1 + native_length(q); q ← link(q); goto check_next;
                end
              end
            else q ← link(q); { we'll also merge if if space-adjustment was applied at this glue, even if it
                  wasn't the font's standard inter-word space }
            if (q ≠ null) ∧ ¬is_char_node(q) ∧ (type(q) = kern_node) ∧ (subtype(q) = space_adjustment)
                  then
              begin q ← link(q); ⟨ Advance q past ignorable nodes 657 ⟩;
              if is_native_word_node(q) ∧ (native_font(q) = native_font(r)) then
                begin p ← q; { record new tail of run in p }
                k ← k + 1 + native_length(q); q ← link(q); goto check_next;
                end
              end;
            goto end_node_run;
            end;
          if is_native_word_node(q) ∧ (native_font(q) = native_font(r)) then
            begin p ← q; { record new tail of run in p }
            q ← link(q); goto check_next;
            end
          end;
      end_node_run: { now r points to first native_word_node of the run, and p to the last }
        if p ≠ r then
          begin { merge nodes from r to p inclusive; total text length is k }
```

```
    str_room(k); k ← 0; { now we'll use this as accumulator for total width }
    q ← r;
    loop
      begin if type(q) = whatsit_node then
        begin if (is_native_word_subtype(q)) then
          begin for j ← 0 to native_length(q) − 1 do append_char(get_native_char(q, j));
          k ← k + width(q);
          end
        end
      else if type(q) = glue_node then
          begin append_char("␣"); g ← glue_ptr(q); k ← k + width(g);
          if g_sign ≠ normal then
            begin if g_sign = stretching then
              begin if stretch_order(g) = g_order then
                begin k ← k + round(float(glue_set(this_box)) ∗ stretch(g))
                end
              end
            else begin if shrink_order(g) = g_order then
                begin k ← k − round(float(glue_set(this_box)) ∗ shrink(g))
                end
              end
            end
          end
        else if type(q) = kern_node then
            begin k ← k + width(q);
            end; { discretionary and deleted nodes can be discarded here }
      if q = p then break
      else q ← link(q);
      end; { create the new merged node q }
    q ← new_native_word_node(native_font(r), cur_length); subtype(q) ← subtype(r);
    for j ← 0 to cur_length − 1 do set_native_char(q, j, str_pool[str_start_macro(str_ptr) + j]);
        { impose the required width on q, and shape its text accordingly }
    width(q) ← k; set_justified_native_glyphs(q); { link q into the list in place of r..p }
    link(prev_p) ← q; link(q) ← link(p); link(p) ← null; { Extract any "invisible" nodes from the
        old list and insert them after the new node, so we don't lose them altogether. Note that the
        first node cannot be one of these, as we always start merging at a native_word node. }
    prev_p ← r; p ← link(r);
    while p ≠ null do
      begin if node_is_invisible_to_interword_space(p) then
        begin link(prev_p) ← link(p); link(p) ← link(q); link(q) ← p; q ← p;
        end;
      prev_p ← p; p ← link(p);
      end; { discard the remains of the old list }
    flush_node_list(r); { clean up and prepare for the next round }
    pool_ptr ← str_start_macro(str_ptr); { flush the temporary string data }
    p ← q;
    end
  end;
 prev_p ← p;
 end;
p ← link(p);
end
```

This code is used in section 655.

657. ⟨ Advance q past ignorable nodes 657 ⟩ ≡

```
while (q ≠ null) ∧ node_is_invisible_to_interword_space(q) do q ← link(q)
```

This code is used in sections 656, 656, and 656.

658. We ought to give special care to the efficiency of one part of *hlist_out*, since it belongs to TeX's inner loop. When a *char_node* is encountered, we save a little time by processing several nodes in succession until reaching a non-*char_node*. The program uses the fact that *set_char_0* = 0.

⟨ Output node p for *hlist_out* and move to the next node, maintaining the condition *cur_v* = *base_line* 658 ⟩ ≡

```
reswitch: if is_char_node(p) then
    begin synch_h; synch_v;
    repeat f ← font(p); c ← character(p);
      if (p ≠ lig_trick) ∧ (font_mapping[f] ≠ nil) then c ← apply_tfm_font_mapping(font_mapping[f], c);
      if f ≠ dvi_f then ⟨ Change font dvi_f to f 659 ⟩;
      if c ≥ qi(128) then dvi_out(set1);
      dvi_out(qo(c));
      cur_h ← cur_h + char_width(f)(char_info(f)(c)); prev_p ← link(prev_p);
          { N.B.: not prev_p ← p, p might be lig_trick }
      p ← link(p);
    until ¬is_char_node(p);
    dvi_h ← cur_h;
    end
  else ⟨ Output the non-char_node p for hlist_out and move to the next node 660 ⟩
```

This code is used in section 655.

659. ⟨ Change font *dvi_f* to f 659 ⟩ ≡

```
begin if ¬font_used[f] then
    begin dvi_font_def(f); font_used[f] ← true;
    end;
  if f ≤ 64 + font_base then dvi_out(f − font_base − 1 + fnt_num_0)
  else begin dvi_out(fnt1); dvi_out(f − font_base − 1);
    end;
  dvi_f ← f;
  end
```

This code is used in sections 658, 1426, and 1430.

660. ⟨ Output the non-*char_node* p for *hlist_out* and move to the next node 660 ⟩ ≡

```
begin case type(p) of
hlist_node, vlist_node: ⟨ Output a box in an hlist 661 ⟩;
rule_node: begin rule_ht ← height(p); rule_dp ← depth(p); rule_wd ← width(p); goto fin_rule;
  end;
whatsit_node: ⟨ Output the whatsit node p in an hlist 1430 ⟩;
glue_node: ⟨ Move right or output leaders 663 ⟩;
margin_kern_node: begin cur_h ← cur_h + width(p);
  end;
kern_node: cur_h ← cur_h + width(p);
math_node: ⟨ Handle a math node in hlist_out 1526 ⟩;
ligature_node: ⟨ Make node p look like a char_node and goto reswitch 692 ⟩;
  ⟨ Cases of hlist_out that arise in mixed direction text only 1530 ⟩
othercases do_nothing
endcases;
goto next_p;
fin_rule: ⟨ Output a rule in an hlist 662 ⟩;
move_past: cur_h ← cur_h + rule_wd;
next_p: prev_p ← p; p ← link(p);
end
```

This code is used in section 658.

661. ⟨ Output a box in an hlist 661 ⟩ ≡

```
if list_ptr(p) = null then cur_h ← cur_h + width(p)
else begin save_h ← dvi_h; save_v ← dvi_v; cur_v ← base_line + shift_amount(p);
      { shift the box down }
  temp_ptr ← p; edge ← cur_h + width(p);
  if cur_dir = right_to_left then cur_h ← edge;
  if type(p) = vlist_node then vlist_out else hlist_out;
  dvi_h ← save_h; dvi_v ← save_v; cur_h ← edge; cur_v ← base_line;
  end
```

This code is used in section 660.

662. ⟨ Output a rule in an hlist 662 ⟩ ≡

```
if is_running(rule_ht) then rule_ht ← height(this_box);
if is_running(rule_dp) then rule_dp ← depth(this_box);
rule_ht ← rule_ht + rule_dp;  { this is the rule thickness }
if (rule_ht > 0) ∧ (rule_wd > 0) then  { we don't output empty rules }
  begin synch_h; cur_v ← base_line + rule_dp; synch_v; dvi_out(set_rule); dvi_four(rule_ht);
  dvi_four(rule_wd); cur_v ← base_line; dvi_h ← dvi_h + rule_wd;
  end
```

This code is used in section 660.

663. **define** *billion* ≡ *float_constant*(1000000000)
define *vet_glue*(#) ≡ *glue_temp* ← #;
if *glue_temp* > *billion* **then** *glue_temp* ← *billion*
else if *glue_temp* < −*billion* **then** *glue_temp* ← −*billion*
define *round_glue* ≡ *g* ← *glue_ptr*(*p*); *rule_wd* ← *width*(*g*) − *cur_g*;
if *g_sign* ≠ *normal* **then**
begin if *g_sign* = *stretching* **then**
begin if *stretch_order*(*g*) = *g_order* **then**
begin *cur_glue* ← *cur_glue* + *stretch*(*g*); *vet_glue*(*float*(*glue_set*(*this_box*)) ∗ *cur_glue*);
cur_g ← *round*(*glue_temp*);
end;
end
else if *shrink_order*(*g*) = *g_order* **then**
begin *cur_glue* ← *cur_glue* − *shrink*(*g*); *vet_glue*(*float*(*glue_set*(*this_box*)) ∗ *cur_glue*);
cur_g ← *round*(*glue_temp*);
end;
end;
rule_wd ← *rule_wd* + *cur_g*

⟨ Move right or output leaders 663 ⟩ ≡
begin *round_glue*;
if *eTeX_ex* **then** ⟨ Handle a glue node for mixed direction typesetting 1509 ⟩;
if *subtype*(*p*) ≥ *a_leaders* **then**
⟨ Output leaders in an hlist, **goto** *fin_rule* if a rule or to *next_p* if done 664 ⟩;
goto *move_past*;
end

This code is used in section 660.

664. ⟨ Output leaders in an hlist, **goto** *fin_rule* if a rule or to *next_p* if done 664 ⟩ ≡
begin *leader_box* ← *leader_ptr*(*p*);
if *type*(*leader_box*) = *rule_node* **then**
begin *rule_ht* ← *height*(*leader_box*); *rule_dp* ← *depth*(*leader_box*); **goto** *fin_rule*;
end;
leader_wd ← *width*(*leader_box*);
if (*leader_wd* > 0) ∧ (*rule_wd* > 0) **then**
begin *rule_wd* ← *rule_wd* + 10; { compensate for floating-point rounding }
if *cur_dir* = *right_to_left* **then** *cur_h* ← *cur_h* − 10;
edge ← *cur_h* + *rule_wd*; *lx* ← 0; ⟨ Let *cur_h* be the position of the first box, and set *leader_wd* + *lx* to the spacing between corresponding parts of boxes 665 ⟩;
while *cur_h* + *leader_wd* ≤ *edge* **do**
⟨ Output a leader box at *cur_h*, then advance *cur_h* by *leader_wd* + *lx* 666 ⟩;
if *cur_dir* = *right_to_left* **then** *cur_h* ← *edge*
else *cur_h* ← *edge* − 10;
goto *next_p*;
end;
end

This code is used in section 663.

665. The calculations related to leaders require a bit of care. First, in the case of *a_leaders* (aligned leaders), we want to move *cur_h* to *left_edge* plus the smallest multiple of *leader_wd* for which the result is not less than the current value of *cur_h*; i.e., *cur_h* should become $left_edge + leader_wd \times \lceil (cur_h - left_edge)/leader_wd \rceil$. The program here should work in all cases even though some implementations of Pascal give nonstandard results for the **div** operation when *cur_h* is less than *left_edge*.

In the case of *c_leaders* (centered leaders), we want to increase *cur_h* by half of the excess space not occupied by the leaders; and in the case of *x_leaders* (expanded leaders) we increase *cur_h* by $1/(q+1)$ of this excess space, where q is the number of times the leader box will be replicated. Slight inaccuracies in the division might accumulate; half of this rounding error is placed at each end of the leaders.

```
⟨ Let cur_h be the position of the first box, and set leader_wd + lx to the spacing between corresponding
      parts of boxes 665 ⟩ ≡
  if subtype(p) = a_leaders then
    begin save_h ← cur_h; cur_h ← left_edge + leader_wd * ((cur_h − left_edge) div leader_wd);
    if cur_h < save_h then cur_h ← cur_h + leader_wd;
    end
  else begin lq ← rule_wd div leader_wd; { the number of box copies }
    lr ← rule_wd mod leader_wd; { the remaining space }
    if subtype(p) = c_leaders then cur_h ← cur_h + (lr div 2)
    else begin lx ← lr div (lq + 1); cur_h ← cur_h + ((lr − (lq − 1) * lx) div 2);
      end;
    end
```

This code is used in section 664.

666. The '*synch*' operations here are intended to decrease the number of bytes needed to specify horizontal and vertical motion in the DVI output.

```
⟨ Output a leader box at cur_h, then advance cur_h by leader_wd + lx 666 ⟩ ≡
  begin cur_v ← base_line + shift_amount(leader_box); synch_v; save_v ← dvi_v;
  synch_h; save_h ← dvi_h; temp_ptr ← leader_box;
  if cur_dir = right_to_left then cur_h ← cur_h + leader_wd;
  outer_doing_leaders ← doing_leaders; doing_leaders ← true;
  if type(leader_box) = vlist_node then vlist_out else hlist_out;
  doing_leaders ← outer_doing_leaders; dvi_v ← save_v; dvi_h ← save_h; cur_v ← base_line;
  cur_h ← save_h + leader_wd + lx;
  end
```

This code is used in section 664.

667. The *vlist_out* routine is similar to *hlist_out*, but a bit simpler.

```
procedure vlist_out; { output a vlist_node box }
  label move_past, fin_rule, next_p;
  var left_edge: scaled; { the left coordinate for this box }
    top_edge: scaled; { the top coordinate for this box }
    save_h, save_v: scaled; { what dvi_h and dvi_v should pop to }
    this_box: pointer; { pointer to containing box }
    g_order: glue_ord; { applicable order of infinity for glue }
    g_sign: normal .. shrinking; { selects type of glue }
    p: pointer; { current position in the vlist }
    save_loc: integer; { DVI byte location upon entry }
    leader_box: pointer; { the leader box being replicated }
    leader_ht: scaled; { height of leader box being replicated }
    lx: scaled; { extra space between leader boxes }
    outer_doing_leaders: boolean; { were we doing leaders? }
    edge: scaled; { bottom boundary of leader space }
    glue_temp: real; { glue value before rounding }
    cur_glue: real; { glue seen so far }
    cur_g: scaled; { rounded equivalent of cur_glue times the glue ratio }
    upwards: boolean; { whether we're stacking upwards }
  begin cur_g ← 0; cur_glue ← float_constant(0); this_box ← temp_ptr; g_order ← glue_order(this_box);
  g_sign ← glue_sign(this_box); p ← list_ptr(this_box);
  upwards ← (subtype(this_box) = min_quarterword + 1); incr(cur_s);
  if cur_s > 0 then dvi_out(push);
  if cur_s > max_push then max_push ← cur_s;
  save_loc ← dvi_offset + dvi_ptr; left_edge ← cur_h;
  if upwards then cur_v ← cur_v + depth(this_box)
  else cur_v ← cur_v − height(this_box);
  top_edge ← cur_v;
  while p ≠ null do ⟨ Output node p for vlist_out and move to the next node, maintaining the condition
      cur_h = left_edge 668 ⟩;
  prune_movements(save_loc);
  if cur_s > 0 then dvi_pop(save_loc);
  decr(cur_s);
  end;
```

668. ⟨ Output node p for *vlist_out* and move to the next node, maintaining the condition $cur_h = left_edge$ 668 ⟩ ≡

```
  begin if is_char_node(p) then confusion("vlistout")
  else ⟨ Output the non-char_node p for vlist_out 669 ⟩;
next_p: p ← link(p);
  end
```

This code is used in section 667.

669. ⟨ Output the non-*char_node* p for *vlist_out* 669 ⟩ ≡

```
begin case type(p) of
hlist_node, vlist_node: ⟨ Output a box in a vlist 670 ⟩;
rule_node: begin rule_ht ← height(p); rule_dp ← depth(p); rule_wd ← width(p); goto fin_rule;
  end;
whatsit_node: ⟨ Output the whatsit node p in a vlist 1426 ⟩;
glue_node: ⟨ Move down or output leaders 672 ⟩;
kern_node: if upwards then cur_v ← cur_v − width(p)
  else cur_v ← cur_v + width(p);
othercases do_nothing
endcases;
goto next_p;
fin_rule: ⟨ Output a rule in a vlist, goto next_p 671 ⟩;
move_past: if upwards then cur_v ← cur_v − rule_ht
  else cur_v ← cur_v + rule_ht;
end
```

This code is used in section 668.

670. The *synch_v* here allows the DVI output to use one-byte commands for adjusting v in most cases, since the baselineskip distance will usually be constant.

⟨ Output a box in a vlist 670 ⟩ ≡

```
if list_ptr(p) = null then cur_v ← cur_v + height(p) + depth(p)
else begin if upwards then cur_v ← cur_v − depth(p)
  else cur_v ← cur_v + height(p);
  synch_v; save_h ← dvi_h; save_v ← dvi_v;
  if cur_dir = right_to_left then cur_h ← left_edge − shift_amount(p)
  else cur_h ← left_edge + shift_amount(p);   { shift the box right }
  temp_ptr ← p;
  if type(p) = vlist_node then vlist_out else hlist_out;
  dvi_h ← save_h; dvi_v ← save_v;
  if upwards then cur_v ← save_v − height(p)
  else cur_v ← save_v + depth(p);
  cur_h ← left_edge;
  end
```

This code is used in section 669.

671. ⟨ Output a rule in a vlist, **goto** *next_p* 671 ⟩ ≡

```
if is_running(rule_wd) then rule_wd ← width(this_box);
rule_ht ← rule_ht + rule_dp;   { this is the rule thickness }
if upwards then cur_v ← cur_v − rule_ht
else cur_v ← cur_v + rule_ht;
if (rule_ht > 0) ∧ (rule_wd > 0) then   { we don't output empty rules }
  begin if cur_dir = right_to_left then cur_h ← cur_h − rule_wd;
  synch_h; synch_v; dvi_out(put_rule); dvi_four(rule_ht); dvi_four(rule_wd); cur_h ← left_edge;
  end;
goto next_p
```

This code is used in section 669.

672. ⟨ Move down or output leaders 672 ⟩ ≡
```
begin g ← glue_ptr(p); rule_ht ← width(g) − cur_g;
if g_sign ≠ normal then
  begin if g_sign = stretching then
    begin if stretch_order(g) = g_order then
      begin cur_glue ← cur_glue + stretch(g); vet_glue(float(glue_set(this_box)) ∗ cur_glue);
      cur_g ← round(glue_temp);
      end;
    end
  else if shrink_order(g) = g_order then
      begin cur_glue ← cur_glue − shrink(g); vet_glue(float(glue_set(this_box)) ∗ cur_glue);
      cur_g ← round(glue_temp);
      end;
  end;
rule_ht ← rule_ht + cur_g;
if subtype(p) ≥ a_leaders then
  ⟨ Output leaders in a vlist, goto fin_rule if a rule or to next_p if done 673 ⟩;
goto move_past;
end
```
This code is used in section 669.

673. ⟨ Output leaders in a vlist, **goto** *fin_rule* if a rule or to *next_p* if done 673 ⟩ ≡
```
begin leader_box ← leader_ptr(p);
if type(leader_box) = rule_node then
  begin rule_wd ← width(leader_box); rule_dp ← 0; goto fin_rule;
  end;
leader_ht ← height(leader_box) + depth(leader_box);
if (leader_ht > 0) ∧ (rule_ht > 0) then
  begin rule_ht ← rule_ht + 10;  { compensate for floating-point rounding }
  edge ← cur_v + rule_ht; lx ← 0; ⟨ Let cur_v be the position of the first box, and set leader_ht + lx to
      the spacing between corresponding parts of boxes 674 ⟩;
  while cur_v + leader_ht ≤ edge do
    ⟨ Output a leader box at cur_v, then advance cur_v by leader_ht + lx 675 ⟩;
  cur_v ← edge − 10; goto next_p;
  end;
end
```
This code is used in section 672.

674. ⟨ Let *cur_v* be the position of the first box, and set *leader_ht* + *lx* to the spacing between corresponding parts of boxes 674 ⟩ ≡
```
if subtype(p) = a_leaders then
  begin save_v ← cur_v; cur_v ← top_edge + leader_ht ∗ ((cur_v − top_edge) div leader_ht);
  if cur_v < save_v then cur_v ← cur_v + leader_ht;
  end
else begin lq ← rule_ht div leader_ht;  { the number of box copies }
  lr ← rule_ht mod leader_ht;  { the remaining space }
  if subtype(p) = c_leaders then cur_v ← cur_v + (lr div 2)
  else begin lx ← lr div (lq + 1); cur_v ← cur_v + ((lr − (lq − 1) ∗ lx) div 2);
    end;
  end
```
This code is used in section 673.

675. When we reach this part of the program, *cur_v* indicates the top of a leader box, not its baseline.

```
⟨ Output a leader box at cur_v, then advance cur_v by leader_ht + lx 675 ⟩ ≡
  begin if cur_dir = right_to_left then cur_h ← left_edge − shift_amount(leader_box)
  else cur_h ← left_edge + shift_amount(leader_box);
  synch_h; save_h ← dvi_h;
  cur_v ← cur_v + height(leader_box); synch_v; save_v ← dvi_v; temp_ptr ← leader_box;
  outer_doing_leaders ← doing_leaders; doing_leaders ← true;
  if type(leader_box) = vlist_node then vlist_out else hlist_out;
  doing_leaders ← outer_doing_leaders; dvi_v ← save_v; dvi_h ← save_h; cur_h ← left_edge;
  cur_v ← save_v − height(leader_box) + leader_ht + lx;
  end
```

This code is used in section 673.

676. The *hlist_out* and *vlist_out* procedures are now complete, so we are ready for the *ship_out* routine that gets them started in the first place.

```
procedure ship_out(p : pointer); { output the box p }
  label done;
  var page_loc: integer; { location of the current bop }
    j, k: 0 .. 9; { indices to first ten count registers }
    s: pool_pointer; { index into str_pool }
    old_setting: 0 .. max_selector; { saved selector setting }
  begin if job_name = 0 then open_log_file;
  if tracing_output > 0 then
    begin print_nl(""); print_ln; print("Completed␣box␣being␣shipped␣out");
    end;
  if term_offset > max_print_line − 9 then print_ln
  else if (term_offset > 0) ∨ (file_offset > 0) then print_char("␣");
  print_char("["); j ← 9;
  while (count(j) = 0) ∧ (j > 0) do decr(j);
  for k ← 0 to j do
    begin print_int(count(k));
    if k < j then print_char(".");
    end;
  update_terminal;
  if tracing_output > 0 then
    begin print_char("]"); begin_diagnostic; show_box(p); end_diagnostic(true);
    end;
  ⟨ Ship box p out 678 ⟩;
  if eTeX_ex then ⟨ Check for LR anomalies at the end of ship_out 1541 ⟩;
  if tracing_output ≤ 0 then print_char("]");
  dead_cycles ← 0; update_terminal; { progress report }
  ⟨ Flush the box from memory, showing statistics if requested 677 ⟩;
  end;
```

677. ⟨ Flush the box from memory, showing statistics if requested 677 ⟩ ≡

```
stat if tracing_stats > 1 then
  begin print_nl("Memory␣usage␣before:␣"); print_int(var_used); print_char("&");
  print_int(dyn_used); print_char(";");
  end;
tats
flush_node_list(p);
stat if tracing_stats > 1 then
  begin print("␣after:␣"); print_int(var_used); print_char("&"); print_int(dyn_used);
  print(";␣still␣untouched:␣"); print_int(hi_mem_min − lo_mem_max − 1); print_ln;
  end;
tats
```

This code is used in section 676.

678. ⟨ Ship box p out 678 ⟩ ≡

```
⟨ Update the values of max_h and max_v; but if the page is too large, goto done 679 ⟩;
⟨ Initialize variables as ship_out begins 653 ⟩;
page_loc ← dvi_offset + dvi_ptr; dvi_out(bop);
for k ← 0 to 9 do dvi_four(count(k));
dvi_four(last_bop); last_bop ← page_loc; { generate a pagesize special at start of page }
old_setting ← selector; selector ← new_string; print("pdf:pagesize␣");
if (pdf_page_width > 0) ∧ (pdf_page_height > 0) then
  begin print("width"); print("␣"); print_scaled(pdf_page_width); print("pt"); print("␣");
  print("height"); print("␣"); print_scaled(pdf_page_height); print("pt");
  end
else print("default");
selector ← old_setting; dvi_out(xxx1); dvi_out(cur_length);
for s ← str_start_macro(str_ptr) to pool_ptr − 1 do dvi_out(so(str_pool[s]));
pool_ptr ← str_start_macro(str_ptr); { erase the string }
cur_v ← height(p) + v_offset; { does this need changing for upwards mode ???? }
temp_ptr ← p;
if type(p) = vlist_node then vlist_out else hlist_out;
dvi_out(eop); incr(total_pages); cur_s ← −1;
if ¬no_pdf_output then fflush(dvi_file);
done:
```

This code is used in section 676.

679. Sometimes the user will generate a huge page because other error messages are being ignored. Such pages are not output to the `dvi` file, since they may confuse the printing software.

```
⟨ Update the values of max_h and max_v; but if the page is too large, goto done 679 ⟩ ≡
  if (height(p) > max_dimen) ∨ (depth(p) > max_dimen) ∨
        (height(p) + depth(p) + v_offset > max_dimen) ∨ (width(p) + h_offset > max_dimen) then
    begin print_err("Huge page cannot be shipped out");
    help2("The page just created is more than 18 feet tall or")
    ("more than 18 feet wide, so I suspect something went wrong."); error;
    if tracing_output ≤ 0 then
      begin begin_diagnostic; print_nl("The following box has been deleted:"); show_box(p);
      end_diagnostic(true);
      end;
    goto done;
    end;
  if height(p) + depth(p) + v_offset > max_v then max_v ← height(p) + depth(p) + v_offset;
  if width(p) + h_offset > max_h then max_h ← width(p) + h_offset
```

This code is used in section 678.

680. At the end of the program, we must finish things off by writing the postamble. If *total_pages* = 0, the `DVI` file was never opened. If *total_pages* ≥ 65536, the `DVI` file will lie. And if *max_push* ≥ 65536, the user deserves whatever chaos might ensue.

An integer variable k will be declared for use by this routine.

```
⟨ Finish the DVI file 680 ⟩ ≡
  while cur_s > −1 do
    begin if cur_s > 0 then dvi_out(pop)
    else begin dvi_out(eop); incr(total_pages);
      end;
    decr(cur_s);
    end;
  if total_pages = 0 then print_nl("No pages of output.")
  else begin dvi_out(post); { beginning of the postamble }
    dvi_four(last_bop); last_bop ← dvi_offset + dvi_ptr − 5; { post location }
    dvi_four(25400000); dvi_four(473628672); { conversion ratio for sp }
    prepare_mag; dvi_four(mag); { magnification factor }
    dvi_four(max_v); dvi_four(max_h);
    dvi_out(max_push div 256); dvi_out(max_push mod 256);
    dvi_out((total_pages div 256) mod 256); dvi_out(total_pages mod 256);
    ⟨ Output the font definitions for all fonts that were used 681 ⟩;
    dvi_out(post_post); dvi_four(last_bop); dvi_out(id_byte);
    k ← 4 + ((dvi_buf_size − dvi_ptr) mod 4); { the number of 223's }
    while k > 0 do
      begin dvi_out(223); decr(k);
      end;
    ⟨ Empty the last bytes out of dvi_buf 635 ⟩;
    print_nl("Output written on "); slow_print(output_file_name); print(" ("); print_int(total_pages);
    print(" page");
    if total_pages ≠ 1 then print_char("s");
    print(", "); print_int(dvi_offset + dvi_ptr); print(" bytes)."); b_close(dvi_file);
    end
```

This code is used in section 1387.

681. ⟨ Output the font definitions for all fonts that were used 681 ⟩ ≡

```
while font_ptr > font_base do
  begin if font_used[font_ptr] then dvi_font_def(font_ptr);
  decr(font_ptr);
  end
```

This code is used in section 680.

682. pdfTEX output low-level subroutines (equivalents).

⟨ Global variables 13 ⟩ +≡
epochseconds: *integer*;
microseconds: *integer*;

683. Packaging. We're essentially done with the parts of TEX that are concerned with the input (*get_next*) and the output (*ship_out*). So it's time to get heavily into the remaining part, which does the real work of typesetting.

After lists are constructed, TEX wraps them up and puts them into boxes. Two major subroutines are given the responsibility for this task: *hpack* applies to horizontal lists (hlists) and *vpack* applies to vertical lists (vlists). The main duty of *hpack* and *vpack* is to compute the dimensions of the resulting boxes, and to adjust the glue if one of those dimensions is pre-specified. The computed sizes normally enclose all of the material inside the new box; but some items may stick out if negative glue is used, if the box is overfull, or if a `\vbox` includes other boxes that have been shifted left.

The subroutine call $hpack(p, w, m)$ returns a pointer to an *hlist_node* for a box containing the hlist that starts at p. Parameter w specifies a width; and parameter m is either '*exactly*' or '*additional*'. Thus, $hpack(p, w, exactly)$ produces a box whose width is exactly w, while $hpack(p, w, additional)$ yields a box whose width is the natural width plus w. It is convenient to define a macro called '*natural*' to cover the most common case, so that we can say $hpack(p, natural)$ to get a box that has the natural width of list p.

Similarly, $vpack(p, w, m)$ returns a pointer to a *vlist_node* for a box containing the vlist that starts at p. In this case w represents a height instead of a width; the parameter m is interpreted as in *hpack*.

```
define exactly = 0  { a box dimension is pre-specified }
define additional = 1  { a box dimension is increased from the natural one }
define natural ≡ 0, additional  { shorthand for parameters to hpack and vpack }
```

684. The parameters to *hpack* and *vpack* correspond to TEX's primitives like '`\hbox to 300pt`', '`\hbox spread 10pt`'; note that '`\hbox`' with no dimension following it is equivalent to '`\hbox spread 0pt`'. The *scan_spec* subroutine scans such constructions in the user's input, including the mandatory left brace that follows them, and it puts the specification onto *save_stack* so that the desired box can later be obtained by executing the following code:

$$save_ptr \leftarrow save_ptr - 2;$$
$$hpack(p, saved(1), saved(0)).$$

Special care is necessary to ensure that the special *save_stack* codes are placed just below the new group code, because scanning can change *save_stack* when `\csname` appears.

```
procedure scan_spec(c : group_code; three_codes : boolean);  { scans a box specification and left brace }
  label found;
  var s: integer;  { temporarily saved value }
    spec_code: exactly .. additional;
  begin if three_codes then s ← saved(0);
  if scan_keyword("to") then spec_code ← exactly
  else if scan_keyword("spread") then spec_code ← additional
    else begin spec_code ← additional; cur_val ← 0; goto found;
      end;
  scan_normal_dimen;
found: if three_codes then
    begin saved(0) ← s; incr(save_ptr);
    end;
  saved(0) ← spec_code; saved(1) ← cur_val; save_ptr ← save_ptr + 2; new_save_level(c); scan_left_brace;
  end;
```

685. To figure out the glue setting, *hpack* and *vpack* determine how much stretchability and shrinkability are present, considering all four orders of infinity. The highest order of infinity that has a nonzero coefficient is then used as if no other orders were present.

For example, suppose that the given list contains six glue nodes with the respective stretchabilities 3pt, 8fill, 5fil, 6pt, −3fil, −8fill. Then the total is essentially 2fil; and if a total additional space of 6pt is to be achieved by stretching, the actual amounts of stretch will be 0pt, 0pt, 15pt, 0pt, −9pt, and 0pt, since only 'fil' glue will be considered. (The 'fill' glue is therefore not really stretching infinitely with respect to 'fil'; nobody would actually want that to happen.)

The arrays *total_stretch* and *total_shrink* are used to determine how much glue of each kind is present. A global variable *last_badness* is used to implement `\badness`.

⟨ Global variables 13 ⟩ +≡
total_stretch, *total_shrink*: **array** [*glue_ord*] **of** *scaled*; { glue found by *hpack* or *vpack* }
last_badness: *integer*; { badness of the most recently packaged box }

686. If the global variable *adjust_tail* is non-null, the *hpack* routine also removes all occurrences of *ins_node*, *mark_node*, and *adjust_node* items and appends the resulting material onto the list that ends at location *adjust_tail*.

⟨ Global variables 13 ⟩ +≡
adjust_tail: *pointer*; { tail of adjustment list }

687. ⟨ Set initial values of key variables 23 ⟩ +≡
adjust_tail ← *null*; *last_badness* ← 0;

688. Some stuff for character protrusion.

```
define left_pw(#) ≡ char_pw(#, left_side)
define right_pw(#) ≡ char_pw(#, right_side)
function char_pw(p : pointer; side : small_number): scaled;
  var f: internal_font_number; c: integer;
  begin char_pw ← 0;
  if side = left_side then last_leftmost_char ← null
  else last_rightmost_char ← null;
  if p = null then return; { native word }
  if is_native_word_node(p) then
    begin if native_glyph_info_ptr(p) ≠ null_ptr then
      begin f ← native_font(p); char_pw ← round_xn_over_d(quad(f), get_native_word_cp(p, side), 1000);
      end;
    return;
    end; { glyph node }
  if is_glyph_node(p) then
    begin f ← native_font(p);
    char_pw ← round_xn_over_d(quad(f), get_cp_code(f, native_glyph(p), side), 1000); return;
    end; { char node or ligature; same like pdftex }
  if ¬is_char_node(p) then
    begin if type(p) = ligature_node then p ← lig_char(p)
    else return;
    end;
  f ← font(p); c ← get_cp_code(f, character(p), side);
  case side of
  left_side: last_leftmost_char ← p;
  right_side: last_rightmost_char ← p;
  endcases;
  if c = 0 then return;
  char_pw ← round_xn_over_d(quad(f), c, 1000);
  end;
function new_margin_kern(w : scaled; p : pointer; side : small_number): pointer;
  var k: pointer;
  begin k ← get_node(margin_kern_node_size); type(k) ← margin_kern_node; subtype(k) ← side;
  width(k) ← w; new_margin_kern ← k;
  end;
```

689. Here now is *hpack*, which contains few if any surprises.

```
function hpack(p : pointer; w : scaled; m : small_number): pointer;
  label reswitch, common_ending, exit, restart;
  var r: pointer; { the box node that will be returned }
    q: pointer; { trails behind p }
    h, d, x: scaled; { height, depth, and natural width }
    s: scaled; { shift amount }
    g: pointer; { points to a glue specification }
    o: glue_ord; { order of infinity }
    f: internal_font_number; { the font in a char_node }
    i: four_quarters; { font information about a char_node }
    hd: eight_bits; { height and depth indices for a character }
    pp, ppp: pointer; total_chars, k: integer;
  begin last_badness ← 0; r ← get_node(box_node_size); type(r) ← hlist_node;
  subtype(r) ← min_quarterword; shift_amount(r) ← 0; q ← r + list_offset; link(q) ← p;
  h ← 0; ⟨ Clear dimensions to zero 690 ⟩;
  if TeXXeT_en then ⟨ Initialize the LR stack 1520 ⟩;
  while p ≠ null do ⟨ Examine node p in the hlist, taking account of its effect on the dimensions of the
      new box, or moving it to the adjustment list; then advance p to the next node 691 ⟩;
  if adjust_tail ≠ null then link(adjust_tail) ← null;
  if pre_adjust_tail ≠ null then link(pre_adjust_tail) ← null;
  height(r) ← h; depth(r) ← d;
  ⟨ Determine the value of width(r) and the appropriate glue setting; then return or goto
      common_ending 699 ⟩;
common_ending: ⟨ Finish issuing a diagnostic message for an overfull or underfull hbox 705 ⟩;
exit: if TeXXeT_en then ⟨ Check for LR anomalies at the end of hpack 1522 ⟩;
  hpack ← r;
  end;
```

690. ⟨ Clear dimensions to zero 690 ⟩ ≡

```
d ← 0; x ← 0; total_stretch[normal] ← 0; total_shrink[normal] ← 0; total_stretch[fil] ← 0;
total_shrink[fil] ← 0; total_stretch[fill] ← 0; total_shrink[fill] ← 0; total_stretch[filll] ← 0;
total_shrink[filll] ← 0
```

This code is used in sections 689 and 710.

691. ⟨ Examine node p in the hlist, taking account of its effect on the dimensions of the new box, or moving it to the adjustment list; then advance p to the next node 691 ⟩ ≡

```
begin reswitch: while is_char_node(p) do ⟨ Incorporate character dimensions into the dimensions of the hbox that will contain it, then move to the next node 694 ⟩;
if p ≠ null then
  begin case type(p) of
  hlist_node, vlist_node, rule_node, unset_node: ⟨ Incorporate box dimensions into the dimensions of the hbox that will contain it 693 ⟩;
  ins_node, mark_node, adjust_node: if (adjust_tail ≠ null) ∨ (pre_adjust_tail ≠ null) then ⟨ Transfer node p to the adjustment list 697 ⟩;
  whatsit_node: ⟨ Incorporate a whatsit node into an hbox 1420 ⟩;
  glue_node: ⟨ Incorporate glue into the horizontal totals 698 ⟩;
  kern_node: x ← x + width(p);
  margin_kern_node: x ← x + width(p);
  math_node: begin x ← x + width(p);
    if TeXXeT_en then ⟨ Adjust the LR stack for the hpack routine 1521 ⟩;
    end;
  ligature_node: ⟨ Make node p look like a char_node and goto reswitch 692 ⟩;
  othercases do_nothing
  endcases;
  p ← link(p);
  end;
end
```

This code is used in section 689.

692. ⟨ Make node p look like a *char_node* and **goto** *reswitch* 692 ⟩ ≡

```
begin mem[lig_trick] ← mem[lig_char(p)]; link(lig_trick) ← link(p); p ← lig_trick;
xtx_ligature_present ← true; goto reswitch;
end
```

This code is used in sections 660, 691, and 1201.

693. The code here implicitly uses the fact that running dimensions are indicated by *null_flag*, which will be ignored in the calculations because it is a highly negative number.

⟨ Incorporate box dimensions into the dimensions of the hbox that will contain it 693 ⟩ ≡

```
begin x ← x + width(p);
if type(p) ≥ rule_node then s ← 0 else s ← shift_amount(p);
if height(p) − s > h then h ← height(p) − s;
if depth(p) + s > d then d ← depth(p) + s;
end
```

This code is used in section 691.

694. The following code is part of TeX's inner loop; i.e., adding another character of text to the user's input will cause each of these instructions to be exercised one more time.

⟨ Incorporate character dimensions into the dimensions of the hbox that will contain it, then move to the next node 694 ⟩ ≡

```
begin f ← font(p); i ← char_info(f)(character(p)); hd ← height_depth(i); x ← x + char_width(f)(i);
s ← char_height(f)(hd); if s > h then h ← s;
s ← char_depth(f)(hd); if s > d then d ← s;
p ← link(p);
end
```

This code is used in section 691.

695. Although node q is not necessarily the immediate predecessor of node p, it always points to some node in the list preceding p. Thus, we can delete nodes by moving q when necessary. The algorithm takes linear time, and the extra computation does not intrude on the inner loop unless it is necessary to make a deletion.

```
⟨ Global variables 13 ⟩ +≡
pre_adjust_tail: pointer;
```

696.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  pre_adjust_tail ← null;
```

697. Materials in `\vadjust` used with `pre` keyword will be appended to *pre_adjust_tail* instead of *adjust_tail*.

```
define update_adjust_list(#) ≡
        begin if # = null then confusion("pre␣vadjust");
        link(#) ← adjust_ptr(p);
        while link(#) ≠ null do # ← link(#);
        end
⟨ Transfer node p to the adjustment list 697 ⟩ ≡
  begin while link(q) ≠ p do q ← link(q);
  if type(p) = adjust_node then
    begin if adjust_pre(p) ≠ 0 then update_adjust_list(pre_adjust_tail)
    else update_adjust_list(adjust_tail);
    p ← link(p); free_node(link(q), small_node_size);
    end
  else begin link(adjust_tail) ← p; adjust_tail ← p; p ← link(p);
    end;
  link(q) ← p; p ← q;
  end
```

This code is used in section 691.

698.
```
⟨ Incorporate glue into the horizontal totals 698 ⟩ ≡
  begin g ← glue_ptr(p); x ← x + width(g);
  o ← stretch_order(g); total_stretch[o] ← total_stretch[o] + stretch(g); o ← shrink_order(g);
  total_shrink[o] ← total_shrink[o] + shrink(g);
  if subtype(p) ≥ a_leaders then
    begin g ← leader_ptr(p);
    if height(g) > h then h ← height(g);
    if depth(g) > d then d ← depth(g);
    end;
  end
```

This code is used in section 691.

699. When we get to the present part of the program, x is the natural width of the box being packaged.

```
⟨ Determine the value of width(r) and the appropriate glue setting; then return or goto
      common_ending 699 ⟩ ≡
  if m = additional then w ← x + w;
  width(r) ← w; x ← w − x; { now x is the excess to be made up }
  if x = 0 then
    begin glue_sign(r) ← normal; glue_order(r) ← normal; set_glue_ratio_zero(glue_set(r)); return;
    end
  else if x > 0 then
      ⟨ Determine horizontal glue stretch setting, then return or goto common_ending 700 ⟩
    else ⟨ Determine horizontal glue shrink setting, then return or goto common_ending 706 ⟩
```

This code is used in section 689.

700.

```
⟨ Determine horizontal glue stretch setting, then return or goto common_ending 700 ⟩ ≡
  begin ⟨ Determine the stretch order 701 ⟩;
  glue_order(r) ← o; glue_sign(r) ← stretching;
  if total_stretch[o] ≠ 0 then glue_set(r) ← unfloat(x/total_stretch[o])
  else begin glue_sign(r) ← normal; set_glue_ratio_zero(glue_set(r)); { there's nothing to stretch }
    end;
  if o = normal then
    if list_ptr(r) ≠ null then
      ⟨ Report an underfull hbox and goto common_ending, if this box is sufficiently bad 702 ⟩;
  return;
  end
```

This code is used in section 699.

701.

```
⟨ Determine the stretch order 701 ⟩ ≡
  if total_stretch[filll] ≠ 0 then o ← filll
  else if total_stretch[fill] ≠ 0 then o ← fill
    else if total_stretch[fil] ≠ 0 then o ← fil
      else o ← normal
```

This code is used in sections 700, 715, and 844.

702.

```
⟨ Report an underfull hbox and goto common_ending, if this box is sufficiently bad 702 ⟩ ≡
  begin last_badness ← badness(x, total_stretch[normal]);
  if last_badness > hbadness then
    begin print_ln;
    if last_badness > 100 then print_nl("Underfull") else print_nl("Loose");
    print("␣\hbox␣(badness␣"); print_int(last_badness); goto common_ending;
    end;
  end
```

This code is used in section 700.

703. In order to provide a decent indication of where an overfull or underfull box originated, we use a global variable *pack_begin_line* that is set nonzero only when *hpack* is being called by the paragraph builder or the alignment finishing routine.

```
⟨ Global variables 13 ⟩ +≡
pack_begin_line: integer; { source file line where the current paragraph or alignment began; a negative
      value denotes alignment }
```

704. ⟨ Set initial values of key variables 23 ⟩ +≡
pack_begin_line ← 0;

705. ⟨ Finish issuing a diagnostic message for an overfull or underfull hbox 705 ⟩ ≡
if *output_active* **then** *print*("␣has␣occurred␣while␣\output␣is␣active")
else begin if *pack_begin_line* ≠ 0 **then**
begin if *pack_begin_line* > 0 **then** *print*("␣in␣paragraph␣at␣lines␣")
else *print*("␣in␣alignment␣at␣lines␣");
print_int(*abs*(*pack_begin_line*)); *print*("--");
end
else *print*("␣detected␣at␣line␣");
print_int(*line*);
end;
print_ln;
font_in_short_display ← *null_font*; *short_display*(*list_ptr*(*r*)); *print_ln*;
begin_diagnostic; *show_box*(*r*); *end_diagnostic*(*true*)

This code is used in section 689.

706. ⟨ Determine horizontal glue shrink setting, then **return** or **goto** *common_ending* 706 ⟩ ≡
begin ⟨ Determine the shrink order 707 ⟩;
glue_order(*r*) ← *o*; *glue_sign*(*r*) ← *shrinking*;
if *total_shrink*[*o*] ≠ 0 **then** *glue_set*(*r*) ← *unfloat*((−*x*)/*total_shrink*[*o*])
else begin *glue_sign*(*r*) ← *normal*; *set_glue_ratio_zero*(*glue_set*(*r*)); { there's nothing to shrink }
end;
if (*total_shrink*[*o*] < −*x*) ∧ (*o* = *normal*) ∧ (*list_ptr*(*r*) ≠ *null*) **then**
begin *last_badness* ← 1000000; *set_glue_ratio_one*(*glue_set*(*r*)); { use the maximum shrinkage }
⟨ Report an overfull hbox and **goto** *common_ending*, if this box is sufficiently bad 708 ⟩;
end
else if *o* = *normal* **then**
if *list_ptr*(*r*) ≠ *null* **then**
⟨ Report a tight hbox and **goto** *common_ending*, if this box is sufficiently bad 709 ⟩;
return;
end

This code is used in section 699.

707. ⟨ Determine the shrink order 707 ⟩ ≡
if *total_shrink*[*filll*] ≠ 0 **then** *o* ← *filll*
else if *total_shrink*[*fill*] ≠ 0 **then** *o* ← *fill*
else if *total_shrink*[*fil*] ≠ 0 **then** *o* ← *fil*
else *o* ← *normal*

This code is used in sections 706, 718, and 844.

708. ⟨ Report an overfull hbox and **goto** *common_ending*, if this box is sufficiently bad 708 ⟩ ≡
if (−*x* − *total_shrink*[*normal*] > *hfuzz*) ∨ (*hbadness* < 100) **then**
begin if (*overfull_rule* > 0) ∧ (−*x* − *total_shrink*[*normal*] > *hfuzz*) **then**
begin while *link*(*q*) ≠ *null* **do** *q* ← *link*(*q*);
link(*q*) ← *new_rule*; *width*(*link*(*q*)) ← *overfull_rule*;
end;
print_ln; *print_nl*("Overfull␣\hbox␣("); *print_scaled*(−*x* − *total_shrink*[*normal*]);
print("pt␣too␣wide"); **goto** *common_ending*;
end

This code is used in section 706.

709. ⟨ Report a tight hbox and **goto** *common_ending*, if this box is sufficiently bad 709 ⟩ ≡

```
begin last_badness ← badness(−x, total_shrink[normal]);
if last_badness > hbadness then
  begin print_ln; print_nl("Tight␣\hbox␣(badness␣"); print_int(last_badness); goto common_ending;
  end;
end
```

This code is used in section 706.

710. The *vpack* subroutine is actually a special case of a slightly more general routine called *vpackage*, which has four parameters. The fourth parameter, which is *max_dimen* in the case of *vpack*, specifies the maximum depth of the page box that is constructed. The depth is first computed by the normal rules; if it exceeds this limit, the reference point is simply moved down until the limiting depth is attained.

```
define vpack(#) ≡ vpackage(#, max_dimen)  { special case of unconstrained depth }
function vpackage(p : pointer; h : scaled; m : small_number; l : scaled): pointer;
  label common_ending, exit;
  var r: pointer;  { the box node that will be returned }
    w, d, x: scaled;  { width, depth, and natural height }
    s: scaled;  { shift amount }
    g: pointer;  { points to a glue specification }
    o: glue_ord;  { order of infinity }
  begin last_badness ← 0; r ← get_node(box_node_size); type(r) ← vlist_node;
  if XeTeX_upwards then subtype(r) ← min_quarterword + 1
  else subtype(r) ← min_quarterword;
  shift_amount(r) ← 0; list_ptr(r) ← p;
  w ← 0; ⟨ Clear dimensions to zero 690 ⟩;
  while p ≠ null do ⟨ Examine node p in the vlist, taking account of its effect on the dimensions of the
      new box; then advance p to the next node 711 ⟩;
  width(r) ← w;
  if d > l then
    begin x ← x + d − l; depth(r) ← l;
    end
  else depth(r) ← d;
  ⟨ Determine the value of height(r) and the appropriate glue setting; then return or goto
      common_ending 714 ⟩;
common_ending: ⟨ Finish issuing a diagnostic message for an overfull or underfull vbox 717 ⟩;
exit: vpackage ← r;
  end;
```

711. ⟨ Examine node p in the vlist, taking account of its effect on the dimensions of the new box; then advance p to the next node 711 ⟩ ≡

```
begin if is_char_node(p) then confusion("vpack")
else case type(p) of
  hlist_node, vlist_node, rule_node, unset_node: ⟨ Incorporate box dimensions into the dimensions of the
      vbox that will contain it 712 ⟩;
  whatsit_node: ⟨ Incorporate a whatsit node into a vbox 1419 ⟩;
  glue_node: ⟨ Incorporate glue into the vertical totals 713 ⟩;
  kern_node: begin x ← x + d + width(p); d ← 0;
    end;
  othercases do_nothing
  endcases;
p ← link(p);
end
```

This code is used in section 710.

712. ⟨ Incorporate box dimensions into the dimensions of the vbox that will contain it 712 ⟩ ≡

```
begin x ← x + d + height(p); d ← depth(p);
if type(p) ≥ rule_node then s ← 0 else s ← shift_amount(p);
if width(p) + s > w then w ← width(p) + s;
end
```

This code is used in section 711.

713. ⟨ Incorporate glue into the vertical totals 713 ⟩ ≡

```
begin x ← x + d; d ← 0;
g ← glue_ptr(p); x ← x + width(g);
o ← stretch_order(g); total_stretch[o] ← total_stretch[o] + stretch(g); o ← shrink_order(g);
total_shrink[o] ← total_shrink[o] + shrink(g);
if subtype(p) ≥ a_leaders then
  begin g ← leader_ptr(p);
  if width(g) > w then w ← width(g);
  end;
end
```

This code is used in section 711.

714. When we get to the present part of the program, x is the natural height of the box being packaged.

⟨ Determine the value of $height(r)$ and the appropriate glue setting; then **return** or **goto** *common_ending* 714 ⟩ ≡

```
if m = additional then h ← x + h;
height(r) ← h; x ← h − x; { now x is the excess to be made up }
if x = 0 then
  begin glue_sign(r) ← normal; glue_order(r) ← normal; set_glue_ratio_zero(glue_set(r)); return;
  end
else if x > 0 then ⟨ Determine vertical glue stretch setting, then return or goto common_ending 715 ⟩
  else ⟨ Determine vertical glue shrink setting, then return or goto common_ending 718 ⟩
```

This code is used in section 710.

715. ⟨ Determine vertical glue stretch setting, then **return** or **goto** *common_ending* 715 ⟩ ≡
```
begin ⟨ Determine the stretch order 701 ⟩;
glue_order(r) ← o; glue_sign(r) ← stretching;
if total_stretch[o] ≠ 0 then glue_set(r) ← unfloat(x/total_stretch[o])
else begin glue_sign(r) ← normal; set_glue_ratio_zero(glue_set(r)); { there's nothing to stretch }
  end;
if o = normal then
  if list_ptr(r) ≠ null then
    ⟨ Report an underfull vbox and goto common_ending, if this box is sufficiently bad 716 ⟩;
return;
end
```
This code is used in section 714.

716. ⟨ Report an underfull vbox and **goto** *common_ending*, if this box is sufficiently bad 716 ⟩ ≡
```
begin last_badness ← badness(x, total_stretch[normal]);
if last_badness > vbadness then
  begin print_ln;
  if last_badness > 100 then print_nl("Underfull") else print_nl("Loose");
  print("␣\vbox␣(badness␣"); print_int(last_badness); goto common_ending;
  end;
end
```
This code is used in section 715.

717. ⟨ Finish issuing a diagnostic message for an overfull or underfull vbox 717 ⟩ ≡
```
if output_active then print(")␣has␣occurred␣while␣\output␣is␣active")
else begin if pack_begin_line ≠ 0 then { it's actually negative }
    begin print(")␣in␣alignment␣at␣lines␣"); print_int(abs(pack_begin_line)); print("--");
    end
  else print(")␣detected␣at␣line␣");
  print_int(line); print_ln;
  end;
begin_diagnostic; show_box(r); end_diagnostic(true)
```
This code is used in section 710.

718. ⟨ Determine vertical glue shrink setting, then **return** or **goto** *common_ending* 718 ⟩ ≡
```
begin ⟨ Determine the shrink order 707 ⟩;
glue_order(r) ← o; glue_sign(r) ← shrinking;
if total_shrink[o] ≠ 0 then glue_set(r) ← unfloat((−x)/total_shrink[o])
else begin glue_sign(r) ← normal; set_glue_ratio_zero(glue_set(r)); { there's nothing to shrink }
  end;
if (total_shrink[o] < −x) ∧ (o = normal) ∧ (list_ptr(r) ≠ null) then
  begin last_badness ← 1000000; set_glue_ratio_one(glue_set(r)); { use the maximum shrinkage }
  ⟨ Report an overfull vbox and goto common_ending, if this box is sufficiently bad 719 ⟩;
  end
else if o = normal then
    if list_ptr(r) ≠ null then
      ⟨ Report a tight vbox and goto common_ending, if this box is sufficiently bad 720 ⟩;
return;
end
```
This code is used in section 714.

719. ⟨ Report an overfull vbox and **goto** *common_ending*, if this box is sufficiently bad 719 ⟩ ≡

```
if (−x − total_shrink[normal] > vfuzz) ∨ (vbadness < 100) then
  begin print_ln; print_nl("Overfull␣\vbox␣("); print_scaled(−x − total_shrink[normal]);
  print("pt␣too␣high"); goto common_ending;
  end
```

This code is used in section 718.

720. ⟨ Report a tight vbox and **goto** *common_ending*, if this box is sufficiently bad 720 ⟩ ≡

```
begin last_badness ← badness(−x, total_shrink[normal]);
if last_badness > vbadness then
  begin print_ln; print_nl("Tight␣\vbox␣(badness␣"); print_int(last_badness); goto common_ending;
  end;
end
```

This code is used in section 718.

721. When a box is being appended to the current vertical list, the baselineskip calculation is handled by the *append_to_vlist* routine.

```
procedure append_to_vlist(b : pointer);
  var d: scaled; { deficiency of space between baselines }
    p: pointer; { a new glue node }
    upwards: boolean;
  begin upwards ← XeTeX_upwards;
  if prev_depth > ignore_depth then
    begin if upwards then d ← width(baseline_skip) − prev_depth − depth(b)
    else d ← width(baseline_skip) − prev_depth − height(b);
    if d < line_skip_limit then p ← new_param_glue(line_skip_code)
    else begin p ← new_skip_param(baseline_skip_code); width(temp_ptr) ← d;
          { temp_ptr = glue_ptr(p) }
      end;
    link(tail) ← p; tail ← p;
    end;
  link(tail) ← b; tail ← b;
  if upwards then prev_depth ← height(b)
  else prev_depth ← depth(b);
  end;
```

722. Data structures for math mode. When TeX reads a formula that is enclosed between `$`'s, it constructs an *mlist*, which is essentially a tree structure representing that formula. An mlist is a linear sequence of items, but we can regard it as a tree structure because mlists can appear within mlists. For example, many of the entries can be subscripted or superscripted, and such "scripts" are mlists in their own right.

An entire formula is parsed into such a tree before any of the actual typesetting is done, because the current style of type is usually not known until the formula has been fully scanned. For example, when the formula '`$a+b \over c+d$`' is being read, there is no way to tell that '`a+b`' will be in script size until '`\over`' has appeared.

During the scanning process, each element of the mlist being built is classified as a relation, a binary operator, an open parenthesis, etc., or as a construct like '`\sqrt`' that must be built up. This classification appears in the mlist data structure.

After a formula has been fully scanned, the mlist is converted to an hlist so that it can be incorporated into the surrounding text. This conversion is controlled by a recursive procedure that decides all of the appropriate styles by a "top-down" process starting at the outermost level and working in towards the subformulas. The formula is ultimately pasted together using combinations of horizontal and vertical boxes, with glue and penalty nodes inserted as necessary.

An mlist is represented internally as a linked list consisting chiefly of "noads" (pronounced "no-adds"), to distinguish them from the somewhat similar "nodes" in hlists and vlists. Certain kinds of ordinary nodes are allowed to appear in mlists together with the noads; TeX tells the difference by means of the *type* field, since a noad's *type* is always greater than that of a node. An mlist does not contain character nodes, hlist nodes, vlist nodes, math nodes, ligature nodes, or unset nodes; in particular, each mlist item appears in the variable-size part of *mem*, so the *type* field is always present.

723. Each noad is four or more words long. The first word contains the *type* and *subtype* and *link* fields that are already so familiar to us; the second, third, and fourth words are called the noad's *nucleus*, *subscr*, and *supscr* fields.

Consider, for example, the simple formula '`$x^2$`', which would be parsed into an mlist containing a single element called an *ord_noad*. The *nucleus* of this noad is a representation of '`x`', the *subscr* is empty, and the *supscr* is a representation of '`2`'.

The *nucleus*, *subscr*, and *supscr* fields are further broken into subfields. If p points to a noad, and if q is one of its principal fields (e.g., $q = subscr(p)$), there are several possibilities for the subfields, depending on the *math_type* of q.

math_type(q) = *math_char* means that *fam*(q) refers to one of the sixteen font families, and *character*(q) is the number of a character within a font of that family, as in a character node.

math_type(q) = *math_text_char* is similar, but the character is unsubscripted and unsuperscripted and it is followed immediately by another character from the same font. (This *math_type* setting appears only briefly during the processing; it is used to suppress unwanted italic corrections.)

math_type(q) = *empty* indicates a field with no value (the corresponding attribute of noad p is not present).

math_type(q) = *sub_box* means that *info*(q) points to a box node (either an *hlist_node* or a *vlist_node*) that should be used as the value of the field. The *shift_amount* in the subsidiary box node is the amount by which that box will be shifted downward.

math_type(q) = *sub_mlist* means that *info*(q) points to an mlist; the mlist must be converted to an hlist in order to obtain the value of this field.

In the latter case, we might have *info*(q) = *null*. This is not the same as *math_type*(q) = *empty*; for example, '`$P_{}$`' and '`$P$`' produce different results (the former will not have the "italic correction" added to the width of P, but the "script skip" will be added).

The definitions of subfields given here are evidently wasteful of space, since a halfword is being used for the *math_type* although only three bits would be needed. However, there are hardly ever many noads present at once, since they are soon converted to nodes that take up even more space, so we can afford to represent them in whatever way simplifies the programming.

define *noad_size* = 4 { number of words in a normal noad }
define *nucleus*(#) ≡ # + 1 { the *nucleus* field of a noad }
define *supscr*(#) ≡ # + 2 { the *supscr* field of a noad }
define *subscr*(#) ≡ # + 3 { the *subscr* field of a noad }
define *math_type* ≡ *link* { a *halfword* in *mem* }
define *plane_and_fam_field* ≡ *font* { a *quarterword* in *mem* }
define *fam*(#) ≡ (*plane_and_fam_field*(#) **mod** "100)
define *math_char* = 1 { *math_type* when the attribute is simple }
define *sub_box* = 2 { *math_type* when the attribute is a box }
define *sub_mlist* = 3 { *math_type* when the attribute is a formula }
define *math_text_char* = 4 { *math_type* when italic correction is dubious }

724. Each portion of a formula is classified as Ord, Op, Bin, Rel, Open, Close, Punct, or Inner, for purposes of spacing and line breaking. An *ord_noad*, *op_noad*, *bin_noad*, *rel_noad*, *open_noad*, *close_noad*, *punct_noad*, or *inner_noad* is used to represent portions of the various types. For example, an '=' sign in a formula leads to the creation of a *rel_noad* whose *nucleus* field is a representation of an equals sign (usually $fam = 0$, $character = "75$). A formula preceded by `\mathrel` also results in a *rel_noad*. When a *rel_noad* is followed by an *op_noad*, say, and possibly separated by one or more ordinary nodes (not noads), TeX will insert a penalty node (with the current *rel_penalty*) just after the formula that corresponds to the *rel_noad*, unless there already was a penalty immediately following; and a "thick space" will be inserted just before the formula that corresponds to the *op_noad*.

A noad of type *ord_noad*, *op_noad*, ..., *inner_noad* usually has a *subtype* = *normal*. The only exception is that an *op_noad* might have *subtype* = *limits* or *no_limits*, if the normal positioning of limits has been overridden for this operator.

define *ord_noad* = *unset_node* + 3 { *type* of a noad classified Ord }
define *op_noad* = *ord_noad* + 1 { *type* of a noad classified Op }
define *bin_noad* = *ord_noad* + 2 { *type* of a noad classified Bin }
define *rel_noad* = *ord_noad* + 3 { *type* of a noad classified Rel }
define *open_noad* = *ord_noad* + 4 { *type* of a noad classified Open }
define *close_noad* = *ord_noad* + 5 { *type* of a noad classified Close }
define *punct_noad* = *ord_noad* + 6 { *type* of a noad classified Punct }
define *inner_noad* = *ord_noad* + 7 { *type* of a noad classified Inner }
define *limits* = 1 { *subtype* of *op_noad* whose scripts are to be above, below }
define *no_limits* = 2 { *subtype* of *op_noad* whose scripts are to be normal }

725. A *radical_noad* is five words long; the fifth word is the *left_delimiter* field, which usually represents a square root sign.

A *fraction_noad* is six words long; it has a *right_delimiter* field as well as a *left_delimiter*.

Delimiter fields are of type *four_quarters*, and they have four subfields called *small_fam*, *small_char*, *large_fam*, *large_char*. These subfields represent variable-size delimiters by giving the "small" and "large" starting characters, as explained in Chapter 17 of *The TEXbook*.

A *fraction_noad* is actually quite different from all other noads. Not only does it have six words, it has *thickness*, *denominator*, and *numerator* fields instead of *nucleus*, *subscr*, and *supscr*. The *thickness* is a scaled value that tells how thick to make a fraction rule; however, the special value *default_code* is used to stand for the *default_rule_thickness* of the current size. The *numerator* and *denominator* point to mlists that define a fraction; we always have

$$math_type(numerator) = math_type(denominator) = sub_mlist.$$

The *left_delimiter* and *right_delimiter* fields specify delimiters that will be placed at the left and right of the fraction. In this way, a *fraction_noad* is able to represent all of TEX's operators `\over`, `\atop`, `\above`, `\overwithdelims`, `\atopwithdelims`, and `\abovewithdelims`.

define *left_delimiter*(#) ≡ # + 4 { first delimiter field of a noad }
define *right_delimiter*(#) ≡ # + 5 { second delimiter field of a fraction noad }
define *radical_noad* = *inner_noad* + 1 { *type* of a noad for square roots }
define *radical_noad_size* = 5 { number of *mem* words in a radical noad }
define *fraction_noad* = *radical_noad* + 1 { *type* of a noad for generalized fractions }
define *fraction_noad_size* = 6 { number of *mem* words in a fraction noad }
define *small_fam*(#) ≡ (*mem*[#].*qqqq.b0* **mod** ″100) { *fam* for "small" delimiter }
define *small_char*(#) ≡ (*mem*[#].*qqqq.b1* + (*mem*[#].*qqqq.b0* **div** ″100) ∗ ″10000)
{ *character* for "small" delimiter }
define *large_fam*(#) ≡ (*mem*[#].*qqqq.b2* **mod** ″100) { *fam* for "large" delimiter }
define *large_char*(#) ≡ (*mem*[#].*qqqq.b3* + (*mem*[#].*qqqq.b2* **div** ″100) ∗ ″10000)
{ *character* for "large" delimiter }
define *small_plane_and_fam_field*(#) ≡ *mem*[#].*qqqq.b0*
define *small_char_field*(#) ≡ *mem*[#].*qqqq.b1*
define *large_plane_and_fam_field*(#) ≡ *mem*[#].*qqqq.b2*
define *large_char_field*(#) ≡ *mem*[#].*qqqq.b3*
define *thickness* ≡ *width* { *thickness* field in a fraction noad }
define *default_code* ≡ ′10000000000 { denotes *default_rule_thickness* }
define *numerator* ≡ *supscr* { *numerator* field in a fraction noad }
define *denominator* ≡ *subscr* { *denominator* field in a fraction noad }

726. The global variable *empty_field* is set up for initialization of empty fields in new noads. Similarly, *null_delimiter* is for the initialization of delimiter fields.

⟨ Global variables 13 ⟩ +≡
empty_field: *two_halves*;
null_delimiter: *four_quarters*;

727. ⟨ Set initial values of key variables 23 ⟩ +≡
empty_field.rh ← *empty*; *empty_field.lh* ← *null*;
null_delimiter.b0 ← 0; *null_delimiter.b1* ← *min_quarterword*;
null_delimiter.b2 ← 0; *null_delimiter.b3* ← *min_quarterword*;

728. The *new_noad* function creates an *ord_noad* that is completely null.

```
function new_noad: pointer;
  var p: pointer;
  begin p ← get_node(noad_size); type(p) ← ord_noad; subtype(p) ← normal;
  mem[nucleus(p)].hh ← empty_field; mem[subscr(p)].hh ← empty_field;
  mem[supscr(p)].hh ← empty_field; new_noad ← p;
  end;
```

729. A few more kinds of noads will complete the set: An *under_noad* has its nucleus underlined; an *over_noad* has it overlined. An *accent_noad* places an accent over its nucleus; the accent character appears as $fam(accent_chr(p))$ and $character(accent_chr(p))$. A *vcenter_noad* centers its nucleus vertically with respect to the axis of the formula; in such noads we always have $math_type(nucleus(p)) = sub_box$.

And finally, we have *left_noad* and *right_noad* types, to implement TeX's `\left` and `\right` as well as ε-TeX's `\middle`. The *nucleus* of such noads is replaced by a *delimiter* field; thus, for example, '`\left(`' produces a *left_noad* such that $delimiter(p)$ holds the family and character codes for all left parentheses. A *left_noad* never appears in an mlist except as the first element, and a *right_noad* never appears in an mlist except as the last element; furthermore, we either have both a *left_noad* and a *right_noad*, or neither one is present. The *subscr* and *supscr* fields are always *empty* in a *left_noad* and a *right_noad*.

```
define under_noad = fraction_noad + 1  { type of a noad for underlining }
define over_noad = under_noad + 1  { type of a noad for overlining }
define accent_noad = over_noad + 1  { type of a noad for accented subformulas }
define fixed_acc = 1  { subtype for non growing math accents }
define bottom_acc = 2  { subtype for bottom math accents }
define is_bottom_acc(#) ≡ ((subtype(#) = bottom_acc) ∨ (subtype(#) = bottom_acc + fixed_acc))
define accent_noad_size = 5  { number of mem words in an accent noad }
define accent_chr(#) ≡ # + 4  { the accent_chr field of an accent noad }
define vcenter_noad = accent_noad + 1  { type of a noad for \vcenter }
define left_noad = vcenter_noad + 1  { type of a noad for \left }
define right_noad = left_noad + 1  { type of a noad for \right }
define delimiter ≡ nucleus  { delimiter field in left and right noads }
define middle_noad ≡ 1  { subtype of right noad representing \middle }
define scripts_allowed(#) ≡ (type(#) ≥ ord_noad) ∧ (type(#) < left_noad)
```

730. Math formulas can also contain instructions like `\textstyle` that override TeX's normal style rules. A *style_node* is inserted into the data structure to record such instructions; it is three words long, so it is considered a node instead of a noad. The *subtype* is either *display_style* or *text_style* or *script_style* or *script_script_style*. The second and third words of a *style_node* are not used, but they are present because a *choice_node* is converted to a *style_node*.

TeX uses even numbers 0, 2, 4, 6 to encode the basic styles *display_style*, ..., *script_script_style*, and adds 1 to get the "cramped" versions of these styles. This gives a numerical order that is backwards from the convention of Appendix G in *The TeXbook*; i.e., a smaller style has a larger numerical value.

define *style_node* = *unset_node* + 1 { *type* of a style node }
define *style_node_size* = 3 { number of words in a style node }
define *display_style* = 0 { *subtype* for `\displaystyle` }
define *text_style* = 2 { *subtype* for `\textstyle` }
define *script_style* = 4 { *subtype* for `\scriptstyle` }
define *script_script_style* = 6 { *subtype* for `\scriptscriptstyle` }
define *cramped* = 1 { add this to an uncramped style if you want to cramp it }

function *new_style*(*s* : *small_number*): *pointer*; { create a style node }
var *p*: *pointer*; { the new node }
begin *p* ← *get_node*(*style_node_size*); *type*(*p*) ← *style_node*; *subtype*(*p*) ← *s*; *width*(*p*) ← 0;
depth(*p*) ← 0; { the *width* and *depth* are not used }
new_style ← *p*;
end;

731. Finally, the `\mathchoice` primitive creates a *choice_node*, which has special subfields *display_mlist*, *text_mlist*, *script_mlist*, and *script_script_mlist* pointing to the mlists for each style.

define *choice_node* = *unset_node* + 2 { *type* of a choice node }
define *display_mlist*(#) ≡ *info*(# + 1) { mlist to be used in display style }
define *text_mlist*(#) ≡ *link*(# + 1) { mlist to be used in text style }
define *script_mlist*(#) ≡ *info*(# + 2) { mlist to be used in script style }
define *script_script_mlist*(#) ≡ *link*(# + 2) { mlist to be used in scriptscript style }

function *new_choice*: *pointer*; { create a choice node }
var *p*: *pointer*; { the new node }
begin *p* ← *get_node*(*style_node_size*); *type*(*p*) ← *choice_node*; *subtype*(*p*) ← 0;
{ the *subtype* is not used }
display_mlist(*p*) ← *null*; *text_mlist*(*p*) ← *null*; *script_mlist*(*p*) ← *null*; *script_script_mlist*(*p*) ← *null*;
new_choice ← *p*;
end;

732. Let's consider now the previously unwritten part of *show_node_list* that displays the things that can only be present in mlists; this program illustrates how to access the data structures just defined.

In the context of the following program, p points to a node or noad that should be displayed, and the current string contains the "recursion history" that leads to this point. The recursion history consists of a dot for each outer level in which p is subsidiary to some node, or in which p is subsidiary to the *nucleus* field of some noad; the dot is replaced by '`_`' or '`^`' or '`/`' or '`\`' if p is descended from the *subscr* or *supscr* or *denominator* or *numerator* fields of noads. For example, the current string would be '`.^._/`' if p points to the *ord_noad* for x in the (ridiculous) formula '`$\sqrt{a^{\mathinner{b_{c\over x+y}}}}$`'.

⟨ Cases of *show_node_list* that arise in mlists only 732 ⟩ ≡
style_node: *print_style*(*subtype*(*p*));
choice_node: ⟨ Display choice node *p* 737 ⟩;
ord_noad, *op_noad*, *bin_noad*, *rel_noad*, *open_noad*, *close_noad*, *punct_noad*,
inner_noad, *radical_noad*, *over_noad*, *under_noad*, *vcenter_noad*, *accent_noad*, *left_noad*, *right_noad*:
⟨ Display normal noad *p* 738 ⟩;
fraction_noad: ⟨ Display fraction noad *p* 739 ⟩;

This code is used in section 209.

733. Here are some simple routines used in the display of noads.

⟨ Declare procedures needed for displaying the elements of mlists 733 ⟩ ≡
procedure *print_fam_and_char*(*p* : *pointer*); { prints family and character }
var *c*: *integer*;
begin *print_esc*("fam"); *print_int*(*fam*(*p*) **mod** ″100); *print_char*("␣");
c ← (*cast_to_ushort*(*character*(*p*)) + ((*plane_and_fam_field*(*p*) **div** ″100) ∗ ″10000));
if *c* < ″10000 **then** *print_ASCII*(*c*)
else *print_char*(*c*); { non-Plane 0 Unicodes can't be sent through *print_ASCII* }
end;
procedure *print_delimiter*(*p* : *pointer*); { prints a delimiter as 24-bit hex value }
var *a*: *integer*; { accumulator }
begin *a* ← *small_fam*(*p*) ∗ 256 + *qo*(*small_char*(*p*));
a ← *a* ∗ ″1000 + *large_fam*(*p*) ∗ 256 + *qo*(*large_char*(*p*));
if *a* < 0 **then** *print_int*(*a*) { this should never happen }
else *print_hex*(*a*);
end;

See also sections 734 and 736.

This code is used in section 205.

734. The next subroutine will descend to another level of recursion when a subsidiary mlist needs to be displayed. The parameter c indicates what character is to become part of the recursion history. An empty mlist is distinguished from a field with $math_type(p) = empty$, because these are not equivalent (as explained above).

```
⟨ Declare procedures needed for displaying the elements of mlists 733 ⟩ +≡
procedure show_info; forward;    { show_node_list(info(temp_ptr)) }
procedure print_subsidiary_data(p : pointer; c : ASCII_code);  { display a noad field }
  begin if cur_length ≥ depth_threshold then
    begin if math_type(p) ≠ empty then print("␣[]");
    end
  else begin append_char(c);  { include c in the recursion history }
    temp_ptr ← p;  { prepare for show_info if recursion is needed }
    case math_type(p) of
    math_char: begin print_ln; print_current_string; print_fam_and_char(p);
      end;
    sub_box: show_info;  { recursive call }
    sub_mlist: if info(p) = null then
        begin print_ln; print_current_string; print("{}");
        end
      else show_info;  { recursive call }
    othercases do_nothing  { empty }
    endcases;
    flush_char;  { remove c from the recursion history }
    end;
  end;
```

735. The inelegant introduction of *show_info* in the code above seems better than the alternative of using Pascal's strange *forward* declaration for a procedure with parameters. The Pascal convention about dropping parameters from a post-*forward* procedure is, frankly, so intolerable to the author of TeX that he would rather stoop to communication via a global temporary variable. (A similar stoopidity occurred with respect to *hlist_out* and *vlist_out* above, and it will occur with respect to *mlist_to_hlist* below.)

```
procedure show_info;  { the reader will kindly forgive this }
  begin show_node_list(info(temp_ptr));
  end;
```

736.

```
⟨ Declare procedures needed for displaying the elements of mlists 733 ⟩ +≡
procedure print_style(c : integer);
  begin case c div 2 of
  0: print_esc("displaystyle");  { display_style = 0 }
  1: print_esc("textstyle");  { text_style = 2 }
  2: print_esc("scriptstyle");  { script_style = 4 }
  3: print_esc("scriptscriptstyle");  { script_script_style = 6 }
  othercases print("Unknown␣style!")
  endcases;
  end;
```

737. ⟨ Display choice node p 737 ⟩ ≡

```
begin print_esc("mathchoice"); append_char("D"); show_node_list(display_mlist(p)); flush_char;
append_char("T"); show_node_list(text_mlist(p)); flush_char; append_char("S");
show_node_list(script_mlist(p)); flush_char; append_char("s"); show_node_list(script_script_mlist(p));
flush_char;
end
```

This code is used in section 732.

738. ⟨ Display normal noad p 738 ⟩ ≡

```
begin case type(p) of
ord_noad: print_esc("mathord");
op_noad: print_esc("mathop");
bin_noad: print_esc("mathbin");
rel_noad: print_esc("mathrel");
open_noad: print_esc("mathopen");
close_noad: print_esc("mathclose");
punct_noad: print_esc("mathpunct");
inner_noad: print_esc("mathinner");
over_noad: print_esc("overline");
under_noad: print_esc("underline");
vcenter_noad: print_esc("vcenter");
radical_noad: begin print_esc("radical"); print_delimiter(left_delimiter(p));
  end;
accent_noad: begin print_esc("accent"); print_fam_and_char(accent_chr(p));
  end;
left_noad: begin print_esc("left"); print_delimiter(delimiter(p));
  end;
right_noad: begin if subtype(p) = normal then print_esc("right")
  else print_esc("middle");
  print_delimiter(delimiter(p));
  end;
end;
if type(p) < left_noad then
  begin if subtype(p) ≠ normal then
    if subtype(p) = limits then print_esc("limits")
    else print_esc("nolimits");
  print_subsidiary_data(nucleus(p), ".");
  end;
print_subsidiary_data(supscr(p), "^"); print_subsidiary_data(subscr(p), "_");
end
```

This code is used in section 732.

739. ⟨ Display fraction noad p 739 ⟩ ≡

```
begin print_esc("fraction, thickness "); if thickness(p) = default_code then print("= default")
else print_scaled(thickness(p)); if (small_fam(left_delimiter(p)) ≠ 0) ∨ (small_char(left_delimiter(p)) ≠ min_quarterword) ∨
    (large_fam(left_delimiter(p)) ≠ 0) ∨ (large_char(left_delimiter(p)) ≠ min_quarterword) then
begin print(", left-delimiter "); print_delimiter(left_delimiter(p));
end; if (small_fam(right_delimiter(p)) ≠ 0) ∨ (small_char(right_delimiter(p)) ≠ min_quarterword) ∨
    (large_fam(right_delimiter(p)) ≠ 0) ∨ (large_char(right_delimiter(p)) ≠ min_quarterword) then
  begin print(", right-delimiter "); print_delimiter(right_delimiter(p));
  end; print_subsidiary_data(numerator(p), "\"); print_subsidiary_data(denominator(p), "/");
end
```

This code is used in section 732.

740. That which can be displayed can also be destroyed.

⟨ Cases of *flush_node_list* that arise in mlists only 740 ⟩ ≡

```
style_node: begin free_node(p, style_node_size); goto done;
  end;
choice_node: begin flush_node_list(display_mlist(p)); flush_node_list(text_mlist(p));
  flush_node_list(script_mlist(p)); flush_node_list(script_script_mlist(p)); free_node(p, style_node_size);
  goto done;
  end;
ord_noad, op_noad, bin_noad, rel_noad, open_noad, close_noad, punct_noad, inner_noad, radical_noad,
    over_noad, under_noad, vcenter_noad, accent_noad:
  begin if math_type(nucleus(p)) ≥ sub_box then flush_node_list(info(nucleus(p)));
  if math_type(supscr(p)) ≥ sub_box then flush_node_list(info(supscr(p)));
  if math_type(subscr(p)) ≥ sub_box then flush_node_list(info(subscr(p)));
  if type(p) = radical_noad then free_node(p, radical_noad_size)
  else if type(p) = accent_noad then free_node(p, accent_noad_size)
    else free_node(p, noad_size);
  goto done;
  end;
left_noad, right_noad: begin free_node(p, noad_size); goto done;
  end;
fraction_noad: begin flush_node_list(info(numerator(p))); flush_node_list(info(denominator(p)));
  free_node(p, fraction_noad_size); goto done;
  end;
```

This code is used in section 228.

741. Subroutines for math mode. In order to convert mlists to hlists, i.e., noads to nodes, we need several subroutines that are conveniently dealt with now.

Let us first introduce the macros that make it easy to get at the parameters and other font information. A size code, which is a multiple of 16, is added to a family number to get an index into the table of internal font numbers for each combination of family and size. (Be alert: Size codes get larger as the type gets smaller.)

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_size(s : integer);
  begin if s = text_size then print_esc("textfont")
  else if s = script_size then print_esc("scriptfont")
    else print_esc("scriptscriptfont");
  end;
```

742. Before an mlist is converted to an hlist, TEX makes sure that the fonts in family 2 have enough parameters to be math-symbol fonts, and that the fonts in family 3 have enough parameters to be math-extension fonts. The math-symbol parameters are referred to by using the following macros, which take a size code as their parameter; for example, *num1*(*cur_size*) gives the value of the *num1* parameter for the current size.

NB: the access functions here must all put the font # into /f/ for mathsy().

The accessors are defined with *define_mathsy_accessor*(*NAME*)(*fontdimen − number*)(*NAME*) because I can't see how to only give the name once, with WEB's limited macro capabilities. This seems a bit ugly, but it works.

define *total_mathsy_params* = 22
{ the following are OpenType MATH constant indices for use with OT math fonts }
define *scriptPercentScaleDown* = 0
define *scriptScriptPercentScaleDown* = 1
define *delimitedSubFormulaMinHeight* = 2
define *displayOperatorMinHeight* = 3
define *mathLeading* = 4
define *firstMathValueRecord* = *mathLeading*
define *axisHeight* = 5
define *accentBaseHeight* = 6
define *flattenedAccentBaseHeight* = 7
define *subscriptShiftDown* = 8
define *subscriptTopMax* = 9
define *subscriptBaselineDropMin* = 10
define *superscriptShiftUp* = 11
define *superscriptShiftUpCramped* = 12
define *superscriptBottomMin* = 13
define *superscriptBaselineDropMax* = 14
define *subSuperscriptGapMin* = 15
define *superscriptBottomMaxWithSubscript* = 16
define *spaceAfterScript* = 17
define *upperLimitGapMin* = 18
define *upperLimitBaselineRiseMin* = 19
define *lowerLimitGapMin* = 20
define *lowerLimitBaselineDropMin* = 21
define *stackTopShiftUp* = 22
define *stackTopDisplayStyleShiftUp* = 23
define *stackBottomShiftDown* = 24
define *stackBottomDisplayStyleShiftDown* = 25
define *stackGapMin* = 26
define *stackDisplayStyleGapMin* = 27
define *stretchStackTopShiftUp* = 28
define *stretchStackBottomShiftDown* = 29
define *stretchStackGapAboveMin* = 30
define *stretchStackGapBelowMin* = 31
define *fractionNumeratorShiftUp* = 32
define *fractionNumeratorDisplayStyleShiftUp* = 33
define *fractionDenominatorShiftDown* = 34
define *fractionDenominatorDisplayStyleShiftDown* = 35
define *fractionNumeratorGapMin* = 36
define *fractionNumDisplayStyleGapMin* = 37
define *fractionRuleThickness* = 38
define *fractionDenominatorGapMin* = 39

```
define fractionDenomDisplayStyleGapMin = 40
define skewedFractionHorizontalGap = 41
define skewedFractionVerticalGap = 42
define overbarVerticalGap = 43
define overbarRuleThickness = 44
define overbarExtraAscender = 45
define underbarVerticalGap = 46
define underbarRuleThickness = 47
define underbarExtraDescender = 48
define radicalVerticalGap = 49
define radicalDisplayStyleVerticalGap = 50
define radicalRuleThickness = 51
define radicalExtraAscender = 52
define radicalKernBeforeDegree = 53
define radicalKernAfterDegree = 54
define lastMathValueRecord = radicalKernAfterDegree
define radicalDegreeBottomRaisePercent = 55
define lastMathConstant = radicalDegreeBottomRaisePercent
define mathsy(#) ≡ font_info[# + param_base[f]].sc
define define_mathsy_end(#) ≡ # ← rval;
    end
define define_mathsy_body(#) ≡
    var f: integer; rval: scaled;
    begin f ← fam_fnt(2 + size_code);
    if is_new_mathfont(f) then rval ← get_native_mathsy_param(f, #)
    else rval ← mathsy(#);
    define_mathsy_end
define define_mathsy_accessor(#) ≡
  function #(size_code : integer): scaled; define_mathsy_body
define_mathsy_accessor(math_x_height)(5)(math_x_height);
define_mathsy_accessor(math_quad)(6)(math_quad); define_mathsy_accessor(num1)(8)(num1);
define_mathsy_accessor(num2)(9)(num2); define_mathsy_accessor(num3)(10)(num3);
define_mathsy_accessor(denom1)(11)(denom1); define_mathsy_accessor(denom2)(12)(denom2);
define_mathsy_accessor(sup1)(13)(sup1); define_mathsy_accessor(sup2)(14)(sup2);
define_mathsy_accessor(sup3)(15)(sup3); define_mathsy_accessor(sub1)(16)(sub1);
define_mathsy_accessor(sub2)(17)(sub2); define_mathsy_accessor(sup_drop)(18)(sup_drop);
define_mathsy_accessor(sub_drop)(19)(sub_drop); define_mathsy_accessor(delim1)(20)(delim1);
define_mathsy_accessor(delim2)(21)(delim2); define_mathsy_accessor(axis_height)(22)(axis_height);
```

743. The math-extension parameters have similar macros, but the size code is omitted (since it is always *cur_size* when we refer to such parameters).

```
define total_mathex_params = 13
define mathex(#) ≡ font_info[# + param_base[f]].sc
define define_mathex_end(#) ≡ # ← rval;
    end
define define_mathex_body(#) ≡
    var f: integer; rval: scaled;
    begin f ← fam_fnt(3 + cur_size);
    if is_new_mathfont(f) then rval ← get_native_mathex_param(f, #)
    else rval ← mathex(#);
    define_mathex_end
define define_mathex_accessor(#) ≡
  function #: scaled; define_mathex_body
define_mathex_accessor(default_rule_thickness)(8)(default_rule_thickness);
define_mathex_accessor(big_op_spacing1)(9)(big_op_spacing1);
define_mathex_accessor(big_op_spacing2)(10)(big_op_spacing2);
define_mathex_accessor(big_op_spacing3)(11)(big_op_spacing3);
define_mathex_accessor(big_op_spacing4)(12)(big_op_spacing4);
define_mathex_accessor(big_op_spacing5)(13)(big_op_spacing5);
```

744. Native font support requires these additional subroutines.

new_native_word_node creates the node, but does not actually set its metrics; call *set_native_metrics*(*node*) if that is required.

```
⟨ Declare subroutines for new_character 616 ⟩ +≡
function new_native_word_node(f : internal_font_number; n : integer): pointer;
  var l: integer; q: pointer;
  begin l ← native_node_size+(n*sizeof(UTF16_code)+sizeof(memory_word)−1)div sizeof(memory_word);
  q ← get_node(l); type(q) ← whatsit_node;
  if XeTeX_generate_actual_text_en then subtype(q) ← native_word_node_AT
  else subtype(q) ← native_word_node;
  native_size(q) ← l; native_font(q) ← f; native_length(q) ← n; native_glyph_count(q) ← 0;
  native_glyph_info_ptr(q) ← null_ptr; new_native_word_node ← q;
  end;
function new_native_character(f : internal_font_number; c : UnicodeScalar): pointer;
  var p: pointer; i, len: integer;
  begin if font_mapping[f] ≠ 0 then
    begin if c > "FFFF then
      begin str_room(2); append_char((c − "10000) div 1024 + "D800);
      append_char((c − "10000) mod 1024 + "DC00);
      end
    else begin str_room(1); append_char(c);
      end;
    len ← apply_mapping(font_mapping[f], addressof(str_pool[str_start_macro(str_ptr)]), cur_length);
    pool_ptr ← str_start_macro(str_ptr); { flush the string, as we'll be using the mapped text instead }
    i ← 0;
    while i < len do
      begin if (mapped_text[i] ≥ "D800) ∧ (mapped_text[i] < "DC00) then
        begin c ← (mapped_text[i] − "D800) * 1024 + mapped_text[i + 1] − "DC00 + "10000;
        if map_char_to_glyph(f, c) = 0 then
          begin char_warning(f, c);
          end;
        i ← i + 2;
        end
      else begin if map_char_to_glyph(f, mapped_text[i]) = 0 then
          begin char_warning(f, mapped_text[i]);
          end;
        i ← i + 1;
        end;
      end;
    p ← new_native_word_node(f, len);
    for i ← 0 to len − 1 do
      begin set_native_char(p, i, mapped_text[i]);
      end
    end
  else begin if tracing_lost_chars > 0 then
      if map_char_to_glyph(f, c) = 0 then
        begin char_warning(f, c);
        end;
    p ← get_node(native_node_size + 1); type(p) ← whatsit_node; subtype(p) ← native_word_node;
    native_size(p) ← native_node_size + 1; native_glyph_count(p) ← 0; native_glyph_info_ptr(p) ← null_ptr;
    native_font(p) ← f;
    if c > "FFFF then
```

```
      begin native_length(p) ← 2; set_native_char(p, 0, (c − "10000) div 1024 + "D800);
      set_native_char(p, 1, (c − "10000) mod 1024 + "DC00);
      end
    else begin native_length(p) ← 1; set_native_char(p, 0, c);
      end;
    end;
  set_native_metrics(p, XeTeX_use_glyph_metrics); new_native_character ← p;
  end;
procedure font_feature_warning(featureNameP : void_pointer; featLen : integer;
        settingNameP : void_pointer; setLen : integer);
  var i: integer;
  begin begin_diagnostic; print_nl("Unknown␣");
  if setLen > 0 then
    begin print("selector␣`"); print_utf8_str(settingNameP, setLen); print("´␣for␣");
    end;
  print("feature␣`"); print_utf8_str(featureNameP, featLen); print("´␣in␣font␣`"); i ← 1;
  while ord(name_of_file[i]) ≠ 0 do
    begin print_visible_char(name_of_file[i]); { this is already UTF-8 }
    incr(i);
    end;
  print("´."); end_diagnostic(false);
  end;
procedure font_mapping_warning(mappingNameP : void_pointer; mappingNameLen : integer;
        warningType : integer); { 0: just logging; 1: file not found; 2: can't load }
  var i: integer;
  begin begin_diagnostic;
  if warningType = 0 then print_nl("Loaded␣mapping␣`")
  else print_nl("Font␣mapping␣`");
  print_utf8_str(mappingNameP, mappingNameLen); print("´␣for␣font␣`"); i ← 1;
  while ord(name_of_file[i]) ≠ 0 do
    begin print_visible_char(name_of_file[i]); { this is already UTF-8 }
    incr(i);
    end;
  case warningType of
  1: print("´␣not␣found.");
  2: begin print("´␣not␣usable;"); print_nl("bad␣mapping␣file␣or␣incorrect␣mapping␣type.");
    end;
  othercases print("´.")
  endcases; end_diagnostic(false);
  end;
procedure graphite_warning;
  var i: integer;
  begin begin_diagnostic; print_nl("Font␣`"); i ← 1;
  while ord(name_of_file[i]) ≠ 0 do
    begin print_visible_char(name_of_file[i]); { this is already UTF-8 }
    incr(i);
    end;
  print("´␣does␣not␣support␣Graphite.␣Trying␣OpenType␣layout␣instead."); end_diagnostic(false);
  end;
function load_native_font(u : pointer; nom, aire : str_number; s : scaled): internal_font_number;
  label done;
  const first_math_fontdimen = 10;
```

```
var k, num_font_dimens: integer; font_engine: void_pointer;
      { really an CFDictionaryRef or XeTeXLayoutEngine }
  actual_size: scaled; { s converted to real size, if it was negative }
  p: pointer; { for temporary native_char node we'll create }
  ascent, descent, font_slant, x_ht, cap_ht: scaled; f: internal_font_number; full_name: str_number;
begin { on entry here, the full name is packed into name_of_file in UTF8 form }
load_native_font ← null_font; font_engine ← find_native_font(name_of_file + 1, s);
if font_engine = 0 then goto done;
if s ≥ 0 then actual_size ← s
else begin if (s ≠ −1000) then actual_size ← xn_over_d(loaded_font_design_size, −s, 1000)
  else actual_size ← loaded_font_design_size;
  end; { look again to see if the font is already loaded, now that we know its canonical name }
str_room(name_length);
for k ← 1 to name_length do append_char(name_of_file[k]);
full_name ← make_string; { not slow_make_string because we'll flush it if the font was already loaded }
for f ← font_base + 1 to font_ptr do
  if (font_area[f] = native_font_type_flag) ∧ str_eq_str(font_name[f], full_name) ∧ (font_size[f] = actual_size)
      then
    begin release_font_engine(font_engine, native_font_type_flag); flush_string; load_native_font ← f;
    goto done;
    end;
if (native_font_type_flag = otgr_font_flag) ∧ isOpenTypeMathFont(font_engine) then
  num_font_dimens ← first_math_fontdimen + lastMathConstant
else num_font_dimens ← 8;
if (font_ptr = font_max) ∨ (fmem_ptr + num_font_dimens > font_mem_size) then
  begin ⟨ Apologize for not loading the font, goto done 602 ⟩;
  end; { we've found a valid installed font, and have room }
incr(font_ptr); font_area[font_ptr] ← native_font_type_flag;
    { set by find_native_font to either aat_font_flag or ot_font_flag }
  { store the canonical name }
font_name[font_ptr] ← full_name; font_check[font_ptr].b0 ← 0; font_check[font_ptr].b1 ← 0;
font_check[font_ptr].b2 ← 0; font_check[font_ptr].b3 ← 0; font_glue[font_ptr] ← null;
font_dsize[font_ptr] ← loaded_font_design_size; font_size[font_ptr] ← actual_size;
if (native_font_type_flag = aat_font_flag) then
  begin aat_get_font_metrics(font_engine, addressof(ascent), addressof(descent), addressof(x_ht),
      addressof(cap_ht), addressof(font_slant))
  end
else begin ot_get_font_metrics(font_engine, addressof(ascent), addressof(descent), addressof(x_ht),
      addressof(cap_ht), addressof(font_slant));
  end;
height_base[font_ptr] ← ascent; depth_base[font_ptr] ← −descent;
font_params[font_ptr] ← num_font_dimens;
    { we add an extra \fontdimen8 ← cap_height; then OT math fonts have a bunch more }
font_bc[font_ptr] ← 0; font_ec[font_ptr] ← 65535; font_used[font_ptr] ← false;
hyphen_char[font_ptr] ← default_hyphen_char; skew_char[font_ptr] ← default_skew_char;
param_base[font_ptr] ← fmem_ptr − 1; font_layout_engine[font_ptr] ← font_engine;
font_mapping[font_ptr] ← 0; { don't use the mapping, if any, when measuring space here }
font_letter_space[font_ptr] ← loaded_font_letter_space;

  { measure the width of the space character and set up font parameters }
p ← new_native_character(font_ptr, "␣"); s ← width(p) + loaded_font_letter_space;
free_node(p, native_size(p)); font_info[fmem_ptr].sc ← font_slant; { slant }
incr(fmem_ptr); font_info[fmem_ptr].sc ← s; { space = width of space character }
```

```
  incr(fmem_ptr); font_info[fmem_ptr].sc ← s div 2;  { space_stretch = 1/2 * space }
  incr(fmem_ptr); font_info[fmem_ptr].sc ← s div 3;  { space_shrink = 1/3 * space }
  incr(fmem_ptr); font_info[fmem_ptr].sc ← x_ht;  { x_height }
  incr(fmem_ptr); font_info[fmem_ptr].sc ← font_size[font_ptr];  { quad = font size }
  incr(fmem_ptr); font_info[fmem_ptr].sc ← s div 3;  { extra_space = 1/3 * space }
  incr(fmem_ptr); font_info[fmem_ptr].sc ← cap_ht;  { cap_height }
  incr(fmem_ptr);
  if num_font_dimens = first_math_fontdimen + lastMathConstant then
    begin font_info[fmem_ptr].int ← num_font_dimens;
        { \fontdimen9 ← number of assigned fontdimens }
    incr(fmem_ptr);
    for k ← 0 to lastMathConstant do
      begin font_info[fmem_ptr].sc ← get_ot_math_constant(font_ptr, k); incr(fmem_ptr);
      end;
    end;
  font_mapping[font_ptr] ← loaded_font_mapping; font_flags[font_ptr] ← loaded_font_flags;
  load_native_font ← font_ptr;
done: end;
procedure do_locale_linebreaks(s : integer; len : integer);
  var offs, prevOffs, i: integer; use_penalty, use_skip: boolean;
  begin if (XeTeX_linebreak_locale = 0) ∨ (len = 1) then
    begin link(tail) ← new_native_word_node(main_f, len); tail ← link(tail);
    for i ← 0 to len − 1 do set_native_char(tail, i, native_text[s + i]);
    set_native_metrics(tail, XeTeX_use_glyph_metrics);
    end
  else begin use_skip ← XeTeX_linebreak_skip ≠ zero_glue;
    use_penalty ← XeTeX_linebreak_penalty ≠ 0 ∨ ¬use_skip;
    linebreak_start(main_f, XeTeX_linebreak_locale, native_text + s, len); offs ← 0;
    repeat prevOffs ← offs; offs ← linebreak_next;
      if offs > 0 then
        begin if prevOffs ≠ 0 then
          begin if use_penalty then tail_append(new_penalty(XeTeX_linebreak_penalty));
          if use_skip then tail_append(new_param_glue(XeTeX_linebreak_skip_code));
          end;
        link(tail) ← new_native_word_node(main_f, offs − prevOffs); tail ← link(tail);
        for i ← prevOffs to offs − 1 do set_native_char(tail, i − prevOffs, native_text[s + i]);
        set_native_metrics(tail, XeTeX_use_glyph_metrics);
        end;
    until offs < 0;
    end
  end;
procedure bad_utf8_warning;
  begin begin_diagnostic; print_nl("Invalid␣UTF-8␣byte␣or␣sequence");
  if terminal_input then print("␣in␣terminal␣input")
  else begin print("␣at␣line␣"); print_int(line);
    end;
  print("␣replaced␣by␣U+FFFD."); end_diagnostic(false);
  end;
function get_input_normalization_state: integer;
  begin if eqtb = nil then get_input_normalization_state ← 0  { may be called before eqtb is initialized }
  else get_input_normalization_state ← XeTeX_input_normalization_state;
  end;
```

```
function get_tracing_fonts_state: integer;
  begin get_tracing_fonts_state ← XeTeX_tracing_fonts_state;
  end;
```

745. We also need to compute the change in style between mlists and their subsidiaries. The following macros define the subsidiary style for an overlined nucleus (*cramped_style*), for a subscript or a superscript (*sub_style* or *sup_style*), or for a numerator or denominator (*num_style* or *denom_style*).

```
define cramped_style(#) ≡ 2 * (# div 2) + cramped    { cramp the style }
define sub_style(#) ≡ 2 * (# div 4) + script_style + cramped    { smaller and cramped }
define sup_style(#) ≡ 2 * (# div 4) + script_style + (# mod 2)    { smaller }
define num_style(#) ≡ # + 2 − 2 * (# div 6)    { smaller unless already script-script }
define denom_style(#) ≡ 2 * (# div 2) + cramped + 2 − 2 * (# div 6)    { smaller, cramped }
```

746. When the style changes, the following piece of program computes associated information:

```
⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩ ≡
  begin if cur_style < script_style then cur_size ← text_size
  else cur_size ← script_size * ((cur_style − text_style) div 2);
  cur_mu ← x_over_n(math_quad(cur_size), 18);
  end
```

This code is used in sections 763, 769, 770, 773, 798, 805, 805, 808, 810, and 811.

747. Here is a function that returns a pointer to a rule node having a given thickness t. The rule will extend horizontally to the boundary of the vlist that eventually contains it.

```
function fraction_rule(t : scaled): pointer;    { construct the bar for a fraction }
  var p: pointer;    { the new node }
  begin p ← new_rule; height(p) ← t; depth(p) ← 0; fraction_rule ← p;
  end;
```

748. The *overbar* function returns a pointer to a vlist box that consists of a given box b, above which has been placed a kern of height k under a fraction rule of thickness t under additional space of height t.

```
function overbar(b : pointer; k, t : scaled): pointer;
  var p, q: pointer;    { nodes being constructed }
  begin p ← new_kern(k); link(p) ← b; q ← fraction_rule(t); link(q) ← p; p ← new_kern(t); link(p) ← q;
  overbar ← vpack(p, natural);
  end;
```

749. The *var_delimiter* function, which finds or constructs a sufficiently large delimiter, is the most interesting of the auxiliary functions that currently concern us. Given a pointer d to a delimiter field in some noad, together with a size code s and a vertical distance v, this function returns a pointer to a box that contains the smallest variant of d whose height plus depth is v or more. (And if no variant is large enough, it returns the largest available variant.) In particular, this routine will construct arbitrarily large delimiters from extensible components, if d leads to such characters.

The value returned is a box whose *shift_amount* has been set so that the box is vertically centered with respect to the axis in the given size. If a built-up symbol is returned, the height of the box before shifting will be the height of its topmost component.

```
⟨ Declare subprocedures for var_delimiter 752 ⟩
procedure stack_glyph_into_box(b : pointer; f : internal_font_number; g : integer);
  var p, q: pointer;
  begin p ← get_node(glyph_node_size); type(p) ← whatsit_node; subtype(p) ← glyph_node;
  native_font(p) ← f; native_glyph(p) ← g; set_native_glyph_metrics(p, 1);
  if type(b) = hlist_node then
    begin q ← list_ptr(b);
    if q = null then list_ptr(b) ← p
    else begin while link(q) ≠ null do q ← link(q);
      link(q) ← p;
      if (height(b) < height(p)) then height(b) ← height(p);
      if (depth(b) < depth(p)) then depth(b) ← depth(p);
      end;
    end
  else begin link(p) ← list_ptr(b); list_ptr(b) ← p; height(b) ← height(p);
    if (width(b) < width(p)) then width(b) ← width(p);
    end;
  end;
procedure stack_glue_into_box(b : pointer; min, max : scaled);
  var p, q: pointer;
  begin q ← new_spec(zero_glue); width(q) ← min; stretch(q) ← max − min; p ← new_glue(q);
  if type(b) = hlist_node then
    begin q ← list_ptr(b);
    if q = null then list_ptr(b) ← p
    else begin while link(q) ≠ null do q ← link(q);
      link(q) ← p;
      end;
    end
  else begin link(p) ← list_ptr(b); list_ptr(b) ← p; height(b) ← height(p); width(b) ← width(p);
    end;
  end;
function build_opentype_assembly(f : internal_font_number; a : void_pointer; s : scaled; horiz : boolean):
        pointer;
        { return a box with height/width at least s, using font f, with glyph assembly info from a }
  var b: pointer;  { the box we're constructing }
    n: integer;  { the number of repetitions of each extender }
    i, j: integer;  { indexes }
    g: integer;  { glyph code }
    p: pointer;  { temp pointer }
    s_max, o, oo, prev_o, min_o: scaled; no_extenders: boolean; nat, str: scaled;  { natural size, stretch }
  begin b ← new_null_box;
  if horiz then type(b) ← hlist_node
  else type(b) ← vlist_node;  { figure out how many repeats of each extender to use }
```

```
n ← −1; no_extenders ← true; min_o ← ot_min_connector_overlap(f);
repeat n ← n + 1; { calc max possible size with this number of extenders }
  s_max ← 0; prev_o ← 0;
  for i ← 0 to ot_part_count(a) − 1 do
    begin if ot_part_is_extender(a, i) then
      begin no_extenders ← false;
      for j ← 1 to n do
        begin o ← ot_part_start_connector(f, a, i);
        if min_o < o then o ← min_o;
        if prev_o < o then o ← prev_o;
        s_max ← s_max − o + ot_part_full_advance(f, a, i); prev_o ← ot_part_end_connector(f, a, i);
        end
      end
    else begin o ← ot_part_start_connector(f, a, i);
      if min_o < o then o ← min_o;
      if prev_o < o then o ← prev_o;
      s_max ← s_max − o + ot_part_full_advance(f, a, i); prev_o ← ot_part_end_connector(f, a, i);
      end;
    end;
until (s_max ≥ s) ∨ no_extenders;
      { assemble box using n copies of each extender, with appropriate glue wherever an overlap occurs }
prev_o ← 0;
for i ← 0 to ot_part_count(a) − 1 do
  begin if ot_part_is_extender(a, i) then
    begin for j ← 1 to n do
      begin o ← ot_part_start_connector(f, a, i);
      if prev_o < o then o ← prev_o;
      oo ← o; { max overlap }
      if min_o < o then o ← min_o;
      if oo > 0 then stack_glue_into_box(b, −oo, −o);
      g ← ot_part_glyph(a, i); stack_glyph_into_box(b, f, g); prev_o ← ot_part_end_connector(f, a, i);
      end
    end
  else begin o ← ot_part_start_connector(f, a, i);
    if prev_o < o then o ← prev_o;
    oo ← o; { max overlap }
    if min_o < o then o ← min_o;
    if oo > 0 then stack_glue_into_box(b, −oo, −o);
    g ← ot_part_glyph(a, i); stack_glyph_into_box(b, f, g); prev_o ← ot_part_end_connector(f, a, i);
    end;
  end; { find natural size and total stretch of the box }
p ← list_ptr(b); nat ← 0; str ← 0;
while p ≠ null do
  begin if type(p) = whatsit_node then
    begin if horiz then nat ← nat + width(p)
    else nat ← nat + height(p) + depth(p);
    end
  else if type(p) = glue_node then
      begin nat ← nat + width(glue_ptr(p)); str ← str + stretch(glue_ptr(p));
      end;
  p ← link(p);
  end; { set glue so as to stretch the connections if needed }
```

```
  o ← 0;
  if (s > nat) ∧ (str > 0) then
    begin o ← (s − nat);  { don't stretch more than str }
    if (o > str) then o ← str;
    glue_order(b) ← normal; glue_sign(b) ← stretching; glue_set(b) ← unfloat(o/str);
    if horiz then width(b) ← nat + round(str ∗ float(glue_set(b)))
    else height(b) ← nat + round(str ∗ float(glue_set(b)));
    end
  else if horiz then width(b) ← nat
    else height(b) ← nat;
  build_opentype_assembly ← b;
  end;
function var_delimiter(d : pointer; s : integer; v : scaled): pointer;
  label found, continue;
  var b: pointer;  { the box that will be constructed }
    ot_assembly_ptr: void_pointer; f, g: internal_font_number;  { best-so-far and tentative font codes }
    c, x, y: quarterword;  { best-so-far and tentative character codes }
    m, n: integer;  { the number of extensible pieces }
    u: scaled;  { height-plus-depth of a tentative character }
    w: scaled;  { largest height-plus-depth so far }
    q: four_quarters;  { character info }
    hd: eight_bits;  { height-depth byte }
    r: four_quarters;  { extensible pieces }
    z: integer;  { runs through font family members }
    large_attempt: boolean;  { are we trying the "large" variant? }
  begin f ← null_font; w ← 0; large_attempt ← false; z ← small_fam(d); x ← small_char(d);
  ot_assembly_ptr ← nil;
  loop begin ⟨ Look at the variants of (z, x); set f and c whenever a better character is found; goto
        found as soon as a large enough variant is encountered 750 ⟩;
    if large_attempt then goto found;  { there were none large enough }
    large_attempt ← true; z ← large_fam(d); x ← large_char(d);
    end;
found: if f ≠ null_font then
    begin if ¬is_ot_font(f) then ⟨ Make variable b point to a box for (f, c) 753 ⟩
    else begin  { for OT fonts, c is the glyph ID to use }
      if ot_assembly_ptr ≠ nil then b ← build_opentype_assembly(f, ot_assembly_ptr, v, 0)
      else begin b ← new_null_box; type(b) ← vlist_node; list_ptr(b) ← get_node(glyph_node_size);
        type(list_ptr(b)) ← whatsit_node; subtype(list_ptr(b)) ← glyph_node; native_font(list_ptr(b)) ← f;
        native_glyph(list_ptr(b)) ← c; set_native_glyph_metrics(list_ptr(b), 1);
        width(b) ← width(list_ptr(b)); height(b) ← height(list_ptr(b)); depth(b) ← depth(list_ptr(b));
        end
      end
    end
  else begin b ← new_null_box; width(b) ← null_delimiter_space;
      { use this width if no delimiter was found }
    end;
  shift_amount(b) ← half(height(b) − depth(b)) − axis_height(s); free_ot_assembly(ot_assembly_ptr);
  var_delimiter ← b;
  end;
```

750. The search process is complicated slightly by the facts that some of the characters might not be present in some of the fonts, and they might not be probed in increasing order of height.

```
⟨ Look at the variants of (z, x); set f and c whenever a better character is found; goto found as soon as a
      large enough variant is encountered 750 ⟩ ≡
  if (z ≠ 0) ∨ (x ≠ min_quarterword) then
    begin z ← z + s + script_size;
    repeat z ← z − script_size; g ← fam_fnt(z);
      if g ≠ null_font then ⟨ Look at the list of characters starting with x in font g; set f and c whenever
          a better character is found; goto found as soon as a large enough variant is encountered 751 ⟩;
    until z < script_size;
    end
```

This code is used in section 749.

751.

```
⟨ Look at the list of characters starting with x in font g; set f and c whenever a better character is
      found; goto found as soon as a large enough variant is encountered 751 ⟩ ≡
  if is_ot_font(g) then
    begin x ← map_char_to_glyph(g, x); f ← g; c ← x; w ← 0; n ← 0;
    repeat y ← get_ot_math_variant(g, x, n, addressof(u), 0);
      if u > w then
        begin c ← y; w ← u;
        if u ≥ v then goto found;
        end;
      n ← n + 1;
    until u < 0; { if we get here, then we didn't find a big enough glyph; check if the char is extensible }
    ot_assembly_ptr ← get_ot_assembly_ptr(g, x, 0);
    if ot_assembly_ptr ≠ nil then goto found;
    end
  else begin y ← x;
    if (qo(y) ≥ font_bc[g]) ∧ (qo(y) ≤ font_ec[g]) then
      begin continue: q ← char_info(g)(y);
      if char_exists(q) then
        begin if char_tag(q) = ext_tag then
          begin f ← g; c ← y; goto found;
          end;
        hd ← height_depth(q); u ← char_height(g)(hd) + char_depth(g)(hd);
        if u > w then
          begin f ← g; c ← y; w ← u;
          if u ≥ v then goto found;
          end;
        if char_tag(q) = list_tag then
          begin y ← rem_byte(q); goto continue;
          end;
        end;
      end;
    end
```

This code is used in section 750.

752. Here is a subroutine that creates a new box, whose list contains a single character, and whose width includes the italic correction for that character. The height or depth of the box will be negative, if the height or depth of the character is negative; thus, this routine may deliver a slightly different result than *hpack* would produce.

```
⟨ Declare subprocedures for var_delimiter 752 ⟩ ≡
function char_box(f : internal_font_number; c : integer): pointer;
  var q: four_quarters; hd: eight_bits;  { height_depth byte }
    b, p: pointer;  { the new box and its character node }
  begin if is_native_font(f) then
    begin b ← new_null_box; p ← new_native_character(f, c); list_ptr(b) ← p; height(b) ← height(p);
    width(b) ← width(p);
    if depth(p) < 0 then depth(b) ← 0
    else depth(b) ← depth(p);
    end
  else begin q ← char_info(f)(c); hd ← height_depth(q); b ← new_null_box;
    width(b) ← char_width(f)(q) + char_italic(f)(q); height(b) ← char_height(f)(hd);
    depth(b) ← char_depth(f)(hd); p ← get_avail; character(p) ← c; font(p) ← f;
    end;
  list_ptr(b) ← p; char_box ← b;
  end;
```

See also sections 754 and 755.

This code is used in section 749.

753. When the following code is executed, $char_tag(q)$ will be equal to *ext_tag* if and only if a built-up symbol is supposed to be returned.

```
⟨ Make variable b point to a box for (f, c) 753 ⟩ ≡
  if char_tag(q) = ext_tag then
    ⟨ Construct an extensible character in a new box b, using recipe rem_byte(q) and font f 756 ⟩
  else b ← char_box(f, c)
```

This code is used in section 749.

754. When we build an extensible character, it's handy to have the following subroutine, which puts a given character on top of the characters already in box b:

```
⟨ Declare subprocedures for var_delimiter 752 ⟩ +≡
procedure stack_into_box(b : pointer; f : internal_font_number; c : quarterword);
  var p: pointer;  { new node placed into b }
  begin p ← char_box(f, c); link(p) ← list_ptr(b); list_ptr(b) ← p; height(b) ← height(p);
  end;
```

755. Another handy subroutine computes the height plus depth of a given character:

```
⟨ Declare subprocedures for var_delimiter 752 ⟩ +≡
function height_plus_depth(f : internal_font_number; c : quarterword): scaled;
  var q: four_quarters; hd: eight_bits;  { height_depth byte }
  begin q ← char_info(f)(c); hd ← height_depth(q);
  height_plus_depth ← char_height(f)(hd) + char_depth(f)(hd);
  end;
```

756. ⟨ Construct an extensible character in a new box b, using recipe $rem_byte(q)$ and font f 756 ⟩ ≡

```
begin b ← new_null_box; type(b) ← vlist_node; r ← font_info[exten_base[f] + rem_byte(q)].qqqq;
⟨ Compute the minimum suitable height, w, and the corresponding number of extension steps, n; also set
    width(b) 757 ⟩;
c ← ext_bot(r);
if c ≠ min_quarterword then stack_into_box(b, f, c);
c ← ext_rep(r);
for m ← 1 to n do stack_into_box(b, f, c);
c ← ext_mid(r);
if c ≠ min_quarterword then
  begin stack_into_box(b, f, c); c ← ext_rep(r);
  for m ← 1 to n do stack_into_box(b, f, c);
  end;
c ← ext_top(r);
if c ≠ min_quarterword then stack_into_box(b, f, c);
depth(b) ← w − height(b);
end
```

This code is used in section 753.

757. The width of an extensible character is the width of the repeatable module. If this module does not have positive height plus depth, we don't use any copies of it, otherwise we use as few as possible (in groups of two if there is a middle part).

⟨ Compute the minimum suitable height, w, and the corresponding number of extension steps, n; also set $width(b)$ 757 ⟩ ≡

```
c ← ext_rep(r); u ← height_plus_depth(f, c); w ← 0; q ← char_info(f)(c);
width(b) ← char_width(f)(q) + char_italic(f)(q);
c ← ext_bot(r); if c ≠ min_quarterword then w ← w + height_plus_depth(f, c);
c ← ext_mid(r); if c ≠ min_quarterword then w ← w + height_plus_depth(f, c);
c ← ext_top(r); if c ≠ min_quarterword then w ← w + height_plus_depth(f, c);
n ← 0;
if u > 0 then
  while w < v do
    begin w ← w + u; incr(n);
    if ext_mid(r) ≠ min_quarterword then w ← w + u;
    end
```

This code is used in section 756.

758. The next subroutine is much simpler; it is used for numerators and denominators of fractions as well as for displayed operators and their limits above and below. It takes a given box b and changes it so that the new box is centered in a box of width w. The centering is done by putting `\hss` glue at the left and right of the list inside b, then packaging the new box; thus, the actual box might not really be centered, if it already contains infinite glue.

The given box might contain a single character whose italic correction has been added to the width of the box; in this case a compensating kern is inserted.

```
function rebox(b : pointer; w : scaled): pointer;
  var p: pointer;  { temporary register for list manipulation }
    f: internal_font_number;  { font in a one-character box }
    v: scaled;  { width of a character without italic correction }
  begin if (width(b) ≠ w) ∧ (list_ptr(b) ≠ null) then
    begin if type(b) = vlist_node then b ← hpack(b, natural);
    p ← list_ptr(b);
    if (is_char_node(p)) ∧ (link(p) = null) then
      begin f ← font(p); v ← char_width(f)(char_info(f)(character(p)));
      if v ≠ width(b) then link(p) ← new_kern(width(b) − v);
      end;
    free_node(b, box_node_size); b ← new_glue(ss_glue); link(b) ← p;
    while link(p) ≠ null do p ← link(p);
    link(p) ← new_glue(ss_glue); rebox ← hpack(b, w, exactly);
    end
  else begin width(b) ← w; rebox ← b;
    end;
  end;
```

759. Here is a subroutine that creates a new glue specification from another one that is expressed in '`mu`', given the value of the math unit.

```
  define mu_mult(#) ≡ nx_plus_y(n, #, xn_over_d(#, f, ´200000))
function math_glue(g : pointer; m : scaled): pointer;
  var p: pointer;  { the new glue specification }
    n: integer;  { integer part of m }
    f: scaled;  { fraction part of m }
  begin n ← x_over_n(m, ´200000); f ← remainder;
  if f < 0 then
    begin decr(n); f ← f + ´200000;
    end;
  p ← get_node(glue_spec_size); width(p) ← mu_mult(width(g));  { convert mu to pt }
  stretch_order(p) ← stretch_order(g);
  if stretch_order(p) = normal then stretch(p) ← mu_mult(stretch(g))
  else stretch(p) ← stretch(g);
  shrink_order(p) ← shrink_order(g);
  if shrink_order(p) = normal then shrink(p) ← mu_mult(shrink(g))
  else shrink(p) ← shrink(g);
  math_glue ← p;
  end;
```

760. The *math_kern* subroutine removes *mu_glue* from a kern node, given the value of the math unit.

```
procedure math_kern(p : pointer; m : scaled);
  var n: integer;  { integer part of m }
    f: scaled;  { fraction part of m }
  begin if subtype(p) = mu_glue then
    begin n ← x_over_n(m, ´200000); f ← remainder;
    if f < 0 then
      begin decr(n); f ← f + ´200000;
      end;
    width(p) ← mu_mult(width(p)); subtype(p) ← explicit;
    end;
  end;
```

761. Sometimes it is necessary to destroy an mlist. The following subroutine empties the current list, assuming that $abs(mode) = mmode$.

```
procedure flush_math;
  begin flush_node_list(link(head)); flush_node_list(incompleat_noad); link(head) ← null; tail ← head;
  incompleat_noad ← null;
  end;
```

762. Typesetting math formulas. TEX's most important routine for dealing with formulas is called *mlist_to_hlist*. After a formula has been scanned and represented as an mlist, this routine converts it to an hlist that can be placed into a box or incorporated into the text of a paragraph. There are three implicit parameters, passed in global variables: *cur_mlist* points to the first node or noad in the given mlist (and it might be *null*); *cur_style* is a style code; and *mlist_penalties* is *true* if penalty nodes for potential line breaks are to be inserted into the resulting hlist. After *mlist_to_hlist* has acted, *link*(*temp_head*) points to the translated hlist.

Since mlists can be inside mlists, the procedure is recursive. And since this is not part of TEX's inner loop, the program has been written in a manner that stresses compactness over efficiency.

```
⟨ Global variables 13 ⟩ +≡
cur_mlist: pointer; { beginning of mlist to be translated }
cur_style: small_number; { style code at current place in the list }
cur_size: integer; { size code corresponding to cur_style }
cur_mu: scaled; { the math unit width corresponding to cur_size }
mlist_penalties: boolean; { should mlist_to_hlist insert penalties? }
```

763. The recursion in *mlist_to_hlist* is due primarily to a subroutine called *clean_box* that puts a given noad field into a box using a given math style; *mlist_to_hlist* can call *clean_box*, which can call *mlist_to_hlist*.

The box returned by *clean_box* is "clean" in the sense that its *shift_amount* is zero.

```
procedure mlist_to_hlist; forward;
function clean_box(p : pointer; s : small_number): pointer;
  label found;
  var q: pointer; { beginning of a list to be boxed }
    save_style: small_number; { cur_style to be restored }
    x: pointer; { box to be returned }
    r: pointer; { temporary pointer }
  begin case math_type(p) of
  math_char: begin cur_mlist ← new_noad; mem[nucleus(cur_mlist)] ← mem[p];
    end;
  sub_box: begin q ← info(p); goto found;
    end;
  sub_mlist: cur_mlist ← info(p);
  othercases begin q ← new_null_box; goto found;
    end
  endcases;
  save_style ← cur_style; cur_style ← s; mlist_penalties ← false;
  mlist_to_hlist; q ← link(temp_head); { recursive call }
  cur_style ← save_style; { restore the style }
  ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
found: if is_char_node(q) ∨ (q = null) then x ← hpack(q, natural)
  else if (link(q) = null) ∧ (type(q) ≤ vlist_node) ∧ (shift_amount(q) = 0) then x ← q
          { it's already clean }
    else x ← hpack(q, natural);
  ⟨ Simplify a trivial box 764 ⟩;
  clean_box ← x;
  end;
```

764. Here we save memory space in a common case.

```
⟨ Simplify a trivial box 764 ⟩ ≡
  q ← list_ptr(x);
  if is_char_node(q) then
    begin r ← link(q);
    if r ≠ null then
      if link(r) = null then
        if ¬is_char_node(r) then
          if type(r) = kern_node then { unneeded italic correction }
            begin free_node(r, small_node_size); link(q) ← null;
            end;
    end
```

This code is used in section 763.

765. It is convenient to have a procedure that converts a *math_char* field to an "unpacked" form. The *fetch* routine sets *cur_f*, *cur_c*, and *cur_i* to the font code, character code, and character information bytes of a given noad field. It also takes care of issuing error messages for nonexistent characters; in such cases, *char_exists*(*cur_i*) will be *false* after *fetch* has acted, and the field will also have been reset to *empty*.

```
procedure fetch(a : pointer); { unpack the math_char field a }
  begin cur_c ← cast_to_ushort(character(a)); cur_f ← fam_fnt(fam(a) + cur_size);
  cur_c ← cur_c + (plane_and_fam_field(a) div "100) * "10000;
  if cur_f = null_font then ⟨ Complain about an undefined family and set cur_i null 766 ⟩
  else if is_native_font(cur_f) then
      begin cur_i ← null_character;
      end
    else begin if (qo(cur_c) ≥ font_bc[cur_f]) ∧ (qo(cur_c) ≤ font_ec[cur_f]) then
        cur_i ← char_info(cur_f)(cur_c)
      else cur_i ← null_character;
      if ¬(char_exists(cur_i)) then
        begin char_warning(cur_f, qo(cur_c)); math_type(a) ← empty; cur_i ← null_character;
        end;
      end;
  end;
```

766.

```
⟨ Complain about an undefined family and set cur_i null 766 ⟩ ≡
  begin print_err(""); print_size(cur_size); print_char("␣"); print_int(fam(a));
  print("␣is␣undefined␣(character␣"); print_ASCII(qo(cur_c)); print_char(")");
  help4("Somewhere␣in␣the␣math␣formula␣just␣ended,␣you␣used␣the")
  ("stated␣character␣from␣an␣undefined␣font␣family.␣For␣example,")
  ("plain␣TeX␣doesn't␣allow␣\it␣or␣\sl␣in␣subscripts.␣Proceed,")
  ("and␣I'll␣try␣to␣forget␣that␣I␣needed␣that␣character."); error; cur_i ← null_character;
  math_type(a) ← empty;
  end
```

This code is used in section 765.

767. The outputs of *fetch* are placed in global variables.

```
⟨ Global variables 13 ⟩ +≡
cur_f: internal_font_number; { the font field of a math_char }
cur_c: integer; { the character field of a math_char }
cur_i: four_quarters; { the char_info of a math_char, or a lig/kern instruction }
```

768. We need to do a lot of different things, so *mlist_to_hlist* makes two passes over the given mlist.

The first pass does most of the processing: It removes "mu" spacing from glue, it recursively evaluates all subsidiary mlists so that only the top-level mlist remains to be handled, it puts fractions and square roots and such things into boxes, it attaches subscripts and superscripts, and it computes the overall height and depth of the top-level mlist so that the size of delimiters for a *left_noad* and a *right_noad* will be known. The hlist resulting from each noad is recorded in that noad's *new_hlist* field, an integer field that replaces the *nucleus* or *thickness*.

The second pass eliminates all noads and inserts the correct glue and penalties between nodes.

define *new_hlist*(#) ≡ *mem*[*nucleus*(#)].*int* { the translation of an mlist }

769. Here is the overall plan of *mlist_to_hlist*, and the list of its local variables.

define *done_with_noad* = 80 { go here when a noad has been fully translated }
define *done_with_node* = 81 { go here when a node has been fully converted }
define *check_dimensions* = 82 { go here to update *max_h* and *max_d* }
define *delete_q* = 83 { go here to delete *q* and move to the next node }

⟨ Declare math construction procedures 777 ⟩
procedure *mlist_to_hlist*;
label *reswitch*, *check_dimensions*, *done_with_noad*, *done_with_node*, *delete_q*, *done*;
var *mlist*: *pointer*; { beginning of the given list }
penalties: *boolean*; { should penalty nodes be inserted? }
style: *small_number*; { the given style }
save_style: *small_number*; { holds *cur_style* during recursion }
q: *pointer*; { runs through the mlist }
r: *pointer*; { the most recent noad preceding *q* }
r_type: *small_number*; { the *type* of noad *r*, or *op_noad* if *r* = *null* }
t: *small_number*; { the effective *type* of noad *q* during the second pass }
p, *x*, *y*, *z*: *pointer*; { temporary registers for list construction }
pen: *integer*; { a penalty to be inserted }
s: *small_number*; { the size of a noad to be deleted }
max_h, *max_d*: *scaled*; { maximum height and depth of the list translated so far }
delta: *scaled*; { offset between subscript and superscript }
begin *mlist* ← *cur_mlist*; *penalties* ← *mlist_penalties*; *style* ← *cur_style*;
{ tuck global parameters away as local variables }
q ← *mlist*; *r* ← *null*; *r_type* ← *op_noad*; *max_h* ← 0; *max_d* ← 0;
⟨ Set up the values of *cur_size* and *cur_mu*, based on *cur_style* 746 ⟩;
while *q* ≠ *null* **do** ⟨ Process node-or-noad *q* as much as possible in preparation for the second pass of *mlist_to_hlist*, then move to the next item in the mlist 770 ⟩;
⟨ Convert a final *bin_noad* to an *ord_noad* 772 ⟩;
⟨ Make a second pass over the mlist, removing all noads and inserting the proper spacing and penalties 808 ⟩;
end;

770. We use the fact that no character nodes appear in an mlist, hence the field *type*(*q*) is always present.

```
⟨ Process node-or-noad q as much as possible in preparation for the second pass of mlist_to_hlist, then move
      to the next item in the mlist 770 ⟩ ≡
  begin ⟨ Do first-pass processing based on type(q); goto done_with_noad if a noad has been fully
      processed, goto check_dimensions if it has been translated into new_hlist(q), or goto done_with_node
      if a node has been fully processed 771 ⟩;
check_dimensions: z ← hpack(new_hlist(q), natural);
  if height(z) > max_h then max_h ← height(z);
  if depth(z) > max_d then max_d ← depth(z);
  free_node(z, box_node_size);
done_with_noad: r ← q; r_type ← type(r);
  if r_type = right_noad then
    begin r_type ← left_noad; cur_style ← style;
    ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
    end;
done_with_node: q ← link(q);
  end
```

This code is used in section 769.

771. One of the things we must do on the first pass is change a *bin_noad* to an *ord_noad* if the *bin_noad* is not in the context of a binary operator. The values of *r* and *r_type* make this fairly easy.

```
⟨ Do first-pass processing based on type(q); goto done_with_noad if a noad has been fully processed, goto
      check_dimensions if it has been translated into new_hlist(q), or goto done_with_node if a node has
      been fully processed 771 ⟩ ≡
reswitch: delta ← 0;
  case type(q) of
  bin_noad: case r_type of
    bin_noad, op_noad, rel_noad, open_noad, punct_noad, left_noad: begin type(q) ← ord_noad;
      goto reswitch;
      end;
    othercases do_nothing
    endcases;
  rel_noad, close_noad, punct_noad, right_noad: begin
    ⟨ Convert a final bin_noad to an ord_noad 772 ⟩;
    if type(q) = right_noad then goto done_with_noad;
    end;
  ⟨ Cases for noads that can follow a bin_noad 776 ⟩
  ⟨ Cases for nodes that can appear in an mlist, after which we goto done_with_node 773 ⟩
  othercases confusion("mlist1")
  endcases;
  ⟨ Convert nucleus(q) to an hlist and attach the sub/superscripts 798 ⟩
```

This code is used in section 770.

772.

```
⟨ Convert a final bin_noad to an ord_noad 772 ⟩ ≡
  if r_type = bin_noad then type(r) ← ord_noad
```

This code is used in sections 769 and 771.

773. ⟨ Cases for nodes that can appear in an mlist, after which we **goto** *done_with_node* 773 ⟩ ≡

```
style_node: begin cur_style ← subtype(q);
  ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
  goto done_with_node;
  end;
choice_node: ⟨ Change this node to a style node followed by the correct choice, then goto
      done_with_node 774 ⟩;
ins_node, mark_node, adjust_node, whatsit_node, penalty_node, disc_node: goto done_with_node;
rule_node: begin if height(q) > max_h then max_h ← height(q);
  if depth(q) > max_d then max_d ← depth(q);
  goto done_with_node;
  end;
glue_node: begin ⟨ Convert math glue to ordinary glue 775 ⟩;
  goto done_with_node;
  end;
kern_node: begin math_kern(q, cur_mu); goto done_with_node;
  end;
```

This code is used in section 771.

774. **define** *choose_mlist*(#) ≡
begin $p \leftarrow$ #(q); #$(q) \leftarrow null$; **end**

⟨ Change this node to a style node followed by the correct choice, then **goto** *done_with_node* 774 ⟩ ≡

```
  begin case cur_style div 2 of
  0: choose_mlist(display_mlist);   { display_style = 0 }
  1: choose_mlist(text_mlist);   { text_style = 2 }
  2: choose_mlist(script_mlist);   { script_style = 4 }
  3: choose_mlist(script_script_mlist);   { script_script_style = 6 }
  end;   { there are no other cases }
  flush_node_list(display_mlist(q)); flush_node_list(text_mlist(q)); flush_node_list(script_mlist(q));
  flush_node_list(script_script_mlist(q));
  type(q) ← style_node; subtype(q) ← cur_style; width(q) ← 0; depth(q) ← 0;
  if p ≠ null then
    begin z ← link(q); link(q) ← p;
    while link(p) ≠ null do p ← link(p);
    link(p) ← z;
    end;
  goto done_with_node;
  end
```

This code is used in section 773.

775. Conditional math glue ('`\nonscript`') results in a *glue_node* pointing to *zero_glue*, with *subtype*(q) = *cond_math_glue*; in such a case the node following will be eliminated if it is a glue or kern node and if the current size is different from *text_size*. Unconditional math glue ('`\muskip`') is converted to normal glue by multiplying the dimensions by *cur_mu*.

```
⟨ Convert math glue to ordinary glue 775 ⟩ ≡
  if subtype(q) = mu_glue then
    begin x ← glue_ptr(q); y ← math_glue(x, cur_mu); delete_glue_ref(x); glue_ptr(q) ← y;
    subtype(q) ← normal;
    end
  else if (cur_size ≠ text_size) ∧ (subtype(q) = cond_math_glue) then
      begin p ← link(q);
      if p ≠ null then
        if (type(p) = glue_node) ∨ (type(p) = kern_node) then
          begin link(q) ← link(p); link(p) ← null; flush_node_list(p);
          end;
      end
```

This code is used in section 773.

776.

```
⟨ Cases for noads that can follow a bin_noad 776 ⟩ ≡
left_noad: goto done_with_noad;
fraction_noad: begin make_fraction(q); goto check_dimensions;
  end;
op_noad: begin delta ← make_op(q);
  if subtype(q) = limits then goto check_dimensions;
  end;
ord_noad: make_ord(q);
open_noad, inner_noad: do_nothing;
radical_noad: make_radical(q);
over_noad: make_over(q);
under_noad: make_under(q);
accent_noad: make_math_accent(q);
vcenter_noad: make_vcenter(q);
```

This code is used in section 771.

777. Most of the actual construction work of *mlist_to_hlist* is done by procedures with names like *make_fraction*, *make_radical*, etc. To illustrate the general setup of such procedures, let's begin with a couple of simple ones.

```
⟨ Declare math construction procedures 777 ⟩ ≡
procedure make_over(q : pointer);
  begin info(nucleus(q)) ← overbar(clean_box(nucleus(q), cramped_style(cur_style)),
      3 * default_rule_thickness, default_rule_thickness); math_type(nucleus(q)) ← sub_box;
  end;
```

See also sections 778, 779, 780, 781, 787, 793, 796, 800, and 810.

This code is used in section 769.

778. ⟨ Declare math construction procedures 777 ⟩ +≡
procedure *make_under*(*q* : *pointer*);
var *p*, *x*, *y*: *pointer*; { temporary registers for box construction }
delta: *scaled*; { overall height plus depth }
begin *x* ← *clean_box*(*nucleus*(*q*), *cur_style*); *p* ← *new_kern*(3 ∗ *default_rule_thickness*); *link*(*x*) ← *p*;
link(*p*) ← *fraction_rule*(*default_rule_thickness*); *y* ← *vpack*(*x*, *natural*);
delta ← *height*(*y*) + *depth*(*y*) + *default_rule_thickness*; *height*(*y*) ← *height*(*x*);
depth(*y*) ← *delta* − *height*(*y*); *info*(*nucleus*(*q*)) ← *y*; *math_type*(*nucleus*(*q*)) ← *sub_box*;
end;

779. ⟨ Declare math construction procedures 777 ⟩ +≡
procedure *make_vcenter*(*q* : *pointer*);
var *v*: *pointer*; { the box that should be centered vertically }
delta: *scaled*; { its height plus depth }
begin *v* ← *info*(*nucleus*(*q*));
if *type*(*v*) ≠ *vlist_node* **then** *confusion*("vcenter");
delta ← *height*(*v*) + *depth*(*v*); *height*(*v*) ← *axis_height*(*cur_size*) + *half*(*delta*);
depth(*v*) ← *delta* − *height*(*v*);
end;

780. According to the rules in the DVI file specifications, we ensure alignment between a square root sign and the rule above its nucleus by assuming that the baseline of the square-root symbol is the same as the bottom of the rule. The height of the square-root symbol will be the thickness of the rule, and the depth of the square-root symbol should exceed or equal the height-plus-depth of the nucleus plus a certain minimum clearance *clr*. The symbol will be placed so that the actual clearance is *clr* plus half the excess.

```
⟨ Declare math construction procedures 777 ⟩ +≡
procedure make_radical(q : pointer);
  var x, y: pointer;  { temporary registers for box construction }
    f: internal_font_number; rule_thickness: scaled;  { rule thickness }
    delta, clr: scaled;  { dimensions involved in the calculation }
  begin f ← fam_fnt(small_fam(left_delimiter(q)) + cur_size);
  if is_new_mathfont(f) then rule_thickness ← get_ot_math_constant(f, radicalRuleThickness)
  else rule_thickness ← default_rule_thickness;
  x ← clean_box(nucleus(q), cramped_style(cur_style));
  if is_new_mathfont(f) then
    begin if cur_style < text_style then  { display style }
      clr ← get_ot_math_constant(f, radicalDisplayStyleVerticalGap)
    else clr ← get_ot_math_constant(f, radicalVerticalGap);
    end
  else begin if cur_style < text_style then  { display style }
      clr ← rule_thickness + (abs(math_x_height(cur_size)) div 4)
    else begin clr ← rule_thickness; clr ← clr + (abs(clr) div 4);
      end;
    end;
  y ← var_delimiter(left_delimiter(q), cur_size, height(x) + depth(x) + clr + rule_thickness);
  if is_new_mathfont(f) then
    begin depth(y) ← height(y) + depth(y) − rule_thickness; height(y) ← rule_thickness;
    end;
  delta ← depth(y) − (height(x) + depth(x) + clr);
  if delta > 0 then clr ← clr + half(delta);  { increase the actual clearance }
  shift_amount(y) ← −(height(x) + clr); link(y) ← overbar(x, clr, height(y));
  info(nucleus(q)) ← hpack(y, natural); math_type(nucleus(q)) ← sub_box;
  end;
```

781. Slants are not considered when placing accents in math mode. The accenter is centered over the accentee, and the accent width is treated as zero with respect to the size of the final box.

```
⟨ Declare math construction procedures 777 ⟩ +≡
function compute_ot_math_accent_pos(p : pointer): scaled;
  var q, r: pointer; s, g: scaled;
  begin if (math_type(nucleus(p)) = math_char) then
    begin fetch(nucleus(p)); q ← new_native_character(cur_f, qo(cur_c)); g ← get_native_glyph(q, 0);
    s ← get_ot_math_accent_pos(cur_f, g);
    end
  else begin if (math_type(nucleus(p)) = sub_mlist) then
      begin r ← info(nucleus(p));
      if (r ≠ null) ∧ (type(r) = accent_noad) then s ← compute_ot_math_accent_pos(r)
      else s ← "7FFFFFFF;
      end
    else s ← "7FFFFFFF;
    end;
  compute_ot_math_accent_pos ← s;
  end;
procedure make_math_accent(q : pointer);
  label done, done1;
  var p, x, y: pointer; { temporary registers for box construction }
    a: integer; { address of lig/kern instruction }
    c, g: integer; { accent character }
    f: internal_font_number; { its font }
    i: four_quarters; { its char_info }
    s, sa: scaled; { amount to skew the accent to the right }
    h: scaled; { height of character being accented }
    delta: scaled; { space to remove between accent and accentee }
    w, w2: scaled; { width of the accentee, not including sub/superscripts }
    ot_assembly_ptr: void_pointer;
  begin fetch(accent_chr(q)); x ← null; ot_assembly_ptr ← nil;
  if is_native_font(cur_f) then
    begin c ← cur_c; f ← cur_f;
    if ¬is_bottom_acc(q) then s ← compute_ot_math_accent_pos(q)
    else s ← 0;
    x ← clean_box(nucleus(q), cramped_style(cur_style)); w ← width(x); h ← height(x);
    end
  else if char_exists(cur_i) then
      begin i ← cur_i; c ← cur_c; f ← cur_f;
      ⟨ Compute the amount of skew 785 ⟩;
      x ← clean_box(nucleus(q), cramped_style(cur_style)); w ← width(x); h ← height(x);
      ⟨ Switch to a larger accent if available and appropriate 784 ⟩;
      end;
  if x ≠ null then
    begin if is_new_mathfont(f) then
      if is_bottom_acc(q) then delta ← 0
      else if h < get_ot_math_constant(f, accentBaseHeight) then
          delta ← h else delta ← get_ot_math_constant(f, accentBaseHeight)
    else if h < x_height(f) then delta ← h else delta ← x_height(f);
    if (math_type(supscr(q)) ≠ empty) ∨ (math_type(subscr(q)) ≠ empty) then
      if math_type(nucleus(q)) = math_char then ⟨ Swap the subscript and superscript into box x 786 ⟩;
    y ← char_box(f, c);
```

```
if is_native_font(f) then
  begin { turn the native_word node into a native_glyph one }
  p ← get_node(glyph_node_size); type(p) ← whatsit_node; subtype(p) ← glyph_node;
  native_font(p) ← f; native_glyph(p) ← get_native_glyph(list_ptr(y), 0); set_native_glyph_metrics(p, 1);
  free_node(list_ptr(y), native_size(list_ptr(y))); list_ptr(y) ← p; ⟨ Switch to a larger native-font accent
      if available and appropriate 783 ⟩; { determine horiz positioning }
  if is_glyph_node(p) then
    begin sa ← get_ot_math_accent_pos(f, native_glyph(p));
    if sa = "7FFFFFFF then sa ← half(width(y));
    end
  else sa ← half(width(y));
  if is_bottom_acc(q) ∨ (s = "7FFFFFFF) then s ← half(w);
  shift_amount(y) ← s − sa;
  end
else shift_amount(y) ← s + half(w − width(y));
width(y) ← 0;
if is_bottom_acc(q) then
  begin link(x) ← y; y ← vpack(x, natural); shift_amount(y) ← −(h − height(y));
  end
else begin p ← new_kern(−delta); link(p) ← x; link(y) ← p; y ← vpack(y, natural);
  if height(y) < h then ⟨ Make the height of box y equal to h 782 ⟩;
  end;
width(y) ← width(x); info(nucleus(q)) ← y; math_type(nucleus(q)) ← sub_box;
end;
free_ot_assembly(ot_assembly_ptr);
end;
```

782. ⟨ Make the height of box y equal to h 782 ⟩ ≡

```
begin p ← new_kern(h − height(y)); link(p) ← list_ptr(y); list_ptr(y) ← p; height(y) ← h;
end
```

This code is used in section 781.

783. ⟨ Switch to a larger native-font accent if available and appropriate 783 ⟩ ≡

```
if odd(subtype(q)) then { non growing accent }
  set_native_glyph_metrics(p, 1)
else begin c ← native_glyph(p); a ← 0;
  repeat g ← get_ot_math_variant(f, c, a, addressof(w2), 1);
    if (w2 > 0) ∧ (w2 ≤ w) then
      begin native_glyph(p) ← g; set_native_glyph_metrics(p, 1); incr(a);
      end;
  until (w2 < 0) ∨ (w2 ≥ w);
  if (w2 < 0) then
    begin ot_assembly_ptr ← get_ot_assembly_ptr(f, c, 1);
    if ot_assembly_ptr ≠ nil then
      begin free_node(p, glyph_node_size); p ← build_opentype_assembly(f, ot_assembly_ptr, w, 1);
      list_ptr(y) ← p; goto found;
      end;
    end
  else set_native_glyph_metrics(p, 1);
  end;
found: width(y) ← width(p); height(y) ← height(p); depth(y) ← depth(p);
if is_bottom_acc(q) then
  begin if height(y) < 0 then height(y) ← 0
  end
else if depth(y) < 0 then depth(y) ← 0;
```

This code is used in section 781.

784. ⟨ Switch to a larger accent if available and appropriate 784 ⟩ ≡

```
loop begin if char_tag(i) ≠ list_tag then goto done;
  y ← rem_byte(i); i ← char_info(f)(y);
  if ¬char_exists(i) then goto done;
  if char_width(f)(i) > w then goto done;
  c ← y;
  end;
done:
```

This code is used in section 781.

785. ⟨ Compute the amount of skew 785 ⟩ ≡

```
s ← 0;
if math_type(nucleus(q)) = math_char then
  begin fetch(nucleus(q));
  if char_tag(cur_i) = lig_tag then
    begin a ← lig_kern_start(cur_f)(cur_i); cur_i ← font_info[a].qqqq;
    if skip_byte(cur_i) > stop_flag then
      begin a ← lig_kern_restart(cur_f)(cur_i); cur_i ← font_info[a].qqqq;
      end;
    loop begin if qo(next_char(cur_i)) = skew_char[cur_f] then
        begin if op_byte(cur_i) ≥ kern_flag then
          if skip_byte(cur_i) ≤ stop_flag then s ← char_kern(cur_f)(cur_i);
        goto done1;
        end;
      if skip_byte(cur_i) ≥ stop_flag then goto done1;
      a ← a + qo(skip_byte(cur_i)) + 1; cur_i ← font_info[a].qqqq;
      end;
    end;
  end;
done1:
```

This code is used in section 781.

786. ⟨ Swap the subscript and superscript into box x 786 ⟩ ≡

```
begin flush_node_list(x); x ← new_noad; mem[nucleus(x)] ← mem[nucleus(q)];
mem[supscr(x)] ← mem[supscr(q)]; mem[subscr(x)] ← mem[subscr(q)];
mem[supscr(q)].hh ← empty_field; mem[subscr(q)].hh ← empty_field;
math_type(nucleus(q)) ← sub_mlist; info(nucleus(q)) ← x; x ← clean_box(nucleus(q), cur_style);
delta ← delta + height(x) − h; h ← height(x);
end
```

This code is used in section 781.

787. The *make_fraction* procedure is a bit different because it sets *new_hlist*(*q*) directly rather than making a sub-box.

```
⟨ Declare math construction procedures 777 ⟩ +≡
procedure make_fraction(q : pointer);
  var p, v, x, y, z: pointer; { temporary registers for box construction }
    delta, delta1, delta2, shift_up, shift_down, clr: scaled; { dimensions for box calculations }
  begin if thickness(q) = default_code then thickness(q) ← default_rule_thickness;
  ⟨ Create equal-width boxes x and z for the numerator and denominator, and compute the default amounts
      shift_up and shift_down by which they are displaced from the baseline 788 ⟩;
  if thickness(q) = 0 then ⟨ Adjust shift_up and shift_down for the case of no fraction line 789 ⟩
  else ⟨ Adjust shift_up and shift_down for the case of a fraction line 790 ⟩;
  ⟨ Construct a vlist box for the fraction, according to shift_up and shift_down 791 ⟩;
  ⟨ Put the fraction into a box with its delimiters, and make new_hlist(q) point to it 792 ⟩;
  end;
```

788. ⟨ Create equal-width boxes x and z for the numerator and denominator, and compute the default amounts *shift_up* and *shift_down* by which they are displaced from the baseline 788 ⟩ ≡

```
x ← clean_box(numerator(q), num_style(cur_style));
z ← clean_box(denominator(q), denom_style(cur_style));
if width(x) < width(z) then x ← rebox(x, width(z))
else z ← rebox(z, width(x));
if cur_style < text_style then { display style }
  begin shift_up ← num1(cur_size); shift_down ← denom1(cur_size);
  end
else begin shift_down ← denom2(cur_size);
  if thickness(q) ≠ 0 then shift_up ← num2(cur_size)
  else shift_up ← num3(cur_size);
  end
```

This code is used in section 787.

789. The numerator and denominator must be separated by a certain minimum clearance, called *clr* in the following program. The difference between *clr* and the actual clearance is twice *delta*.

⟨ Adjust *shift_up* and *shift_down* for the case of no fraction line 789 ⟩ ≡

```
begin if is_new_mathfont(cur_f) then
  begin if cur_style < text_style then clr ← get_ot_math_constant(cur_f, stackDisplayStyleGapMin)
  else clr ← get_ot_math_constant(cur_f, stackGapMin);
  end
else begin if cur_style < text_style then clr ← 7 * default_rule_thickness
  else clr ← 3 * default_rule_thickness;
  end;
delta ← half(clr − ((shift_up − depth(x)) − (height(z) − shift_down)));
if delta > 0 then
  begin shift_up ← shift_up + delta; shift_down ← shift_down + delta;
  end;
end
```

This code is used in section 787.

790. In the case of a fraction line, the minimum clearance depends on the actual thickness of the line.

```
⟨ Adjust shift_up and shift_down for the case of a fraction line 790 ⟩ ≡
  begin if is_new_mathfont(cur_f) then
    begin delta ← half(thickness(q));
    if cur_style < text_style then clr ← get_ot_math_constant(cur_f, fractionNumDisplayStyleGapMin)
    else clr ← get_ot_math_constant(cur_f, fractionNumeratorGapMin);
    delta1 ← clr − ((shift_up − depth(x)) − (axis_height(cur_size) + delta));
    if cur_style < text_style then clr ← get_ot_math_constant(cur_f, fractionDenomDisplayStyleGapMin)
    else clr ← get_ot_math_constant(cur_f, fractionDenominatorGapMin);
    delta2 ← clr − ((axis_height(cur_size) − delta) − (height(z) − shift_down));
    end
  else begin if cur_style < text_style then clr ← 3 * thickness(q)
    else clr ← thickness(q);
    delta ← half(thickness(q)); delta1 ← clr − ((shift_up − depth(x)) − (axis_height(cur_size) + delta));
    delta2 ← clr − ((axis_height(cur_size) − delta) − (height(z) − shift_down));
    end;
  if delta1 > 0 then shift_up ← shift_up + delta1;
  if delta2 > 0 then shift_down ← shift_down + delta2;
  end
```

This code is used in section 787.

791.

```
⟨ Construct a vlist box for the fraction, according to shift_up and shift_down 791 ⟩ ≡
  v ← new_null_box; type(v) ← vlist_node; height(v) ← shift_up + height(x);
  depth(v) ← depth(z) + shift_down; width(v) ← width(x); { this also equals width(z) }
  if thickness(q) = 0 then
    begin p ← new_kern((shift_up − depth(x)) − (height(z) − shift_down)); link(p) ← z;
    end
  else begin y ← fraction_rule(thickness(q));
    p ← new_kern((axis_height(cur_size) − delta) − (height(z) − shift_down));
    link(y) ← p; link(p) ← z;
    p ← new_kern((shift_up − depth(x)) − (axis_height(cur_size) + delta)); link(p) ← y;
    end;
  link(x) ← p; list_ptr(v) ← x
```

This code is used in section 787.

792.

```
⟨ Put the fraction into a box with its delimiters, and make new_hlist(q) point to it 792 ⟩ ≡
  if cur_style < text_style then delta ← delim1(cur_size)
  else delta ← delim2(cur_size);
  x ← var_delimiter(left_delimiter(q), cur_size, delta); link(x) ← v;
  z ← var_delimiter(right_delimiter(q), cur_size, delta); link(v) ← z;
  new_hlist(q) ← hpack(x, natural)
```

This code is used in section 787.

793. If the nucleus of an *op_noad* is a single character, it is to be centered vertically with respect to the axis, after first being enlarged (via a character list in the font) if we are in display style. The normal convention for placing displayed limits is to put them above and below the operator in display style.

The italic correction is removed from the character if there is a subscript and the limits are not being displayed. The *make_op* routine returns the value that should be used as an offset between subscript and superscript.

After *make_op* has acted, *subtype*(q) will be *limits* if and only if the limits have been set above and below the operator. In that case, *new_hlist*(q) will already contain the desired final box.

```
⟨ Declare math construction procedures 777 ⟩ +≡
function make_op(q : pointer): scaled;
  label found;
  var delta: scaled; { offset between subscript and superscript }
    p, v, x, y, z: pointer; { temporary registers for box construction }
    c: quarterword; i: four_quarters; { registers for character examination }
    shift_up, shift_down: scaled; { dimensions for box calculation }
    h1, h2: scaled; { height of original text-style symbol and possible replacement }
    n, g: integer; { potential variant index and glyph code }
    ot_assembly_ptr: void_pointer; save_f: internal_font_number;
  begin if (subtype(q) = normal) ∧ (cur_style < text_style) then subtype(q) ← limits;
  delta ← 0; ot_assembly_ptr ← nil;
  if math_type(nucleus(q)) = math_char then
    begin fetch(nucleus(q));
    if ¬is_ot_font(cur_f) then
      begin if (cur_style < text_style) ∧ (char_tag(cur_i) = list_tag) then { make it larger }
        begin c ← rem_byte(cur_i); i ← char_info(cur_f)(c);
        if char_exists(i) then
          begin cur_c ← c; cur_i ← i; character(nucleus(q)) ← c;
          end;
        end;
      delta ← char_italic(cur_f)(cur_i);
      end;
    x ← clean_box(nucleus(q), cur_style);
    if is_new_mathfont(cur_f) then
      begin p ← list_ptr(x);
      if is_glyph_node(p) then
        begin if cur_style < text_style then
          begin { try to replace the operator glyph with a display-size variant, ensuring it is larger
              than the text size }
          h1 ← get_ot_math_constant(cur_f, displayOperatorMinHeight);
          if h1 < (height(p) + depth(p)) * 5/4 then h1 ← (height(p) + depth(p)) * 5/4;
          c ← native_glyph(p); n ← 0;
          repeat g ← get_ot_math_variant(cur_f, c, n, addressof(h2), 0);
            if h2 > 0 then
              begin native_glyph(p) ← g; set_native_glyph_metrics(p, 1);
              end;
            incr(n);
          until (h2 < 0) ∨ (h2 ≥ h1);
          if (h2 < 0) then
            begin
              { if we get here, then we didn't find a big enough glyph; check if the char is extensible }
            ot_assembly_ptr ← get_ot_assembly_ptr(cur_f, c, 0);
            if ot_assembly_ptr ≠ nil then
```

```
            begin free_node(p, glyph_node_size);
            p ← build_opentype_assembly(cur_f, ot_assembly_ptr, h1, 0); list_ptr(x) ← p; delta ← 0;
            goto found;
            end;
          end
        else set_native_glyph_metrics(p, 1);
        end;
      delta ← get_ot_math_ital_corr(cur_f, native_glyph(p));
    found: width(x) ← width(p); height(x) ← height(p); depth(x) ← depth(p);
      end
    end;
  if (math_type(subscr(q)) ≠ empty) ∧ (subtype(q) ≠ limits) then width(x) ← width(x) − delta;
      { remove italic correction }
  shift_amount(x) ← half(height(x) − depth(x)) − axis_height(cur_size); { center vertically }
  math_type(nucleus(q)) ← sub_box; info(nucleus(q)) ← x;
  end;
save_f ← cur_f;
if subtype(q) = limits then ⟨ Construct a box with limits above and below it, skewed by delta 794 ⟩;
free_ot_assembly(ot_assembly_ptr); make_op ← delta;
end;
```

794. The following program builds a vlist box v for displayed limits. The width of the box is not affected by the fact that the limits may be skewed.

```
⟨ Construct a box with limits above and below it, skewed by delta 794 ⟩ ≡
  begin x ← clean_box(supscr(q), sup_style(cur_style)); y ← clean_box(nucleus(q), cur_style);
  z ← clean_box(subscr(q), sub_style(cur_style)); v ← new_null_box; type(v) ← vlist_node;
  width(v) ← width(y);
  if width(x) > width(v) then width(v) ← width(x);
  if width(z) > width(v) then width(v) ← width(z);
  x ← rebox(x, width(v)); y ← rebox(y, width(v)); z ← rebox(z, width(v));
  shift_amount(x) ← half(delta); shift_amount(z) ← −shift_amount(x); height(v) ← height(y);
  depth(v) ← depth(y);
  ⟨ Attach the limits to y and adjust height(v), depth(v) to account for their presence 795 ⟩;
  new_hlist(q) ← v;
  end
```

This code is used in section 793.

795. We use *shift_up* and *shift_down* in the following program for the amount of glue between the displayed operator y and its limits x and z. The vlist inside box v will consist of x followed by y followed by z, with kern nodes for the spaces between and around them.

```
⟨ Attach the limits to y and adjust height(v), depth(v) to account for their presence 795 ⟩ ≡
  cur_f ← save_f;
  if math_type(supscr(q)) = empty then
    begin free_node(x, box_node_size); list_ptr(v) ← y;
    end
  else begin shift_up ← big_op_spacing3 − depth(x);
    if shift_up < big_op_spacing1 then shift_up ← big_op_spacing1;
    p ← new_kern(shift_up); link(p) ← y; link(x) ← p;
    p ← new_kern(big_op_spacing5); link(p) ← x; list_ptr(v) ← p;
    height(v) ← height(v) + big_op_spacing5 + height(x) + depth(x) + shift_up;
    end;
  if math_type(subscr(q)) = empty then free_node(z, box_node_size)
  else begin shift_down ← big_op_spacing4 − height(z);
    if shift_down < big_op_spacing2 then shift_down ← big_op_spacing2;
    p ← new_kern(shift_down); link(y) ← p; link(p) ← z;
    p ← new_kern(big_op_spacing5); link(z) ← p;
    depth(v) ← depth(v) + big_op_spacing5 + height(z) + depth(z) + shift_down;
    end
```

This code is used in section 794.

796. A ligature found in a math formula does not create a *ligature_node*, because there is no question of hyphenation afterwards; the ligature will simply be stored in an ordinary *char_node*, after residing in an *ord_noad*.

The *math_type* is converted to *math_text_char* here if we would not want to apply an italic correction to the current character unless it belongs to a math font (i.e., a font with *space* = 0).

No boundary characters enter into these ligatures.

```
⟨ Declare math construction procedures 777 ⟩ +≡
procedure make_ord(q : pointer);
  label restart, exit;
  var a: integer;  { address of lig/kern instruction }
    p, r: pointer;  { temporary registers for list manipulation }
  begin restart:
  if math_type(subscr(q)) = empty then
    if math_type(supscr(q)) = empty then
      if math_type(nucleus(q)) = math_char then
        begin p ← link(q);
        if p ≠ null then
          if (type(p) ≥ ord_noad) ∧ (type(p) ≤ punct_noad) then
            if math_type(nucleus(p)) = math_char then
              if fam(nucleus(p)) = fam(nucleus(q)) then
                begin math_type(nucleus(q)) ← math_text_char; fetch(nucleus(q));
                if char_tag(cur_i) = lig_tag then
                  begin a ← lig_kern_start(cur_f)(cur_i); cur_c ← character(nucleus(p));
                  cur_i ← font_info[a].qqqq;
                  if skip_byte(cur_i) > stop_flag then
                    begin a ← lig_kern_restart(cur_f)(cur_i); cur_i ← font_info[a].qqqq;
                    end;
                  loop begin ⟨ If instruction cur_i is a kern with cur_c, attach the kern after q; or if it
                        is a ligature with cur_c, combine noads q and p appropriately; then return if
                        the cursor has moved past a noad, or goto restart 797 ⟩;
                    if skip_byte(cur_i) ≥ stop_flag then return;
                    a ← a + qo(skip_byte(cur_i)) + 1; cur_i ← font_info[a].qqqq;
                    end;
                  end;
                end;
        end;
exit: end;
```

797. Note that a ligature between an *ord_noad* and another kind of noad is replaced by an *ord_noad*, when the two noads collapse into one. But we could make a parenthesis (say) change shape when it follows certain letters. Presumably a font designer will define such ligatures only when this convention makes sense.

```
⟨ If instruction cur_i is a kern with cur_c, attach the kern after q; or if it is a ligature with cur_c,
      combine noads q and p appropriately; then return if the cursor has moved past a noad, or goto
      restart 797 ⟩ ≡
  if next_char(cur_i) = cur_c then
    if skip_byte(cur_i) ≤ stop_flag then
      if op_byte(cur_i) ≥ kern_flag then
        begin p ← new_kern(char_kern(cur_f)(cur_i)); link(p) ← link(q); link(q) ← p; return;
        end
      else begin check_interrupt;  { allow a way out of infinite ligature loop }
        case op_byte(cur_i) of
        qi(1), qi(5): character(nucleus(q)) ← rem_byte(cur_i);  { =:|, =:|> }
        qi(2), qi(6): character(nucleus(p)) ← rem_byte(cur_i);  { |=:, |=:> }
        qi(3), qi(7), qi(11): begin r ← new_noad;  { |=:|, |=:|>, |=:|>> }
          character(nucleus(r)) ← rem_byte(cur_i); plane_and_fam_field(nucleus(r)) ← fam(nucleus(q));
          link(q) ← r; link(r) ← p;
          if op_byte(cur_i) < qi(11) then math_type(nucleus(r)) ← math_char
          else math_type(nucleus(r)) ← math_text_char;  { prevent combination }
          end;
        othercases begin link(q) ← link(p); character(nucleus(q)) ← rem_byte(cur_i);  { =: }
          mem[subscr(q)] ← mem[subscr(p)]; mem[supscr(q)] ← mem[supscr(p)];
          free_node(p, noad_size);
          end
        endcases;
        if op_byte(cur_i) > qi(3) then return;
        math_type(nucleus(q)) ← math_char; goto restart;
        end
```

This code is used in section 796.

798. When we get to the following part of the program, we have "fallen through" from cases that did not lead to *check_dimensions* or *done_with_noad* or *done_with_node*. Thus, q points to a noad whose nucleus may need to be converted to an hlist, and whose subscripts and superscripts need to be appended if they are present.

If *nucleus*(q) is not a *math_char*, the variable *delta* is the amount by which a superscript should be moved right with respect to a subscript when both are present.

```
⟨ Convert nucleus(q) to an hlist and attach the sub/superscripts 798 ⟩ ≡
  case math_type(nucleus(q)) of
  math_char, math_text_char: ⟨ Create a character node p for nucleus(q), possibly followed by a kern node
        for the italic correction, and set delta to the italic correction if a subscript is present 799 ⟩;
  empty: p ← null;
  sub_box: p ← info(nucleus(q));
  sub_mlist: begin cur_mlist ← info(nucleus(q)); save_style ← cur_style; mlist_penalties ← false;
    mlist_to_hlist; { recursive call }
    cur_style ← save_style; ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
    p ← hpack(link(temp_head), natural);
    end;
  othercases confusion("mlist2")
  endcases;
  new_hlist(q) ← p;
  if (math_type(subscr(q)) = empty) ∧ (math_type(supscr(q)) = empty) then goto check_dimensions;
  make_scripts(q, delta)
```

This code is used in section 771.

799. ⟨ Create a character node p for *nucleus*(q), possibly followed by a kern node for the italic correction, and set *delta* to the italic correction if a subscript is present 799 ⟩ ≡

```
  begin fetch(nucleus(q));
  if is_native_font(cur_f) then
    begin z ← new_native_character(cur_f, qo(cur_c)); p ← get_node(glyph_node_size);
    type(p) ← whatsit_node; subtype(p) ← glyph_node; native_font(p) ← cur_f;
    native_glyph(p) ← get_native_glyph(z, 0); set_native_glyph_metrics(p, 1); free_node(z, native_size(z));
    delta ← get_ot_math_ital_corr(cur_f, native_glyph(p));
    if (math_type(nucleus(q)) = math_text_char) ∧ (¬is_new_mathfont(cur_f) ≠ 0) then delta ← 0;
          { no italic correction in mid-word of text font }
    if (math_type(subscr(q)) = empty) ∧ (delta ≠ 0) then
      begin link(p) ← new_kern(delta); delta ← 0;
      end;
    end
  else if char_exists(cur_i) then
      begin delta ← char_italic(cur_f)(cur_i); p ← new_character(cur_f, qo(cur_c));
      if (math_type(nucleus(q)) = math_text_char) ∧ (space(cur_f) ≠ 0) then delta ← 0;
            { no italic correction in mid-word of text font }
      if (math_type(subscr(q)) = empty) ∧ (delta ≠ 0) then
        begin link(p) ← new_kern(delta); delta ← 0;
        end;
      end
    else p ← null;
  end
```

This code is used in section 798.

800. The purpose of *make_scripts*(*q*, *delta*) is to attach the subscript and/or superscript of noad *q* to the list that starts at *new_hlist*(*q*), given that the subscript and superscript aren't both empty. The superscript will appear to the right of the subscript by a given distance *delta*.

We set *shift_down* and *shift_up* to the minimum amounts to shift the baseline of subscripts and superscripts based on the given nucleus.

```
⟨ Declare math construction procedures 777 ⟩ +≡
function attach_hkern_to_new_hlist(q : pointer; delta : scaled): pointer;
  var y, z: pointer;  { temporary registers for box construction }
  begin z ← new_kern(delta);
  if new_hlist(q) = null then new_hlist(q) ← z
  else begin y ← new_hlist(q);
    while link(y) ≠ null do y ← link(y);
    link(y) ← z;
    end;
  attach_hkern_to_new_hlist ← new_hlist(q);
  end;
procedure make_scripts(q : pointer; delta : scaled);
  var p, x, y, z: pointer;  { temporary registers for box construction }
    shift_up, shift_down, clr, sub_kern, sup_kern: scaled;  { dimensions in the calculation }
    script_c: pointer;  { temprary native character for sub/superscript }
    script_g: quarterword;  { temporary register for sub/superscript native glyph id }
    script_f: internal_font_number;  { temporary register for sub/superscript font }
    sup_g: quarterword;  { superscript native glyph id }
    sup_f: internal_font_number;  { superscript font }
    sub_g: quarterword;  { subscript native glyph id }
    sub_f: internal_font_number;  { subscript font }
    t: integer;  { subsidiary size code }
    save_f: internal_font_number; script_head: pointer;  { scratch var for OpenType s*scripts }
    script_ptr: pointer;  { scratch var for OpenType s*scripts }
    saved_math_style: small_number;  { scratch var for OpenType s*scripts }
    this_math_style: small_number;  { scratch var for OpenType s*scripts }
  begin p ← new_hlist(q); script_c ← null; script_g ← 0; script_f ← 0; sup_kern ← 0; sub_kern ← 0;
  if is_char_node(p) ∨ is_glyph_node(p) then
    begin shift_up ← 0; shift_down ← 0;
    end
  else begin z ← hpack(p, natural);
    if cur_style < script_style then t ← script_size else t ← script_script_size;
    shift_up ← height(z) − sup_drop(t); shift_down ← depth(z) + sub_drop(t); free_node(z, box_node_size);
    end;
  if math_type(supscr(q)) = empty then ⟨ Construct a subscript box x when there is no superscript 801 ⟩
  else begin ⟨ Construct a superscript box x 802 ⟩;
    if math_type(subscr(q)) = empty then shift_amount(x) ← −shift_up
    else ⟨ Construct a sub/superscript combination box x, with the superscript offset by delta 803 ⟩;
    end;
  if new_hlist(q) = null then new_hlist(q) ← x
  else begin p ← new_hlist(q);
    while link(p) ≠ null do p ← link(p);
    link(p) ← x;
    end;
  end;
```

801. When there is a subscript without a superscript, the top of the subscript should not exceed the baseline plus four-fifths of the x-height.

```
⟨ Construct a subscript box x when there is no superscript 801 ⟩ ≡
  begin script_head ← subscr(q); ⟨ Fetch first character of a sub/superscript 805 ⟩;
  sub_g ← script_g; sub_f ← script_f; save_f ← cur_f; x ← clean_box(subscr(q), sub_style(cur_style));
  cur_f ← save_f; width(x) ← width(x) + script_space;
  if shift_down < sub1(cur_size) then shift_down ← sub1(cur_size);
  if is_new_mathfont(cur_f) then clr ← height(x) − get_ot_math_constant(cur_f, subscriptTopMax)
  else clr ← height(x) − (abs(math_x_height(cur_size) ∗ 4) div 5);
  if shift_down < clr then shift_down ← clr;
  shift_amount(x) ← shift_down;
  if is_new_mathfont(cur_f) then ⟨ Attach subscript OpenType math kerning 806 ⟩
  end
```

This code is used in section 800.

802. The bottom of a superscript should never descend below the baseline plus one-fourth of the x-height.

```
⟨ Construct a superscript box x 802 ⟩ ≡
  begin script_head ← supscr(q); ⟨ Fetch first character of a sub/superscript 805 ⟩;
  sup_g ← script_g; sup_f ← script_f; save_f ← cur_f; x ← clean_box(supscr(q), sup_style(cur_style));
  cur_f ← save_f; width(x) ← width(x) + script_space;
  if odd(cur_style) then clr ← sup3(cur_size)
  else if cur_style < text_style then clr ← sup1(cur_size)
    else clr ← sup2(cur_size);
  if shift_up < clr then shift_up ← clr;
  if is_new_mathfont(cur_f) then clr ← depth(x) + get_ot_math_constant(cur_f, superscriptBottomMin)
  else clr ← depth(x) + (abs(math_x_height(cur_size)) div 4);
  if shift_up < clr then shift_up ← clr;
  if is_new_mathfont(cur_f) then ⟨ Attach superscript OpenType math kerning 807 ⟩
  end
```

This code is used in section 800.

803. When both subscript and superscript are present, the subscript must be separated from the superscript by at least four times *default_rule_thickness*. If this condition would be violated, the subscript moves down, after which both subscript and superscript move up so that the bottom of the superscript is at least as high as the baseline plus four-fifths of the x-height.

```
⟨ Construct a sub/superscript combination box x, with the superscript offset by delta 803 ⟩ ≡
  begin save_f ← cur_f; script_head ← subscr(q); ⟨ Fetch first character of a sub/superscript 805 ⟩;
  sub_g ← script_g; sub_f ← script_f; y ← clean_box(subscr(q), sub_style(cur_style)); cur_f ← save_f;
  width(y) ← width(y) + script_space;
  if shift_down < sub2(cur_size) then shift_down ← sub2(cur_size);
  if is_new_mathfont(cur_f) then clr ← get_ot_math_constant(cur_f,
        subSuperscriptGapMin) − ((shift_up − depth(x)) − (height(y) − shift_down))
  else clr ← 4 ∗ default_rule_thickness − ((shift_up − depth(x)) − (height(y) − shift_down));
  if clr > 0 then
    begin shift_down ← shift_down + clr;
    if is_new_mathfont(cur_f) then
      clr ← get_ot_math_constant(cur_f, superscriptBottomMaxWithSubscript) − (shift_up − depth(x))
    else clr ← (abs(math_x_height(cur_size) ∗ 4) div 5) − (shift_up − depth(x));
    if clr > 0 then
      begin shift_up ← shift_up + clr; shift_down ← shift_down − clr;
      end;
    end;
  if is_new_mathfont(cur_f) then
    begin ⟨ Attach subscript OpenType math kerning 806 ⟩
    ⟨ Attach superscript OpenType math kerning 807 ⟩
    end
  else begin sup_kern ← 0; sub_kern ← 0;
    end;
  shift_amount(x) ← sup_kern + delta − sub_kern; { superscript is delta to the right of the subscript }
  p ← new_kern((shift_up − depth(x)) − (height(y) − shift_down)); link(x) ← p; link(p) ← y;
  x ← vpack(x, natural); shift_amount(x) ← shift_down;
  end
```

This code is used in section 800.

804. OpenType math fonts provide an additional adjustment for the horizontal position of sub/superscripts called math kerning.

The following definitions should be kept in sync with `XeTeXOTMath.cpp`.

```
define sup_cmd = 0 { superscript kern type for get_ot_math_kern }
define sub_cmd = 1 { subscript kern type for get_ot_math_kern }
define is_valid_pointer(#) ≡ ((# ≥ mem_min) ∧ (# ≤ mem_end))
```

805. ⟨ Fetch first character of a sub/superscript 805 ⟩ ≡

```
script_c ← null; script_g ← qi(0);
script_f ← null_font; this_math_style ← sub_style(cur_style); { Loop through the sub_mlist looking for
    the first character-like thing. Ignore kerns or glue so that, for example, changing P_j to P_j will have a
    predictable effect. Intercept style_nodes and execute them. If we encounter a choice_node, follow the
    appropriate branch. Anything else halts the search and inhibits OpenType kerning. }
  { Don't try to do anything clever if the nucleus of the script_head is empty, e.g., P_j and the such. }
if math_type(script_head) = sub_mlist then
  begin script_ptr ← info(script_head); script_head ← null;
  while is_valid_pointer(script_ptr) do
    begin case type(script_ptr) of
    kern_node, glue_node: do_nothing;
    style_node: begin this_math_style ← subtype(script_ptr);
      end;
    choice_node: do_nothing; { see below }
    ord_noad, op_noad, bin_noad, rel_noad, open_noad, close_noad, punct_noad: begin
        script_head ← nucleus(script_ptr); script_ptr ← null;
      end;
    othercases script_ptr ← null { end the search }
    endcases;
    if is_valid_pointer(script_ptr) then
      if type(script_ptr) = choice_node then
        case this_math_style div 2 of
        0: script_ptr ← display_mlist(script_ptr);
        1: script_ptr ← text_mlist(script_ptr);
        2: script_ptr ← script_mlist(script_ptr);
        3: script_ptr ← script_script_mlist(script_ptr);
        end
      else script_ptr ← link(script_ptr);
    end;
  end;
if is_valid_pointer(script_head) ∧ math_type(script_head) = math_char then
  begin save_f ← cur_f; saved_math_style ← cur_style; cur_style ← this_math_style;
  ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
  fetch(script_head);
  if is_new_mathfont(cur_f) then
    begin script_c ← new_native_character(cur_f, qo(cur_c)); script_g ← get_native_glyph(script_c, 0);
    script_f ← cur_f; { script font }
    end;
  cur_f ← save_f; cur_style ← saved_math_style;
  ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
  end; { The remaining case is math_type(script_head) = sub_box. Although it would be possible to
      deconstruct the box node to find the first glyph, it will most likely be from a text font without
      MATH kerning, so there's probably no point. }
```

This code is used in sections 801, 802, and 803.

806. ⟨ Attach subscript OpenType math kerning 806 ⟩ ≡

```
begin if is_glyph_node(p) then
  begin sub_kern ← get_ot_math_kern(native_font(p), native_glyph(p), sub_f, sub_g, sub_cmd, shift_down);
  if sub_kern ≠ 0 then p ← attach_hkern_to_new_hlist(q, sub_kern);
  end;
end;
```

This code is used in sections 801 and 803.

807. ⟨ Attach superscript OpenType math kerning 807 ⟩ ≡

```
begin { if there is a superscript the kern will be added to shift_amount(x) }
if math_type(subscr(q)) = empty then
  if is_glyph_node(p) then
    begin sup_kern ← get_ot_math_kern(native_font(p), native_glyph(p), sup_f, sup_g, sup_cmd, shift_up);
    if sup_kern ≠ 0 then p ← attach_hkern_to_new_hlist(q, sup_kern);
    end;
end;
```

This code is used in sections 802 and 803.

808. We have now tied up all the loose ends of the first pass of *mlist_to_hlist*. The second pass simply goes through and hooks everything together with the proper glue and penalties. It also handles the *left_noad* and *right_noad* that might be present, since *max_h* and *max_d* are now known. Variable p points to a node at the current end of the final hlist.

⟨ Make a second pass over the mlist, removing all noads and inserting the proper spacing and penalties 808 ⟩ ≡

```
p ← temp_head; link(p) ← null; q ← mlist; r_type ← 0; cur_style ← style;
⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
while q ≠ null do
  begin ⟨ If node q is a style node, change the style and goto delete_q; otherwise if it is not a noad, put
      it into the hlist, advance q, and goto done; otherwise set s to the size of noad q, set t to the
      associated type (ord_noad .. inner_noad), and set pen to the associated penalty 809 ⟩;
  ⟨ Append inter-element spacing based on r_type and t 814 ⟩;
  ⟨ Append any new_hlist entries for q, and any appropriate penalties 815 ⟩;
  if type(q) = right_noad then t ← open_noad;
  r_type ← t;
delete_q: r ← q; q ← link(q); free_node(r, s);
done: end
```

This code is used in section 769.

809. Just before doing the big **case** switch in the second pass, the program sets up default values so that most of the branches are short.

```
⟨ If node q is a style node, change the style and goto delete_q; otherwise if it is not a noad, put it into the
      hlist, advance q, and goto done; otherwise set s to the size of noad q, set t to the associated type
      (ord_noad .. inner_noad), and set pen to the associated penalty 809 ⟩ ≡
  t ← ord_noad; s ← noad_size; pen ← inf_penalty;
  case type(q) of
  op_noad, open_noad, close_noad, punct_noad, inner_noad: t ← type(q);
  bin_noad: begin t ← bin_noad; pen ← bin_op_penalty;
    end;
  rel_noad: begin t ← rel_noad; pen ← rel_penalty;
    end;
  ord_noad, vcenter_noad, over_noad, under_noad: do_nothing;
  radical_noad: s ← radical_noad_size;
  accent_noad: s ← accent_noad_size;
  fraction_noad: s ← fraction_noad_size;
  left_noad, right_noad: t ← make_left_right(q, style, max_d, max_h);
  style_node: ⟨ Change the current style and goto delete_q 811 ⟩;
  whatsit_node, penalty_node, rule_node, disc_node, adjust_node, ins_node, mark_node, glue_node, kern_node:
    begin link(p) ← q; p ← q; q ← link(q); link(p) ← null; goto done;
    end;
  othercases confusion("mlist3")
  endcases
```

This code is used in section 808.

810. The *make_left_right* function constructs a left or right delimiter of the required size and returns the value *open_noad* or *close_noad*. The *right_noad* and *left_noad* will both be based on the original *style*, so they will have consistent sizes.

We use the fact that $right_noad - left_noad = close_noad - open_noad$.

```
⟨ Declare math construction procedures 777 ⟩ +≡
function make_left_right(q : pointer; style : small_number; max_d, max_h : scaled): small_number;
  var delta, delta1, delta2: scaled; { dimensions used in the calculation }
  begin cur_style ← style; ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
  delta2 ← max_d + axis_height(cur_size); delta1 ← max_h + max_d − delta2;
  if delta2 > delta1 then delta1 ← delta2; { delta1 is max distance from axis }
  delta ← (delta1 div 500) * delimiter_factor; delta2 ← delta1 + delta1 − delimiter_shortfall;
  if delta < delta2 then delta ← delta2;
  new_hlist(q) ← var_delimiter(delimiter(q), cur_size, delta);
  make_left_right ← type(q) − (left_noad − open_noad); { open_noad or close_noad }
  end;
```

811.

```
⟨ Change the current style and goto delete_q 811 ⟩ ≡
  begin cur_style ← subtype(q); s ← style_node_size;
  ⟨ Set up the values of cur_size and cur_mu, based on cur_style 746 ⟩;
  goto delete_q;
  end
```

This code is used in section 809.

812. The inter-element spacing in math formulas depends on an 8×8 table that TeX preloads as a 64-digit string. The elements of this string have the following significance:

0 means no space;
1 means a conditional thin space (`\nonscript\mskip\thinmuskip`);
2 means a thin space (`\mskip\thinmuskip`);
3 means a conditional medium space (`\nonscript\mskip\medmuskip`);
4 means a conditional thick space (`\nonscript\mskip\thickmuskip`);
* means an impossible case.

This is all pretty cryptic, but *The TeXbook* explains what is supposed to happen, and the string makes it happen.

A global variable *magic_offset* is computed so that if a and b are in the range *ord_noad* .. *inner_noad*, then *str_pool*$[a * 8 + b + magic_offset]$ is the digit for spacing between noad types a and b.

If Pascal had provided a good way to preload constant arrays, this part of the program would not have been so strange.

```
define math_spacing =
"0234000122*4000133**3**344*0400400*000000234000111*1111112341011"
```

⟨ Global variables 13 ⟩ +≡
magic_offset: *integer*; { used to find inter-element spacing }

813. ⟨ Compute the magic offset 813 ⟩ ≡
magic_offset ← *str_start_macro*(*math_spacing*) − 9 ∗ *ord_noad*
This code is used in section 1391.

814. ⟨ Append inter-element spacing based on *r_type* and *t* 814 ⟩ ≡

```
if r_type > 0 then { not the first noad }
  begin case so(str_pool[r_type * 8 + t + magic_offset]) of
  "0": x ← 0;
  "1": if cur_style < script_style then x ← thin_mu_skip_code else x ← 0;
  "2": x ← thin_mu_skip_code;
  "3": if cur_style < script_style then x ← med_mu_skip_code else x ← 0;
  "4": if cur_style < script_style then x ← thick_mu_skip_code else x ← 0;
  othercases confusion("mlist4")
  endcases;
  if x ≠ 0 then
    begin y ← math_glue(glue_par(x), cur_mu); z ← new_glue(y); glue_ref_count(y) ← null;
    link(p) ← z; p ← z;
    subtype(z) ← x + 1; { store a symbolic subtype }
    end;
  end
```

This code is used in section 808.

815. We insert a penalty node after the hlist entries of noad q if *pen* is not an "infinite" penalty, and if the node immediately following q is not a penalty node or a *rel_noad* or absent entirely.

```
⟨ Append any new_hlist entries for q, and any appropriate penalties 815 ⟩ ≡
  if new_hlist(q) ≠ null then
    begin link(p) ← new_hlist(q);
    repeat p ← link(p);
    until link(p) = null;
    end;
  if penalties then
    if link(q) ≠ null then
      if pen < inf_penalty then
        begin r_type ← type(link(q));
        if r_type ≠ penalty_node then
          if r_type ≠ rel_noad then
            begin z ← new_penalty(pen); link(p) ← z; p ← z;
            end;
        end
```

This code is used in section 808.

816. Alignment. It's sort of a miracle whenever `\halign` and `\valign` work, because they cut across so many of the control structures of TeX.

Therefore the present page is probably not the best place for a beginner to start reading this program; it is better to master everything else first.

Let us focus our thoughts on an example of what the input might be, in order to get some idea about how the alignment miracle happens. The example doesn't do anything useful, but it is sufficiently general to indicate all of the special cases that must be dealt with; please do not be disturbed by its apparent complexity and meaninglessness.

```
\tabskip 2pt plus 3pt
\halign to 300pt{u1#v1&
        \tabskip 1pt plus 1fil u2#v2&
        u3#v3\cr
    a1&\omit a2&\vrule\cr
    \noalign{\vskip 3pt}
    b1\span b2\cr
    \omit&c2\span\omit\cr}
```

Here's what happens:

(0) When '`\halign to 300pt{`' is scanned, the *scan_spec* routine places the 300pt dimension onto the *save_stack*, and an *align_group* code is placed above it. This will make it possible to complete the alignment when the matching '`}`' is found.

(1) The preamble is scanned next. Macros in the preamble are not expanded, except as part of a tabskip specification. For example, if `u2` had been a macro in the preamble above, it would have been expanded, since TeX must look for '`minus...`' as part of the tabskip glue. A "preamble list" is constructed based on the user's preamble; in our case it contains the following seven items:

`\glue 2pt plus 3pt`	(the tabskip preceding column 1)
`\alignrecord, width` $-\infty$	(preamble info for column 1)
`\glue 2pt plus 3pt`	(the tabskip between columns 1 and 2)
`\alignrecord, width` $-\infty$	(preamble info for column 2)
`\glue 1pt plus 1fil`	(the tabskip between columns 2 and 3)
`\alignrecord, width` $-\infty$	(preamble info for column 3)
`\glue 1pt plus 1fil`	(the tabskip following column 3)

These "alignrecord" entries have the same size as an *unset_node*, since they will later be converted into such nodes. However, at the moment they have no *type* or *subtype* fields; they have *info* fields instead, and these *info* fields are initially set to the value *end_span*, for reasons explained below. Furthermore, the alignrecord nodes have no *height* or *depth* fields; these are renamed *u_part* and *v_part*, and they point to token lists for the templates of the alignment. For example, the *u_part* field in the first alignrecord points to the token list '`u1`', i.e., the template preceding the '`#`' for column 1.

(2) TeX now looks at what follows the `\cr` that ended the preamble. It is not '`\noalign`' or '`\omit`', so this input is put back to be read again, and the template '`u1`' is fed to the scanner. Just before reading '`u1`', TeX goes into restricted horizontal mode. Just after reading '`u1`', TeX will see '`a1`', and then (when the `&` is sensed) TeX will see '`v1`'. Then TeX scans an *endv* token, indicating the end of a column. At this point an *unset_node* is created, containing the contents of the current hlist (i.e., '`u1a1v1`'). The natural width of this unset node replaces the *width* field of the alignrecord for column 1; in general, the alignrecords will record the maximum natural width that has occurred so far in a given column.

(3) Since '`\omit`' follows the '`&`', the templates for column 2 are now bypassed. Again TeX goes into restricted horizontal mode and makes an *unset_node* from the resulting hlist; but this time the hlist contains simply '`a2`'. The natural width of the new unset box is remembered in the *width* field of the alignrecord for column 2.

(4) A third *unset_node* is created for column 3, using essentially the mechanism that worked for column 1; this unset box contains '`u3\vrule v3`'. The vertical rule in this case has running dimensions that will later

extend to the height and depth of the whole first row, since each *unset_node* in a row will eventually inherit the height and depth of its enclosing box.

(5) The first row has now ended; it is made into a single unset box comprising the following seven items:

```
\glue 2pt plus 3pt
\unsetbox for 1 column: u1a1v1
\glue 2pt plus 3pt
\unsetbox for 1 column: a2
\glue 1pt plus 1fil
\unsetbox for 1 column: u3\vrule v3
\glue 1pt plus 1fil
```

The width of this unset row is unimportant, but it has the correct height and depth, so the correct baselineskip glue will be computed as the row is inserted into a vertical list.

(6) Since '`\noalign`' follows the current `\cr`, TEX appends additional material (in this case `\vskip 3pt`) to the vertical list. While processing this material, TEX will be in internal vertical mode, and *no_align_group* will be on *save_stack*.

(7) The next row produces an unset box that looks like this:

```
\glue 2pt plus 3pt
\unsetbox for 2 columns: u1b1v1u2b2v2
\glue 1pt plus 1fil
\unsetbox for 1 column: (empty)
\glue 1pt plus 1fil
```

The natural width of the unset box that spans columns 1 and 2 is stored in a "span node," which we will explain later; the *info* field of the alignrecord for column 1 now points to the new span node, and the *info* of the span node points to *end_span*.

(8) The final row produces the unset box

```
\glue 2pt plus 3pt
\unsetbox for 1 column: (empty)
\glue 2pt plus 3pt
\unsetbox for 2 columns: u2c2v2
\glue 1pt plus 1fil
```

A new span node is attached to the alignrecord for column 2.

(9) The last step is to compute the true column widths and to change all the unset boxes to hboxes, appending the whole works to the vertical list that encloses the `\halign`. The rules for deciding on the final widths of each unset column box will be explained below.

Note that as `\halign` is being processed, we fearlessly give up control to the rest of TEX. At critical junctures, an alignment routine is called upon to step in and do some little action, but most of the time these routines just lurk in the background. It's something like post-hypnotic suggestion.

817. We have mentioned that alignrecords contain no *height* or *depth* fields. Their *glue_sign* and *glue_order* are pre-empted as well, since it is necessary to store information about what to do when a template ends. This information is called the *extra_info* field.

define *u_part*(#) ≡ *mem*[# + *height_offset*].*int* { pointer to $\langle u_j \rangle$ token list }
define *v_part*(#) ≡ *mem*[# + *depth_offset*].*int* { pointer to $\langle v_j \rangle$ token list }
define *extra_info*(#) ≡ *info*(# + *list_offset*) { info to remember during template }

818. Alignments can occur within alignments, so a small stack is used to access the alignrecord information. At each level we have a *preamble* pointer, indicating the beginning of the preamble list; a *cur_align* pointer, indicating the current position in the preamble list; a *cur_span* pointer, indicating the value of *cur_align* at the beginning of a sequence of spanned columns; a *cur_loop* pointer, indicating the tabskip glue before an alignrecord that should be copied next if the current list is extended; and the *align_state* variable, which indicates the nesting of braces so that `\cr` and `\span` and tab marks are properly intercepted. There also are pointers *cur_head* and *cur_tail* to the head and tail of a list of adjustments being moved out from horizontal mode to vertical mode.

The current values of these seven quantities appear in global variables; when they have to be pushed down, they are stored in 5-word nodes, and *align_ptr* points to the topmost such node.

define *preamble* ≡ *link*(*align_head*) { the current preamble list }
define *align_stack_node_size* = 6 { number of *mem* words to save alignment states }

⟨ Global variables 13 ⟩ +≡
cur_align: *pointer*; { current position in preamble list }
cur_span: *pointer*; { start of currently spanned columns in preamble list }
cur_loop: *pointer*; { place to copy when extending a periodic preamble }
align_ptr: *pointer*; { most recently pushed-down alignment stack node }
cur_head, *cur_tail*: *pointer*; { adjustment list pointers }
cur_pre_head, *cur_pre_tail*: *pointer*; { pre-adjustment list pointers }

819. The *align_state* and *preamble* variables are initialized elsewhere.

⟨ Set initial values of key variables 23 ⟩ +≡
align_ptr ← *null*; *cur_align* ← *null*; *cur_span* ← *null*; *cur_loop* ← *null*; *cur_head* ← *null*;
cur_tail ← *null*; *cur_pre_head* ← *null*; *cur_pre_tail* ← *null*;

820. Alignment stack maintenance is handled by a pair of trivial routines called *push_alignment* and *pop_alignment*.

procedure *push_alignment*;
var *p*: *pointer*; { the new alignment stack node }
begin *p* ← *get_node*(*align_stack_node_size*); *link*(*p*) ← *align_ptr*; *info*(*p*) ← *cur_align*;
llink(*p*) ← *preamble*; *rlink*(*p*) ← *cur_span*; *mem*[*p* + 2].*int* ← *cur_loop*; *mem*[*p* + 3].*int* ← *align_state*;
info(*p* + 4) ← *cur_head*; *link*(*p* + 4) ← *cur_tail*; *info*(*p* + 5) ← *cur_pre_head*; *link*(*p* + 5) ← *cur_pre_tail*;
align_ptr ← *p*; *cur_head* ← *get_avail*; *cur_pre_head* ← *get_avail*;
end;

procedure *pop_alignment*;
var *p*: *pointer*; { the top alignment stack node }
begin *free_avail*(*cur_head*); *free_avail*(*cur_pre_head*); *p* ← *align_ptr*; *cur_tail* ← *link*(*p* + 4);
cur_head ← *info*(*p* + 4); *cur_pre_tail* ← *link*(*p* + 5); *cur_pre_head* ← *info*(*p* + 5);
align_state ← *mem*[*p* + 3].*int*; *cur_loop* ← *mem*[*p* + 2].*int*; *cur_span* ← *rlink*(*p*); *preamble* ← *llink*(*p*);
cur_align ← *info*(*p*); *align_ptr* ← *link*(*p*); *free_node*(*p*, *align_stack_node_size*);
end;

821. TeX has eight procedures that govern alignments: *init_align* and *fin_align* are used at the very beginning and the very end; *init_row* and *fin_row* are used at the beginning and end of individual rows; *init_span* is used at the beginning of a sequence of spanned columns (possibly involving only one column); *init_col* and *fin_col* are used at the beginning and end of individual columns; and *align_peek* is used after `\cr` to see whether the next item is `\noalign`.

We shall consider these routines in the order they are first used during the course of a complete `\halign`, namely *init_align*, *align_peek*, *init_row*, *init_span*, *init_col*, *fin_col*, *fin_row*, *fin_align*.

822. When \halign or \valign has been scanned in an appropriate mode, TeX calls *init_align*, whose task is to get everything off to a good start. This mostly involves scanning the preamble and putting its information into the preamble list.

```
⟨ Declare the procedure called get_preamble_token 830 ⟩
procedure align_peek; forward;
procedure normal_paragraph; forward;
procedure init_align;
  label done, done1, done2, continue;
  var save_cs_ptr: pointer;  { warning_index value for error messages }
    p: pointer;  { for short-term temporary use }
  begin save_cs_ptr ← cur_cs;  { \halign or \valign, usually }
  push_alignment; align_state ← −1000000;  { enter a new alignment level }
  ⟨ Check for improper alignment in displayed math 824 ⟩;
  push_nest;  { enter a new semantic level }
  ⟨ Change current mode to −vmode for \halign, −hmode for \valign 823 ⟩;
  scan_spec(align_group, false);
  ⟨ Scan the preamble and record it in the preamble list 825 ⟩;
  new_save_level(align_group);
  if every_cr ≠ null then begin_token_list(every_cr, every_cr_text);
  align_peek;  { look for \noalign or \omit }
  end;
```

823. In vertical modes, *prev_depth* already has the correct value. But if we are in *mmode* (displayed formula mode), we reach out to the enclosing vertical mode for the *prev_depth* value that produces the correct baseline calculations.

```
⟨ Change current mode to −vmode for \halign, −hmode for \valign 823 ⟩ ≡
  if mode = mmode then
    begin mode ← −vmode; prev_depth ← nest[nest_ptr − 2].aux_field.sc;
    end
  else if mode > 0 then negate(mode)
```

This code is used in section 822.

824. When \halign is used as a displayed formula, there should be no other pieces of mlists present.

```
⟨ Check for improper alignment in displayed math 824 ⟩ ≡
  if (mode = mmode) ∧ ((tail ≠ head) ∨ (incompleat_noad ≠ null)) then
    begin print_err("Improper␣"); print_esc("halign"); print("␣inside␣$$´s");
    help3("Displays␣can␣use␣special␣alignments␣(like␣\eqalignno)")
    ("only␣if␣nothing␣but␣the␣alignment␣itself␣is␣between␣$$´s.")
    ("So␣I´ve␣deleted␣the␣formulas␣that␣preceded␣this␣alignment."); error; flush_math;
    end
```

This code is used in section 822.

825. ⟨ Scan the preamble and record it in the *preamble* list 825 ⟩ ≡
preamble ← *null*; *cur_align* ← *align_head*; *cur_loop* ← *null*; *scanner_status* ← *aligning*;
warning_index ← *save_cs_ptr*; *align_state* ← −1000000; { at this point, *cur_cmd* = *left_brace* }
loop begin ⟨ Append the current tabskip glue to the preamble list 826 ⟩;
if *cur_cmd* = *car_ret* **then goto** *done*; { `\cr` ends the preamble }
⟨ Scan preamble text until *cur_cmd* is *tab_mark* or *car_ret*, looking for changes in the tabskip glue; append an alignrecord to the preamble list 827 ⟩;
end;
done: *scanner_status* ← *normal*

This code is used in section 822.

826. ⟨ Append the current tabskip glue to the preamble list 826 ⟩ ≡
link(*cur_align*) ← *new_param_glue*(*tab_skip_code*); *cur_align* ← *link*(*cur_align*)

This code is used in section 825.

827. ⟨ Scan preamble text until *cur_cmd* is *tab_mark* or *car_ret*, looking for changes in the tabskip glue; append an alignrecord to the preamble list 827 ⟩ ≡
⟨ Scan the template ⟨u_j⟩, putting the resulting token list in *hold_head* 831 ⟩;
link(*cur_align*) ← *new_null_box*; *cur_align* ← *link*(*cur_align*); { a new alignrecord }
info(*cur_align*) ← *end_span*; *width*(*cur_align*) ← *null_flag*; *u_part*(*cur_align*) ← *link*(*hold_head*);
⟨ Scan the template ⟨v_j⟩, putting the resulting token list in *hold_head* 832 ⟩;
v_part(*cur_align*) ← *link*(*hold_head*)

This code is used in section 825.

828. We enter '`\span`' into *eqtb* with *tab_mark* as its command code, and with *span_code* as the command modifier. This makes TeX interpret it essentially the same as an alignment delimiter like '`&`', yet it is recognizably different when we need to distinguish it from a normal delimiter. It also turns out to be useful to give a special *cr_code* to '`\cr`', and an even larger *cr_cr_code* to '`\crcr`'.

The end of a template is represented by two "frozen" control sequences called `\endtemplate`. The first has the command code *end_template*, which is > *outer_call*, so it will not easily disappear in the presence of errors. The *get_x_token* routine converts the first into the second, which has *endv* as its command code.

define *span_code* = *special_char* { distinct from any character }
define *cr_code* = *span_code* + 1 { distinct from *span_code* and from any character }
define *cr_cr_code* = *cr_code* + 1 { this distinguishes `\crcr` from `\cr` }
define *end_template_token* ≡ *cs_token_flag* + *frozen_end_template*

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("span", *tab_mark*, *span_code*);
primitive("cr", *car_ret*, *cr_code*); *text*(*frozen_cr*) ← "cr"; *eqtb*[*frozen_cr*] ← *eqtb*[*cur_val*];
primitive("crcr", *car_ret*, *cr_cr_code*); *text*(*frozen_end_template*) ← "endtemplate";
text(*frozen_endv*) ← "endtemplate"; *eq_type*(*frozen_endv*) ← *endv*; *equiv*(*frozen_endv*) ← *null_list*;
eq_level(*frozen_endv*) ← *level_one*;
eqtb[*frozen_end_template*] ← *eqtb*[*frozen_endv*]; *eq_type*(*frozen_end_template*) ← *end_template*;

829. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
tab_mark: **if** *chr_code* = *span_code* **then** *print_esc*("span")
else *chr_cmd*("alignment␣tab␣character␣");
car_ret: **if** *chr_code* = *cr_code* **then** *print_esc*("cr")
else *print_esc*("crcr");

830. The preamble is copied directly, except that `\tabskip` causes a change to the tabskip glue, thereby possibly expanding macros that immediately follow it. An appearance of `\span` also causes such an expansion.

Note that if the preamble contains '`\global\tabskip`', the '`\global`' token survives in the preamble and the '`\tabskip`' defines new tabskip glue (locally).

```
⟨ Declare the procedure called get_preamble_token 830 ⟩ ≡
procedure get_preamble_token;
  label restart;
  begin restart: get_token;
  while (cur_chr = span_code) ∧ (cur_cmd = tab_mark) do
    begin get_token;   { this token will be expanded once }
    if cur_cmd > max_command then
      begin expand; get_token;
      end;
    end;
  if cur_cmd = endv then fatal_error("(interwoven␣alignment␣preambles␣are␣not␣allowed)");
  if (cur_cmd = assign_glue) ∧ (cur_chr = glue_base + tab_skip_code) then
    begin scan_optional_equals; scan_glue(glue_val);
    if global_defs > 0 then geq_define(glue_base + tab_skip_code, glue_ref, cur_val)
    else eq_define(glue_base + tab_skip_code, glue_ref, cur_val);
    goto restart;
    end;
  end;
```

This code is used in section 822.

831. Spaces are eliminated from the beginning of a template.

```
⟨ Scan the template ⟨u_j⟩, putting the resulting token list in hold_head 831 ⟩ ≡
  p ← hold_head; link(p) ← null;
  loop begin get_preamble_token;
    if cur_cmd = mac_param then goto done1;
    if (cur_cmd ≤ car_ret) ∧ (cur_cmd ≥ tab_mark) ∧ (align_state = −1000000) then
      if (p = hold_head) ∧ (cur_loop = null) ∧ (cur_cmd = tab_mark) then cur_loop ← cur_align
      else begin print_err("Missing␣#␣inserted␣in␣alignment␣preamble");
        help3("There␣should␣be␣exactly␣one␣#␣between␣&´s,␣when␣an")
        ("\halign␣or␣\valign␣is␣being␣set␣up.␣In␣this␣case␣you␣had")
        ("none,␣so␣I´ve␣put␣one␣in;␣maybe␣that␣will␣work."); back_error; goto done1;
        end
    else if (cur_cmd ≠ spacer) ∨ (p ≠ hold_head) then
        begin link(p) ← get_avail; p ← link(p); info(p) ← cur_tok;
        end;
    end;
done1:
```

This code is used in section 827.

832. ⟨ Scan the template ⟨v_j⟩, putting the resulting token list in *hold_head* 832 ⟩ ≡

```
p ← hold_head; link(p) ← null;
loop begin continue: get_preamble_token;
  if (cur_cmd ≤ car_ret) ∧ (cur_cmd ≥ tab_mark) ∧ (align_state = −1000000) then goto done2;
  if cur_cmd = mac_param then
    begin print_err("Only␣one␣#␣is␣allowed␣per␣tab");
    help3("There␣should␣be␣exactly␣one␣#␣between␣&´s,␣when␣an")
    ("\halign␣or␣\valign␣is␣being␣set␣up.␣In␣this␣case␣you␣had")
    ("more␣than␣one,␣so␣I´m␣ignoring␣all␣but␣the␣first."); error; goto continue;
    end;
  link(p) ← get_avail; p ← link(p); info(p) ← cur_tok;
  end;
done2: link(p) ← get_avail; p ← link(p); info(p) ← end_template_token   { put \endtemplate at the end }
```

This code is used in section 827.

833. The tricky part about alignments is getting the templates into the scanner at the right time, and recovering control when a row or column is finished.

We usually begin a row after each `\cr` has been sensed, unless that `\cr` is followed by `\noalign` or by the right brace that terminates the alignment. The *align_peek* routine is used to look ahead and do the right thing; it either gets a new row started, or gets a `\noalign` started, or finishes off the alignment.

⟨ Declare the procedure called *align_peek* 833 ⟩ ≡

```
procedure align_peek;
  label restart;
  begin restart: align_state ← 1000000;
  repeat get_x_or_protected;
  until cur_cmd ≠ spacer;
  if cur_cmd = no_align then
    begin scan_left_brace; new_save_level(no_align_group);
    if mode = −vmode then normal_paragraph;
    end
  else if cur_cmd = right_brace then fin_align
    else if (cur_cmd = car_ret) ∧ (cur_chr = cr_cr_code) then goto restart   { ignore \crcr }
      else begin init_row;   { start a new row }
        init_col;   { start a new column and replace what we peeked at }
        end;
  end;
```

This code is used in section 848.

834. To start a row (i.e., a 'row' that rhymes with 'dough' but not with 'bough'), we enter a new semantic level, copy the first tabskip glue, and change from internal vertical mode to restricted horizontal mode or vice versa. The *space_factor* and *prev_depth* are not used on this semantic level, but we clear them to zero just to be tidy.

⟨ Declare the procedure called *init_span* 835 ⟩

```
procedure init_row;
  begin push_nest; mode ← (−hmode − vmode) − mode;
  if mode = −hmode then space_factor ← 0 else prev_depth ← 0;
  tail_append(new_glue(glue_ptr(preamble))); subtype(tail) ← tab_skip_code + 1;
  cur_align ← link(preamble); cur_tail ← cur_head; cur_pre_tail ← cur_pre_head; init_span(cur_align);
  end;
```

835. The parameter to *init_span* is a pointer to the alignrecord where the next column or group of columns will begin. A new semantic level is entered, so that the columns will generate a list for subsequent packaging.

```
⟨ Declare the procedure called init_span 835 ⟩ ≡
procedure init_span(p : pointer);
  begin push_nest;
  if mode = −hmode then space_factor ← 1000
  else begin prev_depth ← ignore_depth; normal_paragraph;
    end;
  cur_span ← p;
  end;
```

This code is used in section 834.

836. When a column begins, we assume that *cur_cmd* is either *omit* or else the current token should be put back into the input until the $\langle u_j \rangle$ template has been scanned. (Note that *cur_cmd* might be *tab_mark* or *car_ret*.) We also assume that *align_state* is approximately 1000000 at this time. We remain in the same mode, and start the template if it is called for.

```
procedure init_col;
  begin extra_info(cur_align) ← cur_cmd;
  if cur_cmd = omit then align_state ← 0
  else begin back_input; begin_token_list(u_part(cur_align), u_template);
    end; { now align_state = 1000000 }
  end;
```

837. The scanner sets *align_state* to zero when the $\langle u_j \rangle$ template ends. When a subsequent `\cr` or `\span` or tab mark occurs with *align_state* = 0, the scanner activates the following code, which fires up the $\langle v_j \rangle$ template. We need to remember the *cur_chr*, which is either *cr_cr_code*, *cr_code*, *span_code*, or a character code, depending on how the column text has ended.

This part of the program had better not be activated when the preamble to another alignment is being scanned, or when no alignment preamble is active.

```
⟨ Insert the ⟨v_j⟩ template and goto restart 837 ⟩ ≡
  begin if (scanner_status = aligning) ∨ (cur_align = null) then
    fatal_error("(interwoven␣alignment␣preambles␣are␣not␣allowed)");
  cur_cmd ← extra_info(cur_align); extra_info(cur_align) ← cur_chr;
  if cur_cmd = omit then begin_token_list(omit_template, v_template)
  else begin_token_list(v_part(cur_align), v_template);
  align_state ← 1000000; goto restart;
  end
```

This code is used in section 372.

838. The token list *omit_template* just referred to is a constant token list that contains the special control sequence `\endtemplate` only.

```
⟨ Initialize the special list heads and constant nodes 838 ⟩ ≡
  info(omit_template) ← end_template_token; { link(omit_template) = null }
```

See also sections 845, 868, 1035, and 1042.

This code is used in section 189.

839. When the *endv* command at the end of a ⟨ v_j ⟩ template comes through the scanner, things really start to happen; and it is the *fin_col* routine that makes them happen. This routine returns *true* if a row as well as a column has been finished.

```
function fin_col: boolean;
  label exit;
  var p: pointer;  { the alignrecord after the current one }
    q, r: pointer;  { temporary pointers for list manipulation }
    s: pointer;  { a new span node }
    u: pointer;  { a new unset box }
    w: scaled;  { natural width }
    o: glue_ord;  { order of infinity }
    n: halfword;  { span counter }
  begin if cur_align = null then confusion("endv");
  q ← link(cur_align); if q = null then confusion("endv");
  if align_state < 500000 then fatal_error("(interwoven␣alignment␣preambles␣are␣not␣allowed)");
  p ← link(q); ⟨ If the preamble list has been traversed, check that the row has ended 840 ⟩;
  if extra_info(cur_align) ≠ span_code then
    begin unsave; new_save_level(align_group);
    ⟨ Package an unset box for the current column and record its width 844 ⟩;
    ⟨ Copy the tabskip glue between columns 843 ⟩;
    if extra_info(cur_align) ≥ cr_code then
      begin fin_col ← true; return;
      end;
    init_span(p);
    end;
  align_state ← 1000000;
  repeat get_x_or_protected;
  until cur_cmd ≠ spacer;
  cur_align ← p; init_col; fin_col ← false;
exit: end;
```

840. ⟨ If the preamble list has been traversed, check that the row has ended 840 ⟩ ≡

```
if (p = null) ∧ (extra_info(cur_align) < cr_code) then
  if cur_loop ≠ null then ⟨ Lengthen the preamble periodically 841 ⟩
  else begin print_err("Extra␣alignment␣tab␣has␣been␣changed␣to␣"); print_esc("cr");
    help3("You␣have␣given␣more␣\span␣or␣&␣marks␣than␣there␣were")
    ("in␣the␣preamble␣to␣the␣\halign␣or␣\valign␣now␣in␣progress.")
    ("So␣I´ll␣assume␣that␣you␣meant␣to␣type␣\cr␣instead."); extra_info(cur_align) ← cr_code;
    error;
    end
```

This code is used in section 839.

841. ⟨ Lengthen the preamble periodically 841 ⟩ ≡

```
begin link(q) ← new_null_box; p ← link(q);  { a new alignrecord }
info(p) ← end_span; width(p) ← null_flag; cur_loop ← link(cur_loop);
⟨ Copy the templates from node cur_loop into node p 842 ⟩;
cur_loop ← link(cur_loop); link(p) ← new_glue(glue_ptr(cur_loop)); subtype(link(p)) ← tab_skip_code + 1;
end
```

This code is used in section 840.

842. ⟨ Copy the templates from node *cur_loop* into node *p* 842 ⟩ ≡

```
q ← hold_head; r ← u_part(cur_loop);
while r ≠ null do
  begin link(q) ← get_avail; q ← link(q); info(q) ← info(r); r ← link(r);
  end;
link(q) ← null; u_part(p) ← link(hold_head); q ← hold_head; r ← v_part(cur_loop);
while r ≠ null do
  begin link(q) ← get_avail; q ← link(q); info(q) ← info(r); r ← link(r);
  end;
link(q) ← null; v_part(p) ← link(hold_head)
```

This code is used in section 841.

843. ⟨ Copy the tabskip glue between columns 843 ⟩ ≡

```
tail_append(new_glue(glue_ptr(link(cur_align)))); subtype(tail) ← tab_skip_code + 1
```

This code is used in section 839.

844. ⟨ Package an unset box for the current column and record its width 844 ⟩ ≡

```
begin if mode = −hmode then
  begin adjust_tail ← cur_tail; pre_adjust_tail ← cur_pre_tail; u ← hpack(link(head), natural);
  w ← width(u); cur_tail ← adjust_tail; adjust_tail ← null; cur_pre_tail ← pre_adjust_tail;
  pre_adjust_tail ← null;
  end
else begin u ← vpackage(link(head), natural, 0); w ← height(u);
  end;
n ← min_quarterword; { this represents a span count of 1 }
if cur_span ≠ cur_align then ⟨ Update width entry for spanned columns 846 ⟩
else if w > width(cur_align) then width(cur_align) ← w;
type(u) ← unset_node; span_count(u) ← n;
⟨ Determine the stretch order 701 ⟩;
glue_order(u) ← o; glue_stretch(u) ← total_stretch[o];
⟨ Determine the shrink order 707 ⟩;
glue_sign(u) ← o; glue_shrink(u) ← total_shrink[o];
pop_nest; link(tail) ← u; tail ← u;
end
```

This code is used in section 839.

845. A span node is a 2-word record containing *width*, *info*, and *link* fields. The *link* field is not really a link, it indicates the number of spanned columns; the *info* field points to a span node for the same starting column, having a greater extent of spanning, or to *end_span*, which has the largest possible *link* field; the *width* field holds the largest natural width corresponding to a particular set of spanned columns.

A list of the maximum widths so far, for spanned columns starting at a given column, begins with the *info* field of the alignrecord for that column.

```
define span_node_size = 2  { number of mem words for a span node }
```

⟨ Initialize the special list heads and constant nodes 838 ⟩ +≡

```
link(end_span) ← max_quarterword + 1; info(end_span) ← null;
```

846. ⟨ Update width entry for spanned columns 846 ⟩ ≡

```
begin q ← cur_span;
repeat incr(n); q ← link(link(q));
until q = cur_align;
if n > max_quarterword then confusion("too many spans"); { this can happen, but won't }
q ← cur_span;
while link(info(q)) < n do q ← info(q);
if link(info(q)) > n then
  begin s ← get_node(span_node_size); info(s) ← info(q); link(s) ← n; info(q) ← s; width(s) ← w;
  end
else if width(info(q)) < w then width(info(q)) ← w;
end
```

This code is used in section 844.

847. At the end of a row, we append an unset box to the current vlist (for `\halign`) or the current hlist (for `\valign`). This unset box contains the unset boxes for the columns, separated by the tabskip glue. Everything will be set later.

```
procedure fin_row;
  var p: pointer; { the new unset box }
  begin if mode = −hmode then
    begin p ← hpack(link(head), natural); pop_nest;
    if cur_pre_head ≠ cur_pre_tail then append_list(cur_pre_head)(cur_pre_tail);
    append_to_vlist(p);
    if cur_head ≠ cur_tail then append_list(cur_head)(cur_tail);
    end
  else begin p ← vpack(link(head), natural); pop_nest; link(tail) ← p; tail ← p; space_factor ← 1000;
    end;
  type(p) ← unset_node; glue_stretch(p) ← 0;
  if every_cr ≠ null then begin_token_list(every_cr, every_cr_text);
  align_peek;
  end; { note that glue_shrink(p) = 0 since glue_shrink ≡ shift_amount }
```

848. Finally, we will reach the end of the alignment, and we can breathe a sigh of relief that memory hasn't overflowed. All the unset boxes will now be set so that the columns line up, taking due account of spanned columns.

```
procedure do_assignments; forward;
procedure resume_after_display; forward;
procedure build_page; forward;
procedure fin_align;
  var p,q,r,s,u,v: pointer; { registers for the list operations }
    t,w: scaled; { width of column }
    o: scaled; { shift offset for unset boxes }
    n: halfword; { matching span amount }
    rule_save: scaled; { temporary storage for overfull_rule }
    aux_save: memory_word; { temporary storage for aux }
  begin if cur_group ≠ align_group then confusion("align1");
  unsave; { that align_group was for individual entries }
  if cur_group ≠ align_group then confusion("align0");
  unsave; { that align_group was for the whole alignment }
  if nest[nest_ptr − 1].mode_field = mmode then o ← display_indent
  else o ← 0;
  ⟨ Go through the preamble list, determining the column widths and changing the alignrecords to dummy
      unset boxes 849 ⟩;
  ⟨ Package the preamble list, to determine the actual tabskip glue amounts, and let p point to this
      prototype box 852 ⟩;
  ⟨ Set the glue in all the unset boxes of the current list 853 ⟩;
  flush_node_list(p); pop_alignment; ⟨ Insert the current list into its environment 860 ⟩;
  end;
⟨ Declare the procedure called align_peek 833 ⟩
```

849. It's time now to dismantle the preamble list and to compute the column widths. Let w_{ij} be the maximum of the natural widths of all entries that span columns i through j, inclusive. The alignrecord for column i contains w_{ii} in its *width* field, and there is also a linked list of the nonzero w_{ij} for increasing j, accessible via the *info* field; these span nodes contain the value $j - i + min_quarterword$ in their *link* fields. The values of w_{ii} were initialized to *null_flag*, which we regard as $-\infty$.

The final column widths are defined by the formula

$$w_j = \max_{1\le i\le j}\left(w_{ij} - \sum_{i\le k<j}(t_k + w_k)\right),$$

where t_k is the natural width of the tabskip glue between columns k and $k+1$. However, if $w_{ij} = -\infty$ for all i in the range $1 \le i \le j$ (i.e., if every entry that involved column j also involved column $j+1$), we let $w_j = 0$, and we zero out the tabskip glue after column j.

TeX computes these values by using the following scheme: First $w_1 = w_{11}$. Then replace w_{2j} by $\max(w_{2j}, w_{1j} - t_1 - w_1)$, for all $j > 1$. Then $w_2 = w_{22}$. Then replace w_{3j} by $\max(w_{3j}, w_{2j} - t_2 - w_2)$ for all $j > 2$; and so on. If any w_j turns out to be $-\infty$, its value is changed to zero and so is the next tabskip.

```
⟨ Go through the preamble list, determining the column widths and changing the alignrecords to dummy
      unset boxes 849 ⟩ ≡
  q ← link(preamble);
  repeat flush_list(u_part(q)); flush_list(v_part(q)); p ← link(link(q));
    if width(q) = null_flag then ⟨ Nullify width(q) and the tabskip glue following this column 850 ⟩;
    if info(q) ≠ end_span then
      ⟨ Merge the widths in the span nodes of q with those of p, destroying the span nodes of q 851 ⟩;
    type(q) ← unset_node; span_count(q) ← min_quarterword; height(q) ← 0; depth(q) ← 0;
    glue_order(q) ← normal; glue_sign(q) ← normal; glue_stretch(q) ← 0; glue_shrink(q) ← 0; q ← p;
  until q = null
```

This code is used in section 848.

850.

```
⟨ Nullify width(q) and the tabskip glue following this column 850 ⟩ ≡
  begin width(q) ← 0; r ← link(q); s ← glue_ptr(r);
  if s ≠ zero_glue then
    begin add_glue_ref(zero_glue); delete_glue_ref(s); glue_ptr(r) ← zero_glue;
    end;
  end
```

This code is used in section 849.

851. Merging of two span-node lists is a typical exercise in the manipulation of linearly linked data structures. The essential invariant in the following **repeat** loop is that we want to dispense with node r, in q's list, and u is its successor; all nodes of p's list up to and including s have been processed, and the successor of s matches r or precedes r or follows r, according as $link(r) = n$ or $link(r) > n$ or $link(r) < n$.

```
⟨ Merge the widths in the span nodes of q with those of p, destroying the span nodes of q 851 ⟩ ≡
  begin t ← width(q) + width(glue_ptr(link(q))); r ← info(q); s ← end_span; info(s) ← p;
  n ← min_quarterword + 1;
  repeat width(r) ← width(r) − t; u ← info(r);
    while link(r) > n do
      begin s ← info(s); n ← link(info(s)) + 1;
      end;
    if link(r) < n then
      begin info(r) ← info(s); info(s) ← r; decr(link(r)); s ← r;
      end
    else begin if width(r) > width(info(s)) then width(info(s)) ← width(r);
      free_node(r, span_node_size);
      end;
    r ← u;
  until r = end_span;
  end
```

This code is used in section 849.

852. Now the preamble list has been converted to a list of alternating unset boxes and tabskip glue, where the box widths are equal to the final column sizes. In case of `\valign`, we change the widths to heights, so that a correct error message will be produced if the alignment is overfull or underfull.

```
⟨ Package the preamble list, to determine the actual tabskip glue amounts, and let p point to this prototype
      box 852 ⟩ ≡
  save_ptr ← save_ptr − 2; pack_begin_line ← −mode_line;
  if mode = −vmode then
    begin rule_save ← overfull_rule; overfull_rule ← 0; { prevent rule from being packaged }
    p ← hpack(preamble, saved(1), saved(0)); overfull_rule ← rule_save;
    end
  else begin q ← link(preamble);
    repeat height(q) ← width(q); width(q) ← 0; q ← link(link(q));
    until q = null;
    p ← vpack(preamble, saved(1), saved(0)); q ← link(preamble);
    repeat width(q) ← height(q); height(q) ← 0; q ← link(link(q));
    until q = null;
    end;
  pack_begin_line ← 0
```

This code is used in section 848.

853. ⟨ Set the glue in all the unset boxes of the current list 853 ⟩ ≡
```
q ← link(head); s ← head;
while q ≠ null do
  begin if ¬is_char_node(q) then
    if type(q) = unset_node then ⟨ Set the unset box q and the unset boxes in it 855 ⟩
    else if type(q) = rule_node then
      ⟨ Make the running dimensions in rule q extend to the boundaries of the alignment 854 ⟩;
  s ← q; q ← link(q);
  end
```
This code is used in section 848.

854. ⟨ Make the running dimensions in rule q extend to the boundaries of the alignment 854 ⟩ ≡
```
begin if is_running(width(q)) then width(q) ← width(p);
if is_running(height(q)) then height(q) ← height(p);
if is_running(depth(q)) then depth(q) ← depth(p);
if o ≠ 0 then
  begin r ← link(q); link(q) ← null; q ← hpack(q, natural); shift_amount(q) ← o; link(q) ← r;
  link(s) ← q;
  end;
end
```
This code is used in section 853.

855. The unset box q represents a row that contains one or more unset boxes, depending on how soon `\cr` occurred in that row.

⟨ Set the unset box q and the unset boxes in it 855 ⟩ ≡
```
begin if mode = −vmode then
  begin type(q) ← hlist_node; width(q) ← width(p);
  if nest[nest_ptr − 1].mode_field = mmode then set_box_lr(q)(dlist);   { for ship_out }
  end
else begin type(q) ← vlist_node; height(q) ← height(p);
  end;
glue_order(q) ← glue_order(p); glue_sign(q) ← glue_sign(p); glue_set(q) ← glue_set(p);
shift_amount(q) ← o; r ← link(list_ptr(q)); s ← link(list_ptr(p));
repeat ⟨ Set the glue in node r and change it from an unset node 856 ⟩;
  r ← link(link(r)); s ← link(link(s));
until r = null;
end
```
This code is used in section 853.

856. A box made from spanned columns will be followed by tabskip glue nodes and by empty boxes as if there were no spanning. This permits perfect alignment of subsequent entries, and it prevents values that depend on floating point arithmetic from entering into the dimensions of any boxes.

```
⟨ Set the glue in node r and change it from an unset node 856 ⟩ ≡
  n ← span_count(r); t ← width(s); w ← t; u ← hold_head; set_box_lr(r)(0);  { for ship_out }
  while n > min_quarterword do
    begin decr(n); ⟨ Append tabskip glue and an empty box to list u, and update s and t as the prototype
        nodes are passed 857 ⟩;
    end;
  if mode = −vmode then
    ⟨ Make the unset node r into an hlist_node of width w, setting the glue as if the width were t 858 ⟩
  else ⟨ Make the unset node r into a vlist_node of height w, setting the glue as if the height were t 859 ⟩;
  shift_amount(r) ← 0;
  if u ≠ hold_head then  { append blank boxes to account for spanned nodes }
    begin link(u) ← link(r); link(r) ← link(hold_head); r ← u;
    end
```

This code is used in section 855.

857. ⟨ Append tabskip glue and an empty box to list u, and update s and t as the prototype nodes are passed 857 ⟩ ≡

```
  s ← link(s); v ← glue_ptr(s); link(u) ← new_glue(v); u ← link(u); subtype(u) ← tab_skip_code + 1;
  t ← t + width(v);
  if glue_sign(p) = stretching then
    begin if stretch_order(v) = glue_order(p) then t ← t + round(float(glue_set(p)) ∗ stretch(v));
    end
  else if glue_sign(p) = shrinking then
      begin if shrink_order(v) = glue_order(p) then t ← t − round(float(glue_set(p)) ∗ shrink(v));
      end;
  s ← link(s); link(u) ← new_null_box; u ← link(u); t ← t + width(s);
  if mode = −vmode then width(u) ← width(s) else begin type(u) ← vlist_node; height(u) ← width(s);
    end
```

This code is used in section 856.

858. ⟨ Make the unset node r into an *hlist_node* of width w, setting the glue as if the width were t 858 ⟩ ≡

```
  begin height(r) ← height(q); depth(r) ← depth(q);
  if t = width(r) then
    begin glue_sign(r) ← normal; glue_order(r) ← normal; set_glue_ratio_zero(glue_set(r));
    end
  else if t > width(r) then
      begin glue_sign(r) ← stretching;
      if glue_stretch(r) = 0 then set_glue_ratio_zero(glue_set(r))
      else glue_set(r) ← unfloat((t − width(r))/glue_stretch(r));
      end
    else begin glue_order(r) ← glue_sign(r); glue_sign(r) ← shrinking;
      if glue_shrink(r) = 0 then set_glue_ratio_zero(glue_set(r))
      else if (glue_order(r) = normal) ∧ (width(r) − t > glue_shrink(r)) then
          set_glue_ratio_one(glue_set(r))
        else glue_set(r) ← unfloat((width(r) − t)/glue_shrink(r));
      end;
  width(r) ← w; type(r) ← hlist_node;
  end
```

This code is used in section 856.

859. ⟨ Make the unset node r into a *vlist_node* of height w, setting the glue as if the height were t 859 ⟩ ≡

```
begin width(r) ← width(q);
if t = height(r) then
  begin glue_sign(r) ← normal; glue_order(r) ← normal; set_glue_ratio_zero(glue_set(r));
  end
else if t > height(r) then
    begin glue_sign(r) ← stretching;
    if glue_stretch(r) = 0 then set_glue_ratio_zero(glue_set(r))
    else glue_set(r) ← unfloat((t − height(r))/glue_stretch(r));
    end
  else begin glue_order(r) ← glue_sign(r); glue_sign(r) ← shrinking;
    if glue_shrink(r) = 0 then set_glue_ratio_zero(glue_set(r))
    else if (glue_order(r) = normal) ∧ (height(r) − t > glue_shrink(r)) then
        set_glue_ratio_one(glue_set(r))
      else glue_set(r) ← unfloat((height(r) − t)/glue_shrink(r));
    end;
height(r) ← w; type(r) ← vlist_node;
end
```

This code is used in section 856.

860. We now have a completed alignment, in the list that starts at *head* and ends at *tail*. This list will be merged with the one that encloses it. (In case the enclosing mode is *mmode*, for displayed formulas, we will need to insert glue before and after the display; that part of the program will be deferred until we're more familiar with such operations.)

In restricted horizontal mode, the *clang* part of *aux* is undefined; an over-cautious Pascal runtime system may complain about this.

⟨ Insert the current list into its environment 860 ⟩ ≡

```
aux_save ← aux; p ← link(head); q ← tail; pop_nest;
if mode = mmode then ⟨ Finish an alignment in a display 1260 ⟩
else begin aux ← aux_save; link(tail) ← p;
  if p ≠ null then tail ← q;
  if mode = vmode then build_page;
  end
```

This code is used in section 848.

861. Breaking paragraphs into lines. We come now to what is probably the most interesting algorithm of TEX: the mechanism for choosing the "best possible" breakpoints that yield the individual lines of a paragraph. TEX's line-breaking algorithm takes a given horizontal list and converts it to a sequence of boxes that are appended to the current vertical list. In the course of doing this, it creates a special data structure containing three kinds of records that are not used elsewhere in TEX. Such nodes are created while a paragraph is being processed, and they are destroyed afterwards; thus, the other parts of TEX do not need to know anything about how line-breaking is done.

The method used here is based on an approach devised by Michael F. Plass and the author in 1977, subsequently generalized and improved by the same two people in 1980. A detailed discussion appears in *Software—Practice and Experience* **11** (1981), 1119–1184, where it is shown that the line-breaking problem can be regarded as a special case of the problem of computing the shortest path in an acyclic network. The cited paper includes numerous examples and describes the history of line breaking as it has been practiced by printers through the ages. The present implementation adds two new ideas to the algorithm of 1980: Memory space requirements are considerably reduced by using smaller records for inactive nodes than for active ones, and arithmetic overflow is avoided by using "delta distances" instead of keeping track of the total distance from the beginning of the paragraph to the current point.

862. The *line_break* procedure should be invoked only in horizontal mode; it leaves that mode and places its output into the current vlist of the enclosing vertical mode (or internal vertical mode). There is one explicit parameter: d is true for partial paragraphs preceding display math mode; in this case the amount of additional penalty inserted before the final line is *display_widow_penalty* instead of *widow_penalty*.

There are also a number of implicit parameters: The hlist to be broken starts at *link*(*head*), and it is nonempty. The value of *prev_graf* in the enclosing semantic level tells where the paragraph should begin in the sequence of line numbers, in case hanging indentation or `\parshape` is in use; *prev_graf* is zero unless this paragraph is being continued after a displayed formula. Other implicit parameters, such as the *par_shape_ptr* and various penalties to use for hyphenation, etc., appear in *eqtb*.

After *line_break* has acted, it will have updated the current vlist and the value of *prev_graf*. Furthermore, the global variable *just_box* will point to the final box created by *line_break*, so that the width of this line can be ascertained when it is necessary to decide whether to use *above_display_skip* or *above_display_short_skip* before a displayed formula.

```
⟨ Global variables 13 ⟩ +≡
just_box: pointer; { the hlist_node for the last line of the new paragraph }
```

863. Since *line_break* is a rather lengthy procedure—sort of a small world unto itself—we must build it up little by little, somewhat more cautiously than we have done with the simpler procedures of TEX. Here is the general outline.

```
⟨ Declare subprocedures for line_break 874 ⟩
procedure line_break(d : boolean);
  label done, done1, done2, done3, done4, done5, done6, continue, restart;
  var ⟨ Local variables for line breaking 910 ⟩
  begin pack_begin_line ← mode_line; { this is for over/underfull box messages }
  ⟨ Get ready to start line breaking 864 ⟩;
  ⟨ Find optimal breakpoints 911 ⟩;
  ⟨ Break the paragraph at the chosen breakpoints, justify the resulting lines to the correct widths, and
      append them to the current vertical list 924 ⟩;
  ⟨ Clean up the memory by removing the break nodes 913 ⟩;
  pack_begin_line ← 0;
  end;
⟨ Declare ε-TEX procedures for use by main_control 1466 ⟩
```

864. The first task is to move the list from *head* to *temp_head* and go into the enclosing semantic level. We also append the `\parfillskip` glue to the end of the paragraph, removing a space (or other glue node) if it was there, since spaces usually precede blank lines and instances of '`$$`'. The *par_fill_skip* is preceded by an infinite penalty, so it will never be considered as a potential breakpoint.

This code assumes that a *glue_node* and a *penalty_node* occupy the same number of *mem* words.

```
⟨ Get ready to start line breaking 864 ⟩ ≡
  link(temp_head) ← link(head);
  if is_char_node(tail) then tail_append(new_penalty(inf_penalty))
  else if type(tail) ≠ glue_node then tail_append(new_penalty(inf_penalty))
    else begin type(tail) ← penalty_node; delete_glue_ref(glue_ptr(tail)); flush_node_list(leader_ptr(tail));
      penalty(tail) ← inf_penalty;
      end;
  link(tail) ← new_param_glue(par_fill_skip_code); last_line_fill ← link(tail);
  init_cur_lang ← prev_graf mod ´200000; init_l_hyf ← prev_graf div ´20000000;
  init_r_hyf ← (prev_graf div ´200000) mod ´100; pop_nest;
```

See also sections 875, 882, and 896.

This code is used in section 863.

865. When looking for optimal line breaks, TEX creates a "break node" for each break that is *feasible*, in the sense that there is a way to end a line at the given place without requiring any line to stretch more than a given tolerance. A break node is characterized by three things: the position of the break (which is a pointer to a *glue_node*, *math_node*, *penalty_node*, or *disc_node*); the ordinal number of the line that will follow this breakpoint; and the fitness classification of the line that has just ended, i.e., *tight_fit*, *decent_fit*, *loose_fit*, or *very_loose_fit*.

```
define tight_fit = 3   { fitness classification for lines shrinking 0.5 to 1.0 of their shrinkability }
define loose_fit = 1   { fitness classification for lines stretching 0.5 to 1.0 of their stretchability }
define very_loose_fit = 0   { fitness classification for lines stretching more than their stretchability }
define decent_fit = 2   { fitness classification for all other lines }
```

866. The algorithm essentially determines the best possible way to achieve each feasible combination of position, line, and fitness. Thus, it answers questions like, "What is the best way to break the opening part of the paragraph so that the fourth line is a tight line ending at such-and-such a place?" However, the fact that all lines are to be the same length after a certain point makes it possible to regard all sufficiently large line numbers as equivalent, when the looseness parameter is zero, and this makes it possible for the algorithm to save space and time.

An "active node" and a "passive node" are created in *mem* for each feasible breakpoint that needs to be considered. Active nodes are three words long and passive nodes are two words long. We need active nodes only for breakpoints near the place in the paragraph that is currently being examined, so they are recycled within a comparatively short time after they are created.

867. An active node for a given breakpoint contains six fields:

link points to the next node in the list of active nodes; the last active node has *link* = *last_active*.

break_node points to the passive node associated with this breakpoint.

line_number is the number of the line that follows this breakpoint.

fitness is the fitness classification of the line ending at this breakpoint.

type is either *hyphenated* or *unhyphenated*, depending on whether this breakpoint is a *disc_node*.

total_demerits is the minimum possible sum of demerits over all lines leading from the beginning of the paragraph to this breakpoint.

The value of *link*(*active*) points to the first active node on a linked list of all currently active nodes. This list is in order by *line_number*, except that nodes with *line_number* > *easy_line* may be in any order relative to each other.

```
define active_node_size_normal = 3  { number of words in normal active nodes }
define fitness ≡ subtype  { very_loose_fit .. tight_fit on final line for this break }
define break_node ≡ rlink  { pointer to the corresponding passive node }
define line_number ≡ llink  { line that begins at this breakpoint }
define total_demerits(#) ≡ mem[# + 2].int  { the quantity that TeX minimizes }
define unhyphenated = 0  { the type of a normal active break node }
define hyphenated = 1  { the type of an active node that breaks at a disc_node }
define last_active ≡ active  { the active list ends where it begins }
```

868. ⟨ Initialize the special list heads and constant nodes 838 ⟩ +≡

```
type(last_active) ← hyphenated; line_number(last_active) ← max_halfword; subtype(last_active) ← 0;
    { the subtype is never examined by the algorithm }
```

869. The passive node for a given breakpoint contains only four fields:

link points to the passive node created just before this one, if any, otherwise it is *null*.

cur_break points to the position of this breakpoint in the horizontal list for the paragraph being broken.

prev_break points to the passive node that should precede this one in an optimal path to this breakpoint.

serial is equal to *n* if this passive node is the *n*th one created during the current pass. (This field is used only when printing out detailed statistics about the line-breaking calculations.)

There is a global variable called *passive* that points to the most recently created passive node. Another global variable, *printed_node*, is used to help print out the paragraph when detailed information about the line-breaking computation is being displayed.

```
define passive_node_size = 2  { number of words in passive nodes }
define cur_break ≡ rlink  { in passive node, points to position of this breakpoint }
define prev_break ≡ llink  { points to passive node that should precede this one }
define serial ≡ info  { serial number for symbolic identification }
```

⟨ Global variables 13 ⟩ +≡

```
passive: pointer;  { most recent node on passive list }
printed_node: pointer;  { most recent node that has been printed }
pass_number: halfword;  { the number of passive nodes allocated on this pass }
```

870. The active list also contains "delta" nodes that help the algorithm compute the badness of individual lines. Such nodes appear only between two active nodes, and they have *type* = *delta_node*. If p and r are active nodes and if q is a delta node between them, so that $link(p) = q$ and $link(q) = r$, then q tells the space difference between lines in the horizontal list that start after breakpoint p and lines that start after breakpoint r. In other words, if we know the length of the line that starts after p and ends at our current position, then the corresponding length of the line that starts after r is obtained by adding the amounts in node q. A delta node contains six scaled numbers, since it must record the net change in glue stretchability with respect to all orders of infinity. The natural width difference appears in $mem[q + 1].sc$; the stretch differences in units of pt, fil, fill, and filll appear in $mem[q + 2 \,..\, q + 5].sc$; and the shrink difference appears in $mem[q + 6].sc$. The *subtype* field of a delta node is not used.

define *delta_node_size* = 7 { number of words in a delta node }
define *delta_node* = 2 { *type* field in a delta node }

871. As the algorithm runs, it maintains a set of six delta-like registers for the length of the line following the first active breakpoint to the current position in the given hlist. When it makes a pass through the active list, it also maintains a similar set of six registers for the length following the active breakpoint of current interest. A third set holds the length of an empty line (namely, the sum of `\leftskip` and `\rightskip`); and a fourth set is used to create new delta nodes.

When we pass a delta node we want to do operations like

for $k \leftarrow 1$ **to** 6 **do** $cur_active_width[k] \leftarrow cur_active_width[k] + mem[q + k].sc$;

and we want to do this without the overhead of **for** loops. The *do_all_six* macro makes such six-tuples convenient.

define *do_all_six*(#) ≡ #(1); #(2); #(3); #(4); #(5); #(6)

⟨ Global variables 13 ⟩ +≡
active_width: **array** [1 .. 6] **of** *scaled*; { distance from first active node to *cur_p* }
cur_active_width: **array** [1 .. 6] **of** *scaled*; { distance from current active node }
background: **array** [1 .. 6] **of** *scaled*; { length of an "empty" line }
break_width: **array** [1 .. 6] **of** *scaled*; { length being computed after current break }

872. Let's state the principles of the delta nodes more precisely and concisely, so that the following programs will be less obscure. For each legal breakpoint p in the paragraph, we define two quantities $\alpha(p)$ and $\beta(p)$ such that the length of material in a line from breakpoint p to breakpoint q is $\gamma + \beta(q) - \alpha(p)$, for some fixed γ. Intuitively, $\alpha(p)$ and $\beta(q)$ are the total length of material from the beginning of the paragraph to a point "after" a break at p and to a point "before" a break at q; and γ is the width of an empty line, namely the length contributed by `\leftskip` and `\rightskip`.

Suppose, for example, that the paragraph consists entirely of alternating boxes and glue skips; let the boxes have widths $x_1 \ldots x_n$ and let the skips have widths $y_1 \ldots y_n$, so that the paragraph can be represented by $x_1 y_1 \ldots x_n y_n$. Let p_i be the legal breakpoint at y_i; then $\alpha(p_i) = x_1 + y_1 + \cdots + x_i + y_i$, and $\beta(p_i) = x_1 + y_1 + \cdots + x_i$. To check this, note that the length of material from p_2 to p_5, say, is $\gamma + x_3 + y_3 + x_4 + y_4 + x_5 = \gamma + \beta(p_5) - \alpha(p_2)$.

The quantities α, β, γ involve glue stretchability and shrinkability as well as a natural width. If we were to compute $\alpha(p)$ and $\beta(p)$ for each p, we would need multiple precision arithmetic, and the multiprecise numbers would have to be kept in the active nodes. TeX avoids this problem by working entirely with relative differences or "deltas." Suppose, for example, that the active list contains $a_1\,\delta_1\,a_2\,\delta_2\,a_3$, where the a's are active breakpoints and the δ's are delta nodes. Then $\delta_1 = \alpha(a_1) - \alpha(a_2)$ and $\delta_2 = \alpha(a_2) - \alpha(a_3)$. If the line breaking algorithm is currently positioned at some other breakpoint p, the *active_width* array contains the value $\gamma + \beta(p) - \alpha(a_1)$. If we are scanning through the list of active nodes and considering a tentative line that runs from a_2 to p, say, the *cur_active_width* array will contain the value $\gamma + \beta(p) - \alpha(a_2)$. Thus, when we move from a_2 to a_3, we want to add $\alpha(a_2) - \alpha(a_3)$ to *cur_active_width*; and this is just δ_2, which appears in the active list between a_2 and a_3. The *background* array contains γ. The *break_width* array will be used to calculate values of new delta nodes when the active list is being updated.

873. Glue nodes in a horizontal list that is being paragraphed are not supposed to include "infinite" shrinkability; that is why the algorithm maintains four registers for stretching but only one for shrinking. If the user tries to introduce infinite shrinkability, the shrinkability will be reset to finite and an error message will be issued. A boolean variable *no_shrink_error_yet* prevents this error message from appearing more than once per paragraph.

```
define check_shrinkage(#) ≡
    if (shrink_order(#) ≠ normal) ∧ (shrink(#) ≠ 0) then
      begin # ← finite_shrink(#);
      end
```

⟨ Global variables 13 ⟩ +≡
no_shrink_error_yet: *boolean*; { have we complained about infinite shrinkage? }

874. ⟨ Declare subprocedures for *line_break* 874 ⟩ ≡

```
function finite_shrink(p : pointer): pointer;  { recovers from infinite shrinkage }
  var q: pointer;  { new glue specification }
  begin if no_shrink_error_yet then
    begin no_shrink_error_yet ← false;
    stat if tracing_paragraphs > 0 then end_diagnostic(true);
    tats print_err("Infinite glue shrinkage found in a paragraph");
    help5("The paragraph just ended includes some glue that has")
    ("infinite shrinkability, e.g., `\hskip 0pt minus 1fil´.")
    ("Such glue doesn´t belong there---it allows a paragraph")
    ("of any length to fit on one line. But it´s safe to proceed,")
    ("since the offensive shrinkability has been made finite."); error;
    stat if tracing_paragraphs > 0 then begin_diagnostic;
    tats
    end;
  q ← new_spec(p); shrink_order(q) ← normal; delete_glue_ref(p); finite_shrink ← q;
  end;
```

See also sections 877, 925, 944, and 996.

This code is used in section 863.

875. ⟨ Get ready to start line breaking 864 ⟩ +≡

```
no_shrink_error_yet ← true;
check_shrinkage(left_skip); check_shrinkage(right_skip);
q ← left_skip; r ← right_skip; background[1] ← width(q) + width(r);
background[2] ← 0; background[3] ← 0; background[4] ← 0; background[5] ← 0;
background[2 + stretch_order(q)] ← stretch(q);
background[2 + stretch_order(r)] ← background[2 + stretch_order(r)] + stretch(r);
background[6] ← shrink(q) + shrink(r); ⟨ Check for special treatment of last line of paragraph 1654 ⟩;
```

876. A pointer variable *cur_p* runs through the given horizontal list as we look for breakpoints. This variable is global, since it is used both by *line_break* and by its subprocedure *try_break*.

Another global variable called *threshold* is used to determine the feasibility of individual lines: Breakpoints are feasible if there is a way to reach them without creating lines whose badness exceeds *threshold*. (The badness is compared to *threshold* before penalties are added, so that penalty values do not affect the feasibility of breakpoints, except that no break is allowed when the penalty is 10000 or more.) If *threshold* is 10000 or more, all legal breaks are considered feasible, since the *badness* function specified above never returns a value greater than 10000.

Up to three passes might be made through the paragraph in an attempt to find at least one set of feasible breakpoints. On the first pass, we have *threshold* = *pretolerance* and *second_pass* = *final_pass* = *false*. If this pass fails to find a feasible solution, *threshold* is set to *tolerance*, *second_pass* is set *true*, and an attempt is made to hyphenate as many words as possible. If that fails too, we add *emergency_stretch* to the background stretchability and set *final_pass* = *true*.

⟨ Global variables 13 ⟩ +≡

```
cur_p: pointer;  { the current breakpoint under consideration }
second_pass: boolean;  { is this our second attempt to break this paragraph? }
final_pass: boolean;  { is this our final attempt to break this paragraph? }
threshold: integer;  { maximum badness on feasible lines }
```

877. The heart of the line-breaking procedure is '*try_break*', a subroutine that tests if the current breakpoint *cur_p* is feasible, by running through the active list to see what lines of text can be made from active nodes to *cur_p*. If feasible breaks are possible, new break nodes are created. If *cur_p* is too far from an active node, that node is deactivated.

The parameter *pi* to *try_break* is the penalty associated with a break at *cur_p*; we have *pi* = *eject_penalty* if the break is forced, and *pi* = *inf_penalty* if the break is illegal.

The other parameter, *break_type*, is set to *hyphenated* or *unhyphenated*, depending on whether or not the current break is at a *disc_node*. The end of a paragraph is also regarded as '*hyphenated*'; this case is distinguishable by the condition *cur_p* = *null*.

```
define copy_to_cur_active(#) ≡ cur_active_width[#] ← active_width[#]
define deactivate = 60  { go here when node r should be deactivated }
define cp_skipable(#) ≡  { skipable nodes at the margins during character protrusion }
    (¬is_char_node(#) ∧ ((type(#) = ins_node) ∨ (type(#) = mark_node) ∨ (type(#) =
        adjust_node) ∨ (type(#) = penalty_node) ∨ ((type(#) = disc_node) ∧ (pre_break(#) =
        null) ∧ (post_break(#) = null) ∧ (replace_count(#) = 0))  { an empty disc_node }
    ∨((type(#) = math_node) ∧ (width(#) = 0)) ∨ ((type(#) = kern_node) ∧ ((width(#) =
        0) ∨ (subtype(#) = normal))) ∨ ((type(#) = glue_node) ∧ (glue_ptr(#) = zero_glue)) ∨ ((type(#) =
        hlist_node) ∧ (width(#) = 0) ∧ (height(#) = 0) ∧ (depth(#) = 0) ∧ (list_ptr(#) = null))))
```

⟨ Declare subprocedures for *line_break* 874 ⟩ +≡

```
procedure push_node(p : pointer);
  begin if hlist_stack_level > max_hlist_stack then pdf_error("push_node", "stack␣overflow");
  hlist_stack[hlist_stack_level] ← p; hlist_stack_level ← hlist_stack_level + 1;
  end;
function pop_node: pointer;
  begin hlist_stack_level ← hlist_stack_level − 1;
  if hlist_stack_level < 0 then  { would point to some bug }
    pdf_error("pop_node", "stack␣underflow␣(internal␣error)");
  pop_node ← hlist_stack[hlist_stack_level];
  end;
function find_protchar_left(l : pointer; d : boolean): pointer;
        { searches left to right from list head l, returns 1st non-skipable item }
  var t: pointer; run: boolean;
  begin if (link(l) ≠ null) ∧ (type(l) = hlist_node) ∧ (width(l) = 0) ∧ (height(l) = 0) ∧ (depth(l) =
      0) ∧ (list_ptr(l) = null) then l ← link(l)  { for paragraph start with \parindent = 0pt }
  else if d then
      while (link(l) ≠ null) ∧ (¬(is_char_node(l) ∨ non_discardable(l))) do l ← link(l);
          { std. discardables at line break, TeXbook, p 95 }
  hlist_stack_level ← 0; run ← true;
  repeat t ← l;
    while run ∧ (type(l) = hlist_node) ∧ (list_ptr(l) ≠ null) do
      begin push_node(l); l ← list_ptr(l);
      end;
    while run ∧ cp_skipable(l) do
      begin while (link(l) = null) ∧ (hlist_stack_level > 0) do
        begin l ← pop_node;  { don't visit this node again }
        end;
      if link(l) ≠ null then l ← link(l)
      else if hlist_stack_level = 0 then run ← false
      end;
  until t = l;
  find_protchar_left ← l;
  end;
```

```
function find_protchar_right(l, r : pointer): pointer;
        { searches right to left from list tail r to head l, returns 1st non-skipable item }
  var t: pointer; run: boolean;
  begin find_protchar_right ← null;
  if r = null then return;
  hlist_stack_level ← 0; run ← true;
  repeat t ← r;
    while run ∧ (type(r) = hlist_node) ∧ (list_ptr(r) ≠ null) do
      begin push_node(l); push_node(r); l ← list_ptr(r); r ← l;
      while link(r) ≠ null do r ← link(r);
      end;
    while run ∧ cp_skipable(r) do
      begin while (r = l) ∧ (hlist_stack_level > 0) do
        begin r ← pop_node; { don't visit this node again }
        l ← pop_node;
        end;
      if (r ≠ l) ∧ (r ≠ null) then r ← prev_rightmost(l, r)
      else if (r = l) ∧ (hlist_stack_level = 0) then run ← false
      end;
  until t = r;
  find_protchar_right ← r;
  end;
function total_pw(q, p : pointer): scaled;
        { returns the total width of character protrusion of a line; cur_break(break_node(q)) and p is the
        leftmost resp. rightmost node in the horizontal list representing the actual line }
  var l, r: pointer; n: integer;
  begin if break_node(q) = null then l ← first_p
  else l ← cur_break(break_node(q));
  r ← prev_rightmost(global_prev_p, p); { get link(r) = p }
    { let's look at the right margin first }
  if (p ≠ null) ∧ (type(p) = disc_node) ∧ (pre_break(p) ≠ null) then
        { a disc_node with non-empty pre_break, protrude the last char of pre_break }
    begin r ← pre_break(p);
    while link(r) ≠ null do r ← link(r);
    end
  else r ← find_protchar_right(l, r); { now the left margin }
  if (l ≠ null) ∧ (type(l) = disc_node) then
    begin if post_break(l) ≠ null then
      begin l ← post_break(l); { protrude the first char }
      goto done;
      end
    else { discard replace_count(l) nodes }
    begin n ← replace_count(l); l ← link(l);
    while n > 0 do
      begin if link(l) ≠ null then l ← link(l);
      decr(n);
      end;
    end;
    end;
  l ← find_protchar_left(l, true);
done: total_pw ← left_pw(l) + right_pw(r);
  end;
```

```
procedure try_break(pi : integer; break_type : small_number);
  label exit, done, done1, continue, deactivate, found, not_found;
  var r: pointer;  { runs through the active list }
    prev_r: pointer;  { stays a step behind r }
    old_l: halfword;  { maximum line number in current equivalence class of lines }
    no_break_yet: boolean;  { have we found a feasible break at cur_p? }
    ⟨ Other local variables for try_break 878 ⟩
  begin ⟨ Make sure that pi is in the proper range 879 ⟩;
  no_break_yet ← true; prev_r ← active; old_l ← 0; do_all_six(copy_to_cur_active);
  loop begin continue: r ← link(prev_r); ⟨ If node r is of type delta_node, update cur_active_width, set
      prev_r and prev_prev_r, then goto continue 880 ⟩;
    ⟨ If a line number class has ended, create new active nodes for the best feasible breaks in that class;
      then return if r = last_active, otherwise compute the new line_width 883 ⟩;
    ⟨ Consider the demerits for a line from r to cur_p; deactivate node r if it should no longer be active;
      then goto continue if a line from r to cur_p is infeasible, otherwise record a new feasible
      break 899 ⟩;
    end;
exit: stat ⟨ Update the value of printed_node for symbolic displays 906 ⟩ tats
  end;
```

878. ⟨ Other local variables for *try_break* 878 ⟩ ≡

```
prev_prev_r: pointer;  { a step behind prev_r, if type(prev_r) = delta_node }
s: pointer;  { runs through nodes ahead of cur_p }
q: pointer;  { points to a new node being created }
v: pointer;  { points to a glue specification or a node ahead of cur_p }
t: integer;  { node count, if cur_p is a discretionary node }
f: internal_font_number;  { used in character width calculation }
l: halfword;  { line number of current active node }
node_r_stays_active: boolean;  { should node r remain in the active list? }
line_width: scaled;  { the current line will be justified to this width }
fit_class: very_loose_fit .. tight_fit;  { possible fitness class of test line }
b: halfword;  { badness of test line }
d: integer;  { demerits of test line }
artificial_demerits: boolean;  { has d been forced to zero? }
save_link: pointer;  { temporarily holds value of link(cur_p) }
shortfall: scaled;  { used in badness calculations }
```

See also section 1655.

This code is used in section 877.

879. ⟨ Make sure that *pi* is in the proper range 879 ⟩ ≡

```
if abs(pi) ≥ inf_penalty then
  if pi > 0 then return  { this breakpoint is inhibited by infinite penalty }
  else pi ← eject_penalty  { this breakpoint will be forced }
```

This code is used in section 877.

880. The following code uses the fact that *type*(*last_active*) ≠ *delta_node*.

define *update_width*(#) ≡ *cur_active_width*[#] ← *cur_active_width*[#] + *mem*[*r* + #].*sc*

⟨ If node *r* is of type *delta_node*, update *cur_active_width*, set *prev_r* and *prev_prev_r*, then **goto** *continue* 880 ⟩ ≡

```
if type(r) = delta_node then
  begin do_all_six(update_width); prev_prev_r ← prev_r; prev_r ← r; goto continue;
  end
```

This code is used in section 877.

881. As we consider various ways to end a line at *cur_p*, in a given line number class, we keep track of the best total demerits known, in an array with one entry for each of the fitness classifications. For example, *minimal_demerits*[*tight_fit*] contains the fewest total demerits of feasible line breaks ending at *cur_p* with a *tight_fit* line; *best_place*[*tight_fit*] points to the passive node for the break before *cur_p* that achieves such an optimum; and *best_pl_line*[*tight_fit*] is the *line_number* field in the active node corresponding to *best_place*[*tight_fit*]. When no feasible break sequence is known, the *minimal_demerits* entries will be equal to *awful_bad*, which is $2^{30} - 1$. Another variable, *minimum_demerits*, keeps track of the smallest value in the *minimal_demerits* array.

define *awful_bad* ≡ ´7777777777 { more than a billion demerits }

⟨ Global variables 13 ⟩ +≡

```
minimal_demerits: array [very_loose_fit .. tight_fit] of integer;
    { best total demerits known for current line class and position, given the fitness }
minimum_demerits: integer; { best total demerits known for current line class and position }
best_place: array [very_loose_fit .. tight_fit] of pointer; { how to achieve minimal_demerits }
best_pl_line: array [very_loose_fit .. tight_fit] of halfword; { corresponding line number }
```

882. ⟨ Get ready to start line breaking 864 ⟩ +≡

```
minimum_demerits ← awful_bad; minimal_demerits[tight_fit] ← awful_bad;
minimal_demerits[decent_fit] ← awful_bad; minimal_demerits[loose_fit] ← awful_bad;
minimal_demerits[very_loose_fit] ← awful_bad;
```

883. The first part of the following code is part of TeX's inner loop, so we don't want to waste any time. The current active node, namely node *r*, contains the line number that will be considered next. At the end of the list we have arranged the data structure so that *r* = *last_active* and *line_number*(*last_active*) > *old_l*.

⟨ If a line number class has ended, create new active nodes for the best feasible breaks in that class; then **return** if *r* = *last_active*, otherwise compute the new *line_width* 883 ⟩ ≡

```
begin l ← line_number(r);
if l > old_l then
  begin { now we are no longer in the inner loop }
  if (minimum_demerits < awful_bad) ∧ ((old_l ≠ easy_line) ∨ (r = last_active)) then
    ⟨ Create new active nodes for the best feasible breaks just found 884 ⟩;
  if r = last_active then return;
  ⟨ Compute the new line width 898 ⟩;
  end;
end
```

This code is used in section 877.

884. It is not necessary to create new active nodes having *minimal_demerits* greater than *minimum_demerits* + *abs*(*adj_demerits*), since such active nodes will never be chosen in the final paragraph breaks. This observation allows us to omit a substantial number of feasible breakpoints from further consideration.

```
⟨ Create new active nodes for the best feasible breaks just found 884 ⟩ ≡
  begin if no_break_yet then ⟨ Compute the values of break_width 885 ⟩;
  ⟨ Insert a delta node to prepare for breaks at cur_p 891 ⟩;
  if abs(adj_demerits) ≥ awful_bad − minimum_demerits then minimum_demerits ← awful_bad − 1
  else minimum_demerits ← minimum_demerits + abs(adj_demerits);
  for fit_class ← very_loose_fit to tight_fit do
    begin if minimal_demerits[fit_class] ≤ minimum_demerits then
      ⟨ Insert a new active node from best_place[fit_class] to cur_p 893 ⟩;
    minimal_demerits[fit_class] ← awful_bad;
    end;
  minimum_demerits ← awful_bad; ⟨ Insert a delta node to prepare for the next active node 892 ⟩;
  end
```

This code is used in section 883.

885. When we insert a new active node for a break at *cur_p*, suppose this new node is to be placed just before active node *a*; then we essentially want to insert 'δ *cur_p* δ'' before *a*, where $\delta = \alpha(a) - \alpha(cur_p)$ and $\delta' = \alpha(cur_p) - \alpha(a)$ in the notation explained above. The *cur_active_width* array now holds $\gamma + \beta(cur_p) - \alpha(a)$; so δ can be obtained by subtracting *cur_active_width* from the quantity $\gamma + \beta(cur_p) - \alpha(cur_p)$. The latter quantity can be regarded as the length of a line "from *cur_p* to *cur_p*"; we call it the *break_width* at *cur_p*.

The *break_width* is usually negative, since it consists of the background (which is normally zero) minus the width of nodes following *cur_p* that are eliminated after a break. If, for example, node *cur_p* is a glue node, the width of this glue is subtracted from the background; and we also look ahead to eliminate all subsequent glue and penalty and kern and math nodes, subtracting their widths as well.

Kern nodes do not disappear at a line break unless they are *explicit* or *space_adjustment*.

```
  define set_break_width_to_background(#) ≡ break_width[#] ← background[#]
⟨ Compute the values of break_width 885 ⟩ ≡
  begin no_break_yet ← false; do_all_six(set_break_width_to_background); s ← cur_p;
  if break_type > unhyphenated then
    if cur_p ≠ null then ⟨ Compute the discretionary break_width values 888 ⟩;
  while s ≠ null do
    begin if is_char_node(s) then goto done;
    case type(s) of
    glue_node: ⟨ Subtract glue from break_width 886 ⟩;
    penalty_node: do_nothing;
    math_node: break_width[1] ← break_width[1] − width(s);
    kern_node: if subtype(s) ≠ explicit then goto done
      else break_width[1] ← break_width[1] − width(s);
    othercases goto done
    endcases;
    s ← link(s);
    end;
done: end
```

This code is used in section 884.

886. ⟨ Subtract glue from *break_width* 886 ⟩ ≡
```
begin v ← glue_ptr(s); break_width[1] ← break_width[1] − width(v);
break_width[2 + stretch_order(v)] ← break_width[2 + stretch_order(v)] − stretch(v);
break_width[6] ← break_width[6] − shrink(v);
end
```
This code is used in section 885.

887. When *cur_p* is a discretionary break, the length of a line "from *cur_p* to *cur_p*" has to be defined properly so that the other calculations work out. Suppose that the pre-break text at *cur_p* has length l_0, the post-break text has length l_1, and the replacement text has length l. Suppose also that q is the node following the replacement text. Then length of a line from *cur_p* to q will be computed as $\gamma+\beta(q)-\alpha(cur_p)$, where $\beta(q)=\beta(cur_p)-l_0+l$. The actual length will be the background plus l_1, so the length from *cur_p* to *cur_p* should be $\gamma+l_0+l_1-l$. If the post-break text of the discretionary is empty, a break may also discard q; in that unusual case we subtract the length of q and any other nodes that will be discarded after the discretionary break.

The value of l_0 need not be computed, since *line_break* will put it into the global variable *disc_width* before calling *try_break*.

⟨ Global variables 13 ⟩ +≡
```
disc_width: scaled; { the length of discretionary material preceding a break }
```

888. ⟨ Compute the discretionary *break_width* values 888 ⟩ ≡
```
begin t ← replace_count(cur_p); v ← cur_p; s ← post_break(cur_p);
while t > 0 do
  begin decr(t); v ← link(v); ⟨ Subtract the width of node v from break_width 889 ⟩;
  end;
while s ≠ null do
  begin ⟨ Add the width of node s to break_width 890 ⟩;
  s ← link(s);
  end;
break_width[1] ← break_width[1] + disc_width;
if post_break(cur_p) = null then s ← link(v); { nodes may be discardable after the break }
end
```
This code is used in section 885.

889. Replacement texts and discretionary texts are supposed to contain only character nodes, kern nodes, ligature nodes, and box or rule nodes.

⟨ Subtract the width of node *v* from *break_width* 889 ⟩ ≡
```
if is_char_node(v) then
  begin f ← font(v); break_width[1] ← break_width[1] − char_width(f)(char_info(f)(character(v)));
  end
else case type(v) of
  ligature_node: begin f ← font(lig_char(v));
    xtx_ligature_present ← true;
    break_width[1] ← break_width[1] − char_width(f)(char_info(f)(character(lig_char(v))));
    end;
  hlist_node, vlist_node, rule_node, kern_node: break_width[1] ← break_width[1] − width(v);
  whatsit_node: if (is_native_word_subtype(v)) ∨ (subtype(v) = glyph_node) ∨ (subtype(v) =
        pic_node) ∨ (subtype(v) = pdf_node) then break_width[1] ← break_width[1] − width(v)
    else confusion("disc1a");
  othercases confusion("disc1")
  endcases
```
This code is used in section 888.

890. ⟨ Add the width of node s to *break_width* 890 ⟩ ≡

```
if is_char_node(s) then
  begin f ← font(s); break_width[1] ← break_width[1] + char_width(f)(char_info(f)(character(s)));
  end
else case type(s) of
  ligature_node: begin f ← font(lig_char(s)); xtx_ligature_present ← true;
    break_width[1] ← break_width[1] + char_width(f)(char_info(f)(character(lig_char(s))));
    end;
  hlist_node, vlist_node, rule_node, kern_node: break_width[1] ← break_width[1] + width(s);
  whatsit_node: if (is_native_word_subtype(s)) ∨ (subtype(s) = glyph_node) ∨ (subtype(s) =
        pic_node) ∨ (subtype(s) = pdf_node) then break_width[1] ← break_width[1] + width(s)
    else confusion("disc2a");
  othercases confusion("disc2")
  endcases
```

This code is used in section 888.

891. We use the fact that $type(active) \neq delta_node$.

```
define convert_to_break_width(#) ≡ mem[prev_r + #].sc ←
          mem[prev_r + #].sc − cur_active_width[#] + break_width[#]
define store_break_width(#) ≡ active_width[#] ← break_width[#]
define new_delta_to_break_width(#) ≡ mem[q + #].sc ← break_width[#] − cur_active_width[#]
```

⟨ Insert a delta node to prepare for breaks at *cur_p* 891 ⟩ ≡

```
if type(prev_r) = delta_node then  { modify an existing delta node }
  begin do_all_six(convert_to_break_width);
  end
else if prev_r = active then  { no delta node needed at the beginning }
    begin do_all_six(store_break_width);
    end
  else begin q ← get_node(delta_node_size); link(q) ← r; type(q) ← delta_node;
    subtype(q) ← 0;  { the subtype is not used }
    do_all_six(new_delta_to_break_width); link(prev_r) ← q; prev_prev_r ← prev_r; prev_r ← q;
    end
```

This code is used in section 884.

892. When the following code is performed, we will have just inserted at least one active node before r, so $type(prev_r) \neq delta_node$.

```
define new_delta_from_break_width(#) ≡ mem[q + #].sc ← cur_active_width[#] − break_width[#]
```

⟨ Insert a delta node to prepare for the next active node 892 ⟩ ≡

```
if r ≠ last_active then
  begin q ← get_node(delta_node_size); link(q) ← r; type(q) ← delta_node;
  subtype(q) ← 0;  { the subtype is not used }
  do_all_six(new_delta_from_break_width); link(prev_r) ← q; prev_prev_r ← prev_r; prev_r ← q;
  end
```

This code is used in section 884.

893. When we create an active node, we also create the corresponding passive node.

⟨ Insert a new active node from *best_place*[*fit_class*] to *cur_p* 893 ⟩ ≡
begin $q \leftarrow$ *get_node*(*passive_node_size*); *link*(q) $\leftarrow$ *passive*; *passive* $\leftarrow q$; *cur_break*(q) $\leftarrow$ *cur_p*;
stat *incr*(*pass_number*); *serial*(q) $\leftarrow$ *pass_number*; **tats**
prev_break(q) $\leftarrow$ *best_place*[*fit_class*];
$q \leftarrow$ *get_node*(*active_node_size*); *break_node*(q) $\leftarrow$ *passive*; *line_number*(q) $\leftarrow$ *best_pl_line*[*fit_class*] + 1;
fitness(q) $\leftarrow$ *fit_class*; *type*(q) $\leftarrow$ *break_type*; *total_demerits*(q) $\leftarrow$ *minimal_demerits*[*fit_class*];
if *do_last_line_fit* **then** ⟨ Store additional data in the new active node 1662 ⟩;
link(q) $\leftarrow r$; *link*(*prev_r*) $\leftarrow q$; *prev_r* $\leftarrow q$;
stat if *tracing_paragraphs* > 0 **then** ⟨ Print a symbolic description of the new break node 894 ⟩;
tats
end

This code is used in section 884.

894. ⟨ Print a symbolic description of the new break node 894 ⟩ ≡
begin *print_nl*("@@"); *print_int*(*serial*(*passive*)); *print*(": line "); *print_int*(*line_number*(q) − 1);
print_char("."); *print_int*(*fit_class*);
if *break_type* = *hyphenated* **then** *print_char*("-");
print(" t="); *print_int*(*total_demerits*(q));
if *do_last_line_fit* **then** ⟨ Print additional data in the new active node 1663 ⟩;
print(" -> @@");
if *prev_break*(*passive*) = *null* **then** *print_char*("0")
else *print_int*(*serial*(*prev_break*(*passive*)));
end

This code is used in section 893.

895. The length of lines depends on whether the user has specified `\parshape` or `\hangindent`. If *par_shape_ptr* is not null, it points to a $(2n + 1)$-word record in *mem*, where the *info* in the first word contains the value of n, and the other $2n$ words contain the left margins and line lengths for the first n lines of the paragraph; the specifications for line n apply to all subsequent lines. If *par_shape_ptr* = *null*, the shape of the paragraph depends on the value of n = *hang_after*; if $n \geq 0$, hanging indentation takes place on lines $n + 1$, $n + 2$, ..., otherwise it takes place on lines 1, ..., $|n|$. When hanging indentation is active, the left margin is *hang_indent*, if *hang_indent* ≥ 0, else it is 0; the line length is *hsize* − |*hang_indent*|. The normal setting is *par_shape_ptr* = *null*, *hang_after* = 1, and *hang_indent* = 0. Note that if *hang_indent* = 0, the value of *hang_after* is irrelevant.

⟨ Global variables 13 ⟩ +≡
easy_line: *halfword*; { line numbers > *easy_line* are equivalent in break nodes }
last_special_line: *halfword*; { line numbers > *last_special_line* all have the same width }
first_width: *scaled*; { the width of all lines ≤ *last_special_line*, if no `\parshape` has been specified }
second_width: *scaled*; { the width of all lines > *last_special_line* }
first_indent: *scaled*; { left margin to go with *first_width* }
second_indent: *scaled*; { left margin to go with *second_width* }

896. We compute the values of *easy_line* and the other local variables relating to line length when the *line_break* procedure is initializing itself.

```
⟨ Get ready to start line breaking 864 ⟩ +≡
  if par_shape_ptr = null then
    if hang_indent = 0 then
      begin last_special_line ← 0; second_width ← hsize; second_indent ← 0;
      end
    else ⟨ Set line length parameters in preparation for hanging indentation 897 ⟩
  else begin last_special_line ← info(par_shape_ptr) − 1;
    second_width ← mem[par_shape_ptr + 2 ∗ (last_special_line + 1)].sc;
    second_indent ← mem[par_shape_ptr + 2 ∗ last_special_line + 1].sc;
    end;
  if looseness = 0 then easy_line ← last_special_line
  else easy_line ← max_halfword
```

897.

```
⟨ Set line length parameters in preparation for hanging indentation 897 ⟩ ≡
  begin last_special_line ← abs(hang_after);
  if hang_after < 0 then
    begin first_width ← hsize − abs(hang_indent);
    if hang_indent ≥ 0 then first_indent ← hang_indent
    else first_indent ← 0;
    second_width ← hsize; second_indent ← 0;
    end
  else begin first_width ← hsize; first_indent ← 0; second_width ← hsize − abs(hang_indent);
    if hang_indent ≥ 0 then second_indent ← hang_indent
    else second_indent ← 0;
    end;
  end
```

This code is used in section 896.

898. When we come to the following code, we have just encountered the first active node r whose *line_number* field contains l. Thus we want to compute the length of the lth line of the current paragraph. Furthermore, we want to set *old_l* to the last number in the class of line numbers equivalent to l.

```
⟨ Compute the new line width 898 ⟩ ≡
  if l > easy_line then
    begin line_width ← second_width; old_l ← max_halfword − 1;
    end
  else begin old_l ← l;
    if l > last_special_line then line_width ← second_width
    else if par_shape_ptr = null then line_width ← first_width
      else line_width ← mem[par_shape_ptr + 2 ∗ l ].sc;
    end
```

This code is used in section 883.

899. The remaining part of *try_break* deals with the calculation of demerits for a break from r to *cur_p*.

The first thing to do is calculate the badness, b. This value will always be between zero and $inf_bad + 1$; the latter value occurs only in the case of lines from r to *cur_p* that cannot shrink enough to fit the necessary width. In such cases, node r will be deactivated. We also deactivate node r when a break at *cur_p* is forced, since future breaks must go through a forced break.

```
⟨ Consider the demerits for a line from r to cur_p; deactivate node r if it should no longer be active; then
      goto continue if a line from r to cur_p is infeasible, otherwise record a new feasible break 899 ⟩ ≡
  begin artificial_demerits ← false;
  shortfall ← line_width − cur_active_width[1];  { we're this much too short }
  if XeTeX_protrude_chars > 1 then shortfall ← shortfall + total_pw(r, cur_p);
  if shortfall > 0 then
    ⟨ Set the value of b to the badness for stretching the line, and compute the corresponding fit_class 900 ⟩
  else ⟨ Set the value of b to the badness for shrinking the line, and compute the corresponding
       fit_class 901 ⟩;
  if do_last_line_fit then ⟨ Adjust the additional data for last line 1660 ⟩;
found: if (b > inf_bad) ∨ (pi = eject_penalty) then ⟨ Prepare to deactivate node r, and goto deactivate
       unless there is a reason to consider lines of text from r to cur_p 902 ⟩
  else begin prev_r ← r;
    if b > threshold then goto continue;
    node_r_stays_active ← true;
    end;
  ⟨ Record a new feasible break 903 ⟩;
  if node_r_stays_active then goto continue;  { prev_r has been set to r }
deactivate: ⟨ Deactivate node r 908 ⟩;
  end
```

This code is used in section 877.

900. When a line must stretch, the available stretchability can be found in the subarray *cur_active_width*[2 .. 5], in units of points, fil, fill, and filll.

The present section is part of TeX's inner loop, and it is most often performed when the badness is infinite; therefore it is worth while to make a quick test for large width excess and small stretchability, before calling the *badness* subroutine.

```
⟨ Set the value of b to the badness for stretching the line, and compute the corresponding fit_class 900 ⟩ ≡
  if (cur_active_width[3] ≠ 0) ∨ (cur_active_width[4] ≠ 0) ∨ (cur_active_width[5] ≠ 0) then
    begin if do_last_line_fit then
      begin if cur_p = null then { the last line of a paragraph }
        ⟨ Perform computations for last line and goto found 1657 ⟩;
      shortfall ← 0;
      end;
    b ← 0; fit_class ← decent_fit; { infinite stretch }
    end
  else begin if shortfall > 7230584 then
      if cur_active_width[2] < 1663497 then
        begin b ← inf_bad; fit_class ← very_loose_fit; goto done1;
        end;
    b ← badness(shortfall, cur_active_width[2]);
    if b > 12 then
      if b > 99 then fit_class ← very_loose_fit
      else fit_class ← loose_fit
    else fit_class ← decent_fit;
  done1: end
```

This code is used in section 899.

901. Shrinkability is never infinite in a paragraph; we can shrink the line from r to *cur_p* by at most *cur_active_width*[6].

```
⟨ Set the value of b to the badness for shrinking the line, and compute the corresponding fit_class 901 ⟩ ≡
  begin if −shortfall > cur_active_width[6] then b ← inf_bad + 1
  else b ← badness(−shortfall, cur_active_width[6]);
  if b > 12 then fit_class ← tight_fit else fit_class ← decent_fit;
  end
```

This code is used in section 899.

902. During the final pass, we dare not lose all active nodes, lest we lose touch with the line breaks already found. The code shown here makes sure that such a catastrophe does not happen, by permitting overfull boxes as a last resort. This particular part of TeX was a source of several subtle bugs before the correct program logic was finally discovered; readers who seek to "improve" TeX should therefore think thrice before daring to make any changes here.

```
⟨ Prepare to deactivate node r, and goto deactivate unless there is a reason to consider lines of text from r
      to cur_p 902 ⟩ ≡
  begin if final_pass ∧ (minimum_demerits = awful_bad) ∧ (link(r) = last_active) ∧ (prev_r = active) then
    artificial_demerits ← true { set demerits zero, this break is forced }
  else if b > threshold then goto deactivate;
  node_r_stays_active ← false;
  end
```

This code is used in section 899.

903. When we get to this part of the code, the line from r to *cur_p* is feasible, its badness is b, and its fitness classification is *fit_class*. We don't want to make an active node for this break yet, but we will compute the total demerits and record them in the *minimal_demerits* array, if such a break is the current champion among all ways to get to *cur_p* in a given line-number class and fitness class.

```
⟨ Record a new feasible break 903 ⟩ ≡
  if artificial_demerits then d ← 0
  else ⟨ Compute the demerits, d, from r to cur_p 907 ⟩;
  stat if tracing_paragraphs > 0 then ⟨ Print a symbolic description of this feasible break 904 ⟩;
  tats
  d ← d + total_demerits(r);  { this is the minimum total demerits from the beginning to cur_p via r }
  if d ≤ minimal_demerits[fit_class] then
    begin minimal_demerits[fit_class] ← d; best_place[fit_class] ← break_node(r); best_pl_line[fit_class] ← l;
    if do_last_line_fit then ⟨ Store additional data for this feasible break 1661 ⟩;
    if d < minimum_demerits then minimum_demerits ← d;
    end
```

This code is used in section 899.

904.

```
⟨ Print a symbolic description of this feasible break 904 ⟩ ≡
  begin if printed_node ≠ cur_p then
    ⟨ Print the list between printed_node and cur_p, then set printed_node ← cur_p 905 ⟩;
  print_nl("@");
  if cur_p = null then print_esc("par")
  else if type(cur_p) ≠ glue_node then
      begin if type(cur_p) = penalty_node then print_esc("penalty")
      else if type(cur_p) = disc_node then print_esc("discretionary")
        else if type(cur_p) = kern_node then print_esc("kern")
          else print_esc("math");
      end;
  print("␣via␣@@");
  if break_node(r) = null then print_char("0")
  else print_int(serial(break_node(r)));
  print("␣b=");
  if b > inf_bad then print_char("*") else print_int(b);
  print("␣p="); print_int(pi); print("␣d=");
  if artificial_demerits then print_char("*") else print_int(d);
  end
```

This code is used in section 903.

905.

```
⟨ Print the list between printed_node and cur_p, then set printed_node ← cur_p 905 ⟩ ≡
  begin print_nl("");
  if cur_p = null then short_display(link(printed_node))
  else begin save_link ← link(cur_p); link(cur_p) ← null; print_nl("");
    short_display(link(printed_node)); link(cur_p) ← save_link;
    end;
  printed_node ← cur_p;
  end
```

This code is used in section 904.

906. When the data for a discretionary break is being displayed, we will have printed the *pre_break* and *post_break* lists; we want to skip over the third list, so that the discretionary data will not appear twice. The following code is performed at the very end of *try_break*.

```
⟨ Update the value of printed_node for symbolic displays 906 ⟩ ≡
  if cur_p = printed_node then
    if cur_p ≠ null then
      if type(cur_p) = disc_node then
        begin t ← replace_count(cur_p);
        while t > 0 do
          begin decr(t); printed_node ← link(printed_node);
          end;
        end
```

This code is used in section 877.

907.

```
⟨ Compute the demerits, d, from r to cur_p 907 ⟩ ≡
  begin d ← line_penalty + b;
  if abs(d) ≥ 10000 then d ← 100000000 else d ← d * d;
  if pi ≠ 0 then
    if pi > 0 then d ← d + pi * pi
    else if pi > eject_penalty then d ← d − pi * pi;
  if (break_type = hyphenated) ∧ (type(r) = hyphenated) then
    if cur_p ≠ null then d ← d + double_hyphen_demerits
    else d ← d + final_hyphen_demerits;
  if abs(fit_class − fitness(r)) > 1 then d ← d + adj_demerits;
  end
```

This code is used in section 903.

908. When an active node disappears, we must delete an adjacent delta node if the active node was at the beginning or the end of the active list, or if it was surrounded by delta nodes. We also must preserve the property that *cur_active_width* represents the length of material from *link*(*prev_r*) to *cur_p*.

```
  define combine_two_deltas(#) ≡ mem[prev_r + #].sc ← mem[prev_r + #].sc + mem[r + #].sc
  define downdate_width(#) ≡ cur_active_width[#] ← cur_active_width[#] − mem[prev_r + #].sc

⟨ Deactivate node r 908 ⟩ ≡
  link(prev_r) ← link(r); free_node(r, active_node_size);
  if prev_r = active then ⟨ Update the active widths, since the first active node has been deleted 909 ⟩
  else if type(prev_r) = delta_node then
      begin r ← link(prev_r);
      if r = last_active then
        begin do_all_six(downdate_width); link(prev_prev_r) ← last_active;
        free_node(prev_r, delta_node_size); prev_r ← prev_prev_r;
        end
      else if type(r) = delta_node then
          begin do_all_six(update_width); do_all_six(combine_two_deltas); link(prev_r) ← link(r);
          free_node(r, delta_node_size);
          end;
      end
```

This code is used in section 899.

909. The following code uses the fact that $type(last_active) \neq delta_node$. If the active list has just become empty, we do not need to update the *active_width* array, since it will be initialized when an active node is next inserted.

```
define update_active(#) ≡ active_width[#] ← active_width[#] + mem[r + #].sc
⟨ Update the active widths, since the first active node has been deleted 909 ⟩ ≡
  begin r ← link(active);
  if type(r) = delta_node then
    begin do_all_six(update_active); do_all_six(copy_to_cur_active); link(active) ← link(r);
    free_node(r, delta_node_size);
    end;
  end
```

This code is used in section 908.

910. Breaking paragraphs into lines, continued. So far we have gotten a little way into the *line_break* routine, having covered its important *try_break* subroutine. Now let's consider the rest of the process.

The main loop of *line_break* traverses the given hlist, starting at *link*(*temp_head*), and calls *try_break* at each legal breakpoint. A variable called *auto_breaking* is set to true except within math formulas, since glue nodes are not legal breakpoints when they appear in formulas.

The current node of interest in the hlist is pointed to by *cur_p*. Another variable, *prev_p*, is usually one step behind *cur_p*, but the real meaning of *prev_p* is this: If *type*(*cur_p*) = *glue_node* then *cur_p* is a legal breakpoint if and only if *auto_breaking* is true and *prev_p* does not point to a glue node, penalty node, explicit kern node, or math node.

The following declarations provide for a few other local variables that are used in special calculations.

⟨ Local variables for line breaking 910 ⟩ ≡
auto_breaking: *boolean*; { is node *cur_p* outside a formula? }
prev_p: *pointer*; { helps to determine when glue nodes are breakpoints }
q, *r*, *s*, *prev_s*: *pointer*; { miscellaneous nodes of temporary interest }
f: *internal_font_number*; { used when calculating character widths }

See also sections 942 and 948.

This code is used in section 863.

911. The '**loop**' in the following code is performed at most thrice per call of *line_break*, since it is actually a pass over the entire paragraph.

```
define update_prev_p ≡
        begin prev_p ← cur_p; global_prev_p ← cur_p;
        end
⟨ Find optimal breakpoints 911 ⟩ ≡
  threshold ← pretolerance;
  if threshold ≥ 0 then
    begin stat if tracing_paragraphs > 0 then
      begin begin_diagnostic; print_nl("@firstpass"); end; tats
    second_pass ← false; final_pass ← false;
    end
  else begin threshold ← tolerance; second_pass ← true; final_pass ← (emergency_stretch ≤ 0);
    stat if tracing_paragraphs > 0 then begin_diagnostic;
    tats
    end;
  loop begin if threshold > inf_bad then threshold ← inf_bad;
    if second_pass then ⟨ Initialize for hyphenating a paragraph 939 ⟩;
    ⟨ Create an active breakpoint representing the beginning of the paragraph 912 ⟩;
    cur_p ← link(temp_head); auto_breaking ← true;
    update_prev_p; { glue at beginning is not a legal breakpoint }
    first_p ← cur_p; { to access the first node of paragraph as the first active node has break_node = null }
    while (cur_p ≠ null) ∧ (link(active) ≠ last_active) do ⟨ Call try_break if cur_p is a legal breakpoint;
          on the second pass, also try to hyphenate the next word, if cur_p is a glue node; then advance
          cur_p to the next node of the paragraph that could possibly be a legal breakpoint 914 ⟩;
    if cur_p = null then ⟨ Try the final line break at the end of the paragraph, and goto done if the
          desired breakpoints have been found 921 ⟩;
    ⟨ Clean up the memory by removing the break nodes 913 ⟩;
    if ¬second_pass then
      begin stat if tracing_paragraphs > 0 then print_nl("@secondpass"); tats
      threshold ← tolerance; second_pass ← true; final_pass ← (emergency_stretch ≤ 0);
      end { if at first you don't succeed, ... }
    else begin stat if tracing_paragraphs > 0 then print_nl("@emergencypass"); tats
      background[2] ← background[2] + emergency_stretch; final_pass ← true;
      end;
    end;
done: stat if tracing_paragraphs > 0 then
    begin end_diagnostic(true); normalize_selector;
    end;
  tats
  if do_last_line_fit then ⟨ Adjust the final line of the paragraph 1664 ⟩;
```

This code is used in section 863.

912. The active node that represents the starting point does not need a corresponding passive node.

```
define store_background(#) ≡ active_width[#] ← background[#]
```

```
⟨ Create an active breakpoint representing the beginning of the paragraph 912 ⟩ ≡
  q ← get_node(active_node_size); type(q) ← unhyphenated; fitness(q) ← decent_fit; link(q) ← last_active;
  break_node(q) ← null; line_number(q) ← prev_graf + 1; total_demerits(q) ← 0; link(active) ← q;
  if do_last_line_fit then ⟨ Initialize additional fields of the first active node 1656 ⟩;
  do_all_six(store_background);
  passive ← null; printed_node ← temp_head; pass_number ← 0; font_in_short_display ← null_font
```

This code is used in section 911.

913.

```
⟨ Clean up the memory by removing the break nodes 913 ⟩ ≡
  q ← link(active);
  while q ≠ last_active do
    begin cur_p ← link(q);
    if type(q) = delta_node then free_node(q, delta_node_size)
    else free_node(q, active_node_size);
    q ← cur_p;
    end;
  q ← passive;
  while q ≠ null do
    begin cur_p ← link(q); free_node(q, passive_node_size); q ← cur_p;
    end
```

This code is used in sections 863 and 911.

914. Here is the main switch in the *line_break* routine, where legal breaks are determined. As we move through the hlist, we need to keep the *active_width* array up to date, so that the badness of individual lines is readily calculated by *try_break*. It is convenient to use the short name *act_width* for the component of active width that represents real width as opposed to glue.

```
define act_width ≡ active_width[1]  { length from first active node to current node }
define kern_break ≡
        begin if ¬is_char_node(link(cur_p)) ∧ auto_breaking then
          if type(link(cur_p)) = glue_node then try_break(0, unhyphenated);
        act_width ← act_width + width(cur_p);
        end
```

⟨ Call *try_break* if *cur_p* is a legal breakpoint; on the second pass, also try to hyphenate the next word, if *cur_p* is a glue node; then advance *cur_p* to the next node of the paragraph that could possibly be a legal breakpoint 914 ⟩ ≡

```
begin if is_char_node(cur_p) then
  ⟨ Advance cur_p to the node following the present string of characters 915 ⟩;
case type(cur_p) of
hlist_node, vlist_node, rule_node: act_width ← act_width + width(cur_p);
whatsit_node: ⟨ Advance past a whatsit node in the line_break loop 1422 ⟩;
glue_node: begin ⟨ If node cur_p is a legal breakpoint, call try_break; then update the active widths by
      including the glue in glue_ptr(cur_p) 916 ⟩;
  if second_pass ∧ auto_breaking then ⟨ Try to hyphenate the following word 943 ⟩;
  end;
kern_node: if subtype(cur_p) = explicit then kern_break
  else act_width ← act_width + width(cur_p);
ligature_node: begin f ← font(lig_char(cur_p)); xtx_ligature_present ← true;
  act_width ← act_width + char_width(f)(char_info(f)(character(lig_char(cur_p))));
  end;
disc_node: ⟨ Try to break after a discretionary fragment, then goto done5 917 ⟩;
math_node: begin if subtype(cur_p) < L_code then auto_breaking ← odd(subtype(cur_p));
  kern_break;
  end;
penalty_node: try_break(penalty(cur_p), unhyphenated);
mark_node, ins_node, adjust_node: do_nothing;
othercases confusion("paragraph")
endcases;
update_prev_p; cur_p ← link(cur_p);
done5: end
```

This code is used in section 911.

915. The code that passes over the characters of words in a paragraph is part of TeX's inner loop, so it has been streamlined for speed. We use the fact that '`\parfillskip`' glue appears at the end of each paragraph; it is therefore unnecessary to check if *link*(*cur_p*) = *null* when *cur_p* is a character node.

⟨ Advance *cur_p* to the node following the present string of characters 915 ⟩ ≡

```
begin update_prev_p;
repeat f ← font(cur_p); act_width ← act_width + char_width(f)(char_info(f)(character(cur_p)));
  cur_p ← link(cur_p);
until ¬is_char_node(cur_p);
end
```

This code is used in section 914.

916. When node *cur_p* is a glue node, we look at *prev_p* to see whether or not a breakpoint is legal at *cur_p*, as explained above.

```
⟨ If node cur_p is a legal breakpoint, call try_break; then update the active widths by including the glue in
      glue_ptr(cur_p) 916 ⟩ ≡
  if auto_breaking then
    begin if is_char_node(prev_p) then try_break(0, unhyphenated)
    else if precedes_break(prev_p) then try_break(0, unhyphenated)
      else if (type(prev_p) = kern_node) ∧ (subtype(prev_p) ≠ explicit) then try_break(0, unhyphenated);
    end;
  check_shrinkage(glue_ptr(cur_p)); q ← glue_ptr(cur_p); act_width ← act_width + width(q);
  active_width[2 + stretch_order(q)] ← active_width[2 + stretch_order(q)] + stretch(q);
  active_width[6] ← active_width[6] + shrink(q)
```

This code is used in section 914.

917. The following code knows that discretionary texts contain only character nodes, kern nodes, box nodes, rule nodes, and ligature nodes.

```
⟨ Try to break after a discretionary fragment, then goto done5 917 ⟩ ≡
  begin s ← pre_break(cur_p); disc_width ← 0;
  if s = null then try_break(ex_hyphen_penalty, hyphenated)
  else begin repeat ⟨ Add the width of node s to disc_width 918 ⟩;
      s ← link(s);
    until s = null;
    act_width ← act_width + disc_width; try_break(hyphen_penalty, hyphenated);
    act_width ← act_width − disc_width;
    end;
  r ← replace_count(cur_p); s ← link(cur_p);
  while r > 0 do
    begin ⟨ Add the width of node s to act_width 919 ⟩;
    decr(r); s ← link(s);
    end;
  update_prev_p; cur_p ← s; goto done5;
  end
```

This code is used in section 914.

918.

```
⟨ Add the width of node s to disc_width 918 ⟩ ≡
  if is_char_node(s) then
    begin f ← font(s); disc_width ← disc_width + char_width(f)(char_info(f)(character(s)));
    end
  else case type(s) of
    ligature_node: begin f ← font(lig_char(s)); xtx_ligature_present ← true;
      disc_width ← disc_width + char_width(f)(char_info(f)(character(lig_char(s))));
      end;
    hlist_node, vlist_node, rule_node, kern_node: disc_width ← disc_width + width(s);
    whatsit_node: if (is_native_word_subtype(s)) ∨ (subtype(s) = glyph_node) ∨ (subtype(s) =
          pic_node) ∨ (subtype(s) = pdf_node) then disc_width ← disc_width + width(s)
      else confusion("disc3a");
    othercases confusion("disc3")
    endcases
```

This code is used in section 917.

919. ⟨ Add the width of node s to *act_width* 919 ⟩ ≡

```
if is_char_node(s) then
  begin f ← font(s); act_width ← act_width + char_width(f)(char_info(f)(character(s)));
  end
else case type(s) of
  ligature_node: begin f ← font(lig_char(s)); xtx_ligature_present ← true;
    act_width ← act_width + char_width(f)(char_info(f)(character(lig_char(s))));
    end;
  hlist_node, vlist_node, rule_node, kern_node: act_width ← act_width + width(s);
  whatsit_node: if (is_native_word_subtype(s)) ∨ (subtype(s) = glyph_node) ∨ (subtype(s) =
        pic_node) ∨ (subtype(s) = pdf_node) then act_width ← act_width + width(s)
    else confusion("disc4a");
  othercases confusion("disc4")
  endcases
```

This code is used in section 917.

920. The forced line break at the paragraph's end will reduce the list of breakpoints so that all active nodes represent breaks at $cur_p = null$. On the first pass, we insist on finding an active node that has the correct "looseness." On the final pass, there will be at least one active node, and we will match the desired looseness as well as we can.

The global variable *best_bet* will be set to the active node for the best way to break the paragraph, and a few other variables are used to help determine what is best.

```
⟨ Global variables 13 ⟩ +≡
best_bet: pointer; { use this passive node and its predecessors }
fewest_demerits: integer; { the demerits associated with best_bet }
best_line: halfword; { line number following the last line of the new paragraph }
actual_looseness: integer; { the difference between line_number(best_bet) and the optimum best_line }
line_diff: integer; { the difference between the current line number and the optimum best_line }
```

921. ⟨ Try the final line break at the end of the paragraph, and **goto** *done* if the desired breakpoints have been found 921 ⟩ ≡

```
begin try_break(eject_penalty, hyphenated);
if link(active) ≠ last_active then
  begin ⟨ Find an active node with fewest demerits 922 ⟩;
  if looseness = 0 then goto done;
  ⟨ Find the best active node for the desired looseness 923 ⟩;
  if (actual_looseness = looseness) ∨ final_pass then goto done;
  end;
end
```

This code is used in section 911.

922. ⟨ Find an active node with fewest demerits 922 ⟩ ≡

```
r ← link(active); fewest_demerits ← awful_bad;
repeat if type(r) ≠ delta_node then
    if total_demerits(r) < fewest_demerits then
      begin fewest_demerits ← total_demerits(r); best_bet ← r;
      end;
  r ← link(r);
until r = last_active;
best_line ← line_number(best_bet)
```

This code is used in section 921.

923. The adjustment for a desired looseness is a slightly more complicated version of the loop just considered. Note that if a paragraph is broken into segments by displayed equations, each segment will be subject to the looseness calculation, independently of the other segments.

```
⟨ Find the best active node for the desired looseness 923 ⟩ ≡
  begin r ← link(active); actual_looseness ← 0;
  repeat if type(r) ≠ delta_node then
      begin line_diff ← line_number(r) − best_line;
      if ((line_diff < actual_looseness) ∧ (looseness ≤ line_diff)) ∨
            ((line_diff > actual_looseness) ∧ (looseness ≥ line_diff)) then
        begin best_bet ← r; actual_looseness ← line_diff; fewest_demerits ← total_demerits(r);
        end
      else if (line_diff = actual_looseness) ∧ (total_demerits(r) < fewest_demerits) then
          begin best_bet ← r; fewest_demerits ← total_demerits(r);
          end;
      end;
    r ← link(r);
  until r = last_active;
  best_line ← line_number(best_bet);
  end
```

This code is used in section 921.

924. Once the best sequence of breakpoints has been found (hurray), we call on the procedure *post_line_break* to finish the remainder of the work. (By introducing this subprocedure, we are able to keep *line_break* from getting extremely long.)

```
⟨ Break the paragraph at the chosen breakpoints, justify the resulting lines to the correct widths, and
      append them to the current vertical list 924 ⟩ ≡
  post_line_break(d)
```

This code is used in section 863.

925. The total number of lines that will be set by *post_line_break* is *best_line* − *prev_graf* − 1. The last breakpoint is specified by *break_node*(*best_bet*), and this passive node points to the other breakpoints via the *prev_break* links. The finishing-up phase starts by linking the relevant passive nodes in forward order, changing *prev_break* to *next_break*. (The *next_break* fields actually reside in the same memory space as the *prev_break* fields did, but we give them a new name because of their new significance.) Then the lines are justified, one by one.

```
define next_break ≡ prev_break  { new name for prev_break after links are reversed }
⟨ Declare subprocedures for line_break 874 ⟩ +≡
procedure post_line_break(d : boolean);
  label done, done1;
  var q, r, s: pointer;  { temporary registers for list manipulation }
    p, k: pointer; w: scaled; glue_break: boolean;  { was a break at glue? }
    ptmp: pointer; disc_break: boolean;  { was the current break at a discretionary node? }
    post_disc_break: boolean;  { and did it have a nonempty post-break part? }
    cur_width: scaled;  { width of line number cur_line }
    cur_indent: scaled;  { left margin of line number cur_line }
    t: quarterword;  { used for replacement counts in discretionary nodes }
    pen: integer;  { use when calculating penalties between lines }
    cur_line: halfword;  { the current line number being justified }
    LR_ptr: pointer;  { stack of LR codes }
  begin LR_ptr ← LR_save;
  ⟨ Reverse the links of the relevant passive nodes, setting cur_p to the first breakpoint 926 ⟩;
  cur_line ← prev_graf + 1;
  repeat ⟨ Justify the line ending at breakpoint cur_p, and append it to the current vertical list, together
      with associated penalties and other insertions 928 ⟩;
    incr(cur_line); cur_p ← next_break(cur_p);
    if cur_p ≠ null then
      if ¬post_disc_break then ⟨ Prune unwanted nodes at the beginning of the next line 927 ⟩;
  until cur_p = null;
  if (cur_line ≠ best_line) ∨ (link(temp_head) ≠ null) then confusion("line␣breaking");
  prev_graf ← best_line − 1; LR_save ← LR_ptr;
  end;
```

926. The job of reversing links in a list is conveniently regarded as the job of taking items off one stack and putting them on another. In this case we take them off a stack pointed to by q and having *prev_break* fields; we put them on a stack pointed to by *cur_p* and having *next_break* fields. Node r is the passive node being moved from stack to stack.

```
⟨ Reverse the links of the relevant passive nodes, setting cur_p to the first breakpoint 926 ⟩ ≡
  q ← break_node(best_bet); cur_p ← null;
  repeat r ← q; q ← prev_break(q); next_break(r) ← cur_p; cur_p ← r;
  until q = null
```

This code is used in section 925.

927. Glue and penalty and kern and math nodes are deleted at the beginning of a line, except in the anomalous case that the node to be deleted is actually one of the chosen breakpoints. Otherwise the pruning done here is designed to match the lookahead computation in *try_break*, where the *break_width* values are computed for non-discretionary breakpoints.

```
⟨ Prune unwanted nodes at the beginning of the next line 927 ⟩ ≡
  begin r ← temp_head;
  loop begin q ← link(r);
    if q = cur_break(cur_p) then goto done1;  { cur_break(cur_p) is the next breakpoint }
        { now q cannot be null }
    if is_char_node(q) then goto done1;
    if non_discardable(q) then goto done1;
    if type(q) = kern_node then
      if (subtype(q) ≠ explicit) ∧ (subtype(q) ≠ space_adjustment) then goto done1;
    r ← q;  { now type(q) = glue_node, kern_node, math_node, or penalty_node }
    if type(q) = math_node then
      if TeXXeT_en then ⟨ Adjust the LR stack for the post_line_break routine 1518 ⟩;
    end;
done1: if r ≠ temp_head then
    begin link(r) ← null; flush_node_list(link(temp_head)); link(temp_head) ← q;
    end;
  end
```

This code is used in section 925.

928. The current line to be justified appears in a horizontal list starting at *link*(*temp_head*) and ending at *cur_break*(*cur_p*). If *cur_break*(*cur_p*) is a glue node, we reset the glue to equal the *right_skip* glue; otherwise we append the *right_skip* glue at the right. If *cur_break*(*cur_p*) is a discretionary node, we modify the list so that the discretionary break is compulsory, and we set *disc_break* to *true*. We also append the *left_skip* glue at the left of the line, unless it is zero.

```
⟨ Justify the line ending at breakpoint cur_p, and append it to the current vertical list, together with
      associated penalties and other insertions 928 ⟩ ≡
  if TeXXeT_en then ⟨ Insert LR nodes at the beginning of the current line and adjust the LR stack
      based on LR nodes in this line 1517 ⟩;
  ⟨ Modify the end of the line to reflect the nature of the break and to include \rightskip; also set the
      proper value of disc_break 929 ⟩;
  if TeXXeT_en then ⟨ Insert LR nodes at the end of the current line 1519 ⟩;
  ⟨ Put the \leftskip glue at the left and detach this line 935 ⟩;
  ⟨ Call the packaging subroutine, setting just_box to the justified box 937 ⟩;
  ⟨ Append the new box to the current vertical list, followed by the list of special nodes taken out of the
      box by the packager 936 ⟩;
  ⟨ Append a penalty node, if a nonzero penalty is appropriate 938 ⟩
```

This code is used in section 925.

929. At the end of the following code, q will point to the final node on the list about to be justified.

```
⟨ Modify the end of the line to reflect the nature of the break and to include \rightskip; also set the
      proper value of disc_break 929 ⟩ ≡
  q ← cur_break(cur_p); disc_break ← false; post_disc_break ← false; glue_break ← false;
  if q ≠ null then { q cannot be a char_node }
    if type(q) = glue_node then
      begin delete_glue_ref(glue_ptr(q)); glue_ptr(q) ← right_skip; subtype(q) ← right_skip_code + 1;
      add_glue_ref(right_skip); glue_break ← true; goto done;
      end
    else begin if type(q) = disc_node then
        ⟨ Change discretionary to compulsory and set disc_break ← true 930 ⟩
      else if type(q) = kern_node then width(q) ← 0
        else if type(q) = math_node then
            begin width(q) ← 0;
            if TeXXeT_en then ⟨ Adjust the LR stack for the post_line_break routine 1518 ⟩;
            end;
      end
  else begin q ← temp_head;
    while link(q) ≠ null do q ← link(q);
    end;
done: { at this point q is the rightmost breakpoint; the only exception is the case of a discretionary break
      with non-empty pre_break, then q has been changed to the last node of the pre_break list }
  if XeTeX_protrude_chars > 0 then
    begin
    if disc_break ∧ (is_char_node(q) ∨ (type(q) ≠ disc_node))
        { q has been reset to the last node of pre_break }
    then
    begin p ← q; ptmp ← p;
    end
  else begin p ← prev_rightmost(link(temp_head), q); { get link(p) = q }
    ptmp ← p; p ← find_protchar_right(link(temp_head), p);
    end; w ← right_pw(p);
    if w ≠ 0 then { we have found a marginal kern, append it after ptmp }
      begin k ← new_margin_kern(−w, last_rightmost_char, right_side); link(k) ← link(ptmp);
      link(ptmp) ← k;
      if (ptmp = q) then q ← link(q);
      end;
    end ; { if q was not a breakpoint at glue and has been reset to rightskip then we append rightskip
        after q now }
    if ¬glue_break then
      begin ⟨ Put the \rightskip glue after node q 934 ⟩;
      end;
```

This code is used in section 928.

930.

```
⟨ Change discretionary to compulsory and set disc_break ← true 930 ⟩ ≡
  begin t ← replace_count(q);
  ⟨ Destroy the t nodes following q, and make r point to the following node 931 ⟩;
  if post_break(q) ≠ null then ⟨ Transplant the post-break list 932 ⟩;
  if pre_break(q) ≠ null then ⟨ Transplant the pre-break list 933 ⟩;
  link(q) ← r; disc_break ← true;
  end
```

This code is used in section 929.

931. ⟨ Destroy the t nodes following q, and make r point to the following node 931 ⟩ ≡

```
if t = 0 then r ← link(q)
else begin r ← q;
  while t > 1 do
    begin r ← link(r); decr(t);
    end;
  s ← link(r); r ← link(s); link(s) ← null; flush_node_list(link(q)); replace_count(q) ← 0;
  end
```

This code is used in section 930.

932. We move the post-break list from inside node q to the main list by reattaching it just before the present node r, then resetting r.

⟨ Transplant the post-break list 932 ⟩ ≡

```
begin s ← post_break(q);
while link(s) ≠ null do s ← link(s);
link(s) ← r; r ← post_break(q); post_break(q) ← null; post_disc_break ← true;
end
```

This code is used in section 930.

933. We move the pre-break list from inside node q to the main list by reattaching it just after the present node q, then resetting q.

⟨ Transplant the pre-break list 933 ⟩ ≡

```
begin s ← pre_break(q); link(q) ← s;
while link(s) ≠ null do s ← link(s);
pre_break(q) ← null; q ← s;
end
```

This code is used in section 930.

934. ⟨ Put the `\rightskip` glue after node q 934 ⟩ ≡

```
r ← new_param_glue(right_skip_code); link(r) ← link(q); link(q) ← r; q ← r
```

This code is used in section 929.

935. The following code begins with q at the end of the list to be justified. It ends with q at the beginning of that list, and with $link(temp_head)$ pointing to the remainder of the paragraph, if any.

⟨ Put the `\leftskip` glue at the left and detach this line 935 ⟩ ≡

```
r ← link(q); link(q) ← null; q ← link(temp_head); link(temp_head) ← r;
    { at this point q is the leftmost node; all discardable nodes have been discarded }
if XeTeX_protrude_chars > 0 then
  begin p ← q; p ← find_protchar_left(p, false);   { no more discardables }
  w ← left_pw(p);
  if w ≠ 0 then
    begin k ← new_margin_kern(−w, last_leftmost_char, left_side); link(k) ← q; q ← k;
    end;
  end;
if left_skip ≠ zero_glue then
  begin r ← new_param_glue(left_skip_code); link(r) ← q; q ← r;
  end
```

This code is used in section 928.

936. ⟨ Append the new box to the current vertical list, followed by the list of special nodes taken out of the box by the packager 936 ⟩ ≡

```
if pre_adjust_head ≠ pre_adjust_tail then append_list(pre_adjust_head)(pre_adjust_tail);
pre_adjust_tail ← null; append_to_vlist(just_box);
if adjust_head ≠ adjust_tail then append_list(adjust_head)(adjust_tail);
adjust_tail ← null
```

This code is used in section 928.

937. Now q points to the hlist that represents the current line of the paragraph. We need to compute the appropriate line width, pack the line into a box of this size, and shift the box by the appropriate amount of indentation.

⟨ Call the packaging subroutine, setting *just_box* to the justified box 937 ⟩ ≡

```
if cur_line > last_special_line then
  begin cur_width ← second_width; cur_indent ← second_indent;
  end
else if par_shape_ptr = null then
    begin cur_width ← first_width; cur_indent ← first_indent;
    end
  else begin cur_width ← mem[par_shape_ptr + 2 * cur_line].sc;
    cur_indent ← mem[par_shape_ptr + 2 * cur_line − 1].sc;
    end;
adjust_tail ← adjust_head; pre_adjust_tail ← pre_adjust_head; just_box ← hpack(q, cur_width, exactly);
shift_amount(just_box) ← cur_indent
```

This code is used in section 928.

938. Penalties between the lines of a paragraph come from club and widow lines, from the *inter_line_penalty* parameter, and from lines that end at discretionary breaks. Breaking between lines of a two-line paragraph gets both club-line and widow-line penalties. The local variable *pen* will be set to the sum of all relevant penalties for the current line, except that the final line is never penalized.

```
⟨ Append a penalty node, if a nonzero penalty is appropriate 938 ⟩ ≡
  if cur_line + 1 ≠ best_line then
    begin q ← inter_line_penalties_ptr;
    if q ≠ null then
      begin r ← cur_line;
      if r > penalty(q) then r ← penalty(q);
      pen ← penalty(q + r);
      end
    else pen ← inter_line_penalty;
    q ← club_penalties_ptr;
    if q ≠ null then
      begin r ← cur_line − prev_graf;
      if r > penalty(q) then r ← penalty(q);
      pen ← pen + penalty(q + r);
      end
    else if cur_line = prev_graf + 1 then pen ← pen + club_penalty;
    if d then q ← display_widow_penalties_ptr
    else q ← widow_penalties_ptr;
    if q ≠ null then
      begin r ← best_line − cur_line − 1;
      if r > penalty(q) then r ← penalty(q);
      pen ← pen + penalty(q + r);
      end
    else if cur_line + 2 = best_line then
        if d then pen ← pen + display_widow_penalty
        else pen ← pen + widow_penalty;
    if disc_break then pen ← pen + broken_penalty;
    if pen ≠ 0 then
      begin r ← new_penalty(pen); link(tail) ← r; tail ← r;
      end;
    end
```

This code is used in section 928.

939. Pre-hyphenation. When the line-breaking routine is unable to find a feasible sequence of breakpoints, it makes a second pass over the paragraph, attempting to hyphenate the hyphenatable words. The goal of hyphenation is to insert discretionary material into the paragraph so that there are more potential places to break.

The general rules for hyphenation are somewhat complex and technical, because we want to be able to hyphenate words that are preceded or followed by punctuation marks, and because we want the rules to work for languages other than English. We also must contend with the fact that hyphens might radically alter the ligature and kerning structure of a word.

A sequence of characters will be considered for hyphenation only if it belongs to a "potentially hyphenatable part" of the current paragraph. This is a sequence of nodes $p_0 p_1 \ldots p_m$ where p_0 is a glue node, $p_1 \ldots p_{m-1}$ are either character or ligature or whatsit or implicit kern or text direction nodes, and p_m is a glue or penalty or insertion or adjust or mark or whatsit or explicit kern node. (Therefore hyphenation is disabled by boxes, math formulas, and discretionary nodes already inserted by the user.) The ligature nodes among $p_1 \ldots p_{m-1}$ are effectively expanded into the original non-ligature characters; the kern nodes and whatsits are ignored. Each character c is now classified as either a nonletter (if $lc_code(c) = 0$), a lowercase letter (if $lc_code(c) = c$), or an uppercase letter (otherwise); an uppercase letter is treated as if it were $lc_code(c)$ for purposes of hyphenation. The characters generated by $p_1 \ldots p_{m-1}$ may begin with nonletters; let c_1 be the first letter that is not in the middle of a ligature. Whatsit nodes preceding c_1 are ignored; a whatsit found after c_1 will be the terminating node p_m. All characters that do not have the same font as c_1 will be treated as nonletters. The *hyphen_char* for that font must be between 0 and 255, otherwise hyphenation will not be attempted. TeX looks ahead for as many consecutive letters $c_1 \ldots c_n$ as possible; however, n must be less than $max_hyphenatable_length + 1$, so a character that would otherwise be $c_{max_hyphenatable_length+1}$ is effectively not a letter. Furthermore c_n must not be in the middle of a ligature. In this way we obtain a string of letters $c_1 \ldots c_n$ that are generated by nodes $p_a \ldots p_b$, where $1 \le a \le b+1 \le m$. If $n \ge l_hyf + r_hyf$, this string qualifies for hyphenation; however, *uc_hyph* must be positive, if c_1 is uppercase.

The hyphenation process takes place in three stages. First, the candidate sequence $c_1 \ldots c_n$ is found; then potential positions for hyphens are determined by referring to hyphenation tables; and finally, the nodes $p_a \ldots p_b$ are replaced by a new sequence of nodes that includes the discretionary breaks found.

Fortunately, we do not have to do all this calculation very often, because of the way it has been taken out of TeX's inner loop. For example, when the second edition of the author's 700-page book *Seminumerical Algorithms* was typeset by TeX, only about 1.2 hyphenations needed to be tried per paragraph, since the line breaking algorithm needed to use two passes on only about 5 per cent of the paragraphs.

⟨ Initialize for hyphenating a paragraph 939 ⟩ ≡
```
begin init if trie_not_ready then init_trie;
tini
cur_lang ← init_cur_lang; l_hyf ← init_l_hyf; r_hyf ← init_r_hyf; set_hyph_index;
end
```
This code is used in section 911.

940. The letters $c_1 \ldots c_n$ that are candidates for hyphenation are placed into an array called *hc*; the number n is placed into *hn*; pointers to nodes p_{a-1} and p_b in the description above are placed into variables *ha* and *hb*; and the font number is placed into *hf*.

```
⟨ Global variables 13 ⟩ +≡
hc: array [0 .. hyphenatable_length_limit + 3] of 0 .. number_usvs;  { word to be hyphenated }
      { note that element 0 needs to be a full UnicodeScalar, even though we basically work in UTF16 }
hn: small_number;  { the number of positions occupied in hc, 0..64 in TeX }
ha, hb: pointer;  { nodes ha .. hb should be replaced by the hyphenated result }
hf: internal_font_number;  { font number of the letters in hc }
hu: array [0 .. hyphenatable_length_limit + 1] of 0 .. too_big_char;
        { like hc, before conversion to lowercase }
hyf_char: integer;  { hyphen character of the relevant font }
cur_lang, init_cur_lang: 0 .. biggest_lang;  { current hyphenation table of interest }
l_hyf, r_hyf, init_l_hyf, init_r_hyf: integer;  { limits on fragment sizes }
hyf_bchar: halfword;  { boundary character after c_n }
max_hyph_char: integer;
```

941.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  max_hyph_char ← too_big_lang;
```

942. Hyphenation routines need a few more local variables.

```
⟨ Local variables for line breaking 910 ⟩ +≡
j: small_number;  { an index into hc or hu }
c: UnicodeScalar;  { character being considered for hyphenation }
```

943. When the following code is activated, the *line_break* procedure is in its second pass, and *cur_p* points to a glue node.

```
⟨ Try to hyphenate the following word 943 ⟩ ≡
  begin prev_s ← cur_p; s ← link(prev_s);
  if s ≠ null then
    begin ⟨ Skip to node ha, or goto done1 if no hyphenation should be attempted 949 ⟩;
    if l_hyf + r_hyf > max_hyphenatable_length then goto done1;
    if is_native_word_node(ha) then
      begin ⟨ Check that nodes after native_word permit hyphenation; if not, goto done1 945 ⟩;
      ⟨ Prepare a native_word_node for hyphenation 946 ⟩;
      end
    else begin ⟨ Skip to node hb, putting letters into hu and hc 950 ⟩;
      end;
    ⟨ Check that the nodes following hb permit hyphenation and that at least l_hyf + r_hyf letters have
        been found, otherwise goto done1 952 ⟩;
    hyphenate;
    end;
done1: end
```

This code is used in section 914.

944. ⟨ Declare subprocedures for *line_break* 874 ⟩ +≡
⟨ Declare the function called *reconstitute* 960 ⟩
procedure *hyphenate*;
  label *common_ending*, *done*, *found*, *found1*, *found2*, *not_found*, *exit*;
  var ⟨ Local variables for hyphenation 954 ⟩
  begin ⟨ Find hyphen locations for the word in *hc*, or **return** 977 ⟩;
  ⟨ If no hyphens were found, **return** 955 ⟩;
  ⟨ Replace nodes *ha* .. *hb* by a sequence of nodes that includes the discretionary hyphens 956 ⟩;
exit: **end**;
function *max_hyphenatable_length*: *integer*;
  begin if *XeTeX_hyphenatable_length* > *hyphenatable_length_limit* **then**
    max_hyphenatable_length ← *hyphenatable_length_limit*
  else *max_hyphenatable_length* ← *XeTeX_hyphenatable_length*;
  end;

945. ⟨ Check that nodes after *native_word* permit hyphenation; if not, **goto** *done1* 945 ⟩ ≡
  s ← *link*(*ha*);
  loop begin if ¬(*is_char_node*(*s*)) **then**
      case *type*(*s*) **of**
      ligature_node: *do_nothing*;
      kern_node: **if** *subtype*(*s*) ≠ *normal* **then goto** *done6*;
      whatsit_node, *glue_node*, *penalty_node*, *ins_node*, *adjust_node*, *mark_node*: **goto** *done6*;
      othercases goto *done1*
      endcases;
    s ← *link*(*s*);
    end;
done6:

This code is used in section 943.

946. ⟨ Prepare a *native_word_node* for hyphenation 946 ⟩ ≡

```
{ note that if there are chars with lccode = 0, we split them out into separate native_word nodes }
hn ← 0;
restart: for l ← 0 to native_length(ha) − 1 do
  begin c ← get_native_usv(ha, l); set_lc_code(c);
  if (hc[0] = 0) then
    begin if (hn > 0) then
      begin { we've got some letters, and now found a non-letter, so break off the tail of the
          native_word and link it after this node, and goto done3 }
      ⟨ Split the native_word_node at l and link the second part after ha 947 ⟩;
      goto done3;
      end
    end
  else if (hn = 0) ∧ (l > 0) then
      begin { we've found the first letter after some non-letters, so break off the head of the
          native_word and restart }
      ⟨ Split the native_word_node at l and link the second part after ha 947 ⟩;
      ha ← link(ha); goto restart;
      end
    else if (hn = max_hyphenatable_length) then { reached max hyphenatable length }
        goto done3
      else begin { found a letter that is part of a potentially hyphenatable sequence }
        incr(hn);
        if c < "10000 then
          begin hu[hn] ← c; hc[hn] ← hc[0];
          end
        else begin hu[hn] ← (c − "10000) div "400 + "D800;
          hc[hn] ← (hc[0] − "10000) div "400 + "D800; incr(hn); hu[hn] ← c mod "400 + "DC00;
          hc[hn] ← hc[0] mod "400 + "DC00; incr(l);
          end;
        hyf_bchar ← non_char;
        end
  end;
```

This code is used in section 943.

947. ⟨ Split the *native_word_node* at *l* and link the second part after *ha* 947 ⟩ ≡

```
q ← new_native_word_node(hf, native_length(ha) − l); subtype(q) ← subtype(ha);
for i ← l to native_length(ha) − 1 do set_native_char(q, i − l, get_native_char(ha, i));
set_native_metrics(q, XeTeX_use_glyph_metrics); link(q) ← link(ha); link(ha) ← q;
    { truncate text in node ha }
native_length(ha) ← l; set_native_metrics(ha, XeTeX_use_glyph_metrics);
```

This code is used in sections 946 and 946.

948. ⟨ Local variables for line breaking 910 ⟩ +≡

```
l: integer;
i: integer;
```

949. The first thing we need to do is find the node *ha* just before the first letter.

```
⟨ Skip to node ha, or goto done1 if no hyphenation should be attempted 949 ⟩ ≡
  loop begin if is_char_node(s) then
      begin c ← qo(character(s)); hf ← font(s);
      end
    else if type(s) = ligature_node then
        if lig_ptr(s) = null then goto continue
        else begin q ← lig_ptr(s); c ← qo(character(q)); hf ← font(q);
          end
      else if (type(s) = kern_node) ∧ (subtype(s) = normal) then goto continue
        else if (type(s) = math_node) ∧ (subtype(s) ≥ L_code) then goto continue
          else if type(s) = whatsit_node then
              begin if (is_native_word_subtype(s)) then
                begin
                    { we only consider the node if it contains at least one letter, otherwise we'll skip it }
                for l ← 0 to native_length(s) − 1 do
                  begin c ← get_native_usv(s, l);
                  if lc_code(c) ≠ 0 then
                    begin hf ← native_font(s); prev_s ← s;
                    if (lc_code(c) = c) ∨ (uc_hyph > 0) then goto done2
                    else goto done1;
                    end;
                  if c ≥ "10000 then incr(l);
                  end
                end;
              ⟨ Advance past a whatsit node in the pre-hyphenation loop 1423 ⟩;
              goto continue
              end
            else goto done1;
    set_lc_code(c);
    if hc[0] ≠ 0 then
      if (hc[0] = c) ∨ (uc_hyph > 0) then goto done2
      else goto done1;
  continue: prev_s ← s; s ← link(prev_s);
    end;
done2: hyf_char ← hyphen_char[hf];
  if hyf_char < 0 then goto done1;
  if hyf_char > biggest_char then goto done1;
  ha ← prev_s
```

This code is used in section 943.

950. The word to be hyphenated is now moved to the *hu* and *hc* arrays.

```
⟨ Skip to node hb, putting letters into hu and hc 950 ⟩ ≡
  hn ← 0;
  loop begin if is_char_node(s) then
      begin if font(s) ≠ hf then goto done3;
      hyf_bchar ← character(s); c ← qo(hyf_bchar); set_lc_code(c);
      if hc[0] = 0 then goto done3;
      if hc[0] > max_hyph_char then goto done3;
      if hn = max_hyphenatable_length then goto done3;
      hb ← s; incr(hn); hu[hn] ← c; hc[hn] ← hc[0]; hyf_bchar ← non_char;
      end
    else if type(s) = ligature_node then ⟨ Move the characters of a ligature node to hu and hc; but goto
          done3 if they are not all letters 951 ⟩
      else if (type(s) = kern_node) ∧ (subtype(s) = normal) then
          begin hb ← s; hyf_bchar ← font_bchar[hf];
          end
        else goto done3;
    s ← link(s);
    end;
done3:
```

This code is used in section 943.

951. We let *j* be the index of the character being stored when a ligature node is being expanded, since we do not want to advance *hn* until we are sure that the entire ligature consists of letters. Note that it is possible to get to *done3* with *hn* = 0 and *hb* not set to any value.

```
⟨ Move the characters of a ligature node to hu and hc; but goto done3 if they are not all letters 951 ⟩ ≡
  begin if font(lig_char(s)) ≠ hf then goto done3;
  j ← hn; q ← lig_ptr(s); if q > null then hyf_bchar ← character(q);
  while q > null do
    begin c ← qo(character(q)); set_lc_code(c);
    if hc[0] = 0 then goto done3;
    if hc[0] > max_hyph_char then goto done3;
    if j = max_hyphenatable_length then goto done3;
    incr(j); hu[j] ← c; hc[j] ← hc[0];
    q ← link(q);
    end;
  hb ← s; hn ← j;
  if odd(subtype(s)) then hyf_bchar ← font_bchar[hf] else hyf_bchar ← non_char;
  end
```

This code is used in section 950.

952. ⟨ Check that the nodes following *hb* permit hyphenation and that at least *l_hyf* + *r_hyf* letters have been found, otherwise **goto** *done1* 952 ⟩ ≡

```
if hn < l_hyf + r_hyf then goto done1;  { l_hyf and r_hyf are ≥ 1 }
loop begin if ¬(is_char_node(s)) then
    case type(s) of
    ligature_node: do_nothing;
    kern_node: if subtype(s) ≠ normal then goto done4;
    whatsit_node, glue_node, penalty_node, ins_node, adjust_node, mark_node: goto done4;
    math_node: if subtype(s) ≥ L_code then goto done4 else goto done1;
    othercases goto done1
    endcases;
  s ← link(s);
  end;
done4:
```

This code is used in section 943.

953. Post-hyphenation. If a hyphen may be inserted between $hc[j]$ and $hc[j+1]$, the hyphenation procedure will set $hyf[j]$ to some small odd number. But before we look at TeX's hyphenation procedure, which is independent of the rest of the line-breaking algorithm, let us consider what we will do with the hyphens it finds, since it is better to work on this part of the program before forgetting what ha and hb, etc., are all about.

```
⟨ Global variables 13 ⟩ +≡
hyf: array [0 .. hyphenatable_length_limit + 1] of 0 .. 9;  { odd values indicate discretionary hyphens }
init_list: pointer;  { list of punctuation characters preceding the word }
init_lig: boolean;  { does init_list represent a ligature? }
init_lft: boolean;  { if so, did the ligature involve a left boundary? }
```

954.
```
⟨ Local variables for hyphenation 954 ⟩ ≡
i, j, l: 0 .. hyphenatable_length_limit + 2;  { indices into hc or hu }
q, r, s: pointer;  { temporary registers for list manipulation }
bchar: halfword;  { boundary character of hyphenated word, or non_char }
```
See also sections 966, 976, and 983.

This code is used in section 944.

955. TeX will never insert a hyphen that has fewer than `\lefthyphenmin` letters before it or fewer than `\righthyphenmin` after it; hence, a short word has comparatively little chance of being hyphenated. If no hyphens have been found, we can save time by not having to make any changes to the paragraph.

```
⟨ If no hyphens were found, return 955 ⟩ ≡
  for j ← l_hyf to hn − r_hyf do
    if odd(hyf[j]) then goto found1;
  return;
found1:
```
This code is used in section 944.

956. If hyphens are in fact going to be inserted, TEX first deletes the subsequence of nodes between *ha* and *hb*. An attempt is made to preserve the effect that implicit boundary characters and punctuation marks had on ligatures inside the hyphenated word, by storing a left boundary or preceding character in *hu*[0] and by storing a possible right boundary in *bchar*. We set $j \leftarrow 0$ if *hu*[0] is to be part of the reconstruction; otherwise $j \leftarrow 1$. The variable *s* will point to the tail of the current hlist, and *q* will point to the node following *hb*, so that things can be hooked up after we reconstitute the hyphenated word.

```
⟨ Replace nodes ha .. hb by a sequence of nodes that includes the discretionary hyphens 956 ⟩ ≡
  if is_native_word_node(ha) then
    begin ⟨ Hyphenate the native_word_node at ha 957 ⟩;
    end
  else begin q ← link(hb); link(hb) ← null; r ← link(ha); link(ha) ← null; bchar ← hyf_bchar;
    if is_char_node(ha) then
      if font(ha) ≠ hf then goto found2
      else begin init_list ← ha; init_lig ← false; hu[0] ← qo(character(ha));
        end
    else if type(ha) = ligature_node then
        if font(lig_char(ha)) ≠ hf then goto found2
        else begin init_list ← lig_ptr(ha); init_lig ← true; init_lft ← (subtype(ha) > 1);
          hu[0] ← qo(character(lig_char(ha)));
          if init_list = null then
            if init_lft then
              begin hu[0] ← max_hyph_char; init_lig ← false;
              end; { in this case a ligature will be reconstructed from scratch }
          free_node(ha, small_node_size);
          end
      else begin { no punctuation found; look for left boundary }
        if ¬is_char_node(r) then
          if type(r) = ligature_node then
            if subtype(r) > 1 then goto found2;
        j ← 1; s ← ha; init_list ← null; goto common_ending;
        end;
    s ← cur_p; { we have cur_p ≠ ha because type(cur_p) = glue_node }
    while link(s) ≠ ha do s ← link(s);
    j ← 0; goto common_ending;
  found2: s ← ha; j ← 0; hu[0] ← max_hyph_char; init_lig ← false; init_list ← null;
  common_ending: flush_node_list(r);
    ⟨ Reconstitute nodes for the hyphenated word, inserting discretionary hyphens 967 ⟩;
    flush_list(init_list);
    end
```

This code is used in section 944.

957. ⟨ Hyphenate the *native_word_node* at *ha* 957 ⟩ ≡

```
{ find the node immediately before the word to be hyphenated }
s ← cur_p; { we have cur_p ≠ ha because type(cur_p) = glue_node }
while link(s) ≠ ha do s ← link(s); { for each hyphen position, create a native_word_node fragment for
    the text before this point, and a disc_node for the break, with the hyf_char in the pre_break text }
hyphen_passed ← 0; { location of last hyphen we saw }
for j ← l_hyf to hn − r_hyf do
  begin { if this is a valid break.... }
  if odd(hyf[j]) then
    begin { make a native_word_node for the fragment before the hyphen }
    q ← new_native_word_node(hf, j − hyphen_passed); subtype(q) ← subtype(ha);
    for i ← 0 to j − hyphen_passed − 1 do set_native_char(q, i, get_native_char(ha, i + hyphen_passed));
    set_native_metrics(q, XeTeX_use_glyph_metrics); link(s) ← q; { append the new node }
    s ← q; { make the disc_node for the hyphenation point }
    q ← new_disc; pre_break(q) ← new_native_character(hf, hyf_char); link(s) ← q; s ← q;
    hyphen_passed ← j;
    end
  end; { make a native_word_node for the last fragment of the word }
hn ← native_length(ha); { ensure trailing punctuation is not lost! }
q ← new_native_word_node(hf, hn − hyphen_passed); subtype(q) ← subtype(ha);
for i ← 0 to hn − hyphen_passed − 1 do set_native_char(q, i, get_native_char(ha, i + hyphen_passed));
set_native_metrics(q, XeTeX_use_glyph_metrics); link(s) ← q; { append the new node }
s ← q; q ← link(ha); link(s) ← q; link(ha) ← null; flush_node_list(ha);
```

This code is used in section 956.

958. We must now face the fact that the battle is not over, even though the hyphens have been found: The process of reconstituting a word can be nontrivial because ligatures might change when a hyphen is present. *The TEXbook* discusses the difficulties of the word "difficult", and the discretionary material surrounding a hyphen can be considerably more complex than that. Suppose `abcdef` is a word in a font for which the only ligatures are `bc`, `cd`, `de`, and `ef`. If this word permits hyphenation between `b` and `c`, the two patterns with and without hyphenation are `a b - cd ef` and `a bc de f`. Thus the insertion of a hyphen might cause effects to ripple arbitrarily far into the rest of the word. A further complication arises if additional hyphens appear together with such rippling, e.g., if the word in the example just given could also be hyphenated between `c` and `d`; TEX avoids this by simply ignoring the additional hyphens in such weird cases.

Still further complications arise in the presence of ligatures that do not delete the original characters. When punctuation precedes the word being hyphenated, TEX's method is not perfect under all possible scenarios, because punctuation marks and letters can propagate information back and forth. For example, suppose the original pre-hyphenation pair `*a` changes to `*y` via a `|=:` ligature, which changes to `xy` via a `=:|` ligature; if $p_{a-1} =$ `x` and $p_a =$ `y`, the reconstitution procedure isn't smart enough to obtain `xy` again. In such cases the font designer should include a ligature that goes from `xa` to `xy`.

959. The processing is facilitated by a subroutine called *reconstitute*. Given a string of characters $x_j \dots x_n$, there is a smallest index $m \geq j$ such that the "translation" of $x_j \dots x_n$ by ligatures and kerning has the form $y_1 \dots y_t$ followed by the translation of $x_{m+1} \dots x_n$, where $y_1 \dots y_t$ is some nonempty sequence of character, ligature, and kern nodes. We call $x_j \dots x_m$ a "cut prefix" of $x_j \dots x_n$. For example, if $x_1 x_2 x_3 =$ `fly`, and if the font contains 'fl' as a ligature and a kern between 'fl' and 'y', then $m = 2$, $t = 2$, and y_1 will be a ligature node for 'fl' followed by an appropriate kern node y_2. In the most common case, x_j forms no ligature with x_{j+1} and we simply have $m = j$, $y_1 = x_j$. If $m < n$ we can repeat the procedure on $x_{m+1} \dots x_n$ until the entire translation has been found.

The *reconstitute* function returns the integer m and puts the nodes $y_1 \dots y_t$ into a linked list starting at *link*(*hold_head*), getting the input $x_j \dots x_n$ from the *hu* array. If $x_j = 256$, we consider x_j to be an implicit left boundary character; in this case j must be strictly less than n. There is a parameter *bchar*, which is either 256 or an implicit right boundary character assumed to be present just following x_n. (The value $hu[n+1]$ is never explicitly examined, but the algorithm imagines that *bchar* is there.)

If there exists an index k in the range $j \leq k \leq m$ such that $hyf[k]$ is odd and such that the result of *reconstitute* would have been different if x_{k+1} had been *hchar*, then *reconstitute* sets *hyphen_passed* to the smallest such k. Otherwise it sets *hyphen_passed* to zero.

A special convention is used in the case $j = 0$: Then we assume that the translation of $hu[0]$ appears in a special list of charnodes starting at *init_list*; moreover, if *init_lig* is *true*, then $hu[0]$ will be a ligature character, involving a left boundary if *init_lft* is *true*. This facility is provided for cases when a hyphenated word is preceded by punctuation (like single or double quotes) that might affect the translation of the beginning of the word.

⟨ Global variables 13 ⟩ +≡
hyphen_passed: *small_number*; { first hyphen in a ligature, if any }

960. ⟨ Declare the function called *reconstitute* 960 ⟩ ≡
function *reconstitute*(*j*, *n* : *small_number*; *bchar*, *hchar* : *halfword*): *small_number*;
label *continue*, *done*;
var *p*: *pointer*; { temporary register for list manipulation }
t: *pointer*; { a node being appended to }
q: *four_quarters*; { character information or a lig/kern instruction }
cur_rh: *halfword*; { hyphen character for ligature testing }
test_char: *halfword*; { hyphen or other character for ligature testing }
w: *scaled*; { amount of kerning }
k: *font_index*; { position of current lig/kern instruction }
begin *hyphen_passed* ← 0; *t* ← *hold_head*; *w* ← 0; *link*(*hold_head*) ← *null*;
{ at this point *ligature_present* = *lft_hit* = *rt_hit* = *false* }
⟨ Set up data structures with the cursor following position *j* 962 ⟩;
continue: ⟨ If there's a ligature or kern at the cursor position, update the data structures, possibly advancing *j*; continue until the cursor moves 963 ⟩;
⟨ Append a ligature and/or kern to the translation; **goto** *continue* if the stack of inserted ligatures is nonempty 964 ⟩;
reconstitute ← *j*;
end;

This code is used in section 944.

961. The reconstitution procedure shares many of the global data structures by which TeX has processed the words before they were hyphenated. There is an implied "cursor" between characters *cur_l* and *cur_r*; these characters will be tested for possible ligature activity. If *ligature_present* then *cur_l* is a ligature character formed from the original characters following *cur_q* in the current translation list. There is a "ligature stack" between the cursor and character $j+1$, consisting of pseudo-ligature nodes linked together by their *link* fields. This stack is normally empty unless a ligature command has created a new character that will need to be processed later. A pseudo-ligature is a special node having a *character* field that represents a potential ligature and a *lig_ptr* field that points to a *char_node* or is *null*. We have

$$cur_r = \begin{cases} character(lig_stack), & \text{if } lig_stack > null; \\ qi(hu[j+1]), & \text{if } lig_stack = null \text{ and } j < n; \\ bchar, & \text{if } lig_stack = null \text{ and } j = n. \end{cases}$$

⟨ Global variables 13 ⟩ +≡
cur_l, *cur_r*: *halfword*; { characters before and after the cursor }
cur_q: *pointer*; { where a ligature should be detached }
lig_stack: *pointer*; { unfinished business to the right of the cursor }
ligature_present: *boolean*; { should a ligature node be made for *cur_l*? }
lft_hit, *rt_hit*: *boolean*; { did we hit a ligature with a boundary character? }

962. **define** *append_charnode_to_t*(#) ≡
begin *link*(*t*) ← *get_avail*; *t* ← *link*(*t*); *font*(*t*) ← *hf*; *character*(*t*) ← #;
end
define *set_cur_r* ≡
begin if *j* < *n* **then** *cur_r* ← *qi*(*hu*[*j* + 1]) **else** *cur_r* ← *bchar*;
if *odd*(*hyf*[*j*]) **then** *cur_rh* ← *hchar* **else** *cur_rh* ← *non_char*;
end

⟨ Set up data structures with the cursor following position *j* 962 ⟩ ≡
cur_l ← *qi*(*hu*[*j*]); *cur_q* ← *t*;
if *j* = 0 **then**
begin *ligature_present* ← *init_lig*; *p* ← *init_list*;
if *ligature_present* **then** *lft_hit* ← *init_lft*;
while *p* > *null* **do**
begin *append_charnode_to_t*(*character*(*p*)); *p* ← *link*(*p*);
end;
end
else if *cur_l* < *non_char* **then** *append_charnode_to_t*(*cur_l*);
lig_stack ← *null*; *set_cur_r*

This code is used in section 960.

963. We may want to look at the lig/kern program twice, once for a hyphen and once for a normal letter. (The hyphen might appear after the letter in the program, so we'd better not try to look for both at once.)

```
⟨ If there's a ligature or kern at the cursor position, update the data structures, possibly advancing j;
      continue until the cursor moves 963 ⟩ ≡
  if cur_l = non_char then
    begin k ← bchar_label[hf];
    if k = non_address then goto done else q ← font_info[k].qqqq;
    end
  else begin q ← char_info(hf)(cur_l);
    if char_tag(q) ≠ lig_tag then goto done;
    k ← lig_kern_start(hf)(q); q ← font_info[k].qqqq;
    if skip_byte(q) > stop_flag then
      begin k ← lig_kern_restart(hf)(q); q ← font_info[k].qqqq;
      end;
    end; { now k is the starting address of the lig/kern program }
  if cur_rh < non_char then test_char ← cur_rh else test_char ← cur_r;
  loop begin if next_char(q) = test_char then
      if skip_byte(q) ≤ stop_flag then
        if cur_rh < non_char then
          begin hyphen_passed ← j; hchar ← non_char; cur_rh ← non_char; goto continue;
          end
        else begin if hchar < non_char then
            if odd(hyf[j]) then
              begin hyphen_passed ← j; hchar ← non_char;
              end;
          if op_byte(q) < kern_flag then
            ⟨ Carry out a ligature replacement, updating the cursor structure and possibly advancing j;
                goto continue if the cursor doesn't advance, otherwise goto done 965 ⟩;
          w ← char_kern(hf)(q); goto done; { this kern will be inserted below }
          end;
    if skip_byte(q) ≥ stop_flag then
      if cur_rh = non_char then goto done
      else begin cur_rh ← non_char; goto continue;
        end;
    k ← k + qo(skip_byte(q)) + 1; q ← font_info[k].qqqq;
    end;
done:
```

This code is used in section 960.

964. **define** *wrap_lig*(#) ≡
  if *ligature_present* **then**
    begin $p \leftarrow$ *new_ligature*(*hf*, *cur_l*, *link*(*cur_q*));
    if *lft_hit* **then**
      begin *subtype*(*p*) ← 2; *lft_hit* ← *false*;
      end;
    if # **then**
      if *lig_stack* = *null* **then**
        begin *incr*(*subtype*(*p*)); *rt_hit* ← *false*;
        end;
    link(*cur_q*) ← *p*; *t* ← *p*; *ligature_present* ← *false*;
    end
define *pop_lig_stack* ≡
  begin if *lig_ptr*(*lig_stack*) > *null* **then**
    begin *link*(*t*) ← *lig_ptr*(*lig_stack*); { this is a charnode for *hu*[*j* + 1] }
    t ← *link*(*t*); *incr*(*j*);
    end;
  p ← *lig_stack*; *lig_stack* ← *link*(*p*); *free_node*(*p*, *small_node_size*);
  if *lig_stack* = *null* **then** *set_cur_r* **else** *cur_r* ← *character*(*lig_stack*);
  end { if *lig_stack* isn't *null* we have *cur_rh* = *non_char* }

⟨ Append a ligature and/or kern to the translation; **goto** *continue* if the stack of inserted ligatures is nonempty 964 ⟩ ≡
wrap_lig(*rt_hit*);
if *w* ≠ 0 **then**
  begin *link*(*t*) ← *new_kern*(*w*); *t* ← *link*(*t*); *w* ← 0;
  end;
if *lig_stack* > *null* **then**
  begin *cur_q* ← *t*; *cur_l* ← *character*(*lig_stack*); *ligature_present* ← *true*; *pop_lig_stack*;
  goto *continue*;
  end

This code is used in section 960.

965. ⟨ Carry out a ligature replacement, updating the cursor structure and possibly advancing j; **goto** *continue* if the cursor doesn't advance, otherwise **goto** *done* 965 ⟩ ≡

```
begin if cur_l = non_char then lft_hit ← true;
if j = n then
  if lig_stack = null then rt_hit ← true;
check_interrupt; { allow a way out in case there's an infinite ligature loop }
case op_byte(q) of
qi(1), qi(5): begin cur_l ← rem_byte(q); { =:|, =:|> }
  ligature_present ← true;
  end;
qi(2), qi(6): begin cur_r ← rem_byte(q); { |=:, |=:> }
  if lig_stack > null then character(lig_stack) ← cur_r
  else begin lig_stack ← new_lig_item(cur_r);
    if j = n then bchar ← non_char
    else begin p ← get_avail; lig_ptr(lig_stack) ← p; character(p) ← qi(hu[j + 1]); font(p) ← hf;
      end;
    end;
  end;
qi(3): begin cur_r ← rem_byte(q); { |=:| }
  p ← lig_stack; lig_stack ← new_lig_item(cur_r); link(lig_stack) ← p;
  end;
qi(7), qi(11): begin wrap_lig(false); { |=:|>, |=:|>> }
  cur_q ← t; cur_l ← rem_byte(q); ligature_present ← true;
  end;
othercases begin cur_l ← rem_byte(q); ligature_present ← true; { =: }
  if lig_stack > null then pop_lig_stack
  else if j = n then goto done
    else begin append_charnode_to_t(cur_r); incr(j); set_cur_r;
      end;
  end
endcases;
if op_byte(q) > qi(4) then
  if op_byte(q) ≠ qi(7) then goto done;
goto continue;
end
```

This code is used in section 963.

966. Okay, we're ready to insert the potential hyphenations that were found. When the following program is executed, we want to append the word $hu[1 \,..\, hn]$ after node ha, and node q should be appended to the result. During this process, the variable i will be a temporary index into hu; the variable j will be an index to our current position in hu; the variable l will be the counterpart of j, in a discretionary branch; the variable r will point to new nodes being created; and we need a few new local variables:

```
⟨ Local variables for hyphenation 954 ⟩ +≡
major_tail, minor_tail: pointer;
    { the end of lists in the main and discretionary branches being reconstructed }
c: UnicodeScalar; { character temporarily replaced by a hyphen }
c_loc: 0 .. hyphenatable_length_limit; { where that character came from }
r_count: integer; { replacement count for discretionary }
hyf_node: pointer; { the hyphen, if it exists }
```

967. When the following code is performed, $hyf[0]$ and $hyf[hn]$ will be zero.

```
⟨ Reconstitute nodes for the hyphenated word, inserting discretionary hyphens 967 ⟩ ≡
  repeat l ← j; j ← reconstitute(j, hn, bchar, qi(hyf_char)) + 1;
    if hyphen_passed = 0 then
      begin link(s) ← link(hold_head);
      while link(s) > null do s ← link(s);
      if odd(hyf[j − 1]) then
        begin l ← j; hyphen_passed ← j − 1; link(hold_head) ← null;
        end;
      end;
    if hyphen_passed > 0 then ⟨ Create and append a discretionary node as an alternative to the
        unhyphenated word, and continue to develop both branches until they become equivalent 968 ⟩;
  until j > hn;
  link(s) ← q
```

This code is used in section 956.

968. In this repeat loop we will insert another discretionary if $hyf[j-1]$ is odd, when both branches of the previous discretionary end at position $j-1$. Strictly speaking, we aren't justified in doing this, because we don't know that a hyphen after $j-1$ is truly independent of those branches. But in almost all applications we would rather not lose a potentially valuable hyphenation point. (Consider the word 'difficult', where the letter 'c' is in position j.)

```
  define advance_major_tail ≡
      begin major_tail ← link(major_tail); incr(r_count);
      end

⟨ Create and append a discretionary node as an alternative to the unhyphenated word, and continue to
    develop both branches until they become equivalent 968 ⟩ ≡
  repeat r ← get_node(small_node_size); link(r) ← link(hold_head); type(r) ← disc_node;
    major_tail ← r; r_count ← 0;
    while link(major_tail) > null do advance_major_tail;
    i ← hyphen_passed; hyf[i] ← 0; ⟨ Put the characters hu[l . . i] and a hyphen into pre_break(r) 969 ⟩;
    ⟨ Put the characters hu[i + 1 . .] into post_break(r), appending to this list and to major_tail until
        synchronization has been achieved 970 ⟩;
    ⟨ Move pointer s to the end of the current list, and set replace_count(r) appropriately 972 ⟩;
    hyphen_passed ← j − 1; link(hold_head) ← null;
  until ¬odd(hyf[j − 1])
```

This code is used in section 967.

969. The new hyphen might combine with the previous character via ligature or kern. At this point we have $l - 1 \le i < j$ and $i < hn$.

```
⟨ Put the characters hu[l .. i] and a hyphen into pre_break(r) 969 ⟩ ≡
  minor_tail ← null; pre_break(r) ← null; hyf_node ← new_character(hf, hyf_char);
  if hyf_node ≠ null then
    begin incr(i); c ← hu[i]; hu[i] ← hyf_char; free_avail(hyf_node);
    end;
  while l ≤ i do
    begin l ← reconstitute(l, i, font_bchar[hf], non_char) + 1;
    if link(hold_head) > null then
      begin if minor_tail = null then pre_break(r) ← link(hold_head)
      else link(minor_tail) ← link(hold_head);
      minor_tail ← link(hold_head);
      while link(minor_tail) > null do minor_tail ← link(minor_tail);
      end;
    end;
  if hyf_node ≠ null then
    begin hu[i] ← c; { restore the character in the hyphen position }
    l ← i; decr(i);
    end
```

This code is used in section 968.

970. The synchronization algorithm begins with $l = i + 1 \le j$.

```
⟨ Put the characters hu[i + 1 ..] into post_break(r), appending to this list and to major_tail until
      synchronization has been achieved 970 ⟩ ≡
  minor_tail ← null; post_break(r) ← null; c_loc ← 0;
  if bchar_label[hf] ≠ non_address then { put left boundary at beginning of new line }
    begin decr(l); c ← hu[l]; c_loc ← l; hu[l] ← max_hyph_char;
    end;
  while l < j do
    begin repeat l ← reconstitute(l, hn, bchar, non_char) + 1;
      if c_loc > 0 then
        begin hu[c_loc] ← c; c_loc ← 0;
        end;
      if link(hold_head) > null then
        begin if minor_tail = null then post_break(r) ← link(hold_head)
        else link(minor_tail) ← link(hold_head);
        minor_tail ← link(hold_head);
        while link(minor_tail) > null do minor_tail ← link(minor_tail);
        end;
    until l ≥ j;
    while l > j do ⟨ Append characters of hu[j ..] to major_tail, advancing j 971 ⟩;
    end
```

This code is used in section 968.

971.

```
⟨ Append characters of hu[j ..] to major_tail, advancing j 971 ⟩ ≡
  begin j ← reconstitute(j, hn, bchar, non_char) + 1; link(major_tail) ← link(hold_head);
  while link(major_tail) > null do advance_major_tail;
  end
```

This code is used in section 970.

972. Ligature insertion can cause a word to grow exponentially in size. Therefore we must test the size of *r_count* here, even though the hyphenated text was at most *max_hyphenatable_length* characters long.

⟨ Move pointer s to the end of the current list, and set *replace_count*(r) appropriately 972 ⟩ ≡

```
if r_count > 127 then { we have to forget the discretionary hyphen }
  begin link(s) ← link(r); link(r) ← null; flush_node_list(r);
  end
else begin link(s) ← r; replace_count(r) ← r_count;
  end;
s ← major_tail
```

This code is used in section 968.

973. Hyphenation. When a word $hc[1\,..\,hn]$ has been set up to contain a candidate for hyphenation, TEX first looks to see if it is in the user's exception dictionary. If not, hyphens are inserted based on patterns that appear within the given word, using an algorithm due to Frank M. Liang.

Let's consider Liang's method first, since it is much more interesting than the exception-lookup routine. The algorithm begins by setting $hyf[j]$ to zero for all j, and invalid characters are inserted into $hc[0]$ and $hc[hn+1]$ to serve as delimiters. Then a reasonably fast method is used to see which of a given set of patterns occurs in the word $hc[0\,..\,(hn+1)]$. Each pattern $p_1 \ldots p_k$ of length k has an associated sequence of $k+1$ numbers $n_0 \ldots n_k$; and if the pattern occurs in $hc[(j+1)\,..\,(j+k)]$, TEX will set $hyf[j+i] \leftarrow \max(hyf[j+i], n_i)$ for $0 \le i \le k$. After this has been done for each pattern that occurs, a discretionary hyphen will be inserted between $hc[j]$ and $hc[j+1]$ when $hyf[j]$ is odd, as we have already seen.

The set of patterns $p_1 \ldots p_k$ and associated numbers $n_0 \ldots n_k$ depends, of course, on the language whose words are being hyphenated, and on the degree of hyphenation that is desired. A method for finding appropriate p's and n's, from a given dictionary of words and acceptable hyphenations, is discussed in Liang's Ph.D. thesis (Stanford University, 1983); TEX simply starts with the patterns and works from there.

974. The patterns are stored in a compact table that is also efficient for retrieval, using a variant of "trie memory" [cf. *The Art of Computer Programming* **3** (1973), 481–505]. We can find each pattern $p_1 \ldots p_k$ by letting z_0 be one greater than the relevant language index and then, for $1 \le i \le k$, setting $z_i \leftarrow trie_link(z_{i-1}) + p_i$; the pattern will be identified by the number z_k. Since all the pattern information is packed together into a single *trie_link* array, it is necessary to prevent confusion between the data from inequivalent patterns, so another table is provided such that $trie_char(z_i) = p_i$ for all i. There is also a table $trie_op(z_k)$ to identify the numbers $n_0 \ldots n_k$ associated with $p_1 \ldots p_k$.

Comparatively few different number sequences $n_0 \ldots n_k$ actually occur, since most of the n's are generally zero. Therefore the number sequences are encoded in such a way that $trie_op(z_k)$ is only one byte long. If $trie_op(z_k) \neq min_quarterword$, when $p_1 \ldots p_k$ has matched the letters in $hc[(l-k+1)\,..\,l]$ of language t, we perform all of the required operations for this pattern by carrying out the following little program: Set $v \leftarrow trie_op(z_k)$. Then set $v \leftarrow v + op_start[t]$, $hyf[l-hyf_distance[v]] \leftarrow \max(hyf[l-hyf_distance[v]], hyf_num[v])$, and $v \leftarrow hyf_next[v]$; repeat, if necessary, until $v = min_quarterword$.

⟨ Types in the outer block 18 ⟩ +≡
  trie_pointer = 0 .. *trie_size*; { an index into *trie* }

975. **define** *trie_link*(#) ≡ *trie*[#].*rh* { "downward" link in a trie }
  define *trie_char*(#) ≡ *trie*[#].*b1* { character matched at this trie location }
  define *trie_op*(#) ≡ *trie*[#].*b0* { program for hyphenation at this trie location }

⟨ Global variables 13 ⟩ +≡
trie: **array** [*trie_pointer*] **of** *two_halves*; { *trie_link*, *trie_char*, *trie_op* }
hyf_distance: **array** [1 .. *trie_op_size*] **of** *small_number*; { position $k-j$ of n_j }
hyf_num: **array** [1 .. *trie_op_size*] **of** *small_number*; { value of n_j }
hyf_next: **array** [1 .. *trie_op_size*] **of** *quarterword*; { continuation code }
op_start: **array** [0 .. *biggest_lang*] **of** 0 .. *trie_op_size*; { offset for current language }

976. ⟨ Local variables for hyphenation 954 ⟩ +≡
z: *trie_pointer*; { an index into *trie* }
v: *integer*; { an index into *hyf_distance*, etc. }

977. Assuming that these auxiliary tables have been set up properly, the hyphenation algorithm is quite short. In the following code we set $hc[hn+2]$ to the impossible value 256, in order to guarantee that $hc[hn+3]$ will never be fetched.

```
⟨ Find hyphen locations for the word in hc, or return 977 ⟩ ≡
  for j ← 0 to hn do hyf[j] ← 0;
  ⟨ Look for the word hc[1 .. hn] in the exception table, and goto found (with hyf containing the hyphens)
      if an entry is found 984 ⟩;
  if trie_char(cur_lang + 1) ≠ qi(cur_lang) then return;  { no patterns for cur_lang }
  hc[0] ← 0; hc[hn + 1] ← 0; hc[hn + 2] ← max_hyph_char;  { insert delimiters }
  for j ← 0 to hn − r_hyf + 1 do
    begin z ← trie_link(cur_lang + 1) + hc[j]; l ← j;
    while hc[l] = qo(trie_char(z)) do
      begin if trie_op(z) ≠ min_quarterword then ⟨ Store maximum values in the hyf table 978 ⟩;
      incr(l); z ← trie_link(z) + hc[l];
      end;
    end;
found: for j ← 0 to l_hyf − 1 do hyf[j] ← 0;
  for j ← 0 to r_hyf − 1 do hyf[hn − j] ← 0
```

This code is used in section 944.

978.

```
⟨ Store maximum values in the hyf table 978 ⟩ ≡
  begin v ← trie_op(z);
  repeat v ← v + op_start[cur_lang]; i ← l − hyf_distance[v];
    if hyf_num[v] > hyf[i] then hyf[i] ← hyf_num[v];
    v ← hyf_next[v];
  until v = min_quarterword;
  end
```

This code is used in section 977.

979. The exception table that is built by TeX's `\hyphenation` primitive is organized as an ordered hash table [cf. Amble and Knuth, *The Computer Journal* **17** (1974), 135–142] using linear probing. If α and β are words, we will say that $\alpha < \beta$ if $|\alpha| < |\beta|$ or if $|\alpha| = |\beta|$ and α is lexicographically smaller than β. (The notation $|\alpha|$ stands for the length of α.) The idea of ordered hashing is to arrange the table so that a given word α can be sought by computing a hash address $h = h(\alpha)$ and then looking in table positions h, $h-1$, ..., until encountering the first word $\leq \alpha$. If this word is different from α, we can conclude that α is not in the table.

The words in the table point to lists in *mem* that specify hyphen positions in their *info* fields. The list for $c_1 \ldots c_n$ contains the number k if the word $c_1 \ldots c_n$ has a discretionary hyphen between c_k and c_{k+1}.

```
⟨ Types in the outer block 18 ⟩ +≡
  hyph_pointer = 0 .. hyph_size;  { an index into the ordered hash table }
```

980.

```
⟨ Global variables 13 ⟩ +≡
hyph_word: array [hyph_pointer] of str_number;  { exception words }
hyph_list: array [hyph_pointer] of pointer;  { lists of hyphen positions }
hyph_count: hyph_pointer;  { the number of words in the exception dictionary }
```

981.

```
⟨ Local variables for initialization 19 ⟩ +≡
z: hyph_pointer;  { runs through the exception dictionary }
```

982. ⟨ Set initial values of key variables 23 ⟩ +≡
```
for z ← 0 to hyph_size do
  begin hyph_word[z] ← 0; hyph_list[z] ← null;
  end;
hyph_count ← 0;
```

983. The algorithm for exception lookup is quite simple, as soon as we have a few more local variables to work with.

⟨ Local variables for hyphenation 954 ⟩ +≡
```
h: hyph_pointer; { an index into hyph_word and hyph_list }
k: str_number; { an index into str_start }
u: pool_pointer; { an index into str_pool }
```

984. First we compute the hash code h, then we search until we either find the word or we don't. Words from different languages are kept separate by appending the language code to the string.

⟨ Look for the word $hc[1 \mathinner{..} hn]$ in the exception table, and **goto** *found* (with *hyf* containing the hyphens) if an entry is found 984 ⟩ ≡
```
h ← hc[1]; incr(hn); hc[hn] ← cur_lang;
for j ← 2 to hn do h ← (h + h + hc[j]) mod hyph_size;
loop begin ⟨ If the string hyph_word[h] is less than hc[1 .. hn], goto not_found; but if the two strings
      are equal, set hyf to the hyphen positions and goto found 985 ⟩;
  if h > 0 then decr(h) else h ← hyph_size;
  end;
not_found: decr(hn)
```
This code is used in section 977.

985. ⟨ If the string $hyph_word[h]$ is less than $hc[1 \mathinner{..} hn]$, **goto** *not_found*; but if the two strings are equal, set *hyf* to the hyphen positions and **goto** *found* 985 ⟩ ≡
```
k ← hyph_word[h];
if k = 0 then goto not_found;
if length(k) < hn then goto not_found;
if length(k) = hn then
  begin j ← 1; u ← str_start_macro(k);
  repeat if so(str_pool[u]) < hc[j] then goto not_found;
    if so(str_pool[u]) > hc[j] then goto done;
    incr(j); incr(u);
  until j > hn;
  ⟨ Insert hyphens as specified in hyph_list[h] 986 ⟩;
  decr(hn); goto found;
  end;
done:
```
This code is used in section 984.

986. ⟨ Insert hyphens as specified in $hyph_list[h]$ 986 ⟩ ≡
```
s ← hyph_list[h];
while s ≠ null do
  begin hyf[info(s)] ← 1; s ← link(s);
  end
```
This code is used in section 985.

987. ⟨ Search *hyph_list* for pointers to p 987 ⟩ ≡

```
for q ← 0 to hyph_size do
  begin if hyph_list[q] = p then
    begin print_nl("HYPH("); print_int(q); print_char(")");
    end;
  end
```

This code is used in section 197.

988. We have now completed the hyphenation routine, so the *line_break* procedure is finished at last. Since the hyphenation exception table is fresh in our minds, it's a good time to deal with the routine that adds new entries to it.

When TeX has scanned '`\hyphenation`', it calls on a procedure named *new_hyph_exceptions* to do the right thing.

```
define set_cur_lang ≡
        if language ≤ 0 then cur_lang ← 0
        else if language > biggest_lang then cur_lang ← 0
          else cur_lang ← language
procedure new_hyph_exceptions; { enters new exceptions }
  label reswitch, exit, found, not_found, not_found1;
  var n: 0 .. hyphenatable_length_limit + 1; { length of current word; not always a small_number }
    j: 0 .. hyphenatable_length_limit + 1; { an index into hc }
    h: hyph_pointer; { an index into hyph_word and hyph_list }
    k: str_number; { an index into str_start }
    p: pointer; { head of a list of hyphen positions }
    q: pointer; { used when creating a new node for list p }
    s, t: str_number; { strings being compared or stored }
    u, v: pool_pointer; { indices into str_pool }
  begin scan_left_brace; { a left brace must follow \hyphenation }
  set_cur_lang;
  init if trie_not_ready then
    begin hyph_index ← 0; goto not_found1;
    end;
  tini
  set_hyph_index;
not_found1: ⟨ Enter as many hyphenation exceptions as are listed, until coming to a right brace; then
      return 989 ⟩;
exit: end;
```

989. ⟨ Enter as many hyphenation exceptions as are listed, until coming to a right brace; then **return** 989 ⟩ ≡

```
n ← 0; p ← null;
loop begin get_x_token;
reswitch: case cur_cmd of
  letter, other_char, char_given: ⟨ Append a new letter or hyphen 991 ⟩;
  char_num: begin scan_char_num; cur_chr ← cur_val; cur_cmd ← char_given; goto reswitch;
    end;
  spacer, right_brace: begin if n > 1 then ⟨ Enter a hyphenation exception 993 ⟩;
    if cur_cmd = right_brace then return;
    n ← 0; p ← null;
    end;
  othercases ⟨ Give improper \hyphenation error 990 ⟩
  endcases;
  end
```

This code is used in section 988.

990. ⟨ Give improper `\hyphenation` error 990 ⟩ ≡

```
begin print_err("Improper␣"); print_esc("hyphenation"); print("␣will␣be␣flushed");
help2("Hyphenation␣exceptions␣must␣contain␣only␣letters")
("and␣hyphens.␣But␣continue;␣I´ll␣forgive␣and␣forget."); error;
end
```

This code is used in section 989.

991. ⟨ Append a new letter or hyphen 991 ⟩ ≡

```
if cur_chr = "-" then ⟨ Append the value n to list p 992 ⟩
else begin set_lc_code(cur_chr);
  if hc[0] = 0 then
    begin print_err("Not␣a␣letter");
    help2("Letters␣in␣\hyphenation␣words␣must␣have␣\lccode>0.")
    ("Proceed;␣I´ll␣ignore␣the␣character␣I␣just␣read."); error;
    end
  else if n < max_hyphenatable_length then
      begin incr(n);
      if hc[0] < ″10000 then hc[n] ← hc[0]
      else begin hc[n] ← (hc[0] − ″10000) div ″400 + ″D800; incr(n); hc[n] ← hc[0] mod ″400 + ″DC00;
        end;
      end;
  end
```

This code is used in section 989.

992. ⟨ Append the value n to list p 992 ⟩ ≡

```
begin if n < max_hyphenatable_length then
  begin q ← get_avail; link(q) ← p; info(q) ← n; p ← q;
  end;
end
```

This code is used in section 991.

993. ⟨ Enter a hyphenation exception 993 ⟩ ≡
```
begin incr(n); hc[n] ← cur_lang; str_room(n); h ← 0;
for j ← 1 to n do
  begin h ← (h + h + hc[j]) mod hyph_size; append_char(hc[j]);
  end;
s ← make_string; ⟨ Insert the pair (s, p) into the exception table 994 ⟩;
end
```
This code is used in section 989.

994. ⟨ Insert the pair (s,p) into the exception table 994 ⟩ ≡
```
if hyph_count = hyph_size then overflow("exception␣dictionary", hyph_size);
incr(hyph_count);
while hyph_word[h] ≠ 0 do
  begin ⟨ If the string hyph_word[h] is less than or equal to s, interchange (hyph_word[h], hyph_list[h])
      with (s, p) 995 ⟩;
  if h > 0 then decr(h) else h ← hyph_size;
  end;
hyph_word[h] ← s; hyph_list[h] ← p
```
This code is used in section 993.

995. ⟨ If the string $hyph_word[h]$ is less than or equal to s, interchange $(hyph_word[h], hyph_list[h])$ with (s,p) 995 ⟩ ≡
```
k ← hyph_word[h];
if length(k) < length(s) then goto found;
if length(k) > length(s) then goto not_found;
u ← str_start_macro(k); v ← str_start_macro(s);
repeat if str_pool[u] < str_pool[v] then goto found;
  if str_pool[u] > str_pool[v] then goto not_found;
  incr(u); incr(v);
until u = str_start_macro(k + 1);
found: q ← hyph_list[h]; hyph_list[h] ← p; p ← q;
  t ← hyph_word[h]; hyph_word[h] ← s; s ← t;
not_found:
```
This code is used in section 994.

996. Initializing the hyphenation tables. The trie for TeX's hyphenation algorithm is built from a sequence of patterns following a `\patterns` specification. Such a specification is allowed only in `INITEX`, since the extra memory for auxiliary tables and for the initialization program itself would only clutter up the production version of TeX with a lot of deadwood.

The first step is to build a trie that is linked, instead of packed into sequential storage, so that insertions are readily made. After all patterns have been processed, `INITEX` compresses the linked trie by identifying common subtries. Finally the trie is packed into the efficient sequential form that the hyphenation algorithm actually uses.

⟨ Declare subprocedures for *line_break* 874 ⟩ +≡
init ⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩
tini

997. Before we discuss trie building in detail, let's consider the simpler problem of creating the *hyf_distance*, *hyf_num*, and *hyf_next* arrays.

Suppose, for example, that TeX reads the pattern '`ab2cde1`'. This is a pattern of length 5, with $n_0 \ldots n_5 = 0\,0\,2\,0\,0\,1$ in the notation above. We want the corresponding *trie_op* code v to have $hyf_distance[v] = 3$, $hyf_num[v] = 2$, and $hyf_next[v] = v'$, where the auxiliary *trie_op* code v' has $hyf_distance[v'] = 0$, $hyf_num[v'] = 1$, and $hyf_next[v'] = min_quarterword$.

TeX computes an appropriate value v with the *new_trie_op* subroutine below, by setting

$$v' \leftarrow new_trie_op(0, 1, min_quarterword), \qquad v \leftarrow new_trie_op(3, 2, v').$$

This subroutine looks up its three parameters in a special hash table, assigning a new value only if these three have not appeared before for the current language.

The hash table is called *trie_op_hash*, and the number of entries it contains is *trie_op_ptr*.

⟨ Global variables 13 ⟩ +≡
init *trie_op_hash*: **array** [−*trie_op_size* .. *trie_op_size*] **of** 0 .. *trie_op_size*;
{ trie op codes for quadruples }
trie_used: **array** [*ASCII_code*] **of** *quarterword*; { largest opcode used so far for this language }
trie_op_lang: **array** [1 .. *trie_op_size*] **of** 0 .. *biggest_lang*; { language part of a hashed quadruple }
trie_op_val: **array** [1 .. *trie_op_size*] **of** *quarterword*; { opcode corresponding to a hashed quadruple }
trie_op_ptr: 0 .. *trie_op_size*; { number of stored ops so far }
tini

998. It's tempting to remove the *overflow* stops in the following procedure; *new_trie_op* could return *min_quarterword* (thereby simply ignoring part of a hyphenation pattern) instead of aborting the job. However, that would lead to different hyphenation results on different installations of TeX using the same patterns. The *overflow* stops are necessary for portability of patterns.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ ≡
function new_trie_op(d, n : small_number; v : quarterword): quarterword;
  label exit;
  var h: −trie_op_size .. trie_op_size;  { trial hash location }
    u: quarterword;  { trial op code }
    l: 0 .. trie_op_size;  { pointer to stored data }
  begin h ← abs(n + 313 ∗ d + 361 ∗ v + 1009 ∗ cur_lang) mod (trie_op_size + trie_op_size) − trie_op_size;
  loop begin l ← trie_op_hash[h];
    if l = 0 then  { empty position found for a new op }
      begin if trie_op_ptr = trie_op_size then overflow("pattern␣memory␣ops", trie_op_size);
      u ← trie_used[cur_lang];
      if u = max_quarterword then
        overflow("pattern␣memory␣ops␣per␣language", max_quarterword − min_quarterword);
      incr(trie_op_ptr); incr(u); trie_used[cur_lang] ← u; hyf_distance[trie_op_ptr] ← d;
      hyf_num[trie_op_ptr] ← n; hyf_next[trie_op_ptr] ← v; trie_op_lang[trie_op_ptr] ← cur_lang;
      trie_op_hash[h] ← trie_op_ptr; trie_op_val[trie_op_ptr] ← u; new_trie_op ← u; return;
      end;
    if (hyf_distance[l] = d) ∧ (hyf_num[l] = n) ∧ (hyf_next[l] = v) ∧ (trie_op_lang[l] = cur_lang) then
      begin new_trie_op ← trie_op_val[l]; return;
      end;
    if h > −trie_op_size then decr(h) else h ← trie_op_size;
    end;
exit: end;
```

See also sections 1002, 1003, 1007, 1011, 1013, 1014, and 1020.

This code is used in section 996.

999. After *new_trie_op* has compressed the necessary opcode information, plenty of information is available to unscramble the data into the final form needed by our hyphenation algorithm.

```
⟨ Sort the hyphenation op tables into proper order 999 ⟩ ≡
  op_start[0] ← −min_quarterword;
  for j ← 1 to biggest_lang do op_start[j] ← op_start[j − 1] + qo(trie_used[j − 1]);
  for j ← 1 to trie_op_ptr do trie_op_hash[j] ← op_start[trie_op_lang[j]] + trie_op_val[j];  { destination }
  for j ← 1 to trie_op_ptr do
    while trie_op_hash[j] > j do
      begin k ← trie_op_hash[j];
      t ← hyf_distance[k]; hyf_distance[k] ← hyf_distance[j]; hyf_distance[j] ← t;
      t ← hyf_num[k]; hyf_num[k] ← hyf_num[j]; hyf_num[j] ← t;
      t ← hyf_next[k]; hyf_next[k] ← hyf_next[j]; hyf_next[j] ← t;
      trie_op_hash[j] ← trie_op_hash[k]; trie_op_hash[k] ← k;
      end
```

This code is used in section 1006.

1000. Before we forget how to initialize the data structures that have been mentioned so far, let's write down the code that gets them started.

```
⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
  for k ← −trie_op_size to trie_op_size do trie_op_hash[k] ← 0;
  for k ← 0 to 255 do trie_used[k] ← min_quarterword;
  trie_op_ptr ← 0;
```

1001. The linked trie that is used to preprocess hyphenation patterns appears in several global arrays. Each node represents an instruction of the form "if you see character c, then perform operation o, move to the next character, and go to node l; otherwise go to node r." The four quantities c, o, l, and r are stored in four arrays *trie_c*, *trie_o*, *trie_l*, and *trie_r*. The root of the trie is *trie_l*[0], and the number of nodes is *trie_ptr*. Null trie pointers are represented by zero. To initialize the trie, we simply set *trie_l*[0] and *trie_ptr* to zero. We also set *trie_c*[0] to some arbitrary value, since the algorithm may access it.

The algorithms maintain the condition

$$trie_c[trie_r[z]] > trie_c[z] \qquad \text{whenever } z \neq 0 \text{ and } trie_r[z] \neq 0;$$

in other words, sibling nodes are ordered by their c fields.

```
  define trie_root ≡ trie_l[0]  { root of the linked trie }
⟨ Global variables 13 ⟩ +≡
  init trie_c: packed array [trie_pointer] of packed_ASCII_code;  { characters to match }
  trie_o: packed array [trie_pointer] of quarterword;  { operations to perform }
  trie_l: packed array [trie_pointer] of trie_pointer;  { left subtrie links }
  trie_r: packed array [trie_pointer] of trie_pointer;  { right subtrie links }
  trie_ptr: trie_pointer;  { the number of nodes in the trie }
  trie_hash: packed array [trie_pointer] of trie_pointer;  { used to identify equivalent subtries }
  tini
```

1002. Let us suppose that a linked trie has already been constructed. Experience shows that we can often reduce its size by recognizing common subtries; therefore another hash table is introduced for this purpose, somewhat similar to *trie_op_hash*. The new hash table will be initialized to zero.

The function *trie_node*(p) returns p if p is distinct from other nodes that it has seen, otherwise it returns the number of the first equivalent node that it has seen.

Notice that we might make subtries equivalent even if they correspond to patterns for different languages, in which the trie ops might mean quite different things. That's perfectly all right.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
function trie_node(p : trie_pointer): trie_pointer;  { converts to a canonical form }
  label exit;
  var h: trie_pointer;  { trial hash location }
    q: trie_pointer;  { trial trie node }
  begin h ← abs(trie_c[p] + 1009 * trie_o[p] + 2718 * trie_l[p] + 3142 * trie_r[p]) mod trie_size;
  loop begin q ← trie_hash[h];
    if q = 0 then
      begin trie_hash[h] ← p; trie_node ← p; return;
      end;
    if (trie_c[q] = trie_c[p]) ∧ (trie_o[q] = trie_o[p]) ∧ (trie_l[q] = trie_l[p]) ∧ (trie_r[q] = trie_r[p]) then
      begin trie_node ← q; return;
      end;
    if h > 0 then decr(h) else h ← trie_size;
    end;
exit: end;
```

1003. A neat recursive procedure is now able to compress a trie by traversing it and applying *trie_node* to its nodes in "bottom up" fashion. We will compress the entire trie by clearing *trie_hash* to zero and then saying '*trie_root* ← *compress_trie*(*trie_root*)'.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
function compress_trie(p : trie_pointer): trie_pointer;
  begin if p = 0 then compress_trie ← 0
  else begin trie_l[p] ← compress_trie(trie_l[p]); trie_r[p] ← compress_trie(trie_r[p]);
    compress_trie ← trie_node(p);
    end;
  end;
```

1004. The compressed trie will be packed into the *trie* array using a "top-down first-fit" procedure. This is a little tricky, so the reader should pay close attention: The *trie_hash* array is cleared to zero again and renamed *trie_ref* for this phase of the operation; later on, *trie_ref*[*p*] will be nonzero only if the linked trie node *p* is the smallest character in a family and if the characters *c* of that family have been allocated to locations *trie_ref*[*p*] + *c* in the *trie* array. Locations of *trie* that are in use will have *trie_link* = 0, while the unused holes in *trie* will be doubly linked with *trie_link* pointing to the next larger vacant location and *trie_back* pointing to the next smaller one. This double linking will have been carried out only as far as *trie_max*, where *trie_max* is the largest index of *trie* that will be needed. To save time at the low end of the trie, we maintain array entries *trie_min*[*c*] pointing to the smallest hole that is greater than *c*. Another array *trie_taken* tells whether or not a given location is equal to *trie_ref*[*p*] for some *p*; this array is used to ensure that distinct nodes in the compressed trie will have distinct *trie_ref* entries.

```
  define trie_ref ≡ trie_hash  { where linked trie families go into trie }
  define trie_back(#) ≡ trie[#].lh  { backward links in trie holes }
⟨ Global variables 13 ⟩ +≡
  init trie_taken: packed array [1 .. trie_size] of boolean;  { does a family start here? }
  trie_min: array [ASCII_code] of trie_pointer;  { the first possible slot for each character }
  trie_max: trie_pointer;  { largest location used in trie }
  trie_not_ready: boolean;  { is the trie still in linked form? }
  tini
```

1005. Each time `\patterns` appears, it contributes further patterns to the future trie, which will be built only when hyphenation is attempted or when a format file is dumped. The boolean variable *trie_not_ready* will change to *false* when the trie is compressed; this will disable further patterns.

```
⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
  trie_not_ready ← true; trie_root ← 0; trie_c[0] ← si(0); trie_ptr ← 0;
```

1006. Here is how the trie-compression data structures are initialized. If storage is tight, it would be possible to overlap *trie_op_hash*, *trie_op_lang*, and *trie_op_val* with *trie*, *trie_hash*, and *trie_taken*, because we finish with the former just before we need the latter.

```
⟨ Get ready to compress the trie 1006 ⟩ ≡
  ⟨ Sort the hyphenation op tables into proper order 999 ⟩;
  for p ← 0 to trie_size do trie_hash[p] ← 0;
  hyph_root ← compress_trie(hyph_root); trie_root ← compress_trie(trie_root);
      { identify equivalent subtries }
  for p ← 0 to trie_ptr do trie_ref[p] ← 0;
  for p ← 0 to biggest_char do trie_min[p] ← p + 1;
  trie_link(0) ← 1; trie_max ← 0
```

This code is used in section 1020.

1007. The *first_fit* procedure finds the smallest hole z in *trie* such that a trie family starting at a given node p will fit into vacant positions starting at z. If $c = trie_c[p]$, this means that location $z - c$ must not already be taken by some other family, and that $z - c + c'$ must be vacant for all characters c' in the family. The procedure sets $trie_ref[p]$ to $z - c$ when the first fit has been found.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
procedure first_fit(p : trie_pointer); { packs a family into trie }
  label not_found, found;
  var h: trie_pointer; { candidate for trie_ref[p] }
    z: trie_pointer; { runs through holes }
    q: trie_pointer; { runs through the family starting at p }
    c: ASCII_code; { smallest character in the family }
    l, r: trie_pointer; { left and right neighbors }
    ll: 1 .. too_big_char; { upper limit of trie_min updating }
  begin c ← so(trie_c[p]); z ← trie_min[c]; { get the first conceivably good hole }
  loop begin h ← z − c;
    ⟨ Ensure that trie_max ≥ h + max_hyph_char 1008 ⟩;
    if trie_taken[h] then goto not_found;
    ⟨ If all characters of the family fit relative to h, then goto found, otherwise goto not_found 1009 ⟩;
  not_found: z ← trie_link(z); { move to the next hole }
    end;
found: ⟨ Pack the family into trie relative to h 1010 ⟩;
  end;
```

1008. By making sure that $trie_max$ is at least $h + max_hyph_char$, we can be sure that $trie_max > z$, since $h = z - c$. It follows that location $trie_max$ will never be occupied in *trie*, and we will have $trie_max \geq trie_link(z)$.

```
⟨ Ensure that trie_max ≥ h + max_hyph_char 1008 ⟩ ≡
  if trie_max < h + max_hyph_char then
    begin if trie_size ≤ h + max_hyph_char then overflow("pattern␣memory", trie_size);
    repeat incr(trie_max); trie_taken[trie_max] ← false; trie_link(trie_max) ← trie_max + 1;
      trie_back(trie_max) ← trie_max − 1;
    until trie_max = h + max_hyph_char;
    end
```

This code is used in section 1007.

1009. ⟨ If all characters of the family fit relative to h, then **goto** *found*, otherwise **goto** *not_found* 1009 ⟩ ≡

```
  q ← trie_r[p];
  while q > 0 do
    begin if trie_link(h + so(trie_c[q])) = 0 then goto not_found;
    q ← trie_r[q];
    end;
  goto found
```

This code is used in section 1007.

1010. ⟨ Pack the family into *trie* relative to h 1010 ⟩ ≡

```
trie_taken[h] ← true; trie_ref[p] ← h; q ← p;
repeat z ← h + so(trie_c[q]); l ← trie_back(z); r ← trie_link(z); trie_back(r) ← l; trie_link(l) ← r;
  trie_link(z) ← 0;
  if l < max_hyph_char then
    begin if z < max_hyph_char then ll ← z else ll ← max_hyph_char;
    repeat trie_min[l] ← r; incr(l);
    until l = ll;
    end;
  q ← trie_r[q];
until q = 0
```

This code is used in section 1007.

1011. To pack the entire linked trie, we use the following recursive procedure.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
procedure trie_pack(p : trie_pointer); { pack subtries of a family }
  var q: trie_pointer; { a local variable that need not be saved on recursive calls }
  begin repeat q ← trie_l[p];
    if (q > 0) ∧ (trie_ref[q] = 0) then
      begin first_fit(q); trie_pack(q);
      end;
    p ← trie_r[p];
  until p = 0;
  end;
```

1012. When the whole trie has been allocated into the sequential table, we must go through it once again so that *trie* contains the correct information. Null pointers in the linked trie will be represented by the value 0, which properly implements an "empty" family.

```
⟨ Move the data into trie 1012 ⟩ ≡
  h.rh ← 0; h.b0 ← min_quarterword; h.b1 ← min_quarterword;
        { trie_link ← 0, trie_op ← min_quarterword, trie_char ← qi(0) }
  if trie_max = 0 then { no patterns were given }
    begin for r ← 0 to 256 do trie[r] ← h;
    trie_max ← 256;
    end
  else begin if hyph_root > 0 then trie_fix(hyph_root);
    if trie_root > 0 then trie_fix(trie_root); { this fixes the non-holes in trie }
    r ← 0; { now we will zero out all the holes }
    repeat s ← trie_link(r); trie[r] ← h; r ← s;
    until r > trie_max;
    end;
  trie_char(0) ← qi("?"); { make trie_char(c) ≠ c for all c }
```

This code is used in section 1020.

1013. The fixing-up procedure is, of course, recursive. Since the linked trie usually has overlapping subtries, the same data may be moved several times; but that causes no harm, and at most as much work is done as it took to build the uncompressed trie.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
procedure trie_fix(p : trie_pointer); { moves p and its siblings into trie }
  var q: trie_pointer; { a local variable that need not be saved on recursive calls }
    c: ASCII_code; { another one that need not be saved }
    z: trie_pointer; { trie reference; this local variable must be saved }
  begin z ← trie_ref[p];
  repeat q ← trie_l[p]; c ← so(trie_c[p]); trie_link(z + c) ← trie_ref[q]; trie_char(z + c) ← qi(c);
    trie_op(z + c) ← trie_o[p];
    if q > 0 then trie_fix(q);
    p ← trie_r[p];
  until p = 0;
  end;
```

1014. Now let's go back to the easier problem, of building the linked trie. When `INITEX` has scanned the '`\patterns`' control sequence, it calls on *new_patterns* to do the right thing.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
procedure new_patterns; { initializes the hyphenation pattern data }
  label done, done1;
  var k, l: 0 .. hyphenatable_length_limit + 1;
        { indices into hc and hyf; not always in small_number range }
    digit_sensed: boolean; { should the next digit be treated as a letter? }
    v: quarterword; { trie op code }
    p, q: trie_pointer; { nodes of trie traversed during insertion }
    first_child: boolean; { is p = trie_l[q]? }
    c: ASCII_code; { character being inserted }
  begin if trie_not_ready then
    begin set_cur_lang; scan_left_brace; { a left brace must follow \patterns }
    ⟨ Enter all of the patterns into a linked trie, until coming to a right brace 1015 ⟩;
    if saving_hyph_codes > 0 then ⟨ Store hyphenation codes for current language 1666 ⟩;
    end
  else begin print_err("Too late for \patterns"); print_esc("patterns");
    help1("All patterns must be given before typesetting begins."); error;
    link(garbage) ← scan_toks(false, false); flush_list(def_ref);
    end;
  end;
```

1015. Novices are not supposed to be using `\patterns`, so the error messages are terse. (Note that all error messages appear in TeX's string pool, even if they are used only by `INITEX`.)

```
⟨ Enter all of the patterns into a linked trie, until coming to a right brace 1015 ⟩ ≡
  k ← 0; hyf[0] ← 0; digit_sensed ← false;
  loop begin get_x_token;
    case cur_cmd of
    letter, other_char: ⟨ Append a new letter or a hyphen level 1016 ⟩;
    spacer, right_brace: begin if k > 0 then ⟨ Insert a new pattern into the linked trie 1017 ⟩;
      if cur_cmd = right_brace then goto done;
      k ← 0; hyf[0] ← 0; digit_sensed ← false;
      end;
    othercases begin print_err("Bad␣"); print_esc("patterns"); help1("(See␣Appendix␣H.)"); error;
      end
    endcases;
    end;
done:
```

This code is used in section 1014.

1016.

```
⟨ Append a new letter or a hyphen level 1016 ⟩ ≡
  if digit_sensed ∨ (cur_chr < "0") ∨ (cur_chr > "9") then
    begin if cur_chr = "." then cur_chr ← 0   { edge-of-word delimiter }
    else begin cur_chr ← lc_code(cur_chr);
      if cur_chr = 0 then
        begin print_err("Nonletter"); help1("(See␣Appendix␣H.)"); error;
        end;
      end;
    if cur_chr > max_hyph_char then max_hyph_char ← cur_chr;
    if k < max_hyphenatable_length then
      begin incr(k); hc[k] ← cur_chr; hyf[k] ← 0; digit_sensed ← false;
      end;
    end
  else if k < max_hyphenatable_length then
      begin hyf[k] ← cur_chr − "0"; digit_sensed ← true;
      end
```

This code is used in section 1015.

1017. When the following code comes into play, the pattern $p_1 \ldots p_k$ appears in $hc[1 \,..\, k]$, and the corresponding sequence of numbers $n_0 \ldots n_k$ appears in $hyf[0 \,..\, k]$.

```
⟨ Insert a new pattern into the linked trie 1017 ⟩ ≡
  begin ⟨ Compute the trie op code, v, and set l ← 0 1019 ⟩;
  q ← 0; hc[0] ← cur_lang;
  while l ≤ k do
    begin c ← hc[l]; incr(l); p ← trie_l[q]; first_child ← true;
    while (p > 0) ∧ (c > so(trie_c[p])) do
      begin q ← p; p ← trie_r[q]; first_child ← false;
      end;
    if (p = 0) ∨ (c < so(trie_c[p])) then
      ⟨ Insert a new trie node between q and p, and make p point to it 1018 ⟩;
    q ← p;  { now node q represents p_1 ... p_{l-1} }
    end;
  if trie_o[q] ≠ min_quarterword then
    begin print_err("Duplicate␣pattern"); help1("(See␣Appendix␣H.)"); error;
    end;
  trie_o[q] ← v;
  end
```

This code is used in section 1015.

1018. ⟨ Insert a new trie node between q and p, and make p point to it 1018 ⟩ ≡

```
  begin if trie_ptr = trie_size then overflow("pattern␣memory", trie_size);
  incr(trie_ptr); trie_r[trie_ptr] ← p; p ← trie_ptr; trie_l[p] ← 0;
  if first_child then trie_l[q] ← p else trie_r[q] ← p;
  trie_c[p] ← si(c); trie_o[p] ← min_quarterword;
  end
```

This code is used in sections 1017, 1666, and 1667.

1019. ⟨ Compute the trie op code, v, and set $l \leftarrow 0$ 1019 ⟩ ≡

```
  if hc[1] = 0 then hyf[0] ← 0;
  if hc[k] = 0 then hyf[k] ← 0;
  l ← k; v ← min_quarterword;
  loop begin if hyf[l] ≠ 0 then v ← new_trie_op(k − l, hyf[l], v);
    if l > 0 then decr(l) else goto done1;
    end;
done1:
```

This code is used in section 1017.

1020. Finally we put everything together: Here is how the trie gets to its final, efficient form. The following packing routine is rigged so that the root of the linked tree gets mapped into location 1 of *trie*, as required by the hyphenation algorithm. This happens because the first call of *first_fit* will "take" location 1.

```
⟨ Declare procedures for preprocessing hyphenation patterns 998 ⟩ +≡
procedure init_trie;
  var p: trie_pointer; { pointer for initialization }
    j, k, t: integer; { all-purpose registers for initialization }
    r, s: trie_pointer; { used to clean up the packed trie }
    h: two_halves; { template used to zero out trie's holes }
  begin incr(max_hyph_char); ⟨ Get ready to compress the trie 1006 ⟩;
  if trie_root ≠ 0 then
    begin first_fit(trie_root); trie_pack(trie_root);
    end;
  if hyph_root ≠ 0 then ⟨ Pack all stored hyph_codes 1668 ⟩;
  ⟨ Move the data into trie 1012 ⟩;
  trie_not_ready ← false;
  end;
```

1021. Breaking vertical lists into pages. The *vsplit* procedure, which implements TeX's `\vsplit` operation, is considerably simpler than *line_break* because it doesn't have to worry about hyphenation, and because its mission is to discover a single break instead of an optimum sequence of breakpoints. But before we get into the details of *vsplit*, we need to consider a few more basic things.

1022. A subroutine called *prune_page_top* takes a pointer to a vlist and returns a pointer to a modified vlist in which all glue, kern, and penalty nodes have been deleted before the first box or rule node. However, the first box or rule is actually preceded by a newly created glue node designed so that the topmost baseline will be at distance *split_top_skip* from the top, whenever this is possible without backspacing.

When the second argument s is *false* the deleted nodes are destroyed, otherwise they are collected in a list starting at *split_disc*.

In this routine and those that follow, we make use of the fact that a vertical list contains no character nodes, hence the *type* field exists for each node in the list.

```
function prune_page_top(p : pointer; s : boolean): pointer;  { adjust top after page break }
  var prev_p: pointer;  { lags one step behind p }
    q, r: pointer;  { temporary variables for list manipulation }
  begin prev_p ← temp_head; link(temp_head) ← p;
  while p ≠ null do
    case type(p) of
    hlist_node, vlist_node, rule_node: ⟨ Insert glue for split_top_skip and set p ← null 1023 ⟩;
    whatsit_node, mark_node, ins_node: begin prev_p ← p; p ← link(prev_p);
      end;
    glue_node, kern_node, penalty_node: begin q ← p; p ← link(q); link(q) ← null; link(prev_p) ← p;
      if s then
        begin if split_disc = null then split_disc ← q else link(r) ← q;
        r ← q;
        end
      else flush_node_list(q);
      end;
    othercases confusion("pruning")
    endcases;
  prune_page_top ← link(temp_head);
  end;
```

1023. ⟨ Insert glue for *split_top_skip* and set $p \leftarrow$ *null* 1023 ⟩ ≡

```
  begin q ← new_skip_param(split_top_skip_code); link(prev_p) ← q; link(q) ← p;
      { now temp_ptr = glue_ptr(q) }
  if width(temp_ptr) > height(p) then width(temp_ptr) ← width(temp_ptr) − height(p)
  else width(temp_ptr) ← 0;
  p ← null;
  end
```

This code is used in section 1022.

1024. The next subroutine finds the best place to break a given vertical list so as to obtain a box of height h, with maximum depth d. A pointer to the beginning of the vertical list is given, and a pointer to the optimum breakpoint is returned. The list is effectively followed by a forced break, i.e., a penalty node with the *eject_penalty*; if the best break occurs at this artificial node, the value *null* is returned.

An array of six *scaled* distances is used to keep track of the height from the beginning of the list to the current place, just as in *line_break*. In fact, we use one of the same arrays, only changing its name to reflect its new significance.

```
define active_height ≡ active_width  { new name for the six distance variables }
define cur_height ≡ active_height[1]  { the natural height }
define set_height_zero(#) ≡ active_height[#] ← 0  { initialize the height to zero }
define update_heights = 90  { go here to record glue in the active_height table }
function vert_break(p : pointer; h, d : scaled): pointer;  { finds optimum page break }
  label done, not_found, update_heights;
  var prev_p: pointer;  { if p is a glue node, type(prev_p) determines whether p is a legal breakpoint }
    q, r: pointer;  { glue specifications }
    pi: integer;  { penalty value }
    b: integer;  { badness at a trial breakpoint }
    least_cost: integer;  { the smallest badness plus penalties found so far }
    best_place: pointer;  { the most recent break that leads to least_cost }
    prev_dp: scaled;  { depth of previous box in the list }
    t: small_number;  { type of the node following a kern }
  begin prev_p ← p;  { an initial glue node is not a legal breakpoint }
  least_cost ← awful_bad; do_all_six(set_height_zero); prev_dp ← 0;
  loop begin ⟨ If node p is a legal breakpoint, check if this break is the best known, and goto done if p is
        null or if the page-so-far is already too full to accept more stuff 1026 ⟩;
    prev_p ← p; p ← link(prev_p);
    end;
done: vert_break ← best_place;
  end;
```

1025. A global variable *best_height_plus_depth* will be set to the natural size of the box that corresponds to the optimum breakpoint found by *vert_break*. (This value is used by the insertion-splitting algorithm of the page builder.)

```
⟨ Global variables 13 ⟩ +≡
best_height_plus_depth: scaled;  { height of the best box, without stretching or shrinking }
```

1026. A subtle point to be noted here is that the maximum depth d might be negative, so *cur_height* and *prev_dp* might need to be corrected even after a glue or kern node.

```
⟨ If node p is a legal breakpoint, check if this break is the best known, and goto done if p is null or if the
      page-so-far is already too full to accept more stuff 1026 ⟩ ≡
  if p = null then pi ← eject_penalty
  else ⟨ Use node p to update the current height and depth measurements; if this node is not a legal
      breakpoint, goto not_found or update_heights, otherwise set pi to the associated penalty at the
      break 1027 ⟩;
  ⟨ Check if node p is a new champion breakpoint; then goto done if p is a forced break or if the page-so-far
      is already too full 1028 ⟩;
  if (type(p) < glue_node) ∨ (type(p) > kern_node) then goto not_found;
update_heights: ⟨ Update the current height and depth measurements with respect to a glue or kern
      node p 1030 ⟩;
not_found: if prev_dp > d then
    begin cur_height ← cur_height + prev_dp − d; prev_dp ← d;
    end;
```

This code is used in section 1024.

1027.

```
⟨ Use node p to update the current height and depth measurements; if this node is not a legal
      breakpoint, goto not_found or update_heights, otherwise set pi to the associated penalty at the
      break 1027 ⟩ ≡
  case type(p) of
  hlist_node, vlist_node, rule_node: begin
    cur_height ← cur_height + prev_dp + height(p); prev_dp ← depth(p); goto not_found;
    end;
  whatsit_node: ⟨ Process whatsit p in vert_break loop, goto not_found 1425 ⟩;
  glue_node: if precedes_break(prev_p) then pi ← 0
    else goto update_heights;
  kern_node: begin if link(p) = null then t ← penalty_node
    else t ← type(link(p));
    if t = glue_node then pi ← 0 else goto update_heights;
    end;
  penalty_node: pi ← penalty(p);
  mark_node, ins_node: goto not_found;
  othercases confusion("vertbreak")
  endcases
```

This code is used in section 1026.

1028. **define** *deplorable* ≡ 100000 { more than *inf_bad*, but less than *awful_bad* }

⟨ Check if node p is a new champion breakpoint; then **goto** *done* if p is a forced break or if the page-so-far is already too full 1028 ⟩ ≡

```
if pi < inf_penalty then
  begin ⟨ Compute the badness, b, using awful_bad if the box is too full 1029 ⟩;
  if b < awful_bad then
    if pi ≤ eject_penalty then b ← pi
    else if b < inf_bad then b ← b + pi
      else b ← deplorable;
  if b ≤ least_cost then
    begin best_place ← p; least_cost ← b; best_height_plus_depth ← cur_height + prev_dp;
    end;
  if (b = awful_bad) ∨ (pi ≤ eject_penalty) then goto done;
  end
```

This code is used in section 1026.

1029. ⟨ Compute the badness, b, using *awful_bad* if the box is too full 1029 ⟩ ≡

```
if cur_height < h then
  if (active_height[3] ≠ 0) ∨ (active_height[4] ≠ 0) ∨ (active_height[5] ≠ 0) then b ← 0
  else b ← badness(h − cur_height, active_height[2])
else if cur_height − h > active_height[6] then b ← awful_bad
  else b ← badness(cur_height − h, active_height[6])
```

This code is used in section 1028.

1030. Vertical lists that are subject to the *vert_break* procedure should not contain infinite shrinkability, since that would permit any amount of information to "fit" on one page.

⟨ Update the current height and depth measurements with respect to a glue or kern node p 1030 ⟩ ≡

```
if type(p) = kern_node then q ← p
else begin q ← glue_ptr(p);
  active_height[2 + stretch_order(q)] ← active_height[2 + stretch_order(q)] + stretch(q);
  active_height[6] ← active_height[6] + shrink(q);
  if (shrink_order(q) ≠ normal) ∧ (shrink(q) ≠ 0) then
    begin
    print_err("Infinite glue shrinkage found in box being split");
    help4("The box you are \vsplitting contains some infinitely")
    ("shrinkable glue, e.g., `\vss´ or `\vskip 0pt minus 1fil´.")
    ("Such glue doesn´t belong there; but you can safely proceed,")
    ("since the offensive shrinkability has been made finite."); error; r ← new_spec(q);
    shrink_order(r) ← normal; delete_glue_ref(q); glue_ptr(p) ← r; q ← r;
    end;
  end;
cur_height ← cur_height + prev_dp + width(q); prev_dp ← 0
```

This code is used in section 1026.

1031. Now we are ready to consider *vsplit* itself. Most of its work is accomplished by the two subroutines that we have just considered.

Given the number of a vlist box n, and given a desired page height h, the *vsplit* function finds the best initial segment of the vlist and returns a box for a page of height h. The remainder of the vlist, if any, replaces the original box, after removing glue and penalties and adjusting for *split_top_skip*. Mark nodes in the split-off box are used to set the values of *split_first_mark* and *split_bot_mark*; we use the fact that *split_first_mark* = *null* if and only if *split_bot_mark* = *null*.

The original box becomes "void" if and only if it has been entirely extracted. The extracted box is "void" if and only if the original box was void (or if it was, erroneously, an hlist box).

```
⟨ Declare the function called do_marks 1636 ⟩
function vsplit(n : halfword; h : scaled): pointer;  { extracts a page of height h from box n }
  label exit, done;
  var v: pointer;  { the box to be split }
    p: pointer;  { runs through the vlist }
    q: pointer;  { points to where the break occurs }
  begin cur_val ← n; fetch_box(v); flush_node_list(split_disc); split_disc ← null;
  if sa_mark ≠ null then
    if do_marks(vsplit_init, 0, sa_mark) then sa_mark ← null;
  if split_first_mark ≠ null then
    begin delete_token_ref(split_first_mark); split_first_mark ← null; delete_token_ref(split_bot_mark);
    split_bot_mark ← null;
    end;
  ⟨ Dispense with trivial cases of void or bad boxes 1032 ⟩;
  q ← vert_break(list_ptr(v), h, split_max_depth);
  ⟨ Look at all the marks in nodes before the break, and set the final link to null at the break 1033 ⟩;
  q ← prune_page_top(q, saving_vdiscards > 0); p ← list_ptr(v); free_node(v, box_node_size);
  if q ≠ null then q ← vpack(q, natural);
  change_box(q);  { the eq_level of the box stays the same }
  vsplit ← vpackage(p, h, exactly, split_max_depth);
exit: end;
```

1032. ⟨ Dispense with trivial cases of void or bad boxes 1032 ⟩ ≡

```
if v = null then
  begin vsplit ← null; return;
  end;
if type(v) ≠ vlist_node then
  begin print_err(""); print_esc("vsplit"); print("␣needs␣a␣"); print_esc("vbox");
  help2("The␣box␣you␣are␣trying␣to␣split␣is␣an␣\hbox.")
  ("I␣can´t␣split␣such␣a␣box,␣so␣I´ll␣leave␣it␣alone."); error; vsplit ← null; return;
  end
```

This code is used in section 1031.

1033. It's possible that the box begins with a penalty node that is the "best" break, so we must be careful to handle this special case correctly.

```
⟨ Look at all the marks in nodes before the break, and set the final link to null at the break 1033 ⟩ ≡
  p ← list_ptr(v);
  if p = q then list_ptr(v) ← null
  else loop begin if type(p) = mark_node then
      if mark_class(p) ≠ 0 then ⟨ Update the current marks for vsplit 1638 ⟩
      else if split_first_mark = null then
          begin split_first_mark ← mark_ptr(p); split_bot_mark ← split_first_mark;
          token_ref_count(split_first_mark) ← token_ref_count(split_first_mark) + 2;
          end
        else begin delete_token_ref(split_bot_mark); split_bot_mark ← mark_ptr(p);
          add_token_ref(split_bot_mark);
          end;
      if link(p) = q then
        begin link(p) ← null; goto done;
        end;
      p ← link(p);
      end;
done:
```

This code is used in section 1031.

1034. The page builder. When TEX appends new material to its main vlist in vertical mode, it uses a method something like *vsplit* to decide where a page ends, except that the calculations are done "on line" as new items come in. The main complication in this process is that insertions must be put into their boxes and removed from the vlist, in a more-or-less optimum manner.

We shall use the term "current page" for that part of the main vlist that is being considered as a candidate for being broken off and sent to the user's output routine. The current page starts at *link*(*page_head*), and it ends at *page_tail*. We have *page_head* = *page_tail* if this list is empty.

Utter chaos would reign if the user kept changing page specifications while a page is being constructed, so the page builder keeps the pertinent specifications frozen as soon as the page receives its first box or insertion. The global variable *page_contents* is *empty* when the current page contains only mark nodes and content-less whatsit nodes; it is *inserts_only* if the page contains only insertion nodes in addition to marks and whatsits. Glue nodes, kern nodes, and penalty nodes are discarded until a box or rule node appears, at which time *page_contents* changes to *box_there*. As soon as *page_contents* becomes non-*empty*, the current *vsize* and *max_depth* are squirreled away into *page_goal* and *page_max_depth*; the latter values will be used until the page has been forwarded to the user's output routine. The `\topskip` adjustment is made when *page_contents* changes to *box_there*.

Although *page_goal* starts out equal to *vsize*, it is decreased by the scaled natural height-plus-depth of the insertions considered so far, and by the `\skip` corrections for those insertions. Therefore it represents the size into which the non-inserted material should fit, assuming that all insertions in the current page have been made.

The global variables *best_page_break* and *least_page_cost* correspond respectively to the local variables *best_place* and *least_cost* in the *vert_break* routine that we have already studied; i.e., they record the location and value of the best place currently known for breaking the current page. The value of *page_goal* at the time of the best break is stored in *best_size*.

define *inserts_only* = 1 { *page_contents* when an insert node has been contributed, but no boxes }
define *box_there* = 2 { *page_contents* when a box or rule has been contributed }

⟨ Global variables 13 ⟩ +≡
page_tail: *pointer*; { the final node on the current page }
page_contents: *empty* .. *box_there*; { what is on the current page so far? }
page_max_depth: *scaled*; { maximum box depth on page being built }
best_page_break: *pointer*; { break here to get the best page known so far }
least_page_cost: *integer*; { the score for this currently best page }
best_size: *scaled*; { its *page_goal* }

1035. The page builder has another data structure to keep track of insertions. This is a list of four-word nodes, starting and ending at *page_ins_head*. That is, the first element of the list is node $r_1 = link(page_ins_head)$; node r_j is followed by $r_{j+1} = link(r_j)$; and if there are n items we have $r_{n+1} = page_ins_head$. The *subtype* field of each node in this list refers to an insertion number; for example, '`\insert 250`' would correspond to a node whose *subtype* is $qi(250)$ (the same as the *subtype* field of the relevant *ins_node*). These *subtype* fields are in increasing order, and $subtype(page_ins_head) = qi(255)$, so *page_ins_head* serves as a convenient sentinel at the end of the list. A record is present for each insertion number that appears in the current page.

The *type* field in these nodes distinguishes two possibilities that might occur as we look ahead before deciding on the optimum page break. If $type(r) = inserting$, then $height(r)$ contains the total of the height-plus-depth dimensions of the box and all its inserts seen so far. If $type(r) = split_up$, then no more insertions will be made into this box, because at least one previous insertion was too big to fit on the current page; $broken_ptr(r)$ points to the node where that insertion will be split, if TeX decides to split it, $broken_ins(r)$ points to the insertion node that was tentatively split, and $height(r)$ includes also the natural height plus depth of the part that would be split off.

In both cases, $last_ins_ptr(r)$ points to the last *ins_node* encountered for box $qo(subtype(r))$ that would be at least partially inserted on the next page; and $best_ins_ptr(r)$ points to the last such *ins_node* that should actually be inserted, to get the page with minimum badness among all page breaks considered so far. We have $best_ins_ptr(r) = null$ if and only if no insertion for this box should be made to produce this optimum page.

The data structure definitions here use the fact that the *height* field appears in the fourth word of a box node.

define *page_ins_node_size* = 4 { number of words for a page insertion node }
define *inserting* = 0 { an insertion class that has not yet overflowed }
define *split_up* = 1 { an overflowed insertion class }
define *broken_ptr*(#) ≡ *link*(# + 1) { an insertion for this class will break here if anywhere }
define *broken_ins*(#) ≡ *info*(# + 1) { this insertion might break at *broken_ptr* }
define *last_ins_ptr*(#) ≡ *link*(# + 2) { the most recent insertion for this *subtype* }
define *best_ins_ptr*(#) ≡ *info*(# + 2) { the optimum most recent insertion }

⟨ Initialize the special list heads and constant nodes 838 ⟩ +≡
subtype(*page_ins_head*) ← *qi*(255); *type*(*page_ins_head*) ← *split_up*; *link*(*page_ins_head*) ← *page_ins_head*;

1036. An array *page_so_far* records the heights and depths of everything on the current page. This array contains six *scaled* numbers, like the similar arrays already considered in *line_break* and *vert_break*; and it also contains *page_goal* and *page_depth*, since these values are all accessible to the user via *set_page_dimen* commands. The value of *page_so_far*[1] is also called *page_total*. The stretch and shrink components of the `\skip` corrections for each insertion are included in *page_so_far*, but the natural space components of these corrections are not, since they have been subtracted from *page_goal*.

The variable *page_depth* records the depth of the current page; it has been adjusted so that it is at most *page_max_depth*. The variable *last_glue* points to the glue specification of the most recent node contributed from the contribution list, if this was a glue node; otherwise *last_glue* = *max_halfword*. (If the contribution list is nonempty, however, the value of *last_glue* is not necessarily accurate.) The variables *last_penalty*, *last_kern*, and *last_node_type* are similar. And finally, *insert_penalties* holds the sum of the penalties associated with all split and floating insertions.

```
define page_goal ≡ page_so_far[0]  { desired height of information on page being built }
define page_total ≡ page_so_far[1]  { height of the current page }
define page_shrink ≡ page_so_far[6]  { shrinkability of the current page }
define page_depth ≡ page_so_far[7]  { depth of the current page }
```

```
⟨ Global variables 13 ⟩ +≡
page_so_far: array [0 .. 7] of scaled;  { height and glue of the current page }
last_glue: pointer;  { used to implement \lastskip }
last_penalty: integer;  { used to implement \lastpenalty }
last_kern: scaled;  { used to implement \lastkern }
last_node_type: integer;  { used to implement \lastnodetype }
insert_penalties: integer;  { sum of the penalties for insertions that were held over }
```

1037.
```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("pagegoal", set_page_dimen, 0); primitive("pagetotal", set_page_dimen, 1);
primitive("pagestretch", set_page_dimen, 2); primitive("pagefilstretch", set_page_dimen, 3);
primitive("pagefillstretch", set_page_dimen, 4); primitive("pagefilllstretch", set_page_dimen, 5);
primitive("pageshrink", set_page_dimen, 6); primitive("pagedepth", set_page_dimen, 7);
```

1038.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
set_page_dimen: case chr_code of
  0: print_esc("pagegoal");
  1: print_esc("pagetotal");
  2: print_esc("pagestretch");
  3: print_esc("pagefilstretch");
  4: print_esc("pagefillstretch");
  5: print_esc("pagefilllstretch");
  6: print_esc("pageshrink");
  othercases print_esc("pagedepth")
  endcases;
```

1039. **define** *print_plus_end*(#) ≡ *print*(#); **end**
define *print_plus*(#) ≡
if *page_so_far*[#] ≠ 0 **then**
begin *print*("␣plus␣"); *print_scaled*(*page_so_far*[#]); *print_plus_end*

procedure *print_totals*;
begin *print_scaled*(*page_total*); *print_plus*(2)(""); *print_plus*(3)("fil"); *print_plus*(4)("fill");
print_plus(5)("filll");
if *page_shrink* ≠ 0 **then**
begin *print*("␣minus␣"); *print_scaled*(*page_shrink*);
end;
end;

1040. ⟨ Show the status of the current page 1040 ⟩ ≡
if *page_head* ≠ *page_tail* **then**
begin *print_nl*("###␣current␣page:");
if *output_active* **then** *print*("␣(held␣over␣for␣next␣output)");
show_box(*link*(*page_head*));
if *page_contents* > *empty* **then**
begin *print_nl*("total␣height␣"); *print_totals*; *print_nl*("␣goal␣height␣");
print_scaled(*page_goal*); *r* ← *link*(*page_ins_head*);
while *r* ≠ *page_ins_head* **do**
begin *print_ln*; *print_esc*("insert"); *t* ← *qo*(*subtype*(*r*)); *print_int*(*t*); *print*("␣adds␣");
if *count*(*t*) = 1000 **then** *t* ← *height*(*r*)
else *t* ← *x_over_n*(*height*(*r*), 1000) ∗ *count*(*t*);
print_scaled(*t*);
if *type*(*r*) = *split_up* **then**
begin *q* ← *page_head*; *t* ← 0;
repeat *q* ← *link*(*q*);
if (*type*(*q*) = *ins_node*) ∧ (*subtype*(*q*) = *subtype*(*r*)) **then** *incr*(*t*);
until *q* = *broken_ins*(*r*);
print(",␣#"); *print_int*(*t*); *print*("␣might␣split");
end;
r ← *link*(*r*);
end;
end;
end

This code is used in section 244.

1041. Here is a procedure that is called when the *page_contents* is changing from *empty* to *inserts_only* or *box_there*.

define *set_page_so_far_zero*(#) ≡ *page_so_far*[#] ← 0

procedure *freeze_page_specs*(*s* : *small_number*);
begin *page_contents* ← *s*; *page_goal* ← *vsize*; *page_max_depth* ← *max_depth*; *page_depth* ← 0;
do_all_six(*set_page_so_far_zero*); *least_page_cost* ← *awful_bad*;
stat if *tracing_pages* > 0 **then**
begin *begin_diagnostic*; *print_nl*("%%␣goal␣height="); *print_scaled*(*page_goal*);
print(",␣max␣depth="); *print_scaled*(*page_max_depth*); *end_diagnostic*(*false*);
end; **tats**
end;

1042. Pages are built by appending nodes to the current list in TeX's vertical mode, which is at the outermost level of the semantic nest. This vlist is split into two parts; the "current page" that we have been talking so much about already, and the "contribution list" that receives new nodes as they are created. The current page contains everything that the page builder has accounted for in its data structures, as described above, while the contribution list contains other things that have been generated by other parts of TeX but have not yet been seen by the page builder. The contribution list starts at *link*(*contrib_head*), and it ends at the current node in TeX's vertical mode.

When TeX has appended new material in vertical mode, it calls the procedure *build_page*, which tries to catch up by moving nodes from the contribution list to the current page. This procedure will succeed in its goal of emptying the contribution list, unless a page break is discovered, i.e., unless the current page has grown to the point where the optimum next page break has been determined. In the latter case, the nodes after the optimum break will go back onto the contribution list, and control will effectively pass to the user's output routine.

We make *type*(*page_head*) = *glue_node*, so that an initial glue node on the current page will not be considered a valid breakpoint.

⟨ Initialize the special list heads and constant nodes 838 ⟩ +≡
type(*page_head*) ← *glue_node*; *subtype*(*page_head*) ← *normal*;

1043. The global variable *output_active* is true during the time the user's output routine is driving TeX.

⟨ Global variables 13 ⟩ +≡
output_active: *boolean*; { are we in the midst of an output routine? }

1044. ⟨ Set initial values of key variables 23 ⟩ +≡
output_active ← *false*; *insert_penalties* ← 0;

1045. The page builder is ready to start a fresh page if we initialize the following state variables. (However, the page insertion list is initialized elsewhere.)

⟨ Start a new current page 1045 ⟩ ≡
page_contents ← *empty*; *page_tail* ← *page_head*; *link*(*page_head*) ← *null*;
last_glue ← *max_halfword*; *last_penalty* ← 0; *last_kern* ← 0; *last_node_type* ← −1; *page_depth* ← 0;
page_max_depth ← 0

This code is used in sections 241 and 1071.

1046. At certain times box 255 is supposed to be void (i.e., *null*), or an insertion box is supposed to be ready to accept a vertical list. If not, an error message is printed, and the following subroutine flushes the unwanted contents, reporting them to the user.

procedure *box_error*(*n* : *eight_bits*);
begin *error*; *begin_diagnostic*; *print_nl*("The␣following␣box␣has␣been␣deleted:");
show_box(*box*(*n*)); *end_diagnostic*(*true*); *flush_node_list*(*box*(*n*)); *box*(*n*) ← *null*;
end;

1047. The following procedure guarantees that a given box register does not contain an `\hbox`.

```
procedure ensure_vbox(n : eight_bits);
  var p: pointer; { the box register contents }
  begin p ← box(n);
  if p ≠ null then
    if type(p) = hlist_node then
      begin print_err("Insertions␣can␣only␣be␣added␣to␣a␣vbox");
      help3("Tut␣tut:␣You´re␣trying␣to␣\insert␣into␣a")
      ("\box␣register␣that␣now␣contains␣an␣\hbox.")
      ("Proceed,␣and␣I´ll␣discard␣its␣present␣contents."); box_error(n);
      end;
  end;
```

1048. TeX is not always in vertical mode at the time *build_page* is called; the current mode reflects what TeX should return to, after the contribution list has been emptied. A call on *build_page* should be immediately followed by '**goto** *big_switch*', which is TeX's central control point.

```
  define contribute = 80 { go here to link a node into the current page }
⟨ Declare the procedure called fire_up 1066 ⟩
procedure build_page; { append contributions to the current page }
  label exit, done, done1, continue, contribute, update_heights;
  var p: pointer; { the node being appended }
    q, r: pointer; { nodes being examined }
    b, c: integer; { badness and cost of current page }
    pi: integer; { penalty to be added to the badness }
    n: min_quarterword .. biggest_reg; { insertion box number }
    delta, h, w: scaled; { sizes used for insertion calculations }
  begin if (link(contrib_head) = null) ∨ output_active then return;
  repeat continue: p ← link(contrib_head);
    ⟨ Update the values of last_glue, last_penalty, and last_kern 1050 ⟩;
    ⟨ Move node p to the current page; if it is time for a page break, put the nodes following the break
        back onto the contribution list, and return to the user's output routine if there is one 1051 ⟩;
  until link(contrib_head) = null;
  ⟨ Make the contribution list empty by setting its tail to contrib_head 1049 ⟩;
exit: end;
```

1049. `define contrib_tail ≡ nest[0].tail_field { tail of the contribution list }`

```
⟨ Make the contribution list empty by setting its tail to contrib_head 1049 ⟩ ≡
  if nest_ptr = 0 then tail ← contrib_head { vertical mode }
  else contrib_tail ← contrib_head { other modes }
```

This code is used in section 1048.

1050. ⟨ Update the values of *last_glue*, *last_penalty*, and *last_kern* 1050 ⟩ ≡

```
if last_glue ≠ max_halfword then delete_glue_ref(last_glue);
last_penalty ← 0; last_kern ← 0; last_node_type ← type(p) + 1;
if type(p) = glue_node then
  begin last_glue ← glue_ptr(p); add_glue_ref(last_glue);
  end
else begin last_glue ← max_halfword;
  if type(p) = penalty_node then last_penalty ← penalty(p)
  else if type(p) = kern_node then last_kern ← width(p);
  end
```

This code is used in section 1048.

1051. The code here is an example of a many-way switch into routines that merge together in different places. Some people call this unstructured programming, but the author doesn't see much wrong with it, as long as the various labels have a well-understood meaning.

⟨ Move node *p* to the current page; if it is time for a page break, put the nodes following the break back onto the contribution list, and **return** to the user's output routine if there is one 1051 ⟩ ≡

```
⟨ If the current page is empty and node p is to be deleted, goto done1; otherwise use node p to update
    the state of the current page; if this node is an insertion, goto contribute; otherwise if this node is
    not a legal breakpoint, goto contribute or update_heights; otherwise set pi to the penalty associated
    with this breakpoint 1054 ⟩;
⟨ Check if node p is a new champion breakpoint; then if it is time for a page break, prepare for output,
    and either fire up the user's output routine and return or ship out the page and goto done 1059 ⟩;
if (type(p) < glue_node) ∨ (type(p) > kern_node) then goto contribute;
update_heights: ⟨ Update the current page measurements with respect to the glue or kern specified by
    node p 1058 ⟩;
contribute: ⟨ Make sure that page_max_depth is not exceeded 1057 ⟩;
⟨ Link node p into the current page and goto done 1052 ⟩;
done1: ⟨ Recycle node p 1053 ⟩;
done:
```

This code is used in section 1048.

1052. ⟨ Link node *p* into the current page and **goto** *done* 1052 ⟩ ≡

```
link(page_tail) ← p; page_tail ← p; link(contrib_head) ← link(p); link(p) ← null; goto done
```

This code is used in section 1051.

1053. ⟨ Recycle node *p* 1053 ⟩ ≡

```
link(contrib_head) ← link(p); link(p) ← null;
if saving_vdiscards > 0 then
  begin if page_disc = null then page_disc ← p else link(tail_page_disc) ← p;
  tail_page_disc ← p;
  end
else flush_node_list(p)
```

This code is used in section 1051.

1054. The title of this section is already so long, it seems best to avoid making it more accurate but still longer, by mentioning the fact that a kern node at the end of the contribution list will not be contributed until we know its successor.

⟨ If the current page is empty and node p is to be deleted, **goto** *done1*; otherwise use node p to update the state of the current page; if this node is an insertion, **goto** *contribute*; otherwise if this node is not a legal breakpoint, **goto** *contribute* or *update_heights*; otherwise set *pi* to the penalty associated with this breakpoint 1054 ⟩ ≡

```
case type(p) of
hlist_node, vlist_node, rule_node: if page_contents < box_there then
    ⟨ Initialize the current page, insert the \topskip glue ahead of p, and goto continue 1055 ⟩
  else ⟨ Prepare to move a box or rule node to the current page, then goto contribute 1056 ⟩;
whatsit_node: ⟨ Prepare to move whatsit p to the current page, then goto contribute 1424 ⟩;
glue_node: if page_contents < box_there then goto done1
  else if precedes_break(page_tail) then pi ← 0
    else goto update_heights;
kern_node: if page_contents < box_there then goto done1
  else if link(p) = null then return
    else if type(link(p)) = glue_node then pi ← 0
      else goto update_heights;
penalty_node: if page_contents < box_there then goto done1 else pi ← penalty(p);
mark_node: goto contribute;
ins_node: ⟨ Append an insertion to the current page and goto contribute 1062 ⟩;
othercases confusion("page")
endcases
```

This code is used in section 1051.

1055. ⟨ Initialize the current page, insert the `\topskip` glue ahead of p, and **goto** *continue* 1055 ⟩ ≡

```
begin if page_contents = empty then freeze_page_specs(box_there)
else page_contents ← box_there;
q ← new_skip_param(top_skip_code);  { now temp_ptr = glue_ptr(q) }
if width(temp_ptr) > height(p) then width(temp_ptr) ← width(temp_ptr) − height(p)
else width(temp_ptr) ← 0;
link(q) ← p; link(contrib_head) ← q; goto continue;
end
```

This code is used in section 1054.

1056. ⟨ Prepare to move a box or rule node to the current page, then **goto** *contribute* 1056 ⟩ ≡

```
begin page_total ← page_total + page_depth + height(p); page_depth ← depth(p); goto contribute;
end
```

This code is used in section 1054.

1057. ⟨ Make sure that *page_max_depth* is not exceeded 1057 ⟩ ≡

```
if page_depth > page_max_depth then
  begin page_total ← page_total + page_depth − page_max_depth;
  page_depth ← page_max_depth;
  end;
```

This code is used in section 1051.

1058. ⟨ Update the current page measurements with respect to the glue or kern specified by node p 1058 ⟩ ≡

```
if type(p) = kern_node then q ← p
else begin q ← glue_ptr(p);
  page_so_far[2 + stretch_order(q)] ← page_so_far[2 + stretch_order(q)] + stretch(q);
  page_shrink ← page_shrink + shrink(q);
  if (shrink_order(q) ≠ normal) ∧ (shrink(q) ≠ 0) then
    begin
    print_err("Infinite␣glue␣shrinkage␣found␣on␣current␣page");
    help4("The␣page␣about␣to␣be␣output␣contains␣some␣infinitely")
    ("shrinkable␣glue,␣e.g.,␣`\vss´␣or␣`\vskip␣0pt␣minus␣1fil´.")
    ("Such␣glue␣doesn´t␣belong␣there;␣but␣you␣can␣safely␣proceed,")
    ("since␣the␣offensive␣shrinkability␣has␣been␣made␣finite."); error; r ← new_spec(q);
    shrink_order(r) ← normal; delete_glue_ref(q); glue_ptr(p) ← r; q ← r;
    end;
  end;
page_total ← page_total + page_depth + width(q); page_depth ← 0
```

This code is used in section 1051.

1059. ⟨ Check if node p is a new champion breakpoint; then if it is time for a page break, prepare for output, and either fire up the user's output routine and **return** or ship out the page and **goto** *done* 1059 ⟩ ≡

```
if pi < inf_penalty then
  begin ⟨ Compute the badness, b, of the current page, using awful_bad if the box is too full 1061 ⟩;
  if b < awful_bad then
    if pi ≤ eject_penalty then c ← pi
    else if b < inf_bad then c ← b + pi + insert_penalties
      else c ← deplorable
  else c ← b;
  if insert_penalties ≥ 10000 then c ← awful_bad;
  stat if tracing_pages > 0 then ⟨ Display the page break cost 1060 ⟩;
  tats
  if c ≤ least_page_cost then
    begin best_page_break ← p; best_size ← page_goal; least_page_cost ← c; r ← link(page_ins_head);
    while r ≠ page_ins_head do
      begin best_ins_ptr(r) ← last_ins_ptr(r); r ← link(r);
      end;
    end;
  if (c = awful_bad) ∨ (pi ≤ eject_penalty) then
    begin fire_up(p); { output the current page at the best place }
    if output_active then return; { user's output routine will act }
    goto done; { the page has been shipped out by default output routine }
    end;
  end
```

This code is used in section 1051.

1060. ⟨ Display the page break cost 1060 ⟩ ≡
```
begin begin_diagnostic; print_nl("%"); print("␣t="); print_totals;
print("␣g="); print_scaled(page_goal);
print("␣b=");
if b = awful_bad then print_char("*") else print_int(b);
print("␣p="); print_int(pi); print("␣c=");
if c = awful_bad then print_char("*") else print_int(c);
if c ≤ least_page_cost then print_char("#");
end_diagnostic(false);
end
```
This code is used in section 1059.

1061. ⟨ Compute the badness, b, of the current page, using *awful_bad* if the box is too full 1061 ⟩ ≡
```
if page_total < page_goal then
  if (page_so_far[3] ≠ 0) ∨ (page_so_far[4] ≠ 0) ∨ (page_so_far[5] ≠ 0) then b ← 0
  else b ← badness(page_goal − page_total, page_so_far[2])
else if page_total − page_goal > page_shrink then b ← awful_bad
  else b ← badness(page_total − page_goal, page_shrink)
```
This code is used in section 1059.

1062. ⟨ Append an insertion to the current page and **goto** *contribute* 1062 ⟩ ≡
```
begin if page_contents = empty then freeze_page_specs(inserts_only);
n ← subtype(p); r ← page_ins_head;
while n ≥ subtype(link(r)) do r ← link(r);
n ← qo(n);
if subtype(r) ≠ qi(n) then ⟨ Create a page insertion node with subtype(r) = qi(n), and include the glue
      correction for box n in the current page state 1063 ⟩;
if type(r) = split_up then insert_penalties ← insert_penalties + float_cost(p)
else begin last_ins_ptr(r) ← p; delta ← page_goal − page_total − page_depth + page_shrink;
      { this much room is left if we shrink the maximum }
  if count(n) = 1000 then h ← height(p)
  else h ← x_over_n(height(p), 1000) ∗ count(n);   { this much room is needed }
  if ((h ≤ 0) ∨ (h ≤ delta)) ∧ (height(p) + height(r) ≤ dimen(n)) then
    begin page_goal ← page_goal − h; height(r) ← height(r) + height(p);
    end
  else ⟨ Find the best way to split the insertion, and change type(r) to split_up 1064 ⟩;
  end;
goto contribute;
end
```
This code is used in section 1054.

1063. We take note of the value of `\skip` n and the height plus depth of `\box` n only when the first `\insert` n node is encountered for a new page. A user who changes the contents of `\box` n after that first `\insert` n had better be either extremely careful or extremely lucky, or both.

⟨ Create a page insertion node with $subtype(r) = qi(n)$, and include the glue correction for box n in the current page state 1063 ⟩ ≡

```
begin q ← get_node(page_ins_node_size); link(q) ← link(r); link(r) ← q; r ← q; subtype(r) ← qi(n);
type(r) ← inserting; ensure_vbox(n);
if box(n) = null then height(r) ← 0
else height(r) ← height(box(n)) + depth(box(n));
best_ins_ptr(r) ← null;
q ← skip(n);
if count(n) = 1000 then h ← height(r)
else h ← x_over_n(height(r), 1000) * count(n);
page_goal ← page_goal − h − width(q);
page_so_far[2 + stretch_order(q)] ← page_so_far[2 + stretch_order(q)] + stretch(q);
page_shrink ← page_shrink + shrink(q);
if (shrink_order(q) ≠ normal) ∧ (shrink(q) ≠ 0) then
  begin print_err("Infinite␣glue␣shrinkage␣inserted␣from␣"); print_esc("skip"); print_int(n);
  help3("The␣correction␣glue␣for␣page␣breaking␣with␣insertions")
  ("must␣have␣finite␣shrinkability.␣But␣you␣may␣proceed,")
  ("since␣the␣offensive␣shrinkability␣has␣been␣made␣finite."); error;
  end;
end
```

This code is used in section 1062.

1064. Here is the code that will split a long footnote between pages, in an emergency. The current situation deserves to be recapitulated: Node p is an insertion into box n; the insertion will not fit, in its entirety, either because it would make the total contents of box n greater than `\dimen` n, or because it would make the incremental amount of growth h greater than the available space $delta$, or both. (This amount h has been weighted by the insertion scaling factor, i.e., by `\count` n over 1000.) Now we will choose the best way to break the vlist of the insertion, using the same criteria as in the `\vsplit` operation.

⟨ Find the best way to split the insertion, and change $type(r)$ to $split_up$ 1064 ⟩ ≡

```
begin if count(n) ≤ 0 then w ← max_dimen
else begin w ← page_goal − page_total − page_depth;
  if count(n) ≠ 1000 then w ← x_over_n(w, count(n)) * 1000;
  end;
if w > dimen(n) − height(r) then w ← dimen(n) − height(r);
q ← vert_break(ins_ptr(p), w, depth(p)); height(r) ← height(r) + best_height_plus_depth;
stat if tracing_pages > 0 then ⟨ Display the insertion split cost 1065 ⟩;
tats
if count(n) ≠ 1000 then best_height_plus_depth ← x_over_n(best_height_plus_depth, 1000) * count(n);
page_goal ← page_goal − best_height_plus_depth; type(r) ← split_up; broken_ptr(r) ← q;
broken_ins(r) ← p;
if q = null then insert_penalties ← insert_penalties + eject_penalty
else if type(q) = penalty_node then insert_penalties ← insert_penalties + penalty(q);
end
```

This code is used in section 1062.

1065. ⟨ Display the insertion split cost 1065 ⟩ ≡
```
begin begin_diagnostic; print_nl("% split"); print_int(n); print(" to "); print_scaled(w);
print_char(","); print_scaled(best_height_plus_depth);
print(" p=");
if q = null then print_int(eject_penalty)
else if type(q) = penalty_node then print_int(penalty(q))
  else print_char("0");
end_diagnostic(false);
end
```
This code is used in section 1064.

1066. When the page builder has looked at as much material as could appear before the next page break, it makes its decision. The break that gave minimum badness will be used to put a completed "page" into box 255, with insertions appended to their other boxes.

We also set the values of *top_mark*, *first_mark*, and *bot_mark*. The program uses the fact that *bot_mark* ≠ *null* implies *first_mark* ≠ *null*; it also knows that *bot_mark* = *null* implies *top_mark* = *first_mark* = *null*.

The *fire_up* subroutine prepares to output the current page at the best place; then it fires up the user's output routine, if there is one, or it simply ships out the page. There is one parameter, *c*, which represents the node that was being contributed to the page when the decision to force an output was made.

⟨ Declare the procedure called *fire_up* 1066 ⟩ ≡
```
procedure fire_up(c : pointer);
  label exit;
  var p, q, r, s: pointer;   { nodes being examined and/or changed }
    prev_p: pointer;   { predecessor of p }
    n: min_quarterword .. biggest_reg;   { insertion box number }
    wait: boolean;   { should the present insertion be held over? }
    save_vbadness: integer;   { saved value of vbadness }
    save_vfuzz: scaled;   { saved value of vfuzz }
    save_split_top_skip: pointer;   { saved value of split_top_skip }
  begin ⟨ Set the value of output_penalty 1067 ⟩;
  if sa_mark ≠ null then
    if do_marks(fire_up_init, 0, sa_mark) then sa_mark ← null;
  if bot_mark ≠ null then
    begin if top_mark ≠ null then delete_token_ref(top_mark);
    top_mark ← bot_mark; add_token_ref(top_mark); delete_token_ref(first_mark); first_mark ← null;
    end;
  ⟨ Put the optimal current page into box 255, update first_mark and bot_mark, append insertions to their
      boxes, and put the remaining nodes back on the contribution list 1068 ⟩;
  if sa_mark ≠ null then
    if do_marks(fire_up_done, 0, sa_mark) then sa_mark ← null;
  if (top_mark ≠ null) ∧ (first_mark = null) then
    begin first_mark ← top_mark; add_token_ref(top_mark);
    end;
  if output_routine ≠ null then
    if dead_cycles ≥ max_dead_cycles then
      ⟨ Explain that too many dead cycles have occurred in a row 1078 ⟩
    else ⟨ Fire up the user's output routine and return 1079 ⟩;
  ⟨ Perform the default output routine 1077 ⟩;
exit: end;
```
This code is used in section 1048.

1067. ⟨ Set the value of *output_penalty* 1067 ⟩ ≡
```
if type(best_page_break) = penalty_node then
  begin geq_word_define(int_base + output_penalty_code, penalty(best_page_break));
  penalty(best_page_break) ← inf_penalty;
  end
else geq_word_define(int_base + output_penalty_code, inf_penalty)
```
This code is used in section 1066.

1068. As the page is finally being prepared for output, pointer p runs through the vlist, with *prev_p* trailing behind; pointer q is the tail of a list of insertions that are being held over for a subsequent page.

⟨ Put the optimal current page into box 255, update *first_mark* and *bot_mark*, append insertions to their boxes, and put the remaining nodes back on the contribution list 1068 ⟩ ≡
```
if c = best_page_break then best_page_break ← null;  { c not yet linked in }
⟨ Ensure that box 255 is empty before output 1069 ⟩;
insert_penalties ← 0;  { this will count the number of insertions held over }
save_split_top_skip ← split_top_skip;
if holding_inserts ≤ 0 then ⟨ Prepare all the boxes involved in insertions to act as queues 1072 ⟩;
q ← hold_head; link(q) ← null; prev_p ← page_head; p ← link(prev_p);
while p ≠ best_page_break do
  begin if type(p) = ins_node then
    begin if holding_inserts ≤ 0 then ⟨ Either insert the material specified by node p into the
        appropriate box, or hold it for the next page; also delete node p from the current page 1074 ⟩;
    end
  else if type(p) = mark_node then
      if mark_class(p) ≠ 0 then ⟨ Update the current marks for fire_up 1641 ⟩
      else ⟨ Update the values of first_mark and bot_mark 1070 ⟩;
  prev_p ← p; p ← link(prev_p);
  end;
split_top_skip ← save_split_top_skip; ⟨ Break the current page at node p, put it in box 255, and put the
    remaining nodes on the contribution list 1071 ⟩;
⟨ Delete the page-insertion nodes 1073 ⟩
```
This code is used in section 1066.

1069. ⟨ Ensure that box 255 is empty before output 1069 ⟩ ≡
```
if box(255) ≠ null then
  begin print_err("");  print_esc("box");  print("255␣is␣not␣void");
  help2("You␣shouldn't␣use␣\box255␣except␣in␣\output␣routines.")
  ("Proceed,␣and␣I'll␣discard␣its␣present␣contents.");  box_error(255);
  end
```
This code is used in section 1068.

1070. ⟨ Update the values of *first_mark* and *bot_mark* 1070 ⟩ ≡
```
begin if first_mark = null then
  begin first_mark ← mark_ptr(p);  add_token_ref(first_mark);
  end;
if bot_mark ≠ null then delete_token_ref(bot_mark);
bot_mark ← mark_ptr(p);  add_token_ref(bot_mark);
end
```
This code is used in section 1068.

1071. When the following code is executed, the current page runs from node *link*(*page_head*) to node *prev_p*, and the nodes from *p* to *page_tail* are to be placed back at the front of the contribution list. Furthermore the heldover insertions appear in a list from *link*(*hold_head*) to *q*; we will put them into the current page list for safekeeping while the user's output routine is active. We might have $q = hold_head$; and $p = null$ if and only if $prev_p = page_tail$. Error messages are suppressed within *vpackage*, since the box might appear to be overfull or underfull simply because the stretch and shrink from the `\skip` registers for inserts are not actually present in the box.

```
⟨ Break the current page at node p, put it in box 255, and put the remaining nodes on the contribution
      list 1071 ⟩ ≡
  if p ≠ null then
    begin if link(contrib_head) = null then
      if nest_ptr = 0 then tail ← page_tail
      else contrib_tail ← page_tail;
    link(page_tail) ← link(contrib_head); link(contrib_head) ← p; link(prev_p) ← null;
    end;
  save_vbadness ← vbadness; vbadness ← inf_bad; save_vfuzz ← vfuzz; vfuzz ← max_dimen;
      { inhibit error messages }
  box(255) ← vpackage(link(page_head), best_size, exactly, page_max_depth); vbadness ← save_vbadness;
  vfuzz ← save_vfuzz;
  if last_glue ≠ max_halfword then delete_glue_ref(last_glue);
  ⟨ Start a new current page 1045 ⟩; { this sets last_glue ← max_halfword }
  if q ≠ hold_head then
    begin link(page_head) ← link(hold_head); page_tail ← q;
    end
```

This code is used in section 1068.

1072. If many insertions are supposed to go into the same box, we want to know the position of the last node in that box, so that we don't need to waste time when linking further information into it. The *last_ins_ptr* fields of the page insertion nodes are therefore used for this purpose during the packaging phase.

```
⟨ Prepare all the boxes involved in insertions to act as queues 1072 ⟩ ≡
  begin r ← link(page_ins_head);
  while r ≠ page_ins_head do
    begin if best_ins_ptr(r) ≠ null then
      begin n ← qo(subtype(r)); ensure_vbox(n);
      if box(n) = null then box(n) ← new_null_box;
      p ← box(n) + list_offset;
      while link(p) ≠ null do p ← link(p);
      last_ins_ptr(r) ← p;
      end;
    r ← link(r);
    end;
  end
```

This code is used in section 1068.

1073. ⟨ Delete the page-insertion nodes 1073 ⟩ ≡

```
  r ← link(page_ins_head);
  while r ≠ page_ins_head do
    begin q ← link(r); free_node(r, page_ins_node_size); r ← q;
    end;
  link(page_ins_head) ← page_ins_head
```

This code is used in section 1068.

1074. We will set *best_ins_ptr* ← *null* and package the box corresponding to insertion node r, just after making the final insertion into that box. If this final insertion is '*split_up*', the remainder after splitting and pruning (if any) will be carried over to the next page.

```
⟨ Either insert the material specified by node p into the appropriate box, or hold it for the next page; also
      delete node p from the current page 1074 ⟩ ≡
  begin r ← link(page_ins_head);
  while subtype(r) ≠ subtype(p) do r ← link(r);
  if best_ins_ptr(r) = null then wait ← true
  else begin wait ← false; s ← last_ins_ptr(r); link(s) ← ins_ptr(p);
    if best_ins_ptr(r) = p then ⟨ Wrap up the box specified by node r, splitting node p if called for; set
          wait ← true if node p holds a remainder after splitting 1075 ⟩
    else begin while link(s) ≠ null do s ← link(s);
      last_ins_ptr(r) ← s;
      end;
    end;
  ⟨ Either append the insertion node p after node q, and remove it from the current page, or delete
      node(p) 1076 ⟩;
  end
```

This code is used in section 1068.

1075.

```
⟨ Wrap up the box specified by node r, splitting node p if called for; set wait ← true if node p
      holds a remainder after splitting 1075 ⟩ ≡
  begin if type(r) = split_up then
    if (broken_ins(r) = p) ∧ (broken_ptr(r) ≠ null) then
      begin while link(s) ≠ broken_ptr(r) do s ← link(s);
      link(s) ← null; split_top_skip ← split_top_ptr(p); ins_ptr(p) ← prune_page_top(broken_ptr(r), false);
      if ins_ptr(p) ≠ null then
        begin temp_ptr ← vpack(ins_ptr(p), natural); height(p) ← height(temp_ptr) + depth(temp_ptr);
        free_node(temp_ptr, box_node_size); wait ← true;
        end;
      end;
  best_ins_ptr(r) ← null; n ← qo(subtype(r)); temp_ptr ← list_ptr(box(n));
  free_node(box(n), box_node_size); box(n) ← vpack(temp_ptr, natural);
  end
```

This code is used in section 1074.

1076.

```
⟨ Either append the insertion node p after node q, and remove it from the current page, or delete
      node(p) 1076 ⟩ ≡
  link(prev_p) ← link(p); link(p) ← null;
  if wait then
    begin link(q) ← p; q ← p; incr(insert_penalties);
    end
  else begin delete_glue_ref(split_top_ptr(p)); free_node(p, ins_node_size);
    end;
  p ← prev_p
```

This code is used in section 1074.

1077. The list of heldover insertions, running from *link*(*page_head*) to *page_tail*, must be moved to the contribution list when the user has specified no output routine.

```
⟨ Perform the default output routine 1077 ⟩ ≡
  begin if link(page_head) ≠ null then
    begin if link(contrib_head) = null then
      if nest_ptr = 0 then tail ← page_tail else contrib_tail ← page_tail
    else link(page_tail) ← link(contrib_head);
    link(contrib_head) ← link(page_head); link(page_head) ← null; page_tail ← page_head;
    end;
  flush_node_list(page_disc); page_disc ← null; ship_out(box(255)); box(255) ← null;
  end
```

This code is used in section 1066.

1078.

```
⟨ Explain that too many dead cycles have occurred in a row 1078 ⟩ ≡
  begin print_err("Output␣loop---"); print_int(dead_cycles); print("␣consecutive␣dead␣cycles");
  help3("I´ve␣concluded␣that␣your␣\output␣is␣awry;␣it␣never␣does␣a")
  ("\shipout,␣so␣I´m␣shipping␣\box255␣out␣myself.␣Next␣time")
  ("increase␣\maxdeadcycles␣if␣you␣want␣me␣to␣be␣more␣patient!"); error;
  end
```

This code is used in section 1066.

1079.

```
⟨ Fire up the user's output routine and return 1079 ⟩ ≡
  begin output_active ← true; incr(dead_cycles); push_nest; mode ← −vmode;
  prev_depth ← ignore_depth; mode_line ← −line; begin_token_list(output_routine, output_text);
  new_save_level(output_group); normal_paragraph; scan_left_brace; return;
  end
```

This code is used in section 1066.

1080. When the user's output routine finishes, it has constructed a vlist in internal vertical mode, and TeX will do the following:

```
⟨ Resume the page builder after an output routine has come to an end 1080 ⟩ ≡
  begin if (loc ≠ null) ∨ ((token_type ≠ output_text) ∧ (token_type ≠ backed_up)) then
    ⟨ Recover from an unbalanced output routine 1081 ⟩;
  end_token_list; { conserve stack space in case more outputs are triggered }
  end_graf; unsave; output_active ← false; insert_penalties ← 0;
  ⟨ Ensure that box 255 is empty after output 1082 ⟩;
  if tail ≠ head then { current list goes after heldover insertions }
    begin link(page_tail) ← link(head); page_tail ← tail;
    end;
  if link(page_head) ≠ null then { and both go before heldover contributions }
    begin if link(contrib_head) = null then contrib_tail ← page_tail;
    link(page_tail) ← link(contrib_head); link(contrib_head) ← link(page_head); link(page_head) ← null;
    page_tail ← page_head;
    end;
  flush_node_list(page_disc); page_disc ← null; pop_nest; build_page;
  end
```

This code is used in section 1154.

1081. ⟨ Recover from an unbalanced output routine 1081 ⟩ ≡

```
begin print_err("Unbalanced␣output␣routine");
help2("Your␣sneaky␣output␣routine␣has␣problematic␣{´s␣and/or␣}´s.")
("I␣can´t␣handle␣that␣very␣well;␣good␣luck."); error;
repeat get_token;
until loc = null;
end { loops forever if reading from a file, since null = min_halfword ≤ 0 }
```

This code is used in section 1080.

1082. ⟨ Ensure that box 255 is empty after output 1082 ⟩ ≡

```
if box(255) ≠ null then
  begin print_err("Output␣routine␣didn´t␣use␣all␣of␣"); print_esc("box"); print_int(255);
  help3("Your␣\output␣commands␣should␣empty␣\box255,")
  ("e.g.,␣by␣saying␣`\shipout\box255´.")
  ("Proceed;␣I´ll␣discard␣its␣present␣contents."); box_error(255);
  end
```

This code is used in section 1080.

1083. The chief executive. We come now to the *main_control* routine, which contains the master switch that causes all the various pieces of TeX to do their things, in the right order.

In a sense, this is the grand climax of the program: It applies all the tools that we have worked so hard to construct. In another sense, this is the messiest part of the program: It necessarily refers to other pieces of code all over the place, so that a person can't fully understand what is going on without paging back and forth to be reminded of conventions that are defined elsewhere. We are now at the hub of the web, the central nervous system that touches most of the other parts and ties them together.

The structure of *main_control* itself is quite simple. There's a label called *big_switch*, at which point the next token of input is fetched using *get_x_token*. Then the program branches at high speed into one of about 100 possible directions, based on the value of the current mode and the newly fetched command code; the sum $abs(mode) + cur_cmd$ indicates what to do next. For example, the case '*vmode* + *letter*' arises when a letter occurs in vertical mode (or internal vertical mode); this case leads to instructions that initialize a new paragraph and enter horizontal mode.

The big **case** statement that contains this multiway switch has been labeled *reswitch*, so that the program can **goto** *reswitch* when the next token has already been fetched. Most of the cases are quite short; they call an "action procedure" that does the work for that case, and then they either **goto** *reswitch* or they "fall through" to the end of the **case** statement, which returns control back to *big_switch*. Thus, *main_control* is not an extremely large procedure, in spite of the multiplicity of things it must do; it is small enough to be handled by Pascal compilers that put severe restrictions on procedure size.

One case is singled out for special treatment, because it accounts for most of TeX's activities in typical applications. The process of reading simple text and converting it into *char_node* records, while looking for ligatures and kerns, is part of TeX's "inner loop"; the whole program runs efficiently when its inner loop is fast, so this part has been written with particular care.

1084. We shall concentrate first on the inner loop of *main_control*, deferring consideration of the other cases until later.

```
define big_switch = 60 { go here to branch on the next token of input }
define main_loop = 70 { go here to typeset a string of consecutive characters }
define collect_native = 71 { go here to collect characters in a "native" font string }
define collected = 72
define main_loop_wrapup = 80 { go here to finish a character or ligature }
define main_loop_move = 90 { go here to advance the ligature cursor }
define main_loop_move_lig = 95 { same, when advancing past a generated ligature }
define main_loop_lookahead = 100 { go here to bring in another character, if any }
define main_lig_loop = 110 { go here to check for ligatures or kerning }
define append_normal_space = 120 { go here to append a normal space between words }
define pdfbox_crop = 1 { pdf_box_type passed to find_pic_file }
define pdfbox_media = 2
define pdfbox_bleed = 3
define pdfbox_trim = 4
define pdfbox_art = 5
define pdfbox_none = 6
⟨ Declare action procedures for use by main_control 1097 ⟩
⟨ Declare the procedure called handle_right_brace 1122 ⟩
procedure main_control; { governs TEX's activities }
  label big_switch, reswitch, main_loop, main_loop_wrapup, main_loop_move, main_loop_move + 1,
      main_loop_move + 2, main_loop_move_lig, main_loop_lookahead, main_loop_lookahead + 1,
      main_lig_loop, main_lig_loop + 1, main_lig_loop + 2, collect_native, collected, append_normal_space, exit;
  var t: integer; { general-purpose temporary variable }
  begin if every_job ≠ null then begin_token_list(every_job, every_job_text);
big_switch: get_x_token;
reswitch: ⟨ Give diagnostic information, if requested 1085 ⟩;
  case abs(mode) + cur_cmd of
  hmode + letter, hmode + other_char, hmode + char_given: goto main_loop;
  hmode + char_num: begin scan_usv_num; cur_chr ← cur_val; goto main_loop; end;
  hmode + no_boundary: begin get_x_token;
    if (cur_cmd = letter) ∨ (cur_cmd = other_char) ∨ (cur_cmd = char_given) ∨ (cur_cmd = char_num)
        then cancel_boundary ← true;
    goto reswitch;
    end;
  othercases begin if abs(mode) = hmode then check_for_post_char_toks(big_switch);
    case abs(mode) + cur_cmd of
    hmode + spacer: if space_factor = 1000 then goto append_normal_space
      else app_space;
    hmode + ex_space, mmode + ex_space: goto append_normal_space;
    ⟨ Cases of main_control that are not part of the inner loop 1099 ⟩
    end
    end
  endcases; { of the big case statement }
  goto big_switch;
main_loop: ⟨ Append character cur_chr and the following characters (if any) to the current hlist in the
      current font; goto reswitch when a non-character has been fetched 1088 ⟩;
append_normal_space: check_for_post_char_toks(big_switch);
  ⟨ Append a normal inter-word space to the current list, then goto big_switch 1095 ⟩;
exit: end;
```

1085. When a new token has just been fetched at *big_switch*, we have an ideal place to monitor TeX's activity.

```
⟨ Give diagnostic information, if requested 1085 ⟩ ≡
  if interrupt ≠ 0 then
    if OK_to_interrupt then
      begin back_input; check_interrupt; goto big_switch;
      end;
  debug if panicking then check_mem(false); gubed
  if tracing_commands > 0 then show_cur_cmd_chr
```

This code is used in section 1084.

1086. The following part of the program was first written in a structured manner, according to the philosophy that "premature optimization is the root of all evil." Then it was rearranged into pieces of spaghetti so that the most common actions could proceed with little or no redundancy.

The original unoptimized form of this algorithm resembles the *reconstitute* procedure, which was described earlier in connection with hyphenation. Again we have an implied "cursor" between characters *cur_l* and *cur_r*. The main difference is that the *lig_stack* can now contain a charnode as well as pseudo-ligatures; that stack is now usually nonempty, because the next character of input (if any) has been appended to it. In *main_control* we have

$$cur_r = \begin{cases} character(lig_stack), & \text{if } lig_stack > null; \\ font_bchar[cur_font], & \text{otherwise;} \end{cases}$$

except when $character(lig_stack) = font_false_bchar[cur_font]$. Several additional global variables are needed.

```
⟨ Global variables 13 ⟩ +≡
main_f: internal_font_number; { the current font }
main_i: four_quarters; { character information bytes for cur_l }
main_j: four_quarters; { ligature/kern command }
main_k: font_index; { index into font_info }
main_p: pointer; { temporary register for list manipulation }
main_pp, main_ppp: pointer; { more temporary registers for list manipulation }
main_h: pointer; { temp for hyphen offset in native-font text }
is_hyph: boolean; { whether the last char seen is the font's hyphenchar }
space_class: integer;
prev_class: integer;
main_s: integer; { space factor value }
bchar: halfword; { boundary character of current font, or non_char }
false_bchar: halfword; { nonexistent character matching bchar, or non_char }
cancel_boundary: boolean; { should the left boundary be ignored? }
ins_disc: boolean; { should we insert a discretionary node? }
```

1087. The boolean variables of the main loop are normally false, and always reset to false before the loop is left. That saves us the extra work of initializing each time.

```
⟨ Set initial values of key variables 23 ⟩ +≡
  ligature_present ← false; cancel_boundary ← false; lft_hit ← false; rt_hit ← false; ins_disc ← false;
```

1088. We leave the *space_factor* unchanged if *sf_code*(*cur_chr*) = 0; otherwise we set it equal to *sf_code*(*cur_chr*), except that it should never change from a value less than 1000 to a value exceeding 1000. The most common case is *sf_code*(*cur_chr*) = 1000, so we want that case to be fast.

The overall structure of the main loop is presented here. Some program labels are inside the individual sections.

```
define adjust_space_factor ≡
    main_s ← sf_code(cur_chr) mod "10000;
    if main_s = 1000 then space_factor ← 1000
    else if main_s < 1000 then
        begin if main_s > 0 then space_factor ← main_s;
        end
      else if space_factor < 1000 then space_factor ← 1000
        else space_factor ← main_s
define check_for_inter_char_toks(#) ≡ { check for a spacing token list, goto # if found, or big_switch in
        case of the initial letter of a run }
    cur_ptr ← null; space_class ← sf_code(cur_chr) div "10000;
    if XeTeX_inter_char_tokens_en ∧ space_class ≠ char_class_ignored then
      begin { class 4096 = ignored (for combining marks etc) }
      if prev_class = char_class_boundary then
        begin { boundary }
        if (state ≠ token_list) ∨ (token_type ≠ backed_up_char) then
          begin find_sa_element(inter_char_val, char_class_boundary * char_class_limit + space_class,
              false);
          if (cur_ptr ≠ null) ∧ (sa_ptr(cur_ptr) ≠ null) then
            begin if cur_cmd ≠ letter then cur_cmd ← other_char;
            cur_tok ← (cur_cmd * max_char_val) + cur_chr; back_input;
            token_type ← backed_up_char; begin_token_list(sa_ptr(cur_ptr), inter_char_text);
            goto big_switch;
            end
          end
        end
      else begin find_sa_element(inter_char_val, prev_class * char_class_limit + space_class, false);
        if (cur_ptr ≠ null) ∧ (sa_ptr(cur_ptr) ≠ null) then
          begin if cur_cmd ≠ letter then cur_cmd ← other_char;
          cur_tok ← (cur_cmd * max_char_val) + cur_chr; back_input; token_type ← backed_up_char;
          begin_token_list(sa_ptr(cur_ptr), inter_char_text); prev_class ← char_class_boundary;
          goto #;
          end;
        end;
      prev_class ← space_class;
      end
define check_for_post_char_toks(#) ≡
      if XeTeX_inter_char_tokens_en ∧ (space_class ≠ char_class_ignored) ∧ (prev_class ≠
          char_class_boundary) then
        begin prev_class ← char_class_boundary;
        find_sa_element(inter_char_val, space_class * char_class_limit + char_class_boundary, false);
            { boundary }
        if (cur_ptr ≠ null) ∧ (sa_ptr(cur_ptr) ≠ null) then
          begin if cur_cs = 0 then
            begin if cur_cmd = char_num then cur_cmd ← other_char;
            cur_tok ← (cur_cmd * max_char_val) + cur_chr;
            end
```

```
      else cur_tok ← cs_token_flag + cur_cs;
      back_input; begin_token_list(sa_ptr(cur_ptr), inter_char_text); goto #;
      end;
    end
```

⟨ Append character *cur_chr* and the following characters (if any) to the current hlist in the current font; **goto** *reswitch* when a non-character has been fetched 1088 ⟩ ≡

```
prev_class ← char_class_boundary;  { boundary }
  { added code for native font support }
if is_native_font(cur_font) then
  begin if mode > 0 then
    if language ≠ clang then fix_language;
  main_h ← 0; main_f ← cur_font; native_len ← 0;
collect_native: adjust_space_factor; check_for_inter_char_toks(collected);
  if (cur_chr > "FFFF) then
    begin native_room(2); append_native((cur_chr − "10000) div 1024 + "D800);
    append_native((cur_chr − "10000) mod 1024 + "DC00);
    end
  else begin native_room(1); append_native(cur_chr);
    end;
  is_hyph ← (cur_chr = hyphen_char[main_f]) ∨ (XeTeX_dash_break_en ∧ ((cur_chr = "2014) ∨ (cur_chr =
      "2013)));
  if (main_h = 0) ∧ is_hyph then main_h ← native_len;
      { try to collect as many chars as possible in the same font }
  get_next;
  if (cur_cmd = letter) ∨ (cur_cmd = other_char) ∨ (cur_cmd = char_given) then goto collect_native;
  x_token;
  if (cur_cmd = letter) ∨ (cur_cmd = other_char) ∨ (cur_cmd = char_given) then goto collect_native;
  if cur_cmd = char_num then
    begin scan_usv_num; cur_chr ← cur_val; goto collect_native;
    end;
  check_for_post_char_toks(collected);
collected: if (font_mapping[main_f] ≠ 0) then
    begin main_k ← apply_mapping(font_mapping[main_f], native_text, native_len); native_len ← 0;
    native_room(main_k); main_h ← 0;
    for main_p ← 0 to main_k − 1 do
      begin append_native(mapped_text[main_p]);
      if (main_h = 0) ∧ ((mapped_text[main_p] = hyphen_char[main_f]) ∨ (XeTeX_dash_break_en ∧
          ((mapped_text[main_p] = "2014) ∨ (mapped_text[main_p] = "2013)))) then
        main_h ← native_len;
      end
    end;
  if tracing_lost_chars > 0 then
    begin temp_ptr ← 0;
    while (temp_ptr < native_len) do
      begin main_k ← native_text[temp_ptr]; incr(temp_ptr);
      if (main_k ≥ "D800) ∧ (main_k < "DC00) then
        begin main_k ← "10000 + (main_k − "D800) * 1024;
        main_k ← main_k + native_text[temp_ptr] − "DC00; incr(temp_ptr);
        end;
      if map_char_to_glyph(main_f, main_k) = 0 then char_warning(main_f, main_k);
      end
    end;
```

```
main_k ← native_len; main_pp ← tail;
if mode = hmode then
  begin main_ppp ← head;  { find node preceding tail, skipping discretionaries }
  while (main_ppp ≠ main_pp) ∧ (link(main_ppp) ≠ main_pp) do
    begin if (¬is_char_node(main_ppp)) ∧ (type(main_ppp) = disc_node) then
      begin temp_ptr ← main_ppp;
      for main_p ← 1 to replace_count(temp_ptr) do main_ppp ← link(main_ppp);
      end;
    if main_ppp ≠ main_pp then main_ppp ← link(main_ppp);
    end;
  temp_ptr ← 0;
  repeat if main_h = 0 then main_h ← main_k;
    if is_native_word_node(main_pp) ∧ (native_font(main_pp) = main_f) ∧ (main_ppp ≠
        main_pp) ∧ (¬is_char_node(main_ppp)) ∧ (type(main_ppp) ≠ disc_node) then
      begin  { make a new temp string that contains the concatenated text of tail + the current
          word/fragment }
      main_k ← main_h + native_length(main_pp); native_room(main_k);
      save_native_len ← native_len;
      for main_p ← 0 to native_length(main_pp) − 1 do
        append_native(get_native_char(main_pp, main_p));
      for main_p ← 0 to main_h − 1 do append_native(native_text[temp_ptr + main_p]);
      do_locale_linebreaks(save_native_len, main_k); native_len ← save_native_len;
          { discard the temp string }
      main_k ← native_len − main_h − temp_ptr;
          { and set main_k to remaining length of new word }
      temp_ptr ← main_h;  { pointer to remaining fragment }
      main_h ← 0;
      while (main_h < main_k) ∧ (native_text[temp_ptr + main_h] ≠
          hyphen_char[main_f]) ∧ ((¬XeTeX_dash_break_en) ∨ ((native_text[temp_ptr + main_h] ≠
          "2014) ∧ (native_text[temp_ptr + main_h] ≠ "2013))) do incr(main_h);
          { look for next hyphen or end of text }
      if (main_h < main_k) then incr(main_h);  { remove the preceding node from the list }
      link(main_ppp) ← link(main_pp); link(main_pp) ← null; flush_node_list(main_pp);
      main_pp ← tail;
      while (link(main_ppp) ≠ main_pp) do main_ppp ← link(main_ppp);
      end
    else begin do_locale_linebreaks(temp_ptr, main_h);  { append fragment of current word }
      temp_ptr ← temp_ptr + main_h;  { advance ptr to remaining fragment }
      main_k ← main_k − main_h;  { decrement remaining length }
      main_h ← 0;
      while (main_h < main_k) ∧ (native_text[temp_ptr + main_h] ≠
          hyphen_char[main_f]) ∧ ((¬XeTeX_dash_break_en) ∨ ((native_text[temp_ptr + main_h] ≠
          "2014) ∧ (native_text[temp_ptr + main_h] ≠ "2013))) do incr(main_h);
          { look for next hyphen or end of text }
      if (main_h < main_k) then incr(main_h);
      end;
    if (main_k > 0) ∨ is_hyph then
      begin tail_append(new_disc);  { add a break if we aren't at end of text (must be a hyphen), or
          if last char in original text was a hyphen }
      main_pp ← tail;
      end;
  until main_k = 0;
```

```
    end
  else begin  { must be restricted hmode, so no need for line-breaking or discretionaries }
    { but there might already be explicit disc_nodes in the list }
    main_ppp ← head;  { find node preceding tail, skipping discretionaries }
    while (main_ppp ≠ main_pp) ∧ (link(main_ppp) ≠ main_pp) do
      begin if (¬is_char_node(main_ppp)) ∧ (type(main_ppp) = disc_node) then
        begin temp_ptr ← main_ppp;
        for main_p ← 1 to replace_count(temp_ptr) do main_ppp ← link(main_ppp);
        end;
      if main_ppp ≠ main_pp then main_ppp ← link(main_ppp);
      end;
    if is_native_word_node(main_pp) ∧ (native_font(main_pp) = main_f) ∧ (main_ppp ≠
        main_pp) ∧ (¬is_char_node(main_ppp)) ∧ (type(main_ppp) ≠ disc_node) then
      begin  { total string length for the new merged whatsit }
      link(main_pp) ← new_native_word_node(main_f, main_k + native_length(main_pp));
      tail ← link(main_pp);  { copy text from the old one into the new }
      for main_p ← 0 to native_length(main_pp) − 1 do
        set_native_char(tail, main_p, get_native_char(main_pp, main_p));  { append the new text }
      for main_p ← 0 to main_k − 1 do
        set_native_char(tail, main_p + native_length(main_pp), native_text[main_p]);
      set_native_metrics(tail, XeTeX_use_glyph_metrics);  { remove the preceding node from the list }
      main_p ← head;
      if main_p ≠ main_pp then
        while link(main_p) ≠ main_pp do main_p ← link(main_p);
      link(main_p) ← link(main_pp); link(main_pp) ← null; flush_node_list(main_pp);
      end
    else begin  { package the current string into a native_word whatsit }
      link(main_pp) ← new_native_word_node(main_f, main_k); tail ← link(main_pp);
      for main_p ← 0 to main_k − 1 do set_native_char(tail, main_p, native_text[main_p]);
      set_native_metrics(tail, XeTeX_use_glyph_metrics);
      end
    end;
  if XeTeX_interword_space_shaping_state > 0 then
    begin  { tail is a word we have just appended. If it is preceded by another word with a normal
        inter-word space between (all in the same font), then we will measure that space in context and
        replace it with an adjusted glue value if it differs from the font's normal space. }
      { First we look for the most recent native_word in the list and set main_pp to it. This is potentially
        expensive, in the case of very long paragraphs, but in practice it's negligible compared to the
        cost of shaping and measurement. }
    main_p ← head; main_pp ← null;
    while main_p ≠ tail do
      begin if is_native_word_node(main_p) then main_pp ← main_p;
      main_p ← link(main_p);
      end;
    if (main_pp ≠ null) then
      begin  { check if the font matches; if so, check the intervening nodes }
      if (native_font(main_pp) = main_f) then
        begin main_p ← link(main_pp);
          { Skip nodes that should be invisible to inter-word spacing, so that e.g., '\nobreak\ '
          doesn't prevent contextual measurement. This loop is guaranteed to end safely because it'll
          eventually hit tail, which is a native_word node, if nothing else intervenes. }
        while node_is_invisible_to_interword_space(main_p) do main_p ← link(main_p);
```

```
    if ¬is_char_node(main_p) ∧ (type(main_p) = glue_node) then
      begin { We found a glue node: we might have an inter-word space to deal with. Again,
          skip nodes that should be invisible to inter-word spacing. We leave main_p pointing to
          the glue node; main_pp is the preceding word. }
      main_ppp ← link(main_p);
      while node_is_invisible_to_interword_space(main_ppp) do main_ppp ← link(main_ppp);
      if main_ppp = tail then
        begin { We found a candidate inter-word space! Collect the characters of both words,
            separated by a single space, into a native_word node and measure its overall width. }
        temp_ptr ← new_native_word_node(main_f, native_length(main_pp)+1+native_length(tail));
        main_k ← 0;
        for t ← 0 to native_length(main_pp) − 1 do
          begin set_native_char(temp_ptr, main_k, get_native_char(main_pp, t)); incr(main_k);
          end;
        set_native_char(temp_ptr, main_k, "␣"); incr(main_k);
        for t ← 0 to native_length(tail) − 1 do
          begin set_native_char(temp_ptr, main_k, get_native_char(tail, t)); incr(main_k);
          end;
        set_native_metrics(temp_ptr, XeTeX_use_glyph_metrics); { The contextual space width is
            the difference between this width and the sum of the two words measured separately. }
        t ← width(temp_ptr) − width(main_pp) − width(tail);
        free_node(temp_ptr, native_size(temp_ptr)); { If the desired width differs from the font's
            default word space, we will insert a suitable kern after the existing glue. Because kerns
            are discardable, this will behave OK during line breaking, and it's easier than actually
            modifying/replacing the glue node. }
        if t ≠ width(font_glue[main_f]) then
          begin temp_ptr ← new_kern(t − width(font_glue[main_f]));
          subtype(temp_ptr) ← space_adjustment; link(temp_ptr) ← link(main_p);
          link(main_p) ← temp_ptr;
          end
        end
      end
    end;
  if cur_ptr ≠ null then goto big_switch
  else goto reswitch;
  end; { End of added code for native fonts }
adjust_space_factor;
check_for_inter_char_toks(big_switch); main_f ← cur_font; bchar ← font_bchar[main_f];
false_bchar ← font_false_bchar[main_f];
if mode > 0 then
  if language ≠ clang then fix_language;
fast_get_avail(lig_stack); font(lig_stack) ← main_f; cur_l ← qi(cur_chr); character(lig_stack) ← cur_l;
cur_q ← tail;
if cancel_boundary then
  begin cancel_boundary ← false; main_k ← non_address;
  end
else main_k ← bchar_label[main_f];
if main_k = non_address then goto main_loop_move + 2; { no left boundary processing }
cur_r ← cur_l; cur_l ← non_char; goto main_lig_loop + 1; { begin with cursor after left boundary }
```

main_loop_wrapup: ⟨ Make a ligature node, if *ligature_present*; insert a null discretionary, if

appropriate 1089⟩;
main_loop_move: ⟨If the cursor is immediately followed by the right boundary, **goto** *reswitch*; if it's followed by an invalid character, **goto** *big_switch*; otherwise move the cursor one step to the right and **goto** *main_lig_loop* 1090⟩;
main_loop_lookahead: ⟨Look ahead for another character, or leave *lig_stack* empty if there's none there 1092⟩;
main_lig_loop: ⟨If there's a ligature/kern command relevant to *cur_l* and *cur_r*, adjust the text appropriately; exit to *main_loop_wrapup* 1093⟩;
main_loop_move_lig: ⟨Move the cursor past a pseudo-ligature, then **goto** *main_loop_lookahead* or *main_lig_loop* 1091⟩

This code is used in section 1084.

1089. If *link*(*cur_q*) is nonnull when *wrapup* is invoked, *cur_q* points to the list of characters that were consumed while building the ligature character *cur_l*.

A discretionary break is not inserted for an explicit hyphen when we are in restricted horizontal mode. In particular, this avoids putting discretionary nodes inside of other discretionaries.

```
define pack_lig(#) ≡   { the parameter is either rt_hit or false }
    begin main_p ← new_ligature(main_f, cur_l, link(cur_q));
    if lft_hit then
      begin subtype(main_p) ← 2; lft_hit ← false;
      end;
    if # then
      if lig_stack = null then
        begin incr(subtype(main_p)); rt_hit ← false;
        end;
    link(cur_q) ← main_p; tail ← main_p; ligature_present ← false;
    end
define wrapup(#) ≡
    if cur_l < non_char then
      begin if link(cur_q) > null then
        if character(tail) = qi(hyphen_char[main_f]) then ins_disc ← true;
      if ligature_present then pack_lig(#);
      if ins_disc then
        begin ins_disc ← false;
        if mode > 0 then tail_append(new_disc);
        end;
      end
```

⟨Make a ligature node, if *ligature_present*; insert a null discretionary, if appropriate 1089⟩ ≡
wrapup(*rt_hit*)

This code is used in section 1088.

1090. ⟨ If the cursor is immediately followed by the right boundary, **goto** *reswitch*; if it's followed by an invalid character, **goto** *big_switch*; otherwise move the cursor one step to the right and **goto** *main_lig_loop* 1090 ⟩ ≡

```
  if lig_stack = null then goto reswitch;
  cur_q ← tail; cur_l ← character(lig_stack);
main_loop_move + 1: if ¬is_char_node(lig_stack) then goto main_loop_move_lig;
main_loop_move + 2: if (cur_chr < font_bc[main_f]) ∨ (cur_chr > font_ec[main_f]) then
    begin char_warning(main_f, cur_chr); free_avail(lig_stack); goto big_switch;
    end;
  main_i ← char_info(main_f)(cur_l);
  if ¬char_exists(main_i) then
    begin char_warning(main_f, cur_chr); free_avail(lig_stack); goto big_switch;
    end;
  link(tail) ← lig_stack; tail ← lig_stack   { main_loop_lookahead is next }
```

This code is used in section 1088.

1091. Here we are at *main_loop_move_lig*. When we begin this code we have *cur_q* = *tail* and *cur_l* = *character*(*lig_stack*).

⟨ Move the cursor past a pseudo-ligature, then **goto** *main_loop_lookahead* or *main_lig_loop* 1091 ⟩ ≡

```
  main_p ← lig_ptr(lig_stack);
  if main_p > null then tail_append(main_p);   { append a single character }
  temp_ptr ← lig_stack; lig_stack ← link(temp_ptr); free_node(temp_ptr, small_node_size);
  main_i ← char_info(main_f)(cur_l); ligature_present ← true;
  if lig_stack = null then
    if main_p > null then goto main_loop_lookahead
    else cur_r ← bchar
  else cur_r ← character(lig_stack);
  goto main_lig_loop
```

This code is used in section 1088.

1092. The result of `\char` can participate in a ligature or kern, so we must look ahead for it.

⟨ Look ahead for another character, or leave *lig_stack* empty if there's none there 1092 ⟩ ≡

```
  get_next;   { set only cur_cmd and cur_chr, for speed }
  if cur_cmd = letter then goto main_loop_lookahead + 1;
  if cur_cmd = other_char then goto main_loop_lookahead + 1;
  if cur_cmd = char_given then goto main_loop_lookahead + 1;
  x_token;   { now expand and set cur_cmd, cur_chr, cur_tok }
  if cur_cmd = letter then goto main_loop_lookahead + 1;
  if cur_cmd = other_char then goto main_loop_lookahead + 1;
  if cur_cmd = char_given then goto main_loop_lookahead + 1;
  if cur_cmd = char_num then
    begin scan_char_num; cur_chr ← cur_val; goto main_loop_lookahead + 1;
    end;
  if cur_cmd = no_boundary then bchar ← non_char;
  cur_r ← bchar; lig_stack ← null; goto main_lig_loop;
main_loop_lookahead + 1: adjust_space_factor; check_for_inter_char_toks(big_switch);
  fast_get_avail(lig_stack); font(lig_stack) ← main_f; cur_r ← qi(cur_chr); character(lig_stack) ← cur_r;
  if cur_r = false_bchar then cur_r ← non_char   { this prevents spurious ligatures }
```

This code is used in section 1088.

1093. Even though comparatively few characters have a lig/kern program, several of the instructions here count as part of TeX's inner loop, since a potentially long sequential search must be performed. For example, tests with Computer Modern Roman showed that about 40 per cent of all characters actually encountered in practice had a lig/kern program, and that about four lig/kern commands were investigated for every such character.

At the beginning of this code we have $main_i = char_info(main_f)(cur_l)$.

```
⟨ If there's a ligature/kern command relevant to cur_l and cur_r, adjust the text appropriately; exit to
      main_loop_wrapup 1093 ⟩ ≡
  if char_tag(main_i) ≠ lig_tag then goto main_loop_wrapup;
  if cur_r = non_char then goto main_loop_wrapup;
  main_k ← lig_kern_start(main_f)(main_i); main_j ← font_info[main_k].qqqq;
  if skip_byte(main_j) ≤ stop_flag then goto main_lig_loop + 2;
  main_k ← lig_kern_restart(main_f)(main_j);
main_lig_loop + 1: main_j ← font_info[main_k].qqqq;
main_lig_loop + 2: if next_char(main_j) = cur_r then
    if skip_byte(main_j) ≤ stop_flag then ⟨ Do ligature or kern command, returning to main_lig_loop or
        main_loop_wrapup or main_loop_move 1094 ⟩;
  if skip_byte(main_j) = qi(0) then incr(main_k)
  else begin if skip_byte(main_j) ≥ stop_flag then goto main_loop_wrapup;
    main_k ← main_k + qo(skip_byte(main_j)) + 1;
    end;
  goto main_lig_loop + 1
```

This code is used in section 1088.

1094. When a ligature or kern instruction matches a character, we know from *read_font_info* that the character exists in the font, even though we haven't verified its existence in the normal way.

This section could be made into a subroutine, if the code inside *main_control* needs to be shortened.

```
⟨ Do ligature or kern command, returning to main_lig_loop or main_loop_wrapup or main_loop_move 1094 ⟩ ≡
  begin if op_byte(main_j) ≥ kern_flag then
    begin wrapup(rt_hit); tail_append(new_kern(char_kern(main_f)(main_j))); goto main_loop_move;
    end;
  if cur_l = non_char then lft_hit ← true
  else if lig_stack = null then rt_hit ← true;
  check_interrupt; { allow a way out in case there's an infinite ligature loop }
  case op_byte(main_j) of
  qi(1), qi(5): begin cur_l ← rem_byte(main_j); { =:|, =:|> }
    main_i ← char_info(main_f)(cur_l); ligature_present ← true;
    end;
  qi(2), qi(6): begin cur_r ← rem_byte(main_j); { |=:, |=:> }
    if lig_stack = null then { right boundary character is being consumed }
      begin lig_stack ← new_lig_item(cur_r); bchar ← non_char;
      end
    else if is_char_node(lig_stack) then { link(lig_stack) = null }
        begin main_p ← lig_stack; lig_stack ← new_lig_item(cur_r); lig_ptr(lig_stack) ← main_p;
        end
      else character(lig_stack) ← cur_r;
    end;
  qi(3): begin cur_r ← rem_byte(main_j); { |=:| }
    main_p ← lig_stack; lig_stack ← new_lig_item(cur_r); link(lig_stack) ← main_p;
    end;
  qi(7), qi(11): begin wrapup(false); { |=:|>, |=:|>> }
    cur_q ← tail; cur_l ← rem_byte(main_j); main_i ← char_info(main_f)(cur_l);
    ligature_present ← true;
    end;
  othercases begin cur_l ← rem_byte(main_j); ligature_present ← true; { =: }
    if lig_stack = null then goto main_loop_wrapup
    else goto main_loop_move + 1;
    end
  endcases;
  if op_byte(main_j) > qi(4) then
    if op_byte(main_j) ≠ qi(7) then goto main_loop_wrapup;
  if cur_l < non_char then goto main_lig_loop;
  main_k ← bchar_label[main_f]; goto main_lig_loop + 1;
  end
```

This code is used in section 1093.

1095. The occurrence of blank spaces is almost part of TeX's inner loop, since we usually encounter about one space for every five non-blank characters. Therefore *main_control* gives second-highest priority to ordinary spaces.

When a glue parameter like `\spaceskip` is set to '`0pt`', we will see to it later that the corresponding glue specification is precisely *zero_glue*, not merely a pointer to some specification that happens to be full of zeroes. Therefore it is simple to test whether a glue parameter is zero or not.

```
⟨ Append a normal inter-word space to the current list, then goto big_switch 1095 ⟩ ≡
  if space_skip = zero_glue then
    begin ⟨ Find the glue specification, main_p, for text spaces in the current font 1096 ⟩;
    temp_ptr ← new_glue(main_p);
    end
  else temp_ptr ← new_param_glue(space_skip_code);
  link(tail) ← temp_ptr; tail ← temp_ptr; goto big_switch
```

This code is used in section 1084.

1096. Having *font_glue* allocated for each text font saves both time and memory. If any of the three spacing parameters are subsequently changed by the use of `\fontdimen`, the *find_font_dimen* procedure deallocates the *font_glue* specification allocated here.

```
⟨ Find the glue specification, main_p, for text spaces in the current font 1096 ⟩ ≡
  begin main_p ← font_glue[cur_font];
  if main_p = null then
    begin main_p ← new_spec(zero_glue); main_k ← param_base[cur_font] + space_code;
    width(main_p) ← font_info[main_k].sc; { that's space(cur_font) }
    stretch(main_p) ← font_info[main_k + 1].sc; { and space_stretch(cur_font) }
    shrink(main_p) ← font_info[main_k + 2].sc; { and space_shrink(cur_font) }
    font_glue[cur_font] ← main_p;
    end;
  end
```

This code is used in sections 1095 and 1097.

1097.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ ≡
procedure app_space; { handle spaces when space_factor ≠ 1000 }
  var q: pointer; { glue node }
  begin if (space_factor ≥ 2000) ∧ (xspace_skip ≠ zero_glue) then q ← new_param_glue(xspace_skip_code)
  else begin if space_skip ≠ zero_glue then main_p ← space_skip
    else ⟨ Find the glue specification, main_p, for text spaces in the current font 1096 ⟩;
    main_p ← new_spec(main_p);
    ⟨ Modify the glue specification in main_p according to the space factor 1098 ⟩;
    q ← new_glue(main_p); glue_ref_count(main_p) ← null;
    end;
  link(tail) ← q; tail ← q;
  end;
```

See also sections 1101, 1103, 1104, 1105, 1108, 1114, 1115, 1118, 1123, 1124, 1129, 1133, 1138, 1140, 1145, 1147, 1149, 1150, 1153, 1155, 1157, 1159, 1164, 1167, 1171, 1173, 1177, 1181, 1183, 1185, 1189, 1190, 1192, 1196, 1205, 1209, 1213, 1214, 1217, 1219, 1226, 1228, 1230, 1235, 1245, 1248, 1254, 1265, 1324, 1329, 1333, 1342, 1347, 1356, 1403, and 1439.

This code is used in section 1084.

1098.
```
⟨ Modify the glue specification in main_p according to the space factor 1098 ⟩ ≡
  if space_factor ≥ 2000 then width(main_p) ← width(main_p) + extra_space(cur_font);
  stretch(main_p) ← xn_over_d(stretch(main_p), space_factor, 1000);
  shrink(main_p) ← xn_over_d(shrink(main_p), 1000, space_factor)
```

This code is used in section 1097.

1099. Whew—that covers the main loop. We can now proceed at a leisurely pace through the other combinations of possibilities.

define *any_mode*(#) ≡ *vmode* + #, *hmode* + #, *mmode* + # { for mode-independent commands }

⟨ Cases of *main_control* that are not part of the inner loop 1099 ⟩ ≡
any_mode(*relax*), *vmode* + *spacer*, *mmode* + *spacer*, *mmode* + *no_boundary*: *do_nothing*;
any_mode(*ignore_spaces*): **begin if** *cur_chr* = 0 **then**
begin ⟨ Get the next non-blank non-call token 440 ⟩;
goto *reswitch*;
end
else begin *t* ← *scanner_status*; *scanner_status* ← *normal*; *get_next*; *scanner_status* ← *t*;
if *cur_cs* < *hash_base* **then** *cur_cs* ← *prim_lookup*(*cur_cs* − *single_base*)
else *cur_cs* ← *prim_lookup*(*text*(*cur_cs*));
if *cur_cs* ≠ *undefined_primitive* **then**
begin *cur_cmd* ← *prim_eq_type*(*cur_cs*); *cur_chr* ← *prim_equiv*(*cur_cs*);
cur_tok ← *cs_token_flag* + *prim_eqtb_base* + *cur_cs*; **goto** *reswitch*;
end;
end;
end;
vmode + *stop*: **if** *its_all_over* **then return**; { this is the only way out }
⟨ Forbidden cases detected in *main_control* 1102 ⟩ *any_mode*(*mac_param*): *report_illegal_case*;
⟨ Math-only cases in non-math modes, or vice versa 1100 ⟩: *insert_dollar_sign*;
⟨ Cases of *main_control* that build boxes and lists 1110 ⟩
⟨ Cases of *main_control* that don't depend on *mode* 1264 ⟩
⟨ Cases of *main_control* that are for extensions to TeX 1402 ⟩

This code is used in section 1084.

1100. Here is a list of cases where the user has probably gotten into or out of math mode by mistake. TeX will insert a dollar sign and rescan the current token.

define *non_math*(#) ≡ *vmode* + #, *hmode* + #

⟨ Math-only cases in non-math modes, or vice versa 1100 ⟩ ≡
non_math(*sup_mark*), *non_math*(*sub_mark*), *non_math*(*math_char_num*), *non_math*(*math_given*),
non_math(*XeTeX_math_given*), *non_math*(*math_comp*), *non_math*(*delim_num*), *non_math*(*left_right*),
non_math(*above*), *non_math*(*radical*), *non_math*(*math_style*), *non_math*(*math_choice*),
non_math(*vcenter*), *non_math*(*non_script*), *non_math*(*mkern*), *non_math*(*limit_switch*),
non_math(*mskip*), *non_math*(*math_accent*), *mmode* + *endv*, *mmode* + *par_end*, *mmode* + *stop*,
mmode + *vskip*, *mmode* + *un_vbox*, *mmode* + *valign*, *mmode* + *hrule*

This code is used in section 1099.

1101. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *insert_dollar_sign*;
begin *back_input*; *cur_tok* ← *math_shift_token* + "$"; *print_err*("Missing␣$␣inserted");
help2("I´ve␣inserted␣a␣begin-math/end-math␣symbol␣since␣I␣think")
("you␣left␣one␣out.␣Proceed,␣with␣fingers␣crossed."); *ins_error*;
end;

1102. When erroneous situations arise, TeX usually issues an error message specific to the particular error. For example, '`\noalign`' should not appear in any mode, since it is recognized by the *align_peek* routine in all of its legitimate appearances; a special error message is given when '`\noalign`' occurs elsewhere. But sometimes the most appropriate error message is simply that the user is not allowed to do what he or she has attempted. For example, '`\moveleft`' is allowed only in vertical mode, and '`\lower`' only in non-vertical modes. Such cases are enumerated here and in the other sections referred to under 'See also ....'

⟨ Forbidden cases detected in *main_control* 1102 ⟩ ≡
vmode + *vmove*, *hmode* + *hmove*, *mmode* + *hmove*, *any_mode*(*last_item*),

See also sections 1152, 1165, and 1198.

This code is used in section 1099.

1103. The '*you_cant*' procedure prints a line saying that the current command is illegal in the current mode; it identifies these things symbolically.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *you_cant*;
begin *print_err*("You␣can´t␣use␣`"); *print_cmd_chr*(*cur_cmd*, *cur_chr*); *print*("´␣in␣");
print_mode(*mode*);
end;

1104. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *report_illegal_case*;
begin *you_cant*; *help4*("Sorry,␣but␣I´m␣not␣programmed␣to␣handle␣this␣case;")
("I´ll␣just␣pretend␣that␣you␣didn´t␣ask␣for␣it.")
("If␣you´re␣in␣the␣wrong␣mode,␣you␣might␣be␣able␣to")
("return␣to␣the␣right␣one␣by␣typing␣`I}´␣or␣`I$´␣or␣`I\par´.");
error;
end;

1105. Some operations are allowed only in privileged modes, i.e., in cases that *mode* > 0. The *privileged* function is used to detect violations of this rule; it issues an error message and returns *false* if the current *mode* is negative.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
function *privileged*: *boolean*;
begin if *mode* > 0 **then** *privileged* ← *true*
else begin *report_illegal_case*; *privileged* ← *false*;
end;
end;

1106. Either `\dump` or `\end` will cause *main_control* to enter the endgame, since both of them have '*stop*' as their command code.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("end", *stop*, 0);
primitive("dump", *stop*, 1);

1107. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
stop: **if** *chr_code* = 1 **then** *print_esc*("dump") **else** *print_esc*("end");

1108. We don't want to leave *main_control* immediately when a *stop* command is sensed, because it may be necessary to invoke an `\output` routine several times before things really grind to a halt. (The output routine might even say '`\gdef\end{...}`', to prolong the life of the job.) Therefore *its_all_over* is *true* only when the current page and contribution list are empty, and when the last output was not a "dead cycle."

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
function its_all_over: boolean; { do this when \end or \dump occurs }
  label exit;
  begin if privileged then
    begin if (page_head = page_tail) ∧ (head = tail) ∧ (dead_cycles = 0) then
      begin its_all_over ← true; return;
      end;
    back_input; { we will try to end again after ejecting residual material }
    tail_append(new_null_box); width(tail) ← hsize; tail_append(new_glue(fill_glue));
    tail_append(new_penalty(−'10000000000));
    build_page; { append \hbox to \hsize{}\vfill\penalty-'10000000000 }
    end;
  its_all_over ← false;
exit: end;
```

1109. Building boxes and lists. The most important parts of *main_control* are concerned with TeX's chief mission of box-making. We need to control the activities that put entries on vlists and hlists, as well as the activities that convert those lists into boxes. All of the necessary machinery has already been developed; it remains for us to "push the buttons" at the right times.

1110. As an introduction to these routines, let's consider one of the simplest cases: What happens when '`\hrule`' occurs in vertical mode, or '`\vrule`' in horizontal mode or math mode? The code in *main_control* is short, since the *scan_rule_spec* routine already does most of what is required; thus, there is no need for a special action procedure.

Note that baselineskip calculations are disabled after a rule in vertical mode, by setting *prev_depth* ← *ignore_depth*.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ ≡
vmode + *hrule*, *hmode* + *vrule*, *mmode* + *vrule*: **begin** *tail_append*(*scan_rule_spec*);
if *abs*(*mode*) = *vmode* **then** *prev_depth* ← *ignore_depth*
else if *abs*(*mode*) = *hmode* **then** *space_factor* ← 1000;
end;

See also sections 1111, 1117, 1121, 1127, 1144, 1146, 1148, 1151, 1156, 1158, 1163, 1166, 1170, 1176, 1180, 1184, 1188, 1191, 1194, 1204, 1208, 1212, 1216, 1218, 1221, 1225, 1229, 1234, 1244, and 1247.

This code is used in section 1099.

1111. The processing of things like `\hskip` and `\vskip` is slightly more complicated. But the code in *main_control* is very short, since it simply calls on the action routine *append_glue*. Similarly, `\kern` activates *append_kern*.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
vmode + *vskip*, *hmode* + *hskip*, *mmode* + *hskip*, *mmode* + *mskip*: *append_glue*;
any_mode(*kern*), *mmode* + *mkern*: *append_kern*;

1112. The *hskip* and *vskip* command codes are used for control sequences like `\hss` and `\vfil` as well as for `\hskip` and `\vskip`. The difference is in the value of *cur_chr*.

define *fil_code* = 0 { identifies `\hfil` and `\vfil` }
define *fill_code* = 1 { identifies `\hfill` and `\vfill` }
define *ss_code* = 2 { identifies `\hss` and `\vss` }
define *fil_neg_code* = 3 { identifies `\hfilneg` and `\vfilneg` }
define *skip_code* = 4 { identifies `\hskip` and `\vskip` }
define *mskip_code* = 5 { identifies `\mskip` }

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("hskip", *hskip*, *skip_code*);
primitive("hfil", *hskip*, *fil_code*); *primitive*("hfill", *hskip*, *fill_code*);
primitive("hss", *hskip*, *ss_code*); *primitive*("hfilneg", *hskip*, *fil_neg_code*);
primitive("vskip", *vskip*, *skip_code*);
primitive("vfil", *vskip*, *fil_code*); *primitive*("vfill", *vskip*, *fill_code*);
primitive("vss", *vskip*, *ss_code*); *primitive*("vfilneg", *vskip*, *fil_neg_code*);
primitive("mskip", *mskip*, *mskip_code*);
primitive("kern", *kern*, *explicit*); *primitive*("mkern", *mkern*, *mu_glue*);

1113. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
hskip: case chr_code of
  skip_code: print_esc("hskip");
  fil_code: print_esc("hfil");
  fill_code: print_esc("hfill");
  ss_code: print_esc("hss");
  othercases print_esc("hfilneg")
  endcases;
vskip: case chr_code of
  skip_code: print_esc("vskip");
  fil_code: print_esc("vfil");
  fill_code: print_esc("vfill");
  ss_code: print_esc("vss");
  othercases print_esc("vfilneg")
  endcases;
mskip: print_esc("mskip");
kern: print_esc("kern");
mkern: print_esc("mkern");
```

1114. All the work relating to glue creation has been relegated to the following subroutine. It does not call *build_page*, because it is used in at least one place where that would be a mistake.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure append_glue;
  var s: small_number;  { modifier of skip command }
  begin s ← cur_chr;
  case s of
  fil_code: cur_val ← fil_glue;
  fill_code: cur_val ← fill_glue;
  ss_code: cur_val ← ss_glue;
  fil_neg_code: cur_val ← fil_neg_glue;
  skip_code: scan_glue(glue_val);
  mskip_code: scan_glue(mu_val);
  end;  { now cur_val points to the glue specification }
  tail_append(new_glue(cur_val));
  if s ≥ skip_code then
    begin decr(glue_ref_count(cur_val));
    if s > skip_code then subtype(tail) ← mu_glue;
    end;
  end;
```

1115. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure append_kern;
  var s: quarterword;  { subtype of the kern node }
  begin s ← cur_chr; scan_dimen(s = mu_glue, false, false); tail_append(new_kern(cur_val));
  subtype(tail) ← s;
  end;
```

1116. Many of the actions related to box-making are triggered by the appearance of braces in the input. For example, when the user says '`\hbox to 100pt{`⟨ hlist ⟩`}`' in vertical mode, the information about the box size (100pt, *exactly*) is put onto *save_stack* with a level boundary word just above it, and *cur_group* ← *adjusted_hbox_group*; TeX enters restricted horizontal mode to process the hlist. The right brace eventually causes *save_stack* to be restored to its former state, at which time the information about the box size (100pt, *exactly*) is available once again; a box is packaged and we leave restricted horizontal mode, appending the new box to the current list of the enclosing mode (in this case to the current list of vertical mode), followed by any vertical adjustments that were removed from the box by *hpack*.

The next few sections of the program are therefore concerned with the treatment of left and right curly braces.

1117. If a left brace occurs in the middle of a page or paragraph, it simply introduces a new level of grouping, and the matching right brace will not have such a drastic effect. Such grouping affects neither the mode nor the current list.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
non_math(left_brace): new_save_level(simple_group);
any_mode(begin_group): new_save_level(semi_simple_group);
any_mode(end_group): if cur_group = semi_simple_group then unsave
  else off_save;
```

1118. We have to deal with errors in which braces and such things are not properly nested. Sometimes the user makes an error of commission by inserting an extra symbol, but sometimes the user makes an error of omission. TeX can't always tell one from the other, so it makes a guess and tries to avoid getting into a loop.

The *off_save* routine is called when the current group code is wrong. It tries to insert something into the user's input that will help clean off the top level.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure off_save;
  var p: pointer; { inserted token }
  begin if cur_group = bottom_level then ⟨ Drop current token and complain that it was unmatched 1120 ⟩
  else begin back_input; p ← get_avail; link(temp_head) ← p; print_err("Missing␣");
    ⟨ Prepare to insert a token that matches cur_group, and print what it is 1119 ⟩;
    print("␣inserted"); ins_list(link(temp_head));
    help5("I´ve␣inserted␣something␣that␣you␣may␣have␣forgotten.")
    ("(See␣the␣<inserted␣text>␣above.)")
    ("With␣luck,␣this␣will␣get␣me␣unwedged.␣But␣if␣you")
    ("really␣didn´t␣forget␣anything,␣try␣typing␣`2´␣now;␣then")
    ("my␣insertion␣and␣my␣current␣dilemma␣will␣both␣disappear."); error;
    end;
  end;
```

1119. At this point, *link*(*temp_head*) = *p*, a pointer to an empty one-word node.

```
⟨ Prepare to insert a token that matches cur_group, and print what it is 1119 ⟩ ≡
  case cur_group of
  semi_simple_group: begin info(p) ← cs_token_flag + frozen_end_group; print_esc("endgroup");
    end;
  math_shift_group: begin info(p) ← math_shift_token + "$"; print_char("$");
    end;
  math_left_group: begin info(p) ← cs_token_flag + frozen_right; link(p) ← get_avail; p ← link(p);
    info(p) ← other_token + "."; print_esc("right.");
    end;
  othercases begin info(p) ← right_brace_token + "}"; print_char("}");
    end
  endcases
```

This code is used in section 1118.

1120.

```
⟨ Drop current token and complain that it was unmatched 1120 ⟩ ≡
  begin print_err("Extra␣"); print_cmd_chr(cur_cmd, cur_chr);
  help1("Things␣are␣pretty␣mixed␣up,␣but␣I␣think␣the␣worst␣is␣over.");
  error;
  end
```

This code is used in section 1118.

1121. The routine for a *right_brace* character branches into many subcases, since a variety of things may happen, depending on *cur_group*. Some types of groups are not supposed to be ended by a right brace; error messages are given in hopes of pinpointing the problem. Most branches of this routine will be filled in later, when we are ready to understand them; meanwhile, we must prepare ourselves to deal with such errors.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
any_mode(right_brace): handle_right_brace;
```

1122.

```
⟨ Declare the procedure called handle_right_brace 1122 ⟩ ≡
procedure handle_right_brace;
  var p, q: pointer; { for short-term use }
    d: scaled; { holds split_max_depth in insert_group }
    f: integer; { holds floating_penalty in insert_group }
  begin case cur_group of
  simple_group: unsave;
  bottom_level: begin print_err("Too␣many␣}´s");
    help2("You´ve␣closed␣more␣groups␣than␣you␣opened.")
    ("Such␣booboos␣are␣generally␣harmless,␣so␣keep␣going."); error;
    end;
  semi_simple_group, math_shift_group, math_left_group: extra_right_brace;
  ⟨ Cases of handle_right_brace where a right_brace triggers a delayed action 1139 ⟩
  othercases confusion("rightbrace")
  endcases;
  end;
```

This code is used in section 1084.

1123. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure extra_right_brace;
  begin print_err("Extra␣},␣or␣forgotten␣");
  case cur_group of
  semi_simple_group: print_esc("endgroup");
  math_shift_group: print_char("$");
  math_left_group: print_esc("right");
  end;
  help5("I´ve␣deleted␣a␣group-closing␣symbol␣because␣it␣seems␣to␣be")
  ("spurious,␣as␣in␣`$x}$´.␣But␣perhaps␣the␣}␣is␣legitimate␣and")
  ("you␣forgot␣something␣else,␣as␣in␣`\hbox{$x}´.␣In␣such␣cases")
  ("the␣way␣to␣recover␣is␣to␣insert␣both␣the␣forgotten␣and␣the")
  ("deleted␣material,␣e.g.,␣by␣typing␣`I$}´."); error; incr(align_state);
  end;
```

1124. Here is where we clear the parameters that are supposed to revert to their default values after every paragraph and when internal vertical mode is entered.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure normal_paragraph;
  begin if looseness ≠ 0 then eq_word_define(int_base + looseness_code, 0);
  if hang_indent ≠ 0 then eq_word_define(dimen_base + hang_indent_code, 0);
  if hang_after ≠ 1 then eq_word_define(int_base + hang_after_code, 1);
  if par_shape_ptr ≠ null then eq_define(par_shape_loc, shape_ref, null);
  if inter_line_penalties_ptr ≠ null then eq_define(inter_line_penalties_loc, shape_ref, null);
  end;
```

1125. Now let's turn to the question of how `\hbox` is treated. We actually need to consider also a slightly larger context, since constructions like '`\setbox3=\hbox...`' and '`\leaders\hbox...`' and '`\lower3.8pt\hbox...`' are supposed to invoke quite different actions after the box has been packaged. Conversely, constructions like '`\setbox3=`' can be followed by a variety of different kinds of boxes, and we would like to encode such things in an efficient way.

In other words, there are two problems: to represent the context of a box, and to represent its type.

The first problem is solved by putting a "context code" on the *save_stack*, just below the two entries that give the dimensions produced by *scan_spec*. The context code is either a (signed) shift amount, or it is a large integer $\geq$ *box_flag*, where *box_flag* $= 2^{30}$. Codes *box_flag* through *global_box_flag* $- 1$ represent '`\setbox0`' through '`\setbox32767`'; codes *global_box_flag* through *ship_out_flag* $- 1$ represent '`\global\setbox0`' through '`\global\setbox32767`'; code *ship_out_flag* represents '`\shipout`'; and codes *leader_flag* through *leader_flag* $+ 2$ represent '`\leaders`', '`\cleaders`', and '`\xleaders`'.

The second problem is solved by giving the command code *make_box* to all control sequences that produce a box, and by using the following *chr_code* values to distinguish between them: *box_code*, *copy_code*, *last_box_code*, *vsplit_code*, *vtop_code*, *vtop_code* $+$ *vmode*, and *vtop_code* $+$ *hmode*, where the latter two are used to denote `\vbox` and `\hbox`, respectively.

define *box_flag* $\equiv$ *'10000000000* { context code for '`\setbox0`' }
define *global_box_flag* $\equiv$ *'10000100000* { context code for '`\global\setbox0`' }
define *ship_out_flag* $\equiv$ *'10000200000* { context code for '`\shipout`' }
define *leader_flag* $\equiv$ *'10000200001* { context code for '`\leaders`' }
define *box_code* $= 0$ { *chr_code* for '`\box`' }
define *copy_code* $= 1$ { *chr_code* for '`\copy`' }
define *last_box_code* $= 2$ { *chr_code* for '`\lastbox`' }
define *vsplit_code* $= 3$ { *chr_code* for '`\vsplit`' }
define *vtop_code* $= 4$ { *chr_code* for '`\vtop`' }

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive(`"moveleft"`, *hmove*, 1); *primitive*(`"moveright"`, *hmove*, 0);
primitive(`"raise"`, *vmove*, 1); *primitive*(`"lower"`, *vmove*, 0);
primitive(`"box"`, *make_box*, *box_code*); *primitive*(`"copy"`, *make_box*, *copy_code*);
primitive(`"lastbox"`, *make_box*, *last_box_code*); *primitive*(`"vsplit"`, *make_box*, *vsplit_code*);
primitive(`"vtop"`, *make_box*, *vtop_code*);
primitive(`"vbox"`, *make_box*, *vtop_code* + *vmode*); *primitive*(`"hbox"`, *make_box*, *vtop_code* + *hmode*);
primitive(`"shipout"`, *leader_ship*, *a_leaders* $- 1$); { *ship_out_flag* $=$ *leader_flag* $- 1$ }
primitive(`"leaders"`, *leader_ship*, *a_leaders*); *primitive*(`"cleaders"`, *leader_ship*, *c_leaders*);
primitive(`"xleaders"`, *leader_ship*, *x_leaders*);

1126. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
hmove: if chr_code = 1 then print_esc("moveleft") else print_esc("moveright");
vmove: if chr_code = 1 then print_esc("raise") else print_esc("lower");
make_box: case chr_code of
  box_code: print_esc("box");
  copy_code: print_esc("copy");
  last_box_code: print_esc("lastbox");
  vsplit_code: print_esc("vsplit");
  vtop_code: print_esc("vtop");
  vtop_code + vmode: print_esc("vbox");
  othercases print_esc("hbox")
  endcases;
leader_ship: if chr_code = a_leaders then print_esc("leaders")
  else if chr_code = c_leaders then print_esc("cleaders")
    else if chr_code = x_leaders then print_esc("xleaders")
      else print_esc("shipout");
```

1127. Constructions that require a box are started by calling *scan_box* with a specified context code. The *scan_box* routine verifies that a *make_box* command comes next and then it calls *begin_box*.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡

```
vmode + hmove, hmode + vmove, mmode + vmove: begin t ← cur_chr; scan_normal_dimen;
  if t = 0 then scan_box(cur_val) else scan_box(−cur_val);
  end;
any_mode(leader_ship): scan_box(leader_flag − a_leaders + cur_chr);
any_mode(make_box): begin_box(0);
```

1128. The global variable *cur_box* will point to a newly made box. If the box is void, we will have *cur_box* = *null*. Otherwise we will have *type*(*cur_box*) = *hlist_node* or *vlist_node* or *rule_node*; the *rule_node* case can occur only with leaders.

⟨ Global variables 13 ⟩ +≡

```
cur_box: pointer;  { box to be placed into its context }
```

1129. The *box_end* procedure does the right thing with *cur_box*, if *box_context* represents the context as explained above.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure box_end(box_context : integer);
  var p: pointer;  { ord_noad for new box in math mode }
    a: small_number;  { global prefix }
  begin if box_context < box_flag then
    ⟨ Append box cur_box to the current list, shifted by box_context 1130 ⟩
  else if box_context < ship_out_flag then ⟨ Store cur_box in a box register 1131 ⟩
    else if cur_box ≠ null then
        if box_context > ship_out_flag then ⟨ Append a new leader node that uses cur_box 1132 ⟩
        else ship_out(cur_box);
  end;
```

1130. The global variable *adjust_tail* will be non-null if and only if the current box might include adjustments that should be appended to the current vertical list.

```
⟨ Append box cur_box to the current list, shifted by box_context 1130 ⟩ ≡
  begin if cur_box ≠ null then
    begin shift_amount(cur_box) ← box_context;
    if abs(mode) = vmode then
      begin if pre_adjust_tail ≠ null then
        begin if pre_adjust_head ≠ pre_adjust_tail then append_list(pre_adjust_head)(pre_adjust_tail);
        pre_adjust_tail ← null;
        end;
      append_to_vlist(cur_box);
      if adjust_tail ≠ null then
        begin if adjust_head ≠ adjust_tail then append_list(adjust_head)(adjust_tail);
        adjust_tail ← null;
        end;
      if mode > 0 then build_page;
      end
    else begin if abs(mode) = hmode then space_factor ← 1000
      else begin p ← new_noad; math_type(nucleus(p)) ← sub_box; info(nucleus(p)) ← cur_box;
        cur_box ← p;
        end;
      link(tail) ← cur_box; tail ← cur_box;
      end;
    end
  end
```

This code is used in section 1129.

1131.

```
⟨ Store cur_box in a box register 1131 ⟩ ≡
  begin if box_context < global_box_flag then
    begin cur_val ← box_context − box_flag; a ← 0;
    end
  else begin cur_val ← box_context − global_box_flag; a ← 4;
    end;
  if cur_val < 256 then define(box_base + cur_val, box_ref, cur_box)
  else sa_def_box;
  end
```

This code is used in section 1129.

1132.

```
⟨ Append a new leader node that uses cur_box 1132 ⟩ ≡
  begin ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
  if ((cur_cmd = hskip) ∧ (abs(mode) ≠ vmode)) ∨ ((cur_cmd = vskip) ∧ (abs(mode) = vmode)) then
    begin append_glue; subtype(tail) ← box_context − (leader_flag − a_leaders);
    leader_ptr(tail) ← cur_box;
    end
  else begin print_err("Leaders␣not␣followed␣by␣proper␣glue");
    help3("You␣should␣say␣`\leaders␣<box␣or␣rule><hskip␣or␣vskip>´.")
    ("I␣found␣the␣<box␣or␣rule>,␣but␣there´s␣no␣suitable")
    ("<hskip␣or␣vskip>,␣so␣I´m␣ignoring␣these␣leaders."); back_error; flush_node_list(cur_box);
    end;
  end
```

This code is used in section 1129.

1133. Now that we can see what eventually happens to boxes, we can consider the first steps in their creation. The *begin_box* routine is called when *box_context* is a context specification, *cur_chr* specifies the type of box desired, and *cur_cmd* = *make_box*.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure begin_box(box_context : integer);
  label exit, done;
  var p, q: pointer;  { run through the current list }
    r: pointer;  { running behind p }
    fm: boolean;  { a final \beginM \endM node pair? }
    tx: pointer;  { effective tail node }
    m: quarterword;  { the length of a replacement list }
    k: halfword;  { 0 or vmode or hmode }
    n: halfword;  { a box number }
  begin case cur_chr of
  box_code: begin scan_register_num; fetch_box(cur_box); change_box(null);
      { the box becomes void, at the same level }
    end;
  copy_code: begin scan_register_num; fetch_box(q); cur_box ← copy_node_list(q);
    end;
  last_box_code: ⟨ If the current list ends with a box node, delete it from the list and make cur_box point to
      it; otherwise set cur_box ← null 1134 ⟩;
  vsplit_code: ⟨ Split off part of a vertical box, make cur_box point to it 1136 ⟩;
  othercases ⟨ Initiate the construction of an hbox or vbox, then return 1137 ⟩
  endcases;
  box_end(box_context);  { in simple cases, we use the box immediately }
exit: end;
```

1134. Note that the condition ¬*is_char_node*(*tail*) implies that *head* ≠ *tail*, since *head* is a one-word node.

```
define fetch_effective_tail_eTeX(#) ≡ { extract tx, drop \beginM \endM pair }
    q ← head; p ← null;
    repeat r ← p; p ← q; fm ← false;
      if ¬is_char_node(q) then
        if type(q) = disc_node then
          begin for m ← 1 to replace_count(q) do p ← link(p);
          if p = tx then #;
          end
        else if (type(q) = math_node) ∧ (subtype(q) = begin_M_code) then fm ← true;
      q ← link(p);
    until q = tx; { found r..p..q = tx }
    q ← link(tx); link(p) ← q; link(tx) ← null;
    if q = null then
      if fm then confusion("tail1")
      else tail ← p
    else if fm then { r..p = begin_M..q = end_M }
        begin tail ← r; link(r) ← null; flush_node_list(p); end
define check_effective_tail(#) ≡ find_effective_tail_eTeX
define fetch_effective_tail ≡ fetch_effective_tail_eTeX
```

⟨ If the current list ends with a box node, delete it from the list and make *cur_box* point to it; otherwise set *cur_box* ← *null* 1134 ⟩ ≡

```
begin cur_box ← null;
if abs(mode) = mmode then
  begin you_cant; help1("Sorry; this \lastbox will be void."); error;
  end
else if (mode = vmode) ∧ (head = tail) then
    begin you_cant; help2("Sorry... I usually can't take things from the current page.")
    ("This \lastbox will therefore be void."); error;
    end
  else begin check_effective_tail(goto done);
    if ¬is_char_node(tx) then
      if (type(tx) = hlist_node) ∨ (type(tx) = vlist_node) then
        ⟨ Remove the last box, unless it's part of a discretionary 1135 ⟩;
  done: end;
end
```

This code is used in section 1133.

1135. ⟨ Remove the last box, unless it's part of a discretionary 1135 ⟩ ≡

```
begin fetch_effective_tail(goto done); cur_box ← tx; shift_amount(cur_box) ← 0;
end
```

This code is used in section 1134.

1136. Here we deal with things like '`\vsplit 13 to 100pt`'.

```
⟨ Split off part of a vertical box, make cur_box point to it 1136 ⟩ ≡
  begin scan_register_num; n ← cur_val;
  if ¬scan_keyword("to") then
    begin print_err("Missing␣`to´␣inserted");
    help2("I´m␣working␣on␣`\vsplit<box␣number>␣to␣<dimen>´;")
    ("will␣look␣for␣the␣<dimen>␣next."); error;
    end;
  scan_normal_dimen; cur_box ← vsplit(n, cur_val);
  end
```

This code is used in section 1133.

1137. Here is where we enter restricted horizontal mode or internal vertical mode, in order to make a box.

```
⟨ Initiate the construction of an hbox or vbox, then return 1137 ⟩ ≡
  begin k ← cur_chr − vtop_code; saved(0) ← box_context;
  if k = hmode then
    if (box_context < box_flag) ∧ (abs(mode) = vmode) then scan_spec(adjusted_hbox_group, true)
    else scan_spec(hbox_group, true)
  else begin if k = vmode then scan_spec(vbox_group, true)
    else begin scan_spec(vtop_group, true); k ← vmode;
      end;
    normal_paragraph;
    end;
  push_nest; mode ← −k;
  if k = vmode then
    begin prev_depth ← ignore_depth;
    if every_vbox ≠ null then begin_token_list(every_vbox, every_vbox_text);
    end
  else begin space_factor ← 1000;
    if every_hbox ≠ null then begin_token_list(every_hbox, every_hbox_text);
    end;
  return;
  end
```

This code is used in section 1133.

1138.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure scan_box(box_context : integer); { the next input should specify a box or perhaps a rule }
  begin ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
  if cur_cmd = make_box then begin_box(box_context)
  else if (box_context ≥ leader_flag) ∧ ((cur_cmd = hrule) ∨ (cur_cmd = vrule)) then
      begin cur_box ← scan_rule_spec; box_end(box_context);
      end
    else begin
      print_err("A␣<box>␣was␣supposed␣to␣be␣here");
      help3("I␣was␣expecting␣to␣see␣\hbox␣or␣\vbox␣or␣\copy␣or␣\box␣or")
      ("something␣like␣that.␣So␣you␣might␣find␣something␣missing␣in")
      ("your␣output.␣But␣keep␣trying;␣you␣can␣fix␣this␣later."); back_error;
      end;
  end;
```

1139. When the right brace occurs at the end of an `\hbox` or `\vbox` or `\vtop` construction, the *package* routine comes into action. We might also have to finish a paragraph that hasn't ended.

```
⟨ Cases of handle_right_brace where a right_brace triggers a delayed action 1139 ⟩ ≡
hbox_group: package(0);
adjusted_hbox_group: begin adjust_tail ← adjust_head; pre_adjust_tail ← pre_adjust_head; package(0);
  end;
vbox_group: begin end_graf; package(0);
  end;
vtop_group: begin end_graf; package(vtop_code);
  end;
```

See also sections 1154, 1172, 1186, 1187, 1222, 1227, and 1240.

This code is used in section 1122.

1140.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure package(c : small_number);
  var h: scaled; { height of box }
    p: pointer; { first node in a box }
    d: scaled; { max depth }
    u, v: integer; { saved values for upwards mode flag }
  begin d ← box_max_depth; u ← XeTeX_upwards_state; unsave; save_ptr ← save_ptr − 3;
  v ← XeTeX_upwards_state; XeTeX_upwards_state ← u;
  if mode = −hmode then cur_box ← hpack(link(head), saved(2), saved(1))
  else begin cur_box ← vpackage(link(head), saved(2), saved(1), d);
    if c = vtop_code then ⟨ Readjust the height and depth of cur_box, for \vtop 1141 ⟩;
    end;
  XeTeX_upwards_state ← v; pop_nest; box_end(saved(0));
  end;
```

1141. The height of a '`\vtop`' box is inherited from the first item on its list, if that item is an *hlist_node*, *vlist_node*, or *rule_node*; otherwise the `\vtop` height is zero.

```
⟨ Readjust the height and depth of cur_box, for \vtop 1141 ⟩ ≡
  begin h ← 0; p ← list_ptr(cur_box);
  if p ≠ null then
    if type(p) ≤ rule_node then h ← height(p);
  depth(cur_box) ← depth(cur_box) − h + height(cur_box); height(cur_box) ← h;
  end
```

This code is used in section 1140.

1142. A paragraph begins when horizontal-mode material occurs in vertical mode, or when the paragraph is explicitly started by '`\indent`' or '`\noindent`'.

```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("indent", start_par, 1); primitive("noindent", start_par, 0);
```

1143.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
start_par: if chr_code = 0 then print_esc("noindent") else print_esc("indent");
```

1144. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
vmode + *start_par*: *new_graf*(*cur_chr* > 0);
vmode + *letter*, *vmode* + *other_char*, *vmode* + *char_num*, *vmode* + *char_given*, *vmode* + *math_shift*, *vmode* + *un_hbox*, *vmode* + *vrule*, *vmode* + *accent*, *vmode* + *discretionary*, *vmode* + *hskip*, *vmode* + *valign*, *vmode* + *ex_space*, *vmode* + *no_boundary*:
begin *back_input*; *new_graf*(*true*);
end;

1145. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
function *norm_min*(*h* : *integer*): *small_number*;
begin if *h* ≤ 0 **then** *norm_min* ← 1 **else if** *h* ≥ 63 **then** *norm_min* ← 63 **else** *norm_min* ← *h*;
end;
procedure *new_graf*(*indented* : *boolean*);
begin *prev_graf* ← 0;
if (*mode* = *vmode*) ∨ (*head* ≠ *tail*) **then** *tail_append*(*new_param_glue*(*par_skip_code*));
push_nest; *mode* ← *hmode*; *space_factor* ← 1000; *set_cur_lang*; *clang* ← *cur_lang*;
prev_graf ← (*norm_min*(*left_hyphen_min*) * ´100 + *norm_min*(*right_hyphen_min*)) * ´200000 + *cur_lang*;
if *indented* **then**
begin *tail* ← *new_null_box*; *link*(*head*) ← *tail*; *width*(*tail*) ← *par_indent*; **end**;
if *every_par* ≠ *null* **then** *begin_token_list*(*every_par*, *every_par_text*);
if *nest_ptr* = 1 **then** *build_page*; { put *par_skip* glue on current page }
end;

1146. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
hmode + *start_par*, *mmode* + *start_par*: *indent_in_hmode*;

1147. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *indent_in_hmode*;
var *p*, *q*: *pointer*;
begin if *cur_chr* > 0 **then** { `\indent` }
begin *p* ← *new_null_box*; *width*(*p*) ← *par_indent*;
if *abs*(*mode*) = *hmode* **then** *space_factor* ← 1000
else begin *q* ← *new_noad*; *math_type*(*nucleus*(*q*)) ← *sub_box*; *info*(*nucleus*(*q*)) ← *p*; *p* ← *q*;
end;
tail_append(*p*);
end;
end;

1148. A paragraph ends when a *par_end* command is sensed, or when we are in horizontal mode when reaching the right brace of vertical-mode routines like `\vbox`, `\insert`, or `\output`.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
vmode + *par_end*: **begin** *normal_paragraph*;
if *mode* > 0 **then** *build_page*;
end;
hmode + *par_end*: **begin if** *align_state* < 0 **then** *off_save*;
{ this tries to recover from an alignment that didn't end properly }
end_graf; { this takes us to the enclosing mode, if *mode* > 0 }
if *mode* = *vmode* **then** *build_page*;
end;
hmode + *stop*, *hmode* + *vskip*, *hmode* + *hrule*, *hmode* + *un_vbox*, *hmode* + *halign*: *head_for_vmode*;

1149. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *head_for_vmode*;
begin if *mode* < 0 **then**
if *cur_cmd* ≠ *hrule* **then** *off_save*
else begin *print_err*("You␣can´t␣use␣`"); *print_esc*("hrule");
print("´␣here␣except␣with␣leaders");
help2("To␣put␣a␣horizontal␣rule␣in␣an␣hbox␣or␣an␣alignment,")
("you␣should␣use␣\leaders␣or␣\hrulefill␣(see␣The␣TeXbook).");*error*;
end
else begin *back_input*; *cur_tok* ← *par_token*; *back_input*; *token_type* ← *inserted*;
end;
end;

1150. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *end_graf*;
begin if *mode* = *hmode* **then**
begin if *head* = *tail* **then** *pop_nest* { null paragraphs are ignored }
else *line_break*(*false*);
if *LR_save* ≠ *null* **then**
begin *flush_list*(*LR_save*); *LR_save* ← *null*;
end;
normal_paragraph; *error_count* ← 0;
end;
end;

1151. Insertion and adjustment and mark nodes are constructed by the following pieces of the program.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
any_mode(*insert*), *hmode* + *vadjust*, *mmode* + *vadjust*: *begin_insert_or_adjust*;
any_mode(*mark*): *make_mark*;

1152. ⟨ Forbidden cases detected in *main_control* 1102 ⟩ +≡
vmode + *vadjust*,

1153. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *begin_insert_or_adjust*;
begin if *cur_cmd* = *vadjust* **then** *cur_val* ← 255
else begin *scan_eight_bit_int*;
if *cur_val* = 255 **then**
begin *print_err*("You␣can´t␣"); *print_esc*("insert"); *print_int*(255);
help1("I´m␣changing␣to␣\insert0;␣box␣255␣is␣special."); *error*; *cur_val* ← 0;
end;
end;
saved(0) ← *cur_val*;
if (*cur_cmd* = *vadjust*) ∧ *scan_keyword*("pre") **then** *saved*(1) ← 1
else *saved*(1) ← 0;
save_ptr ← *save_ptr* + 2; *new_save_level*(*insert_group*); *scan_left_brace*; *normal_paragraph*; *push_nest*;
mode ← −*vmode*; *prev_depth* ← *ignore_depth*;
end;

1154. ⟨ Cases of *handle_right_brace* where a *right_brace* triggers a delayed action 1139 ⟩ +≡
insert_group: **begin** *end_graf*; $q \leftarrow$ *split_top_skip*; *add_glue_ref*(*q*); $d \leftarrow$ *split_max_depth*;
  $f \leftarrow$ *floating_penalty*; *unsave*; *save_ptr* $\leftarrow$ *save_ptr* − 2;
    { now *saved*(0) is the insertion number, or 255 for *vadjust* }
  $p \leftarrow$ *vpack*(*link*(*head*), *natural*); *pop_nest*;
  if *saved*(0) < 255 **then**
    begin *tail_append*(*get_node*(*ins_node_size*)); *type*(*tail*) $\leftarrow$ *ins_node*; *subtype*(*tail*) $\leftarrow$ *qi*(*saved*(0));
    height(*tail*) $\leftarrow$ *height*(*p*) + *depth*(*p*); *ins_ptr*(*tail*) $\leftarrow$ *list_ptr*(*p*); *split_top_ptr*(*tail*) $\leftarrow$ *q*;
    depth(*tail*) $\leftarrow$ *d*; *float_cost*(*tail*) $\leftarrow$ *f*;
    end
  else begin *tail_append*(*get_node*(*small_node_size*)); *type*(*tail*) $\leftarrow$ *adjust_node*;
    adjust_pre(*tail*) $\leftarrow$ *saved*(1); { the *subtype* is used for *adjust_pre* }
    adjust_ptr(*tail*) $\leftarrow$ *list_ptr*(*p*); *delete_glue_ref*(*q*);
    end;
  free_node(*p*, *box_node_size*);
  if *nest_ptr* = 0 **then** *build_page*;
  end;
output_group: ⟨ Resume the page builder after an output routine has come to an end 1080 ⟩;

1155. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *make_mark*;
  var *p*: *pointer*; { new node }
    c: *halfword*; { the mark class }
  begin if *cur_chr* = 0 **then** $c \leftarrow 0$
  else begin *scan_register_num*; $c \leftarrow$ *cur_val*;
    end;
  $p \leftarrow$ *scan_toks*(*false*, *true*); $p \leftarrow$ *get_node*(*small_node_size*); *mark_class*(*p*) $\leftarrow c$; *type*(*p*) $\leftarrow$ *mark_node*;
  subtype(*p*) $\leftarrow 0$; { the *subtype* is not used }
  mark_ptr(*p*) $\leftarrow$ *def_ref*; *link*(*tail*) $\leftarrow p$; *tail* $\leftarrow p$;
  end;

1156. Penalty nodes get into a list via the *break_penalty* command.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
any_mode(*break_penalty*): *append_penalty*;

1157. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *append_penalty*;
  begin *scan_int*; *tail_append*(*new_penalty*(*cur_val*));
  if *mode* = *vmode* **then** *build_page*;
  end;

1158. The *remove_item* command removes a penalty, kern, or glue node if it appears at the tail of the current list, using a brute-force linear scan. Like `\lastbox`, this command is not allowed in vertical mode (except internal vertical mode), since the current list in vertical mode is sent to the page builder. But if we happen to be able to implement it in vertical mode, we do.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
any_mode(*remove_item*): *delete_last*;

1159. When *delete_last* is called, *cur_chr* is the *type* of node that will be deleted, if present.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure delete_last;
  label exit;
  var p, q: pointer;  { run through the current list }
    r: pointer;  { running behind p }
    fm: boolean;  { a final \beginM \endM node pair? }
    tx: pointer;  { effective tail node }
    m: quarterword;  { the length of a replacement list }
  begin if (mode = vmode) ∧ (tail = head) then
    ⟨ Apologize for inability to do the operation now, unless \unskip follows non-glue 1160 ⟩
  else begin check_effective_tail(return);
    if ¬is_char_node(tx) then
      if type(tx) = cur_chr then
        begin fetch_effective_tail(return); flush_node_list(tx);
        end;
    end;
exit: end;
```

1160. ⟨ Apologize for inability to do the operation now, unless \unskip follows non-glue 1160 ⟩ ≡

```
  begin if (cur_chr ≠ glue_node) ∨ (last_glue ≠ max_halfword) then
    begin you_cant; help2("Sorry...I␣usually␣can't␣take␣things␣from␣the␣current␣page.")
    ("Try␣`I\vskip-\lastskip'␣instead.");
    if cur_chr = kern_node then help_line[0] ← ("Try␣`I\kern-\lastkern'␣instead.")
    else if cur_chr ≠ glue_node then
        help_line[0] ← ("Perhaps␣you␣can␣make␣the␣output␣routine␣do␣it.");
    error;
    end;
  end
```

This code is used in section 1159.

1161. ⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡

```
  primitive("unpenalty", remove_item, penalty_node);
  primitive("unkern", remove_item, kern_node);
  primitive("unskip", remove_item, glue_node);
  primitive("unhbox", un_hbox, box_code);
  primitive("unhcopy", un_hbox, copy_code);
  primitive("unvbox", un_vbox, box_code);
  primitive("unvcopy", un_vbox, copy_code);
```

1162. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
remove_item: if chr_code = glue_node then print_esc("unskip")
  else if chr_code = kern_node then print_esc("unkern")
    else print_esc("unpenalty");
un_hbox: if chr_code = copy_code then print_esc("unhcopy")
  else print_esc("unhbox");
un_vbox: if chr_code = copy_code then print_esc("unvcopy") ⟨ Cases of un_vbox for print_cmd_chr 1673 ⟩
  else print_esc("unvbox");
```

1163. The *un_hbox* and *un_vbox* commands unwrap one of the 256 current boxes.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
vmode + un_vbox, hmode + un_hbox, mmode + un_hbox: unpackage;
```

1164. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure unpackage;
  label done, exit;
  var p: pointer;  { the box }
    r: pointer;  { to remove marginal kern nodes }
    c: box_code .. copy_code;  { should we copy? }
  begin if cur_chr > copy_code then ⟨ Handle saved items and goto done 1674 ⟩;
  c ← cur_chr; scan_register_num; fetch_box(p);
  if p = null then return;
  if (abs(mode) = mmode) ∨ ((abs(mode) = vmode) ∧ (type(p) ≠ vlist_node)) ∨
        ((abs(mode) = hmode) ∧ (type(p) ≠ hlist_node)) then
    begin print_err("Incompatible␣list␣can't␣be␣unboxed");
    help3("Sorry,␣Pandora.␣(You␣sneaky␣devil.)")
    ("I␣refuse␣to␣unbox␣an␣\hbox␣in␣vertical␣mode␣or␣vice␣versa.")
    ("And␣I␣can't␣open␣any␣boxes␣in␣math␣mode.");
    error; return;
    end;
  if c = copy_code then link(tail) ← copy_node_list(list_ptr(p))
  else begin link(tail) ← list_ptr(p); change_box(null); free_node(p, box_node_size);
    end;
done: while link(tail) ≠ null do
    begin r ← link(tail);
    if ¬is_char_node(r) ∧ (type(r) = margin_kern_node) then
      begin link(tail) ← link(r); free_node(r, margin_kern_node_size);
      end;
    tail ← link(tail);
    end;
exit: end;
```

1165. ⟨ Forbidden cases detected in *main_control* 1102 ⟩ +≡

```
  vmode + ital_corr,
```

1166. Italic corrections are converted to kern nodes when the *ital_corr* command follows a character. In math mode the same effect is achieved by appending a kern of zero here, since italic corrections are supplied later.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡

```
hmode + ital_corr: append_italic_correction;
mmode + ital_corr: tail_append(new_kern(0));
```

1167. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure append_italic_correction;
  label exit;
  var p: pointer;  { char_node at the tail of the current list }
    f: internal_font_number;  { the font in the char_node }
  begin if tail ≠ head then
    begin if is_char_node(tail) then p ← tail
    else if type(tail) = ligature_node then p ← lig_char(tail)
      else if (type(tail) = whatsit_node) then
          begin if is_native_word_subtype(tail) then
            begin tail_append(new_kern(get_native_italic_correction(tail))); subtype(tail) ← explicit;
            end
          else if (subtype(tail) = glyph_node) then
              begin tail_append(new_kern(get_native_glyph_italic_correction(tail)));
              subtype(tail) ← explicit;
              end;
          return;
          end
        else return;
    f ← font(p); tail_append(new_kern(char_italic(f)(char_info(f)(character(p)))));
    subtype(tail) ← explicit;
    end;
exit: end;
```

1168. Discretionary nodes are easy in the common case '`\-`', but in the general case we must process three braces full of items.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡

```
  primitive("-", discretionary, 1); primitive("discretionary", discretionary, 0);
```

1169. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
discretionary: if chr_code = 1 then print_esc("-") else print_esc("discretionary");
```

1170. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡

```
hmode + discretionary, mmode + discretionary: append_discretionary;
```

1171. The space factor does not change when we append a discretionary node, but it starts out as 1000 in the subsidiary lists.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure append_discretionary;
  var c: integer;  { hyphen character }
  begin tail_append(new_disc);
  if cur_chr = 1 then
    begin c ← hyphen_char[cur_font];
    if c ≥ 0 then
      if c ≤ biggest_char then pre_break(tail) ← new_character(cur_font, c);
    end
  else begin incr(save_ptr); saved(−1) ← 0; new_save_level(disc_group); scan_left_brace; push_nest;
    mode ← −hmode; space_factor ← 1000;
    end;
  end;
```

1172. The three discretionary lists are constructed somewhat as if they were hboxes. A subroutine called *build_discretionary* handles the transitions. (This is sort of fun.)

```
⟨ Cases of handle_right_brace where a right_brace triggers a delayed action 1139 ⟩ +≡
disc_group: build_discretionary;
```

1173.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure build_discretionary;
  label done, exit;
  var p, q: pointer;  { for link manipulation }
    n: integer;  { length of discretionary list }
  begin unsave;
  ⟨ Prune the current list, if necessary, until it contains only char_node, kern_node, hlist_node, vlist_node,
      rule_node, and ligature_node items; set n to the length of the list, and set q to the list's tail 1175 ⟩;
  p ← link(head); pop_nest;
  case saved(−1) of
  0: pre_break(tail) ← p;
  1: post_break(tail) ← p;
  2: ⟨ Attach list p to the current list, and record its length; then finish up and return 1174 ⟩;
  end;  { there are no other cases }
  incr(saved(−1)); new_save_level(disc_group); scan_left_brace; push_nest; mode ← −hmode;
  space_factor ← 1000;
exit: end;
```

1174.
```
⟨ Attach list p to the current list, and record its length; then finish up and return 1174 ⟩ ≡
  begin if (n > 0) ∧ (abs(mode) = mmode) then
    begin print_err("Illegal␣math␣"); print_esc("discretionary");
    help2("Sorry:␣The␣third␣part␣of␣a␣discretionary␣break␣must␣be")
    ("empty,␣in␣math␣formulas.␣I␣had␣to␣delete␣your␣third␣part."); flush_node_list(p); n ← 0;
    error;
    end
  else link(tail) ← p;
  if n ≤ max_quarterword then replace_count(tail) ← n
  else begin print_err("Discretionary␣list␣is␣too␣long");
    help2("Wow---I␣never␣thought␣anybody␣would␣tweak␣me␣here.")
    ("You␣can´t␣seriously␣need␣such␣a␣huge␣discretionary␣list?"); error;
    end;
  if n > 0 then tail ← q;
  decr(save_ptr); return;
  end
```

This code is used in section 1173.

1175. During this loop, $p = link(q)$ and there are n items preceding p.

```
⟨ Prune the current list, if necessary, until it contains only char_node, kern_node, hlist_node, vlist_node,
      rule_node, and ligature_node items; set n to the length of the list, and set q to the list's tail 1175 ⟩ ≡
  q ← head; p ← link(q); n ← 0;
  while p ≠ null do
    begin if ¬is_char_node(p) then
      if type(p) > rule_node then
        if type(p) ≠ kern_node then
          if type(p) ≠ ligature_node then
            if (type(p) ≠ whatsit_node) ∨ (¬is_native_word_subtype(p) ∧ (subtype(p) ≠ glyph_node)) then
              begin print_err("Improper␣discretionary␣list");
              help1("Discretionary␣lists␣must␣contain␣only␣boxes␣and␣kerns.");
              error; begin_diagnostic;
              print_nl("The␣following␣discretionary␣sublist␣has␣been␣deleted:"); show_box(p);
              end_diagnostic(true); flush_node_list(p); link(q) ← null; goto done;
              end;
    q ← p; p ← link(q); incr(n);
    end;
done:
```

This code is used in section 1173.

1176. We need only one more thing to complete the horizontal mode routines, namely the `\accent` primitive.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
hmode + accent: make_accent;
```

1177. The positioning of accents is straightforward but tedious. Given an accent of width a, designed for characters of height x and slant s; and given a character of width w, height h, and slant t: We will shift the accent down by $x - h$, and we will insert kern nodes that have the effect of centering the accent over the character and shifting the accent to the right by $\delta = \frac{1}{2}(w - a) + h \cdot t - x \cdot s$. If either character is absent from the font, we will simply use the other, without shifting.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure make_accent;
  var s,t: real; { amount of slant }
    p,q,r: pointer; { character, box, and kern nodes }
    f: internal_font_number; { relevant font }
    a,h,x,w,delta,lsb,rsb: scaled; { heights and widths, as explained above }
    i: four_quarters; { character information }
  begin scan_char_num; f ← cur_font; p ← new_character(f, cur_val);
  if p ≠ null then
    begin x ← x_height(f); s ← slant(f)/float_constant(65536);
    if is_native_font(f) then
      begin a ← width(p);
      if a = 0 then get_native_char_sidebearings(f, cur_val, addressof(lsb), addressof(rsb))
      end
    else a ← char_width(f)(char_info(f)(character(p)));
    do_assignments;
    ⟨ Create a character node q for the next character, but set q ← null if problems arise 1178 ⟩;
    if q ≠ null then ⟨ Append the accent with appropriate kerns, then set p ← q 1179 ⟩;
    link(tail) ← p; tail ← p; space_factor ← 1000;
    end;
  end;
```

1178.

```
⟨ Create a character node q for the next character, but set q ← null if problems arise 1178 ⟩ ≡
  q ← null; f ← cur_font;
  if (cur_cmd = letter) ∨ (cur_cmd = other_char) ∨ (cur_cmd = char_given) then
    begin q ← new_character(f, cur_chr); cur_val ← cur_chr
    end
  else if cur_cmd = char_num then
      begin scan_char_num; q ← new_character(f, cur_val);
      end
    else back_input
```

This code is used in section 1177.

1179. The kern nodes appended here must be distinguished from other kerns, lest they be wiped away by the hyphenation algorithm or by a previous line break.

The two kerns are computed with (machine-dependent) *real* arithmetic, but their sum is machine-independent; the net effect is machine-independent, because the user cannot remove these nodes nor access them via `\lastkern`.

```
⟨ Append the accent with appropriate kerns, then set p ← q 1179 ⟩ ≡
  begin t ← slant(f)/float_constant(65536);
  if is_native_font(f) then
    begin w ← width(q); get_native_char_height_depth(f, cur_val, addressof(h), addressof(delta))
        { using delta as scratch space for the unneeded depth value }
    end
  else begin i ← char_info(f)(character(q)); w ← char_width(f)(i); h ← char_height(f)(height_depth(i))
    end;
  if h ≠ x then   { the accent must be shifted up or down }
    begin p ← hpack(p, natural); shift_amount(p) ← x − h;
    end;
  if is_native_font(f) ∧ (a = 0) then   { special case for non-spacing marks }
    delta ← round((w − lsb + rsb)/float_constant(2) + h ∗ t − x ∗ s)
  else delta ← round((w − a)/float_constant(2) + h ∗ t − x ∗ s);
  r ← new_kern(delta); subtype(r) ← acc_kern; link(tail) ← r; link(r) ← p;
  tail ← new_kern(−a − delta); subtype(tail) ← acc_kern; link(p) ← tail; p ← q;
  end
```

This code is used in section 1177.

1180. When '`\cr`' or '`\span`' or a tab mark comes through the scanner into *main_control*, it might be that the user has foolishly inserted one of them into something that has nothing to do with alignment. But it is far more likely that a left brace or right brace has been omitted, since *get_next* takes actions appropriate to alignment only when '`\cr`' or '`\span`' or tab marks occur with *align_state* = 0. The following program attempts to make an appropriate recovery.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
any_mode(car_ret), any_mode(tab_mark): align_error;
any_mode(no_align): no_align_error;
any_mode(omit): omit_error;
```

1181.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure align_error;
  begin if abs(align_state) > 2 then
    ⟨ Express consternation over the fact that no alignment is in progress 1182 ⟩
  else begin back_input;
    if align_state < 0 then
      begin print_err("Missing␣{␣inserted"); incr(align_state); cur_tok ← left_brace_token + "{";
      end
    else begin print_err("Missing␣}␣inserted"); decr(align_state); cur_tok ← right_brace_token + "}";
      end;
    help3("I´ve␣put␣in␣what␣seems␣to␣be␣necessary␣to␣fix")
    ("the␣current␣column␣of␣the␣current␣alignment.")
    ("Try␣to␣go␣on,␣since␣this␣might␣almost␣work."); ins_error;
    end;
  end;
```

1182. ⟨ Express consternation over the fact that no alignment is in progress 1182 ⟩ ≡

```
begin print_err("Misplaced␣"); print_cmd_chr(cur_cmd, cur_chr);
if cur_tok = tab_token + "&" then
  begin help6("I␣can´t␣figure␣out␣why␣you␣would␣want␣to␣use␣a␣tab␣mark")
  ("here.␣If␣you␣just␣want␣an␣ampersand,␣the␣remedy␣is")
  ("simple:␣Just␣type␣`I\&´␣now.␣But␣if␣some␣right␣brace")
  ("up␣above␣has␣ended␣a␣previous␣alignment␣prematurely,")
  ("you´re␣probably␣due␣for␣more␣error␣messages,␣and␣you")
  ("might␣try␣typing␣`S´␣now␣just␣to␣see␣what␣is␣salvageable.");
  end
else begin help5("I␣can´t␣figure␣out␣why␣you␣would␣want␣to␣use␣a␣tab␣mark")
  ("or␣\cr␣or␣\span␣just␣now.␣If␣something␣like␣a␣right␣brace")
  ("up␣above␣has␣ended␣a␣previous␣alignment␣prematurely,")
  ("you´re␣probably␣due␣for␣more␣error␣messages,␣and␣you")
  ("might␣try␣typing␣`S´␣now␣just␣to␣see␣what␣is␣salvageable.");
  end;
error;
end
```

This code is used in section 1181.

1183. The help messages here contain a little white lie, since `\noalign` and `\omit` are allowed also after '`\noalign{...}`'.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure no_align_error;
  begin print_err("Misplaced␣"); print_esc("noalign");
  help2("I␣expect␣to␣see␣\noalign␣only␣after␣the␣\cr␣of")
  ("an␣alignment.␣Proceed,␣and␣I´ll␣ignore␣this␣case."); error;
  end;
procedure omit_error;
  begin print_err("Misplaced␣"); print_esc("omit");
  help2("I␣expect␣to␣see␣\omit␣only␣after␣tab␣marks␣or␣the␣\cr␣of")
  ("an␣alignment.␣Proceed,␣and␣I´ll␣ignore␣this␣case."); error;
  end;
```

1184. We've now covered most of the abuses of `\halign` and `\valign`. Let's take a look at what happens when they are used correctly.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡

```
vmode + halign: init_align;
hmode + valign: ⟨ Cases of main_control for hmode + valign 1513 ⟩
  init_align;
mmode + halign: if privileged then
    if cur_group = math_shift_group then init_align
    else off_save;
vmode + endv, hmode + endv: do_endv;
```

1185. An *align_group* code is supposed to remain on the *save_stack* during an entire alignment, until *fin_align* removes it.

A devious user might force an *endv* command to occur just about anywhere; we must defeat such hacks.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure do_endv;
  begin base_ptr ← input_ptr; input_stack[base_ptr] ← cur_input;
  while (input_stack[base_ptr].index_field ≠ v_template) ∧ (input_stack[base_ptr].loc_field =
      null) ∧ (input_stack[base_ptr].state_field = token_list) do decr(base_ptr);
  if (input_stack[base_ptr].index_field ≠ v_template) ∨ (input_stack[base_ptr].loc_field ≠
      null) ∨ (input_stack[base_ptr].state_field ≠ token_list) then
    fatal_error("(interwoven␣alignment␣preambles␣are␣not␣allowed)");
  if cur_group = align_group then
    begin end_graf;
    if fin_col then fin_row;
    end
  else off_save;
  end;
```

1186. ⟨ Cases of *handle_right_brace* where a *right_brace* triggers a delayed action 1139 ⟩ +≡

```
align_group: begin back_input; cur_tok ← cs_token_flag + frozen_cr; print_err("Missing␣");
  print_esc("cr"); print("␣inserted");
  help1("I´m␣guessing␣that␣you␣meant␣to␣end␣an␣alignment␣here."); ins_error;
  end;
```

1187. ⟨ Cases of *handle_right_brace* where a *right_brace* triggers a delayed action 1139 ⟩ +≡

```
no_align_group: begin end_graf; unsave; align_peek;
  end;
```

1188. Finally, `\endcsname` is not supposed to get through to *main_control*.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
any_mode(end_cs_name): cs_error;
```

1189. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure cs_error;
  begin print_err("Extra␣"); print_esc("endcsname");
  help1("I´m␣ignoring␣this,␣since␣I␣wasn´t␣doing␣a␣\csname."); error;
  end;
```

1190. Building math lists. The routines that TEX uses to create mlists are similar to those we have just seen for the generation of hlists and vlists. But it is necessary to make "noads" as well as nodes, so the reader should review the discussion of math mode data structures before trying to make sense out of the following program.

Here is a little routine that needs to be done whenever a subformula is about to be processed. The parameter is a code like *math_group*.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure push_math(c : group_code);
  begin push_nest; mode ← −mmode; incompleat_noad ← null; new_save_level(c);
  end;
```

1191. We get into math mode from horizontal mode when a '$' (i.e., a *math_shift* character) is scanned. We must check to see whether this '$' is immediately followed by another, in case display math mode is called for.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
hmode + math_shift: init_math;
```

1192.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
⟨ Declare subprocedures for init_math 1544 ⟩
procedure init_math;
  label reswitch, found, not_found, done;
  var w: scaled; { new or partial pre_display_size }
    j: pointer; { prototype box for display }
    x: integer; { new pre_display_direction }
    l: scaled; { new display_width }
    s: scaled; { new display_indent }
    p: pointer; { current node when calculating pre_display_size }
    q: pointer; { glue specification when calculating pre_display_size }
    f: internal_font_number; { font in current char_node }
    n: integer; { scope of paragraph shape specification }
    v: scaled; { w plus possible glue amount }
    d: scaled; { increment to v }
  begin get_token; { get_x_token would fail on \ifmmode! }
  if (cur_cmd = math_shift) ∧ (mode > 0) then ⟨ Go into display math mode 1199 ⟩
  else begin back_input; ⟨ Go into ordinary math mode 1193 ⟩;
    end;
  end;
```

1193.

```
⟨ Go into ordinary math mode 1193 ⟩ ≡
  begin push_math(math_shift_group); eq_word_define(int_base + cur_fam_code, −1);
  if every_math ≠ null then begin_token_list(every_math, every_math_text);
  end
```

This code is used in sections 1192 and 1196.

1194. We get into ordinary math mode from display math mode when '\eqno' or '\leqno' appears. In such cases *cur_chr* will be 0 or 1, respectively; the value of *cur_chr* is placed onto *save_stack* for safe keeping.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
mmode + eq_no: if privileged then
    if cur_group = math_shift_group then start_eq_no
    else off_save;
```

1195. ⟨ Put each of TEX's primitives into the hash table 252 ⟩ +≡
primitive("eqno", *eq_no*, 0); *primitive*("leqno", *eq_no*, 1);

1196. When TEX is in display math mode, *cur_group* = *math_shift_group*, so it is not necessary for the *start_eq_no* procedure to test for this condition.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *start_eq_no*;
begin *saved*(0) ← *cur_chr*; *incr*(*save_ptr*); ⟨ Go into ordinary math mode 1193 ⟩;
end;

1197. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
eq_no: **if** *chr_code* = 1 **then** *print_esc*("leqno") **else** *print_esc*("eqno");

1198. ⟨ Forbidden cases detected in *main_control* 1102 ⟩ +≡
non_math(*eq_no*),

1199. When we enter display math mode, we need to call *line_break* to process the partial paragraph that has just been interrupted by the display. Then we can set the proper values of *display_width* and *display_indent* and *pre_display_size*.

⟨ Go into display math mode 1199 ⟩ ≡
begin *j* ← *null*; *w* ← −*max_dimen*;
if *head* = *tail* **then** { '\noindent$$' or '$$ $$' }
⟨ Prepare for display after an empty paragraph 1543 ⟩
else begin *line_break*(*true*);
⟨ Calculate the natural width, *w*, by which the characters of the final line extend to the right of the reference point, plus two ems; or set *w* ← *max_dimen* if the non-blank information on that line is affected by stretching or shrinking 1200 ⟩;
end; { now we are in vertical mode, working on the list that will contain the display }
⟨ Calculate the length, *l*, and the shift amount, *s*, of the display lines 1203 ⟩;
push_math(*math_shift_group*); *mode* ← *mmode*; *eq_word_define*(*int_base* + *cur_fam_code*, −1);
eq_word_define(*dimen_base* + *pre_display_size_code*, *w*); *LR_box* ← *j*;
if *eTeX_ex* **then** *eq_word_define*(*int_base* + *pre_display_direction_code*, *x*);
eq_word_define(*dimen_base* + *display_width_code*, *l*); *eq_word_define*(*dimen_base* + *display_indent_code*, *s*);
if *every_display* ≠ *null* **then** *begin_token_list*(*every_display*, *every_display_text*);
if *nest_ptr* = 1 **then** *build_page*;
end

This code is used in section 1192.

1200. ⟨ Calculate the natural width, w, by which the characters of the final line extend to the right of the reference point, plus two ems; or set $w \leftarrow max_dimen$ if the non-blank information on that line is affected by stretching or shrinking 1200 ⟩ ≡

```
⟨ Prepare for display after a non-empty paragraph 1545 ⟩;
while p ≠ null do
  begin ⟨ Let d be the natural width of node p; if the node is "visible," goto found; if the node is glue
      that stretches or shrinks, set v ← max_dimen 1201 ⟩;
  if v < max_dimen then v ← v + d;
  goto not_found;
found: if v < max_dimen then
    begin v ← v + d; w ← v;
    end
  else begin w ← max_dimen; goto done;
    end;
not_found: p ← link(p);
  end;
done: ⟨ Finish the natural width computation 1546 ⟩
```

This code is used in section 1199.

1201. ⟨ Let d be the natural width of node p; if the node is "visible," **goto** *found*; if the node is glue that stretches or shrinks, set $v \leftarrow max_dimen$ 1201 ⟩ ≡

```
reswitch: if is_char_node(p) then
    begin f ← font(p); d ← char_width(f)(char_info(f)(character(p))); goto found;
    end;
  case type(p) of
  hlist_node, vlist_node, rule_node: begin d ← width(p); goto found;
    end;
  ligature_node: ⟨ Make node p look like a char_node and goto reswitch 692 ⟩;
  kern_node: d ← width(p);
  margin_kern_node: d ← width(p);
  ⟨ Cases of 'Let d be the natural width' that need special treatment 1547 ⟩
  glue_node: ⟨ Let d be the natural width of this glue; if stretching or shrinking, set v ← max_dimen; goto
      found in the case of leaders 1202 ⟩;
  whatsit_node: ⟨ Let d be the width of the whatsit p, and goto found if "visible" 1421 ⟩;
  othercases d ← 0
  endcases
```

This code is used in section 1200.

1202. We need to be careful that w, v, and d do not depend on any *glue_set* values, since such values are subject to system-dependent rounding. System-dependent numbers are not allowed to infiltrate parameters like *pre_display_size*, since TeX82 is supposed to make the same decisions on all machines.

```
⟨ Let d be the natural width of this glue; if stretching or shrinking, set v ← max_dimen; goto found in the
      case of leaders 1202 ⟩ ≡
  begin q ← glue_ptr(p); d ← width(q);
  if glue_sign(just_box) = stretching then
    begin if (glue_order(just_box) = stretch_order(q)) ∧ (stretch(q) ≠ 0) then v ← max_dimen;
    end
  else if glue_sign(just_box) = shrinking then
      begin if (glue_order(just_box) = shrink_order(q)) ∧ (shrink(q) ≠ 0) then v ← max_dimen;
      end;
  if subtype(p) ≥ a_leaders then goto found;
  end
```

This code is used in section 1201.

1203. A displayed equation is considered to be three lines long, so we calculate the length and offset of line number *prev_graf* + 2.

```
⟨ Calculate the length, l, and the shift amount, s, of the display lines 1203 ⟩ ≡
  if par_shape_ptr = null then
    if (hang_indent ≠ 0) ∧ (((hang_after ≥ 0) ∧ (prev_graf + 2 > hang_after)) ∨
          (prev_graf + 1 < −hang_after)) then
      begin l ← hsize − abs(hang_indent);
      if hang_indent > 0 then s ← hang_indent else s ← 0;
      end
    else begin l ← hsize; s ← 0;
      end
  else begin n ← info(par_shape_ptr);
    if prev_graf + 2 ≥ n then p ← par_shape_ptr + 2 ∗ n
    else p ← par_shape_ptr + 2 ∗ (prev_graf + 2);
    s ← mem[p − 1].sc; l ← mem[p].sc;
    end
```

This code is used in section 1199.

1204. Subformulas of math formulas cause a new level of math mode to be entered, on the semantic nest as well as the save stack. These subformulas arise in several ways: (1) A left brace by itself indicates the beginning of a subformula that will be put into a box, thereby freezing its glue and preventing line breaks. (2) A subscript or superscript is treated as a subformula if it is not a single character; the same applies to the nucleus of things like `\underline`. (3) The `\left` primitive initiates a subformula that will be terminated by a matching `\right`. The group codes placed on *save_stack* in these three cases are *math_group*, *math_group*, and *math_left_group*, respectively.

Here is the code that handles case (1); the other cases are not quite as trivial, so we shall consider them later.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
mmode + left_brace: begin tail_append(new_noad); back_input; scan_math(nucleus(tail));
  end;
```

1205. Recall that the *nucleus*, *subscr*, and *supscr* fields in a noad are broken down into subfields called *math_type* and either *info* or (*fam*, *character*). The job of *scan_math* is to figure out what to place in one of these principal fields; it looks at the subformula that comes next in the input, and places an encoding of that subformula into a given word of *mem*.

```
define fam_in_range ≡ ((cur_fam ≥ 0) ∧ (cur_fam < number_math_families))
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure scan_math(p : pointer);
  label restart, reswitch, exit;
  var c: integer;  { math character code }
  begin restart: ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
reswitch: case cur_cmd of
  letter, other_char, char_given: begin c ← ho(math_code(cur_chr));
    if is_active_math_char(c) then
      begin ⟨ Treat cur_chr as an active character 1206 ⟩;
      goto restart;
      end;
    end;
  char_num: begin scan_char_num; cur_chr ← cur_val; cur_cmd ← char_given; goto reswitch;
    end;
  math_char_num: if cur_chr = 2 then
      begin  { \Umathchar }
      scan_math_class_int; c ← set_class_field(cur_val); scan_math_fam_int;
      c ← c + set_family_field(cur_val); scan_usv_num; c ← c + cur_val;
      end
    else if cur_chr = 1 then
        begin  { \Umathcharnum }
        scan_xetex_math_char_int; c ← cur_val;
        end
      else begin scan_fifteen_bit_int;
        c ← set_class_field(cur_val div "1000) + set_family_field((cur_val mod "1000) div "100) +
            (cur_val mod "100);
        end;
  math_given: begin c ← set_class_field(cur_chr div "1000) + set_family_field((cur_chr mod "1000) div
        "100) + (cur_chr mod "100);
    end;
  XeTeX_math_given: c ← cur_chr;
  delim_num: begin if cur_chr = 1 then
      begin  { \Udelimiter <class> <fam> <usv> }
      scan_math_class_int; c ← set_class_field(cur_val); scan_math_fam_int;
      c ← c + set_family_field(cur_val); scan_usv_num; c ← c + cur_val;
      end
    else begin  { \delimiter <27-bit delcode> }
      scan_delimiter_int; c ← cur_val div '10000;  { get the 'small' delimiter field }
      c ← set_class_field(c div "1000) + set_family_field((c mod "1000) div "100) + (c mod "100);
          { and convert it to a XeTeX mathchar code }
      end;
    end;
  othercases ⟨ Scan a subformula enclosed in braces and return 1207 ⟩
  endcases;
  math_type(p) ← math_char; character(p) ← qi(c mod "10000);
  if (is_var_family(c)) ∧ fam_in_range then plane_and_fam_field(p) ← cur_fam
  else plane_and_fam_field(p) ← (math_fam_field(c));
```

plane_and_fam_field(*p*) ← *plane_and_fam_field*(*p*) + (*math_char_field*(*c*) **div** ″10000) ∗ ″100;
exit: **end**;

1206. An active character that is an *outer_call* is allowed here.

⟨ Treat *cur_chr* as an active character 1206 ⟩ ≡
begin *cur_cs* ← *cur_chr* + *active_base*; *cur_cmd* ← *eq_type*(*cur_cs*); *cur_chr* ← *equiv*(*cur_cs*); *x_token*; *back_input*;
end

This code is used in sections 1205 and 1209.

1207. The pointer *p* is placed on *save_stack* while a complex subformula is being scanned.

⟨ Scan a subformula enclosed in braces and **return** 1207 ⟩ ≡
begin *back_input*; *scan_left_brace*;
saved(0) ← *p*; *incr*(*save_ptr*); *push_math*(*math_group*); **return**;
end

This code is used in section 1205.

1208. The simplest math formula is, of course, '`$ $`', when no noads are generated. The next simplest cases involve a single character, e.g., '`$x$`'. Even though such cases may not seem to be very interesting, the reader can perhaps understand how happy the author was when '`$x$`' was first properly typeset by TeX. The code in this section was used.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
mmode + letter, mmode + other_char, mmode + char_given: set_math_char(ho(math_code(cur_chr)));
mmode + char_num: begin scan_char_num; cur_chr ← cur_val; set_math_char(ho(math_code(cur_chr)));
  end;
mmode + math_char_num: if cur_chr = 2 then
    begin { \Umathchar }
    scan_math_class_int; t ← set_class_field(cur_val); scan_math_fam_int; t ← t + set_family_field(cur_val);
    scan_usv_num; t ← t + cur_val; set_math_char(t);
    end
  else if cur_chr = 1 then
      begin { \Umathcharnum }
      scan_xetex_math_char_int; set_math_char(cur_val);
      end
    else begin scan_fifteen_bit_int;
      set_math_char(set_class_field(cur_val div "1000) + set_family_field((cur_val mod "1000) div "100) +
          (cur_val mod "100));
      end;
mmode + math_given: begin set_math_char(set_class_field(cur_chr div "1000) + set_family_field((cur_chr mod
      "1000) div "100) + (cur_chr mod "100));
  end;
mmode + XeTeX_math_given: set_math_char(cur_chr);
mmode + delim_num: begin if cur_chr = 1 then
    begin { \Udelimiter }
    scan_math_class_int; t ← set_class_field(cur_val); scan_math_fam_int; t ← t + set_family_field(cur_val);
    scan_usv_num; t ← t + cur_val; set_math_char(t);
    end
  else begin scan_delimiter_int; cur_val ← cur_val div '10000; { discard the large delimiter code }
    set_math_char(set_class_field(cur_val div "1000) + set_family_field((cur_val mod "1000) div "100) +
        (cur_val mod "100));
    end;
  end;
```

1209. The *set_math_char* procedure creates a new noad appropriate to a given math code, and appends it to the current mlist. However, if the math code is sufficiently large, the *cur_chr* is treated as an active character and nothing is appended.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure set_math_char(c : integer);
  var p: pointer; { the new noad }
    ch: UnicodeScalar;
  begin if is_active_math_char(c) then ⟨ Treat cur_chr as an active character 1206 ⟩
  else begin p ← new_noad; math_type(nucleus(p)) ← math_char; ch ← math_char_field(c);
    character(nucleus(p)) ← qi(ch mod "10000); plane_and_fam_field(nucleus(p)) ← math_fam_field(c);
    if is_var_family(c) then
      begin if fam_in_range then plane_and_fam_field(nucleus(p)) ← cur_fam;
      type(p) ← ord_noad;
      end
    else type(p) ← ord_noad + math_class_field(c);
    plane_and_fam_field(nucleus(p)) ← plane_and_fam_field(nucleus(p)) + (ch div "10000) * "100;
    link(tail) ← p; tail ← p;
    end;
  end;
```

1210. Primitive math operators like `\mathop` and `\underline` are given the command code *math_comp*, supplemented by the noad type that they generate.

```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("mathord", math_comp, ord_noad); primitive("mathop", math_comp, op_noad);
  primitive("mathbin", math_comp, bin_noad); primitive("mathrel", math_comp, rel_noad);
  primitive("mathopen", math_comp, open_noad); primitive("mathclose", math_comp, close_noad);
  primitive("mathpunct", math_comp, punct_noad); primitive("mathinner", math_comp, inner_noad);
  primitive("underline", math_comp, under_noad); primitive("overline", math_comp, over_noad);
  primitive("displaylimits", limit_switch, normal); primitive("limits", limit_switch, limits);
  primitive("nolimits", limit_switch, no_limits);
```

1211.

```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
math_comp: case chr_code of
  ord_noad: print_esc("mathord");
  op_noad: print_esc("mathop");
  bin_noad: print_esc("mathbin");
  rel_noad: print_esc("mathrel");
  open_noad: print_esc("mathopen");
  close_noad: print_esc("mathclose");
  punct_noad: print_esc("mathpunct");
  inner_noad: print_esc("mathinner");
  under_noad: print_esc("underline");
  othercases print_esc("overline")
  endcases;
limit_switch: if chr_code = limits then print_esc("limits")
  else if chr_code = no_limits then print_esc("nolimits")
    else print_esc("displaylimits");
```

1212.

```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
mmode + math_comp: begin tail_append(new_noad); type(tail) ← cur_chr; scan_math(nucleus(tail));
  end;
mmode + limit_switch: math_limit_switch;
```

1213. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure math_limit_switch;
  label exit;
  begin if head ≠ tail then
    if type(tail) = op_noad then
      begin subtype(tail) ← cur_chr; return;
      end;
  print_err("Limit␣controls␣must␣follow␣a␣math␣operator");
  help1("I´m␣ignoring␣this␣misplaced␣\limits␣or␣\nolimits␣command."); error;
exit: end;
```

1214. Delimiter fields of noads are filled in by the *scan_delimiter* routine. The first parameter of this procedure is the *mem* address where the delimiter is to be placed; the second tells if this delimiter follows `\radical` or not.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure scan_delimiter(p : pointer; r : boolean);
  begin if r then
    begin if cur_chr = 1 then
      begin { \Uradical }
      cur_val1 ← "40000000; { extended delimiter code flag }
      scan_math_fam_int; cur_val1 ← cur_val1 + cur_val * "200000; scan_usv_num;
      cur_val ← cur_val1 + cur_val;
      end
    else { radical }
    scan_delimiter_int;
    end
  else begin ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
    case cur_cmd of
    letter, other_char: begin cur_val ← del_code(cur_chr);
      end;
    delim_num: if cur_chr = 1 then
        begin { \Udelimiter }
        cur_val1 ← "40000000; { extended delimiter code flag }
        scan_math_class_int; { discarded }
        scan_math_fam_int; cur_val1 ← cur_val1 + cur_val * "200000; scan_usv_num;
        cur_val ← cur_val1 + cur_val;
        end
      else scan_delimiter_int; { normal delimiter }
    othercases begin cur_val ← −1;
      end;
    endcases;
    end;
  if cur_val < 0 then
    begin ⟨ Report that an invalid delimiter code is being changed to null; set cur_val ← 0 1215 ⟩;
    end;
  if cur_val ≥ "40000000 then
    begin { extended delimiter code, only one size }
    small_plane_and_fam_field(p) ← ((cur_val mod "200000) div "10000) * "100 { plane }
    +(cur_val div "200000) mod "100; { family }
    small_char_field(p) ← qi(cur_val mod "10000); large_plane_and_fam_field(p) ← 0;
    large_char_field(p) ← 0;
    end
  else begin { standard delimiter code, 4-bit families and 8-bit char codes }
    small_plane_and_fam_field(p) ← (cur_val div ´4000000) mod 16;
    small_char_field(p) ← qi((cur_val div ´10000) mod 256);
    large_plane_and_fam_field(p) ← (cur_val div 256) mod 16; large_char_field(p) ← qi(cur_val mod 256);
    end;
  end;
```

1215. ⟨ Report that an invalid delimiter code is being changed to null; set *cur_val* ← 0 1215 ⟩ ≡
```
begin print_err("Missing␣delimiter␣(.␣inserted)");
help6("I␣was␣expecting␣to␣see␣something␣like␣`(´␣or␣`\{´␣or")
("`\}´␣here.␣If␣you␣typed,␣e.g.,␣`{´␣instead␣of␣`\{´,␣you")
("should␣probably␣delete␣the␣`{´␣by␣typing␣`1´␣now,␣so␣that")
("braces␣don´t␣get␣unbalanced.␣Otherwise␣just␣proceed.")
("Acceptable␣delimiters␣are␣characters␣whose␣\delcode␣is")
("nonnegative,␣or␣you␣can␣use␣`\delimiter␣<delimiter␣code>´."); back_error; cur_val ← 0;
end
```
This code is used in section 1214.

1216. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
```
mmode + radical: math_radical;
```

1217. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
```
procedure math_radical;
  begin tail_append(get_node(radical_noad_size)); type(tail) ← radical_noad; subtype(tail) ← normal;
  mem[nucleus(tail)].hh ← empty_field; mem[subscr(tail)].hh ← empty_field;
  mem[supscr(tail)].hh ← empty_field; scan_delimiter(left_delimiter(tail), true); scan_math(nucleus(tail));
  end;
```

1218. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
```
mmode + accent, mmode + math_accent: math_ac;
```

1219. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
```
procedure math_ac;
  var c: integer;
  begin if cur_cmd = accent then ⟨ Complain that the user should have said \mathaccent 1220 ⟩;
  tail_append(get_node(accent_noad_size)); type(tail) ← accent_noad; subtype(tail) ← normal;
  mem[nucleus(tail)].hh ← empty_field; mem[subscr(tail)].hh ← empty_field;
  mem[supscr(tail)].hh ← empty_field; math_type(accent_chr(tail)) ← math_char;
  if cur_chr = 1 then
    begin if scan_keyword("fixed") then subtype(tail) ← fixed_acc
    else if scan_keyword("bottom") then
        begin if scan_keyword("fixed") then subtype(tail) ← bottom_acc + fixed_acc
        else subtype(tail) ← bottom_acc;
        end;
    scan_math_class_int; c ← set_class_field(cur_val); scan_math_fam_int;
    c ← c + set_family_field(cur_val); scan_usv_num; cur_val ← cur_val + c;
    end
  else begin scan_fifteen_bit_int;
    cur_val ← set_class_field(cur_val div "1000) + set_family_field((cur_val mod "1000) div "100) +
        (cur_val mod "100);
    end;
  character(accent_chr(tail)) ← qi(cur_val mod "10000);
  if (is_var_family(cur_val)) ∧ fam_in_range then plane_and_fam_field(accent_chr(tail)) ← cur_fam
  else plane_and_fam_field(accent_chr(tail)) ← math_fam_field(cur_val);
  plane_and_fam_field(accent_chr(tail)) ← plane_and_fam_field(accent_chr(tail)) +
      (math_char_field(cur_val) div "10000) * "100; scan_math(nucleus(tail));
  end;
```

1220. ⟨ Complain that the user should have said `\mathaccent` 1220 ⟩ ≡
begin *print_err*("Please␣use␣"); *print_esc*("mathaccent"); *print*("␣for␣accents␣in␣math␣mode");
help2("I´m␣changing␣\accent␣to␣\mathaccent␣here;␣wish␣me␣luck.")
("(Accents␣are␣not␣the␣same␣in␣formulas␣as␣they␣are␣in␣text.)"); *error*;
end
This code is used in section 1219.

1221. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
mmode + *vcenter*: **begin** *scan_spec*(*vcenter_group*, *false*); *normal_paragraph*; *push_nest*; *mode* ← −*vmode*;
prev_depth ← *ignore_depth*;
if *every_vbox* ≠ *null* **then** *begin_token_list*(*every_vbox*, *every_vbox_text*);
end;

1222. ⟨ Cases of *handle_right_brace* where a *right_brace* triggers a delayed action 1139 ⟩ +≡
vcenter_group: **begin** *end_graf*; *unsave*; *save_ptr* ← *save_ptr* − 2;
p ← *vpack*(*link*(*head*), *saved*(1), *saved*(0)); *pop_nest*; *tail_append*(*new_noad*); *type*(*tail*) ← *vcenter_noad*;
math_type(*nucleus*(*tail*)) ← *sub_box*; *info*(*nucleus*(*tail*)) ← *p*;
end;

1223. The routine that inserts a *style_node* holds no surprises.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive("displaystyle", *math_style*, *display_style*); *primitive*("textstyle", *math_style*, *text_style*);
primitive("scriptstyle", *math_style*, *script_style*);
primitive("scriptscriptstyle", *math_style*, *script_script_style*);

1224. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
math_style: *print_style*(*chr_code*);

1225. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
mmode + *math_style*: *tail_append*(*new_style*(*cur_chr*));
mmode + *non_script*: **begin** *tail_append*(*new_glue*(*zero_glue*)); *subtype*(*tail*) ← *cond_math_glue*;
end;
mmode + *math_choice*: *append_choices*;

1226. The routine that scans the four mlists of a `\mathchoice` is very much like the routine that builds discretionary nodes.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *append_choices*;
begin *tail_append*(*new_choice*); *incr*(*save_ptr*); *saved*(−1) ← 0; *push_math*(*math_choice_group*);
scan_left_brace;
end;

1227. ⟨ Cases of *handle_right_brace* where a *right_brace* triggers a delayed action 1139 ⟩ +≡
math_choice_group: *build_choices*;

1228. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
⟨ Declare the function called *fin_mlist* 1238 ⟩
procedure *build_choices*;
  label *exit*;
  var *p*: *pointer*; { the current mlist }
  begin *unsave*; *p* ← *fin_mlist*(*null*);
  case *saved*(−1) **of**
  0: *display_mlist*(*tail*) ← *p*;
  1: *text_mlist*(*tail*) ← *p*;
  2: *script_mlist*(*tail*) ← *p*;
  3: **begin** *script_script_mlist*(*tail*) ← *p*; *decr*(*save_ptr*); **return**;
    end;
  end; { there are no other cases }
  incr(*saved*(−1)); *push_math*(*math_choice_group*); *scan_left_brace*;
exit: **end**;

1229. Subscripts and superscripts are attached to the previous nucleus by the action procedure called *sub_sup*. We use the facts that *sub_mark* = *sup_mark* + 1 and *subscr*(*p*) = *supscr*(*p*) + 1.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡
mmode + *sub_mark*, *mmode* + *sup_mark*: *sub_sup*;

1230. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
procedure *sub_sup*;
  var *t*: *small_number*; { type of previous sub/superscript }
    p: *pointer*; { field to be filled by *scan_math* }
  begin *t* ← *empty*; *p* ← *null*;
  if *tail* ≠ *head* **then**
    if *scripts_allowed*(*tail*) **then**
      begin *p* ← *supscr*(*tail*) + *cur_cmd* − *sup_mark*; { *supscr* or *subscr* }
      t ← *math_type*(*p*);
      end;
  if (*p* = *null*) ∨ (*t* ≠ *empty*) **then** ⟨ Insert a dummy noad to be sub/superscripted 1231 ⟩;
  scan_math(*p*);
  end;

1231. ⟨ Insert a dummy noad to be sub/superscripted 1231 ⟩ ≡
  begin *tail_append*(*new_noad*); *p* ← *supscr*(*tail*) + *cur_cmd* − *sup_mark*; { *supscr* or *subscr* }
  if *t* ≠ *empty* **then**
    begin if *cur_cmd* = *sup_mark* **then**
      begin *print_err*("Double␣superscript");
      help1("I␣treat␣`x^1^2´␣essentially␣like␣`x^1{}^2´.");
      end
    else begin *print_err*("Double␣subscript");
      help1("I␣treat␣`x_1_2´␣essentially␣like␣`x_1{}_2´.");
      end;
    error;
    end;
  end

This code is used in section 1230.

1232. An operation like '`\over`' causes the current mlist to go into a state of suspended animation: *incompleat_noad* points to a *fraction_noad* that contains the mlist-so-far as its numerator, while the denominator is yet to come. Finally when the mlist is finished, the denominator will go into the incompleat fraction noad, and that noad will become the whole formula, unless it is surrounded by '`\left`' and '`\right`' delimiters.

```
define above_code = 0  { '\above' }
define over_code = 1  { '\over' }
define atop_code = 2  { '\atop' }
define delimited_code = 3  { '\abovewithdelims', etc. }
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("above", above, above_code);
  primitive("over", above, over_code);
  primitive("atop", above, atop_code);
  primitive("abovewithdelims", above, delimited_code + above_code);
  primitive("overwithdelims", above, delimited_code + over_code);
  primitive("atopwithdelims", above, delimited_code + atop_code);
```

1233.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
above: case chr_code of
  over_code: print_esc("over");
  atop_code: print_esc("atop");
  delimited_code + above_code: print_esc("abovewithdelims");
  delimited_code + over_code: print_esc("overwithdelims");
  delimited_code + atop_code: print_esc("atopwithdelims");
  othercases print_esc("above")
  endcases;
```

1234.
```
⟨ Cases of main_control that build boxes and lists 1110 ⟩ +≡
mmode + above: math_fraction;
```

1235.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure math_fraction;
  var c: small_number;  { the type of generalized fraction we are scanning }
  begin c ← cur_chr;
  if incompleat_noad ≠ null then
    ⟨ Ignore the fraction operation and complain about this ambiguous case 1237 ⟩
  else begin incompleat_noad ← get_node(fraction_noad_size); type(incompleat_noad) ← fraction_noad;
    subtype(incompleat_noad) ← normal; math_type(numerator(incompleat_noad)) ← sub_mlist;
    info(numerator(incompleat_noad)) ← link(head);
    mem[denominator(incompleat_noad)].hh ← empty_field;
    mem[left_delimiter(incompleat_noad)].qqqq ← null_delimiter;
    mem[right_delimiter(incompleat_noad)].qqqq ← null_delimiter;
    link(head) ← null; tail ← head; ⟨ Use code c to distinguish between generalized fractions 1236 ⟩;
    end;
  end;
```

1236. ⟨ Use code c to distinguish between generalized fractions 1236 ⟩ ≡

```
if c ≥ delimited_code then
  begin scan_delimiter(left_delimiter(incompleat_noad), false);
  scan_delimiter(right_delimiter(incompleat_noad), false);
  end;
case c mod delimited_code of
above_code: begin scan_normal_dimen; thickness(incompleat_noad) ← cur_val;
  end;
over_code: thickness(incompleat_noad) ← default_code;
atop_code: thickness(incompleat_noad) ← 0;
end { there are no other cases }
```

This code is used in section 1235.

1237. ⟨ Ignore the fraction operation and complain about this ambiguous case 1237 ⟩ ≡

```
begin if c ≥ delimited_code then
  begin scan_delimiter(garbage, false); scan_delimiter(garbage, false);
  end;
if c mod delimited_code = above_code then scan_normal_dimen;
print_err("Ambiguous;␣you␣need␣another␣{␣and␣}");
help3("I´m␣ignoring␣this␣fraction␣specification,␣since␣I␣don´t")
("know␣whether␣a␣construction␣like␣`x␣\over␣y␣\over␣z´")
("means␣`{x␣\over␣y}␣\over␣z´␣or␣`x␣\over␣{y␣\over␣z}´.");  error;
end
```

This code is used in section 1235.

1238. At the end of a math formula or subformula, the *fin_mlist* routine is called upon to return a pointer to the newly completed mlist, and to pop the nest back to the enclosing semantic level. The parameter to *fin_mlist*, if not null, points to a *right_noad* that ends the current mlist; this *right_noad* has not yet been appended.

⟨ Declare the function called *fin_mlist* 1238 ⟩ ≡

```
function fin_mlist(p : pointer): pointer;
  var q: pointer;   { the mlist to return }
  begin if incompleat_noad ≠ null then ⟨ Compleat the incompleat noad 1239 ⟩
  else begin link(tail) ← p; q ← link(head);
    end;
  pop_nest; fin_mlist ← q;
  end;
```

This code is used in section 1228.

1239. ⟨ Compleat the incompleat noad 1239 ⟩ ≡

```
begin math_type(denominator(incompleat_noad)) ← sub_mlist;
info(denominator(incompleat_noad)) ← link(head);
if p = null then q ← incompleat_noad
else begin q ← info(numerator(incompleat_noad));
  if (type(q) ≠ left_noad) ∨ (delim_ptr = null) then confusion("right");
  info(numerator(incompleat_noad)) ← link(delim_ptr); link(delim_ptr) ← incompleat_noad;
  link(incompleat_noad) ← p;
  end;
end
```

This code is used in section 1238.

1240. Now at last we're ready to see what happens when a right brace occurs in a math formula. Two special cases are simplified here: Braces are effectively removed when they surround a single Ord without sub/superscripts, or when they surround an accent that is the nucleus of an Ord atom.

```
⟨ Cases of handle_right_brace where a right_brace triggers a delayed action 1139 ⟩ +≡
math_group: begin unsave; decr(save_ptr);
  math_type(saved(0)) ← sub_mlist; p ← fin_mlist(null); info(saved(0)) ← p;
  if p ≠ null then
    if link(p) = null then
      if type(p) = ord_noad then
        begin if math_type(subscr(p)) = empty then
          if math_type(supscr(p)) = empty then
            begin mem[saved(0)].hh ← mem[nucleus(p)].hh; free_node(p, noad_size);
            end;
        end
      else if type(p) = accent_noad then
          if saved(0) = nucleus(tail) then
            if type(tail) = ord_noad then ⟨ Replace the tail of the list by p 1241 ⟩;
  end;
```

1241. ⟨ Replace the tail of the list by p 1241 ⟩ ≡

```
  begin q ← head;
  while link(q) ≠ tail do q ← link(q);
  link(q) ← p; free_node(tail, noad_size); tail ← p;
  end
```

This code is used in section 1240.

1242. We have dealt with all constructions of math mode except '`\left`' and '`\right`', so the picture is completed by the following sections of the program.

```
⟨ Put each of TEX's primitives into the hash table 252 ⟩ +≡
  primitive("left", left_right, left_noad); primitive("right", left_right, right_noad);
  text(frozen_right) ← "right"; eqtb[frozen_right] ← eqtb[cur_val];
```

1243. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
left_right: if chr_code = left_noad then print_esc("left")
  ⟨ Cases of left_right for print_cmd_chr 1508 ⟩
else print_esc("right");
```

1244. ⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡

```
mmode + left_right: math_left_right;
```

1245. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure math_left_right;
  var t: small_number;  { left_noad or right_noad }
    p: pointer;  { new noad }
    q: pointer;  { resulting mlist }
  begin t ← cur_chr;
  if (t ≠ left_noad) ∧ (cur_group ≠ math_left_group) then ⟨ Try to recover from mismatched \right 1246 ⟩
  else begin p ← new_noad; type(p) ← t; scan_delimiter(delimiter(p), false);
    if t = middle_noad then
      begin type(p) ← right_noad; subtype(p) ← middle_noad;
      end;
    if t = left_noad then q ← p
    else begin q ← fin_mlist(p); unsave;  { end of math_left_group }
      end;
    if t ≠ right_noad then
      begin push_math(math_left_group); link(head) ← q; tail ← p; delim_ptr ← p;
      end
    else begin tail_append(new_noad); type(tail) ← inner_noad; math_type(nucleus(tail)) ← sub_mlist;
      info(nucleus(tail)) ← q;
      end;
    end;
  end;
```

1246. ⟨ Try to recover from mismatched `\right` 1246 ⟩ ≡

```
  begin if cur_group = math_shift_group then
    begin scan_delimiter(garbage, false); print_err("Extra␣");
    if t = middle_noad then
      begin print_esc("middle"); help1("I´m␣ignoring␣a␣\middle␣that␣had␣no␣matching␣\left.");
      end
    else begin print_esc("right"); help1("I´m␣ignoring␣a␣\right␣that␣had␣no␣matching␣\left.");
      end;
    error;
    end
  else off_save;
  end
```

This code is used in section 1245.

1247. Here is the only way out of math mode.

⟨ Cases of *main_control* that build boxes and lists 1110 ⟩ +≡

```
mmode + math_shift: if cur_group = math_shift_group then after_math
  else off_save;
```

1248. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
⟨ Declare subprocedures for *after_math* 1555 ⟩
procedure *after_math*;
  var *l*: *boolean*; { '`\leqno`' instead of '`\eqno`' }
    danger: *boolean*; { not enough symbol fonts are present }
    m: *integer*; { *mmode* or −*mmode* }
    p: *pointer*; { the formula }
    a: *pointer*; { box containing equation number }
    ⟨ Local variables for finishing a displayed formula 1252 ⟩
  begin *danger* ← *false*; ⟨ Retrieve the prototype box 1553 ⟩;
  ⟨ Check that the necessary fonts for math symbols are present; if not, flush the current math lists and set *danger* ← *true* 1249 ⟩;
  m ← *mode*; *l* ← *false*; *p* ← *fin_mlist*(*null*); { this pops the nest }
  if *mode* = −*m* **then** { end of equation number }
    begin ⟨ Check that another `$` follows 1251 ⟩;
    cur_mlist ← *p*; *cur_style* ← *text_style*; *mlist_penalties* ← *false*; *mlist_to_hlist*;
    a ← *hpack*(*link*(*temp_head*), *natural*); *set_box_lr*(*a*)(*dlist*); *unsave*; *decr*(*save_ptr*);
      { now *cur_group* = *math_shift_group* }
    if *saved*(0) = 1 **then** *l* ← *true*;
    danger ← *false*; ⟨ Retrieve the prototype box 1553 ⟩;
    ⟨ Check that the necessary fonts for math symbols are present; if not, flush the current math lists and set *danger* ← *true* 1249 ⟩;
    m ← *mode*; *p* ← *fin_mlist*(*null*);
    end
  else *a* ← *null*;
  if *m* < 0 **then** ⟨ Finish math in text 1250 ⟩
  else begin if *a* = *null* **then** ⟨ Check that another `$` follows 1251 ⟩;
    ⟨ Finish displayed math 1253 ⟩;
    end;
  end;

1249. ⟨ Check that the necessary fonts for math symbols are present; if not, flush the current math lists and set *danger* ← *true* 1249 ⟩ ≡

```
if ((font_params[fam_fnt(2 + text_size)] < total_mathsy_params) ∧ (¬is_new_mathfont(fam_fnt(2 +
    text_size)))) ∨ ((font_params[fam_fnt(2 + script_size)] < total_mathsy_params) ∧
    (¬is_new_mathfont(fam_fnt(2 + script_size)))) ∨ ((font_params[fam_fnt(2 + script_script_size)] <
    total_mathsy_params) ∧ (¬is_new_mathfont(fam_fnt(2 + script_script_size)))) then
  begin print_err("Math␣formula␣deleted:␣Insufficient␣symbol␣fonts");
  help3("Sorry,␣but␣I␣can´t␣typeset␣math␣unless␣\textfont␣2")
  ("and␣\scriptfont␣2␣and␣\scriptscriptfont␣2␣have␣all")
  ("the␣\fontdimen␣values␣needed␣in␣math␣symbol␣fonts."); error; flush_math; danger ← true;
  end
else if ((font_params[fam_fnt(3 + text_size)] < total_mathex_params) ∧ (¬is_new_mathfont(fam_fnt(3 +
    text_size)))) ∨ ((font_params[fam_fnt(3 + script_size)] < total_mathex_params) ∧
    (¬is_new_mathfont(fam_fnt(3 + script_size)))) ∨ ((font_params[fam_fnt(3 + script_script_size)] <
    total_mathex_params) ∧ (¬is_new_mathfont(fam_fnt(3 + script_script_size)))) then
  begin print_err("Math␣formula␣deleted:␣Insufficient␣extension␣fonts");
  help3("Sorry,␣but␣I␣can´t␣typeset␣math␣unless␣\textfont␣3")
  ("and␣\scriptfont␣3␣and␣\scriptscriptfont␣3␣have␣all")
  ("the␣\fontdimen␣values␣needed␣in␣math␣extension␣fonts."); error; flush_math;
  danger ← true;
  end
```

This code is used in sections 1248 and 1248.

1250. The *unsave* is done after everything else here; hence an appearance of '`\mathsurround`' inside of '`$...$`' affects the spacing at these particular `$`'s. This is consistent with the conventions of '`$$...$$`', since '`\abovedisplayskip`' inside a display affects the space above that display.

⟨ Finish math in text 1250 ⟩ ≡

```
begin tail_append(new_math(math_surround, before)); cur_mlist ← p; cur_style ← text_style;
mlist_penalties ← (mode > 0); mlist_to_hlist; link(tail) ← link(temp_head);
while link(tail) ≠ null do tail ← link(tail);
tail_append(new_math(math_surround, after)); space_factor ← 1000; unsave;
end
```

This code is used in section 1248.

1251. TeX gets to the following part of the program when the first '`$`' ending a display has been scanned.

⟨ Check that another `$` follows 1251 ⟩ ≡

```
begin get_x_token;
if cur_cmd ≠ math_shift then
  begin print_err("Display␣math␣should␣end␣with␣$$");
  help2("The␣`$´␣that␣I␣just␣saw␣supposedly␣matches␣a␣previous␣`$$´.")
  ("So␣I␣shall␣assume␣that␣you␣typed␣`$$´␣both␣times."); back_error;
  end;
end
```

This code is used in sections 1248, 1248, and 1260.

1252. We have saved the worst for last: The fussiest part of math mode processing occurs when a displayed formula is being centered and placed with an optional equation number.

```
⟨ Local variables for finishing a displayed formula 1252 ⟩ ≡
b: pointer; { box containing the equation }
w: scaled; { width of the equation }
z: scaled; { width of the line }
e: scaled; { width of equation number }
q: scaled; { width of equation number plus space to separate from equation }
d: scaled; { displacement of equation in the line }
s: scaled; { move the line right this much }
g1, g2: small_number; { glue parameter codes for before and after }
r: pointer; { kern node used to position the display }
t: pointer; { tail of adjustment list }
pre_t: pointer; { tail of pre-adjustment list }
```

See also section 1552.

This code is used in section 1248.

1253. At this time p points to the mlist for the formula; a is either *null* or it points to a box containing the equation number; and we are in vertical mode (or internal vertical mode).

```
⟨ Finish displayed math 1253 ⟩ ≡
  cur_mlist ← p; cur_style ← display_style; mlist_penalties ← false; mlist_to_hlist; p ← link(temp_head);
  adjust_tail ← adjust_head; pre_adjust_tail ← pre_adjust_head; b ← hpack(p, natural); p ← list_ptr(b);
  t ← adjust_tail; adjust_tail ← null;
  pre_t ← pre_adjust_tail; pre_adjust_tail ← null;
  w ← width(b); z ← display_width; s ← display_indent;
  if pre_display_direction < 0 then s ← −s − z;
  if (a = null) ∨ danger then
    begin e ← 0; q ← 0;
    end
  else begin e ← width(a); q ← e + math_quad(text_size);
    end;
  if w + q > z then ⟨ Squeeze the equation as much as possible; if there is an equation number that should
      go on a separate line by itself, set e ← 0 1255 ⟩;
  ⟨ Determine the displacement, d, of the left edge of the equation, with respect to the line size z, assuming
      that l = false 1256 ⟩;
  ⟨ Append the glue or equation number preceding the display 1257 ⟩;
  ⟨ Append the display and perhaps also the equation number 1258 ⟩;
  ⟨ Append the glue or equation number following the display 1259 ⟩;
  ⟨ Flush the prototype box 1554 ⟩;
  resume_after_display
```

This code is used in section 1248.

1254.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure resume_after_display;
  begin if cur_group ≠ math_shift_group then confusion("display");
  unsave; prev_graf ← prev_graf + 3; push_nest; mode ← hmode; space_factor ← 1000; set_cur_lang;
  clang ← cur_lang;
  prev_graf ← (norm_min(left_hyphen_min) * ´100 + norm_min(right_hyphen_min)) * ´200000 + cur_lang;
  ⟨ Scan an optional space 477 ⟩;
  if nest_ptr = 1 then build_page;
  end;
```

1255. The user can force the equation number to go on a separate line by causing its width to be zero.

```
⟨ Squeeze the equation as much as possible; if there is an equation number that should go on a separate line
      by itself, set e ← 0 1255 ⟩ ≡
  begin if (e ≠ 0) ∧ ((w − total_shrink[normal] + q ≤ z) ∨
        (total_shrink[fil] ≠ 0) ∨ (total_shrink[fill] ≠ 0) ∨ (total_shrink[filll] ≠ 0)) then
    begin free_node(b, box_node_size); b ← hpack(p, z − q, exactly);
    end
  else begin e ← 0;
    if w > z then
      begin free_node(b, box_node_size); b ← hpack(p, z, exactly);
      end;
    end;
  w ← width(b);
  end
```

This code is used in section 1253.

1256. We try first to center the display without regard to the existence of the equation number. If that would make it too close (where "too close" means that the space between display and equation number is less than the width of the equation number), we either center it in the remaining space or move it as far from the equation number as possible. The latter alternative is taken only if the display begins with glue, since we assume that the user put glue there to control the spacing precisely.

```
⟨ Determine the displacement, d, of the left edge of the equation, with respect to the line size z, assuming
      that l = false 1256 ⟩ ≡
  set_box_lr(b)(dlist); d ← half(z − w);
  if (e > 0) ∧ (d < 2 ∗ e) then { too close }
    begin d ← half(z − w − e);
    if p ≠ null then
      if ¬is_char_node(p) then
        if type(p) = glue_node then d ← 0;
    end
```

This code is used in section 1253.

1257. If the equation number is set on a line by itself, either before or after the formula, we append an infinite penalty so that no page break will separate the display from its number; and we use the same size and displacement for all three potential lines of the display, even though '`\parshape`' may specify them differently.

```
⟨ Append the glue or equation number preceding the display 1257 ⟩ ≡
  tail_append(new_penalty(pre_display_penalty));
  if (d + s ≤ pre_display_size) ∨ l then { not enough clearance }
    begin g1 ← above_display_skip_code; g2 ← below_display_skip_code;
    end
  else begin g1 ← above_display_short_skip_code; g2 ← below_display_short_skip_code;
    end;
  if l ∧ (e = 0) then { it follows that type(a) = hlist_node }
    begin app_display(j, a, 0); tail_append(new_penalty(inf_penalty));
    end
  else tail_append(new_param_glue(g1))
```

This code is used in section 1253.

1258. ⟨ Append the display and perhaps also the equation number 1258 ⟩ ≡

```
if e ≠ 0 then
  begin r ← new_kern(z − w − e − d);
  if l then
    begin link(a) ← r; link(r) ← b; b ← a; d ← 0;
    end
  else begin link(b) ← r; link(r) ← a;
    end;
  b ← hpack(b, natural);
  end;
app_display(j, b, d)
```

This code is used in section 1253.

1259. ⟨ Append the glue or equation number following the display 1259 ⟩ ≡

```
if (a ≠ null) ∧ (e = 0) ∧ ¬l then
  begin tail_append(new_penalty(inf_penalty)); app_display(j, a, z − width(a)); g2 ← 0;
  end;
if t ≠ adjust_head then { migrating material comes after equation number }
  begin link(tail) ← link(adjust_head); tail ← t;
  end;
if pre_t ≠ pre_adjust_head then
  begin link(tail) ← link(pre_adjust_head); tail ← pre_t;
  end;
tail_append(new_penalty(post_display_penalty));
if g2 > 0 then tail_append(new_param_glue(g2))
```

This code is used in section 1253.

1260. When `\halign` appears in a display, the alignment routines operate essentially as they do in vertical mode. Then the following program is activated, with p and q pointing to the beginning and end of the resulting list, and with *aux_save* holding the *prev_depth* value.

⟨ Finish an alignment in a display 1260 ⟩ ≡

```
begin do_assignments;
if cur_cmd ≠ math_shift then ⟨ Pontificate about improper alignment in display 1261 ⟩
else ⟨ Check that another $ follows 1251 ⟩;
flush_node_list(LR_box); pop_nest; tail_append(new_penalty(pre_display_penalty));
tail_append(new_param_glue(above_display_skip_code)); link(tail) ← p;
if p ≠ null then tail ← q;
tail_append(new_penalty(post_display_penalty)); tail_append(new_param_glue(below_display_skip_code));
prev_depth ← aux_save.sc; resume_after_display;
end
```

This code is used in section 860.

1261. ⟨ Pontificate about improper alignment in display 1261 ⟩ ≡

```
begin print_err("Missing␣$$␣inserted");
help2("Displays␣can␣use␣special␣alignments␣(like␣\eqalignno)")
("only␣if␣nothing␣but␣the␣alignment␣itself␣is␣between␣$$´s.");  back_error;
end
```

This code is used in section 1260.

1262. Mode-independent processing. The long *main_control* procedure has now been fully specified, except for certain activities that are independent of the current mode. These activities do not change the current vlist or hlist or mlist; if they change anything, it is the value of a parameter or the meaning of a control sequence.

Assignments to values in *eqtb* can be global or local. Furthermore, a control sequence can be defined to be '`\long`', '`\protected`', or '`\outer`', and it might or might not be expanded. The prefixes '`\global`', '`\long`', '`\protected`', and '`\outer`' can occur in any order. Therefore we assign binary numeric codes, making it possible to accumulate the union of all specified prefixes by adding the corresponding codes. (Pascal's **set** operations could also have been used.)

```
⟨ Put each of TEX's primitives into the hash table 252 ⟩ +≡
  primitive("long", prefix, 1); primitive("outer", prefix, 2); primitive("global", prefix, 4);
  primitive("def", def, 0); primitive("gdef", def, 1); primitive("edef", def, 2); primitive("xdef", def, 3);
```

1263.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
prefix: if chr_code = 1 then print_esc("long")
  else if chr_code = 2 then print_esc("outer")
  ⟨ Cases of prefix for print_cmd_chr 1582 ⟩
else print_esc("global");
def: if chr_code = 0 then print_esc("def")
  else if chr_code = 1 then print_esc("gdef")
    else if chr_code = 2 then print_esc("edef")
      else print_esc("xdef");
```

1264. Every prefix, and every command code that might or might not be prefixed, calls the action procedure *prefixed_command*. This routine accumulates a sequence of prefixes until coming to a non-prefix, then it carries out the command.

```
⟨ Cases of main_control that don't depend on mode 1264 ⟩ ≡
any_mode(toks_register), any_mode(assign_toks), any_mode(assign_int), any_mode(assign_dimen),
    any_mode(assign_glue), any_mode(assign_mu_glue), any_mode(assign_font_dimen),
    any_mode(assign_font_int), any_mode(set_aux), any_mode(set_prev_graf), any_mode(set_page_dimen),
    any_mode(set_page_int), any_mode(set_box_dimen), any_mode(set_shape), any_mode(def_code),
    any_mode(XeTeX_def_code), any_mode(def_family), any_mode(set_font), any_mode(def_font),
    any_mode(register), any_mode(advance), any_mode(multiply), any_mode(divide), any_mode(prefix),
    any_mode(let), any_mode(shorthand_def), any_mode(read_to_cs), any_mode(def), any_mode(set_box),
    any_mode(hyph_data), any_mode(set_interaction): prefixed_command;
```

See also sections 1322, 1325, 1328, 1330, 1339, and 1344.

This code is used in section 1099.

1265. If the user says, e.g., '`\global\global`', the redundancy is silently accepted.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
⟨ Declare subprocedures for prefixed_command 1269 ⟩
procedure prefixed_command;
  label done, exit;
  var a: small_number; { accumulated prefix codes so far }
    f: internal_font_number; { identifies a font }
    j: halfword; { index into a \parshape specification }
    k: font_index; { index into font_info }
    p, q: pointer; { for temporary short-term use }
    n: integer; { ditto }
    e: boolean; { should a definition be expanded? or was \let not done? }
  begin a ← 0;
  while cur_cmd = prefix do
    begin if ¬odd(a div cur_chr) then a ← a + cur_chr;
    ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
    if cur_cmd ≤ max_non_prefixed_command then ⟨ Discard erroneous prefixes and return 1266 ⟩;
    if tracing_commands > 2 then
      if eTeX_ex then show_cur_cmd_chr;
    end;
  ⟨ Discard the prefixes \long and \outer if they are irrelevant 1267 ⟩;
  ⟨ Adjust for the setting of \globaldefs 1268 ⟩;
  case cur_cmd of
  ⟨ Assignments 1271 ⟩
  othercases confusion("prefix")
  endcases;
done: ⟨ Insert a token saved by \afterassignment, if any 1323 ⟩;
exit: end;
```

1266. ⟨ Discard erroneous prefixes and **return** 1266 ⟩ ≡

```
  begin print_err("You can't use a prefix with `"); print_cmd_chr(cur_cmd, cur_chr);
  print_char("'"); help1("I'll pretend you didn't say \long or \outer or \global.");
  if eTeX_ex then
    help_line[0] ← "I'll pretend you didn't say \long or \outer or \global or \protected.";
  back_error; return;
  end
```

This code is used in section 1265.

1267. ⟨ Discard the prefixes `\long` and `\outer` if they are irrelevant 1267 ⟩ ≡

```
if a ≥ 8 then
  begin j ← protected_token; a ← a − 8;
  end
else j ← 0;
if (cur_cmd ≠ def) ∧ ((a mod 4 ≠ 0) ∨ (j ≠ 0)) then
  begin print_err("You␣can´t␣use␣`"); print_esc("long"); print("´␣or␣`"); print_esc("outer");
  help1("I´ll␣pretend␣you␣didn´t␣say␣\long␣or␣\outer␣here.");
  if eTeX_ex then
    begin help_line[0] ← "I´ll␣pretend␣you␣didn´t␣say␣\long␣or␣\outer␣or␣\protected␣here.";
    print("´␣or␣`"); print_esc("protected");
    end;
  print("´␣with␣`"); print_cmd_chr(cur_cmd, cur_chr); print_char("´"); error;
  end
```

This code is used in section 1265.

1268. The previous routine does not have to adjust a so that a **mod** $4 = 0$, since the following routines test for the `\global` prefix as follows.

```
define global ≡ (a ≥ 4)
define define(#) ≡
          if global then geq_define(#) else eq_define(#)
define word_define(#) ≡
          if global then geq_word_define(#) else eq_word_define(#)
define word_define1(#) ≡
          if global then geq_word_define1(#) else eq_word_define1(#)
```

⟨ Adjust for the setting of `\globaldefs` 1268 ⟩ ≡

```
if global_defs ≠ 0 then
  if global_defs < 0 then
    begin if global then a ← a − 4;
    end
  else begin if ¬global then a ← a + 4;
    end
```

This code is used in section 1265.

1269. When a control sequence is to be defined, by `\def` or `\let` or something similar, the *get_r_token* routine will substitute a special control sequence for a token that is not redefinable.

```
⟨ Declare subprocedures for prefixed_command 1269 ⟩ ≡
procedure get_r_token;
  label restart;
  begin restart: repeat get_token;
  until cur_tok ≠ space_token;
  if (cur_cs = 0) ∨ (cur_cs > frozen_control_sequence) then
    begin print_err("Missing␣control␣sequence␣inserted");
    help5("Please␣don't␣say␣`\def␣cs{...}',␣say␣`\def\cs{...}'.")
    ("I've␣inserted␣an␣inaccessible␣control␣sequence␣so␣that␣your")
    ("definition␣will␣be␣completed␣without␣mixing␣me␣up␣too␣badly.")
    ("You␣can␣recover␣graciously␣from␣this␣error,␣if␣you're")
    ("careful;␣see␣exercise␣27.2␣in␣The␣TeXbook.");
    if cur_cs = 0 then back_input;
    cur_tok ← cs_token_flag + frozen_protection; ins_error; goto restart;
    end;
  end;
```

See also sections 1283, 1290, 1297, 1298, 1299, 1300, 1301, 1311, and 1319.

This code is used in section 1265.

1270.
```
⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
  text(frozen_protection) ← "inaccessible";
```

1271. Here's an example of the way many of the following routines operate. (Unfortunately, they aren't all as simple as this.)

```
⟨ Assignments 1271 ⟩ ≡
set_font: define(cur_font_loc, data, cur_chr);
```

See also sections 1272, 1275, 1278, 1279, 1280, 1282, 1286, 1288, 1289, 1295, 1296, 1302, 1306, 1307, 1310, and 1318.

This code is used in section 1265.

1272. When a *def* command has been scanned, *cur_chr* is odd if the definition is supposed to be global, and *cur_chr* ≥ 2 if the definition is supposed to be expanded.

```
⟨ Assignments 1271 ⟩ +≡
def: begin if odd(cur_chr) ∧ ¬global ∧ (global_defs ≥ 0) then a ← a + 4;
  e ← (cur_chr ≥ 2); get_r_token; p ← cur_cs; q ← scan_toks(true, e);
  if j ≠ 0 then
    begin q ← get_avail; info(q) ← j; link(q) ← link(def_ref); link(def_ref) ← q;
    end;
  define(p, call + (a mod 4), def_ref);
  end;
```

1273. Both `\let` and `\futurelet` share the command code *let*.

```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("let", let, normal);
  primitive("futurelet", let, normal + 1);
```

1274.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
let: if chr_code ≠ normal then print_esc("futurelet") else print_esc("let");
```

1275. ⟨ Assignments 1271 ⟩ +≡

```
let: begin n ← cur_chr; get_r_token; p ← cur_cs;
  if n = normal then
    begin repeat get_token;
    until cur_cmd ≠ spacer;
    if cur_tok = other_token + "=" then
      begin get_token;
      if cur_cmd = spacer then get_token;
      end;
    end
  else begin get_token; q ← cur_tok; get_token; back_input; cur_tok ← q; back_input;
        { look ahead, then back up }
    end; { note that back_input doesn't affect cur_cmd, cur_chr }
  if cur_cmd ≥ call then add_token_ref(cur_chr)
  else if (cur_cmd = register) ∨ (cur_cmd = toks_register) then
      if (cur_chr < mem_bot) ∨ (cur_chr > lo_mem_stat_max) then add_sa_ref(cur_chr);
  define(p, cur_cmd, cur_chr);
  end;
```

1276. A `\chardef` creates a control sequence whose *cmd* is *char_given*; a `\mathchardef` creates a control sequence whose *cmd* is *math_given*; and the corresponding *chr* is the character code or math code. A `\countdef` or `\dimendef` or `\skipdef` or `\muskipdef` creates a control sequence whose *cmd* is *assign_int* or ... or *assign_mu_glue*, and the corresponding *chr* is the *eqtb* location of the internal register in question.

```
  define char_def_code = 0   { shorthand_def for \chardef }
  define math_char_def_code = 1   { shorthand_def for \mathchardef }
  define count_def_code = 2   { shorthand_def for \countdef }
  define dimen_def_code = 3   { shorthand_def for \dimendef }
  define skip_def_code = 4   { shorthand_def for \skipdef }
  define mu_skip_def_code = 5   { shorthand_def for \muskipdef }
  define toks_def_code = 6   { shorthand_def for \toksdef }
  define XeTeX_math_char_num_def_code = 8
  define XeTeX_math_char_def_code = 9
```

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡

```
  primitive("chardef", shorthand_def, char_def_code);
  primitive("mathchardef", shorthand_def, math_char_def_code);
  primitive("XeTeXmathcharnumdef", shorthand_def, XeTeX_math_char_num_def_code);
  primitive("Umathcharnumdef", shorthand_def, XeTeX_math_char_num_def_code);
  primitive("XeTeXmathchardef", shorthand_def, XeTeX_math_char_def_code);
  primitive("Umathchardef", shorthand_def, XeTeX_math_char_def_code);
  primitive("countdef", shorthand_def, count_def_code);
  primitive("dimendef", shorthand_def, dimen_def_code);
  primitive("skipdef", shorthand_def, skip_def_code);
  primitive("muskipdef", shorthand_def, mu_skip_def_code);
  primitive("toksdef", shorthand_def, toks_def_code);
```

1277. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
shorthand_def: case chr_code of
  char_def_code: print_esc("chardef");
  math_char_def_code: print_esc("mathchardef");
  XeTeX_math_char_def_code: print_esc("Umathchardef");
  XeTeX_math_char_num_def_code: print_esc("Umathcharnumdef");
  count_def_code: print_esc("countdef");
  dimen_def_code: print_esc("dimendef");
  skip_def_code: print_esc("skipdef");
  mu_skip_def_code: print_esc("muskipdef");
  othercases print_esc("toksdef")
  endcases;
char_given: begin print_esc("char"); print_hex(chr_code);
  end;
math_given: begin print_esc("mathchar"); print_hex(chr_code);
  end;
XeTeX_math_given: begin print_esc("Umathchar"); print_hex(math_class_field(chr_code));
  print_hex(math_fam_field(chr_code)); print_hex(math_char_field(chr_code));
  end;
```

1278. We temporarily define p to be *relax*, so that an occurrence of p while scanning the definition will simply stop the scanning instead of producing an "undefined control sequence" error or expanding the previous meaning. This allows, for instance, '`\chardef\foo=123\foo`'.

```
⟨ Assignments 1271 ⟩ +≡
shorthand_def: begin n ← cur_chr; get_r_token; p ← cur_cs; define(p, relax, 256); scan_optional_equals;
  case n of
  char_def_code: begin scan_usv_num; define(p, char_given, cur_val);
    end;
  math_char_def_code: begin scan_fifteen_bit_int; define(p, math_given, cur_val);
    end;
  XeTeX_math_char_num_def_code: begin scan_xetex_math_char_int; define(p, XeTeX_math_given, cur_val);
    end;
  XeTeX_math_char_def_code: begin scan_math_class_int; n ← set_class_field(cur_val); scan_math_fam_int;
    n ← n + set_family_field(cur_val); scan_usv_num; n ← n + cur_val; define(p, XeTeX_math_given, n);
    end;
  othercases begin scan_register_num;
    if cur_val > 255 then
      begin j ← n − count_def_code; { int_val .. box_val }
      if j > mu_val then j ← tok_val; { int_val .. mu_val or tok_val }
      find_sa_element(j, cur_val, true); add_sa_ref(cur_ptr);
      if j = tok_val then j ← toks_register else j ← register;
      define(p, j, cur_ptr);
      end
    else case n of
      count_def_code: define(p, assign_int, count_base + cur_val);
      dimen_def_code: define(p, assign_dimen, scaled_base + cur_val);
      skip_def_code: define(p, assign_glue, skip_base + cur_val);
      mu_skip_def_code: define(p, assign_mu_glue, mu_skip_base + cur_val);
      toks_def_code: define(p, assign_toks, toks_base + cur_val);
      end; { there are no other cases }
    end
  endcases;
  end;
```

1279.

```
⟨ Assignments 1271 ⟩ +≡
read_to_cs: begin j ← cur_chr; scan_int; n ← cur_val;
  if ¬scan_keyword("to") then
    begin print_err("Missing␣`to´␣inserted");
    help2("You␣should␣have␣said␣`\read<number>␣to␣\cs´.")
    ("I´m␣going␣to␣look␣for␣the␣\cs␣now."); error;
    end;
  get_r_token; p ← cur_cs; read_toks(n, p, j); define(p, call, cur_val);
  end;
```

1280. The token-list parameters, `\output` and `\everypar`, etc., receive their values in the following way. (For safety's sake, we place an enclosing pair of braces around an `\output` list.)

```
⟨ Assignments 1271 ⟩ +≡
toks_register, assign_toks: begin q ← cur_cs; e ← false;
      { just in case, will be set true for sparse array elements }
  if cur_cmd = toks_register then
    if cur_chr = mem_bot then
      begin scan_register_num;
      if cur_val > 255 then
        begin find_sa_element(tok_val, cur_val, true); cur_chr ← cur_ptr; e ← true;
        end
      else cur_chr ← toks_base + cur_val;
      end
    else e ← true
  else if cur_chr = XeTeX_inter_char_loc then
      begin scan_char_class_not_ignored; cur_ptr ← cur_val; scan_char_class_not_ignored;
      find_sa_element(inter_char_val, cur_ptr * char_class_limit + cur_val, true); cur_chr ← cur_ptr;
      e ← true;
      end;
  p ← cur_chr; { p = every_par_loc or output_routine_loc or ... }
  scan_optional_equals; ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
  if cur_cmd ≠ left_brace then ⟨ If the right-hand side is a token parameter or token register, finish the
        assignment and goto done 1281 ⟩;
  back_input; cur_cs ← q; q ← scan_toks(false, false);
  if link(def_ref) = null then { empty list: revert to the default }
    begin sa_define(p, null)(p, undefined_cs, null); free_avail(def_ref);
    end
  else begin if (p = output_routine_loc) ∧ ¬e then { enclose in curlies }
      begin link(q) ← get_avail; q ← link(q); info(q) ← right_brace_token + "}"; q ← get_avail;
      info(q) ← left_brace_token + "{"; link(q) ← link(def_ref); link(def_ref) ← q;
      end;
    sa_define(p, def_ref)(p, call, def_ref);
    end;
  end;
```

1281. ⟨ If the right-hand side is a token parameter or token register, finish the assignment and **goto** *done* 1281 ⟩ ≡

```
if (cur_cmd = toks_register) ∨ (cur_cmd = assign_toks) then
  begin if cur_cmd = toks_register then
    if cur_chr = mem_bot then
      begin scan_register_num;
      if cur_val < 256 then q ← equiv(toks_base + cur_val)
      else begin find_sa_element(tok_val, cur_val, false);
        if cur_ptr = null then q ← null
        else q ← sa_ptr(cur_ptr);
        end;
      end
    else q ← sa_ptr(cur_chr)
  else if cur_chr = XeTeX_inter_char_loc then
      begin scan_char_class_not_ignored; cur_ptr ← cur_val; scan_char_class_not_ignored;
      find_sa_element(inter_char_val, cur_ptr * char_class_limit + cur_val, false);
      if cur_ptr = null then q ← null
      else q ← sa_ptr(cur_ptr);
      end
    else q ← equiv(cur_chr);
  if q = null then sa_define(p, null)(p, undefined_cs, null)
  else begin add_token_ref(q); sa_define(p, q)(p, call, q);
    end;
  goto done;
  end
```

This code is used in section 1280.

1282. Similar routines are used to assign values to the numeric parameters.

⟨ Assignments 1271 ⟩ +≡

```
assign_int: begin p ← cur_chr; scan_optional_equals; scan_int; word_define(p, cur_val);
  end;
assign_dimen: begin p ← cur_chr; scan_optional_equals; scan_normal_dimen; word_define(p, cur_val);
  end;
assign_glue, assign_mu_glue: begin p ← cur_chr; n ← cur_cmd; scan_optional_equals;
  if n = assign_mu_glue then scan_glue(mu_val) else scan_glue(glue_val);
  trap_zero_glue; define(p, glue_ref, cur_val);
  end;
```

1283. When a glue register or parameter becomes zero, it will always point to *zero_glue* because of the following procedure. (Exception: The tabskip glue isn't trapped while preambles are being scanned.)

⟨ Declare subprocedures for *prefixed_command* 1269 ⟩ +≡

```
procedure trap_zero_glue;
  begin if (width(cur_val) = 0) ∧ (stretch(cur_val) = 0) ∧ (shrink(cur_val) = 0) then
    begin add_glue_ref(zero_glue); delete_glue_ref(cur_val); cur_val ← zero_glue;
    end;
  end;
```

1284. The various character code tables are changed by the *def_code* commands, and the font families are declared by *def_family*.

```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("catcode", def_code, cat_code_base); primitive("mathcode", def_code, math_code_base);
  primitive("XeTeXmathcodenum", XeTeX_def_code, math_code_base);
  primitive("Umathcodenum", XeTeX_def_code, math_code_base);
  primitive("XeTeXmathcode", XeTeX_def_code, math_code_base + 1);
  primitive("Umathcode", XeTeX_def_code, math_code_base + 1);
  primitive("lccode", def_code, lc_code_base); primitive("uccode", def_code, uc_code_base);
  primitive("sfcode", def_code, sf_code_base); primitive("XeTeXcharclass", XeTeX_def_code, sf_code_base);
  primitive("delcode", def_code, del_code_base);
  primitive("XeTeXdelcodenum", XeTeX_def_code, del_code_base);
  primitive("Udelcodenum", XeTeX_def_code, del_code_base);
  primitive("XeTeXdelcode", XeTeX_def_code, del_code_base + 1);
  primitive("Udelcode", XeTeX_def_code, del_code_base + 1);
  primitive("textfont", def_family, math_font_base);
  primitive("scriptfont", def_family, math_font_base + script_size);
  primitive("scriptscriptfont", def_family, math_font_base + script_script_size);
```

1285.

```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
def_code: if chr_code = cat_code_base then print_esc("catcode")
  else if chr_code = math_code_base then print_esc("mathcode")
    else if chr_code = lc_code_base then print_esc("lccode")
      else if chr_code = uc_code_base then print_esc("uccode")
        else if chr_code = sf_code_base then print_esc("sfcode")
          else print_esc("delcode");
XeTeX_def_code: if chr_code = sf_code_base then print_esc("XeTeXcharclass")
  else if chr_code = math_code_base then print_esc("Umathcodenum")
    else if chr_code = math_code_base + 1 then print_esc("Umathcode")
      else if chr_code = del_code_base then print_esc("Udelcodenum")
        else print_esc("Udelcode");
def_family: print_size(chr_code − math_font_base);
```

1286. The different types of code values have different legal ranges; the following program is careful to check each case properly.

```
⟨ Assignments 1271 ⟩ +≡
XeTeX_def_code: begin if cur_chr = sf_code_base then
    begin p ← cur_chr; scan_usv_num; p ← p + cur_val; n ← sf_code(cur_val) mod "10000;
    scan_optional_equals; scan_char_class; define(p, data, cur_val * "10000 + n);
    end
  else if cur_chr = math_code_base then
      begin p ← cur_chr; scan_usv_num; p ← p + cur_val; scan_optional_equals;
      scan_xetex_math_char_int; define(p, data, hi(cur_val));
      end
    else if cur_chr = math_code_base + 1 then
        begin p ← cur_chr − 1; scan_usv_num; p ← p + cur_val; scan_optional_equals;
        scan_math_class_int; n ← set_class_field(cur_val); scan_math_fam_int;
        n ← n + set_family_field(cur_val); scan_usv_num; n ← n + cur_val; define(p, data, hi(n));
        end
      else if cur_chr = del_code_base then
          begin p ← cur_chr; scan_usv_num; p ← p + cur_val; scan_optional_equals; scan_int;
              { scan_xetex_del_code_int; !!FIXME!! }
          word_define(p, hi(cur_val));
          end
        else begin p ← cur_chr − 1; scan_usv_num; p ← p + cur_val; scan_optional_equals;
          n ← "40000000;  { extended delimiter code flag }
          scan_math_fam_int; n ← n + cur_val * "200000;  { extended delimiter code family }
          scan_usv_num; n ← n + cur_val;  { extended delimiter code USV }
          word_define(p, hi(n));
          end;
  end;
def_code: begin ⟨ Let n be the largest legal code value, based on cur_chr 1287 ⟩;
  p ← cur_chr; scan_usv_num; p ← p + cur_val; scan_optional_equals; scan_int;
  if ((cur_val < 0) ∧ (p < del_code_base)) ∨ (cur_val > n) then
    begin print_err("Invalid␣code␣("); print_int(cur_val);
    if p < del_code_base then print("),␣should␣be␣in␣the␣range␣0..")
    else print("),␣should␣be␣at␣most␣");
    print_int(n); help1("I´m␣going␣to␣use␣0␣instead␣of␣that␣illegal␣code␣value.");
    error; cur_val ← 0;
    end;
  if p < math_code_base then
    begin if p ≥ sf_code_base then
      begin n ← equiv(p) div "10000; define(p, data, n * "10000 + cur_val);
      end
    else define(p, data, cur_val)
    end
  else if p < del_code_base then
      begin if cur_val = "8000 then cur_val ← active_math_char
      else cur_val ← set_class_field(cur_val div "1000) + set_family_field((cur_val mod "1000) div "100) +
          (cur_val mod "100);  { !!FIXME!! check how this is used }
      define(p, data, hi(cur_val));
      end
    else word_define(p, cur_val);
  end;
```

1287. ⟨ Let n be the largest legal code value, based on *cur_chr* 1287 ⟩ ≡

```
if cur_chr = cat_code_base then n ← max_char_code
else if cur_chr = math_code_base then n ← ´100000
  else if cur_chr = sf_code_base then n ← ´77777
    else if cur_chr = del_code_base then n ← ´77777777
      else n ← biggest_usv
```

This code is used in section 1286.

1288. ⟨ Assignments 1271 ⟩ +≡

```
def_family: begin p ← cur_chr; scan_math_fam_int; p ← p + cur_val; scan_optional_equals;
  scan_font_ident; define(p, data, cur_val);
  end;
```

1289. Next we consider changes to TEX's numeric registers.

⟨ Assignments 1271 ⟩ +≡

```
register, advance, multiply, divide: do_register_command(a);
```

1290. We use the fact that *register* < *advance* < *multiply* < *divide*.

⟨ Declare subprocedures for *prefixed_command* 1269 ⟩ +≡

```
procedure do_register_command(a : small_number);
  label found, exit;
  var l, q, r, s: pointer; { for list manipulation }
    p: int_val .. mu_val; { type of register involved }
    e: boolean; { does l refer to a sparse array element? }
    w: integer; { integer or dimen value of l }
  begin q ← cur_cmd; e ← false; { just in case, will be set true for sparse array elements }
  ⟨ Compute the register location l and its type p; but return if invalid 1291 ⟩;
  if q = register then scan_optional_equals
  else if scan_keyword("by") then do_nothing; { optional 'by' }
  arith_error ← false;
  if q < multiply then ⟨ Compute result of register or advance, put it in cur_val 1292 ⟩
  else ⟨ Compute result of multiply or divide, put it in cur_val 1294 ⟩;
  if arith_error then
    begin print_err("Arithmetic␣overflow");
    help2("I␣can´t␣carry␣out␣that␣multiplication␣or␣division,")
    ("since␣the␣result␣is␣out␣of␣range.");
    if p ≥ glue_val then delete_glue_ref(cur_val);
    error; return;
    end;
  if p < glue_val then sa_word_define(l, cur_val)
  else begin trap_zero_glue; sa_define(l, cur_val)(l, glue_ref, cur_val);
    end;
exit: end;
```

1291. Here we use the fact that the consecutive codes *int_val* .. *mu_val* and *assign_int* .. *assign_mu_glue* correspond to each other nicely.

```
⟨ Compute the register location l and its type p; but return if invalid 1291 ⟩ ≡
  begin if q ≠ register then
    begin get_x_token;
    if (cur_cmd ≥ assign_int) ∧ (cur_cmd ≤ assign_mu_glue) then
      begin l ← cur_chr; p ← cur_cmd − assign_int; goto found;
      end;
    if cur_cmd ≠ register then
      begin print_err("You can't use `"); print_cmd_chr(cur_cmd, cur_chr); print("' after");
      print_cmd_chr(q, 0); help1("I'm forgetting what you said and not changing anything.");
      error; return;
      end;
    end;
  if (cur_chr < mem_bot) ∨ (cur_chr > lo_mem_stat_max) then
    begin l ← cur_chr; p ← sa_type(l); e ← true;
    end
  else begin p ← cur_chr − mem_bot; scan_register_num;
    if cur_val > 255 then
      begin find_sa_element(p, cur_val, true); l ← cur_ptr; e ← true;
      end
    else case p of
      int_val: l ← cur_val + count_base;
      dimen_val: l ← cur_val + scaled_base;
      glue_val: l ← cur_val + skip_base;
      mu_val: l ← cur_val + mu_skip_base;
      end; { there are no other cases }
    end;
  end;
found: if p < glue_val then if e then w ← sa_int(l) else w ← eqtb[l].int
  else if e then s ← sa_ptr(l) else s ← equiv(l)
```

This code is used in section 1290.

1292.

```
⟨ Compute result of register or advance, put it in cur_val 1292 ⟩ ≡
  if p < glue_val then
    begin if p = int_val then scan_int else scan_normal_dimen;
    if q = advance then cur_val ← cur_val + w;
    end
  else begin scan_glue(p);
    if q = advance then ⟨ Compute the sum of two glue specs 1293 ⟩;
    end
```

This code is used in section 1290.

1293. ⟨ Compute the sum of two glue specs 1293 ⟩ ≡

```
begin q ← new_spec(cur_val); r ← s; delete_glue_ref(cur_val); width(q) ← width(q) + width(r);
if stretch(q) = 0 then stretch_order(q) ← normal;
if stretch_order(q) = stretch_order(r) then stretch(q) ← stretch(q) + stretch(r)
else if (stretch_order(q) < stretch_order(r)) ∧ (stretch(r) ≠ 0) then
    begin stretch(q) ← stretch(r); stretch_order(q) ← stretch_order(r);
    end;
if shrink(q) = 0 then shrink_order(q) ← normal;
if shrink_order(q) = shrink_order(r) then shrink(q) ← shrink(q) + shrink(r)
else if (shrink_order(q) < shrink_order(r)) ∧ (shrink(r) ≠ 0) then
    begin shrink(q) ← shrink(r); shrink_order(q) ← shrink_order(r);
    end;
cur_val ← q;
end
```

This code is used in section 1292.

1294. ⟨ Compute result of *multiply* or *divide*, put it in *cur_val* 1294 ⟩ ≡

```
begin scan_int;
if p < glue_val then
  if q = multiply then
    if p = int_val then cur_val ← mult_integers(w, cur_val)
    else cur_val ← nx_plus_y(w, cur_val, 0)
  else cur_val ← x_over_n(w, cur_val)
else begin r ← new_spec(s);
  if q = multiply then
    begin width(r) ← nx_plus_y(width(s), cur_val, 0); stretch(r) ← nx_plus_y(stretch(s), cur_val, 0);
    shrink(r) ← nx_plus_y(shrink(s), cur_val, 0);
    end
  else begin width(r) ← x_over_n(width(s), cur_val); stretch(r) ← x_over_n(stretch(s), cur_val);
    shrink(r) ← x_over_n(shrink(s), cur_val);
    end;
  cur_val ← r;
  end;
end
```

This code is used in section 1290.

1295. The processing of boxes is somewhat different, because we may need to scan and create an entire box before we actually change the value of the old one.

⟨ Assignments 1271 ⟩ +≡

```
set_box: begin scan_register_num;
  if global then n ← global_box_flag + cur_val else n ← box_flag + cur_val;
  scan_optional_equals;
  if set_box_allowed then scan_box(n)
  else begin print_err("Improper␣"); print_esc("setbox");
    help2("Sorry,␣\setbox␣is␣not␣allowed␣after␣\halign␣in␣a␣display,")
    ("or␣between␣\accent␣and␣an␣accented␣character."); error;
    end;
  end;
```

1296. The *space_factor* or *prev_depth* settings are changed when a *set_aux* command is sensed. Similarly, *prev_graf* is changed in the presence of *set_prev_graf*, and *dead_cycles* or *insert_penalties* in the presence of *set_page_int*. These definitions are always global.

When some dimension of a box register is changed, the change isn't exactly global; but TeX does not look at the `\global` switch.

```
⟨ Assignments 1271 ⟩ +≡
set_aux: alter_aux;
set_prev_graf: alter_prev_graf;
set_page_dimen: alter_page_so_far;
set_page_int: alter_integer;
set_box_dimen: alter_box_dimen;
```

1297.
```
⟨ Declare subprocedures for prefixed_command 1269 ⟩ +≡
procedure alter_aux;
  var c: halfword;  { hmode or vmode }
  begin if cur_chr ≠ abs(mode) then report_illegal_case
  else begin c ← cur_chr; scan_optional_equals;
    if c = vmode then
      begin scan_normal_dimen; prev_depth ← cur_val;
      end
    else begin scan_int;
      if (cur_val ≤ 0) ∨ (cur_val > 32767) then
        begin print_err("Bad space factor");
        help1("I allow only values in the range 1..32767 here."); int_error(cur_val);
        end
      else space_factor ← cur_val;
      end;
    end;
  end;
```

1298.
```
⟨ Declare subprocedures for prefixed_command 1269 ⟩ +≡
procedure alter_prev_graf;
  var p: 0 .. nest_size;  { index into nest }
  begin nest[nest_ptr] ← cur_list; p ← nest_ptr;
  while abs(nest[p].mode_field) ≠ vmode do decr(p);
  scan_optional_equals; scan_int;
  if cur_val < 0 then
    begin print_err("Bad "); print_esc("prevgraf");
    help1("I allow only nonnegative values here."); int_error(cur_val);
    end
  else begin nest[p].pg_field ← cur_val; cur_list ← nest[nest_ptr];
    end;
  end;
```

1299.
```
⟨ Declare subprocedures for prefixed_command 1269 ⟩ +≡
procedure alter_page_so_far;
  var c: 0 .. 7;  { index into page_so_far }
  begin c ← cur_chr; scan_optional_equals; scan_normal_dimen; page_so_far[c] ← cur_val;
  end;
```

1300. ⟨ Declare subprocedures for *prefixed_command* 1269 ⟩ +≡
```
procedure alter_integer;
  var c: small_number;  { 0 for \deadcycles, 1 for \insertpenalties, etc. }
  begin c ← cur_chr; scan_optional_equals; scan_int;
  if c = 0 then dead_cycles ← cur_val
  ⟨ Cases for alter_integer 1506 ⟩
else insert_penalties ← cur_val;
  end;
```

1301. ⟨ Declare subprocedures for *prefixed_command* 1269 ⟩ +≡
```
procedure alter_box_dimen;
  var c: small_number;  { width_offset or height_offset or depth_offset }
    b: pointer;  { box register }
  begin c ← cur_chr; scan_register_num; fetch_box(b); scan_optional_equals; scan_normal_dimen;
  if b ≠ null then mem[b + c].sc ← cur_val;
  end;
```

1302. Paragraph shapes are set up in the obvious way.

⟨ Assignments 1271 ⟩ +≡
```
set_shape: begin q ← cur_chr; scan_optional_equals; scan_int; n ← cur_val;
  if n ≤ 0 then p ← null
  else if q > par_shape_loc then
      begin n ← (cur_val div 2) + 1; p ← get_node(2 * n + 1); info(p) ← n; n ← cur_val;
      mem[p + 1].int ← n;  { number of penalties }
      for j ← p + 2 to p + n + 1 do
        begin scan_int; mem[j].int ← cur_val;  { penalty values }
        end;
      if ¬odd(n) then mem[p + n + 2].int ← 0;  { unused }
      end
    else begin p ← get_node(2 * n + 1); info(p) ← n;
      for j ← 1 to n do
        begin scan_normal_dimen; mem[p + 2 * j − 1].sc ← cur_val;  { indentation }
        scan_normal_dimen; mem[p + 2 * j].sc ← cur_val;  { width }
        end;
      end;
  define(q, shape_ref, p);
  end;
```

1303. Here's something that isn't quite so obvious. It guarantees that *info*(*par_shape_ptr*) can hold any positive n for which *get_node*$(2 * n + 1)$ doesn't overflow the memory capacity.

⟨ Check the "constant" values for consistency 14 ⟩ +≡
```
  if 2 * max_halfword < mem_top − mem_min then bad ← 41;
```

1304. New hyphenation data is loaded by the *hyph_data* command.

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
```
  primitive("hyphenation", hyph_data, 0); primitive("patterns", hyph_data, 1);
```

1305. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
```
hyph_data: if chr_code = 1 then print_esc("patterns")
  else print_esc("hyphenation");
```

1306. ⟨ Assignments 1271 ⟩ +≡
```
hyph_data: if cur_chr = 1 then
    begin init new_patterns; goto done; tini
    print_err("Patterns␣can␣be␣loaded␣only␣by␣INITEX"); help0; error;
    repeat get_token;
    until cur_cmd = right_brace;  { flush the patterns }
    return;
    end
  else begin new_hyph_exceptions; goto done;
    end;
```

1307. All of TEX's parameters are kept in *eqtb* except the font information, the interaction mode, and the hyphenation tables; these are strictly global.

⟨ Assignments 1271 ⟩ +≡
```
assign_font_dimen: begin find_font_dimen(true); k ← cur_val; scan_optional_equals; scan_normal_dimen;
  font_info[k].sc ← cur_val;
  end;
assign_font_int: begin n ← cur_chr; scan_font_ident; f ← cur_val;
  if n < lp_code_base then
    begin scan_optional_equals; scan_int;
    if n = 0 then hyphen_char[f] ← cur_val else skew_char[f] ← cur_val;
    end
  else begin if is_native_font(f) then scan_glyph_number(f)  { for native fonts, the value is a glyph id }
    else scan_char_num;  { for tfm fonts it's the same like pdftex }
    p ← cur_val; scan_optional_equals; scan_int;
    case n of
    lp_code_base: set_cp_code(f, p, left_side, cur_val);
    rp_code_base: set_cp_code(f, p, right_side, cur_val);
    endcases;
    end;
  end;
```

1308. ⟨ Put each of TEX's primitives into the hash table 252 ⟩ +≡
```
primitive("hyphenchar", assign_font_int, 0); primitive("skewchar", assign_font_int, 1);
primitive("lpcode", assign_font_int, lp_code_base); primitive("rpcode", assign_font_int, rp_code_base);
```

1309. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
```
assign_font_int: case chr_code of
  0: print_esc("hyphenchar");
  1: print_esc("skewchar");
  lp_code_base: print_esc("lpcode");
  rp_code_base: print_esc("rpcode");
  endcases;
```

1310. Here is where the information for a new font gets loaded.

⟨ Assignments 1271 ⟩ +≡
```
def_font: new_font(a);
```

1311. ⟨ Declare subprocedures for *prefixed_command* 1269 ⟩ +≡

```
procedure new_font(a : small_number);
  label common_ending;
  var u: pointer; { user's font identifier }
    s: scaled; { stated "at" size, or negative of scaled magnification }
    f: internal_font_number; { runs through existing fonts }
    t: str_number; { name for the frozen font identifier }
    old_setting: 0 .. max_selector; { holds selector setting }
    flushable_string: str_number; { string not yet referenced }
  begin if job_name = 0 then open_log_file; { avoid confusing texput with the font name }
  get_r_token; u ← cur_cs;
  if u ≥ hash_base then t ← text(u)
  else if u ≥ single_base then
      if u = null_cs then t ← "FONT" else t ← u − single_base
    else begin old_setting ← selector; selector ← new_string; print("FONT"); print(u − active_base);
      selector ← old_setting; str_room(1); t ← make_string;
      end;
  define(u, set_font, null_font); scan_optional_equals; scan_file_name;
  ⟨ Scan the font size specification 1312 ⟩;
  ⟨ If this font has already been loaded, set f to the internal font number and goto common_ending 1314 ⟩;
  f ← read_font_info(u, cur_name, cur_area, s);
common_ending: define(u, set_font, f); eqtb[font_id_base + f] ← eqtb[u]; font_id_text(f) ← t;
  end;
```

1312. ⟨ Scan the font size specification 1312 ⟩ ≡

```
  name_in_progress ← true; { this keeps cur_name from being changed }
  if scan_keyword("at") then ⟨ Put the (positive) 'at' size into s 1313 ⟩
  else if scan_keyword("scaled") then
      begin scan_int; s ← −cur_val;
      if (cur_val ≤ 0) ∨ (cur_val > 32768) then
        begin print_err("Illegal␣magnification␣has␣been␣changed␣to␣1000");
        help1("The␣magnification␣ratio␣must␣be␣between␣1␣and␣32768."); int_error(cur_val);
        s ← −1000;
        end;
      end
    else s ← −1000;
  name_in_progress ← false
```

This code is used in section 1311.

1313. ⟨ Put the (positive) 'at' size into s 1313 ⟩ ≡

```
  begin scan_normal_dimen; s ← cur_val;
  if (s ≤ 0) ∨ (s ≥ ´1000000000) then
    begin print_err("Improper␣`at´␣size␣("); print_scaled(s); print("pt),␣replaced␣by␣10pt");
    help2("I␣can␣only␣handle␣fonts␣at␣positive␣sizes␣that␣are")
    ("less␣than␣2048pt,␣so␣I´ve␣changed␣what␣you␣said␣to␣10pt."); error; s ← 10 ∗ unity;
    end;
  end
```

This code is used in section 1312.

1314. When the user gives a new identifier to a font that was previously loaded, the new name becomes the font identifier of record. Font names '`xyz`' and '`XYZ`' are considered to be different.

```
⟨ If this font has already been loaded, set f to the internal font number and goto common_ending 1314 ⟩ ≡
  flushable_string ← str_ptr − 1;
  for f ← font_base + 1 to font_ptr do
    begin if str_eq_str(font_name[f],
          cur_name) ∧ (((cur_area = "") ∧ is_native_font(f)) ∨ str_eq_str(font_area[f], cur_area)) then
      begin if cur_name = flushable_string then
        begin flush_string; cur_name ← font_name[f];
        end;
      if s > 0 then
        begin if s = font_size[f] then goto common_ending;
        end
      else if font_size[f] = xn_over_d(font_dsize[f], −s, 1000) then goto common_ending;
      end; { could be a native font whose "name" ended up partly in area or extension }
    append_str(cur_area); append_str(cur_name); append_str(cur_ext);
    if str_eq_str(font_name[f], make_string) then
      begin flush_string;
      if is_native_font(f) then
        begin if s > 0 then
          begin if s = font_size[f] then goto common_ending;
          end
        else if font_size[f] = xn_over_d(font_dsize[f], −s, 1000) then goto common_ending;
        end
      end
    else flush_string;
    end
```

This code is used in section 1311.

1315.

```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
set_font: begin print("select␣font␣"); font_name_str ← font_name[chr_code];
  if is_native_font(chr_code) then
    begin quote_char ← """";
    for n ← 0 to length(font_name_str) − 1 do
      if str_pool[str_start_macro(font_name_str) + n] = """" then quote_char ← "´";
    print_char(quote_char); slow_print(font_name_str); print_char(quote_char);
    end
  else slow_print(font_name_str);
  if font_size[chr_code] ≠ font_dsize[chr_code] then
    begin print("␣at␣"); print_scaled(font_size[chr_code]); print("pt");
    end;
  end;
```

1316.

```
⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
  primitive("batchmode", set_interaction, batch_mode);
  primitive("nonstopmode", set_interaction, nonstop_mode);
  primitive("scrollmode", set_interaction, scroll_mode);
  primitive("errorstopmode", set_interaction, error_stop_mode);
```

1317. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
```
set_interaction: case chr_code of
  batch_mode: print_esc("batchmode");
  nonstop_mode: print_esc("nonstopmode");
  scroll_mode: print_esc("scrollmode");
  othercases print_esc("errorstopmode")
  endcases;
```

1318. ⟨ Assignments 1271 ⟩ +≡
```
set_interaction: new_interaction;
```

1319. ⟨ Declare subprocedures for *prefixed_command* 1269 ⟩ +≡
```
procedure new_interaction;
  begin print_ln; interaction ← cur_chr; ⟨ Initialize the print selector based on interaction 79 ⟩;
  if log_opened then selector ← selector + 2;
  end;
```

1320. The `\afterassignment` command puts a token into the global variable *after_token*. This global variable is examined just after every assignment has been performed.

⟨ Global variables 13 ⟩ +≡
```
after_token: halfword; { zero, or a saved token }
```

1321. ⟨ Set initial values of key variables 23 ⟩ +≡
```
  after_token ← 0;
```

1322. ⟨ Cases of *main_control* that don't depend on *mode* 1264 ⟩ +≡
```
any_mode(after_assignment): begin get_token; after_token ← cur_tok;
  end;
```

1323. ⟨ Insert a token saved by `\afterassignment`, if any 1323 ⟩ ≡
```
  if after_token ≠ 0 then
    begin cur_tok ← after_token; back_input; after_token ← 0;
    end
```
This code is used in section 1265.

1324. Here is a procedure that might be called 'Get the next non-blank non-relax non-call non-assignment token'.

⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
```
procedure do_assignments;
  label exit;
  begin loop
    begin ⟨ Get the next non-blank non-relax non-call token 438 ⟩;
    if cur_cmd ≤ max_non_prefixed_command then return;
    set_box_allowed ← false; prefixed_command; set_box_allowed ← true;
    end;
exit: end;
```

1325. ⟨ Cases of *main_control* that don't depend on *mode* 1264 ⟩ +≡
```
any_mode(after_group): begin get_token; save_for_after(cur_tok);
  end;
```

1326. Files for `\read` are opened and closed by the *in_stream* command.

```
⟨ Put each of TEX's primitives into the hash table 252 ⟩ +≡
  primitive("openin", in_stream, 1); primitive("closein", in_stream, 0);
```

1327.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
in_stream: if chr_code = 0 then print_esc("closein")
  else print_esc("openin");
```

1328.
```
⟨ Cases of main_control that don't depend on mode 1264 ⟩ +≡
any_mode(in_stream): open_or_close_in;
```

1329.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure open_or_close_in;
  var c: 0 .. 1; { 1 for \openin, 0 for \closein }
    n: 0 .. 15; { stream number }
  begin c ← cur_chr; scan_four_bit_int; n ← cur_val;
  if read_open[n] ≠ closed then
    begin u_close(read_file[n]); read_open[n] ← closed;
    end;
  if c ≠ 0 then
    begin scan_optional_equals; scan_file_name;
    if cur_ext = "" then cur_ext ← ".tex";
    pack_cur_name;
    if a_open_in(read_file[n]) then read_open[n] ← just_open;
    end;
  end;
```

1330. The user can issue messages to the terminal, regardless of the current mode.

```
⟨ Cases of main_control that don't depend on mode 1264 ⟩ +≡
any_mode(message): issue_message;
```

1331.
```
⟨ Put each of TEX's primitives into the hash table 252 ⟩ +≡
  primitive("message", message, 0); primitive("errmessage", message, 1);
```

1332.
```
⟨ Cases of print_cmd_chr for symbolic printing of primitives 253 ⟩ +≡
message: if chr_code = 0 then print_esc("message")
  else print_esc("errmessage");
```

1333.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure issue_message;
  var old_setting: 0 .. max_selector; { holds selector setting }
    c: 0 .. 1; { identifies \message and \errmessage }
    s: str_number; { the message }
  begin c ← cur_chr; link(garbage) ← scan_toks(false, true); old_setting ← selector;
  selector ← new_string; token_show(def_ref); selector ← old_setting; flush_list(def_ref); str_room(1);
  s ← make_string;
  if c = 0 then ⟨ Print string s on the terminal 1334 ⟩
  else ⟨ Print string s as an error message 1337 ⟩;
  flush_string;
  end;
```

1334. ⟨ Print string s on the terminal 1334 ⟩ ≡
```
begin if term_offset + length(s) > max_print_line − 2 then print_ln
else if (term_offset > 0) ∨ (file_offset > 0) then print_char("␣");
slow_print(s); update_terminal;
end
```
This code is used in section 1333.

1335. If `\errmessage` occurs often in *scroll_mode*, without user-defined `\errhelp`, we don't want to give a long help message each time. So we give a verbose explanation only once.

⟨ Global variables 13 ⟩ +≡
```
long_help_seen: boolean; { has the long \errmessage help been used? }
```

1336. ⟨ Set initial values of key variables 23 ⟩ +≡
```
long_help_seen ← false;
```

1337. ⟨ Print string s as an error message 1337 ⟩ ≡
```
begin print_err(""); slow_print(s);
if err_help ≠ null then use_err_help ← true
else if long_help_seen then help1("(That␣was␣another␣\errmessage.)")
  else begin if interaction < error_stop_mode then long_help_seen ← true;
    help4("This␣error␣message␣was␣generated␣by␣an␣\errmessage")
    ("command,␣so␣I␣can't␣give␣any␣explicit␣help.")
    ("Pretend␣that␣you're␣Hercule␣Poirot:␣Examine␣all␣clues,")
    ("and␣deduce␣the␣truth␣by␣order␣and␣method.");
    end;
error; use_err_help ← false;
end
```
This code is used in section 1333.

1338. The *error* routine calls on *give_err_help* if help is requested from the *err_help* parameter.

```
procedure give_err_help;
  begin token_show(err_help);
  end;
```

1339. The `\uppercase` and `\lowercase` commands are implemented by building a token list and then changing the cases of the letters in it.

⟨ Cases of *main_control* that don't depend on *mode* 1264 ⟩ +≡
```
any_mode(case_shift): shift_case;
```

1340. ⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
```
primitive("lowercase", case_shift, lc_code_base); primitive("uppercase", case_shift, uc_code_base);
```

1341. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
```
case_shift: if chr_code = lc_code_base then print_esc("lowercase")
  else print_esc("uppercase");
```

1342. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡
```
procedure shift_case;
  var b: pointer; { lc_code_base or uc_code_base }
    p: pointer; { runs through the token list }
    t: halfword; { token }
    c: integer; { character code }
  begin b ← cur_chr; p ← scan_toks(false, false); p ← link(def_ref);
  while p ≠ null do
    begin ⟨ Change the case of the token in p, if a change is appropriate 1343 ⟩;
    p ← link(p);
    end;
  back_list(link(def_ref)); free_avail(def_ref); { omit reference count }
  end;
```

1343. When the case of a *chr_code* changes, we don't change the *cmd*. We also change active characters, using the fact that *cs_token_flag* + *active_base* is a multiple of 256.

⟨ Change the case of the token in *p*, if a change is appropriate 1343 ⟩ ≡
```
t ← info(p);
if t < cs_token_flag + single_base then
  begin c ← t mod max_char_val;
  if equiv(b + c) ≠ 0 then info(p) ← t − c + equiv(b + c);
  end
```
This code is used in section 1342.

1344. We come finally to the last pieces missing from *main_control*, namely the '`\show`' commands that are useful when debugging.

⟨ Cases of *main_control* that don't depend on *mode* 1264 ⟩ +≡
```
any_mode(xray): show_whatever;
```

1345.
```
define show_code = 0 { \show }
define show_box_code = 1 { \showbox }
define show_the_code = 2 { \showthe }
define show_lists_code = 3 { \showlists }
```

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
```
primitive("show", xray, show_code); primitive("showbox", xray, show_box_code);
primitive("showthe", xray, show_the_code); primitive("showlists", xray, show_lists_code);
```

1346. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡
```
xray: case chr_code of
  show_box_code: print_esc("showbox");
  show_the_code: print_esc("showthe");
  show_lists_code: print_esc("showlists");
    ⟨ Cases of xray for print_cmd_chr 1486 ⟩
  othercases print_esc("show")
  endcases;
```

1347. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
procedure show_whatever;
  label common_ending;
  var p: pointer;  { tail of a token list to show }
    t: small_number;  { type of conditional being shown }
    m: normal .. or_code;  { upper bound on fi_or_else codes }
    l: integer;  { line where that conditional began }
    n: integer;  { level of \if...\fi nesting }
  begin case cur_chr of
  show_lists_code: begin begin_diagnostic; show_activities;
    end;
  show_box_code: ⟨ Show the current contents of a box 1350 ⟩;
  show_code: ⟨ Show the current meaning of a token, then goto common_ending 1348 ⟩;
    ⟨ Cases for show_whatever 1487 ⟩
  othercases ⟨ Show the current value of some parameter or register, then goto common_ending 1351 ⟩
  endcases;
  ⟨ Complete a potentially long \show command 1352 ⟩;
common_ending: if interaction < error_stop_mode then
    begin help0; decr(error_count);
    end
  else if tracing_online > 0 then
      begin
      help3("This␣isn´t␣an␣error␣message;␣I´m␣just␣\showing␣something.")
      ("Type␣`I\show...´␣to␣show␣more␣(e.g.,␣\show\cs,")
      ("\showthe\count10,␣\showbox255,␣\showlists).");
      end
    else begin
      help5("This␣isn´t␣an␣error␣message;␣I´m␣just␣\showing␣something.")
      ("Type␣`I\show...´␣to␣show␣more␣(e.g.,␣\show\cs,")
      ("\showthe\count10,␣\showbox255,␣\showlists).")
      ("And␣type␣`I\tracingonline=1\show...´␣to␣show␣boxes␣and")
      ("lists␣on␣your␣terminal␣as␣well␣as␣in␣the␣transcript␣file.");
      end;
  error;
  end;
```

1348. ⟨ Show the current meaning of a token, then **goto** *common_ending* 1348 ⟩ ≡

```
  begin get_token;
  if interaction = error_stop_mode then wake_up_terminal;
  print_nl(">␣");
  if cur_cs ≠ 0 then
    begin sprint_cs(cur_cs); print_char("=");
    end;
  print_meaning; goto common_ending;
  end
```

This code is used in section 1347.

1349. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
undefined_cs: print("undefined");
call, long_call, outer_call, long_outer_call: begin n ← cmd − call;
  if info(link(chr_code)) = protected_token then n ← n + 4;
  if odd(n div 4) then print_esc("protected");
  if odd(n) then print_esc("long");
  if odd(n div 2) then print_esc("outer");
  if n > 0 then print_char("␣");
  print("macro");
  end;
end_template: print_esc("outer␣endtemplate");
```

1350. ⟨ Show the current contents of a box 1350 ⟩ ≡

```
begin scan_register_num; fetch_box(p); begin_diagnostic; print_nl(">␣\box"); print_int(cur_val);
print_char("=");
if p = null then print("void") else show_box(p);
end
```

This code is used in section 1347.

1351. ⟨ Show the current value of some parameter or register, then **goto** *common_ending* 1351 ⟩ ≡

```
begin p ← the_toks;
if interaction = error_stop_mode then wake_up_terminal;
print_nl(">␣"); token_show(temp_head); flush_list(link(temp_head)); goto common_ending;
end
```

This code is used in section 1347.

1352. ⟨ Complete a potentially long `\show` command 1352 ⟩ ≡

```
end_diagnostic(true); print_err("OK");
if selector = term_and_log then
  if tracing_online ≤ 0 then
    begin selector ← term_only; print("␣(see␣the␣transcript␣file)"); selector ← term_and_log;
    end
```

This code is used in section 1347.

1353. Dumping and undumping the tables. After INITEX has seen a collection of fonts and macros, it can write all the necessary information on an auxiliary file so that production versions of TEX are able to initialize their memory at high speed. The present section of the program takes care of such output and input. We shall consider simultaneously the processes of storing and restoring, so that the inverse relation between them is clear.

The global variable *format_ident* is a string that is printed right after the *banner* line when TEX is ready to start. For INITEX this string says simply ‘ (INITEX)’; for other versions of TEX it says, for example, ‘ (preloaded format=plain 1982.11.19)’, showing the year, month, and day that the format file was created. We have *format_ident* = 0 before TEX’s tables are loaded.

```
⟨ Global variables 13 ⟩ +≡
format_ident: str_number;
```

1354.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  format_ident ← 0;
```

1355.
```
⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
  format_ident ← "␣(INITEX)";
```

1356.
```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
  init procedure store_fmt_file;
  label found1, found2, done1, done2;
  var j, k, l: integer;  { all-purpose indices }
    p, q: pointer;  { all-purpose pointers }
    x: integer;  { something to dump }
    w: four_quarters;  { four ASCII codes }
  begin ⟨ If dumping is not allowed, abort 1358 ⟩;
  ⟨ Create the format_ident, open the format file, and inform the user that dumping has begun 1382 ⟩;
  ⟨ Dump constants for consistency check 1361 ⟩;
  ⟨ Dump the string pool 1363 ⟩;
  ⟨ Dump the dynamic memory 1365 ⟩;
  ⟨ Dump the table of equivalents 1367 ⟩;
  ⟨ Dump the font information 1374 ⟩;
  ⟨ Dump the hyphenation tables 1378 ⟩;
  ⟨ Dump a couple more things and the closing check word 1380 ⟩;
  ⟨ Close the format file 1383 ⟩;
  end;
  tini
```

1357. Corresponding to the procedure that dumps a format file, we have a function that reads one in. The function returns *false* if the dumped format is incompatible with the present TeX table sizes, etc.

```
define bad_fmt = 6666 { go here if the format file is unacceptable }
define too_small(#) ≡
        begin wake_up_terminal; wterm_ln('---! Must increase the ', #); goto bad_fmt;
        end
⟨ Declare the function called open_fmt_file 559 ⟩
function load_fmt_file: boolean;
  label bad_fmt, exit;
  var j, k: integer; { all-purpose indices }
    p, q: pointer; { all-purpose pointers }
    x: integer; { something undumped }
    w: four_quarters; { four ASCII codes }
  begin ⟨ Undump constants for consistency check 1362 ⟩;
  ⟨ Undump the string pool 1364 ⟩;
  ⟨ Undump the dynamic memory 1366 ⟩;
  ⟨ Undump the table of equivalents 1368 ⟩;
  ⟨ Undump the font information 1375 ⟩;
  ⟨ Undump the hyphenation tables 1379 ⟩;
  ⟨ Undump a couple more things and the closing check word 1381 ⟩;
  load_fmt_file ← true; return; { it worked! }
bad_fmt: wake_up_terminal; wterm_ln('(Fatal format file error; I''m stymied)');
  load_fmt_file ← false;
exit: end;
```

1358. The user is not allowed to dump a format file unless *save_ptr* = 0. This condition implies that *cur_level* = *level_one*, hence the *xeq_level* array is constant and it need not be dumped.

```
⟨ If dumping is not allowed, abort 1358 ⟩ ≡
  if save_ptr ≠ 0 then
    begin print_err("You can't dump inside a group"); help1("`{...\dump}' is a no-no.");
    succumb;
    end
```

This code is used in section 1356.

1359. Format files consist of *memory_word* items, and we use the following macros to dump words of different types:

```
define dump_wd(#) ≡
        begin fmt_file↑ ← #; put(fmt_file); end
define dump_int(#) ≡
        begin fmt_file↑.int ← #; put(fmt_file); end
define dump_hh(#) ≡
        begin fmt_file↑.hh ← #; put(fmt_file); end
define dump_qqqq(#) ≡
        begin fmt_file↑.qqqq ← #; put(fmt_file); end
⟨ Global variables 13 ⟩ +≡
fmt_file: word_file; { for input or output of format information }
```

1360. The inverse macros are slightly more complicated, since we need to check the range of the values we are reading in. We say '*undump*(*a*)(*b*)(*x*)' to read an integer value x that is supposed to be in the range $a \le x \le b$. System error messages should be suppressed when undumping.

```
define undump_wd(#) ≡
        begin get(fmt_file); # ← fmt_file↑; end
define undump_int(#) ≡
        begin get(fmt_file); # ← fmt_file↑.int; end
define undump_hh(#) ≡
        begin get(fmt_file); # ← fmt_file↑.hh; end
define undump_qqqq(#) ≡
        begin get(fmt_file); # ← fmt_file↑.qqqq; end
define undump_end_end(#) ≡ # ← x; end
define undump_end(#) ≡ (x > #) then goto bad_fmt else undump_end_end
define undump(#) ≡
      begin undump_int(x);
      if (x < #) ∨ undump_end
define undump_size_end_end(#) ≡ too_small(#) else undump_end_end
define undump_size_end(#) ≡
        if x > # then undump_size_end_end
define undump_size(#) ≡
      begin undump_int(x);
      if x < # then goto bad_fmt;
      undump_size_end
```

1361. The next few sections of the program should make it clear how we use the dump/undump macros.

```
⟨ Dump constants for consistency check 1361 ⟩ ≡
  dump_int(@$);
  ⟨ Dump the ε-TeX state 1464 ⟩
  dump_int(mem_bot);
  dump_int(mem_top);
  dump_int(eqtb_size);
  dump_int(hash_prime);
  dump_int(hyph_size)
```

This code is used in section 1356.

1362. Sections of a WEB program that are "commented out" still contribute strings to the string pool; therefore INITEX and TEX will have the same strings. (And it is, of course, a good thing that they do.)

```
⟨ Undump constants for consistency check 1362 ⟩ ≡
  x ← fmt_file↑.int;
  if x ≠ @$ then goto bad_fmt;  { check that strings are the same }
  ⟨ Undump the ε-TEX state 1465 ⟩
  undump_int(x);
  if x ≠ mem_bot then goto bad_fmt;
  undump_int(x);
  if x ≠ mem_top then goto bad_fmt;
  undump_int(x);
  if x ≠ eqtb_size then goto bad_fmt;
  undump_int(x);
  if x ≠ hash_prime then goto bad_fmt;
  undump_int(x);
  if x ≠ hyph_size then goto bad_fmt
```

This code is used in section 1357.

1363.
```
define dump_four_ASCII ≡ w.b0 ← qi(so(str_pool[k])); w.b1 ← qi(so(str_pool[k + 1]));
    w.b2 ← qi(so(str_pool[k + 2])); w.b3 ← qi(so(str_pool[k + 3])); dump_qqqq(w)
⟨ Dump the string pool 1363 ⟩ ≡
  dump_int(pool_ptr); dump_int(str_ptr);
  for k ← 0 to str_ptr do dump_int(str_start[k]);
  k ← 0;
  while k + 4 < pool_ptr do
    begin dump_four_ASCII; k ← k + 4;
    end;
  k ← pool_ptr − 4; dump_four_ASCII; print_ln; print_int(str_ptr);
  print("␣strings␣of␣total␣length␣"); print_int(pool_ptr)
```

This code is used in section 1356.

1364.
```
define undump_four_ASCII ≡ undump_qqqq(w); str_pool[k] ← si(qo(w.b0));
    str_pool[k + 1] ← si(qo(w.b1)); str_pool[k + 2] ← si(qo(w.b2)); str_pool[k + 3] ← si(qo(w.b3))
⟨ Undump the string pool 1364 ⟩ ≡
  undump_size(0)(pool_size)('string␣pool␣size')(pool_ptr);
  undump_size(0)(max_strings)('max␣strings')(str_ptr);
  for k ← 0 to str_ptr do undump(0)(pool_ptr)(str_start[k]);
  k ← 0;
  while k + 4 < pool_ptr do
    begin undump_four_ASCII; k ← k + 4;
    end;
  k ← pool_ptr − 4; undump_four_ASCII; init_str_ptr ← str_ptr; init_pool_ptr ← pool_ptr
```

This code is used in section 1357.

1365. By sorting the list of available spaces in the variable-size portion of *mem*, we are usually able to get by without having to dump very much of the dynamic memory.

We recompute *var_used* and *dyn_used*, so that INITEX dumps valid information even when it has not been gathering statistics.

```
⟨ Dump the dynamic memory 1365 ⟩ ≡
  sort_avail; var_used ← 0; dump_int(lo_mem_max); dump_int(rover);
  if eTeX_ex then
    for k ← int_val to inter_char_val do dump_int(sa_root[k]);
  p ← mem_bot; q ← rover; x ← 0;
  repeat for k ← p to q + 1 do dump_wd(mem[k]);
    x ← x + q + 2 − p; var_used ← var_used + q − p; p ← q + node_size(q); q ← rlink(q);
  until q = rover;
  var_used ← var_used + lo_mem_max − p; dyn_used ← mem_end + 1 − hi_mem_min;
  for k ← p to lo_mem_max do dump_wd(mem[k]);
  x ← x + lo_mem_max + 1 − p; dump_int(hi_mem_min); dump_int(avail);
  for k ← hi_mem_min to mem_end do dump_wd(mem[k]);
  x ← x + mem_end + 1 − hi_mem_min; p ← avail;
  while p ≠ null do
    begin decr(dyn_used); p ← link(p);
    end;
  dump_int(var_used); dump_int(dyn_used); print_ln; print_int(x);
  print("␣memory␣locations␣dumped;␣current␣usage␣is␣"); print_int(var_used); print_char("&");
  print_int(dyn_used)
```

This code is used in section 1356.

1366. ⟨ Undump the dynamic memory 1366 ⟩ ≡

```
  undump(lo_mem_stat_max + 1000)(hi_mem_stat_min − 1)(lo_mem_max);
  undump(lo_mem_stat_max + 1)(lo_mem_max)(rover);
  if eTeX_ex then
    for k ← int_val to inter_char_val do undump(null)(lo_mem_max)(sa_root[k]);
  p ← mem_bot; q ← rover;
  repeat for k ← p to q + 1 do undump_wd(mem[k]);
    p ← q + node_size(q);
    if (p > lo_mem_max) ∨ ((q ≥ rlink(q)) ∧ (rlink(q) ≠ rover)) then goto bad_fmt;
    q ← rlink(q);
  until q = rover;
  for k ← p to lo_mem_max do undump_wd(mem[k]);
  if mem_min < mem_bot − 2 then { make more low memory available }
    begin p ← llink(rover); q ← mem_min + 1; link(mem_min) ← null; info(mem_min) ← null;
        { we don't use the bottom word }
    rlink(p) ← q; llink(rover) ← q;
    rlink(q) ← rover; llink(q) ← p; link(q) ← empty_flag; node_size(q) ← mem_bot − q;
    end;
  undump(lo_mem_max + 1)(hi_mem_stat_min)(hi_mem_min); undump(null)(mem_top)(avail);
  mem_end ← mem_top;
  for k ← hi_mem_min to mem_end do undump_wd(mem[k]);
  undump_int(var_used); undump_int(dyn_used)
```

This code is used in section 1357.

1367. ⟨ Dump the table of equivalents 1367 ⟩ ≡
⟨ Dump regions 1 to 4 of *eqtb* 1369 ⟩;
⟨ Dump regions 5 and 6 of *eqtb* 1370 ⟩;
dump_int(*par_loc*); *dump_int*(*write_loc*);
⟨ Dump the hash table 1372 ⟩
This code is used in section 1356.

1368. ⟨ Undump the table of equivalents 1368 ⟩ ≡
⟨ Undump regions 1 to 6 of *eqtb* 1371 ⟩;
undump(*hash_base*)(*frozen_control_sequence*)(*par_loc*); *par_token* ← *cs_token_flag* + *par_loc*;
undump(*hash_base*)(*frozen_control_sequence*)(*write_loc*);
⟨ Undump the hash table 1373 ⟩
This code is used in section 1357.

1369. The table of equivalents usually contains repeated information, so we dump it in compressed form: The sequence of $n+2$ values $(n, x_1, \ldots, x_n, m)$ in the format file represents $n+m$ consecutive entries of *eqtb*, with m extra copies of x_n, namely $(x_1, \ldots, x_n, x_n, \ldots, x_n)$.

```
⟨ Dump regions 1 to 4 of eqtb 1369 ⟩ ≡
  k ← active_base;
  repeat j ← k;
    while j < int_base − 1 do
      begin if (equiv(j) = equiv(j + 1)) ∧ (eq_type(j) = eq_type(j + 1)) ∧ (eq_level(j) = eq_level(j + 1))
          then goto found1;
      incr(j);
      end;
    l ← int_base; goto done1; { j = int_base − 1 }
  found1: incr(j); l ← j;
    while j < int_base − 1 do
      begin if (equiv(j) ≠ equiv(j + 1)) ∨ (eq_type(j) ≠ eq_type(j + 1)) ∨ (eq_level(j) ≠ eq_level(j + 1))
          then goto done1;
      incr(j);
      end;
  done1: dump_int(l − k);
    while k < l do
      begin dump_wd(eqtb[k]); incr(k);
      end;
    k ← j + 1; dump_int(k − l);
  until k = int_base
```
This code is used in section 1367.

1370. ⟨ Dump regions 5 and 6 of *eqtb* 1370 ⟩ ≡

```
repeat j ← k;
  while j < eqtb_size do
    begin if eqtb[j].int = eqtb[j + 1].int then goto found2;
    incr(j);
    end;
  l ← eqtb_size + 1; goto done2; { j = eqtb_size }
found2: incr(j); l ← j;
  while j < eqtb_size do
    begin if eqtb[j].int ≠ eqtb[j + 1].int then goto done2;
    incr(j);
    end;
done2: dump_int(l − k);
  while k < l do
    begin dump_wd(eqtb[k]); incr(k);
    end;
  k ← j + 1; dump_int(k − l);
until k > eqtb_size
```

This code is used in section 1367.

1371. ⟨ Undump regions 1 to 6 of *eqtb* 1371 ⟩ ≡

```
k ← active_base;
repeat undump_int(x);
  if (x < 1) ∨ (k + x > eqtb_size + 1) then goto bad_fmt;
  for j ← k to k + x − 1 do undump_wd(eqtb[j]);
  k ← k + x; undump_int(x);
  if (x < 0) ∨ (k + x > eqtb_size + 1) then goto bad_fmt;
  for j ← k to k + x − 1 do eqtb[j] ← eqtb[k − 1];
  k ← k + x;
until k > eqtb_size
```

This code is used in section 1368.

1372. A different scheme is used to compress the hash table, since its lower region is usually sparse. When *text*(*p*) ≠ 0 for *p* ≤ *hash_used*, we output two words, *p* and *hash*[*p*]. The hash table is, of course, densely packed for *p* ≥ *hash_used*, so the remaining entries are output in a block.

⟨ Dump the hash table 1372 ⟩ ≡

```
for p ← 0 to prim_size do dump_hh(prim[p]);
dump_int(hash_used); cs_count ← frozen_control_sequence − 1 − hash_used;
for p ← hash_base to hash_used do
  if text(p) ≠ 0 then
    begin dump_int(p); dump_hh(hash[p]); incr(cs_count);
    end;
for p ← hash_used + 1 to undefined_control_sequence − 1 do dump_hh(hash[p]);
dump_int(cs_count);
print_ln; print_int(cs_count); print(" multiletter control sequences")
```

This code is used in section 1367.

1373. ⟨ Undump the hash table 1373 ⟩ ≡
```
for p ← 0 to prim_size do undump_hh(prim[p]);
undump(hash_base)(frozen_control_sequence)(hash_used); p ← hash_base − 1;
repeat undump(p + 1)(hash_used)(p); undump_hh(hash[p]);
until p = hash_used;
for p ← hash_used + 1 to undefined_control_sequence − 1 do undump_hh(hash[p]);
undump_int(cs_count)
```
This code is used in section 1368.

1374. ⟨ Dump the font information 1374 ⟩ ≡
```
dump_int(fmem_ptr);
for k ← 0 to fmem_ptr − 1 do dump_wd(font_info[k]);
dump_int(font_ptr);
for k ← null_font to font_ptr do ⟨ Dump the array info for internal font number k 1376 ⟩;
print_ln; print_int(fmem_ptr − 7); print("␣words␣of␣font␣info␣for␣");
print_int(font_ptr − font_base); print("␣preloaded␣font");
if font_ptr ≠ font_base + 1 then print_char("s")
```
This code is used in section 1356.

1375. ⟨ Undump the font information 1375 ⟩ ≡
```
undump_size(7)(font_mem_size)(´font␣mem␣size´)(fmem_ptr);
for k ← 0 to fmem_ptr − 1 do undump_wd(font_info[k]);
undump_size(font_base)(font_max)(´font␣max´)(font_ptr);
for k ← null_font to font_ptr do ⟨ Undump the array info for internal font number k 1377 ⟩
```
This code is used in section 1357.

1376. ⟨ Dump the array info for internal font number k 1376 ⟩ ≡
```
begin dump_qqqq(font_check[k]); dump_int(font_size[k]); dump_int(font_dsize[k]);
dump_int(font_params[k]);
dump_int(hyphen_char[k]); dump_int(skew_char[k]);
dump_int(font_name[k]); dump_int(font_area[k]);
dump_int(font_bc[k]); dump_int(font_ec[k]);
dump_int(char_base[k]); dump_int(width_base[k]); dump_int(height_base[k]);
dump_int(depth_base[k]); dump_int(italic_base[k]); dump_int(lig_kern_base[k]);
dump_int(kern_base[k]); dump_int(exten_base[k]); dump_int(param_base[k]);
dump_int(font_glue[k]);
dump_int(bchar_label[k]); dump_int(font_bchar[k]); dump_int(font_false_bchar[k]);
print_nl("\font"); print_esc(font_id_text(k)); print_char("=");
print_file_name(font_name[k], font_area[k], "");
if font_size[k] ≠ font_dsize[k] then
  begin print("␣at␣"); print_scaled(font_size[k]); print("pt");
  end;
end
```
This code is used in section 1374.

1377. ⟨ Undump the array info for internal font number k 1377 ⟩ ≡

```
begin undump_qqqq(font_check[k]);
undump_int(font_size[k]); undump_int(font_dsize[k]);
undump(min_halfword)(max_halfword)(font_params[k]);
undump_int(hyphen_char[k]); undump_int(skew_char[k]);
undump(0)(str_ptr)(font_name[k]); undump(0)(str_ptr)(font_area[k]);
undump(0)(255)(font_bc[k]); undump(0)(255)(font_ec[k]);
undump_int(char_base[k]); undump_int(width_base[k]); undump_int(height_base[k]);
undump_int(depth_base[k]); undump_int(italic_base[k]); undump_int(lig_kern_base[k]);
undump_int(kern_base[k]); undump_int(exten_base[k]); undump_int(param_base[k]);
undump(min_halfword)(lo_mem_max)(font_glue[k]);
undump(0)(fmem_ptr - 1)(bchar_label[k]); undump(min_quarterword)(non_char)(font_bchar[k]);
undump(min_quarterword)(non_char)(font_false_bchar[k]);
end
```

This code is used in section 1375.

1378. ⟨ Dump the hyphenation tables 1378 ⟩ ≡

```
dump_int(hyph_count);
for k ← 0 to hyph_size do
  if hyph_word[k] ≠ 0 then
    begin dump_int(k); dump_int(hyph_word[k]); dump_int(hyph_list[k]);
    end;
print_ln; print_int(hyph_count); print("␣hyphenation␣exception");
if hyph_count ≠ 1 then print_char("s");
if trie_not_ready then init_trie;
dump_int(trie_max); dump_int(hyph_start);
for k ← 0 to trie_max do dump_hh(trie[k]);
dump_int(max_hyph_char); dump_int(trie_op_ptr);
for k ← 1 to trie_op_ptr do
  begin dump_int(hyf_distance[k]); dump_int(hyf_num[k]); dump_int(hyf_next[k]);
  end;
print_nl("Hyphenation␣trie␣of␣length␣"); print_int(trie_max); print("␣has␣");
print_int(trie_op_ptr); print("␣op");
if trie_op_ptr ≠ 1 then print_char("s");
print("␣out␣of␣"); print_int(trie_op_size);
for k ← biggest_lang downto 0 do
  if trie_used[k] > min_quarterword then
    begin print_nl("␣␣"); print_int(qo(trie_used[k])); print("␣for␣language␣"); print_int(k);
    dump_int(k); dump_int(qo(trie_used[k]));
    end
```

This code is used in section 1356.

1379. Only "nonempty" parts of *op_start* need to be restored.

```
⟨ Undump the hyphenation tables 1379 ⟩ ≡
  undump(0)(hyph_size)(hyph_count);
  for k ← 1 to hyph_count do
    begin undump(0)(hyph_size)(j); undump(0)(str_ptr)(hyph_word[j]);
    undump(min_halfword)(max_halfword)(hyph_list[j]);
    end;
  undump_size(0)(trie_size)('trie␣size')(j); init trie_max ← j; tini undump(0)(j)(hyph_start);
  for k ← 0 to j do undump_hh(trie[k]);
  undump_int(max_hyph_char);
  undump_size(0)(trie_op_size)('trie␣op␣size')(j); init trie_op_ptr ← j; tini
  for k ← 1 to j do
    begin undump(0)(63)(hyf_distance[k]); { a small_number }
    undump(0)(63)(hyf_num[k]); undump(min_quarterword)(max_quarterword)(hyf_next[k]);
    end;
  init for k ← 0 to biggest_lang do trie_used[k] ← min_quarterword;
  tini
  k ← biggest_lang + 1;
  while j > 0 do
    begin undump(0)(k − 1)(k); undump(1)(j)(x); init trie_used[k] ← qi(x); tini
    j ← j − x; op_start[k] ← qo(j);
    end;
  init trie_not_ready ← false tini
```

This code is used in section 1357.

1380. We have already printed a lot of statistics, so we set *tracing_stats* ← 0 to prevent them from appearing again.

```
⟨ Dump a couple more things and the closing check word 1380 ⟩ ≡
  dump_int(interaction); dump_int(format_ident); dump_int(69069); tracing_stats ← 0
```

This code is used in section 1356.

1381.

```
⟨ Undump a couple more things and the closing check word 1381 ⟩ ≡
  undump(batch_mode)(error_stop_mode)(interaction); undump(0)(str_ptr)(format_ident); undump_int(x);
  if (x ≠ 69069) ∨ eof(fmt_file) then goto bad_fmt
```

This code is used in section 1357.

1382.

```
⟨ Create the format_ident, open the format file, and inform the user that dumping has
    begun 1382 ⟩ ≡
  selector ← new_string; print("␣(preloaded␣format="); print(job_name); print_char("␣");
  print_int(year); print_char("."); print_int(month); print_char("."); print_int(day); print_char(")");
  if interaction = batch_mode then selector ← log_only
  else selector ← term_and_log;
  str_room(1); format_ident ← make_string; pack_job_name(format_extension);
  while ¬w_open_out(fmt_file) do prompt_file_name("format␣file␣name", format_extension);
  print_nl("Beginning␣to␣dump␣on␣file␣"); slow_print(w_make_name_string(fmt_file)); flush_string;
  print_nl(""); slow_print(format_ident)
```

This code is used in section 1356.

1383.

```
⟨ Close the format file 1383 ⟩ ≡
  w_close(fmt_file)
```

This code is used in section 1356.

1384. The main program. This is it: the part of TeX that executes all those procedures we have written.

Well—almost. Let's leave space for a few more routines that we may have forgotten.

⟨ Last-minute procedures 1387 ⟩

1385. We have noted that there are two versions of TeX82. One, called `INITEX`, has to be run first; it initializes everything from scratch, without reading a format file, and it has the capability of dumping a format file. The other one is called '`VIRTEX`'; it is a "virgin" program that needs to input a format file in order to get started. `VIRTEX` typically has more memory capacity than `INITEX`, because it does not need the space consumed by the auxiliary hyphenation tables and the numerous calls on *primitive*, etc.

The `VIRTEX` program cannot read a format file instantaneously, of course; the best implementations therefore allow for production versions of TeX that not only avoid the loading routine for Pascal object code, they also have a format file pre-loaded. This is impossible to do if we stick to standard Pascal; but there is a simple way to fool many systems into avoiding the initialization, as follows: (1) We declare a global integer variable called *ready_already*. The probability is negligible that this variable holds any particular value like 314159 when `VIRTEX` is first loaded. (2) After we have read in a format file and initialized everything, we set *ready_already* ← 314159. (3) Soon `VIRTEX` will print '`*`', waiting for more input; and at this point we interrupt the program and save its core image in some form that the operating system can reload speedily. (4) When that core image is activated, the program starts again at the beginning; but now *ready_already* = 314159 and all the other global variables have their initial values too. The former chastity has vanished!

In other words, if we allow ourselves to test the condition *ready_already* = 314159, before *ready_already* has been assigned a value, we can avoid the lengthy initialization. Dirty tricks rarely pay off so handsomely.

On systems that allow such preloading, the standard program called `TeX` should be the one that has `plain` format preloaded, since that agrees with *The TeXbook*. Other versions, e.g., `AmSTeX`, should also be provided for commonly used formats.

```
⟨ Global variables 13 ⟩ +≡
ready_already: integer;  { a sacrifice of purity for economy }
```

1386. Now this is really it: TEX starts and ends here.

The initial test involving *ready_already* should be deleted if the Pascal runtime system is smart enough to detect such a "mistake."

```
  begin { start_here }
  history ← fatal_error_stop; { in case we quit during initialization }
  t_open_out; { open the terminal for output }
  if ready_already = 314159 then goto start_of_TEX;
  ⟨ Check the "constant" values for consistency 14 ⟩
  if bad > 0 then
    begin wterm_ln('Ouch---my␣internal␣constants␣have␣been␣clobbered!', '---case␣', bad : 1);
    goto final_end;
    end;
  initialize; { set global variables to their starting values }
  init if ¬get_strings_started then goto final_end;
  init_prim; { call primitive for each primitive }
  init_str_ptr ← str_ptr; init_pool_ptr ← pool_ptr; fix_date_and_time;
  tini
  ready_already ← 314159;
start_of_TEX: ⟨ Initialize the output routines 55 ⟩;
  ⟨ Get the first line of input and prepare to start 1391 ⟩;
  history ← spotless; { ready to go! }
  main_control; { come to life }
  final_cleanup; { prepare for death }
end_of_TEX: close_files_and_terminate;
final_end: ready_already ← 0;
  end.
```

1387. Here we do whatever is needed to complete TEX's job gracefully on the local operating system. The code here might come into play after a fatal error; it must therefore consist entirely of "safe" operations that cannot produce error messages. For example, it would be a mistake to call *str_room* or *make_string* at this time, because a call on *overflow* might lead to an infinite loop. (Actually there's one way to get error messages, via *prepare_mag*; but that can't cause infinite recursion.)

If *final_cleanup* is bypassed, this program doesn't bother to close the input files that may still be open.

```
⟨ Last-minute procedures 1387 ⟩ ≡
procedure close_files_and_terminate;
  var k: integer; { all-purpose index }
  begin ⟨ Finish the extensions 1441 ⟩;
  new_line_char ← −1;
  stat if tracing_stats > 0 then ⟨ Output statistics about this job 1388 ⟩; tats
  wake_up_terminal; ⟨ Finish the DVI file 680 ⟩;
  if log_opened then
    begin wlog_cr; a_close(log_file); selector ← selector − 2;
    if selector = term_only then
      begin print_nl("Transcript␣written␣on␣"); slow_print(log_name); print_char(".");
      end;
    end;
  end;
```

See also sections 1389, 1390, and 1392.

This code is used in section 1384.

1388. The present section goes directly to the log file instead of using *print* commands, because there's no need for these strings to take up *str_pool* memory when a non-**stat** version of TeX is being used.

```
⟨ Output statistics about this job 1388 ⟩ ≡
  if log_opened then
    begin wlog_ln(´␣´); wlog_ln(´Here␣is␣how␣much␣of␣TeX´´s␣memory´, ´␣you␣used:´);
    wlog(´␣´, str_ptr − init_str_ptr : 1, ´␣string´);
    if str_ptr ≠ init_str_ptr + 1 then wlog(´s´);
    wlog_ln(´␣out␣of␣´, max_strings − init_str_ptr : 1);
    wlog_ln(´␣´, pool_ptr − init_pool_ptr : 1, ´␣string␣characters␣out␣of␣´, pool_size − init_pool_ptr : 1);
    wlog_ln(´␣´, lo_mem_max − mem_min + mem_end − hi_mem_min + 2 : 1,
        ´␣words␣of␣memory␣out␣of␣´, mem_end + 1 − mem_min : 1);
    wlog_ln(´␣´, cs_count : 1, ´␣multiletter␣control␣sequences␣out␣of␣´, hash_size : 1);
    wlog(´␣´, fmem_ptr : 1, ´␣words␣of␣font␣info␣for␣´, font_ptr − font_base : 1, ´␣font´);
    if font_ptr ≠ font_base + 1 then wlog(´s´);
    wlog_ln(´,␣out␣of␣´, font_mem_size : 1, ´␣for␣´, font_max − font_base : 1);
    wlog(´␣´, hyph_count : 1, ´␣hyphenation␣exception´);
    if hyph_count ≠ 1 then wlog(´s´);
    wlog_ln(´␣out␣of␣´, hyph_size : 1);
    wlog_ln(´␣´, max_in_stack : 1, ´i,´, max_nest_stack : 1, ´n,´, max_param_stack : 1, ´p,´,
        max_buf_stack + 1 : 1, ´b,´, max_save_stack + 6 : 1, ´s␣stack␣positions␣out␣of␣´,
        stack_size : 1, ´i,´, nest_size : 1, ´n,´, param_size : 1, ´p,´, buf_size : 1, ´b,´, save_size : 1, ´s´);
    end
```

This code is used in section 1387.

1389. We get to the *final_cleanup* routine when `\end` or `\dump` has been scanned and *its_all_over*.

```
⟨ Last-minute procedures 1387 ⟩ +≡
procedure final_cleanup;
  label exit;
  var c: small_number;  { 0 for \end, 1 for \dump }
  begin c ← cur_chr;
  if c ≠ 1 then new_line_char ← −1;
  if job_name = 0 then open_log_file;
  while input_ptr > 0 do
    if state = token_list then end_token_list else end_file_reading;
  while open_parens > 0 do
    begin print("␣)"); decr(open_parens);
    end;
  if cur_level > level_one then
    begin print_nl("("); print_esc("end␣occurred␣"); print("inside␣a␣group␣at␣level␣");
    print_int(cur_level − level_one); print_char(")");
    if eTeX_ex then show_save_groups;
    end;
  while cond_ptr ≠ null do
    begin print_nl("("); print_esc("end␣occurred␣"); print("when␣"); print_cmd_chr(if_test, cur_if);
    if if_line ≠ 0 then
      begin print("␣on␣line␣"); print_int(if_line);
      end;
    print("␣was␣incomplete)"); if_line ← if_line_field(cond_ptr); cur_if ← subtype(cond_ptr);
    temp_ptr ← cond_ptr; cond_ptr ← link(cond_ptr); free_node(temp_ptr, if_node_size);
    end;
  if history ≠ spotless then
    if ((history = warning_issued) ∨ (interaction < error_stop_mode)) then
      if selector = term_and_log then
        begin selector ← term_only;
        print_nl("(see␣the␣transcript␣file␣for␣additional␣information)");
        selector ← term_and_log;
        end;
  if c = 1 then
    begin init for c ← top_mark_code to split_bot_mark_code do
      if cur_mark[c] ≠ null then delete_token_ref(cur_mark[c]);
    if sa_mark ≠ null then
      if do_marks(destroy_marks, 0, sa_mark) then sa_mark ← null;
    for c ← last_box_code to vsplit_code do flush_node_list(disc_ptr[c]);
    if last_glue ≠ max_halfword then delete_glue_ref(last_glue);
    store_fmt_file; return; tini
    print_nl("(\dump␣is␣performed␣only␣by␣INITEX)"); return;
    end;
exit: end;
```

1390.

```
⟨ Last-minute procedures 1387 ⟩ +≡
  init procedure init_prim;  { initialize all the primitives }
  begin no_new_control_sequence ← false; first ← 0;
  ⟨ Put each of TeX's primitives into the hash table 252 ⟩;
  no_new_control_sequence ← true;
  end;
  tini
```

1391. When we begin the following code, TEX's tables may still contain garbage; the strings might not even be present. Thus we must proceed cautiously to get bootstrapped in.

But when we finish this part of the program, TEX is ready to call on the *main_control* routine to do its work.

```
⟨ Get the first line of input and prepare to start 1391 ⟩ ≡
  begin ⟨ Initialize the input routines 361 ⟩;
  ⟨ Enable ε-TEX, if requested 1451 ⟩
  if (format_ident = 0) ∨ (buffer[loc] = "&") then
    begin if format_ident ≠ 0 then initialize; { erase preloaded format }
    if ¬open_fmt_file then goto final_end;
    if ¬load_fmt_file then
      begin w_close(fmt_file); goto final_end;
      end;
    w_close(fmt_file);
    while (loc < limit) ∧ (buffer[loc] = "␣") do incr(loc);
    end;
  if eTeX_ex then wterm_ln('entering␣extended␣mode');
  if end_line_char_inactive then decr(limit)
  else buffer[limit] ← end_line_char;
  fix_date_and_time;
  random_seed ← (microseconds * 1000) + (epochseconds mod 1000000);
  init_randoms(random_seed);
  ⟨ Compute the magic offset 813 ⟩;
  ⟨ Initialize the print selector based on interaction 79 ⟩;
  if (loc < limit) ∧ (cat_code(buffer[loc]) ≠ escape) then start_input; { \input assumed }
  end
```

This code is used in section 1386.

1392. Debugging. Once TeX is working, you should be able to diagnose most errors with the `\show` commands and other diagnostic features. But for the initial stages of debugging, and for the revelation of really deep mysteries, you can compile TeX with a few more aids, including the Pascal runtime checks and its debugger. An additional routine called *debug_help* will also come into play when you type '`D`' after an error message; *debug_help* also occurs just before a fatal error causes TeX to succumb.

The interface to *debug_help* is primitive, but it is good enough when used with a Pascal debugger that allows you to set breakpoints and to read variables and change their values. After getting the prompt '`debug #`', you type either a negative number (this exits *debug_help*), or zero (this goes to a location where you can set a breakpoint, thereby entering into dialog with the Pascal debugger), or a positive number m followed by an argument n. The meaning of m and n will be clear from the program below. (If $m = 13$, there is an additional argument, l.)

```
define breakpoint = 888  { place where a breakpoint is desirable }
⟨ Last-minute procedures 1387 ⟩ +≡
  debug procedure debug_help;  { routine to display various things }
  label breakpoint, exit;
  var k, l, m, n: integer;
  begin clear_terminal;
  loop
    begin wake_up_terminal; print_nl("debug␣#␣(-1␣to␣exit):"); update_terminal; read(term_in, m);
    if m < 0 then return
    else if m = 0 then
        begin goto breakpoint;
          { go to every declared label at least once }
      breakpoint: m ← 0; @{´BREAKPOINT´@}
        end
      else begin read(term_in, n);
        case m of
        ⟨ Numbered cases for debug_help 1393 ⟩
        othercases print("?")
        endcases;
        end;
    end;
exit: end;
  gubed
```

1393. ⟨ Numbered cases for *debug_help* 1393 ⟩ ≡

```
1: print_word(mem[n]);  { display mem[n] in all forms }
2: print_int(info(n));
3: print_int(link(n));
4: print_word(eqtb[n]);
5: print_word(font_info[n]);
6: print_word(save_stack[n]);
7: show_box(n);  { show a box, abbreviated by show_box_depth and show_box_breadth }
8: begin breadth_max ← 10000; depth_threshold ← pool_size − pool_ptr − 10; show_node_list(n);
       { show a box in its entirety }
   end;
9: show_token_list(n, null, 1000);
10: slow_print(n);
11: check_mem(n > 0);  { check wellformedness; print new busy locations if n > 0 }
12: search_mem(n);  { look for pointers to n }
13: begin read(term_in, l); print_cmd_chr(n, l);
   end;
14: for k ← 0 to n do print(buffer[k]);
15: begin font_in_short_display ← null_font; short_display(n);
   end;
16: panicking ← ¬panicking;
```

This code is used in section 1392.

1394. Extensions. The program above includes a bunch of "hooks" that allow further capabilities to be added without upsetting TeX's basic structure. Most of these hooks are concerned with "whatsit" nodes, which are intended to be used for special purposes; whenever a new extension to TeX involves a new kind of whatsit node, a corresponding change needs to be made to the routines below that deal with such nodes, but it will usually be unnecessary to make many changes to the other parts of this program.

In order to demonstrate how extensions can be made, we shall treat '`\write`', '`\openout`', '`\closeout`', '`\immediate`', '`\special`', and '`\setlanguage`' as if they were extensions. These commands are actually primitives of TeX, and they should appear in all implementations of the system; but let's try to imagine that they aren't. Then the program below illustrates how a person could add them.

Sometimes, of course, an extension will require changes to TeX itself; no system of hooks could be complete enough for all conceivable extensions. The features associated with '`\write`' are almost all confined to the following paragraphs, but there are small parts of the *print_ln* and *print_char* procedures that were introduced specifically to `\write` characters. Furthermore one of the token lists recognized by the scanner is a *write_text*; and there are a few other miscellaneous places where we have already provided for some aspect of `\write`. The goal of a TeX extender should be to minimize alterations to the standard parts of the program, and to avoid them completely if possible. He or she should also be quite sure that there's no easy way to accomplish the desired goals with the standard features that TeX already has. "Think thrice before extending," because that may save a lot of work, and it will also keep incompatible extensions of TeX from proliferating.

1395. First let's consider the format of whatsit nodes that are used to represent the data associated with `\write` and its relatives. Recall that a whatsit has *type* = *whatsit_node*, and the *subtype* is supposed to distinguish different kinds of whatsits. Each node occupies two or more words; the exact number is immaterial, as long as it is readily determined from the *subtype* or other data.

We shall introduce five *subtype* values here, corresponding to the control sequences `\openout`, `\write`, `\closeout`, `\special`, and `\setlanguage`. The second word of I/O whatsits has a *write_stream* field that identifies the write-stream number (0 to 15, or 16 for out-of-range and positive, or 17 for out-of-range and negative). In the case of `\write` and `\special`, there is also a field that points to the reference count of a token list that should be sent. In the case of `\openout`, we need three words and three auxiliary subfields to hold the string numbers for name, area, and extension.

define *write_node_size* = 2 { number of words in a write/whatsit node }
define *open_node_size* = 3 { number of words in an open/whatsit node }
define *open_node* = 0 { *subtype* in whatsits that represent files to `\openout` }
define *write_node* = 1 { *subtype* in whatsits that represent things to `\write` }
define *close_node* = 2 { *subtype* in whatsits that represent streams to `\closeout` }
define *special_node* = 3 { *subtype* in whatsits that represent `\special` things }
define *language_node* = 4 { *subtype* in whatsits that change the current language }
define *what_lang*(#) ≡ *link*(# + 1) { language number, in the range 0 .. 255 }
define *what_lhm*(#) ≡ *type*(# + 1) { minimum left fragment, in the range 1 .. 63 }
define *what_rhm*(#) ≡ *subtype*(# + 1) { minimum right fragment, in the range 1 .. 63 }
define *write_tokens*(#) ≡ *link*(# + 1) { reference count of token list to write }
define *write_stream*(#) ≡ *info*(# + 1) { stream number (0 to 17) }
define *open_name*(#) ≡ *link*(# + 1) { string number of file name to open }
define *open_area*(#) ≡ *info*(# + 2) { string number of file area for *open_name* }
define *open_ext*(#) ≡ *link*(# + 2) { string number of file extension for *open_name* }

1396. The sixteen possible `\write` streams are represented by the *write_file* array. The jth file is open if and only if *write_open*[j] = *true*. The last two streams are special; *write_open*[16] represents a stream number greater than 15, while *write_open*[17] represents a negative stream number, and both of these variables are always *false*.

⟨ Global variables 13 ⟩ +≡
write_file: **array** [0 .. 15] **of** *alpha_file*;
write_open: **array** [0 .. 17] **of** *boolean*;

1397. ⟨ Set initial values of key variables 23 ⟩ +≡
for $k \leftarrow 0$ **to** 17 **do** *write_open*[k] ← *false*;

1398. Extensions might introduce new command codes; but it's best to use *extension* with a modifier, whenever possible, so that *main_control* stays the same.

define *immediate_code* = 4 { command modifier for `\immediate` }
define *set_language_code* = 5 { command modifier for `\setlanguage` }
define *pdftex_first_extension_code* = 6
define *pdf_save_pos_node* ≡ *pdftex_first_extension_code* + 15
define *reset_timer_code* ≡ *pdftex_first_extension_code* + 25
define *set_random_seed_code* ≡ *pdftex_first_extension_code* + 27
define *pic_file_code* = 41 { command modifier for `\XeTeXpicfile`, skipping codes pdfTeX might use }
define *pdf_file_code* = 42 { command modifier for `\XeTeXpdffile` }
define *glyph_code* = 43 { command modifier for `\XeTeXglyph` }
define *XeTeX_input_encoding_extension_code* = 44
define *XeTeX_default_encoding_extension_code* = 45
define *XeTeX_linebreak_locale_extension_code* = 46

⟨ Put each of TeX's primitives into the hash table 252 ⟩ +≡
primitive(`"openout"`, *extension*, *open_node*);
primitive(`"write"`, *extension*, *write_node*); *write_loc* ← *cur_val*;
primitive(`"closeout"`, *extension*, *close_node*);
primitive(`"special"`, *extension*, *special_node*);
primitive(`"immediate"`, *extension*, *immediate_code*);
primitive(`"setlanguage"`, *extension*, *set_language_code*);
primitive(`"resettimer"`, *extension*, *reset_timer_code*);
primitive(`"setrandomseed"`, *extension*, *set_random_seed_code*);

1399. The `\XeTeXpicfile` and `\XeTeXpdffile` primitives are only defined in extended mode.

⟨ Generate all ε-TeX primitives 1399 ⟩ ≡
primitive(`"XeTeXpicfile"`, *extension*, *pic_file_code*);
primitive(`"XeTeXpdffile"`, *extension*, *pdf_file_code*);
primitive(`"XeTeXglyph"`, *extension*, *glyph_code*);
primitive(`"XeTeXlinebreaklocale"`, *extension*, *XeTeX_linebreak_locale_extension_code*);
primitive(`"XeTeXinterchartoks"`, *assign_toks*, *XeTeX_inter_char_loc*);
primitive(`"pdfsavepos"`, *extension*, *pdf_save_pos_node*);

See also sections 1452, 1467, 1473, 1476, 1479, 1482, 1485, 1494, 1496, 1499, 1502, 1507, 1511, 1558, 1570, 1573, 1581, 1589, 1612, 1616, 1620, 1672, and 1675.

This code is used in section 1451.

1400. The variable *write_loc* just introduced is used to provide an appropriate error message in case of "runaway" write texts.

⟨ Global variables 13 ⟩ +≡
write_loc: *pointer*; { *eqtb* address of `\write` }

1401. ⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253 ⟩ +≡

```
extension: case chr_code of
  open_node: print_esc("openout");
  write_node: print_esc("write");
  close_node: print_esc("closeout");
  special_node: print_esc("special");
  immediate_code: print_esc("immediate");
  set_language_code: print_esc("setlanguage");
  pdf_save_pos_node: print_esc("pdfsavepos");
  reset_timer_code: print_esc("resettimer");
  set_random_seed_code: print_esc("setrandomseed");
  pic_file_code: print_esc("XeTeXpicfile");
  pdf_file_code: print_esc("XeTeXpdffile");
  glyph_code: print_esc("XeTeXglyph");
  XeTeX_linebreak_locale_extension_code: print_esc("XeTeXlinebreaklocale");
  XeTeX_input_encoding_extension_code: print_esc("XeTeXinputencoding");
  XeTeX_default_encoding_extension_code: print_esc("XeTeXdefaultencoding");
  othercases print("[unknown␣extension!]")
  endcases;
```

1402. When an *extension* command occurs in *main_control*, in any mode, the *do_extension* routine is called.

```
⟨ Cases of main_control that are for extensions to TeX 1402 ⟩ ≡
any_mode(extension): do_extension;
```

This code is used in section 1099.

1403. ⟨ Declare action procedures for use by *main_control* 1097 ⟩ +≡

```
⟨ Declare procedures needed in do_extension 1404 ⟩
procedure do_extension;
  var i, j, k: integer; { all-purpose integers }
    p, q, r: pointer; { all-purpose pointers }
  begin case cur_chr of
  open_node: ⟨ Implement \openout 1406 ⟩;
  write_node: ⟨ Implement \write 1407 ⟩;
  close_node: ⟨ Implement \closeout 1408 ⟩;
  special_node: ⟨ Implement \special 1409 ⟩;
  immediate_code: ⟨ Implement \immediate 1438 ⟩;
  set_language_code: ⟨ Implement \setlanguage 1440 ⟩;
  pdf_save_pos_node: ⟨ Implement \pdfsavepos 1450 ⟩;
  reset_timer_code: ⟨ Implement \resettimer 1414 ⟩;
  set_random_seed_code: ⟨ Implement \setrandomseed 1413 ⟩;
  pic_file_code: ⟨ Implement \XeTeXpicfile 1442 ⟩;
  pdf_file_code: ⟨ Implement \XeTeXpdffile 1443 ⟩;
  glyph_code: ⟨ Implement \XeTeXglyph 1444 ⟩;
  XeTeX_input_encoding_extension_code: ⟨ Implement \XeTeXinputencoding 1446 ⟩;
  XeTeX_default_encoding_extension_code: ⟨ Implement \XeTeXdefaultencoding 1447 ⟩;
  XeTeX_linebreak_locale_extension_code: ⟨ Implement \XeTeXlinebreaklocale 1448 ⟩;
  othercases confusion("ext1")
  endcases;
  end;
```

1404. Here is a subroutine that creates a whatsit node having a given *subtype* and a given number of words. It initializes only the first word of the whatsit, and appends it to the current list.

```
⟨ Declare procedures needed in do_extension 1404 ⟩ ≡
procedure new_whatsit(s : small_number; w : small_number);
  var p: pointer;  { the new node }
  begin p ← get_node(w); type(p) ← whatsit_node; subtype(p) ← s; link(tail) ← p; tail ← p;
  end;
```

See also sections 1405, 1445, and 1456.

This code is used in section 1403.

1405. The next subroutine uses *cur_chr* to decide what sort of whatsit is involved, and also inserts a *write_stream* number.

```
⟨ Declare procedures needed in do_extension 1404 ⟩ +≡
procedure new_write_whatsit(w : small_number);
  begin new_whatsit(cur_chr, w);
  if w ≠ write_node_size then scan_four_bit_int
  else begin scan_int;
    if cur_val < 0 then cur_val ← 17
    else if cur_val > 15 then cur_val ← 16;
    end;
  write_stream(tail) ← cur_val;
  end;
```

1406.

```
⟨ Implement \openout 1406 ⟩ ≡
  begin new_write_whatsit(open_node_size); scan_optional_equals; scan_file_name;
  open_name(tail) ← cur_name; open_area(tail) ← cur_area; open_ext(tail) ← cur_ext;
  end
```

This code is used in section 1403.

1407. When '`\write 12{...}`' appears, we scan the token list '`{...}`' without expanding its macros; the macros will be expanded later when this token list is rescanned.

```
⟨ Implement \write 1407 ⟩ ≡
  begin k ← cur_cs; new_write_whatsit(write_node_size);
  cur_cs ← k; p ← scan_toks(false, false); write_tokens(tail) ← def_ref;
  end
```

This code is used in section 1403.

1408.

```
⟨ Implement \closeout 1408 ⟩ ≡
  begin new_write_whatsit(write_node_size); write_tokens(tail) ← null;
  end
```

This code is used in section 1403.

1409. When '`\special{...}`' appears, we expand the macros in the token list as in `\xdef` and `\mark`.

```
⟨ Implement \special 1409 ⟩ ≡
  begin new_whatsit(special_node, write_node_size); write_stream(tail) ← null; p ← scan_toks(false, true);
  write_tokens(tail) ← def_ref;
  end
```

This code is used in section 1403.

1410. **define** *call_func*(#) ≡
  begin if # ≠ 0 **then** *do_nothing*
  end
define *flushable*(#) ≡ (# = *str_ptr* − 1)
define *max_integer* ≡ "7FFFFFFF { $2^{31}-1$ }
procedure *flush_str*(*s* : *str_number*); { flush a string if possible }
  begin if *flushable*(*s*) **then** *flush_string*;
  end;
function *tokens_to_string*(*p* : *pointer*): *str_number*; { return a string from tokens list }
  begin if *selector* = *new_string* **then**
    pdf_error("tokens", "tokens_to_string() called while selector = new_string");
  old_setting ← *selector*; *selector* ← *new_string*; *show_token_list*(*link*(*p*), *null*, *pool_size* − *pool_ptr*);
  selector ← *old_setting*; *tokens_to_string* ← *make_string*;
  end;
procedure *scan_pdf_ext_toks*;
  begin *call_func*(*scan_toks*(*false*, *true*)); { like \special }
  end;
procedure *compare_strings*; { to implement \strcmp }
  label *done*;
  var *s1*, *s2*: *str_number*; *i1*, *i2*, *j1*, *j2*: *pool_pointer*; *save_cur_cs*: *pointer*;
  begin *save_cur_cs* ← *cur_cs*; *call_func*(*scan_toks*(*false*, *true*)); *s1* ← *tokens_to_string*(*def_ref*);
  delete_token_ref(*def_ref*); *cur_cs* ← *save_cur_cs*; *call_func*(*scan_toks*(*false*, *true*));
  s2 ← *tokens_to_string*(*def_ref*); *delete_token_ref*(*def_ref*); *i1* ← *str_start_macro*(*s1*);
  j1 ← *str_start_macro*(*s1* + 1); *i2* ← *str_start_macro*(*s2*); *j2* ← *str_start_macro*(*s2* + 1);
  while (*i1* < *j1*) ∧ (*i2* < *j2*) **do**
    begin if *str_pool*[*i1*] < *str_pool*[*i2*] **then**
      begin *cur_val* ← −1; **goto** *done*;
      end;
    if *str_pool*[*i1*] > *str_pool*[*i2*] **then**
      begin *cur_val* ← 1; **goto** *done*;
      end;
    incr(*i1*); *incr*(*i2*);
    end;
  if (*i1* = *j1*) ∧ (*i2* = *j2*) **then** *cur_val* ← 0
  else if *i1* < *j1* **then** *cur_val* ← 1
    else *cur_val* ← −1;
done: *flush_str*(*s2*); *flush_str*(*s1*); *cur_val_level* ← *int_val*;
  end;

1411. ⟨ Declare procedures that need to be declared forward for pdfTeX 1411 ⟩ ≡
function *get_microinterval*: *integer*;
  var *s*, *m*: *integer*; { seconds and microseconds }
  begin *seconds_and_micros*(*s*, *m*);
  if (*s* − *epochseconds*) > 32767 **then** *get_microinterval* ← *max_integer*
  else if (*microseconds* > *m*) **then**
      get_microinterval ← ((*s*−1−*epochseconds*)∗65536)+(((*m*+1000000−*microseconds*)/100)∗65536)/10000
    else *get_microinterval* ← ((*s* − *epochseconds*) ∗ 65536) + (((*m* − *microseconds*)/100) ∗ 65536)/10000;
  end;
This code is used in section 198.

1412. ⟨ Set initial values of key variables 23 ⟩ +≡
  seconds_and_micros(*epochseconds*, *microseconds*); *init_start_time*;

1413. Negative random seed values are silently converted to positive ones

```
⟨ Implement \setrandomseed 1413 ⟩ ≡
  begin scan_int;
  if cur_val < 0 then negate(cur_val);
  random_seed ← cur_val; init_randoms(random_seed);
  end
```

This code is used in section 1403.

1414.

```
⟨ Implement \resettimer 1414 ⟩ ≡
  begin seconds_and_micros(epochseconds, microseconds);
  end
```

This code is used in section 1403.

1415. Each new type of node that appears in our data structure must be capable of being displayed, copied, destroyed, and so on. The routines that we need for write-oriented whatsits are somewhat like those for mark nodes; other extensions might, of course, involve more subtlety here.

```
⟨ Basic printing procedures 57 ⟩ +≡
procedure print_write_whatsit(s : str_number; p : pointer);
  begin print_esc(s);
  if write_stream(p) < 16 then print_int(write_stream(p))
  else if write_stream(p) = 16 then print_char("*")
    else print_char("-");
  end;
procedure print_native_word(p : pointer);
  var i, c, cc: integer;
  begin for i ← 0 to native_length(p) − 1 do
    begin c ← get_native_char(p, i);
    if (c ≥ "D800) ∧ (c ≤ "DBFF) then
      begin if i < native_length(p) − 1 then
        begin cc ← get_native_char(p, i + 1);
        if (cc ≥ "DC00) ∧ (cc ≤ "DFFF) then
          begin c ← "10000 + (c − "D800) ∗ "400 + (cc − "DC00); print_char(c); incr(i);
          end
        else print(".");
        end
      else print(".");
      end
    else print_char(c);
    end
  end;
```

1416. ⟨ Display the whatsit node p 1416 ⟩ ≡

```
case subtype(p) of
open_node: begin print_write_whatsit("openout", p); print_char("=");
  print_file_name(open_name(p), open_area(p), open_ext(p));
  end;
write_node: begin print_write_whatsit("write", p); print_mark(write_tokens(p));
  end;
close_node: print_write_whatsit("closeout", p);
special_node: begin print_esc("special"); print_mark(write_tokens(p));
  end;
language_node: begin print_esc("setlanguage"); print_int(what_lang(p)); print("␣(hyphenmin␣");
  print_int(what_lhm(p)); print_char(","); print_int(what_rhm(p)); print_char(")");
  end;
pdf_save_pos_node: print_esc("pdfsavepos");
native_word_node, native_word_node_AT: begin print_esc(font_id_text(native_font(p))); print_char("␣");
  print_native_word(p);
  end;
glyph_node: begin print_esc(font_id_text(native_font(p))); print("␣glyph#"); print_int(native_glyph(p));
  end;
pic_node, pdf_node: begin if subtype(p) = pic_node then print_esc("XeTeXpicfile")
  else print_esc("XeTeXpdffile");
  print("␣""");
  for i ← 0 to pic_path_length(p) − 1 do print_visible_char(pic_path_byte(p, i));
  print("""");
  end;
othercases print("whatsit?")
endcases
```

This code is used in section 209.

1417. Picture nodes are tricky in that they are variable size.

```
define total_pic_node_size(#) ≡ (pic_node_size + (pic_path_length(#) + sizeof(memory_word) − 1) div
        sizeof(memory_word))
⟨ Make a partial copy of the whatsit node p and make r point to it; set words to the number of initial words
      not yet copied 1417 ⟩ ≡
  case subtype(p) of
  open_node: begin r ← get_node(open_node_size); words ← open_node_size;
    end;
  write_node, special_node: begin r ← get_node(write_node_size); add_token_ref(write_tokens(p));
    words ← write_node_size;
    end;
  close_node, language_node: begin r ← get_node(small_node_size); words ← small_node_size;
    end;
  native_word_node, native_word_node_AT: begin words ← native_size(p); r ← get_node(words);
    while words > 0 do
      begin decr(words); mem[r + words] ← mem[p + words];
      end;
    native_glyph_info_ptr(r) ← null_ptr; native_glyph_count(r) ← 0; copy_native_glyph_info(p, r);
    end;
  glyph_node: begin r ← get_node(glyph_node_size); words ← glyph_node_size;
    end;
  pic_node, pdf_node: begin words ← total_pic_node_size(p); r ← get_node(words);
    end;
  pdf_save_pos_node: r ← get_node(small_node_size);
  othercases confusion("ext2")
  endcases
```

This code is used in sections 232 and 1544.

1418. ⟨ Wipe out the whatsit node p and **goto** *done* 1418 ⟩ ≡

```
  begin case subtype(p) of
  open_node: free_node(p, open_node_size);
  write_node, special_node: begin delete_token_ref(write_tokens(p)); free_node(p, write_node_size);
    goto done;
    end;
  close_node, language_node: free_node(p, small_node_size);
  native_word_node, native_word_node_AT: begin free_native_glyph_info(p); free_node(p, native_size(p));
    end;
  glyph_node: free_node(p, glyph_node_size);
  pic_node, pdf_node: free_node(p, total_pic_node_size(p));
  pdf_save_pos_node: free_node(p, small_node_size);
  othercases confusion("ext3")
  endcases;
  goto done;
  end
```

This code is used in section 228.

1419. ⟨ Incorporate a whatsit node into a vbox 1419 ⟩ ≡

```
begin if (subtype(p) = pic_node) ∨ (subtype(p) = pdf_node) then
  begin x ← x + d + height(p); d ← depth(p);
  if width(p) > w then w ← width(p);
  end;
end
```

This code is used in section 711.

1420. ⟨ Incorporate a whatsit node into an hbox 1420 ⟩ ≡

```
begin case subtype(p) of
native_word_node, native_word_node_AT: begin
        { merge with any following word fragments in same font, discarding discretionary breaks }
  if (q ≠ r + list_offset) ∧ (type(q) = disc_node) then k ← replace_count(q)
  else k ← 0;
  while (link(q) ≠ p) do
    begin decr(k); q ← link(q);  { bring q up in preparation for deletion of nodes starting at p }
    if type(q) = disc_node then k ← replace_count(q);
    end;
  pp ← link(p);
restart: if (k ≤ 0) ∧ (pp ≠ null) ∧ (¬is_char_node(pp)) then
    begin if (type(pp) = whatsit_node)∧(is_native_word_subtype(pp))∧(native_font(pp) = native_font(p))
          then
      begin pp ← link(pp); goto restart;
      end
    else if (type(pp) = disc_node) then
        begin ppp ← link(pp);
        if is_native_word_node(ppp) ∧ (native_font(ppp) = native_font(p)) then
          begin pp ← link(ppp); goto restart;
          end
        end
    end;  { now pp points to the non-native_word node that ended the chain, or null }
      { we can just check type(p)=whatsit_node below, as we know that the chain contains only
        discretionaries and native_word nodes, no other whatsits or char_nodes }
  if (pp ≠ link(p)) then
    begin  { found a chain of at least two pieces starting at p }
    total_chars ← 0; p ← link(q);  { the first fragment }
    while (p ≠ pp) do
      begin if (type(p) = whatsit_node) then total_chars ← total_chars + native_length(p);
            { accumulate char count }
      ppp ← p;  { remember last node seen }
      p ← link(p);  { point to next fragment or discretionary or terminator }
      end;
    p ← link(q);  { the first fragment again }
    pp ← new_native_word_node(native_font(p), total_chars);  { make new node for merged word }
    subtype(pp) ← subtype(p); link(q) ← pp;  { link to preceding material }
    link(pp) ← link(ppp);  { attach remainder of hlist to it }
    link(ppp) ← null;  { and detach from the old fragments }
      { copy the chars into new node }
    total_chars ← 0; ppp ← p;
    repeat if (type(ppp) = whatsit_node) then
        for k ← 0 to native_length(ppp) − 1 do
          begin set_native_char(pp, total_chars, get_native_char(ppp, k)); incr(total_chars);
          end;
      ppp ← link(ppp);
    until (ppp = null);
    flush_node_list(p);  { delete the fragments }
    p ← link(q);  { update p to point to the new node }
    set_native_metrics(p, XeTeX_use_glyph_metrics);  { and measure it (i.e., re-do the OT layout) }
    end;  { now incorporate the native_word node measurements into the box we're packing }
  if height(p) > h then h ← height(p);
```

```
    if depth(p) > d then d ← depth(p);
    x ← x + width(p);
    end;
  glyph_node, pic_node, pdf_node: begin if height(p) > h then h ← height(p);
    if depth(p) > d then d ← depth(p);
    x ← x + width(p);
    end;
  othercases do_nothing
  endcases;
  end
```

This code is used in section 691.

1421. ⟨ Let d be the width of the whatsit p, and **goto** *found* if "visible" 1421 ⟩ ≡

```
if (is_native_word_subtype(p)) ∨ (subtype(p) = glyph_node) ∨ (subtype(p) = pic_node) ∨ (subtype(p) =
      pdf_node) then
  begin d ← width(p); goto found;
  end
else d ← 0
```

This code is used in section 1201.

1422.

```
define adv_past_linebreak(#) ≡ if subtype(#) = language_node then
    begin cur_lang ← what_lang(#); l_hyf ← what_lhm(#); r_hyf ← what_rhm(#); set_hyph_index;
    end
  else if (is_native_word_subtype(#)) ∨ (subtype(#) = glyph_node) ∨ (subtype(#) =
      pic_node) ∨ (subtype(#) = pdf_node) then
    begin act_width ← act_width + width(#);
    end
```

⟨ Advance past a whatsit node in the *line_break* loop 1422 ⟩ ≡ *adv_past_linebreak*(*cur_p*)

This code is used in section 914.

1423.

```
define adv_past_prehyph(#) ≡ if subtype(#) = language_node then
    begin cur_lang ← what_lang(#); l_hyf ← what_lhm(#); r_hyf ← what_rhm(#); set_hyph_index;
    end
```

⟨ Advance past a whatsit node in the pre-hyphenation loop 1423 ⟩ ≡ *adv_past_prehyph*(*s*)

This code is used in section 949.

1424. ⟨ Prepare to move whatsit p to the current page, then **goto** *contribute* 1424 ⟩ ≡

```
begin if (subtype(p) = pic_node) ∨ (subtype(p) = pdf_node) then
  begin page_total ← page_total + page_depth + height(p); page_depth ← depth(p);
  end;
goto contribute;
end
```

This code is used in section 1054.

1425. ⟨ Process whatsit p in *vert_break* loop, **goto** *not_found* 1425 ⟩ ≡

```
begin if (subtype(p) = pic_node) ∨ (subtype(p) = pdf_node) then
  begin cur_height ← cur_height + prev_dp + height(p); prev_dp ← depth(p);
  end;
goto not_found;
end
```

This code is used in section 1027.

1426. ⟨ Output the whatsit node p in a vlist 1426 ⟩ ≡

```
begin case subtype(p) of
glyph_node: begin cur_v ← cur_v + height(p); cur_h ← left_edge; synch_h; synch_v;
      { Sync DVI state to TeX state }
  f ← native_font(p);
  if f ≠ dvi_f then ⟨ Change font dvi_f to f 659 ⟩;
  dvi_out(set_glyphs); dvi_four(0);  { width }
  dvi_two(1);  { glyph count }
  dvi_four(0);  { x-offset as fixed point }
  dvi_four(0);  { y-offset as fixed point }
  dvi_two(native_glyph(p)); cur_v ← cur_v + depth(p); cur_h ← left_edge;
  end;
pic_node, pdf_node: begin save_h ← dvi_h; save_v ← dvi_v; cur_v ← cur_v + height(p); pic_out(p);
  dvi_h ← save_h; dvi_v ← save_v; cur_v ← save_v + depth(p); cur_h ← left_edge;
  end;
pdf_save_pos_node: ⟨ Save current position to pdf_last_x_pos, pdf_last_y_pos 1427 ⟩;
othercases out_what(p)
endcases
end
```

This code is used in section 669.

1427. ⟨ Save current position to *pdf_last_x_pos*, *pdf_last_y_pos* 1427 ⟩ ≡

```
begin pdf_last_x_pos ← cur_h + 4736286; pdf_last_y_pos ← cur_page_height − cur_v − 4736286;
end
```

This code is used in sections 1426 and 1430.

1428. ⟨ Calculate page dimensions and margins 1428 ⟩ ≡

```
cur_h_offset ← h_offset + (unity * 7227)/100; cur_v_offset ← v_offset + (unity * 7227)/100;
if pdf_page_width ≠ 0 then cur_page_width ← pdf_page_width
else cur_page_width ← width(p) + 2 * cur_h_offset;
if pdf_page_height ≠ 0 then cur_page_height ← pdf_page_height
else cur_page_height ← height(p) + depth(p) + 2 * cur_v_offset
```

This code is used in section 653.

1429. ⟨ Global variables 13 ⟩ +≡

```
cur_page_width: scaled;  { width of page being shipped }
cur_page_height: scaled;  { height of page being shipped }
cur_h_offset: scaled;  { horizontal offset of page being shipped }
cur_v_offset: scaled;  { vertical offset of page being shipped }
```

1430. ⟨ Output the whatsit node p in an hlist 1430 ⟩ ≡

```
begin case subtype(p) of
native_word_node, native_word_node_AT, glyph_node: begin synch_h; synch_v;
        { Sync DVI state to TeX state }
    f ← native_font(p);
    if f ≠ dvi_f then ⟨ Change font dvi_f to f 659 ⟩;
    if subtype(p) = glyph_node then
      begin dvi_out(set_glyphs); dvi_four(width(p)); dvi_two(1);   { glyph count }
      dvi_four(0);   { x-offset as fixed point }
      dvi_four(0);   { y-offset as fixed point }
      dvi_two(native_glyph(p)); cur_h ← cur_h + width(p);
      end
    else begin if subtype(p) = native_word_node_AT then
        begin if (native_length(p) > 0) ∨ (native_glyph_info_ptr(p) ≠ null_ptr) then
          begin dvi_out(set_text_and_glyphs); len ← native_length(p); dvi_two(len);
          for k ← 0 to len − 1 do
            begin dvi_two(get_native_char(p, k));
            end;
          len ← make_xdv_glyph_array_data(p);
          for k ← 0 to len − 1 do dvi_out(xdv_buffer_byte(k));
          end
        end
      else begin if native_glyph_info_ptr(p) ≠ null_ptr then
          begin dvi_out(set_glyphs); len ← make_xdv_glyph_array_data(p);
          for k ← 0 to len − 1 do dvi_out(xdv_buffer_byte(k));
          end
        end;
      cur_h ← cur_h + width(p);
      end;
    dvi_h ← cur_h;
    end;
pic_node, pdf_node: begin save_h ← dvi_h; save_v ← dvi_v; cur_v ← base_line; edge ← cur_h + width(p);
    pic_out(p); dvi_h ← save_h; dvi_v ← save_v; cur_h ← edge; cur_v ← base_line;
    end;
pdf_save_pos_node: ⟨ Save current position to pdf_last_x_pos, pdf_last_y_pos 1427 ⟩;
othercases out_what(p)
endcases
end
```

This code is used in section 660.

1431. After all this preliminary shuffling, we come finally to the routines that actually send out the requested data. Let's do `\special` first (it's easier).

```
⟨ Declare procedures needed in hlist_out, vlist_out 1431 ⟩ ≡
procedure special_out(p : pointer);
  var old_setting: 0 .. max_selector;  { holds print selector }
    k: pool_pointer;  { index into str_pool }
  begin synch_h; synch_v;
  doing_special ← true; old_setting ← selector; selector ← new_string;
  show_token_list(link(write_tokens(p)), null, pool_size − pool_ptr); selector ← old_setting; str_room(1);
  if cur_length < 256 then
    begin dvi_out(xxx1); dvi_out(cur_length);
    end
  else begin dvi_out(xxx4); dvi_four(cur_length);
    end;
  for k ← str_start_macro(str_ptr) to pool_ptr − 1 do dvi_out(so(str_pool[k]));
  pool_ptr ← str_start_macro(str_ptr);  { erase the string }
  doing_special ← false;
  end;
```

See also sections 1433, 1436, 1529, and 1533.

This code is used in section 655.

1432. To write a token list, we must run it through TeX's scanner, expanding macros and `\the` and `\number`, etc. This might cause runaways, if a delimited macro parameter isn't matched, and runaways would be extremely confusing since we are calling on TeX's scanner in the middle of a `\shipout` command. Therefore we will put a dummy control sequence as a "stopper," right after the token list. This control sequence is artificially defined to be `\outer`.

```
⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡
  text(end_write) ← "endwrite"; eq_level(end_write) ← level_one; eq_type(end_write) ← outer_call;
  equiv(end_write) ← null;
```

1433.

```
⟨ Declare procedures needed in hlist_out, vlist_out 1431 ⟩ +≡
procedure write_out(p : pointer);
  var old_setting: 0 .. max_selector;  { holds print selector }
    old_mode: integer;  { saved mode }
    j: small_number;  { write stream number }
    k: integer; q, r: pointer;  { temporary variables for list manipulation }
  begin ⟨ Expand macros in the token list and make link(def_ref) point to the result 1434 ⟩;
  old_setting ← selector; j ← write_stream(p);
  if write_open[j] then selector ← j
  else begin  { write to the terminal if file isn't open }
    if (j = 17) ∧ (selector = term_and_log) then selector ← log_only;
    print_nl("");
    end;
  token_show(def_ref); print_ln; flush_list(def_ref); selector ← old_setting;
  end;
```

1434. The final line of this routine is slightly subtle; at least, the author didn't think about it until getting burnt! There is a used-up token list on the stack, namely the one that contained *end_write_token*. (We insert this artificial '`\endwrite`' to prevent runaways, as explained above.) If it were not removed, and if there were numerous writes on a single page, the stack would overflow.

define *end_write_token* ≡ *cs_token_flag* + *end_write*

⟨ Expand macros in the token list and make *link*(*def_ref*) point to the result 1434 ⟩ ≡
q ← *get_avail*; *info*(*q*) ← *right_brace_token* + "}";
r ← *get_avail*; *link*(*q*) ← *r*; *info*(*r*) ← *end_write_token*; *ins_list*(*q*);
begin_token_list(*write_tokens*(*p*), *write_text*);
q ← *get_avail*; *info*(*q*) ← *left_brace_token* + "{"; *ins_list*(*q*);
{ now we're ready to scan '{⟨ token list ⟩} `\endwrite`' }
old_mode ← *mode*; *mode* ← 0; { disable `\prevdepth`, `\spacefactor`, `\lastskip`, `\prevgraf` }
cur_cs ← *write_loc*; *q* ← *scan_toks*(*false*, *true*); { expand macros, etc. }
get_token; **if** *cur_tok* ≠ *end_write_token* **then** ⟨ Recover from an unbalanced write command 1435 ⟩;
mode ← *old_mode*; *end_token_list* { conserve stack space }

This code is used in section 1433.

1435. ⟨ Recover from an unbalanced write command 1435 ⟩ ≡
begin *print_err*("Unbalanced␣`write`␣command");
help2("On␣`this`␣`page`␣`there´s`␣`a`␣`\write`␣`with`␣`fewer`␣`real`␣{´s␣`than`␣}´`s.`")
("I␣`can´t`␣`handle`␣`that`␣`very`␣`well;`␣`good`␣`luck.`"); *error*;
repeat *get_token*;
until *cur_tok* = *end_write_token*;
end

This code is used in section 1434.

1436. The *out_what* procedure takes care of outputting whatsit nodes for *vlist_out* and *hlist_out*.

```
⟨ Declare procedures needed in hlist_out, vlist_out 1431 ⟩ +≡
procedure pic_out(p : pointer);
  var old_setting: 0 .. max_selector; { holds print selector }
    i: integer; k: pool_pointer; { index into str_pool }
  begin synch_h; synch_v; old_setting ← selector; selector ← new_string; print("pdf:image␣");
  print("matrix␣"); print_scaled(pic_transform1(p)); print("␣"); print_scaled(pic_transform2(p));
  print("␣"); print_scaled(pic_transform3(p)); print("␣"); print_scaled(pic_transform4(p)); print("␣");
  print_scaled(pic_transform5(p)); print("␣"); print_scaled(pic_transform6(p)); print("␣");
  print("page␣"); print_int(pic_page(p)); print("␣");
  case pic_pdf_box(p) of
  pdfbox_crop: print("pagebox␣cropbox␣");
  pdfbox_media: print("pagebox␣mediabox␣");
  pdfbox_bleed: print("pagebox␣bleedbox␣");
  pdfbox_art: print("pagebox␣artbox␣");
  pdfbox_trim: print("pagebox␣trimbox␣");
  others: do_nothing;
  endcases; print("(");
  for i ← 0 to pic_path_length(p) − 1 do print_visible_char(pic_path_byte(p, i));
  print(")"); selector ← old_setting;
  if cur_length < 256 then
    begin dvi_out(xxx1); dvi_out(cur_length);
    end
  else begin dvi_out(xxx4); dvi_four(cur_length);
    end;
  for k ← str_start_macro(str_ptr) to pool_ptr − 1 do dvi_out(so(str_pool[k]));
  pool_ptr ← str_start_macro(str_ptr); { erase the string }
  end;
procedure out_what(p : pointer);
  var j: small_number; { write stream number }
  begin case subtype(p) of
  open_node, write_node, close_node: ⟨ Do some work that has been queued up for \write 1437 ⟩;
  special_node: special_out(p);
  language_node: do_nothing;
  othercases confusion("ext4")
  endcases;
  end;
```

1437. We don't implement `\write` inside of leaders. (The reason is that the number of times a leader box appears might be different in different implementations, due to machine-dependent rounding in the glue calculations.)

```
⟨ Do some work that has been queued up for \write 1437 ⟩ ≡
  if ¬doing_leaders then
    begin j ← write_stream(p);
    if subtype(p) = write_node then write_out(p)
    else begin if write_open[j] then a_close(write_file[j]);
      if subtype(p) = close_node then write_open[j] ← false
      else if j < 16 then
          begin cur_name ← open_name(p); cur_area ← open_area(p); cur_ext ← open_ext(p);
          if cur_ext = "" then cur_ext ← ".tex";
          pack_cur_name;
          while ¬a_open_out(write_file[j]) do prompt_file_name("output␣file␣name", ".tex");
          write_open[j] ← true;
          end;
      end;
    end
```

This code is used in section 1436.

1438. The presence of '`\immediate`' causes the *do_extension* procedure to descend to one level of recursion. Nothing happens unless `\immediate` is followed by '`\openout`', '`\write`', or '`\closeout`'.

```
⟨ Implement \immediate 1438 ⟩ ≡
  begin get_x_token;
  if (cur_cmd = extension) ∧ (cur_chr ≤ close_node) then
    begin p ← tail; do_extension; { append a whatsit node }
    out_what(tail); { do the action immediately }
    flush_node_list(tail); tail ← p; link(p) ← null;
    end
  else back_input;
  end
```

This code is used in section 1403.

1439. The `\language` extension is somewhat different. We need a subroutine that comes into play when a character of a non-*clang* language is being appended to the current paragraph.

```
⟨ Declare action procedures for use by main_control 1097 ⟩ +≡
procedure fix_language;
  var l: ASCII_code; { the new current language }
  begin if language ≤ 0 then l ← 0
  else if language > 255 then l ← 0
    else l ← language;
  if l ≠ clang then
    begin new_whatsit(language_node, small_node_size); what_lang(tail) ← l; clang ← l;
    what_lhm(tail) ← norm_min(left_hyphen_min); what_rhm(tail) ← norm_min(right_hyphen_min);
    end;
  end;
```

1440. ⟨ Implement `\setlanguage` 1440 ⟩ ≡

```
if abs(mode) ≠ hmode then report_illegal_case
else begin new_whatsit(language_node, small_node_size); scan_int;
  if cur_val ≤ 0 then clang ← 0
  else if cur_val > 255 then clang ← 0
    else clang ← cur_val;
  what_lang(tail) ← clang; what_lhm(tail) ← norm_min(left_hyphen_min);
  what_rhm(tail) ← norm_min(right_hyphen_min);
  end
```

This code is used in section 1403.

1441. ⟨ Finish the extensions 1441 ⟩ ≡

```
terminate_font_manager;
for k ← 0 to 15 do
  if write_open[k] then a_close(write_file[k])
```

This code is used in section 1387.

1442. ⟨ Implement `\XeTeXpicfile` 1442 ⟩ ≡

```
if abs(mode) = mmode then report_illegal_case
else load_picture(false)
```

This code is used in section 1403.

1443. ⟨ Implement `\XeTeXpdffile` 1443 ⟩ ≡

```
if abs(mode) = mmode then report_illegal_case
else load_picture(true)
```

This code is used in section 1403.

1444. ⟨ Implement `\XeTeXglyph` 1444 ⟩ ≡

```
begin if abs(mode) = vmode then
  begin back_input; new_graf(true);
  end
else if abs(mode) = mmode then report_illegal_case
  else begin if is_native_font(cur_font) then
      begin new_whatsit(glyph_node, glyph_node_size); scan_int;
      if (cur_val < 0) ∨ (cur_val > 65535) then
        begin print_err("Bad␣glyph␣number");
        help2("A␣glyph␣number␣must␣be␣between␣0␣and␣65535.")
        ("I␣changed␣this␣one␣to␣zero."); int_error(cur_val); cur_val ← 0;
        end;
      native_font(tail) ← cur_font; native_glyph(tail) ← cur_val;
      set_native_glyph_metrics(tail, XeTeX_use_glyph_metrics);
      end
    else not_native_font_error(extension, glyph_code, cur_font);
    end
end
```

This code is used in section 1403.

1445. Load a picture file and handle following keywords.

```
define calc_min_and_max ≡
        begin xmin ← 1000000.0; xmax ← −xmin; ymin ← xmin; ymax ← xmax;
        for i ← 0 to 3 do
          begin if xCoord(corners[i]) < xmin then xmin ← xCoord(corners[i]);
          if xCoord(corners[i]) > xmax then xmax ← xCoord(corners[i]);
          if yCoord(corners[i]) < ymin then ymin ← yCoord(corners[i]);
          if yCoord(corners[i]) > ymax then ymax ← yCoord(corners[i]);
          end;
        end
define update_corners ≡
        for i ← 0 to 3 do transform_point(addressof(corners[i]), addressof(t2))
define do_size_requests ≡
        begin { calculate current width and height }
        calc_min_and_max;
        if x_size_req = 0.0 then
          begin make_scale(addressof(t2), y_size_req/(ymax − ymin), y_size_req/(ymax − ymin));
          end
        else if y_size_req = 0.0 then
            begin make_scale(addressof(t2), x_size_req/(xmax − xmin), x_size_req/(xmax − xmin));
            end
          else begin make_scale(addressof(t2), x_size_req/(xmax − xmin), y_size_req/(ymax − ymin));
            end;
        update_corners; x_size_req ← 0.0; y_size_req ← 0.0;
        transform_concat(addressof(t), addressof(t2));
        end
```

⟨ Declare procedures needed in *do_extension* 1404 ⟩ +≡

```
procedure load_picture(is_pdf : boolean);
  var pic_path: ↑char; bounds: real_rect; t, t2: transform; corners: array [0 .. 3] of real_point;
    x_size_req, y_size_req: real; check_keywords: boolean; xmin, xmax, ymin, ymax: real; i: small_number;
    page: integer; pdf_box_type: integer; result: integer;
  begin { scan the filename and pack into name_of_file }
  scan_file_name; pack_cur_name; pdf_box_type ← 0; page ← 0;
  if is_pdf then
    begin if scan_keyword("page") then
      begin scan_int; page ← cur_val;
      end;
    pdf_box_type ← pdfbox_none;
    if scan_keyword("crop") then pdf_box_type ← pdfbox_crop
    else if scan_keyword("media") then pdf_box_type ← pdfbox_media
      else if scan_keyword("bleed") then pdf_box_type ← pdfbox_bleed
        else if scan_keyword("trim") then pdf_box_type ← pdfbox_trim
          else if scan_keyword("art") then pdf_box_type ← pdfbox_art;
    end; { access the picture file and check its size }
  if pdf_box_type = pdfbox_none then
    result ← find_pic_file(addressof(pic_path), addressof(bounds), pdfbox_crop, page)
  else result ← find_pic_file(addressof(pic_path), addressof(bounds), pdf_box_type, page);
  setPoint(corners[0], xField(bounds), yField(bounds));
  setPoint(corners[1], xField(corners[0]), yField(bounds) + htField(bounds));
  setPoint(corners[2], xField(bounds) + wdField(bounds), yField(corners[1]));
  setPoint(corners[3], xField(corners[2]), yField(corners[0])); x_size_req ← 0.0; y_size_req ← 0.0;
      { look for any scaling requests for this picture }
```

```
make_identity(addressof(t)); check_keywords ← true;
while check_keywords do
  begin if scan_keyword("scaled") then
    begin scan_int;
    if (x_size_req = 0.0) ∧ (y_size_req = 0.0) then
      begin make_scale(addressof(t2), float(cur_val)/1000.0, float(cur_val)/1000.0); update_corners;
      transform_concat(addressof(t), addressof(t2));
      end
    end
  else if scan_keyword("xscaled") then
      begin scan_int;
      if (x_size_req = 0.0) ∧ (y_size_req = 0.0) then
        begin make_scale(addressof(t2), float(cur_val)/1000.0, 1.0); update_corners;
        transform_concat(addressof(t), addressof(t2));
        end
      end
    else if scan_keyword("yscaled") then
        begin scan_int;
        if (x_size_req = 0.0) ∧ (y_size_req = 0.0) then
          begin make_scale(addressof(t2), 1.0, float(cur_val)/1000.0); update_corners;
          transform_concat(addressof(t), addressof(t2));
          end
        end
      else if scan_keyword("width") then
          begin scan_normal_dimen;
          if cur_val ≤ 0 then
            begin print_err("Improper␣image␣"); print("size␣("); print_scaled(cur_val);
            print("pt)␣will␣be␣ignored");
            help2("I␣can´t␣scale␣images␣to␣zero␣or␣negative␣sizes,")
            ("so␣I´m␣ignoring␣this."); error;
            end
          else x_size_req ← Fix2D(cur_val);
          end
        else if scan_keyword("height") then
            begin scan_normal_dimen;
            if cur_val ≤ 0 then
              begin print_err("Improper␣image␣"); print("size␣("); print_scaled(cur_val);
              print("pt)␣will␣be␣ignored");
              help2("I␣can´t␣scale␣images␣to␣zero␣or␣negative␣sizes,")
              ("so␣I´m␣ignoring␣this."); error;
              end
            else y_size_req ← Fix2D(cur_val);
            end
          else if scan_keyword("rotated") then
              begin scan_decimal;
              if (x_size_req ≠ 0.0) ∨ (y_size_req ≠ 0.0) then do_size_requests;
              make_rotation(addressof(t2), Fix2D(cur_val) * 3.141592653589793/180.0);
              update_corners; calc_min_and_max; setPoint(corners[0], xmin, ymin);
              setPoint(corners[1], xmin, ymax); setPoint(corners[2], xmax, ymax);
              setPoint(corners[3], xmax, ymin); transform_concat(addressof(t), addressof(t2));
              end
            else check_keywords ← false;
```

```
    end;
  if (x_size_req ≠ 0.0) ∨ (y_size_req ≠ 0.0) then do_size_requests;
  calc_min_and_max; make_translation(addressof(t2), −xmin * 72/72.27, −ymin * 72/72.27);
  transform_concat(addressof(t), addressof(t2));
  if result = 0 then
    begin new_whatsit(pic_node,
        pic_node_size + (strlen(pic_path) + sizeof(memory_word) − 1) div sizeof(memory_word));
    if is_pdf then
      begin subtype(tail) ← pdf_node;
      end;
    pic_path_length(tail) ← strlen(pic_path); pic_page(tail) ← page; pic_pdf_box(tail) ← pdf_box_type;
    width(tail) ← D2Fix(xmax − xmin); height(tail) ← D2Fix(ymax − ymin); depth(tail) ← 0;
    pic_transform1(tail) ← D2Fix(aField(t)); pic_transform2(tail) ← D2Fix(bField(t));
    pic_transform3(tail) ← D2Fix(cField(t)); pic_transform4(tail) ← D2Fix(dField(t));
    pic_transform5(tail) ← D2Fix(xField(t)); pic_transform6(tail) ← D2Fix(yField(t));
    memcpy(addressof(mem[tail + pic_node_size]), pic_path, strlen(pic_path)); libc_free(pic_path);
    end
  else begin print_err("Unable to load picture or PDF file ´");
    print_file_name(cur_name, cur_area, cur_ext); print("´");
    if result = −43 then
      begin { Mac OS file not found error }
      help2("The requested image couldn´t be read because")
      ("the file was not found.");
      end
    else begin { otherwise assume GraphicImport failed }
      help2("The requested image couldn´t be read because")
      ("it was not a recognized image format.");
      end;
    error;
    end;
  end;
```

1446. ⟨ Implement `\XeTeXinputencoding` 1446 ⟩ ≡

```
  begin scan_and_pack_name; { scan a filename-like arg for the input encoding }
  i ← get_encoding_mode_and_info(addressof(j)); { convert it to mode and encoding values }
  if i = XeTeX_input_mode_auto then
    begin print_err("Encoding mode `auto´ is not valid for \XeTeXinputencoding");
    help2("You can´t use `auto´ encoding here, only for \XeTeXdefaultencoding.")
    ("I´ll ignore this and leave the current encoding unchanged.");
    error;
    end
  else set_input_file_encoding(input_file[in_open], i, j); { apply them to the current input file }
  end
```

This code is used in section 1403.

1447. ⟨ Implement `\XeTeXdefaultencoding` 1447 ⟩ ≡

```
  begin scan_and_pack_name; { scan a filename-like arg for the input encoding }
  i ← get_encoding_mode_and_info(addressof(j)); { convert it to mode and encoding values }
  XeTeX_default_input_mode ← i; { store them as defaults for new input files }
  XeTeX_default_input_encoding ← j;
  end
```

This code is used in section 1403.

1448. ⟨ Implement `\XeTeXlinebreaklocale` 1448 ⟩ ≡
begin *scan_file_name*; { scan a filename-like arg for the locale name }
if *length*(*cur_name*) = 0 **then** *XeTeX_linebreak_locale* ← 0
else *XeTeX_linebreak_locale* ← *cur_name*; { we ignore the area and extension! }
end
This code is used in section 1403.

1449. ⟨ Global variables 13 ⟩ +≡
pdf_last_x_pos: *integer*;
pdf_last_y_pos: *integer*;

1450. ⟨ Implement `\pdfsavepos` 1450 ⟩ ≡
begin *new_whatsit*(*pdf_save_pos_node*, *small_node_size*);
end
This code is used in section 1403.

1451. The extended features of ε-TEX. The program has two modes of operation: (1) In TEX compatibility mode it fully deserves the name TEX and there are neither extended features nor additional primitive commands. There are, however, a few modifications that would be legitimate in any implementation of TEX such as, e.g., preventing inadequate results of the glue to `DVI` unit conversion during *ship_out*. (2) In extended mode there are additional primitive commands and the extended features of ε-TEX are available.

The distinction between these two modes of operation initially takes place when a 'virgin' `eINITEX` starts without reading a format file. Later on the values of all ε-TEX state variables are inherited when `eVIRTEX` (or `eINITEX`) reads a format file.

The code below is designed to work for cases where '**init** ... **tini**' is a run-time switch.

```
⟨ Enable ε-TEX, if requested 1451 ⟩ ≡
  init if (buffer[loc] = "*") ∧ (format_ident = "␣(INITEX)") then
    begin no_new_control_sequence ← false; ⟨ Generate all ε-TEX primitives 1399 ⟩
    incr(loc); eTeX_mode ← 1; { enter extended mode }
    ⟨ Initialize variables for ε-TEX extended mode 1624 ⟩
    end;
  tini
  if ¬no_new_control_sequence then { just entered extended mode ? }
    no_new_control_sequence ← true else
```

This code is used in section 1391.

1452. The ε-TEX features available in extended mode are grouped into two categories: (1) Some of them are permanently enabled and have no semantic effect as long as none of the additional primitives are executed. (2) The remaining ε-TEX features are optional and can be individually enabled and disabled. For each optional feature there is an ε-TEX state variable named `\...state`; the feature is enabled, resp. disabled by assigning a positive, resp. non-positive value to that integer.

define *eTeX_state_base* = *int_base* + *eTeX_state_code*
define *eTeX_state*(#) ≡ *eqtb*[*eTeX_state_base* + #].*int* { an ε-TEX state variable }

define *eTeX_version_code* = *eTeX_int* { code for `\eTeXversion` }

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("lastnodetype", *last_item*, *last_node_type_code*);
primitive("eTeXversion", *last_item*, *eTeX_version_code*);
primitive("eTeXrevision", *convert*, *eTeX_revision_code*);
primitive("XeTeXversion", *last_item*, *XeTeX_version_code*);
primitive("XeTeXrevision", *convert*, *XeTeX_revision_code*);
primitive("XeTeXcountglyphs", *last_item*, *XeTeX_count_glyphs_code*);
primitive("XeTeXcountvariations", *last_item*, *XeTeX_count_variations_code*);
primitive("XeTeXvariation", *last_item*, *XeTeX_variation_code*);
primitive("XeTeXfindvariationbyname", *last_item*, *XeTeX_find_variation_by_name_code*);
primitive("XeTeXvariationmin", *last_item*, *XeTeX_variation_min_code*);
primitive("XeTeXvariationmax", *last_item*, *XeTeX_variation_max_code*);
primitive("XeTeXvariationdefault", *last_item*, *XeTeX_variation_default_code*);
primitive("XeTeXcountfeatures", *last_item*, *XeTeX_count_features_code*);
primitive("XeTeXfeaturecode", *last_item*, *XeTeX_feature_code_code*);
primitive("XeTeXfindfeaturebyname", *last_item*, *XeTeX_find_feature_by_name_code*);
primitive("XeTeXisexclusivefeature", *last_item*, *XeTeX_is_exclusive_feature_code*);
primitive("XeTeXcountselectors", *last_item*, *XeTeX_count_selectors_code*);
primitive("XeTeXselectorcode", *last_item*, *XeTeX_selector_code_code*);
primitive("XeTeXfindselectorbyname", *last_item*, *XeTeX_find_selector_by_name_code*);
primitive("XeTeXisdefaultselector", *last_item*, *XeTeX_is_default_selector_code*);
primitive("XeTeXvariationname", *convert*, *XeTeX_variation_name_code*);
primitive("XeTeXfeaturename", *convert*, *XeTeX_feature_name_code*);
primitive("XeTeXselectorname", *convert*, *XeTeX_selector_name_code*);
primitive("XeTeXOTcountscripts", *last_item*, *XeTeX_OT_count_scripts_code*);
primitive("XeTeXOTcountlanguages", *last_item*, *XeTeX_OT_count_languages_code*);
primitive("XeTeXOTcountfeatures", *last_item*, *XeTeX_OT_count_features_code*);
primitive("XeTeXOTscripttag", *last_item*, *XeTeX_OT_script_code*);
primitive("XeTeXOTlanguagetag", *last_item*, *XeTeX_OT_language_code*);
primitive("XeTeXOTfeaturetag", *last_item*, *XeTeX_OT_feature_code*);
primitive("XeTeXcharglyph", *last_item*, *XeTeX_map_char_to_glyph_code*);
primitive("XeTeXglyphindex", *last_item*, *XeTeX_glyph_index_code*);
primitive("XeTeXglyphbounds", *last_item*, *XeTeX_glyph_bounds_code*);
primitive("XeTeXglyphname", *convert*, *XeTeX_glyph_name_code*);
primitive("XeTeXfonttype", *last_item*, *XeTeX_font_type_code*);
primitive("XeTeXfirstfontchar", *last_item*, *XeTeX_first_char_code*);
primitive("XeTeXlastfontchar", *last_item*, *XeTeX_last_char_code*);
primitive("XeTeXpdfpagecount", *last_item*, *XeTeX_pdf_page_count_code*);

1453. ⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩ ≡

```
last_node_type_code: print_esc("lastnodetype");
eTeX_version_code: print_esc("eTeXversion");
XeTeX_version_code: print_esc("XeTeXversion");
XeTeX_count_glyphs_code: print_esc("XeTeXcountglyphs");
XeTeX_count_variations_code: print_esc("XeTeXcountvariations");
XeTeX_variation_code: print_esc("XeTeXvariation");
XeTeX_find_variation_by_name_code: print_esc("XeTeXfindvariationbyname");
XeTeX_variation_min_code: print_esc("XeTeXvariationmin");
XeTeX_variation_max_code: print_esc("XeTeXvariationmax");
XeTeX_variation_default_code: print_esc("XeTeXvariationdefault");
XeTeX_count_features_code: print_esc("XeTeXcountfeatures");
XeTeX_feature_code_code: print_esc("XeTeXfeaturecode");
XeTeX_find_feature_by_name_code: print_esc("XeTeXfindfeaturebyname");
XeTeX_is_exclusive_feature_code: print_esc("XeTeXisexclusivefeature");
XeTeX_count_selectors_code: print_esc("XeTeXcountselectors");
XeTeX_selector_code_code: print_esc("XeTeXselectorcode");
XeTeX_find_selector_by_name_code: print_esc("XeTeXfindselectorbyname");
XeTeX_is_default_selector_code: print_esc("XeTeXisdefaultselector");
XeTeX_OT_count_scripts_code: print_esc("XeTeXOTcountscripts");
XeTeX_OT_count_languages_code: print_esc("XeTeXOTcountlanguages");
XeTeX_OT_count_features_code: print_esc("XeTeXOTcountfeatures");
XeTeX_OT_script_code: print_esc("XeTeXOTscripttag");
XeTeX_OT_language_code: print_esc("XeTeXOTlanguagetag");
XeTeX_OT_feature_code: print_esc("XeTeXOTfeaturetag");
XeTeX_map_char_to_glyph_code: print_esc("XeTeXcharglyph");
XeTeX_glyph_index_code: print_esc("XeTeXglyphindex");
XeTeX_glyph_bounds_code: print_esc("XeTeXglyphbounds");
XeTeX_font_type_code: print_esc("XeTeXfonttype");
XeTeX_first_char_code: print_esc("XeTeXfirstfontchar");
XeTeX_last_char_code: print_esc("XeTeXlastfontchar");
XeTeX_pdf_page_count_code: print_esc("XeTeXpdfpagecount");
```

See also sections 1474, 1477, 1480, 1483, 1590, 1613, and 1617.

This code is used in section 451.

1454. ⟨ Cases for fetching an integer value 1454 ⟩ ≡

```
eTeX_version_code: cur_val ← eTeX_version;
XeTeX_version_code: cur_val ← XeTeX_version;
XeTeX_count_glyphs_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then cur_val ← aat_font_get(m − XeTeX_int, font_layout_engine[n])
  else if is_otgr_font(n) then cur_val ← ot_font_get(m − XeTeX_int, font_layout_engine[n])
    else cur_val ← 0;
  end;
XeTeX_count_features_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then cur_val ← aat_font_get(m − XeTeX_int, font_layout_engine[n])
  else if is_gr_font(n) then cur_val ← ot_font_get(m − XeTeX_int, font_layout_engine[n])
    else cur_val ← 0;
  end;
XeTeX_variation_code, XeTeX_variation_min_code, XeTeX_variation_max_code,
      XeTeX_variation_default_code, XeTeX_count_variations_code: begin scan_font_ident; n ← cur_val;
  cur_val ← 0; { Deprecated }
  end;
XeTeX_feature_code_code, XeTeX_is_exclusive_feature_code, XeTeX_count_selectors_code: begin
      scan_font_ident; n ← cur_val;
  if is_aat_font(n) then
    begin scan_int; k ← cur_val; cur_val ← aat_font_get_1(m − XeTeX_int, font_layout_engine[n], k);
    end
  else if is_gr_font(n) then
      begin scan_int; k ← cur_val; cur_val ← ot_font_get_1(m − XeTeX_int, font_layout_engine[n], k);
      end
    else begin not_aat_gr_font_error(last_item, m, n); cur_val ← −1;
      end;
  end;
XeTeX_selector_code_code, XeTeX_is_default_selector_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then
    begin scan_int; k ← cur_val; scan_int;
    cur_val ← aat_font_get_2(m − XeTeX_int, font_layout_engine[n], k, cur_val);
    end
  else if is_gr_font(n) then
      begin scan_int; k ← cur_val; scan_int;
      cur_val ← ot_font_get_2(m − XeTeX_int, font_layout_engine[n], k, cur_val);
      end
    else begin not_aat_gr_font_error(last_item, m, n); cur_val ← −1;
      end;
  end;
XeTeX_find_variation_by_name_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then
    begin scan_and_pack_name; cur_val ← aat_font_get_named(m − XeTeX_int, font_layout_engine[n]);
    end
  else begin not_aat_font_error(last_item, m, n); cur_val ← −1;
    end;
  end;
XeTeX_find_feature_by_name_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then
    begin scan_and_pack_name; cur_val ← aat_font_get_named(m − XeTeX_int, font_layout_engine[n]);
    end
  else if is_gr_font(n) then
```

```
      begin scan_and_pack_name; cur_val ← gr_font_get_named(m − XeTeX_int, font_layout_engine[n]);
      end
    else begin not_aat_gr_font_error(last_item, m, n); cur_val ← −1;
      end;
  end;
XeTeX_find_selector_by_name_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then
    begin scan_int; k ← cur_val; scan_and_pack_name;
    cur_val ← aat_font_get_named_1(m − XeTeX_int, font_layout_engine[n], k);
    end
  else if is_gr_font(n) then
      begin scan_int; k ← cur_val; scan_and_pack_name;
      cur_val ← gr_font_get_named_1(m − XeTeX_int, font_layout_engine[n], k);
      end
    else begin not_aat_gr_font_error(last_item, m, n); cur_val ← −1;
      end;
  end;
XeTeX_OT_count_scripts_code: begin scan_font_ident; n ← cur_val;
  if is_ot_font(n) then cur_val ← ot_font_get(m − XeTeX_int, font_layout_engine[n])
  else begin cur_val ← 0;
    end;
  end;
XeTeX_OT_count_languages_code, XeTeX_OT_script_code: begin scan_font_ident; n ← cur_val;
  if is_ot_font(n) then
    begin scan_int; cur_val ← ot_font_get_1(m − XeTeX_int, font_layout_engine[n], cur_val);
    end
  else begin not_ot_font_error(last_item, m, n); cur_val ← −1;
    end;
  end;
XeTeX_OT_count_features_code, XeTeX_OT_language_code: begin scan_font_ident; n ← cur_val;
  if is_ot_font(n) then
    begin scan_int; k ← cur_val; scan_int;
    cur_val ← ot_font_get_2(m − XeTeX_int, font_layout_engine[n], k, cur_val);
    end
  else begin not_ot_font_error(last_item, m, n); cur_val ← −1;
    end;
  end;
XeTeX_OT_feature_code: begin scan_font_ident; n ← cur_val;
  if is_ot_font(n) then
    begin scan_int; k ← cur_val; scan_int; kk ← cur_val; scan_int;
    cur_val ← ot_font_get_3(m − XeTeX_int, font_layout_engine[n], k, kk, cur_val);
    end
  else begin not_ot_font_error(last_item, m, n); cur_val ← −1;
    end;
  end;
XeTeX_map_char_to_glyph_code: begin if is_native_font(cur_font) then
    begin scan_int; n ← cur_val; cur_val ← map_char_to_glyph(cur_font, n)
    end
  else begin not_native_font_error(last_item, m, cur_font); cur_val ← 0
    end
  end;
XeTeX_glyph_index_code: begin if is_native_font(cur_font) then
```

```
    begin scan_and_pack_name; cur_val ← map_glyph_to_index(cur_font)
    end
  else begin not_native_font_error(last_item, m, cur_font); cur_val ← 0
    end
  end;
XeTeX_font_type_code: begin scan_font_ident; n ← cur_val;
  if is_aat_font(n) then cur_val ← 1
  else if is_ot_font(n) then cur_val ← 2
    else if is_gr_font(n) then cur_val ← 3
      else cur_val ← 0;
  end;
XeTeX_first_char_code, XeTeX_last_char_code: begin scan_font_ident; n ← cur_val;
  if is_native_font(n) then cur_val ← get_font_char_range(n, m = XeTeX_first_char_code)
  else begin if m = XeTeX_first_char_code then cur_val ← font_bc[n]
    else cur_val ← font_ec[n];
    end
  end;
pdf_last_x_pos_code: cur_val ← pdf_last_x_pos;
pdf_last_y_pos_code: cur_val ← pdf_last_y_pos;
XeTeX_pdf_page_count_code: begin scan_and_pack_name; cur_val ← count_pdf_file_pages;
  end;
```

See also sections 1475, 1478, and 1614.

This code is used in section 458.

1455. Slip in an extra procedure here and there....

```
⟨ Error handling procedures 82 ⟩ +≡
procedure scan_and_pack_name; forward;
```

1456.

```
⟨ Declare procedures needed in do_extension 1404 ⟩ +≡
procedure scan_and_pack_name;
  begin scan_file_name; pack_cur_name;
  end;
```

1457.

```
⟨ Declare the procedure called print_cmd_chr 328 ⟩ +≡
procedure not_aat_font_error(cmd, c : integer; f : integer);
  begin print_err("Cannot␣use␣"); print_cmd_chr(cmd, c); print("␣with␣"); print(font_name[f]);
  print(";␣not␣an␣AAT␣font"); error;
  end;
procedure not_aat_gr_font_error(cmd, c : integer; f : integer);
  begin print_err("Cannot␣use␣"); print_cmd_chr(cmd, c); print("␣with␣"); print(font_name[f]);
  print(";␣not␣an␣AAT␣or␣Graphite␣font"); error;
  end;
procedure not_ot_font_error(cmd, c : integer; f : integer);
  begin print_err("Cannot␣use␣"); print_cmd_chr(cmd, c); print("␣with␣"); print(font_name[f]);
  print(";␣not␣an␣OpenType␣Layout␣font"); error;
  end;
procedure not_native_font_error(cmd, c : integer; f : integer);
  begin print_err("Cannot␣use␣"); print_cmd_chr(cmd, c); print("␣with␣"); print(font_name[f]);
  print(";␣not␣a␣native␣platform␣font"); error;
  end;
```

1458. ⟨ Cases for fetching a dimension value 1458 ⟩ ≡

```
XeTeX_glyph_bounds_code: begin if is_native_font(cur_font) then
    begin scan_int; n ← cur_val; { which edge: 1=left, 2=top, 3=right, 4=bottom }
    if (n < 1) ∨ (n > 4) then
      begin print_err("\\XeTeXglyphbounds requires an edge index from 1 to 4;");
      print_nl("I don't know anything about edge ");
      print_int(n); error; cur_val ← 0;
      end
    else begin scan_int; { glyph number }
      cur_val ← get_glyph_bounds(cur_font, n, cur_val);
      end
    end
  else begin not_native_font_error(last_item, m, cur_font); cur_val ← 0
    end
  end;
```

See also sections 1481, 1484, and 1615.

This code is used in section 458.

1459. ⟨ Cases of *convert* for *print_cmd_chr* 1459 ⟩ ≡

```
XeTeX_revision_code: print_esc("XeTeXrevision");
XeTeX_variation_name_code: print_esc("XeTeXvariationname");
XeTeX_feature_name_code: print_esc("XeTeXfeaturename");
XeTeX_selector_name_code: print_esc("XeTeXselectorname");
XeTeX_glyph_name_code: print_esc("XeTeXglyphname");
XeTeX_Uchar_code: print_esc("Uchar");
XeTeX_Ucharcat_code: print_esc("Ucharcat");
```

This code is used in section 504.

1460. ⟨ Cases of 'Scan the argument for command c' 1460 ⟩ ≡

```
XeTeX_revision_code: do_nothing;
XeTeX_variation_name_code: begin scan_font_ident; fnt ← cur_val;
  if is_aat_font(fnt) then
    begin scan_int; arg1 ← cur_val; arg2 ← 0;
    end
  else not_aat_font_error(convert, c, fnt);
  end;
XeTeX_feature_name_code: begin scan_font_ident; fnt ← cur_val;
  if is_aat_font(fnt) ∨ is_gr_font(fnt) then
    begin scan_int; arg1 ← cur_val; arg2 ← 0;
    end
  else not_aat_gr_font_error(convert, c, fnt);
  end;
XeTeX_selector_name_code: begin scan_font_ident; fnt ← cur_val;
  if is_aat_font(fnt) ∨ is_gr_font(fnt) then
    begin scan_int; arg1 ← cur_val; scan_int; arg2 ← cur_val;
    end
  else not_aat_gr_font_error(convert, c, fnt);
  end;
XeTeX_glyph_name_code: begin scan_font_ident; fnt ← cur_val;
  if is_native_font(fnt) then
    begin scan_int; arg1 ← cur_val;
    end
  else not_native_font_error(convert, c, fnt);
  end;
```

This code is used in section 506.

1461. ⟨ Cases of 'Print the result of command c' 1461 ⟩ ≡

```
XeTeX_revision_code: print(XeTeX_revision);
XeTeX_variation_name_code: if is_aat_font(fnt) then
    aat_print_font_name(c, font_layout_engine[fnt], arg1, arg2);
XeTeX_feature_name_code, XeTeX_selector_name_code: if is_aat_font(fnt) then
    aat_print_font_name(c, font_layout_engine[fnt], arg1, arg2)
  else if is_gr_font(fnt) then gr_print_font_name(c, font_layout_engine[fnt], arg1, arg2);
XeTeX_glyph_name_code: if is_native_font(fnt) then print_glyph_name(fnt, arg1);
```

This code is used in section 507.

1462. **define** $eTeX_ex \equiv (eTeX_mode = 1)$ { is this extended mode? }

⟨ Global variables 13 ⟩ +≡

```
eTeX_mode: 0 .. 1; { identifies compatibility and extended mode }
```

1463. ⟨ Initialize table entries (done by INITEX only) 189 ⟩ +≡

```
  eTeX_mode ← 0; { initially we are in compatibility mode }
  ⟨ Initialize variables for ε-TEX compatibility mode 1623 ⟩
```

1464. ⟨ Dump the ε-TEX state 1464 ⟩ ≡

```
  dump_int(eTeX_mode);
      { in a deliberate change from e-TeX, we allow non-zero state variables to be dumped }
```

See also section 1569.

This code is used in section 1361.

1465. ⟨ Undump the ε-TEX state 1465 ⟩ ≡
```
undump(0)(1)(eTeX_mode);
if eTeX_ex then
  begin ⟨ Initialize variables for ε-TEX extended mode 1624 ⟩
  end
else begin ⟨ Initialize variables for ε-TEX compatibility mode 1623 ⟩
  end;
```
This code is used in section 1362.

1466. The *eTeX_enabled* function simply returns its first argument as result. This argument is *true* if an optional ε-TEX feature is currently enabled; otherwise, if the argument is *false*, the function gives an error message.

⟨ Declare ε-TEX procedures for use by *main_control* 1466 ⟩ ≡
```
function eTeX_enabled(b : boolean; j : quarterword; k : halfword): boolean;
  begin if ¬b then
    begin print_err("Improper␣"); print_cmd_chr(j, k);
    help1("Sorry,␣this␣optional␣e-TeX␣feature␣has␣been␣disabled."); error;
    end;
  eTeX_enabled ← b;
  end;
```
See also sections 1489 and 1505.

This code is used in section 863.

1467. First we implement the additional ε-TEX parameters in the table of equivalents.

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
```
primitive("everyeof", assign_toks, every_eof_loc);
primitive("tracingassigns", assign_int, int_base + tracing_assigns_code);
primitive("tracinggroups", assign_int, int_base + tracing_groups_code);
primitive("tracingifs", assign_int, int_base + tracing_ifs_code);
primitive("tracingscantokens", assign_int, int_base + tracing_scan_tokens_code);
primitive("tracingnesting", assign_int, int_base + tracing_nesting_code);
primitive("predisplaydirection", assign_int, int_base + pre_display_direction_code);
primitive("lastlinefit", assign_int, int_base + last_line_fit_code);
primitive("savingvdiscards", assign_int, int_base + saving_vdiscards_code);
primitive("savinghyphcodes", assign_int, int_base + saving_hyph_codes_code);
```

1468. **define** *every_eof* ≡ *equiv*(*every_eof_loc*)

⟨ Cases of *assign_toks* for *print_cmd_chr* 1468 ⟩ ≡
```
every_eof_loc: print_esc("everyeof");
XeTeX_inter_char_loc: print_esc("XeTeXinterchartoks");
```
This code is used in section 257.

1469. ⟨ Cases for *print_param* 1469 ⟩ ≡
tracing_assigns_code: *print_esc*("tracingassigns");
tracing_groups_code: *print_esc*("tracinggroups");
tracing_ifs_code: *print_esc*("tracingifs");
tracing_scan_tokens_code: *print_esc*("tracingscantokens");
tracing_nesting_code: *print_esc*("tracingnesting");
pre_display_direction_code: *print_esc*("predisplaydirection");
last_line_fit_code: *print_esc*("lastlinefit");
saving_vdiscards_code: *print_esc*("savingvdiscards");
saving_hyph_codes_code: *print_esc*("savinghyphcodes");

See also section 1510.

This code is used in section 263.

1470. In order to handle `\everyeof` we need an array *eof_seen* of boolean variables.

⟨ Global variables 13 ⟩ +≡
eof_seen: **array** [1 .. *max_in_open*] **of** *boolean*; { has eof been seen? }

1471. The *print_group* procedure prints the current level of grouping and the name corresponding to *cur_group*.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure print_group(e : boolean);
  label exit;
  begin case cur_group of
  bottom_level: begin print("bottom␣level"); return;
    end;
  simple_group, semi_simple_group: begin if cur_group = semi_simple_group then print("semi␣");
    print("simple");
    end;
  hbox_group, adjusted_hbox_group: begin if cur_group = adjusted_hbox_group then print("adjusted␣");
    print("hbox");
    end;
  vbox_group: print("vbox");
  vtop_group: print("vtop");
  align_group, no_align_group: begin if cur_group = no_align_group then print("no␣");
    print("align");
    end;
  output_group: print("output");
  disc_group: print("disc");
  insert_group: print("insert");
  vcenter_group: print("vcenter");
  math_group, math_choice_group, math_shift_group, math_left_group: begin print("math");
    if cur_group = math_choice_group then print("␣choice")
    else if cur_group = math_shift_group then print("␣shift")
      else if cur_group = math_left_group then print("␣left");
    end;
  end; { there are no other cases }
  print("␣group␣(level␣"); print_int(qo(cur_level)); print_char(")");
  if saved(−1) ≠ 0 then
    begin if e then print("␣entered␣at␣line␣")
    else print("␣at␣line␣");
    print_int(saved(−1));
    end;
exit: end;
```

1472. The *group_trace* procedure is called when a new level of grouping begins ($e = false$) or ends ($e = true$) with $saved(-1)$ containing the line number.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
  stat procedure group_trace(e : boolean);
  begin begin_diagnostic; print_char("{");
  if e then print("leaving␣")
  else print("entering␣");
  print_group(e); print_char("}"); end_diagnostic(false);
  end;
  tats
```

1473. The `\currentgrouplevel` and `\currentgrouptype` commands return the current level of grouping and the type of the current group respectively.

define *current_group_level_code* = *eTeX_int* + 1 { code for `\currentgrouplevel` }
define *current_group_type_code* = *eTeX_int* + 2 { code for `\currentgrouptype` }

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("currentgrouplevel", *last_item*, *current_group_level_code*);
primitive("currentgrouptype", *last_item*, *current_group_type_code*);

1474. ⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩ +≡
current_group_level_code: *print_esc*("currentgrouplevel");
current_group_type_code: *print_esc*("currentgrouptype");

1475. ⟨ Cases for fetching an integer value 1454 ⟩ +≡
current_group_level_code: *cur_val* ← *cur_level* − *level_one*;
current_group_type_code: *cur_val* ← *cur_group*;

1476. The `\currentiflevel`, `\currentiftype`, and `\currentifbranch` commands return the current level of conditionals and the type and branch of the current conditional.

define *current_if_level_code* = *eTeX_int* + 3 { code for `\currentiflevel` }
define *current_if_type_code* = *eTeX_int* + 4 { code for `\currentiftype` }
define *current_if_branch_code* = *eTeX_int* + 5 { code for `\currentifbranch` }

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("currentiflevel", *last_item*, *current_if_level_code*);
primitive("currentiftype", *last_item*, *current_if_type_code*);
primitive("currentifbranch", *last_item*, *current_if_branch_code*);

1477. ⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩ +≡
current_if_level_code: *print_esc*("currentiflevel");
current_if_type_code: *print_esc*("currentiftype");
current_if_branch_code: *print_esc*("currentifbranch");

1478. ⟨ Cases for fetching an integer value 1454 ⟩ +≡
current_if_level_code: **begin** *q* ← *cond_ptr*; *cur_val* ← 0;
while *q* ≠ *null* **do**
begin *incr*(*cur_val*); *q* ← *link*(*q*);
end;
end;
current_if_type_code: **if** *cond_ptr* = *null* **then** *cur_val* ← 0
else if *cur_if* < *unless_code* **then** *cur_val* ← *cur_if* + 1
else *cur_val* ← −(*cur_if* − *unless_code* + 1);
current_if_branch_code: **if** (*if_limit* = *or_code*) ∨ (*if_limit* = *else_code*) **then** *cur_val* ← 1
else if *if_limit* = *fi_code* **then** *cur_val* ← −1
else *cur_val* ← 0;

1479. The `\fontcharwd`, `\fontcharht`, `\fontchardp`, and `\fontcharic` commands return information about a character in a font.

define *font_char_wd_code* = *eTeX_dim* { code for `\fontcharwd` }
define *font_char_ht_code* = *eTeX_dim* + 1 { code for `\fontcharht` }
define *font_char_dp_code* = *eTeX_dim* + 2 { code for `\fontchardp` }
define *font_char_ic_code* = *eTeX_dim* + 3 { code for `\fontcharic` }

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("fontcharwd", *last_item*, *font_char_wd_code*);
primitive("fontcharht", *last_item*, *font_char_ht_code*);
primitive("fontchardp", *last_item*, *font_char_dp_code*);
primitive("fontcharic", *last_item*, *font_char_ic_code*);

1480. ⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩ +≡
font_char_wd_code: *print_esc*("fontcharwd");
font_char_ht_code: *print_esc*("fontcharht");
font_char_dp_code: *print_esc*("fontchardp");
font_char_ic_code: *print_esc*("fontcharic");

1481. ⟨ Cases for fetching a dimension value 1458 ⟩ +≡
font_char_wd_code, *font_char_ht_code*, *font_char_dp_code*, *font_char_ic_code*: **begin** *scan_font_ident*;
$q \leftarrow cur_val$; *scan_usv_num*;
if *is_native_font*(*q*) **then**
begin case *m* **of**
font_char_wd_code: *cur_val* ← *getnativecharwd*(*q*, *cur_val*);
font_char_ht_code: *cur_val* ← *getnativecharht*(*q*, *cur_val*);
font_char_dp_code: *cur_val* ← *getnativechardp*(*q*, *cur_val*);
font_char_ic_code: *cur_val* ← *getnativecharic*(*q*, *cur_val*);
end; { there are no other cases }
end
else begin if (*font_bc*[*q*] ≤ *cur_val*) ∧ (*font_ec*[*q*] ≥ *cur_val*) **then**
begin *i* ← *char_info*(*q*)(*qi*(*cur_val*));
case *m* **of**
font_char_wd_code: *cur_val* ← *char_width*(*q*)(*i*);
font_char_ht_code: *cur_val* ← *char_height*(*q*)(*height_depth*(*i*));
font_char_dp_code: *cur_val* ← *char_depth*(*q*)(*height_depth*(*i*));
font_char_ic_code: *cur_val* ← *char_italic*(*q*)(*i*);
end; { there are no other cases }
end
else *cur_val* ← 0;
end
end;

1482. The `\parshapedimen`, `\parshapeindent`, and `\parshapelength` commands return the indent and length parameters of the current `\parshape` specification.

define *par_shape_length_code* = *eTeX_dim* + 4 { code for `\parshapelength` }
define *par_shape_indent_code* = *eTeX_dim* + 5 { code for `\parshapeindent` }
define *par_shape_dimen_code* = *eTeX_dim* + 6 { code for `\parshapedimen` }

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("parshapelength", *last_item*, *par_shape_length_code*);
primitive("parshapeindent", *last_item*, *par_shape_indent_code*);
primitive("parshapedimen", *last_item*, *par_shape_dimen_code*);

1483. ⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩ +≡
```
par_shape_length_code: print_esc("parshapelength");
par_shape_indent_code: print_esc("parshapeindent");
par_shape_dimen_code: print_esc("parshapedimen");
```

1484. ⟨ Cases for fetching a dimension value 1458 ⟩ +≡
```
par_shape_length_code, par_shape_indent_code, par_shape_dimen_code: begin
    q ← cur_chr − par_shape_length_code; scan_int;
  if (par_shape_ptr = null) ∨ (cur_val ≤ 0) then cur_val ← 0
  else begin if q = 2 then
    begin q ← cur_val mod 2; cur_val ← (cur_val + q) div 2;
    end;
   if cur_val > info(par_shape_ptr) then cur_val ← info(par_shape_ptr);
   cur_val ← mem[par_shape_ptr + 2 ∗ cur_val − q].sc;
   end;
  cur_val_level ← dimen_val;
  end;
```

1485. The `\showgroups` command displays all currently active grouping levels.
```
  define show_groups = 4  { \showgroups }
```
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
```
  primitive("showgroups", xray, show_groups);
```

1486. ⟨ Cases of *xray* for *print_cmd_chr* 1486 ⟩ ≡
```
show_groups: print_esc("showgroups");
```
See also sections 1495 and 1500.

This code is used in section 1346.

1487. ⟨ Cases for *show_whatever* 1487 ⟩ ≡
```
show_groups: begin begin_diagnostic; show_save_groups;
  end;
```
See also section 1501.

This code is used in section 1347.

1488. ⟨ Types in the outer block 18 ⟩ +≡
```
  save_pointer = 0 .. save_size;  { index into save_stack }
```

1489. The modifications of TEX required for the display produced by the *show_save_groups* procedure were first discussed by Donald E. Knuth in *TUGboat* **11**, 165–170 and 499–511, 1990.

In order to understand a group type we also have to know its mode. Since unrestricted horizontal modes are not associated with grouping, they are skipped when traversing the semantic nest.

```
⟨ Declare ε-TEX procedures for use by main_control 1466 ⟩ +≡
procedure show_save_groups;
  label found1, found2, found, done;
  var p: 0 .. nest_size; { index into nest }
    m: −mmode .. mmode; { mode }
    v: save_pointer; { saved value of save_ptr }
    l: quarterword; { saved value of cur_level }
    c: group_code; { saved value of cur_group }
    a: −1 .. 1; { to keep track of alignments }
    i: integer; j: quarterword; s: str_number;
  begin p ← nest_ptr; nest[p] ← cur_list; { put the top level into the array }
  v ← save_ptr; l ← cur_level; c ← cur_group; save_ptr ← cur_boundary; decr(cur_level);
  a ← 1; print_nl("␣␣␣"); print_ln;
  loop begin print_nl("###␣"); print_group(true);
    if cur_group = bottom_level then goto done;
    repeat m ← nest[p].mode_field;
      if p > 0 then decr(p)
      else m ← vmode;
    until m ≠ hmode;
    print("␣(");
    case cur_group of
    simple_group: begin incr(p); goto found2;
      end;
    hbox_group, adjusted_hbox_group: s ← "hbox";
    vbox_group: s ← "vbox";
    vtop_group: s ← "vtop";
    align_group: if a = 0 then
        begin if m = −vmode then s ← "halign"
        else s ← "valign";
        a ← 1; goto found1;
        end
      else begin if a = 1 then print("align␣entry")
        else print_esc("cr");
        if p ≥ a then p ← p − a;
        a ← 0; goto found;
        end;
    no_align_group: begin incr(p); a ← −1; print_esc("noalign"); goto found2;
      end;
    output_group: begin print_esc("output"); goto found;
      end;
    math_group: goto found2;
    disc_group, math_choice_group: begin if cur_group = disc_group then print_esc("discretionary")
      else print_esc("mathchoice");
      for i ← 1 to 3 do
        if i ≤ saved(−2) then print("{}");
      goto found2;
      end;
    insert_group: begin if saved(−2) = 255 then print_esc("vadjust")
```

```
    else begin print_esc("insert"); print_int(saved(−2));
      end;
    goto found2;
    end;
  vcenter_group: begin s ← "vcenter"; goto found1;
    end;
  semi_simple_group: begin incr(p); print_esc("begingroup"); goto found;
    end;
  math_shift_group: begin if m = mmode then print_char("$")
    else if nest[p].mode_field = mmode then
        begin print_cmd_chr(eq_no, saved(−2)); goto found;
        end;
    print_char("$"); goto found;
    end;
  math_left_group: begin if type(nest[p + 1].eTeX_aux_field) = left_noad then print_esc("left")
    else print_esc("middle");
    goto found;
    end;
  end; { there are no other cases }
  ⟨ Show the box context 1491 ⟩;
found1: print_esc(s); ⟨ Show the box packaging info 1490 ⟩;
found2: print_char("{");
found: print_char(")"); decr(cur_level); cur_group ← save_level(save_ptr);
  save_ptr ← save_index(save_ptr)
  end;
done: save_ptr ← v; cur_level ← l; cur_group ← c;
end;
```

1490. ⟨ Show the box packaging info 1490 ⟩ ≡

```
if saved(−2) ≠ 0 then
  begin print_char("␣");
  if saved(−3) = exactly then print("to")
  else print("spread");
  print_scaled(saved(−2)); print("pt");
  end
```

This code is used in section 1489.

1491. ⟨ Show the box context 1491 ⟩ ≡

```
i ← saved(−4);
if i ≠ 0 then
  if i < box_flag then
    begin if abs(nest[p].mode_field) = vmode then j ← hmove
    else j ← vmove;
    if i > 0 then print_cmd_chr(j, 0)
    else print_cmd_chr(j, 1);
    print_scaled(abs(i)); print("pt");
    end
  else if i < ship_out_flag then
      begin if i ≥ global_box_flag then
        begin print_esc("global"); i ← i − (global_box_flag − box_flag);
        end;
      print_esc("setbox"); print_int(i − box_flag); print_char("=");
      end
    else print_cmd_chr(leader_ship, i − (leader_flag − a_leaders))
```

This code is used in section 1489.

1492. The *scan_general_text* procedure is much like *scan_toks*(*false*, *false*), but will be invoked via *expand*, i.e., recursively.

⟨ Declare ε-TEX procedures for scanning 1492 ⟩ ≡

```
procedure scan_general_text; forward;
```

See also sections 1583, 1592, and 1597.

This code is used in section 443.

1493. The token list (balanced text) created by *scan_general_text* begins at *link*(*temp_head*) and ends at *cur_val*. (If *cur_val* = *temp_head*, the list is empty.)

```
⟨ Declare ε-TEX procedures for token lists 1493 ⟩ ≡
procedure scan_general_text;
  label found;
  var s: normal .. absorbing;  { to save scanner_status }
    w: pointer;  { to save warning_index }
    d: pointer;  { to save def_ref }
    p: pointer;  { tail of the token list being built }
    q: pointer;  { new node being added to the token list via store_new_token }
    unbalance: halfword;  { number of unmatched left braces }
  begin s ← scanner_status; w ← warning_index; d ← def_ref; scanner_status ← absorbing;
  warning_index ← cur_cs; def_ref ← get_avail; token_ref_count(def_ref) ← null; p ← def_ref;
  scan_left_brace;  { remove the compulsory left brace }
  unbalance ← 1;
  loop begin get_token;
    if cur_tok < right_brace_limit then
      if cur_cmd < right_brace then incr(unbalance)
      else begin decr(unbalance);
        if unbalance = 0 then goto found;
        end;
    store_new_token(cur_tok);
    end;
found: q ← link(def_ref); free_avail(def_ref);  { discard reference count }
  if q = null then cur_val ← temp_head else cur_val ← p;
  link(temp_head) ← q; scanner_status ← s; warning_index ← w; def_ref ← d;
  end;
```

See also section 1564.

This code is used in section 499.

1494. The `\showtokens` command displays a token list.

```
  define show_tokens = 5  { \showtokens , must be odd! }
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("showtokens", xray, show_tokens);
```

1495.

```
⟨ Cases of xray for print_cmd_chr 1486 ⟩ +≡
show_tokens: print_esc("showtokens");
```

1496. The `\unexpanded` primitive prevents expansion of tokens much as the result from `\the` applied to a token variable. The `\detokenize` primitive converts a token list into a list of character tokens much as if the token list were written to a file. We use the fact that the command modifiers for `\unexpanded` and `\detokenize` are odd whereas those for `\the` and `\showthe` are even.

```
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("unexpanded", the, 1);
  primitive("detokenize", the, show_tokens);
```

1497.

```
⟨ Cases of the for print_cmd_chr 1497 ⟩ ≡
else if chr_code = 1 then print_esc("unexpanded")
  else print_esc("detokenize")
```

This code is used in section 296.

1498. ⟨ Handle `\unexpanded` or `\detokenize` and **return** 1498 ⟩ ≡

```
if odd(cur_chr) then
  begin c ← cur_chr; scan_general_text;
  if c = 1 then the_toks ← cur_val
  else begin old_setting ← selector; selector ← new_string; b ← pool_ptr; p ← get_avail;
    link(p) ← link(temp_head); token_show(p); flush_list(p); selector ← old_setting;
    the_toks ← str_toks(b);
    end;
  return;
  end
```

This code is used in section 500.

1499. The `\showifs` command displays all currently active conditionals.

```
define show_ifs = 6 { \showifs }
```

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡

```
primitive("showifs", xray, show_ifs);
```

1500. ⟨ Cases of *xray* for *print_cmd_chr* 1486 ⟩ +≡

```
show_ifs: print_esc("showifs");
```

1501.

```
define print_if_line(#) ≡
        if # ≠ 0 then
          begin print("␣entered␣on␣line␣"); print_int(#);
          end
```

⟨ Cases for *show_whatever* 1487 ⟩ +≡

```
show_ifs: begin begin_diagnostic; print_nl(""); print_ln;
  if cond_ptr = null then
    begin print_nl("###␣"); print("no␣active␣conditionals");
    end
  else begin p ← cond_ptr; n ← 0;
    repeat incr(n); p ← link(p); until p = null;
    p ← cond_ptr; t ← cur_if; l ← if_line; m ← if_limit;
    repeat print_nl("###␣level␣"); print_int(n); print(":␣"); print_cmd_chr(if_test, t);
      if m = fi_code then print_esc("else");
      print_if_line(l); decr(n); t ← subtype(p); l ← if_line_field(p); m ← type(p); p ← link(p);
    until p = null;
    end;
  end;
```

1502. The `\interactionmode` primitive allows to query and set the interaction mode.

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡

```
primitive("interactionmode", set_page_int, 2);
```

1503. ⟨ Cases of *set_page_int* for *print_cmd_chr* 1503 ⟩ ≡

```
else if chr_code = 2 then print_esc("interactionmode")
```

This code is used in section 451.

1504. ⟨ Cases for 'Fetch the *dead_cycles* or the *insert_penalties*' 1504 ⟩ ≡

```
else if m = 2 then cur_val ← interaction
```

This code is used in section 453.

1505. ⟨ Declare ε-TEX procedures for use by *main_control* 1466 ⟩ +≡
procedure *new_interaction*; *forward*;

1506. ⟨ Cases for *alter_integer* 1506 ⟩ ≡
else if $c = 2$ **then**
  begin if (*cur_val* < *batch_mode*) ∨ (*cur_val* > *error_stop_mode*) **then**
    begin *print_err*("Bad␣interaction␣mode");
    help2("Modes␣are␣0=batch,␣1=nonstop,␣2=scroll,␣and")
    ("3=errorstop.␣Proceed,␣and␣I´ll␣ignore␣this␣case."); *int_error*(*cur_val*);
    end
  else begin *cur_chr* ← *cur_val*; *new_interaction*;
    end;
  end

This code is used in section 1300.

1507. The *middle* feature of ε-TEX allows one ore several `\middle` delimiters to appear between `\left` and `\right`.

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("middle", *left_right*, *middle_noad*);

1508. ⟨ Cases of *left_right* for *print_cmd_chr* 1508 ⟩ ≡
else if *chr_code* = *middle_noad* **then** *print_esc*("middle")

This code is used in section 1243.

1509. In constructions such as

```
\hbox to \hsize{
    \hskip 0pt plus 0.0001fil
    ...
    \hfil\penalty-200\hfilneg
    ...}
```

the stretch components of **\hfil** and **\hfilneg** compensate; they may, however, get modified in order to prevent arithmetic overflow during *hlist_out* when each of them is multiplied by a large *glue_set* value.

Since this "glue rounding" depends on state variables *cur_g* and *cur_glue* and TEX--XET is supposed to emulate the behaviour of TEX-XeTeX (plus a suitable postprocessor) as close as possible the glue rounding cannot be postponed until (segments of) an hlist has been reversed.

The code below is invoked after the effective width, *rule_wd*, of a glue node has been computed. The glue node is either converted into a kern node or, for leaders, the glue specification is replaced by an equivalent rigid one; the subtype of the glue node remains unchanged.

⟨ Handle a glue node for mixed direction typesetting 1509 ⟩ ≡

```
if (((g_sign = stretching) ∧ (stretch_order(g) = g_order)) ∨ ((g_sign = shrinking) ∧ (shrink_order(g) =
      g_order))) then
  begin fast_delete_glue_ref(g);
  if subtype(p) < a_leaders then
    begin type(p) ← kern_node; width(p) ← rule_wd;
    end
  else begin g ← get_node(glue_spec_size);
    stretch_order(g) ← filll + 1; shrink_order(g) ← filll + 1; { will never match }
    width(g) ← rule_wd; stretch(g) ← 0; shrink(g) ← 0; glue_ptr(p) ← g;
    end;
  end
```

This code is used in sections 663 and 1537.

1510. The optional *TeXXeT* feature of ε-TEX contains the code for mixed left-to-right and right-to-left typesetting. This code is inspired by but different from TEX-XET as presented by Donald E. Knuth and Pierre MacKay in *TUGboat* **8**, 14–25, 1987.

In order to avoid confusion with TEX-XET the present implementation of mixed direction typesetting is called TEX--XET. It differs from TEX-XET in several important aspects: (1) Right-to-left text is reversed explicitly by the *ship_out* routine and is written to a normal DVI file without any *begin_reflect* or *end_reflect* commands; (2) a *math_node* is (ab)used instead of a *whatsit_node* to record the \beginL, \endL, \beginR, and \endR text direction primitives in order to keep the influence on the line breaking algorithm for pure left-to-right text as small as possible; (3) right-to-left text interrupted by a displayed equation is automatically resumed after that equation; and (4) the *valign* command code with a non-zero command modifier is (ab)used for the text direction primitives.

Nevertheless there is a subtle difference between TEX and TEX--XET that may influence the line breaking algorithm for pure left-to-right text. When a paragraph containing math mode material is broken into lines TEX may generate lines where math mode material is not enclosed by properly nested \mathon and \mathoff nodes. Unboxing such lines as part of a new paragraph may have the effect that hyphenation is attempted for 'words' originating from math mode or that hyphenation is inhibited for words originating from horizontal mode.

In TEX--XET additional \beginM, resp. \endM math nodes are supplied at the start, resp. end of lines such that math mode material inside a horizontal list always starts with either \mathon or \beginM and ends with \mathoff or \endM. These additional nodes are transparent to operations such as \unskip, \lastpenalty, or \lastbox but they do have the effect that hyphenation is never attempted for 'words' originating from math mode and is never inhibited for words originating from horizontal mode.

define *TeXXeT_state* ≡ *eTeX_state*(*TeXXeT_code*)
define *TeXXeT_en* ≡ (*TeXXeT_state* > 0) { is TEX--XET enabled? }
define *XeTeX_upwards_state* ≡ *eTeX_state*(*XeTeX_upwards_code*)
define *XeTeX_upwards* ≡ (*XeTeX_upwards_state* > 0)
define *XeTeX_use_glyph_metrics_state* ≡ *eTeX_state*(*XeTeX_use_glyph_metrics_code*)
define *XeTeX_use_glyph_metrics* ≡ (*XeTeX_use_glyph_metrics_state* > 0)
define *XeTeX_inter_char_tokens_state* ≡ *eTeX_state*(*XeTeX_inter_char_tokens_code*)
define *XeTeX_inter_char_tokens_en* ≡ (*XeTeX_inter_char_tokens_state* > 0)
define *XeTeX_dash_break_state* ≡ *eTeX_state*(*XeTeX_dash_break_code*)
define *XeTeX_dash_break_en* ≡ (*XeTeX_dash_break_state* > 0)
define *XeTeX_input_normalization_state* ≡ *eTeX_state*(*XeTeX_input_normalization_code*)
define *XeTeX_tracing_fonts_state* ≡ *eTeX_state*(*XeTeX_tracing_fonts_code*)
define *XeTeX_interword_space_shaping_state* ≡ *eTeX_state*(*XeTeX_interword_space_shaping_code*)
define *XeTeX_generate_actual_text_state* ≡ *eTeX_state*(*XeTeX_generate_actual_text_code*)
define *XeTeX_generate_actual_text_en* ≡ (*XeTeX_generate_actual_text_state* > 0)
define *XeTeX_default_input_mode* ≡ *eTeX_state*(*XeTeX_default_input_mode_code*)
define *XeTeX_default_input_encoding* ≡ *eTeX_state*(*XeTeX_default_input_encoding_code*)
define *XeTeX_hyphenatable_length* ≡ *eTeX_state*(*XeTeX_hyphenatable_length_code*)

⟨ Cases for *print_param* 1469 ⟩ +≡
suppress_fontnotfound_error_code: *print_esc*("suppressfontnotfounderror");
eTeX_state_code + *TeXXeT_code*: *print_esc*("TeXXeTstate");
eTeX_state_code + *XeTeX_upwards_code*: *print_esc*("XeTeXupwardsmode");
eTeX_state_code + *XeTeX_use_glyph_metrics_code*: *print_esc*("XeTeXuseglyphmetrics");
eTeX_state_code + *XeTeX_inter_char_tokens_code*: *print_esc*("XeTeXinterchartokenstate");
eTeX_state_code + *XeTeX_dash_break_code*: *print_esc*("XeTeXdashbreakstate");
eTeX_state_code + *XeTeX_input_normalization_code*: *print_esc*("XeTeXinputnormalization");
eTeX_state_code + *XeTeX_tracing_fonts_code*: *print_esc*("XeTeXtracingfonts");
eTeX_state_code + *XeTeX_interword_space_shaping_code*: *print_esc*("XeTeXinterwordspaceshaping");
eTeX_state_code + *XeTeX_generate_actual_text_code*: *print_esc*("XeTeXgenerateactualtext");
eTeX_state_code + *XeTeX_hyphenatable_length_code*: *print_esc*("XeTeXhyphenatablelength");

1511. ⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("suppressfontnotfounderror", *assign_int*, *int_base* + *suppress_fontnotfound_error_code*);
primitive("TeXXeTstate", *assign_int*, *eTeX_state_base* + *TeXXeT_code*);
primitive("XeTeXupwardsmode", *assign_int*, *eTeX_state_base* + *XeTeX_upwards_code*);
primitive("XeTeXuseglyphmetrics", *assign_int*, *eTeX_state_base* + *XeTeX_use_glyph_metrics_code*);
primitive("XeTeXinterchartokenstate", *assign_int*, *eTeX_state_base* + *XeTeX_inter_char_tokens_code*);
primitive("XeTeXdashbreakstate", *assign_int*, *eTeX_state_base* + *XeTeX_dash_break_code*);
primitive("XeTeXinputnormalization", *assign_int*, *eTeX_state_base* + *XeTeX_input_normalization_code*);
primitive("XeTeXtracingfonts", *assign_int*, *eTeX_state_base* + *XeTeX_tracing_fonts_code*);
primitive("XeTeXinterwordspaceshaping", *assign_int*,
eTeX_state_base + *XeTeX_interword_space_shaping_code*);
primitive("XeTeXgenerateactualtext", *assign_int*, *eTeX_state_base* + *XeTeX_generate_actual_text_code*);
primitive("XeTeXhyphenatablelength", *assign_int*, *eTeX_state_base* + *XeTeX_hyphenatable_length_code*);
primitive("XeTeXinputencoding", *extension*, *XeTeX_input_encoding_extension_code*);
primitive("XeTeXdefaultencoding", *extension*, *XeTeX_default_encoding_extension_code*);
primitive("beginL", *valign*, *begin_L_code*); *primitive*("endL", *valign*, *end_L_code*);
primitive("beginR", *valign*, *begin_R_code*); *primitive*("endR", *valign*, *end_R_code*);

1512. ⟨ Cases of *valign* for *print_cmd_chr* 1512 ⟩ ≡
else case *chr_code* **of**
begin_L_code: *print_esc*("beginL");
end_L_code: *print_esc*("endL");
begin_R_code: *print_esc*("beginR");
othercases *print_esc*("endR")
endcases
This code is used in section 296.

1513. ⟨ Cases of *main_control* for *hmode* + *valign* 1513 ⟩ ≡
if *cur_chr* > 0 **then**
begin if *eTeX_enabled*(*TeXXeT_en*, *cur_cmd*, *cur_chr*) **then** *tail_append*(*new_math*(0, *cur_chr*));
end
else
This code is used in section 1184.

1514. An hbox with subtype dlist will never be reversed, even when embedded in right-to-left text.

⟨ Display if this box is never to be reversed 1514 ⟩ ≡
if (*type*(*p*) = *hlist_node*) ∧ (*box_lr*(*p*) = *dlist*) **then** *print*(",␣display")
This code is used in section 210.

1515. A number of routines are based on a stack of one-word nodes whose *info* fields contain *end_M_code*, *end_L_code*, or *end_R_code*. The top of the stack is pointed to by *LR_ptr*.

When the stack manipulation macros of this section are used below, variable *LR_ptr* might be the global variable declared here for *hpack* and *ship_out*, or might be local to *post_line_break*.

```
define put_LR(#) ≡
        begin temp_ptr ← get_avail; info(temp_ptr) ← #; link(temp_ptr) ← LR_ptr;
        LR_ptr ← temp_ptr;
        end
define push_LR(#) ≡ put_LR(end_LR_type(#))
define pop_LR ≡
        begin temp_ptr ← LR_ptr; LR_ptr ← link(temp_ptr); free_avail(temp_ptr);
        end
```

⟨ Global variables 13 ⟩ +≡
```
LR_ptr: pointer; { stack of LR codes for hpack, ship_out, and init_math }
LR_problems: integer; { counts missing begins and ends }
cur_dir: small_number; { current text direction }
```

1516. ⟨ Set initial values of key variables 23 ⟩ +≡
```
LR_ptr ← null; LR_problems ← 0; cur_dir ← left_to_right;
```

1517. ⟨ Insert LR nodes at the beginning of the current line and adjust the LR stack based on LR nodes in this line 1517 ⟩ ≡
```
begin q ← link(temp_head);
if LR_ptr ≠ null then
  begin temp_ptr ← LR_ptr; r ← q;
  repeat s ← new_math(0, begin_LR_type(info(temp_ptr))); link(s) ← r; r ← s;
    temp_ptr ← link(temp_ptr);
  until temp_ptr = null;
  link(temp_head) ← r;
  end;
while q ≠ cur_break(cur_p) do
  begin if ¬is_char_node(q) then
    if type(q) = math_node then ⟨ Adjust the LR stack for the post_line_break routine 1518 ⟩;
  q ← link(q);
  end;
end
```
This code is used in section 928.

1518. ⟨ Adjust the LR stack for the *post_line_break* routine 1518 ⟩ ≡
```
if end_LR(q) then
  begin if LR_ptr ≠ null then
    if info(LR_ptr) = end_LR_type(q) then pop_LR;
  end
else push_LR(q)
```
This code is used in sections 927, 929, and 1517.

1519. We use the fact that q now points to the node with `\rightskip` glue.

```
⟨ Insert LR nodes at the end of the current line 1519 ⟩ ≡
  if LR_ptr ≠ null then
    begin s ← temp_head; r ← link(s);
    while r ≠ q do
      begin s ← r; r ← link(s);
      end;
    r ← LR_ptr;
    while r ≠ null do
      begin temp_ptr ← new_math(0, info(r)); link(s) ← temp_ptr; s ← temp_ptr; r ← link(r);
      end;
    link(s) ← q;
    end
```

This code is used in section 928.

1520.
```
⟨ Initialize the LR stack 1520 ⟩ ≡
  put_LR(before)  { this will never match }
```

This code is used in sections 689, 1524, and 1545.

1521.
```
⟨ Adjust the LR stack for the hpack routine 1521 ⟩ ≡
  if end_LR(p) then
    if info(LR_ptr) = end_LR_type(p) then pop_LR
    else begin incr(LR_problems); type(p) ← kern_node; subtype(p) ← explicit;
      end
  else push_LR(p)
```

This code is used in section 691.

1522.
```
⟨ Check for LR anomalies at the end of hpack 1522 ⟩ ≡
  begin if info(LR_ptr) ≠ before then
    begin while link(q) ≠ null do q ← link(q);
    repeat temp_ptr ← q; q ← new_math(0, info(LR_ptr)); link(temp_ptr) ← q;
      LR_problems ← LR_problems + 10000; pop_LR;
    until info(LR_ptr) = before;
    end;
  if LR_problems > 0 then
    begin ⟨ Report LR problems 1523 ⟩;
    goto common_ending;
    end;
  pop_LR;
  if LR_ptr ≠ null then confusion("LR1");
  end
```

This code is used in section 689.

1523.
```
⟨ Report LR problems 1523 ⟩ ≡
  begin print_ln; print_nl("\endL␣or␣\endR␣problem␣(");
  print_int(LR_problems div 10000); print("␣missing,␣");
  print_int(LR_problems mod 10000); print("␣extra");
  LR_problems ← 0;
  end
```

This code is used in sections 1522 and 1541.

1524. ⟨ Initialize *hlist_out* for mixed direction typesetting 1524 ⟩ ≡
```
if eTeX_ex then
  begin ⟨ Initialize the LR stack 1520 ⟩;
  if box_lr(this_box) = dlist then
    if cur_dir = right_to_left then
      begin cur_dir ← left_to_right; cur_h ← cur_h − width(this_box);
      end
    else set_box_lr(this_box)(0);
  if (cur_dir = right_to_left) ∧ (box_lr(this_box) ≠ reversed) then
    ⟨ Reverse the complete hlist and set the subtype to reversed 1531 ⟩;
  end
```
This code is used in section 655.

1525. ⟨ Finish *hlist_out* for mixed direction typesetting 1525 ⟩ ≡
```
if eTeX_ex then
  begin ⟨ Check for LR anomalies at the end of hlist_out 1528 ⟩;
  if box_lr(this_box) = dlist then cur_dir ← right_to_left;
  end
```
This code is used in section 655.

1526. ⟨ Handle a math node in *hlist_out* 1526 ⟩ ≡
```
begin if eTeX_ex then ⟨ Adjust the LR stack for the hlist_out routine; if necessary reverse an hlist
      segment and goto reswitch 1527 ⟩;
cur_h ← cur_h + width(p);
end
```
This code is used in section 660.

1527. Breaking a paragraph into lines while TEX--XET is disabled may result in lines whith unpaired math nodes. Such hlists are silently accepted in the absence of text direction directives.

```
define LR_dir(#) ≡ (subtype(#) div R_code)   { text direction of a 'math node' }
```
⟨ Adjust the LR stack for the *hlist_out* routine; if necessary reverse an hlist segment and **goto** *reswitch* 1527 ⟩ ≡
```
begin if end_LR(p) then
  if info(LR_ptr) = end_LR_type(p) then pop_LR
  else begin if subtype(p) > L_code then incr(LR_problems);
    end
else begin push_LR(p);
  if LR_dir(p) ≠ cur_dir then ⟨ Reverse an hlist segment and goto reswitch 1532 ⟩;
  end;
type(p) ← kern_node;
end
```
This code is used in section 1526.

1528. ⟨ Check for LR anomalies at the end of *hlist_out* 1528 ⟩ ≡
```
begin while info(LR_ptr) ≠ before do
  begin if info(LR_ptr) > L_code then LR_problems ← LR_problems + 10000;
  pop_LR;
  end;
pop_LR;
end
```
This code is used in section 1525.

1529. **define** *edge_node* = *style_node* { a *style_node* does not occur in hlists }
define *edge_node_size* = *style_node_size* { number of words in an edge node }
define *edge_dist*(#) ≡ *depth*(#)
{ new *left_edge* position relative to *cur_h* (after *width* has been taken into account) }

⟨ Declare procedures needed in *hlist_out*, *vlist_out* 1431 ⟩ +≡
function *new_edge*(*s* : *small_number*; *w* : *scaled*): *pointer*; { create an edge node }
var *p*: *pointer*; { the new node }
begin *p* ← *get_node*(*edge_node_size*); *type*(*p*) ← *edge_node*; *subtype*(*p*) ← *s*; *width*(*p*) ← *w*;
edge_dist(*p*) ← 0; { the *edge_dist* field will be set later }
new_edge ← *p*;
end;

1530. ⟨ Cases of *hlist_out* that arise in mixed direction text only 1530 ⟩ ≡
edge_node: **begin** *cur_h* ← *cur_h* + *width*(*p*); *left_edge* ← *cur_h* + *edge_dist*(*p*); *cur_dir* ← *subtype*(*p*);
end;
This code is used in section 660.

1531. We detach the hlist, start a new one consisting of just one kern node, append the reversed list, and set the width of the kern node.

⟨ Reverse the complete hlist and set the subtype to *reversed* 1531 ⟩ ≡
begin *save_h* ← *cur_h*; *temp_ptr* ← *p*; *p* ← *new_kern*(0); *link*(*prev_p*) ← *p*; *cur_h* ← 0;
link(*p*) ← *reverse*(*this_box*, *null*, *cur_g*, *cur_glue*); *width*(*p*) ← −*cur_h*; *cur_h* ← *save_h*;
set_box_lr(*this_box*)(*reversed*);
end
This code is used in section 1524.

1532. We detach the remainder of the hlist, replace the math node by an edge node, and append the reversed hlist segment to it; the tail of the reversed segment is another edge node and the remainder of the original list is attached to it.

⟨ Reverse an hlist segment and **goto** *reswitch* 1532 ⟩ ≡
begin *save_h* ← *cur_h*; *temp_ptr* ← *link*(*p*); *rule_wd* ← *width*(*p*); *free_node*(*p*, *small_node_size*);
cur_dir ← *reflected*; *p* ← *new_edge*(*cur_dir*, *rule_wd*); *link*(*prev_p*) ← *p*;
cur_h ← *cur_h* − *left_edge* + *rule_wd*; *link*(*p*) ← *reverse*(*this_box*, *new_edge*(*reflected*, 0), *cur_g*, *cur_glue*);
edge_dist(*p*) ← *cur_h*; *cur_dir* ← *reflected*; *cur_h* ← *save_h*; **goto** *reswitch*;
end
This code is used in section 1527.

1533. The *reverse* function defined here is responsible to reverse the nodes of an hlist (segment). The first parameter *this_box* is the enclosing hlist node, the second parameter t is to become the tail of the reversed list, and the global variable *temp_ptr* is the head of the list to be reversed. Finally *cur_g* and *cur_glue* are the current glue rounding state variables, to be updated by this function. We remove nodes from the original list and add them to the head of the new one.

```
⟨ Declare procedures needed in hlist_out, vlist_out 1431 ⟩ +≡
function reverse(this_box, t : pointer; var cur_g : scaled; var cur_glue : real): pointer;
  label reswitch, next_p, done;
  var l: pointer;  { the new list }
    p: pointer;  { the current node }
    q: pointer;  { the next node }
    g_order: glue_ord;  { applicable order of infinity for glue }
    g_sign: normal .. shrinking;  { selects type of glue }
    glue_temp: real;  { glue value before rounding }
    m, n: halfword;  { count of unmatched math nodes }
  begin g_order ← glue_order(this_box); g_sign ← glue_sign(this_box); l ← t; p ← temp_ptr;
  m ← min_halfword; n ← min_halfword;
  loop begin while p ≠ null do ⟨ Move node p to the new list and go to the next node; or goto done if
          the end of the reflected segment has been reached 1534 ⟩;
    if (t = null) ∧ (m = min_halfword) ∧ (n = min_halfword) then goto done;
    p ← new_math(0, info(LR_ptr)); LR_problems ← LR_problems + 10000;
      { manufacture one missing math node }
    end;
done: reverse ← l;
  end;
```

1534. ⟨ Move node p to the new list and go to the next node; or **goto** *done* if the end of the reflected segment has been reached 1534 ⟩ ≡

```
reswitch: if is_char_node(p) then
    repeat f ← font(p); c ← character(p); cur_h ← cur_h + char_width(f)(char_info(f)(c)); q ← link(p);
      link(p) ← l; l ← p; p ← q;
    until ¬is_char_node(p)
  else ⟨ Move the non-char_node p to the new list 1535 ⟩
```

This code is used in section 1533.

1535. ⟨ Move the non-*char_node* p to the new list 1535 ⟩ ≡

```
  begin q ← link(p);
  case type(p) of
  hlist_node, vlist_node, rule_node, kern_node: rule_wd ← width(p);
  ⟨ Cases of reverse that need special treatment 1536 ⟩
  edge_node: confusion("LR2");
  othercases goto next_p
  endcases;
  cur_h ← cur_h + rule_wd;
next_p: link(p) ← l;
  if type(p) = kern_node then
    if (rule_wd = 0) ∨ (l = null) then
      begin free_node(p, small_node_size); p ← l;
      end;
  l ← p; p ← q;
  end
```

This code is used in section 1534.

1536. Need to measure *native_word* and picture nodes when reversing!

```
⟨ Cases of reverse that need special treatment 1536 ⟩ ≡
whatsit_node: if (is_native_word_subtype(p)) ∨ (subtype(p) = glyph_node) ∨ (subtype(p) =
        pic_node) ∨ (subtype(p) = pdf_node) then rule_wd ← width(p)
  else goto next_p;
```

See also sections 1537, 1538, and 1539.

This code is used in section 1535.

1537. Here we compute the effective width of a glue node as in *hlist_out*.

```
⟨ Cases of reverse that need special treatment 1536 ⟩ +≡
glue_node: begin round_glue; ⟨ Handle a glue node for mixed direction typesetting 1509 ⟩;
  end;
```

1538. A ligature node is replaced by a char node.

```
⟨ Cases of reverse that need special treatment 1536 ⟩ +≡
ligature_node: begin flush_node_list(lig_ptr(p)); temp_ptr ← p; p ← get_avail;
  mem[p] ← mem[lig_char(temp_ptr)]; link(p) ← q; free_node(temp_ptr, small_node_size); goto reswitch;
  end;
```

1539. Math nodes in an inner reflected segment are modified, those at the outer level are changed into kern nodes.

```
⟨ Cases of reverse that need special treatment 1536 ⟩ +≡
math_node: begin rule_wd ← width(p);
  if end_LR(p) then
    if info(LR_ptr) ≠ end_LR_type(p) then
      begin type(p) ← kern_node; incr(LR_problems);
      end
    else begin pop_LR;
      if n > min_halfword then
        begin decr(n); decr(subtype(p));   { change after into before }
        end
      else begin type(p) ← kern_node;
        if m > min_halfword then decr(m)
        else ⟨ Finish the reversed hlist segment and goto done 1540 ⟩;
        end;
      end
  else begin push_LR(p);
    if (n > min_halfword) ∨ (LR_dir(p) ≠ cur_dir) then
      begin incr(n); incr(subtype(p));   { change before into after }
      end
    else begin type(p) ← kern_node; incr(m);
      end;
    end;
  end;
```

1540. Finally we have found the end of the hlist segment to be reversed; the final math node is released and the remaining list attached to the edge node terminating the reversed segment.

⟨ Finish the reversed hlist segment and **goto** *done* 1540 ⟩ ≡
```
begin free_node(p, small_node_size); link(t) ← q; width(t) ← rule_wd; edge_dist(t) ← −cur_h − rule_wd;
goto done;
end
```
This code is used in section 1539.

1541. ⟨ Check for LR anomalies at the end of *ship_out* 1541 ⟩ ≡
```
begin if LR_problems > 0 then
  begin ⟨ Report LR problems 1523 ⟩;
  print_char("("); print_ln;
  end;
if (LR_ptr ≠ null) ∨ (cur_dir ≠ left_to_right) then confusion("LR3");
end
```
This code is used in section 676.

1542. Some special actions are required for displayed equation in paragraphs with mixed direction texts. First of all we have to set the text direction preceding the display.

⟨ Set the value of x to the text direction before the display 1542 ⟩ ≡
```
if LR_save = null then x ← 0
else if info(LR_save) ≥ R_code then x ← −1 else x ← 1
```
This code is used in sections 1543 and 1545.

1543. ⟨ Prepare for display after an empty paragraph 1543 ⟩ ≡
```
begin pop_nest; ⟨ Set the value of x to the text direction before the display 1542 ⟩;
end
```
This code is used in section 1199.

1544. When calculating the natural width, w, of the final line preceding the display, we may have to copy all or part of its hlist. We copy, however, only those parts of the original list that are relevant for the computation of *pre_display_size*.

```
⟨ Declare subprocedures for init_math 1544 ⟩ ≡
procedure just_copy(p, h, t : pointer);
  label found, not_found;
  var r: pointer;  { current node being fabricated for new list }
    words: 0 .. 5;  { number of words remaining to be copied }
  begin while p ≠ null do
    begin words ← 1;  { this setting occurs in more branches than any other }
    if is_char_node(p) then r ← get_avail
    else case type(p) of
      hlist_node, vlist_node: begin r ← get_node(box_node_size); mem[r + 6] ← mem[p + 6];
        mem[r + 5] ← mem[p + 5];  { copy the last two words }
        words ← 5; list_ptr(r) ← null;  { this affects mem[r + 5] }
        end;
      rule_node: begin r ← get_node(rule_node_size); words ← rule_node_size;
        end;
      ligature_node: begin r ← get_avail;  { only font and character are needed }
        mem[r] ← mem[lig_char(p)]; goto found;
        end;
      kern_node, math_node: begin r ← get_node(small_node_size); words ← small_node_size;
        end;
      glue_node: begin r ← get_node(small_node_size); add_glue_ref(glue_ptr(p));
        glue_ptr(r) ← glue_ptr(p); leader_ptr(r) ← null;
        end;
      whatsit_node: ⟨ Make a partial copy of the whatsit node p and make r point to it; set words to the
          number of initial words not yet copied 1417 ⟩;
      othercases goto not_found
      endcases;
    while words > 0 do
      begin decr(words); mem[r + words] ← mem[p + words];
      end;
  found: link(h) ← r; h ← r;
  not_found: p ← link(p);
    end;
  link(h) ← t;
  end;
```

See also section 1549.

This code is used in section 1192.

1545. When the final line ends with R-text, the value w refers to the line reflected with respect to the left edge of the enclosing vertical list.

```
⟨ Prepare for display after a non-empty paragraph 1545 ⟩ ≡
  if eTeX_ex then ⟨ Let j be the prototype box for the display 1551 ⟩;
  v ← shift_amount(just_box); ⟨ Set the value of x to the text direction before the display 1542 ⟩;
  if x ≥ 0 then
    begin p ← list_ptr(just_box); link(temp_head) ← null;
    end
  else begin v ← −v − width(just_box); p ← new_math(0, begin_L_code); link(temp_head) ← p;
    just_copy(list_ptr(just_box), p, new_math(0, end_L_code)); cur_dir ← right_to_left;
    end;
  v ← v + 2 ∗ quad(cur_font);
  if TeXXeT_en then ⟨ Initialize the LR stack 1520 ⟩
```

This code is used in section 1200.

1546.

```
⟨ Finish the natural width computation 1546 ⟩ ≡
  if TeXXeT_en then
    begin while LR_ptr ≠ null do pop_LR;
    if LR_problems ≠ 0 then
      begin w ← max_dimen; LR_problems ← 0;
      end;
    end;
  cur_dir ← left_to_right; flush_node_list(link(temp_head))
```

This code is used in section 1200.

1547. In the presence of text direction directives we assume that any LR problems have been fixed by the *hpack* routine. If the final line contains, however, text direction directives while TEX--XETEX is disabled, then we set $w \leftarrow max_dimen$.

```
⟨ Cases of 'Let d be the natural width' that need special treatment 1547 ⟩ ≡
math_node: begin d ← width(p);
  if TeXXeT_en then ⟨ Adjust the LR stack for the init_math routine 1548 ⟩
  else if subtype(p) ≥ L_code then
      begin w ← max_dimen; goto done;
      end;
  end;
edge_node: begin d ← width(p); cur_dir ← subtype(p);
  end;
```

This code is used in section 1201.

1548. ⟨ Adjust the LR stack for the *init_math* routine 1548 ⟩ ≡

```
if end_LR(p) then
  begin if info(LR_ptr) = end_LR_type(p) then pop_LR
  else if subtype(p) > L_code then
      begin w ← max_dimen; goto done;
      end
  end
else begin push_LR(p);
  if LR_dir(p) ≠ cur_dir then
    begin just_reverse(p); p ← temp_head;
    end;
  end
```

This code is used in section 1547.

1549. ⟨ Declare subprocedures for *init_math* 1544 ⟩ +≡

```
procedure just_reverse(p : pointer);
  label found, done;
  var l: pointer;   { the new list }
    t: pointer;   { tail of reversed segment }
    q: pointer;   { the next node }
    m, n: halfword;   { count of unmatched math nodes }
  begin m ← min_halfword; n ← min_halfword;
  if link(temp_head) = null then
    begin just_copy(link(p), temp_head, null); q ← link(temp_head);
    end
  else begin q ← link(p); link(p) ← null; flush_node_list(link(temp_head));
    end;
  t ← new_edge(cur_dir, 0); l ← t; cur_dir ← reflected;
  while q ≠ null do
    if is_char_node(q) then
      repeat p ← q; q ← link(p); link(p) ← l; l ← p;
      until ¬is_char_node(q)
    else begin p ← q; q ← link(p);
      if type(p) = math_node then ⟨ Adjust the LR stack for the just_reverse routine 1550 ⟩;
      link(p) ← l; l ← p;
      end;
  goto done;
found: width(t) ← width(p); link(t) ← q; free_node(p, small_node_size);
done: link(temp_head) ← l;
  end;
```

1550. ⟨ Adjust the LR stack for the *just_reverse* routine 1550 ⟩ ≡

```
if end_LR(p) then
  if info(LR_ptr) ≠ end_LR_type(p) then
    begin type(p) ← kern_node; incr(LR_problems);
    end
  else begin pop_LR;
    if n > min_halfword then
      begin decr(n); decr(subtype(p)); { change after into before }
      end
    else begin if m > min_halfword then decr(m) else goto found;
      type(p) ← kern_node;
      end;
    end
else begin push_LR(p);
  if (n > min_halfword) ∨ (LR_dir(p) ≠ cur_dir) then
    begin incr(n); incr(subtype(p)); { change before into after }
    end
  else begin type(p) ← kern_node; incr(m);
    end;
  end
```

This code is used in section 1549.

1551. The prototype box is an hlist node with the width, glue set, and shift amount of *just_box*, i.e., the last line preceding the display. Its hlist reflects the current `\leftskip` and `\rightskip`.

⟨ Let j be the prototype box for the display 1551 ⟩ ≡

```
begin if right_skip = zero_glue then j ← new_kern(0)
else j ← new_param_glue(right_skip_code);
if left_skip = zero_glue then p ← new_kern(0)
else p ← new_param_glue(left_skip_code);
link(p) ← j; j ← new_null_box; width(j) ← width(just_box); shift_amount(j) ← shift_amount(just_box);
list_ptr(j) ← p; glue_order(j) ← glue_order(just_box); glue_sign(j) ← glue_sign(just_box);
glue_set(j) ← glue_set(just_box);
end
```

This code is used in section 1545.

1552. At the end of a displayed equation we retrieve the prototype box.

⟨ Local variables for finishing a displayed formula 1252 ⟩ +≡

```
j: pointer; { prototype box }
```

1553. ⟨ Retrieve the prototype box 1553 ⟩ ≡

```
if mode = mmode then j ← LR_box
```

This code is used in sections 1248 and 1248.

1554. ⟨ Flush the prototype box 1554 ⟩ ≡

```
flush_node_list(j)
```

This code is used in section 1253.

1555. The *app_display* procedure used to append the displayed equation and/or equation number to the current vertical list has three parameters: the prototype box, the hbox to be appended, and the displacement of the hbox in the display line.

```
⟨ Declare subprocedures for after_math 1555 ⟩ ≡
procedure app_display(j, b : pointer; d : scaled);
  var z: scaled; { width of the line }
    s: scaled; { move the line right this much }
    e: scaled; { distance from right edge of box to end of line }
    x: integer; { pre_display_direction }
    p, q, r, t, u: pointer; { for list manipulation }
  begin s ← display_indent; x ← pre_display_direction;
  if x = 0 then shift_amount(b) ← s + d
  else begin z ← display_width; p ← b; ⟨ Set up the hlist for the display line 1556 ⟩;
    ⟨ Package the display line 1557 ⟩;
    end;
  append_to_vlist(b);
  end;
```

This code is used in section 1248.

1556. Here we construct the hlist for the display, starting with node p and ending with node q. We also set d and e to the amount of kerning to be added before and after the hlist (adjusted for the prototype box).

```
⟨ Set up the hlist for the display line 1556 ⟩ ≡
  if x > 0 then e ← z − d − width(p)
  else begin e ← d; d ← z − e − width(p);
    end;
  if j ≠ null then
    begin b ← copy_node_list(j); height(b) ← height(p); depth(b) ← depth(p); s ← s − shift_amount(b);
    d ← d + s; e ← e + width(b) − z − s;
    end;
  if box_lr(p) = dlist then q ← p { display or equation number }
  else begin { display and equation number }
    r ← list_ptr(p); free_node(p, box_node_size);
    if r = null then confusion("LR4");
    if x > 0 then
      begin p ← r;
      repeat q ← r; r ← link(r); { find tail of list }
      until r = null;
      end
    else begin p ← null; q ← r;
      repeat t ← link(r); link(r) ← p; p ← r; r ← t; { reverse list }
      until r = null;
      end;
    end
```

This code is used in section 1555.

1557. In the presence of a prototype box we use its shift amount and width to adjust the values of kerning and add these values to the glue nodes inserted to cancel the `\leftskip` and `\rightskip`. If there is no prototype box (because the display is preceded by an empty paragraph), or if the skip parameters are zero, we just add kerns.

The *cancel_glue* macro creates and links a glue node that is, together with another glue node, equivalent to a given amount of kerning. We can use j as temporary pointer, since all we need is $j \neq null$.

```
define cancel_glue(#) ≡ j ← new_skip_param(#); cancel_glue_cont
define cancel_glue_cont(#) ≡ link(#) ← j; cancel_glue_cont_cont
define cancel_glue_cont_cont(#) ≡ link(j) ← #; cancel_glue_end
define cancel_glue_end(#) ≡ j ← glue_ptr(#); cancel_glue_end_end
define cancel_glue_end_end(#) ≡ stretch_order(temp_ptr) ← stretch_order(j);
    shrink_order(temp_ptr) ← shrink_order(j); width(temp_ptr) ← # − width(j);
    stretch(temp_ptr) ← −stretch(j); shrink(temp_ptr) ← −shrink(j)
```

⟨ Package the display line 1557 ⟩ ≡

```
if j = null then
  begin r ← new_kern(0); t ← new_kern(0);   { the widths will be set later }
  end
else begin r ← list_ptr(b); t ← link(r);
  end;
u ← new_math(0, end_M_code);
if type(t) = glue_node then   { t is \rightskip glue }
  begin cancel_glue(right_skip_code)(q)(u)(t)(e); link(u) ← t;
  end
else begin width(t) ← e; link(t) ← u; link(q) ← t;
  end;
u ← new_math(0, begin_M_code);
if type(r) = glue_node then   { r is \leftskip glue }
  begin cancel_glue(left_skip_code)(u)(p)(r)(d); link(r) ← u;
  end
else begin width(r) ← d; link(r) ← p; link(u) ← r;
  if j = null then
    begin b ← hpack(u, natural); shift_amount(b) ← s;
    end
  else list_ptr(b) ← u;
  end
```

This code is used in section 1555.

1558. The *scan_tokens* feature of ε-TEX defines the `\scantokens` primitive.

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡

```
primitive("scantokens", input, 2);
```

1559. ⟨ Cases of *input* for *print_cmd_chr* 1559 ⟩ ≡

```
else if chr_code = 2 then print_esc("scantokens")
```

This code is used in section 411.

1560. ⟨ Cases for *input* 1560 ⟩ ≡

```
else if cur_chr = 2 then pseudo_start
```

This code is used in section 412.

1561. The global variable *pseudo_files* is used to maintain a stack of pseudo files. The *info* field of each pseudo file points to a linked list of variable size nodes representing lines not yet processed: the *info* field of the first word contains the size of this node, all the following words contain ASCII codes.

```
⟨ Global variables 13 ⟩ +≡
pseudo_files: pointer; { stack of pseudo files }
```

1562.
```
⟨ Set initial values of key variables 23 ⟩ +≡
  pseudo_files ← null;
```

1563. The *pseudo_start* procedure initiates reading from a pseudo file.

```
⟨ Declare ε-TEX procedures for expanding 1563 ⟩ ≡
procedure pseudo_start; forward;
```

See also sections 1621, 1626, and 1630.

This code is used in section 396.

1564.
```
⟨ Declare ε-TEX procedures for token lists 1493 ⟩ +≡
procedure pseudo_start;
  var old_setting: 0 .. max_selector; { holds selector setting }
    s: str_number; { string to be converted into a pseudo file }
    l, m: pool_pointer; { indices into str_pool }
    p, q, r: pointer; { for list construction }
    w: four_quarters; { four ASCII codes }
    nl, sz: integer;
  begin scan_general_text; old_setting ← selector; selector ← new_string; token_show(temp_head);
  selector ← old_setting; flush_list(link(temp_head)); str_room(1); s ← make_string;
  ⟨ Convert string s into a new pseudo file 1565 ⟩;
  flush_string; ⟨ Initiate input from new pseudo file 1566 ⟩;
  end;
```

1565. ⟨ Convert string s into a new pseudo file 1565 ⟩ ≡

```
str_pool[pool_ptr] ← si("␣"); l ← str_start_macro(s); nl ← si(new_line_char); p ← get_avail; q ← p;
while l < pool_ptr do
  begin m ← l;
  while (l < pool_ptr) ∧ (str_pool[l] ≠ nl) do incr(l);
  sz ← (l − m + 7) div 4;
  if sz = 1 then sz ← 2;
  r ← get_node(sz); link(q) ← r; q ← r; info(q) ← hi(sz);
  while sz > 2 do
    begin decr(sz); incr(r); w.b0 ← qi(so(str_pool[m])); w.b1 ← qi(so(str_pool[m + 1]));
    w.b2 ← qi(so(str_pool[m + 2])); w.b3 ← qi(so(str_pool[m + 3])); mem[r].qqqq ← w; m ← m + 4;
    end;
  w.b0 ← qi("␣"); w.b1 ← qi("␣"); w.b2 ← qi("␣"); w.b3 ← qi("␣");
  if l > m then
    begin w.b0 ← qi(so(str_pool[m]));
    if l > m + 1 then
      begin w.b1 ← qi(so(str_pool[m + 1]));
      if l > m + 2 then
        begin w.b2 ← qi(so(str_pool[m + 2]));
        if l > m + 3 then w.b3 ← qi(so(str_pool[m + 3]));
        end;
      end;
    end;
  mem[r + 1].qqqq ← w;
  if str_pool[l] = nl then incr(l);
  end;
info(p) ← link(p); link(p) ← pseudo_files; pseudo_files ← p
```

This code is used in section 1564.

1566. ⟨ Initiate input from new pseudo file 1566 ⟩ ≡

```
begin_file_reading; { set up cur_file and new level of input }
line ← 0; limit ← start; loc ← limit + 1; { force line read }
if tracing_scan_tokens > 0 then
  begin if term_offset > max_print_line − 3 then print_ln
  else if (term_offset > 0) ∨ (file_offset > 0) then print_char("␣");
  name ← 19; print("(␣"); incr(open_parens); update_terminal;
  end
else name ← 18
```

This code is used in section 1564.

1567. Here we read a line from the current pseudo file into *buffer*.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
function pseudo_input: boolean; { inputs the next line or returns false }
  var p: pointer; { current line from pseudo file }
    sz: integer; { size of node p }
    w: four_quarters; { four ASCII codes }
    r: pointer; { loop index }
  begin last ← first; { cf. Matthew 19 : 30 }
  p ← info(pseudo_files);
  if p = null then pseudo_input ← false
  else begin info(pseudo_files) ← link(p); sz ← ho(info(p));
    if 4 * sz − 3 ≥ buf_size − last then ⟨ Report overflow of the input buffer, and abort 35 ⟩;
    last ← first;
    for r ← p + 1 to p + sz − 1 do
      begin w ← mem[r].qqqq; buffer[last] ← w.b0; buffer[last + 1] ← w.b1; buffer[last + 2] ← w.b2;
      buffer[last + 3] ← w.b3; last ← last + 4;
      end;
    if last ≥ max_buf_stack then max_buf_stack ← last + 1;
    while (last > first) ∧ (buffer[last − 1] = "␣") do decr(last);
    free_node(p, sz); pseudo_input ← true;
    end;
  end;
```

1568. When we are done with a pseudo file we 'close' it.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure pseudo_close; { close the top level pseudo file }
  var p, q: pointer;
  begin p ← link(pseudo_files); q ← info(pseudo_files); free_avail(pseudo_files); pseudo_files ← p;
  while q ≠ null do
    begin p ← q; q ← link(p); free_node(p, ho(info(p)));
    end;
  end;
```

1569.

```
⟨ Dump the ε-TEX state 1464 ⟩ +≡
  while pseudo_files ≠ null do pseudo_close; { flush pseudo files }
```

1570.

```
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("readline", read_to_cs, 1);
```

1571.

```
⟨ Cases of read for print_cmd_chr 1571 ⟩ ≡
else print_esc("readline")
```

This code is used in section 296.

1572. ⟨ Handle `\readline` and **goto** *done* 1572 ⟩ ≡

```
if j = 1 then
  begin while loc ≤ limit do { current line not yet finished }
    begin cur_chr ← buffer[loc]; incr(loc);
    if cur_chr = "␣" then cur_tok ← space_token else cur_tok ← cur_chr + other_token;
    store_new_token(cur_tok);
    end;
  goto done;
  end
```

This code is used in section 518.

1573. Here we define the additional conditionals of ε-TEX as well as the `\unless` prefix.

```
define if_def_code = 17 { '\ifdefined' }
define if_cs_code = 18 { '\ifcsname' }
define if_font_char_code = 19 { '\iffontchar' }
define if_in_csname_code = 20 { '\ifincsname' }
```

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡

```
primitive("unless", expand_after, 1);
primitive("ifdefined", if_test, if_def_code); primitive("ifcsname", if_test, if_cs_code);
primitive("iffontchar", if_test, if_font_char_code); primitive("ifincsname", if_test, if_in_csname_code);
```

1574. ⟨ Cases of *expandafter* for *print_cmd_chr* 1574 ⟩ ≡

```
else print_esc("unless")
```

This code is used in section 296.

1575. ⟨ Cases of *if_test* for *print_cmd_chr* 1575 ⟩ ≡

```
if_def_code: print_esc("ifdefined");
if_cs_code: print_esc("ifcsname");
if_font_char_code: print_esc("iffontchar");
if_in_csname_code: print_esc("ifincsname");
```

This code is used in section 523.

1576. The result of a boolean condition is reversed when the conditional is preceded by `\unless`.

⟨ Negate a boolean conditional and **goto** *reswitch* 1576 ⟩ ≡

```
begin get_token;
if (cur_cmd = if_test) ∧ (cur_chr ≠ if_case_code) then
  begin cur_chr ← cur_chr + unless_code; goto reswitch;
  end;
print_err("You␣can´t␣use␣`"); print_esc("unless"); print("´␣before␣`");
print_cmd_chr(cur_cmd, cur_chr); print_char("´");
help1("Continue,␣and␣I´ll␣forget␣that␣it␣ever␣happened."); back_error;
end
```

This code is used in section 399.

1577. The conditional `\ifdefined` tests if a control sequence is defined.

We need to reset *scanner_status*, since `\outer` control sequences are allowed, but we might be scanning a macro definition or preamble.

```
⟨ Cases for conditional 1577 ⟩ ≡
if_def_code: begin save_scanner_status ← scanner_status; scanner_status ← normal; get_next;
  b ← (cur_cmd ≠ undefined_cs); scanner_status ← save_scanner_status;
  end;
```

See also sections 1578 and 1580.

This code is used in section 536.

1578. The conditional `\ifcsname` is equivalent to `{\expandafter }\expandafter \ifdefined \csname`, except that no new control sequence will be entered into the hash table (once all tokens preceding the mandatory `\endcsname` have been expanded).

```
⟨ Cases for conditional 1577 ⟩ +≡
if_cs_code: begin n ← get_avail; p ← n;  { head of the list of characters }
  e ← is_in_csname; is_in_csname ← true;
  repeat get_x_token;
    if cur_cs = 0 then store_new_token(cur_tok);
  until cur_cs ≠ 0;
  if cur_cmd ≠ end_cs_name then ⟨ Complain about missing \endcsname 407 ⟩;
  ⟨ Look up the characters of list n in the hash table, and set cur_cs 1579 ⟩;
  flush_list(n); b ← (eq_type(cur_cs) ≠ undefined_cs); is_in_csname ← e;
  end;
```

1579.

```
⟨ Look up the characters of list n in the hash table, and set cur_cs 1579 ⟩ ≡
  m ← first; p ← link(n);
  while p ≠ null do
    begin if m ≥ max_buf_stack then
      begin max_buf_stack ← m + 1;
      if max_buf_stack = buf_size then overflow("buffer␣size", buf_size);
      end;
    buffer[m] ← info(p) mod max_char_val; incr(m); p ← link(p);
    end;
  if m > first + 1 then cur_cs ← id_lookup(first, m − first)  { no_new_control_sequence is true }
  else if m = first then cur_cs ← null_cs  { the list is empty }
    else cur_cs ← single_base + buffer[first]  { the list has length one }
```

This code is used in section 1578.

1580. The conditional `\iffontchar` tests the existence of a character in a font.

```
⟨ Cases for conditional 1577 ⟩ +≡
if_in_csname_code: b ← is_in_csname;
if_font_char_code: begin scan_font_ident; n ← cur_val; scan_usv_num;
  if is_native_font(n) then b ← (map_char_to_glyph(n, cur_val) > 0)
  else begin if (font_bc[n] ≤ cur_val) ∧ (font_ec[n] ≥ cur_val) then
      b ← char_exists(char_info(n)(qi(cur_val)))
    else b ← false;
    end;
  end;
```

1581. The *protected* feature of ε-TEX defines the `\protected` prefix command for macro definitions. Such macros are protected against expansions when lists of expanded tokens are built, e.g., for `\edef` or during `\write`.

```
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("protected", prefix, 8);
```

1582.
```
⟨ Cases of prefix for print_cmd_chr 1582 ⟩ ≡
else if chr_code = 8 then print_esc("protected")
```
This code is used in section 1263.

1583. The *get_x_or_protected* procedure is like *get_x_token* except that protected macros are not expanded.

```
⟨ Declare ε-TEX procedures for scanning 1492 ⟩ +≡
procedure get_x_or_protected; { sets cur_cmd, cur_chr, cur_tok, and expands non-protected macros }
  label exit;
  begin loop begin get_token;
    if cur_cmd ≤ max_command then return;
    if (cur_cmd ≥ call) ∧ (cur_cmd < end_template) then
      if info(link(cur_chr)) = protected_token then return;
    expand;
    end;
exit: end;
```

1584. A group entered (or a conditional started) in one file may end in a different file. Such slight anomalies, although perfectly legitimate, may cause errors that are difficult to locate. In order to be able to give a warning message when such anomalies occur, ε-TEX uses the *grp_stack* and *if_stack* arrays to record the initial *cur_boundary* and *cond_ptr* values for each input file.

```
⟨ Global variables 13 ⟩ +≡
grp_stack: array [0 .. max_in_open] of save_pointer; { initial cur_boundary }
if_stack: array [0 .. max_in_open] of pointer; { initial cond_ptr }
```

1585. When a group ends that was apparently entered in a different input file, the *group_warning* procedure is invoked in order to update the *grp_stack*. If moreover `\tracingnesting` is positive we want to give a warning message. The situation is, however, somewhat complicated by two facts: (1) There may be *grp_stack* elements without a corresponding `\input` file or `\scantokens` pseudo file (e.g., error insertions from the terminal); and (2) the relevant information is recorded in the *name_field* of the *input_stack* only loosely synchronized with the *in_open* variable indexing *grp_stack*.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure group_warning;
  var i: 0 .. max_in_open; { index into grp_stack }
    w: boolean; { do we need a warning? }
  begin base_ptr ← input_ptr; input_stack[base_ptr] ← cur_input; { store current state }
  i ← in_open; w ← false;
  while (grp_stack[i] = cur_boundary) ∧ (i > 0) do
    begin ⟨ Set variable w to indicate if this case should be reported 1586 ⟩;
    grp_stack[i] ← save_index(save_ptr); decr(i);
    end;
  if w then
    begin print_nl("Warning: end of "); print_group(true); print(" of a different file"); print_ln;
    if tracing_nesting > 1 then show_context;
    if history = spotless then history ← warning_issued;
    end;
  end;
```

1586. This code scans the input stack in order to determine the type of the current input file.

```
⟨ Set variable w to indicate if this case should be reported 1586 ⟩ ≡
  if tracing_nesting > 0 then
    begin while (input_stack[base_ptr].state_field = token_list) ∨ (input_stack[base_ptr].index_field > i) do
      decr(base_ptr);
    if input_stack[base_ptr].name_field > 17 then w ← true;
    end
```

This code is used in sections 1585 and 1587.

1587. When a conditional ends that was apparently started in a different input file, the *if_warning* procedure is invoked in order to update the *if_stack*. If moreover `\tracingnesting` is positive we want to give a warning message (with the same complications as above).

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure if_warning;
  var i: 0 .. max_in_open; { index into if_stack }
    w: boolean; { do we need a warning? }
  begin base_ptr ← input_ptr; input_stack[base_ptr] ← cur_input; { store current state }
  i ← in_open; w ← false;
  while if_stack[i] = cond_ptr do
    begin ⟨ Set variable w to indicate if this case should be reported 1586 ⟩;
    if_stack[i] ← link(cond_ptr); decr(i);
    end;
  if w then
    begin print_nl("Warning:␣end␣of␣"); print_cmd_chr(if_test, cur_if); print_if_line(if_line);
    print("␣of␣a␣different␣file"); print_ln;
    if tracing_nesting > 1 then show_context;
    if history = spotless then history ← warning_issued;
    end;
  end;
```

1588. Conversely, the *file_warning* procedure is invoked when a file ends and some groups entered or conditionals started while reading from that file are still incomplete.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure file_warning;
  var p: pointer; { saved value of save_ptr or cond_ptr }
    l: quarterword; { saved value of cur_level or if_limit }
    c: quarterword; { saved value of cur_group or cur_if }
    i: integer; { saved value of if_line }
  begin p ← save_ptr; l ← cur_level; c ← cur_group; save_ptr ← cur_boundary;
  while grp_stack[in_open] ≠ save_ptr do
    begin decr(cur_level); print_nl("Warning:␣end␣of␣file␣when␣"); print_group(true);
    print("␣is␣incomplete");
    cur_group ← save_level(save_ptr); save_ptr ← save_index(save_ptr)
    end;
  save_ptr ← p; cur_level ← l; cur_group ← c; { restore old values }
  p ← cond_ptr; l ← if_limit; c ← cur_if; i ← if_line;
  while if_stack[in_open] ≠ cond_ptr do
    begin print_nl("Warning:␣end␣of␣file␣when␣"); print_cmd_chr(if_test, cur_if);
    if if_limit = fi_code then print_esc("else");
    print_if_line(if_line); print("␣is␣incomplete");
    if_line ← if_line_field(cond_ptr); cur_if ← subtype(cond_ptr); if_limit ← type(cond_ptr);
    cond_ptr ← link(cond_ptr);
    end;
  cond_ptr ← p; if_limit ← l; cur_if ← c; if_line ← i; { restore old values }
  print_ln;
  if tracing_nesting > 1 then show_context;
  if history = spotless then history ← warning_issued;
  end;
```

1589. Here are the additional ε-TEX primitives for expressions.

```
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("numexpr", last_item, eTeX_expr − int_val + int_val);
  primitive("dimexpr", last_item, eTeX_expr − int_val + dimen_val);
  primitive("glueexpr", last_item, eTeX_expr − int_val + glue_val);
  primitive("muexpr", last_item, eTeX_expr − int_val + mu_val);
```

1590.
```
⟨ Cases of last_item for print_cmd_chr 1453 ⟩ +≡
eTeX_expr − int_val + int_val: print_esc("numexpr");
eTeX_expr − int_val + dimen_val: print_esc("dimexpr");
eTeX_expr − int_val + glue_val: print_esc("glueexpr");
eTeX_expr − int_val + mu_val: print_esc("muexpr");
```

1591. This code for reducing *cur_val_level* and/or negating the result is similar to the one for all the other cases of *scan_something_internal*, with the difference that *scan_expr* has already increased the reference count of a glue specification.

```
⟨ Process an expression and return 1591 ⟩ ≡
  begin if m < eTeX_mu then
    begin case m of
      ⟨ Cases for fetching a glue value 1618 ⟩
    end; { there are no other cases }
    cur_val_level ← glue_val;
    end
  else if m < eTeX_expr then
      begin case m of
        ⟨ Cases for fetching a mu value 1619 ⟩
      end; { there are no other cases }
      cur_val_level ← mu_val;
      end
    else begin cur_val_level ← m − eTeX_expr + int_val; scan_expr;
      end;
  while cur_val_level > level do
    begin if cur_val_level = glue_val then
      begin m ← cur_val; cur_val ← width(m); delete_glue_ref(m);
      end
    else if cur_val_level = mu_val then mu_error;
    decr(cur_val_level);
    end;
  if negative then
    if cur_val_level ≥ glue_val then
      begin m ← cur_val; cur_val ← new_spec(m); delete_glue_ref(m);
      ⟨ Negate all three glue components of cur_val 465 ⟩;
      end
    else negate(cur_val);
  return;
  end
```

This code is used in section 458.

1592.
```
⟨ Declare ε-TEX procedures for scanning 1492 ⟩ +≡
procedure scan_expr; forward;
```

1593. The *scan_expr* procedure scans and evaluates an expression.

```
⟨ Declare procedures needed for expressions 1593 ⟩ ≡
⟨ Declare subprocedures for scan_expr 1604 ⟩
procedure scan_expr; { scans and evaluates an expression }
  label restart, continue, found;
  var a, b: boolean; { saved values of arith_error }
    l: small_number; { type of expression }
    r: small_number; { state of expression so far }
    s: small_number; { state of term so far }
    o: small_number; { next operation or type of next factor }
    e: integer; { expression so far }
    t: integer; { term so far }
    f: integer; { current factor }
    n: integer; { numerator of combined multiplication and division }
    p: pointer; { top of expression stack }
    q: pointer; { for stack manipulations }
  begin l ← cur_val_level; a ← arith_error; b ← false; p ← null; incr(expand_depth_count);
  if expand_depth_count ≥ expand_depth then overflow("expansion␣depth", expand_depth);
  ⟨ Scan and evaluate an expression e of type l 1594 ⟩;
  decr(expand_depth_count);
  if b then
    begin print_err("Arithmetic␣overflow"); help2("I␣can't␣evaluate␣this␣expression,")
    ("since␣the␣result␣is␣out␣of␣range."); error;
    if l ≥ glue_val then
      begin delete_glue_ref(e); e ← zero_glue; add_glue_ref(e);
      end
    else e ← 0;
    end;
  arith_error ← a; cur_val ← e; cur_val_level ← l;
  end;
```

See also section 1598.

This code is used in section 496.

1594. Evaluating an expression is a recursive process: When the left parenthesis of a subexpression is scanned we descend to the next level of recursion; the previous level is resumed with the matching right parenthesis.

```
define expr_none = 0  { ( seen, or ( ⟨expr⟩ ) seen }
define expr_add = 1  { ( ⟨expr⟩ + seen }
define expr_sub = 2  { ( ⟨expr⟩ - seen }
define expr_mult = 3  { ⟨term⟩ * seen }
define expr_div = 4  { ⟨term⟩ / seen }
define expr_scale = 5  { ⟨term⟩ * ⟨factor⟩ / seen }
```

```
⟨ Scan and evaluate an expression e of type l 1594 ⟩ ≡
restart: r ← expr_none; e ← 0; s ← expr_none; t ← 0; n ← 0;
continue: if s = expr_none then o ← l else o ← int_val;
  ⟨ Scan a factor f of type o or start a subexpression 1596 ⟩;
found: ⟨ Scan the next operator and set o 1595 ⟩;
  arith_error ← b; ⟨ Make sure that f is in the proper range 1601 ⟩;
  case s of
    ⟨ Cases for evaluation of the current term 1602 ⟩
  end; { there are no other cases }
  if o > expr_sub then s ← o else ⟨ Evaluate the current expression 1603 ⟩;
  b ← arith_error;
  if o ≠ expr_none then goto continue;
  if p ≠ null then ⟨ Pop the expression stack and goto found 1600 ⟩
```

This code is used in section 1593.

1595.

```
⟨ Scan the next operator and set o 1595 ⟩ ≡
  ⟨ Get the next non-blank non-call token 440 ⟩;
  if cur_tok = other_token + "+" then o ← expr_add
  else if cur_tok = other_token + "-" then o ← expr_sub
    else if cur_tok = other_token + "*" then o ← expr_mult
      else if cur_tok = other_token + "/" then o ← expr_div
        else begin o ← expr_none;
          if p = null then
            begin if cur_cmd ≠ relax then back_input;
            end
          else if cur_tok ≠ other_token + ")" then
              begin print_err("Missing␣)␣inserted␣for␣expression");
              help1("I␣was␣expecting␣to␣see␣`+´,␣`-´,␣`*´,␣`/´,␣or␣`)´.␣Didn´t.");
              back_error;
              end;
          end
```

This code is used in section 1594.

1596.

```
⟨ Scan a factor f of type o or start a subexpression 1596 ⟩ ≡
  ⟨ Get the next non-blank non-call token 440 ⟩;
  if cur_tok = other_token + "(" then ⟨ Push the expression stack and goto restart 1599 ⟩;
  back_input;
  if o = int_val then scan_int
  else if o = dimen_val then scan_normal_dimen
    else if o = glue_val then scan_normal_glue
      else scan_mu_glue;
  f ← cur_val
```

This code is used in section 1594.

1597. ⟨ Declare ε-TEX procedures for scanning 1492 ⟩ +≡
procedure *scan_normal_glue*; *forward*;
procedure *scan_mu_glue*; *forward*;

1598. Here we declare two trivial procedures in order to avoid mutually recursive procedures with parameters.

⟨ Declare procedures needed for expressions 1593 ⟩ +≡
procedure *scan_normal_glue*;
  begin *scan_glue*(*glue_val*);
  end;
procedure *scan_mu_glue*;
  begin *scan_glue*(*mu_val*);
  end;

1599. Parenthesized subexpressions can be inside expressions, and this nesting has a stack. Seven local variables represent the top of the expression stack: p points to pushed-down entries, if any; l specifies the type of expression currently beeing evaluated; e is the expression so far and r is the state of its evaluation; t is the term so far and s is the state of its evaluation; finally n is the numerator for a combined multiplication and division, if any.

  define *expr_node_size* = 4 { number of words in stack entry for subexpressions }
  define *expr_e_field*(#) ≡ *mem*[# + 1].*int* { saved expression so far }
  define *expr_t_field*(#) ≡ *mem*[# + 2].*int* { saved term so far }
  define *expr_n_field*(#) ≡ *mem*[# + 3].*int* { saved numerator }

⟨ Push the expression stack and **goto** *restart* 1599 ⟩ ≡
  begin $q \leftarrow$ *get_node*(*expr_node_size*); *link*(q) $\leftarrow p$; *type*(q) $\leftarrow l$; *subtype*(q) $\leftarrow 4 * s + r$;
  expr_e_field(q) $\leftarrow e$; *expr_t_field*(q) $\leftarrow t$; *expr_n_field*(q) $\leftarrow n$; $p \leftarrow q$; $l \leftarrow o$; **goto** *restart*;
  end

This code is used in section 1596.

1600. ⟨ Pop the expression stack and **goto** *found* 1600 ⟩ ≡
  begin $f \leftarrow e$; $q \leftarrow p$; $e \leftarrow$ *expr_e_field*(q); $t \leftarrow$ *expr_t_field*(q); $n \leftarrow$ *expr_n_field*(q); $s \leftarrow$ *subtype*(q) **div** 4;
  $r \leftarrow$ *subtype*(q) **mod** 4; $l \leftarrow$ *type*(q); $p \leftarrow$ *link*(q); *free_node*(q, *expr_node_size*); **goto** *found*;
  end

This code is used in section 1594.

1601. We want to make sure that each term and (intermediate) result is in the proper range. Integer values must not exceed *infinity* ($2^{31}-1$) in absolute value, dimensions must not exceed *max_dimen* ($2^{30}-1$). We avoid the absolute value of an integer, because this might fail for the value -2^{31} using 32-bit arithmetic.

```
define num_error(#) ≡   { clear a number or dimension and set arith_error }
        begin arith_error ← true; # ← 0;
        end
define glue_error(#) ≡   { clear a glue spec and set arith_error }
        begin arith_error ← true; delete_glue_ref(#); # ← new_spec(zero_glue);
        end
```

```
⟨ Make sure that f is in the proper range 1601 ⟩ ≡
  if (l = int_val) ∨ (s > expr_sub) then
    begin if (f > infinity) ∨ (f < −infinity) then num_error(f);
    end
  else if l = dimen_val then
      begin if abs(f) > max_dimen then num_error(f);
      end
    else begin if (abs(width(f)) > max_dimen) ∨ (abs(stretch(f)) > max_dimen) ∨
            (abs(shrink(f)) > max_dimen) then glue_error(f);
      end
```

This code is used in section 1594.

1602. Applying the factor f to the partial term t (with the operator s) is delayed until the next operator o has been scanned. Here we handle the first factor of a partial term. A glue spec has to be copied unless the next operator is a right parenthesis; this allows us later on to simply modify the glue components.

```
define normalize_glue(#) ≡
          if stretch(#) = 0 then stretch_order(#) ← normal;
        if shrink(#) = 0 then shrink_order(#) ← normal
```

```
⟨ Cases for evaluation of the current term 1602 ⟩ ≡
expr_none: if (l ≥ glue_val) ∧ (o ≠ expr_none) then
    begin t ← new_spec(f); delete_glue_ref(f); normalize_glue(t);
    end
  else t ← f;
```

See also sections 1606, 1607, and 1609.

This code is used in section 1594.

1603. When a term t has been completed it is copied to, added to, or subtracted from the expression e.

```
define expr_add_sub(#) ≡ add_or_sub(#, r = expr_sub)
define expr_a(#) ≡ expr_add_sub(#, max_dimen)
```

```
⟨ Evaluate the current expression 1603 ⟩ ≡
  begin s ← expr_none;
  if r = expr_none then e ← t
  else if l = int_val then e ← expr_add_sub(e, t, infinity)
    else if l = dimen_val then e ← expr_a(e, t)
      else ⟨ Compute the sum or difference of two glue specs 1605 ⟩;
  r ← o;
  end
```

This code is used in section 1594.

1604. The function *add_or_sub*(*x*, *y*, *max_answer*, *negative*) computes the sum (for *negative* = *false*) or difference (for *negative* = *true*) of *x* and *y*, provided the absolute value of the result does not exceed *max_answer*.

```
⟨ Declare subprocedures for scan_expr 1604 ⟩ ≡
function add_or_sub(x, y, max_answer : integer; negative : boolean): integer;
  var a: integer;  { the answer }
  begin if negative then negate(y);
  if x ≥ 0 then
    if y ≤ max_answer − x then a ← x + y else num_error(a)
  else if y ≥ −max_answer − x then a ← x + y else num_error(a);
  add_or_sub ← a;
  end;
```

See also sections 1608 and 1610.

This code is used in section 1593.

1605. We know that *stretch_order*(*e*) > *normal* implies *stretch*(*e*) ≠ 0 and *shrink_order*(*e*) > *normal* implies *shrink*(*e*) ≠ 0.

```
⟨ Compute the sum or difference of two glue specs 1605 ⟩ ≡
  begin width(e) ← expr_a(width(e), width(t));
  if stretch_order(e) = stretch_order(t) then stretch(e) ← expr_a(stretch(e), stretch(t))
  else if (stretch_order(e) < stretch_order(t)) ∧ (stretch(t) ≠ 0) then
      begin stretch(e) ← stretch(t); stretch_order(e) ← stretch_order(t);
      end;
  if shrink_order(e) = shrink_order(t) then shrink(e) ← expr_a(shrink(e), shrink(t))
  else if (shrink_order(e) < shrink_order(t)) ∧ (shrink(t) ≠ 0) then
      begin shrink(e) ← shrink(t); shrink_order(e) ← shrink_order(t);
      end;
  delete_glue_ref(t); normalize_glue(e);
  end
```

This code is used in section 1603.

1606. If a multiplication is followed by a division, the two operations are combined into a 'scaling' operation. Otherwise the term *t* is multiplied by the factor *f*.

```
  define expr_m(#) ≡ # ← nx_plus_y(#, f, 0)
⟨ Cases for evaluation of the current term 1602 ⟩ +≡
expr_mult: if o = expr_div then
    begin n ← f; o ← expr_scale;
    end
  else if l = int_val then t ← mult_integers(t, f)
    else if l = dimen_val then expr_m(t)
      else begin expr_m(width(t)); expr_m(stretch(t)); expr_m(shrink(t));
        end;
```

1607. Here we divide the term *t* by the factor *f*.

```
  define expr_d(#) ≡ # ← quotient(#, f)
⟨ Cases for evaluation of the current term 1602 ⟩ +≡
expr_div: if l < glue_val then expr_d(t)
  else begin expr_d(width(t)); expr_d(stretch(t)); expr_d(shrink(t));
    end;
```

1608. The function *quotient*(n,d) computes the rounded quotient $q=\lfloor n/d+\frac{1}{2}\rfloor$, when n and d are positive.

```
⟨ Declare subprocedures for scan_expr 1604 ⟩ +≡
function quotient(n,d : integer): integer;
  var negative: boolean; { should the answer be negated? }
    a: integer; { the answer }
  begin if d = 0 then num_error(a)
  else begin if d > 0 then negative ← false
    else begin negate(d); negative ← true;
      end;
    if n < 0 then
      begin negate(n); negative ← ¬negative;
      end;
    a ← n div d; n ← n − a ∗ d; d ← n − d; { avoid certain compiler optimizations! }
    if d + n ≥ 0 then incr(a);
    if negative then negate(a);
    end;
  quotient ← a;
  end;
```

1609. Here the term t is multiplied by the quotient n/f.

```
  define expr_s(#) ≡ # ← fract(#,n,f,max_dimen)
⟨ Cases for evaluation of the current term 1602 ⟩ +≡
expr_scale: if l = int_val then t ← fract(t,n,f,infinity)
  else if l = dimen_val then expr_s(t)
    else begin expr_s(width(t)); expr_s(stretch(t)); expr_s(shrink(t));
      end;
```

1610. Finally, the function *fract*(x, n, d, max_answer) computes the integer $q = \lfloor xn/d + \frac{1}{2} \rfloor$, when x, n, and d are positive and the result does not exceed *max_answer*. We can't use floating point arithmetic since the routine must produce identical results in all cases; and it would be too dangerous to multiply by n and then divide by d, in separate operations, since overflow might well occur. Hence this subroutine simulates double precision arithmetic, somewhat analogous to METAFONT's *make_fraction* and *take_fraction* routines.

```
define too_big = 88  { go here when the result is too big }
⟨ Declare subprocedures for scan_expr 1604 ⟩ +≡
function fract(x, n, d, max_answer : integer): integer;
  label found, found1, too_big, done;
  var negative: boolean;  { should the answer be negated? }
    a: integer;  { the answer }
    f: integer;  { a proper fraction }
    h: integer;  { smallest integer such that 2 * h ≥ d }
    r: integer;  { intermediate remainder }
    t: integer;  { temp variable }
  begin if d = 0 then goto too_big;
  a ← 0;
  if d > 0 then negative ← false
  else begin negate(d); negative ← true;
    end;
  if x < 0 then
    begin negate(x); negative ← ¬negative;
    end
  else if x = 0 then goto done;
  if n < 0 then
    begin negate(n); negative ← ¬negative;
    end;
  t ← n div d;
  if t > max_answer div x then goto too_big;
  a ← t * x; n ← n − t * d;
  if n = 0 then goto found;
  t ← x div d;
  if t > (max_answer − a) div n then goto too_big;
  a ← a + t * n; x ← x − t * d;
  if x = 0 then goto found;
  if x < n then
    begin t ← x; x ← n; n ← t;
    end;  { now 0 < n ≤ x < d }
  ⟨ Compute f = ⌊xn/d + 1/2⌋ 1611 ⟩
  if f > (max_answer − a) then goto too_big;
  a ← a + f;
found: if negative then negate(a);
  goto done;
too_big: num_error(a);
done: fract ← a;
  end;
```

1611. The loop here preserves the following invariant relations between f, x, n, and r: (i) $f + \lfloor (xn + (r + d))/d \rfloor = \lfloor x_0 n_0/d + \frac{1}{2} \rfloor$; (ii) $-d \le r < 0 < n \le x < d$, where x_0, n_0 are the original values of x and n.

Notice that the computation specifies $(x - d) + x$ instead of $(x + x) - d$, because the latter could overflow.

```
⟨ Compute f = ⌊xn/d + 1/2⌋ 1611 ⟩ ≡
  f ← 0; r ← (d div 2) − d; h ← −r;
  loop begin if odd(n) then
      begin r ← r + x;
      if r ≥ 0 then
        begin r ← r − d; incr(f);
        end;
      end;
    n ← n div 2;
    if n = 0 then goto found1;
    if x < h then x ← x + x
    else begin t ← x − d; x ← t + x; f ← f + n;
      if x < n then
        begin if x = 0 then goto found1;
        t ← x; x ← n; n ← t;
        end;
      end;
    end;
found1:
```

This code is used in section 1610.

1612. The `\gluestretch`, `\glueshrink`, `\gluestretchorder`, and `\glueshrinkorder` commands return the stretch and shrink components and their orders of "infinity" of a glue specification.

```
define glue_stretch_order_code = eTeX_int + 6   { code for \gluestretchorder }
define glue_shrink_order_code = eTeX_int + 7    { code for \glueshrinkorder }
define glue_stretch_code = eTeX_dim + 7         { code for \gluestretch }
define glue_shrink_code = eTeX_dim + 8          { code for \glueshrink }
```

```
⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
  primitive("gluestretchorder", last_item, glue_stretch_order_code);
  primitive("glueshrinkorder", last_item, glue_shrink_order_code);
  primitive("gluestretch", last_item, glue_stretch_code);
  primitive("glueshrink", last_item, glue_shrink_code);
```

1613.

```
⟨ Cases of last_item for print_cmd_chr 1453 ⟩ +≡
glue_stretch_order_code: print_esc("gluestretchorder");
glue_shrink_order_code: print_esc("glueshrinkorder");
glue_stretch_code: print_esc("gluestretch");
glue_shrink_code: print_esc("glueshrink");
```

1614.

```
⟨ Cases for fetching an integer value 1454 ⟩ +≡
glue_stretch_order_code, glue_shrink_order_code: begin scan_normal_glue; q ← cur_val;
  if m = glue_stretch_order_code then cur_val ← stretch_order(q)
  else cur_val ← shrink_order(q);
  delete_glue_ref(q);
  end;
```

1615. ⟨ Cases for fetching a dimension value 1458 ⟩ +≡
glue_stretch_code, *glue_shrink_code*: **begin** *scan_normal_glue*; $q \leftarrow cur_val$;
if $m = glue_stretch_code$ **then** $cur_val \leftarrow stretch(q)$
else $cur_val \leftarrow shrink(q)$;
delete_glue_ref(*q*);
end;

1616. The `\mutoglue` and `\gluetomu` commands convert "math" glue into normal glue and vice versa; they allow to manipulate math glue with `\gluestretch` etc.

define *mu_to_glue_code* = *eTeX_glue* { code for `\mutoglue` }
define *glue_to_mu_code* = *eTeX_mu* { code for `\gluetomu` }

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive(`"mutoglue"`, *last_item*, *mu_to_glue_code*); *primitive*(`"gluetomu"`, *last_item*, *glue_to_mu_code*);

1617. ⟨ Cases of *last_item* for *print_cmd_chr* 1453 ⟩ +≡
mu_to_glue_code: *print_esc*(`"mutoglue"`);
glue_to_mu_code: *print_esc*(`"gluetomu"`);

1618. ⟨ Cases for fetching a glue value 1618 ⟩ ≡
mu_to_glue_code: *scan_mu_glue*;
This code is used in section 1591.

1619. ⟨ Cases for fetching a mu value 1619 ⟩ ≡
glue_to_mu_code: *scan_normal_glue*;
This code is used in section 1591.

1620. ε-TEX (in extended mode) supports 32768 (i.e., 2^{15}) count, dimen, skip, muskip, box, and token registers. As in TEX the first 256 registers of each kind are realized as arrays in the table of equivalents; the additional registers are realized as tree structures built from variable-size nodes with individual registers existing only when needed. Default values are used for nonexistent registers: zero for count and dimen values, *zero_glue* for glue (skip and muskip) values, void for boxes, and *null* for token lists (and current marks discussed below).

Similarly there are 32768 mark classes; the command `\marks`*n* creates a mark node for a given mark class $0 \le n \le 32767$ (where `\marks0` is synonymous to `\mark`). The page builder (actually the *fire_up* routine) and the *vsplit* routine maintain the current values of *top_mark*, *first_mark*, *bot_mark*, *split_first_mark*, and *split_bot_mark* for each mark class. They are accessed as `\topmarks`*n* etc., and `\topmarks0` is again synonymous to `\topmark`. As in TEX the five current marks for mark class zero are realized as *cur_mark* array. The additional current marks are again realized as tree structure with individual mark classes existing only when needed.

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive(`"marks"`, *mark*, *marks_code*);
primitive(`"topmarks"`, *top_bot_mark*, *top_mark_code* + *marks_code*);
primitive(`"firstmarks"`, *top_bot_mark*, *first_mark_code* + *marks_code*);
primitive(`"botmarks"`, *top_bot_mark*, *bot_mark_code* + *marks_code*);
primitive(`"splitfirstmarks"`, *top_bot_mark*, *split_first_mark_code* + *marks_code*);
primitive(`"splitbotmarks"`, *top_bot_mark*, *split_bot_mark_code* + *marks_code*);

1621. The *scan_register_num* procedure scans a register number that must not exceed 255 in compatibility mode resp. 32767 in extended mode.

⟨ Declare ε-TEX procedures for expanding 1563 ⟩ +≡
procedure *scan_register_num*; *forward*;

1622. ⟨ Declare procedures that scan restricted classes of integers 467 ⟩ +≡
procedure *scan_register_num*;
begin *scan_int*;
if (*cur_val* < 0) ∨ (*cur_val* > *max_reg_num*) **then**
begin *print_err*("Bad␣register␣code");
help2(*max_reg_help_line*)("I␣changed␣this␣one␣to␣zero."); *int_error*(*cur_val*); *cur_val* ← 0;
end;
end;

1623. ⟨ Initialize variables for ε-TEX compatibility mode 1623 ⟩ ≡
max_reg_num ← 255; *max_reg_help_line* ← "A␣register␣number␣must␣be␣between␣0␣and␣255.";
This code is used in sections 1463 and 1465.

1624. ⟨ Initialize variables for ε-TEX extended mode 1624 ⟩ ≡
max_reg_num ← 32767; *max_reg_help_line* ← "A␣register␣number␣must␣be␣between␣0␣and␣32767.";
This code is used in sections 1451 and 1465.

1625. ⟨ Global variables 13 ⟩ +≡
max_reg_num: *halfword*; { largest allowed register number }
max_reg_help_line: *str_number*; { first line of help message }

1626. There are eight almost identical doubly linked trees, one for the sparse array of the up to 32512 additional registers of each kind, one for inter-character token lists at specified class transitions, and one for the sparse array of the up to 32767 additional mark classes. The root of each such tree, if it exists, is an index node containing 64 pointers to subtrees for 64^4 consecutive array elements. Similar index nodes are the starting points for all nonempty subtrees for 64^3, 64^2, and 64 consecutive array elements. These four levels of index nodes are followed by a fifth level with nodes for the individual array elements.

Each index node is 33 words long. The pointers to the 64 possible subtrees or nodes are kept in the *info* and *link* fields of the last 32 words. (It would be both elegant and efficient to declare them as array, unfortunately Pascal doesn't allow this.)

The fields in the first word of each index node and in the nodes for the array elements are closely related. The *link* field points to the next lower index node and the *sa_index* field contains four bits (one hexadecimal digit) of the register number or mark class. For the lowest index node the *link* field is *null* and the *sa_index* field indicates the type of quantity (*int_val*, *dimen_val*, *glue_val*, *mu_val*, *box_val*, *tok_val*, *inter_char_val* or *mark_val*). The *sa_used* field in the index nodes counts how many of the 64 pointers are non-null.

The *sa_index* field in the nodes for array elements contains the six bits plus 64 times the type. Therefore such a node represents a count or dimen register if and only if $sa_index < dimen_val_limit$; it represents a skip or muskip register if and only if $dimen_val_limit \le sa_index < mu_val_limit$; it represents a box register if and only if $mu_val_limit \le sa_index < box_val_limit$; it represents a token list register if and only if $box_val_limit \le sa_index < tok_val_limit$; finally it represents a mark class if and only if $tok_val_limit \le sa_index$.

The *new_index* procedure creates an index node (returned in *cur_ptr*) having given contents of the *sa_index* and *link* fields.

```
define box_val ≡ 4  { the additional box registers }
define mark_val = 7  { the additional mark classes }
define dimen_val_limit = "80  { 2^6 · (dimen_val + 1) }
define mu_val_limit = "100  { 2^6 · (mu_val + 1) }
define box_val_limit = "140  { 2^6 · (box_val + 1) }
define tok_val_limit = "180  { 2^6 · (tok_val + 1) }
define index_node_size = 33  { size of an index node }
define sa_index ≡ type  { a four-bit address or a type or both }
define sa_used ≡ subtype  { count of non-null pointers }
⟨ Declare ε-TEX procedures for expanding 1563 ⟩ +≡
procedure new_index(i : quarterword; q : pointer);
  var k: small_number;  { loop index }
  begin cur_ptr ← get_node(index_node_size); sa_index(cur_ptr) ← i; sa_used(cur_ptr) ← 0;
  link(cur_ptr) ← q;
  for k ← 1 to index_node_size − 1 do  { clear all 64 pointers }
    mem[cur_ptr + k] ← sa_null;
  end;
```

1627. The roots of the eight trees for the additional registers and mark classes are kept in the *sa_root* array. The first seven locations must be dumped and undumped; the last one is also known as *sa_mark*.

```
define sa_mark ≡ sa_root[mark_val]  { root for mark classes }
⟨ Global variables 13 ⟩ +≡
sa_root: array [int_val .. mark_val] of pointer;  { roots of sparse arrays }
cur_ptr: pointer;  { value returned by new_index and find_sa_element }
sa_null: memory_word;  { two null pointers }
```

1628. ⟨ Set initial values of key variables 23 ⟩ +≡

```
sa_mark ← null; sa_null.hh.lh ← null; sa_null.hh.rh ← null;
```

1629. ⟨ Initialize table entries (done by `INITEX` only) 189 ⟩ +≡
for $i \leftarrow$ *int_val* **to** *inter_char_val* **do** *sa_root*[i] $\leftarrow$ *null*;

1630. Given a type t and a twenty-four-bit number n, the *find_sa_element* procedure returns (in *cur_ptr*) a pointer to the node for the corresponding array element, or *null* when no such element exists. The third parameter w is set *true* if the element must exist, e.g., because it is about to be modified. The procedure has two main branches: one follows the existing tree structure, the other (only used when w is *true*) creates the missing nodes.

We use macros to extract the six-bit pieces from a twenty-four-bit register number or mark class and to fetch or store one of the 64 pointers from an index node. (Note that the *hex_dig* macros are mis-named since the conversion from 4-bit to 6-bit fields for XETEX!)

```
define if_cur_ptr_is_null_then_return_or_goto(#) ≡   { some tree element is missing }
        begin if cur_ptr = null then
          if w then goto # else return;
        end

define hex_dig1(#) ≡ # div "40000   { the fourth lowest 6-bit field }
define hex_dig2(#) ≡ (# div "1000) mod "40   { the third lowest 6-bit field }
define hex_dig3(#) ≡ (# div "40) mod "40   { the second lowest 6-bit field }
define hex_dig4(#) ≡ # mod "40   { the lowest 6-bit field }

define get_sa_ptr ≡
          if odd(i) then cur_ptr ← link(q + (i div 2) + 1)
          else cur_ptr ← info(q + (i div 2) + 1)
                { set cur_ptr to the pointer indexed by i from index node q }
define put_sa_ptr(#) ≡
          if odd(i) then link(q + (i div 2) + 1) ← #
          else info(q + (i div 2) + 1) ← #   { store the pointer indexed by i in index node q }
define add_sa_ptr ≡
          begin put_sa_ptr(cur_ptr); incr(sa_used(q));
          end   { add cur_ptr as the pointer indexed by i in index node q }
define delete_sa_ptr ≡
          begin put_sa_ptr(null); decr(sa_used(q));
          end   { delete the pointer indexed by i in index node q }

⟨ Declare ε-TEX procedures for expanding 1563 ⟩ +≡
procedure find_sa_element(t : small_number; n : halfword; w : boolean);
        { sets cur_val to sparse array element location or null }
  label not_found, not_found1, not_found2, not_found3, not_found4, exit;
  var q: pointer;   { for list manipulations }
    i: small_number;   { a six bit index }
  begin cur_ptr ← sa_root[t]; if_cur_ptr_is_null_then_return_or_goto(not_found);
  q ← cur_ptr; i ← hex_dig1(n); get_sa_ptr; if_cur_ptr_is_null_then_return_or_goto(not_found1);
  q ← cur_ptr; i ← hex_dig2(n); get_sa_ptr; if_cur_ptr_is_null_then_return_or_goto(not_found2);
  q ← cur_ptr; i ← hex_dig3(n); get_sa_ptr; if_cur_ptr_is_null_then_return_or_goto(not_found3);
  q ← cur_ptr; i ← hex_dig4(n); get_sa_ptr;
  if (cur_ptr = null) ∧ w then goto not_found4;
  return;
not_found: new_index(t, null);   { create first level index node }
  sa_root[t] ← cur_ptr; q ← cur_ptr; i ← hex_dig1(n);
not_found1: new_index(i, q);   { create second level index node }
  add_sa_ptr; q ← cur_ptr; i ← hex_dig2(n);
not_found2: new_index(i, q);   { create third level index node }
  add_sa_ptr; q ← cur_ptr; i ← hex_dig3(n);
not_found3: new_index(i, q);   { create fourth level index node }
  add_sa_ptr; q ← cur_ptr; i ← hex_dig4(n);
not_found4: ⟨ Create a new array element of type t with index i 1631 ⟩;
```

```
  link(cur_ptr) ← q; add_sa_ptr;
exit: end;
```

1631. The array elements for registers are subject to grouping and have an *sa_lev* field (quite analogous to *eq_level*) instead of *sa_used*. Since saved values as well as shorthand definitions (created by e.g., `\countdef`) refer to the location of the respective array element, we need a reference count that is kept in the *sa_ref* field. An array element can be deleted (together with all references to it) when its *sa_ref* value is *null* and its value is the default value.

Skip, muskip, box, and token registers use two word nodes, their values are stored in the *sa_ptr* field. Count and dimen registers use three word nodes, their values are stored in the *sa_int* resp. *sa_dim* field in the third word; the *sa_ptr* field is used under the name *sa_num* to store the register number. Mark classes use four word nodes. The last three words contain the five types of current marks

```
define sa_lev ≡ sa_used { grouping level for the current value }
define pointer_node_size = 2 { size of an element with a pointer value }
define sa_type(#) ≡ (sa_index(#) div 64) { type part of combined type/index }
define sa_ref(#) ≡ info(# + 1) { reference count of a sparse array element }
define sa_ptr(#) ≡ link(# + 1) { a pointer value }
define word_node_size = 3 { size of an element with a word value }
define sa_num ≡ sa_ptr { the register number }
define sa_int(#) ≡ mem[# + 2].int { an integer }
define sa_dim(#) ≡ mem[# + 2].sc { a dimension (a somewhat esotheric distinction) }
define mark_class_node_size = 4 { size of an element for a mark class }
define fetch_box(#) ≡ { fetch box(cur_val) }
    if cur_val < 256 then # ← box(cur_val)
    else begin find_sa_element(box_val, cur_val, false);
      if cur_ptr = null then # ← null else # ← sa_ptr(cur_ptr);
      end
```

⟨ Create a new array element of type t with index i 1631 ⟩ ≡

```
if t = mark_val then { a mark class }
  begin cur_ptr ← get_node(mark_class_node_size); mem[cur_ptr + 1] ← sa_null;
  mem[cur_ptr + 2] ← sa_null; mem[cur_ptr + 3] ← sa_null;
  end
else begin if t ≤ dimen_val then { a count or dimen register }
    begin cur_ptr ← get_node(word_node_size); sa_int(cur_ptr) ← 0; sa_num(cur_ptr) ← n;
    end
  else begin cur_ptr ← get_node(pointer_node_size);
    if t ≤ mu_val then { a skip or muskip register }
      begin sa_ptr(cur_ptr) ← zero_glue; add_glue_ref(zero_glue);
      end
    else sa_ptr(cur_ptr) ← null; { a box or token list register }
    end;
  sa_ref(cur_ptr) ← null; { all registers have a reference count }
  end;
sa_index(cur_ptr) ← 64 * t + i; sa_lev(cur_ptr) ← level_one
```

This code is used in section 1630.

1632. The *delete_sa_ref* procedure is called when a pointer to an array element representing a register is being removed; this means that the reference count should be decreased by one. If the reduced reference count is *null* and the register has been (globally) assigned its default value the array element should disappear, possibly together with some index nodes. This procedure will never be used for mark class nodes.

```
define add_sa_ref(#) ≡ incr(sa_ref(#))  { increase reference count }
define change_box(#) ≡  { change box(cur_val), the eq_level stays the same }
    if cur_val < 256 then box(cur_val) ← # else set_sa_box(#)
define set_sa_box(#) ≡
    begin find_sa_element(box_val, cur_val, false);
    if cur_ptr ≠ null then
      begin sa_ptr(cur_ptr) ← #; add_sa_ref(cur_ptr); delete_sa_ref(cur_ptr);
      end;
    end
```

⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡

```
procedure delete_sa_ref(q : pointer);  { reduce reference count }
  label exit;
  var p: pointer;  { for list manipulations }
    i: small_number;  { a four bit index }
    s: small_number;  { size of a node }
  begin decr(sa_ref(q));
  if sa_ref(q) ≠ null then return;
  if sa_index(q) < dimen_val_limit then
    if sa_int(q) = 0 then s ← word_node_size
    else return
  else begin if sa_index(q) < mu_val_limit then
      if sa_ptr(q) = zero_glue then delete_glue_ref(zero_glue)
      else return
    else if sa_ptr(q) ≠ null then return;
    s ← pointer_node_size;
    end;
  repeat i ← hex_dig4(sa_index(q)); p ← q; q ← link(p); free_node(p, s);
    if q = null then  { the whole tree has been freed }
      begin sa_root[i] ← null; return;
      end;
    delete_sa_ptr; s ← index_node_size;  { node q is an index node }
  until sa_used(q) > 0;
exit: end;
```

1633. The *print_sa_num* procedure prints the register number corresponding to an array element.

⟨ Basic printing procedures 57 ⟩ +≡

```
procedure print_sa_num(q : pointer);  { print register number }
  var n: halfword;  { the register number }
  begin if sa_index(q) < dimen_val_limit then n ← sa_num(q)  { the easy case }
  else begin n ← hex_dig4(sa_index(q)); q ← link(q); n ← n + 64 * sa_index(q); q ← link(q);
    n ← n + 64 * 64 * (sa_index(q) + 64 * sa_index(link(q)));
    end;
  print_int(n);
  end;
```

1634. Here is a procedure that displays the contents of an array element symbolically. It is used under similar circumstances as is *restore_trace* (together with *show_eqtb*) for the quantities kept in the *eqtb* array.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
  stat procedure show_sa(p : pointer; s : str_number);
  var t: small_number;  { the type of element }
  begin begin_diagnostic; print_char("{"); print(s); print_char("␣");
  if p = null then print_char("?")  { this can't happen }
  else begin t ← sa_type(p);
    if t < box_val then print_cmd_chr(register, p)
    else if t = box_val then
        begin print_esc("box"); print_sa_num(p);
        end
      else if t = tok_val then print_cmd_chr(toks_register, p)
        else print_char("?");  { this can't happen either }
    print_char("=");
    if t = int_val then print_int(sa_int(p))
    else if t = dimen_val then
        begin print_scaled(sa_dim(p)); print("pt");
        end
      else begin p ← sa_ptr(p);
        if t = glue_val then print_spec(p, "pt")
        else if t = mu_val then print_spec(p, "mu")
          else if t = box_val then
              if p = null then print("void")
              else begin depth_threshold ← 0; breadth_max ← 1; show_node_list(p);
                end
            else if t = tok_val then
                begin if p ≠ null then show_token_list(link(p), null, 32);
                end
              else print_char("?");  { this can't happen either }
        end;
    end;
  print_char("}"); end_diagnostic(false);
  end;
  tats
```

1635. Here we compute the pointer to the current mark of type t and mark class *cur_val*.

```
⟨ Compute the mark pointer for mark type t and class cur_val 1635 ⟩ ≡
  begin find_sa_element(mark_val, cur_val, false);
  if cur_ptr ≠ null then
    if odd(t) then cur_ptr ← link(cur_ptr + (t div 2) + 1)
    else cur_ptr ← info(cur_ptr + (t div 2) + 1);
  end
```

This code is used in section 420.

1636. The current marks for all mark classes are maintained by the *vsplit* and *fire_up* routines and are finally destroyed (for INITEX only) by the *final_cleanup* routine. Apart from updating the current marks when mark nodes are encountered, these routines perform certain actions on all existing mark classes. The recursive *do_marks* procedure walks through the whole tree or a subtree of existing mark class nodes and preforms certain actions indicted by its first parameter a, the action code. The second parameter l indicates the level of recursion (at most four); the third parameter points to a nonempty tree or subtree. The result is *true* if the complete tree or subtree has been deleted.

```
define vsplit_init ≡ 0  { action code for vsplit initialization }
define fire_up_init ≡ 1  { action code for fire_up initialization }
define fire_up_done ≡ 2  { action code for fire_up completion }
define destroy_marks ≡ 3  { action code for final_cleanup }

define sa_top_mark(#) ≡ info(# + 1)  { \topmarksn }
define sa_first_mark(#) ≡ link(# + 1)  { \firstmarksn }
define sa_bot_mark(#) ≡ info(# + 2)  { \botmarksn }
define sa_split_first_mark(#) ≡ link(# + 2)  { \splitfirstmarksn }
define sa_split_bot_mark(#) ≡ info(# + 3)  { \splitbotmarksn }
```

⟨ Declare the function called *do_marks* 1636 ⟩ ≡

```
function do_marks(a, l : small_number; q : pointer): boolean;
  var i: small_number;  { a four bit index }
  begin if l < 4 then  { q is an index node }
    begin for i ← 0 to 15 do
      begin get_sa_ptr;
      if cur_ptr ≠ null then
        if do_marks(a, l + 1, cur_ptr) then delete_sa_ptr;
      end;
    if sa_used(q) = 0 then
      begin free_node(q, index_node_size); q ← null;
      end;
    end
  else  { q is the node for a mark class }
  begin case a of
    ⟨ Cases for do_marks 1637 ⟩
  end;  { there are no other cases }
  if sa_bot_mark(q) = null then
    if sa_split_bot_mark(q) = null then
      begin free_node(q, mark_class_node_size); q ← null;
      end;
  end; do_marks ← (q = null);
  end;
```

This code is used in section 1031.

1637. At the start of the *vsplit* routine the existing *split_fist_mark* and *split_bot_mark* are discarded.

⟨ Cases for *do_marks* 1637 ⟩ ≡

```
vsplit_init: if sa_split_first_mark(q) ≠ null then
    begin delete_token_ref(sa_split_first_mark(q)); sa_split_first_mark(q) ← null;
    delete_token_ref(sa_split_bot_mark(q)); sa_split_bot_mark(q) ← null;
    end;
```

See also sections 1639, 1640, and 1642.

This code is used in section 1636.

1638. We use again the fact that *split_first_mark* = *null* if and only if *split_bot_mark* = *null*.

```
⟨ Update the current marks for vsplit 1638 ⟩ ≡
  begin find_sa_element(mark_val, mark_class(p), true);
  if sa_split_first_mark(cur_ptr) = null then
    begin sa_split_first_mark(cur_ptr) ← mark_ptr(p); add_token_ref(mark_ptr(p));
    end
  else delete_token_ref(sa_split_bot_mark(cur_ptr));
  sa_split_bot_mark(cur_ptr) ← mark_ptr(p); add_token_ref(mark_ptr(p));
  end
```

This code is used in section 1033.

1639. At the start of the *fire_up* routine the old *top_mark* and *first_mark* are discarded, whereas the old *bot_mark* becomes the new *top_mark*. An empty new *top_mark* token list is, however, discarded as well in order that mark class nodes can eventually be released. We use again the fact that *bot_mark* ≠ *null* implies *first_mark* ≠ *null*; it also knows that *bot_mark* = *null* implies *top_mark* = *first_mark* = *null*.

```
⟨ Cases for do_marks 1637 ⟩ +≡
fire_up_init: if sa_bot_mark(q) ≠ null then
    begin if sa_top_mark(q) ≠ null then delete_token_ref(sa_top_mark(q));
    delete_token_ref(sa_first_mark(q)); sa_first_mark(q) ← null;
    if link(sa_bot_mark(q)) = null then { an empty token list }
      begin delete_token_ref(sa_bot_mark(q)); sa_bot_mark(q) ← null;
      end
    else add_token_ref(sa_bot_mark(q));
    sa_top_mark(q) ← sa_bot_mark(q);
    end;
```

1640.

```
⟨ Cases for do_marks 1637 ⟩ +≡
fire_up_done: if (sa_top_mark(q) ≠ null) ∧ (sa_first_mark(q) = null) then
    begin sa_first_mark(q) ← sa_top_mark(q); add_token_ref(sa_top_mark(q));
    end;
```

1641.

```
⟨ Update the current marks for fire_up 1641 ⟩ ≡
  begin find_sa_element(mark_val, mark_class(p), true);
  if sa_first_mark(cur_ptr) = null then
    begin sa_first_mark(cur_ptr) ← mark_ptr(p); add_token_ref(mark_ptr(p));
    end;
  if sa_bot_mark(cur_ptr) ≠ null then delete_token_ref(sa_bot_mark(cur_ptr));
  sa_bot_mark(cur_ptr) ← mark_ptr(p); add_token_ref(mark_ptr(p));
  end
```

This code is used in section 1068.

1642. Here we use the fact that the five current mark pointers in a mark class node occupy the same locations as the the first five pointers of an index node. For systems using a run-time switch to distinguish between VIRTEX and INITEX, the codewords '**init** ... **tini**' surrounding the following piece of code should be removed.

```
⟨ Cases for do_marks 1637 ⟩ +≡
  init destroy_marks: for i ← top_mark_code to split_bot_mark_code do
    begin get_sa_ptr;
    if cur_ptr ≠ null then
      begin delete_token_ref(cur_ptr); put_sa_ptr(null);
      end;
    end;
  tini
```

1643. The command code *register* is used for '\count', '\dimen', etc., as well as for references to sparse array elements defined by '\countdef', etc.

```
⟨ Cases of register for print_cmd_chr 1643 ⟩ ≡
  begin if (chr_code < mem_bot) ∨ (chr_code > lo_mem_stat_max) then cmd ← sa_type(chr_code)
  else begin cmd ← chr_code − mem_bot; chr_code ← null;
    end;
  if cmd = int_val then print_esc("count")
  else if cmd = dimen_val then print_esc("dimen")
    else if cmd = glue_val then print_esc("skip")
      else print_esc("muskip");
  if chr_code ≠ null then print_sa_num(chr_code);
  end
```

This code is used in section 446.

1644. Similarly the command code *toks_register* is used for '\toks' as well as for references to sparse array elements defined by '\toksdef'.

```
⟨ Cases of toks_register for print_cmd_chr 1644 ⟩ ≡
  begin print_esc("toks");
  if chr_code ≠ mem_bot then print_sa_num(chr_code);
  end
```

This code is used in section 296.

1645. When a shorthand definition for an element of one of the sparse arrays is destroyed, we must reduce the reference count.

```
⟨ Cases for eq_destroy 1645 ⟩ ≡
toks_register, register: if (equiv_field(w) < mem_bot) ∨ (equiv_field(w) > lo_mem_stat_max) then
    delete_sa_ref(equiv_field(w));
```

This code is used in section 305.

1646. The task to maintain (change, save, and restore) register values is essentially the same when the register is realized as sparse array element or entry in *eqtb*. The global variable *sa_chain* is the head of a linked list of entries saved at the topmost level *sa_level*; the lists for lowel levels are kept in special save stack entries.

```
⟨ Global variables 13 ⟩ +≡
sa_chain: pointer; { chain of saved sparse array entries }
sa_level: quarterword; { group level for sa_chain }
```

1647. ⟨ Set initial values of key variables 23 ⟩ +≡
sa_chain ← *null*; *sa_level* ← *level_zero*;

1648. The individual saved items are kept in pointer or word nodes similar to those used for the array elements: a word node with value zero is, however, saved as pointer node with the otherwise impossible *sa_index* value *tok_val_limit*.

define *sa_loc* ≡ *sa_ref* { location of saved item }

⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure *sa_save*(*p* : *pointer*); { saves value of *p* }
var *q*: *pointer*; { the new save node }
i: *quarterword*; { index field of node }
begin if *cur_level* ≠ *sa_level* **then**
begin *check_full_save_stack*; *save_type*(*save_ptr*) ← *restore_sa*; *save_level*(*save_ptr*) ← *sa_level*;
save_index(*save_ptr*) ← *sa_chain*; *incr*(*save_ptr*); *sa_chain* ← *null*; *sa_level* ← *cur_level*;
end;
i ← *sa_index*(*p*);
if *i* < *dimen_val_limit* **then**
begin if *sa_int*(*p*) = 0 **then**
begin *q* ← *get_node*(*pointer_node_size*); *i* ← *tok_val_limit*;
end
else begin *q* ← *get_node*(*word_node_size*); *sa_int*(*q*) ← *sa_int*(*p*);
end;
sa_ptr(*q*) ← *null*;
end
else begin *q* ← *get_node*(*pointer_node_size*); *sa_ptr*(*q*) ← *sa_ptr*(*p*);
end;
sa_loc(*q*) ← *p*; *sa_index*(*q*) ← *i*; *sa_lev*(*q*) ← *sa_lev*(*p*); *link*(*q*) ← *sa_chain*; *sa_chain* ← *q*; *add_sa_ref*(*p*);
end;

1649. ⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure *sa_destroy*(*p* : *pointer*); { destroy value of *p* }
begin if *sa_index*(*p*) < *mu_val_limit* **then** *delete_glue_ref*(*sa_ptr*(*p*))
else if *sa_ptr*(*p*) ≠ *null* **then**
if *sa_index*(*p*) < *box_val_limit* **then** *flush_node_list*(*sa_ptr*(*p*))
else *delete_token_ref*(*sa_ptr*(*p*));
end;

1650. The procedure *sa_def* assigns a new value to sparse array elements, and saves the former value if appropriate. This procedure is used only for skip, muskip, box, and token list registers. The counterpart of *sa_def* for count and dimen registers is called *sa_w_def*.

```
define sa_define(#) ≡
        if e then
          if global then gsa_def(#) else sa_def(#)
        else define
define sa_def_box ≡ { assign cur_box to box(cur_val) }
      begin find_sa_element(box_val, cur_val, true);
      if global then gsa_def(cur_ptr, cur_box) else sa_def(cur_ptr, cur_box);
      end
define sa_word_define(#) ≡
        if e then
          if global then gsa_w_def(#) else sa_w_def(#)
        else word_define(#)
```

⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡

```
procedure sa_def(p : pointer; e : halfword); { new data for sparse array elements }
  begin add_sa_ref(p);
  if sa_ptr(p) = e then
    begin stat if tracing_assigns > 0 then show_sa(p, "reassigning");
    tats
    sa_destroy(p);
    end
  else begin stat if tracing_assigns > 0 then show_sa(p, "changing");
    tats
    if sa_lev(p) = cur_level then sa_destroy(p) else sa_save(p);
    sa_lev(p) ← cur_level; sa_ptr(p) ← e;
    stat if tracing_assigns > 0 then show_sa(p, "into");
    tats
    end;
  delete_sa_ref(p);
  end;
procedure sa_w_def(p : pointer; w : integer);
  begin add_sa_ref(p);
  if sa_int(p) = w then
    begin stat if tracing_assigns > 0 then show_sa(p, "reassigning");
    tats
    end
  else begin stat if tracing_assigns > 0 then show_sa(p, "changing");
    tats
    if sa_lev(p) ≠ cur_level then sa_save(p);
    sa_lev(p) ← cur_level; sa_int(p) ← w;
    stat if tracing_assigns > 0 then show_sa(p, "into");
    tats
    end;
  delete_sa_ref(p);
  end;
```

1651. The *sa_def* and *sa_w_def* routines take care of local definitions. Global definitions are done in almost the same way, but there is no need to save old values, and the new value is associated with *level_one*.

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure gsa_def(p : pointer; e : halfword); { global sa_def }
  begin add_sa_ref(p);
  stat if tracing_assigns > 0 then show_sa(p, "globally␣changing");
  tats
  sa_destroy(p); sa_lev(p) ← level_one; sa_ptr(p) ← e;
  stat if tracing_assigns > 0 then show_sa(p, "into");
  tats
  delete_sa_ref(p);
  end;
procedure gsa_w_def(p : pointer; w : integer); { global sa_w_def }
  begin add_sa_ref(p);
  stat if tracing_assigns > 0 then show_sa(p, "globally␣changing");
  tats
  sa_lev(p) ← level_one; sa_int(p) ← w;
  stat if tracing_assigns > 0 then show_sa(p, "into");
  tats
  delete_sa_ref(p);
  end;
```

1652. The *sa_restore* procedure restores the sparse array entries pointed at by *sa_chain*

```
⟨ Declare ε-TEX procedures for tracing and input 314 ⟩ +≡
procedure sa_restore;
  var p: pointer; { sparse array element }
  begin repeat p ← sa_loc(sa_chain);
    if sa_lev(p) = level_one then
      begin if sa_index(p) ≥ dimen_val_limit then sa_destroy(sa_chain);
      stat if tracing_restores > 0 then show_sa(p, "retaining");
      tats
      end
    else begin if sa_index(p) < dimen_val_limit then
        if sa_index(sa_chain) < dimen_val_limit then sa_int(p) ← sa_int(sa_chain)
        else sa_int(p) ← 0
      else begin sa_destroy(p); sa_ptr(p) ← sa_ptr(sa_chain);
        end;
      sa_lev(p) ← sa_lev(sa_chain);
      stat if tracing_restores > 0 then show_sa(p, "restoring");
      tats
      end;
    delete_sa_ref(p); p ← sa_chain; sa_chain ← link(p);
    if sa_index(p) < dimen_val_limit then free_node(p, word_node_size)
    else free_node(p, pointer_node_size);
  until sa_chain = null;
  end;
```

1653. When the value of *last_line_fit* is positive, the last line of a (partial) paragraph is treated in a special way and we need additional fields in the active nodes.

```
define active_node_size_extended = 5  { number of words in extended active nodes }
define active_short(#) ≡ mem[# + 3].sc  { shortfall of this line }
define active_glue(#) ≡ mem[# + 4].sc  { corresponding glue stretch or shrink }
⟨ Global variables 13 ⟩ +≡
last_line_fill: pointer;  { the par_fill_skip glue node of the new paragraph }
do_last_line_fit: boolean;  { special algorithm for last line of paragraph? }
active_node_size: small_number;  { number of words in active nodes }
fill_width: array [0 .. 2] of scaled;  { infinite stretch components of par_fill_skip }
best_pl_short: array [very_loose_fit .. tight_fit] of scaled;  { shortfall corresponding to minimal_demerits }
best_pl_glue: array [very_loose_fit .. tight_fit] of scaled;  { corresponding glue stretch or shrink }
```

1654. The new algorithm for the last line requires that the stretchability of *par_fill_skip* is infinite and the stretchability of *left_skip* plus *right_skip* is finite.

```
⟨ Check for special treatment of last line of paragraph 1654 ⟩ ≡
  do_last_line_fit ← false; active_node_size ← active_node_size_normal;  { just in case }
  if last_line_fit > 0 then
    begin q ← glue_ptr(last_line_fill);
    if (stretch(q) > 0) ∧ (stretch_order(q) > normal) then
      if (background[3] = 0) ∧ (background[4] = 0) ∧ (background[5] = 0) then
        begin do_last_line_fit ← true; active_node_size ← active_node_size_extended; fill_width[0] ← 0;
        fill_width[1] ← 0; fill_width[2] ← 0; fill_width[stretch_order(q) − 1] ← stretch(q);
        end;
    end
```

This code is used in section 875.

1655.
```
⟨ Other local variables for try_break 878 ⟩ +≡
g: scaled;  { glue stretch or shrink of test line, adjustment for last line }
```

1656. Here we initialize the additional fields of the first active node representing the beginning of the paragraph.

```
⟨ Initialize additional fields of the first active node 1656 ⟩ ≡
  begin active_short(q) ← 0; active_glue(q) ← 0;
  end
```

This code is used in section 912.

1657. Here we compute the adjustment g and badness b for a line from r to the end of the paragraph. When any of the criteria for adjustment is violated we fall through to the normal algorithm.

The last line must be too short, and have infinite stretch entirely due to *par_fill_skip*.

```
⟨ Perform computations for last line and goto found 1657 ⟩ ≡
  begin if (active_short(r) = 0) ∨ (active_glue(r) ≤ 0) then goto not_found;
        { previous line was neither stretched nor shrunk, or was infinitely bad }
  if (cur_active_width[3] ≠ fill_width[0]) ∨ (cur_active_width[4] ≠ fill_width[1]) ∨
        (cur_active_width[5] ≠ fill_width[2]) then goto not_found;
        { infinite stretch of this line not entirely due to par_fill_skip }
  if active_short(r) > 0 then g ← cur_active_width[2]
  else g ← cur_active_width[6];
  if g ≤ 0 then goto not_found;  { no finite stretch resp. no shrink }
  arith_error ← false; g ← fract(g, active_short(r), active_glue(r), max_dimen);
  if last_line_fit < 1000 then g ← fract(g, last_line_fit, 1000, max_dimen);
  if arith_error then
    if active_short(r) > 0 then g ← max_dimen else g ← −max_dimen;
  if g > 0 then ⟨ Set the value of b to the badness of the last line for stretching, compute the corresponding
        fit_class, and goto found 1658 ⟩
  else if g < 0 then ⟨ Set the value of b to the badness of the last line for shrinking, compute the
        corresponding fit_class, and goto found 1659 ⟩;
not_found: end
```

This code is used in section 900.

1658. These badness computations are rather similar to those of the standard algorithm, with the adjustment amount g replacing the *shortfall*.

```
⟨ Set the value of b to the badness of the last line for stretching, compute the corresponding fit_class, and
      goto found 1658 ⟩ ≡
  begin if g > shortfall then g ← shortfall;
  if g > 7230584 then
    if cur_active_width[2] < 1663497 then
      begin b ← inf_bad; fit_class ← very_loose_fit; goto found;
      end;
  b ← badness(g, cur_active_width[2]);
  if b > 12 then
    if b > 99 then fit_class ← very_loose_fit
    else fit_class ← loose_fit
  else fit_class ← decent_fit;
  goto found;
  end
```

This code is used in section 1657.

1659. ⟨ Set the value of b to the badness of the last line for shrinking, compute the corresponding *fit_class*, and **goto** *found* 1659 ⟩ ≡

```
  begin if −g > cur_active_width[6] then g ← −cur_active_width[6];
  b ← badness(−g, cur_active_width[6]);
  if b > 12 then fit_class ← tight_fit else fit_class ← decent_fit;
  goto found;
  end
```

This code is used in section 1657.

1660. Vanishing values of *shortfall* and g indicate that the last line is not adjusted.

⟨ Adjust the additional data for last line 1660 ⟩ ≡
```
begin if cur_p = null then shortfall ← 0;
if shortfall > 0 then g ← cur_active_width[2]
else if shortfall < 0 then g ← cur_active_width[6]
  else g ← 0;
end
```
This code is used in section 899.

1661. For each feasible break we record the shortfall and glue stretch or shrink (or adjustment).

⟨ Store additional data for this feasible break 1661 ⟩ ≡
```
begin best_pl_short[fit_class] ← shortfall; best_pl_glue[fit_class] ← g;
end
```
This code is used in section 903.

1662. Here we save these data in the active node representing a potential line break.

⟨ Store additional data in the new active node 1662 ⟩ ≡
```
begin active_short(q) ← best_pl_short[fit_class]; active_glue(q) ← best_pl_glue[fit_class];
end
```
This code is used in section 893.

1663. ⟨ Print additional data in the new active node 1663 ⟩ ≡
```
begin print("␣s="); print_scaled(active_short(q));
if cur_p = null then print("␣a=") else print("␣g=");
print_scaled(active_glue(q));
end
```
This code is used in section 894.

1664. Here we either reset *do_last_line_fit* or adjust the *par_fill_skip* glue.

⟨ Adjust the final line of the paragraph 1664 ⟩ ≡
```
if active_short(best_bet) = 0 then do_last_line_fit ← false
else begin q ← new_spec(glue_ptr(last_line_fill)); delete_glue_ref(glue_ptr(last_line_fill));
  width(q) ← width(q) + active_short(best_bet) − active_glue(best_bet); stretch(q) ← 0;
  glue_ptr(last_line_fill) ← q;
  end
```
This code is used in section 911.

1665. When reading `\patterns` while `\savinghyphcodes` is positive the current *lc_code* values are stored together with the hyphenation patterns for the current language. They will later be used instead of the *lc_code* values for hyphenation purposes.

The *lc_code* values are stored in the linked trie analogous to patterns p_1 of length 1, with *hyph_root* = *trie_r*[0] replacing *trie_root* and *lc_code*(*p_1*) replacing the *trie_op* code. This allows to compress and pack them together with the patterns with minimal changes to the existing code.

```
define hyph_root ≡ trie_r[0]  { root of the linked trie for hyph_codes }
```

⟨ Initialize table entries (done by `INITEX` only) 189 ⟩ +≡
```
XeTeX_hyphenatable_length ← 63;  { for backward compatibility with standard TeX by default }
```

1666. ⟨ Store hyphenation codes for current language 1666 ⟩ ≡
```
begin c ← cur_lang; first_child ← false; p ← 0;
repeat q ← p; p ← trie_r[q];
until (p = 0) ∨ (c ≤ so(trie_c[p]));
if (p = 0) ∨ (c < so(trie_c[p])) then
  ⟨ Insert a new trie node between q and p, and make p point to it 1018 ⟩;
q ← p; { now node q represents cur_lang }
⟨ Store all current lc_code values 1667 ⟩;
end
```
This code is used in section 1014.

1667. We store all nonzero *lc_code* values, overwriting any previously stored values (and possibly wasting a few trie nodes that were used previously and are not needed now). We always store at least one *lc_code* value such that *hyph_index* (defined below) will not be zero.

⟨ Store all current *lc_code* values 1667 ⟩ ≡
```
p ← trie_l[q]; first_child ← true;
for c ← 0 to 255 do
  if (lc_code(c) > 0) ∨ ((c = 255) ∧ first_child) then
    begin if p = 0 then ⟨ Insert a new trie node between q and p, and make p point to it 1018 ⟩
    else trie_c[p] ← si(c);
    trie_o[p] ← qi(lc_code(c)); q ← p; p ← trie_r[q]; first_child ← false;
    end;
if first_child then trie_l[q] ← 0 else trie_r[q] ← 0
```
This code is used in section 1666.

1668. We must avoid to "take" location 1, in order to distinguish between *lc_code* values and patterns.

⟨ Pack all stored *hyph_codes* 1668 ⟩ ≡
```
begin if trie_root = 0 then
  for p ← 0 to 255 do trie_min[p] ← p + 2;
first_fit(hyph_root); trie_pack(hyph_root); hyph_start ← trie_ref[hyph_root];
end
```
This code is used in section 1020.

1669. The global variable *hyph_index* will point to the hyphenation codes for the current language.

```
define set_hyph_index ≡ { set hyph_index for current language }
    if trie_char(hyph_start + cur_lang) ≠ qi(cur_lang) then hyph_index ← 0
        { no hyphenation codes for cur_lang }
    else hyph_index ← trie_link(hyph_start + cur_lang)
define set_lc_code(#) ≡ { set hc[0] to hyphenation or lc code for # }
    if (hyph_index = 0) ∨ ((#) > 255) then hc[0] ← lc_code(#)
    else if trie_char(hyph_index + #) ≠ qi(#) then hc[0] ← 0
      else hc[0] ← qo(trie_op(hyph_index + #))
```

⟨ Global variables 13 ⟩ +≡
```
hyph_start: trie_pointer; { root of the packed trie for hyph_codes }
hyph_index: trie_pointer; { pointer to hyphenation codes for cur_lang }
```

1670. When *saving_vdiscards* is positive then the glue, kern, and penalty nodes removed by the page builder or by `\vsplit` from the top of a vertical list are saved in special lists instead of being discarded.

define *tail_page_disc* ≡ *disc_ptr*[*copy_code*] { last item removed by page builder }
define *page_disc* ≡ *disc_ptr*[*last_box_code*] { first item removed by page builder }
define *split_disc* ≡ *disc_ptr*[*vsplit_code*] { first item removed by `\vsplit` }

⟨ Global variables 13 ⟩ +≡
disc_ptr: **array** [*copy_code* .. *vsplit_code*] **of** *pointer*; { list pointers }

1671. ⟨ Set initial values of key variables 23 ⟩ +≡
page_disc ← *null*; *split_disc* ← *null*;

1672. The `\pagediscards` and `\splitdiscards` commands share the command code *un_vbox* with `\unvbox` and `\unvcopy`, they are distinguished by their *chr_code* values *last_box_code* and *vsplit_code*. These *chr_code* values are larger than *box_code* and *copy_code*.

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("pagediscards", *un_vbox*, *last_box_code*);
primitive("splitdiscards", *un_vbox*, *vsplit_code*);

1673. ⟨ Cases of *un_vbox* for *print_cmd_chr* 1673 ⟩ ≡
else if *chr_code* = *last_box_code* **then** *print_esc*("pagediscards")
else if *chr_code* = *vsplit_code* **then** *print_esc*("splitdiscards")

This code is used in section 1162.

1674. ⟨ Handle saved items and **goto** *done* 1674 ⟩ ≡
begin *link*(*tail*) ← *disc_ptr*[*cur_chr*]; *disc_ptr*[*cur_chr*] ← *null*; **goto** *done*;
end

This code is used in section 1164.

1675. The `\interlinepenalties`, `\clubpenalties`, `\widowpenalties`, and `\displaywidowpenalties` commands allow to define arrays of penalty values to be used instead of the corresponding single values.

define *inter_line_penalties_ptr* ≡ *equiv*(*inter_line_penalties_loc*)
define *club_penalties_ptr* ≡ *equiv*(*club_penalties_loc*)
define *widow_penalties_ptr* ≡ *equiv*(*widow_penalties_loc*)
define *display_widow_penalties_ptr* ≡ *equiv*(*display_widow_penalties_loc*)

⟨ Generate all ε-TEX primitives 1399 ⟩ +≡
primitive("interlinepenalties", *set_shape*, *inter_line_penalties_loc*);
primitive("clubpenalties", *set_shape*, *club_penalties_loc*);
primitive("widowpenalties", *set_shape*, *widow_penalties_loc*);
primitive("displaywidowpenalties", *set_shape*, *display_widow_penalties_loc*);

1676. ⟨ Cases of *set_shape* for *print_cmd_chr* 1676 ⟩ ≡
inter_line_penalties_loc: *print_esc*("interlinepenalties");
club_penalties_loc: *print_esc*("clubpenalties");
widow_penalties_loc: *print_esc*("widowpenalties");
display_widow_penalties_loc: *print_esc*("displaywidowpenalties");

This code is used in section 296.

1677. ⟨ Fetch a penalties array element 1677 ⟩ ≡

```
begin scan_int;
if (equiv(m) = null) ∨ (cur_val < 0) then cur_val ← 0
else begin if cur_val > penalty(equiv(m)) then cur_val ← penalty(equiv(m));
  cur_val ← penalty(equiv(m) + cur_val);
  end;
end
```

This code is used in section 457.

1678. System-dependent changes. This section should be replaced, if necessary, by any special modifications of the program that are necessary to make TeX work at a particular installation. It is usually best to design your change file so that all changes to previous sections preserve the section numbering; then everybody's version will be consistent with the published program. More extensive changes, which introduce new sections, can be inserted here; then only the index itself will get a new section number.

1679. Index. Here is where you can find all uses of each identifier in the program, with underlined entries pointing to where the identifier was defined. If the identifier is only one letter long, however, you get to see only the underlined entries. *All references are to section numbers instead of page numbers.*

This index also lists error messages and other aspects of the program that you might want to look up some day. For example, the entry for "system dependencies" lists all sections that should receive special attention from people who are installing TeX in a new operating environment. A list of various things that can't happen appears under "this can't happen". Approximately 40 sections are listed under "inner loop"; these account for about 60% of TeX's running time, exclusive of input and output.

`**`: 37, 569.
`*`: 200, 202, 204, 343, 390, 904, 1060, 1415.
`->`: 324.
`=>`: 393.
`???`: 63.
`?`: 87.
`@`: 904.
`@@`: 894.
a: 47, 106, 126, 244, 311, 553, 554, 558, 595, 633, 733, 749, 765, 781, 796, 1129, 1177, 1248, 1265, 1290, 1311, 1489, 1593, 1604, 1608, 1610, 1636.
`A <box> was supposed to...`: 1138.
a_close: 28, 51, 1387, 1437, 1441.
a_leaders: 173, 215, 663, 665, 672, 674, 698, 713, 1125, 1126, 1127, 1132, 1202, 1491, 1509.
a_make_name_string: 560, 569, 572.
a_open_in: 27, 51, 572, 1329.
a_open_out: 27, 569, 1437.
A_token: 479.
aat_font_flag: 584, 744.
aat_font_get: 1454.
aat_font_get_named: 1454.
aat_font_get_named_1: 1454.
aat_font_get_1: 1454.
aat_font_get_2: 1454.
aat_get_font_metrics: 744.
aat_print_font_name: 1461.
ab_vs_cd: 126, 131.
abort: 595, 599, 600, 603, 604, 605, 606, 608, 610.
above: 234, 1100, 1232, 1233, 1234.
`\above` primitive: 1232.
above_code: 1232, 1233, 1236, 1237.
above_display_short_skip: 250, 862.
`\abovedisplayshortskip` primitive: 252.
above_display_short_skip_code: 250, 251, 252, 1257.
above_display_skip: 250, 862.
`\abovedisplayskip` primitive: 252.
above_display_skip_code: 250, 251, 252, 1257, 1260.
`\abovewithdelims` primitive: 1232.
abs: 70, 129, 130, 131, 212, 237, 244, 245, 452, 456, 482, 536, 646, 705, 717, 761, 780, 801, 802, 803, 879, 884, 897, 907, 998, 1002, 1083, 1084, 1110, 1130, 1132, 1134, 1137, 1147, 1164, 1174, 1181, 1203, 1297, 1298, 1440, 1442, 1443, 1444, 1491, 1601.
absorbing: 335, 336, 369, 508, 1493.
acc_kern: 179, 217, 1179.
accent: 234, 295, 296, 1144, 1176, 1218, 1219.
`\accent` primitive: 295.
accent_chr: 729, 738, 781, 1219.
accent_noad: 729, 732, 738, 740, 776, 781, 809, 1219, 1240.
accent_noad_size: 729, 740, 809, 1219.
accentBaseHeight: 742, 781.
act_width: 914, 915, 916, 917, 919, 1422.
action procedure: 1083.
active: 187, 867, 877, 891, 902, 908, 909, 911, 912, 913, 921, 922, 923.
active_base: 246, 248, 278, 279, 281, 292, 293, 383, 476, 499, 541, 1206, 1311, 1343, 1369, 1371.
active_char: 233, 374, 499, 506, 541.
active_glue: 1653, 1656, 1657, 1662, 1663, 1664.
active_height: 1024, 1029, 1030.
active_math_char: 258, 469, 1286.
active_node_size: 893, 908, 912, 913, 1653, 1654.
active_node_size_extended: 1653, 1654.
active_node_size_normal: 867, 1654.
active_short: 1653, 1656, 1657, 1662, 1663, 1664.
active_width: 871, 872, 877, 891, 909, 912, 914, 916, 1024.
actual_looseness: 920, 921, 923.
actual_size: 744.
add_delims_to: 377.
add_glue_ref: 229, 232, 464, 850, 929, 1050, 1154, 1283, 1544, 1593, 1631.
add_or_sub: 1603, 1604.
add_sa_ptr: 1630.
add_sa_ref: 1275, 1278, 1632, 1648, 1650, 1651.
add_token_ref: 229, 232, 353, 1033, 1066, 1070, 1275, 1281, 1417, 1638, 1639, 1640, 1641.
additional: 683, 684, 699, 714.
addressof: 744, 751, 783, 793, 1177, 1179, 1445, 1446, 1447.
adj_demerits: 262, 884, 907.
`\adjdemerits` primitive: 264.
adj_demerits_code: 262, 263, 264.
adjust: 611.

adjust_head: 187, 936, 937, 1130, 1139, 1253, 1259.
adjust_node: 164, 172, 201, 209, 228, 232, 656, 686, 691, 697, 773, 809, 877, 914, 945, 952, 1154.
adjust_pre: 164, 223, 697, 1154.
adjust_ptr: 164, 223, 228, 232, 697, 1154.
adjust_space_factor: 1088, 1092.
adjust_tail: 686, 687, 689, 691, 697, 844, 936, 937, 1130, 1139, 1253.
adjusted_hbox_group: 299, 1116, 1137, 1139, 1471, 1489.
adv_past_linebreak: 1422.
adv_past_prehyph: 1423.
advance: 235, 295, 296, 1264, 1289, 1290, 1292.
`\advance` primitive: 295.
advance_major_tail: 968, 971.
aField: 1445.
after: 171, 218, 1250, 1539, 1550.
after_assignment: 234, 295, 296, 1322.
`\afterassignment` primitive: 295.
after_group: 234, 295, 296, 1325.
`\aftergroup` primitive: 295.
after_math: 1247, 1248.
after_token: 1320, 1321, 1322, 1323.
aire: 595, 596, 598, 611, 744.
align_error: 1180, 1181.
align_group: 299, 816, 822, 839, 848, 1185, 1186, 1471, 1489.
align_head: 187, 818, 825.
align_peek: 821, 822, 833, 847, 1102, 1187.
align_ptr: 818, 819, 820.
align_stack_node_size: 818, 820.
align_state: 92, 339, 354, 355, 356, 361, 369, 372, 377, 387, 428, 429, 430, 437, 476, 510, 517, 518, 521, 818, 819, 820, 822, 825, 831, 832, 833, 836, 837, 839, 1123, 1148, 1180, 1181.
aligning: 335, 336, 369, 825, 837.
alignment of rules with characters: 625.
alpha: 595, 606, 607.
alpha_file: 25, 27, 28, 31, 32, 50, 54, 334, 560, 1396.
alpha_token: 472, 474.
alter_aux: 1296, 1297.
alter_box_dimen: 1296, 1301.
alter_integer: 1296, 1300.
alter_page_so_far: 1296, 1299.
alter_prev_graf: 1296, 1298.
`Ambiguous...`: 1237.
Amble, Ole: 979.
`AmSTeX`: 1385.
any_mode: 1099, 1102, 1111, 1117, 1121, 1127, 1151, 1156, 1158, 1180, 1188, 1264, 1322, 1325, 1328, 1330, 1339, 1344, 1402.
any_state_plus: 374, 375, 377.
app_display: 1257, 1258, 1259, 1555.
app_space: 1084, 1097.
append_char: 42, 44, 52, 58, 206, 221, 287, 551, 560, 656, 734, 737, 744, 993.
append_charnode_to_t: 962, 965.
append_choices: 1225, 1226.
append_discretionary: 1170, 1171.
append_glue: 1111, 1114, 1132.
append_italic_correction: 1166, 1167.
append_kern: 1111, 1115.
append_list: 164, 847, 936, 1130.
append_list_end: 164.
append_native: 60, 1088.
append_normal_space: 1084.
append_penalty: 1156, 1157.
append_str: 44, 1314.
append_to_name: 554, 558.
append_to_vlist: 721, 847, 936, 1130, 1555.
apply_mapping: 744, 1088.
apply_tfm_font_mapping: 658.
area_delimiter: 548, 550, 551, 552.
`Argument of \x has...`: 429.
arg1: 505, 1460, 1461.
arg2: 505, 1460, 1461.
arith_error: 108, 109, 110, 111, 116, 118, 198, 482, 488, 495, 1290, 1593, 1594, 1601, 1657.
`Arithmetic overflow`: 1290, 1593.
artificial_demerits: 878, 899, 902, 903, 904.
ascent: 744.
ASCII code: 17, 538.
ASCII_code: 18, 19, 20, 29, 30, 31, 38, 42, 54, 58, 59, 86, 322, 423, 551, 554, 558, 734, 997, 1004, 1007, 1013, 1014, 1439.
assign_dimen: 235, 274, 275, 447, 1264, 1278, 1282.
assign_font_dimen: 235, 295, 296, 447, 1264, 1307.
assign_font_int: 235, 447, 1264, 1307, 1308, 1309.
assign_glue: 235, 252, 253, 447, 830, 1264, 1278, 1282.
assign_int: 235, 264, 265, 447, 1264, 1276, 1278, 1282, 1291, 1467, 1511.
assign_mu_glue: 235, 252, 253, 447, 1264, 1276, 1278, 1282, 1291.
assign_toks: 235, 256, 257, 259, 353, 447, 449, 1264, 1278, 1280, 1281, 1399, 1467.
assign_trace: 307, 308, 309.
`at`: 1312.
`\atop` primitive: 1232.
atop_code: 1232, 1233, 1236.
`\atopwithdelims` primitive: 1232.
attach_fraction: 482, 488, 489, 491.
attach_hkern_to_new_hlist: 800, 806, 807.

attach_sign: 482, 484, 490.
auto_breaking: 910, 911, 914, 916.
aux: 238, 239, 242, 848, 860.
aux_field: 238, 239, 244, 823.
aux_save: 848, 860, 1260.
avail: 140, 142, 143, 144, 145, 189, 193, 1365, 1366.
`AVAIL list clobbered...`: 193.
awful_bad: 881, 882, 883, 884, 902, 922, 1024, 1028, 1029, 1041, 1059, 1060, 1061.
axis_height: 742, 749, 779, 790, 791, 793, 810.
axisHeight: 742.
b: 396, 499, 500, 505, 533, 558, 595, 633, 721, 748, 749, 752, 754, 758, 878, 1024, 1048, 1252, 1301, 1342, 1466, 1555, 1593.
b_close: 28, 595, 680.
b_make_name_string: 560, 567.
b_open_in: 27, 598.
b_open_out: 27.
back_error: 357, 407, 430, 437, 449, 476, 480, 511, 514, 538, 612, 831, 1132, 1138, 1215, 1251, 1261, 1266, 1576, 1595.
back_input: 311, 355, 356, 357, 400, 401, 402, 406, 409, 413, 429, 439, 441, 449, 477, 478, 482, 487, 490, 496, 561, 836, 1085, 1088, 1101, 1108, 1118, 1144, 1149, 1178, 1181, 1186, 1192, 1204, 1206, 1207, 1269, 1275, 1280, 1323, 1438, 1444, 1595, 1596.
back_list: 353, 355, 367, 441, 1342.
backed_up: 337, 341, 342, 344, 353, 354, 355, 1080.
backed_up_char: 337, 344, 1088.
background: 871, 872, 875, 885, 911, 912, 1654.
backup_backup: 396.
backup_head: 187, 396, 441.
`BAD`: 323, 324.
bad: 13, 14, 133, 320, 557, 1303, 1386.
`Bad \patterns`: 1015.
`Bad \prevgraf`: 1298.
`Bad character code`: 467, 468.
`Bad delcode`: 448.
`Bad delimiter code`: 471.
`Bad dump length`: 506.
`Bad file offset`: 506.
`Bad flag...`: 195.
`Bad interaction mode`: 1506.
`Bad link...`: 208.
`Bad mathchar`: 448, 470.
`Bad number`: 469.
`Bad register code`: 467, 1622.
`Bad space factor`: 1297.
bad_fmt: 1357, 1360, 1362, 1366, 1371, 1381.
bad_pool: 51, 52, 53.
bad_tfm: 595.
bad_utf8_warning: 744.
badness: 112, 702, 709, 716, 720, 876, 900, 901, 1029, 1061, 1658, 1659.
`\badness` primitive: 450.
badness_code: 450, 458.
banner: 2, 65, 571, 1353.
base_line: 655, 661, 662, 666, 1430.
base_ptr: 88, 89, 340, 341, 342, 343, 1185, 1585, 1586, 1587.
baseline_skip: 250, 273, 721.
`\baselineskip` primitive: 252.
baseline_skip_code: 173, 250, 251, 252, 721.
batch_mode: 77, 79, 90, 94, 96, 97, 570, 1316, 1317, 1381, 1382, 1506.
`\batchmode` primitive: 1316.
bc: 575, 576, 578, 580, 595, 600, 601, 605, 611.
bch_label: 595, 608, 611.
bchar: 595, 608, 611, 954, 956, 959, 960, 962, 965, 967, 970, 971, 1086, 1088, 1091, 1092, 1094.
bchar_label: 584, 587, 611, 963, 970, 1088, 1094, 1376, 1377.
be_careful: 116, 117, 118.
before: 171, 218, 1250, 1520, 1522, 1528, 1539, 1550.
begin: 7, 8.
begin_box: 1127, 1133, 1138.
begin_diagnostic: 80, 271, 314, 329, 353, 434, 435, 537, 544, 595, 616, 676, 679, 705, 717, 744, 874, 911, 1041, 1046, 1060, 1065, 1175, 1347, 1350, 1472, 1487, 1501, 1634.
begin_file_reading: 82, 91, 358, 518, 572, 1566.
begin_group: 234, 295, 296, 1117.
`\begingroup` primitive: 295.
begin_insert_or_adjust: 1151, 1153.
begin_L_code: 171, 1511, 1512, 1545.
begin_LR_type: 171, 1517.
begin_M: 1134.
begin_M_code: 171, 1134, 1557.
begin_name: 547, 550, 561, 562, 566.
begin_pseudoprint: 346, 348, 349.
begin_R_code: 171, 1511, 1512.
begin_reflect: 1510.
begin_token_list: 353, 389, 392, 420, 424, 822, 836, 837, 847, 1079, 1084, 1088, 1137, 1145, 1193, 1199, 1221, 1434.
`\beginL` primitive: 1511.
`Beginning to dump...`: 1382.
`\beginR` primitive: 1511.
below_display_short_skip: 250.
`\belowdisplayshortskip` primitive: 252.
below_display_short_skip_code: 250, 251, 252, 1257.
below_display_skip: 250.

\belowdisplayskip primitive: 252.
below_display_skip_code: 250, 251, 252, 1257, 1260.
best_bet: 920, 922, 923, 925, 926, 1664.
best_height_plus_depth: 1025, 1028, 1064, 1065.
best_ins_ptr: 1035, 1059, 1063, 1072, 1074, 1075.
best_line: 920, 922, 923, 925, 938.
best_page_break: 1034, 1059, 1067, 1068.
best_pl_glue: 1653, 1661, 1662.
best_pl_line: 881, 893, 903.
best_pl_short: 1653, 1661, 1662.
best_place: 881, 893, 903, 1024, 1028, 1034.
best_size: 1034, 1059, 1071.
beta: 595, 606, 607.
bField: 1445.
big_op_spacing1: 743, 795.
big_op_spacing2: 743, 795.
big_op_spacing3: 743, 795.
big_op_spacing4: 743, 795.
big_op_spacing5: 743, 795.
big_switch: 235, 262, 1048, 1083, 1084, 1085, 1088, 1090, 1092, 1095.
BigEndian order: 575.
biggest_char: 12, 18, 19, 38, 63, 289, 468, 561, 949, 1006, 1171.
biggest_lang: 12, 940, 975, 988, 997, 999, 1378, 1379.
biggest_reg: 12, 273, 281, 1048, 1066.
biggest_usv: 12, 18, 67, 382, 385, 468, 469, 476, 541, 1287.
billion: 663.
bin_noad: 724, 732, 738, 740, 771, 772, 805, 809, 1210, 1211.
bin_op_penalty: 262, 809.
\binoppenalty primitive: 264.
bin_op_penalty_code: 262, 263, 264.
blank_line: 271.
boolean: 27, 31, 37, 45, 46, 47, 58, 61, 80, 83, 100, 108, 110, 111, 116, 118, 190, 192, 198, 271, 282, 311, 341, 391, 396, 397, 441, 447, 474, 482, 496, 505, 508, 533, 551, 559, 562, 567, 584, 595, 613, 628, 655, 667, 684, 721, 744, 749, 762, 769, 839, 863, 873, 876, 877, 878, 910, 925, 953, 961, 1004, 1014, 1022, 1043, 1066, 1086, 1105, 1108, 1133, 1145, 1159, 1214, 1248, 1265, 1290, 1335, 1357, 1396, 1445, 1466, 1470, 1471, 1472, 1567, 1585, 1587, 1593, 1604, 1608, 1610, 1630, 1636, 1653.
boolvar: 505, 506.
bop: 619, 621, 622, 624, 626, 628, 676, 678.
Bosshard, Hans Rudolf: 493.
bot: 581.
bot_mark: 416, 417, 1066, 1070, 1620, 1639.
\botmark primitive: 418.
bot_mark_code: 416, 418, 419, 1620.
\botmarks primitive: 1620.
bottom_acc: 729, 1219.
bottom_level: 299, 302, 311, 1118, 1122, 1471, 1489.
bottom_line: 341.
bounds: 1445.
bowels: 628.
box: 256, 258, 1046, 1047, 1063, 1069, 1071, 1072, 1075, 1077, 1082, 1631, 1632, 1650.
\box primitive: 1125.
box_base: 256, 258, 259, 281, 1131.
box_code: 1125, 1126, 1133, 1161, 1164, 1672.
box_context: 1129, 1130, 1131, 1132, 1133, 1137, 1138.
box_end: 1129, 1133, 1138, 1140.
box_error: 1046, 1047, 1069, 1082.
box_flag: 1125, 1129, 1131, 1137, 1295, 1491.
box_lr: 157, 652, 1514, 1524, 1525, 1556.
box_max_depth: 273, 1140.
\boxmaxdepth primitive: 274.
box_max_depth_code: 273, 274.
box_node: 169, 170.
box_node_size: 157, 158, 228, 232, 689, 710, 758, 770, 795, 800, 1031, 1075, 1154, 1164, 1255, 1544, 1556.
box_ref: 236, 258, 305, 1131.
box_there: 1034, 1041, 1054, 1055.
box_val: 1278, 1626, 1631, 1632, 1634, 1650.
box_val_limit: 1626, 1649.
\box255 is not void: 1069.
bp: 493.
brain: 1083.
breadth_max: 207, 208, 224, 259, 262, 1393, 1634.
break: 34, 656.
break_in: 34.
break_node: 867, 877, 893, 903, 904, 911, 912, 925, 926.
break_penalty: 234, 295, 296, 1156.
break_type: 877, 885, 893, 894, 907.
break_width: 871, 872, 885, 886, 888, 889, 890, 891, 892, 927.
breakpoint: 1392.
broken_ins: 1035, 1040, 1064, 1075.
broken_penalty: 262, 938.
\brokenpenalty primitive: 264.
broken_penalty_code: 262, 263, 264.
broken_ptr: 1035, 1064, 1075.
buf_size: 11, 30, 31, 35, 75, 133, 294, 345, 358, 361, 371, 393, 396, 408, 559, 565, 569, 1388, 1567, 1579.

buffer: 30, 31, 36, 37, 45, 75, 87, 91, 92, 286, 287, 288, 294, 332, 333, 345, 348, 361, 371, 373, 382, 384, 385, 386, 390, 392, 393, 396, 408, 518, 519, 558, 559, 565, 566, 569, 573, 1391, 1393, 1451, 1567, 1572, 1579.
`Buffer size exceeded`: 35.
build_choices: 1227, 1228.
build_discretionary: 1172, 1173.
build_opentype_assembly: 749, 783, 793.
build_page: 848, 860, 1042, 1048, 1080, 1108, 1114, 1130, 1145, 1148, 1154, 1157, 1199, 1254.
`by`: 1290.
bypass_eoln: 31.
byte_file: 25, 27, 28, 560, 567, 574.
b0: 132, 135, 136, 155, 169, 170, 247, 283, 298, 580, 581, 585, 589, 591, 599, 638, 725, 727, 744, 975, 1012, 1363, 1364, 1565, 1567.
b1: 132, 135, 136, 155, 169, 170, 247, 283, 298, 580, 581, 589, 591, 599, 638, 725, 727, 744, 975, 1012, 1363, 1364, 1565, 1567.
b2: 132, 135, 136, 169, 580, 581, 589, 591, 599, 638, 725, 727, 744, 1363, 1364, 1565, 1567.
b3: 132, 135, 136, 169, 580, 581, 591, 599, 638, 725, 727, 744, 1363, 1364, 1565, 1567.
c: 47, 67, 86, 166, 294, 304, 322, 371, 500, 505, 551, 554, 558, 595, 616, 618, 628, 684, 733, 734, 736, 744, 749, 752, 754, 755, 781, 793, 942, 966, 1007, 1013, 1014, 1048, 1066, 1140, 1155, 1164, 1171, 1190, 1205, 1209, 1219, 1235, 1297, 1299, 1300, 1301, 1329, 1333, 1342, 1389, 1489, 1588.
c_leaders: 173, 216, 665, 674, 1125, 1126.
`\cleaders` primitive: 1125.
c_loc: 966, 970.
calc_min_and_max: 1445.
call: 236, 249, 305, 326, 396, 414, 421, 429, 430, 513, 542, 1272, 1275, 1279, 1280, 1281, 1349, 1583.
call_func: 1410.
cancel_boundary: 1084, 1086, 1087, 1088.
cancel_glue: 1557.
cancel_glue_cont: 1557.
cancel_glue_cont_cont: 1557.
cancel_glue_end: 1557.
cancel_glue_end_end: 1557.
`cannot \read`: 519.
cap_height: 744.
cap_ht: 744.
car_ret: 233, 258, 372, 377, 825, 828, 829, 831, 832, 833, 836, 1180.
carriage_return: 22, 49, 233, 258, 266, 393.
case_shift: 234, 1339, 1340, 1341.
cast_to_ushort: 733, 765.
cat: 371, 384, 385, 386, 499, 505, 506.
cat_code: 256, 258, 262, 292, 371, 373, 384, 385, 386, 1391.
`\catcode` primitive: 1284.
cat_code_base: 256, 258, 259, 261, 1284, 1285, 1287.
cc: 382, 1415.
`cc`: 493.
ccc: 382.
cccc: 382.
cField: 1445.
ch: 1209.
change_box: 1031, 1133, 1164, 1632.
change_if_limit: 532, 533, 544.
char: 19, 26, 59, 169, 555, 569, 1445.
`\char` primitive: 295.
char_base: 585, 587, 589, 601, 605, 611, 1376, 1377.
char_box: 752, 753, 754, 781.
char_class_boundary: 447, 1088.
char_class_ignored: 447, 1088.
char_class_limit: 447, 449, 467, 1088, 1280, 1281.
`\chardef` primitive: 1276.
char_def_code: 1276, 1277, 1278.
char_depth: 589, 694, 751, 752, 755, 1481.
char_depth_end: 589.
char_exists: 589, 608, 611, 618, 751, 765, 781, 784, 793, 799, 1090, 1580.
char_given: 234, 447, 989, 1084, 1088, 1092, 1144, 1178, 1205, 1208, 1276, 1277, 1278.
char_height: 589, 694, 751, 752, 755, 1179, 1481.
char_height_end: 589.
char_info: 578, 585, 589, 590, 592, 605, 608, 611, 618, 658, 694, 751, 752, 755, 757, 758, 765, 767, 781, 784, 793, 889, 890, 914, 915, 918, 919, 963, 1090, 1091, 1093, 1094, 1167, 1177, 1179, 1201, 1481, 1534, 1580.
char_info_end: 589.
char_info_word: 576, 578, 579.
char_italic: 589, 752, 757, 793, 799, 1167, 1481.
char_italic_end: 589.
char_kern: 592, 785, 797, 963, 1094.
char_kern_end: 592.
char_node: 156, 165, 167, 187, 202, 583, 628, 658, 689, 796, 929, 961, 1083, 1167, 1192, 1420.
char_num: 234, 295, 296, 989, 1084, 1088, 1092, 1144, 1178, 1205, 1208.
char_pw: 688.
char_tag: 589, 605, 751, 753, 784, 785, 793, 796, 963, 1093.
char_warning: 616, 618, 744, 765, 1088, 1090.
char_width: 589, 658, 694, 752, 757, 758, 784, 889, 890, 914, 915, 918, 919, 1177, 1179, 1201, 1481, 1534.

char_width_end: 589.
character: 156, 165, 166, 200, 202, 232, 618, 658, 688, 694, 723, 724, 725, 729, 733, 752, 758, 765, 767, 793, 796, 797, 889, 890, 914, 915, 918, 919, 949, 950, 951, 956, 961, 962, 964, 965, 1086, 1088, 1089, 1090, 1091, 1092, 1094, 1167, 1177, 1179, 1201, 1205, 1209, 1219, 1534, 1544.
character set dependencies: 23, 49.
check sum: 53, 577, 624.
check_byte_range: 605, 608.
check_dimensions: 769, 770, 776, 798.
check_effective_tail: 1134, 1159.
check_existence: 608, 609.
check_for_inter_char_toks: 1088, 1092.
check_for_post_char_toks: 1084, 1088.
check_for_tfm_font_mapping: 598.
check_full_save_stack: 303, 304, 306, 310, 1648.
check_interrupt: 100, 354, 373, 797, 965, 1085, 1094.
check_keywords: 1445.
check_mem: 190, 192, 1085, 1393.
check_next: 655, 656.
check_outer_validity: 366, 381, 383, 384, 387, 392, 409.
check_shrinkage: 873, 875, 916.
Chinese characters: 156, 621.
choice_node: 730, 731, 732, 740, 773, 805.
choose_mlist: 774.
chr: 19, 20, 23, 1276.
chr_cmd: 328, 829.
chr_code: 253, 257, 265, 275, 296, 328, 411, 419, 445, 447, 451, 504, 523, 527, 829, 1038, 1107, 1113, 1125, 1126, 1143, 1162, 1169, 1197, 1211, 1224, 1233, 1243, 1263, 1274, 1277, 1285, 1305, 1309, 1315, 1317, 1327, 1332, 1341, 1343, 1346, 1349, 1401, 1497, 1503, 1508, 1512, 1559, 1582, 1643, 1644, 1672, 1673.
clang: 238, 239, 860, 1088, 1145, 1254, 1439, 1440.
clean_box: 763, 777, 778, 780, 781, 786, 788, 793, 794, 801, 802, 803.
clear_for_error_prompt: 82, 87, 360, 376.
clear_terminal: 34, 360, 565, 1392.
clobbered: 192, 193, 194.
`CLOBBERED`: 323.
close: 28.
close_files_and_terminate: 82, 85, 1386, 1387.
`\closein` primitive: 1326.
close_noad: 724, 732, 738, 740, 771, 805, 809, 810, 1210, 1211.
close_node: 1395, 1398, 1401, 1403, 1416, 1417, 1418, 1436, 1437, 1438.
`\closeout` primitive: 1398.
closed: 515, 516, 518, 520, 521, 536, 1329.
clr: 780, 787, 789, 790, 800, 801, 802, 803.
`\clubpenalties` primitive: 1675.
club_penalties_loc: 256, 1675, 1676.
club_penalties_ptr: 938, 1675.
club_penalty: 262, 938.
`\clubpenalty` primitive: 264.
club_penalty_code: 262, 263, 264.
`cm`: 493.
cmd: 328, 1276, 1343, 1349, 1457, 1643.
co_backup: 396.
collect_native: 1084, 1088.
collected: 1084, 1088.
COLORED: 621.
combine_two_deltas: 908.
comment: 233, 258, 377.
common_ending: 15, 533, 535, 544, 689, 702, 708, 709, 710, 716, 719, 720, 944, 956, 1311, 1314, 1347, 1348, 1351, 1522.
compare_strings: 506, 1410.
`Completed box...`: 676.
compress_trie: 1003, 1006.
compute_ot_math_accent_pos: 781.
cond_math_glue: 173, 215, 775, 1225.
cond_ptr: 329, 358, 392, 524, 525, 530, 531, 532, 533, 535, 544, 1389, 1478, 1501, 1584, 1587, 1588.
conditional: 396, 399, 533.
confusion: 99, 116, 228, 232, 311, 532, 668, 697, 711, 771, 779, 798, 809, 814, 839, 846, 848, 889, 890, 914, 918, 919, 925, 1022, 1027, 1054, 1122, 1134, 1239, 1254, 1265, 1403, 1417, 1418, 1436, 1522, 1535, 1541, 1556.
continental_point_token: 472, 482.
continue: 15, 86, 87, 88, 92, 93, 423, 426, 427, 428, 429, 431, 508, 509, 511, 749, 751, 822, 832, 863, 877, 880, 899, 949, 960, 963, 964, 965, 1048, 1055, 1593, 1594.
contrib_head: 187, 241, 244, 1042, 1048, 1049, 1052, 1053, 1055, 1071, 1077, 1080.
contrib_tail: 1049, 1071, 1077, 1080.
contribute: 1048, 1051, 1054, 1056, 1062, 1424.
conv_toks: 396, 399, 505.
conventions for representing stacks: 330.
convert: 236, 396, 399, 503, 504, 505, 1452, 1460.
convert_to_break_width: 891.
`\copy` primitive: 1125.
copy_code: 1125, 1126, 1133, 1161, 1162, 1164, 1670, 1672.
copy_native_glyph_info: 169, 1417.
copy_node_list: 186, 229, 230, 232, 1133, 1164, 1556.

copy_to_cur_active: 877, 909.
corners: 1445.
count: 262, 461, 676, 678, 1040, 1062, 1063, 1064.
\count primitive: 445.
count_base: 262, 265, 268, 1278, 1291.
\countdef primitive: 1276.
count_def_code: 1276, 1277, 1278.
count_pdf_file_pages: 1454.
cp_skipable: 507, 877.
\cr primitive: 828.
cr_code: 828, 829, 837, 839, 840.
\crcr primitive: 828.
cr_cr_code: 828, 833, 837.
cramped: 730, 745.
cramped_style: 745, 777, 780, 781.
cs_count: 282, 285, 287, 1372, 1373, 1388.
cs_error: 1188, 1189.
cs_name: 236, 295, 296, 396, 399.
\csname primitive: 295.
cs_token_flag: 319, 320, 323, 364, 366, 367, 369, 387, 388, 395, 401, 402, 403, 406, 409, 413, 414, 415, 474, 476, 499, 501, 541, 828, 1088, 1099, 1119, 1186, 1269, 1343, 1368, 1434.
cur_active_width: 871, 872, 877, 880, 885, 891, 892, 899, 900, 901, 908, 1657, 1658, 1659, 1660.
cur_align: 818, 819, 820, 825, 826, 827, 831, 834, 836, 837, 839, 840, 843, 844, 846.
cur_area: 547, 552, 564, 565, 572, 1311, 1314, 1406, 1437, 1445.
cur_boundary: 300, 301, 302, 304, 312, 358, 392, 1489, 1584, 1585, 1588.
cur_box: 1128, 1129, 1130, 1131, 1132, 1133, 1134, 1135, 1136, 1138, 1140, 1141, 1650.
cur_break: 869, 877, 893, 927, 928, 929, 1517.
cur_c: 765, 766, 767, 781, 793, 796, 797, 799, 805.
cur_chr: 92, 326, 327, 329, 362, 367, 371, 373, 378, 379, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 394, 395, 399, 402, 403, 412, 414, 415, 420, 421, 423, 437, 441, 447, 449, 458, 462, 476, 500, 505, 507, 509, 511, 513, 514, 518, 529, 530, 533, 535, 536, 541, 542, 543, 544, 545, 561, 612, 830, 833, 837, 989, 991, 1016, 1084, 1088, 1090, 1092, 1099, 1103, 1112, 1114, 1115, 1120, 1127, 1133, 1137, 1144, 1147, 1155, 1159, 1160, 1164, 1171, 1178, 1182, 1194, 1196, 1205, 1206, 1208, 1209, 1212, 1213, 1214, 1219, 1225, 1235, 1245, 1265, 1266, 1267, 1271, 1272, 1275, 1278, 1279, 1280, 1281, 1282, 1286, 1287, 1288, 1291, 1297, 1299, 1300, 1301, 1302, 1306, 1307, 1319, 1329, 1333, 1342, 1347, 1389, 1403, 1405, 1438, 1484, 1498, 1506, 1513, 1560, 1572, 1576, 1583, 1674.
cur_cmd: 92, 237, 326, 327, 329, 362, 367, 371, 372, 373, 374, 378, 379, 381, 383, 384, 387, 388, 390, 394, 395, 396, 399, 400, 402, 403, 406, 414, 415, 420, 421, 437, 438, 440, 441, 447, 449, 462, 474, 476, 477, 478, 482, 487, 490, 496, 498, 509, 512, 513, 514, 518, 529, 536, 541, 542, 561, 612, 825, 830, 831, 832, 833, 836, 837, 839, 989, 1015, 1083, 1084, 1088, 1092, 1099, 1103, 1120, 1132, 1133, 1138, 1149, 1153, 1178, 1182, 1192, 1205, 1206, 1214, 1219, 1230, 1231, 1251, 1260, 1265, 1266, 1267, 1275, 1280, 1281, 1282, 1290, 1291, 1306, 1324, 1438, 1493, 1513, 1576, 1577, 1578, 1583, 1595.
cur_cs: 327, 362, 363, 366, 367, 368, 371, 381, 383, 384, 386, 387, 388, 395, 402, 403, 406, 408, 413, 414, 415, 423, 425, 441, 507, 508, 536, 542, 822, 1088, 1099, 1206, 1269, 1272, 1275, 1278, 1279, 1280, 1311, 1348, 1407, 1410, 1434, 1493, 1578, 1579.
cur_dir: 652, 661, 664, 666, 670, 671, 675, 1515, 1516, 1524, 1525, 1527, 1530, 1532, 1539, 1541, 1545, 1546, 1547, 1548, 1549, 1550.
cur_ext: 547, 552, 564, 565, 572, 595, 596, 1314, 1329, 1406, 1437, 1445.
cur_f: 765, 767, 781, 785, 789, 790, 793, 795, 796, 797, 799, 801, 802, 803, 805.
cur_fam: 262, 1205, 1209, 1219.
cur_fam_code: 262, 263, 264, 1193, 1199.
cur_file: 334, 359, 392, 572, 573, 1566.
cur_font: 256, 258, 593, 594, 612, 1086, 1088, 1096, 1098, 1171, 1177, 1178, 1444, 1454, 1458, 1545.
cur_font_loc: 256, 258, 259, 260, 1271.
cur_g: 655, 663, 667, 672, 1509, 1531, 1532, 1533.
cur_glue: 655, 663, 667, 672, 1509, 1531, 1532, 1533.
cur_group: 300, 301, 302, 304, 311, 312, 848, 1116, 1117, 1118, 1119, 1121, 1122, 1123, 1184, 1185, 1194, 1196, 1245, 1246, 1247, 1248, 1254, 1471, 1475, 1489, 1588.
cur_h: 652, 653, 654, 655, 658, 660, 661, 664, 665, 666, 667, 670, 671, 675, 1426, 1427, 1430, 1524, 1526, 1529, 1530, 1531, 1532, 1534, 1535, 1540.
cur_h_offset: 1428, 1429.
cur_head: 818, 819, 820, 834, 847.
cur_height: 1024, 1026, 1027, 1028, 1029, 1030, 1425.
cur_i: 765, 766, 767, 781, 785, 793, 796, 797, 799.
cur_if: 329, 366, 524, 525, 530, 531, 1389, 1478, 1501, 1587, 1588.
cur_indent: 925, 937.
cur_input: 35, 36, 91, 331, 332, 341, 351, 352, 569, 1185, 1585, 1587.
cur_l: 961, 962, 963, 964, 965, 1086, 1088, 1089,

1090, 1091, 1093, 1094.
cur_lang: 939, 940, 977, 978, 984, 988, 993, 998, 1017, 1145, 1254, 1422, 1423, 1666, 1669.
cur_length: 41, 206, 208, 287, 289, 551, 560, 653, 656, 678, 734, 744, 1431, 1436.
cur_level: 300, 301, 302, 304, 307, 308, 310, 311, 1358, 1389, 1471, 1475, 1489, 1588, 1648, 1650.
cur_line: 925, 937, 938.
cur_list: 239, 242, 243, 244, 456, 1298, 1489.
cur_loop: 818, 819, 820, 825, 831, 840, 841, 842.
cur_mark: 326, 416, 420, 1389, 1620.
cur_mlist: 762, 763, 769, 798, 1248, 1250, 1253.
cur_mu: 746, 762, 773, 775, 814.
cur_name: 547, 552, 564, 565, 572, 1311, 1312, 1314, 1406, 1437, 1445, 1448.
cur_order: 396, 473, 481, 482, 489, 497.
cur_p: 871, 876, 877, 878, 881, 885, 887, 888, 893, 899, 900, 901, 903, 904, 905, 906, 907, 908, 910, 911, 913, 914, 915, 916, 917, 920, 925, 926, 927, 928, 929, 943, 956, 957, 1422, 1517, 1660, 1663.
cur_page_height: 1427, 1428, 1429.
cur_page_width: 1428, 1429.
cur_pre_head: 818, 819, 820, 834, 847.
cur_pre_tail: 818, 819, 820, 834, 844, 847.
cur_ptr: 420, 449, 461, 1088, 1278, 1280, 1281, 1291, 1626, 1627, 1630, 1631, 1632, 1635, 1636, 1638, 1641, 1642, 1650.
cur_q: 961, 962, 964, 965, 1088, 1089, 1090, 1091, 1094.
cur_r: 961, 962, 963, 964, 965, 1086, 1088, 1091, 1092, 1093, 1094.
cur_rh: 960, 962, 963, 964.
cur_s: 629, 652, 655, 667, 678, 680.
cur_size: 742, 743, 746, 762, 765, 766, 775, 779, 780, 788, 790, 791, 792, 793, 801, 802, 803, 810.
cur_span: 818, 819, 820, 835, 844, 846.
cur_style: 746, 762, 763, 769, 770, 773, 774, 777, 778, 780, 781, 786, 788, 789, 790, 792, 793, 794, 798, 800, 801, 802, 803, 805, 808, 810, 811, 814, 1248, 1250, 1253.
cur_tail: 818, 819, 820, 834, 844, 847.
cur_tok: 92, 311, 327, 355, 356, 357, 366, 394, 395, 396, 400, 401, 402, 403, 406, 409, 413, 414, 415, 426, 427, 428, 429, 431, 433, 437, 439, 441, 474, 475, 476, 478, 479, 482, 487, 509, 511, 512, 514, 518, 529, 538, 541, 831, 832, 1088, 1092, 1099, 1101, 1149, 1181, 1182, 1186, 1269, 1275, 1322, 1323, 1325, 1434, 1435, 1493, 1572, 1578, 1583, 1595, 1596.
cur_v: 652, 654, 655, 661, 662, 666, 667, 669, 670, 671, 673, 674, 675, 678, 1426, 1427, 1430.
cur_v_offset: 1428, 1429.
cur_val: 294, 295, 364, 396, 420, 444, 447, 448, 449, 453, 454, 455, 457, 458, 459, 460, 461, 463, 464, 465, 467, 468, 469, 470, 471, 472, 473, 474, 476, 478, 479, 481, 482, 483, 485, 486, 488, 490, 492, 493, 495, 496, 497, 498, 500, 501, 506, 507, 517, 526, 536, 538, 539, 544, 588, 612, 613, 614, 615, 684, 828, 830, 989, 1031, 1084, 1088, 1092, 1114, 1115, 1127, 1131, 1136, 1153, 1155, 1157, 1177, 1178, 1179, 1205, 1208, 1214, 1215, 1219, 1236, 1242, 1278, 1279, 1280, 1281, 1282, 1283, 1286, 1288, 1290, 1291, 1292, 1293, 1294, 1295, 1297, 1298, 1299, 1300, 1301, 1302, 1307, 1312, 1313, 1329, 1350, 1398, 1405, 1410, 1413, 1440, 1444, 1445, 1454, 1458, 1460, 1475, 1478, 1481, 1484, 1493, 1498, 1504, 1506, 1580, 1591, 1593, 1596, 1614, 1615, 1622, 1630, 1631, 1632, 1635, 1650, 1677.
cur_val_level: 396, 444, 447, 449, 453, 454, 455, 457, 458, 461, 463, 464, 473, 484, 486, 490, 496, 500, 501, 1410, 1484, 1591, 1593.
cur_val1: 444, 448, 1214.
cur_width: 925, 937.
current page: 1034.
current_character_being_worked_on: 605.
`\currentgrouplevel` primitive: 1473.
current_group_level_code: 1473, 1474, 1475.
`\currentgrouptype` primitive: 1473.
current_group_type_code: 1473, 1474, 1475.
`\currentifbranch` primitive: 1476.
current_if_branch_code: 1476, 1477, 1478.
`\currentiflevel` primitive: 1476.
current_if_level_code: 1476, 1477, 1478.
`\currentiftype` primitive: 1476.
current_if_type_code: 1476, 1477, 1478.
cv_backup: 396.
cvl_backup: 396.
d: 111, 198, 202, 203, 286, 371, 474, 595, 689, 710, 721, 749, 863, 878, 925, 998, 1024, 1122, 1140, 1192, 1252, 1493, 1555, 1608, 1610.
d_fixed: 644, 645.
danger: 1248, 1249, 1253.
data: 236, 258, 1271, 1286, 1288.
data structure assumptions: 186, 189, 230, 652, 864, 1022, 1035, 1343, 1544.
day: 262, 267, 653, 1382.
`\day` primitive: 264.
day_code: 262, 263, 264.
`dd`: 493.
deactivate: 877, 899, 902.
dead_cycles: 453, 628, 629, 676, 1066, 1078, 1079, 1108, 1296, 1300.
`\deadcycles` primitive: 450.

debug: 7, 9, 82, 88, 97, 136, 190, 191, 192, 197, 1085, 1392.
`debug #`: 1392.
debug_help: 82, 88, 97, 1392.
debugging: 7, 88, 100, 136, 190, 208, 1085, 1392.
decent_fit: 865, 882, 900, 901, 912, 1658, 1659.
decr: 16, 42, 44, 68, 75, 90, 92, 93, 94, 96, 106, 128, 142, 143, 145, 201, 203, 226, 227, 231, 243, 271, 287, 290, 311, 312, 341, 352, 354, 355, 356, 359, 361, 377, 386, 387, 390, 392, 428, 433, 456, 463, 476, 505, 512, 518, 529, 544, 569, 573, 603, 611, 637, 655, 667, 676, 680, 681, 759, 760, 851, 856, 877, 888, 906, 917, 931, 969, 970, 984, 985, 994, 998, 1002, 1019, 1114, 1174, 1181, 1185, 1228, 1240, 1248, 1298, 1347, 1365, 1389, 1391, 1417, 1420, 1489, 1493, 1501, 1539, 1544, 1550, 1565, 1567, 1585, 1586, 1587, 1588, 1591, 1593, 1630, 1632.
def: 235, 1262, 1263, 1264, 1267, 1272.
`\def` primitive: 1262.
def_code: 235, 447, 1264, 1284, 1285, 1286.
def_family: 235, 447, 612, 1264, 1284, 1285, 1288.
def_font: 235, 295, 296, 447, 612, 1264, 1310.
def_ref: 335, 336, 506, 508, 517, 1014, 1155, 1272, 1280, 1333, 1342, 1407, 1409, 1410, 1433, 1493.
default_code: 725, 739, 787, 1236.
default_hyphen_char: 262, 611, 744.
`\defaulthyphenchar` primitive: 264.
default_hyphen_char_code: 262, 263, 264.
default_rule: 498.
default_rule_thickness: 725, 743, 777, 778, 780, 787, 789, 803.
default_skew_char: 262, 611, 744.
`\defaultskewchar` primitive: 264.
default_skew_char_code: 262, 263, 264.
defecation: 633.
define: 1131, 1268, 1271, 1272, 1275, 1278, 1279, 1282, 1286, 1288, 1302, 1311, 1650.
define_mathex_accessor: 743.
define_mathex_body: 743.
define_mathex_end: 743.
define_mathsy_accessor: 742.
define_mathsy_body: 742.
define_mathsy_end: 742.
define_native_font: 621, 622, 638.
defining: 335, 336, 369, 508, 517.
del_code: 262, 266, 447, 448, 1214.
`\delcode` primitive: 1284.
del_code_base: 262, 266, 268, 447, 448, 1284, 1285, 1286, 1287.
delete_glue_ref: 227, 228, 305, 486, 500, 613, 775, 850, 864, 874, 929, 1030, 1050, 1058, 1071, 1076, 1154, 1283, 1290, 1293, 1389, 1591, 1593, 1601, 1602, 1605, 1614, 1615, 1632, 1649, 1664.
delete_last: 1158, 1159.
delete_q: 769, 808, 811.
delete_sa_ptr: 1630, 1632, 1636.
delete_sa_ref: 1632, 1645, 1650, 1651, 1652.
delete_token_ref: 226, 228, 305, 354, 506, 1031, 1033, 1066, 1070, 1389, 1410, 1418, 1637, 1638, 1639, 1641, 1642, 1649.
deletions_allowed: 80, 81, 88, 89, 102, 366, 376.
delim_num: 233, 295, 296, 1100, 1205, 1208, 1214.
delim_ptr: 238, 239, 1239, 1245.
delimited_code: 1232, 1233, 1236, 1237.
delimitedSubFormulaMinHeight: 742.
delimiter: 729, 738, 810, 1245.
`\delimiter` primitive: 295.
delimiter_factor: 262, 810.
`\delimiterfactor` primitive: 264.
delimiter_factor_code: 262, 263, 264.
delimiter_shortfall: 273, 810.
`\delimitershortfall` primitive: 274.
delimiter_shortfall_code: 273, 274.
delim1: 742, 792.
delim2: 742, 792.
delta: 107, 769, 771, 776, 778, 779, 780, 781, 786, 787, 789, 790, 791, 792, 793, 794, 798, 799, 800, 803, 810, 1048, 1062, 1064, 1177, 1179.
delta_node: 870, 878, 880, 891, 892, 908, 909, 913, 922, 923.
delta_node_size: 870, 891, 892, 908, 909, 913.
delta1: 787, 790, 810.
delta2: 787, 790, 810.
den: 621, 623, 626.
denom: 485, 493.
denom_style: 745, 788.
denominator: 725, 732, 739, 740, 788, 1235, 1239.
denom1: 742, 788.
denom2: 742, 788.
deplorable: 1028, 1059.
`depth`: 498.
depth: 157, 158, 160, 161, 162, 169, 170, 210, 213, 214, 498, 589, 660, 662, 664, 667, 669, 670, 673, 679, 689, 693, 698, 710, 712, 721, 730, 747, 749, 752, 756, 770, 773, 774, 778, 779, 780, 783, 789, 790, 791, 793, 794, 795, 800, 802, 803, 816, 817, 849, 854, 858, 877, 1027, 1056, 1063, 1064, 1075, 1141, 1154, 1419, 1420, 1424, 1425, 1426, 1428, 1445, 1529, 1556.
depth_base: 585, 587, 589, 601, 606, 744, 1376, 1377.
depth_index: 578, 589.
depth_offset: 157, 450, 817, 1301.

depth_threshold: 207, 208, 224, 259, 262, 734, 1393, 1634.
descent: 744.
dest: 169.
destroy_marks: 1389, 1636, 1642.
\detokenize primitive: 1496.
dField: 1445.
dig: 54, 68, 69, 71, 106, 487.
digit_sensed: 1014, 1015, 1016.
\dimexpr primitive: 1589.
dimen: 273, 461, 1062, 1064.
\dimen primitive: 445.
dimen_base: 246, 262, 273, 274, 275, 276, 277, 278, 1124, 1199.
\dimendef primitive: 1276.
dimen_def_code: 1276, 1277, 1278.
dimen_par: 273.
dimen_pars: 273.
dimen_val: 444, 445, 447, 449, 450, 451, 452, 454, 455, 458, 459, 461, 462, 463, 484, 490, 500, 1291, 1484, 1589, 1590, 1596, 1601, 1603, 1606, 1609, 1626, 1631, 1634, 1643.
dimen_val_limit: 1626, 1632, 1633, 1648, 1652.
Dimension too large: 495.
dirty Pascal: 3, 136, 197, 208, 212, 315, 860, 1385.
disc_break: 925, 928, 929, 930, 938.
disc_group: 299, 1171, 1172, 1173, 1471, 1489.
disc_node: 167, 172, 201, 209, 228, 232, 773, 809, 865, 867, 877, 904, 906, 914, 929, 957, 968, 1088, 1134, 1420.
disc_ptr: 1389, 1670, 1674.
disc_width: 887, 888, 917, 918.
discretionary: 234, 1144, 1168, 1169, 1170.
Discretionary list is too long: 1174.
\discretionary primitive: 1168.
Display math...with $$: 1251.
display_indent: 273, 848, 1192, 1199, 1253, 1555.
\displayindent primitive: 274.
display_indent_code: 273, 274, 1199.
\displaylimits primitive: 1210.
display_mlist: 731, 737, 740, 774, 805, 1228.
display_style: 730, 736, 774, 1223, 1253.
\displaystyle primitive: 1223.
\displaywidowpenalties primitive: 1675.
display_widow_penalties_loc: 256, 1675, 1676.
display_widow_penalties_ptr: 938, 1675.
display_widow_penalty: 262, 862, 938.
\displaywidowpenalty primitive: 264.
display_widow_penalty_code: 262, 263, 264.
display_width: 273, 1192, 1199, 1253, 1555.
\displaywidth primitive: 274.
display_width_code: 273, 274, 1199.
displayOperatorMinHeight: 742, 793.
div: 104, 665, 674.
divide: 235, 295, 296, 1264, 1289, 1290.
\divide primitive: 295.
dlist: 652, 855, 1248, 1256, 1514, 1524, 1525, 1556.
do_all_six: 871, 877, 880, 885, 891, 892, 908, 909, 912, 1024, 1041.
do_assignments: 848, 1177, 1260, 1324.
do_endv: 1184, 1185.
do_extension: 1402, 1403, 1438.
do_last_line_fit: 893, 894, 899, 900, 903, 911, 912, 1653, 1654, 1664.
do_locale_linebreaks: 744, 1088.
do_marks: 1031, 1066, 1389, 1636.
do_nothing: 16, 34, 57, 58, 88, 201, 228, 305, 374, 387, 506, 573, 604, 645, 647, 648, 660, 669, 691, 711, 734, 771, 776, 805, 809, 885, 914, 945, 952, 1099, 1290, 1410, 1420, 1436, 1460.
do_register_command: 1289, 1290.
do_size_requests: 1445.
doing_leaders: 628, 629, 666, 675, 1437.
doing_special: 59, 61, 62, 1431.
done: 15, 47, 53, 228, 311, 312, 341, 414, 423, 431, 474, 479, 482, 488, 493, 508, 509, 511, 517, 518, 529, 561, 565, 566, 572, 595, 602, 611, 651, 676, 678, 679, 740, 744, 769, 781, 784, 808, 809, 822, 825, 863, 877, 885, 911, 921, 925, 929, 944, 960, 963, 965, 985, 1014, 1015, 1024, 1028, 1031, 1033, 1048, 1051, 1052, 1059, 1133, 1134, 1135, 1164, 1173, 1175, 1192, 1200, 1265, 1281, 1306, 1410, 1418, 1489, 1533, 1540, 1547, 1548, 1549, 1572, 1610, 1674.
done_with_noad: 769, 770, 771, 776, 798.
done_with_node: 769, 770, 773, 774, 798.
done1: 15, 192, 193, 423, 433, 482, 487, 508, 509, 781, 785, 822, 831, 863, 877, 900, 925, 927, 943, 945, 949, 952, 1014, 1019, 1048, 1051, 1054, 1356, 1369.
done2: 15, 192, 194, 482, 493, 494, 508, 513, 822, 832, 863, 949, 1356, 1370.
done3: 15, 863, 946, 950, 951.
done4: 15, 863, 952.
done5: 15, 863, 914, 917.
done6: 15, 863, 945.
dont_expand: 236, 285, 387, 401.
double: 115, 117, 123.
Double subscript: 1231.
Double superscript: 1231.
double_hyphen_demerits: 262, 907.
\doublehyphendemerits primitive: 264.
double_hyphen_demerits_code: 262, 263, 264.
Doubly free location...: 194.

down_ptr: 641, 642, 643, 651.
downdate_width: 908.
down1: 621, 622, 643, 645, 646, 649, 650, 652.
down2: 621, 630, 646.
down3: 621, 646.
down4: 621, 646.
\dp primitive: 450.
dry rot: 99.
\dump...only by INITEX: 1389.
\dump primitive: 1106.
dump_four_ASCII: 1363.
dump_hh: 1359, 1372, 1378.
dump_int: 1359, 1361, 1363, 1365, 1367, 1369, 1370, 1372, 1374, 1376, 1378, 1380, 1464.
dump_qqqq: 1359, 1363, 1376.
dump_wd: 1359, 1365, 1369, 1370, 1374.
Duplicate pattern: 1017.
dvi_buf: 630, 631, 633, 634, 643, 649, 650.
dvi_buf_size: 11, 14, 630, 631, 632, 634, 635, 643, 649, 650, 680.
dvi_f: 652, 653, 658, 659, 1426, 1430.
dvi_file: 567, 628, 631, 633, 678, 680.
DVI files: 619.
dvi_font_def: 638, 659, 681.
dvi_four: 636, 638, 646, 653, 662, 671, 678, 680, 1426, 1430, 1431, 1436.
dvi_gone: 630, 631, 632, 634, 648.
dvi_h: 652, 653, 655, 658, 661, 662, 666, 667, 670, 675, 1426, 1430.
dvi_index: 630, 631, 633.
dvi_limit: 630, 631, 632, 634, 635.
dvi_native_font_def: 638.
dvi_offset: 630, 631, 632, 634, 637, 641, 643, 649, 650, 655, 667, 678, 680.
dvi_open_out: 567.
dvi_out: 634, 636, 637, 638, 639, 645, 646, 653, 655, 658, 659, 662, 667, 671, 678, 680, 1426, 1430, 1431, 1436.
dvi_pop: 637, 655, 667.
dvi_ptr: 630, 631, 632, 634, 635, 637, 643, 655, 667, 678, 680.
dvi_swap: 634.
dvi_two: 636, 1426, 1430.
dvi_v: 652, 653, 655, 661, 666, 667, 670, 675, 1426, 1430.
dyn_used: 139, 142, 143, 144, 145, 189, 677, 1365, 1366.
D2Fix: 1445.
e: 307, 309, 533, 553, 554, 565, 1252, 1265, 1290, 1471, 1472, 1555, 1593, 1650, 1651.
easy_line: 867, 883, 895, 896, 898.
ec: 575, 576, 578, 580, 595, 600, 601, 605, 611.
\edef primitive: 1262.
edge: 655, 661, 664, 667, 673, 1430.
edge_dist: 1529, 1530, 1532, 1540.
edge_node: 652, 1529, 1530, 1535, 1547.
edge_node_size: 1529.
eight_bit_p: 59.
eight_bits: 25, 68, 134, 327, 584, 595, 618, 631, 643, 689, 749, 752, 755, 1046, 1047.
eject_penalty: 182, 877, 879, 899, 907, 921, 1024, 1026, 1028, 1059, 1064, 1065.
el_gordo: 115, 116, 118.
\elapsedtime primitive: 450.
elapsed_time_code: 450, 451, 458.
else: 10.
\else primitive: 526.
else_code: 524, 526, 533, 1478.
em: 490.
EMBOLDEN: 621.
embolden: 621.
Emergency stop: 97.
emergency_stretch: 273, 876, 911.
\emergencystretch primitive: 274.
emergency_stretch_code: 273, 274.
empty: 16, 455, 723, 727, 729, 734, 765, 766, 781, 793, 795, 796, 798, 799, 800, 807, 1034, 1040, 1041, 1045, 1055, 1062, 1230, 1231, 1240.
empty line at end of file: 521, 573.
empty_field: 726, 727, 728, 786, 1217, 1219, 1235.
empty_flag: 146, 148, 152, 174, 189, 1366.
encoding: 1446, 1447.
end: 7, 8, 10.
End of file on the terminal: 37, 75.
(\end occurred...): 1389.
\end primitive: 1106.
end_cs_name: 234, 295, 296, 406, 1188, 1578.
\endcsname primitive: 295.
end_diagnostic: 271, 314, 329, 353, 434, 435, 537, 544, 595, 616, 676, 679, 705, 717, 744, 874, 911, 1041, 1046, 1060, 1065, 1175, 1352, 1472, 1634.
end_file_reading: 359, 360, 390, 392, 518, 572, 1389.
end_graf: 1080, 1139, 1148, 1150, 1154, 1185, 1187, 1222.
end_group: 234, 295, 296, 1117.
\endgroup primitive: 295.
\endinput primitive: 410.
end_L_code: 171, 1511, 1512, 1515, 1545.
end_line_char: 91, 262, 266, 333, 348, 362, 390, 392, 518, 569, 573, 1391.
\endlinechar primitive: 264.
end_line_char_code: 262, 263, 264.
end_line_char_inactive: 390, 392, 518, 573, 1391.

end_LR: 171, 218, 1518, 1521, 1527, 1539, 1548, 1550.
end_LR_type: 171, 1515, 1518, 1521, 1527, 1539, 1548, 1550.
end_M: 1134.
end_M_code: 171, 458, 1515, 1557.
end_match: 233, 319, 321, 324, 425, 426, 428.
end_match_token: 319, 423, 425, 426, 427, 428, 509, 511, 517.
end_name: 547, 552, 561, 566.
end_node_run: 655, 656.
end_of_TEX: 6, 85, 1386.
end_R_code: 171, 1511, 1515.
end_reflect: 1510.
end_span: 187, 816, 827, 841, 845, 849, 851.
end_template: 236, 396, 409, 414, 828, 1349, 1583.
end_template_token: 828, 832, 838.
end_token_list: 354, 355, 387, 424, 1080, 1389, 1434.
end_write: 248, 1432, 1434.
`\endwrite`: 1432.
end_write_token: 1434, 1435.
endcases: 10.
`\endL` primitive: 1511.
`\endR` primitive: 1511.
`endtemplate`: 828.
endv: 233, 328, 409, 414, 816, 828, 830, 839, 1100, 1184, 1185.
ensure_dvi_open: 567, 653.
ensure_vbox: 1047, 1063, 1072.
eof: 26, 31, 52, 599, 610, 1381.
eof_seen: 358, 392, 1470.
eoln: 31, 52.
eop: 619, 621, 622, 624, 678, 680.
epochseconds: 682, 1391, 1411, 1412, 1414.
eq_define: 307, 308, 309, 406, 830, 1124, 1268.
eq_destroy: 305, 307, 309, 313.
eq_level: 247, 248, 254, 258, 262, 279, 285, 294, 307, 309, 313, 828, 1031, 1369, 1432, 1631, 1632.
eq_level_field: 247.
eq_no: 234, 1194, 1195, 1197, 1198, 1489.
`\eqno` primitive: 1195.
eq_save: 306, 307, 308.
eq_type: 236, 247, 248, 249, 254, 258, 279, 285, 294, 295, 297, 307, 309, 381, 383, 384, 387, 388, 406, 423, 425, 828, 1206, 1369, 1432, 1578.
eq_type_field: 247, 305.
eq_word_define: 308, 309, 1124, 1193, 1199, 1268.
eq_word_define1: 1268.
eqtb: 2, 137, 188, 246, 247, 248, 249, 250, 254, 256, 258, 262, 266, 268, 273, 276, 277, 278, 279, 281, 283, 292, 294, 295, 296, 297, 298, 300, 302, 304, 305, 306, 307, 308, 309, 311, 312, 313, 314, 315, 316, 319, 321, 327, 328, 335, 337, 362, 363, 384, 423, 447, 448, 508, 526, 583, 588, 744, 828, 862, 1242, 1262, 1276, 1291, 1307, 1311, 1369, 1370, 1371, 1393, 1400, 1452, 1634, 1646.
eqtb_size: 246, 273, 276, 278, 279, 280, 1361, 1362, 1370, 1371.
equiv: 247, 248, 249, 250, 254, 255, 256, 258, 259, 260, 261, 279, 281, 285, 294, 295, 297, 305, 307, 309, 381, 383, 384, 387, 388, 447, 448, 449, 543, 612, 828, 1206, 1281, 1286, 1291, 1343, 1369, 1432, 1468, 1675, 1677.
equiv_field: 247, 305, 315, 1645.
err_help: 83, 256, 1337, 1338.
`\errhelp` primitive: 256.
err_help_loc: 256.
`\errmessage` primitive: 1331.
error: 76, 79, 80, 82, 83, 86, 92, 95, 97, 102, 125, 357, 368, 376, 404, 432, 442, 447, 448, 452, 462, 479, 489, 491, 494, 495, 506, 510, 511, 521, 535, 545, 558, 570, 596, 602, 614, 679, 766, 824, 832, 840, 874, 990, 991, 1014, 1015, 1016, 1017, 1030, 1032, 1046, 1058, 1063, 1078, 1081, 1104, 1118, 1120, 1122, 1123, 1134, 1136, 1149, 1153, 1160, 1164, 1174, 1175, 1182, 1183, 1189, 1213, 1220, 1231, 1237, 1246, 1249, 1267, 1279, 1286, 1290, 1291, 1295, 1306, 1313, 1337, 1338, 1347, 1435, 1445, 1446, 1457, 1458, 1466, 1593.
error_context_lines: 262, 341.
`\errorcontextlines` primitive: 264.
error_context_lines_code: 262, 263, 264.
error_count: 80, 81, 86, 90, 1150, 1347.
error_line: 11, 14, 54, 58, 336, 341, 345, 346, 347.
error_message_issued: 80, 86, 99.
error_stop_mode: 76, 77, 78, 86, 87, 97, 102, 1316, 1337, 1347, 1348, 1351, 1381, 1389, 1506.
`\errorstopmode` primitive: 1316.
erstat: 27.
escape: 233, 258, 374, 1391.
escape_char: 262, 266, 269.
`\escapechar` primitive: 264.
escape_char_code: 262, 263, 264.
`ETC`: 322.
`etc`: 208.
eTeX_aux: 238, 239, 241, 242.
eTeX_aux_field: 238, 239, 1489.
etex_convert_base: 503.
etex_convert_codes: 503.
eTeX_dim: 450, 1479, 1482, 1612.
eTeX_enabled: 1466, 1513.
eTeX_ex: 210, 304, 307, 308, 312, 356, 571, 616, 663, 676, 1199, 1265, 1266, 1267, 1365, 1366,

1389, 1391, 1462, 1465, 1524, 1525, 1526, 1545.
eTeX_expr: 450, 1589, 1590, 1591.
eTeX_glue: 450, 458, 1616.
eTeX_int: 450, 1452, 1473, 1476, 1612.
etex_int_base: 262.
etex_int_pars: 262.
eTeX_mode: 1451, 1462, 1463, 1464, 1465.
eTeX_mu: 450, 1591, 1616.
etex_pen_base: 256, 258, 259.
etex_pens: 256, 258, 259.
eTeX_revision: 2, 507.
\eTeXrevision primitive: 1452.
eTeX_revision_code: 503, 504, 506, 507, 1452.
eTeX_state: 1452, 1510.
eTeX_state_base: 1452, 1511.
eTeX_state_code: 262, 1452, 1510.
eTeX_states: 2, 262.
eTeX_text_offset: 337.
etex_toks: 256.
etex_toks_base: 256.
eTeX_version: 2, 1454.
\eTeXversion primitive: 1452.
eTeX_version_code: 450, 1452, 1453, 1454.
eTeX_version_string: 2.
every_cr: 256, 822, 847.
\everycr primitive: 256.
every_cr_loc: 256, 257.
every_cr_text: 337, 344, 822, 847.
every_display: 256, 1199.
\everydisplay primitive: 256.
every_display_loc: 256, 257.
every_display_text: 337, 344, 1199.
every_eof: 392, 1468.
\everyeof primitive: 1467.
every_eof_loc: 256, 337, 1467, 1468.
every_eof_text: 337, 344, 392.
every_hbox: 256, 1137.
\everyhbox primitive: 256.
every_hbox_loc: 256, 257.
every_hbox_text: 337, 344, 1137.
every_job: 256, 1084.
\everyjob primitive: 256.
every_job_loc: 256, 257.
every_job_text: 337, 344, 1084.
every_math: 256, 1193.
\everymath primitive: 256.
every_math_loc: 256, 257.
every_math_text: 337, 344, 1193.
every_par: 256, 1145.
\everypar primitive: 256.
every_par_loc: 256, 257, 337, 1280.
every_par_text: 337, 344, 1145.
every_vbox: 256, 1137, 1221.
\everyvbox primitive: 256.
every_vbox_loc: 256, 257.
every_vbox_text: 337, 344, 1137, 1221.
ex: 490.
ex_hyphen_penalty: 167, 262, 917.
\exhyphenpenalty primitive: 264.
ex_hyphen_penalty_code: 262, 263, 264.
ex_space: 234, 295, 296, 1084, 1144.
exactly: 683, 684, 758, 937, 1031, 1071, 1116, 1255, 1490.
exit: 15, 16, 37, 47, 58, 59, 63, 73, 86, 126, 147, 208, 307, 308, 322, 371, 423, 441, 447, 496, 500, 532, 533, 559, 618, 643, 651, 689, 710, 796, 839, 877, 944, 988, 998, 1002, 1031, 1048, 1066, 1084, 1108, 1133, 1159, 1164, 1167, 1173, 1205, 1213, 1228, 1265, 1290, 1324, 1357, 1389, 1392, 1471, 1583, 1630, 1632.
expand: 388, 396, 400, 402, 405, 414, 415, 473, 502, 513, 533, 545, 830, 1492, 1583.
expand_after: 236, 295, 296, 396, 399, 1573.
\expandafter primitive: 295.
expand_depth: 1593.
expand_depth_count: 1593.
\expanded primitive: 503.
expanded_code: 503, 504, 506.
explicit: 179, 760, 885, 914, 916, 927, 1112, 1167, 1521.
expr_a: 1603, 1605.
expr_add: 1594, 1595.
expr_add_sub: 1603.
expr_d: 1607.
expr_div: 1594, 1595, 1606, 1607.
expr_e_field: 1599, 1600.
expr_m: 1606.
expr_mult: 1594, 1595, 1606.
expr_n_field: 1599, 1600.
expr_node_size: 1599, 1600.
expr_none: 1594, 1595, 1602, 1603.
expr_s: 1609.
expr_scale: 1594, 1606, 1609.
expr_sub: 1594, 1595, 1601, 1603.
expr_t_field: 1599, 1600.
ext_bot: 581, 756, 757.
ext_delimiter: 548, 550, 551, 552.
ext_mid: 581, 756, 757.
ext_rep: 581, 756, 757.
ext_tag: 579, 604, 751, 753.
ext_top: 581, 756, 757.
exten: 579.
exten_base: 585, 587, 601, 608, 609, 611, 756, 1376, 1377.

extend: 621.
EXTEND: 621.
extensible_recipe: 576, 581.
extension: 234, 1398, 1399, 1401, 1402, 1438, 1444, 1511.
extensions to TeX: 2, 168, 1394.
Extra \else: 545.
Extra \endcsname: 1189.
Extra \fi: 545.
Extra \middle.: 1246.
Extra \or: 535, 545.
Extra \right.: 1246.
Extra }, or forgotten x: 1123.
Extra alignment tab...: 840.
Extra x: 1120.
extra_info: 817, 836, 837, 839, 840.
extra_right_brace: 1122, 1123.
extra_space: 582, 593, 744, 1098.
extra_space_code: 582, 593.
eyes and mouth: 362.
f: 27, 28, 31, 116, 118, 166, 482, 560, 595, 612, 613, 616, 618, 628, 638, 688, 689, 742, 743, 744, 749, 752, 754, 755, 758, 759, 760, 781, 878, 910, 1122, 1167, 1177, 1192, 1265, 1311, 1593, 1610.
false: 27, 31, 37, 45, 46, 47, 51, 58, 59, 62, 80, 84, 92, 93, 102, 110, 111, 116, 119, 191, 192, 193, 194, 198, 294, 304, 311, 314, 329, 341, 353, 357, 358, 361, 366, 376, 391, 392, 395, 398, 408, 434, 435, 441, 449, 459, 461, 474, 475, 479, 481, 482, 483, 484, 490, 495, 496, 497, 500, 520, 536, 537, 540, 542, 544, 547, 551, 559, 561, 563, 573, 586, 595, 616, 629, 744, 749, 763, 765, 798, 822, 839, 874, 876, 877, 885, 899, 902, 911, 929, 935, 956, 960, 964, 965, 1005, 1008, 1014, 1015, 1016, 1017, 1020, 1022, 1041, 1044, 1060, 1065, 1074, 1075, 1080, 1085, 1087, 1088, 1089, 1094, 1105, 1108, 1115, 1134, 1150, 1155, 1221, 1236, 1237, 1245, 1246, 1248, 1253, 1280, 1281, 1290, 1312, 1324, 1333, 1336, 1337, 1342, 1357, 1379, 1390, 1396, 1397, 1407, 1409, 1410, 1431, 1434, 1437, 1442, 1445, 1451, 1466, 1472, 1492, 1567, 1580, 1585, 1587, 1593, 1604, 1608, 1610, 1631, 1632, 1634, 1635, 1654, 1657, 1664, 1666, 1667.
false_bchar: 1086, 1088, 1092.
fam: 723, 724, 725, 729, 733, 765, 766, 796, 797, 1205.
\fam primitive: 264.
fam_fnt: 256, 742, 743, 750, 765, 780, 1249.
fam_in_range: 1205, 1209, 1219.
fast_delete_glue_ref: 227, 228, 1509.
fast_get_avail: 144, 405, 1088, 1092.
fast_store_new_token: 405, 433, 499, 501.
Fatal format file error: 1357.
fatal_error: 75, 97, 354, 390, 519, 565, 570, 830, 837, 839, 1185.
fatal_error_stop: 80, 81, 86, 97, 1386.
fbyte: 599, 603, 606, 610.
featLen: 744.
featureNameP: 744.
Ferguson, Michael John: 2.
fetch: 765, 767, 781, 785, 793, 796, 799, 805.
fetch_box: 454, 506, 540, 1031, 1133, 1164, 1301, 1350, 1631.
fetch_effective_tail: 1134, 1135, 1159.
fetch_effective_tail_eTeX: 1134.
fewest_demerits: 920, 922, 923.
fflush: 678.
fget: 599, 600, 603, 606, 610.
\fi primitive: 526.
fi_code: 524, 526, 527, 529, 533, 535, 544, 545, 1478, 1501, 1588.
fi_or_else: 236, 329, 396, 399, 524, 526, 527, 529, 545, 1347.
fil: 489.
fil: 157, 174, 189, 203, 489, 690, 701, 707, 1255.
fil_code: 1112, 1113, 1114.
fil_glue: 187, 189, 1114.
fil_neg_code: 1112, 1114.
fil_neg_glue: 187, 189, 1114.
File ended while scanning...: 368.
File ended within \read: 521.
file_name_quote_char: 548, 550, 596.
file_name_size: 11, 26, 554, 557, 558, 560.
file_offset: 54, 55, 57, 58, 66, 572, 676, 1334, 1566.
file_opened: 595, 596, 598.
file_warning: 392, 1588.
fill: 157, 174, 189, 690, 701, 707, 1255.
fill_code: 1112, 1113, 1114.
fill_glue: 187, 189, 1108, 1114.
fill_width: 1653, 1654, 1657.
filll: 157, 174, 203, 489, 690, 701, 707, 1255, 1509.
fin_align: 821, 833, 848, 1185.
fin_col: 821, 839, 1185.
fin_mlist: 1228, 1238, 1240, 1245, 1248.
fin_row: 821, 847, 1185.
fin_rule: 655, 660, 664, 667, 669, 673.
final_cleanup: 1386, 1387, 1389, 1636.
final_end: 6, 35, 361, 1386, 1391.
final_hyphen_demerits: 262, 907.
\finalhyphendemerits primitive: 264.
final_hyphen_demerits_code: 262, 263, 264.
final_pass: 876, 902, 911, 921.
find_effective_tail: 458.
find_effective_tail_eTeX: 458, 1134.

find_font_dimen: 459, 613, 1096, 1307.
find_native_font: 744.
find_pic_file: 1084, 1445.
find_protchar_left: 181, 877, 935.
find_protchar_right: 181, 877, 929.
find_sa_element: 449, 461, 1088, 1278, 1280, 1281, 1291, 1627, 1630, 1631, 1632, 1635, 1638, 1641, 1650.
fingers: 546.
finite_shrink: 873, 874.
fire_up: 1059, 1066, 1620, 1636, 1639.
fire_up_done: 1066, 1636, 1640.
fire_up_init: 1066, 1636, 1639.
firm_up_the_line: 370, 392, 393, 573.
first: 30, 31, 35, 36, 37, 75, 87, 91, 92, 294, 358, 359, 361, 390, 392, 393, 408, 518, 566, 573, 1390, 1567, 1579.
first_child: 1014, 1017, 1018, 1666, 1667.
first_count: 54, 345, 346, 347.
first_fit: 1007, 1011, 1020, 1668.
first_indent: 895, 897, 937.
first_mark: 416, 417, 1066, 1070, 1620, 1639.
`\firstmark` primitive: 418.
first_mark_code: 416, 418, 419, 1620.
`\firstmarks` primitive: 1620.
first_math_fontdimen: 744.
first_p: 181, 877, 911.
first_text_char: 19.
first_width: 895, 897, 898, 937.
firstMathValueRecord: 742.
fit_class: 878, 884, 893, 894, 900, 901, 903, 907, 1658, 1659, 1661, 1662.
fitness: 867, 893, 907, 912.
fix_date_and_time: 267, 1386, 1391.
fix_language: 1088, 1439.
fix_word: 576, 577, 582, 583, 606.
fixed_acc: 729, 1219.
Fix2D: 1445.
flags: 621.
flattenedAccentBaseHeight: 742.
float: 113, 136, 212, 656, 663, 672, 749, 857, 1445.
float_constant: 113, 212, 655, 663, 667, 1177, 1179.
float_cost: 162, 214, 1062, 1154.
floating_penalty: 162, 262, 1122, 1154.
`\floatingpenalty` primitive: 264.
floating_penalty_code: 262, 263, 264.
flush_char: 42, 206, 221, 734, 737.
flush_list: 145, 226, 354, 406, 430, 441, 849, 956, 1014, 1150, 1333, 1351, 1433, 1498, 1564, 1578.
flush_math: 761, 824, 1249.
flush_node_list: 225, 228, 305, 656, 677, 740, 761, 774, 775, 786, 848, 864, 927, 931, 956, 957, 972, 1022, 1031, 1046, 1053, 1077, 1080, 1088, 1132, 1134, 1159, 1174, 1175, 1260, 1389, 1420, 1438, 1538, 1546, 1549, 1554, 1649.
flush_str: 1410.
flush_string: 44, 294, 506, 572, 744, 1314, 1333, 1382, 1410, 1564.
flushable: 506, 1410.
flushable_string: 1311, 1314.
fm: 1133, 1134, 1159.
fmem_ptr: 459, 584, 587, 601, 604, 605, 611, 613, 614, 615, 744, 1374, 1375, 1377, 1388.
fmt_file: 559, 1359, 1360, 1362, 1381, 1382, 1383, 1391.
fnt: 505, 1460, 1461.
fnt_def1: 621, 622, 638.
fnt_def2: 621.
fnt_def3: 621.
fnt_def4: 621.
fnt_num_0: 621, 622, 659.
fnt1: 621, 622, 659.
fnt2: 621.
fnt3: 621.
fnt4: 621.
font: 156, 165, 166, 200, 202, 219, 232, 297, 583, 618, 658, 688, 694, 723, 752, 758, 767, 889, 890, 914, 915, 918, 919, 949, 950, 951, 956, 962, 965, 1088, 1092, 1167, 1201, 1534, 1544.
font metric files: 574.
font parameters: 742, 743.
`Font x has only...`: 614.
`Font x=xx not loadable...`: 596.
`Font x=xx not loaded...`: 602.
`\font` primitive: 295.
font_area: 584, 587, 611, 638, 639, 744, 1314, 1376, 1377.
font_base: 11, 12, 133, 156, 200, 202, 248, 258, 583, 586, 638, 659, 681, 744, 1314, 1374, 1375, 1388.
font_bc: 584, 587, 611, 618, 744, 751, 765, 1090, 1376, 1377, 1454, 1481, 1580.
font_bchar: 584, 587, 611, 950, 951, 969, 1086, 1088, 1376, 1377.
font_biggest: 12.
`\fontchardp` primitive: 1479.
font_char_dp_code: 1479, 1480, 1481.
`\fontcharht` primitive: 1479.
font_char_ht_code: 1479, 1480, 1481.
`\fontcharic` primitive: 1479.
font_char_ic_code: 1479, 1480, 1481.
`\fontcharwd` primitive: 1479.
font_char_wd_code: 1479, 1480, 1481.
font_check: 584, 603, 638, 744, 1376, 1377.
font_def_length: 638.

\fontdimen primitive: 295.
font_dsize: 507, 584, 587, 603, 638, 744, 1314, 1315, 1376, 1377.
font_ec: 584, 587, 611, 618, 744, 751, 765, 1090, 1376, 1377, 1454, 1481, 1580.
font_engine: 744.
font_false_bchar: 584, 587, 611, 1086, 1088, 1376, 1377.
font_feature_warning: 744.
font_flags: 744.
font_glue: 584, 587, 611, 613, 656, 744, 1088, 1096, 1376, 1377.
font_id_base: 248, 260, 282, 449, 583, 1311.
font_id_text: 201, 260, 282, 297, 614, 1311, 1376, 1416.
font_in_short_display: 199, 200, 201, 219, 705, 912, 1393.
font_index: 583, 584, 595, 960, 1086, 1265.
font_info: 11, 459, 583, 584, 585, 587, 589, 592, 593, 595, 601, 604, 606, 608, 609, 610, 613, 615, 742, 743, 744, 756, 785, 796, 963, 1086, 1093, 1096, 1265, 1307, 1374, 1375, 1393.
font_layout_engine: 584, 744, 1454, 1461.
font_letter_space: 656, 744.
font_mapping: 611, 658, 744, 1088.
font_mapping_warning: 744.
font_max: 11, 133, 200, 202, 583, 586, 601, 744, 1375, 1388.
font_mem_size: 11, 583, 601, 615, 744, 1375, 1388.
font_name: 507, 584, 587, 611, 616, 639, 744, 1314, 1315, 1376, 1377, 1457.
\fontname primitive: 503.
font_name_code: 503, 504, 506, 507.
font_name_str: 328, 505, 507, 1315.
font_params: 584, 587, 611, 613, 614, 615, 744, 1249, 1376, 1377.
font_ptr: 584, 587, 601, 611, 613, 681, 744, 1314, 1374, 1375, 1388.
font_size: 507, 584, 587, 603, 638, 744, 1314, 1315, 1376, 1377.
font_slant: 744.
font_used: 584, 586, 659, 681, 744.
fontdimen: 742.
FONTx: 1311.
for accent: 217.
Forbidden control sequence...: 368.
force_eof: 361, 391, 392, 412.
format_area_length: 555, 559.
format_default_length: 555, 557, 558, 559.
format_ext_length: 555, 558, 559.
format_extension: 555, 564, 1382.
format_ident: 35, 65, 571, 1353, 1354, 1355, 1380, 1381, 1382, 1391, 1451.
forward: 82, 244, 311, 370, 396, 443, 654, 734, 735, 763, 822, 848, 1455, 1492, 1505, 1563, 1592, 1597, 1621.
found: 15, 147, 150, 151, 286, 289, 371, 384, 386, 423, 426, 428, 482, 490, 508, 510, 512, 559, 643, 645, 648, 649, 650, 684, 749, 751, 763, 783, 793, 877, 899, 944, 977, 985, 988, 995, 1007, 1009, 1192, 1200, 1201, 1202, 1290, 1291, 1421, 1489, 1493, 1544, 1549, 1550, 1593, 1594, 1600, 1610, 1658, 1659.
found1: 15, 944, 955, 1356, 1369, 1489, 1610, 1611.
found2: 15, 944, 956, 1356, 1370, 1489.
four_choices: 135.
four_quarters: 135, 447, 583, 584, 589, 590, 595, 689, 725, 726, 749, 752, 755, 767, 781, 793, 960, 1086, 1177, 1356, 1357, 1564, 1567.
fract: 1609, 1610, 1657.
fraction: 114, 116.
fraction_four: 114, 115, 120, 123, 124.
fraction_half: 115, 120, 131.
fraction_noad: 114, 725, 729, 732, 740, 776, 809, 1232, 1235.
fraction_noad_size: 725, 740, 809, 1235.
fraction_one: 114, 115, 116, 117, 118, 128, 129.
fraction_rule: 747, 748, 778, 791.
fractionDenomDisplayStyleGapMin: 742, 790.
fractionDenominatorDisplayStyleShiftDown: 742.
fractionDenominatorGapMin: 742, 790.
fractionDenominatorShiftDown: 742.
fractionNumDisplayStyleGapMin: 742, 790.
fractionNumeratorDisplayStyleShiftUp: 742.
fractionNumeratorGapMin: 742, 790.
fractionNumeratorShiftUp: 742.
fractionRuleThickness: 742.
free: 190, 192, 193, 194, 195, 196.
free_avail: 143, 228, 230, 243, 434, 487, 506, 820, 969, 1090, 1280, 1342, 1493, 1515, 1568.
free_native_glyph_info: 169, 1418.
free_node: 152, 227, 228, 305, 531, 651, 697, 740, 744, 758, 764, 770, 781, 783, 793, 795, 797, 799, 800, 808, 820, 851, 908, 909, 913, 956, 964, 1031, 1073, 1075, 1076, 1088, 1091, 1154, 1164, 1240, 1241, 1255, 1389, 1418, 1532, 1535, 1538, 1540, 1549, 1556, 1567, 1568, 1600, 1632, 1636, 1652.
free_ot_assembly: 749, 781, 793.
freeze_page_specs: 1041, 1055, 1062.
frozen_control_sequence: 248, 285, 1269, 1368, 1372, 1373.
frozen_cr: 248, 369, 828, 1186.

frozen_dont_expand: 248, 285, 401.
frozen_end_group: 248, 295, 1119.
frozen_end_template: 248, 409, 828.
frozen_endv: 248, 409, 414, 828.
frozen_fi: 248, 366, 526.
frozen_null_font: 248, 292, 293, 588.
frozen_primitive: 248, 285, 402, 474.
frozen_protection: 248, 1269, 1270.
frozen_relax: 248, 295, 403, 413.
frozen_right: 248, 1119, 1242.
Fuchs, David Raymond: 2, 619, 627.
full_name: 744.
\futurelet primitive: 1273.
g: 47, 208, 595, 628, 689, 710, 749, 759, 781, 793, 1655.
g_order: 655, 656, 663, 667, 672, 1509, 1533.
g_sign: 655, 656, 663, 667, 672, 1509, 1533.
garbage: 187, 502, 505, 506, 1014, 1237, 1246, 1333.
\gdef primitive: 1262.
geq_define: 309, 830, 1268.
geq_word_define: 309, 318, 1067, 1268.
geq_word_define1: 1268.
get: 26, 29, 31, 33, 520, 573, 599, 1360.
get_avail: 142, 144, 230, 231, 242, 355, 356, 367, 369, 401, 402, 405, 406, 487, 508, 517, 618, 752, 820, 831, 832, 842, 962, 965, 992, 1118, 1119, 1272, 1280, 1434, 1493, 1498, 1515, 1538, 1544, 1565, 1578.
get_cp_code: 460, 688.
get_encoding_mode_and_info: 1446, 1447.
get_font_char_range: 1454.
get_glyph_bounds: 1458.
get_input_normalization_state: 744.
get_microinterval: 458, 1411.
get_native_char: 656, 947, 957, 1088, 1415, 1420, 1430.
get_native_char_height_depth: 1179.
get_native_char_sidebearings: 1177.
get_native_glyph: 781, 799, 805.
get_native_glyph_italic_correction: 1167.
get_native_italic_correction: 1167.
get_native_mathex_param: 743.
get_native_mathsy_param: 742.
get_native_usv: 946, 949.
get_native_word_cp: 688.
get_next: 80, 327, 362, 366, 370, 371, 387, 390, 394, 395, 396, 401, 414, 415, 421, 423, 513, 529, 536, 542, 683, 1088, 1092, 1099, 1180, 1577.
get_node: 147, 153, 158, 161, 166, 167, 171, 175, 176, 177, 180, 183, 232, 530, 643, 688, 689, 710, 728, 730, 731, 744, 749, 759, 781, 799, 820, 846, 891, 892, 893, 912, 968, 1063, 1154, 1155, 1217, 1219, 1235, 1302, 1303, 1404, 1417, 1509, 1529, 1544, 1565, 1599, 1626, 1631, 1648.
get_ot_assembly_ptr: 751, 783, 793.
get_ot_math_accent_pos: 781.
get_ot_math_constant: 744, 780, 781, 789, 790, 793, 801, 802, 803.
get_ot_math_ital_corr: 793, 799.
get_ot_math_kern: 804, 806, 807.
get_ot_math_variant: 751, 783, 793.
get_preamble_token: 830, 831, 832.
get_r_token: 1269, 1272, 1275, 1278, 1279, 1311.
get_sa_ptr: 1630, 1636, 1642.
get_strings_started: 47, 51, 1386.
get_token: 80, 82, 92, 394, 395, 400, 401, 402, 403, 426, 433, 476, 487, 506, 508, 509, 511, 512, 514, 518, 830, 1081, 1192, 1269, 1275, 1306, 1322, 1325, 1348, 1434, 1435, 1493, 1576, 1583.
get_tracing_fonts_state: 744.
get_x_or_protected: 833, 839, 1583.
get_x_token: 394, 396, 406, 414, 415, 436, 438, 440, 441, 477, 478, 479, 487, 500, 514, 541, 561, 828, 989, 1015, 1083, 1084, 1192, 1251, 1291, 1438, 1578, 1583.
get_x_token_or_active_char: 541.
getcreationdate: 506.
getfiledump: 506.
getfilemoddate: 506.
getfilesize: 506.
getmd5sum: 506.
getnativechardp: 1481.
getnativecharht: 1481.
getnativecharic: 1481.
getnativecharwd: 1481.
give_err_help: 82, 93, 94, 1338.
global: 1268, 1272, 1295, 1650.
global definitions: 247, 309, 313, 1651.
\global primitive: 1262.
global_box_flag: 1125, 1131, 1295, 1491.
global_defs: 262, 830, 1268, 1272.
\globaldefs primitive: 264.
global_defs_code: 262, 263, 264.
global_prev_p: 181, 877, 911.
glue_base: 246, 248, 250, 252, 253, 254, 255, 278, 830.
glue_break: 925, 929.
glue_error: 1601.
\glueexpr primitive: 1589.
glue_node: 173, 176, 177, 201, 209, 228, 232, 458, 507, 656, 660, 669, 691, 711, 749, 773, 775, 805, 809, 864, 865, 877, 885, 904, 910, 914, 927, 929, 945, 952, 956, 957, 1022, 1026, 1027,

1042, 1050, 1051, 1054, 1088, 1160, 1161, 1162, 1201, 1256, 1537, 1544, 1557.
glue_offset: 157, 184, 212.
glue_ord: 174, 481, 655, 667, 685, 689, 710, 839, 1533.
glue_order: 157, 158, 184, 211, 212, 655, 667, 699, 700, 706, 714, 715, 718, 749, 817, 844, 849, 855, 857, 858, 859, 1202, 1533, 1551.
glue_par: 250, 814.
glue_pars: 250.
glue_ptr: 173, 176, 177, 201, 215, 216, 228, 232, 458, 656, 663, 672, 698, 713, 721, 749, 775, 834, 841, 843, 850, 851, 857, 864, 877, 886, 916, 929, 1023, 1030, 1050, 1055, 1058, 1202, 1509, 1544, 1557, 1654, 1664.
glue_ratio: 113, 132, 135, 157, 212.
glue_ref: 236, 254, 305, 830, 1282, 1290.
glue_ref_count: 174, 175, 176, 177, 178, 189, 227, 229, 254, 814, 1097, 1114.
glue_set: 157, 158, 184, 212, 656, 663, 672, 699, 700, 706, 714, 715, 718, 749, 855, 857, 858, 859, 1202, 1509, 1551.
glue_shrink: 184, 211, 844, 847, 849, 858, 859.
`\glueshrink` primitive: 1612.
glue_shrink_code: 1612, 1613, 1615.
`\glueshrinkorder` primitive: 1612.
glue_shrink_order_code: 1612, 1613, 1614.
glue_sign: 157, 158, 184, 211, 212, 655, 667, 699, 700, 706, 714, 715, 718, 749, 817, 844, 849, 855, 857, 858, 859, 1202, 1533, 1551.
glue_spec_size: 174, 175, 187, 189, 227, 759, 1509.
glue_stretch: 184, 211, 844, 847, 849, 858, 859.
`\gluestretch` primitive: 1612.
glue_stretch_code: 1612, 1613, 1615.
`\gluestretchorder` primitive: 1612.
glue_stretch_order_code: 1612, 1613, 1614.
glue_temp: 655, 663, 667, 672, 1533.
`\gluetomu` primitive: 1616.
glue_to_mu_code: 1616, 1617, 1619.
glue_val: 444, 445, 447, 450, 451, 458, 461, 463, 464, 486, 496, 500, 830, 1114, 1282, 1290, 1291, 1292, 1294, 1589, 1590, 1591, 1593, 1596, 1598, 1602, 1607, 1626, 1634, 1643.
glyph_code: 1398, 1399, 1401, 1403, 1444.
glyph_count: 169.
glyph_node: 169, 656, 749, 781, 799, 889, 890, 918, 919, 1167, 1175, 1416, 1417, 1418, 1420, 1421, 1422, 1426, 1430, 1444, 1536.
glyph_node_size: 169, 749, 781, 783, 793, 799, 1417, 1418, 1444.
`goal height`: 1040, 1041.
goto: 35, 85.
gr: 132, 135, 136, 157.
gr_font_get_named: 1454.
gr_font_get_named_1: 1454.
gr_print_font_name: 1461.
graphite_warning: 744.
group_code: 299, 301, 304, 684, 1190, 1489.
group_trace: 304, 312, 1472.
group_warning: 312, 1585.
grp_stack: 312, 358, 361, 392, 1584, 1585, 1588.
gsa_def: 1650, 1651.
gsa_w_def: 1650, 1651.
gubed: 7.
Guibas, Leonidas Ioannis: 2.
gzFile: 135.
g1: 1252, 1257.
g2: 1252, 1257, 1259.
h: 230, 286, 289, 689, 710, 781, 983, 988, 998, 1002, 1007, 1020, 1024, 1031, 1048, 1140, 1145, 1177, 1544, 1610.
h_offset: 273, 653, 679, 1428.
`\hoffset` primitive: 274.
h_offset_code: 273, 274.
ha: 940, 943, 945, 946, 947, 949, 953, 956, 957, 966.
half: 104, 749, 779, 780, 781, 789, 790, 793, 794, 1256.
half_buf: 630, 631, 632, 634, 635.
half_error_line: 11, 14, 341, 345, 346, 347.
halfp: 115, 120, 124, 129.
halfword: 112, 132, 135, 137, 152, 294, 307, 309, 310, 311, 327, 328, 330, 363, 371, 396, 423, 447, 499, 508, 517, 584, 595, 612, 723, 839, 848, 869, 877, 878, 881, 895, 920, 925, 940, 954, 960, 961, 1031, 1086, 1133, 1155, 1265, 1297, 1320, 1342, 1466, 1493, 1533, 1549, 1625, 1630, 1633, 1650, 1651.
halign: 234, 295, 296, 1148, 1184.
`\halign` primitive: 295.
handle_right_brace: 1121, 1122.
hang_after: 262, 266, 895, 897, 1124, 1203.
`\hangafter` primitive: 264.
hang_after_code: 262, 263, 264, 1124.
hang_indent: 273, 895, 896, 897, 1124, 1203.
`\hangindent` primitive: 274.
hang_indent_code: 273, 274, 1124.
hanging indentation: 895.
hash: 260, 282, 284, 286, 287, 289, 1372, 1373.
hash_base: 246, 248, 282, 284, 286, 292, 293, 402, 403, 536, 1099, 1311, 1368, 1372, 1373.
hash_brace: 508, 511.
hash_is_full: 282, 287.
hash_prime: 12, 14, 286, 288, 1361, 1362.
hash_size: 12, 14, 248, 287, 288, 1388.

hash_used: 282, 285, 287, 1372, 1373.
hb: 940, 950, 951, 953, 956.
hbadness: 262, 702, 708, 709.
\hbadness primitive: 264.
hbadness_code: 262, 263, 264.
\hbox primitive: 1125.
hbox_group: 299, 304, 1137, 1139, 1471, 1489.
hc: 940, 942, 946, 949, 950, 951, 953, 954, 973, 974, 977, 984, 985, 988, 991, 993, 1014, 1016, 1017, 1019, 1669.
hchar: 959, 960, 962, 963.
hd: 689, 694, 749, 751, 752, 755.
head: 238, 239, 241, 242, 243, 458, 761, 824, 844, 847, 853, 860, 862, 864, 1080, 1088, 1108, 1134, 1140, 1145, 1150, 1154, 1159, 1167, 1173, 1175, 1199, 1213, 1222, 1230, 1235, 1238, 1239, 1241, 1245.
head_field: 238, 239, 244.
head_for_vmode: 1148, 1149.
header: 577.
Hedrick, Charles Locke: 3.
height: 157, 158, 160, 161, 162, 169, 170, 210, 213, 214, 498, 589, 660, 662, 664, 667, 669, 670, 673, 675, 678, 679, 689, 693, 698, 712, 714, 721, 747, 749, 752, 754, 756, 770, 773, 778, 779, 780, 781, 782, 783, 786, 789, 790, 791, 793, 794, 795, 800, 801, 803, 816, 817, 844, 849, 852, 854, 855, 857, 858, 859, 877, 1023, 1027, 1035, 1040, 1055, 1056, 1062, 1063, 1064, 1075, 1141, 1154, 1419, 1420, 1424, 1425, 1426, 1428, 1445, 1556.
height: 498.
height_base: 585, 587, 589, 601, 606, 744, 1376, 1377.
height_depth: 589, 694, 751, 752, 755, 1179, 1481.
height_index: 578, 589.
height_offset: 157, 450, 451, 817, 1301.
height_plus_depth: 755, 757.
held over for next output: 1040.
help_line: 83, 93, 94, 366, 1160, 1266, 1267.
help_ptr: 83, 84, 93, 94.
help0: 83, 1306, 1347.
help1: 83, 97, 99, 318, 442, 462, 489, 506, 521, 535, 538, 545, 1014, 1015, 1016, 1017, 1120, 1134, 1153, 1175, 1186, 1189, 1213, 1231, 1246, 1266, 1267, 1286, 1291, 1297, 1298, 1312, 1337, 1358, 1466, 1576, 1595.
help2: 76, 83, 92, 93, 98, 99, 125, 318, 376, 407, 447, 448, 467, 468, 469, 470, 471, 476, 479, 495, 506, 510, 511, 612, 614, 679, 990, 991, 1032, 1069, 1081, 1101, 1122, 1134, 1136, 1149, 1160, 1174, 1183, 1220, 1251, 1261, 1279, 1290, 1295, 1313, 1435, 1444, 1445, 1446, 1506, 1593, 1622.
help3: 76, 83, 102, 366, 430, 449, 480, 514, 824, 831, 832, 840, 1047, 1063, 1078, 1082, 1132, 1138, 1164, 1181, 1237, 1249, 1347.
help4: 83, 93, 368, 432, 437, 452, 491, 602, 766, 1030, 1058, 1104, 1337.
help5: 83, 404, 596, 874, 1118, 1123, 1182, 1269, 1347.
help6: 83, 429, 494, 1182, 1215.
Here is how much...: 1388.
hex_dig: 1630.
hex_dig1: 1630.
hex_dig2: 1630.
hex_dig3: 1630.
hex_dig4: 1630, 1632, 1633.
hex_to_cur_chr: 382.
hex_token: 472, 478.
hf: 940, 947, 949, 950, 951, 956, 957, 962, 963, 964, 965, 969, 970.
\hfil primitive: 1112.
\hfilneg primitive: 1112.
\hfill primitive: 1112.
hfuzz: 273, 708.
\hfuzz primitive: 274.
hfuzz_code: 273, 274.
hh: 132, 135, 136, 140, 155, 170, 208, 239, 245, 247, 283, 298, 728, 786, 1217, 1219, 1235, 1240, 1359, 1360, 1628.
hi: 134, 258, 1286, 1565.
hi_mem_min: 138, 140, 142, 147, 148, 156, 189, 190, 192, 193, 196, 197, 202, 323, 677, 1365, 1366, 1388.
hi_mem_stat_min: 187, 189, 1366.
hi_mem_stat_usage: 187, 189.
history: 80, 81, 86, 97, 99, 271, 1386, 1389, 1585, 1587, 1588.
hlist_node: 157, 158, 159, 160, 172, 184, 201, 209, 210, 228, 232, 506, 540, 652, 654, 655, 660, 669, 683, 689, 691, 711, 723, 749, 855, 858, 862, 877, 889, 890, 914, 918, 919, 1022, 1027, 1047, 1054, 1128, 1134, 1141, 1164, 1201, 1257, 1514, 1535, 1544.
hlist_out: 628, 651, 652, 654, 655, 658, 661, 666, 667, 670, 675, 676, 678, 735, 1436, 1509, 1537.
hlist_stack: 179, 181, 877.
hlist_stack_level: 181, 877.
hlp1: 83.
hlp2: 83.
hlp3: 83.
hlp4: 83.
hlp5: 83.
hlp6: 83.

hmode: 237, 244, 450, 536, 834, 835, 844, 847, 1084, 1088, 1099, 1100, 1102, 1110, 1111, 1125, 1127, 1130, 1133, 1137, 1140, 1145, 1146, 1147, 1148, 1150, 1151, 1163, 1164, 1166, 1170, 1171, 1173, 1176, 1184, 1191, 1254, 1297, 1440, 1489.
hmove: 234, 1102, 1125, 1126, 1127, 1491.
hn: 940, 946, 950, 951, 952, 955, 957, 966, 967, 969, 970, 971, 973, 977, 984, 985.
ho: 134, 261, 447, 448, 1205, 1208, 1567, 1568.
hold_head: 187, 336, 827, 831, 832, 842, 856, 959, 960, 967, 968, 969, 970, 971, 1068, 1071.
holding_inserts: 262, 1068.
`\holdinginserts` primitive: 264.
holding_inserts_code: 262, 263, 264.
horiz: 749.
hpack: 187, 262, 683, 684, 685, 686, 689, 703, 752, 758, 763, 770, 780, 792, 798, 800, 844, 847, 852, 854, 937, 1116, 1140, 1179, 1248, 1253, 1255, 1258, 1515, 1547, 1557.
hrule: 234, 295, 296, 498, 1100, 1110, 1138, 1148, 1149.
`\hrule` primitive: 295.
hsize: 273, 895, 896, 897, 1108, 1203.
`\hsize` primitive: 274.
hsize_code: 273, 274.
hskip: 234, 1111, 1112, 1113, 1132, 1144.
`\hskip` primitive: 1112.
`\hss` primitive: 1112.
`\ht` primitive: 450.
htField: 1445.
hu: 940, 942, 946, 950, 951, 954, 956, 959, 961, 962, 964, 965, 966, 969, 970.
`Huge page...`: 679.
hyf: 953, 955, 957, 959, 962, 963, 967, 968, 973, 974, 977, 978, 986, 1014, 1015, 1016, 1017, 1019.
hyf_bchar: 940, 946, 950, 951, 956.
hyf_char: 940, 949, 957, 967, 969.
hyf_distance: 974, 975, 976, 978, 997, 998, 999, 1378, 1379.
hyf_next: 974, 975, 978, 997, 998, 999, 1378, 1379.
hyf_node: 966, 969.
hyf_num: 974, 975, 978, 997, 998, 999, 1378, 1379.
hyph_codes: 1665, 1669.
hyph_count: 980, 982, 994, 1378, 1379, 1388.
hyph_data: 235, 1264, 1304, 1305, 1306.
hyph_index: 988, 1667, 1669.
hyph_list: 980, 982, 983, 986, 987, 988, 994, 995, 1378, 1379.
hyph_pointer: 979, 980, 981, 983, 988.
hyph_root: 1006, 1012, 1020, 1665, 1668.
hyph_size: 12, 979, 982, 984, 987, 993, 994, 1361, 1362, 1378, 1379, 1388.
hyph_start: 1378, 1379, 1668, 1669.
hyph_word: 980, 982, 983, 985, 988, 994, 995, 1378, 1379.
hyphen_char: 179, 460, 584, 587, 611, 744, 939, 949, 1088, 1089, 1171, 1307, 1376, 1377.
`\hyphenchar` primitive: 1308.
hyphen_passed: 957, 959, 960, 963, 967, 968.
hyphen_penalty: 167, 262, 917.
`\hyphenpenalty` primitive: 264.
hyphen_penalty_code: 262, 263, 264.
hyphenatable_length_limit: 12, 105, 940, 944, 953, 954, 966, 988, 1014.
hyphenate: 943, 944.
hyphenated: 867, 868, 877, 894, 907, 917, 921.
`Hyphenation trie...`: 1378.
`\hyphenation` primitive: 1304.
h1: 793.
h2: 793.
i: 19, 44, 129, 345, 447, 505, 623, 689, 744, 781, 793, 954, 1177, 1403, 1415, 1489, 1585, 1587, 1588, 1626, 1630, 1632, 1636, 1648.
`I can't find file x`: 565.
`I can't find PLAIN...`: 559.
`I can't go on...`: 99.
`I can't read TEX.POOL`: 51.
`I can't write on file x`: 565.
id_byte: 623, 653, 680.
id_lookup: 286, 294, 384, 386, 408, 1579.
ident_val: 444, 449, 500, 501.
`\ifcase` primitive: 522.
if_case_code: 522, 523, 536, 1576.
if_cat_code: 522, 523, 536.
`\ifcat` primitive: 522.
`\if` primitive: 522.
if_char_code: 522, 536, 541.
if_code: 524, 530, 545.
if_cs_code: 1573, 1575, 1578.
`\ifcsname` primitive: 1573.
if_cur_ptr_is_null_then_return_or_goto: 1630.
if_def_code: 1573, 1575, 1577.
`\ifdefined` primitive: 1573.
`\ifdim` primitive: 522.
if_dim_code: 522, 523, 536.
`\ifeof` primitive: 522.
if_eof_code: 522, 523, 536.
`\iffalse` primitive: 522.
if_false_code: 522, 523, 536.
`\iffontchar` primitive: 1573.
if_font_char_code: 1573, 1575, 1580.
`\ifhbox` primitive: 522.
if_hbox_code: 522, 523, 536, 540.
`\ifhmode` primitive: 522.

if_hmode_code: 522, 523, 536.
\ifincsname primitive: 1573.
if_in_csname_code: 1573, 1575, 1580.
\ifinner primitive: 522.
if_inner_code: 522, 523, 536.
\ifnum primitive: 522.
if_int_code: 522, 523, 536, 538.
if_limit: 524, 525, 530, 531, 532, 533, 545, 1478, 1501, 1588.
if_line: 329, 524, 525, 530, 531, 1389, 1501, 1587, 1588.
if_line_field: 524, 530, 531, 1389, 1501, 1588.
\ifmmode primitive: 522.
if_mmode_code: 522, 523, 536.
if_node_size: 524, 530, 531, 1389.
\ifodd primitive: 522.
if_odd_code: 522, 523, 536.
\ifprimitive primitive: 522.
if_primitive_code: 522, 523, 536.
if_stack: 358, 361, 392, 531, 1584, 1587, 1588.
if_test: 236, 329, 366, 396, 399, 522, 523, 529, 533, 538, 1389, 1501, 1573, 1576, 1587, 1588.
\iftrue primitive: 522.
if_true_code: 522, 523, 536.
\ifvbox primitive: 522.
if_vbox_code: 522, 523, 536.
\ifvmode primitive: 522.
if_vmode_code: 522, 523, 536.
\ifvoid primitive: 522.
if_void_code: 522, 523, 536, 540.
if_warning: 531, 1587.
\ifx primitive: 522.
ifx_code: 522, 523, 536.
ignore: 233, 258, 362, 375, 506.
ignore_depth: 238, 241, 245, 721, 835, 1079, 1110, 1137, 1153, 1221.
ignore_spaces: 234, 285, 295, 296, 402, 447, 1099.
\ignorespaces primitive: 295.
Illegal magnification...: 318, 1312.
Illegal math \disc...: 1174.
Illegal parameter number...: 514.
Illegal unit of measure: 489, 491, 494.
illegal_Ucharcat_catcode: 506.
\immediate primitive: 1398.
immediate_code: 1398, 1401, 1403.
IMPOSSIBLE: 292.
Improper \beginL: 1513.
Improper \beginR: 1513.
Improper \endL: 1513.
Improper \endR: 1513.
Improper \halign...: 824.
Improper \hyphenation...: 990.
Improper \prevdepth: 452.
Improper \setbox: 1295.
Improper \spacefactor: 452.
Improper 'at' size...: 1313.
Improper alphabetic constant: 476.
Improper discretionary list: 1175.
in: 493.
in_open: 312, 334, 343, 358, 359, 361, 392, 531, 1446, 1585, 1587, 1588.
in_state_record: 330, 331.
in_stream: 234, 1326, 1327, 1328.
inaccessible: 1270.
Incompatible glue units: 442.
Incompatible list...: 1164.
Incompatible magnification: 318.
incompleat_noad: 238, 239, 761, 824, 1190, 1232, 1235, 1236, 1238, 1239.
Incomplete \if...: 366.
incr: 16, 31, 37, 42, 43, 44, 45, 46, 53, 58, 60, 63, 69, 71, 74, 75, 86, 94, 102, 117, 142, 144, 176, 177, 195, 198, 208, 229, 242, 286, 287, 304, 306, 310, 324, 329, 341, 342, 351, 355, 356, 358, 373, 377, 382, 384, 385, 386, 387, 390, 392, 408, 426, 429, 431, 433, 434, 437, 441, 476, 487, 489, 499, 510, 511, 512, 529, 552, 554, 559, 566, 572, 615, 634, 639, 655, 667, 678, 680, 684, 744, 757, 783, 793, 846, 893, 925, 946, 949, 950, 951, 964, 965, 968, 969, 977, 984, 985, 991, 993, 994, 995, 998, 1008, 1010, 1016, 1017, 1018, 1020, 1040, 1076, 1079, 1088, 1089, 1093, 1123, 1171, 1173, 1175, 1181, 1196, 1207, 1226, 1228, 1369, 1370, 1372, 1391, 1410, 1415, 1420, 1451, 1478, 1489, 1493, 1501, 1521, 1527, 1539, 1550, 1565, 1566, 1572, 1579, 1593, 1608, 1611, 1630, 1632, 1648.
incr_offset: 58.
\indent primitive: 1142.
indent_in_hmode: 1146, 1147.
indented: 1145.
index: 330, 332, 333, 334, 337, 343, 358, 359, 361, 392.
index_field: 330, 332, 1185, 1586.
index_node_size: 1626, 1632, 1636.
inf: 481, 482, 488.
inf_bad: 112, 182, 899, 900, 901, 904, 911, 1028, 1059, 1071, 1658.
inf_penalty: 182, 809, 815, 864, 877, 879, 1028, 1059, 1067, 1257, 1259.
Infinite glue shrinkage...: 874, 1030, 1058, 1063.
infinity: 479, 1601, 1603, 1609.
info: 140, 146, 148, 162, 163, 179, 189, 197, 226, 259, 305, 321, 323, 355, 356, 367, 369, 387, 388,

401, 402, 405, 408, 423, 425, 426, 427, 428, 431, 434, 457, 487, 501, 513, 543, 641, 644, 645, 646, 647, 648, 649, 650, 651, 723, 731, 734, 735, 740, 763, 777, 778, 779, 780, 781, 786, 793, 798, 805, 816, 817, 820, 827, 831, 832, 838, 841, 842, 845, 846, 849, 851, 869, 895, 896, 979, 986, 992, 1035, 1119, 1130, 1147, 1203, 1205, 1222, 1235, 1239, 1240, 1245, 1272, 1280, 1302, 1303, 1343, 1349, 1366, 1393, 1395, 1434, 1484, 1515, 1517, 1518, 1519, 1521, 1522, 1527, 1528, 1533, 1539, 1542, 1548, 1550, 1561, 1565, 1567, 1568, 1579, 1583, 1626, 1630, 1631, 1635, 1636.
init: 8, 47, 50, 153, 294, 939, 988, 996, 997, 1001, 1004, 1306, 1356, 1379, 1386, 1389, 1390, 1451, 1642.
init_align: 821, 822, 1184.
init_col: 821, 833, 836, 839.
init_cur_lang: 864, 939, 940.
init_l_hyf: 864, 939, 940.
init_lft: 953, 956, 959, 962.
init_lig: 953, 956, 959, 962.
init_list: 953, 956, 959, 962.
init_math: 1191, 1192, 1515.
init_pool_ptr: 39, 42, 1364, 1386, 1388.
init_prim: 1386, 1390.
init_r_hyf: 864, 939, 940.
init_randoms: 129, 1391, 1413.
init_row: 821, 833, 834.
init_span: 821, 834, 835, 839.
init_start_time: 1412.
init_str_ptr: 39, 43, 552, 1364, 1386, 1388.
init_terminal: 37, 361.
init_trie: 939, 1020, 1378.
INITEX: 8, 11, 12, 47, 50, 138, 1353, 1385, 1636, 1642.
initialize: 4, 1386, 1391.
inner loop: 31, 116, 117, 120, 134, 142, 143, 144, 145, 147, 149, 150, 152, 228, 354, 355, 371, 372, 373, 387, 395, 414, 433, 441, 589, 633, 647, 658, 691, 694, 695, 880, 883, 899, 900, 915, 1084, 1088, 1089, 1090, 1093, 1095.
inner_noad: 724, 725, 732, 738, 740, 776, 809, 812, 1210, 1211, 1245.
input: 236, 396, 399, 410, 411, 1558.
\input primitive: 410.
input_file: 334, 1446.
\inputlineno primitive: 450.
input_line_no_code: 450, 451, 458.
input_ln: 30, 31, 37, 58, 75, 392, 520, 521, 573.
input_ptr: 331, 341, 342, 351, 352, 360, 361, 390, 569, 1185, 1389, 1585, 1587.
input_stack: 88, 89, 331, 341, 351, 352, 569, 1185, 1585, 1586, 1587.
ins_disc: 1086, 1087, 1089.
ins_error: 357, 366, 429, 1101, 1181, 1186, 1269.
ins_list: 353, 369, 502, 505, 506, 1118, 1434.
ins_node: 162, 172, 201, 209, 228, 232, 656, 686, 691, 773, 809, 877, 914, 945, 952, 1022, 1027, 1035, 1040, 1054, 1068, 1154.
ins_node_size: 162, 228, 232, 1076, 1154.
ins_ptr: 162, 214, 228, 232, 1064, 1074, 1075, 1154.
ins_the_toks: 396, 399, 502.
insert: 234, 295, 296, 1151.
insert>: 91.
\insert primitive: 295.
insert_dollar_sign: 1099, 1101.
insert_group: 299, 1122, 1153, 1154, 1471, 1489.
insert_penalties: 453, 1036, 1044, 1059, 1062, 1064, 1068, 1076, 1080, 1296, 1300.
\insertpenalties primitive: 450.
insert_relax: 412, 413, 545.
insert_token: 298, 310, 312.
inserted: 337, 344, 353, 354, 357, 413, 1149.
inserting: 1035, 1063.
Insertions can only...: 1047.
inserts_only: 1034, 1041, 1062.
int: 132, 135, 136, 162, 164, 182, 212, 239, 245, 262, 266, 268, 304, 308, 309, 447, 448, 524, 641, 744, 768, 817, 820, 867, 1291, 1302, 1359, 1360, 1362, 1370, 1452, 1599, 1631.
int_base: 246, 256, 258, 262, 264, 265, 266, 268, 278, 279, 280, 298, 313, 318, 1067, 1124, 1193, 1199, 1369, 1452, 1467, 1511.
int_error: 95, 318, 448, 467, 468, 469, 470, 471, 506, 1297, 1298, 1312, 1444, 1506, 1622.
int_par: 262.
int_pars: 262.
int_val: 444, 445, 447, 448, 450, 451, 452, 453, 456, 457, 458, 460, 461, 462, 463, 467, 473, 474, 484, 496, 500, 1278, 1290, 1291, 1292, 1294, 1365, 1366, 1410, 1589, 1590, 1591, 1594, 1596, 1601, 1603, 1606, 1609, 1626, 1627, 1629, 1634, 1643.
integer: 3, 13, 19, 40, 44, 45, 47, 54, 59, 61, 63, 67, 69, 70, 71, 73, 86, 95, 98, 100, 104, 105, 106, 109, 110, 111, 112, 113, 114, 116, 118, 121, 123, 126, 128, 129, 130, 131, 132, 135, 139, 147, 169, 183, 188, 197, 198, 199, 200, 202, 203, 204, 207, 208, 224, 237, 238, 244, 251, 263, 272, 273, 282, 286, 289, 292, 294, 308, 309, 316, 322, 328, 329, 334, 338, 339, 341, 345, 396, 444, 447, 474, 482, 485, 505, 517, 524, 528, 529, 533, 553, 554, 558, 584, 585, 595, 613, 616, 628, 631, 636, 637, 638, 643, 651, 652, 655, 667, 676, 682, 684,

685, 688, 689, 703, 733, 736, 741, 742, 743, 744, 749, 752, 759, 760, 762, 767, 769, 781, 793, 796, 800, 812, 876, 877, 878, 881, 920, 925, 940, 944, 948, 966, 976, 1020, 1024, 1034, 1036, 1048, 1066, 1084, 1086, 1122, 1129, 1133, 1138, 1140, 1145, 1171, 1173, 1192, 1205, 1209, 1219, 1248, 1265, 1290, 1342, 1347, 1356, 1357, 1385, 1387, 1392, 1403, 1411, 1415, 1433, 1436, 1445, 1449, 1457, 1489, 1515, 1555, 1564, 1567, 1588, 1593, 1604, 1608, 1610, 1650, 1651.
inter_char_text: 337, 344, 1088.
inter_char_val: 444, 449, 1088, 1280, 1281, 1365, 1366, 1626, 1629.
`\interlinepenalties` primitive: 1675.
inter_line_penalties_loc: 256, 1124, 1675, 1676.
inter_line_penalties_ptr: 938, 1124, 1675.
inter_line_penalty: 262, 938.
`\interlinepenalty` primitive: 264.
inter_line_penalty_code: 262, 263, 264.
interaction: 75, 76, 77, 78, 79, 86, 87, 88, 90, 94, 96, 97, 102, 390, 393, 519, 565, 1319, 1337, 1347, 1348, 1351, 1380, 1381, 1382, 1389, 1504.
`\interactionmode` primitive: 1502.
internal_font_number: 467, 583, 584, 585, 595, 612, 613, 616, 618, 638, 652, 688, 689, 744, 749, 752, 754, 755, 758, 767, 780, 781, 793, 800, 878, 910, 940, 1086, 1167, 1177, 1192, 1265, 1311.
interrupt: 100, 101, 102, 1085.
`Interruption`: 102.
`interwoven alignment preambles...`: 354, 830, 837, 839, 1185.
`Invalid code`: 506, 1286.
invalid_char: 233, 258, 374.
invalid_code: 22, 258.
is_aat_font: 584, 1454, 1460, 1461.
is_active_math_char: 258, 448, 469, 1205, 1209.
is_bottom_acc: 729, 781, 783.
is_char_node: 156, 200, 209, 228, 231, 458, 507, 656, 658, 668, 688, 691, 711, 758, 763, 764, 800, 853, 864, 877, 885, 889, 890, 914, 915, 916, 918, 919, 927, 929, 945, 949, 950, 952, 956, 1088, 1090, 1094, 1134, 1159, 1164, 1167, 1175, 1201, 1256, 1420, 1517, 1534, 1544, 1549.
is_empty: 146, 149, 194, 195.
is_glyph_node: 656, 688, 781, 793, 800, 806, 807.
is_gr_font: 584, 1454, 1460, 1461.
is_hex: 382, 385.
is_hyph: 1086, 1088.
is_in_csname: 397, 398, 406, 1578, 1580.
is_native_font: 460, 507, 584, 638, 752, 765, 781, 799, 1088, 1177, 1179, 1307, 1314, 1315, 1444, 1454, 1458, 1460, 1461, 1481, 1580.
is_native_word_node: 656, 688, 943, 956, 1088, 1420.
is_native_word_subtype: 169, 656, 889, 890, 918, 919, 949, 1167, 1175, 1420, 1421, 1422, 1536.
is_new_mathfont: 584, 742, 743, 780, 781, 789, 790, 793, 799, 801, 802, 803, 805, 1249.
is_ot_font: 584, 749, 751, 793, 1454.
is_otgr_font: 584, 1454.
is_pdf: 1445.
is_running: 160, 202, 662, 671, 854.
is_unless: 533.
is_valid_pointer: 804, 805.
is_var_family: 258, 1205, 1209, 1219.
isOpenTypeMathFont: 584, 744.
issue_message: 1330, 1333.
ital_corr: 234, 295, 296, 1165, 1166.
italic correction: 578.
italic_base: 585, 587, 589, 601, 606, 1376, 1377.
italic_index: 578.
its_all_over: 1099, 1108, 1389.
i1: 1410.
i2: 1410.
j: 45, 46, 63, 73, 74, 129, 286, 289, 294, 345, 396, 505, 517, 554, 558, 559, 655, 676, 942, 954, 960, 988, 1020, 1192, 1265, 1356, 1357, 1403, 1433, 1436, 1466, 1489, 1552, 1555.
j_random: 114, 128, 130, 131.
Japanese characters: 156, 621.
Jensen, Kathleen: 10.
jj: 129.
`job aborted`: 390.
`job aborted, file error...`: 565.
job_name: 96, 506, 507, 562, 563, 564, 567, 569, 572, 676, 1311, 1382, 1389.
`\jobname` primitive: 503.
job_name_code: 503, 506, 507.
jump_out: 85, 86, 88, 97.
just_box: 862, 936, 937, 1202, 1545, 1551.
just_copy: 1544, 1545, 1549.
just_open: 515, 518, 1329.
just_reverse: 1548, 1549.
j1: 1410.
j2: 1410.
k: 45, 46, 68, 69, 71, 73, 75, 106, 123, 128, 129, 188, 286, 289, 294, 371, 393, 441, 485, 499, 554, 558, 560, 565, 569, 595, 623, 633, 638, 643, 655, 676, 688, 689, 744, 748, 960, 983, 988, 1014, 1020, 1133, 1265, 1356, 1357, 1387, 1392, 1403, 1431, 1433, 1466, 1626.
kern: 234, 580, 1111, 1112, 1113.
`\kern` primitive: 1112.
kern_base: 585, 587, 592, 601, 608, 611, 1376, 1377.

kern_base_offset: 592, 601, 608.
kern_break: 914.
kern_flag: 580, 785, 797, 963, 1094.
kern_node: 179, 180, 209, 228, 232, 458, 656, 660, 669, 691, 711, 764, 773, 775, 805, 809, 877, 885, 889, 890, 904, 914, 916, 918, 919, 927, 929, 945, 949, 950, 952, 1022, 1026, 1027, 1030, 1050, 1051, 1054, 1058, 1160, 1161, 1162, 1175, 1201, 1509, 1521, 1527, 1535, 1539, 1544, 1550.
kk: 447, 485, 487, 1454.
Knuth, Donald Ervin: 2, 90, 735, 861, 939, 979, 1051, 1208, 1434, 1489, 1510.
l: 59, 131, 286, 289, 294, 306, 311, 322, 329, 345, 529, 532, 569, 637, 651, 710, 744, 877, 878, 954, 998, 1007, 1014, 1192, 1248, 1290, 1347, 1356, 1392, 1439, 1489, 1533, 1549, 1564, 1588, 1593, 1636.
L_code: 171, 201, 218, 914, 949, 952, 1527, 1528, 1547, 1548.
l_hyf: 939, 940, 943, 952, 955, 957, 977, 1422, 1423.
language: 262, 988, 1088, 1439.
`\language` primitive: 264.
language_code: 262, 263, 264.
language_node: 1395, 1416, 1417, 1418, 1422, 1423, 1436, 1439, 1440.
large_attempt: 749.
large_char: 725, 733, 739, 749.
large_char_field: 725, 1214.
large_fam: 725, 733, 739, 749.
large_plane_and_fam_field: 725, 1214.
last: 30, 31, 35, 36, 37, 75, 87, 91, 92, 361, 390, 393, 518, 559, 566, 1567.
last_active: 867, 868, 880, 883, 892, 902, 908, 909, 911, 912, 913, 921, 922, 923.
last_badness: 458, 685, 687, 689, 702, 706, 709, 710, 716, 718, 720.
last_bop: 628, 629, 678, 680.
`\lastbox` primitive: 1125.
last_box_code: 1125, 1126, 1133, 1389, 1670, 1672, 1673.
last_glue: 458, 1036, 1045, 1050, 1071, 1160, 1389.
last_ins_ptr: 1035, 1059, 1062, 1072, 1074.
last_item: 234, 447, 450, 451, 1102, 1452, 1454, 1458, 1473, 1476, 1479, 1482, 1589, 1612, 1616.
last_kern: 458, 1036, 1045, 1050.
`\lastkern` primitive: 450.
last_leftmost_char: 181, 688, 935.
last_line_fill: 864, 1653, 1654, 1664.
last_line_fit: 262, 1653, 1654, 1657.
`\lastlinefit` primitive: 1467.
last_line_fit_code: 262, 1467, 1469.
last_node_type: 458, 1036, 1045, 1050.
`\lastnodetype` primitive: 1452.
last_node_type_code: 450, 458, 1452, 1453.
last_nonblank: 31.
last_penalty: 458, 1036, 1045, 1050.
`\lastpenalty` primitive: 450.
last_rightmost_char: 181, 688, 929.
`\lastskip` primitive: 450.
last_special_line: 895, 896, 897, 898, 937.
last_text_char: 19.
lastMathConstant: 742, 744.
lastMathValueRecord: 742.
lc_code: 256, 258, 939, 949, 1016, 1665, 1667, 1668, 1669.
`\lccode` primitive: 1284.
lc_code_base: 256, 261, 1284, 1285, 1340, 1341, 1342.
lccode: 946.
leader_box: 655, 664, 666, 667, 673, 675.
leader_flag: 1125, 1127, 1132, 1138, 1491.
leader_ht: 667, 673, 674, 675.
leader_ptr: 173, 176, 177, 216, 228, 232, 664, 673, 698, 713, 864, 1132, 1544.
leader_ship: 234, 1125, 1126, 1127, 1491.
leader_wd: 655, 664, 665, 666.
leaders: 1437.
`Leaders not followed by...`: 1132.
`\leaders` primitive: 1125.
least_cost: 1024, 1028, 1034.
least_page_cost: 1034, 1041, 1059, 1060.
`\left` primitive: 1242.
left_brace: 233, 319, 324, 328, 377, 387, 437, 506, 508, 825, 1117, 1204, 1280.
left_brace_limit: 319, 355, 356, 426, 428, 433, 511.
left_brace_token: 319, 437, 1181, 1280, 1434.
left_delimiter: 725, 738, 739, 780, 792, 1217, 1235, 1236.
left_edge: 655, 665, 667, 670, 671, 675, 1426, 1529, 1530, 1532.
left_hyphen_min: 262, 1145, 1254, 1439, 1440.
`\lefthyphenmin` primitive: 264.
left_hyphen_min_code: 262, 263, 264.
`\leftmarginkern` primitive: 503.
left_margin_kern_code: 503, 504, 506, 507.
left_noad: 238, 729, 732, 738, 740, 768, 770, 771, 776, 808, 809, 810, 1239, 1242, 1243, 1245, 1489.
left_pw: 688, 877, 935.
left_right: 234, 1100, 1242, 1243, 1244, 1507.
left_side: 179, 209, 460, 507, 688, 935, 1307.
left_skip: 250, 875, 928, 935, 1551, 1654.
`\leftskip` primitive: 252.
left_skip_code: 250, 251, 252, 507, 935, 1551, 1557.
left_to_right: 652, 1516, 1524, 1541, 1546.

len: 655, 744, 1430.
length: 40, 44, 46, 169, 286, 289, 507, 572, 638, 639, 985, 995, 1315, 1334, 1448.
length of lines: 895.
`\leqno` primitive: 1195.
let: 235, 1264, 1273, 1274, 1275.
`\let` primitive: 1273.
letter: 233, 258, 292, 319, 321, 324, 328, 377, 384, 386, 989, 1015, 1083, 1084, 1088, 1092, 1144, 1178, 1205, 1208, 1214.
letter_token: 319, 479.
level: 444, 447, 449, 452, 462, 496, 1591.
level_boundary: 298, 300, 304, 312.
level_one: 247, 254, 258, 280, 285, 294, 302, 307, 308, 309, 310, 311, 313, 828, 1358, 1389, 1432, 1475, 1631, 1651, 1652.
level_zero: 247, 248, 302, 306, 310, 1647.
lf: 575, 595, 600, 601, 610, 611.
lft_hit: 960, 961, 962, 964, 965, 1087, 1089, 1094.
lh: 132, 135, 136, 140, 170, 239, 245, 282, 283, 575, 576, 595, 600, 601, 603, 727, 1004, 1628.
Liang, Franklin Mark: 2, 973.
libc_free: 169, 1445.
lig_char: 165, 166, 219, 232, 688, 692, 889, 890, 914, 918, 919, 951, 956, 1167, 1538, 1544.
lig_kern: 579, 580, 584.
lig_kern_base: 585, 587, 592, 601, 606, 608, 611, 1376, 1377.
lig_kern_command: 576, 580.
lig_kern_restart: 592, 785, 796, 963, 1093.
lig_kern_restart_end: 592.
lig_kern_start: 592, 785, 796, 963, 1093.
lig_ptr: 165, 166, 201, 219, 228, 232, 949, 951, 956, 961, 964, 965, 1091, 1094, 1538.
lig_stack: 961, 962, 964, 965, 1086, 1088, 1089, 1090, 1091, 1092, 1094.
lig_tag: 579, 604, 785, 796, 963, 1093.
lig_trick: 187, 658, 692.
ligature_node: 165, 166, 172, 201, 209, 228, 232, 660, 688, 691, 796, 889, 890, 914, 918, 919, 945, 949, 950, 952, 956, 1167, 1175, 1201, 1538, 1544.
ligature_present: 960, 961, 962, 964, 965, 1087, 1089, 1091, 1094.
limit: 330, 332, 333, 337, 348, 358, 360, 361, 373, 378, 380, 381, 382, 384, 385, 386, 390, 392, 393, 518, 521, 572, 573, 1391, 1566, 1572.
`Limit controls must follow...`: 1213.
limit_field: 35, 91, 330, 332, 569.
limit_switch: 234, 1100, 1210, 1211, 1212.
limits: 724, 738, 776, 793, 1210, 1211.
`\limits` primitive: 1210.
line: 88, 242, 304, 329, 334, 343, 358, 359, 361, 392, 458, 529, 530, 573, 705, 717, 744, 1079, 1566.
line_break: 181, 187, 862, 863, 876, 887, 896, 910, 911, 914, 924, 943, 988, 1021, 1024, 1036, 1150, 1199.
line_diff: 920, 923.
line_number: 867, 868, 881, 883, 893, 894, 898, 912, 920, 922, 923.
line_penalty: 262, 907.
`\linepenalty` primitive: 264.
line_penalty_code: 262, 263, 264.
line_skip: 250, 273.
`\lineskip` primitive: 252.
line_skip_code: 173, 176, 250, 251, 252, 721.
line_skip_limit: 273, 721.
`\lineskiplimit` primitive: 274.
line_skip_limit_code: 273, 274.
line_stack: 334, 343, 358, 359.
line_width: 878, 898, 899.
linebreak_next: 744.
linebreak_start: 744.
link: 140, 142, 143, 144, 145, 146, 147, 148, 152, 155, 156, 157, 162, 163, 164, 165, 174, 189, 193, 197, 198, 200, 201, 202, 208, 228, 230, 238, 240, 244, 249, 259, 322, 325, 329, 336, 349, 353, 356, 369, 387, 388, 396, 401, 402, 405, 408, 423, 424, 425, 428, 430, 431, 434, 441, 458, 487, 499, 501, 502, 505, 506, 507, 513, 524, 530, 531, 532, 543, 641, 643, 645, 647, 651, 656, 657, 658, 660, 668, 689, 691, 692, 694, 697, 708, 711, 721, 723, 731, 744, 748, 749, 754, 758, 761, 762, 763, 764, 770, 774, 775, 778, 780, 781, 782, 791, 792, 795, 796, 797, 798, 799, 800, 803, 805, 808, 809, 814, 815, 818, 820, 826, 827, 831, 832, 834, 838, 839, 841, 842, 843, 844, 845, 846, 847, 849, 850, 851, 852, 853, 854, 855, 856, 857, 860, 862, 864, 867, 869, 870, 877, 878, 885, 888, 891, 892, 893, 902, 905, 906, 908, 909, 910, 911, 912, 913, 914, 915, 917, 921, 922, 923, 925, 927, 928, 929, 930, 931, 932, 933, 934, 935, 938, 943, 945, 946, 947, 949, 950, 951, 952, 956, 957, 959, 960, 961, 962, 964, 965, 967, 968, 969, 970, 971, 972, 986, 992, 1014, 1022, 1023, 1024, 1027, 1033, 1034, 1035, 1040, 1042, 1045, 1048, 1052, 1053, 1054, 1055, 1059, 1062, 1063, 1068, 1071, 1072, 1073, 1074, 1075, 1076, 1077, 1080, 1088, 1089, 1090, 1091, 1094, 1095, 1097, 1118, 1119, 1130, 1134, 1140, 1145, 1154, 1155, 1164, 1173, 1174, 1175, 1177, 1179, 1200, 1209, 1222, 1235, 1238, 1239, 1240, 1241, 1245, 1248, 1250, 1253, 1258, 1259, 1260, 1272, 1280, 1333, 1342, 1349, 1351, 1365, 1366, 1389, 1393, 1395, 1404, 1410, 1420,

1431, 1434, 1438, 1478, 1493, 1498, 1501, 1515, 1517, 1519, 1522, 1531, 1532, 1534, 1535, 1538, 1540, 1544, 1545, 1546, 1549, 1551, 1556, 1557, 1564, 1565, 1567, 1568, 1579, 1583, 1587, 1588, 1599, 1600, 1626, 1630, 1631, 1632, 1633, 1634, 1635, 1636, 1639, 1648, 1652, 1674.
list_offset: 157, 655, 656, 689, 817, 1072, 1420.
list_ptr: 157, 158, 210, 228, 232, 507, 655, 656, 661, 667, 670, 700, 705, 706, 710, 715, 718, 749, 752, 754, 758, 764, 781, 782, 783, 791, 793, 795, 855, 877, 1031, 1033, 1075, 1141, 1154, 1164, 1253, 1544, 1545, 1551, 1556, 1557.
list_state_record: 238, 239.
list_tag: 579, 604, 605, 751, 784, 793.
ll: 286, 287, 1007, 1010.
llink: 146, 148, 149, 151, 152, 153, 167, 173, 189, 194, 820, 867, 869, 1366.
lo_mem_max: 138, 142, 147, 148, 189, 190, 192, 194, 195, 196, 197, 204, 677, 1365, 1366, 1377, 1388.
lo_mem_stat_max: 187, 189, 461, 1275, 1291, 1366, 1643, 1645.
load_fmt_file: 1357, 1391.
load_native_font: 595, 744.
load_picture: 1442, 1443, 1445.
load_tfm_font_mapping: 611.
loaded_font_design_size: 744.
loaded_font_flags: 744.
loaded_font_letter_space: 744.
loaded_font_mapping: 744.
loc: 36, 37, 91, 330, 332, 333, 337, 342, 344, 348, 349, 353, 355, 356, 358, 360, 361, 373, 378, 380, 381, 382, 384, 386, 387, 388, 390, 392, 401, 402, 424, 518, 559, 572, 573, 1080, 1081, 1391, 1451, 1566, 1572.
loc_field: 35, 36, 330, 332, 1185.
local_base: 246, 250, 254, 256, 278.
location: 641, 643, 648, 649, 650, 651.
log_file: 54, 56, 79, 569, 1387.
log_name: 567, 569, 1387.
log_only: 54, 57, 58, 66, 79, 102, 390, 569, 1382, 1433.
log_opened: 96, 97, 562, 563, 569, 570, 1319, 1387, 1388.
`Logarithm...replaced by 0`: 125.
`\long` primitive: 1262.
long_call: 236, 305, 396, 421, 423, 426, 433, 1349.
long_help_seen: 1335, 1336, 1337.
long_hex_to_cur_chr: 382.
long_outer_call: 236, 305, 396, 421, 423, 1349.
long_state: 369, 421, 425, 426, 429, 430, 433.
loop: 15, 16.
`Loose \hbox...`: 702.
`Loose \vbox...`: 716.
loose_fit: 865, 882, 900, 1658.
looseness: 262, 896, 921, 923, 1124.
`\looseness` primitive: 264.
looseness_code: 262, 263, 264, 1124.
lower: 371, 373.
`\lower` primitive: 1125.
`\lowercase` primitive: 1340.
lowerLimitBaselineDropMin: 742.
lowerLimitGapMin: 742.
`\lpcode` primitive: 1308.
lp_code_base: 179, 460, 1307, 1308, 1309.
lq: 628, 665, 674.
lr: 628, 665, 674.
LR_box: 238, 239, 1199, 1260, 1553.
LR_dir: 1527, 1539, 1548, 1550.
LR_problems: 1515, 1516, 1521, 1522, 1523, 1527, 1528, 1533, 1539, 1541, 1546, 1550.
LR_ptr: 925, 1515, 1516, 1517, 1518, 1519, 1521, 1522, 1527, 1528, 1533, 1539, 1541, 1546, 1548, 1550.
LR_save: 238, 239, 925, 1150, 1542.
lsb: 1177, 1179.
lx: 655, 664, 665, 666, 667, 673, 674, 675.
m: 47, 69, 183, 237, 244, 322, 345, 423, 447, 474, 517, 533, 612, 689, 710, 749, 759, 760, 1133, 1159, 1248, 1347, 1392, 1411, 1489, 1533, 1549, 1564.
M_code: 171.
m_exp: 114.
m_log: 114, 123, 125, 131.
mac_param: 233, 321, 324, 328, 377, 509, 512, 514, 831, 832, 1099.
MacKay, Pierre: 1510.
macro: 337, 344, 349, 353, 354, 424.
macro_call: 321, 396, 414, 416, 421, 422, 423, 425.
macro_def: 508, 512.
mag: 262, 266, 318, 492, 621, 623, 624, 626, 653, 680.
`\mag` primitive: 264.
mag_code: 262, 263, 264, 318.
mag_set: 316, 317, 318.
magic_offset: 812, 813, 814.
main_control: 1083, 1084, 1086, 1094, 1095, 1106, 1108, 1109, 1110, 1111, 1180, 1188, 1262, 1344, 1386, 1391, 1398, 1402.
main_f: 744, 1086, 1088, 1089, 1090, 1091, 1092, 1093, 1094.
main_h: 1086, 1088.
main_i: 1086, 1090, 1091, 1093, 1094.
main_j: 1086, 1093, 1094.

main_k: 1086, 1088, 1093, 1094, 1096.
main_lig_loop: 1084, 1088, 1091, 1092, 1093, 1094.
main_loop: 1084.
main_loop_lookahead: 1084, 1088, 1090, 1091, 1092.
main_loop_move: 1084, 1088, 1090, 1094.
main_loop_move_lig: 1084, 1088, 1090, 1091.
main_loop_wrapup: 1084, 1088, 1093, 1094.
main_p: 1086, 1088, 1089, 1091, 1094, 1095, 1096, 1097, 1098.
main_pp: 1086, 1088.
main_ppp: 1086, 1088.
main_s: 1086, 1088.
major_tail: 966, 968, 971, 972.
make_accent: 1176, 1177.
make_box: 234, 1125, 1126, 1127, 1133, 1138.
make_font_def: 638.
make_frac: 114, 116, 131.
make_fraction: 114, 776, 777, 787, 1610.
make_identity: 1445.
make_left_right: 809, 810.
make_mark: 1151, 1155.
make_math_accent: 776, 781.
make_name_string: 560.
make_op: 776, 793.
make_ord: 776, 796.
make_over: 776, 777.
make_radical: 776, 777, 780.
make_rotation: 1445.
make_scale: 1445.
make_scripts: 798, 800.
make_string: 43, 52, 287, 505, 506, 552, 560, 744, 993, 1311, 1314, 1333, 1382, 1387, 1410, 1564.
make_translation: 1445.
make_under: 776, 778.
make_utf16_name: 560.
make_vcenter: 776, 779.
make_xdv_glyph_array_data: 1430.
map_char_to_glyph: 467, 744, 751, 1088, 1454, 1580.
map_glyph_to_index: 467, 1454.
mapped_text: 744, 1088.
mappingNameLen: 744.
mappingNameP: 744.
margin_char: 179.
margin_kern_node: 179, 209, 228, 232, 507, 660, 688, 691, 1164, 1201.
margin_kern_node_size: 179, 228, 232, 688, 1164.
mark: 234, 295, 296, 1151, 1620.
`\mark` primitive: 295.
mark_class: 163, 222, 1033, 1068, 1155, 1638, 1641.
mark_class_node_size: 1631, 1636.
mark_node: 163, 172, 201, 209, 228, 232, 656, 686, 691, 773, 809, 877, 914, 945, 952, 1022, 1027, 1033, 1054, 1068, 1155.
mark_ptr: 163, 222, 228, 232, 1033, 1070, 1155, 1638, 1641.
mark_text: 337, 344, 353, 420.
mark_val: 1626, 1627, 1631, 1635, 1638, 1641.
`\marks` primitive: 1620.
marks_code: 326, 416, 419, 420, 1620.
mastication: 371.
match: 233, 319, 321, 322, 324, 425, 426.
match_chr: 322, 324, 423, 425, 434.
match_token: 319, 425, 426, 427, 428, 511.
matching: 335, 336, 369, 425.
`Math formula deleted...`: 1249.
math_ac: 1218, 1219.
math_accent: 234, 295, 296, 1100, 1218.
`\mathaccent` primitive: 295.
`\mathbin` primitive: 1210.
math_char: 723, 734, 763, 765, 767, 781, 785, 793, 796, 797, 798, 805, 1205, 1209, 1219.
`\mathchar` primitive: 295.
`\mathchardef` primitive: 1276.
math_char_def_code: 1276, 1277, 1278.
math_char_field: 258, 448, 469, 1205, 1209, 1219, 1277.
math_char_num: 234, 295, 296, 1100, 1205, 1208.
math_choice: 234, 295, 296, 1100, 1225.
`\mathchoice` primitive: 295.
math_choice_group: 299, 1226, 1227, 1228, 1471, 1489.
math_class_field: 258, 448, 1209, 1277.
`\mathclose` primitive: 1210.
math_code: 256, 258, 262, 447, 448, 1205, 1208.
`\mathcode` primitive: 1284.
math_code_base: 256, 261, 447, 448, 1284, 1285, 1286, 1287.
math_comp: 234, 1100, 1210, 1211, 1212.
math_fam_field: 448, 1205, 1209, 1219, 1277.
math_font_base: 256, 258, 260, 1284, 1285.
math_font_biggest: 12.
math_fraction: 1234, 1235.
math_given: 234, 447, 1100, 1205, 1208, 1276, 1277, 1278.
math_glue: 759, 775, 814.
math_group: 299, 1190, 1204, 1207, 1240, 1471, 1489.
`\mathinner` primitive: 1210.
math_kern: 760, 773.
math_left_group: 238, 299, 1119, 1122, 1123, 1204, 1245, 1471, 1489.
math_left_right: 1244, 1245.

math_limit_switch: 1212, 1213.
math_node: 171, 172, 201, 209, 228, 232, 458, 660, 691, 865, 877, 885, 914, 927, 929, 949, 952, 1134, 1510, 1517, 1539, 1544, 1547, 1549.
\mathop primitive: 1210.
\mathopen primitive: 1210.
\mathord primitive: 1210.
\mathpunct primitive: 1210.
math_quad: 742, 746, 1253.
math_radical: 1216, 1217.
\mathrel primitive: 1210.
math_shift: 233, 319, 324, 328, 377, 1144, 1191, 1192, 1247, 1251, 1260.
math_shift_group: 299, 1119, 1122, 1123, 1184, 1193, 1194, 1196, 1199, 1246, 1247, 1248, 1254, 1471, 1489.
math_shift_token: 319, 1101, 1119.
math_spacing: 812, 813.
math_style: 234, 1100, 1223, 1224, 1225.
math_surround: 273, 1250.
\mathsurround primitive: 274.
math_surround_code: 273, 274.
math_text_char: 723, 796, 797, 798, 799.
math_type: 723, 725, 729, 734, 740, 763, 765, 766, 777, 778, 780, 781, 785, 786, 793, 795, 796, 797, 798, 799, 800, 805, 807, 1130, 1147, 1205, 1209, 1219, 1222, 1230, 1235, 1239, 1240, 1245.
math_x_height: 742, 780, 801, 802, 803.
mathex: 743.
mathLeading: 742.
mathsy: 742.
max: 749.
max_answer: 109, 1604, 1610.
max_buf_stack: 30, 31, 361, 408, 1388, 1567, 1579.
max_char_code: 233, 333, 371, 374, 1287.
max_char_val: 319, 323, 387, 395, 402, 408, 414, 415, 499, 1088, 1343, 1579.
max_command: 235, 236, 237, 245, 388, 396, 400, 402, 414, 415, 513, 830, 1583.
max_d: 769, 770, 773, 808, 809, 810.
max_dead_cycles: 262, 266, 1066.
\maxdeadcycles primitive: 264.
max_dead_cycles_code: 262, 263, 264.
max_depth: 273, 1034, 1041.
\maxdepth primitive: 274.
max_depth_code: 273, 274.
max_dimen: 455, 495, 679, 710, 1064, 1071, 1199, 1200, 1202, 1546, 1547, 1548, 1601, 1603, 1609, 1657.
max_group_code: 299.
max_h: 628, 629, 679, 680, 769, 770, 773, 808, 809, 810.
max_halfword: 11, 14, 132, 133, 135, 146, 147, 148, 153, 154, 319, 320, 458, 868, 896, 898, 1036, 1045, 1050, 1071, 1160, 1303, 1377, 1379, 1389.
max_hlist_stack: 179, 181, 877.
max_hyph_char: 940, 941, 950, 951, 956, 970, 977, 1008, 1010, 1016, 1020, 1378, 1379.
max_hyphenatable_length: 12, 939, 943, 944, 946, 950, 951, 972, 991, 992, 1016.
max_in_open: 11, 14, 334, 358, 1470, 1584, 1585, 1587.
max_in_stack: 331, 351, 361, 1388.
max_integer: 1410, 1411.
max_internal: 235, 447, 474, 482, 490, 496.
max_nest_stack: 239, 241, 242, 1388.
max_non_prefixed_command: 234, 1265, 1324.
max_param_stack: 338, 361, 424, 1388.
max_print_line: 11, 14, 54, 58, 76, 202, 572, 676, 1334, 1566.
max_push: 628, 629, 655, 667, 680.
max_quarterword: 11, 12, 132, 133, 135, 179, 304, 845, 846, 998, 1174, 1379.
max_reg_help_line: 1622, 1623, 1624, 1625.
max_reg_num: 1622, 1623, 1624, 1625.
max_save_stack: 301, 302, 303, 1388.
max_selector: 54, 272, 341, 500, 505, 569, 676, 1311, 1333, 1431, 1433, 1436, 1564.
max_strings: 11, 38, 43, 133, 552, 560, 1364, 1388.
max_v: 628, 629, 679, 680.
maxdimen: 114.
\meaning primitive: 503.
meaning_code: 503, 504, 506, 507.
med_mu_skip: 250.
\medmuskip primitive: 252.
med_mu_skip_code: 250, 251, 252, 814.
mem: 11, 12, 137, 138, 140, 146, 148, 153, 155, 156, 157, 162, 164, 169, 170, 174, 175, 182, 184, 187, 188, 189, 190, 192, 197, 208, 212, 229, 231, 232, 247, 250, 305, 321, 421, 454, 524, 641, 692, 722, 723, 725, 728, 729, 763, 768, 786, 797, 817, 818, 820, 845, 864, 866, 867, 870, 871, 880, 891, 892, 895, 896, 898, 908, 909, 937, 979, 1203, 1205, 1214, 1217, 1219, 1235, 1240, 1301, 1302, 1365, 1366, 1393, 1417, 1445, 1484, 1538, 1544, 1565, 1567, 1599, 1626, 1631, 1653.
mem_bot: 11, 12, 14, 133, 138, 147, 148, 187, 189, 295, 445, 449, 461, 1275, 1280, 1281, 1291, 1361, 1362, 1365, 1366, 1643, 1644, 1645.
mem_end: 138, 140, 142, 189, 190, 192, 193, 196, 197, 200, 202, 208, 323, 804, 1365, 1366, 1388.
mem_max: 11, 12, 14, 132, 133, 138, 142, 146, 147, 190, 191.
mem_min: 11, 12, 133, 138, 142, 147, 190, 191,

192, 194, 195, 196, 197, 200, 204, 208, 804, 1303, 1366, 1388.
mem_top: 11, 12, 14, 133, 138, 187, 189, 1303, 1361, 1362, 1366.
memcpy: 169, 1445.
Memory usage...: 677.
memory_word: 132, 135, 136, 138, 208, 238, 244, 247, 279, 298, 301, 305, 583, 584, 744, 848, 1359, 1417, 1445, 1627.
message: 234, 1330, 1331, 1332.
\message primitive: 1331.
METAFONT: 625.
microseconds: 682, 1391, 1411, 1412, 1414.
mid: 581.
mid_line: 91, 333, 358, 374, 377, 382, 383, 384.
middle: 1507.
\middle primitive: 1507.
middle_noad: 238, 729, 1245, 1246, 1507, 1508.
min: 749.
min_halfword: 11, 132, 133, 134, 135, 137, 256, 1081, 1377, 1379, 1533, 1539, 1549, 1550.
min_internal: 234, 447, 474, 482, 490, 496.
min_o: 749.
min_quarterword: 12, 132, 133, 134, 135, 156, 158, 162, 211, 247, 304, 584, 585, 589, 591, 592, 601, 611, 667, 689, 710, 727, 739, 750, 756, 757, 844, 849, 851, 856, 974, 977, 978, 997, 998, 999, 1000, 1012, 1017, 1018, 1019, 1048, 1066, 1377, 1378, 1379.
minimal_demerits: 881, 882, 884, 893, 903, 1653.
minimum_demerits: 881, 882, 883, 884, 902, 903.
minor_tail: 966, 969, 970.
minus: 497.
Misplaced &: 1182.
Misplaced \cr: 1182.
Misplaced \noalign: 1183.
Misplaced \omit: 1183.
Misplaced \span: 1182.
Missing) inserted: 1595.
Missing = inserted: 538.
Missing # inserted...: 831.
Missing $ inserted: 1101, 1119.
Missing \cr inserted: 1186.
Missing \endcsname...: 407.
Missing \endgroup inserted: 1119.
Missing \right. inserted: 1119.
Missing { inserted: 437, 510, 1181.
Missing } inserted: 1119, 1181.
Missing 'to' inserted: 1136.
Missing 'to'...: 1279.
Missing $$ inserted: 1261.
Missing character: 616.
Missing control...: 1269.
Missing delimiter...: 1215.
Missing font identifier: 612.
Missing number...: 449, 480.
mkern: 234, 1100, 1111, 1112, 1113.
\mkern primitive: 1112.
ml_field: 238, 239, 244.
mlist: 769, 808.
mlist_penalties: 762, 763, 769, 798, 1248, 1250, 1253.
mlist_to_hlist: 735, 762, 763, 768, 769, 777, 798, 808, 1248, 1250, 1253.
mm: 493.
mmode: 237, 238, 239, 244, 536, 761, 823, 824, 848, 855, 860, 1084, 1099, 1100, 1102, 1110, 1111, 1127, 1134, 1146, 1151, 1163, 1164, 1166, 1170, 1174, 1184, 1190, 1194, 1199, 1204, 1208, 1212, 1216, 1218, 1221, 1225, 1229, 1234, 1244, 1247, 1248, 1442, 1443, 1444, 1489, 1553.
mode: 2, 237, 238, 239, 241, 242, 329, 452, 456, 458, 536, 761, 823, 824, 833, 834, 835, 844, 847, 852, 855, 856, 857, 860, 1079, 1083, 1084, 1088, 1089, 1103, 1105, 1110, 1130, 1132, 1134, 1137, 1140, 1145, 1147, 1148, 1149, 1150, 1153, 1157, 1159, 1164, 1171, 1173, 1174, 1190, 1192, 1199, 1221, 1248, 1250, 1254, 1297, 1433, 1434, 1440, 1442, 1443, 1444, 1446, 1447, 1553.
mode_field: 238, 239, 244, 456, 848, 855, 1298, 1489, 1491.
mode_line: 238, 239, 241, 242, 334, 852, 863, 1079.
month: 262, 267, 653, 1382.
\month primitive: 264.
month_code: 262, 263, 264.
months: 569, 571.
more_name: 547, 551, 561, 566.
\moveleft primitive: 1125.
move_past: 655, 660, 663, 667, 669, 672.
\moveright primitive: 1125.
movement: 643, 645, 652.
movement_node_size: 641, 643, 651.
mskip: 234, 1100, 1111, 1112, 1113.
\mskip primitive: 1112.
mskip_code: 1112, 1114.
mstate: 643, 647, 648.
mtype: 4.
mu: 481, 482, 484, 488, 490, 496, 497.
mu: 491.
mu_error: 442, 463, 484, 490, 496, 1591.
\muexpr primitive: 1589.
mu_glue: 173, 179, 217, 458, 760, 775, 1112, 1114, 1115.
mu_mult: 759, 760.

mu_skip: 250, 461.
\muskip primitive: 445.
mu_skip_base: 250, 253, 255, 1278, 1291.
\muskipdef primitive: 1276.
mu_skip_def_code: 1276, 1277, 1278.
\mutoglue primitive: 1616.
mu_to_glue_code: 1616, 1617, 1618.
mu_val: 444, 445, 447, 458, 461, 463, 464, 484, 486, 490, 496, 500, 1114, 1278, 1282, 1290, 1291, 1589, 1590, 1591, 1598, 1626, 1631, 1634.
mu_val_limit: 1626, 1632, 1649.
mult_and_add: 109.
mult_integers: 109, 1294, 1606.
multiply: 235, 295, 296, 1264, 1289, 1290, 1294.
\multiply primitive: 295.
Must increase the x: 1357.
n: 47, 69, 70, 71, 73, 95, 98, 109, 110, 111, 116, 118, 176, 178, 198, 200, 208, 251, 263, 273, 278, 322, 328, 329, 345, 423, 517, 533, 553, 554, 558, 613, 744, 749, 759, 760, 793, 839, 848, 960, 988, 998, 1031, 1046, 1047, 1048, 1066, 1133, 1173, 1192, 1265, 1329, 1347, 1392, 1533, 1549, 1593, 1608, 1610, 1630, 1633.
name: 330, 332, 333, 334, 337, 341, 343, 344, 353, 358, 359, 361, 367, 390, 392, 424, 518, 572, 1566.
NAME: 742.
name_field: 88, 89, 330, 332, 1585, 1586.
name_in_progress: 412, 561, 562, 563, 1312.
name_length: 26, 51, 554, 558, 560, 744.
name_length16: 26, 560.
name_of_file: 26, 27, 51, 554, 558, 560, 565, 595, 744, 1445.
name_of_file16: 26, 560.
nameoffile: 467.
nat: 749.
native_char: 744.
native_font: 169, 201, 656, 688, 744, 749, 781, 799, 806, 807, 949, 1088, 1416, 1420, 1426, 1430, 1444.
native_font_type_flag: 744.
native_glyph: 169, 688, 749, 781, 783, 793, 799, 806, 807, 1416, 1426, 1430, 1444.
native_glyph_count: 169, 744, 1417.
native_glyph_info_ptr: 169, 688, 744, 1417, 1430.
native_glyph_info_size: 169.
native_len: 60, 61, 1088.
native_length: 169, 656, 744, 946, 947, 949, 957, 1088, 1415, 1420, 1430.
native_node_size: 169, 744.
native_room: 60, 1088.
native_size: 169, 744, 781, 799, 1088, 1417, 1418.
native_text: 60, 61, 62, 744, 1088.
native_text_size: 60, 61, 62.
native_word: 169, 656, 781, 946, 1088, 1420, 1536.
native_word_node: 169, 201, 656, 744, 957, 1416, 1417, 1418, 1420, 1430.
native_word_node_AT: 169, 201, 744, 1416, 1417, 1418, 1420, 1430.
natural: 683, 748, 758, 763, 770, 778, 780, 781, 792, 798, 800, 803, 844, 847, 854, 1031, 1075, 1154, 1179, 1248, 1253, 1258, 1557.
nd: 575, 576, 595, 600, 601, 604.
ne: 575, 576, 595, 600, 601, 604.
negate: 16, 69, 107, 109, 110, 111, 116, 119, 127, 198, 464, 465, 474, 482, 496, 823, 1413, 1591, 1604, 1608, 1610.
negative: 110, 116, 118, 119, 447, 464, 474, 475, 482, 496, 1591, 1604, 1608, 1610.
nest: 238, 239, 242, 243, 244, 245, 447, 456, 823, 848, 855, 1049, 1298, 1489, 1491.
nest_ptr: 239, 241, 242, 243, 244, 456, 823, 848, 855, 1049, 1071, 1077, 1145, 1154, 1199, 1254, 1298, 1489.
nest_size: 11, 239, 242, 244, 447, 1298, 1388, 1489.
new_character: 617, 618, 799, 969, 1171, 1177, 1178.
new_choice: 731, 1226.
new_delta_from_break_width: 892.
new_delta_to_break_width: 891.
new_disc: 167, 957, 1088, 1089, 1171.
new_edge: 1529, 1532, 1549.
new_font: 1310, 1311.
new_glue: 177, 178, 749, 758, 814, 834, 841, 843, 857, 1095, 1097, 1108, 1114, 1225.
new_graf: 1144, 1145, 1444.
new_hlist: 768, 770, 787, 792, 793, 794, 798, 800, 810, 815.
new_hyph_exceptions: 988, 1306.
new_index: 1626, 1627, 1630.
new_interaction: 1318, 1319, 1505, 1506.
new_kern: 180, 748, 758, 778, 781, 782, 791, 795, 797, 799, 800, 803, 964, 1088, 1094, 1115, 1166, 1167, 1179, 1258, 1531, 1551, 1557.
new_lig_item: 166, 965, 1094.
new_ligature: 166, 964, 1089.
new_line: 333, 361, 373, 374, 375, 377, 518, 572.
new_line_char: 63, 262, 270, 1387, 1389, 1565.
\newlinechar primitive: 264.
new_line_char_code: 262, 263, 264.
new_margin_kern: 688, 929, 935.
new_math: 171, 1250, 1513, 1517, 1519, 1522, 1533, 1545, 1557.
new_native_character: 744, 752, 781, 799, 805, 957.

new_native_word_node: 656, 744, 947, 957, 1088, 1420.
new_noad: 728, 763, 786, 797, 1130, 1147, 1204, 1209, 1212, 1222, 1231, 1245.
new_null_box: 158, 749, 752, 756, 763, 791, 794, 827, 841, 857, 1072, 1108, 1145, 1147, 1551.
new_param_glue: 176, 178, 721, 744, 826, 864, 934, 935, 1095, 1097, 1145, 1257, 1259, 1260, 1551.
new_patterns: 1014, 1306.
new_penalty: 183, 744, 815, 864, 938, 1108, 1157, 1257, 1259, 1260.
new_randoms: 114, 128, 129.
new_rule: 161, 498, 708, 747.
new_save_level: 304, 684, 822, 833, 839, 1079, 1117, 1153, 1171, 1173, 1190.
new_skip_param: 178, 721, 1023, 1055, 1557.
new_spec: 175, 178, 464, 497, 749, 874, 1030, 1058, 1096, 1097, 1293, 1294, 1591, 1601, 1602, 1664.
new_string: 54, 57, 58, 500, 505, 506, 653, 678, 1311, 1333, 1382, 1410, 1431, 1436, 1498, 1564.
new_style: 730, 1225.
new_trie_op: 997, 998, 999, 1019.
new_whatsit: 1404, 1405, 1409, 1439, 1440, 1444, 1445, 1450.
new_write_whatsit: 1405, 1406, 1407, 1408.
next: 282, 284, 286, 287.
next_break: 925, 926.
next_char: 580, 785, 797, 963, 1093.
next_p: 655, 660, 664, 667, 668, 669, 671, 673, 1533, 1535, 1536.
next_random: 128, 130, 131.
nh: 575, 576, 595, 600, 601, 604.
ni: 575, 576, 595, 600, 601, 604.
nil: 16.
nk: 575, 576, 595, 600, 601, 608.
nl: 63, 575, 576, 580, 595, 600, 601, 604, 608, 611, 1564, 1565.
nn: 341, 342.
`No pages of output`: 680.
no_align: 234, 295, 296, 833, 1180.
`\noalign` primitive: 295.
no_align_error: 1180, 1183.
no_align_group: 299, 816, 833, 1187, 1471, 1489.
no_boundary: 234, 295, 296, 1084, 1092, 1099, 1144.
`\noboundary` primitive: 295.
no_break_yet: 877, 884, 885.
no_expand: 236, 295, 296, 396, 399.
`\noexpand` primitive: 295.
no_expand_flag: 388, 513, 541.
no_extenders: 749.
`\noindent` primitive: 1142.
no_limits: 724, 1210, 1211.
`\nolimits` primitive: 1210.
no_new_control_sequence: 282, 284, 286, 289, 294, 395, 408, 1390, 1451, 1579.
no_pdf_output: 567, 568, 678.
no_print: 54, 57, 58, 79, 102.
no_shrink_error_yet: 873, 874, 875.
no_tag: 579, 604.
noad_size: 723, 728, 740, 797, 809, 1240, 1241.
node: 744.
node_is_invisible_to_interword_space: 656, 657, 1088.
node_list_display: 206, 210, 214, 216, 221, 223.
node_r_stays_active: 878, 899, 902.
node_size: 146, 148, 149, 150, 152, 189, 194, 1365, 1366.
nom: 595, 596, 598, 611, 744.
non_address: 584, 587, 611, 963, 970, 1088.
non_char: 584, 587, 611, 946, 950, 951, 954, 962, 963, 964, 965, 969, 970, 971, 1086, 1088, 1089, 1092, 1093, 1094, 1377.
non_discardable: 172, 877, 927.
non_math: 1100, 1117, 1198.
non_script: 234, 295, 296, 1100, 1225.
`\nonscript` primitive: 295, 775.
none_seen: 647, 648.
`NONEXISTENT`: 292.
`Nonletter`: 1016.
nonnegative_integer: 73, 105, 111, 198.
nonstop_mode: 77, 90, 390, 393, 519, 1316, 1317.
`\nonstopmode` primitive: 1316.
nop: 619, 621, 622, 624, 626.
norm_min: 1145, 1254, 1439, 1440.
norm_rand: 114, 131, 507.
normal: 157, 158, 173, 174, 177, 179, 180, 189, 203, 212, 215, 217, 335, 361, 366, 401, 402, 473, 482, 506, 508, 515, 517, 520, 524, 525, 536, 542, 655, 656, 663, 667, 672, 690, 699, 700, 701, 702, 706, 707, 708, 709, 714, 715, 716, 718, 719, 720, 724, 728, 738, 749, 759, 775, 793, 825, 849, 858, 859, 873, 874, 877, 945, 949, 950, 952, 1030, 1042, 1058, 1063, 1099, 1210, 1217, 1219, 1235, 1255, 1273, 1274, 1275, 1293, 1347, 1493, 1533, 1577, 1602, 1605, 1654.
`\normaldeviate` primitive: 503.
normal_deviate_code: 503, 504, 506, 507.
normal_paragraph: 822, 833, 835, 1079, 1124, 1137, 1148, 1150, 1153, 1221.
normalize_glue: 1602, 1605.
normalize_selector: 82, 96, 97, 98, 99, 198, 911.
`Not a letter`: 991.
not_aat_font_error: 1454, 1457, 1460.

not_aat_gr_font_error: 1454, 1457, 1460.
not_exp: 371, 382.
not_found: 15, 45, 46, 482, 490, 595, 605, 643, 647, 648, 877, 944, 984, 985, 988, 995, 1007, 1009, 1024, 1026, 1027, 1192, 1200, 1425, 1544, 1630, 1657.
not_found1: 15, 988, 1630.
not_found2: 15, 1630.
not_found3: 15, 1630.
not_found4: 15, 1630.
not_native_font_error: 1444, 1454, 1457, 1458, 1460.
not_ot_font_error: 1454, 1457.
`notexpanded:`: 285.
np: 575, 576, 595, 600, 601, 610, 611.
nucleus: 723, 724, 725, 728, 729, 732, 738, 740, 763, 768, 777, 778, 779, 780, 781, 785, 786, 793, 794, 796, 797, 798, 799, 805, 1130, 1147, 1204, 1205, 1209, 1212, 1217, 1219, 1222, 1240, 1245.
null: 137, 138, 140, 142, 144, 145, 147, 148, 157, 158, 166, 167, 173, 174, 175, 176, 177, 178, 189, 193, 194, 198, 201, 202, 208, 226, 227, 228, 230, 236, 238, 241, 242, 244, 245, 248, 249, 258, 259, 305, 322, 325, 329, 336, 337, 342, 344, 355, 361, 387, 388, 392, 405, 408, 416, 417, 420, 424, 425, 426, 431, 434, 441, 444, 449, 454, 457, 461, 487, 499, 501, 506, 507, 508, 513, 517, 524, 525, 532, 540, 543, 584, 587, 611, 613, 618, 642, 647, 651, 655, 656, 657, 661, 667, 670, 687, 688, 689, 691, 696, 697, 700, 706, 708, 710, 715, 718, 723, 727, 731, 734, 744, 749, 758, 761, 762, 763, 764, 769, 774, 775, 781, 796, 798, 799, 800, 805, 808, 809, 814, 815, 819, 822, 824, 825, 831, 832, 837, 838, 839, 840, 842, 844, 845, 847, 849, 852, 853, 854, 855, 860, 869, 877, 885, 888, 894, 895, 896, 898, 900, 904, 905, 906, 907, 911, 912, 913, 915, 917, 920, 925, 926, 927, 929, 930, 931, 932, 933, 935, 936, 937, 938, 943, 949, 951, 956, 957, 960, 961, 962, 964, 965, 967, 968, 969, 970, 971, 972, 982, 986, 989, 1022, 1023, 1024, 1026, 1027, 1031, 1032, 1033, 1035, 1045, 1046, 1047, 1048, 1052, 1053, 1054, 1063, 1064, 1065, 1066, 1068, 1069, 1070, 1071, 1072, 1074, 1075, 1076, 1077, 1080, 1081, 1082, 1084, 1086, 1088, 1089, 1090, 1091, 1092, 1094, 1096, 1097, 1124, 1128, 1129, 1130, 1133, 1134, 1137, 1141, 1145, 1150, 1164, 1175, 1177, 1178, 1185, 1190, 1193, 1199, 1200, 1203, 1221, 1228, 1230, 1235, 1238, 1239, 1240, 1248, 1250, 1253, 1256, 1259, 1260, 1280, 1281, 1301, 1302, 1337, 1342, 1350, 1365, 1366, 1389, 1393, 1408, 1409, 1410, 1420, 1431, 1432, 1438, 1478, 1484, 1493, 1501, 1516, 1517, 1518, 1519, 1522, 1531, 1533, 1535, 1541, 1542, 1544, 1545, 1546, 1549, 1556, 1557, 1562, 1567, 1568, 1569, 1579, 1593, 1594, 1595, 1620, 1626, 1627, 1628, 1629, 1630, 1631, 1632, 1634, 1635, 1636, 1637, 1638, 1639, 1640, 1641, 1642, 1643, 1647, 1648, 1649, 1652, 1660, 1663, 1671, 1674, 1677.
null delimiter: 266, 1119.
null_character: 590, 591, 765, 766.
null_code: 22, 258.
null_cs: 248, 292, 293, 384, 408, 1311, 1579.
null_delimiter: 726, 727, 1235.
null_delimiter_space: 273, 749.
`\nulldelimiterspace` primitive: 274.
null_delimiter_space_code: 273, 274.
null_flag: 160, 161, 498, 693, 827, 841, 849.
null_font: 258, 587, 588, 595, 612, 653, 705, 744, 749, 750, 765, 805, 912, 1311, 1374, 1375, 1393.
`\nullfont` primitive: 588.
null_list: 14, 187, 414, 828.
null_ptr: 169, 688, 744, 1417, 1430.
num: 485, 493, 621, 623, 626.
num_error: 1601, 1604, 1608, 1610.
`\numexpr` primitive: 1589.
num_font_dimens: 744.
num_style: 745, 788.
number: 742.
`Number too big`: 479.
`\number` primitive: 503.
number_code: 503, 504, 506, 507.
number_fonts: 12.
number_math_families: 12, 469, 1205.
number_math_fonts: 12, 256, 258.
number_regs: 12, 250, 256, 258, 262, 273.
number_usvs: 12, 248, 256, 258, 262, 266, 940.
numerator: 725, 732, 739, 740, 788, 1235, 1239.
num1: 742, 788.
num2: 742, 788.
num3: 742, 788.
nw: 575, 576, 595, 600, 601, 604.
nx_plus_y: 109, 490, 759, 1294, 1606.
o: 294, 643, 689, 710, 839, 848, 1593.
octal_token: 472, 478.
odd: 66, 104, 120, 171, 219, 539, 783, 802, 914, 951, 955, 957, 962, 963, 967, 968, 1265, 1272, 1302, 1349, 1498, 1611, 1630, 1635.
off_save: 1117, 1118, 1148, 1149, 1184, 1185, 1194, 1246, 1247.
offs: 744.
`OK`: 1352.
OK_so_far: 474, 479.
OK_to_interrupt: 92, 100, 101, 102, 357, 1085.
old_l: 877, 883, 898.

old_mode: 1433, 1434.
old_rover: 153.
old_setting: 271, 272, 341, 342, 500, 505, 506, 569, 616, 653, 676, 678, 1311, 1333, 1410, 1431, 1433, 1436, 1498, 1564.
omit: 234, 295, 296, 836, 837, 1180.
`\omit` primitive: 295.
omit_error: 1180, 1183.
omit_template: 187, 837, 838.
`Only one # is allowed...`: 832.
oo: 749.
op_byte: 580, 592, 785, 797, 963, 965, 1094.
op_noad: 724, 732, 738, 740, 769, 771, 776, 793, 805, 809, 1210, 1211, 1213.
op_start: 974, 975, 978, 999, 1379.
open_area: 1395, 1406, 1416, 1437.
open_ext: 1395, 1406, 1416, 1437.
open_fmt_file: 559, 1391.
`\openin` primitive: 1326.
open_log_file: 82, 96, 390, 506, 567, 569, 570, 572, 676, 1311, 1389.
open_name: 1395, 1406, 1416, 1437.
open_noad: 724, 732, 738, 740, 771, 776, 805, 808, 809, 810, 1210, 1211.
open_node: 1395, 1398, 1401, 1403, 1416, 1417, 1418, 1436.
open_node_size: 1395, 1406, 1417, 1418.
open_or_close_in: 1328, 1329.
`\openout` primitive: 1398.
open_parens: 334, 361, 392, 572, 1389, 1566.
`\or` primitive: 526.
or_code: 524, 526, 527, 535, 544, 1347, 1478.
ord: 20, 744.
ord_noad: 723, 724, 728, 729, 732, 738, 740, 771, 772, 776, 796, 797, 805, 809, 812, 813, 1129, 1209, 1210, 1211, 1240.
order: 203.
oriental characters: 156, 621.
ot_assembly_ptr: 749, 751, 781, 783, 793.
ot_font_flag: 744.
ot_font_get: 1454.
ot_font_get_1: 1454.
ot_font_get_2: 1454.
ot_font_get_3: 1454.
ot_get_font_metrics: 744.
ot_min_connector_overlap: 749.
ot_part_count: 749.
ot_part_end_connector: 749.
ot_part_full_advance: 749.
ot_part_glyph: 749.
ot_part_is_extender: 749.
ot_part_start_connector: 749.
otgr_font_flag: 584, 744.
other_A_token: 479.
other_char: 233, 258, 319, 321, 324, 328, 377, 479, 499, 505, 561, 989, 1015, 1084, 1088, 1092, 1144, 1178, 1205, 1208, 1214.
other_token: 319, 439, 472, 475, 479, 499, 538, 1119, 1275, 1572, 1595, 1596.
othercases: 10.
others: 10, 1436.
`Ouch...clobbered`: 1386.
out_param: 233, 319, 321, 324, 387, 506.
out_param_token: 319, 514.
out_what: 1426, 1430, 1436, 1438.
`\outer` primitive: 1262.
outer_call: 236, 305, 369, 381, 383, 384, 387, 396, 421, 425, 430, 828, 1206, 1349, 1432.
outer_doing_leaders: 655, 666, 667, 675.
output: 4.
`Output loop...`: 1078.
`Output routine didn't use...`: 1082.
`Output written on x`: 680.
`\output` primitive: 256.
output_active: 455, 705, 717, 1040, 1043, 1044, 1048, 1059, 1079, 1080.
output_file_extension: 564, 567, 568.
output_file_name: 567, 568, 680.
output_group: 299, 1079, 1154, 1471, 1489.
output_penalty: 262.
`\outputpenalty` primitive: 264.
output_penalty_code: 262, 263, 264, 1067.
output_routine: 256, 1066, 1079.
output_routine_loc: 256, 257, 258, 337, 353, 1280.
output_text: 337, 344, 353, 1079, 1080.
`\over` primitive: 1232.
over_code: 1232, 1233, 1236.
over_noad: 729, 732, 738, 740, 776, 809, 1210.
`\overwithdelims` primitive: 1232.
overbar: 748, 777, 780.
overbarExtraAscender: 742.
overbarRuleThickness: 742.
overbarVerticalGap: 742.
overflow: 35, 42, 43, 98, 142, 147, 242, 287, 290, 294, 303, 304, 351, 358, 408, 424, 552, 615, 994, 998, 1008, 1018, 1387, 1579, 1593.
overflow in arithmetic: 9, 108.
`Overfull \hbox...`: 708.
`Overfull \vbox...`: 719.
overfull boxes: 902.
overfull_rule: 273, 708, 848, 852.
`\overfullrule` primitive: 274.
overfull_rule_code: 273, 274.
`\overline` primitive: 1210.

p: 116, 118, 142, 145, 147, 152, 153, 158, 161, 166, 167, 171, 175, 176, 177, 178, 180, 183, 192, 197, 198, 200, 202, 204, 208, 224, 226, 227, 228, 230, 244, 286, 289, 292, 293, 306, 307, 308, 309, 311, 314, 322, 325, 329, 336, 345, 353, 355, 366, 396, 423, 441, 447, 485, 499, 500, 508, 517, 532, 533, 618, 643, 651, 655, 667, 676, 689, 710, 721, 728, 730, 731, 733, 734, 744, 747, 748, 749, 752, 754, 758, 759, 760, 763, 769, 778, 781, 787, 793, 796, 800, 820, 822, 835, 839, 847, 848, 874, 960, 988, 1002, 1003, 1007, 1011, 1013, 1014, 1020, 1022, 1024, 1047, 1048, 1066, 1118, 1122, 1129, 1133, 1140, 1147, 1155, 1159, 1164, 1167, 1173, 1177, 1192, 1205, 1209, 1214, 1228, 1230, 1238, 1245, 1248, 1265, 1290, 1298, 1342, 1347, 1356, 1357, 1403, 1404, 1415, 1431, 1433, 1436, 1489, 1493, 1529, 1533, 1544, 1549, 1555, 1564, 1567, 1568, 1588, 1593, 1632, 1634, 1648, 1649, 1650, 1651, 1652.
p_1: 1665.
pack_begin_line: 703, 704, 705, 717, 852, 863.
pack_buffered_name: 558, 559.
pack_cur_name: 564, 565, 572, 1329, 1437, 1445, 1456.
pack_file_name: 554, 564, 572, 595, 598.
pack_job_name: 564, 567, 569, 1382.
pack_lig: 1089.
package: 1139, 1140.
packed_ASCII_code: 18, 38, 39, 1001.
packed_UTF16_code: 18.
page: 334, 1445.
page_contents: 455, 1034, 1040, 1041, 1045, 1054, 1055, 1062.
page_depth: 1036, 1041, 1045, 1056, 1057, 1058, 1062, 1064, 1424.
\pagedepth primitive: 1037.
page_disc: 1053, 1077, 1080, 1670, 1671.
\pagediscards primitive: 1672.
\pagefilstretch primitive: 1037.
\pagefillstretch primitive: 1037.
\pagefilllstretch primitive: 1037.
page_goal: 1034, 1036, 1040, 1041, 1059, 1060, 1061, 1062, 1063, 1064.
\pagegoal primitive: 1037.
page_head: 187, 241, 1034, 1040, 1042, 1045, 1068, 1071, 1077, 1080, 1108.
page_ins_head: 187, 1035, 1040, 1059, 1062, 1072, 1073, 1074.
page_ins_node_size: 1035, 1063, 1073.
page_loc: 676, 678.
page_max_depth: 1034, 1036, 1041, 1045, 1057, 1071.
page_shrink: 1036, 1039, 1058, 1061, 1062, 1063.
\pageshrink primitive: 1037.
page_so_far: 455, 1036, 1039, 1041, 1058, 1061, 1063, 1299.
page_stack: 334.
\pagestretch primitive: 1037.
page_tail: 241, 1034, 1040, 1045, 1052, 1054, 1071, 1077, 1080, 1108.
page_total: 1036, 1039, 1056, 1057, 1058, 1061, 1062, 1064, 1424.
\pagetotal primitive: 1037.
panicking: 190, 191, 1085, 1393.
\par primitive: 364.
par_end: 233, 364, 365, 1100, 1148.
par_fill_skip: 250, 864, 1653, 1654, 1657, 1664.
\parfillskip primitive: 252.
par_fill_skip_code: 250, 251, 252, 864.
par_indent: 273, 1145, 1147.
\parindent primitive: 274.
par_indent_code: 273, 274.
par_loc: 363, 364, 381, 1367, 1368.
\parshape primitive: 295.
\parshapedimen primitive: 1482.
par_shape_dimen_code: 1482, 1483, 1484.
\parshapeindent primitive: 1482.
par_shape_indent_code: 1482, 1483, 1484.
\parshapelength primitive: 1482.
par_shape_length_code: 1482, 1483, 1484.
par_shape_loc: 256, 258, 259, 295, 296, 457, 1124, 1302.
par_shape_ptr: 256, 258, 259, 457, 862, 895, 896, 898, 937, 1124, 1203, 1303, 1484.
par_skip: 250, 1145.
\parskip primitive: 252.
par_skip_code: 250, 251, 252, 1145.
par_token: 363, 364, 369, 426, 429, 433, 1149, 1368.
Paragraph ended before...: 430.
param: 577, 582, 593.
param_base: 585, 587, 593, 601, 609, 610, 611, 613, 615, 742, 743, 744, 1096, 1376, 1377.
param_end: 593.
param_ptr: 338, 353, 354, 361, 424.
param_size: 11, 338, 424, 1388.
param_stack: 337, 338, 354, 389, 422, 423, 424.
param_start: 337, 353, 354, 389.
parameter: 337, 344, 389.
parameters for symbols: 742, 743.
Parameters...consecutively: 511.
Pascal-H: 3, 4, 9, 10, 27, 28, 33, 34.
Pascal: 1, 10, 735, 812.
pass_number: 869, 893, 912.
pass_text: 396, 529, 535, 544, 545.

passive: 869, 893, 894, 912, 913.
passive_node_size: 869, 893, 913.
Patterns can be...: 1306.
\patterns primitive: 1304.
pause_for_instructions: 100, 102.
pausing: 262, 393.
\pausing primitive: 264.
pausing_code: 262, 263, 264.
pc: 493.
pdf_box_type: 1084, 1445.
\creationdate primitive: 503.
pdf_creation_date_code: 503, 504, 506.
pdf_error: 198, 506, 877, 1410.
pdf_file_code: 1398, 1399, 1401, 1403.
\filedump primitive: 503.
pdf_file_dump_code: 503, 504, 506.
\filemoddate primitive: 503.
pdf_file_mod_date_code: 503, 504, 506.
\filesize primitive: 503.
pdf_file_size_code: 503, 504, 506.
pdf_last_x_pos: 1427, 1449, 1454.
\pdflastxpos primitive: 450.
pdf_last_x_pos_code: 450, 451, 1454.
pdf_last_y_pos: 1427, 1449, 1454.
\pdflastypos primitive: 450.
pdf_last_y_pos_code: 450, 451, 1454.
\mdfivesum primitive: 503.
pdf_mdfive_sum_code: 503, 504, 506.
pdf_node: 170, 889, 890, 918, 919, 1416, 1417, 1418, 1419, 1420, 1421, 1422, 1424, 1425, 1426, 1430, 1445, 1536.
pdf_page_height: 273, 678, 1428.
\pdfpageheight primitive: 274.
pdf_page_height_code: 273, 274.
pdf_page_width: 273, 678, 1428.
\pdfpagewidth primitive: 274.
pdf_page_width_code: 273, 274.
\pdfsavepos primitive: 1399.
pdf_save_pos_node: 1398, 1399, 1401, 1403, 1416, 1417, 1418, 1426, 1430, 1450.
pdf_scan_ext_toks: 505.
\shellescape primitive: 450.
pdf_shell_escape_code: 450, 451, 458.
\strcmp primitive: 503.
pdf_strcmp_code: 503, 504, 506, 507.
pdfbox_art: 1084, 1436, 1445.
pdfbox_bleed: 1084, 1436, 1445.
pdfbox_crop: 1084, 1436, 1445.
pdfbox_media: 1084, 1436, 1445.
pdfbox_none: 1084, 1445.
pdfbox_trim: 1084, 1436, 1445.
pdftex_convert_codes: 503.
pdftex_first_expand_code: 503.
pdftex_first_extension_code: 1398.
pdftex_first_rint_code: 450.
pdftex_last_item_codes: 450.
pen: 769, 809, 815, 925, 938.
penalties: 1156.
penalties: 769, 815.
penalty: 182, 183, 220, 259, 458, 864, 914, 938, 1027, 1050, 1054, 1064, 1065, 1067, 1677.
\penalty primitive: 295.
penalty_node: 182, 183, 209, 228, 232, 458, 656, 773, 809, 815, 864, 865, 877, 885, 904, 914, 927, 945, 952, 1022, 1027, 1050, 1054, 1064, 1065, 1067, 1161.
pg_field: 238, 239, 244, 245, 456, 1298.
pi: 877, 879, 899, 904, 907, 1024, 1026, 1027, 1028, 1048, 1054, 1059, 1060.
pic_file_code: 1398, 1399, 1401, 1403.
pic_node: 170, 889, 890, 918, 919, 1416, 1417, 1418, 1419, 1420, 1421, 1422, 1424, 1425, 1426, 1430, 1445, 1536.
pic_node_size: 170, 1417, 1445.
pic_out: 1426, 1430, 1436.
pic_page: 170, 1436, 1445.
pic_path: 1445.
pic_path_byte: 1416, 1436.
pic_path_length: 170, 1416, 1417, 1436, 1445.
pic_pdf_box: 170, 1436, 1445.
pic_transform1: 170, 1436, 1445.
pic_transform2: 170, 1436, 1445.
pic_transform3: 170, 1436, 1445.
pic_transform4: 170, 1436, 1445.
pic_transform5: 170, 1436, 1445.
pic_transform6: 170, 1436, 1445.
plain: 556, 559, 1385.
plane_and_fam_field: 723, 733, 765, 797, 1205, 1209, 1219.
Plass, Michael Frederick: 2, 861.
Please type...: 390, 565.
Please use \mathaccent...: 1220.
PLtoTF: 596.
plus: 497.
point_token: 472, 474, 482, 487.
pointer: 137, 138, 140, 142, 145, 146, 147, 152, 153, 158, 161, 166, 167, 169, 171, 175, 176, 177, 178, 180, 181, 183, 190, 192, 197, 198, 224, 226, 227, 228, 230, 238, 244, 278, 282, 283, 286, 289, 293, 305, 306, 307, 308, 309, 311, 314, 325, 327, 329, 335, 336, 338, 353, 355, 363, 366, 396, 416, 422, 423, 441, 447, 485, 496, 498, 499, 500, 505, 508, 517, 524, 532, 533, 584, 595, 618, 628, 641, 643, 651, 655, 667, 676, 686, 688, 689, 695, 710,

721, 728, 730, 731, 733, 734, 744, 747, 748, 749, 752, 754, 758, 759, 760, 762, 763, 765, 769, 777, 778, 779, 780, 781, 787, 793, 796, 800, 810, 818, 820, 822, 835, 839, 847, 848, 862, 869, 874, 876, 877, 878, 881, 910, 920, 925, 940, 953, 954, 960, 961, 966, 980, 988, 1022, 1024, 1031, 1034, 1036, 1047, 1048, 1066, 1086, 1097, 1118, 1122, 1128, 1129, 1133, 1140, 1147, 1155, 1159, 1164, 1167, 1173, 1177, 1192, 1205, 1209, 1214, 1228, 1230, 1238, 1245, 1248, 1252, 1265, 1290, 1301, 1311, 1342, 1347, 1356, 1357, 1400, 1403, 1404, 1410, 1415, 1431, 1433, 1436, 1493, 1515, 1529, 1533, 1544, 1549, 1552, 1555, 1561, 1564, 1567, 1568, 1584, 1588, 1593, 1626, 1627, 1630, 1632, 1633, 1634, 1636, 1646, 1648, 1649, 1650, 1651, 1652, 1653, 1670.
pointer_node_size: 1631, 1632, 1648, 1652.
Poirot, Hercule: 1337.
pool_file: 47, 50, 51, 52, 53.
pool_name: 11, 51.
pool_pointer: 38, 39, 44, 45, 46, 63, 73, 74, 294, 441, 499, 500, 505, 548, 554, 638, 676, 983, 988, 1410, 1431, 1436, 1564.
pool_ptr: 38, 39, 41, 42, 43, 44, 47, 48, 52, 58, 74, 224, 287, 499, 500, 505, 506, 551, 560, 653, 656, 678, 744, 1363, 1364, 1386, 1388, 1393, 1410, 1431, 1436, 1498, 1565.
pool_size: 11, 38, 42, 52, 58, 224, 506, 560, 1364, 1388, 1393, 1410, 1431.
pop: 620, 621, 622, 626, 637, 644, 680.
pop_alignment: 820, 848.
pop_input: 352, 354, 359.
pop_lig_stack: 964, 965.
pop_LR: 1515, 1518, 1521, 1522, 1527, 1528, 1539, 1546, 1548, 1550.
pop_nest: 243, 844, 847, 860, 864, 1080, 1140, 1150, 1154, 1173, 1222, 1238, 1260, 1543.
pop_node: 877.
positive: 111, 198.
post: 619, 621, 622, 626, 627, 680.
post_break: 167, 201, 221, 228, 232, 877, 888, 906, 930, 932, 970, 1173.
post_disc_break: 925, 929, 932.
post_display_penalty: 262, 1259, 1260.
`\postdisplaypenalty` primitive: 264.
post_display_penalty_code: 262, 263, 264.
post_line_break: 924, 925, 1515.
post_post: 621, 622, 626, 627, 680.
pp: 689, 1420.
ppp: 689, 1420.
pre: 619, 621, 622, 653.
pre_adjust_head: 187, 936, 937, 1130, 1139, 1253, 1259.
pre_adjust_tail: 689, 691, 695, 696, 697, 844, 936, 937, 1130, 1139, 1253.
pre_break: 167, 201, 221, 228, 232, 877, 906, 917, 929, 930, 933, 957, 969, 1171, 1173.
pre_display_direction: 262, 1192, 1253, 1555.
`\predisplaydirection` primitive: 1467.
pre_display_direction_code: 262, 1199, 1467, 1469.
pre_display_penalty: 262, 1257, 1260.
`\predisplaypenalty` primitive: 264.
pre_display_penalty_code: 262, 263, 264.
pre_display_size: 273, 1192, 1199, 1202, 1257, 1544.
`\predisplaysize` primitive: 274.
pre_display_size_code: 273, 274, 1199.
pre_t: 1252, 1253, 1259.
preamble: 816, 822.
preamble: 818, 819, 820, 825, 834, 849, 852.
preamble of `DVI` file: 653.
precedes_break: 172, 916, 1027, 1054.
prefix: 235, 1262, 1263, 1264, 1265, 1581.
prefixed_command: 1264, 1265, 1324.
prepare_mag: 318, 492, 653, 680, 1387.
pretolerance: 262, 876, 911.
`\pretolerance` primitive: 264.
pretolerance_code: 262, 263, 264.
prev_break: 869, 893, 894, 925, 926.
prev_class: 1086, 1088.
prev_depth: 238, 239, 241, 452, 721, 823, 834, 835, 1079, 1110, 1137, 1153, 1221, 1260, 1296, 1297.
`\prevdepth` primitive: 450.
prev_dp: 1024, 1026, 1027, 1028, 1030, 1425.
prev_graf: 238, 239, 241, 242, 456, 862, 864, 912, 925, 938, 1145, 1203, 1254, 1296.
`\prevgraf` primitive: 295.
prev_o: 749.
prev_p: 181, 655, 656, 658, 660, 910, 911, 916, 1022, 1023, 1024, 1027, 1066, 1068, 1071, 1076, 1531, 1532.
prev_prev_r: 878, 880, 891, 892, 908.
prev_r: 877, 878, 880, 891, 892, 893, 899, 902, 908.
prev_rightmost: 198, 507, 877, 929.
prev_s: 910, 943, 949.
prevOffs: 744.
prim: 283, 284, 290, 1372, 1373.
prim_base: 283, 289.
prim_eq_level: 283, 294.
prim_eq_level_field: 283.
prim_eq_type: 283, 294, 402, 403, 536, 1099.
prim_eq_type_field: 283.
prim_eqtb: 294.
prim_eqtb_base: 248, 283, 292, 293, 403, 1099.

prim_equiv: 283, 294, 402, 403, 536, 1099.
prim_equiv_field: 283.
prim_is_full: 283, 290.
prim_lookup: 289, 294, 402, 403, 536, 1099.
prim_next: 283, 284, 289, 290.
prim_prime: 283, 289, 291.
prim_size: 248, 283, 284, 285, 290, 291, 1372, 1373.
prim_text: 283, 284, 289, 290, 292, 293.
prim_used: 283, 285, 290.
prim_val: 294.
primitive: 252, 256, 264, 274, 294, 295, 296, 328, 364, 410, 418, 445, 450, 503, 522, 526, 588, 828, 1037, 1106, 1112, 1125, 1142, 1161, 1168, 1195, 1210, 1223, 1232, 1242, 1262, 1273, 1276, 1284, 1304, 1308, 1316, 1326, 1331, 1340, 1345, 1385, 1386, 1398, 1399, 1452, 1467, 1473, 1476, 1479, 1482, 1485, 1494, 1496, 1499, 1502, 1507, 1511, 1558, 1570, 1573, 1581, 1589, 1612, 1616, 1620, 1672, 1675.
`\primitive` primitive: 295.
`\primitive` primitive (internalized): 402.
primitive_size: 283.
print: 54, 63, 64, 66, 72, 74, 75, 77, 88, 89, 90, 93, 95, 98, 99, 125, 198, 201, 203, 204, 208, 209, 210, 211, 212, 213, 214, 216, 217, 218, 219, 221, 223, 237, 244, 245, 251, 259, 260, 263, 273, 277, 292, 314, 318, 324, 328, 329, 336, 347, 353, 366, 368, 369, 393, 407, 429, 430, 432, 434, 462, 489, 491, 494, 500, 506, 507, 537, 544, 565, 569, 571, 595, 596, 602, 614, 616, 653, 676, 677, 678, 680, 702, 705, 708, 716, 717, 719, 734, 736, 739, 744, 766, 824, 894, 904, 990, 1032, 1039, 1040, 1041, 1060, 1065, 1069, 1078, 1103, 1118, 1149, 1186, 1220, 1267, 1286, 1291, 1311, 1313, 1315, 1349, 1350, 1352, 1363, 1365, 1372, 1374, 1376, 1378, 1382, 1388, 1389, 1392, 1393, 1401, 1415, 1416, 1436, 1445, 1457, 1461, 1471, 1472, 1489, 1490, 1491, 1501, 1514, 1523, 1566, 1576, 1585, 1587, 1588, 1634, 1663.
print_ASCII: 72, 200, 202, 328, 616, 733, 766.
print_c_string: 595.
print_char: 58, 59, 63, 64, 67, 68, 69, 70, 71, 73, 74, 86, 95, 98, 99, 107, 136, 196, 197, 200, 201, 202, 203, 204, 210, 212, 213, 214, 215, 216, 217, 218, 219, 222, 244, 245, 249, 255, 259, 260, 261, 268, 277, 278, 281, 292, 293, 296, 314, 315, 324, 326, 328, 329, 336, 343, 347, 348, 392, 419, 507, 544, 571, 572, 596, 616, 653, 676, 677, 680, 733, 766, 894, 904, 987, 1060, 1065, 1119, 1123, 1266, 1267, 1315, 1334, 1348, 1349, 1350, 1365, 1374, 1376, 1378, 1382, 1387, 1389, 1394, 1415, 1416, 1471, 1472, 1489, 1490, 1491, 1541, 1566, 1576, 1634.
print_cmd_chr: 249, 259, 296, 326, 328, 329, 353, 366, 452, 462, 538, 545, 1103, 1120, 1182, 1266, 1267, 1291, 1389, 1393, 1457, 1466, 1489, 1491, 1501, 1576, 1587, 1588, 1634.
print_cs: 292, 323, 344, 435.
print_current_string: 74, 208, 734.
print_delimiter: 733, 738, 739.
print_err: 76, 77, 97, 98, 99, 102, 125, 198, 318, 366, 368, 376, 404, 407, 429, 430, 432, 437, 442, 447, 448, 449, 452, 462, 467, 468, 469, 470, 471, 476, 479, 480, 489, 491, 494, 495, 506, 510, 511, 514, 521, 535, 538, 545, 565, 596, 612, 614, 679, 766, 824, 831, 832, 840, 874, 990, 991, 1014, 1015, 1016, 1017, 1030, 1032, 1047, 1058, 1063, 1069, 1078, 1081, 1082, 1101, 1103, 1118, 1120, 1122, 1123, 1132, 1136, 1138, 1149, 1153, 1164, 1174, 1175, 1181, 1182, 1183, 1186, 1189, 1213, 1215, 1220, 1231, 1237, 1246, 1249, 1251, 1261, 1266, 1267, 1269, 1279, 1286, 1290, 1291, 1295, 1297, 1298, 1306, 1312, 1313, 1337, 1352, 1358, 1435, 1444, 1445, 1446, 1457, 1458, 1466, 1506, 1576, 1593, 1595, 1622.
print_esc: 67, 90, 201, 202, 209, 210, 213, 214, 215, 216, 217, 218, 220, 221, 222, 223, 251, 253, 255, 257, 259, 260, 261, 263, 265, 268, 273, 275, 277, 292, 293, 296, 297, 322, 323, 324, 353, 365, 407, 411, 419, 451, 462, 504, 521, 523, 527, 535, 614, 733, 736, 737, 738, 739, 741, 824, 829, 840, 904, 990, 1014, 1015, 1032, 1038, 1040, 1063, 1069, 1082, 1107, 1113, 1119, 1123, 1126, 1143, 1149, 1153, 1162, 1169, 1174, 1183, 1186, 1189, 1197, 1211, 1220, 1233, 1243, 1246, 1263, 1267, 1274, 1277, 1285, 1295, 1298, 1305, 1309, 1317, 1327, 1332, 1341, 1346, 1349, 1376, 1389, 1401, 1415, 1416, 1453, 1459, 1468, 1469, 1474, 1477, 1480, 1483, 1486, 1489, 1491, 1495, 1497, 1500, 1501, 1503, 1508, 1510, 1512, 1559, 1571, 1574, 1575, 1576, 1582, 1588, 1590, 1613, 1617, 1634, 1643, 1644, 1673, 1676.
print_fam_and_char: 733, 734, 738.
print_file_name: 507, 553, 565, 596, 1376, 1416, 1445.
print_font_and_char: 202, 209, 219.
print_glue: 203, 204, 211, 212.
print_glyph_name: 1461.
print_group: 1471, 1472, 1489, 1585, 1588.
print_hex: 71, 733, 1277.
print_if_line: 329, 1501, 1587, 1588.
print_int: 69, 88, 95, 98, 107, 136, 193, 194, 195, 196, 197, 211, 214, 220, 221, 222, 244, 245, 253, 255, 257, 259, 260, 261, 265, 268, 275, 277, 281,

315, 318, 329, 343, 366, 434, 500, 506, 507, 544, 571, 595, 596, 614, 653, 676, 677, 680, 702, 705, 709, 716, 717, 720, 733, 744, 766, 894, 904, 987, 1040, 1060, 1063, 1065, 1078, 1082, 1153, 1286, 1350, 1363, 1365, 1372, 1374, 1378, 1382, 1389, 1393, 1415, 1416, 1436, 1458, 1471, 1489, 1491, 1501, 1523, 1633, 1634.
print_lc_hex: 59.
print_length_param: 273, 275, 277.
print_ln: 57, 58, 59, 63, 65, 66, 75, 90, 93, 94, 136, 208, 224, 244, 262, 271, 326, 336, 344, 347, 360, 390, 393, 435, 519, 569, 572, 676, 677, 702, 705, 708, 709, 716, 717, 719, 720, 734, 1040, 1319, 1334, 1363, 1365, 1372, 1374, 1378, 1394, 1433, 1489, 1501, 1523, 1541, 1566, 1585, 1587, 1588.
print_locs: 192.
print_mark: 202, 222, 1416.
print_meaning: 326, 507, 1348.
print_mode: 237, 244, 329, 1103.
print_native_word: 201, 1415, 1416.
print_nl: 66, 77, 86, 88, 89, 94, 193, 194, 195, 196, 197, 244, 245, 271, 281, 315, 318, 329, 336, 341, 343, 344, 353, 390, 434, 565, 569, 595, 616, 676, 677, 679, 680, 702, 708, 709, 716, 719, 720, 744, 894, 904, 905, 911, 987, 1040, 1041, 1046, 1060, 1065, 1175, 1348, 1350, 1351, 1376, 1378, 1382, 1387, 1389, 1392, 1433, 1458, 1489, 1501, 1523, 1585, 1587, 1588.
print_param: 263, 265, 268.
print_plus: 1039.
print_plus_end: 1039.
print_raw_char: 58, 59.
print_roman_int: 73, 507.
print_rule_dimen: 202, 213.
print_sa_num: 1633, 1634, 1643, 1644.
print_scaled: 107, 125, 136, 202, 203, 204, 209, 210, 214, 217, 218, 245, 277, 500, 507, 595, 596, 678, 708, 719, 739, 1039, 1040, 1041, 1060, 1065, 1313, 1315, 1376, 1436, 1445, 1490, 1491, 1634, 1663.
print_size: 741, 766, 1285.
print_skip_param: 215, 251, 253, 255.
print_spec: 204, 214, 215, 216, 255, 500, 1634.
print_style: 732, 736, 1224.
print_subsidiary_data: 734, 738, 739.
print_the_digs: 68, 69, 71.
print_totals: 244, 1039, 1040, 1060.
print_two: 70, 571, 653.
print_utf8_str: 744.
print_visible_char: 58, 59, 64, 347, 744, 1416, 1436.
print_word: 136, 1393.
print_write_whatsit: 1415, 1416.
printed_node: 869, 904, 905, 906, 912.
privileged: 1105, 1108, 1184, 1194.
prompt_file_name: 565, 567, 570, 572, 1382, 1437.
prompt_input: 75, 87, 91, 390, 393, 519, 565.
protected: 1581.
`\protected` primitive: 1581.
protected_token: 319, 423, 513, 1267, 1349, 1583.
prune_movements: 651, 655, 667.
prune_page_top: 1022, 1031, 1075.
pseudo: 54, 57, 58, 59, 63, 346.
pseudo_close: 359, 1568, 1569.
pseudo_files: 1561, 1562, 1565, 1567, 1568, 1569.
pseudo_input: 392, 1567.
pseudo_start: 1560, 1563, 1564.
pstack: 422, 424, 430, 434.
`pt`: 488.
ptmp: 925, 929.
ptr: 169.
punct_noad: 724, 732, 738, 740, 771, 796, 805, 809, 1210, 1211.
push: 620, 621, 622, 626, 628, 637, 644, 652, 655, 667.
push_alignment: 820, 822.
push_input: 351, 353, 355, 358.
push_LR: 1515, 1518, 1521, 1527, 1539, 1548, 1550.
push_math: 1190, 1193, 1199, 1207, 1226, 1228, 1245.
push_nest: 242, 822, 834, 835, 1079, 1137, 1145, 1153, 1171, 1173, 1190, 1221, 1254.
push_node: 877.
put: 26, 29, 1359.
put_LR: 1515, 1520.
put_rule: 621, 622, 671.
put_sa_ptr: 1630, 1642.
put1: 621.
put2: 621.
put3: 621.
put4: 621.
q: 116, 118, 126, 145, 147, 152, 153, 166, 175, 176, 177, 192, 197, 228, 230, 244, 305, 322, 345, 366, 396, 423, 441, 447, 485, 496, 498, 499, 500, 508, 517, 532, 533, 643, 655, 689, 748, 749, 752, 755, 763, 769, 777, 778, 779, 780, 781, 787, 793, 796, 800, 810, 839, 848, 874, 878, 910, 925, 954, 960, 988, 1002, 1007, 1011, 1013, 1014, 1022, 1024, 1048, 1066, 1097, 1122, 1133, 1147, 1159, 1173, 1177, 1192, 1238, 1245, 1252, 1265, 1290, 1356, 1357, 1403, 1433, 1493, 1533, 1549, 1555, 1564, 1568, 1593, 1626, 1630, 1632, 1633, 1636, 1648.
qi: 134, 580, 584, 599, 605, 608, 611, 618, 652, 658, 797, 805, 961, 962, 965, 967, 977, 1012, 1013, 1035, 1062, 1063, 1088, 1089, 1092, 1093,

1094, 1154, 1205, 1209, 1214, 1219, 1363, 1379, 1481, 1565, 1580, 1667, 1669.
qo: 134, 184, 200, 202, 211, 214, 589, 605, 611, 638, 652, 658, 733, 751, 765, 766, 781, 785, 796, 799, 805, 949, 950, 951, 956, 963, 977, 999, 1035, 1040, 1062, 1072, 1075, 1093, 1364, 1378, 1379, 1471, 1669.
qqqq: 132, 135, 136, 169, 585, 589, 604, 608, 609, 725, 756, 785, 796, 963, 1093, 1235, 1359, 1360, 1565, 1567.
quad: 582, 593, 688, 744, 1545.
quad_code: 582, 593.
quarterword: 132, 135, 166, 279, 294, 301, 306, 307, 309, 311, 328, 330, 353, 628, 723, 749, 754, 755, 793, 800, 925, 975, 997, 998, 1001, 1014, 1115, 1133, 1159, 1466, 1489, 1588, 1626, 1646, 1648.
quote_char: 328, 505, 507, 1315.
quoted_filename: 595.
quotient: 1607, 1608.
qw: 595, 599, 605, 608, 611.
r: 112, 126, 145, 147, 153, 230, 244, 396, 423, 447, 500, 517, 533, 655, 689, 710, 749, 763, 769, 781, 796, 839, 848, 877, 910, 925, 954, 1007, 1020, 1022, 1024, 1048, 1066, 1133, 1159, 1177, 1214, 1252, 1290, 1403, 1433, 1544, 1555, 1564, 1567, 1593, 1610.
R_code: 171, 218, 1527, 1542.
r_count: 966, 968, 972.
r_hyf: 939, 940, 943, 952, 955, 957, 977, 1422, 1423.
r_type: 769, 770, 771, 772, 808, 814, 815.
radical: 234, 295, 296, 1100, 1216.
`\radical` primitive: 295.
radical_noad: 725, 732, 738, 740, 776, 809, 1217.
radical_noad_size: 725, 740, 809, 1217.
radicalDegreeBottomRaisePercent: 742.
radicalDisplayStyleVerticalGap: 742, 780.
radicalExtraAscender: 742.
radicalKernAfterDegree: 742.
radicalKernBeforeDegree: 742.
radicalRuleThickness: 742, 780.
radicalVerticalGap: 742, 780.
radix: 396, 472, 473, 474, 478, 479, 482.
radix_backup: 396.
`\raise` primitive: 1125.
Ramshaw, Lyle Harold: 574.
random_seed: 114, 458, 1391, 1413.
`\randomseed` primitive: 450.
random_seed_code: 450, 451, 458.
randoms: 114, 128, 129, 130, 131.
rbrace_ptr: 423, 433, 434.
read: 52, 53, 1392, 1393.
`\read` primitive: 295.
read_file: 515, 520, 521, 1329.
read_font_info: 595, 599, 1094, 1311.
`\readline` primitive: 1570.
read_ln: 52.
read_open: 515, 516, 518, 520, 521, 536, 1329.
read_sixteen: 599, 600, 603.
read_to_cs: 235, 295, 296, 1264, 1279, 1570.
read_toks: 333, 517, 1279.
ready_already: 1385, 1386.
real: 3, 113, 132, 208, 212, 655, 667, 1177, 1179, 1445, 1533.
real addition: 1179.
real division: 700, 706, 715, 718, 858, 859, 1177, 1179.
real multiplication: 136, 212, 663, 672, 857, 1179.
real_point: 1445.
real_rect: 1445.
rebox: 758, 788, 794.
reconstitute: 959, 960, 967, 969, 970, 971, 1086.
recursion: 80, 82, 199, 206, 224, 228, 229, 396, 436, 441, 533, 562, 628, 654, 734, 762, 763, 768, 798, 1003, 1011, 1013, 1387, 1438, 1492.
ref_count: 423, 424, 435.
reference counts: 174, 226, 227, 229, 305, 321, 337, 1631, 1632.
reflected: 652, 1532, 1549.
register: 235, 445, 446, 447, 1264, 1275, 1278, 1289, 1290, 1291, 1634, 1643, 1645.
rel_noad: 724, 732, 738, 740, 771, 805, 809, 815, 1210, 1211.
rel_penalty: 262, 724, 809.
`\relpenalty` primitive: 264.
rel_penalty_code: 262, 263, 264.
relax: 233, 295, 296, 388, 403, 406, 438, 513, 541, 1099, 1278, 1595.
`\relax` primitive: 295.
release_font_engine: 744.
rem_byte: 580, 589, 592, 605, 751, 756, 784, 793, 797, 965, 1094.
remainder: 108, 110, 111, 492, 493, 578, 579, 580, 759, 760.
remove_item: 234, 1158, 1161, 1162.
rep: 581.
replace_count: 167, 201, 221, 877, 888, 906, 917, 930, 931, 972, 1088, 1134, 1174, 1420.
report_illegal_case: 1099, 1104, 1105, 1297, 1440, 1442, 1443, 1444.
requires_units: 482.
reset: 26, 27, 33.
reset_OK: 27.

\resettimer primitive: 1398.
reset_timer_code: 1398, 1401, 1403.
restart: 15, 147, 148, 371, 376, 387, 389, 390, 392, 403, 414, 447, 474, 689, 796, 797, 830, 833, 837, 863, 946, 1205, 1269, 1420, 1593, 1594, 1599.
restore_cur_string: 505, 506.
restore_old_value: 298, 306, 312.
restore_sa: 298, 312, 1648.
restore_trace: 307, 313, 314, 1634.
restore_zero: 298, 306, 308.
restrictedshell: 458.
result: 45, 46, 1445.
resume_after_display: 848, 1253, 1254, 1260.
reswitch: 15, 371, 373, 382, 396, 402, 498, 655, 658, 689, 691, 692, 769, 771, 988, 989, 1083, 1084, 1088, 1090, 1099, 1192, 1201, 1205, 1532, 1533, 1534, 1538, 1576.
return: 15, 16.
return_sign: 126, 127.
reverse: 3, 1531, 1532, 1533.
reversed: 652, 1524, 1531.
rewrite: 26, 27, 33.
rewrite_OK: 27.
rgba: 621.
rh: 132, 135, 136, 140, 170, 239, 245, 247, 260, 282, 283, 298, 727, 975, 1012, 1628.
\right primitive: 1242.
right_brace: 233, 319, 324, 328, 377, 387, 423, 476, 509, 512, 833, 989, 1015, 1121, 1306, 1493.
right_brace_limit: 319, 355, 356, 426, 433, 434, 509, 512, 1493.
right_brace_token: 319, 369, 1119, 1181, 1280, 1434.
right_delimiter: 725, 739, 792, 1235, 1236.
right_hyphen_min: 262, 1145, 1254, 1439, 1440.
\righthyphenmin primitive: 264.
right_hyphen_min_code: 262, 263, 264.
\rightmarginkern primitive: 503.
right_margin_kern_code: 503, 504, 506, 507.
right_noad: 729, 732, 738, 740, 768, 770, 771, 808, 809, 810, 1238, 1242, 1245.
right_ptr: 641, 642, 643, 651.
right_pw: 688, 877, 929.
right_side: 179, 460, 507, 688, 929, 1307.
right_skip: 250, 875, 928, 929, 1551, 1654.
\rightskip primitive: 252.
right_skip_code: 250, 251, 252, 507, 929, 934, 1551, 1557.
right_to_left: 652, 661, 664, 666, 670, 671, 675, 1524, 1525, 1545.
rightskip: 929.
right1: 621, 622, 643, 646, 652.
right2: 621, 646.
right3: 621, 646.
right4: 621, 646.
rlink: 146, 147, 148, 149, 151, 152, 153, 154, 167, 173, 189, 194, 820, 867, 869, 1365, 1366.
\romannumeral primitive: 503.
roman_numeral_code: 503, 504, 506, 507.
round: 3, 136, 212, 656, 663, 672, 749, 857, 1179.
round_decimals: 106, 107, 487.
round_glue: 663, 1537.
round_xn_over_d: 198, 688.
rover: 146, 147, 148, 149, 150, 151, 152, 153, 154, 189, 194, 1365, 1366.
\rpcode primitive: 1308.
rp_code_base: 179, 460, 1307, 1308, 1309.
rsb: 1177, 1179.
rt_hit: 960, 961, 964, 965, 1087, 1089, 1094.
rule_dp: 628, 660, 662, 664, 669, 671, 673.
rule_ht: 628, 660, 662, 664, 669, 671, 672, 673, 674.
rule_node: 160, 161, 172, 201, 209, 228, 232, 660, 664, 669, 673, 691, 693, 711, 712, 773, 809, 853, 889, 890, 914, 918, 919, 1022, 1027, 1054, 1128, 1141, 1175, 1201, 1535, 1544.
rule_node_size: 160, 161, 228, 232, 1544.
rule_save: 848, 852.
rule_thickness: 780.
rule_wd: 628, 660, 662, 663, 664, 665, 669, 671, 673, 1509, 1532, 1535, 1536, 1539, 1540.
rules aligning with characters: 625.
run: 877.
runaway: 142, 336, 368, 430, 521.
Runaway...: 336.
rval: 742, 743.
s: 44, 45, 46, 58, 59, 63, 66, 67, 97, 98, 99, 107, 112, 147, 152, 171, 203, 204, 289, 294, 314, 423, 441, 505, 508, 517, 564, 565, 595, 676, 684, 689, 710, 730, 741, 749, 763, 769, 781, 839, 848, 878, 910, 925, 954, 988, 1020, 1022, 1041, 1066, 1114, 1115, 1177, 1192, 1252, 1290, 1311, 1333, 1404, 1411, 1415, 1489, 1493, 1529, 1555, 1564, 1593, 1632, 1634.
s_max: 749.
sa: 781.
sa_bot_mark: 1636, 1639, 1641.
sa_chain: 298, 312, 1646, 1647, 1648, 1652.
sa_def: 1650, 1651.
sa_def_box: 1131, 1650.
sa_define: 1280, 1281, 1290, 1650.
sa_destroy: 1649, 1650, 1651, 1652.
sa_dim: 1631, 1634.
sa_first_mark: 1636, 1639, 1640, 1641.
sa_index: 1626, 1631, 1632, 1633, 1648, 1649, 1652.

sa_int: 461, 1291, 1631, 1632, 1634, 1648, 1650, 1651, 1652.
sa_lev: 1631, 1648, 1650, 1651, 1652.
sa_level: 298, 312, 1646, 1647, 1648.
sa_loc: 1648, 1652.
sa_mark: 1031, 1066, 1389, 1627, 1628.
sa_null: 1626, 1627, 1628, 1631.
sa_num: 1631, 1633.
sa_ptr: 449, 461, 1088, 1281, 1291, 1631, 1632, 1634, 1648, 1649, 1650, 1651, 1652.
sa_ref: 1631, 1632, 1648.
sa_restore: 312, 1652.
sa_root: 1365, 1366, 1627, 1629, 1630, 1632.
sa_save: 1648, 1650.
sa_split_bot_mark: 1636, 1637, 1638.
sa_split_first_mark: 1636, 1637, 1638.
sa_top_mark: 1636, 1639, 1640.
sa_type: 461, 1291, 1631, 1634, 1643.
sa_used: 1626, 1630, 1631, 1632, 1636.
sa_w_def: 1650, 1651.
sa_word_define: 1290, 1650.
save_cond_ptr: 533, 535, 544.
save_cs_ptr: 822, 825.
save_cur_cs: 441, 1410.
save_cur_string: 505, 506.
save_cur_val: 485, 490.
save_def_ref: 505, 506.
save_f: 793, 795, 800, 801, 802, 803, 805.
save_for_after: 310, 1325.
save_h: 655, 661, 665, 666, 667, 670, 675, 1426, 1430, 1531, 1532.
save_index: 298, 304, 306, 310, 312, 1489, 1585, 1588, 1648.
save_level: 298, 299, 304, 306, 310, 312, 1489, 1588, 1648.
save_link: 878, 905.
save_loc: 655, 667.
save_native_len: 61, 1088.
save_pointer: 1488, 1489, 1584.
save_ptr: 298, 301, 302, 303, 304, 306, 310, 312, 313, 315, 684, 852, 1140, 1153, 1154, 1171, 1174, 1196, 1207, 1222, 1226, 1228, 1240, 1248, 1358, 1489, 1585, 1588, 1648.
save_scanner_status: 396, 401, 402, 423, 505, 506, 529, 533, 536, 542, 1577.
save_size: 11, 133, 301, 303, 1388, 1488.
save_split_top_skip: 1066, 1068.
save_stack: 229, 298, 300, 301, 303, 304, 305, 306, 307, 311, 312, 313, 315, 330, 406, 524, 684, 816, 1116, 1125, 1185, 1194, 1204, 1207, 1393, 1488.
save_style: 763, 769, 798.
save_type: 298, 304, 306, 310, 312, 1648.
save_v: 655, 661, 666, 667, 670, 674, 675, 1426, 1430.
save_vbadness: 1066, 1071.
save_vfuzz: 1066, 1071.
save_warning_index: 423, 505, 506.
saved: 304, 684, 852, 1137, 1140, 1153, 1154, 1171, 1173, 1196, 1207, 1222, 1226, 1228, 1240, 1248, 1471, 1472, 1489, 1490, 1491.
saved_chr: 505, 506.
saved_math_style: 800, 805.
saving_hyph_codes: 262, 1014.
`\savinghyphcodes` primitive: 1467.
saving_hyph_codes_code: 262, 1467, 1469.
saving_vdiscards: 262, 1031, 1053, 1670.
`\savingvdiscards` primitive: 1467.
saving_vdiscards_code: 262, 1467, 1469.
sc: 132, 135, 136, 157, 174, 184, 189, 239, 245, 273, 276, 277, 447, 454, 459, 585, 587, 589, 592, 593, 606, 608, 610, 615, 742, 743, 744, 823, 870, 871, 880, 891, 892, 896, 898, 908, 909, 937, 1096, 1203, 1260, 1301, 1302, 1307, 1484, 1631, 1653.
scaled: 105, 106, 107, 108, 109, 110, 111, 112, 114, 130, 132, 135, 171, 174, 180, 198, 202, 203, 481, 482, 485, 488, 583, 584, 595, 620, 628, 643, 652, 655, 667, 685, 688, 689, 710, 721, 742, 743, 744, 747, 748, 749, 755, 758, 759, 760, 762, 769, 778, 779, 780, 781, 787, 793, 800, 810, 839, 848, 871, 877, 878, 887, 895, 925, 960, 1024, 1025, 1031, 1034, 1036, 1048, 1066, 1122, 1140, 1177, 1192, 1252, 1311, 1429, 1529, 1533, 1555, 1653, 1655.
`scaled`: 1312.
scaled_base: 273, 275, 277, 1278, 1291.
scan_and_pack_name: 467, 1446, 1447, 1454, 1455, 1456.
scan_box: 1127, 1138, 1295.
scan_char_class: 467, 1286.
scan_char_class_not_ignored: 449, 467, 1280, 1281.
scan_char_num: 460, 467, 468, 989, 1092, 1177, 1178, 1205, 1208, 1307.
scan_decimal: 483, 1445.
scan_delimiter: 1214, 1217, 1236, 1237, 1245, 1246.
scan_delimiter_int: 471, 1205, 1208, 1214.
scan_dimen: 444, 474, 481, 482, 483, 496, 497, 1115.
scan_eight_bit_int: 467, 1153.
scan_expr: 1591, 1592, 1593.
scan_fifteen_bit_int: 470, 1205, 1208, 1219, 1278.
scan_file_name: 295, 364, 561, 562, 572, 1311, 1329, 1406, 1445, 1448, 1456.
scan_font_ident: 449, 460, 506, 612, 613, 1288, 1307, 1454, 1460, 1481, 1580.
scan_four_bit_int: 469, 536, 1329, 1405.

scan_general_text: 1492, 1493, 1498, 1564.
scan_glue: 444, 496, 830, 1114, 1282, 1292, 1598.
scan_glyph_number: 460, 467, 1307.
scan_int: 443, 444, 466, 467, 468, 469, 470, 471, 472, 474, 481, 482, 496, 506, 538, 539, 544, 613, 1157, 1279, 1282, 1286, 1292, 1294, 1297, 1298, 1300, 1302, 1307, 1312, 1405, 1413, 1440, 1444, 1445, 1454, 1458, 1460, 1484, 1596, 1622, 1677.
scan_keyword: 187, 441, 467, 488, 489, 490, 491, 493, 497, 498, 506, 684, 1136, 1153, 1219, 1279, 1290, 1312, 1445.
scan_left_brace: 437, 508, 684, 833, 988, 1014, 1079, 1153, 1171, 1173, 1207, 1226, 1228, 1493.
scan_math: 1204, 1205, 1212, 1217, 1219, 1230.
scan_math_class_int: 469, 1205, 1208, 1214, 1219, 1278, 1286.
scan_math_fam_int: 469, 612, 1205, 1208, 1214, 1219, 1278, 1286, 1288.
scan_mu_glue: 1596, 1597, 1598, 1618.
scan_normal_dimen: 482, 498, 538, 684, 1127, 1136, 1236, 1237, 1282, 1292, 1297, 1299, 1301, 1302, 1307, 1313, 1445, 1596.
scan_normal_glue: 1596, 1597, 1598, 1614, 1615, 1619.
scan_optional_equals: 439, 830, 1278, 1280, 1282, 1286, 1288, 1290, 1295, 1297, 1298, 1299, 1300, 1301, 1302, 1307, 1311, 1329, 1406.
scan_pdf_ext_toks: 506, 1410.
scan_register_num: 420, 449, 454, 461, 506, 540, 1133, 1136, 1155, 1164, 1278, 1280, 1281, 1291, 1295, 1301, 1350, 1621, 1622.
scan_rule_spec: 498, 1110, 1138.
scan_something_internal: 443, 444, 447, 466, 474, 484, 486, 490, 496, 500, 1591.
scan_spec: 684, 816, 822, 1125, 1137, 1221.
scan_tokens: 1558.
`\scantokens` primitive: 1558.
scan_toks: 321, 499, 508, 1014, 1155, 1272, 1280, 1333, 1342, 1407, 1409, 1410, 1434, 1492.
scan_usv_num: 447, 448, 468, 506, 1084, 1088, 1205, 1208, 1214, 1219, 1278, 1286, 1481, 1580.
scan_xetex_del_code_int: 1286.
scan_xetex_math_char_int: 469, 1205, 1208, 1278, 1286.
scanned_result: 447, 448, 449, 452, 456, 459, 460, 462, 467.
scanned_result_end: 447.
scanner_status: 335, 336, 361, 366, 369, 396, 401, 402, 423, 425, 505, 506, 508, 517, 529, 533, 536, 542, 825, 837, 1099, 1493, 1577.
script_c: 800, 805.
script_f: 800, 801, 802, 803, 805.
`\scriptfont` primitive: 1284.
script_g: 800, 801, 802, 803, 805.
script_head: 800, 801, 802, 803, 805.
script_mlist: 731, 737, 740, 774, 805, 1228.
script_ptr: 800, 805.
`\scriptscriptfont` primitive: 1284.
script_script_mlist: 731, 737, 740, 774, 805, 1228.
script_script_size: 12, 260, 800, 1249, 1284.
script_script_style: 730, 736, 774, 1223.
`\scriptscriptstyle` primitive: 1223.
script_size: 12, 260, 741, 746, 750, 800, 1249, 1284.
script_space: 273, 801, 802, 803.
`\scriptspace` primitive: 274.
script_space_code: 273, 274.
script_style: 730, 736, 745, 746, 774, 800, 814, 1223.
`\scriptstyle` primitive: 1223.
scriptPercentScaleDown: 742.
scripts_allowed: 729, 1230.
scriptScriptPercentScaleDown: 742.
scroll_mode: 75, 77, 88, 90, 97, 565, 1316, 1317, 1335.
`\scrollmode` primitive: 1316.
search_mem: 190, 197, 281, 1393.
second_indent: 895, 896, 897, 937.
second_pass: 876, 911, 914.
second_width: 895, 896, 897, 898, 937.
seconds_and_micros: 1411, 1412, 1414.
Sedgewick, Robert: 2.
`see the transcript file...`: 1389.
seed: 129.
selector: 54, 55, 57, 58, 59, 63, 66, 75, 79, 90, 94, 96, 102, 271, 341, 342, 346, 390, 500, 505, 506, 569, 570, 653, 676, 678, 1311, 1319, 1333, 1352, 1382, 1387, 1389, 1410, 1431, 1433, 1436, 1498, 1564.
semi_simple_group: 299, 1117, 1119, 1122, 1123, 1471, 1489.
serial: 869, 893, 894, 904.
set_aux: 235, 447, 450, 451, 452, 1264, 1296.
set_box: 235, 295, 296, 1264, 1295.
`\setbox` primitive: 295.
set_box_allowed: 80, 81, 1295, 1324.
set_box_dimen: 235, 447, 450, 451, 1264, 1296.
set_box_lr: 652, 855, 856, 1248, 1256, 1524, 1531.
set_box_lr_end: 652.
set_break_width_to_background: 885.
set_char_0: 621, 622, 658.
set_class_field: 258, 1205, 1208, 1219, 1278, 1286.
set_conversion: 493.
set_conversion_end: 493.
set_cp_code: 1307.

set_cur_lang: 988, 1014, 1145, 1254.
set_cur_r: 962, 964, 965.
set_family_field: 258, 1205, 1208, 1219, 1278, 1286.
set_font: 235, 447, 588, 612, 1264, 1271, 1311, 1315.
set_glue_ratio_one: 113, 706, 718, 858, 859.
set_glue_ratio_zero: 113, 158, 699, 700, 706, 714, 715, 718, 858, 859.
set_glyphs: 621, 622, 1426, 1430.
set_height_zero: 1024.
set_hyph_index: 939, 988, 1422, 1423, 1669.
set_input_file_encoding: 1446.
set_interaction: 235, 1264, 1316, 1317, 1318.
set_justified_native_glyphs: 656.
\setlanguage primitive: 1398.
set_language_code: 1398, 1401, 1403.
set_lc_code: 946, 949, 950, 951, 991, 1669.
set_math_char: 1208, 1209.
set_native_char: 656, 744, 947, 957, 1088, 1420.
set_native_glyph_metrics: 749, 781, 783, 793, 799, 1444.
set_native_metrics: 169, 744, 947, 957, 1088, 1420.
set_page_dimen: 235, 447, 1036, 1037, 1038, 1264, 1296.
set_page_int: 235, 447, 450, 451, 1264, 1296, 1502.
set_page_so_far_zero: 1041.
set_prev_graf: 235, 295, 296, 447, 1264, 1296.
set_random_seed_code: 1398, 1401, 1403.
\setrandomseed primitive: 1398.
set_rule: 619, 621, 622, 662.
set_sa_box: 1632.
set_shape: 235, 259, 295, 296, 447, 1264, 1302, 1675.
set_text_and_glyphs: 621, 622, 1430.
set_trick_count: 346, 347, 348, 350.
setLen: 744.
setPoint: 1445.
settingNameP: 744.
set1: 621, 622, 658.
set2: 621.
set3: 621.
set4: 621.
sf_code: 256, 258, 447, 1088, 1286.
\sfcode primitive: 1284.
sf_code_base: 256, 261, 447, 448, 1284, 1285, 1286, 1287.
shape_ref: 236, 258, 305, 1124, 1302.
shellenabledp: 458.
shift_amount: 157, 158, 184, 210, 661, 666, 670, 675, 689, 693, 710, 712, 723, 749, 763, 780, 781, 793, 794, 800, 801, 803, 807, 847, 854, 855, 856, 937, 1130, 1135, 1179, 1545, 1551, 1555, 1556, 1557.
shift_case: 1339, 1342.
shift_down: 787, 788, 789, 790, 791, 793, 795, 800, 801, 803, 806.
shift_up: 787, 788, 789, 790, 791, 793, 795, 800, 802, 803, 807.
ship_out: 628, 676, 683, 855, 856, 1077, 1129, 1451, 1510, 1515.
\shipout primitive: 1125.
ship_out_flag: 1125, 1129, 1491.
short_display: 199, 200, 201, 219, 705, 905, 1393.
short_display_n: 224.
short_real: 113, 132.
shortcut: 481, 482.
shortfall: 878, 899, 900, 901, 1653, 1658, 1660, 1661.
shorthand_def: 235, 1264, 1276, 1277, 1278.
\show primitive: 1345.
show_activities: 244, 1347.
show_box: 206, 208, 224, 244, 245, 262, 676, 679, 705, 717, 1040, 1046, 1175, 1350, 1393.
\showbox primitive: 1345.
show_box_breadth: 262, 1393.
\showboxbreadth primitive: 264.
show_box_breadth_code: 262, 263, 264.
show_box_code: 1345, 1346, 1347.
show_box_depth: 262, 1393.
\showboxdepth primitive: 264.
show_box_depth_code: 262, 263, 264.
show_code: 1345, 1347.
show_context: 54, 82, 86, 92, 340, 341, 348, 565, 570, 572, 1585, 1587, 1588.
show_cur_cmd_chr: 329, 399, 529, 533, 545, 1085, 1265.
show_eqtb: 278, 314, 1634.
show_groups: 1485, 1486, 1487.
\showgroups primitive: 1485.
show_ifs: 1499, 1500, 1501.
\showifs primitive: 1499.
show_info: 734, 735.
show_lists_code: 1345, 1346, 1347.
\showlists primitive: 1345.
show_node_list: 199, 202, 206, 207, 208, 221, 224, 259, 732, 734, 735, 737, 1393, 1634.
show_sa: 1634, 1650, 1651, 1652.
show_save_groups: 1389, 1487, 1489.
\showthe primitive: 1345.
show_the_code: 1345, 1346.
show_token_list: 202, 249, 259, 322, 325, 336, 349, 350, 434, 506, 1393, 1410, 1431, 1634.
show_tokens: 1494, 1495, 1496.

\showtokens primitive: 1494.
show_whatever: 1344, 1347.
shown_mode: 239, 241, 329.
shrink: 174, 175, 189, 204, 465, 497, 656, 663, 672, 698, 713, 759, 857, 873, 875, 886, 916, 1030, 1058, 1063, 1096, 1098, 1202, 1283, 1293, 1294, 1509, 1557, 1601, 1602, 1605, 1606, 1607, 1609, 1615.
shrink_order: 174, 189, 204, 497, 656, 663, 672, 698, 713, 759, 857, 873, 874, 1030, 1058, 1063, 1202, 1293, 1509, 1557, 1602, 1605, 1614.
shrinking: 157, 212, 655, 667, 706, 718, 857, 858, 859, 1202, 1509, 1533.
si: 38, 42, 73, 1005, 1018, 1364, 1565, 1667.
side: 688.
simple_group: 299, 1117, 1122, 1471, 1489.
Single-character primitives: 297.
\-: 1168.
\/: 295.
\␣: 295.
single_base: 248, 292, 293, 294, 384, 402, 403, 408, 476, 536, 1099, 1311, 1343, 1579.
size_code: 742.
sizeof: 60, 62, 744, 1417, 1445.
skew_char: 179, 460, 584, 587, 611, 744, 785, 1307, 1376, 1377.
\skewchar primitive: 1308.
skewedFractionHorizontalGap: 742.
skewedFractionVerticalGap: 742.
skip: 250, 461, 1063.
\skip primitive: 445.
skip_base: 250, 253, 255, 1278, 1291.
skip_blanks: 333, 374, 375, 377, 379, 384.
skip_byte: 580, 592, 785, 796, 797, 963, 1093.
skip_code: 1112, 1113, 1114.
\skipdef primitive: 1276.
skip_def_code: 1276, 1277, 1278.
skip_line: 366, 528, 529.
skipping: 335, 336, 366, 529.
SLANT: 621.
slant: 582, 593, 610, 621, 744, 1177, 1179.
slant_code: 582, 593.
slow_make_string: 744.
slow_print: 64, 65, 67, 88, 553, 571, 572, 616, 680, 1315, 1334, 1337, 1382, 1387, 1393.
small_char: 725, 733, 739, 749.
small_char_field: 725, 1214.
small_fam: 725, 733, 739, 749, 780.
small_node_size: 163, 166, 167, 171, 176, 177, 180, 183, 228, 232, 697, 764, 956, 964, 968, 1091, 1154, 1155, 1417, 1418, 1439, 1440, 1450, 1532, 1535, 1538, 1540, 1544, 1549.
small_number: 59, 105, 106, 171, 176, 178, 294, 371, 396, 423, 447, 472, 474, 485, 496, 499, 500, 505, 517, 524, 529, 532, 533, 558, 643, 688, 689, 710, 730, 762, 763, 769, 800, 810, 877, 940, 942, 959, 960, 975, 988, 998, 1014, 1024, 1041, 1114, 1129, 1140, 1145, 1230, 1235, 1245, 1252, 1265, 1290, 1300, 1301, 1311, 1347, 1379, 1389, 1404, 1405, 1433, 1436, 1445, 1515, 1529, 1593, 1626, 1630, 1632, 1634, 1636, 1653.
small_plane_and_fam_field: 725, 1214.
so: 38, 45, 63, 73, 74, 294, 441, 499, 554, 639, 653, 678, 814, 985, 1007, 1009, 1010, 1013, 1017, 1363, 1431, 1436, 1565, 1666.
Sorry, I can't find...: 559.
sort_avail: 153, 1365.
sp: 108, 623.
sp: 493.
space: 582, 593, 744, 796, 799, 1096.
space_adjustment: 179, 217, 656, 885, 927, 1088.
space_class: 1086, 1088.
space_code: 582, 593, 613, 1096.
space_factor: 238, 239, 452, 834, 835, 847, 1084, 1088, 1097, 1098, 1110, 1130, 1137, 1145, 1147, 1171, 1173, 1177, 1250, 1254, 1296, 1297.
\spacefactor primitive: 450.
space_shrink: 582, 593, 744, 1096.
space_shrink_code: 582, 593, 613.
space_skip: 250, 1095, 1097.
\spaceskip primitive: 252.
space_skip_code: 250, 251, 252, 1095.
space_stretch: 582, 593, 744, 1096.
space_stretch_code: 582, 593.
space_token: 319, 427, 499, 1269, 1572.
spaceAfterScript: 742.
spacer: 233, 234, 258, 319, 321, 324, 328, 333, 367, 375, 377, 378, 379, 384, 438, 440, 441, 477, 478, 487, 499, 505, 831, 833, 839, 989, 1015, 1084, 1099, 1275.
\span primitive: 828.
span_code: 828, 829, 830, 837, 839.
span_count: 184, 211, 844, 849, 856.
span_node_size: 845, 846, 851.
spec_code: 684.
spec_log: 121, 122, 124.
\special primitive: 1398.
special_char: 12, 388, 828.
special_node: 1395, 1398, 1401, 1403, 1409, 1416, 1417, 1418, 1436.
special_out: 1431, 1436.
split: 1065.
split_bot_mark: 416, 417, 1031, 1033, 1620, 1637, 1638.

\splitbotmark primitive: 418.
split_bot_mark_code: 416, 418, 419, 1389, 1620, 1642.
\splitbotmarks primitive: 1620.
split_disc: 1022, 1031, 1670, 1671.
\splitdiscards primitive: 1672.
split_first_mark: 416, 417, 1031, 1033, 1620, 1638.
\splitfirstmark primitive: 418.
split_first_mark_code: 416, 418, 419, 1620.
\splitfirstmarks primitive: 1620.
split_fist_mark: 1637.
split_max_depth: 162, 273, 1031, 1122, 1154.
\splitmaxdepth primitive: 274.
split_max_depth_code: 273, 274.
split_top_ptr: 162, 214, 228, 232, 1075, 1076, 1154.
split_top_skip: 162, 250, 1022, 1031, 1066, 1068, 1075, 1154.
\splittopskip primitive: 252.
split_top_skip_code: 250, 251, 252, 1023.
split_up: 1035, 1040, 1062, 1064, 1074, 1075.
spotless: 80, 81, 271, 1386, 1389, 1585, 1587, 1588.
spread: 684.
sprint_cs: 249, 293, 368, 429, 430, 432, 507, 514, 519, 596, 1348.
square roots: 780.
src: 169.
ss_code: 1112, 1113, 1114.
ss_glue: 187, 189, 758, 1114.
stack conventions: 330.
stack_glue_into_box: 749.
stack_glyph_into_box: 749.
stack_into_box: 754, 756.
stack_size: 11, 331, 340, 351, 1388.
stackBottomDisplayStyleShiftDown: 742.
stackBottomShiftDown: 742.
stackDisplayStyleGapMin: 742, 789.
stackGapMin: 742, 789.
stackTopDisplayStyleShiftUp: 742.
stackTopShiftUp: 742.
start: 330, 332, 333, 337, 348, 349, 353, 354, 355, 356, 358, 359, 361, 390, 392, 393, 401, 402, 518, 573, 1566.
start_cs: 371, 384, 385.
start_eq_no: 1194, 1196.
start_field: 330, 332.
start_font_error_message: 596, 602.
start_here: 5, 1386.
start_input: 396, 410, 412, 572, 1391.
start_of_TEX: 6, 1386.
start_par: 234, 1142, 1143, 1144, 1146.
stat: 7, 139, 142, 143, 144, 145, 147, 152, 278, 287, 304, 307, 312, 313, 314, 677, 874, 877, 893, 903, 911, 1041, 1059, 1064, 1387, 1472, 1634, 1650, 1651, 1652.
state: 91, 330, 332, 333, 337, 341, 342, 353, 355, 358, 360, 361, 367, 371, 373, 374, 376, 377, 379, 382, 383, 384, 424, 518, 572, 1088, 1389.
state_field: 330, 332, 1185, 1586.
stomach: 436.
stop: 233, 1099, 1100, 1106, 1107, 1108, 1148.
stop_flag: 580, 592, 785, 796, 797, 963, 1093.
store_background: 912.
store_break_width: 891.
store_fmt_file: 1356, 1389.
store_four_quarters: 599, 603, 604, 608, 609.
store_new_token: 405, 406, 427, 431, 433, 441, 499, 501, 508, 509, 511, 512, 517, 518, 1493, 1572, 1578.
store_scaled: 606, 608, 610.
str: 749.
str_eq_buf: 45, 286.
str_eq_str: 46, 289, 744, 1314.
str_number: 2, 38, 39, 40, 43, 44, 45, 46, 47, 66, 67, 83, 97, 98, 99, 198, 203, 204, 289, 294, 314, 328, 441, 505, 547, 554, 560, 562, 564, 565, 567, 584, 595, 744, 980, 983, 988, 1311, 1333, 1353, 1410, 1415, 1489, 1564, 1625, 1634.
str_pool: 38, 39, 42, 43, 44, 45, 46, 47, 63, 73, 74, 282, 287, 291, 294, 333, 441, 499, 507, 554, 638, 639, 653, 656, 676, 678, 744, 812, 814, 983, 985, 988, 995, 1315, 1363, 1364, 1388, 1410, 1431, 1436, 1564, 1565.
str_ptr: 38, 39, 41, 43, 44, 47, 48, 63, 74, 287, 289, 292, 505, 552, 560, 572, 653, 656, 678, 744, 1314, 1363, 1364, 1377, 1379, 1381, 1386, 1388, 1410, 1431, 1436.
str_room: 42, 44, 206, 287, 499, 551, 560, 656, 744, 993, 1311, 1333, 1382, 1387, 1431, 1564.
str_start: 38, 39, 47, 282, 983, 988, 1363, 1364.
str_start_macro: 38, 40, 41, 43, 44, 45, 46, 48, 63, 73, 74, 287, 289, 294, 441, 505, 507, 552, 554, 639, 653, 656, 678, 744, 813, 985, 995, 1315, 1410, 1431, 1436, 1565.
str_toks: 499, 500, 505, 506, 1498.
str_toks_cat: 499, 505.
stretch: 174, 175, 189, 204, 465, 497, 656, 663, 672, 698, 713, 749, 759, 857, 875, 886, 916, 1030, 1058, 1063, 1096, 1098, 1202, 1283, 1293, 1294, 1509, 1557, 1601, 1602, 1605, 1606, 1607, 1609, 1615, 1654, 1664.
stretch_order: 174, 189, 204, 497, 656, 663, 672, 698, 713, 759, 857, 875, 886, 916, 1030, 1058, 1063, 1202, 1293, 1509, 1557, 1602, 1605, 1614, 1654.

stretching: 157, 656, 663, 672, 700, 715, 749, 857, 858, 859, 1202, 1509.
stretchStackBottomShiftDown: 742.
stretchStackGapAboveMin: 742.
stretchStackGapBelowMin: 742.
stretchStackTopShiftUp: 742.
string pool: 47, 1362.
`\string` primitive: 503.
string_code: 503, 504, 506, 507.
string_vacancies: 11, 52.
stringcast: 595.
strlen: 1445.
style: 769, 770, 808, 809, 810.
style_node: 185, 730, 732, 740, 773, 774, 805, 809, 1223, 1529.
style_node_size: 730, 731, 740, 811, 1529.
sub_box: 723, 729, 734, 740, 763, 777, 778, 780, 781, 793, 798, 805, 1130, 1147, 1222.
sub_cmd: 804, 806.
sub_drop: 742, 800.
sub_f: 800, 801, 803, 806.
sub_g: 800, 801, 803, 806.
sub_kern: 800, 803, 806.
sub_mark: 233, 324, 328, 377, 1100, 1229.
sub_mlist: 723, 725, 734, 763, 781, 786, 798, 805, 1235, 1239, 1240, 1245.
sub_style: 745, 794, 801, 803, 805.
sub_sup: 1229, 1230.
subscr: 723, 725, 728, 729, 732, 738, 740, 781, 786, 793, 794, 795, 796, 797, 798, 799, 800, 801, 803, 807, 1205, 1217, 1219, 1229, 1230, 1231, 1240.
subscriptBaselineDropMin: 742.
subscripts: 798, 1229.
subscriptShiftDown: 742.
subscriptTopMax: 742, 801.
subSuperscriptGapMin: 742, 803.
subtype: 155, 156, 157, 158, 161, 162, 164, 165, 166, 167, 168, 169, 170, 171, 173, 174, 176, 177, 178, 179, 180, 183, 184, 201, 209, 214, 215, 216, 217, 218, 219, 458, 507, 524, 530, 531, 652, 656, 663, 665, 667, 672, 674, 688, 689, 698, 710, 713, 723, 724, 728, 729, 730, 731, 732, 738, 744, 749, 760, 773, 774, 775, 776, 781, 783, 793, 799, 805, 811, 814, 816, 834, 841, 843, 857, 867, 868, 870, 877, 885, 889, 890, 891, 892, 914, 916, 918, 919, 927, 929, 945, 947, 949, 950, 951, 952, 956, 957, 964, 1035, 1040, 1042, 1062, 1063, 1072, 1074, 1075, 1088, 1089, 1114, 1115, 1132, 1134, 1154, 1155, 1167, 1175, 1179, 1202, 1213, 1217, 1219, 1225, 1235, 1245, 1389, 1395, 1404, 1416, 1417, 1418, 1419, 1420, 1421, 1422, 1423, 1424, 1425, 1426, 1430, 1436, 1437, 1445, 1501, 1509, 1521, 1527, 1529, 1530, 1536, 1539, 1547, 1548, 1550, 1588, 1599, 1600, 1626.
sub1: 742, 801.
sub2: 742, 803.
succumb: 97, 98, 99, 198, 1358.
sup_cmd: 804, 807.
sup_count: 371, 382, 385.
sup_drop: 742, 800.
sup_f: 800, 802, 807.
sup_g: 800, 802, 807.
sup_kern: 800, 803, 807.
sup_mark: 233, 324, 328, 371, 374, 385, 1100, 1229, 1230, 1231.
sup_style: 745, 794, 802.
superscriptBaselineDropMax: 742.
superscriptBottomMaxWithSubscript: 742, 803.
superscriptBottomMin: 742, 802.
superscripts: 798, 1229.
superscriptShiftUp: 742.
superscriptShiftUpCramped: 742.
suppress_fontnotfound_error: 262, 595.
suppress_fontnotfound_error_code: 262, 1510, 1511.
supscr: 723, 725, 728, 729, 732, 738, 740, 781, 786, 794, 795, 796, 797, 798, 800, 802, 1205, 1217, 1219, 1229, 1230, 1231, 1240.
sup1: 742, 802.
sup2: 742, 802.
sup3: 742, 802.
sw: 595, 606, 610.
switch: 371, 373, 374, 376, 380.
synch_h: 652, 658, 662, 666, 671, 675, 1426, 1430, 1431, 1436.
synch_v: 652, 658, 662, 666, 670, 671, 675, 1426, 1430, 1431, 1436.
sys_day: 267, 272, 571.
sys_month: 267, 272, 571.
sys_time: 267, 272, 571.
sys_year: 267, 272, 571.
system dependencies: 2, 3, 4, 9, 10, 11, 12, 19, 21, 23, 26, 27, 28, 32, 33, 34, 35, 37, 38, 49, 56, 63, 65, 76, 85, 88, 100, 113, 116, 132, 134, 135, 186, 212, 267, 334, 343, 358, 520, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 558, 560, 572, 573, 592, 599, 627, 631, 633, 846, 1360, 1385, 1386, 1387, 1392, 1394, 1642, 1678.
sz: 1564, 1565, 1567.
s1: 86, 92, 1410.
s2: 86, 92, 1410.
s3: 86, 92.
s4: 86, 92.

t: 46, 111, 112, 147, 198, 244, 307, 309, 310, 311, 353, 371, 396, 423, 499, 508, 747, 748, 769, 800, 848, 877, 878, 925, 960, 988, 1020, 1024, 1084, 1177, 1230, 1245, 1252, 1311, 1342, 1347, 1533, 1544, 1549, 1555, 1593, 1610, 1630, 1634.
t_open_in: 33, 37.
t_open_out: 33, 1386.
tab_mark: 233, 319, 324, 372, 377, 828, 829, 830, 831, 832, 836, 1180.
tab_skip: 250.
`\tabskip` primitive: 252.
tab_skip_code: 250, 251, 252, 826, 830, 834, 841, 843, 857.
tab_token: 319, 1182.
tag: 578, 579, 589.
tail: 164, 238, 239, 240, 241, 242, 458, 721, 744, 761, 824, 834, 843, 844, 847, 860, 864, 938, 1049, 1071, 1077, 1080, 1088, 1089, 1090, 1091, 1094, 1095, 1097, 1108, 1114, 1115, 1130, 1132, 1134, 1145, 1150, 1154, 1155, 1159, 1164, 1167, 1171, 1173, 1174, 1177, 1179, 1199, 1204, 1209, 1212, 1213, 1217, 1219, 1222, 1225, 1228, 1230, 1231, 1235, 1238, 1240, 1241, 1245, 1250, 1259, 1260, 1404, 1405, 1406, 1407, 1408, 1409, 1438, 1439, 1440, 1444, 1445, 1674.
tail_append: 240, 744, 834, 843, 864, 1088, 1089, 1091, 1094, 1108, 1110, 1114, 1115, 1145, 1147, 1154, 1157, 1166, 1167, 1171, 1204, 1212, 1217, 1219, 1222, 1225, 1226, 1231, 1245, 1250, 1257, 1259, 1260, 1513.
tail_field: 238, 239, 1049.
tail_page_disc: 1053, 1670.
take_frac: 118, 130, 131.
take_fraction: 1610.
tally: 54, 55, 57, 58, 322, 342, 345, 346, 347.
tats: 7.
temp_head: 187, 336, 425, 430, 434, 499, 501, 502, 505, 506, 513, 762, 763, 798, 808, 864, 910, 911, 912, 925, 927, 928, 929, 935, 1022, 1118, 1119, 1248, 1250, 1253, 1351, 1493, 1498, 1517, 1519, 1545, 1546, 1548, 1549, 1564.
temp_ptr: 137, 178, 654, 655, 661, 666, 667, 670, 675, 678, 721, 734, 735, 1023, 1055, 1075, 1088, 1091, 1095, 1389, 1515, 1517, 1519, 1522, 1531, 1532, 1533, 1538, 1557.
term_and_log: 54, 57, 58, 75, 79, 96, 271, 569, 1352, 1382, 1389, 1433.
term_in: 32, 33, 34, 36, 37, 75, 1392, 1393.
term_input: 75, 82.
term_offset: 54, 55, 57, 58, 65, 66, 75, 572, 676, 1334, 1566.
term_only: 54, 55, 57, 58, 75, 79, 96, 570, 1352, 1387, 1389.
term_out: 32, 33, 34, 35, 36, 37, 51, 56.
terminal_input: 334, 343, 358, 360, 390, 744.
terminate_font_manager: 1441.
test_char: 960, 963.
TEX: 2, 4.
`TeX capacity exceeded ...`: 98.
  buffer size: 35, 294, 358, 408, 1579.
  exception dictionary: 994.
  font memory: 615.
  grouping levels: 304.
  hash size: 287.
  input stack size: 351.
  main memory size: 142, 147.
  number of strings: 43, 552.
  parameter stack size: 424.
  pattern memory: 1008, 1018.
  pool size: 42.
  primitive size: 290.
  save size: 303.
  semantic nest size: 242.
  text input levels: 358.
`TEX.POOL check sum...`: 53.
`TEX.POOL doesn't match`: 53.
`TEX.POOL has no check sum`: 52.
`TEX.POOL line doesn't...`: 52.
TEX_area: 549, 572.
TEX_font_area: 549, 598.
TEX_format_default: 555, 556, 558.
tex_int_pars: 262.
tex_toks: 256.
The TEXbook: 1, 23, 49, 112, 233, 449, 480, 491, 494, 725, 730, 812, 1269, 1385.
`TeXfonts`: 549.
`TeXformats`: 11, 556.
`TeXinputs`: 549.
`texput`: 35, 569, 1311.
text: 282, 284, 285, 286, 287, 292, 293, 294, 295, 402, 403, 526, 536, 588, 828, 1099, 1242, 1270, 1311, 1372, 1432.
`Text line contains...`: 376.
text_char: 19, 20, 25, 47.
`\textfont` primitive: 1284.
text_mlist: 731, 737, 740, 774, 805, 1228.
text_size: 12, 741, 746, 775, 1249, 1253.
text_style: 730, 736, 746, 774, 780, 788, 789, 790, 792, 793, 802, 1223, 1248, 1250.
`\textstyle` primitive: 1223.
TeXXeT: 1510.
TeXXeT_code: 2, 1510, 1511.
TeXXeT_en: 689, 691, 927, 928, 929, 1510, 1513, 1545, 1546, 1547.

TeXXeT_state: 1510.
\TeXXeT_state primitive: 1511.
TEX82: 1, 103.
tfm: 1307.
TFM files: 574.
tfm_file: 574, 595, 598, 599, 610.
TFtoPL: 596.
That makes 100 errors...: 86.
the: 236, 295, 296, 396, 399, 513, 1496.
The following...deleted: 679, 1046, 1175.
\the primitive: 295.
the_toks: 500, 501, 502, 513, 1351, 1498.
thick_mu_skip: 250.
\thickmuskip primitive: 252.
thick_mu_skip_code: 250, 251, 252, 814.
thickness: 725, 739, 768, 787, 788, 790, 791, 1236.
thin_mu_skip: 250.
\thinmuskip primitive: 252.
thin_mu_skip_code: 250, 251, 252, 255, 814.
This can't happen: 99.
/: 116.
align: 848.
copying: 232.
curlevel: 311.
disc1: 889.
disc2: 890.
disc3: 918.
disc4: 919.
display: 1254.
endv: 839.
ext1: 1403.
ext2: 1417.
ext3: 1418.
ext4: 1436.
flushing: 228.
if: 532.
line breaking: 925.
LR1: 1522.
LR2: 1535.
LR3: 1541.
mlist1: 771.
mlist2: 798.
mlist3: 809.
mlist4: 814.
page: 1054.
paragraph: 914.
prefix: 1265.
pruning: 1022.
right: 1239.
rightbrace: 1122.
tail1: 1134.
too many spans: 846.
vcenter: 779.
vertbreak: 1027.
vlistout: 668.
vpack: 711.
this_box: 655, 656, 662, 663, 667, 671, 672, 1524, 1525, 1531, 1532, 1533.
this_if: 533, 536, 538, 540, 541.
this_math_style: 800, 805.
three_codes: 684.
threshold: 876, 899, 902, 911.
Tight \hbox...: 709.
Tight \vbox...: 720.
tight_fit: 865, 867, 878, 881, 882, 884, 901, 1653, 1659.
time: 262, 267, 653.
\time primitive: 264.
time_code: 262, 263, 264.
tini: 8.
to: 684, 1136, 1279.
tok_val: 444, 449, 452, 462, 500, 1278, 1280, 1281, 1626, 1634.
tok_val_limit: 1626, 1648.
token: 319.
token_list: 337, 341, 342, 353, 355, 360, 367, 371, 376, 424, 1088, 1185, 1389, 1586.
token_ref_count: 226, 229, 321, 508, 517, 1033, 1493.
token_show: 325, 326, 353, 435, 1333, 1338, 1351, 1433, 1498, 1564.
token_type: 337, 341, 342, 344, 349, 353, 354, 355, 357, 413, 424, 1080, 1088, 1149.
tokens_to_string: 1410.
toks: 256.
\toks primitive: 295.
toks_base: 256, 257, 258, 259, 337, 449, 1278, 1280, 1281.
\toksdef primitive: 1276.
toks_def_code: 1276, 1278.
toks_register: 235, 295, 296, 447, 449, 1264, 1275, 1278, 1280, 1281, 1634, 1644, 1645.
tolerance: 262, 266, 876, 911.
\tolerance primitive: 264.
tolerance_code: 262, 263, 264.
Too many }'s: 1122.
too_big: 1610.
too_big_char: 12, 38, 48, 441, 584, 940, 1007.
too_big_lang: 12, 941.
too_big_usv: 12, 295, 364, 406, 541.
too_small: 1357, 1360.
top: 581.
top_bot_mark: 236, 326, 396, 399, 418, 419, 420, 1620.

top_edge: 667, 674.
top_mark: 416, 417, 1066, 1620, 1639.
\topmark primitive: 418.
top_mark_code: 416, 418, 420, 1389, 1620, 1642.
\topmarks primitive: 1620.
top_skip: 250.
\topskip primitive: 252.
top_skip_code: 250, 251, 252, 1055.
total_chars: 689, 1420.
total_demerits: 867, 893, 894, 903, 912, 922, 923.
total height: 1040.
total_mathex_params: 743, 1249.
total_mathsy_params: 742, 1249.
total_pages: 628, 629, 653, 678, 680.
total_pic_node_size: 1417, 1418.
total_pw: 877, 899.
total_shrink: 685, 690, 698, 706, 707, 708, 709, 713, 718, 719, 720, 844, 1255.
total_stretch: 685, 690, 698, 700, 701, 702, 713, 715, 716, 844.
Trabb Pardo, Luis Isidoro: 2.
tracing_assigns: 262, 307, 1650, 1651.
\tracingassigns primitive: 1467.
tracing_assigns_code: 262, 1467, 1469.
tracing_commands: 262, 399, 533, 544, 545, 1085, 1265.
\tracingcommands primitive: 264.
tracing_commands_code: 262, 263, 264.
tracing_groups: 262, 304, 312.
\tracinggroups primitive: 1467.
tracing_groups_code: 262, 1467, 1469.
tracing_ifs: 262, 329, 529, 533, 545.
\tracingifs primitive: 1467.
tracing_ifs_code: 262, 1467, 1469.
tracing_lost_chars: 262, 616, 744, 1088.
\tracinglostchars primitive: 264.
tracing_lost_chars_code: 262, 263, 264.
tracing_macros: 262, 353, 423, 434.
\tracingmacros primitive: 264.
tracing_macros_code: 262, 263, 264.
tracing_nesting: 262, 392, 1585, 1586, 1587, 1588.
\tracingnesting primitive: 1467.
tracing_nesting_code: 262, 1467, 1469.
tracing_online: 262, 271, 616, 1347, 1352.
\tracingonline primitive: 264.
tracing_online_code: 262, 263, 264.
tracing_output: 262, 676, 679.
\tracingoutput primitive: 264.
tracing_output_code: 262, 263, 264.
tracing_pages: 262, 1041, 1059, 1064.
\tracingpages primitive: 264.
tracing_pages_code: 262, 263, 264.
tracing_paragraphs: 262, 874, 893, 903, 911.
\tracingparagraphs primitive: 264.
tracing_paragraphs_code: 262, 263, 264.
tracing_restores: 262, 313, 1652.
\tracingrestores primitive: 264.
tracing_restores_code: 262, 263, 264.
tracing_scan_tokens: 262, 1566.
\tracingscantokens primitive: 1467.
tracing_scan_tokens_code: 262, 1467, 1469.
tracing_stats: 139, 262, 677, 1380, 1387.
\tracingstats primitive: 264.
tracing_stats_code: 262, 263, 264.
Transcript written...: 1387.
transform: 1445.
transform_concat: 1445.
transform_point: 1445.
trap_zero_glue: 1282, 1283, 1290.
trick_buf: 54, 58, 59, 345, 347.
trick_count: 54, 58, 345, 346, 347.
Trickey, Howard Wellington: 2.
trie: 974, 975, 976, 1004, 1006, 1007, 1008, 1012, 1013, 1020, 1378, 1379.
trie_back: 1004, 1008, 1010.
trie_c: 1001, 1002, 1005, 1007, 1009, 1010, 1013, 1017, 1018, 1666, 1667.
trie_char: 974, 975, 977, 1012, 1013, 1669.
trie_fix: 1012, 1013.
trie_hash: 1001, 1002, 1003, 1004, 1006.
trie_l: 1001, 1002, 1003, 1011, 1013, 1014, 1017, 1018, 1667.
trie_link: 974, 975, 977, 1004, 1006, 1007, 1008, 1009, 1010, 1012, 1013, 1669.
trie_max: 1004, 1006, 1008, 1012, 1378, 1379.
trie_min: 1004, 1006, 1007, 1010, 1668.
trie_node: 1002, 1003.
trie_not_ready: 939, 988, 1004, 1005, 1014, 1020, 1378, 1379.
trie_o: 1001, 1002, 1013, 1017, 1018, 1667.
trie_op: 974, 975, 977, 978, 997, 1012, 1013, 1665, 1669.
trie_op_hash: 997, 998, 999, 1000, 1002, 1006.
trie_op_lang: 997, 998, 999, 1006.
trie_op_ptr: 997, 998, 999, 1000, 1378, 1379.
trie_op_size: 11, 975, 997, 998, 1000, 1378, 1379.
trie_op_val: 997, 998, 999, 1006.
trie_pack: 1011, 1020, 1668.
trie_pointer: 974, 975, 976, 1001, 1002, 1003, 1004, 1007, 1011, 1013, 1014, 1020, 1669.
trie_ptr: 1001, 1005, 1006, 1018.
trie_r: 1001, 1002, 1003, 1009, 1010, 1011, 1013, 1017, 1018, 1665, 1666, 1667.
trie_ref: 1004, 1006, 1007, 1010, 1011, 1013, 1668.

trie_root: 1001, 1003, 1005, 1006, 1012, 1020, 1665, 1668.
trie_size: 11, 974, 1002, 1004, 1006, 1008, 1018, 1379.
trie_taken: 1004, 1006, 1007, 1008, 1010.
trie_used: 997, 998, 999, 1000, 1378, 1379.
true: 4, 16, 31, 34, 37, 45, 46, 49, 51, 53, 59, 75, 81, 92, 101, 102, 108, 109, 110, 111, 116, 118, 119, 193, 194, 198, 282, 284, 286, 312, 341, 357, 358, 366, 376, 391, 392, 395, 406, 408, 412, 441, 447, 464, 474, 478, 481, 482, 488, 496, 497, 521, 536, 543, 547, 551, 559, 561, 569, 598, 613, 628, 659, 666, 675, 676, 679, 692, 705, 717, 749, 762, 839, 874, 875, 876, 877, 889, 890, 899, 902, 911, 914, 918, 919, 928, 929, 930, 932, 956, 959, 964, 965, 1005, 1010, 1016, 1017, 1046, 1074, 1075, 1079, 1084, 1089, 1091, 1094, 1105, 1108, 1134, 1137, 1144, 1155, 1175, 1199, 1217, 1248, 1249, 1272, 1278, 1280, 1290, 1291, 1307, 1312, 1324, 1333, 1337, 1352, 1357, 1390, 1396, 1409, 1410, 1431, 1434, 1437, 1443, 1444, 1445, 1451, 1466, 1472, 1489, 1567, 1578, 1579, 1585, 1586, 1588, 1601, 1604, 1608, 1610, 1630, 1636, 1638, 1641, 1650, 1654, 1667.
`true`: 488.
try_break: 876, 877, 887, 899, 906, 910, 914, 916, 917, 921, 927.
two: 105, 106.
two_choices: 135.
two_halves: 135, 140, 146, 197, 247, 282, 283, 726, 975, 1020.
two_to_the: 121, 122, 124.
tx: 447, 458, 1133, 1134, 1135, 1159.
type: 4, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 171, 172, 173, 174, 176, 177, 179, 180, 182, 183, 184, 185, 201, 209, 210, 228, 232, 458, 506, 507, 524, 530, 531, 532, 540, 656, 660, 661, 664, 666, 669, 670, 673, 675, 678, 688, 689, 691, 693, 697, 710, 711, 712, 722, 723, 724, 725, 728, 729, 730, 731, 738, 740, 744, 749, 756, 758, 763, 764, 769, 770, 771, 772, 774, 775, 779, 781, 791, 794, 796, 799, 805, 808, 809, 810, 815, 816, 844, 847, 849, 853, 855, 857, 858, 859, 864, 867, 868, 870, 877, 878, 880, 885, 889, 890, 891, 892, 893, 904, 906, 907, 908, 909, 910, 912, 913, 914, 916, 918, 919, 922, 923, 927, 929, 945, 949, 950, 952, 956, 957, 968, 1022, 1024, 1026, 1027, 1030, 1032, 1033, 1035, 1040, 1042, 1047, 1050, 1051, 1054, 1058, 1062, 1063, 1064, 1065, 1067, 1068, 1075, 1088, 1128, 1134, 1141, 1154, 1155, 1159, 1164, 1167, 1175, 1201, 1209, 1212, 1213, 1217, 1219, 1222, 1235, 1239, 1240, 1245, 1256, 1257, 1395, 1404, 1420, 1489, 1501, 1509, 1514, 1517, 1521, 1527, 1529, 1535, 1539, 1544, 1549, 1550, 1557, 1588, 1599, 1600, 1626.
`Type <return> to proceed...`: 89.
t2: 1445.
u: 73, 111, 131, 198, 423, 505, 595, 749, 839, 848, 983, 988, 998, 1140, 1311, 1555.
u_close: 359, 520, 521, 1329.
`\Udelcode` primitive: 1284.
`\Udelcodenum` primitive: 1284.
`\Udelimiter` primitive: 295.
u_make_name_string: 560.
`\Umathaccent` primitive: 295.
`\Umathchar` primitive: 295.
`\Umathchardef` primitive: 1276.
`\Umathcharnum` primitive: 295.
`\Umathcharnumdef` primitive: 1276.
`\Umathcode` primitive: 1284.
`\Umathcodenum` primitive: 1284.
u_part: 816, 817, 827, 836, 842, 849.
`\Uradical` primitive: 295.
u_template: 337, 344, 354, 836.
uc_code: 256, 258, 441.
`\uccode` primitive: 1284.
uc_code_base: 256, 261, 1284, 1285, 1340, 1342.
uc_hyph: 262, 939, 949.
`\uchyph` primitive: 264.
uc_hyph_code: 262, 263, 264.
un_hbox: 234, 1144, 1161, 1162, 1163.
`\unhbox` primitive: 1161.
`\unhcopy` primitive: 1161.
`\unkern` primitive: 1161.
`\unpenalty` primitive: 1161.
`\unskip` primitive: 1161.
un_vbox: 234, 1100, 1148, 1161, 1162, 1163, 1672.
`\unvbox` primitive: 1161.
`\unvcopy` primitive: 1161.
unbalance: 423, 425, 430, 433, 508, 512, 1493.
`Unbalanced output routine`: 1081.
`Unbalanced write...`: 1435.
`Undefined control sequence`: 404.
undefined_control_sequence: 248, 258, 282, 284, 286, 292, 298, 312, 320, 1372, 1373.
undefined_cs: 236, 248, 396, 406, 536, 1280, 1281, 1349, 1577, 1578.
undefined_primitive: 283, 289, 402, 403, 536, 1099.
under_noad: 729, 732, 738, 740, 776, 809, 1210, 1211.
underbarExtraDescender: 742.
underbarRuleThickness: 742.
underbarVerticalGap: 742.
`Underfull \hbox...`: 702.

Underfull \vbox...: 716.
\underline primitive: 1210.
undump: 1360, 1364, 1366, 1368, 1373, 1377, 1379, 1381, 1465.
undump_end: 1360.
undump_end_end: 1360.
undump_four_ASCII: 1364.
undump_hh: 1360, 1373, 1379.
undump_int: 1360, 1362, 1366, 1371, 1373, 1377, 1379, 1381.
undump_qqqq: 1360, 1364, 1377.
undump_size: 1360, 1364, 1375, 1379.
undump_size_end: 1360.
undump_size_end_end: 1360.
undump_wd: 1360, 1366, 1371, 1375.
\unexpanded primitive: 1496.
unfloat: 113, 700, 706, 715, 718, 749, 858, 859.
unhyphenated: 867, 877, 885, 912, 914, 916.
unicode_file: 515, 560.
UnicodeScalar: 18, 371, 505, 744, 942, 966, 1209.
unif_rand: 130, 507.
\uniformdeviate primitive: 503.
uniform_deviate_code: 503, 504, 506, 507.
unity: 105, 107, 123, 136, 189, 212, 482, 488, 603, 1313, 1428.
\unless primitive: 1573.
unless_code: 522, 523, 533, 1478, 1576.
unpackage: 1163, 1164.
unsave: 311, 313, 839, 848, 1080, 1117, 1122, 1140, 1154, 1173, 1187, 1222, 1228, 1240, 1245, 1248, 1250, 1254.
unset_node: 184, 201, 209, 210, 228, 232, 458, 691, 711, 724, 730, 731, 816, 844, 847, 849, 853.
update_active: 909.
update_adjust_list: 697.
update_corners: 1445.
update_heights: 1024, 1026, 1027, 1048, 1051, 1054.
update_prev_p: 181, 911, 914, 915, 917.
update_terminal: 34, 37, 65, 75, 90, 392, 559, 572, 676, 1334, 1392, 1566.
update_width: 880, 908.
\uppercase primitive: 1340.
upperLimitBaselineRiseMin: 742.
upperLimitGapMin: 742.
upwards: 667, 669, 670, 671, 721.
Use of x doesn't match...: 432.
use_err_help: 83, 84, 93, 94, 1337.
use_penalty: 744.
use_skip: 744.
usingGraphite: 584.
usingOpenType: 584.
UTF16_code: 18, 26, 60, 61, 62, 328, 371, 505, 548, 636, 744.
UTF8_code: 18.
v: 73, 111, 198, 423, 485, 749, 758, 779, 787, 793, 848, 878, 976, 988, 998, 1014, 1031, 1192, 1489.
v_offset: 273, 678, 679, 1428.
\voffset primitive: 274.
v_offset_code: 273, 274.
v_part: 816, 817, 827, 837, 842, 849.
v_template: 337, 344, 355, 424, 837, 1185.
vacuous: 474, 478, 479.
vadjust: 234, 295, 296, 1151, 1152, 1153, 1154.
\vadjust primitive: 295.
valign: 234, 295, 296, 1100, 1144, 1184, 1510, 1511.
\valign primitive: 295.
var_delimiter: 749, 780, 792, 810.
var_fam_class: 258.
var_used: 139, 147, 152, 189, 677, 1365, 1366.
vbadness: 262, 716, 719, 720, 1066, 1071.
\vbadness primitive: 264.
vbadness_code: 262, 263, 264.
\vbox primitive: 1125.
vbox_group: 299, 1137, 1139, 1471, 1489.
vcenter: 234, 295, 296, 1100, 1221.
\vcenter primitive: 295.
vcenter_group: 299, 1221, 1222, 1471, 1489.
vcenter_noad: 729, 732, 738, 740, 776, 809, 1222.
vert_break: 1024, 1025, 1030, 1031, 1034, 1036, 1064.
very_loose_fit: 865, 867, 878, 881, 882, 884, 900, 1653, 1658.
vet_glue: 663, 672.
\vfil primitive: 1112.
\vfilneg primitive: 1112.
\vfill primitive: 1112.
vfuzz: 273, 719, 1066, 1071.
\vfuzz primitive: 274.
vfuzz_code: 273, 274.
VIRTEX: 1385.
virtual memory: 148.
Vitter, Jeffrey Scott: 288.
vlist_node: 159, 172, 184, 201, 209, 210, 228, 232, 540, 654, 660, 661, 666, 667, 669, 670, 675, 678, 683, 691, 710, 711, 723, 749, 756, 758, 763, 779, 791, 794, 855, 857, 859, 889, 890, 914, 918, 919, 1022, 1027, 1032, 1054, 1128, 1134, 1141, 1164, 1201, 1535, 1544.
vlist_out: 628, 651, 652, 654, 655, 661, 666, 667, 670, 675, 676, 678, 735, 1436.
vmode: 237, 241, 450, 451, 452, 456, 458, 536, 823, 833, 834, 852, 855, 856, 857, 860, 1079, 1083, 1099, 1100, 1102, 1110, 1111, 1125, 1126, 1127,

1130, 1132, 1133, 1134, 1137, 1144, 1145, 1148, 1152, 1153, 1157, 1159, 1163, 1164, 1165, 1184, 1221, 1297, 1298, 1444, 1489, 1491.
vmove: 234, 1102, 1125, 1126, 1127, 1491.
void_pointer: 744, 749, 781, 793.
vpack: 262, 683, 684, 685, 710, 748, 778, 781, 803, 847, 852, 1031, 1075, 1154, 1222.
vpackage: 710, 844, 1031, 1071, 1140.
vrule: 234, 295, 296, 498, 1110, 1138, 1144.
`\vrule` primitive: 295.
vsize: 273, 1034, 1041.
`\vsize` primitive: 274.
vsize_code: 273, 274.
vskip: 234, 1100, 1111, 1112, 1113, 1132, 1148.
`\vskip` primitive: 1112.
vsplit: 1021, 1031, 1032, 1034, 1136, 1620, 1636, 1637.
`\vsplit needs a \vbox`: 1032.
`\vsplit` primitive: 1125.
vsplit_code: 1125, 1126, 1133, 1389, 1670, 1672, 1673.
vsplit_init: 1031, 1636, 1637.
`\vss` primitive: 1112.
`\vtop` primitive: 1125.
vtop_code: 1125, 1126, 1137, 1139, 1140.
vtop_group: 299, 1137, 1139, 1471, 1489.
w: 136, 171, 180, 305, 308, 309, 643, 689, 710, 749, 758, 781, 839, 848, 960, 1048, 1177, 1192, 1252, 1290, 1356, 1357, 1404, 1405, 1493, 1529, 1564, 1567, 1585, 1587, 1630, 1650, 1651.
w_close: 28, 1383, 1391.
w_make_name_string: 560, 1382.
w_open_in: 27, 559.
w_open_out: 27, 1382.
wait: 1066, 1074, 1075, 1076.
wake_up_terminal: 34, 37, 51, 75, 77, 393, 519, 559, 565, 1348, 1351, 1357, 1387, 1392.
`Warning: end of file when...`: 1588.
`Warning: end of...`: 1585, 1587.
warning_index: 335, 361, 368, 423, 424, 429, 430, 432, 435, 506, 508, 514, 517, 822, 825, 1493.
warning_issued: 80, 271, 1389, 1585, 1587, 1588.
warningType: 744.
was_free: 190, 192, 196.
was_hi_min: 190, 191, 192, 196.
was_lo_max: 190, 191, 192, 196.
was_mem_end: 190, 191, 192, 196.
`\wd` primitive: 450.
wdField: 1445.
`WEB`: 1, 4, 38, 40, 50, 1362.
what_lang: 1395, 1416, 1422, 1423, 1439, 1440.
what_lhm: 1395, 1416, 1422, 1423, 1439, 1440.
what_rhm: 1395, 1416, 1422, 1423, 1439, 1440.
whatsit_node: 168, 172, 201, 209, 228, 232, 656, 660, 669, 691, 711, 744, 749, 773, 781, 799, 809, 889, 890, 914, 918, 919, 945, 949, 952, 1022, 1027, 1054, 1167, 1175, 1201, 1395, 1404, 1420, 1510, 1536, 1544.
`\widowpenalties` primitive: 1675.
widow_penalties_loc: 256, 1675, 1676.
widow_penalties_ptr: 938, 1675.
widow_penalty: 262, 862, 938.
`\widowpenalty` primitive: 264.
widow_penalty_code: 262, 263, 264.
`width`: 498.
width: 157, 158, 160, 161, 169, 170, 171, 174, 175, 179, 180, 204, 209, 210, 213, 217, 218, 458, 463, 465, 486, 497, 498, 507, 589, 641, 643, 647, 656, 660, 661, 663, 664, 669, 671, 672, 673, 679, 688, 691, 693, 698, 699, 708, 710, 711, 712, 713, 721, 725, 730, 744, 749, 752, 757, 758, 759, 760, 774, 781, 783, 788, 791, 793, 794, 801, 802, 803, 816, 827, 841, 844, 845, 846, 849, 850, 851, 852, 854, 855, 856, 857, 858, 859, 875, 877, 885, 886, 889, 890, 914, 916, 918, 919, 929, 1023, 1030, 1050, 1055, 1058, 1063, 1088, 1096, 1098, 1108, 1145, 1147, 1177, 1179, 1201, 1202, 1253, 1255, 1259, 1283, 1293, 1294, 1419, 1420, 1421, 1422, 1428, 1430, 1445, 1509, 1524, 1526, 1529, 1530, 1531, 1532, 1535, 1536, 1539, 1540, 1545, 1547, 1549, 1551, 1556, 1557, 1591, 1601, 1605, 1606, 1607, 1609, 1664.
width_base: 585, 587, 589, 601, 604, 606, 611, 1376, 1377.
width_index: 578, 585.
width_offset: 157, 450, 451, 1301.
Wirth, Niklaus: 10.
wlog: 56, 58, 571, 1388.
wlog_cr: 56, 57, 58, 571, 1387.
wlog_ln: 56, 1388.
word_define: 1268, 1282, 1286, 1650.
word_define1: 1268.
word_file: 25, 27, 28, 135, 560, 1359.
word_node_size: 1631, 1632, 1648, 1652.
words: 230, 231, 232, 1417, 1544.
wrap_lig: 964, 965.
wrapup: 1089, 1094.
write: 37, 56, 58, 633.
`\write` primitive: 1398.
write_dvi: 633, 634, 635.
write_file: 57, 58, 1396, 1437, 1441.
write_ln: 35, 37, 51, 56, 57.
write_loc: 1367, 1368, 1398, 1400, 1434.

write_node: 1395, 1398, 1401, 1403, 1416, 1417, 1418, 1436, 1437.
write_node_size: 1395, 1405, 1407, 1408, 1409, 1417, 1418.
write_open: 1396, 1397, 1433, 1437, 1441.
write_out: 1433, 1437.
write_stream: 1395, 1405, 1409, 1415, 1433, 1437.
write_text: 337, 344, 353, 1394, 1434.
write_tokens: 1395, 1407, 1408, 1409, 1416, 1417, 1418, 1431, 1434.
writing: 613.
wterm: 56, 58, 65.
wterm_cr: 56, 57, 58.
wterm_ln: 56, 65, 559, 1357, 1386, 1391.
Wyatt, Douglas Kirk: 2.
w0: 621, 622, 640, 645.
w1: 621, 622, 643.
w2: 621, 781, 783.
w3: 621.
w4: 621.
x: 104, 109, 110, 111, 123, 128, 130, 131, 198, 623, 636, 689, 710, 749, 763, 769, 778, 780, 781, 787, 793, 800, 1177, 1192, 1356, 1357, 1555, 1604, 1610.
x_height: 582, 593, 594, 744, 781, 1177.
x_height_code: 582, 593.
x_ht: 744.
x_leaders: 173, 216, 665, 1125, 1126.
`\xleaders` primitive: 1125.
x_over_n: 110, 746, 759, 760, 1040, 1062, 1063, 1064, 1294.
x_size_req: 1445.
x_token: 394, 415, 513, 1088, 1092, 1206.
xchr: 20, 21, 23, 24, 38, 49, 58, 554.
xclause: 16.
xCoord: 1445.
`\xdef` primitive: 1262.
xdv_buffer: 638.
xdv_buffer_byte: 1430.
xeq_level: 279, 280, 298, 308, 309, 313, 1358.
XETEX: 2.
XeTeX_banner: 2.
`\XeTeXcharclass` primitive: 1284.
XeTeX_convert_codes: 503.
XeTeX_count_features_code: 450, 1452, 1453, 1454.
XeTeX_count_glyphs_code: 450, 1452, 1453, 1454.
XeTeX_count_selectors_code: 450, 1452, 1453, 1454.
XeTeX_count_variations_code: 450, 1452, 1453, 1454.
XeTeX_dash_break_code: 2, 1510, 1511.
XeTeX_dash_break_en: 1088, 1510.
XeTeX_dash_break_state: 1510.
`\XeTeXdashbreakstate` primitive: 1511.
XeTeX_def_code: 235, 447, 1264, 1284, 1285, 1286.
XeTeX_default_encoding_extension_code: 1398, 1401, 1403, 1511.
XeTeX_default_input_encoding: 1447, 1510.
XeTeX_default_input_encoding_code: 2, 1510.
XeTeX_default_input_mode: 1447, 1510.
XeTeX_default_input_mode_code: 2, 1510.
XeTeX_dim: 450, 458.
XeTeX_feature_code_code: 450, 1452, 1453, 1454.
XeTeX_feature_name_code: 503, 1452, 1459, 1460, 1461.
XeTeX_find_feature_by_name_code: 450, 1452, 1453, 1454.
XeTeX_find_selector_by_name_code: 450, 1452, 1453, 1454.
XeTeX_find_variation_by_name_code: 450, 1452, 1453, 1454.
XeTeX_first_char_code: 450, 1452, 1453, 1454.
XeTeX_first_expand_code: 503.
XeTeX_font_type_code: 450, 1452, 1453, 1454.
`\XeTeXgenerateactualtext` primitive: 1511.
XeTeX_generate_actual_text_code: 2, 1510, 1511.
XeTeX_generate_actual_text_en: 744, 1510.
XeTeX_generate_actual_text_state: 1510.
`\XeTeXglyph` primitive: 1399.
XeTeX_glyph_bounds_code: 450, 1452, 1453, 1458.
XeTeX_glyph_index_code: 450, 1452, 1453, 1454.
XeTeX_glyph_name_code: 503, 1452, 1459, 1460, 1461.
XeTeX_hyphenatable_length: 944, 1510, 1665.
`\XeTeXhyphenatablelength` primitive: 1511.
XeTeX_hyphenatable_length_code: 2, 1510, 1511.
XeTeX_input_encoding_extension_code: 1398, 1401, 1403, 1511.
XeTeX_input_mode_auto: 2, 1446.
XeTeX_input_mode_icu_mapping: 2.
XeTeX_input_mode_raw: 2.
XeTeX_input_mode_utf16be: 2.
XeTeX_input_mode_utf16le: 2.
XeTeX_input_mode_utf8: 2.
`\XeTeXinputnormalization` primitive: 1511.
XeTeX_input_normalization_code: 2, 1510, 1511.
XeTeX_input_normalization_state: 744, 1510.
XeTeX_int: 450, 1454.
XeTeX_inter_char_loc: 256, 337, 449, 1280, 1281, 1399, 1468.
XeTeX_inter_char_tokens_code: 2, 1510, 1511.
XeTeX_inter_char_tokens_en: 1088, 1510.
XeTeX_inter_char_tokens_state: 1510.
`\XeTeXinterchartoks` primitive: 1399.

\XeTeXinterwordspaceshaping primitive: 1511.
XeTeX_interword_space_shaping_code: 2, 1510, 1511.
XeTeX_interword_space_shaping_state: 655, 1088, 1510.
XeTeX_is_default_selector_code: 450, 1452, 1453, 1454.
XeTeX_is_exclusive_feature_code: 450, 1452, 1453, 1454.
XeTeX_last_char_code: 450, 1452, 1453, 1454.
XeTeX_last_dim_codes: 450.
XeTeX_last_item_codes: 450.
XeTeX_linebreak_locale: 262, 744, 1448.
\XeTeXlinebreaklocale primitive: 1399.
XeTeX_linebreak_locale_code: 262.
XeTeX_linebreak_locale_extension_code: 1398, 1399, 1401, 1403.
XeTeX_linebreak_penalty: 262, 744.
\XeTeXlinebreakpenalty primitive: 264.
XeTeX_linebreak_penalty_code: 262, 263, 264.
XeTeX_linebreak_skip: 250, 744.
\XeTeXlinebreakskip primitive: 252.
XeTeX_linebreak_skip_code: 250, 251, 252, 744.
XeTeX_map_char_to_glyph_code: 450, 1452, 1453, 1454.
XeTeX_math_char_def_code: 1276, 1277, 1278.
XeTeX_math_char_num_def_code: 1276, 1277, 1278.
XeTeX_math_given: 234, 447, 1100, 1205, 1208, 1277, 1278.
XeTeX_OT_count_features_code: 450, 1452, 1453, 1454.
XeTeX_OT_count_languages_code: 450, 1452, 1453, 1454.
XeTeX_OT_count_scripts_code: 450, 1452, 1453, 1454.
XeTeX_OT_feature_code: 450, 1452, 1453, 1454.
XeTeX_OT_language_code: 450, 1452, 1453, 1454.
XeTeX_OT_script_code: 450, 1452, 1453, 1454.
\XeTeXpdffile primitive: 1399.
XeTeX_pdf_page_count_code: 450, 1452, 1453, 1454.
\XeTeXpicfile primitive: 1399.
XeTeX_protrude_chars: 262, 899, 929, 935.
\XeTeXprotrudechars primitive: 264.
XeTeX_protrude_chars_code: 262, 263, 264.
XeTeX_revision: 2, 1461.
XeTeX_revision_code: 503, 1452, 1459, 1460, 1461.
xetex_scan_dimen: 482, 483.
XeTeX_selector_code_code: 450, 1452, 1453, 1454.
XeTeX_selector_name_code: 503, 1452, 1459, 1460, 1461.
XeTeX_tracing_fonts_code: 2, 1510, 1511.
XeTeX_tracing_fonts_state: 595, 744, 1510.
\Uchar primitive: 503.
XeTeX_Uchar_code: 503, 506, 507, 1459.
\Ucharcat primitive: 503.
XeTeX_Ucharcat_code: 503, 506, 507, 1459.
XeTeX_upwards: 710, 721, 1510.
XeTeX_upwards_code: 2, 1510, 1511.
\XeTeXupwardsmode primitive: 1511.
XeTeX_upwards_state: 1140, 1510.
XeTeX_use_glyph_metrics: 744, 947, 957, 1088, 1420, 1444, 1510.
\XeTeXuseglyphmetrics primitive: 1511.
XeTeX_use_glyph_metrics_code: 2, 1510, 1511.
XeTeX_use_glyph_metrics_state: 1510.
\XeTeXinterchartokenstate primitive: 1511.
XeTeX_variation_code: 450, 1452, 1453, 1454.
XeTeX_variation_default_code: 450, 1452, 1453, 1454.
XeTeX_variation_max_code: 450, 1452, 1453, 1454.
XeTeX_variation_min_code: 450, 1452, 1453, 1454.
XeTeX_variation_name_code: 503, 1452, 1459, 1460, 1461.
XeTeX_version: 2, 1454.
\XeTeXversion primitive: 1452.
XeTeX_version_code: 450, 1452, 1453, 1454.
XeTeX_version_string: 2.
\XeTeXrevision primitive: 1452.
xField: 1445.
xmalloc: 62.
xmalloc_array: 169.
xmax: 1445.
xmin: 1445.
xn_over_d: 111, 490, 492, 493, 603, 744, 759, 1098, 1314.
xord: 20, 24, 31, 52, 53, 558.
xpand: 508, 512, 514.
xray: 234, 1344, 1345, 1346, 1485, 1494, 1499.
xrealloc: 60.
xspace_skip: 250, 1097.
\xspaceskip primitive: 252.
xspace_skip_code: 250, 251, 252, 1097.
xtx_ligature_present: 692, 889, 890, 914, 918, 919.
xxx1: 621, 622, 678, 1431, 1436.
xxx2: 621.
xxx3: 621.
xxx4: 621, 622, 1431, 1436.
xy: 621.
x0: 621, 622, 640, 645.
x1: 621, 622, 643.
x2: 621.
x3: 621.

x4: 621.
y: 109, 123, 130, 749, 769, 778, 780, 781, 787, 793, 800, 1604.
y_here: 644, 645, 647, 648, 649.
y_OK: 644, 645, 648.
y_seen: 647, 648.
y_size_req: 1445.
yCoord: 1445.
year: 262, 267, 653, 1382.
\year primitive: 264.
year_code: 262, 263, 264.
yField: 1445.
ymax: 1445.
ymin: 1445.
You already have nine...: 511.
You can't \insert255: 1153.
You can't dump...: 1358.
You can't use \hrule...: 1149.
You can't use \long...: 1267.
You can't use \unless...: 1576.
You can't use a prefix with x: 1266.
You can't use x after ...: 462, 1291.
You can't use x in y mode: 1103.
You have to increase POOLSIZE: 52.
You want to edit file x: 88.
you_cant: 1103, 1104, 1134, 1160.
yz_OK: 644, 645, 646, 648.
y0: 621, 622, 630, 640, 645.
y1: 621, 622, 643, 649.
y2: 621, 630.
y3: 621.
y4: 621.
z: 123, 595, 749, 769, 787, 793, 800, 976, 981, 1007, 1013, 1252, 1555.
z_here: 644, 645, 647, 648, 650.
z_OK: 644, 645, 648.
z_seen: 647, 648.
Zabala Salelles, Ignacio Andrés: 2.
zero_glue: 187, 201, 250, 254, 458, 461, 497, 744, 749, 775, 850, 877, 935, 1095, 1096, 1097, 1225, 1283, 1551, 1593, 1601, 1620, 1631, 1632.
zero_token: 479, 487, 508, 511, 514.
z0: 621, 622, 640, 645.
z1: 621, 622, 643, 650.
z2: 621.
z3: 621.
z4: 621.

⟨ Accumulate the constant until *cur_tok* is not a suitable digit 479 ⟩ Used in section 478.
⟨ Add the width of node *s* to *act_width* 919 ⟩ Used in section 917.
⟨ Add the width of node *s* to *break_width* 890 ⟩ Used in section 888.
⟨ Add the width of node *s* to *disc_width* 918 ⟩ Used in section 917.
⟨ Adjust for the magnification ratio 492 ⟩ Used in section 488.
⟨ Adjust for the setting of `\globaldefs` 1268 ⟩ Used in section 1265.
⟨ Adjust *shift_up* and *shift_down* for the case of a fraction line 790 ⟩ Used in section 787.
⟨ Adjust *shift_up* and *shift_down* for the case of no fraction line 789 ⟩ Used in section 787.
⟨ Adjust the LR stack for the *hlist_out* routine; if necessary reverse an hlist segment and **goto** *reswitch* 1527 ⟩ Used in section 1526.
⟨ Adjust the LR stack for the *hpack* routine 1521 ⟩ Used in section 691.
⟨ Adjust the LR stack for the *init_math* routine 1548 ⟩ Used in section 1547.
⟨ Adjust the LR stack for the *just_reverse* routine 1550 ⟩ Used in section 1549.
⟨ Adjust the LR stack for the *post_line_break* routine 1518 ⟩ Used in sections 927, 929, and 1517.
⟨ Adjust the additional data for last line 1660 ⟩ Used in section 899.
⟨ Adjust the final line of the paragraph 1664 ⟩ Used in section 911.
⟨ Advance *cur_p* to the node following the present string of characters 915 ⟩ Used in section 914.
⟨ Advance past a whatsit node in the *line_break* loop 1422 ⟩ Used in section 914.
⟨ Advance past a whatsit node in the pre-hyphenation loop 1423 ⟩ Used in section 949.
⟨ Advance *r*; **goto** *found* if the parameter delimiter has been fully matched, otherwise **goto** *continue* 428 ⟩ Used in section 426.
⟨ Advance *q* past ignorable nodes 657 ⟩ Used in sections 656, 656, and 656.
⟨ Allocate entire node *p* and **goto** *found* 151 ⟩ Used in section 149.
⟨ Allocate from the top of node *p* and **goto** *found* 150 ⟩ Used in section 149.
⟨ Apologize for inability to do the operation now, unless `\unskip` follows non-glue 1160 ⟩ Used in section 1159.
⟨ Apologize for not loading the font, **goto** *done* 602 ⟩ Used in sections 601 and 744.
⟨ Append a ligature and/or kern to the translation; **goto** *continue* if the stack of inserted ligatures is nonempty 964 ⟩ Used in section 960.
⟨ Append a new leader node that uses *cur_box* 1132 ⟩ Used in section 1129.
⟨ Append a new letter or a hyphen level 1016 ⟩ Used in section 1015.
⟨ Append a new letter or hyphen 991 ⟩ Used in section 989.
⟨ Append a normal inter-word space to the current list, then **goto** *big_switch* 1095 ⟩ Used in section 1084.
⟨ Append a penalty node, if a nonzero penalty is appropriate 938 ⟩ Used in section 928.
⟨ Append an insertion to the current page and **goto** *contribute* 1062 ⟩ Used in section 1054.
⟨ Append any *new_hlist* entries for *q*, and any appropriate penalties 815 ⟩ Used in section 808.
⟨ Append box *cur_box* to the current list, shifted by *box_context* 1130 ⟩ Used in section 1129.
⟨ Append character *cur_chr* and the following characters (if any) to the current hlist in the current font; **goto** *reswitch* when a non-character has been fetched 1088 ⟩ Used in section 1084.
⟨ Append characters of $hu[j\,..]$ to *major_tail*, advancing *j* 971 ⟩ Used in section 970.
⟨ Append inter-element spacing based on *r_type* and *t* 814 ⟩ Used in section 808.
⟨ Append tabskip glue and an empty box to list *u*, and update *s* and *t* as the prototype nodes are passed 857 ⟩ Used in section 856.
⟨ Append the accent with appropriate kerns, then set $p \leftarrow q$ 1179 ⟩ Used in section 1177.
⟨ Append the current tabskip glue to the preamble list 826 ⟩ Used in section 825.
⟨ Append the display and perhaps also the equation number 1258 ⟩ Used in section 1253.
⟨ Append the glue or equation number following the display 1259 ⟩ Used in section 1253.
⟨ Append the glue or equation number preceding the display 1257 ⟩ Used in section 1253.
⟨ Append the new box to the current vertical list, followed by the list of special nodes taken out of the box by the packager 936 ⟩ Used in section 928.
⟨ Append the value *n* to list *p* 992 ⟩ Used in section 991.
⟨ Assign the values *depth_threshold* ← *show_box_depth* and *breadth_max* ← *show_box_breadth* 262 ⟩ Used in section 224.

⟨ Assignments 1271, 1272, 1275, 1278, 1279, 1280, 1282, 1286, 1288, 1289, 1295, 1296, 1302, 1306, 1307, 1310, 1318 ⟩ Used in section 1265.
⟨ Attach list p to the current list, and record its length; then finish up and **return** 1174 ⟩ Used in section 1173.
⟨ Attach subscript OpenType math kerning 806 ⟩ Used in sections 801 and 803.
⟨ Attach superscript OpenType math kerning 807 ⟩ Used in sections 802 and 803.
⟨ Attach the limits to y and adjust $height(v)$, $depth(v)$ to account for their presence 795 ⟩ Used in section 794.
⟨ Back up an outer control sequence so that it can be reread 367 ⟩ Used in section 366.
⟨ Basic printing procedures 57, 58, 59, 63, 66, 67, 68, 69, 292, 293, 553, 741, 1415, 1633 ⟩ Used in section 4.
⟨ Break the current page at node p, put it in box 255, and put the remaining nodes on the contribution list 1071 ⟩ Used in section 1068.
⟨ Break the paragraph at the chosen breakpoints, justify the resulting lines to the correct widths, and append them to the current vertical list 924 ⟩ Used in section 863.
⟨ Calculate page dimensions and margins 1428 ⟩ Used in section 653.
⟨ Calculate the length, l, and the shift amount, s, of the display lines 1203 ⟩ Used in section 1199.
⟨ Calculate the natural width, w, by which the characters of the final line extend to the right of the reference point, plus two ems; or set $w \leftarrow max_dimen$ if the non-blank information on that line is affected by stretching or shrinking 1200 ⟩ Used in section 1199.
⟨ Call the packaging subroutine, setting *just_box* to the justified box 937 ⟩ Used in section 928.
⟨ Call *try_break* if *cur_p* is a legal breakpoint; on the second pass, also try to hyphenate the next word, if *cur_p* is a glue node; then advance *cur_p* to the next node of the paragraph that could possibly be a legal breakpoint 914 ⟩ Used in section 911.
⟨ Carry out a ligature replacement, updating the cursor structure and possibly advancing j; **goto** *continue* if the cursor doesn't advance, otherwise **goto** *done* 965 ⟩ Used in section 963.
⟨ Case statement to copy different types and set *words* to the number of initial words not yet copied 232 ⟩ Used in section 231.
⟨ Cases for 'Fetch the *dead_cycles* or the *insert_penalties*' 1504 ⟩ Used in section 453.
⟨ Cases for evaluation of the current term 1602, 1606, 1607, 1609 ⟩ Used in section 1594.
⟨ Cases for fetching a dimension value 1458, 1481, 1484, 1615 ⟩ Used in section 458.
⟨ Cases for fetching a glue value 1618 ⟩ Used in section 1591.
⟨ Cases for fetching a mu value 1619 ⟩ Used in section 1591.
⟨ Cases for fetching an integer value 1454, 1475, 1478, 1614 ⟩ Used in section 458.
⟨ Cases for noads that can follow a *bin_noad* 776 ⟩ Used in section 771.
⟨ Cases for nodes that can appear in an mlist, after which we **goto** *done_with_node* 773 ⟩ Used in section 771.
⟨ Cases for *alter_integer* 1506 ⟩ Used in section 1300.
⟨ Cases for *conditional* 1577, 1578, 1580 ⟩ Used in section 536.
⟨ Cases for *do_marks* 1637, 1639, 1640, 1642 ⟩ Used in section 1636.
⟨ Cases for *eq_destroy* 1645 ⟩ Used in section 305.
⟨ Cases for *input* 1560 ⟩ Used in section 412.
⟨ Cases for *print_param* 1469, 1510 ⟩ Used in section 263.
⟨ Cases for *show_whatever* 1487, 1501 ⟩ Used in section 1347.
⟨ Cases of 'Let d be the natural width' that need special treatment 1547 ⟩ Used in section 1201.
⟨ Cases of 'Print the result of command c' 1461 ⟩ Used in section 507.
⟨ Cases of 'Scan the argument for command c' 1460 ⟩ Used in section 506.
⟨ Cases of *assign_toks* for *print_cmd_chr* 1468 ⟩ Used in section 257.
⟨ Cases of *convert* for *print_cmd_chr* 1459 ⟩ Used in section 504.
⟨ Cases of *expandafter* for *print_cmd_chr* 1574 ⟩ Used in section 296.
⟨ Cases of *flush_node_list* that arise in mlists only 740 ⟩ Used in section 228.
⟨ Cases of *handle_right_brace* where a *right_brace* triggers a delayed action 1139, 1154, 1172, 1186, 1187, 1222, 1227, 1240 ⟩ Used in section 1122.
⟨ Cases of *hlist_out* that arise in mixed direction text only 1530 ⟩ Used in section 660.
⟨ Cases of *if_test* for *print_cmd_chr* 1575 ⟩ Used in section 523.
⟨ Cases of *input* for *print_cmd_chr* 1559 ⟩ Used in section 411.

⟨ Cases of *last_item* for *print_cmd_chr* 1453, 1474, 1477, 1480, 1483, 1590, 1613, 1617 ⟩ Used in section 451.
⟨ Cases of *left_right* for *print_cmd_chr* 1508 ⟩ Used in section 1243.
⟨ Cases of *main_control* for *hmode* + *valign* 1513 ⟩ Used in section 1184.
⟨ Cases of *main_control* that are for extensions to TeX 1402 ⟩ Used in section 1099.
⟨ Cases of *main_control* that are not part of the inner loop 1099 ⟩ Used in section 1084.
⟨ Cases of *main_control* that build boxes and lists 1110, 1111, 1117, 1121, 1127, 1144, 1146, 1148, 1151, 1156, 1158, 1163, 1166, 1170, 1176, 1180, 1184, 1188, 1191, 1194, 1204, 1208, 1212, 1216, 1218, 1221, 1225, 1229, 1234, 1244, 1247 ⟩ Used in section 1099.
⟨ Cases of *main_control* that don't depend on *mode* 1264, 1322, 1325, 1328, 1330, 1339, 1344 ⟩ Used in section 1099.
⟨ Cases of *prefix* for *print_cmd_chr* 1582 ⟩ Used in section 1263.
⟨ Cases of *print_cmd_chr* for symbolic printing of primitives 253, 257, 265, 275, 296, 365, 411, 419, 446, 451, 504, 523, 527, 829, 1038, 1107, 1113, 1126, 1143, 1162, 1169, 1197, 1211, 1224, 1233, 1243, 1263, 1274, 1277, 1285, 1305, 1309, 1315, 1317, 1327, 1332, 1341, 1346, 1349, 1401 ⟩ Used in section 328.
⟨ Cases of *read* for *print_cmd_chr* 1571 ⟩ Used in section 296.
⟨ Cases of *register* for *print_cmd_chr* 1643 ⟩ Used in section 446.
⟨ Cases of *reverse* that need special treatment 1536, 1537, 1538, 1539 ⟩ Used in section 1535.
⟨ Cases of *set_page_int* for *print_cmd_chr* 1503 ⟩ Used in section 451.
⟨ Cases of *set_shape* for *print_cmd_chr* 1676 ⟩ Used in section 296.
⟨ Cases of *show_node_list* that arise in mlists only 732 ⟩ Used in section 209.
⟨ Cases of *the* for *print_cmd_chr* 1497 ⟩ Used in section 296.
⟨ Cases of *toks_register* for *print_cmd_chr* 1644 ⟩ Used in section 296.
⟨ Cases of *un_vbox* for *print_cmd_chr* 1673 ⟩ Used in section 1162.
⟨ Cases of *valign* for *print_cmd_chr* 1512 ⟩ Used in section 296.
⟨ Cases of *xray* for *print_cmd_chr* 1486, 1495, 1500 ⟩ Used in section 1346.
⟨ Cases where character is ignored 375 ⟩ Used in section 374.
⟨ Change buffered instruction to *y* or *w* and **goto** *found* 649 ⟩ Used in section 648.
⟨ Change buffered instruction to *z* or *x* and **goto** *found* 650 ⟩ Used in section 648.
⟨ Change current mode to −*vmode* for `\halign`, −*hmode* for `\valign` 823 ⟩ Used in section 822.
⟨ Change discretionary to compulsory and set *disc_break* ← *true* 930 ⟩ Used in section 929.
⟨ Change font *dvi_f* to *f* 659 ⟩ Used in sections 658, 1426, and 1430.
⟨ Change state if necessary, and **goto** *switch* if the current character should be ignored, or **goto** *reswitch* if the current character changes to another 374 ⟩ Used in section 373.
⟨ Change the case of the token in *p*, if a change is appropriate 1343 ⟩ Used in section 1342.
⟨ Change the current style and **goto** *delete_q* 811 ⟩ Used in section 809.
⟨ Change the interaction level and **return** 90 ⟩ Used in section 88.
⟨ Change this node to a style node followed by the correct choice, then **goto** *done_with_node* 774 ⟩ Used in section 773.
⟨ Character *s* is the current new-line character 270 ⟩ Used in sections 59 and 63.
⟨ Check flags of unavailable nodes 195 ⟩ Used in section 192.
⟨ Check for LR anomalies at the end of *hlist_out* 1528 ⟩ Used in section 1525.
⟨ Check for LR anomalies at the end of *hpack* 1522 ⟩ Used in section 689.
⟨ Check for LR anomalies at the end of *ship_out* 1541 ⟩ Used in section 676.
⟨ Check for charlist cycle 605 ⟩ Used in section 604.
⟨ Check for improper alignment in displayed math 824 ⟩ Used in section 822.
⟨ Check for special treatment of last line of paragraph 1654 ⟩ Used in section 875.
⟨ Check if node *p* is a new champion breakpoint; then **goto** *done* if *p* is a forced break or if the page-so-far is already too full 1028 ⟩ Used in section 1026.
⟨ Check if node *p* is a new champion breakpoint; then if it is time for a page break, prepare for output, and either fire up the user's output routine and **return** or ship out the page and **goto** *done* 1059 ⟩ Used in section 1051.
⟨ Check single-word *avail* list 193 ⟩ Used in section 192.
⟨ Check that another $ follows 1251 ⟩ Used in sections 1248, 1248, and 1260.

⟨ Check that nodes after *native_word* permit hyphenation; if not, **goto** *done1* 945 ⟩ Used in section 943.
⟨ Check that the necessary fonts for math symbols are present; if not, flush the current math lists and set *danger* ← *true* 1249 ⟩ Used in sections 1248 and 1248.
⟨ Check that the nodes following *hb* permit hyphenation and that at least *l_hyf* + *r_hyf* letters have been found, otherwise **goto** *done1* 952 ⟩ Used in section 943.
⟨ Check the "constant" values for consistency 14, 133, 320, 557, 1303 ⟩ Used in section 1386.
⟨ Check the pool check sum 53 ⟩ Used in section 52.
⟨ Check variable-size *avail* list 194 ⟩ Used in section 192.
⟨ Clean up the memory by removing the break nodes 913 ⟩ Used in sections 863 and 911.
⟨ Clear dimensions to zero 690 ⟩ Used in sections 689 and 710.
⟨ Clear off top level from *save_stack* 312 ⟩ Used in section 311.
⟨ Close the format file 1383 ⟩ Used in section 1356.
⟨ Coerce glue to a dimension 486 ⟩ Used in sections 484 and 490.
⟨ Compiler directives 9 ⟩ Used in section 4.
⟨ Complain about an undefined family and set *cur_i* null 766 ⟩ Used in section 765.
⟨ Complain about an undefined macro 404 ⟩ Used in section 399.
⟨ Complain about missing `\endcsname` 407 ⟩ Used in sections 406 and 1578.
⟨ Complain about unknown unit and **goto** *done2* 494 ⟩ Used in section 493.
⟨ Complain that `\the` can't do this; give zero result 462 ⟩ Used in section 447.
⟨ Complain that the user should have said `\mathaccent` 1220 ⟩ Used in section 1219.
⟨ Compleat the incompleat noad 1239 ⟩ Used in section 1238.
⟨ Complete a potentially long `\show` command 1352 ⟩ Used in section 1347.
⟨ Compute $f = \lfloor 2^{28}(1 + p/q) + \frac{1}{2} \rfloor$ 117 ⟩ Used in section 116.
⟨ Compute $p = \lfloor qf/2^{28} + \frac{1}{2} \rfloor - q$ 120 ⟩ Used in section 118.
⟨ Compute $f = \lfloor xn/d + \frac{1}{2} \rfloor$ 1611 ⟩ Used in section 1610.
⟨ Compute result of *multiply* or *divide*, put it in *cur_val* 1294 ⟩ Used in section 1290.
⟨ Compute result of *register* or *advance*, put it in *cur_val* 1292 ⟩ Used in section 1290.
⟨ Compute the amount of skew 785 ⟩ Used in section 781.
⟨ Compute the badness, b, of the current page, using *awful_bad* if the box is too full 1061 ⟩ Used in section 1059.
⟨ Compute the badness, b, using *awful_bad* if the box is too full 1029 ⟩ Used in section 1028.
⟨ Compute the demerits, d, from r to *cur_p* 907 ⟩ Used in section 903.
⟨ Compute the discretionary *break_width* values 888 ⟩ Used in section 885.
⟨ Compute the hash code h 288 ⟩ Used in section 286.
⟨ Compute the magic offset 813 ⟩ Used in section 1391.
⟨ Compute the mark pointer for mark type t and class *cur_val* 1635 ⟩ Used in section 420.
⟨ Compute the minimum suitable height, w, and the corresponding number of extension steps, n; also set *width*(b) 757 ⟩ Used in section 756.
⟨ Compute the new line width 898 ⟩ Used in section 883.
⟨ Compute the primitive code h 291 ⟩ Used in section 289.
⟨ Compute the register location l and its type p; but **return** if invalid 1291 ⟩ Used in section 1290.
⟨ Compute the sum of two glue specs 1293 ⟩ Used in section 1292.
⟨ Compute the sum or difference of two glue specs 1605 ⟩ Used in section 1603.
⟨ Compute the trie op code, v, and set $l \leftarrow 0$ 1019 ⟩ Used in section 1017.
⟨ Compute the values of *break_width* 885 ⟩ Used in section 884.
⟨ Consider a node with matching width; **goto** *found* if it's a hit 648 ⟩ Used in section 647.
⟨ Consider the demerits for a line from r to *cur_p*; deactivate node r if it should no longer be active; then **goto** *continue* if a line from r to *cur_p* is infeasible, otherwise record a new feasible break 899 ⟩ Used in section 877.
⟨ Constants in the outer block 11 ⟩ Used in section 4.
⟨ Construct a box with limits above and below it, skewed by *delta* 794 ⟩ Used in section 793.
⟨ Construct a sub/superscript combination box x, with the superscript offset by *delta* 803 ⟩ Used in section 800.
⟨ Construct a subscript box x when there is no superscript 801 ⟩ Used in section 800.

⟨ Construct a superscript box x 802 ⟩ Used in section 800.
⟨ Construct a vlist box for the fraction, according to *shift_up* and *shift_down* 791 ⟩ Used in section 787.
⟨ Construct an extensible character in a new box b, using recipe *rem_byte*(q) and font f 756 ⟩ Used in section 753.
⟨ Contribute an entire group to the current parameter 433 ⟩ Used in section 426.
⟨ Contribute the recently matched tokens to the current parameter, and **goto** *continue* if a partial match is still in effect; but abort if $s = null$ 431 ⟩ Used in section 426.
⟨ Convert a final *bin_noad* to an *ord_noad* 772 ⟩ Used in sections 769 and 771.
⟨ Convert *cur_val* to a lower level 463 ⟩ Used in section 447.
⟨ Convert math glue to ordinary glue 775 ⟩ Used in section 773.
⟨ Convert *nucleus*(q) to an hlist and attach the sub/superscripts 798 ⟩ Used in section 771.
⟨ Convert string s into a new pseudo file 1565 ⟩ Used in section 1564.
⟨ Copy the tabskip glue between columns 843 ⟩ Used in section 839.
⟨ Copy the templates from node *cur_loop* into node p 842 ⟩ Used in section 841.
⟨ Copy the token list 501 ⟩ Used in section 500.
⟨ Create a character node p for *nucleus*(q), possibly followed by a kern node for the italic correction, and set *delta* to the italic correction if a subscript is present 799 ⟩ Used in section 798.
⟨ Create a character node q for the next character, but set $q \leftarrow null$ if problems arise 1178 ⟩ Used in section 1177.
⟨ Create a new array element of type t with index i 1631 ⟩ Used in section 1630.
⟨ Create a new glue specification whose width is *cur_val*; scan for its stretch and shrink components 497 ⟩ Used in section 496.
⟨ Create a page insertion node with *subtype*$(r) = qi(n)$, and include the glue correction for box n in the current page state 1063 ⟩ Used in section 1062.
⟨ Create an active breakpoint representing the beginning of the paragraph 912 ⟩ Used in section 911.
⟨ Create and append a discretionary node as an alternative to the unhyphenated word, and continue to develop both branches until they become equivalent 968 ⟩ Used in section 967.
⟨ Create equal-width boxes x and z for the numerator and denominator, and compute the default amounts *shift_up* and *shift_down* by which they are displaced from the baseline 788 ⟩ Used in section 787.
⟨ Create new active nodes for the best feasible breaks just found 884 ⟩ Used in section 883.
⟨ Create the *format_ident*, open the format file, and inform the user that dumping has begun 1382 ⟩ Used in section 1356.
⟨ Current *mem* equivalent of glue parameter number n 250 ⟩ Used in sections 176 and 178.
⟨ Deactivate node r 908 ⟩ Used in section 899.
⟨ Declare ε-TeX procedures for expanding 1563, 1621, 1626, 1630 ⟩ Used in section 396.
⟨ Declare ε-TeX procedures for scanning 1492, 1583, 1592, 1597 ⟩ Used in section 443.
⟨ Declare ε-TeX procedures for token lists 1493, 1564 ⟩ Used in section 499.
⟨ Declare ε-TeX procedures for tracing and input 314, 1471, 1472, 1567, 1568, 1585, 1587, 1588, 1632, 1634, 1648, 1649, 1650, 1651, 1652 ⟩ Used in section 298.
⟨ Declare ε-TeX procedures for use by *main_control* 1466, 1489, 1505 ⟩ Used in section 863.
⟨ Declare action procedures for use by *main_control* 1097, 1101, 1103, 1104, 1105, 1108, 1114, 1115, 1118, 1123, 1124, 1129, 1133, 1138, 1140, 1145, 1147, 1149, 1150, 1153, 1155, 1157, 1159, 1164, 1167, 1171, 1173, 1177, 1181, 1183, 1185, 1189, 1190, 1192, 1196, 1205, 1209, 1213, 1214, 1217, 1219, 1226, 1228, 1230, 1235, 1245, 1248, 1254, 1265, 1324, 1329, 1333, 1342, 1347, 1356, 1403, 1439 ⟩ Used in section 1084.
⟨ Declare math construction procedures 777, 778, 779, 780, 781, 787, 793, 796, 800, 810 ⟩ Used in section 769.
⟨ Declare procedures for preprocessing hyphenation patterns 998, 1002, 1003, 1007, 1011, 1013, 1014, 1020 ⟩ Used in section 996.
⟨ Declare procedures needed for displaying the elements of mlists 733, 734, 736 ⟩ Used in section 205.
⟨ Declare procedures needed for expressions 1593, 1598 ⟩ Used in section 496.
⟨ Declare procedures needed in *do_extension* 1404, 1405, 1445, 1456 ⟩ Used in section 1403.
⟨ Declare procedures needed in *hlist_out*, *vlist_out* 1431, 1433, 1436, 1529, 1533 ⟩ Used in section 655.
⟨ Declare procedures that need to be declared forward for pdfTeX 1411 ⟩ Used in section 198.

⟨ Declare procedures that scan font-related stuff 612, 613 ⟩ Used in section 443.
⟨ Declare procedures that scan restricted classes of integers 467, 468, 469, 470, 471, 1622 ⟩ Used in section 443.
⟨ Declare subprocedures for *after_math* 1555 ⟩ Used in section 1248.
⟨ Declare subprocedures for *init_math* 1544, 1549 ⟩ Used in section 1192.
⟨ Declare subprocedures for *line_break* 874, 877, 925, 944, 996 ⟩ Used in section 863.
⟨ Declare subprocedures for *prefixed_command* 1269, 1283, 1290, 1297, 1298, 1299, 1300, 1301, 1311, 1319 ⟩ Used in section 1265.
⟨ Declare subprocedures for *scan_expr* 1604, 1608, 1610 ⟩ Used in section 1593.
⟨ Declare subprocedures for *var_delimiter* 752, 754, 755 ⟩ Used in section 749.
⟨ Declare subroutines for *new_character* 616, 744 ⟩ Used in section 617.
⟨ Declare the function called *do_marks* 1636 ⟩ Used in section 1031.
⟨ Declare the function called *fin_mlist* 1238 ⟩ Used in section 1228.
⟨ Declare the function called *open_fmt_file* 559 ⟩ Used in section 1357.
⟨ Declare the function called *reconstitute* 960 ⟩ Used in section 944.
⟨ Declare the procedure called *align_peek* 833 ⟩ Used in section 848.
⟨ Declare the procedure called *fire_up* 1066 ⟩ Used in section 1048.
⟨ Declare the procedure called *get_preamble_token* 830 ⟩ Used in section 822.
⟨ Declare the procedure called *handle_right_brace* 1122 ⟩ Used in section 1084.
⟨ Declare the procedure called *init_span* 835 ⟩ Used in section 834.
⟨ Declare the procedure called *insert_relax* 413 ⟩ Used in section 396.
⟨ Declare the procedure called *macro_call* 423 ⟩ Used in section 396.
⟨ Declare the procedure called *print_cmd_chr* 328, 1457 ⟩ Used in section 278.
⟨ Declare the procedure called *print_skip_param* 251 ⟩ Used in section 205.
⟨ Declare the procedure called *runaway* 336 ⟩ Used in section 141.
⟨ Declare the procedure called *show_token_list* 322 ⟩ Used in section 141.
⟨ Decry the invalid character and **goto** *restart* 376 ⟩ Used in section 374.
⟨ Delete $c -$ `"0"` tokens and **goto** *continue* 92 ⟩ Used in section 88.
⟨ Delete the page-insertion nodes 1073 ⟩ Used in section 1068.
⟨ Destroy the t nodes following q, and make r point to the following node 931 ⟩ Used in section 930.
⟨ Determine horizontal glue shrink setting, then **return** or **goto** *common_ending* 706 ⟩ Used in section 699.
⟨ Determine horizontal glue stretch setting, then **return** or **goto** *common_ending* 700 ⟩ Used in section 699.
⟨ Determine the displacement, d, of the left edge of the equation, with respect to the line size z, assuming that $l = false$ 1256 ⟩ Used in section 1253.
⟨ Determine the shrink order 707 ⟩ Used in sections 706, 718, and 844.
⟨ Determine the stretch order 701 ⟩ Used in sections 700, 715, and 844.
⟨ Determine the value of $height(r)$ and the appropriate glue setting; then **return** or **goto** *common_ending* 714 ⟩ Used in section 710.
⟨ Determine the value of $width(r)$ and the appropriate glue setting; then **return** or **goto** *common_ending* 699 ⟩ Used in section 689.
⟨ Determine vertical glue shrink setting, then **return** or **goto** *common_ending* 718 ⟩ Used in section 714.
⟨ Determine vertical glue stretch setting, then **return** or **goto** *common_ending* 715 ⟩ Used in section 714.
⟨ Discard erroneous prefixes and **return** 1266 ⟩ Used in section 1265.
⟨ Discard the prefixes `\long` and `\outer` if they are irrelevant 1267 ⟩ Used in section 1265.
⟨ Dispense with trivial cases of void or bad boxes 1032 ⟩ Used in section 1031.
⟨ Display adjustment p 223 ⟩ Used in section 209.
⟨ Display box p 210 ⟩ Used in section 209.
⟨ Display choice node p 737 ⟩ Used in section 732.
⟨ Display discretionary p 221 ⟩ Used in section 209.
⟨ Display fraction noad p 739 ⟩ Used in section 732.
⟨ Display glue p 215 ⟩ Used in section 209.
⟨ Display if this box is never to be reversed 1514 ⟩ Used in section 210.
⟨ Display insertion p 214 ⟩ Used in section 209.

⟨ Display kern p 217 ⟩ Used in section 209.
⟨ Display leaders p 216 ⟩ Used in section 215.
⟨ Display ligature p 219 ⟩ Used in section 209.
⟨ Display mark p 222 ⟩ Used in section 209.
⟨ Display math node p 218 ⟩ Used in section 209.
⟨ Display node p 209 ⟩ Used in section 208.
⟨ Display normal noad p 738 ⟩ Used in section 732.
⟨ Display penalty p 220 ⟩ Used in section 209.
⟨ Display rule p 213 ⟩ Used in section 209.
⟨ Display special fields of the unset node p 211 ⟩ Used in section 210.
⟨ Display the current context 342 ⟩ Used in section 341.
⟨ Display the insertion split cost 1065 ⟩ Used in section 1064.
⟨ Display the page break cost 1060 ⟩ Used in section 1059.
⟨ Display the token (m, c) 324 ⟩ Used in section 323.
⟨ Display the value of b 537 ⟩ Used in section 533.
⟨ Display the value of *glue_set*(p) 212 ⟩ Used in section 210.
⟨ Display the whatsit node p 1416 ⟩ Used in section 209.
⟨ Display token p, and **return** if there are problems 323 ⟩ Used in section 322.
⟨ Do first-pass processing based on *type*(q); **goto** *done_with_noad* if a noad has been fully processed, **goto** *check_dimensions* if it has been translated into *new_hlist*(q), or **goto** *done_with_node* if a node has been fully processed 771 ⟩ Used in section 770.
⟨ Do ligature or kern command, returning to *main_lig_loop* or *main_loop_wrapup* or *main_loop_move* 1094 ⟩ Used in section 1093.
⟨ Do magic computation 350 ⟩ Used in section 322.
⟨ Do some work that has been queued up for `\write` 1437 ⟩ Used in section 1436.
⟨ Drop current token and complain that it was unmatched 1120 ⟩ Used in section 1118.
⟨ Dump a couple more things and the closing check word 1380 ⟩ Used in section 1356.
⟨ Dump constants for consistency check 1361 ⟩ Used in section 1356.
⟨ Dump regions 1 to 4 of *eqtb* 1369 ⟩ Used in section 1367.
⟨ Dump regions 5 and 6 of *eqtb* 1370 ⟩ Used in section 1367.
⟨ Dump the ε-TeX state 1464, 1569 ⟩ Used in section 1361.
⟨ Dump the array info for internal font number k 1376 ⟩ Used in section 1374.
⟨ Dump the dynamic memory 1365 ⟩ Used in section 1356.
⟨ Dump the font information 1374 ⟩ Used in section 1356.
⟨ Dump the hash table 1372 ⟩ Used in section 1367.
⟨ Dump the hyphenation tables 1378 ⟩ Used in section 1356.
⟨ Dump the string pool 1363 ⟩ Used in section 1356.
⟨ Dump the table of equivalents 1367 ⟩ Used in section 1356.
⟨ Either append the insertion node p after node q, and remove it from the current page, or delete *node*(p) 1076 ⟩ Used in section 1074.
⟨ Either insert the material specified by node p into the appropriate box, or hold it for the next page; also delete node p from the current page 1074 ⟩ Used in section 1068.
⟨ Either process `\ifcase` or set b to the value of a boolean condition 536 ⟩ Used in section 533.
⟨ Empty the last bytes out of *dvi_buf* 635 ⟩ Used in section 680.
⟨ Enable ε-TeX, if requested 1451 ⟩ Used in section 1391.
⟨ Ensure that box 255 is empty after output 1082 ⟩ Used in section 1080.
⟨ Ensure that box 255 is empty before output 1069 ⟩ Used in section 1068.
⟨ Ensure that *trie_max* $\geq h +$ *max_hyph_char* 1008 ⟩ Used in section 1007.
⟨ Enter a hyphenation exception 993 ⟩ Used in section 989.
⟨ Enter all of the patterns into a linked trie, until coming to a right brace 1015 ⟩ Used in section 1014.
⟨ Enter as many hyphenation exceptions as are listed, until coming to a right brace; then **return** 989 ⟩ Used in section 988.

⟨ Enter *skip_blanks* state, emit a space 379 ⟩ Used in section 377.
⟨ Error handling procedures 82, 85, 86, 97, 98, 99, 1455 ⟩ Used in section 4.
⟨ Evaluate the current expression 1603 ⟩ Used in section 1594.
⟨ Examine node p in the hlist, taking account of its effect on the dimensions of the new box, or moving it to the adjustment list; then advance p to the next node 691 ⟩ Used in section 689.
⟨ Examine node p in the vlist, taking account of its effect on the dimensions of the new box; then advance p to the next node 711 ⟩ Used in section 710.
⟨ Expand a nonmacro 399 ⟩ Used in section 396.
⟨ Expand macros in the token list and make *link*(*def_ref*) point to the result 1434 ⟩ Used in section 1433.
⟨ Expand the next part of the input 513 ⟩ Used in section 512.
⟨ Expand the token after the next token 400 ⟩ Used in section 399.
⟨ Explain that too many dead cycles have occurred in a row 1078 ⟩ Used in section 1066.
⟨ Express astonishment that no number was here 480 ⟩ Used in section 478.
⟨ Express consternation over the fact that no alignment is in progress 1182 ⟩ Used in section 1181.
⟨ Express shock at the missing left brace; **goto** *found* 510 ⟩ Used in section 509.
⟨ Feed the macro body and its parameters to the scanner 424 ⟩ Used in section 423.
⟨ Fetch a box dimension 454 ⟩ Used in section 447.
⟨ Fetch a character code from some table 448 ⟩ Used in section 447.
⟨ Fetch a font dimension 459 ⟩ Used in section 447.
⟨ Fetch a font integer 460 ⟩ Used in section 447.
⟨ Fetch a penalties array element 1677 ⟩ Used in section 457.
⟨ Fetch a register 461 ⟩ Used in section 447.
⟨ Fetch a token list or font identifier, provided that *level* = *tok_val* 449 ⟩ Used in section 447.
⟨ Fetch an internal dimension and **goto** *attach_sign*, or fetch an internal integer 484 ⟩ Used in section 482.
⟨ Fetch an item in the current node, if appropriate 458 ⟩ Used in section 447.
⟨ Fetch first character of a sub/superscript 805 ⟩ Used in sections 801, 802, and 803.
⟨ Fetch something on the *page_so_far* 455 ⟩ Used in section 447.
⟨ Fetch the *dead_cycles* or the *insert_penalties* 453 ⟩ Used in section 447.
⟨ Fetch the *par_shape* size 457 ⟩ Used in section 447.
⟨ Fetch the *prev_graf* 456 ⟩ Used in section 447.
⟨ Fetch the *space_factor* or the *prev_depth* 452 ⟩ Used in section 447.
⟨ Find an active node with fewest demerits 922 ⟩ Used in section 921.
⟨ Find hyphen locations for the word in *hc*, or **return** 977 ⟩ Used in section 944.
⟨ Find optimal breakpoints 911 ⟩ Used in section 863.
⟨ Find the best active node for the desired looseness 923 ⟩ Used in section 921.
⟨ Find the best way to split the insertion, and change *type*(*r*) to *split_up* 1064 ⟩ Used in section 1062.
⟨ Find the glue specification, *main_p*, for text spaces in the current font 1096 ⟩ Used in sections 1095 and 1097.
⟨ Finish an alignment in a display 1260 ⟩ Used in section 860.
⟨ Finish displayed math 1253 ⟩ Used in section 1248.
⟨ Finish issuing a diagnostic message for an overfull or underfull hbox 705 ⟩ Used in section 689.
⟨ Finish issuing a diagnostic message for an overfull or underfull vbox 717 ⟩ Used in section 710.
⟨ Finish line, emit a `\par` 381 ⟩ Used in section 377.
⟨ Finish line, emit a space 378 ⟩ Used in section 377.
⟨ Finish line, **goto** *switch* 380 ⟩ Used in section 377.
⟨ Finish math in text 1250 ⟩ Used in section 1248.
⟨ Finish the `DVI` file 680 ⟩ Used in section 1387.
⟨ Finish the extensions 1441 ⟩ Used in section 1387.
⟨ Finish the natural width computation 1546 ⟩ Used in section 1200.
⟨ Finish the reversed hlist segment and **goto** *done* 1540 ⟩ Used in section 1539.
⟨ Finish *hlist_out* for mixed direction typesetting 1525 ⟩ Used in section 655.
⟨ Fire up the user's output routine and **return** 1079 ⟩ Used in section 1066.
⟨ Fix the reference count, if any, and negate *cur_val* if *negative* 464 ⟩ Used in section 447.

⟨ Flush the box from memory, showing statistics if requested 677 ⟩ Used in section 676.
⟨ Flush the prototype box 1554 ⟩ Used in section 1253.
⟨ Forbidden cases detected in *main_control* 1102, 1152, 1165, 1198 ⟩ Used in section 1099.
⟨ Generate a *down* or *right* command for w and **return** 646 ⟩ Used in section 643.
⟨ Generate a *y0* or *z0* command in order to reuse a previous appearance of w 645 ⟩ Used in section 643.
⟨ Generate all ε-TeX primitives 1399, 1452, 1467, 1473, 1476, 1479, 1482, 1485, 1494, 1496, 1499, 1502, 1507, 1511, 1558, 1570, 1573, 1581, 1589, 1612, 1616, 1620, 1672, 1675 ⟩ Used in section 1451.
⟨ Get ready to compress the trie 1006 ⟩ Used in section 1020.
⟨ Get ready to start line breaking 864, 875, 882, 896 ⟩ Used in section 863.
⟨ Get the first line of input and prepare to start 1391 ⟩ Used in section 1386.
⟨ Get the next non-blank non-call token 440 ⟩ Used in sections 439, 475, 490, 538, 561, 612, 1099, 1595, and 1596.
⟨ Get the next non-blank non-relax non-call token 438 ⟩ Used in sections 437, 1132, 1138, 1205, 1214, 1265, 1280, and 1324.
⟨ Get the next non-blank non-sign token; set *negative* appropriately 475 ⟩ Used in sections 474, 482, and 496.
⟨ Get the next token, suppressing expansion 388 ⟩ Used in section 387.
⟨ Get user's advice and **return** 87 ⟩ Used in section 86.
⟨ Give diagnostic information, if requested 1085 ⟩ Used in section 1084.
⟨ Give improper `\hyphenation` error 990 ⟩ Used in section 989.
⟨ Global variables 13, 20, 26, 30, 32, 39, 50, 54, 61, 77, 80, 83, 100, 108, 114, 121, 137, 138, 139, 140, 146, 181, 190, 199, 207, 239, 272, 279, 282, 283, 301, 316, 327, 331, 334, 335, 338, 339, 340, 363, 391, 397, 416, 421, 422, 444, 472, 481, 515, 524, 528, 547, 548, 555, 562, 567, 574, 584, 585, 590, 628, 631, 641, 652, 682, 685, 686, 695, 703, 726, 762, 767, 812, 818, 862, 869, 871, 873, 876, 881, 887, 895, 920, 940, 953, 959, 961, 975, 980, 997, 1001, 1004, 1025, 1034, 1036, 1043, 1086, 1128, 1320, 1335, 1353, 1359, 1385, 1396, 1400, 1429, 1449, 1462, 1470, 1515, 1561, 1584, 1625, 1627, 1646, 1653, 1669, 1670 ⟩ Used in section 4.
⟨ Go into display math mode 1199 ⟩ Used in section 1192.
⟨ Go into ordinary math mode 1193 ⟩ Used in sections 1192 and 1196.
⟨ Go through the preamble list, determining the column widths and changing the alignrecords to dummy unset boxes 849 ⟩ Used in section 848.
⟨ Grow more variable-size memory and **goto** *restart* 148 ⟩ Used in section 147.
⟨ Handle `\readline` and **goto** *done* 1572 ⟩ Used in section 518.
⟨ Handle `\unexpanded` or `\detokenize` and **return** 1498 ⟩ Used in section 500.
⟨ Handle a glue node for mixed direction typesetting 1509 ⟩ Used in sections 663 and 1537.
⟨ Handle a math node in *hlist_out* 1526 ⟩ Used in section 660.
⟨ Handle non-positive logarithm 125 ⟩ Used in section 123.
⟨ Handle saved items and **goto** *done* 1674 ⟩ Used in section 1164.
⟨ Handle situations involving spaces, braces, changes of state 377 ⟩ Used in section 374.
⟨ Hyphenate the *native_word_node* at *ha* 957 ⟩ Used in section 956.
⟨ If a line number class has ended, create new active nodes for the best feasible breaks in that class; then **return** if $r = last_active$, otherwise compute the new *line_width* 883 ⟩ Used in section 877.
⟨ If all characters of the family fit relative to h, then **goto** *found*, otherwise **goto** *not_found* 1009 ⟩ Used in section 1007.
⟨ If an alignment entry has just ended, take appropriate action 372 ⟩ Used in section 371.
⟨ If an expanded code is present, reduce it and **goto** *start_cs* 385 ⟩ Used in sections 384 and 386.
⟨ If dumping is not allowed, abort 1358 ⟩ Used in section 1356.
⟨ If instruction *cur_i* is a kern with *cur_c*, attach the kern after q; or if it is a ligature with *cur_c*, combine noads q and p appropriately; then **return** if the cursor has moved past a noad, or **goto** *restart* 797 ⟩ Used in section 796.
⟨ If no hyphens were found, **return** 955 ⟩ Used in section 944.
⟨ If node *cur_p* is a legal breakpoint, call *try_break*; then update the active widths by including the glue in *glue_ptr*(*cur_p*) 916 ⟩ Used in section 914.
⟨ If node p is a legal breakpoint, check if this break is the best known, and **goto** *done* if p is null or if the page-so-far is already too full to accept more stuff 1026 ⟩ Used in section 1024.

⟨ If node q is a style node, change the style and **goto** *delete_q*; otherwise if it is not a noad, put it into the hlist, advance q, and **goto** *done*; otherwise set s to the size of noad q, set t to the associated type (*ord_noad* .. *inner_noad*), and set *pen* to the associated penalty 809 ⟩ Used in section 808.
⟨ If node r is of type *delta_node*, update *cur_active_width*, set *prev_r* and *prev_prev_r*, then **goto** *continue* 880 ⟩ Used in section 877.
⟨ If the current list ends with a box node, delete it from the list and make *cur_box* point to it; otherwise set *cur_box* ← *null* 1134 ⟩ Used in section 1133.
⟨ If the current page is empty and node p is to be deleted, **goto** *done1*; otherwise use node p to update the state of the current page; if this node is an insertion, **goto** *contribute*; otherwise if this node is not a legal breakpoint, **goto** *contribute* or *update_heights*; otherwise set *pi* to the penalty associated with this breakpoint 1054 ⟩ Used in section 1051.
⟨ If the cursor is immediately followed by the right boundary, **goto** *reswitch*; if it's followed by an invalid character, **goto** *big_switch*; otherwise move the cursor one step to the right and **goto** *main_lig_loop* 1090 ⟩ Used in section 1088.
⟨ If the next character is a parameter number, make *cur_tok* a *match* token; but if it is a left brace, store '*left_brace*, *end_match*', set *hash_brace*, and **goto** *done* 511 ⟩ Used in section 509.
⟨ If the preamble list has been traversed, check that the row has ended 840 ⟩ Used in section 839.
⟨ If the right-hand side is a token parameter or token register, finish the assignment and **goto** *done* 1281 ⟩ Used in section 1280.
⟨ If the string *hyph_word*[h] is less than *hc*[1 .. *hn*], **goto** *not_found*; but if the two strings are equal, set *hyf* to the hyphen positions and **goto** *found* 985 ⟩ Used in section 984.
⟨ If the string *hyph_word*[h] is less than or equal to s, interchange (*hyph_word*[h], *hyph_list*[h]) with (s, p) 995 ⟩ Used in section 994.
⟨ If there's a ligature or kern at the cursor position, update the data structures, possibly advancing j; continue until the cursor moves 963 ⟩ Used in section 960.
⟨ If there's a ligature/kern command relevant to *cur_l* and *cur_r*, adjust the text appropriately; exit to *main_loop_wrapup* 1093 ⟩ Used in section 1088.
⟨ If this font has already been loaded, set f to the internal font number and **goto** *common_ending* 1314 ⟩ Used in section 1311.
⟨ If this *sup_mark* starts an expanded character like `^^A` or `^^df`, then **goto** *reswitch*, otherwise set *state* ← *mid_line* 382 ⟩ Used in section 374.
⟨ Ignore the fraction operation and complain about this ambiguous case 1237 ⟩ Used in section 1235.
⟨ Implement `\XeTeXdefaultencoding` 1447 ⟩ Used in section 1403.
⟨ Implement `\XeTeXglyph` 1444 ⟩ Used in section 1403.
⟨ Implement `\XeTeXinputencoding` 1446 ⟩ Used in section 1403.
⟨ Implement `\XeTeXlinebreaklocale` 1448 ⟩ Used in section 1403.
⟨ Implement `\XeTeXpdffile` 1443 ⟩ Used in section 1403.
⟨ Implement `\XeTeXpicfile` 1442 ⟩ Used in section 1403.
⟨ Implement `\closeout` 1408 ⟩ Used in section 1403.
⟨ Implement `\immediate` 1438 ⟩ Used in section 1403.
⟨ Implement `\openout` 1406 ⟩ Used in section 1403.
⟨ Implement `\pdfsavepos` 1450 ⟩ Used in section 1403.
⟨ Implement `\primitive` 402 ⟩ Used in section 399.
⟨ Implement `\resettimer` 1414 ⟩ Used in section 1403.
⟨ Implement `\setlanguage` 1440 ⟩ Used in section 1403.
⟨ Implement `\setrandomseed` 1413 ⟩ Used in section 1403.
⟨ Implement `\special` 1409 ⟩ Used in section 1403.
⟨ Implement `\write` 1407 ⟩ Used in section 1403.
⟨ Incorporate a whatsit node into a vbox 1419 ⟩ Used in section 711.
⟨ Incorporate a whatsit node into an hbox 1420 ⟩ Used in section 691.
⟨ Incorporate box dimensions into the dimensions of the hbox that will contain it 693 ⟩ Used in section 691.
⟨ Incorporate box dimensions into the dimensions of the vbox that will contain it 712 ⟩ Used in section 711.

⟨ Incorporate character dimensions into the dimensions of the hbox that will contain it, then move to the next node 694 ⟩ Used in section 691.
⟨ Incorporate glue into the horizontal totals 698 ⟩ Used in section 691.
⟨ Incorporate glue into the vertical totals 713 ⟩ Used in section 711.
⟨ Increase the number of parameters in the last font 615 ⟩ Used in section 613.
⟨ Increase k until x can be multiplied by a factor of 2^{-k}, and adjust y accordingly 124 ⟩ Used in section 123.
⟨ Initialize additional fields of the first active node 1656 ⟩ Used in section 912.
⟨ Initialize for hyphenating a paragraph 939 ⟩ Used in section 911.
⟨ Initialize table entries (done by INITEX only) 189, 248, 254, 258, 266, 276, 285, 587, 1000, 1005, 1270, 1355, 1432, 1463, 1629, 1665 ⟩ Used in section 8.
⟨ Initialize the LR stack 1520 ⟩ Used in sections 689, 1524, and 1545.
⟨ Initialize the current page, insert the \topskip glue ahead of p, and **goto** *continue* 1055 ⟩ Used in section 1054.
⟨ Initialize the input routines 361 ⟩ Used in section 1391.
⟨ Initialize the output routines 55, 65, 563, 568 ⟩ Used in section 1386.
⟨ Initialize the print *selector* based on *interaction* 79 ⟩ Used in sections 1319 and 1391.
⟨ Initialize the special list heads and constant nodes 838, 845, 868, 1035, 1042 ⟩ Used in section 189.
⟨ Initialize variables as *ship_out* begins 653 ⟩ Used in section 678.
⟨ Initialize variables for ε-TeX compatibility mode 1623 ⟩ Used in sections 1463 and 1465.
⟨ Initialize variables for ε-TeX extended mode 1624 ⟩ Used in sections 1451 and 1465.
⟨ Initialize whatever TeX might access 8 ⟩ Used in section 4.
⟨ Initialize *hlist_out* for mixed direction typesetting 1524 ⟩ Used in section 655.
⟨ Initiate input from new pseudo file 1566 ⟩ Used in section 1564.
⟨ Initiate or terminate input from a file 412 ⟩ Used in section 399.
⟨ Initiate the construction of an hbox or vbox, then **return** 1137 ⟩ Used in section 1133.
⟨ Input and store tokens from the next line of the file 518 ⟩ Used in section 517.
⟨ Input for \read from the terminal 519 ⟩ Used in section 518.
⟨ Input from external file, **goto** *restart* if no input found 373 ⟩ Used in section 371.
⟨ Input from token list, **goto** *restart* if end of list or if a parameter needs to be expanded 387 ⟩ Used in section 371.
⟨ Input the first line of *read_file*[m] 520 ⟩ Used in section 518.
⟨ Input the next line of *read_file*[m] 521 ⟩ Used in section 518.
⟨ Insert LR nodes at the beginning of the current line and adjust the LR stack based on LR nodes in this line 1517 ⟩ Used in section 928.
⟨ Insert LR nodes at the end of the current line 1519 ⟩ Used in section 928.
⟨ Insert a delta node to prepare for breaks at *cur_p* 891 ⟩ Used in section 884.
⟨ Insert a delta node to prepare for the next active node 892 ⟩ Used in section 884.
⟨ Insert a dummy noad to be sub/superscripted 1231 ⟩ Used in section 1230.
⟨ Insert a new active node from *best_place*[*fit_class*] to *cur_p* 893 ⟩ Used in section 884.
⟨ Insert a new control sequence after p, then make p point to it 287 ⟩ Used in section 286.
⟨ Insert a new pattern into the linked trie 1017 ⟩ Used in section 1015.
⟨ Insert a new primitive after p, then make p point to it 290 ⟩ Used in section 289.
⟨ Insert a new trie node between q and p, and make p point to it 1018 ⟩ Used in sections 1017, 1666, and 1667.
⟨ Insert a token containing *frozen_endv* 409 ⟩ Used in section 396.
⟨ Insert a token saved by \afterassignment, if any 1323 ⟩ Used in section 1265.
⟨ Insert glue for *split_top_skip* and set $p \leftarrow$ *null* 1023 ⟩ Used in section 1022.
⟨ Insert hyphens as specified in *hyph_list*[h] 986 ⟩ Used in section 985.
⟨ Insert macro parameter and **goto** *restart* 389 ⟩ Used in section 387.
⟨ Insert the appropriate mark text into the scanner 420 ⟩ Used in section 399.
⟨ Insert the current list into its environment 860 ⟩ Used in section 848.
⟨ Insert the pair (s, p) into the exception table 994 ⟩ Used in section 993.
⟨ Insert the ⟨v_j⟩ template and **goto** *restart* 837 ⟩ Used in section 372.

⟨ Insert token p into TEX's input 356 ⟩ Used in section 312.
⟨ Interpret code c and **return** if done 88 ⟩ Used in section 87.
⟨ Introduce new material from the terminal and **return** 91 ⟩ Used in section 88.
⟨ Issue an error message if $cur_val = fmem_ptr$ 614 ⟩ Used in section 613.
⟨ Justify the line ending at breakpoint cur_p, and append it to the current vertical list, together with associated penalties and other insertions 928 ⟩ Used in section 925.
⟨ Labels in the outer block 6 ⟩ Used in section 4.
⟨ Last-minute procedures 1387, 1389, 1390, 1392 ⟩ Used in section 1384.
⟨ Lengthen the preamble periodically 841 ⟩ Used in section 840.
⟨ Let cur_h be the position of the first box, and set $leader_wd + lx$ to the spacing between corresponding parts of boxes 665 ⟩ Used in section 664.
⟨ Let cur_v be the position of the first box, and set $leader_ht + lx$ to the spacing between corresponding parts of boxes 674 ⟩ Used in section 673.
⟨ Let d be the natural width of node p; if the node is "visible," **goto** *found*; if the node is glue that stretches or shrinks, set $v \leftarrow max_dimen$ 1201 ⟩ Used in section 1200.
⟨ Let d be the natural width of this glue; if stretching or shrinking, set $v \leftarrow max_dimen$; **goto** *found* in the case of leaders 1202 ⟩ Used in section 1201.
⟨ Let d be the width of the whatsit p, and **goto** *found* if "visible" 1421 ⟩ Used in section 1201.
⟨ Let j be the prototype box for the display 1551 ⟩ Used in section 1545.
⟨ Let n be the largest legal code value, based on cur_chr 1287 ⟩ Used in section 1286.
⟨ Link node p into the current page and **goto** *done* 1052 ⟩ Used in section 1051.
⟨ Local variables for dimension calculations 485 ⟩ Used in section 482.
⟨ Local variables for finishing a displayed formula 1252, 1552 ⟩ Used in section 1248.
⟨ Local variables for formatting calculations 345 ⟩ Used in section 341.
⟨ Local variables for hyphenation 954, 966, 976, 983 ⟩ Used in section 944.
⟨ Local variables for initialization 19, 188, 981 ⟩ Used in section 4.
⟨ Local variables for line breaking 910, 942, 948 ⟩ Used in section 863.
⟨ Look ahead for another character, or leave lig_stack empty if there's none there 1092 ⟩ Used in section 1088.
⟨ Look at all the marks in nodes before the break, and set the final link to *null* at the break 1033 ⟩ Used in section 1031.
⟨ Look at the list of characters starting with x in font g; set f and c whenever a better character is found; **goto** *found* as soon as a large enough variant is encountered 751 ⟩ Used in section 750.
⟨ Look at the other stack entries until deciding what sort of DVI command to generate; **goto** *found* if node p is a "hit" 647 ⟩ Used in section 643.
⟨ Look at the variants of (z, x); set f and c whenever a better character is found; **goto** *found* as soon as a large enough variant is encountered 750 ⟩ Used in section 749.
⟨ Look for parameter number or **##** 514 ⟩ Used in section 512.
⟨ Look for the word $hc[1 \mathinner{..} hn]$ in the exception table, and **goto** *found* (with hyf containing the hyphens) if an entry is found 984 ⟩ Used in section 977.
⟨ Look up the characters of list n in the hash table, and set cur_cs 1579 ⟩ Used in section 1578.
⟨ Look up the characters of list r in the hash table, and set cur_cs 408 ⟩ Used in section 406.
⟨ Make a copy of node p in node r 231 ⟩ Used in section 230.
⟨ Make a ligature node, if $ligature_present$; insert a null discretionary, if appropriate 1089 ⟩ Used in section 1088.
⟨ Make a partial copy of the whatsit node p and make r point to it; set $words$ to the number of initial words not yet copied 1417 ⟩ Used in sections 232 and 1544.
⟨ Make a second pass over the mlist, removing all noads and inserting the proper spacing and penalties 808 ⟩ Used in section 769.
⟨ Make final adjustments and **goto** *done* 611 ⟩ Used in section 597.
⟨ Make node p look like a $char_node$ and **goto** *reswitch* 692 ⟩ Used in sections 660, 691, and 1201.
⟨ Make sure that f is in the proper range 1601 ⟩ Used in section 1594.
⟨ Make sure that $page_max_depth$ is not exceeded 1057 ⟩ Used in section 1051.
⟨ Make sure that pi is in the proper range 879 ⟩ Used in section 877.

⟨ Make the contribution list empty by setting its tail to *contrib_head* 1049 ⟩ Used in section 1048.
⟨ Make the first 256 strings 48 ⟩ Used in section 47.
⟨ Make the height of box y equal to h 782 ⟩ Used in section 781.
⟨ Make the running dimensions in rule q extend to the boundaries of the alignment 854 ⟩ Used in section 853.
⟨ Make the unset node r into a *vlist_node* of height w, setting the glue as if the height were t 859 ⟩ Used in section 856.
⟨ Make the unset node r into an *hlist_node* of width w, setting the glue as if the width were t 858 ⟩ Used in section 856.
⟨ Make variable b point to a box for (f, c) 753 ⟩ Used in section 749.
⟨ Manufacture a control sequence name 406 ⟩ Used in section 399.
⟨ Math-only cases in non-math modes, or vice versa 1100 ⟩ Used in section 1099.
⟨ Merge sequences of words using native fonts and inter-word spaces into single nodes 656 ⟩ Used in section 655.
⟨ Merge the widths in the span nodes of q with those of p, destroying the span nodes of q 851 ⟩ Used in section 849.
⟨ Modify the end of the line to reflect the nature of the break and to include `\rightskip`; also set the proper value of *disc_break* 929 ⟩ Used in section 928.
⟨ Modify the glue specification in *main_p* according to the space factor 1098 ⟩ Used in section 1097.
⟨ Move down or output leaders 672 ⟩ Used in section 669.
⟨ Move node p to the current page; if it is time for a page break, put the nodes following the break back onto the contribution list, and **return** to the user's output routine if there is one 1051 ⟩ Used in section 1048.
⟨ Move node p to the new list and go to the next node; or **goto** *done* if the end of the reflected segment has been reached 1534 ⟩ Used in section 1533.
⟨ Move pointer s to the end of the current list, and set *replace_count*(r) appropriately 972 ⟩ Used in section 968.
⟨ Move right or output leaders 663 ⟩ Used in section 660.
⟨ Move the characters of a ligature node to *hu* and *hc*; but **goto** *done3* if they are not all letters 951 ⟩ Used in section 950.
⟨ Move the cursor past a pseudo-ligature, then **goto** *main_loop_lookahead* or *main_lig_loop* 1091 ⟩ Used in section 1088.
⟨ Move the data into *trie* 1012 ⟩ Used in section 1020.
⟨ Move the non-*char_node* p to the new list 1535 ⟩ Used in section 1534.
⟨ Move to next line of file, or **goto** *restart* if there is no next line, or **return** if a `\read` line has finished 390 ⟩ Used in section 373.
⟨ Negate a boolean conditional and **goto** *reswitch* 1576 ⟩ Used in section 399.
⟨ Negate all three glue components of *cur_val* 465 ⟩ Used in sections 464 and 1591.
⟨ Nullify *width*(q) and the tabskip glue following this column 850 ⟩ Used in section 849.
⟨ Numbered cases for *debug_help* 1393 ⟩ Used in section 1392.
⟨ Open *tfm_file* for input and **begin** 598 ⟩ Used in section 597.
⟨ Other local variables for *try_break* 878, 1655 ⟩ Used in section 877.
⟨ Output a box in a vlist 670 ⟩ Used in section 669.
⟨ Output a box in an hlist 661 ⟩ Used in section 660.
⟨ Output a leader box at *cur_h*, then advance *cur_h* by *leader_wd* + *lx* 666 ⟩ Used in section 664.
⟨ Output a leader box at *cur_v*, then advance *cur_v* by *leader_ht* + *lx* 675 ⟩ Used in section 673.
⟨ Output a rule in a vlist, **goto** *next_p* 671 ⟩ Used in section 669.
⟨ Output a rule in an hlist 662 ⟩ Used in section 660.
⟨ Output leaders in a vlist, **goto** *fin_rule* if a rule or to *next_p* if done 673 ⟩ Used in section 672.
⟨ Output leaders in an hlist, **goto** *fin_rule* if a rule or to *next_p* if done 664 ⟩ Used in section 663.
⟨ Output node p for *hlist_out* and move to the next node, maintaining the condition *cur_v* = *base_line* 658 ⟩ Used in section 655.
⟨ Output node p for *vlist_out* and move to the next node, maintaining the condition *cur_h* = *left_edge* 668 ⟩ Used in section 667.
⟨ Output statistics about this job 1388 ⟩ Used in section 1387.
⟨ Output the font definitions for all fonts that were used 681 ⟩ Used in section 680.

⟨ Output the font name whose internal number is f 639 ⟩ Used in section 638.
⟨ Output the non-*char_node* p for *hlist_out* and move to the next node 660 ⟩ Used in section 658.
⟨ Output the non-*char_node* p for *vlist_out* 669 ⟩ Used in section 668.
⟨ Output the whatsit node p in a vlist 1426 ⟩ Used in section 669.
⟨ Output the whatsit node p in an hlist 1430 ⟩ Used in section 660.
⟨ Pack all stored *hyph_codes* 1668 ⟩ Used in section 1020.
⟨ Pack the family into *trie* relative to h 1010 ⟩ Used in section 1007.
⟨ Package an unset box for the current column and record its width 844 ⟩ Used in section 839.
⟨ Package the display line 1557 ⟩ Used in section 1555.
⟨ Package the preamble list, to determine the actual tabskip glue amounts, and let p point to this prototype box 852 ⟩ Used in section 848.
⟨ Perform computations for last line and **goto** *found* 1657 ⟩ Used in section 900.
⟨ Perform the default output routine 1077 ⟩ Used in section 1066.
⟨ Pontificate about improper alignment in display 1261 ⟩ Used in section 1260.
⟨ Pop the condition stack 531 ⟩ Used in sections 533, 535, 544, and 545.
⟨ Pop the expression stack and **goto** *found* 1600 ⟩ Used in section 1594.
⟨ Prepare a *native_word_node* for hyphenation 946 ⟩ Used in section 943.
⟨ Prepare all the boxes involved in insertions to act as queues 1072 ⟩ Used in section 1068.
⟨ Prepare for display after a non-empty paragraph 1545 ⟩ Used in section 1200.
⟨ Prepare for display after an empty paragraph 1543 ⟩ Used in section 1199.
⟨ Prepare to deactivate node r, and **goto** *deactivate* unless there is a reason to consider lines of text from r to *cur_p* 902 ⟩ Used in section 899.
⟨ Prepare to insert a token that matches *cur_group*, and print what it is 1119 ⟩ Used in section 1118.
⟨ Prepare to move a box or rule node to the current page, then **goto** *contribute* 1056 ⟩ Used in section 1054.
⟨ Prepare to move whatsit p to the current page, then **goto** *contribute* 1424 ⟩ Used in section 1054.
⟨ Print a short indication of the contents of node p 201 ⟩ Used in section 200.
⟨ Print a symbolic description of the new break node 894 ⟩ Used in section 893.
⟨ Print a symbolic description of this feasible break 904 ⟩ Used in section 903.
⟨ Print additional data in the new active node 1663 ⟩ Used in section 894.
⟨ Print either '`definition`' or '`use`' or '`preamble`' or '`text`', and insert tokens that should lead to recovery 369 ⟩ Used in section 368.
⟨ Print location of current line 343 ⟩ Used in section 342.
⟨ Print newly busy locations 196 ⟩ Used in section 192.
⟨ Print string s as an error message 1337 ⟩ Used in section 1333.
⟨ Print string s on the terminal 1334 ⟩ Used in section 1333.
⟨ Print the banner line, including the date and time 571 ⟩ Used in section 569.
⟨ Print the font identifier for *font*(p) 297 ⟩ Used in sections 200 and 202.
⟨ Print the help information and **goto** *continue* 93 ⟩ Used in section 88.
⟨ Print the list between *printed_node* and *cur_p*, then set *printed_node* ← *cur_p* 905 ⟩ Used in section 904.
⟨ Print the menu of available options 89 ⟩ Used in section 88.
⟨ Print the result of command c 507 ⟩ Used in section 505.
⟨ Print two lines using the tricky pseudoprinted information 347 ⟩ Used in section 342.
⟨ Print type of token list 344 ⟩ Used in section 342.
⟨ Process an active-character control sequence and set *state* ← *mid_line* 383 ⟩ Used in section 374.
⟨ Process an expression and **return** 1591 ⟩ Used in section 458.
⟨ Process node-or-noad q as much as possible in preparation for the second pass of *mlist_to_hlist*, then move to the next item in the mlist 770 ⟩ Used in section 769.
⟨ Process whatsit p in *vert_break* loop, **goto** *not_found* 1425 ⟩ Used in section 1027.
⟨ Prune the current list, if necessary, until it contains only *char_node*, *kern_node*, *hlist_node*, *vlist_node*, *rule_node*, and *ligature_node* items; set n to the length of the list, and set q to the list's tail 1175 ⟩ Used in section 1173.
⟨ Prune unwanted nodes at the beginning of the next line 927 ⟩ Used in section 925.

⟨ Pseudoprint the line 348 ⟩ Used in section 342.
⟨ Pseudoprint the token list 349 ⟩ Used in section 342.
⟨ Push the condition stack 530 ⟩ Used in section 533.
⟨ Push the expression stack and **goto** *restart* 1599 ⟩ Used in section 1596.
⟨ Put each of TEX's primitives into the hash table 252, 256, 264, 274, 295, 364, 410, 418, 445, 450, 503, 522, 526, 588, 828, 1037, 1106, 1112, 1125, 1142, 1161, 1168, 1195, 1210, 1223, 1232, 1242, 1262, 1273, 1276, 1284, 1304, 1308, 1316, 1326, 1331, 1340, 1345, 1398 ⟩ Used in section 1390.
⟨ Put help message on the transcript file 94 ⟩ Used in section 86.
⟨ Put the characters $hu[i+1\,..]$ into *post_break*(r), appending to this list and to *major_tail* until synchronization has been achieved 970 ⟩ Used in section 968.
⟨ Put the characters $hu[l\,..\,i]$ and a hyphen into *pre_break*(r) 969 ⟩ Used in section 968.
⟨ Put the fraction into a box with its delimiters, and make *new_hlist*(q) point to it 792 ⟩ Used in section 787.
⟨ Put the `\leftskip` glue at the left and detach this line 935 ⟩ Used in section 928.
⟨ Put the optimal current page into box 255, update *first_mark* and *bot_mark*, append insertions to their boxes, and put the remaining nodes back on the contribution list 1068 ⟩ Used in section 1066.
⟨ Put the (positive) 'at' size into s 1313 ⟩ Used in section 1312.
⟨ Put the `\rightskip` glue after node q 934 ⟩ Used in section 929.
⟨ Read and check the font data if file exists; *abort* if the `TFM` file is malformed; if there's no room for this font, say so and **goto** *done*; otherwise *incr*(*font_ptr*) and **goto** *done* 597 ⟩ Used in section 595.
⟨ Read box dimensions 606 ⟩ Used in section 597.
⟨ Read character data 604 ⟩ Used in section 597.
⟨ Read extensible character recipes 609 ⟩ Used in section 597.
⟨ Read font parameters 610 ⟩ Used in section 597.
⟨ Read ligature/kern program 608 ⟩ Used in section 597.
⟨ Read next line of file into *buffer*, or **goto** *restart* if the file has ended 392 ⟩ Used in section 390.
⟨ Read one string, but return *false* if the string memory space is getting too tight for comfort 52 ⟩ Used in section 51.
⟨ Read the first line of the new file 573 ⟩ Used in section 572.
⟨ Read the other strings from the `TEX.POOL` file and return *true*, or give an error message and return *false* 51 ⟩ Used in section 47.
⟨ Read the `TFM` header 603 ⟩ Used in section 597.
⟨ Read the `TFM` size fields 600 ⟩ Used in section 597.
⟨ Readjust the height and depth of *cur_box*, for `\vtop` 1141 ⟩ Used in section 1140.
⟨ Reconstitute nodes for the hyphenated word, inserting discretionary hyphens 967 ⟩ Used in section 956.
⟨ Record a new feasible break 903 ⟩ Used in section 899.
⟨ Recover from an unbalanced output routine 1081 ⟩ Used in section 1080.
⟨ Recover from an unbalanced write command 1435 ⟩ Used in section 1434.
⟨ Recycle node p 1053 ⟩ Used in section 1051.
⟨ Reduce to the case that $a, c \geq 0$, $b, d > 0$ 127 ⟩ Used in section 126.
⟨ Reduce to the case that $f \geq 0$ and $q > 0$ 119 ⟩ Used in section 118.
⟨ Remove the last box, unless it's part of a discretionary 1135 ⟩ Used in section 1134.
⟨ Replace nodes $ha\,..\,hb$ by a sequence of nodes that includes the discretionary hyphens 956 ⟩ Used in section 944.
⟨ Replace the tail of the list by p 1241 ⟩ Used in section 1240.
⟨ Replace z by z' and compute α, β 607 ⟩ Used in section 606.
⟨ Report LR problems 1523 ⟩ Used in sections 1522 and 1541.
⟨ Report a runaway argument and abort 430 ⟩ Used in sections 426 and 433.
⟨ Report a tight hbox and **goto** *common_ending*, if this box is sufficiently bad 709 ⟩ Used in section 706.
⟨ Report a tight vbox and **goto** *common_ending*, if this box is sufficiently bad 720 ⟩ Used in section 718.
⟨ Report an extra right brace and **goto** *continue* 429 ⟩ Used in section 426.
⟨ Report an improper use of the macro and abort 432 ⟩ Used in section 431.
⟨ Report an overfull hbox and **goto** *common_ending*, if this box is sufficiently bad 708 ⟩ Used in section 706.

⟨ Report an overfull vbox and **goto** *common_ending*, if this box is sufficiently bad 719 ⟩ Used in section 718.
⟨ Report an underfull hbox and **goto** *common_ending*, if this box is sufficiently bad 702 ⟩ Used in section 700.
⟨ Report an underfull vbox and **goto** *common_ending*, if this box is sufficiently bad 716 ⟩ Used in section 715.
⟨ Report overflow of the input buffer, and abort 35 ⟩ Used in sections 31 and 1567.
⟨ Report that an invalid delimiter code is being changed to null; set $cur_val \leftarrow 0$ 1215 ⟩ Used in section 1214.
⟨ Report that the font won't be loaded 596 ⟩ Used in section 595.
⟨ Report that this dimension is out of range 495 ⟩ Used in section 482.
⟨ Reset *cur_tok* for unexpandable primitives, goto restart 403 ⟩ Used in sections 447 and 474.
⟨ Resume the page builder after an output routine has come to an end 1080 ⟩ Used in section 1154.
⟨ Retrieve the prototype box 1553 ⟩ Used in sections 1248 and 1248.
⟨ Reverse an hlist segment and **goto** *reswitch* 1532 ⟩ Used in section 1527.
⟨ Reverse the complete hlist and set the subtype to *reversed* 1531 ⟩ Used in section 1524.
⟨ Reverse the links of the relevant passive nodes, setting *cur_p* to the first breakpoint 926 ⟩ Used in section 925.
⟨ Save current position to *pdf_last_x_pos*, *pdf_last_y_pos* 1427 ⟩ Used in sections 1426 and 1430.
⟨ Scan a control sequence and set *state* ← *skip_blanks* or *mid_line* 384 ⟩ Used in section 374.
⟨ Scan a factor f of type o or start a subexpression 1596 ⟩ Used in section 1594.
⟨ Scan a numeric constant 478 ⟩ Used in section 474.
⟨ Scan a parameter until its delimiter string has been found; or, if $s = null$, simply scan the delimiter string 426 ⟩ Used in section 425.
⟨ Scan a subformula enclosed in braces and **return** 1207 ⟩ Used in section 1205.
⟨ Scan ahead in the buffer until finding a nonletter; if an expanded code is encountered, reduce it and **goto** *start_cs*; otherwise if a multiletter control sequence is found, adjust *cur_cs* and *loc*, and **goto** *found* 386 ⟩ Used in section 384.
⟨ Scan an alphabetic character code into *cur_val* 476 ⟩ Used in section 474.
⟨ Scan an optional space 477 ⟩ Used in sections 476, 482, 490, and 1254.
⟨ Scan and build the body of the token list; **goto** *found* when finished 512 ⟩ Used in section 508.
⟨ Scan and build the parameter part of the macro definition 509 ⟩ Used in section 508.
⟨ Scan and evaluate an expression e of type l 1594 ⟩ Used in section 1593.
⟨ Scan decimal fraction 487 ⟩ Used in section 482.
⟨ Scan file name in the buffer 566 ⟩ Used in section 565.
⟨ Scan for all other units and adjust *cur_val* and f accordingly; **goto** *done* in the case of scaled points 493 ⟩ Used in section 488.
⟨ Scan for `fil` units; **goto** *attach_fraction* if found 489 ⟩ Used in section 488.
⟨ Scan for `mu` units and **goto** *attach_fraction* 491 ⟩ Used in section 488.
⟨ Scan for units that are internal dimensions; **goto** *attach_sign* with *cur_val* set if found 490 ⟩ Used in section 488.
⟨ Scan preamble text until *cur_cmd* is *tab_mark* or *car_ret*, looking for changes in the tabskip glue; append an alignrecord to the preamble list 827 ⟩ Used in section 825.
⟨ Scan the argument for command c 506 ⟩ Used in section 505.
⟨ Scan the font size specification 1312 ⟩ Used in section 1311.
⟨ Scan the next operator and set o 1595 ⟩ Used in section 1594.
⟨ Scan the parameters and make *link*(r) point to the macro body; but **return** if an illegal `\par` is detected 425 ⟩ Used in section 423.
⟨ Scan the preamble and record it in the *preamble* list 825 ⟩ Used in section 822.
⟨ Scan the template $\langle u_j \rangle$, putting the resulting token list in *hold_head* 831 ⟩ Used in section 827.
⟨ Scan the template $\langle v_j \rangle$, putting the resulting token list in *hold_head* 832 ⟩ Used in section 827.
⟨ Scan units and set *cur_val* to $x \cdot (cur_val + f/2^{16})$, where there are x sp per unit; **goto** *attach_sign* if the units are internal 488 ⟩ Used in section 482.
⟨ Search *eqtb* for equivalents equal to p 281 ⟩ Used in section 197.
⟨ Search *hyph_list* for pointers to p 987 ⟩ Used in section 197.
⟨ Search *save_stack* for equivalents that point to p 315 ⟩ Used in section 197.
⟨ Select the appropriate case and **return** or **goto** *common_ending* 544 ⟩ Used in section 536.

⟨ Set initial values of key variables 23, 24, 62, 78, 81, 84, 101, 122, 191, 241, 280, 284, 302, 317, 398, 417, 473, 516, 525, 556, 586, 591, 629, 632, 642, 687, 696, 704, 727, 819, 941, 982, 1044, 1087, 1321, 1336, 1354, 1397, 1412, 1516, 1562, 1628, 1647, 1671 ⟩ Used in section 8.
⟨ Set line length parameters in preparation for hanging indentation 897 ⟩ Used in section 896.
⟨ Set the glue in all the unset boxes of the current list 853 ⟩ Used in section 848.
⟨ Set the glue in node r and change it from an unset node 856 ⟩ Used in section 855.
⟨ Set the unset box q and the unset boxes in it 855 ⟩ Used in section 853.
⟨ Set the value of b to the badness for shrinking the line, and compute the corresponding *fit_class* 901 ⟩ Used in section 899.
⟨ Set the value of b to the badness for stretching the line, and compute the corresponding *fit_class* 900 ⟩ Used in section 899.
⟨ Set the value of b to the badness of the last line for shrinking, compute the corresponding *fit_class*, and **goto** *found* 1659 ⟩ Used in section 1657.
⟨ Set the value of b to the badness of the last line for stretching, compute the corresponding *fit_class*, and **goto** *found* 1658 ⟩ Used in section 1657.
⟨ Set the value of *output_penalty* 1067 ⟩ Used in section 1066.
⟨ Set the value of x to the text direction before the display 1542 ⟩ Used in sections 1543 and 1545.
⟨ Set up data structures with the cursor following position j 962 ⟩ Used in section 960.
⟨ Set up the hlist for the display line 1556 ⟩ Used in section 1555.
⟨ Set up the values of *cur_size* and *cur_mu*, based on *cur_style* 746 ⟩ Used in sections 763, 769, 770, 773, 798, 805, 805, 808, 810, and 811.
⟨ Set variable c to the current escape character 269 ⟩ Used in section 67.
⟨ Set variable w to indicate if this case should be reported 1586 ⟩ Used in sections 1585 and 1587.
⟨ Ship box p out 678 ⟩ Used in section 676.
⟨ Show equivalent n, in region 1 or 2 249 ⟩ Used in section 278.
⟨ Show equivalent n, in region 3 255 ⟩ Used in section 278.
⟨ Show equivalent n, in region 4 259 ⟩ Used in section 278.
⟨ Show equivalent n, in region 5 268 ⟩ Used in section 278.
⟨ Show equivalent n, in region 6 277 ⟩ Used in section 278.
⟨ Show the auxiliary field, a 245 ⟩ Used in section 244.
⟨ Show the box context 1491 ⟩ Used in section 1489.
⟨ Show the box packaging info 1490 ⟩ Used in section 1489.
⟨ Show the current contents of a box 1350 ⟩ Used in section 1347.
⟨ Show the current meaning of a token, then **goto** *common_ending* 1348 ⟩ Used in section 1347.
⟨ Show the current value of some parameter or register, then **goto** *common_ending* 1351 ⟩ Used in section 1347.
⟨ Show the font identifier in $eqtb[n]$ 260 ⟩ Used in section 259.
⟨ Show the halfword code in $eqtb[n]$ 261 ⟩ Used in section 259.
⟨ Show the status of the current page 1040 ⟩ Used in section 244.
⟨ Show the text of the macro being expanded 435 ⟩ Used in section 423.
⟨ Simplify a trivial box 764 ⟩ Used in section 763.
⟨ Skip to `\else` or `\fi`, then **goto** *common_ending* 535 ⟩ Used in section 533.
⟨ Skip to node *ha*, or **goto** *done1* if no hyphenation should be attempted 949 ⟩ Used in section 943.
⟨ Skip to node *hb*, putting letters into *hu* and *hc* 950 ⟩ Used in section 943.
⟨ Sort p into the list starting at *rover* and advance p to $rlink(p)$ 154 ⟩ Used in section 153.
⟨ Sort the hyphenation op tables into proper order 999 ⟩ Used in section 1006.
⟨ Split off part of a vertical box, make *cur_box* point to it 1136 ⟩ Used in section 1133.
⟨ Split the *native_word_node* at l and link the second part after *ha* 947 ⟩ Used in sections 946 and 946.
⟨ Squeeze the equation as much as possible; if there is an equation number that should go on a separate line by itself, set $e \leftarrow 0$ 1255 ⟩ Used in section 1253.
⟨ Start a new current page 1045 ⟩ Used in sections 241 and 1071.
⟨ Store additional data for this feasible break 1661 ⟩ Used in section 903.
⟨ Store additional data in the new active node 1662 ⟩ Used in section 893.

⟨ Store *cur_box* in a box register 1131 ⟩ Used in section 1129.
⟨ Store maximum values in the *hyf* table 978 ⟩ Used in section 977.
⟨ Store *save_stack*[*save_ptr*] in *eqtb*[*p*], unless *eqtb*[*p*] holds a global value 313 ⟩ Used in section 312.
⟨ Store all current *lc_code* values 1667 ⟩ Used in section 1666.
⟨ Store hyphenation codes for current language 1666 ⟩ Used in section 1014.
⟨ Store the current token, but **goto** *continue* if it is a blank space that would become an undelimited parameter 427 ⟩ Used in section 426.
⟨ Subtract glue from *break_width* 886 ⟩ Used in section 885.
⟨ Subtract the width of node *v* from *break_width* 889 ⟩ Used in section 888.
⟨ Suppress expansion of the next token 401 ⟩ Used in section 399.
⟨ Swap the subscript and superscript into box *x* 786 ⟩ Used in section 781.
⟨ Switch to a larger accent if available and appropriate 784 ⟩ Used in section 781.
⟨ Switch to a larger native-font accent if available and appropriate 783 ⟩ Used in section 781.
⟨ Tell the user what has run away and try to recover 368 ⟩ Used in section 366.
⟨ Terminate the current conditional and skip to `\fi` 545 ⟩ Used in section 399.
⟨ Test box register status 540 ⟩ Used in section 536.
⟨ Test if an integer is odd 539 ⟩ Used in section 536.
⟨ Test if two characters match 541 ⟩ Used in section 536.
⟨ Test if two macro texts match 543 ⟩ Used in section 542.
⟨ Test if two tokens match 542 ⟩ Used in section 536.
⟨ Test relation between integers or dimensions 538 ⟩ Used in section 536.
⟨ The em width for *cur_font* 593 ⟩ Used in section 490.
⟨ The x-height for *cur_font* 594 ⟩ Used in section 490.
⟨ Tidy up the parameter just scanned, and tuck it away 434 ⟩ Used in section 426.
⟨ Transfer node *p* to the adjustment list 697 ⟩ Used in section 691.
⟨ Transplant the post-break list 932 ⟩ Used in section 930.
⟨ Transplant the pre-break list 933 ⟩ Used in section 930.
⟨ Treat *cur_chr* as an active character 1206 ⟩ Used in sections 1205 and 1209.
⟨ Try the final line break at the end of the paragraph, and **goto** *done* if the desired breakpoints have been found 921 ⟩ Used in section 911.
⟨ Try to allocate within node *p* and its physical successors, and **goto** *found* if allocation was possible 149 ⟩ Used in section 147.
⟨ Try to break after a discretionary fragment, then **goto** *done5* 917 ⟩ Used in section 914.
⟨ Try to get a different log file name 570 ⟩ Used in section 569.
⟨ Try to hyphenate the following word 943 ⟩ Used in section 914.
⟨ Try to recover from mismatched `\right` 1246 ⟩ Used in section 1245.
⟨ Types in the outer block 18, 25, 38, 105, 113, 135, 174, 238, 299, 330, 583, 630, 974, 979, 1488 ⟩ Used in section 4.
⟨ Undump a couple more things and the closing check word 1381 ⟩ Used in section 1357.
⟨ Undump constants for consistency check 1362 ⟩ Used in section 1357.
⟨ Undump regions 1 to 6 of *eqtb* 1371 ⟩ Used in section 1368.
⟨ Undump the ε-TeX state 1465 ⟩ Used in section 1362.
⟨ Undump the array info for internal font number *k* 1377 ⟩ Used in section 1375.
⟨ Undump the dynamic memory 1366 ⟩ Used in section 1357.
⟨ Undump the font information 1375 ⟩ Used in section 1357.
⟨ Undump the hash table 1373 ⟩ Used in section 1368.
⟨ Undump the hyphenation tables 1379 ⟩ Used in section 1357.
⟨ Undump the string pool 1364 ⟩ Used in section 1357.
⟨ Undump the table of equivalents 1368 ⟩ Used in section 1357.
⟨ Update the active widths, since the first active node has been deleted 909 ⟩ Used in section 908.
⟨ Update the current height and depth measurements with respect to a glue or kern node *p* 1030 ⟩ Used in section 1026.
⟨ Update the current marks for *fire_up* 1641 ⟩ Used in section 1068.

⟨ Update the current marks for *vsplit* 1638 ⟩ Used in section 1033.
⟨ Update the current page measurements with respect to the glue or kern specified by node p 1058 ⟩ Used in section 1051.
⟨ Update the value of *printed_node* for symbolic displays 906 ⟩ Used in section 877.
⟨ Update the values of *first_mark* and *bot_mark* 1070 ⟩ Used in section 1068.
⟨ Update the values of *last_glue*, *last_penalty*, and *last_kern* 1050 ⟩ Used in section 1048.
⟨ Update the values of *max_h* and *max_v*; but if the page is too large, **goto** *done* 679 ⟩ Used in section 678.
⟨ Update width entry for spanned columns 846 ⟩ Used in section 844.
⟨ Use code c to distinguish between generalized fractions 1236 ⟩ Used in section 1235.
⟨ Use node p to update the current height and depth measurements; if this node is not a legal breakpoint, **goto** *not_found* or *update_heights*, otherwise set *pi* to the associated penalty at the break 1027 ⟩ Used in section 1026.
⟨ Use size fields to allocate font information 601 ⟩ Used in section 597.
⟨ Wipe out the whatsit node p and **goto** *done* 1418 ⟩ Used in section 228.
⟨ Wrap up the box specified by node r, splitting node p if called for; set *wait* ← *true* if node p holds a remainder after splitting 1075 ⟩ Used in section 1074.